2024 年 Java 程序员找工作最新面试攻略

这个文档是帮助正在找工作以及准备找工作的同学，在面试之前去复习和突击的一种方式。

适合已经在技术领域有一定积累，然后不确定面试切入点，所以可以通过这个面试文档来预热和巩固。

想直接通过刷面试文档找到工作的同学也要注意，面试文档的内容是静态的，但是面试过程是动态的，

面试官对于某一个领域的考察，通常是通过连环问的方式去问，所以在面试之前，求职者要对 Java 相

关技术有一个体系化的了解，从而更好地突出自己的综合能力。

在科技日新月异的今天，软件开发行业正经历着前所未有的变革。Java，作为企业级应用开发的中流砥柱，其生态系统也在不断进化，从微服务架构的普及到云原生技术的兴起，再到 AI 与大数据的深度融合，Java 程序员的角色和技能需求随之迭代升级。面对这样的行业背景，如何在求职路上脱颖而出，成为每位开发者必须深思的问题。

随着 Java 这个赛道的不断内卷，这两年，Java 程序员的面试，从原来的常规八股文（有标准答案）到现在，以项目、场景问题、技术深度思考为主，逐步转变成没有标准答案，需要大家基于自己的理解和技术底蕴来回答。

那针对市场中新的需求，有没有最新的面试攻略呢？ 其实也是有的，虽然说没有标准答案，但是我们可以针对如今市场的面试变化，来针对性的设计一些面试回答的思路，让大家有一个清晰和明确的方向。

这里有什么？

1. 针对 2024 年面试行情的变化设计的面试场景题以及回答思路

2. 如何快速通过面试的详细攻略
3. 简历优化技巧

1. 知己知彼才能百战百胜，如何做好面试前的准备工作

2024 年的行情，和 3~4 年前不同，通过海量简历投递和海量面试找工作的时代已经过去了。

在如今面试机会较少，并且面试难度较大的情况下。

充分做好面试的准备才是快速通过面试最有效的方法！

切忌把真实面试当靶场，最后带来的代价是非常巨大的！

面试无非就两个部分，投简历、面试！

很多人把重心放在投简历上，忽略了准备面试的重要性，最后的结果是获得了面试机会但是在面试过程中被刷下来了。

1.1 怎么写简历

着手准备的第一步聚焦于简历的打造。

简历是他人初步了解你的窗口，其重要性不言而喻，因此精心构思简历至关重要。

理想的简历应当围绕你的亲身经历构建，正如某些杰出人士仅凭一句“Unix 的创造者”便足以令人印象深刻。

尽管并非所有人都拥有如此耀眼的成就，但这一例证强调了个人经历在简历中的核心地位。

因此，选择那些能丰富你简历含金量的职位，从事能让你简历熠熠生辉的项目，乃是简历撰写的黄金法则——让经历说话，而非仅仅堆砌文字。

然而，简历的文字表达同样不可小觑，尤其是当我们缺乏显赫背景时。简历应当精炼，避免冗余信息导致焦点模糊，着重突出你的优势与技能。一般而言，简历应涵盖以下核心部分：

- 个人简介：用不超过 200 字简明扼要地概括自己，例如：“拥有超十年软件开发经验，带领团队逾四年，专长于构建高可用性、高性能的分布式系统，背景横跨互联网与金融业，目前担任 XXX 公司 XX 职位，负责 XXX 平台。”
- 联系信息：推荐使用专业的电子邮件地址（如 Gmail），避免使用过于休闲的邮箱服务。若拥有个人网站、博客、GitHub 或 Stack Overflow 账号，务必附上，尤其是个性化域名更能彰显主动性和审美品味，且确保页面内容组织得当，真诚无伪。
- 技术作品与展示：展示你的成果，尤其是对于前端开发者更为直接。后端开发者则可通过 GitHub 展示项目。专业证书或奖项同样值得提及，如微软或 Oracle 认证。
- 技能概览：
  - 技术栈：明确列出掌握的编程语言、框架、技术工具及设计理念，如 Java、Spring Boot、Redis、分布式系统架构等。
  - 专业领域：涵盖前端、算法、机器学习、分布式系统、数据库管理等。
  - 行业经验：涉及的行业如金融、电子商务，及特定业务如客户关系管理、支付系统等。
  - 软实力与经验：团队管理规模、项目管理经验、个人特质等，简洁明了即可。

确保简历内容与目标职位的技能要求对齐，以便快速建立匹配印象。工作与教育背景需精挑细选，强调关键项目、挑战、责任及所获成就，同时，这些内容应紧密支撑你的技能陈述，避免离题。

简历长度以不超过两页 A4 纸为宜，采用 PDF 格式以保证格式一致性。

可借鉴 LinkedIn 或 Microsoft Office 模板美化外观，力求内容精炼，重点突出。

记住，简历的目标是凸显你的独特之处，哪怕仅展示两三个亮点，也足以引起注意。

最后，简历是打开机遇之门的第一步，尤其在竞争激烈的就业市场中，除了实质性的技能和经验，别出心裁的简历设计与正面积极的自我评价亦能增加脱颖而出的机会。

即便初始条件有限，展现出积极的态度、持续的学习意愿和解决问题的能力，同样能传达出你是一个值得投资的潜力股。

1.2 技术知识储备

在准备简历时，切记实事求是，你所列出的每一项技能都可能成为面试对话的起点。"精通"、"熟悉"、"了解"需准确区分，以免自相矛盾。

对于提及的每项技术，务必把握其基础及核心概念，因为面试过程中，面试官往往会逐步深入探讨，以此评估你的实际水平。因此，系统性复习相关书籍和资料是不可或缺的步骤，以备不时之需。

例如：

- 如你标明熟练掌握 Java，那么不仅限于基础语法，还应涵盖并发编程、NIO、JVM 等进阶知识，同时对 Spring、Netty 等流行框架的基本原理有所认识。
- 提及 Go 语言，意味着你应至少阅读过官方的《Effective Go》，理解其核心理念。
- 当列举 Redis 时，对其数据结构、性能调优策略、高可用部署方式及分布式锁机制等，通过官方文档的研读应达到一定的理解深度。
- 如声称掌握面向对象设计，熟悉《设计模式》中的经典 23 种模式将是基本要求。
- 对于分布式架构的宣称，则需对 CAP 原则、微服务架构、弹性设计以及 Spring Cloud、Cloud Native 等相关技术框架有深刻理解。
- 关于网络编程的技能，理解 TCP/IP 协议的三次握手、四次挥手过程，Socket 编程基础，以及 select、poll、epoll 等 I/O 多路复用技术，都是必不可少的知识点。

综上所述，你简历上的每一项技术标注，都应当基于你对该技术核心知识点的掌握之上。这好比备考期末考试，你需要全面回顾教材，确保掌握大多数关键知识点，即使不必面面俱到，但对于 80%以上的重点内容，你都应做到心中有数。这样的准备不仅是为了应对面试，更是对自己技术深度和广度的负责态度体现。

1.3 项目准备（非常重要）

在面试过程中，分享个人项目经历或解决过的挑战几乎是每个面试官必问的环节，但令人诧异的是，许多候选人并未对此做好充分准备。以下四个经典问题频繁出现于面试之中：

1. 分享一个你最为自豪或最近完成的项目。
2. 讲述一次你攻克的最复杂或技术含量最高的难题。
3. 描述一个你经历过的最具挑战性或最艰难的项目。
4. 谈谈你曾犯下的最大技术失误或引发的技术故障。

这些问题背后，面试官的意图各异：

- 第一个问题旨在探查你的成就顶峰、兴趣所在；

- 第二、三题侧重于你的问题解决能力和面对逆境时的心态韧性；
- 而第四题则关注你对待错误的态度，以及是否具备反思与成长的能力。

值得注意的是，面试官会通过连续追问细节来验证信息的真实性，因为虚构的情节难以在严密的追问下自圆其说。

为有效应对这类问题，以下建议或许能帮助你更好地准备：

- 构建故事框架：运用 STAR 法则（情境 Situation、任务 Task、行动 Action、结果 Result）来组织你的叙述，确保内容条理清晰，避免冗长繁杂。
- 添加细节：丰富的技术细节是说服力的关键，它能让故事显得更加真实可信。
- 注入情感：真挚的情感表达能传递你的热情、自豪与坚持，确保情感源自真实的体验。
- 融入反思：在叙述中穿插你的思考、教训总结及后续的改进措施，展现你的成长和成熟。

达到这样的叙述水平并非易事，需要持续的练习与积累。日常工作中，培养即时总结的习惯，对经历进行记录与反思，是避免临阵磨枪的有效方法。

此外，提升语言组织能力与逻辑思维同样重要。通过撰写工作文档和经营个人技术博客，不仅可以锻炼写作技巧，还能深化对技术的理解和反思。

实际上，这些平日里的功夫，正是在关键时刻展示你专业素养与个人魅力的基石。因此，成功的面试表现，源于平日里一点一滴的积累与修炼。

> 面试建议是，**一定要自信，敢于表达**，面试的时候我们对知识的掌握有时候很难面面俱到，把自己的思路说出来，而不是直接告诉面试官自己不懂，这也是可以加分的。

1.4 程序员职业规划的利器-终局思维

没有人知道未来是怎样的，但未来并非虚无飘渺，而是以不同的概率形式活在今天，“终局思维”就是以时间为矢，找到其中概率最大的可能。

在和朋友的饭后闲聊的时候，常常会因为没有限定的发散，引发一些争论或者进入僵局，一般这个时候，我经常会引入一个词语**“终局思维”**，基于未来的角度去回看事情的发展，往往会得到一些有趣的结论。

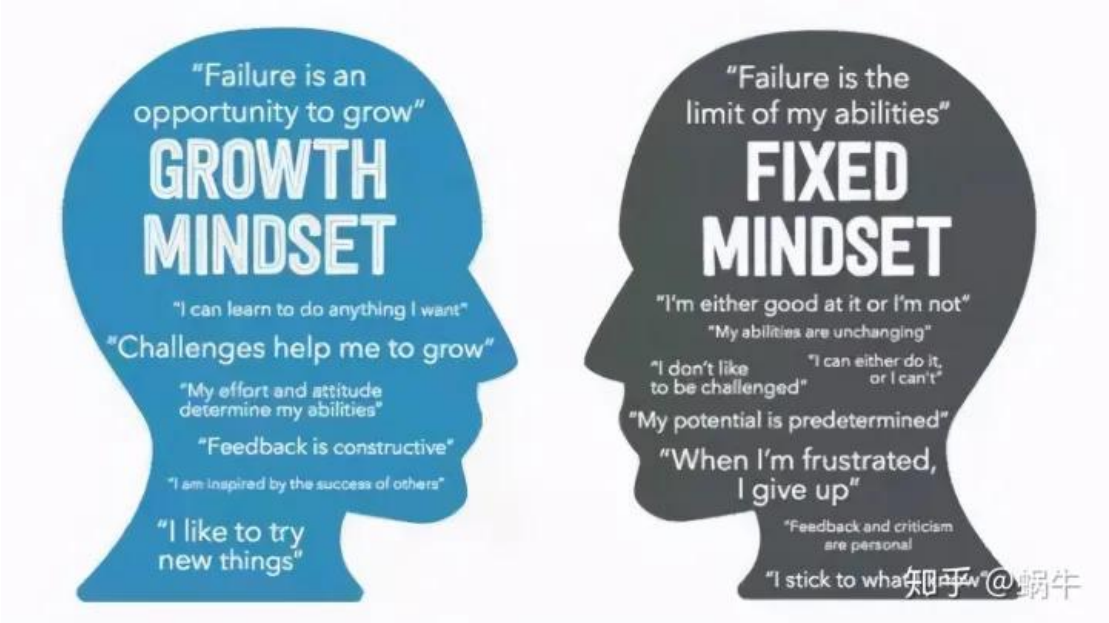
在做职业规划的过程中，面对职场迷茫的咨询者，我也常常会用到这种方式，基于理想职业，去规划 Ta 的职业发展路径，再从众多路径中选择中最舒服或者最适合的去执行。

那么什么是终局思维呢？这不是什么新的理论，在一本十多年前的畅销书**《高效能人士的七个习惯》**中，就提出了**“以终为始”**的观点，这其实就是我们现在说的“终局思维”

> “从现在开始，以你的人生目标作为衡量一切的标准，你的一言一行，一举一动，无论发生在何时，都必须遵循这一原则，即由个人最重视的期许或价值观来决定一切。牢记自己的目标或者使命，就能确信日常的所作所为是否与之南辕北辙。”

而这个词条的最早提出者，大部分认为是湖畔大学的曾鸣教授，关于终局思维他是这样表述的：**“在面对选择时，从终点出发考虑问题，来决定你当下的选择。”**

人的一生中，总有一些比较复杂且时间跨度很长的事业，我们常常做着做着就忘了自己为什么要做这件事，而“终局思维”就是在做一件事的过程中，通过想象最终的结局，以愿景为导向来反推现在的每一个行为、每一项选择，矫正自己是否正沿着正确的路线奔向终点站而没有跑偏，清晰接下来的每一寸时光都要以什么方式度过，然后“以终为始”，**站在未来看现在，修正自己当下正在做的事情。**



似乎听起来很简单，但实际很少有人能做到，因为更多的时候只是把“终局思维”作为知识去理解，而不是一种的思维的工具去运用。

职场问题概述来说无非就是**定位、适应、发展、转型**这几大类，处在不同职业发展阶段的人，一般都会遇到不同的困境，而中年危机的由来，大部分都可以划分到转型问题，对于这个阶段的人生来说，要放弃的就不仅仅是一个岗位而已，还有你在行业中已经获得的资历；在转型过程中，最大的障碍就是你的薪酬待遇上。但是，我们要知道的是，当一个环境缺乏更多的想象空间时，给多少薪酬已经不重要了。

而采用终极思维的思考方式，会让你重新审视现在的经历，跳出 **“当下利益优先”**的思维模式，用一个长期的眼光去重新判断，对个人而言，面临转型期，你需要思考的是：

1、现在做的哪些事情是对未来 5-10 年有帮助的?

2、现在做的哪些事情对未来没有帮助?

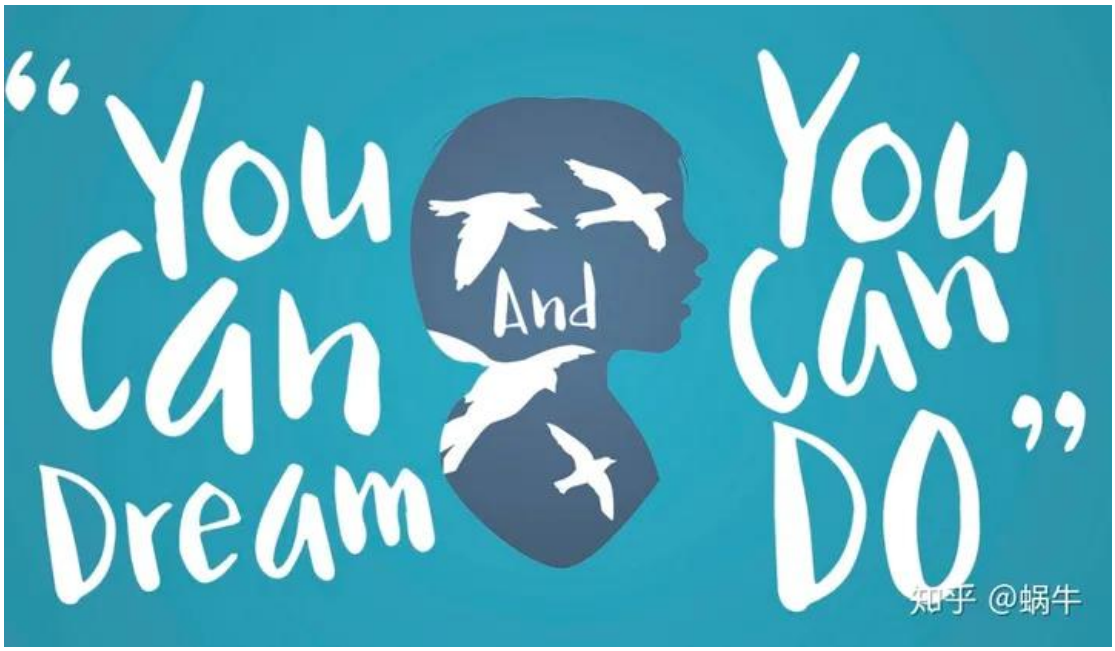
3、对未来是否有帮助，你的判断依据是什么?

4、你的个人商业模式是什么?

回归主题，人到中年，该怎样规划职业生涯？首先要做到的应该是不断审视自己，思考从职业危机中得到了什么，而非失去了什么。当你重新筛选出的自己的核心价值观时，再去着手规划自己的职业：

1.4.1 重新制定整体的职业生涯规划

把目光投向未来，制定 5-10 的长期规划，然后制定一个 3-5 年的可操作性强、详细完备、动态性的中期职业生涯规划。



如果你对现在工作意味着什么，未来的职业的真实性都不清楚时，那你对你来说最重要问题，不是转型，而是把当下的工作做好，在工作中不断地明确未来的人生目标。

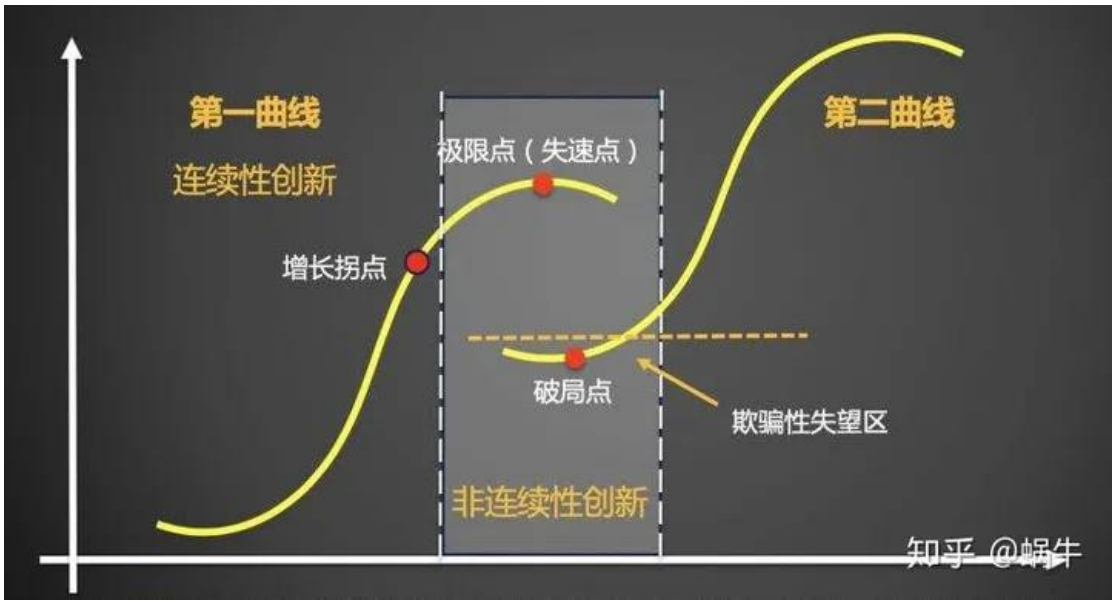
在制定长期规划时，你需要时刻提醒自己：面对快速增长、激荡变化的未来，你今后想要哪种类型的工作生活？什么样的工作或活动会更让你觉得开心？面对不可预测的未来，你未雨绸缪了吗？你有什么样的远大抱负？个人或生活偏好需要再改变什么以更适将来？在职业满意度方面，什么对你来说最重要...

1.4.2 实现思维方式的转变

转型并不是要是全盘否定过去的职业经历，从 0 开始，而是基于新的规划，重新归纳整理过去的经历，梳理出可复用的职场技能、管理经验、人脉资源、思维逻辑等，要相信没有任何一段经历是无用的，多余的，不断的思考，提炼，用唯一的经历打造独有的优势，从技术思维向管理思维转变；

1.4.3 探索职业第二曲线

走出岗位、平台、行业的舒适区，不局限服务于某一企业、行业，打开思路，积极探索职业第二曲线，用最小的试错成本去尝试一些你认为有趣的事情，去找到真正决定 “未来”的小概率事件。



终局思维可以使你考虑问题更加长远和深入，有时会领先同龄人和竞争对手几个维度；终局思维可以很复杂，也可以很简单，只要换一个未来的视角，从最大的问题开始，用倒序思考，你将会发现更多的可能性。

2. 电商平台中订单未支付过期如何实现自动关单?

日常开发中，我们经常遇到这种业务场景，如：外卖订单超 30 分钟未支付，则自动取订单；用户注册成功 15 分钟后，发短信息通知用户等等。这就延时任务处理场景。

在电商，支付等系统中，一设都是先创建订单（支付单）,再给用户一定的时间进行支付，如果没有按时支付的话，就需要把之前的订单（支付单）取消掉。这种类以的场景有很多，还有比如到期自动收货，超时自动退款，下单后自动发送短信等

等都是类似的业务问题。

2.1 定时任务

通过定时任务关闭订单，是一种成本很低，实现也很容易的方案。通过简单的几行代码，写一个定时任务，定期扫描数据库中的订单，如果时间过期，就将其状态更新为关闭即可。

```
@Scheduled(cron = "0 0 22 * * ?")
public void pmNotify() {
    this.pmService.todoNotify();
}
```

优点：实现容易，成本低，基本不依赖其他组件。

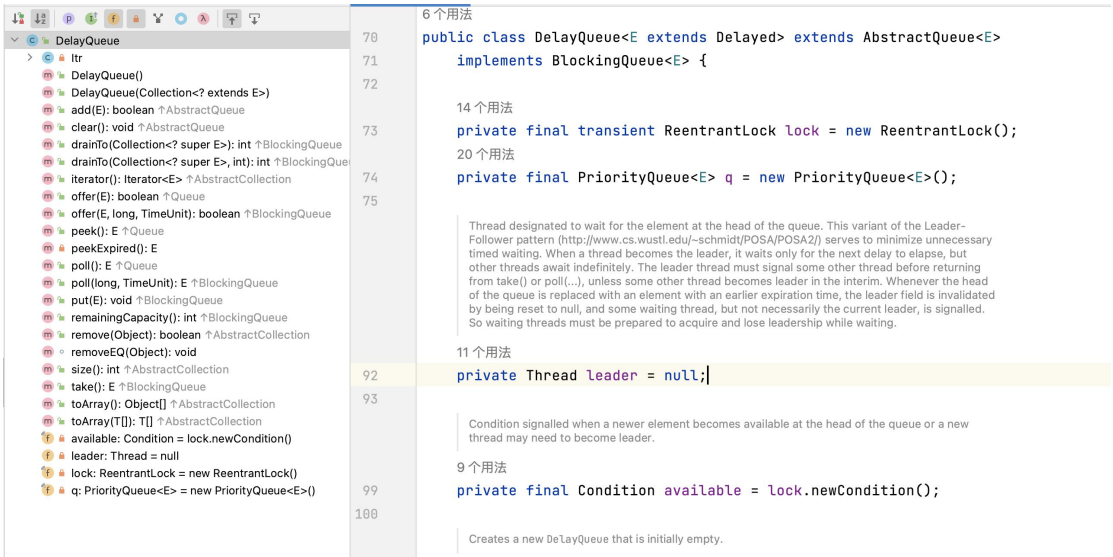
缺点：

时间可能不够精确。由于定时任务扫描的间隔是固定的，所以可能造成一些订单已经过期了一段时间才被扫描到，订单关闭的时间比正常时间晚一些。

增加了数据库的压力。随着订单的数量越来越多，扫描的成本也会越来越大，执行时间也会被拉长，可能导致某些应该被关闭的订单迟迟没有被关闭。

总结：采用定时任务的方案比较适合对时间要求不是很敏感，并且数据量不太多的业务场景。

2.2 JDK 延迟队列 DelayQueue



DelayQueue 是 JDK 提供的一个无界队列，我们可以看到，DelayQueue 队列中的元素需要实现 Delayed，它只提供了一个方法，就是获取过期时间。

```
package java.util.concurrent;

A mix-in style interface for marking objects that should be acted upon after a given delay.
An implementation of this interface must define a compareTo method that provides an ordering consistent
with its getDelay method.
自:  1.5
作者: Doug Lea
public interface Delayed extends Comparable<Delayed> {

    Returns the remaining delay associated with this object, in the given time unit.
    形参:  unit – the time unit
    返回值: the remaining delay; zero or negative values indicate that the delay has already elapsed
    17 个实现
    long getDelay( @NotNull TimeUnit unit);
}
```

用户的订单生成以后，设置过期时间比如 30 分钟，放入定义好的 DelayQueue，然后创建一个线程，在线程中通过 while(true)不断的从 DelayQueue 中获取过期的数据。

优点：不依赖任何第三方组件，连数据库也不需要了，实现起来也方便。

缺点：

因为 DelayQueue 是一个无界队列，如果放入的订单过多，会造成 JVM OOM。

DelayQueue 基于 JVM 内存，如果 JVM 重启了，那所有数据就丢失了。

总结：DelayQueue 适用于数据量较小，且丢失也不影响主业务的场景，比如内部系统的一些非重要通知，就算丢失，也不会有太大影响。

2.3 redis 过期监听

redis 是一个高性能的 KV 数据库，除了用作缓存以外，其实还提供了过期监听的功能。

在 redis.conf 中，配置 notify-keyspace-events Ex 即可开启此功能。

然后在代码中继承 KeyspaceEventMessageListener，实现 onMessage 就可以监听过期的数据量。

```
1  public abstract class KeyspaceEventMessageListener implements MessageListener,
   InitializingBean, DisposableBean {
2
3      private static final Topic TOPIC_ALL_KEYEVENTS = new PatternTopic("__keyevent@*");
```

```
4
5     //...省略部分代码
6     public void init() {
7         if (StringUtils.hasText(keyspaceNotificationsConfigParameter)) {
8             RedisConnection connection =
  listenerContainer.getConnectionFactory().getConnection();
9             try {
10                Properties config = connection.getConfig("notify-keyspace-events");
11                if (!StringUtils.hasText(config.getProperty("notify-keyspace-events"))) {
12                    connection.setConfig("notify-keyspace-events",
  keyspaceNotificationsConfigParameter);
13                }
14            } finally {
15                connection.close();
16            }
17        }
18        doRegister(listenerContainer);
19    }
20
21    protected void doRegister(RedisMessageListenerContainer container) {
22        listenerContainer.addMessageListener(this, TOPIC_ALL_KEYEVENTS);
23    }
24    //...省略部分代码
25    @Override
26    public void afterPropertiesSet() throws Exception {
27        init();
28    }
29 }
30
```

通过以上源码，我们可以发现，其本质也是注册一个 listener，利用 redis 的发布订阅，当 key 过期时，发布过期消息（key）到 Channel ：**keyevent@***:expired 中。在实际的业务中，我们可以将订单的过期时间设置比如 30 分钟，然后放入到 redis。30 分钟之后，就可以消费这个 key，然后做一些业务上的后置动作，比如检查用户是否支付。

优点： 由于 redis 的高性能，所以我们**在设置 key，或者消费 key 时，速度上是可以保证的。**

缺点：由于 redis 的 key 过期策略原因，当一个 key 过期时，redis 无法保证立刻将其删除，自然我们的监听事件也无法第一时间消费到这个 key，所以会存在一定的延迟。另外，在 redis5.0 之前，订阅发布中的消息并没有被持久化，自然也没有所谓

的确认机制。所以一旦消费消息的过程中我们的客户端发生了宕机，这条消息就彻底丢失了。

总结：redis 的过期订阅相比于其他方案没有太大的优势，在实际生产环境中，用得相对较少。

2.4 Redisson 分布式延迟队列

Redisson 是一个基于 redis 实现的 Java 驻内存数据网格，它不仅提供了一系列的分布式的 Java 常用对象，还提供了许多分布式服务。

Redisson 除了提供我们常用的分布式锁外，还提供了一个分布式延迟队列 RDelayedQueue，他是一种基于 zset 结构实现的延迟队列，其实现类是 RedissonDelayedQueue。

```
public interface RDelayedQueue<V> extends RQueue<V>, RDestroyable {

    Inserts element into this queue with specified transfer delay to destination queue.
    形参: e – the element to add
          delay – for transition
          timeUnit – for delay
    1 个实现
    void offer(V e, long delay, TimeUnit timeUnit);

    Inserts element into this queue with specified transfer delay to destination queue.
    形参:  e – the element to add
           delay – for transition
           timeUnit – for delay
    返回值: void
    1 个用法   1 个实现
    RFuture<Void> offerAsync(V e, long delay, TimeUnit timeUnit);

}
```

优点：使用简单，并且其实现类中大量使用 lua 脚本保证其原子性，不会有并发重复问题。

缺点：需要依赖 redis（如果这算一种缺点的话）。

总结：Redisson 是 redis 官方推荐的 JAVA 客户端，提供了很多常用的功能，使用简单、高效，推荐大家尝试使用。

2.5 RocketMQ 延迟消息

> 延迟消息，当消息写入到 Broker 后，不会立刻被消费者消费，需要等待指定的时长后才可被消费处理的消息，称为延时消息。

在订单创建之后，我们就可以把订单作为一条消息投递到 rocketmq，并将延迟时间设置为 30 分钟，这样，30 分钟后我们定义的 consumer 就可以消费到这条消息，然后检查用户是否支付了这个订单。

通过延迟消息，我们就可以将业务解耦，极大地简化我们的代码逻辑。

优点：可以使代码逻辑清晰，系统之间完全解耦，只需关注生产及消费消息即可。另外其吞吐量极高，最多可以支撑万亿级的数据量。

缺点：相对来说 mq 是重量级的组件，引入 mq 之后，随之而来的消息丢失、幂等性问题等都加深了系统的复杂度。

总结：通过 mq 进行系统业务解耦，以及对系统性能削峰填谷已经是当前高性能系统的标配。

2.6 RabbitMQ 死信队列

除了 RocketMQ 的延迟队列，RabbitMQ 的死信队列也可以实现消息延迟功能。

当 RabbitMQ 中的一条正常消息，因为过了存活时间（TTL 过期）、队列长度超限、被消费者拒绝等原因无法被消费时，就会被当成一条死信消息，投递到死信队列。

基于这样的机制，我们可以给消息设置一个 ttl，然后故意不消费消息，等消息过期就会进入死信队列，我们再消费死信队列即可。

通过这样的方式，就可以达到同 RocketMQ 延迟消息一样的效果。

优点：同 RocketMQ 一样，RabbitMQ 同样可以使业务解耦，基于其集群的扩展性，也可以实现高可用、高性能的目标。

缺点：死信队列本质还是一个队列，队列都是先进先出，如果队头的消息过期时间比较长，就会导致后面过期的消息无法得到及时消费，造成消息阻塞。

总结：除了增加系统复杂度之外，死信队列的阻塞问题也是需要我们重点关注的。

3. 如何设计一个秒杀系统

说起秒杀，我想你肯定不陌生，这两年，从双十一购物到春节抢红包，再到 12306

抢火车票，“秒杀”的场景处处可见。简单来说，秒杀就是在同一个时刻有大量的请求争抢购买同一个商品并完成交易的过程，用技术的行话来说就是大量的并发读和并发写。

不管是哪一门语言，并发都是程序员们最为头疼的部分。同样，对于一个软件而言也是这样，你可以很快增删改查做出一个秒杀系统，但是要让它支持高并发访问就没那么容易了。比如说，如何让系统面对百万级的请求流量不出故障？如何保证高并发情况下数据的一致性写？完全靠堆服务器来解决吗？这显然不是最好的解决方案。

在我看来，**秒杀系统本质上就是一个满足大并发、高性能和高可用的分布式系统。**今天，我们就来聊聊，如何在满足一个良好架构的分布式系统基础上，针对秒杀这种业务做到极致的性能改进。

如何才能更好地理解秒杀系统呢？我觉得作为一个程序员，你首先需要从高维度出发，从整体上思考问题。在我看来，**秒杀其实主要解决两个问题，一个是并发读，一个是并发写。**并发读的核心优化理念是尽量减少用户到服务端来“读”数据，或者让他们读更少的数据；并发写的处理原则也一样，它要求我们在数据库层面独立出来一个库，做特殊的处理。另外，我们还要针对秒杀系统做一些保护，针对意料之外的情况设计兜底方案，以防止最坏的情况发生。

而从一个架构师的角度来看，要想打造并维护一个超大流量并发读写、高性能、高可用的系统，在整个用户请求路径上从浏览器到服务端我们要遵循几个原则，就是要保证用户请求的数据尽量少、请求数尽量少、路径尽量短、依赖尽量少，并且不要有单点。这些关键点我会在后面的文章里重点讲解。

其实，秒杀的整体架构可以概括为“稳、准、快”几个关键字。

所谓“稳”，就是整个系统架构要满足高可用，流量符合预期时肯定要稳定，就是超出预期时也同样不能掉链子，你要保证秒杀活动顺利完成，即秒杀商品顺利地卖出去，这个是最基本的前提。

然后就是“准”，就是秒杀 10 台 iPhone，那就只能成交 10 台，多一台少一台都不行。一旦库存不对，那平台就要承担损失，所以“准”就是要求保证数据的一致性。

最后再看“快”，“快”其实很好理解，它就是说系统的性能要足够高，否则你怎么支撑这么大的流量呢？不光是服务端要做极致的性能优化，而且在整个请求链路上都要做协同的优化，每个地方快一点，整个系统就完美了。

所以从技术角度上看“稳、准、快”，就对应了我们架构上的高可用、一致性和高性能的要求，我们的专栏也将主要围绕这几个方面来展开，具体如下。

- **高性能。** 秒杀涉及大量的并发读和并发写，因此支持高并发访问这点非常关键。本专栏将从设计数据的动静分离方案、热点的发现与隔离、请求的削峰与分层过滤、服务端的极致优化这 4 个方面重点介绍。
- **一致性。** 秒杀中商品减库存的实现方式同样关键。可想而知，有限数量的商品在同一时刻被很多倍的请求同时来减库存，减库存又分为“拍下减库存”“付款减库存”以及预扣等几种，在大并发更新的过程中都要保证数据的准确性，其难度可想而知。因此，我将用一篇文章来专门讲解如何设计秒杀减库存方案。
- **高可用。** 虽然我介绍了很多极致的优化思路，但现实中总难免出现一些我们考虑不到的情况，所以要保证系统的高可用和正确性，我们还要设计一个 PlanB 来兜底，以便在最坏情况发生时仍然能够从容应对。专栏的最后，我将带你思考可以从哪些环节来设计兜底方案。

3.1 设计秒杀系统应该注意的 5 个架构原则

不管是哪一门语言，并发都是程序员们最为头疼的部分。同样，对于一个软件而言也是这样，你可以很快增删改查做出一个秒杀系统，但是要让它支持高并发访问就没那么容易了。比如说，如何让系统面对百万级的请求流量不出故障？如何保证高并发情况下数据的一致性写？完全靠堆服务器来解决吗？这显然不是最好的解决方案。

在我看来，**秒杀系统本质上就是一个满足大并发、高性能和高可用的分布式系统。**今天，我们就来聊聊，如何在满足一个良好架构的分布式系统基础上，针对秒杀这种业务做到极致的性能改进。

3.1.1 架构原则：“4 要 1 不要”

如果你是一个架构师，你首先要勾勒出一个轮廓，想一想如何构建一个超大流量并发读写、高性能，以及高可用的系统，这其中有哪些要素需要考虑。我把这些要素总结为“4要 1 不要”。

1. **数据要尽量少**

所谓“数据要尽量少”，首先是指用户请求的数据能少就少。请求的数据包括上传给系统的数据和系统返回给用户的数据（通常就是网页）。

为啥“数据要尽量少”呢？因为首先这些数据在网络上传输需要时间，其次不管是请求数据还是返回数据都需要服务器做处理，而服务器在写网络时通常都要做压缩和字符编码，这些都非常消耗 CPU，所以减少传输的数据量可以显著减少 CPU 的使用。例如，我们可以简化秒杀页面的大小，去掉不必要的页面装修效果，等等。

其次，“数据要尽量少”还要求系统依赖的数据能少就少，包括系统完成某些业务逻辑需要读取和保存的数据，这些数据一般是和后台服务以及数据库打交道的。调用其他服务会涉及数据的序列化和反序列化，而这也是 CPU 的一大杀手，同样也会增加延时。而且，数据库本身也容易成为一个瓶颈，所以和数据库打交道越少越好，数据越简单、越小则越好。

2. **请求数要尽量少**

用户请求的页面返回后，浏览器渲染这个页面还要包含其他的额外请求，比如说，这个页面依赖的 CSS/JavaScript、图片，以及 Ajax 请求等等都定义为“额外请求”，这些额外请求应该尽量少。因为浏览器每发出一个请求都多少会有一些消耗，例如建立连接要做三次握手，有的时候有页面依赖或者连接数限制，一些请求（例如 JavaScript）还需要串行加载等。另外，如果不同请求的域名不一样的话，还涉及这些域名的 DNS 解析，可能会耗时更久。所以你要记住的是，减少请求数可以显著减少以上这些因素导致的资源消耗。

例如，减少请求数最常用的一个实践就是合并 CSS 和 JavaScript 文件，把多个 JavaScript 文件合并成一个文件，在 URL 中用逗号隔开（https://g.xxx.com/tm/xx-b/4.0.94/mods/??module-preview/index.xtpl.js,module-jhs/index.xtpl.js,module-focus/index.xtpl.js）。这种方式在服务端仍然是单个文件各自存放，只是服务端会有一个组件解析这个 URL，然后动态把这些文件合并起来一起返回。

3. **路径要尽量短**

所谓“路径”，就是用户发出请求到返回数据这个过程中，需求经过的中间的节点数。通常，这些节点可以表示为一个系统或者一个新的 Socket 连接（比如代理服务器只是创建一个新的 Socket 连接来转发请求）。每经过一个节点，一般都会产生一个新的 Socket 连接。

然而，每增加一个连接都会增加新的不确定性。从概率统计上来说，假如一次请求经过 5 个节点，每个节点的可用性是 99.9%的话，那么整个请求的可用性是：99.9%的 5 次方，约等于 99.5%。

所以缩短请求路径不仅可以增加可用性，同样可以有效提升性能（减少中间节点可以减少数据的序列化与反序列化），并减少延时（可以减少网络传输耗时）。

要缩短访问路径有一种办法，就是多个相互强依赖的应用合并部署在一起，把远程过程调用（RPC）变成 JVM 内部之间的方法调用。在《大型网站技术架构演进与性能优化》一书中，我也有一章介绍了这种技术的详细实现。

4. **依赖要尽量少**

所谓依赖，指的是要完成一次用户请求必须依赖的系统或者服务，这里的依赖指的是强依赖。

举个例子，比如说你要展示秒杀页面，而这个页面必须强依赖商品信息、用户信息，还有其他如优惠券、成交列表等这些对秒杀不是非要不可的信息（弱依赖），这些弱依赖在紧急情况下就可以去掉。

要减少依赖，我们可以给系统进行分级，比如 0 级系统、1 级系统、2 级系统、3 级系统，0 级系统如果是最重要的系统，那么 0 级系统强依赖的系统也同样是最重要的系统，以此类推。

注意，0 级系统要尽量减少对 1 级系统的强依赖，防止重要的系统被不重要的系统拖垮。例如支付系统是 0 级系统，而优惠券是 1 级系统的话，在极端情况下可以把优惠券给降级，防止支付系统被优惠券这个 1 级系统给拖垮。

5. **不要有单点**

系统中的单点可以说是系统架构上的一个大忌，因为单点意味着没有备份，风险不可控，我们设计分布式系统最重要的原则就是“消除单点”。

那如何避免单点呢？我认为关键点是避免将服务的状态和机器绑定，即把服务无状态化，这样服务就可以在机器中随意移动。

如何那把服务的状态和机器解耦呢？这里也有很多实现方式。例如把和机器相关的配置动态化，这些参数可以通过配置中心来动态推送，在服务启动时动态拉取下来，我们在这些配置中心设置一些规则来方便地改变这些映射关系。

应用无状态化是有效避免单点的一种方式，但是像存储服务本身很难无状态化，因为数据要存储在磁盘上，本身就要和机器绑定，那么这种场景一般要通过冗余多个备份的方式来解决单点问题。

前面介绍了这些设计上的一些原则，但是你有没有发现，我一直说的是“尽量”而不是“绝对”？

我想你肯定会问是不是请求最少就一定最好，我的答案是“不一定”。我们曾经把有些 CSS 内联进页面里，这样做可以减少依赖一个 CSS 的请求从而加快首页的渲染，但是同样也增大了页面的大小，又不符合“数据要尽量少”的原则，这种情况下我们为了提升首屏的渲染速度，只把首屏的 HTML 依赖的 CSS 内联进来，其他 CSS 仍然放到文件中作为依赖加载，尽量实现首屏的打开速度与整个页面加载性能的平衡。

所以说，**架构是一种平衡的艺术，而最好的架构一旦脱离了它所适应的场景，一切都将是空谈。**我希望你记住的是，这里所说的几点都只是一个个方向，你应该尽量往这些方向上去努力，但也要考虑平衡其他因素。

3.1.2 不同场景下的不同架构案例

前面我说了一些架构上的原则，那么针对“秒杀”这个场景，怎样才是一个好的架构呢？下面我以淘宝早期秒杀系统架构的演进为主线，来帮你梳理不同的请求体量下，我认为的最佳秒杀系统架构。

如果你想快速搭建一个简单的秒杀系统，只需要把你的商品购买页面增加一个“定时上架”功能，仅在秒杀开始时才让用户看到购买按钮，当商品的库存卖完了也就结束了。这就是当时第一个版本的秒杀系统实现方式。

但随着请求量的加大（比如从 1w/s 到了 10w/s 的量级），这个简单的架构很快就遇到了瓶颈，因此需要做架构改造来提升系统性能。这些架构改造包括：

1. 把秒杀系统独立出来单独打造一个系统，这样可以有针对性地做优化，例如这个独立出来的系统就减少了店铺装修的功能，减少了页面的复杂度；
2. 在系统部署上也独立做一个机器集群，这样秒杀的大流量就不会影响到正常的商品购买集群的机器负载；
3. 将热点数据（如库存数据）单独放到一个缓存系统中，以提高“读性能”；
4. 增加秒杀答题，防止有秒杀器抢单。

此时的系统架构变成了下图这个样子。最重要的就是，秒杀详情成为了一个独立的新系统，另外核心的一些数据放到了缓存（Cache）中，其他的关联系统也都以独立集群的方式进行部署。

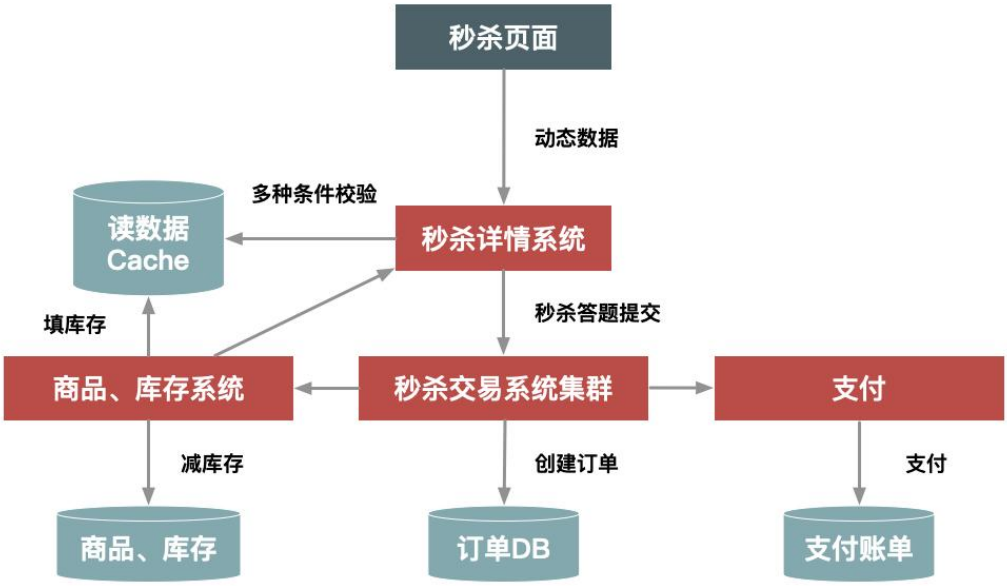


图 1 改造后的系统架构

然而这个架构仍然支持不了超过 100w/s 的请求量，所以为了进一步提升秒杀系统的性能，我们又对架构做进一步升级，比如：

5. 对页面进行彻底的动静分离，使得用户秒杀时不需要刷新整个页面，而只需要点击抢宝按钮，借此把页面刷新的数据降到最少；

6. 在服务端对秒杀商品进行本地缓存，不需要再调用依赖系统的后台服务获取数据，甚至不需要去公共的缓存集群中查询数据，这样不仅可以减少系统调用，而且能够避免压垮公共缓存集群。
7. 增加系统限流保护，防止最坏情况发生。

经过这些优化，系统架构变成了下图中的样子。在这里，我们对页面进行了进一步的静态化，秒杀过程中不需要刷新整个页面，而只需要向服务端请求很少的动态数据。而且，最关键的详情和交易系统都增加了本地缓存，来提前缓存秒杀商品的信息，热点数据库也做了独立部署，等等。

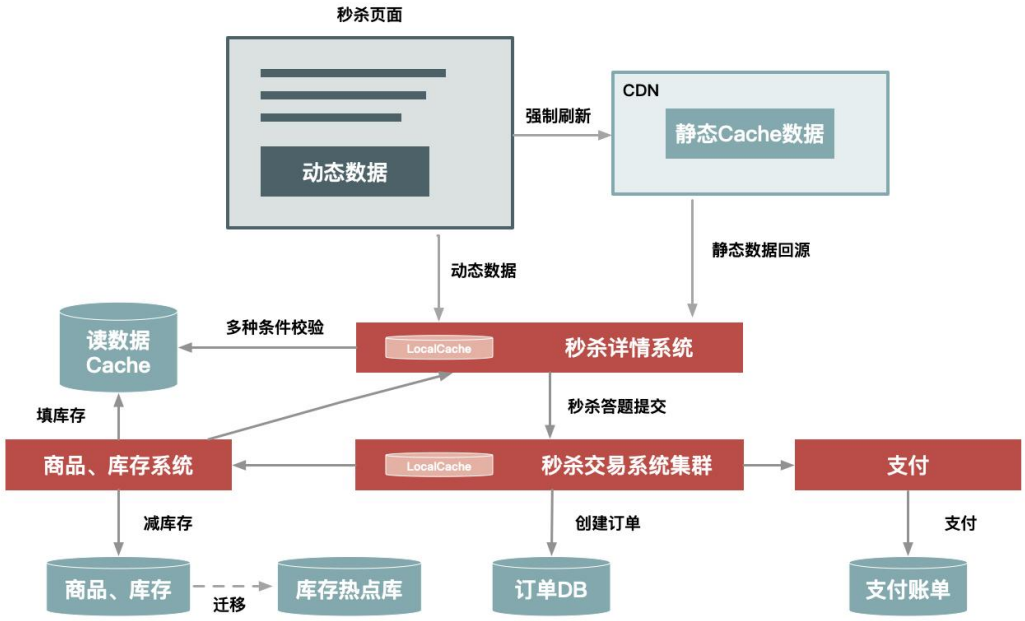


图 2 进一步改造后的系统架构

从前面的几次升级来看，其实越到后面需要定制的地方越多，也就是越“不通用”。例如，把秒杀商品缓存在每台机器的内存中，这种方式显然不适合太多的商品同时进行秒杀的情况，因为单机的内存始终有限。所以要取得极致的性能，就要在其他地方（比如，通用性、易用性、成本等方面）有所牺牲。

3.2 如何才能做好动静分离？有哪些方案可选？

如果你在一个业务飞速发展的公司里，并且你在深度参与公司内类秒杀类系统的架构或者开发工作，那么你迟早会想到动静分离的方案。为什么？很简单，**秒杀的场景中，对于系统的要求其实就三个字：快、准、稳。**

那怎么才能“快”起来呢？我觉得抽象起来讲，就只有两点，一点是提高单次请求的效率，一点是减少没必要的请求。今天我们聊到的“动静分离”其实就是瞄着这个大方向去的。

3.2.1 何为动静数据

那到底什么才是动静分离呢？所谓“动静分离”，其实就是把用户请求的数据（如 HTML 页面）划分为“动态数据”和“静态数据”。

简单来说，**“动态数据”和“静态数据”的主要区别就是看页面中输出的数据是否和 URL、浏览者、时间、地域相关，以及是否含有 Cookie 等私密数据。**比如说：

1. 很多媒体类的网站，某一篇文章的内容不管是你访问还是我访问，它都是一样的。所以它就是一个典型的静态数据，但是它是个动态页面。
2. 我们如果现在访问淘宝的首页，每个人看到的页面可能都是不一样的，淘宝首页中包含了很多根据访问者特征推荐的信息，而这些个性化的数据就可以理解为动态数据了。

这里再强调一下，我们所说的静态数据，不能仅仅理解为传统意义上完全存在磁盘上的 HTML 页面，它也可能是经过 Java 系统产生的页面，但是它输出的页面本身不包含上面所说的那些因素。也就是所谓“动态”还是“静态”，并不是说数据本身是否动静，而是数据中是否含有和访问者相关的个性化数据。

还有一点要注意，就是页面中“不包含”，指的是“页面的 HTML 源码中不含有”，这一点务必要清楚。

理解了静态数据和动态数据，我估计你很容易就能想明白“动静分离”这个方案的来龙去脉了。分离了动静数据，我们就可以对分离出来的静态数据做缓存，有了缓存之后，静态数据的“访问效率”自然就提高了。

那么，怎样对静态数据做缓存呢？我在这里总结了几个重点。

第一，你应该把静态数据缓存到离用户最近的地方。静态数据就是那些相对不会变化的数据，因此我们可以把它们缓存起来。缓存到哪里呢？常见的就三种，用户浏览器里、CDN 上或者在服务端的 Cache 中。你应该根据情况，把它们尽量缓存到离用户最近的地方。

第二，静态化改造就是要直接缓存 HTTP 连接。相较于普通的数据缓存而言，你肯定还听过系统的静态化改造。静态化改造是直接缓存 HTTP 连接而不是仅仅缓存数据，如下图所示，Web 代理服务器根据请求 URL，直接取出对应的 HTTP 响应头和响应体然后直接返回，这个响应过程简单得连 HTTP 协议都不用重新组装，甚至连 HTTP 请求头也不需要解析。

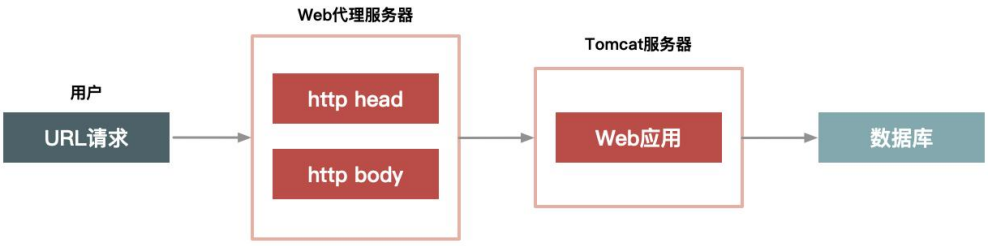


图 1 静态化改造

第三，让谁来缓存静态数据也很重要。不同语言写的 Cache 软件处理缓存数据的效率也各不相同。以 Java 为例，因为 Java 系统本身也有其弱点（比如不擅长处理大量连接请求，每个连接消耗的内存较多，Servlet 容器解析 HTTP 协议较慢），所以你可以不在 Java 层做缓存，而是直接在 Web 服务器层上做，这样你就可以屏蔽 Java 语言层面的一些弱点；而相比起来，Web 服务器（如 Nginx、Apache、Varnish）也更擅长处理大并发的静态文件请求。

3.2.2 如何做动静分离的改造

理解了动静态数据的“why”和“what”，接下来我们就要看“how”了。我们如何把动态页面改造成适合缓存的静态页面呢？其实也很简单，就是去除前面所说的那几个影响因素，把它们单独分离出来，做动静分离。

下面，我以典型的商品详情系统为例来详细介绍。这里，你可以先打开京东或者淘宝的商品详情页，看看这个页面里都有哪些动静数据。我们从以下 5 个方面来分离出动态内容。

1. **URL 唯一化。**商品详情系统天然地就可以做到 URL 唯一化，比如每个商品都由 ID 来标识，那么 http://item.xxx.com/item.htm?id=xxxx 就可以作为唯一的 URL 标

识。为啥要 URL 唯一呢？前面说了我们是要缓存整个 HTTP 连接，那么以什么作为 Key 呢？就以 URL 作为缓存的 Key，例如以 id=xxx 这个格式进行区分。

2. **分离浏览者相关的因素。**浏览者相关的因素包括是否已登录，以及登录身份等，这些相关因素我们可以单独拆分出来，通过动态请求来获取。
3. **分离时间因素。**服务端输出的时间也通过动态请求获取。
4. **异步化地域因素。**详情页面上与地域相关的因素做成异步方式获取，当然你也可以通过动态请求方式获取，只是这里通过异步获取更合适。
5. **去掉 Cookie。**服务端输出的页面包含的 Cookie 可以通过代码软件来删除，如 Web 服务器 Varnish 可以通过 unset req.http.cookie 命令去掉 Cookie。注意，这里说的去掉 Cookie 并不是用户端收到的页面就不含 Cookie 了，而是说，在缓存的静态数据中不含有 Cookie。

分离出动态内容之后，如何组织这些内容页就变得非常关键了。这里我要提醒你一点，因为这其中很多动态内容都会被页面中的其他模块用到，如判断该用户是否已登录、用户 ID 是否匹配等，所以这个时候我们应该将这些信息 JSON 化（用 JSON 格式组织这些数据），以方便前端获取。

前面我们介绍里用缓存的方式来处理静态数据。而动态内容的处理通常有两种方案：ESI（Edge Side Includes）方案和 CSI（Client Side Include）方案。

6. **ESI 方案（或者 SSI）**：即在 Web 代理服务器上做动态内容请求，并将请求插入到静态页面中，当用户拿到页面时已经是一个完整的页面了。这种方式对服务端性能有些影响，但是用户体验较好。
7. **CSI 方案。**即单独发起一个异步 JavaScript 请求，以向服务端获取动态内容。这种方式服务端性能更佳，但是用户端页面可能会延时，体验稍差。

3.2.3 动静分离的几种架构方案

前面我们通过改造把静态数据和动态数据做了分离，那么如何在系统架构上进一步对这些动态和静态数据重新组合，再完整地输出给用户呢？

这就涉及对用户请求路径进行合理的架构了。根据架构上的复杂度，有 3 种方案可选：

1. 实体机单机部署；
2. 统一 Cache 层；

3. 上 CDN。

方案 1：实体机单机部署

这种方案是将虚拟机改为实体机，以增大 Cache 的容量，并且采用了一致性 Hash 分组的方式来提升命中率。这里将 Cache 分成若干组，是希望能达到命中率和访问热点的平衡。Hash 分组越少，缓存的命中率肯定就会越高，但短板是也会使单个商品集中在一个分组中，容易导致 Cache 被击穿，所以我们应该适当增加多个相同的分组，来平衡访问热点和命中率的问题。

这里我给出了实体机单机部署方案的结构图，如下：

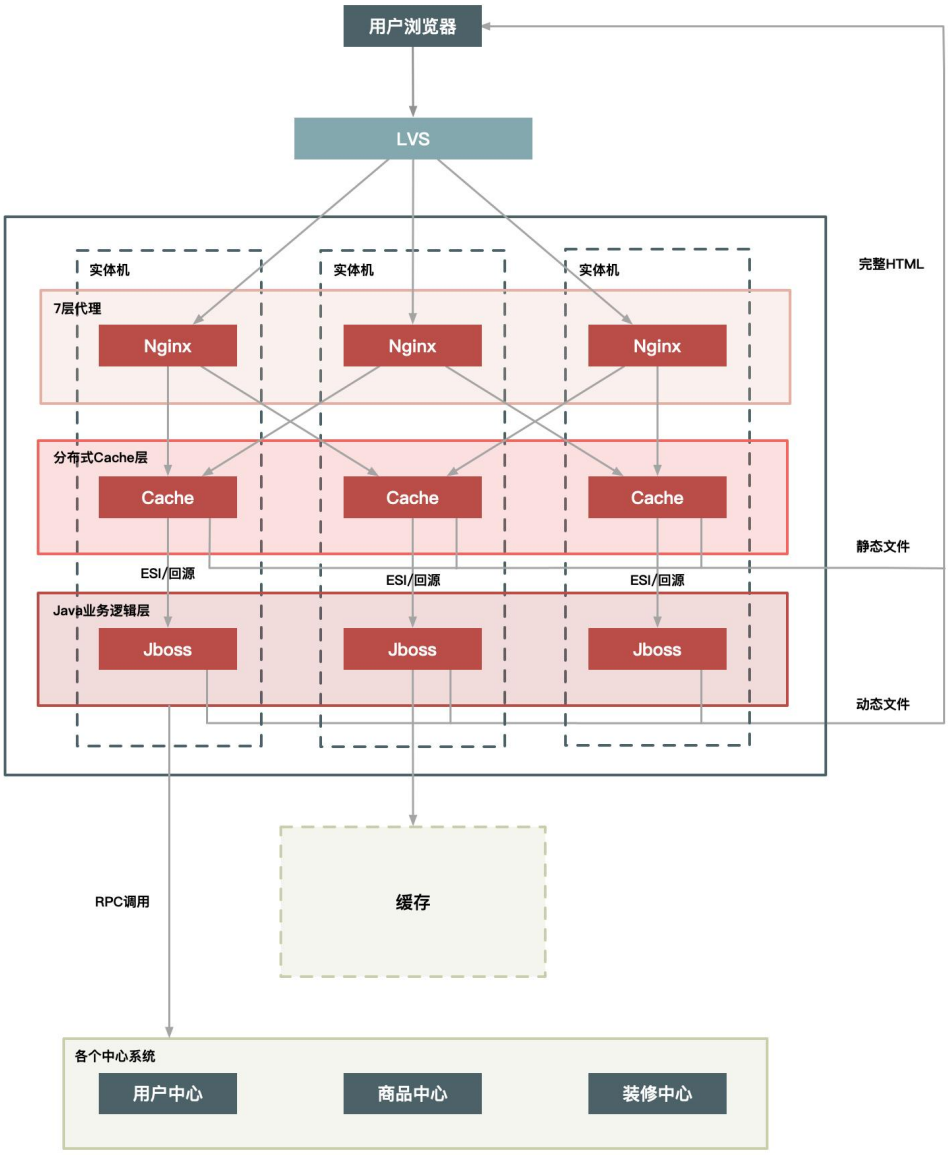


图 2 Nginx+Cache+Java 结构实体机单机部署

实体机单机部署有以下几个优点：

4. 没有网络瓶颈，而且能使用大内存；
5. 既能提升命中率，又能减少 Gzip 压缩；
6. 减少 Cache 失效压力，因为采用定时失效方式，例如只缓存 3 秒钟，过期即自动失效。

这个方案中，虽然把通常只需要虚拟机或者容器运行的 Java 应用换成实体机，优势很明显，它会增加单机的内存容量，但是一定程度上也造成了 CPU 的浪费，因为单个的 Java 进程很难用完整个实体机的 CPU。

另外就是，一个实体机上部署了 Java 应用又作为 Cache 来使用，这造成了运维上的高复杂度，所以这是一个折中的方案。如果你的公司里，没有更多的系统有类似需求，那么这样做也比较合适，如果你们有多个业务系统都有静态化改造的需求，那还是建议把 Cache 层单独抽出来公用比较合理，如下面的方案 2 所示。

方案 2：统一 Cache 层

所谓统一 Cache 层，就是将单机的 Cache 统一分离出来，形成一个单独的 Cache 集群。统一 Cache 层是个更理想的可推广方案，该方案的结构图如下：

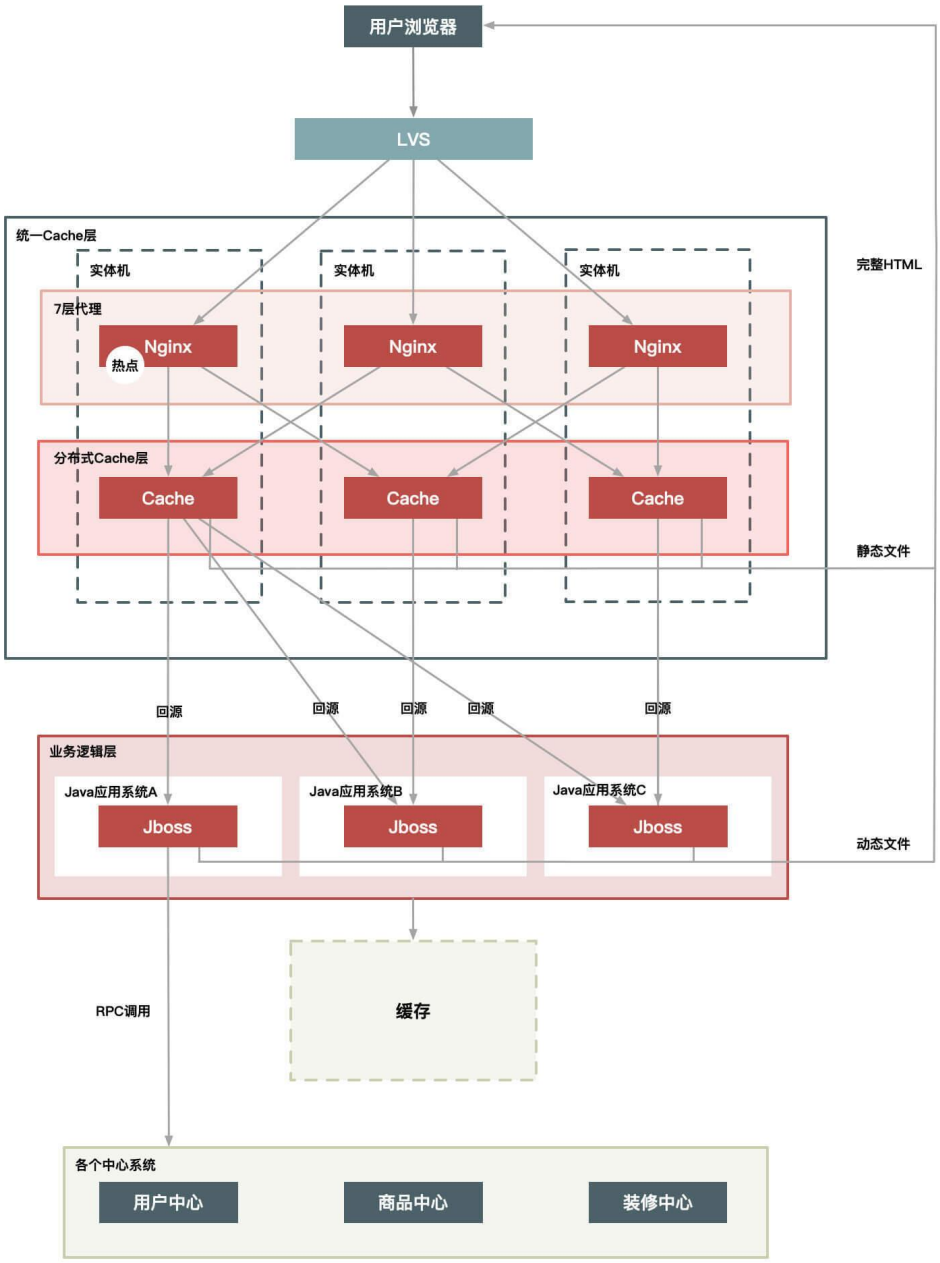


图 3 统一 Cache

将 Cache 层单独拿出来统一管理可以减少运维成本，同时也方便接入其他静态化系统。此外，它还有一些优点。

7. 单独一个 Cache 层，可以减少多个应用接入时使用 Cache 的成本。这样接入的应用只要维护自己的 Java 系统就好，不需要单独维护 Cache，而只关心如何使用即可。

8. 统一 Cache 的方案更易于维护，如后面加强监控、配置的自动化，只需要一套解决方案就行，统一起来维护升级也比较方便。
9. 可以共享内存，最大化利用内存，不同系统之间的内存可以动态切换，从而能够有效应对各种攻击。

这种方案虽然维护上更方便了，但是也带来了其他一些问题，比如缓存更加集中，导致：

10. Cache 层内部交换网络成为瓶颈；
11. 缓存服务器的网卡也会是瓶颈；
12. 机器少风险较大，挂掉一台就会影响很大一部分缓存数据。

要解决上面这些问题，可以再对 Cache 做 Hash 分组，即一组 Cache 缓存的内容相同，这样能够避免热点数据过度集中导致新的瓶颈产生。

方案 3：上 CDN

在将整个系统做动静分离后，我们自然会想到更进一步的方案，就是将 Cache 进一步前移到 CDN 上，因为 CDN 离用户最近，效果会更好。

但是要想这么做，有以下几个问题需要解决。

13. **失效问题。**前面我们也有提到过缓存时效的问题，不知道你有没有理解，我再来解释一下。谈到静态数据时，我说过一个关键词叫“相对不变”，它的言外之意是“可能会变化”。比如一篇文章，现在不变，但如果你发现个错别字，是不是就会变化了？如果你的缓存时效很长，那用户端在很长一段时间内看到的都是错的。所以，这个方案中也是，我们需要保证 CDN 可以在秒级时间内，让分布在全国各地的 Cache 同时失效，这对 CDN 的失效系统要求很高。
14. **命中率问题。**Cache 最重要的一个衡量指标就是“高命中率”，不然 Cache 的存在就失去了意义。同样，如果将数据全部放到全国的 CDN 上，必然导致 Cache 分散，而 Cache 分散又会导致访问请求命中同一个 Cache 的可能性降低，那么命中率就成为一个问题。
15. **发布更新问题。**如果一个业务系统每周都有日常业务需要发布，那么发布系统必须足够简洁高效，而且你还要考虑有问题时快速回滚和排查问题的简便性。

从前面的分析来看，将商品详情系统放到全国的所有 CDN 节点上是不太现实的，因为存在失效问题、命中率问题以及系统的发布更新问题。那么是否可以选择若干个节点来尝试实施呢？答案是“可以”，但是这样的节点需要满足几个条件：

16. 靠近访问量比较集中的地区；

17. 离主站相对较远；

18. 节点到主站间的网络比较好，而且稳定；

19. 节点容量比较大，不会占用其他 CDN 太多的资源。

最后，还有一点也很重要，那就是：节点不要太多。

基于上面几个因素，选择 CDN 的二级 Cache 比较合适，因为二级 Cache 数量偏少，容量也更大，让用户的请求先回源的 CDN 的二级 Cache 中，如果没命中再回源站获取数据，部署方式如下图所示：

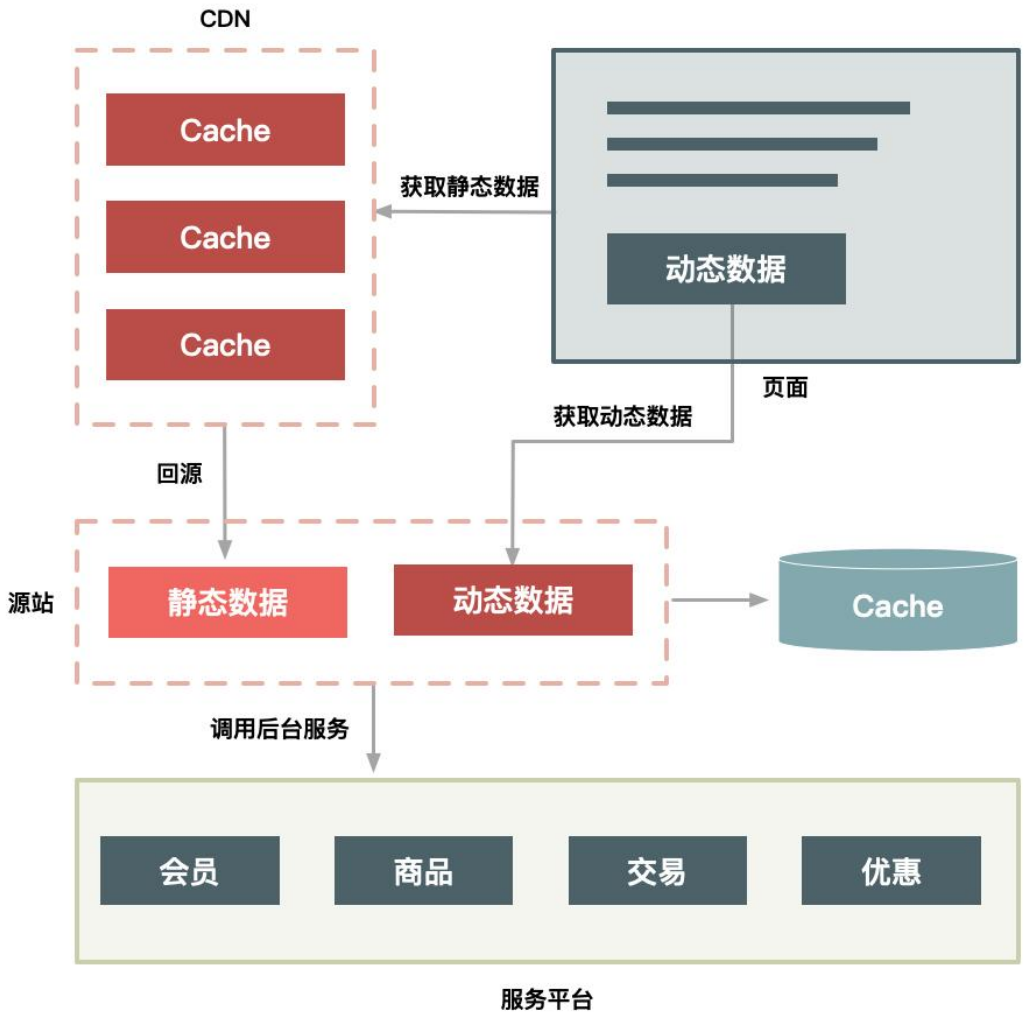


图 4 CDN 化部署方案

使用 CDN 的二级 Cache 作为缓存，可以达到和当前服务端静态化 Cache 类似的命中率，因为节点数不多，Cache 不是很分散，访问量也比较集中，这样也就解决了命中率问题，同时能够给用户最好的访问体验，是当前比较理想的一种 CDN 化方案。

除此之外，CDN 化部署方案还有以下几个特点：

20. 把整个页面缓存在用户浏览器中；
21. 如果强制刷新整个页面，也会请求 CDN；
22. 实际有效请求，只是用户对“刷新抢宝”按钮的点击。

这样就把 90%的静态数据缓存在了用户端或者 CDN 上，当真正秒杀时，用户只需要点击特殊的“刷新抢宝”按钮，而不需要刷新整个页面。这样一来，系统只是向服务端请求很少的有效数据，而不需要重复请求大量的静态数据。

秒杀的动态数据和普通详情页面的动态数据相比更少，性能也提升了 3 倍以上。所以“抢宝”这种设计思路，让我们不用刷新页面就能够很好地请求到服务端最新的动态数据。

3.3 有针对性地处理好系统的“热点数据”

假设你的系统中存储有几十亿上百亿的商品，而每天有千万级的商品被上亿的用户访问，那么肯定有一部分被大量用户访问的热卖商品，这就是我们常说的“热点商品”。这些热点商品中最极端的例子就是秒杀商品，它们在很短时间内被大量用户执行访问、添加购物车、下单等操作，这些操作我们就称为“热点操作”。那么问题来了：这些热点对系统有啥影响，我们非要关注这些热点吗？

3.3.1 为什么要关注热点

我们一定要关注热点，因为热点会对系统产生一系列的影响。

首先，热点请求会大量占用服务器处理资源，虽然这个热点可能只占请求总量的亿分之一，然而却可能抢占 90%的服务器资源，如果这个热点请求还是没有价值的无效请求，那么对系统资源来说完全是浪费。

其次，即使这些热点是有效的请求，我们也要识别出来做针对性的优化，从而用更低的代价来支撑这些热点请求。

既然热点对系统来说这么重要，那么热点到底包含哪些内容呢？

3.3.2 什么是“热点”

热点分为**热点操作**和**热点数据**。所谓“热点操作”，例如大量的刷新页面、大量的添加购物车、双十一零点大量的下单等都属于此类操作。对系统来说，这些操作可以抽象为“读请求”和“写请求”，这两种热点请求的处理方式大相径庭，读请求的优化空间要大一些，而写请求的瓶颈一般都在存储层，优化的思路就是根据 CAP 理论做平衡，这个内容我在“减库存”一文再详细介绍。

而“热点数据”比较好理解，那就是用户的热点请求对应的数据。而热点数据又分为“静态热点数据”和“动态热点数据”。

所谓“静态热点数据”，就是能够提前预测的热点数据。例如，我们可以通过卖家报名的方式提前筛选出来，通过报名系统对这些热点商品进行打标。另外，我们还可以通过大数据分析来提前发现热点商品，比如我们分析历史成交记录、用户的购物车记录，来发现哪些商品可能更热门、更好卖，这些都是可以提前分析出来的热点。

所谓“动态热点数据”，就是不能被提前预测到的，系统在运行过程中临时产生的热点。例如，卖家在抖音上做了广告，然后商品一下就火了，导致它在短时间内被大量购买。

由于热点操作是用户的行为，我们不好改变，但能做一些限制和保护，所以本文我主要针对热点数据来介绍如何进行优化。

3.3.3 发现热点数据

前面，我介绍了如何对单个秒杀商品的页面数据进行动静分离，以便针对性地对静态数据做优化处理，那么另外一个关键的问题来了：如何发现这些秒杀商品，或者更准确地说，如何发现热点商品呢？

你可能会说“参加秒杀的商品就是秒杀商品啊”，没错，关键是系统怎么知道哪些商品参加了秒杀活动呢？所以，你要有一个机制提前来区分普通商品和秒杀商品。

我们从发现静态热点和发现动态热点两个方面来看一下。

发现静态热点数据

如前面讲的，静态热点数据可以通过商业手段，例如强制让卖家通过报名参加的方式提前把热点商品筛选出来，实现方式是通过一个运营系统，把参加活动的商品数据进行打标，然后通过一个后台系统对这些热点商品进行预处理，如提前进行缓存。但是这种通

过报名提前筛选的方式也会带来新的问题，即增加卖家的使用成本，而且实时性较差，也不太灵活。

不过，除了提前报名筛选这种方式，你还可以通过技术手段提前预测，例如对买家每天访问的商品进行大数据计算，然后统计出 TOP N 的商品，我们可以认为这些 TOP N 的商品就是热点商品。

发现动态热点数据

我们可以通过卖家报名或者大数据预测这些手段来提前预测静态热点数据，但这其中有一个痛点，就是实时性较差，如果我们的系统能在秒级内自动发现热点商品那就完美了。能够动态地实时发现热点不仅对秒杀商品，对其他热卖商品也同样有价值，所以我们需要想办法实现热点的动态发现功能。

这里我给出一个动态热点发现系统的具体实现。

1. 构建一个异步的系统，它可以收集交易链路上各个环节中的中间件产品的热点 Key，如 Nginx、缓存、RPC 服务框架等这些中间件（一些中间件产品本身已经有热点统计模块）。
2. 建立一个热点上报和可以按照需求订阅的热点服务的下发规范，主要目的是通过交易链路上各个系统（包括详情、购物车、交易、优惠、库存、物流等）访问的时间差，把上游已经发现的热点透传给下游系统，提前做好保护。比如，对于大促高峰期，详情系统是最早知道的，在统一接入层上 Nginx 模块统计的热点 URL。
3. 将上游系统收集的热点数据发送到热点服务台，然后下游系统（如交易系统）就会知道哪些商品会被频繁调用，然后做热点保护。

这里我给出了一个图，其中用户访问商品时经过的路径有很多，我们主要是依赖前面的导购页面（包括首页、搜索页面、商品详情、购物车等）提前识别哪些商品的访问量高，通过这些系统中的中间件来收集热点数据，并记录到日志中。

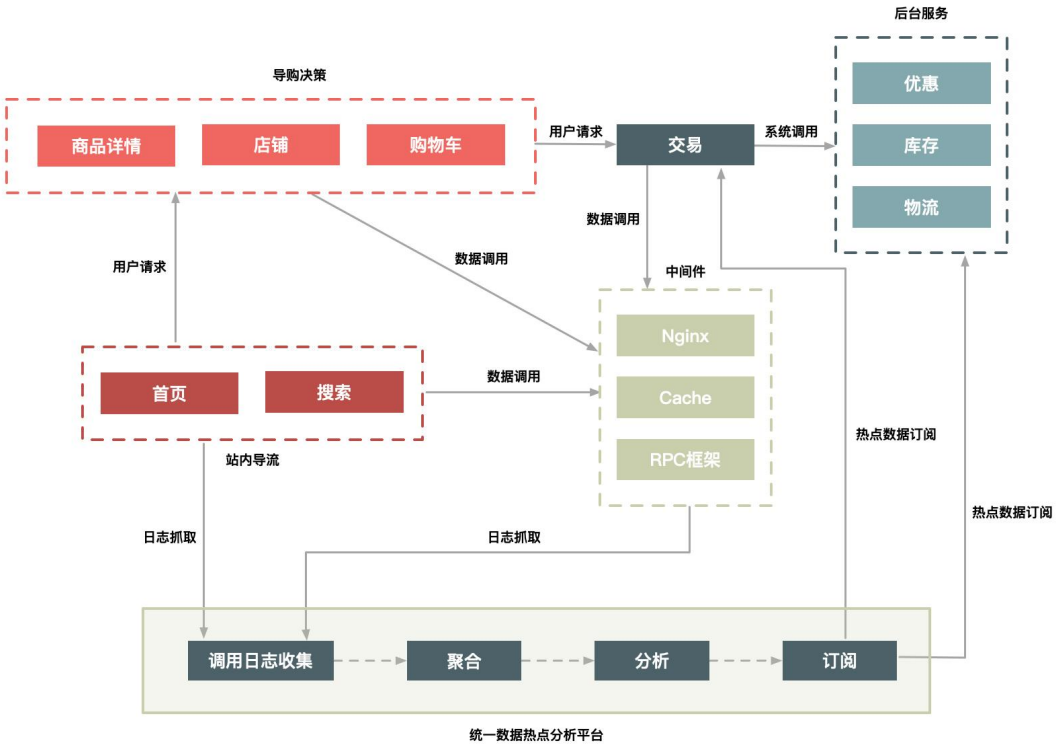


图 1 一个动态热点发现系统

我们通过部署在每台机器上的 Agent 把日志汇总到聚合和分析集群中，然后把符合一定规则的热点数据，通过订阅分发系统再推送到相应的系统中。你可以是把热点数据填充到 Cache 中，或者直接推送到应用服务器的内存中，还可以对这些数据进行拦截，总之下游系统可以订阅这些数据，然后根据自己的需求决定如何处理这些数据。

打造热点发现系统时，我根据以往经验总结了几点注意事项。

4. 这个热点服务后台抓取热点数据日志最好采用异步方式，因为“异步”一方面便于保证通用性，另一方面又不影响业务系统和中间件产品的主流程。
5. 热点服务发现和中间件自身的热点保护模块并存，每个中间件和应用还需要保护自己。热点服务台提供热点数据的收集和订阅服务，便于把各个系统的热点数据透明出来。
6. 热点发现要做到接近实时（3s 内完成热点数据的发现），因为只有做到接近实时，动态发现才有意义，才能实时地对下游系统提供保护。

3.3.4 处理热点数据

处理热点数据通常有几种思路：一是优化，二是限制，三是隔离。

先来说说优化。优化热点数据最有效的办法就是缓存热点数据，如果热点数据做了动静分离，那么可以长期缓存静态数据。但是，缓存热点数据更多的是“临时”缓存，即不管是静态数据还是动态数据，都用一个队列短暂地缓存数秒钟，由于队列长度有限，可以采用 LRU 淘汰算法替换。

再来说说限制。限制更多的是一种保护机制，限制的办法也有很多，例如对被访问商品的 ID 做一致性 Hash，然后根据 Hash 做分桶，每个分桶设置一个处理队列，这样可以把热点商品限制在一个请求队列里，防止因某些热点商品占用太多的服务器资源，而使其他请求始终得不到服务器的处理资源。

最后介绍一下隔离。秒杀系统设计的第一个原则就是将这种热点数据隔离出来，不要让1%的请求影响到另外的 99%，隔离出来后也更方便对这 1%的请求做针对性的优化。

具体到“秒杀”业务，我们可以在以下几个层次实现隔离。

1. **业务隔离。**把秒杀做成一种营销活动，卖家要参加秒杀这种营销活动需要单独报名，从技术上来说，卖家报名后对我们来说就有了已知热点，因此可以提前做好预热。
2. **系统隔离。**系统隔离更多的是运行时的隔离，可以通过分组部署的方式和另外 99%分开。秒杀可以申请单独的域名，目的也是让请求落到不同的集群中。
3. **数据隔离。**秒杀所调用的数据大部分都是热点数据，比如会启用单独的 Cache 集群或者 MySQL 数据库来放热点数据，目的也是不想 0.01%的数据有机会影响 99.99%数据。

当然了，实现隔离有很多种办法。比如，你可以按照用户来区分，给不同的用户分配不同的 Cookie，在接入层，路由到不同的服务接口中；再比如，你还可以在接入层针对 URL 中的不同 Path 来设置限流策略。服务层调用不同的服务接口，以及数据层通过给数据打标来区分等等这些措施，其目的都是把已经识别出来的热点请求和普通的请求区分开。

3.4 流量削峰应该怎么做?

如果你看过秒杀系统的流量监控图的话，你会发现它是一条直线，就在秒杀开始那一秒是一条很直很直的线，这是因为秒杀请求在时间上高度集中于某一特定的时间点。这样一来，就会导致一个特别高的流量峰值，它对资源的消耗是瞬时的。

但是对秒杀这个场景来说，最终能够抢到商品的人数是固定的，也就是说 100 人和 10000 人发起请求的结果都是一样的，并发度越高，无效请求也越多。

但是从业务上来说，秒杀活动是希望更多的人来参与的，也就是开始之前希望有更多的人来刷页面，但是真正开始下单时，秒杀请求并不是越多越好。因此我们可以设计一些规则，让并发的请求更多地延缓，而且我们甚至可以过滤掉一些无效请求。

3.4.1 为什么要削峰

为什么要削峰呢？或者说峰值会带来哪些坏处？

我们知道服务器的处理资源是恒定的，你用或者不用它的处理能力都是一样的，所以出现峰值的话，很容易导致忙到处理不过来，闲的时候却又没有什么要处理。但是由于要保证服务质量，我们的很多处理资源只能按照忙的时候来预估，而这会导致资源的一个浪费。

这就好比因为存在早高峰和晚高峰的问题，所以有了错峰限行的解决方案。削峰的存在，一是可以让服务端处理变得更加平稳，二是可以节省服务器的资源成本。针对秒杀这一场景，削峰从本质上来说就是更多地延缓用户请求的发出，以便减少和过滤掉一些无效请求，它遵从“请求数要尽量少”的原则。

今天，我就来介绍一下流量削峰的一些操作思路：排队、答题、分层过滤。这几种方式都是无损（即不会损失用户的发出请求）的实现方案，当然还有些有损的实现方案，包括我们后面要介绍的关于稳定性的一些办法，比如限流和机器负载保护等一些强制措施也能达到削峰保护的目的，当然这都是不得已的一些措施，因此就不归类到这里了。

3.4.2 排队

要对流量进行削峰，最容易想到的解决方案就是用消息队列来缓冲瞬时流量，把同步的直接调用转换成异步的间接推送，中间通过一个队列在一端承接瞬时的流量洪峰，在另一端平滑地将消息推送出去。在这里，消息队列就像“水库”一样， 拦蓄上游的洪水，削减进入下游河道的洪峰流量，从而达到减免洪水灾害的目的。

用消息队列来缓冲瞬时流量的方案，如下图所示：

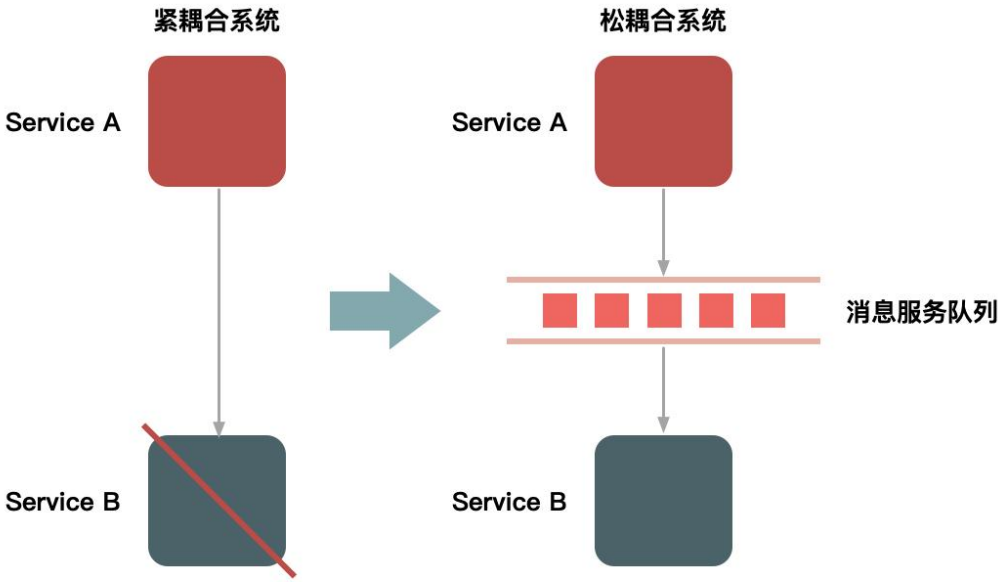


图 1 用消息队列来缓冲瞬时流量

但是，如果流量峰值持续一段时间达到了消息队列的处理上限，例如本机的消息积压达到了存储空间的上限，消息队列同样也会被压垮，这样虽然保护了下游的系统，但是和直接把请求丢弃也没多大的区别。就像遇到洪水爆发时，即使是有水库恐怕也无济于事。

除了消息队列，类似的排队方式还有很多，例如：

1. 利用线程池加锁等待也是一种常用的排队方式；
2. 先进先出、先进后出等常用的内存排队算法的实现方式；
3. 把请求序列化到文件中，然后再顺序地读文件（例如基于 MySQL binlog 的同步机制）来恢复请求等方式。

可以看到，这些方式都有一个共同特征，就是把“一步的操作”变成“两步的操作”，其中增加的一步操作用来起到缓冲的作用。

说到这里你可能会说，这样一来增加了访问请求的路径啊，并不符合我们介绍的“4 要 1 不要”原则。没错，的确看起来不太合理，但是如果不增加一个缓冲步骤，那么在一些场景下系统很可能会直接崩溃，所以最终还是需要你做出妥协和平衡。

3.4.3 答题

你是否还记得，最早期的秒杀只是纯粹地刷新页面和点击购买按钮，它是后来才增加了答题功能的。那么，为什么要增加答题功能呢？

这主要是为了增加购买的复杂度，从而达到两个目的。

第一个目的是防止部分买家使用秒杀器在参加秒杀时作弊。2011 年秒杀非常火的时候，秒杀器也比较猖獗，因而没有达到全民参与和营销的目的，所以系统增加了答题来限制秒杀器。增加答题后，下单的时间基本控制在 2s 后，秒杀器的下单比例也大大下降。答题页面如下图所示。

图 2 答题页面

第二个目的其实就是延缓请求，起到对请求流量进行削峰的作用，从而让系统能够更好地支持瞬时的流量高峰。这个重要的功能就是把峰值的下单请求拉长，从以前的 1s 之内延长到 2s~10s。这样一来，请求峰值基于时间分片了。这个时间的分片对服务端处理并发非常重要，会大大减轻压力。而且，由于请求具有先后顺序，靠后的请求到来时自然也就没有库存了，因此根本到不了最后的下单步骤，所以真正的并发写就非常有限了。这种设计思路目前用得非常普遍，如当年支付宝的“咻一咻”、微信的“摇一摇”都是类似的方式。

这里，我重点说一下秒杀答题的设计思路。

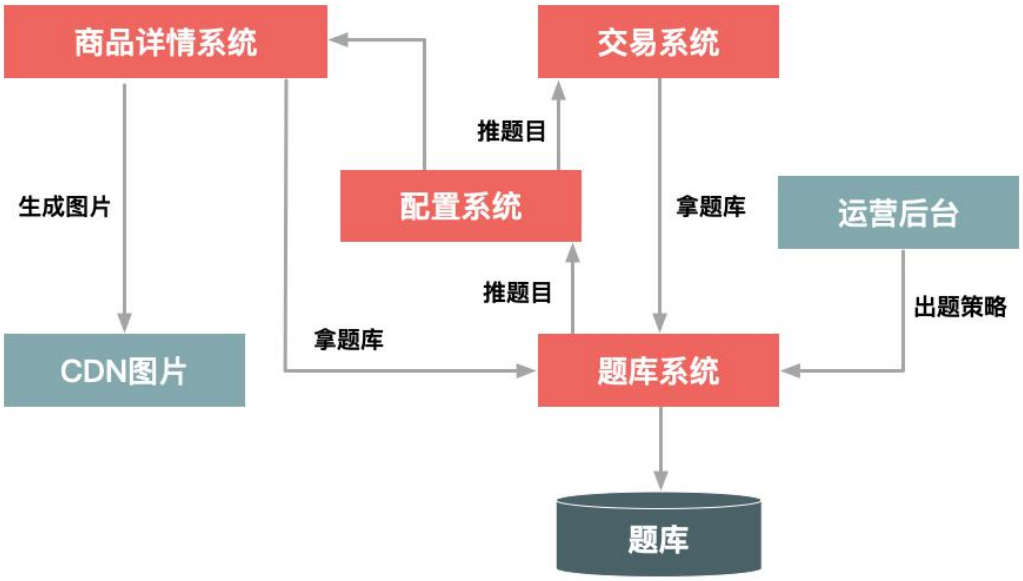


图 3 秒杀答题

如上图所示，整个秒杀答题的逻辑主要分为 3 部分。

1. **题库生成模块**，这个部分主要就是生成一个个问题和答案，其实题目和答案本身并不需要很复杂，重要的是能够防止由机器来算出结果，即防止秒杀器来答题。
2. **题库的推送模块**，用于在秒杀答题前，把题目提前推送给详情系统和交易系统。题库的推送主要是为了保证每次用户请求的题目是唯一的，目的也是防止答题作弊。
3. **题目的图片生成模块**，用于把题目生成为图片格式，并且在图片里增加一些干扰因素。这也同样是为防止机器直接来答题，它要求只有人才能理解题目本身的含义。这里还要注意一点，由于答题时网络比较拥挤，我们应该把题目的图片提前推送到 CDN 上并且要进行预热，不然的话当用户真正请求题目时，图片可能加载比较慢，从而影响答题的体验。

其实真正答题的逻辑比较简单，很好理解：当用户提交的答案和题目对应的答案做比较，如果通过了就继续进行下一步的下单逻辑，否则就失败。我们可以把问题和答案用下面这样的 key 来进行 MD5 加密：

- 问题 key：userId+itemId+question_Id+time+PK
- 答案 key：userId+itemId+answer+PK

验证的逻辑如下图所示：

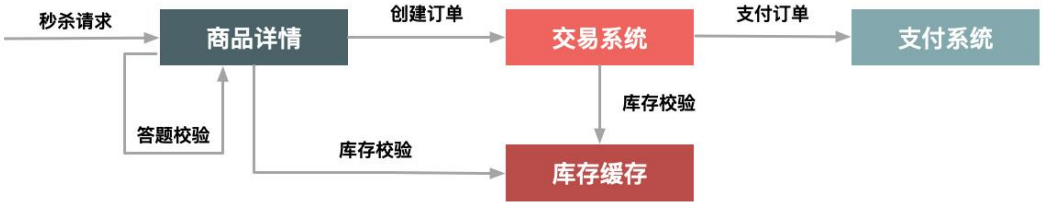


图 4 答题的验证逻辑

注意，这里面的验证逻辑，除了验证问题的答案以外，还包括用户本身身份的验证，例如是否已经登录、用户的 Cookie 是否完整、用户是否重复频繁提交等。

除了做正确性验证，我们还可以对提交答案的时间做些限制，例如从开始答题到接受答案要超过 1s，因为小于 1s 是人为操作的可能性很小，这样也能防止机器答题的情况。

3.4.4 分层过滤

前面介绍的排队和答题要么是少发请求，要么对发出来的请求进行缓冲，而针对秒杀场景还有一种方法，就是对请求进行分层过滤，从而过滤掉一些无效的请求。分层过滤其实就是采用“漏斗”式设计来处理请求的，如下图所示。

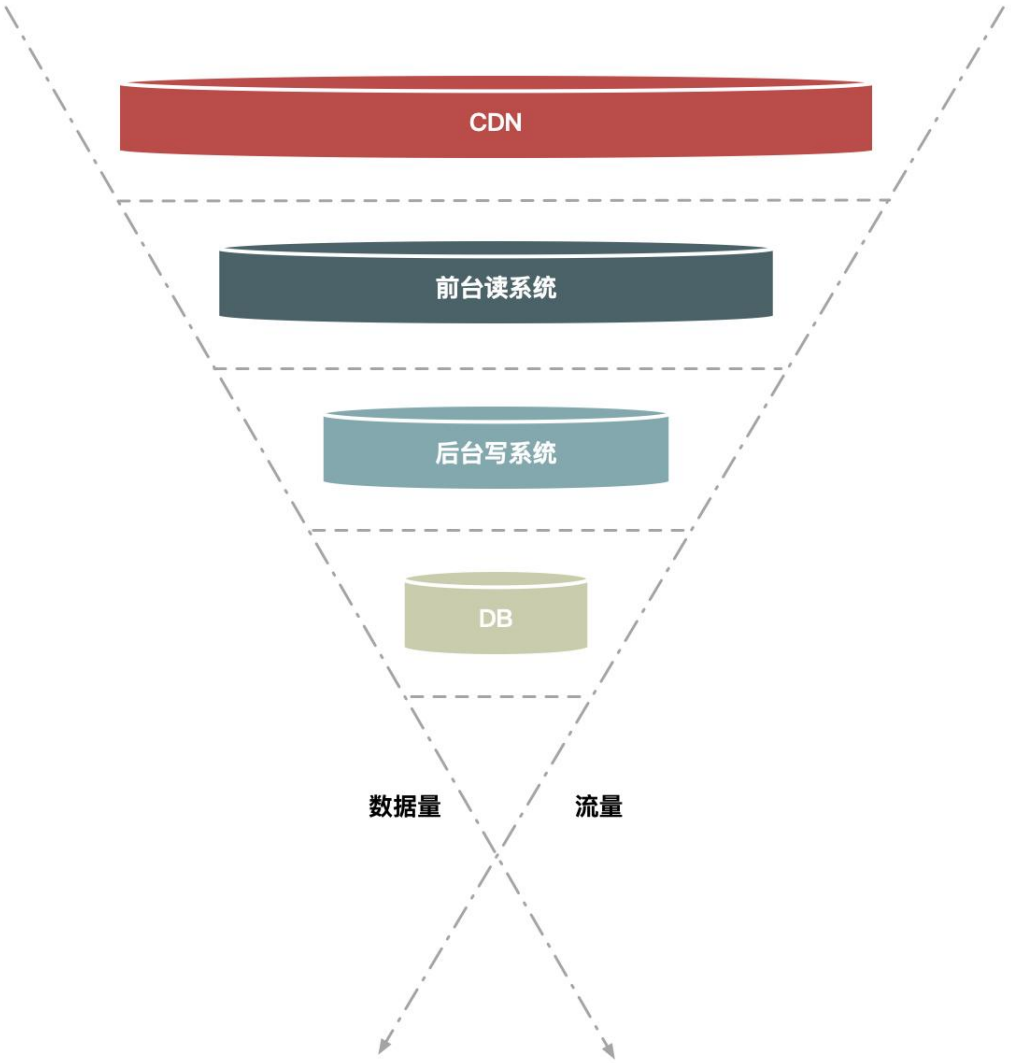


图 5 分层过滤

假如请求分别经过 CDN、前台读系统（如商品详情系统）、后台系统（如交易系统）和数据库这几层，那么：

- 大部分数据和流量在用户浏览器或者 CDN 上获取，这一层可以拦截大部分数据的读取；
- 经过第二层（即前台系统）时数据（包括强一致性的数据）尽量得走 Cache，过滤一些无效的请求；
- 再到第三层后台系统，主要做数据的二次检验，对系统做好保护和限流，这样数据量和请求就进一步减少；
- 最后在数据层完成数据的强一致性校验。

这样就像漏斗一样，尽量把数据量和请求量一层一层地过滤和减少了。

分层过滤的核心思想是：在不同的层次尽可能地过滤掉无效请求，让“漏斗”最末端的才是有效请求。而要达到这种效果，我们就必须对数据做分层的校验。

分层校验的基本原则是：

1. 将动态请求的读数据缓存（Cache）在 Web 端，过滤掉无效的数据读；
2. 对读数据不做强一致性校验，减少因为一致性校验产生瓶颈的问题；
3. 对写数据进行基于时间的合理分片，过滤掉过期的失效请求；
4. 对写请求做限流保护，将超出系统承载能力的请求过滤掉；
5. 对写数据进行强一致性校验，只保留最后有效的数据。

分层校验的目的是：在读系统中，尽量减少由于一致性校验带来的系统瓶颈，但是尽量将不影响性能的检查条件提前，如用户是否具有秒杀资格、商品状态是否正常、用户答题是否正确、秒杀是否已经结束、是否非法请求、营销等价物是否充足等；在写数据系统中，主要对写的数据（如“库存”）做一致性检查，最后在数据库层保证数据的最终准确性（如“库存”不能减为负数）。

3.5 影响性能的因素有哪些？又该如何提高系统的性能？

你想要提升性能，首先肯定要知道哪些因素对于系统性能的影响最大，然后再针对这些具体的因素想办法做优化，是不是这个逻辑？

那么，哪些因素对性能有影响呢？在回答这个问题之前，我们先定义一下“性能”，服务设备不同对性能的定义也是不一样的，例如 CPU 主要看主频、磁盘主要看 IOPS（Input/Output Operations Per Second，即每秒进行读写操作的次数）。

而今天我们讨论的主要是系统服务端性能，一般用 QPS（Query Per Second，每秒请求数）来衡量，还有一个影响和 QPS 也息息相关，那就是响应时间（Response Time，RT），它可以理解为服务器处理响应的耗时。

正常情况下响应时间（RT）越短，一秒钟处理的请求数（QPS）自然也就会越多，这在单线程处理的情况下看起来是线性的关系，即我们只要把每个请求的响应时间降到最低，那么性能就会最高。

但是你可能想到响应时间总有一个极限，不可能无限下降，所以又出现了另外一个维度，即通过多线程，来处理请求。这样理论上就变成了“总 QPS =（1000ms / 响应时间）

× 线程数量”，这样性能就和两个因素相关了，一个是一次响应的服务端耗时，一个是处理请求的线程数。

接下来，我们一起看看这个两个因素到底会造成什么样的影响。

首先，我们先来看看响应时间和 QPS 有啥关系。

对于大部分的 Web 系统而言，响应时间一般都是由 CPU 执行时间和线程等待时间 (比如 RPC、IO 等待、Sleep、Wait 等) 组成，即服务器在处理一个请求时，一部分是 CPU 本身在做运算，还有一部分是在各种等待。

理解了服务器处理请求的逻辑，估计你会说为什么我们不去减少这种等待时间。很遗憾，根据我们实际的测试发现，减少线程等待时间对提升性能的影响没有我们想象得那么大，它并不是线性的提升关系，这点在很多代理服务器（Proxy）上可以做验证。

如果代理服务器本身没有 CPU 消耗，我们在每次给代理服务器代理的请求加个延时，即增加响应时间，但是这对代理服务器本身的吞吐量并没有多大的影响，因为代理服务器本身的资源并没有被消耗，可以通过增加代理服务器的处理线程数，来弥补响应时间对代理服务器的 QPS 的影响。

其实，真正对性能有影响的是 CPU 的执行时间。这也很好理解，因为 CPU 的执行真正消耗了服务器的资源。经过实际的测试，如果减少 CPU 一半的执行时间，就可以增加一倍的 QPS。

也就是说，我们应该致力于减少 CPU 的执行时间。

其次，我们再来看看线程数对 QPS 的影响。

单看“总 QPS”的计算公式，你会觉得线程数越多 QPS 也就会越高，但这会一直正确吗？显然不是，线程数不是越多越好，因为线程本身也消耗资源，也受到其他因素的制约。例如，线程越多系统的线程切换成本就会越高，而且每个线程也都会耗费一定内存。

那么，设置什么样的线程数最合理呢？其实**很多多线程的场景都有一个默认配置，即“线程数 = 2 * CPU 核数 + 1”**。除去这个配置，还有一个根据最佳实践得出来的公式：

> 线程数 = [(线程等待时间 + 线程 CPU 时间) / 线程 CPU 时间] × CPU 数量

当然，最好的办法是通过性能测试来发现最佳的线程数。

换句话说，要提升性能我们就要减少 CPU 的执行时间，另外就是要设置一个合理的并发线程数，通过这两方面来显著提升服务器的性能。

现在，你知道了如何来快速提升性能，那接下来你估计会问，我应该怎么发现系统哪里最消耗 CPU 资源呢？

3.5.1 如何发现瓶颈

就服务器而言，会出现瓶颈的地方有很多，例如 CPU、内存、磁盘以及网络等都可能会导致瓶颈。此外，不同的系统对瓶颈的关注度也不一样，例如对缓存系统而言，制约它的是内存，而对存储型系统来说 I/O 更容易是瓶颈。

这个专栏中，我们定位的场景是秒杀，它的瓶颈更多地发生在 CPU 上。

那么，如何发现 CPU 的瓶颈呢？其实有很多 CPU 诊断工具可以发现 CPU 的消耗，最常用的就是 JProfiler 和 Yourkit 这两个工具，它们可以列出整个请求中每个函数的 CPU 执行时间，可以发现哪个函数消耗的 CPU 时间最多，以便你有针对性地做优化。

当然还有一些办法也可以近似地统计 CPU 的耗时，例如通过 jstack 定时地打印调用栈，如果某些函数调用频繁或者耗时较多，那么那些函数就会多次出现在系统调用栈里，这样相当于采样的方式也能够发现耗时较多的函数。

虽说秒杀系统的瓶颈大部分在 CPU，但这并不表示其他方面就一定不出现瓶颈。例如，如果海量请求涌过来，你的页面又比较大，那么网络就有可能出现瓶颈。

怎样简单地判断 CPU 是不是瓶颈呢？一个办法就是看当 QPS 达到极限时，你的服务器的 CPU 使用率是不是超过了 95%，如果没有超过，那么表示 CPU 还有提升的空间，要么是有锁限制，要么是有过多的本地 I/O 等待发生。

现在你知道了优化哪些因素，又发现了瓶颈，那么接下来就要关注如何优化了。

3.5.2 如何优化系统

对 Java 系统来说，可以优化的地方很多，这里我重点说一下比较有效的几种手段，供你参考，它们是：减少编码、减少序列化、Java 极致优化、并发读优化。接下来，我们分别来看一下。

1. 减少编码

Java 的编码运行比较慢，这是 Java 的一大硬伤。在很多场景下，只要涉及字符串的操作（如输入输出操作、I/O 操作）都比较耗 CPU 资源，不管它是磁盘 I/O 还是网络 I/O，因为都需要将字符转换成字节，而这个转换必须编码。

每个字符的编码都需要查表，而这种查表的操作非常耗资源，所以减少字符到字节或者相反的转换、减少字符编码会非常有成效。减少编码就可以大大提升性能。

那么如何才能减少编码呢？例如，网页输出是可以直接进行流输出的，即用resp.getOutputStream()函数写数据，把一些静态的数据提前转化成字节，等到真正往外写的时候再直接用 OutputStream()函数写，就可以减少静态数据的编码转换。

我在《深入分析 Java Web 技术内幕》一书中介绍的“Velocity 优化实践”一章的内容，就是基于把静态的字符串提前编码成字节并缓存，然后直接输出字节内容到页面，从而大大减少编码的性能消耗的，网页输出的性能比没有提前进行字符到字节转换时提升了30%左右。

2. 减少序列化

序列化也是 Java 性能的一大天敌，减少 Java 中的序列化操作也能大大提升性能。又因为序列化往往是和编码同时发生的，所以减少序列化也就减少了编码。

序列化大部分是在 RPC 中发生的，因此避免或者减少 RPC 就可以减少序列化，当然当前的序列化协议也已经做了很多优化来提升性能。有一种新的方案，就是可以将多个关联性比较强的应用进行“合并部署”，而减少不同应用之间的 RPC 也可以减少序列化的消耗。

所谓“合并部署”，就是把两个原本在不同机器上的不同应用合并部署到一台机器上，当然不仅仅是部署在一台机器上，还要在同一个 Tomcat 容器中，且不能走本机的 Socket，这样才能避免序列化的产生。

另外针对秒杀场景，我们还可以做得更极致一些，接下来我们来看第 3 点：Java 极致优化。

3. Java 极致优化

Java 和通用的 Web 服务器(如 Nginx 或 Apache 服务器)相比，在处理大并发的 HTTP 请求时要弱一点，所以一般我们都会对大流量的 Web 系统做静态化改造，让大部分请求和数据直接在 Nginx 服务器或者 Web 代理服务器（如 Varnish、Squid 等）上直接返回（这样可以减少数据的序列化与反序列化），而 Java 层只需处理少量数据的动态请求。针对这些请求，我们可以使用以下手段进行优化：

- 直接使用 Servlet 处理请求。避免使用传统的 MVC 框架，这样可以绕过一大堆复杂且用处不大的处理逻辑，节省1ms时间(具体取决于你对MVC框架的依赖程度)。

- 直接输出流数据。使用 resp.getOutputStream()而不是 resp.getWriter()函数，可以省掉一些不变字符数据的编码，从而提升性能；数据输出时推荐使用 JSON 而不是模板引擎（一般都是解释执行）来输出页面。

4. 并发读优化

也许有读者会觉得这个问题很容易解决，无非就是放到 Tair 缓存里面。集中式缓存为了保证命中率一般都会采用一致性 Hash，所以同一个 key 会落到同一台机器上。虽然单台缓存机器也能支撑 30w/s 的请求，但还是远不足以应对像“大秒”这种级别的热点商品。那么，该如何彻底解决单点的瓶颈呢？

答案是采用应用层的 LocalCache，即在秒杀系统的单机上缓存商品相关的数据。

那么，又如何缓存（Cache）数据呢？你需要划分成动态数据和静态数据分别进行处理：

- 像商品中的“标题”和“描述”这些本身不变的数据，会在秒杀开始之前全量推送到秒杀机器上，并一直缓存到秒杀结束；
- 像库存这类动态数据，会采用“被动失效”的方式缓存一定时间（一般是数秒），失效后再去缓存拉取最新的数据。

你可能还会有疑问：像库存这种频繁更新的数据，一旦数据不一致，会不会导致超卖？这就要用到前面介绍的读数据的分层校验原则了，读的场景可以允许一定的脏数据，因为这里的误判只会导致少量原本无库存的下单请求被误认为有库存，可以等到真正写数据时再保证最终的一致性，通过在数据的高可用性和一致性之间的平衡，来解决高并发的数据读取问题。

3.6 秒杀系统“减库存”设计的核心逻辑

如果要设计一套秒杀系统，那我想你的老板肯定会先对你说：千万不要超卖，这是大前提。

如果你第一次接触秒杀，那你可能还不太理解，库存 100 件就卖 100 件，在数据库里减到 0 就好了啊，这有什么麻烦的？是的，理论上是这样，但是具体到业务场景中，“减库存”就不是这么简单了。

例如，我们平常购物都是这样，看到喜欢的商品然后下单，但并不是每个下单请求你都最后付款了。你说系统是用户下单了就算这个商品卖出去了，还是等到用户真正付款了才算卖出了呢？这的确是个问题！

我们可以先根据减库存是发生在下单阶段还是付款阶段，把减库存做一下划分。

3.6.1 减库存有哪几种方式

在正常的电商平台购物场景中，用户的实际购买过程一般分为两步：下单和付款。你想买一台 iPhone 手机，在商品页面点了“立即购买”按钮，核对信息之后点击“提交订单”，这一步称为下单操作。下单之后，你只有真正完成付款操作才能算真正购买，也就是俗话说的“落袋为安”。

那如果你是架构师，你会在哪个环节完成减库存的操作呢？总结来说，减库存操作一般有如下几个方式：

- **下单减库存**，即当买家下单后，在商品的总库存中减去买家购买数量。下单减库存是最简单的减库存方式，也是控制最精确的一种，下单时直接通过数据库的事务机制控制商品库存，这样一定不会出现超卖的情况。但是你要知道，有些人下完单可能并不会付款。
- **付款减库存**，即买家下单后，并不立即减库存，而是等到有用户付款后才真正减库存，否则库存一直保留给其他买家。但因为付款时才减库存，如果并发比较高，有可能出现买家下单后付不了款的情况，因为可能商品已经被其他人买走了。
- **预扣库存**，这种方式相对复杂一些，买家下单后，库存为其保留一定的时间（如 10 分钟），超过这个时间，库存将会自动释放，释放后其他买家就可以继续购买。在买家付款前，系统会校验该订单的库存是否还有保留：如果没有保留，则再次尝试预扣；如果库存不足（也就是预扣失败）则不允许继续付款；如果预扣成功，则完成付款并实际地减去库存。

以上这几种减库存的方式都会存在一些问题，下面我们一起来看下。

3.6.2 减库存可能存在的问题

由于购物过程中存在两步或者多步的操作，因此在不同的操作步骤中减库存，就会存在一些可能被恶意买家利用的漏洞，例如发生恶意下单的情况。

假如我们采用“下单减库存”的方式，即用户下单后就减去库存，正常情况下，买家下单后付款的概率会很高，所以不会有太大问题。但是有一种场景例外，就是当卖家参加某个活动时，此时活动的有效时间是商品的黄金售卖时间，如果有竞争对手通过恶意下

单的方式将该卖家的商品全部下单，让这款商品的库存减为零，那么这款商品就不能正常售卖了。要知道，这些恶意下单的人是不会真正付款的，这正是“下单减库存”方式的不足之处。

既然“下单减库存”可能导致恶意下单，从而影响卖家的商品销售，那么有没有办法解决呢？你可能会想，采用“付款减库存”的方式是不是就可以了？的确可以。但是，“付款减库存”又会导致另外一个问题：库存超卖。

假如有 100 件商品，就可能出现 300 人下单成功的情况，因为下单时不会减库存，所以也就可能出现下单成功数远远超过真正库存数的情况，这尤其会发生在做活动的热门商品上。这样一来，就会导致很多买家下单成功但是付不了款，买家的购物体验自然比较差。

可以看到，不管是“下单减库存”还是“付款减库存”，都会导致商品库存不能完全和实际售卖情况对应起来的情况，看来要把商品准确地卖出去还真是不容易啊！

那么，既然“下单减库存”和“付款减库存”都有缺点，我们能否把两者相结合，将两次操作进行前后关联起来，下单时先预扣，在规定时间内不付款再释放库存，即采用“预扣库存”这种方式呢？

这种方案确实可以在一定程度上缓解上面的问题。但是否就彻底解决了呢？其实没有！针对恶意下单这种情况，虽然把有效的付款时间设置为 10 分钟，但是恶意买家完全可以在 10 分钟后再次下单，或者采用一次下单很多件的方式把库存减完。针对这种情况，解决办法还是要结合安全和反作弊的措施来制止。

例如，给经常下单不付款的买家进行识别打标（可以在被打标的买家下单时不减库存）、给某些类目设置最大购买件数（例如，参加活动的商品一人最多只能买 3 件），以及对重复下单不付款的操作进行次数限制等。

针对“库存超卖”这种情况，在 10 分钟时间内下单的数量仍然有可能超过库存数量，遇到这种情况我们只能区别对待：对普通的商品下单数量超过库存数量的情况，可以通过补货来解决；但是有些卖家完全不允许库存为负数的情况，那只能在买家付款时提示库存不足。

3.6.3 大型秒杀中如何减库存?

目前来看，业务系统中最常见的就是预扣库存方案，像你在买机票、买电影票时，下单后一般都有个“有效付款时间”，超过这个时间订单自动释放，这都是典型的预扣库存方案。而具体到秒杀这个场景，应该采用哪种方案比较好呢？

由于参加秒杀的商品，一般都是“抢到就是赚到”，所以成功下单后却不付款的情况比较少，再加上卖家对秒杀商品的库存有严格限制，所以秒杀商品采用“下单减库存”更加合理。另外，理论上由于“下单减库存”比“预扣库存”以及涉及第三方支付的“付款减库存”在逻辑上更为简单，所以性能上更占优势。

“下单减库存”在数据一致性上，主要就是保证大并发请求时库存数据不能为负数，也就是要保证数据库中的库存字段值不能为负数，一般我们有多种解决方案：一种是在应用程序中通过事务来判断，即保证减后库存不能为负数，否则就回滚；另一种办法是直接设置数据库的字段数据为无符号整数，这样减后库存字段值小于零时会直接执行 SQL 语句来报错；再有一种就是使用 CASE WHEN 判断语句，例如这样的 SQL 语句：

```
UPDATE item SET inventory = CASE WHEN inventory >= xxx THEN inventory-xxx ELSE inventory END
```

3.6.4 秒杀减库存的极致优化

在交易环节中，“库存”是个关键数据，也是个热点数据，因为交易的各个环节中都可能涉及对库存的查询。但是，我在前面介绍分层过滤时提到过，秒杀中并不需要对库存有精确的一致性读，把库存数据放到缓存（Cache）中，可以大大提升读性能。

解决大并发读问题，可以采用 LocalCache（即在秒杀系统的单机上缓存商品相关的数据）和对数据进行分层过滤的方式，但是像减库存这种大并发写无论如何还是避免不了，这也是秒杀场景下最为核心的一个技术难题。

因此，这里我想专门来说一下秒杀场景下减库存的极致优化思路，包括如何在缓存中减库存以及如何在数据库中减库存。

秒杀商品和普通商品的减库存还是有些差异的，例如商品数量比较少，交易时间段也比较短，因此这里有一个大胆的假设，即能否把秒杀商品减库存直接放到缓存系统中实现，也就是直接在缓存中减库存或者在一个带有持久化功能的缓存系统（如 Redis）中完成呢？

如果你的秒杀商品的减库存逻辑非常单一，比如没有复杂的 SKU 库存和总库存这种联动关系的话，我觉得完全可以。但是如果有比较复杂的减库存逻辑，或者需要使用事务，你还是必须在数据库中完成减库存。

由于 MySQL 存储数据的特点，同一数据在数据库里肯定是一行存储（MySQL），因此会有大量线程来竞争 InnoDB 行锁，而并发度越高时等待线程会越多，TPS（Transaction Per Second，即每秒处理的消息数）会下降，响应时间（RT）会上升，数据库的吞吐量就会严重受影响。

这就可能引发一个问题，就是单个热点商品会影响整个数据库的性能， 导致 0.01%的商品影响 99.99%的商品的售卖，这是我们不愿意看到的情况。一个解决思路是遵循前面介绍的原则进行隔离，把热点商品放到单独的热点库中。但是这无疑会带来维护上的麻烦，比如要做热点数据的动态迁移以及单独的数据库等。

而分离热点商品到单独的数据库还是没有解决并发锁的问题，我们应该怎么办呢？要解决并发锁的问题，有两种办法：

- **应用层做排队。**按照商品维度设置队列顺序执行，这样能减少同一台机器对数据库同一行记录进行操作的并发度，同时也能控制单个商品占用数据库连接的数量，防止热点商品占用太多的数据库连接。
- **数据库层做排队。**应用层只能做到单机的排队，但是应用机器数本身很多，这种排队方式控制并发的能力仍然有限，所以如果能在数据库层做全局排队是最理想的。阿里的数据库团队开发了针对这种 MySQL 的 InnoDB 层上的补丁程序（patch），可以在数据库层上对单行记录做到并发排队。

你可能有疑问了，排队和锁竞争不都是要等待吗，有啥区别？

如果熟悉 MySQL 的话，你会知道 InnoDB 内部的死锁检测，以及 MySQL Server 和 InnoDB 的切换会比较消耗性能，淘宝的 MySQL 核心团队还做了很多其他方面的优化，如 COMMIT_ON_SUCCESS 和 ROLLBACK_ON_FAIL 的补丁程序，配合在 SQL 里面加提示（hint），在事务里不需要等待应用层提交（COMMIT），而在数据执行完最后一条 SQL 后，直接根据 TARGET_AFFECT_ROW 的结果进行提交或回滚，可以减少网络等待时间（平均约 0.7ms）。据我所知，目前阿里 MySQL 团队已经将包含这些补丁程序的 MySQL 开源。

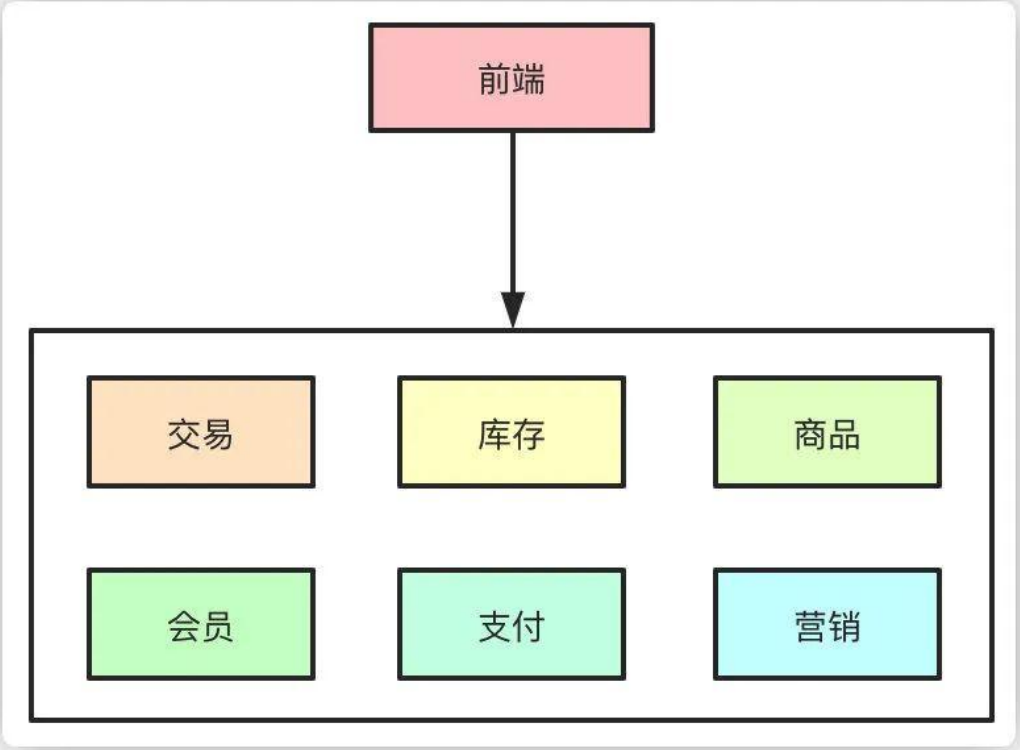
另外，数据更新问题除了前面介绍的热点隔离和排队处理之外，还有些场景（如对商品的 lastmodifytime 字段的）更新会非常频繁，在某些场景下这些多条 SQL 是可以合并的，一定时间内只要执行最后一条 SQL 就行了，以便减少对数据库的更新操作。

4. 如果你的系统的 QPS 突然提升 10 倍你会怎么设计?

4.1 硬件的扩展+微服务的拆分

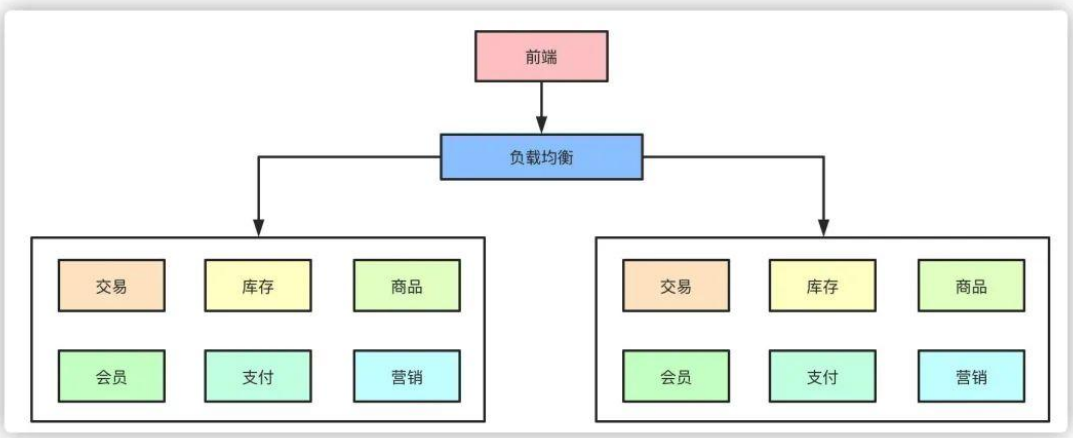
在互联网早期的时候，单体架构就足以支撑起日常的业务需求，大家的所有业务服务都在一个项目里，部署在一台物理机器上。

所有的业务包括你的交易系统、会员信息、库存、商品等等都夹杂在一起，当流量一旦起来之后，单体架构的问题就暴露出来了，机器挂了所有的业务全部无法使用了。



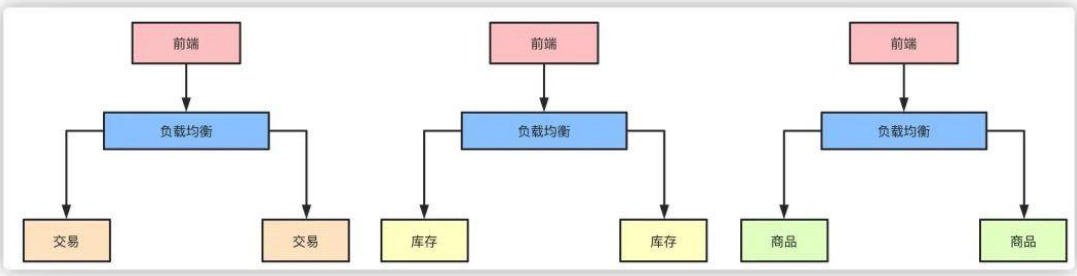
于是，集群架构的架构开始出现，单机无法抗住的压力，最简单的办法就是水平拓展横向扩容了，

这样，通过负载均衡把压力流量分摊到不同的机器上，暂时是解决了单点导致服务不可用的问题。



但是随着业务的发展，在一个项目里维护所有的业务场景使开发和代码维护变得越来越困难，

一个简单的需求改动都需要发布整个服务，代码的合并冲突也会变得越来越频繁，同时线上故障出现的可能性越大。微服务的架构模式就诞生了。



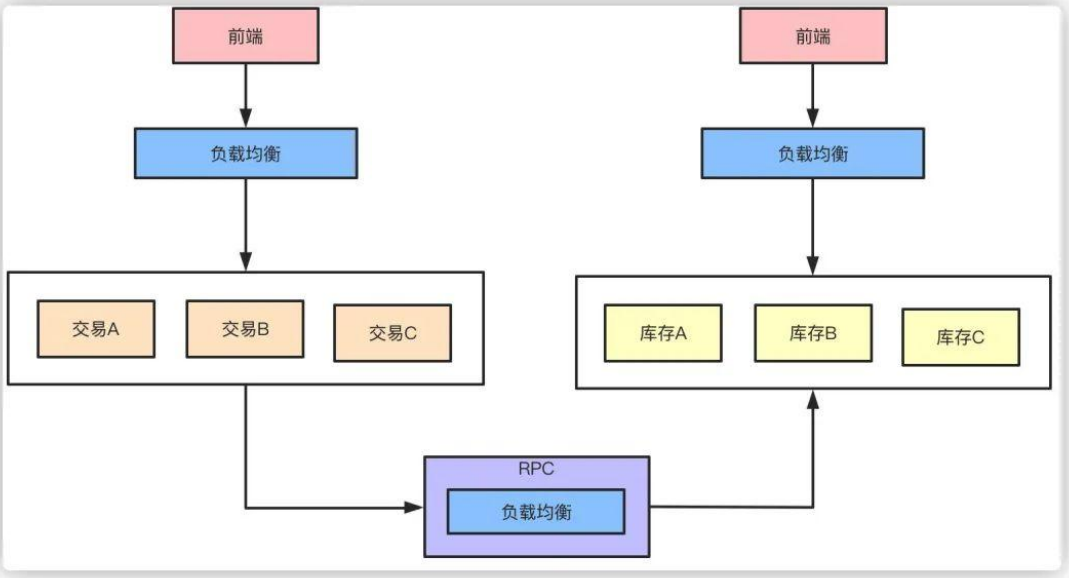
把每个独立的业务拆分开独立部署，开发和维护的成本降低，集群能承受的压力也提高了，再也不会出现一个小小的改动点需要牵一发而动全身了。

以上的点从高并发的角度而言，似乎都可以归类为通过服务拆分和集群物理机器的扩展提高了整体的系统抗压能力，那么，随之拆分而带来的问题也就是高并发系统需要解决的问题。

4.2 高性能 RPC

微服务化的拆分带来的好处和便利性是显而易见的，但是与此同时各个微服务之间的通信就需要考虑了。

传统 HTTP 的通信方式性能首先并不太好，大量的请求头之类无效的信息是对性能的浪费，这时候就需要引入诸如 Dubbo 类的 RPC 框架。



有小伙伴进行对比测试，Dubbo RPC 的性能，是 Feign RPC 的性能 10 倍。

我们假设原来来自客户端的 QPS 是 9000 的话，那么通过负载均衡策略分散到每台机器就是 3000，而 Feign HTTP RPC 改为 Dubbo RPC 之后，接口的耗时缩短了，单体服务和整体的 QPS 就提升了。

而 RPC 框架本身一般都自带负载均衡、熔断降级的机制，可以更好的维护整个系统的高可用性。

那么说完 RPC，作为基本上国内普遍的选择 Dubbo 的一些基本原理就是接下来的问题。

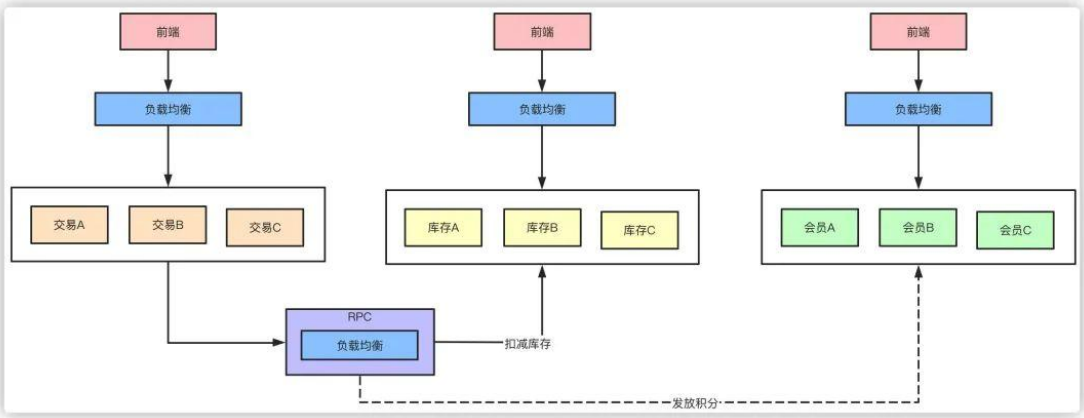
4.3 消息队列消峰解耦

对于 MQ 的作用大家都应该很了解了，主要功能：

- 削峰填谷、解耦。
- 同步转异步的方式，可以降低微服务之间的耦合。

对于一些不需要同步执行的接口，可以通过引入消息队列的方式异步执行以提高接口响应时间。在交易完成之后需要扣库存，然后可能需要给会员发放积分，本质上，发积分的动作应该属于履约服务，对实时性的要求也不高，我们只要保证最终一致性也就是能履约成功就行了。

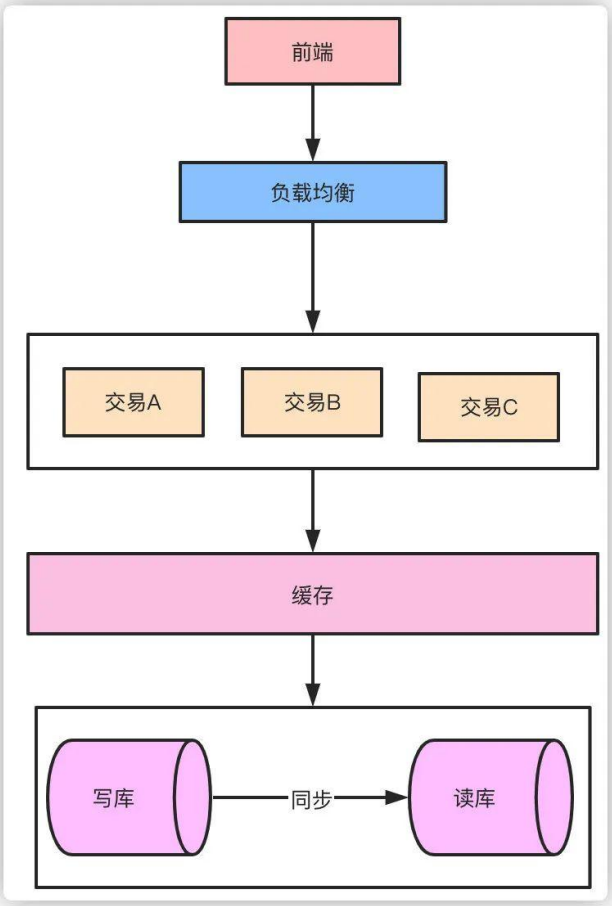
对于这种同类性质的请求就可以走 MQ 异步，也就提高了系统抗压能力了。



4.4 三级缓存架构

缓存作为高性能的代表，在某些特殊业务可能承担 90%以上的热点流量。

对于一些活动比如秒杀这种并发 QPS 可能几十万的场景，引入缓存事先预热可以大幅降低对数据库的压力，10 万的 QPS 对于单机的数据库来说可能就挂了，但是对于如 redis 这样的缓存来说就完全不是问题。



以秒杀系统举例，活动预热商品信息可以提前缓存提供查询服务，库存数据可以提前缓存，下单流程可以完全走缓存扣减，秒杀结束后再异步写入数据库，数据库承担的压力就小的太多了。

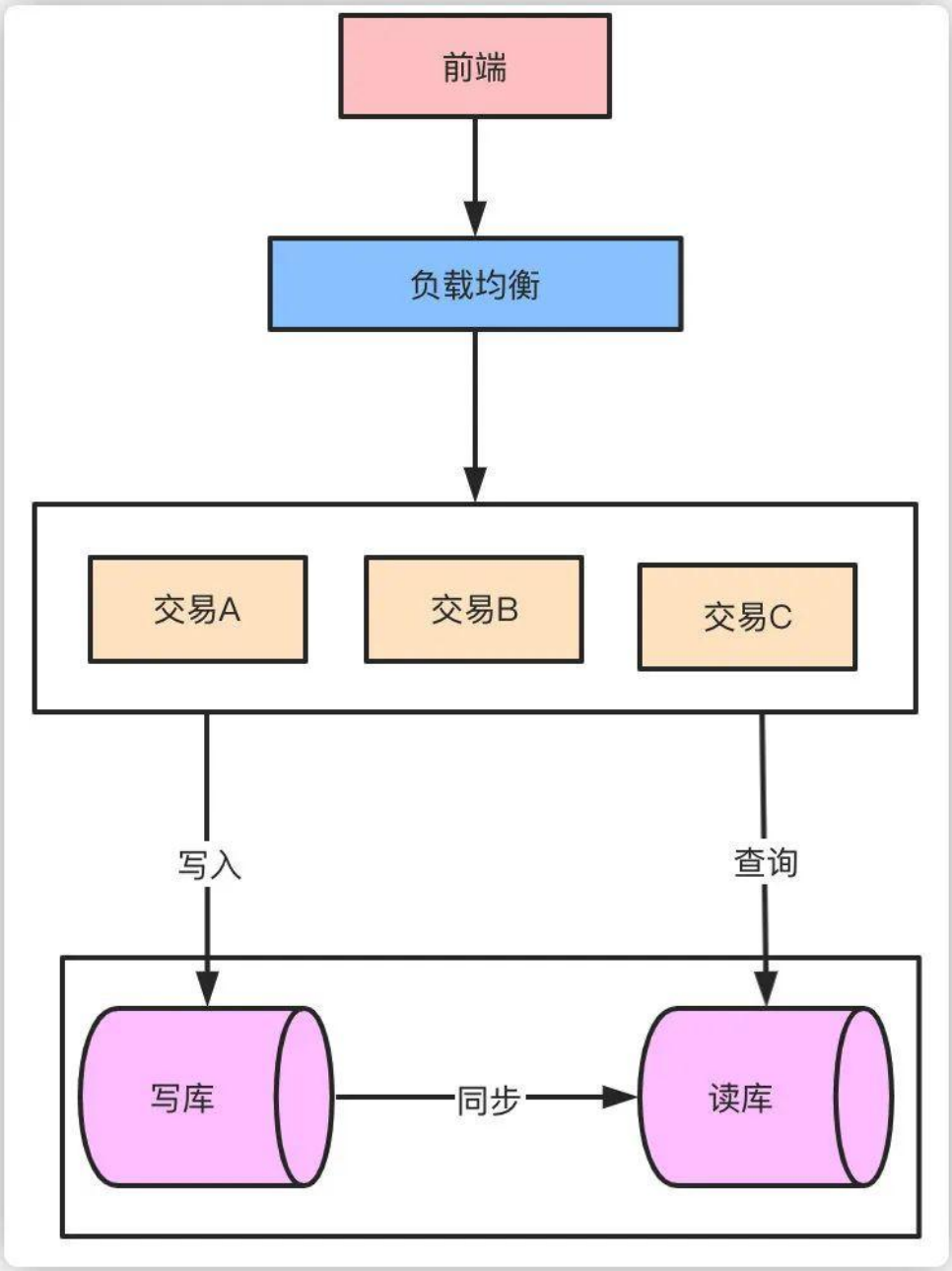
4.5 数据库分库分表

对于整个系统而言，最终所有的流量的查询和写入都落在数据库上，数据库是支撑系统高并发能力的核心。

怎么降低数据库的压力，提升数据库的性能是支撑高并发的基石。主要的方式就是通过读写分离和分库分表来解决这个问题。

对于整个系统而言，流量应该是一个漏斗的形式。比如我们的日活用户 DAU 有 20 万，实际可能每天来到提单页的用户只有 3 万 QPS，最终转化到下单支付成功的 QPS 只有 1 万。

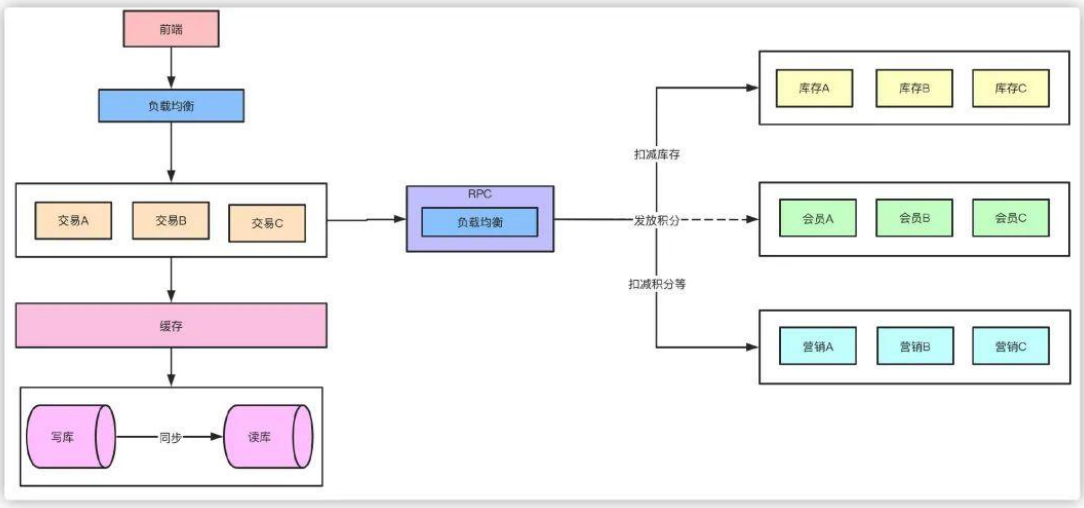
那么对于系统来说读是大于写的，这时候可以通过读写分离的方式来降低数据库的压力。



读写分离也就相当于数据库集群的方式降低了单节点的压力。而面对数据的急剧增长，原来的单库单表的存储方式已经无法支撑整个业务的发展，这时候就需要对数据库进行分库分表了。

针对微服务而言垂直的分库本身已经是做过的，剩下大部分都是分表的方案了。

4.6 高可用



4.6.1 熔断

比如营销服务挂了或者接口大量超时的异常情况，不能影响下单的主链路，涉及到积分的扣减一些操作可以在事后做补救。

4.6.2 限流

对突发如大促秒杀类的高并发，如果一些接口不做限流处理，可能直接就把服务打挂了，针对每个接口的压测性能的评估做出合适的限流尤为重要。

4.6.3 降级

熔断之后实际上可以说就是降级的一种，以熔断的举例来说营销接口熔断之后降级方案就是短时间内不再调用营销的服务，等到营销恢复之后再调用。

4.6.4 预案

一般来说，就算是有统一配置中心，在业务的高峰期也是不允许做出任何的变更的，但是通过配置合理的预案可以在紧急的时候做一些修改。

4.6.5 核对

针对各种分布式系统产生的分布式事务一致性或者受到攻击导致的数据异常，非常需要核对平台来做最后的兜底的数据验证。比如下游支付系统和订单系统的金额做核对是否正确，如果收到中间人攻击落库的数据是否保证正确性。

4.7 总结

其实可以看到，怎么设计高并发系统这个问题本身他是不难的，无非是基于你知道的知识点，从物理硬件层面到软件的架构、代码层面的优化，使用什么中间件来不断提高系统的抗压能力。

但是这个问题本身会带来更多的问题，微服务本身的拆分带来了分布式事务的问题，http、RPC 框架的使用带来了通信效率、路由、容错的问题，MQ 的引入带来了消息丢失、积压、事务消息、顺序消息的问题，

缓存的引入又会带来一致性、雪崩、击穿的问题，

数据库的读写分离、分库分表又会带来主从同步延迟、分布式 ID、事务一致性的问题，而为了解决这些问题我们又要不断的加入各种措施熔断、限流、降级、离线核对、预案处理等等来防止和追溯这些问题。

5. 字节二面：如何设计一个支持 10 万 QPS 的会员系统

会员系统是一种基础系统，跟公司所有业务线的下单主流程密切相关。如果会员系统出故障，会导致用户无法下单，影响范围是全公司所有业务线。所以，会员系统必须保证高性能、高可用，提供稳定、高效的基础服务。

随着同程和艺龙两家公司的合并，越来越多的系统需要打通同程 APP、艺龙 APP、同程微信小程序、艺龙微信小程序等多平台会员体系。

例如微信小程序的交叉营销，用户买了一张火车票，此时想给他发酒店红包，这就需要查询该用户的统一会员关系。

因为火车票用的是同程会员体系，酒店用的是艺龙会员体系，只有查到对应的艺龙会员卡号后，才能将红包挂载到该会员账号。

除了上述讲的交叉营销，还有许多场景需要查询统一会员关系，例如订单中心、会员等级、里程、红包、常旅、实名，以及各类营销活动等等。

所以，会员系统的请求量越来越大，并发量越来越高，今年清明小长假的秒并发 tps 甚至超过 2 万多。

在如此大流量的冲击下，会员系统是如何做到高性能和高可用的呢？这就是本文着重要讲述的内容。

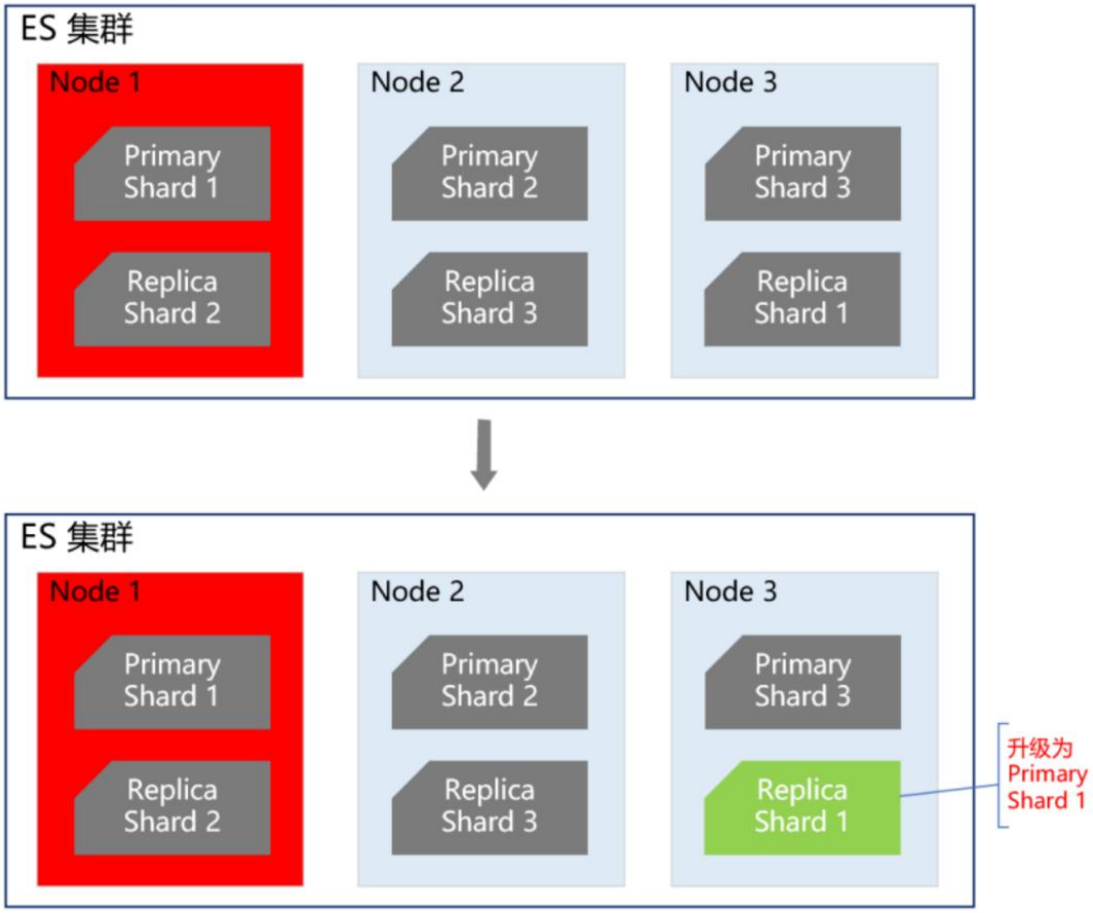
5.1 ES 高可用方案

5.1.1 ES 双中心主备集群架构

同程和艺龙两家公司融合后，全平台所有体系的会员总量是十多亿。在这么大的数据体量下，业务线的查询维度也比较复杂。

有的业务线基于手机号，有的基于微信 unionid，也有的基于艺龙卡号等查询会员信息。这么大的数据量，又有这么多的查询维度，基于此，我们选择 ES 用来存储统一会员关系。ES 集群在整个会员系统架构中非常重要，那么如何保证 ES 的高可用呢？

首先我们知道，ES 集群本身就是保证高可用的，如下图所示：



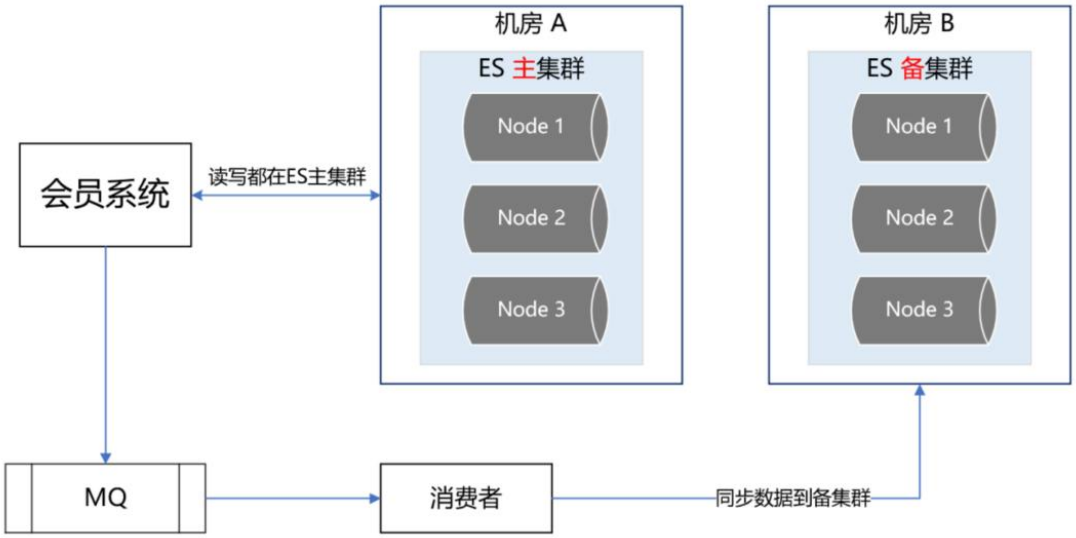
当 ES 集群有一个节点宕机了，会将其他节点对应的 Replica Shard 升级为 Primary Shard，继续提供服务。

但即使是这样，还远远不够。例如 ES 集群都部署在机房 A，现在机房 A 突然断电了，怎么办？

例如服务器硬件故障，ES 集群大部分机器宕机了，怎么办？或者突然有个非常热门的抢购秒杀活动，带来了一波非常大的流量，直接把 ES 集群打死了，怎么办？面对这些情况，让运维兄弟冲到机房去解决？

这个非常不现实，因为会员系统直接影响全公司所有业务线的下单主流程，故障恢复的时间必须非常短，如果需要运维兄弟人工介入，那这个时间就太长了，是绝对不能容忍的。

那 ES 的高可用如何做呢？我们的方案是 ES 双中心主备集群架构。



我们有两个机房，分别是机房 A 和机房 B。我们把 ES 主集群部署在机房 A，把 ES 备集群部署在机房 B。会员系统的读写都在 ES 主集群，通过 MQ 将数据同步到 ES 备集群。

此时，如果 ES 主集群崩了，通过统一配置，将会员系统的读写切到机房 B 的 ES 备集群上，这样即使 ES 主集群挂了，也能在很短的时间内实现故障转移，确保会员系统的稳定运行。

最后，等 ES 主集群故障恢复后，打开开关，将故障期间的数据同步到 ES 主集群，等数据同步一致后，再将会员系统的读写切到 ES 主集群。

5.1.2 ES 流量隔离三集群架构

双中心 ES 主备集群做到这一步，感觉应该没啥大问题了，但去年的一次恐怖流量冲击让我们改变了想法。

那是一个节假日，某个业务上线了一个营销活动，在用户的一次请求中，循环 10 多次调用了会员系统，导致会员系统的 tps 暴涨，差点把 ES 集群打爆。

这件事让我们后怕不已，它让我们意识到，一定要对调用方进行优先级分类，实施更精细的隔离、熔断、降级、限流策略。

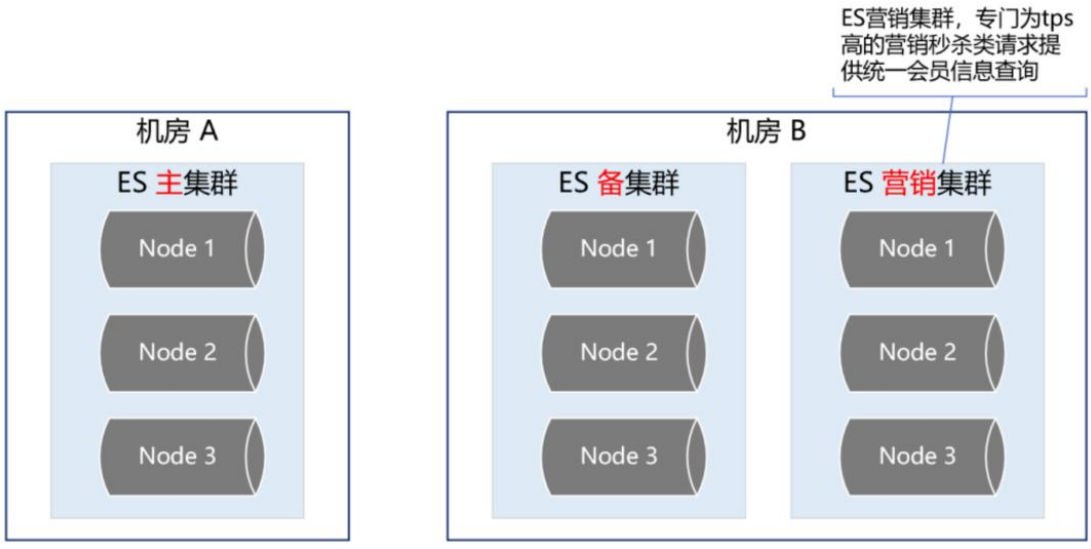
首先，我们梳理了所有调用方，分出两大类请求类型：

第一类是跟用户的下单主流程密切相关的请求，这类请求非常重要，应该高优先级保障。

第二类是营销活动相关的，这类请求有个特点，他们的请求量很大，tps 很高，但不影响下单主流程。

基于此，我们又构建了一个 ES 集群，专门用来应对高 tps 的营销秒杀类请求，这样就跟 ES 主集群隔离开来，不会因为某个营销活动的流量冲击而影响用户的下单主流程。

如下图所示：



5.1.3 ES 集群深度优化提升

讲完了 ES 的双中心主备集群高可用架构，接下来我们深入讲解一下 ES 主集群的优化工作。

有一段时间，我们特别痛苦，就是每到饭点，ES 集群就开始报警，搞得每次吃饭都心慌慌的，生怕 ES 集群一个扛不住，就全公司炸锅了。

那为什么一到饭点就报警呢？因为流量比较大，导致 ES 线程数飙高，cpu 直往上窜，查询耗时增加，并传导给所有调用方，导致更大范围的延时。那么如何解决这个问题呢？

关注公众号：码猿技术专栏，回复关键词：1111 获取阿里内部 Java 性能调优手册

通过深入 ES 集群，我们发现了以下几个问题：

ES 负载不合理，热点问题严重。ES 主集群一共有几十个节点，有的节点上部署的 shard 数偏多，有的节点部署的 shard 数很少，导致某些服务器的负载很高，每到流量高峰期，就经常预警。

ES 线程池的大小设置得太高，导致 cpu 飙高。我们知道，设置 ES 的 threadpool，一般将线程数设置为服务器的 cpu 核数，即使 ES 的查询压力很大，需要增加线程数，那最好也不要超过“cpu core * 3 / 2 + 1”。如果设置的线程数过多，会导致 cpu 在多个线程上下文之间频繁来回切换，浪费大量 cpu 资源。

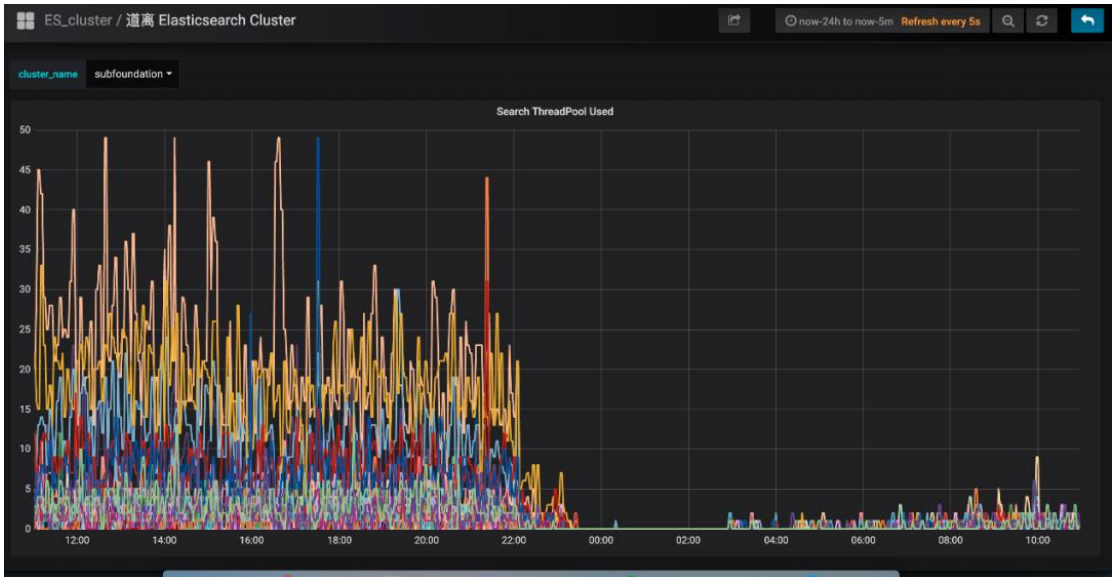
shard 分配的内存太大，100g，导致查询变慢。我们知道，ES 的索引要合理分配 shard 数，要控制一个 shard 的内存大小在 50g 以内。如果一个 shard 分配的内存过大，会导致查询变慢，耗时增加，严重拖累性能。

string 类型的字段设置了双字段，既是 text，又是 keyword，导致存储容量增大了一倍。会员信息的查询不需要关联度打分，直接根据 keyword 查询就行，所以完全可以将 text 字段去掉，这样就能节省很大一部分存储空间，提升性能。

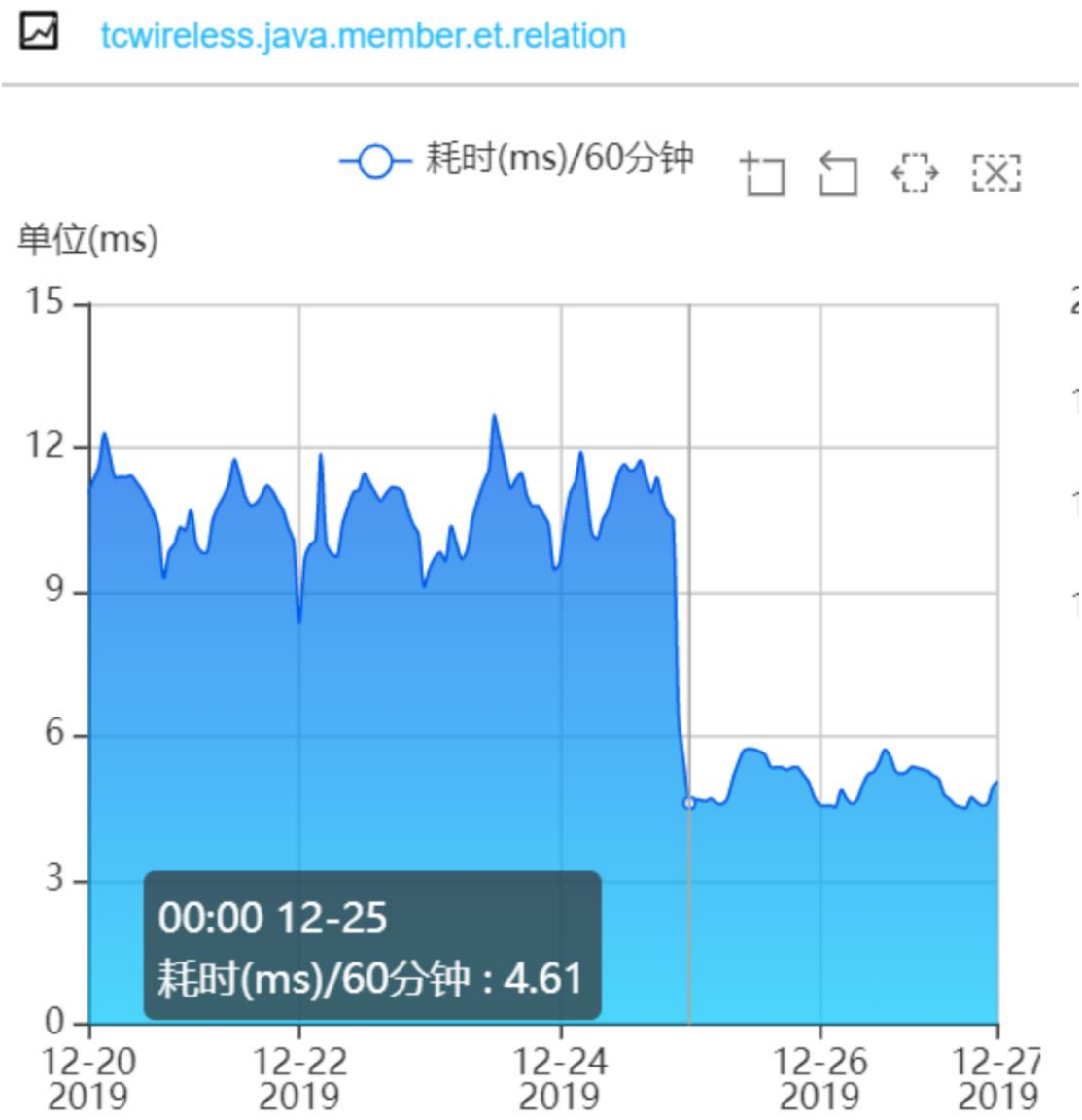
ES 查询，使用 filter，不使用 query。因为 query 会对搜索结果进行相关度算分，比较耗 cpu，而会员信息的查询是不需要算分的，这部分的性能损耗完全可以避免。节约 ES 算力，将 ES 的搜索结果排序放在会员系统的 jvm 内存中进行。

增加 routing key。我们知道，一次 ES 查询，会将请求分发给所有 shard，等所有 shard 返回结果后再聚合数据，最后将结果返回给调用方。如果我们事先已经知道数据分布在哪些 shard 上，那么就可以减少大量不必要的请求，提升查询性能。

经过以上优化，成果非常显著，ES 集群的 cpu 大幅下降，查询性能大幅提升。ES 集群的 cpu 使用率：


ES_cluster / 道离 Elasticsearch Cluster
now-24h to now-5m
Refresh every 5s
cluster_name
subfoundation
Search ThreadPool Used
50
45
40
35
30
25
20
15
10
5
0
12:00
14:00
16:00
18:00
20:00
22:00
00:00
02:00
04:00
06:00
08:00
10:00

会员系统的接口耗时：



5.2 会员 Redis 缓存方案

一直以来，会员系统是不做缓存的，原因主要有两个：

第一个，前面讲的 ES 集群性能很好，秒并发 3 万多，99 线耗时 5 毫秒左右，已经足够应付各种棘手的场景。

第二个，有的业务对会员的绑定关系要求实时一致，而会员是一个发展了 10 多年的老系统，是一个由好多接口、好多系统组成的分布式系统。

所以，只要有一个接口没有考虑到位，没有及时去更新缓存，就会导致脏数据，进而引发一系列的问题。

例如：用户在 APP 上看不到微信订单、APP 和微信的会员等级、里程等没合并、微信和 APP 无法交叉营销等等。

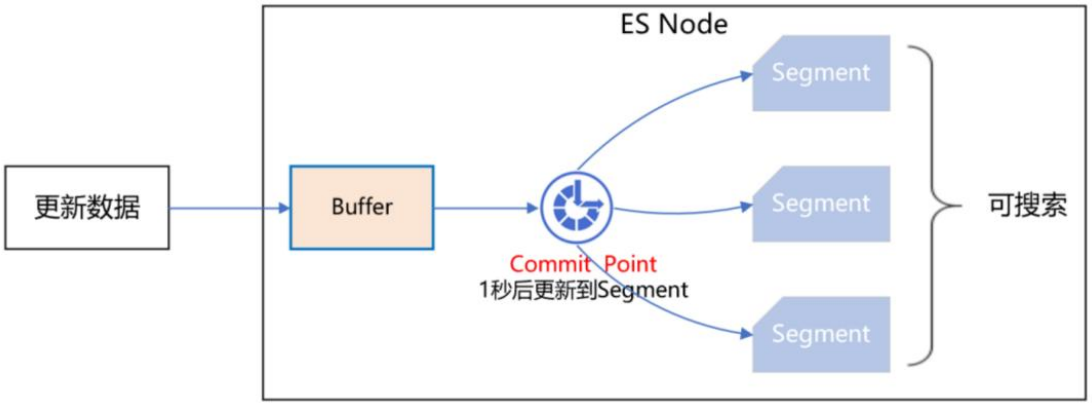
那后来为什么又要做缓存呢？是因为今年机票的盲盒活动，它带来的瞬时并发太高了。虽然会员系统安然无恙，但还是有点心有余悸，稳妥起见，最终还是决定实施缓存方案。

5.2.1 ES 近一秒延时导致的 Redis 缓存数据不一致问题的解决方案

在做会员缓存方案的过程中，遇到一个 ES 引发的问题，该问题会导致缓存数据的不一致。

我们知道，ES 操作数据是近实时的，往 ES 新增一个 Document，此时立即去查，是查不到的，需要等待 1 秒后才能查询到。

如下图所示：

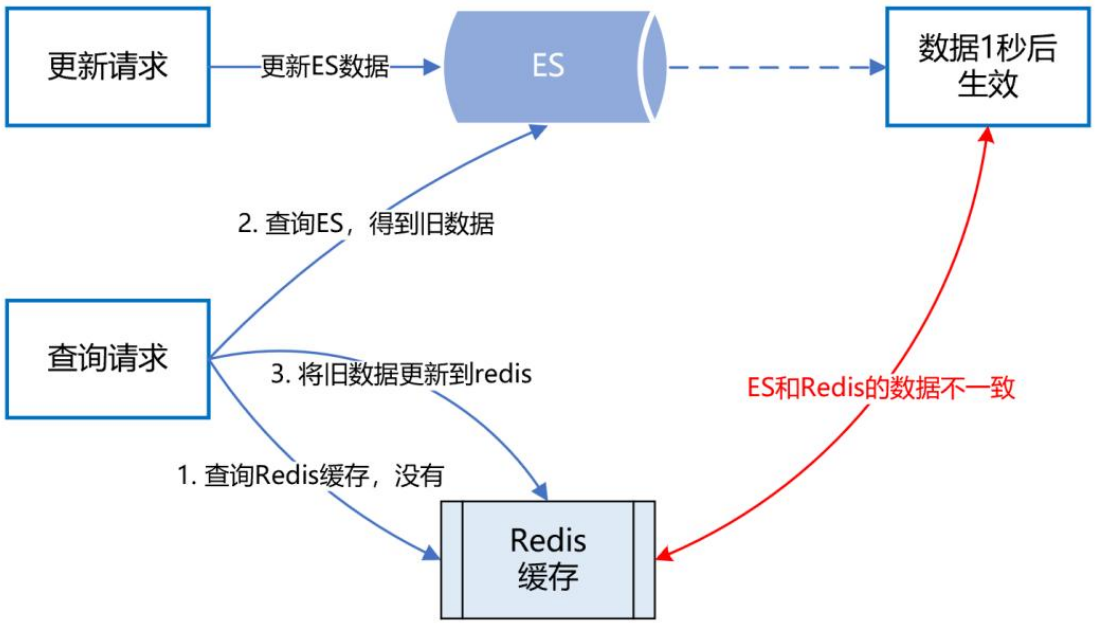


ES 的近实时机制为什么会导致 Redis 缓存数据不一致呢？具体来讲，假设一个用户注销了自己的 APP 账号，此时需要更新 ES，删除 APP 账号和微信账号的绑定关系。而 ES 的数据更新是近实时的，也就是说，1 秒后你才能查询到更新后的数据。

而就在这 1 秒内，有个请求来查询该用户的会员绑定关系，它先到 Redis 缓存中查，发现没有，然后到 ES 查，查到了，但查到的是更新前的旧数据。

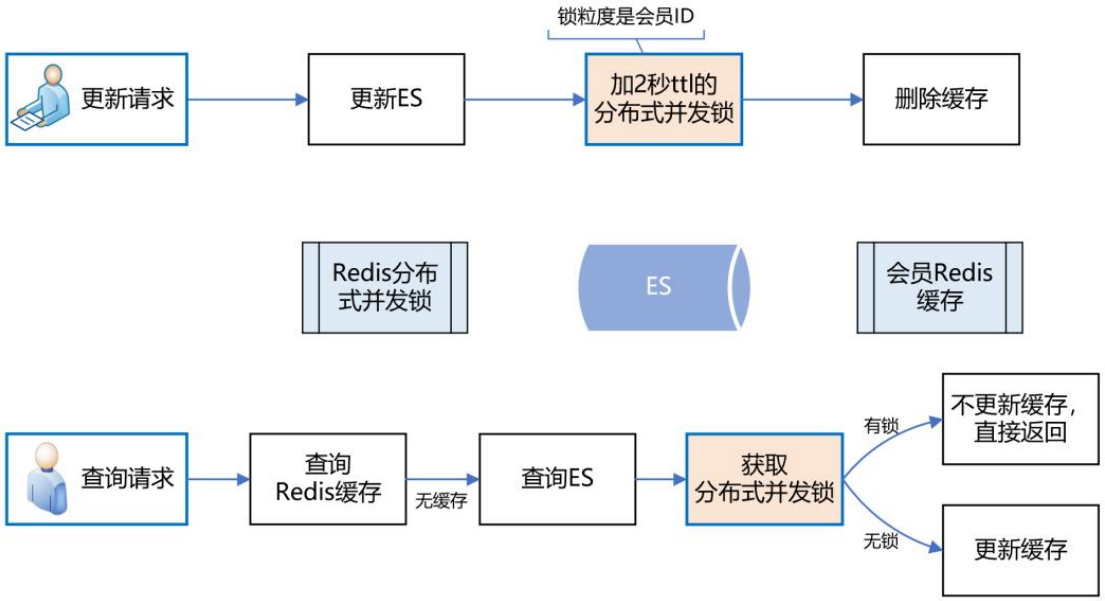
最后，该请求把查询到的旧数据更新到 Redis 缓存并返回。就这样，1 秒后，ES 中该用户的会员数据更新了，但 Redis 缓存的数据还是旧数据，导致了 Redis 缓存跟 ES 的数据不一致。

如下图所示：



面对该问题，如何解决呢？我们的思路是，在更新 ES 数据时，加一个 2 秒的 Redis 分布式并发锁，为了保证缓存数据的一致性，接着再删除 Redis 中该会员的缓存数据。如果此时有请求来查询数据，先获取分布式锁，发现该会员 ID 已经上锁了，说明 ES 刚刚更新的数据尚未生效，那么此时查询完数据后就不更新 Redis 缓存了，直接返回，这样就避免了缓存数据的不一致问题。

如下图所示：

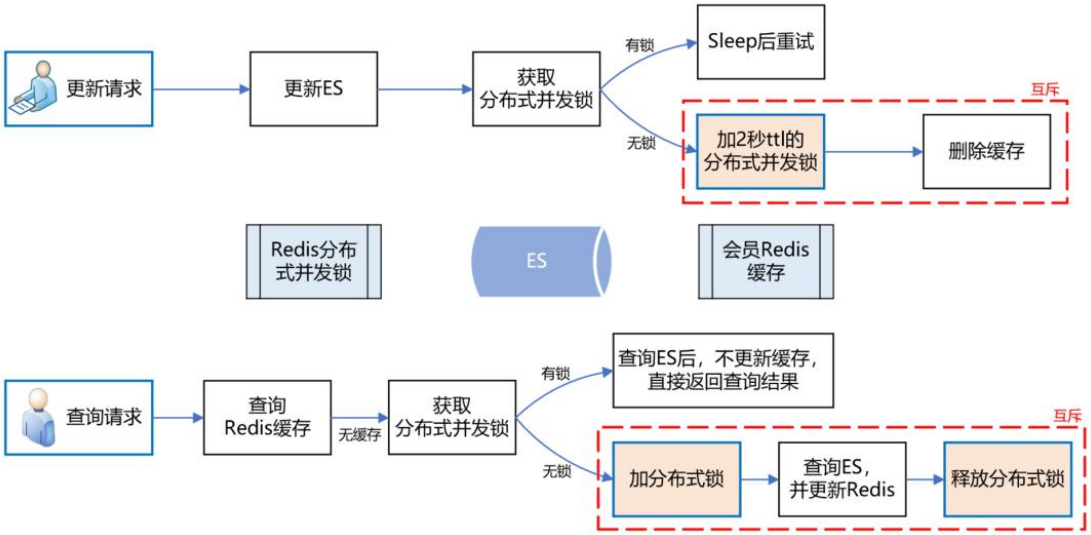


上述方案，乍一看似乎没什么问题了，但仔细分析，还是有可能导致缓存数据的不一致。例如，在更新请求加分布式锁之前，恰好有一个查询请求获取分布式锁，而此时是没有锁的，所以它可以继续更新缓存。

但就在他更新缓存之前，线程 block 了，此时更新请求来了，加了分布式锁，并删除了缓存。当更新请求完成操作后，查询请求的线程活过来了，此时它再执行更新缓存，就把脏数据写到缓存中了。

发现没有？主要的问题症结就在于“删除缓存”和“更新缓存”发生了并发冲突，只要将它们互斥，就能解决问题。

如下图所示：

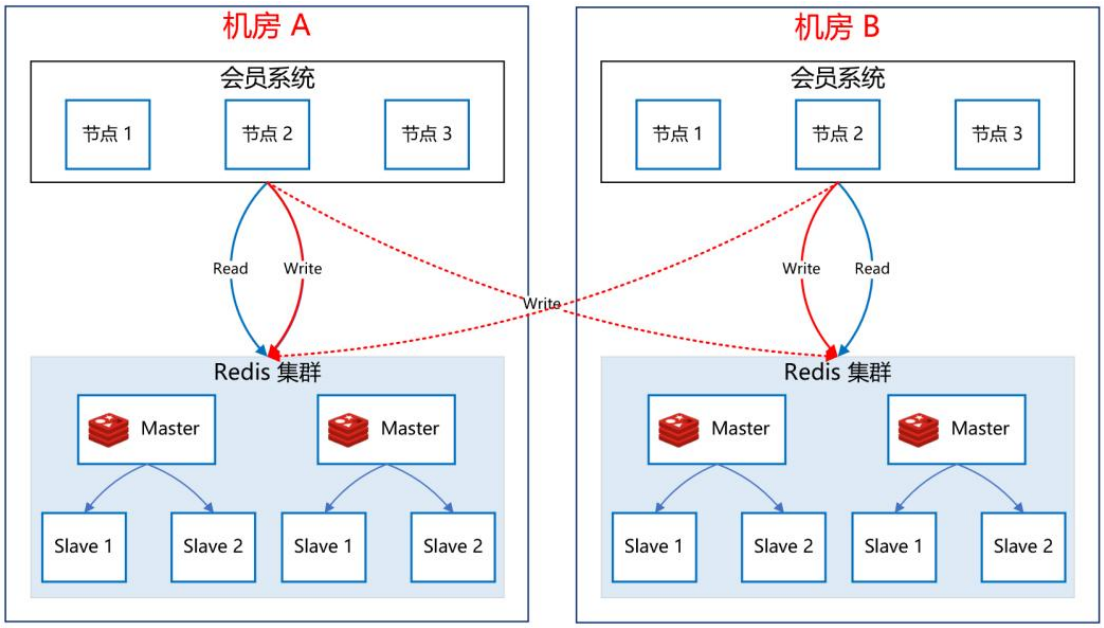


实施了缓存方案后，经统计，缓存命中率 90%+，极大缓解了 ES 的压力，会员系统整体性能得到了很大提升。

5.2.2 Redis 双中心多集群架构

接下来，我们看一下如何保障 Redis 集群的高可用。

如下图所示：



关于 Redis 集群的高可用，我们采用了双中心多集群的模式。在机房 A 和机房 B 各部署一套 Redis 集群。

更新缓存数据时，双写，只有两个机房的 Redis 集群都写成功了，才返回成功。查询缓存数据时，机房内就近查询，降低延时。这样，即使机房 A 整体故障，机房 B 还能提供完整的会员服务。

5.3 高可用会员主库方案

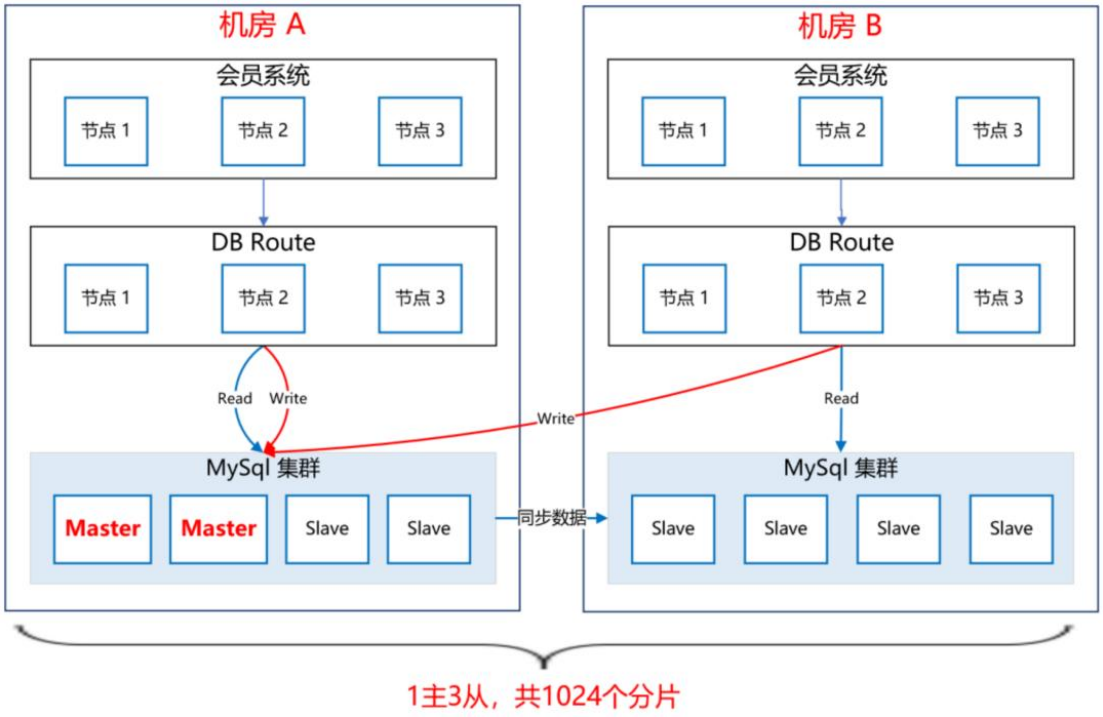
上述讲到，全平台会员的绑定关系数据存在 ES，而会员的注册明细数据存在关系型数据库。

最早，会员使用的数据库是 SqlServer，直到有一天，DBA 找到我们说，单台 SqlServer 数据库已经存储了十多亿的会员数据，服务器已达到物理极限，不能再扩展了。按照现在的增长趋势，过不了多久，整个 SqlServer 数据库就崩了。

你想想，那是一种什么样的灾难场景：会员数据库崩了，会员系统就崩了；会员系统崩了，全公司所有业务线就崩了。想想就不寒而栗，酸爽无比，为此我们立刻开启了迁移 DB 的工作。

5.3.1 MySQL 双中心 Partition 集群方案

经过调研，我们选择了双中心分库分表的 MySQL 集群方案，如下图所示：



会员一共有十多亿的数据，我们把会员主库分了 1000 多个分片，平分到每个分片大概百万的量级，足够使用了。

MySQL 集群采用 1 主 3 从的架构，主库放在机房 A，从库放在机房 B，两个机房之间通过专线同步数据，延迟在 1 毫秒内。

会员系统通过 DBRoute 读写数据，写数据都路由到 master 节点所在的机房 A，读数据都路由到本地机房，就近访问，减少网络延迟

这样，采用双中心的 MySQL 集群架构，极大提高了可用性，即使机房 A 整体都崩了，还可以将机房 B 的 Slave 升级为 Master，继续提供服务。

双中心 MySQL 集群搭建好后，我们进行了压测，测试下来，秒并发能达到 2 万多，平均耗时在 10 毫秒内，性能达标。

5.3.2 会员主库平滑迁移方案

接下来的工作，就是把会员系统的底层存储从 SqlServer 切到 MySQL 上，这是个风险极高的工作。

主要有以下几个难点：

会员系统是一刻都不能停机的，要在不停机的情况下完成 SqlServer 到 MySQL 的切换，就像是在给高速行驶的汽车换轮子。

会员系统是由很多个系统和接口组成的，毕竟发展了 10 多年，由于历史原因，遗留了大量老接口，逻辑错综复杂。这么多系统，必须一个不落的全部梳理清楚，DAL 层代码必须重写，而且不能出任何问题，否则将是灾难性的。

数据的迁移要做到无缝迁移，不仅是存量 10 多亿数据的迁移，实时产生的数据也要无缝同步到 MySQL。另外，除了要保障数据同步的实时性，还要保证数据的正确性，以及 SqlServer 和 MySQL 数据的一致性。

基于以上痛点，我们设计了“全量同步、增量同步、实时流量灰度切换”的技术方案。

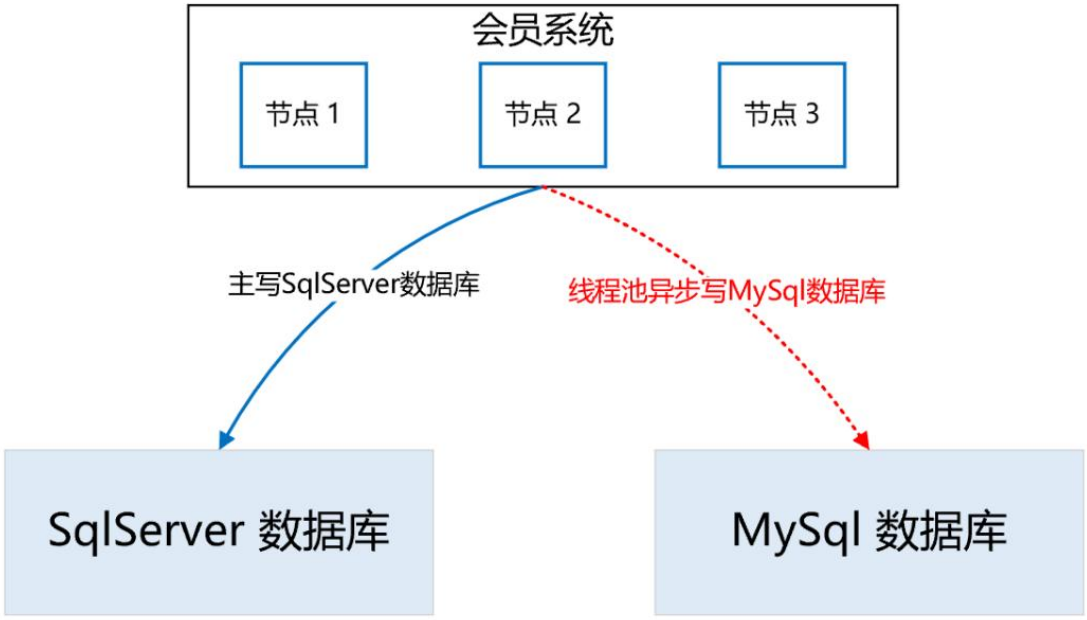
首先，为了保证数据的无缝切换，采用实时双写的方案。因为业务逻辑的复杂，以及 SqlServer 和 MySQL 的技术差异性，在双写 MySQL 的过程中，不一定会写成功，而一旦写失败，就会导致 SqlServer 和 MySQL 的数据不一致，这是绝不允许的。

所以，我们采取的策略是，在试运行期间，主写 SqlServer，然后通过线程池异步写 MySQL，如果写失败了，重试三次，如果依然失败，则记日志，然后人工排查原因，解决后，继续双写，直到运行一段时间，没有双写失败的情况。

通过上述策略，可以确保在绝大部分情况下，双写操作的正确性和稳定性，即使在试运行期间出现了 SqlServer 和 MySQL 的数据不一致的情况，也可以基于 SqlServer 再次全量构建出 MySQL 的数据。

因为我们在设计双写策略时，会确保 SqlServer 一定能写成功，也就是说，SqlServer 中的数据是全量最完整、最正确的。

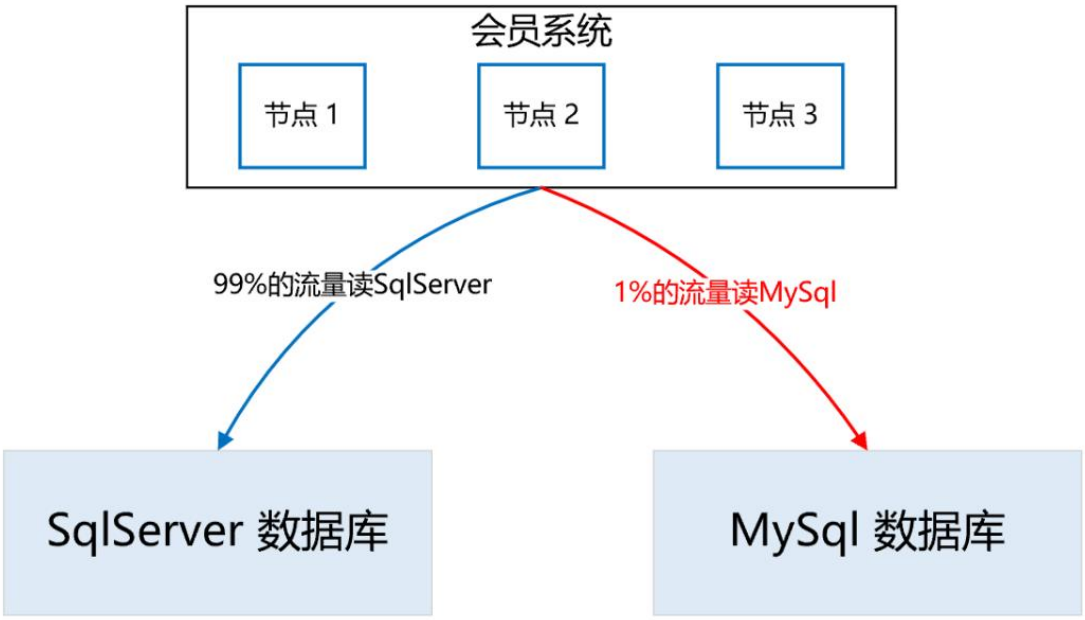
如下图所示：



讲完了双写，接下来我们看一下“读数据”如何灰度。整体思路是，通过 A/B 平台逐步灰度流量，刚开始 100% 的流量读取 SqlServer 数据库，然后逐步切流量读取 MySQL 数据库，先 1%，如果没有问题，再逐步放流量，最终 100% 的流量都走 MySQL 数据库。

在逐步灰度流量的过程中，需要有验证机制，只有验证没问题了，才能进一步放大流量。那么这个验证机制如何实施呢？方案是，在一次查询请求里，通过异步线程，比较 SqlServer 和 MySQL 的查询结果是否一致，如果不一致，记日志，再人工检查不一致的原因，直到彻底解决不一致的问题后，再逐步灰度流量。

如下图所示：



所以，整体的实施流程如下：

首先，在一个夜黑风高的深夜，流量最小的时候，完成 SqlServer 到 MySQL 数据库的全量数据同步。

接着，开启双写，此时，如果有用户注册，就会实时双写到两个数据库。那么，在全量同步和实时双写开启之间，两个数据库还相差这段时间的数据，所以需要再次增量同步，把数据补充完整，以防数据的不一致。

剩下的时间，就是各种日志监控，看双写是否有问题，看数据比对是否一致等等。

这段时间是耗时最长的，也是最容易发生问题的，如果有的问题比较严重，导致数据不一致了，就需要从头再来，再次基于 SqlServer 全量构建 MySQL 数据库，然后重新灰度流量。

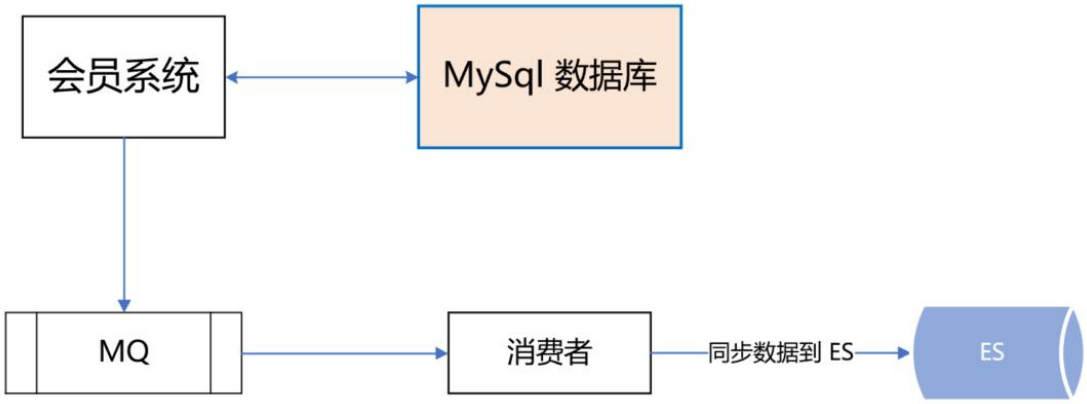
直到最后，100% 的流量全部灰度到 MySQL，此时就大功告成了，下线灰度逻辑，所有读写都切到 MySQL 集群。

5.3.3 MySQL 和 ES 主备集群方案

做到这一步，感觉会员主库应该没问题了，可 dal 组件的一次严重故障改变了我们的想法。

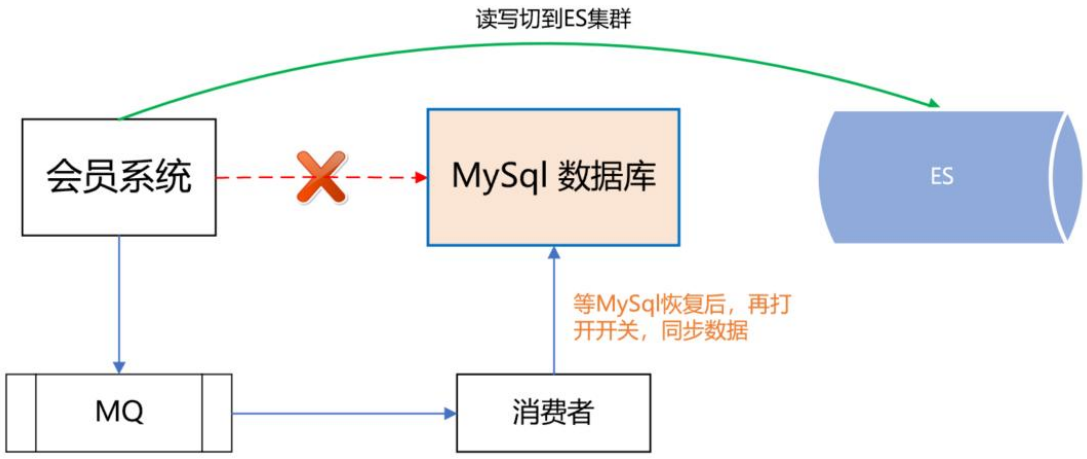
那次故障很恐怖，公司很多应用连接不上数据库了，创单量直线往下掉，这让我们意识到，即使数据库是好的，但 dal 组件异常，依然能让会员系统挂掉。

所以，我们再次异构了会员主库的数据源，双写数据到 ES，如下所示：



如果 dal 组件故障或 MySQL 数据库挂了，可以把读写切到 ES，等 MySQL 恢复了，再把数据同步到 MySQL，最后把读写再切回到 MySQL 数据库。

如下图所示：



5.4 异常会员关系治理

会员系统不仅仅要保证系统的稳定和高可用，数据的精准和正确也同样重要。

举个例子，一个分布式并发故障，导致一名用户的 APP 账户绑定了别人的微信小程序账户，这将会带来非常恶劣的影响。

首先，一旦这两个账号绑定了，那么这两个用户下的酒店、机票、火车票订单是互相可以看到的。

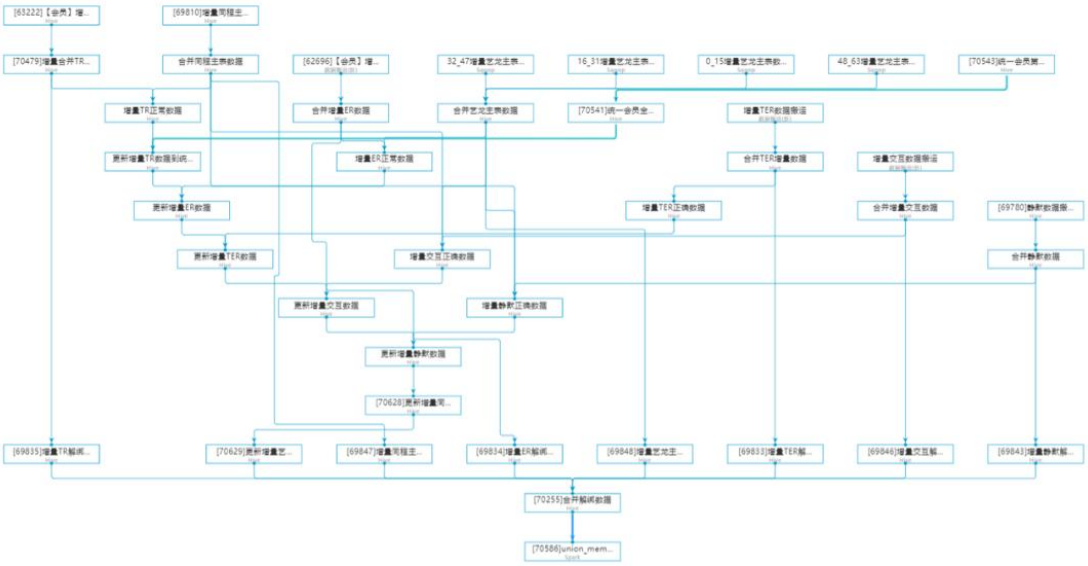
你想想，别人能看到你订的酒店订单，你火不火，会不会投诉？除了能看到别人的订单，你还能操作订单。

例如，一个用户在 APP 的订单中心，看到了别人订的机票订单，他觉得不是自己的订单，就把订单取消了。

这将会带来非常严重的客诉，大家知道，机票退订费用是挺高的，这不仅影响了该用户的正常出行，还导致了比较大的经济损失，非常糟糕。

针对这些异常会员账号，我们进行了详细的梳理，通过非常复杂烧脑的逻辑识别出这些账号，并对会员接口进行了深度优化治理，在代码逻辑层堵住了相关漏洞，完成了异常会员的治理工作。

如下图所示：



展望：更精细化的流控和降级策略

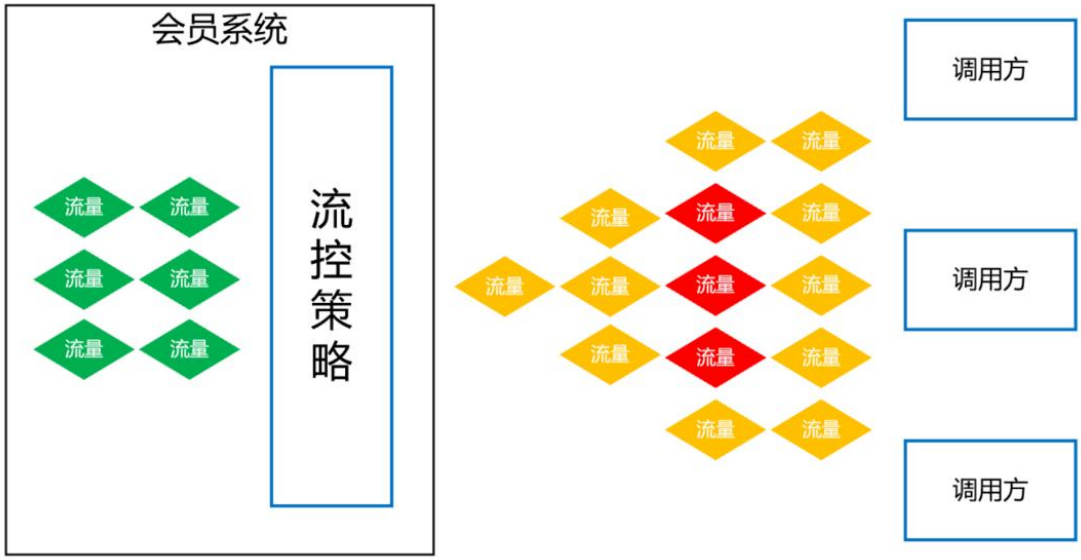
任何一个系统，都不能保证百分之一百不出问题，所以我们要有面向失败的设计，那就是更精细化的流控和降级策略。

5.5 更精细化的流控策略

热点控制。针对黑产刷单的场景，同一个会员 id 会有大量重复的请求，形成热点账号，当这些账号的访问超过设定阈值时，实施限流策略。

基于调用账号的流控规则。这个策略主要是防止调用方的代码 bug 导致的大流量。例如，调用方在一次用户请求中，循环很多次来调用会员接口，导致会员系统流量暴增很多倍。所以，要针对每个调用账号设置流控规则，当超过阈值时，实施限流策略。

全局流控规则。我们会员系统能抗下 tps 3 万多的秒并发请求量，如果此时，有个很恐怖的流量打过来，tps 高达 10 万，与其让这波流量把会员数据库、ES 全部打死，还不如把超过会员系统承受范围之外的流量快速失败，至少 tps 3 万内的会员请求能正常响应，不会让整个会员系统全部崩溃。



5.6 更精细化的降级策略

基于平均响应时间的降级。会员接口也有依赖其他接口，当调用其他接口的平均响应时间超过阈值，进入准降级状态。

如果接下来 1s 内进入的请求，它们的平均响应时间都持续超过阈值，那么在接下的时间窗口内，自动地熔断。

基于异常数和异常比例的降级。当会员接口依赖的其他接口发生异常，如果 1 分钟内的异常数超过阈值，或者每秒异常总数占通过量的比值超过阈值，进入降级状态，在接下的时间窗口之内，自动熔断。

目前，我们最大的痛点是会员调用账号的治理。公司内，想要调用会员接口，必须申请一个调用账号，我们会记录该账号的使用场景，并设置流控、降级策略的规则。

但在实际使用的过程中，申请了该账号的同事，可能异动到其他部门了，此时他可能也会调用会员系统，为了省事，他不会再次申请会员账号，而是直接沿用以前的账号过来调用，这导致我们无法判断一个会员账号的具体使用场景是什么，也就无法实施更精细的流控和降级策略。

6. 如何从零搭建 10 万级 QPS 大流量、高并发优惠券系统

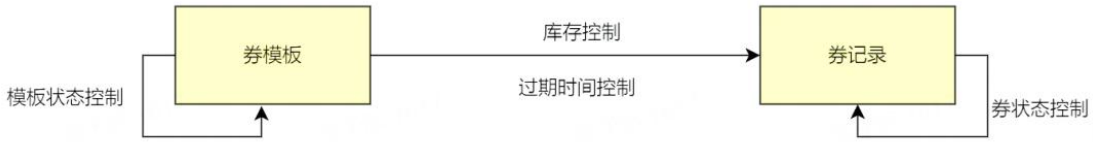
春节活动中，多个业务方都有发放优惠券的需求，且对发券的 QPS 量级有明确的需求。所有的优惠券发放、核销、查询都需要一个新系统来承载。因此，我们需要设计、开发一个能够支持十万级 QPS 的券系统，并且对优惠券完整的生命周期进行维护。

6.1 需求拆解及技术选型

6.1.1 需求拆解

- 要配置券，会涉及到券批次（券模板）创建，券模板的有效期以及券的库存信息
- 要发券，会涉及到券记录的创建和管理（过期时间，状态）

因此，我们可以将需求先简单拆解为两部分：



同时，无论是券模板还是券记录，都需要开放查询接口，支持券模板/券记录的查询。

6.1.2 系统选型及中间件

确定了基本的需求，我们根据需求，进一步分析可能会用到的中间件，以及系统整体的组织方式。

存储

由于券模板、券记录这些都是需要持久化的数据，同时还需要支持条件查询，所以我们选用通用的结构化存储 MySQL 作为存储中间件。

缓存

- 由于发券时需要券模板信息，大流量情况下，不可能每次都从 MySQL 获取券模板信息，因此考虑引入缓存
- 同理，券的库存管理，或者叫库存扣减，也是一个高频、实时的操作，因此也考虑放入缓存中

主流的缓存 Redis 可以满足我们的需求，因此我们选用 Redis 作为缓存中间件。

消息队列

由于券模板/券记录都需要展示过期状态，并且根据不同的状态进行业务逻辑处理，因此有必要引入延迟消息队列来对券模板/券状态进行处理。RocketMQ 支持延时消息，因此我们选用 RocketMQ 作为消息队列。

系统框架

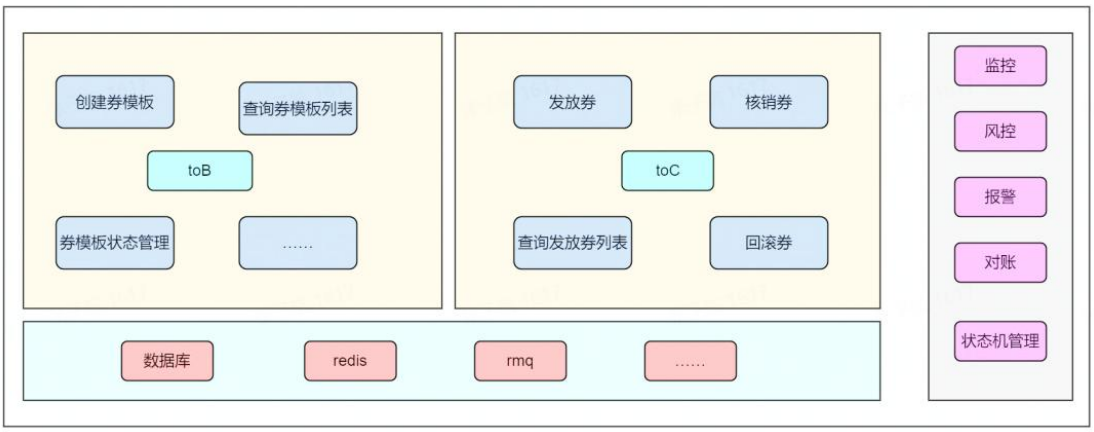
发券系统作为下游服务，是需要被上游服务所调用的。公司内部服务之间，采用的都是 RPC 服务调用，系统开发语言使用的是 golang，因此我们使用 golang 服务的 RPC 框架 kitex 进行代码编写。

我们采用 kitex+MySQL+Redis+RocketMQ 来实现发券系统，RPC 服务部署在公司的 docker 容器中。

6.2 系统开发与实践

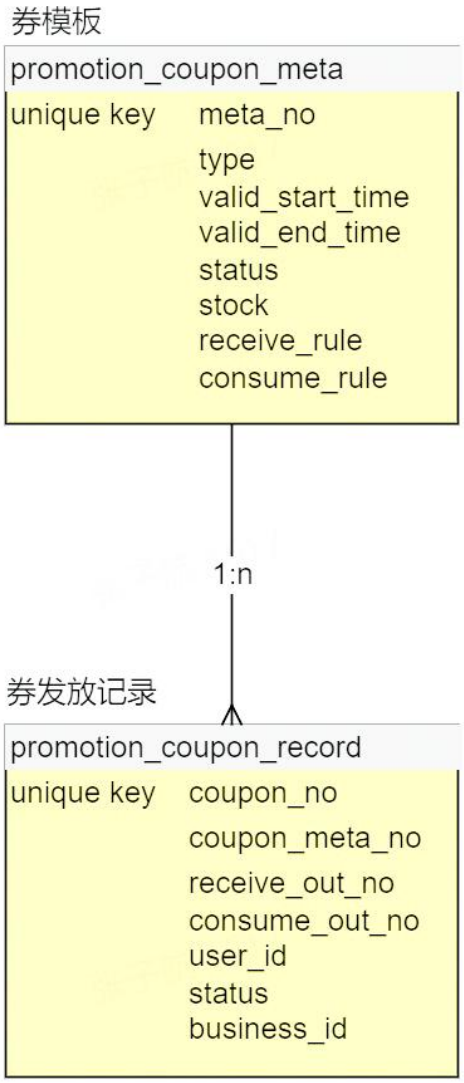
6.2.1 系统整体架构

从需求拆解部分我们对大致要开发的系统有了一个了解，下面给出整体的一个系统架构，包含了一些具体的功能。



6.2.2 数据结构 ER 图

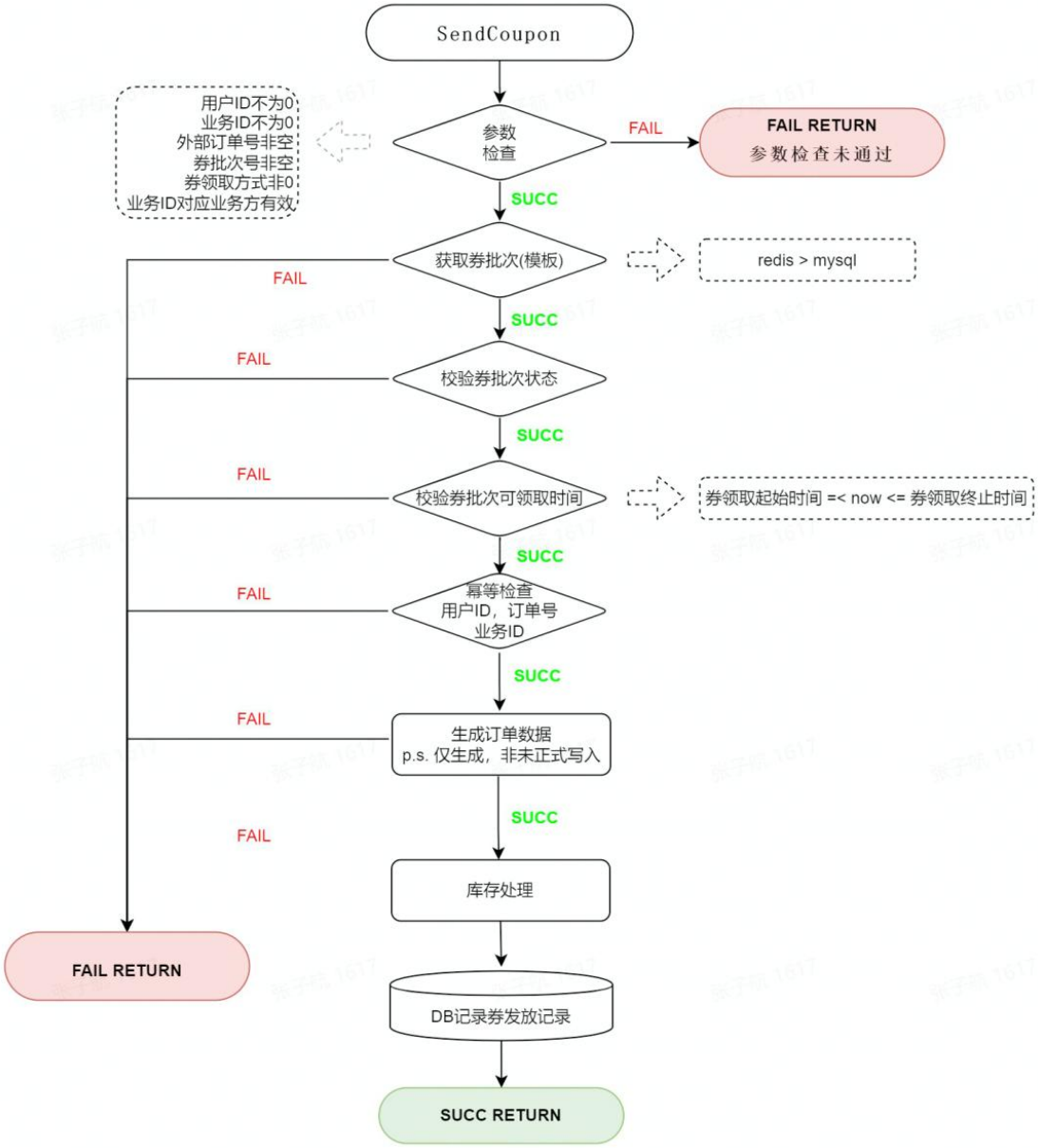
与系统架构对应的，我们需要建立对应的 MySQL 数据存储表。



6.2.3 核心逻辑实现

6.2.3.1 发券：

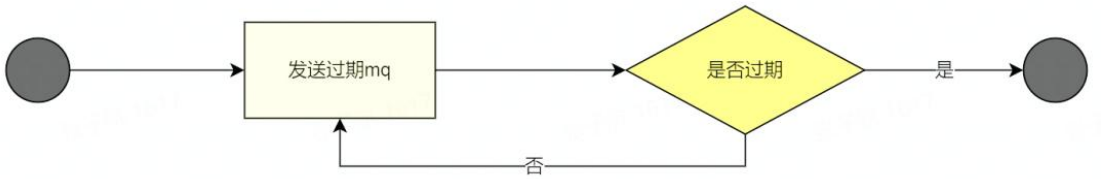
发券流程分为三部分：参数校验、幂等校验、库存扣减。



幂等操作用于保证发券请求不正确的情况下，业务方通过重试、补偿的方式再次请求，可以最终只发出一张券，防止资金损失。

6.2.3.2 券过期：

券过期是一个状态推进的过程，这里我们使用 RocketMQ 来实现。



由于 RocketMQ 支持的延时消息有最大限制，而卡券的有效期不固定，有可能会超过限制，所以我们将卡券过期消息循环处理，直到卡券过期。

6.3 大流量、高并发场景下的问题及解决方案

实现了系统的基本功能后，我们来讨论一下，如果在大流量、高并发的场景下，系统可能会遇到的一些问题及解决方案。

6.3.1 存储瓶颈及解决方案

瓶颈：

在系统架构中，我们使用了 MySQL、Redis 作为存储组件。我们知道，单个服务器的 I/O 能力终是有限的，在实际测试过程中，能够得到如下的数据：

- 单个 MySQL 的每秒写入在 4000 QPS 左右，超过这个数字，MySQL 的 I/O 时延会剧量增长。
- MySQL 单表记录到达了千万级别，查询效率会大大降低，如果过亿的话，数据查询会成为一个问题。
- Redis 单分片的写入瓶颈在 2w 左右，读瓶颈在 10w 左右

解决方案：

1. 读写分离。在查询券模板、查询券记录等场景下，我们可以将 MySQL 进行读写分离，让这部分查询流量走 MySQL 的读库，从而减轻 MySQL 写库的查询压力。
2. 分治。在软件设计中，有一种分治的思想，对于存储瓶颈的问题，业界常用的方案就是分而治之：流量分散、存储分散，即：分库分表。
3. 发券，归根结底是要对用户的领券记录做持久化存储。对于 MySQL 本身 I/O 瓶颈来说，我们可以在不同服务器上部署 MySQL 的不同分片，对 MySQL

做水平扩容，这样一来，写请求就会分布在不同的 MySQL 主机上，这样就能够大幅提升 MySQL 整体的吞吐量。

4. 给用户发了券，那么用户肯定需要查询自己获得的券。基于这个逻辑，我们以 user_id 后四位为分片键，对用户领取的记录表做水平拆分，以支持用户维度的领券记录的查询。
5. 每种券都有对应的数量，在给用户发券的过程中，我们是将发券数记录在 Redis 中的，大流量的情况下，我们也需要对 Redis 做水平扩容，减轻 Redis 单机的压力。

6.3.1.1 容量预估

基于上述思路，在要满足发券 12w QPS 的需求下，我们预估一下存储资源。

a. MySQL 资源

在实际测试中，单次发券对 MySQL 有一次非事务性写入，MySQL 的单机的写入瓶颈为 4000，据此可以计算我们需要的 MySQL 主库资源为：

120000/4000 = 30

b. Redis 资源

假设 12w 的发券 QPS，均为同一券模板，单分片的写入瓶颈为 2w，则需要的最少 Redis 分片为：

120000/20000 = 6

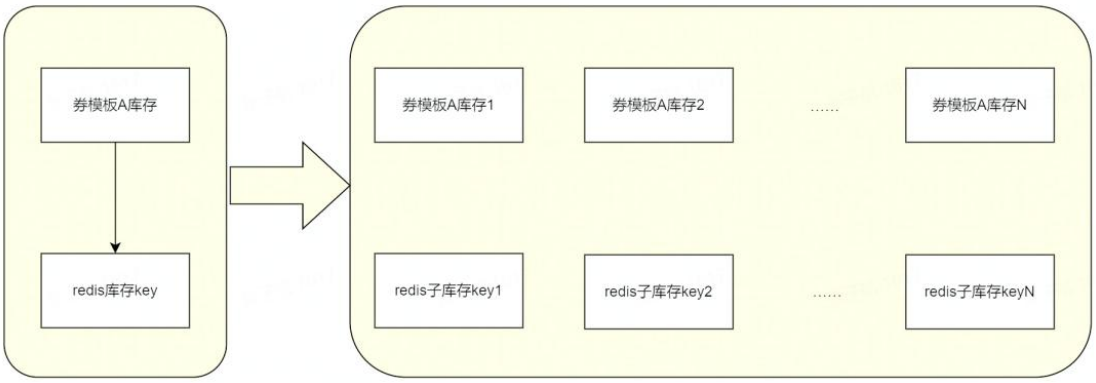
6.3.1.2 热点库存问题及解决方案

问题

大流量发券场景下，如果我们使用的券模板为一个，那么每次扣减库存时，访问到的 Redis 必然是特定的一个分片，因此，一定会达到这个分片的写入瓶颈，更严重的，可能会导致整个 Redis 集群不可用。

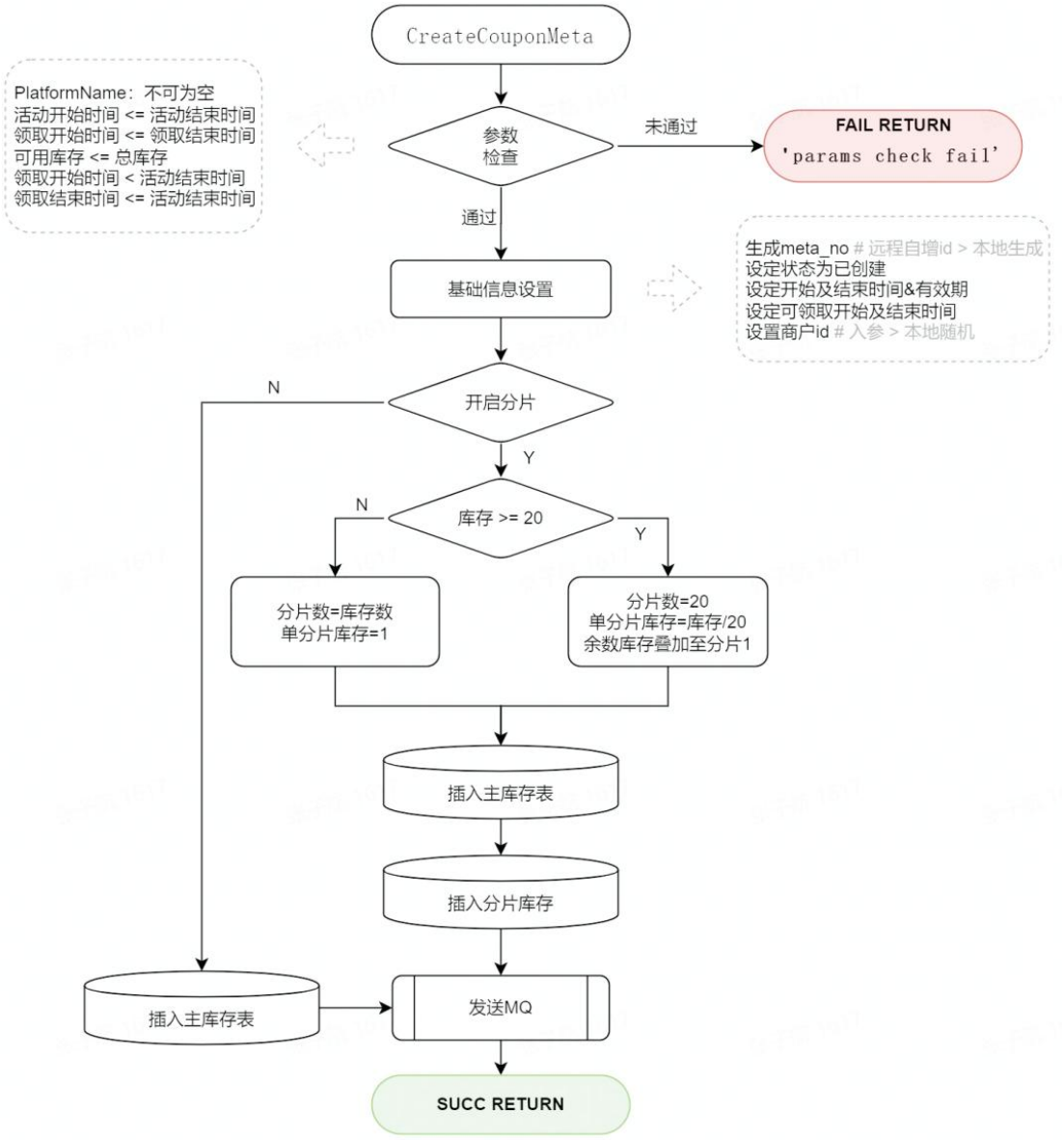
解决方案

热点库存的问题，业界有通用的方案：即，扣减的库存 key 不要集中在某一个分片上。如何保证这一个券模板的 key 不集中在某一个分片上呢，我们拆 key（拆库存）即可。如图：

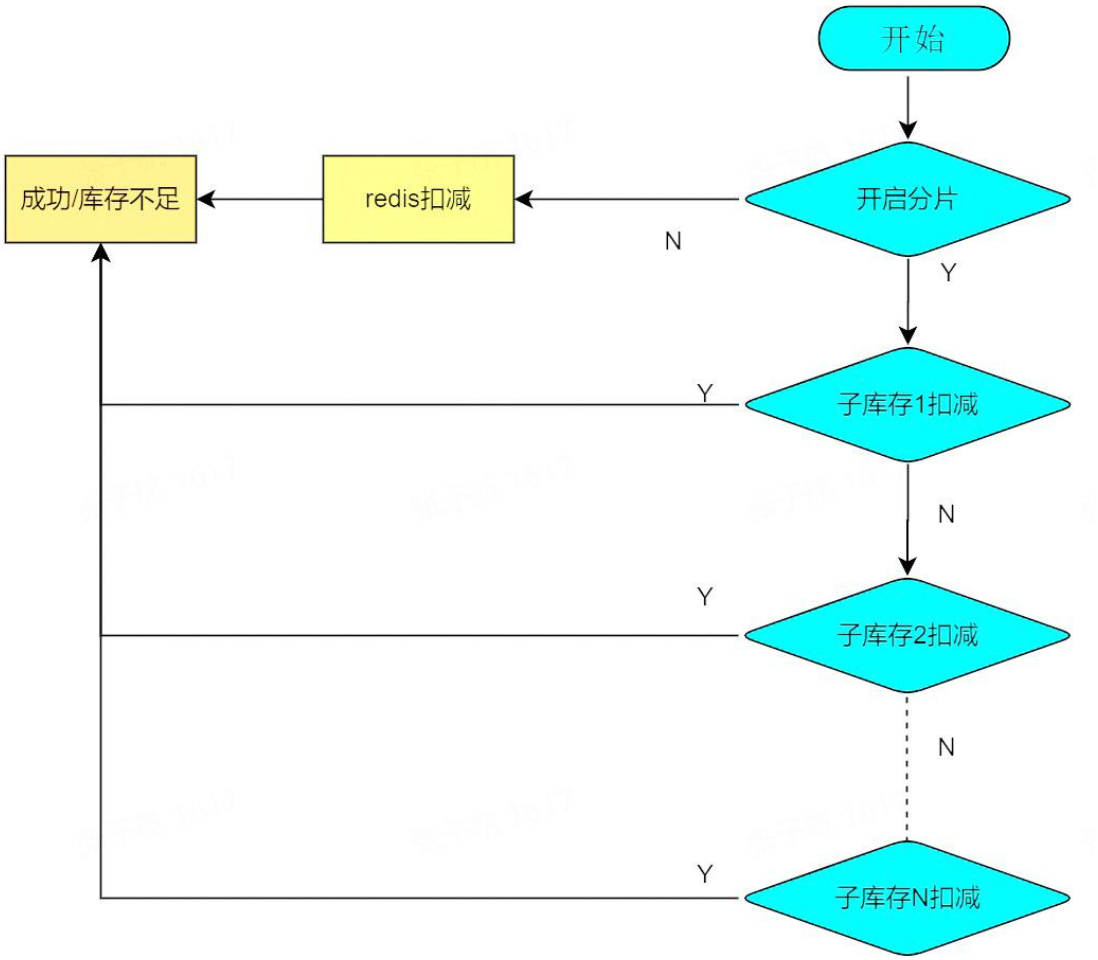


在业务逻辑中，我们在建券模板的时候，就将这种热点券模板做库存拆分，后续扣减库存时，也扣减相应的子库存即可。

6.3.1.3 建券



6.3.1.4 库存扣减



这里还剩下一个问题，即：扣减子库存，每次都是从 1 开始进行的话，那对 Redis 对应分片的压力其实并没有减轻，因此，我们需要做到：每次请求，随机不重复的轮询子库存。以下是本项目采取的一个具体思路：

> Redis 子库存的 key 的最后一位是分片的编号，如：xxx_stock_key1、xxx_stock_key2……，在扣减子库存时，我们先生成对应分片总数的随机不重复数组，如第一次是[1,2,3]，第二次可能是[3,1,2]，这样，每次扣减子库存的请求，就会分布到不同的 Redis 分片上，缓轻 Redis 单分片压力的同时，也能支持更高 QPS 的扣减请求。

这种思路的一个问题是，当我们库存接近耗尽的情况下，很多分片子库存的轮询将变得毫无意义，因此我们可以在每次请求的时候，将子库存的剩余量记录下来，当某一个券模板的子库存耗尽后，随机不重复的轮询操作直接跳过这个子库存分片，这样能够优化系统在库存即将耗尽情况下的响应速度。

业界针对 Redis 热点 key 的处理，除了分 key 以外，还有一种 key 备份的思路：即，将相同的 key，用某种策略备份到不同的 Redis 分片上去，这样就能将热点打散。这种思路适用于那种读多写少的场景，不适合应对发券这种大流量写的场景。在面对具体的业务场景时，我们需要根据业务需求，选用恰当的方案来解决问题。

6.3.2 券模板获取失败问题及解决方案

问题

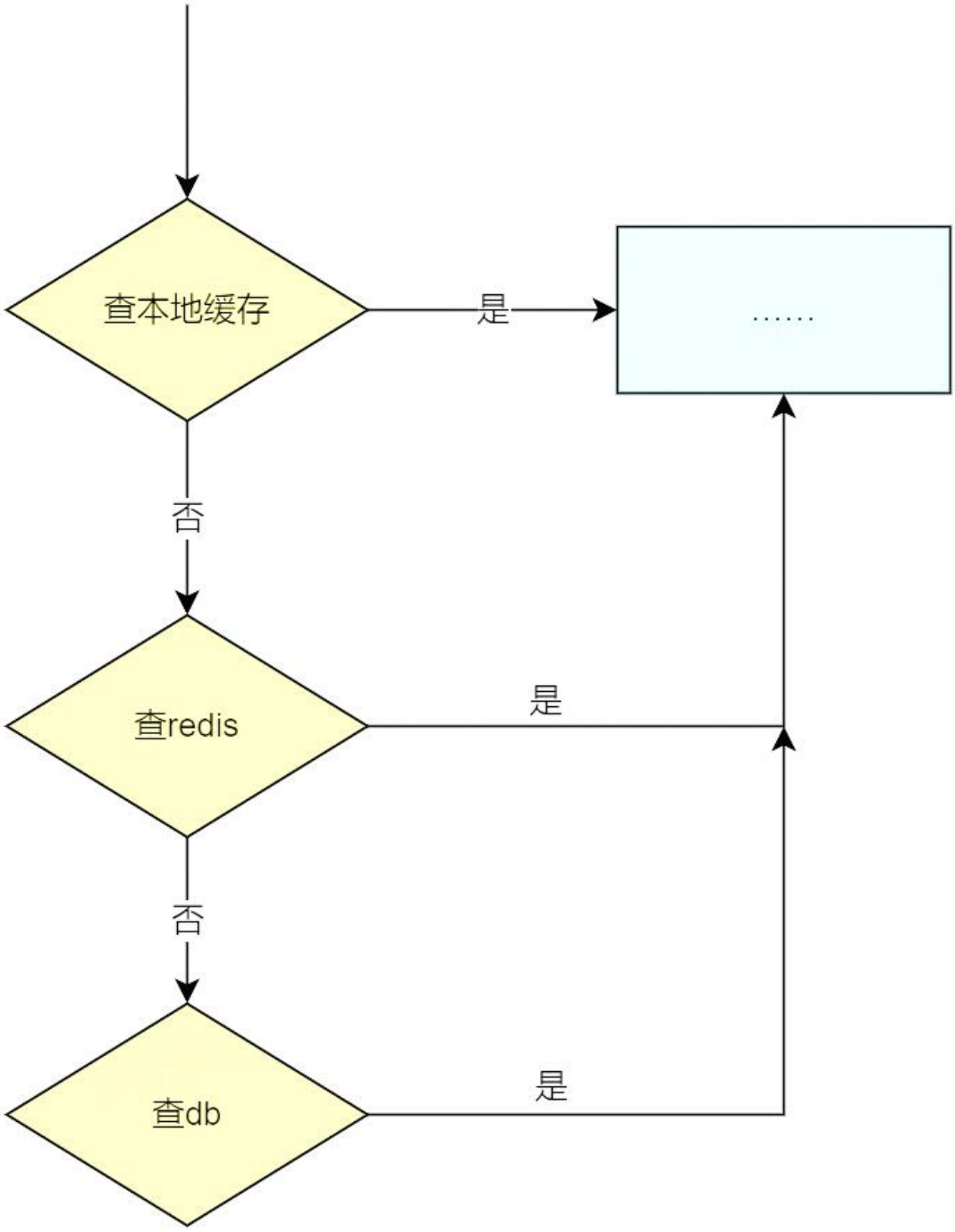
> 高 QPS，高并发的场景下，即使我们能将接口的成功率提升 0.01%，实际表现也是可观的。现在回过头来看下整个发券的流程：查券模板（Redis）-->校验-->幂等（MySQL）--> 发券（MySQL）。在查券模板信息时，我们会请求 Redis，这是强依赖，在实际的观测中，我们会发现，Redis 超时的概率大概在万分之 2、3。因此，这部分发券请求是必然失败的。

解决方案

为了提高这部分请求的成功率，我们有两种方案。

一是从 Redis 获取券模板失败时，内部进行重试；二是将券模板信息缓存到实例的本地内存中，即引入二级缓存。

内部重试可以提高一部分请求的成功率，但无法从根本上解决 Redis 存在超时的问题，同时重试的次数也和接口响应的时长成正比。二级缓存的引入，可以从根本上避免 Redis 超时造成的发券请求失败。因此我们选用二级缓存方案：



当然，引入了本地缓存，我们还需要在每个服务实例中启动一个定时任务来将最新的券模板信息刷入到本地缓存和 Redis 中，将模板信息刷入 Redis 中时，要加分布式锁，防止多个实例同时写 Redis 给 Redis 造成不必要的压力。

6.3.3 服务治理

系统开发完成后，还需要通过一系列操作保障系统的可靠运行。

1. 超时设置。优惠券系统是一个 RPC 服务，因此我们需要设置合理的 RPC 超时时间，保证系统不会因为上游系统的故障而被拖垮。例如发券的接口，我们内部执行时间不超过 100ms，因此接口超时我们可以设置为 500ms，如果有异常请求，在 500ms 后，就会被拒绝，从而保障我们服务稳定的运行。
2. 监控与报警。对于一些核心接口的监控、稳定性、重要数据，以及系统 CPU、内存等的监控，我们会在 Grafana 上建立对应的可视化图表，在春节活动期间，实时观测 Grafana 仪表盘，以保证能够最快观测到系统异常。同时，对于一些异常情况，我们还有完善的报警机制，从而能够第一时间感知到系统的异常。
3. 限流。优惠券系统是一个底层服务，实际业务场景下会被多个上游服务所调用，因此，合理的对这些上游服务进行限流，也是保证优惠券系统本身稳定性必不可少的一环。
4. 资源隔离。因为我们服务都是部署在 docker 集群中的，因此为了保证服务的高可用，服务部署的集群资源尽量分布在不同的物理区域上，以避免由集群导致的服务不可用。

6.4 系统压测及实际表现

做完了上述一系列的工作后，是时候检验我们服务在生产环境中的表现了。当然，新服务上线前，首先需要对服务进行压测。这里总结一下压测可能需要注意的一些问题及压测结论。

注意事项

首先是压测思路，由于我们一开始无法确定 docker 的瓶颈、存储组件的瓶颈等。所以我们的压测思路一般是：

1. 找到单实例瓶颈
2. 找到 MySQL 一主的写瓶颈、读瓶颈
3. 找到 Redis 单分片写瓶颈、读瓶颈

得到了上述数据后，我们就可以粗略估算所需要的资源数，进行服务整体的压测了。

1. 压测资源也很重要，提前申请到足量的压测资源，才能合理制定压测计划。

2. 压测过程中，要注意服务和资源的监控，对不符合预期的部分要深入思考，优化代码。
3. 适时记录压测数据，才能更好的复盘。
4. 实际的使用资源，一般是压测数据的 1.5 倍，我们需要保证线上有部分资源冗余以应对突发的流量增长。

6.5 结论

系统在 13w QPS 的发券请求下，请求成功率达到 99.9%以上，系统监控正常。春节红包雨期间，该优惠券系统承载了两次红包雨的全部流量，期间未出现异常，圆满完成了发放优惠券的任务。

系统的业务思考

目前的系统，只是单纯支持了高并发的发券功能，对于券的业务探索并不足够。后续需要结合业务，尝试批量发券（券包）、批量核销等功能

发券系统只是一个最底层的业务中台，可以适配各种场景，后续可以探索支持更多业务。

从零搭建一个大流量、高并发的优惠券系统，首先应该充分理解业务需求，然后对需求进行拆解，根据拆解后的需求，合理选用各种中间件；本文主要是要建设一套优惠券系统，因此会使用各类存储组件和消息队列，来完成优惠券的存储、查询、过期操作；

在系统开发实现过程中，对核心的发券、券过期实现流程进行了阐述，并针对大流量、高并发场景下可能遇到的存储瓶颈、热点库存、券模板缓存获取超时的问题提出了对应的解决方案。其中，我们使用了分治的思想，对存储中间件进行水平扩容以解决存储瓶颈；采取库存拆分子库存思路解决热点库存问题；引入本地缓存解决券模板从 Redis 获取超时的问题。最终保证了优惠券系统在大流量高并发的情景下稳定可用；

除开服务本身，我们还从服务超时设置、监控报警、限流、资源隔离等方面对服务进行了治理，保障服务的高可用；

压测是一个新服务不可避免的一个环节，通过压测我们能够对服务的整体情况有个明确的了解，并且压测期间暴露的问题也会是线上可能遇到的，通过压测，我们能够对新服务的整体情况做到心里有数，对服务上线正式投产就更有信心了。

7. 设计一个支持 10W QPS 的评论中台，你会怎么设计?

在 B 站，UP 主每天都会发布海量的视频、动态、专栏等内容，随之而来的是弹幕和评论区的各种讨论。

播放器中直接滚动播放的弹幕，如同调味剂，重在提升视频观看体验；

而点进评论区，相对而言评论文本更长，内容的观点、形式都更丰富，更像是饭后甜点。

随着业务不断发展，B 站的评论系统逐渐组件化、平台化；

通过持续演进架构设计，管理不断上升的系统复杂度，从而更好地满足各类用户的需求。

评论的基础功能模块是相对稳定的。

1. 发布评论：支持无限盖楼回复。
2. 读取评论：按照时间、热度排序；显示评论数、楼中楼等。
3. 删除评论：用户删除、UP 主删除等。
4. 评论互动：点赞、点踩、举报等。
5. 管理评论：置顶、精选、后台运营管理（搜索、删除、审核等）。

结合 B 站以及其他互联网平台的评论产品特点，评论一般还包括一些更高阶的基础功能：

6. 评论富文本展示：例如表情、@、分享链接、广告等。
7. 评论标签：例如 UP 主点赞、UP 主回复、好友点赞等。
8. 评论装扮：一般用于凸显发评人的身份等。
9. 热评管理：结合 AI 和人工，为用户营造更好的评论区氛围。

7.1 总体的架构设计

评论系统 中台，从总体的架构上来区分，分为：

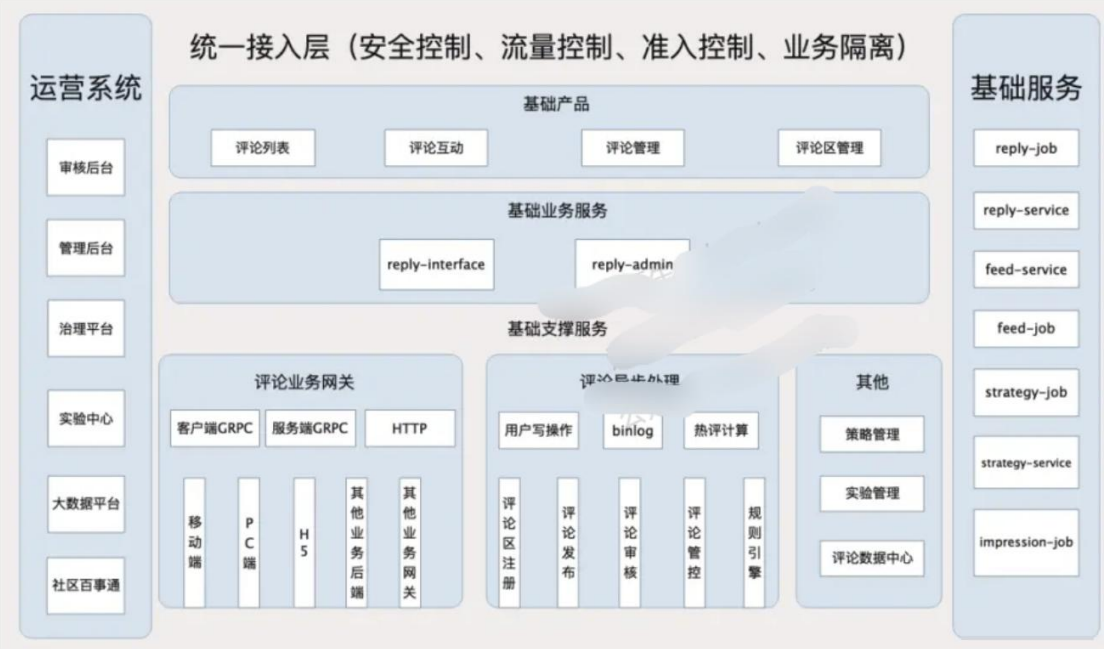
（1）接入层

（2）服务层

（3）异步任务层

（4）cache 层

（5）DB 层



7.2 接入层架构 reply-interface

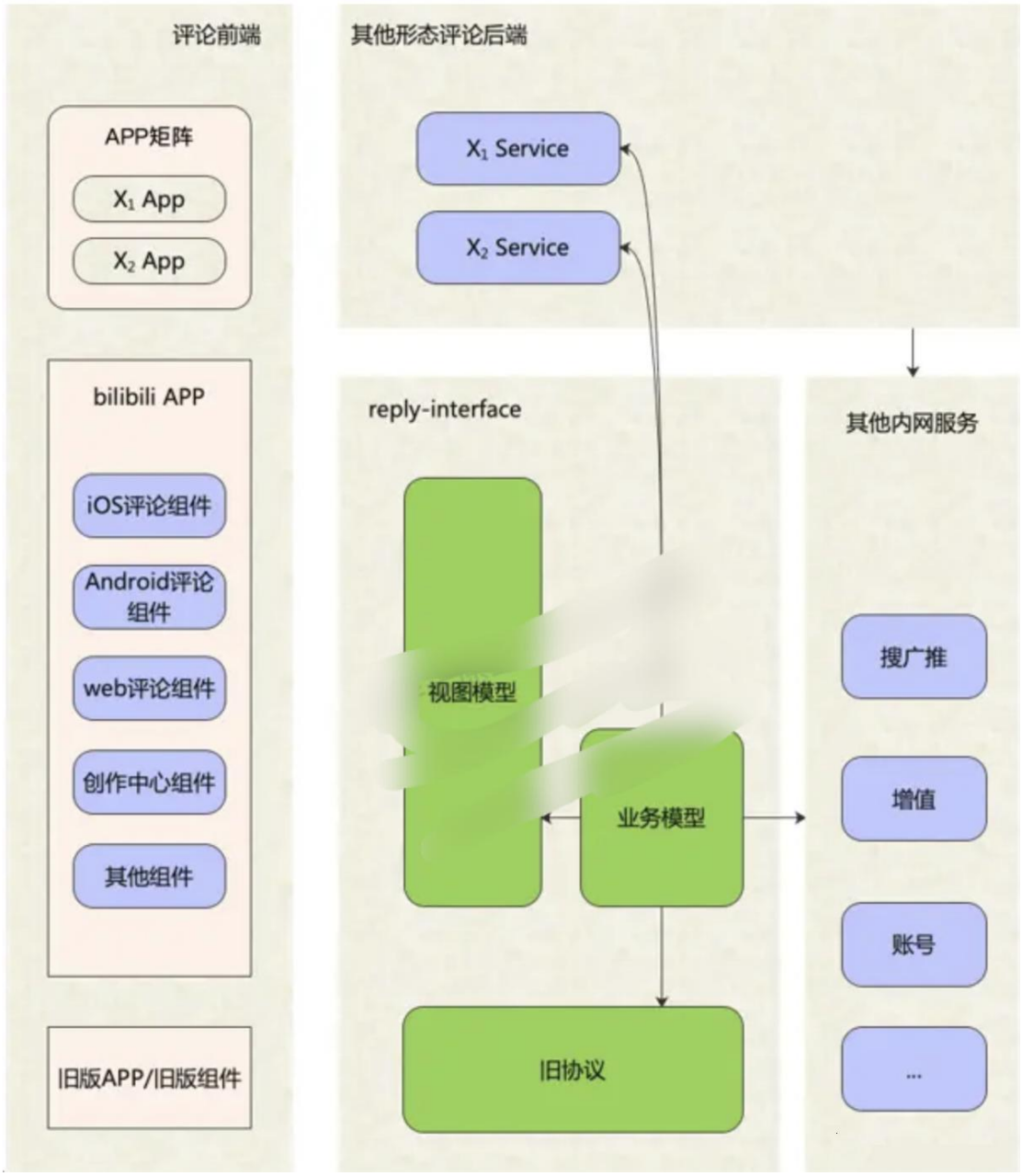
reply-interface 是评论系统的接入层，主要服务于两种调用者：

一是客户端的评论组件，

二是基于评论系统做二次开发或存在业务关联的其他业务后端。

面向移动端/WEB 场景，设计一套基于视图模型的 API，利用客户端提供的布局能力，接入层负责组织业务数据模型，并转换为视图模型，编排后下发给客户端。

面向服务端场景，接入层设计的 API 需要体现清晰的系统边界，最小可用原则对外提供数据，同时做好安全校验和流量控制。



接入层整个业务数据模型组装，分为两个步骤：

一是服务编排，

二是数据组装。

服务编排拆的架构为：

（1）对服务进行分层，分为若干个层级，

（2）前置依赖通过流水线调用，

（3）同一层级的可以并发调用，结构性提升了复杂调用场景下的接口性能下限；

（4）针对不同依赖服务所提供的 SLA 不同，设置不同的降级处理、超时控制和服务限流方案，保证少数弱依赖抖动甚至完全不可用情况下评论服务可用。

SLA 一般指服务级别协议。服务级别协议是指提供服务的企业与客户之间就服务的品质、水准、性能等方面所达成的双方共同认可的协议或契约。

7.3 服务层架构

7.3.1 评论管理服务层 reply-admin

评论管理服务层，为多个内部管理后台提供服务。

运营人员的数据查询具有：

1. 组合、关联查询条件复杂；
2. 刚需关键词检索能力；
3. 写后读的可靠性与实时性要求高等特征。

此类查询需求，ES 几乎是不二选择。

但是由于业务数据量较大，需要为多个不同的查询场景建立多种索引分片，且数据更新实时性不高。

因此，我们基于 ES 做了一层封装，提供统一化的数据检索能力，并结合在线数据库刷新部分实时性要求较高的字段。

7.3.2 评论基础服务 reply-service 架构设计

评论基础服务层，专注于评论功能的原子功能，例如：

- 查询评论列表
- 删除评论等。

这一层的特点是：

- 较少做业务逻辑变更的，
- 极高的可用性
- 极高性能吞吐。

这一层采用了多种高性能方案：

- 多级缓存
- 布隆过滤器
- 热点探测等。

7.3.3 异步任务层 reply-job 架构设计

异步任务层，主要有两个职责：

1. 为原子的业务操作操作，提供异步协助

与 reply-service 协同，为评论基础功能的原子化实现做架构上的补充。

2. 异步削峰处理

为 长耗时/高吞吐的调用， 做异步化/削峰处理

职责 1：提供异步协助

为原子的业务操作操作，提供异步协助， 最典型的案例就是缓存的更新。

一般采用 Cache Aside 模式，先读缓存，再读 DB；

Cache Aside 模式下的缓存的重建策略：就是读请求未命中缓存穿透到 DB，从 DB 读取到内容之后反写缓存。

这一套流程对外提供了一个原子化的数据读取功能。

但由于部分缓存数据项的重建代价较高，比如评论列表。

为啥呢？

由于列表是分页的，缓存重建时会启用预加载，也就是要多加载几页，

如果短时间内大量请求缓存未命中，并且多个服务节点的同时重建缓存，容易造成 DB 抖动。

解决方案是啥？

利用消息队列+reply-job ，实现单个评论列表异步重建，只重建一次缓存。

另外呢，reply-job 还作为数据库 binlog 的消费者，执行缓存的更新操作。

职责 2：异步削峰处理

与 reply-interface 协同，为 长耗时/高吞吐的调用，做异步化/削峰处理

诸如评论发布等操作，基于安全/策略考量，会有非常重的前置调用逻辑。

对于用户来说，这个长耗时几乎是不可接受的。同时，时事热点容易造成发评论的瞬间峰值流量。

因此，reply-interface 在处理完一些必要校验逻辑之后，会通过消息队列送至 reply-job 异步处理，包括送审、写 DB、发通知等。

那么异步处理后用户体验是如何保证的呢?

首先是当次交互，返回最新数据。

C 端的发评接口会返回展示新评论所需的数据内容，客户端据此展示新评论，完成一次用户交互。

其次，控制延迟时长，如果太长则进行预警和调优

若用户重新刷新页面，因为发评的异步处理端到端延迟基本在 2s 以内，此时所有数据已准备好，不会影响用户体验。

7.3.4 消息队列的保证有序

利用了消息队列的「有序」特性，将单个评论区内的发评串行处理，避免了并行处理导致的一些数据错乱风险。

一个有趣的问题是，早年间评论显示楼层号，楼层号实际是计数器，且在一个评论区范围内不能出现重复。

因此，这个楼层发号操作必须是在一个评论区范围内串行的（或者用更复杂的锁实现），否则两条同时发布的评论，获取的楼层号就是重复的。

而分布式部署+负载均衡的网关，处理发评论请求是无法实现这种串行的，因此需要放到消息队列中处理。

7.4 数据存储架构

7.4.1 结构化模型设计

结合评论的产品功能要求，评论需要至少两张表：

（1）首先是评论表，主键是评论 id，关键索引是评论区 id；

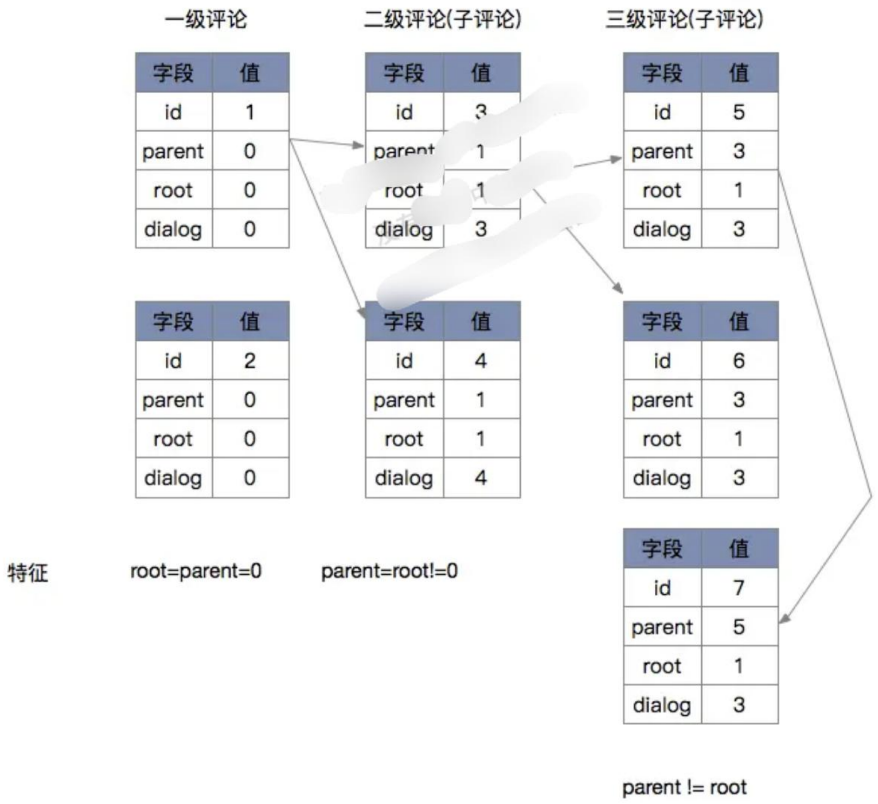
（2）其次是评论区表，主键是评论区 id，平台化之后增加一个评论区 type 字段，与评论区 id 组成一个”联合主键“。

（3）评论内容表. 由于评论内容是大字段，且相对独立、很少修改，因此独立设计第 3 张表。主键也是评论 id。

评论表和评论区表的字段主要包括 4 种：

1. 关系类，包括发布人、父评论等，这些关系型数据是发布时已经确定的，基本不会修改。
2. 计数类，包括总评论数、根评论数、子评论数等，一般会在有评论发布或者删除时修改。
3. 状态类，包括评论/评论区状态、评论/评论区属性等，评论/评论区状态是一个枚举值，描述的是正常、审核、删除等可见性状态；评论/评论区属性是一个整型的bitmap，可用于描述评论/评论区的一些关键属性，例如 UP 主点赞等。
4. 其他，包括 meta 等，可用于存储一些关键的附属信息。

评论回复的树形关系，如下图所示：



以评论列表的访问为例，我们的查询 SQL 可能是（已简化）：

1. 查询评论区基础信息：SELECT * FROM subject WHERE obj_id=? AND obj_type=?
2. 查询时间序一级评论列表：SELECT id FROM reply_index WHERE obj_id=? AND obj_type=? AND root=0 AND state=0 ORDER BY floor=? LIMIT 0,20

3. 批量查询根评论基础信息：SELECT * FROM reply_index,reply_content WHERE rpid in (?,?,...)
4. 并发查询楼中楼评论列表：SELECT id FROM reply_index WHERE obj_id=? AND obj_type=? AND root=? ORDER BY like_count LIMIT 0,3
5. 批量查询楼中楼评论基础信息：SELECT * FROM reply_index,reply_content WHERE rpid in (?,?,...)

7.4.2 分库分表架构

评论系统对数据库的选型要求，有两个基本且重要的特征：

6. 必须有事务；
7. 必须容量大。

一开始，B 站采用的是 MySQL 分表来满足这两个需求。MySQL 分库分表数据量起来之后，原来的 MySQL 分表架构很快到达存储瓶颈。

> 提示：
> mysql 不停服在线扩容实际非常复杂，很多公司选择停服切换， 估计 B 站为了不停服， 或者不愿意发生停服的风险， 选择了 专门的商用 分布式 TiDB， 毕竟这个是花了钱的。

于是从 2020 年起，我们逐步迁移到 TiDB，从而具备了在线水平扩容能力。

7.4.3 高并发写入架构，TPS 提升了 10 倍以上

面对 10Wqps 的并发写入超大规模吞吐量，做了如下优化：

方案一：内存聚合+ 批量写入评论区评论计数的更新，先做内存合并再更新，可以减少热点场景下的 SQL 执行条数；评论表的插入，改成批量写入。

方案二：核心逻辑和非核心异步化，为核心操作瘦身

非数据库写操作的其他业务逻辑，拆分为前置和后置两部分，

其他业务逻辑从数据写入主线程中剥离，交由其他的线程池并发执行。

总之，采用新的高并发写入架构之后，性能得到极大提升。

写入架构调整之后，系统的并发处理能力有了极大提升，同时支持配置并行度/聚合粒度，在吞吐方面具备更大的弹性，热点评论区发评论的 TPS 提升了 10 倍以上。

7.5 缓存层架构

7.5.1 数据的缓存模型架构

主要有 3 项缓存：

8. subject，对应于「查询评论区基础信息」，

redis string 类型，value 使用 JSON 序列化方式存入。

9. reply_index，对应于「查询 xxx 评论列表」，

redis sorted set 类型。

member 是评论 id，score 对应于 ORDER BY 的字段，如 floor、like_count 等。

10. reply_content，对应于「查询 xxx 评论基础信息」

存储内容包括同一个评论 id 对应的 reply_index 和 reply_content 表的两部分字段。

7.5.2 缓存的一致性架构

缓存的一致性依赖 binlog 刷新，主要两个要点：

11. 消息队列，保证 同一个评论区内有序

binlog 投递到消息队列，分片 key 选择的是评论区，保证单个评论区和单个评论的更新操作是串行的，消费者顺序执行，保证对同一个 member 的 zadd 和 zrem 操作不会顺序错乱。

12. 采用删除缓存而非直接更新的方式

程序主动写缓存和 binlog 刷缓存，都采用删除缓存而非直接更新的方式，避免并发写操作时，特别是诸如 binlog 延迟、网络抖动等异常场景下的数据错乱。

7.5.3 缓存击穿解决方案

那大量写操作后读操作缓存命中率低的问题如何解决呢？

读缓存的时候， 可以利用 锁的机制，进行同步控制，防止缓存击穿。

7.5.4 热点探测架构

除了写热点，评论的读热点也有一些典型的特征：

13. 由于大量接口都需要读取评论区基础信息，存在**读放大**，因此该操作是最先感知到读热点存在的。
14. 由于评论业务的下游依赖较多，且多是批量查询，对下游来说也是读放大。此外，很多依赖是体量相对小的业务单元，数据稀疏，难以承载评论的大流量。
15. 评论的读热点集中在评论列表的第一页，以及热评的热评。
16. 评论列表的业务数据模型也包含部分个性化信息。

在读取评论区基础信息阶段探测热点，并将热点标识传递至服务层；

服务层实现了页面请求级的热点本地缓存，感知到热点后即读取本地缓存，然后再加载个性化信息。

热点探测的实现基于单机的滑动窗口+LFU，那么如何定义、计算相应的热点条件阈值呢?

首先，我们进行系统容量设计，列出容量计算的数学公式，主要包括各接口 QPS 的关系、服务集群总 QPS 与节点数的关系、接口 QPS 与 CPU/网络吞吐的关系等；

然后，收集系统内部以及相应依赖方的一些的热点相关统计信息，通过公式，计算出探测数据项的单机 QPS 热点阈值。

最后通过热点压测，来验证相应的热点配置与代码实现是符合预期的。

7.6 高可用架构

包括:

（1）缓存降级与 DB 降级

（2）同城读双活 + 双机房独立部署 架构

（3）副本数据延迟优化架构

（4）限流熔断策略的优化

7.6.1 缓存降级与 DB 降级

基础服务层集成了多级缓存，在上一级缓存未命中或者出现网络错误后，降级至下一级缓存，

缓存没有命中，就降级到 DB，保证系统的的可用性。

7.6.2 同城读双活 + 双机房独立部署 架构

评论系统是一个同城读双活+ 双机房独立部署 的架构。

DB 和 redis 均支持多副本，具备水平扩容的弹性。

7.6.3 双机房独立部署 ：

数据库与缓存均是双机房独立部署的，通过 db-proxy，或者 db-redis 进行访问

7.6.3.1 副本数据延迟优化架构

双机房架构场景下，存在跨机房数据延迟问题，

采用如下的策略解决：

- 入口层切流
- 应用层补偿
- 跨机房重试

尽可能保证极端情况下， 用户没有延迟感。

7.6.3.2 限流熔断策略的优化

为了尽可能 保证系统可用， 在功能层面，做了级别划分：

> 把依赖划分为强依赖（如审核）、弱依赖（如粉丝勋章）。

首先，在如果强依赖出现异常，下坚决限流熔断，尽可能 保证 强依赖的可用性。

另外，对于弱依赖，通过超时控制、请求预过滤、优化调用编排， 持续优化提升非核心功能的可用性，

7.7 重点：Redis 的双机房部署方案

> 双机房部署 redis 该怎么做 ？

顺着这个文章， 给大家 把双机房部署 redis 方案说一下。

采用 redis-cluster-proxy + redis cluster 的架构方案。

redis cluster 是 redis 的官方集群方案，但是他要求客户端自己做重定向，

redis-cluster-proxy 是 redis 的官方集群代理，经过这个 proxy 的代理后，连接 redis 集群就和连接单机 redis 一样了。

7.7.1 proxy+cluster 架构

架构说明：

1、redis-cluster 采用了同城双活架构，

其中，图中的 AZ1 和 AZ2 表示为可用区 1、可用区 2，

主节点（AZ1）按 3 主 3 从部署，备节点（AZ2）作为 Cluster 的 6 从，整个集群为 3 主 9 从；

2、单个 master 节点发生故障，redis 集群自动感知并进行选主，完成主从切换，不影响业务正常使用；

3、应用服务连接所有 Redis 集群主从节点，以便自动感知主从切换情况。

其实只要连接到 redis-proxy 一个节点，应用服务便可以获取到集群信息，某些节点宕机后，客户端不会收到影响。

注意：单 AZ1 池资源出现问题，可以使用已准备好的脚本进行快速切换 AZ2 节点，完成集群恢复。

当然，上面的方案，可以考虑把 master 节点打散到两个 AZ，避免集群超半数 master 节点宕机；

7.7.2 proxy+cluster 架构问题

redis-proxy 模式采用官方的 redis-proxy+cluster 模式，优点就是应用只需要连接 proxy 节点即可，不需要配置更多的 node 节点，生产环境 proxy 需要考虑高可用，而 proxy 也可以考虑用 lvs+keepalived 作为代理取代，

7.7.2.1 自动切换脚本

先使用 `cluster failover force` 命令执行强制切库，如果试了 3 次都不行，就使用 `cluster failover takeover` 更强制的切库，

手动故障转移是一种特殊的故障转移，通常在没有实际故障的情况下执行，我们希望将当前主节点与其中一个 slave 从节点（我们发送该命令的节点）交换（安全地，而不会有数据丢失的窗口）。

> 当前 slave 从节点通知主节点停止处理客户端的请求。
>
> 主节点回复 slave 从节点当前的 *同步偏移量*。

> slave 从节点等待同步偏移量在 slave 从节点的侧匹配，以确保它已经处理了所有主节点的数据，然后继续。
>
> slave 从节点开始故障转移，从主节点的大多数主节点获取新的配置纪元值 epoch，并广播新的配置。
>
> 旧的主节点接收配置更新：解除对客户端访问的阻止，并开始回复重定向消息，以便它们继续与新的主节点通信。

两个选项：

> FORCE option: manual failover when the master is down　当主节点停止时手动故障转移
>
> 如果选择 **FORCE** 选项，slave 从节点不会与 master 主节点进行协商（master 节点可能不可达），而是直接尽快从上文的故障转移步骤中的第 4 步开始做故障转移。
>
> 当主节点不可达时，**FORCE** 选项对于我们做手动故障转移非常有用。
>
> TAKEOVER option: manual failover without cluster consensus　在集群数据不一致的场景下，也要人工故障转移
>
> **TACKOVER** 选项实现了 **FORCE** 选项的所有实现，但是无需集群一致性验证来进行故障转移。

如果需要故障转移，在从节点上，可以输入以下命令

redis-cli-p7001-cCLUSTERFAILOVER

也可以结合 linux 上的定时器，使用脚本，配合使用

（1）探活脚本：redis_check.sh

（2）切换脚本：redis_task.sh

redis_check.sh 脚本为检测 redis 节点是否存活

```
#!/bin/bash

LOGDIR="/root"
BINDIR="/data/redis/7001/bin/redis-cli"
PASSWD="123456"
IPDIR="172.16.0.8:7001　172.16.0.8:7002 172.16.0.8:7003 172.16.0.8:7004 172.16.0.8:7005 172.16.0.8:7006"
DATE=`date`
LOGFILE=${LOGDIR}/logs
ERROR_LOG=${LOGDIR}/error.logs
```

```

cd${LOGDIR}
if [ ! -d bak ] ; then
            mkdir-p bak
fi

for i in$IPDIR
do
   port=${i#*:}
   ip=${i%:*}

   ALIVE=`$BINDIR -h $ip -p $port    PING`
   #ALIVE=`$BINDIR -h $ip -p $port -a $PASSWD PING`

   if [ "$ALIVE"=="PONG" ]; then
      echo"${DATE} Success: redis-cli -h $ip -p $port     PING $ALIVE" >> $LOGFILE2>&1
   else
      echo"${DATE} Failed:redis-cli -h $ip -p $port    PING $ALIVE " >> $ERROR_LOG2>&1
   fi

done

```

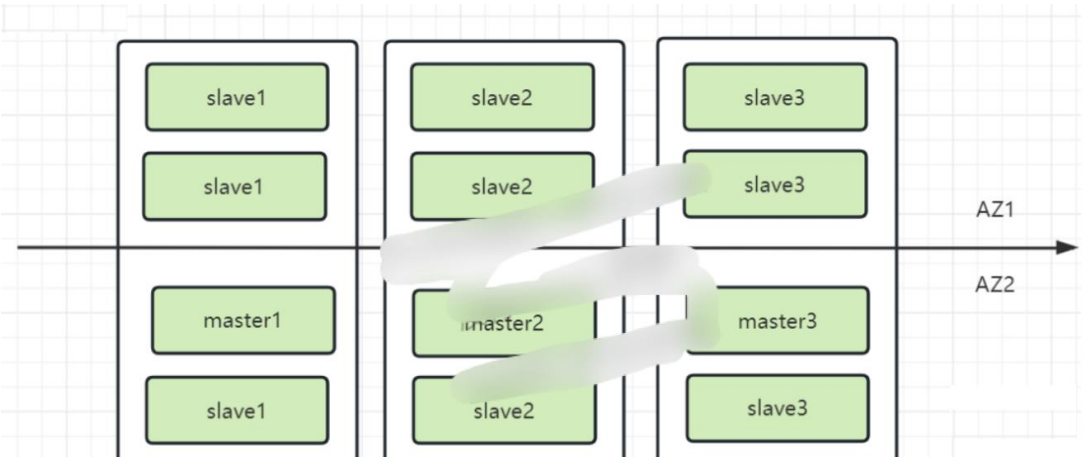
redis_task.sh 为 AZ1 节点宕机后，执行脚本切换到 AZ2 节点

```
#!/bin/bash

IPMASTER="172.16.0.8:7001    172.16.0.8:7002 172.16.0.8:7003"
BINDIR="/data/redis/7001/bin/redis-cli"
PASSWD="123456"
DATE=`date`
LOGDIR="/root"
LOGFILE=${LOGDIR}/logs
ERROR_LOG=${LOGDIR}/error.logs

for i in$IPMASTER
do
   port=${i#*:}
   ip=${i%:*}

   status=`$BINDIR -h $ip -p $port -c cluster failover takeover`
   #status=` $BINDIR -h $ip -p $port -a $PASSWD -c cluster failover takeover`

   if [ "$status"=="OK" ]; then
      echo"${DATE} Success: $i  成功切换成 master 节点" >> $LOGFILE2>&1
   else
```

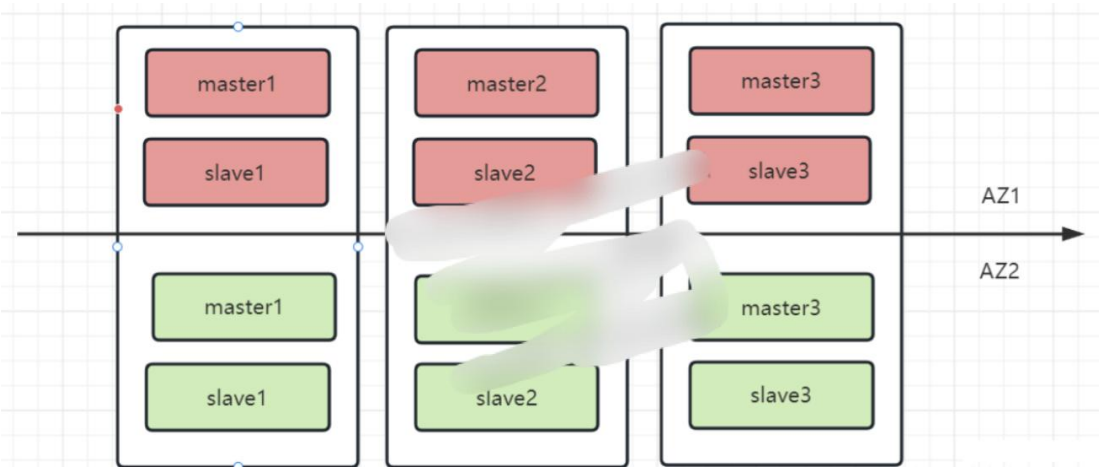
```
22        echo"${DATE} Failed: $i  切换 master 节点失败 " >> $ERROR_LOG2>&1
23    fi
24
25  done
```

7.7.2.2 机房故障的 redis 故障转移过程

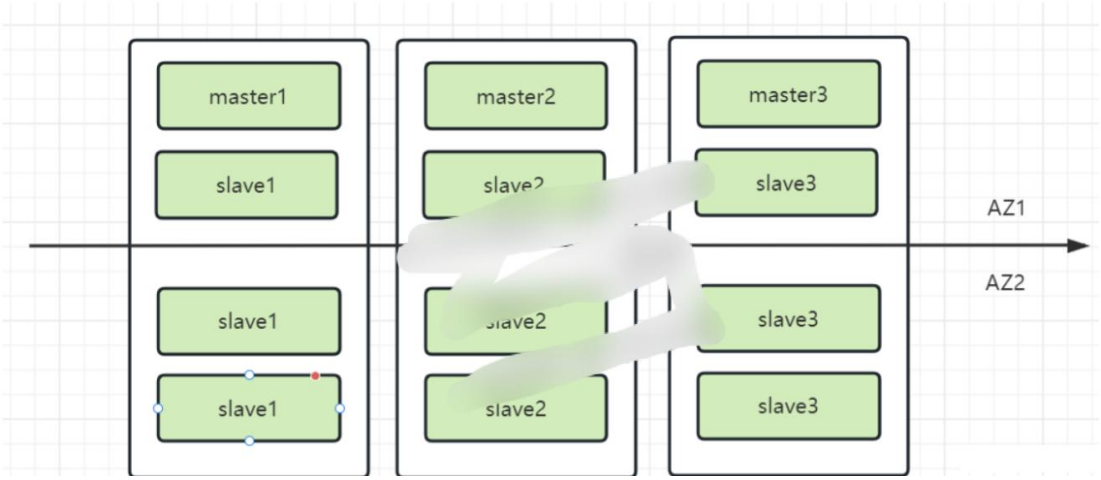
最开始 cluster 搭建方式如下图所示



在 AZ1 机房全部宕机后，我们需要通过执行 cluster failover takeover 命令将 AZ2 机房切换为 Master，如下图所示：



在 AZ1 机房恢复正常之后，如下图所示：



7.8 安全性架构

7.8.1 一、数据安全

满足数据安全法要求，除了数据安全法所要求的以外，评论系统的数据安全还包括「合规性要求」。

评论数据合规，一方面是审核和风控，另一方面对工程侧的要求主要是「状态一致性」。

例如，有害评论被删除后，在客户端不能展现，也不能通过 API 等对外暴露。

这就对数据一致性，包括缓存，提出了较高要求。

在设计层面主要有两方面实践：

17. 数据读写阶段均考虑了一致性风险，严格保证时序性。
18. 对各类数据写操作，定义了优先级，避免高优先级操作被低优先级操作覆盖，例如审核删除的有害评论，不能被其他普通运营人员/自动化策略放出。
19. 通过冗余校验，避免风险数据外泄。

例如评论列表的露出，读取 sorted set 中的 id 列表后，还需要校验对应评论的状态，是可见态才允许下发。

7.8.2 二、舆论安全

舆论安全问题更为泛化。

接口错误导致用户操作失败、关闭评论区、评论计数不准，甚至新功能上线、用户不满意的评论被顶到热评前排等问题均可能引发舆情问题。

在系统设计层面，我们主要通过几方面规避。

20. **不对用户暴露用户无法处理和不值得处理的错误。**

例如评论点赞点踩、某个数据项读取失败这一类的轻量级操作，不值得用户重试，此时告知用户操作失败也没有意义。系统可以考虑自行重试，甚至直接忽略。

21. 优化产品功能及其技术实现，例如评论计数、热评排序等。

7.9 热评设计架构

7.9.1 什么是热评

早期的热评，实际就是按照评论点赞数降序。

后来衍生了更为复杂的热评：

> 既包括类似「妙评」这种用户推荐、运营精选且带 logo 突出展示的产品形态，
> 也包括各类热评排序算法，且热评排序算法应用场景也不仅局限于评论主列表的热度序，
> 还包括楼中楼（外露子评论）、动态外露评论等。

热评排序逻辑一般包括点赞数、回复数、内容相关、负反馈数、“时间衰退因子”、字数加权、用户等级加权等等。

咬文嚼字来说，我们对「热」的理解，大致分为几个阶段：

22. 阶段 1 ： 点赞高，就代表热度高。→ 解决热评的有无问题
23. 阶段 2 ： 基于用户正负样本投票的，加权平均高，就代表热度高。→ 解决高赞高踩的负面热评问题
24. 阶段 3：短时间内点赞率高，就代表热度高。→ 解决高赞永远高赞的马太效应
25. 阶段 4 ： 热评用户流量大，社区影响也大。→ 追求用户价值平衡， 要权衡社会价值观引导、公司战略导向、商业利益、UP 主与用户的「情绪」等。

7.9.1.1 阶段 1 ： 按照点赞绝对值排序

按照点赞绝对值排序，即要实现 ORDER BY like_count 的分页排序。

点赞数是一个频繁更新的值，MySQL，特别是 TiDB，由于扫描行数约等于 OFFSET，因此在 OFFSET 较大时查询性能特别差，很难找到一个完美的优化方案。

此外，由于 like_count 的分布可能出现同一个值堆叠多个元素，比如评论区所有的评论都没有赞，

我们更多依赖 redis 的 sorted set 来执行分页查询，这就要求 **缓存命中率非常高。**

7.9.1.2 阶段 2 ： 按照正负样本加权平均排序

按照正负样本加权平均的，即 **Reddit：威尔逊排序[6]**，

到这个阶段，数据库已经无法实现这样复杂的 ORDER BY，热评开始几乎完全依赖 sorted set 这样的数据结构，预先计算好排序分数并写入。

于是在架构设计上，新增了 feed-service 和 feed-job 来支撑热评列表的读写。

Reddit 最早成立于 2005 年，两名创始人是史蒂夫·霍夫曼和阿里克西斯·奥哈尼安，当时他们刚刚从弗吉尼亚大学毕业。他们的创业想法获得了美国知名创业孵化器 Y Combinator 的天使投资。

Reddit 在美国的影响力非常大，它的信息展示形式像论坛，又像贴吧。是由不同版块下的帖子组成的交流平台，用户可以选择对帖子点击“上涨”或“下沉”来决定帖子的排名顺序。

Reddit 在 2009 年公开过自己如何用威尔逊区间(wilson interval)对评论排序的，代码也开源过。

Wilson 算法要点有两个：

26. 把“所有正负反馈中正反馈的比例”作为对评论质量的考核指标。

在 Reddit 的情况中，正负反馈分别为点赞和反对。

简单来说，就是 正为点赞， 负为反对

27. 对在冷启动过程中的评论（即正负反馈总数很少）做降权处理。

具体算法就是：假设观测到的正反馈率符合真实正反馈率的正态分布，求当前观测得到的正反馈率恰好位于 95%置信上区间时的真实反馈率。

7.9.1.3 阶段 3：按照点赞率排序

按照点赞率排序，需要实现点赞率的近实时计算。

点赞率=点赞数/曝光数，曝光的数据来源是客户端上报的展现日志，量级非常大，

可以说是一个写多读少的场景：只有重算排序的时候才会读取曝光数。

7.9.1.4 阶段 4 ： 追求用户价值平衡

追求用户价值平衡，需要处理各种细分场景下的差异化需求。

热评排序与 feed 排序很像，但也有一点根本性差异：

> feed 排序是个性化的，每个人看到的都不相同，
>
> 但评论排序往往不会如此激进，一般来说会希望大家看到的评论排序都大致相同。

由于排序问题的解决方案是探索型的，因此系统设计层面需要提供更多元、更易扩展的工程化能力，

> 包括算法和策略的快速迭代、实验能力等，
>
> 并提升整个热评模块的可观测水平，监控完善、数据报表丰富、排序过程可解释等等。

在架构上，新增了 strategy-service 和 strategy-job 来承担这部分策略探索型业务。

此外，数据量级规模的增加，也对系统的吞吐能力提出了更高要求：

不管热评的算法如何变化，一般来说，热评列表都需要能够访问到全部评论，且基本维持相同的热评排序逻辑。

在评论数过百万甚至千万的评论区，热评排序的挑战点主要在于：

28. **大 key 问题**：

例如单个 sorted set 过大，读写性能都受影响（时间复杂度的基数可以认为都是 O(logN)）；全量更新时，还可能遇到 redis pipeline 的瓶颈。

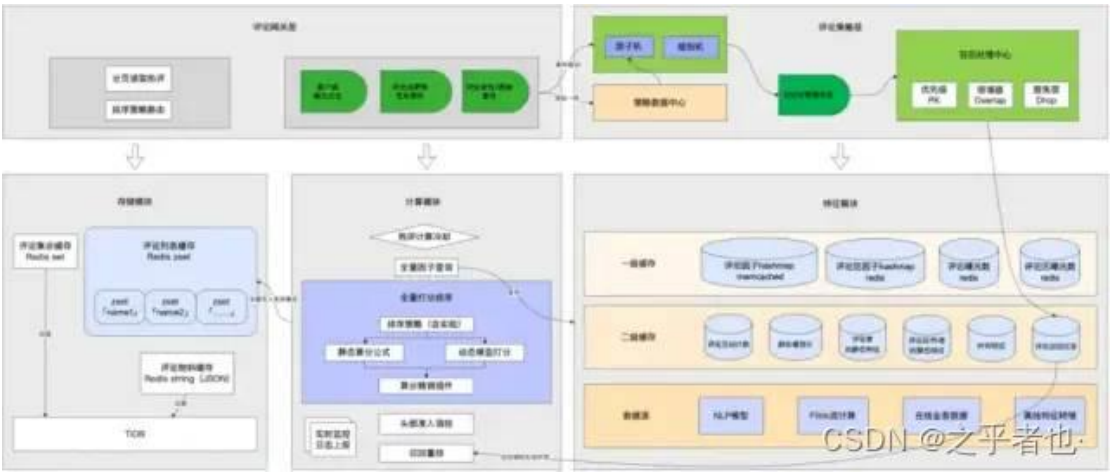
29. **实时性放大存储压力**：

多样化的数据源，对特征的导入与更新都提出了挑战，需要支持较丰富的数据结构，和尽可能高的写吞吐（想象一下曝光数作为排序特征的变态要求）；

与推荐排序不同，热评排序是全排序，此时需要读取全部评论的全部特征，查询压力也会非常大。

这一阶段，我们仍然在持续优化，在工程落地层面尽可能还原理想的排序算法设计，保障用户的热评浏览体验。

目前形成的系统架构总体如下图所示：



图示的「评论策略层」，负责建立一套热评调控体系化能力，通过召回机制来实现想要的“balance“。

即先通过策略工程，召回一批应该沉底的不良评论或者应该进前排的优秀评论，然后在排序分计算阶段根据召回结果实现这样的效果。

这样做的好处是，可以**保留一套通用的底层排序算法，然后通过迭代细分场景下的召回策略，来实现差异化评论排序的平衡。**

召回策略的工程设计，按照分层设计的原则拆分为 3 个部分：

30. **因子机。**

主要职责是维护策略所需的全部「因子」，包括一些已有的在线/离线数据，也包括为了策略迭代而需要新开发的流式的窗口聚合数据。

因子机的重难点是需要管理各种数据获取的拓扑关系，以及通过缓存来保护下游（数据提供方很难也不应该承受热评业务的巨大流量）。

所有的因子可以构成一个有向无环图，通过梳理依赖关系和推导计算，实现并发提效、减少冗余。

31. **规则机。**

实现了一套声明式规则语法，可以直接引用因子机预定义的因子，结合各种逻辑算子构成一个规则表达式。

规则机执行命中后，会向下游传递预先声明的召回决策，例如排序提权。

32. **召回处理中心。**

这一层的职责就是接收规则机返回的各种决策并执行，需要处理不同决策的优先级 PK、不同规则的决策叠加作用、决策豁免等。

热评排序涉及的特征，是多数据源的，数据更新方式、更新频率、查询性能也天差万别。因此我们针对数据源的特点做了多级缓存，通过**多级冗余与跨级合并**，提升了特征读取的稳定性与性能上限。

当然，其中的数据实时性、一致性、可用性，仍然处于一个动态权衡取舍的过程。

8. 短 URL 生成器设计：百亿短 URL 怎样做到无冲突?

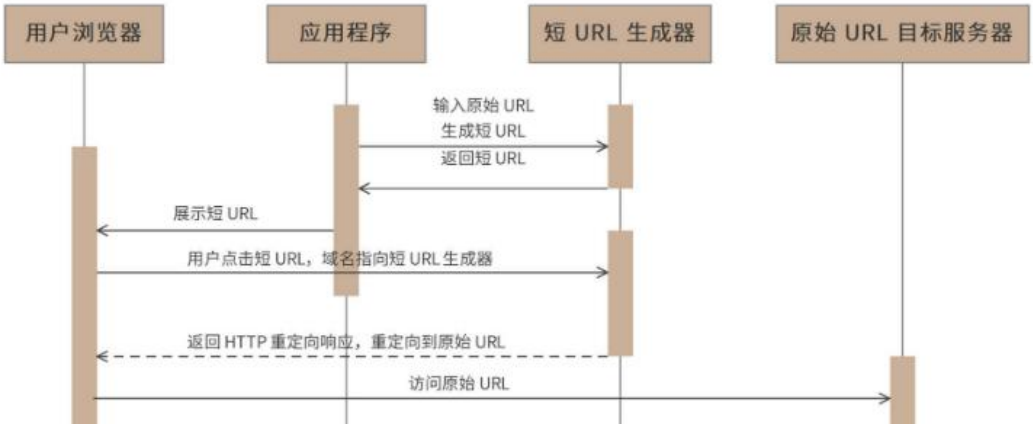
在社交媒体上，人们经常需要分享一些 URL，但是有些 URL 可能会很长，比如：

`https://geek.qq.org/hybrid/pvip?utm_source=geek-pc-discover-banner&utm_term=geek-pc-discover-banner`

这样长的 URL 显然体验并不友好。我们期望分享的是一些更短、更易于阅读的短 URL，比如像 http://1.cn/ScW4dt 这样的。当用户点击这个短 URL 的时候，可以重定向访问到原始的链接地址。为此我们将设计开发一个短 URL 生成器，产品名称是“Fuxi(伏羲)”。我们预计 Fuxi 需要管理的短 URL 规模在百亿级别，并发吞吐量达到数万级别。这个量级的数据对应的存储方案是什么样的？用传统的关系数据库存储，还是有其他更简单的办法？此外，如何提升系统的并发处理能力呢？这些是我们今天要重点考虑的问题。

8.1 需求分析

短 URL 生成器，也称作短链接生成器，就是将一个比较长的 URL 生成一个比较短的 URL，当浏览器通过短 URL 生成器访问这个短 URL 的时候，重定向访问到原始的长 URL 目标服务器，访问时序图如下。



对于需要展示短 URL 的应用程序，由该应用调用短 URL 生成器生成短 URL，并将该短 URL 展示给用户，用户在浏览器中点击该短 URL 的时候，请求发送到短 URL 生成器（短 URL 生成器以 HTTP 服务器的方式对外提供服务，短 URL 域名指向短 URL 生成器），短 URL 生成器返回 HTTP 重定向响应，将用户请求重定向到最初的原始长 URL，浏览器访问长 URL 服务器，完成请求服务。

8.1.1 短 URL 生成器的用例图

1. 用户 client 程序可以使用短 URL 生成器 Fuxi 为每个长 URL 生成唯一的短 URL，并存储起来。
2. 用户可以访问这个短 URL，Fuxi 将请求重定向到原始长 URL。
3. 生成的短 URL 可以是 Fuxi 自动生成的，也可以是用户自定义的。用户可以指定一个长 URL 对应的短 URL 内容，只要这个短 URL 还没有被使用。

4. 管理员可以通过 web 后台检索、查看 Fuxi 的使用情况。
5. 短 URL 有有效期（2 年），后台定时任务会清理超过有效期的 URL，以节省存储资源，同时回收短 URL 地址链接资源。

8.1.2 性能指标估算

Fuxi 的**存储容量**和**并发量**估算如下。

预计每月新生成短 URL 5 亿条，短 URL 有效期 2 年，那么总 URL 数量 120 亿。

\(\\small 5 亿\\times12 月\\times2 年=120 亿\)

- **存储空间**- 每条短 URL 数据库记录大约 1KB，那么需要总存储空间 12TB（不含数据冗余备份）。

\(\\small 120 亿\\times1KB=12TB\)

- **吞吐量**- 每条短 URL 平均读取次数 100 次，那么平均访问吞吐量（每秒访问次数）2 万。

\(\\small（5 亿\\times100）\\div（30\\times24\\times60\\times60）\\approx20000\)

一般系统高峰期访问量是平均访问量的 2 倍，因此系统架构需要支持的吞吐能力应为 4 万。

- **网络带宽**- 短 URL 的重定向响应包含长 URL 地址内容，长 URL 地址大约 500B，HTTP 响应头其他内容大约 500B，所以每个响应 1KB，高峰期需要的响应网络带宽 320Mb。

\(\\small 4 万（每秒）次请求\\times1KB=40MB\\times8bit=320Mb\)

Fuxi 的**短 URL 长度估算**如下。

短 URL 采用 Base64 编码，如果短 URL 长度是 7 个字符的话，大约可以编码 4 万亿个短 URL。

\(\\small 64^{7}\\approx4 万亿\)

如果短 URL 长度是 6 个字符的话，大约可以编码 680 亿个短 URL。

\(\\small 64^{6}\\approx680 亿\)

按我们前面评估，总 URL 数 120 亿，6 个字符的编码就可以满足需求。因此 Fuxi 的短 URL 编码长度 6 个字符，形如 http://l.cn/ScW4dt 。

8.1.3 非功能需求

1. 系统需要保持高可用，不因为服务器、数据库宕机而引起服务失效。
2. 系统需要保持高性能，服务端 80%请求响应时间应小于 5ms，99%请求响应时间小于 20ms，平均响应时间小于 10ms。
3. 短 URL 应该是不可猜测的，即不能猜测某个短 URL 是否存在，也不能猜测短 URL 可能对应的长 URL 地址内容。

8.2 概要设计

短 URL 生成器的设计核心就是短 URL 的生成，即长 URL 通过某种函数，计算得到一个 6 个字符的短 URL。短 URL 有几种不同的生成算法。

8.2.1 单项散列函数生成短 URL

通常的设计方案是，将长 URL 利用 MD5 或者 SHA256 等单项散列算法，进行 Hash 计算，得到 128bit 或者 256bit 的 Hash 值。然后对该 Hash 值进行 Base64 编码，得到 22 个或者 43 个 Base64 字符，再截取前面的 6 个字符，就得到短 URL 了，如图。

但是这样得到的短 URL，可能会发生 Hash 冲突，即不同的长 URL，计算得到的短 URL 是相同的（MD5 或者 SHA256 计算得到的 Hash 值几乎不会冲突，但是 Base64 编码后再截断的 6 个字符有可能会冲突）。所以在生成的时候，需要先校验该短 URL 是否已经映射为其他的长 URL，如果是，那么需要重新计算(换单向散列算法，或者换 Base64 编码截断位置）。重新计算得到的短 URL 依然可能冲突，需要再重新计算。
但是这样的冲突处理需要多次到存储中查找 URL，无法保证 Fuxi 的性能要求。

8.2.2 自增长短 URL

一种免冲突的算法是用自增长自然数来实现，即维持一个自增长的二进制自然数，然后将该自然数进行 Base64 编码即可得到一系列的短 URL。这样生成的的短 URL 必然唯一，而且还可以生成小于 6 个字符的短 URL，比如自然数 0 的 Base64 编码是字符“A”，就可以用 http://1.cn/A 作为短 URL。

但是这种算法将导致短 URL 是可猜测的，如果某个应用在某个时间段内生成了一批短 URL，那么这批短 URL 就会集中在一个自然数区间内。只要知道了其中一个短 URL，就可以通过自增（以及自减）的方式请求访问其他 URL。Fuxi 的需求是不允许短 URL 可预测。

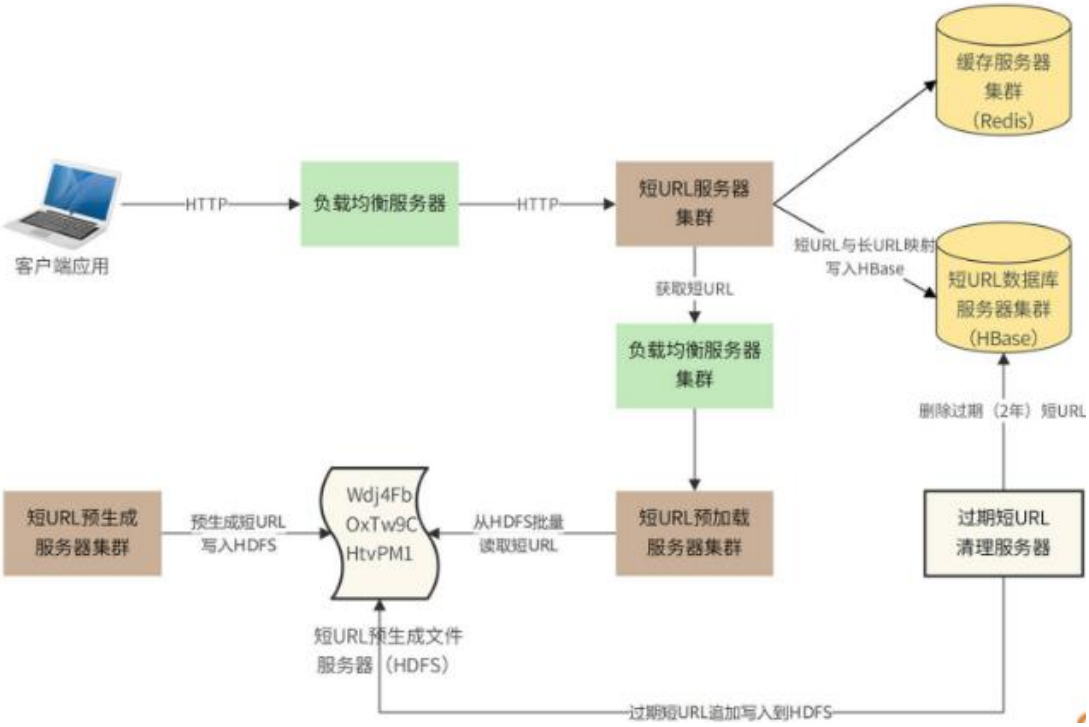
8.2.3 预生成短 URL

因此，Fuxi 采用预生成短 URL 的方案。即预先生成一批没有冲突的短 URL 字符串，当外部请求输入长 URL 需要生成短 URL 的时候，直接从预先生成好的短 URL 字符串池中获取一个即可。

预生成短 URL 的算法可以采用随机数来实现，6 个字符，每个字符都用随机数产生（用 0~63 的随机数产生一个 Base64 编码字符）。为了避免随机数产生的短 URL 冲突，需要在预生成的时候检查该 URL 是否已经存在（用布隆过滤器检查）。因为预生成短 URL 是离线的，所以这时不会有性能方面的问题。事实上，Fuxi 在上线之前就已经生成全部需要的 144 亿条短 URL 并存储在文件系统中（预估需要短 URL120 亿，Fuxi 预生成的时候进行了 20%的冗余，即 144 亿。）

8.2.4 Fuxi 的整体部署模型

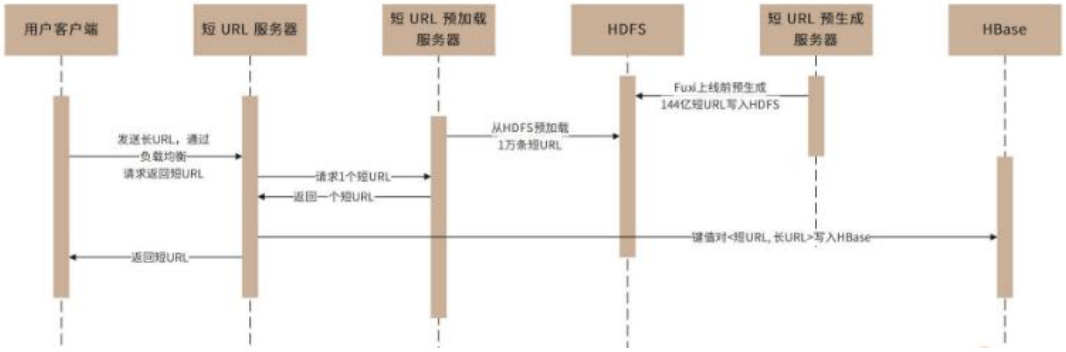
Fuxi 的业务逻辑比较简单，相对比较有挑战的就是**高并发的读请求如何处理、预生成的短 URL 如何存储以及访问**。高并发访问主要通过负载均衡与分布式缓存解决，而海量数据存储则通过 HDFS 以及 HBase 来完成。具体架构图如下。



系统调用可以分成两种情况，一种是用户请求生成短 URL 的过程；另一种是用户访问短 URL，通过 Fuxi 跳转到长 URL 的过程。

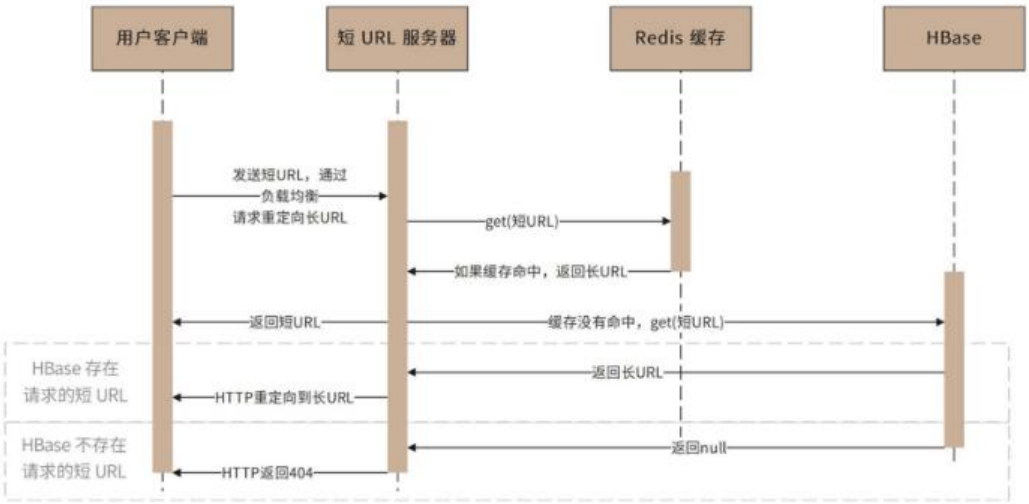
对于用户请求生成短 URL 的过程，在短 URL 系统 Fuxi 上线前，已经通过随机数算法预生成 144 亿条短 URL 并将其存储在 HDFS 文件系统中。系统上线运行后，应用程序请求生成短 URL 的时候（即输入长 URL，请求返回短 URL），请求通过负载均衡服务器被发送到短 URL 服务器集群，短 URL 服务器再通过负载均衡服务器调用短 URL 预加载服务器集群。

短 URL 预加载服务器此前已经从短 URL 预生成文件服务器（HDFS）中加载了一批短 URL 存放在自己的内存中，这时，只需要从内存中返回一个短 URL 即可，同时将短 URL 与长 URL 的映射关系存储在 HBase 数据库中，时序图如下。



对于用户通过客户端请求访问短 URL 的过程（即输入短 URL，请求返回长 URL），请求通过负载均衡服务器发送到短 URL 服务器集群，短 URL 服务器首先到缓存服务器中查找是否有该短 URL，如果有，立即返回对应的长 URL，短 URL 生成服务器构造重定向响应返回给客户端应用。

如果缓存没有用户请求访问的短 URL，短 URL 服务器将访问 HBase 短 URL 数据库服务器集群。如果数据库中存在该短 URL，短 URL 服务器会将该短 URL 写入缓存服务器集群，并构造重定向响应返回给客户端应用。如果 HBase 中没有该短 URL，短 URL 服务器将构造 404 响应返回给客户端应用，时序图如下。



过期短 URL 清理服务器会每个月启动一次，将已经超过有效期（2 年）的 URL 数据删除，并将这些短 URL 追加写入到短 URL 预生成文件中。

为了保证系统**高可用**，Fuxi 的应用服务器、文件服务器、数据库服务器都采用集群部署方案，单个服务器故障不会影响 Fuxi 短 URL 的可用性。

对于 Fuxi 的**高性能**要求，80%以上的访问请求将被设计为通过缓存返回。Redis 的缓存响应时间 1ms 左右，服务器端请求响应时间小于 3ms，满足 80%请求小于 5ms 的性能目标。对于缓存没有命中的数据，通过 HBase 获取，HBase 平均响应时间 10ms，也可以满足设计目标中的性能指标。

对于 Redis 缓存**内存空间**估算，业界一般认为，超过 80%请求集中在最近 6 天生成的短 URL 上，Fuxi 主要缓存最近六天生成的短 URL 即可。根据需求容量估计，最近 6 天

生成的短 URL 数量约 1 亿条，因此需要 Redis 缓存服务器内存空间：\(\\small 1 亿 \\times1KB=100GB\)。

8.3 详细设计

详细设计关注重定向响应码、短 URL 预生成文件及加载、用户自定义短 URL 等几个关键设计点。

8.3.1 重定向响应码

满足短 URL 重定向要求的 HTTP 重定向响应码有 301 和 302 两种，其中 301 表示永久重定向，即浏览器一旦访问过该短 URL，就将重定向的原始长 URL 缓存在本地，此后不再请求短 URL 生成器，直接根据缓存在浏览器（HTTP 客户端）的长 URL 路径进行访问。

302 表示临时重定向，每次访问短 URL 都需要访问短 URL 生成器。

一般说来，使用 301 状态码可以降低 Fuxi 服务器的负载压力，但无法统计短 URL 的使用情况，而 Fuxi 的架构设计完全可以承受这些负载压力，因此 Fuxi 使用 302 状态码构造重定向响应。

8.3.2 短 URL 预生成文件及预加载

Fuxi 的短 URL 是在系统上线前全部预生成的，并存储在 HDFS 文件中。共 144 亿个短 URL，每个短 URL 6 个字符，文件大小\(\\small 144 亿\\times6B=86.4GB\)。

文件格式就是直接将 144 亿个短 URL 的 ASC 码无分割地存储在文件中，如下是存储了 3 个短 URL 的文件示例：

```
Wdj4FbOxTw9CHtvPM1
```

所以如果短 URL 预加载服务器第一次启动的时候加载 1 万个短 URL，那么就从文件头读取 60K 数据，并标记当前文件偏移量 60K。下次再加载 1 万个短 URL 的时候，再从文件 60K 偏移位置继续读取 60K 数据即可。

因此，Fuxi 除了需要一个在 HDFS 记录预生成短 URL 的文件外，还需要一个记录偏移量的文件，记录偏移量的文件也存储在 HDFS 中。同时，由于预加载短 URL 服务器集

群部署多台服务器，会出现多台服务器同时加载相同短 URL 的情况，所以还需要利用偏移量文件对多个服务器进行互斥操作，即**利用文件系统写操作锁的互斥性实现多服务器访问互斥。**

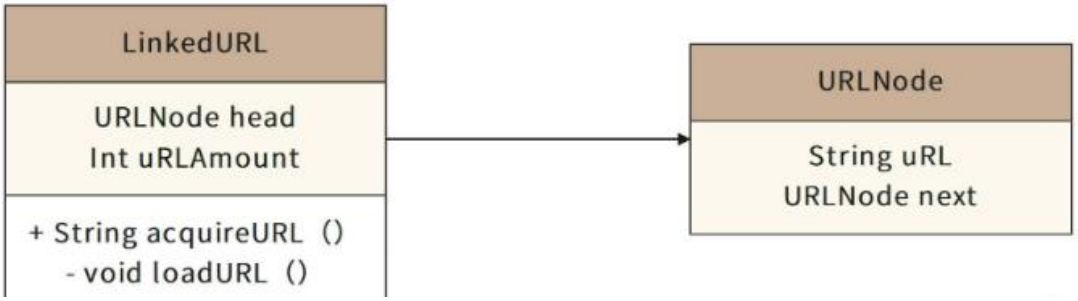
应用程序的文件访问流程应该是：写打开偏移量文件 -> 读偏移量 -> 读打开短 URL 文件 -> 从偏移量开始读取 60K 数据 -> 关闭短 URL 文件 -> 修改偏移量文件 -> 关闭偏移量文件。

由于写打开偏移量文件是一个互斥操作，所以第一个预加载短 URL 服务器写打开偏移量文件以后，其他预加载短 URL 服务器无法再写打开该文件，也就无法完成读 60K 短 URL 数据及修改偏移量的操作，这样就能保证这两个操作是并发安全的。

加载到预加载短 URL 服务器的 1 万个短 URL 会以链表的方式存储，每使用一个短 URL，链表头指针就向后移动一位，并设置前一个链表元素的 next 对象为 null。这样用过的短 URL 对象可以被垃圾回收。

当剩余链表长度不足 2000 的时候，触发一个异步线程，从文件中加载 1 万个新的短 URL，并链接到链表的尾部。

与之对应的 URL 链表类图如下。



URLNode：URL 链表元素类，成员变量 uRL 即短 URL 字符串，next 指向下一个链表元素。

LinkedURL：URL 链表主类，成员变量 head 指向链表头指针元素，uRLAmount 表示当前链表剩余元素个数。acquireURL()方法从链表头指针指向的元素中取出短 URL 字符串，并执行 urlAmount– 操作。当 urlAmount < 2000 的时候，调用私有方法 loadURL()，该方法调用一个线程从文件中加载 1 万个短 URL 并构造成链表添加到当前链表的尾部，并重置 uRLAmount。

8.3.2.1 用户自定义短 URL

Fuxi 允许用户自己定义短 URL，即在生成短 URL 的时候，由用户指定短 URL 的内容。为了避免预生成的短 URL 和用户指定的短 URL 冲突，Fuxi 限制用户自定义短 URL 的字符个数，不允许用户使用 6 个字符的自定义短 URL，且 URL 长度不得超过 20 个字符。

但是用户自定义短 URL 依然可能和其他用户自定义短 URL 冲突，所以 Fuxi 生成自定义短 URL 的时候需要到数据库中检查冲突，是否指定的 URL 已经被使用，如果发生冲突，要求用户重新指定。

8.3.2.2 URL Base64 编码

标准 Base64 编码表如下。

数值	字符	数值	字符	数值	字符	数值	字符
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

其中“+”和“/”在 URL 中会被编码为“%2B”以及“%2F”，而“%”在写入数据库的时候又和 SQL 编码规则冲突，需要进行再编码，因此直接使用标准 Base64 编码进行短 URL 编码并不合适。URL 保留字符编码表如下。

!	#	$	&	'	(	)	*	+	,	/	:	;	=	?	@	[	]
%21	%23	%24	%26	%27	%28	%29	%2A	%2B	%2C	%2F	%3A	%3B	%3D	%3F	%40	%5B	%5D

所以，我们需要针对 URL 场景对 Base64 编码进行改造，使用 URL 保留字符表以外的字符对 Base64 编码表中的 62，63 进行编码：将“+”改为“-”，将“/”改为“_”，Fuxi 最终采用的 URL Base64 编码表如下。

数值	字符	数值	字符	数值	字符	数值	字符
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	-
15	P	31	f	47	v	63	_

9. 如何设计一个支持万亿 GB 网盘实现秒传与限速的系统

网盘，又称云盘，是提供文件托管和文件上传、下载服务的网站（File hosting service）。人们通过网盘保管自己拍摄的照片、视频，通过网盘和他人共享文件，已经成为了一种习惯。我们准备开发一个自己的网盘应用系统，应用名称为“DBox”。

十几年前曾经有个段子，技术人员对老板说：您不能在公司电脑打开您家里电脑的文件，再贵的电脑也不能。事实上，随着网盘技术的成熟，段子中老板的需求已经成为现实：网盘可以自动将家里电脑的文件同步到公司电脑，老板可以在公司的电脑打开家里电脑的文件了。

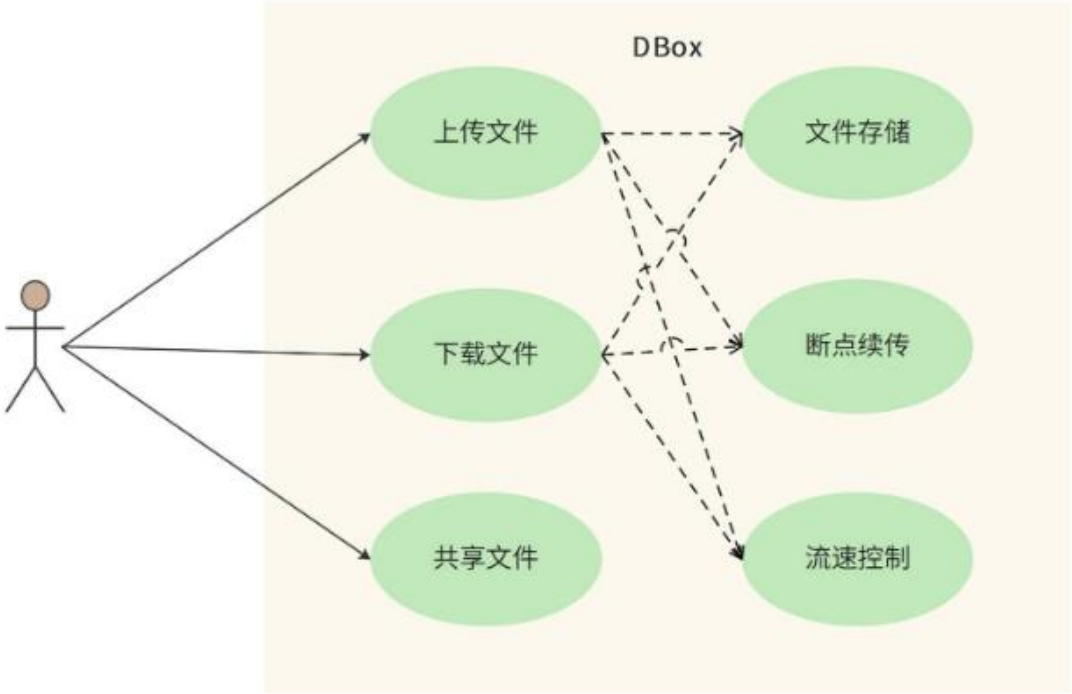
网盘的主要技术挑战是**海量数据的高并发读写访问**。**用户上传的海量数据如何存储？如何避免部分用户频繁读写文件，消耗太多资源，而导致其他的用户体验不佳？我们看下 DBox 的技术架构以及如何解决这些问题。

9.1 需求分析

DBox 的核心功能是提供文件上传和下载服务。基于核心功能，DBox 需要在服务器端保存这些文件，并在下载和上传过程中实现断点续传。也就是说，如果上传或下载过程被中断了，恢复之后，还能从中断的地方重新上传或者下载，而不是从头再来。

DBox 还需要实现文件共享的需求。使用 DBox 的不同用户之间可以共享文件，一个用户上传的文件共享给其他用户后，其他用户也可以下载这个文件。

此外，网盘是一个存储和网络密集型的应用，用户文件占据大量**硬盘资源**，上传、下载需要占用大量网络带宽，并因此产生较高的运营成本。所以用户体验需要向付费用户倾斜，DBox 需要对上传和下载进行**流速控制**，保证付费用户得到更多的网络资源。DBox 用例图如下。



9.1.1 负载指标估算

DBox 的设计目标是支持 10 亿用户注册使用，免费用户最大可拥有 1TB 存储空间。预计日活用户占总用户的 20%，即 2 亿用户。每个活跃用户平均每天上传、下载 4 个文件。

DBox 的**存储量**、**吞吐量**、**带宽负载**估算如下。

- **总存储量**- 理论上，总存储空间估算为 10 亿 TB，即 1 万亿 GB。

\(\\small 10 亿\\times1TB=10 亿 TB\)

但考虑到大多数用户并不会完全用掉这个空间，还有很多用户存储的文件其实是和别人重复的（电影、电子书、软件安装包等），真正需要的存储空间大约是这个估算值的 10%，即 1 亿 TB。

- **QPS**- 系统需要满足的平均 QPS 约为 10000。

\(\\small 2 亿\\times4\\div（24\\times60\\times60）\\approx1 万\)

高峰期 QPS 约为平均 QPS 的两倍，即 2 万。

- **带宽负载**- 每次上传下载文件平均大小 1MB，所以需要网络带宽负载 10GB/s，即 80Gb/s。

\(\\small 1 万\\times1MB=10GB/s=80Gb/s\)

同样，高峰期带宽负载为 160Gb/s。

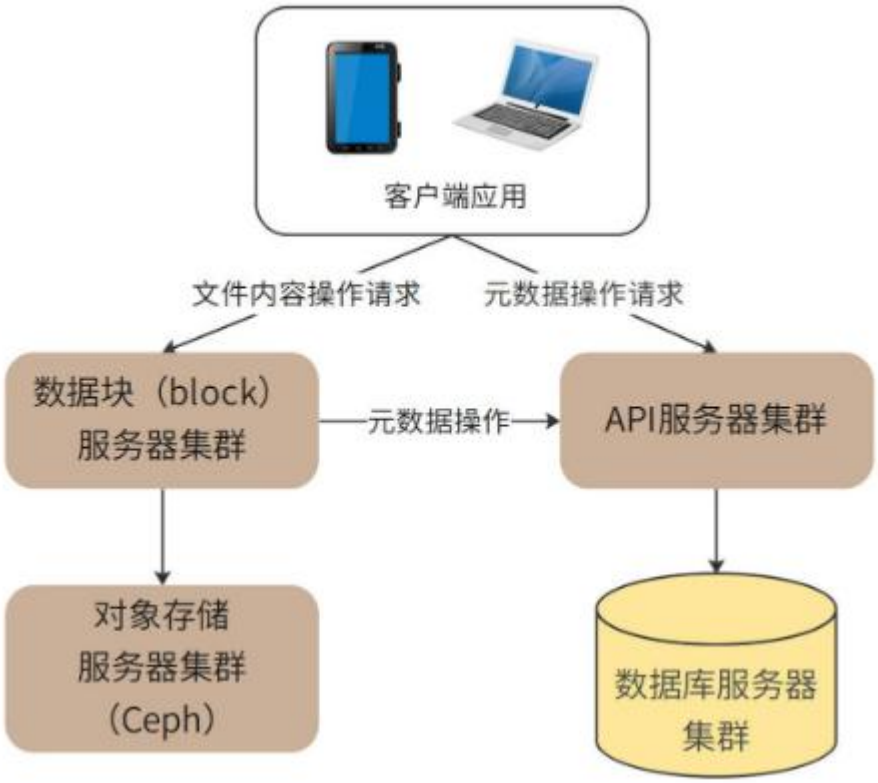
9.1.2 非功能需求

1. 大数据量存储：10 亿注册用户，1000 亿个文件，约 1 亿 TB 的存储空间。
2. 高并发访问：平均 1 万 QPS，高峰期 2 万 QPS。
3. 大流量负载：平均网络带宽负载 80Gb/S，高峰期带宽负载 160Gb/s。
4. 高可靠存储：文件不丢失，持久存储可靠性达到 99.9999% ，即 100 万个文件最多丢失（或损坏）1 个文件。
5. 高可用服务：用户正常上传、下载服务可用性在 99.99%以上，即一年最多 53 分钟不可用。
6. 数据安全性：文件需要加密存储，用户本人及共享文件外，其他人不能查看文件内容。
7. 不重复上传：相同文件内容不重复上传，也就是说，如果用户上传的文件内容已经被其他用户上传过了，该用户不需要再上传一次文件内容，进而实现“秒传”功能。从用户视角来看，不到一秒就可以完成一个大文件的上传。

9.2 概要设计

网盘设计的关键是**元数据与文件内容的分离存储与管理。**所谓文件元数据就是文件所有者、文件属性、访问控制这些文件的基础信息，事实上，传统文件系统也是元数据与文件内容分离管理的，比如 Linux 的文件元数据记录在文件控制块 FCB 中，Windows 的文件元数据记录在文件分配表 FAB 中，Hadoop 分布式文件系统 HDFS 的元数据记录在 NameNode 中。

而 DBox 是将元信息存储在数据库中，文件内容则使用另外专门的存储体系。但是由于 DBox 是一个互联网应用，出于安全和访问管理的目的，并不适合由客户端直接访问存储元数据的数据库和存储文件内容的存储集群，而是通过 API 服务器集群和数据块服务器集群分别进行访问管理。整体架构如下图。

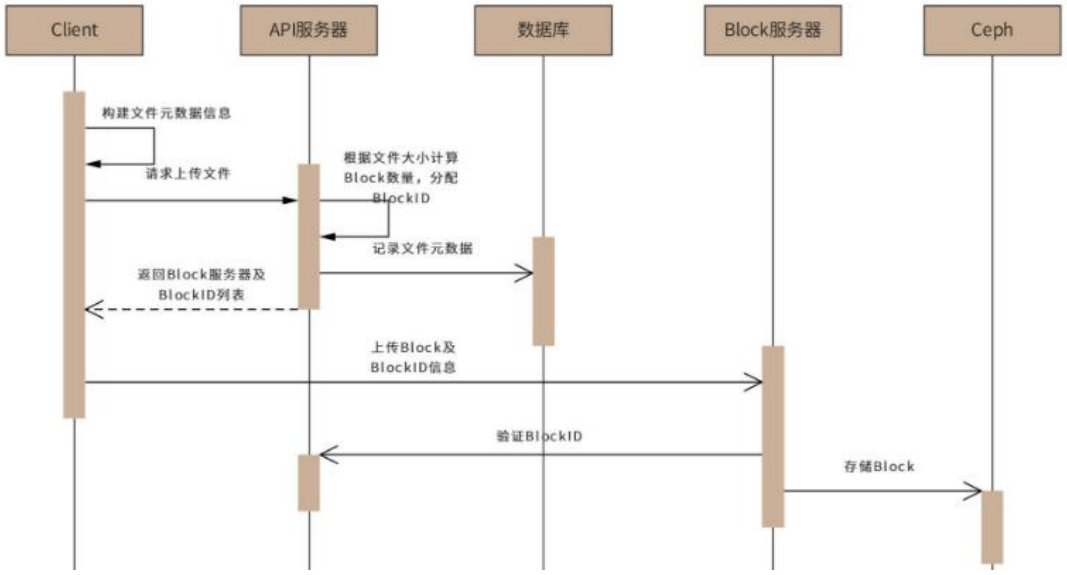


对于大文件，DBox 不会上传、存储一整个的文件，而是将这个文件进行切分，变成一个个单独的 Block，再将它们分别上传并存储起来。

这样做的核心原因是，DBox 采用对象存储作为最终的文件存储方案，而对象存储不适合存储大文件，需要进行切分。而大文件进行切分还带来其他的好处：可以**以 Block 为单位进行上传和下载，提高文件传输速度**；客户端或者网络故障导致文件传输失败，也只需要重新传输失败的 Block 就可以，进而实现**断点续传**功能。

Block 服务器就是负责 Block 上传和管理的。客户端应用程序根据 API 服务器的返回指令，将文件切分成一些 Block，然后将这些 Block 分别发送给 Block 服务器，Block 服务器再调用对象存储服务器集群，将 Block 存储在对象存储服务器中（DBox 选择 Ceph 作为对象存储）。

用户上传文件的时序图如下。

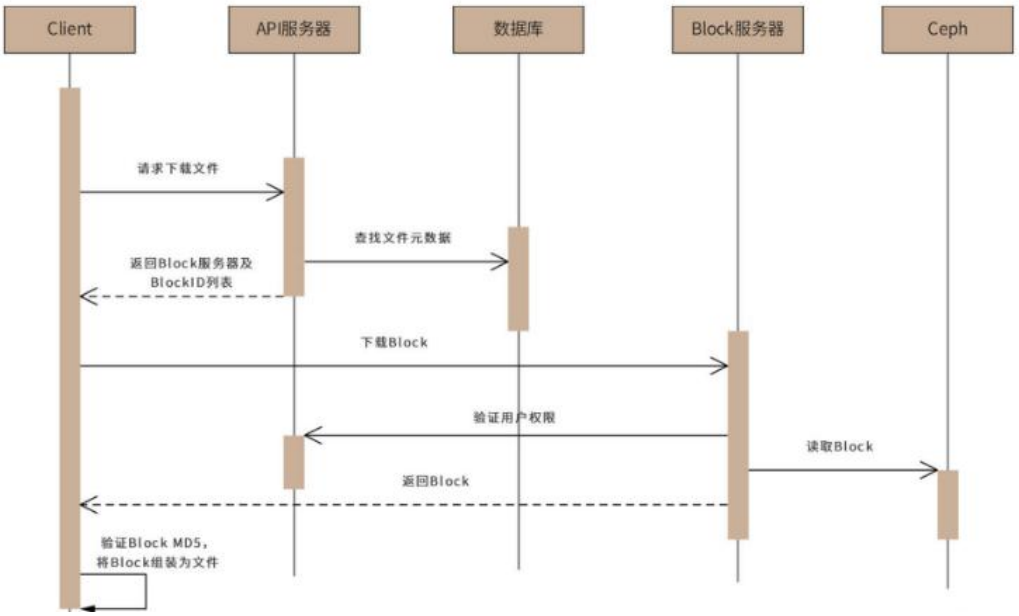


用户上传文件时，客户端应用程序收集文件元数据，包括文件名、文件内容 MD5、文件大小等等，并根据文件大小计算 Block 的数量（DBox 设定每个 block 大小 4MB），以及每个 Block 的 MD5 值。

然后客户端应用程序将全部元数据（包括所有 Block 的 MD5 值列表）发送给 API 服务器。API 服务器收到文件元数据后，为每个 Block 分配全局唯一的 BlockID（BlockID 为严格递增的 64 位正整数，总可记录数据大小\(\\small 2^{64}\\times4MB=180 亿 PB\)，足以满足 DBox 的应用场景）。

下一步，API 服务器将文件元数据与 BlockID 记录在数据库中，并将 BlockID 列表和应用程序可以连接的 Block 服务器列表返回客户端。客户端连接 Block 服务器请求上传 Block，Block 服务器连接 API 服务器进行权限和文件元数据验证。验证通过后，客户端上传 Block 数据，Block 服务器再次验证 Block 数据的 MD5 值，确认数据完整后，将 BlockID 和 Block 数据保存到对象存储集群 Ceph 中。

类似的，用户下载文件的时序图如下。



客户端程序访问 API 服务器，请求下载文件。然后 API 服务器会查找数据库，获得文件的元数据信息，再将元数据信息中的文件 BlockID 列表及可以访问的 Block 服务器列表返回给客户端。

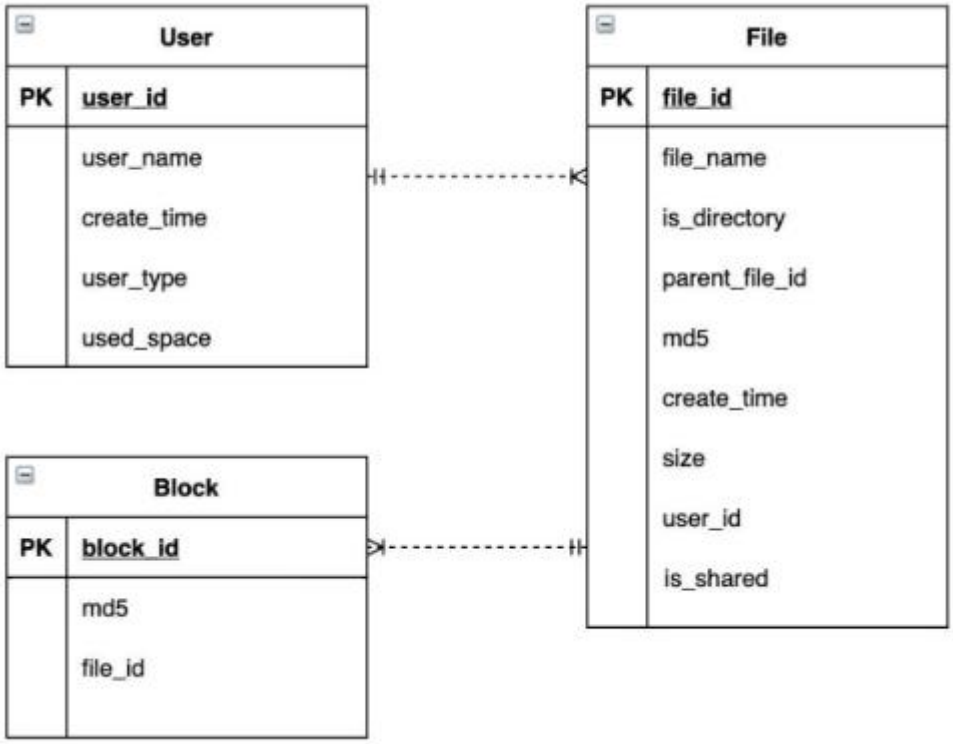
下一步，客户端访问 Block 服务器，请求下载 Block。Block 服务器验证用户权限后，从 Ceph 中读取 Block 数据，返回给客户端，客户端再将返回的 Block 组装为文件。

9.3 详细设计

为解决网盘的三个重要问题：元数据如何管理？网络资源如何向付费用户倾斜？如何做到不重复上传？DBox 详细设计将关注元数据库、上传下载限速、秒传的设计实现。

9.3.1 元数据库设计

元数据库表结构设计如下。



从图中可以看出，元数据库表结构中主要包括三个表，分别是 User 用户表、File 文件表和 Block 数据块表，表的用途和包含的主要字段如下：

1. User 用户表记录用户基本信息：用户名、创建时间、用户类型（免费、VIP）、用户已用空间、电话号码、头像等等。
2. File 文件表记录文件元信息：文件名、是否为文件夹、上级文件夹、文件 MD5、创建时间、文件大小、文件所属用户、是否为共享文件等。
3. Block 数据块表记录 Block 数据，包括 BlockID、Block MD5、对应文件等。

其中，User 表和 File 表为一对多的关系，File 表和 Block 表也是一对多的关系。

这 3 种表的记录数都是百亿级以上，所以元数据表采用**分片的关系数据库**存储。

因为查询的主要场景是根据用户 ID 查找用户信息和文件信息，以及根据文件 ID 查询 block 信息，所以 User 和 File 表都采用 user_id 作为分片键，Block 表采用 file_id 作为分片键。

9.3.2 限速

DBox 根据用户付费类型决定用户的上传、下载速度。而要控制上传、下载速度，可以通过限制并发 Block 服务器数目，以及限制 Block 服务器内的线程数来实现。

具体过程是，客户端程序访问 API 服务器，请求上传、下载文件的时候，API 服务器可以根据用户类型，决定分配的 Block 服务器数目和 Block 服务器内的服务线程数，以及每个线程的上传、下载速率。

Block 服务器会根据 API 服务器的返回值，来控制客户端能够同时上传、下载的 Block 数量以及传输速率，以此对不同用户进行限速。

9.3.3 秒传

秒传是用户快速上传文件的一种功能。

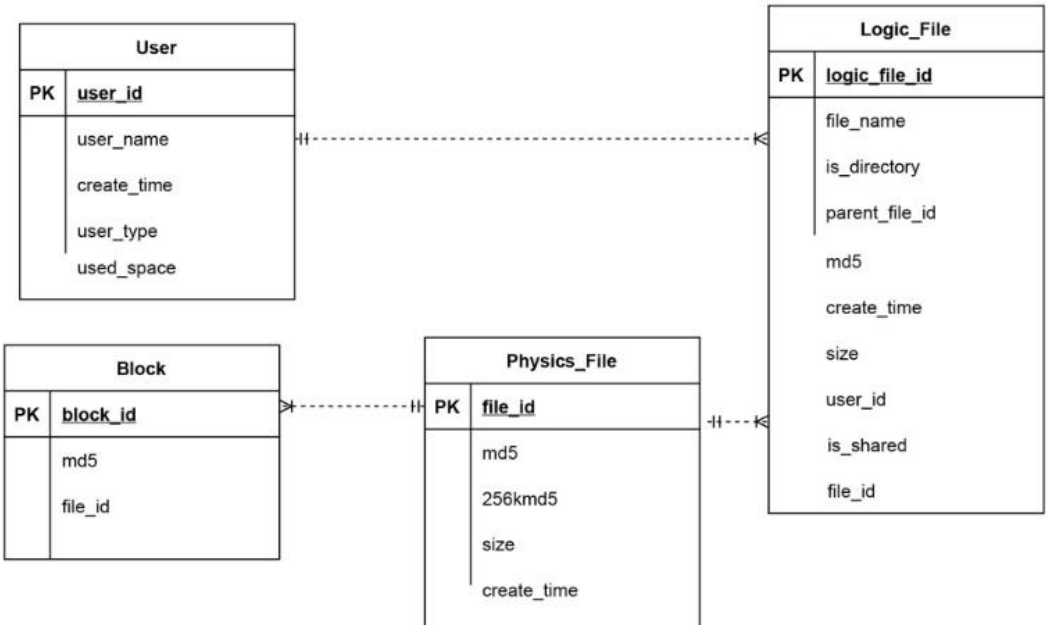
事实上，网盘保存的很多文件，内容其实是重复的，比如电影、电子书等等。一方面，重复上传这些文件会加大网盘的存储负载压力；另一方面，每次都要重新上传重复的内容，会导致用户网络带宽的浪费和用户等待时间过长的问题。

所以，在设计中，物理上相同的文件，DBox 只会保存一份。用户每次上传文件时，DBox 都会先在客户端计算文件的 MD5 值，再根据 MD5 值判断该文件是否已经存在。对于已经存在的文件，只需要建立用户文件和该物理文件的关联即可，并不需要用户真正上传该文件，这样就可以实现秒传的功能。

但是，计算 MD5 可能会发生 Hash 冲突，也就是不同文件算出来的 MD5 值是相同的，这样会导致 DBox 误判，将本不相同的文件关联到一个物理文件上。不但会使上传者丢失自己的文件，还会被黑客利用：上传一个和目标文件 MD5 相同的文件，然后就可以下载目标文件了。

所以，DBox 需要通过更多信息判断文件是否相同：**只有文件长度、文件开头 256KB 的 MD5 值、文件的 MD5 值，三个值都相同，才会认为文件相同。**当文件长度小于 256KB，则直接上传文件，不启用秒传功能。

为此，我们需要将上面的元数据库表结构进行一些改动，将原来的 File 表拆分成物理文件表 Physics_File 和逻辑文件表 Logic_File。其中，Logic_File 记录用户文件的元数据，并和物理文件表 Physics_File 建立多对 1 关联关系，而 Block 表关联的则是 Physics_File 表，如下。



Physics_File 中字段 md5 和 256kmd5 字段分别记录了文件 MD5 和文件头 256KB 的 MD5 数据，而 size 记录了文件长度，只有这三个字段都相同才会启用秒传。

10. 如何设计一个支持三千万用户同时在线的短视频系统?

短视频（short video）通常时长在 15 分钟以内，主要是在移动智能终端上进行拍摄、美化编辑或加特效，并可以在网络社交平台上进行实时分享的一种新型视频形式。短视频具有时间短、信息承载量高等特点，更符合当下网民手机使用行为习惯，短视频的用户流量创造了巨大的商机。

我们准备开发一个面向全球用户的短视频应用，用户总量预计 20 亿，应用名称：QuickTok。

视频文件和其他媒体文件相比，会更大一点，这就意味着存储短视频文件需要更大的存储空间，播放短视频也需要更多的网络带宽。因此，QuickTok 的主要技术挑战是：如何应对高并发用户访问时的网络带宽压力，以及如何存储海量的短视频文件。接下来我们就来看看 QuickTok 的需求与技术架构。

10.1 需求分析

QuickTok 的核心功能需求非常简单：用户上传视频、搜索视频、观看视频。我们将主要分析非功能需求。

QuickTok 预计用户总量为 20 亿，日活用户约 10 亿，每个用户平均每天浏览 10 个短视频，由此可以预估，短视频日播放量为 100 亿：

\(\\small 10 亿\\times10=100 亿\)

平均播放 QPS 为 11 万/秒：

\(\\small 100 亿\\div（24\\times60\\times60）\\approx11 万/秒\)

每秒 11 万用户点击视频，假设用户平均观看 5 分钟，那么同时在观看的视频数就是：

\(\\small 11 万/秒\\times5\\times60 秒=3 千万\)

假设每个短视频的平均播放次数 200 次，那么为了支撑这样体量的播放量，平均需要每秒上传视频数：

\(\\small 11 万/秒\\div200=550/秒\)

每个短视频平均大小 100MB，每秒上传至服务器的文件大小为：

\(\\small 100MB\\times550=55GB\)

（视频虽然不是一秒内上传至服务器的，但是这样计算依然没有问题。）

每年新增视频需要的存储空间：

\(\\small 55GB\\times60\\times60\\times24\\times365=1700PB\)

事实上，为了保证视频数据的高可用，不会因为硬盘损坏导致数据丢失，视频文件需要备份存储，QuickTok 采用双副本的备份存储策略，也就是每个视频文件存储三份，需要的总存储空间：

\(\\small 1700PB\\times3=5200PB\)

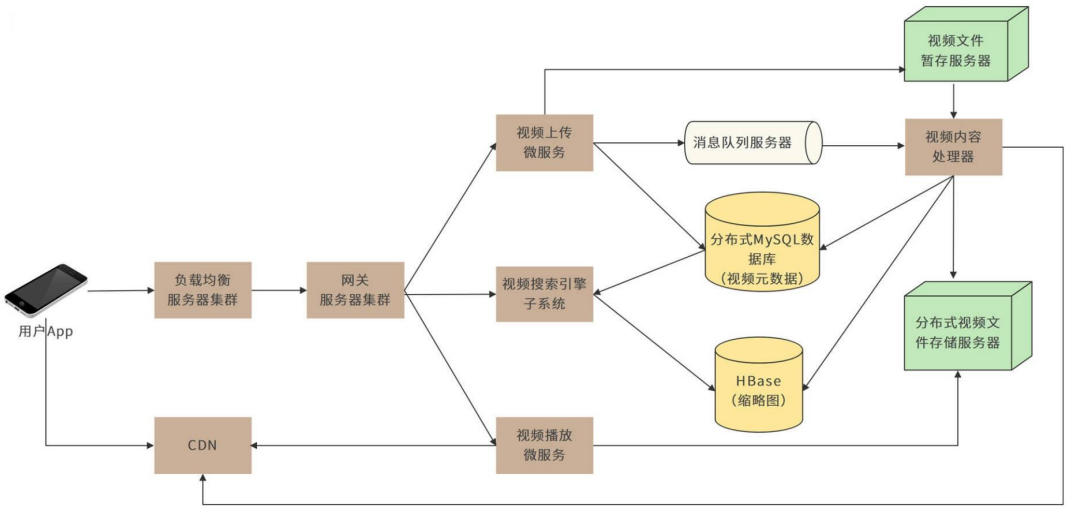
而播放视频需要的总带宽：

\(\\small 11 万\\times100MB\\times8bit=88Tb\)

因此，我们需要设计的短视频应用是一个每秒上传 550 个视频文件、11 万次播放、新增 165GB 存储以及 88Tb 总带宽的**高并发**应用系统。这个系统呢需要是**高性能的**，能迅速响应用户的上传和播放操作，也需要是**高可用**的，能面向全球用户提供 7 * 24 小时稳定的服务。

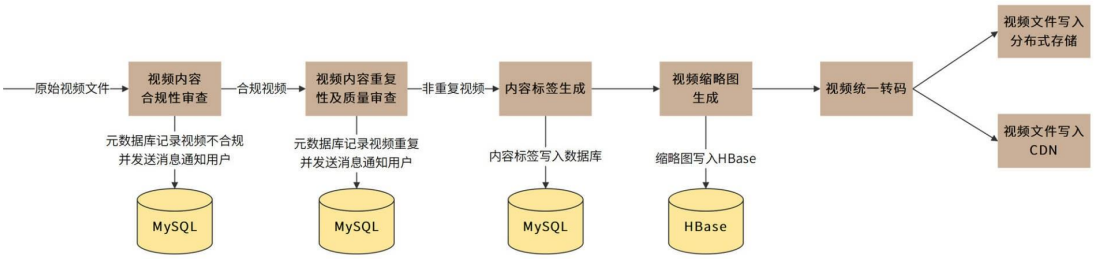
10.2 概要设计

QuickTok 的核心部署模型如下图。

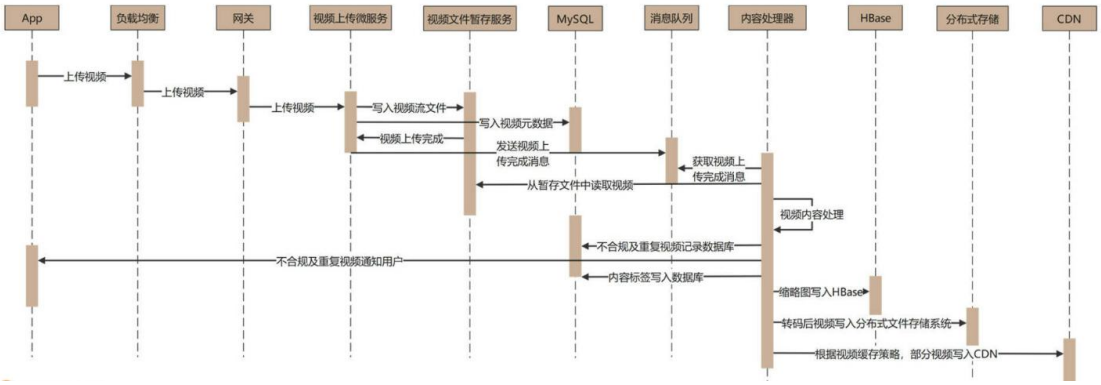


用户上传视频时，上传请求会通过负载均衡服务器和网关服务器，到达视频上传微服务。视频上传微服务需要做两件事：一是把上传文件数据流写入视频文件暂存服务器；二是把用户名、上传时间、视频时长、视频标题等视频元数据写入分布式 MySQL 数据库。视频文件上传完成后，视频上传微服务会生成一个视频上传完成消息，并将其写入到消息队列服务器。视频内容处理器将消费这个上传完成消息，并根据消息内容，从视频文件暂存服务器获取视频文件数据，进行处理。

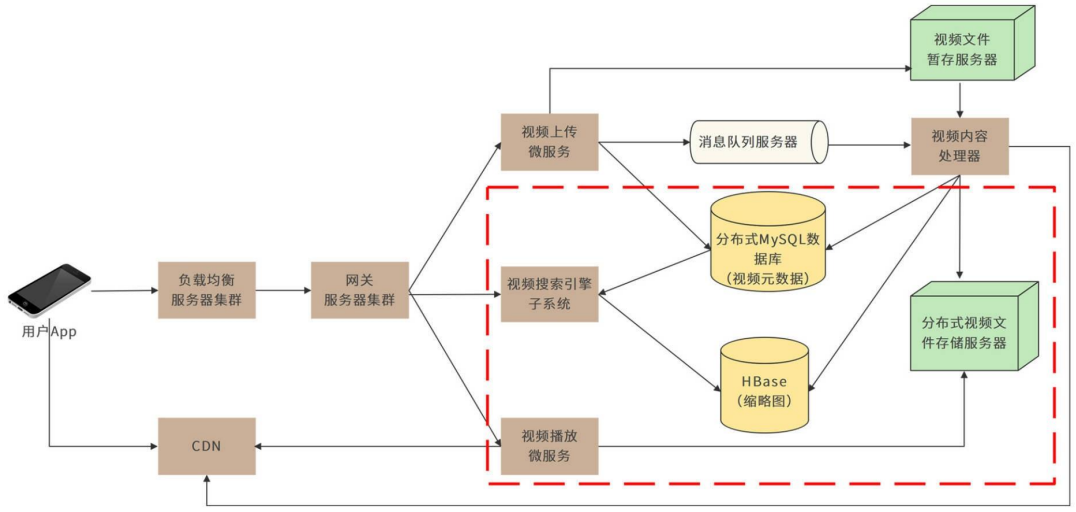
视频内容处理器是一个由责任链模式构建起来的管道。在这个管道中，视频将会被顺序进行内容合规性审查、内容重复性及质量审查、内容标签生成、视频缩略图生成、统一视频转码处理等操作，如下图。



合规且非重复的视频会经过统一转码，最终被写入分布式文件存储和 CDN。这样视频上传处理就完成了，具体时序图如下。



以上就是对视频上传环节的设计，接下来我们将讨论对视频搜索及播放部分的设计，即核心部署模型图中标红的部分，如下。



视频搜索引擎会根据用户提交的视频标题、上传用户等元数据，以及视频内容处理器生成的内容标签构建**倒排索引**。当用户搜索视频时，系统会根据倒排索引来检索符合条件的视频，并返回结果列表。结果列表在 App 端向用户呈现时，会将此前视频内容处理器生成的缩略图展现给用户，使用户对视频内容有个初步而直观的感受。

当用户点击缩略图时，App 开始播放视频。App 并不需要下载完整个视频文件才开始播放，而是**以流的方式一边下载视频数据，一边播放**，使用户尽量减少等待，获得良好的观看体验。QuickTok 使用 MPEG–DASH 流媒体传输协议进行视频流传输，因为这个协议具有自适应能力，而且支持 HTTP，可以应对 QuickTok 的视频播放需求。

10.3 详细设计

为解决 QuickTok 的两个重要问题：如何存储海量视频文件？如何解决高并发视频播放导致的带宽压力？详细设计将关注视频存储系统、性能优化与 CDN。

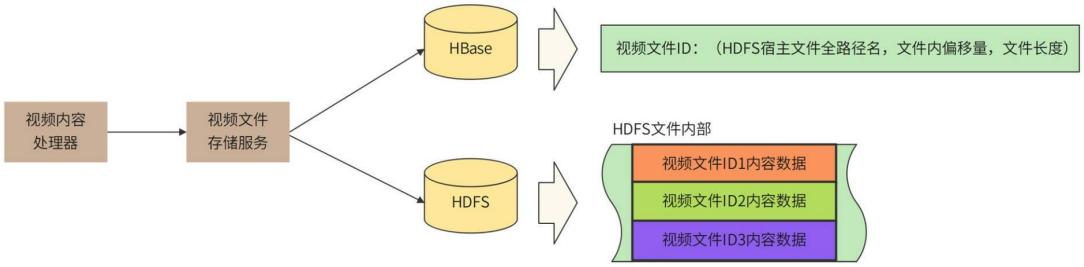
此外，“如何生成更吸引用户的缩略图”是短视频应用用户体验的一个关键问题，详细设计也会关注缩略图生成与推荐的设计实现。

10.3.1 视频存储系统设计

由需求分析可知，QuickTok 每年新增 5200PB 的存储。因此，“如何存储海量视频文件”就是 QuickTok 设计的重要挑战之一。对此，我们可以尝试与[网盘]相同的存储技术方案，将视频文件拆分成若干 block，使用对象存储服务进行存储。

但 QuickTok 最终采用了另一种存储方案，即使用 **Hadoop 分布式文件系统 HDFS** 进行存储。HDFS 适合大文件存储的一次写入多次读取的场景，满足视频一次上传多次播放的需求；同时，它还可以自动进行数据备份（缺省配置下，每个文件存储三份），也满足我们关于数据存储高可用的需求。

HDFS 适合存储大文件，大文件减少磁盘碎片，更有利于存储空间的利用，同时 HDFS NameNode 的访问压力也更小，所以我们需要把若干个视频文件合并成一个 HDFS 文件进行存储，并将存储相关的细节记录到 HBase 中。



举个例子，当用户上传一个视频文件，系统会自动生成一个视频 ID，这里假设这个 ID 是 123。视频内容处理器先对视频进行一系列处理，再调用视频文件存储服务来进行存储。

存储服务首先通过 HDFS 创建一个文件，比如/data/videos/clust0/p0/000000001，然后将视频文件数据顺序写入到 HDFS 中。写完后，存储服务就可以得到这个 HDFS

文件的全路径名(/data/videos/clust0/p0/000000001)、视频文件在 HDFS 中的偏移量 0、文件大小 99,000,000B。

然后，视频文件存储服务再将这些信息记录到 HBase 中，主键就是视频 ID<123>，value 就是。

假设另一个用户上传的视频 ID 为 456，文件大小 100,000,000B，紧随着上一个视频文件，也保存到同一个 HDFS 文件中。那么 HBase 中就可以记录主键<456>，value。

当其他用户播放视频 456 时，播放微服务根据主键 ID 在 HBase 中查找 value 值，得到 HDFS 文件路径/data/videos/clust0/p0/000000001，从该文件 99,000,000 偏移位置开始读取 100,000,000Byte 数据，就是视频 ID 456 完整的文件数据了。

10.3.2 性能优化与 CDN 设计

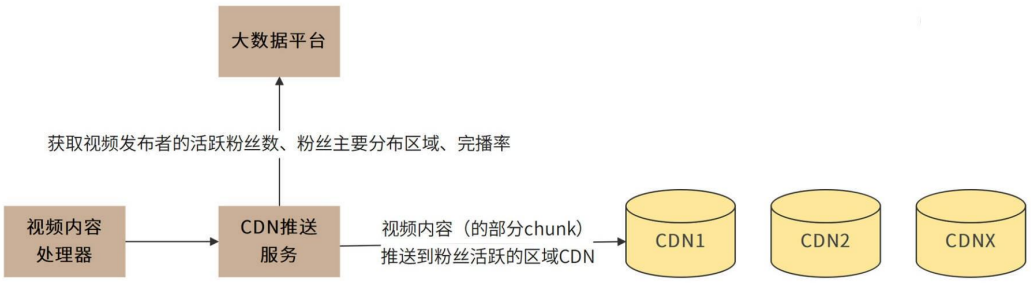
我们前面分析过，QuickTok 需要的总带宽是 88Tb，这是一个非常巨大的数字。如果单纯靠 QuickTok 自己的数据中心来承担这个带宽压力，技术挑战和成本都非常巨大。只有通过 CDN 将用户的网络通信请求就近返回，才能缓解数据中心的带宽压力。

App 请求获取视频数据流的时候，会优先检查离自己比较近的 CDN 中是否有视频数据。如果有，直接从 CDN 加载数据，如果没有，才会从 QuickTok 数据中心获取视频数据流。

如果用户的大部分请求都可以通过 CDN 返回，那么一方面可以极大**加快用户请求的响应速度**，另一方面又可以较大**缓解数据中心的网络和硬盘负载压力**，进一步提升应用整体的性能。

通常的 CDN 设计，是在 CDN 中没有用户请求的数据时，进行回源，即由 CDN 请求数据中心返回需要的数据，然后缓存在 CDN 本地。

但 QuickTok 考虑到了短视频的特点：大 V、网红们发布的短视频会被更快速、更广泛地播放。因此针对粉丝量超过 10 万的用户，系统将采用主动推送 CDN 的方法，以提高 CDN 的命中率，优化用户体验，如图：



从图中可以看出，视频内容处理器进行完视频处理后，一方面会将视频存储到前面说过的视频存储系统中，另一方面又会调用 CDN 推送服务。然后，CDN 推送服务将调用大数据平台，获取视频上传者的活跃粉丝数、粉丝分布区域等数据。如果是 10 万粉丝以上的用户发布了短视频，CDN 推送服务会根据其粉丝活跃的区域，将视频推送到对应区域的 CDN 服务器上。

短视频的完播率通常不足 30%，所以 QuickTok 也不需要将完整视频推送到 CDN，只需要根据视频发布者的历史播放记录，计算其完播率和播放期望进度，然后将短视频切分成若干 chunk，将部分 chunk 推送到 CDN 即可。

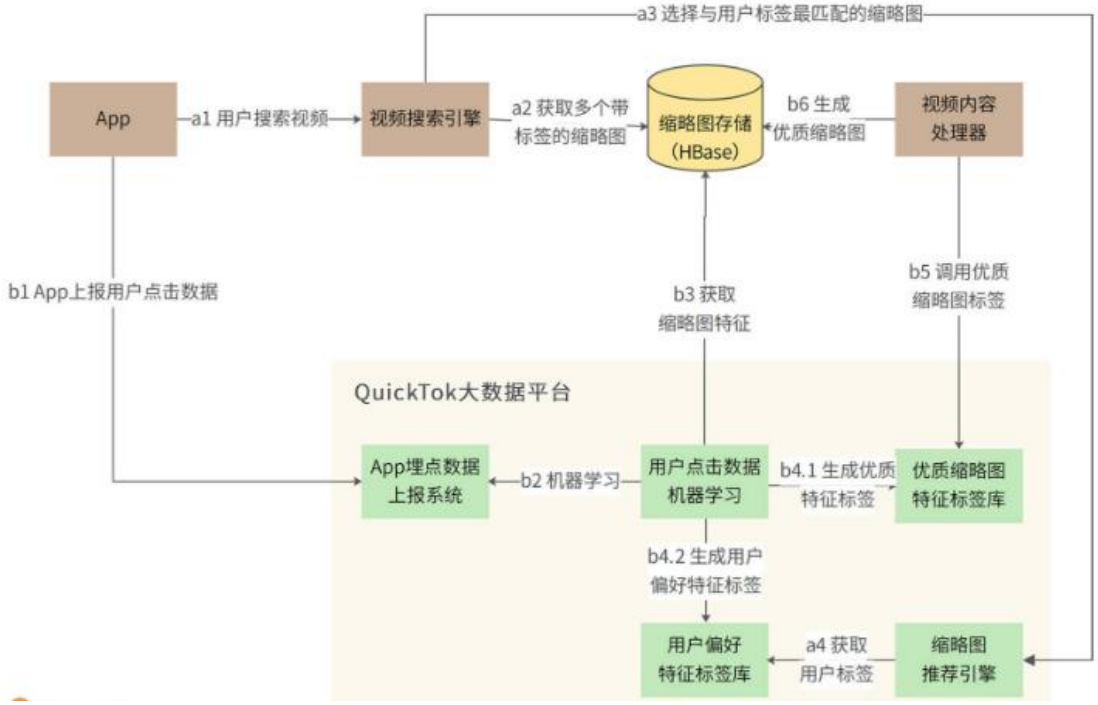
业界一般共识，**视频应用 CDN 处理的带宽大约占总带宽的 95%以上**，也就是说，通过合理使用 CDN，QuickTok 数据中心需要处理的带宽压力不到 4Tb。

10.3.3 缩略图生成与推荐设计

用户可以通过 App 主页、搜索结果页、视频推荐页等页面看到视频列表，其中每个视频都需要有个缩略图。用户点击缩略图，就开始播放视频。

缩略图通常是由视频的某一帧画面缩略而生成的。事实上，缩略图的选择会极大地影响用户点击、播放视频的意愿。一个 10 分钟的视频大约包含 3 万帧画面，选择哪一帧画面，才能使用户点击视频的可能性最大？以及，针对不同的用户分类，是否选择不同的缩略图会产生更高的点击率？

我们需要通过**大数据平台的机器学习引擎**来完成缩略图的生成和推荐，如下图。



缩略图的生成和推荐可以分为两个具体过程：

- 实时在线的缩略图推荐过程 a；
- 利用离线机器学习生成优质缩略图的过程 b。

a 过程中，用户通过搜索引擎搜索视频，搜索引擎产生搜索结果视频列表后，根据视频 ID 从缩略图存储中获取对应的缩略图。

但是，一个视频可能对应很多个缩略图，如果想要显示最吸引当前用户的那个，搜索引擎就需要调用 QuickTok 大数据平台的缩略图推荐引擎进行推荐。

推荐引擎可以获取当前用户的偏好特征标签以及视频对应的多个缩略图特征，使用 XGboost 算法训练好的模型，将用户特征标签和缩略图特征进行匹配，然后返回最有可能被当前用户点击的缩略图 ID。搜索引擎再按照 ID，将对应的缩略图构建到搜索结果页面，返回给用户。

用户浏览搜索结果列表，点击某些缩略图进行播放。App 应用会将用户的浏览与点击数据发送给 QuickTok 大数据平台，这样就进入了利用机器学习来生成优质缩略图的过程 b。

机器学习系统获取到了海量用户的浏览和点击数据，同时获取每个缩略图的特征。一方面，机器可以学习到，哪些特征的缩略图更容易获得用户点击，从而生成优质缩略图特

征标签库；另一方面，机器还可以学习到每个用户自身更偏好的图像特征标签，供前面提到的推荐引擎使用。

有了机器学习系统的加持，视频内容处理器就可以使用优质特征标签库来处理上传的视频内容，抽取符合优质特征的帧，进而生成缩略图。

以上的 a、b 两个过程不断循环迭代，系统就可以不断优化优质特征标签库，不断使缩略图更符合用户喜好。

那最开始没有特征库的时候怎么办呢？视频内容处理器可以使用随机的办法，抽取一些帧作为缩略图，进行冷启动。机器学习再从这些随机抽取的缩略图上开始学习，从而进入循环优化过程。

11. 需要设计一款基于 LBS 的交友系统，如何设计地理空间邻近算法?

交友与婚恋是人们最基本的需求之一。随着互联网时代的不断发展，移动社交软件已经成为了人们生活中必不可少的一部分。然而，熟人社交并不能完全满足年轻人的社交与情感需求，于是陌生人交友平台悄然兴起。

我们决定开发一款基于地理位置服务（LBS）的应用，为用户匹配邻近的、互相感兴趣的好友，应用名称为“Liao”。

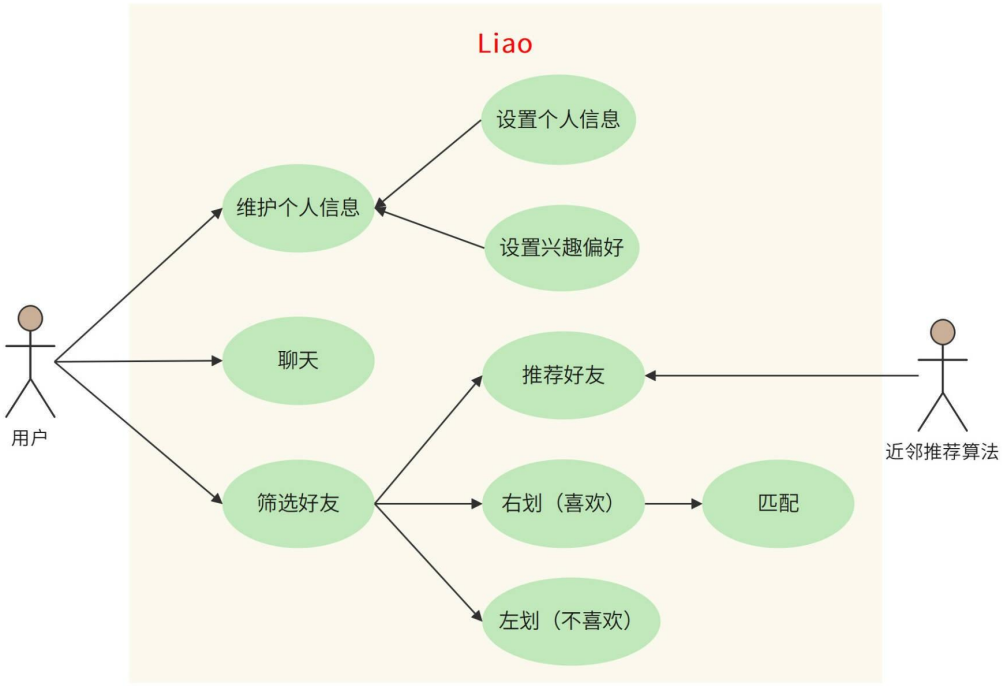
Liao 面临的技术挑战包括：面对海量的用户，如何为其快速找到邻近的人，可以选择的地理空间邻近算法有哪些？Liao 如何在这些算法中选择出最合适的那个？

11.1 需求分析

Liao 的客户端是一个移动 App，用户打开 App 后，上传、编辑自己的基本信息，然后系统（推荐算法）根据其地理位置和个人信息，为其推荐位置邻近的用户。用户在手机上查看对方的照片和资料，如果感兴趣，希望进一步联系，就向右滑动照片；如果不感兴趣，就向左滑动照片。

如果两个人都向右滑动了对方，就表示他们互相感兴趣。系统就通知他们配对成功，并为他们开启聊天功能，可以更进一步了解对方，决定是否建立更深入的关系。

Liao 的功能用例图如下。

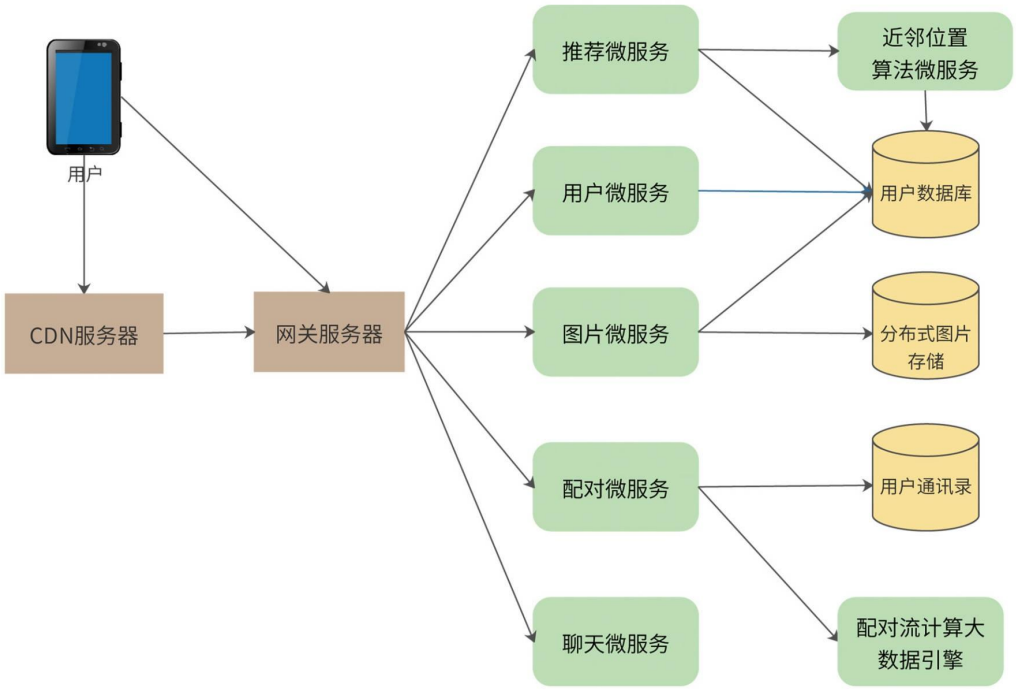


用户规模分析

Liao 的目标用户是全球范围内的中青年单身男女，预估目标用户超过 10 亿，系统按 10 亿用户进行设计。

11.2 概要设计

Liao 的系统架构采用典型的**微服务架构**设计方案，用户通过网关服务器访问具体的微服务，如下图。



由上图可知，首先，用户所有请求都通过统一的**网关服务器**处理。网关服务器负责限流、防攻击、用户身份识别及权限验证、微服务调用及数据聚合封装等，而真正的业务逻辑则通过访问微服务来完成。Liao 的关键微服务有：用户微服务、图片微服务、配对微服务、聊天微服务、推荐微服务、邻近算法微服务等。Liao 的网关预计将承担每天百亿次规模的访问压力。

用户微服务管理用户的个人信息、兴趣爱好以及交友偏好等，此外也负责用户登录服务，只有登录用户才能访问系统。因为需要存储十亿条用户数据，所以用户数据库采用分片的 MySQL 数据库。

图片微服务用于管理用户照片，提供用户照片存储及展示的功能。Liao 需要存储的图片数大约几百亿张。我们使用 Nginx 作为图片服务器，图片服务器可以线性扩容，每写满一台服务器（及其 Slave 服务器），就继续写入下一台服务器。服务器 IP、图片路径则记录在用户数据库中。同时，购买 CDN 服务，缓存热门的用户照片。

配对微服务负责将互相喜欢的用户配对，通知用户，并加入彼此的通讯录中。用户每次右划操作都调用该微服务。系统设置一个用户每天可以喜欢（右划）的人是有上限的，但是，对于活跃用户而言，长期积累下来，喜欢的人的数量还是非常大的，因此配对微服务会将数据发送给一个流式大数据引擎进行计算。

推荐微服务负责向用户展示其可能感兴趣的、邻近的用户。因此，一方面，推荐微服务需要根据用户操作、个人兴趣、交友偏好调用协同过滤等推荐算法进行推荐，另一方面必须保证推荐的用户在当前用户的附近。

11.3 详细设计

详细设计主要关注邻近位置算法，也就是，如何根据用户的地理位置寻找距其一定范围内的其他用户。

我们可以通过 Liao App 获取用户当前经、纬度坐标，然后根据经、纬度，计算两个用户之间的距离，距离计算公式采用半正矢公式：

$$d = 2r\arcsin\left(\sqrt{\operatorname{hav}(\varphi_2 - \varphi_1) + \cos(\varphi_1)\cos(\varphi_2)\operatorname{hav}(\lambda_2 - \lambda_1)}\right)$$
$$= 2r\arcsin\left(\sqrt{\sin^2\left(\frac{\varphi_2 - \varphi_1}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\lambda_2 - \lambda_1}{2}\right)}\right)$$

其中 r 代表地球半径，\(\\small \\varphi\)表示纬度，\(\\small \\lambda\)表示经度。

但是，当我们有 10 亿用户的时候，如果每次进行当前用户匹配都要和其他所有用户做一次距离计算，然后再进行排序，那么需要的计算量至少也是千亿级别，这样的计算量是我们不能承受的。通常的空间邻近算法有以下 4 种，我们一一进行分析，最终选择出最合适的方案。

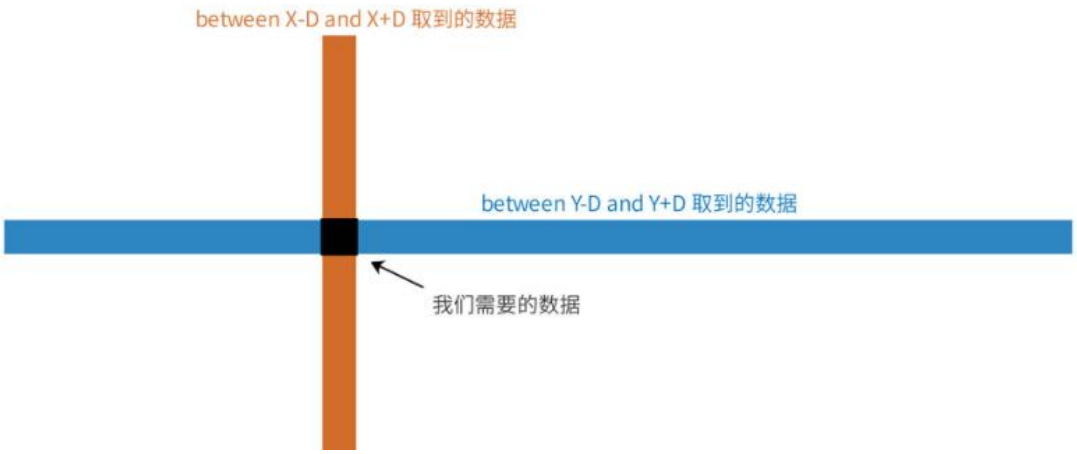
11.3.1 SQL 邻近算法

我们可以将用户经、纬度直接记录到数据库中，纬度记录在 latitude 字段，经度记录在 longitude 字段，用户当前的纬度和经度为 X，Y，如果我们想要查找和当前用户经、纬度距离 D 之内的其他用户，可以通过如下 SQL 实现。

```
1  select*from users where latitude between X-D and X+D and longitude between Y-D and Y+D;
```

这样的 SQL 实现起来比较简单，但是如果有十亿用户，数据分片在几百台服务器上，SQL 执行效率就会很低。而且我们用经、纬度距离进行近似计算，在高纬度地区，这种近似计算的偏差还是非常大的。

同时“between X-D and X+D”以及“between Y-D and Y+D”也会产生大量中间计算数据，这两个 betwen 会先返回经度和纬度各自区间内的所有用户，再进行交集 and 处理，如下图。



我们的用户量非常大，而计算邻近好友又是一个非常高频的访问，同时，分片数据库进行集合计算需要在中间代理服务器或应用程序服务器完成计算，因此，这样的交集计算带来计算负载压力是我们的系统完全不能承受的。所以这个方案可以被放弃。

11.3.2 地理网格邻近算法

为了减少上述交集计算使用的中间数据量，我们将整个地球用网格进行划分，如下图。

事实上，我们划分的网格远比图中示意的要密集得多，赤道附近，经、纬度方向每 10 公里一个网格。

这样每个用户必然会落入到一个网格中，我们在用户表中记录用户所在的网格 ID（gridID），然后借助这个字段进行辅助查找，将查找范围限制在用户所在的网格（gridIDx0）及其周围 8 个网格（gridIDx1 ~ gridIDx8）中，可以极大降低中间数据量，SQL 如下。

```
select*from users where latitude between X-D and X+D and longitude between Y-D and Y+D and gridID in (gridIDx0,gridIDx1,gridIDx2,gridIDx3,gridIDx4,gridIDx5,gridIDx6,gridIDx7,gridIDx8);
```

这条 SQL 要比上面 SQL 的计算负载压力小得多，但是对于高频访问的分片数据库而言，用这样的 SQL 进行邻近好友查询依然是不能承受的，同样距离精度也不满足要求。

但是基于这种网格设计思想，我们发现，我们可以不通过数据库就能实现邻近好友查询：我们可以**将所有的网格及其包含的用户都记录在内存中**。当我们进行邻近查询时，只需要在内存中计算用户及其邻近的 8 个网格内的所有用户的距离即可。

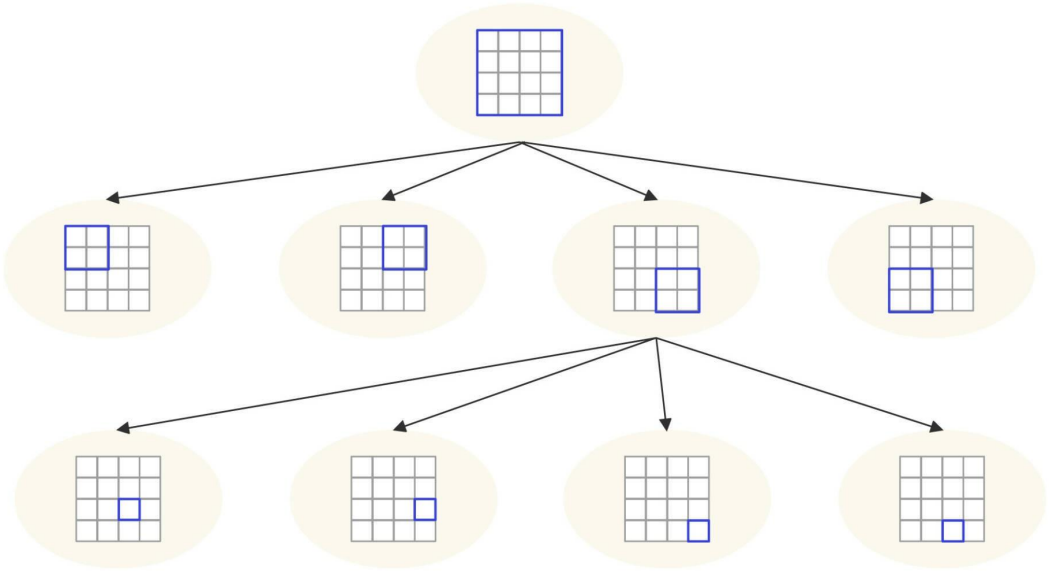
我们可以估算下所有用户经、纬度都加载到内存中需要的内存量：\(\\small 1G\\times3\\times4B=12GB\)（用户 ID、经度、纬度，都采用 4 个字节编码，总用户数 1G）。这个内存量是完全可以接受的。

实际上，通过恰当地选择网格的大小，我们不停访问当前用户位置周边的网格就可以由近及远不断得到邻近的其他用户，而不需要再通过 SQL 来得到。那么如何选择网格大小？如何根据用户位置得到其所在的网格？又如何得到当前用户位置周边的其他网格呢？我们看下实践中更常用的动态网格和 GeoHash 算法。

11.3.3 动态网格算法

事实上，不管如何选择网格大小，可能都不合适。因为在陆家嘴即使很小的网格可能就包含近百万的用户，而在可可西里，非常大的网格也包含不了几个用户。

因此，我们希望能够动态设定网格的大小，如果一个网格内用户太多，就把它分裂成几个小网格，小网格内如果用户还是太多，继续分裂更小的网格，如下图。



这是一个四叉树网格结构，开始的时候整个地球只有一个网格，当用户增加，超过阈值（500 个用户）的时候，就分裂出 4 个子树，4 个子树对应父节点网格的 4 个地理子网格。同时，将用户根据位置信息重新分配到 4 个子树中。同样，如图中所示，如果某个子树中的用户增加，超过了阈值，该子树继续分裂成 4 个子树。

因此，我们可以将全球用户分配在这样一个 4 叉树网格结构中，所有的用户都必然在这个 4 叉树的叶子节点中，而且每个节点内包含的用户数不超过 500 个。那么，陆家嘴的网格可能就会很小，而可可西里的网格就会很大，太平洋对应的网格可能有几千公里。

当给定当前用户的经、纬度，查询邻近用户的时候，首先从根节点开始查找，如果根节点就是叶子节点，那么直接遍历根节点中的所有用户，计算距离即可。如果根节点不是叶子节点，那么根据给定的经、纬度判断其在网格中的位置，左上、右上、右下、左下，4 个位置，顺序对应 4 个子树，根据网格位置访问对应的子树。如果子树是叶子节点，那么在叶子节点中查找，如果不是叶子节点，继续上面的过程，直到叶子节点。

上面的过程只能找到当前用户所在网格的好友，如何查找邻近网格的其他用户呢？事实上，我们只需要将 4 叉树所有的叶子节点顺序组成一个双向链表，每个节点在链表上的若干个前驱和后继节点正好就是其地理位置邻近的节点。

动态网格也叫 4 叉树网格，在空间邻近算法中较为常用，也能满足 Liao 的需求。但是编程实现稍稍有点麻烦，而且如果网格大小设计不合适，导致树的高度太高，每次查找

需要遍历的路径太长，性能结果也比较差。我们再看下性能和灵活性更好的 GeoHash 算法。

11.3.4 GeoHash 算法

除了动态网格算法，GeoHash 事实上是另外一种变形了的网格算法，同时也是 Redis 中 Geo 函数使用的算法。GeoHash 是将网格进行编码，然后根据编码进行 Hash 存储的一种算法。

经、纬度数字的不同精度，意味着经、纬度的误差范围，比如保留经、纬度到小数点后第 1 位，那么误差范围最大可能会达到 11 公里（在赤道附近）。也就是说，小数点后 1 位精度的经、纬度，其覆盖范围是一个 11km * 11km 的网格。

那么，我们用小数点后 1 位精度的经、纬度做 key，网格内的用户集合做 value，就可以构建一个 Hash 表的<key, value="" style="box-sizing: inherit;">对。通过查找这个 KV 对及其周围 8 个网格的 KV 对，计算这些 value 内所有用户和当前用户的距离，就可以找到邻近 11 公里内的所有用户。

实践中，redis 的 GeoHash 并不会直接用经、纬度做 key，而是采用一种基于 Z 阶曲线的编码方式，将二维的经、纬度，转化为一维的二进制数字，再进行 base32 编码，具体过程如下。

首先，分别针对经度和纬度，求取当前区间（对于纬度而言，开始的区间就是[-90, 90]，对于经度而言，开始区间就是[-180, 180]）的平均值，将当前区间分为两个区间。然后用用户的经、纬度和区间平均值进行比较，用户经、纬度必然落在两个区间中的一个，如果大于平均值，那么取 1，如果小于平均值，那么取 0。继续求取当前区间的平均值，进一步将当前区间分为两个区间。如此不断重复，可以在经度和纬度方向上，得到两个二进制数。这个二进制数越长，其所在的区间越小，精度越高。

下图表示经、纬度<42.60411, -5.59041>的二进制编码过程，最终得到纬度 12 位编码，经度 13 位编码。

Latitude code 101111001001						
bit position	bit value	min	mid	max	mean value	maximum error
0	1	-90.000	0.000	90.000	45.000	45.000
1	0	0.000	45.000	90.000	22.500	22.500
2	1	0.000	22.500	45.000	33.750	11.250
3	1	22.500	33.750	45.000	39.375	5.625
4	1	33.750	39.375	45.000	42.188	2.813
5	1	39.375	42.188	45.000	43.594	1.406
6	0	42.188	43.594	45.000	42.891	0.703
7	0	42.188	42.891	43.594	42.539	0.352
8	1	42.188	42.539	42.891	42.715	0.176
9	0	42.539	42.715	42.891	42.627	0.088
10	0	42.539	42.627	42.715	42.583	0.044
11	1	42.539	42.583	42.627	42.605	0.022

Longitude code 0111110000000						
bit position	bit value	min	mid	max	mean value	maximum error
0	0	-180.000	0.000	180.000	-90.000	90.000
1	1	-180.000	-90.000	0.000	-45.000	45.000
2	1	-90.000	-45.000	0.000	-22.500	22.500
3	1	-45.000	-22.500	0.000	-11.250	11.250
4	1	-22.500	-11.250	0.000	-5.625	5.625
5	1	-11.250	-5.625	0.000	-2.813	2.813
6	0	-5.625	-2.813	0.000	-4.219	1.406
7	0	-5.625	-4.219	-2.813	-4.922	0.703
8	0	-5.625	-4.922	-4.219	-5.273	0.352
9	0	-5.625	-5.273	-4.922	-5.449	0.176
10	0	-5.625	-5.449	-5.273	-5.537	0.088
11	0	-5.625	-5.537	-5.449	-5.581	0.044
12	0	-5.625	-5.581	-5.537	-5.603	0.022

得到两个二进制数后，再将它们合并成一个二进制数。合并规则是，从第一位开始，奇数位为经度，偶数位为纬度，上面例子合并后的结果为 01101 11111 11000 00100 00010 ，共 25 位二进制数。

将 25 位二进制数划分成 5 组，每组 5 个二进制数，对应的 10 进制数是 0-31，采用 Base32 编码，可以得到一个 5 位字符串，Base32 编码表如下。

Decimal	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Base 32	0	1	2	3	4	5	6	7	8	9	b	c	d	e	f	g
Decimal	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Base 32	h	j	k	m	n	p	q	r	s	t	u	v	w	x	y	z

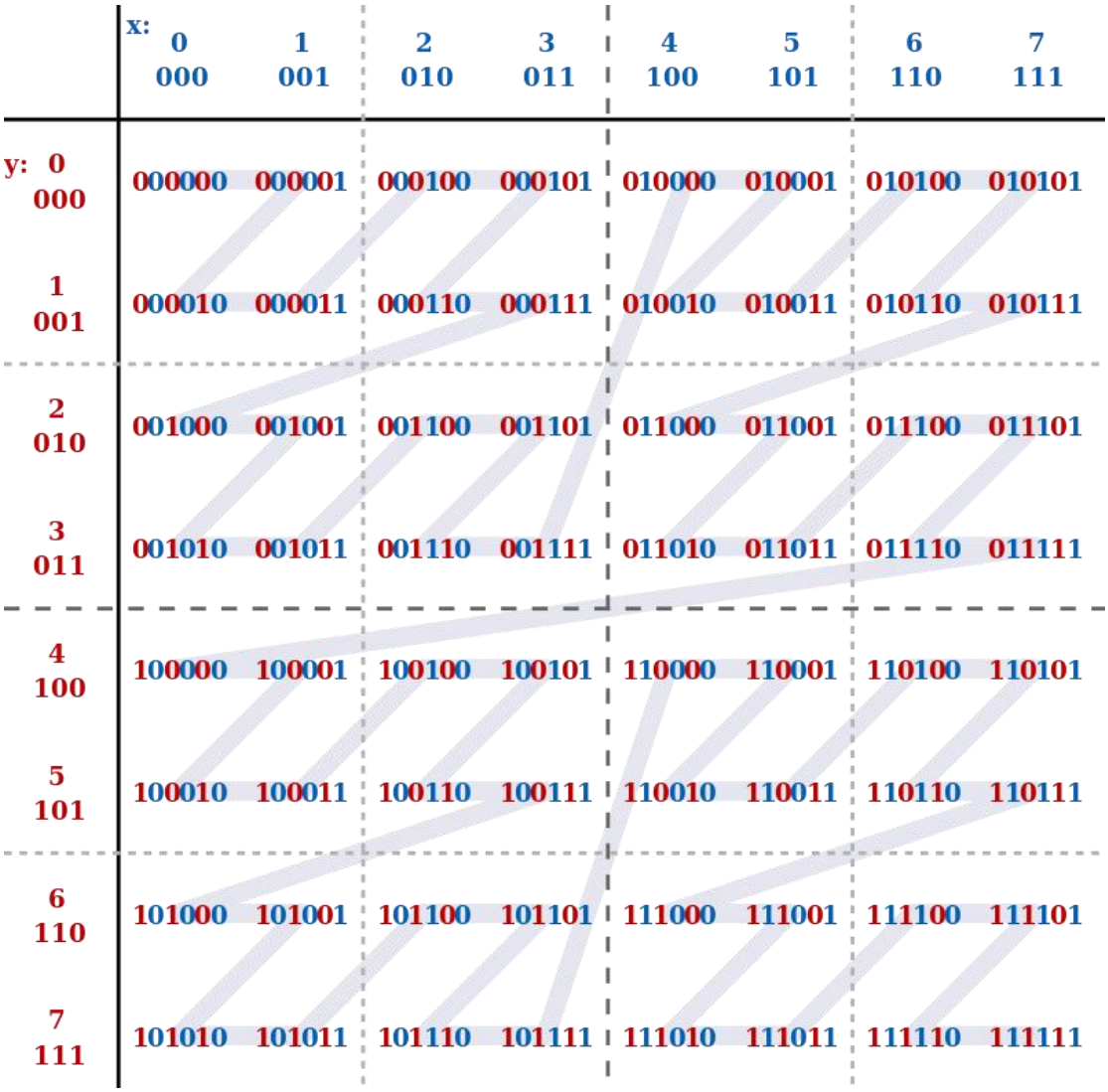
编码计算过程如下。

$$
\begin{aligned}
[e]_{32ghs} &= [13]_{10} = [01101]_2 \\
[z]_{32ghs} &= [31]_{10} = [11111]_2 \\
[s]_{32ghs} &= [24]_{10} = [11000]_2 \\
[4]_{32ghs} &= [4]_{10} = [00100]_2 \\
[2]_{32ghs} &= [2]_{10} = [00010]_2 .
\end{aligned}
$$

最后得到一个字符串“ezs42”，作为 Hash 表的 key。25 位二进制的 GeoHash 编码，其误差范围大概 2.4 公里，即对应一个\(\\small 2.4km\\times2.4km\)的网格。网格内的用户都作为 value 放入到 Hash 表中。

一般说来，通过选择 GeoHash 的编码长度，实现不同大小的网格，就可以满足我们邻近交友的应用场景了。但是在 Redis 中，需要面对更通用的地理位置计算场景，所以 Redis 中的 GeoHash 并没有用 Hash 表存储，而是用跳表存储。

Redis 使用 52 位二进制的 GeoHash 编码，误差范围 0.6 米。Redis 将编码后的二进制数按照 Z 阶曲线的布局，进行一维化展开。即将二维的经、纬度上的点，用一条 Z 型曲线连接起来，Z 阶曲线布局示例如下图。



事实上，所谓的 Z 阶曲线布局，本质其实就是基于 GeoHash 的二进制排序。将这些经过编码的 2 进制数据用跳表存储。查找用户的时候，可以快速找到该用户，沿着跳表前后检索，得到的就是邻近的用户。

11.3.5 Liao 的最终算法选择

Liao 的邻近算法最终选择使用 Hash 表存储的 GeoHash 算法，经度采用 13bit 编码，纬度采用 12bit 编码，即最后的 GeoHash 编码 5 个字符，每个网格\(\\small 4.9km\\times4.9km\\approx 25km^{2}\)，将整个地球分为\(\\small 2^{25}\\approx3300 万\)个网格，去掉海洋和几乎无人生存的荒漠极地，需要存储的 Hash 键不到 500 万个，采用 Hash 表存储。Hash 表的 key 是 GeoHash 编码，value 是一个 List，其中包含了所有相同 GeoHash 编码的用户 ID。

查找邻近好友的时候，Liao 将先计算用户当前位置的 GeoHash 值（5 个字符），然后从 Hash 表中读取该 Hash 值对应的所有用户，即在同一个网格内的用户，进行匹配，将满足匹配条件的对象返回给用户。如果一个网格内匹配的对象数量不足，计算周围 8 个网格的 GeoHash 值，读取这些 Hash 值对应的用户列表，继续匹配。

12. 搜索引擎设计：信息搜索怎么避免大海捞针？

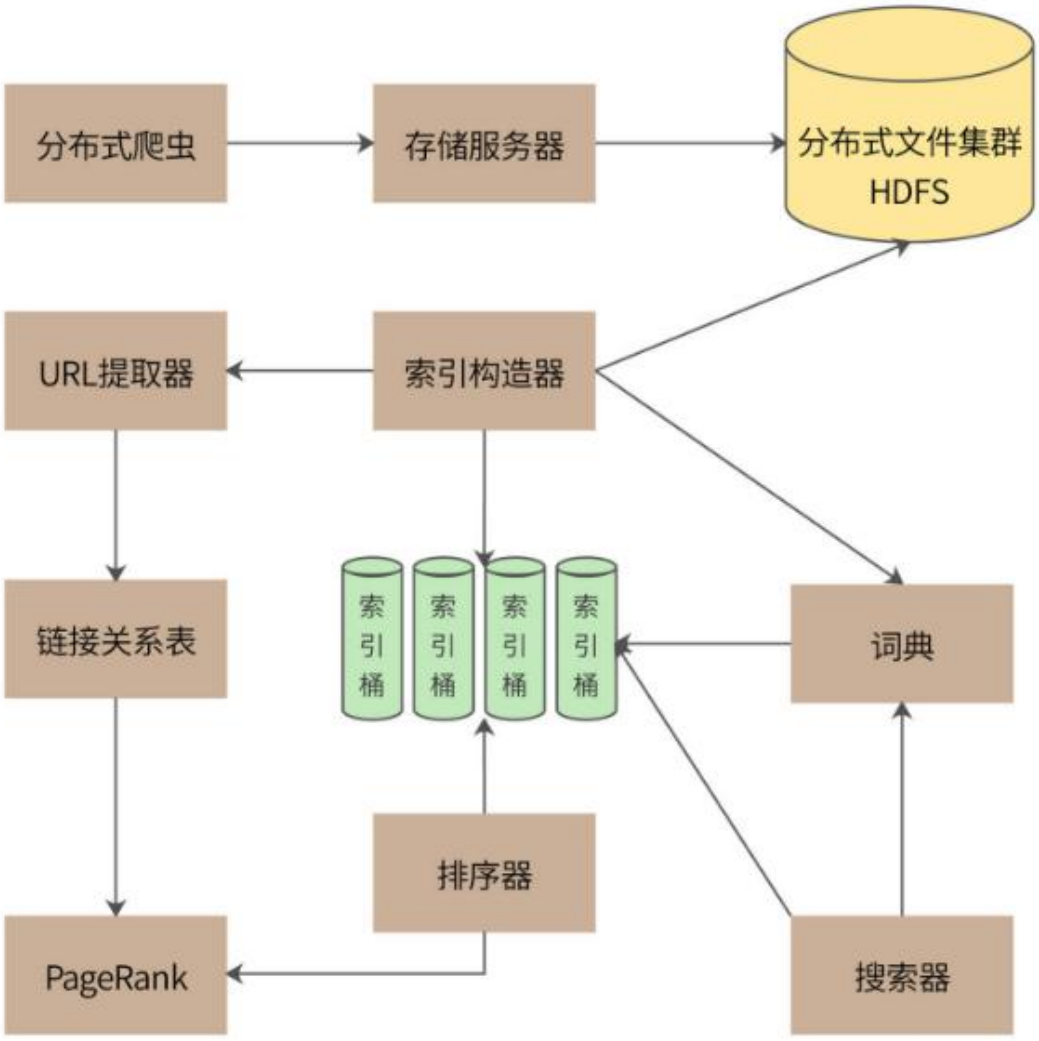
我们准备开发一个针对全网内容的搜索引擎，产品名称为“Bingoo”。

Bingoo 的主要技术挑战包括：

1. 针对爬虫获取的海量数据，如何高效地进行数据管理；
2. 当用户输入搜索词的时候，如何快速查找包含搜索词的网页内容；
3. 如何对搜索结果的网页内容进行排序，使排在搜索结果列表前面的网页，正好是用户期望看到的内容。

12.1 概要设计

一个完整的搜索引擎包括分布式爬虫、索引构造器、网页排名算法、搜索器等组成部分，Bingoo 的系统架构如下。



分布式爬虫通过存储服务器将爬取的网页存储到分布式文件集群 HDFS，为了提高存储效率，网页将被压缩后存储。存储的时候，网页一个文件挨着一个文件地连续存储，存储格式如下。

docID	URL长度	数据长度（压缩后）	URL	网页内容数据（压缩后）

每个网页被分配得到一个 8 字节长整型 docID，docID 之后用 2 个字节记录网页的 URL 的长度，之后 4 个字节记录压缩后网页内容数据的长度，所有存储的网页的头 14 个字节都是同样的格式。之后存储 URL 字符串和压缩后的网页内容数据。读取文件的时候，

先读 14 个字节的头信息，根据头信息中记录的 URL 长度和数据长度，再读取对应长度的 URL 和网页内容数据。

搜索引擎能够快速查找的核心就是利用索引，根据用户的查询内容查找匹配的索引，根据索引列表构建结果页面。索引的构造主要通过索引构造器完成，索引构造器读取 HDFS 中的网页内容，解压缩后提取网页中的单词，构建一个“docID->单词列表”的正排索引。然后，索引构造器再根据这个正排索引构建一个“单词->docID 列表”的倒排索引，“docID 列表”就是包含了这个单词的所有网页列表。利用这个倒排索引，搜索器可以快速获得用户搜索词对应的所有网页。

网页中所有的单词构成了一个词典，实际上，词典就是一个 Hash 表，key 就是单词，value 就是倒排索引的网页列表。虽然互联网页的内容非常庞大，但是使用到的单词其实是非常有限的。根据 Google 的报告，256M 内存可以存放 1400 万个单词，这差不多就是英文单词的全部了。

在构建索引的过程中，因为要不断修改索引列表，还要进行排序，所以，有很多操作是需要进行加锁同步完成的。对于海量的互联网页的计算，这样的索引构建速度太慢了。因此我们设计了 64 个索引桶，根据 docID 取模，将不同网页分配到不同的桶中，在每个桶中分别进行索引构建，通过并行计算来加快索引处理速度。

索引构造器在读取网页内容、构造索引的时候，还会调用 URL 提取器，将网页中包含的 URL 提取出来，构建一个链接关系表。链接关系表的格式是“docID->docID”，前一个 docID 是当前网页的 docID，后一个 docID 是当前网页中包含的 URL 对应的 docID。一个网页中会包含很多个 URL，也就是会构建出很多个这样的链接关系。后面会利用这个链接关系表，使用 PageRank 排名算法对所有网页进行打分排名，当索引器得到查找的网页列表时，利用 PageRank 值进行排名，最终呈现给用户，保证用户最先看到的网页是最接近用户期望的结果页面。

12.2 详细设计

一个运行良好的搜索引擎的核心技术就是索引和排名，所以我们将分别说明这两种技术要点。

12.2.1 索引

索引构造器从 HDFS 读取网页内容后，解析每个页面，提取网页里的每个单词。如果是英文，那么每个单词都用空格分隔，比较容易；如果是中文，需要使用中文分词器才能提取到每个单词，比如“高并发架构”，使用中文分词器得到的就是“高并发”、“架构”两个词。

首先，索引构造器将所有的网页都读取完，构建出所有的“docID->单词列表”正排索引。

1	架构，分布式，缓存，队列，微服务
2	高并发，架构，设计，文档，写作

然后遍历所有的正排索引，再按照“单词→docID 列表”的方式组织起来，就是倒排索引了。

单词	网页列表
高并发	2、4、5、7
架构	1、2、4

我们这个例子中只有两个单词、7 个网页。事实上，Bingoo 数以千亿的网页就是这样通过倒排索引组织起来的，网页数量虽然庞大，但是单词数却是比较有限的。所以，整个倒排索引的大小相比于网页数量要小得多。Bingoo 将每个单词对应的网页列表存储在硬盘中，而单词则存储在内存的 Hash 表，也就是词典中，词典示例：

单词	网页列表地址
高并发	0xa46fc960
架构	0x8a8f29be

对于部分热门的单词，整个网页列表也可以存储在内存中，相当于缓存。在词典中，每个单词记录下硬盘或者内存中的网页列表地址，这样只要搜索单词，就可以快速得到对应的网页地址列表。Bingoo 根据列表中的网页编号 docID，展示对应的网页信息摘要，就完成了海量数据的快速检索。

如果用户的搜索词正好是一个单词，比如“高并发”，那么直接查找词典，得到网页列表就完成查找了。但是如果用户输入的是一个句话，那么搜索器就需要将这句话拆分成几个单词，然后分别查找倒排索引。这样的话，得到的就是几个网页列表，还需要对这几个网页列表求交集，才能得到最终的结果列表。

比如，用户输入“高并发架构”进行搜索，那么搜索器就会拆分成两个词：“高并发”、“架构”，得到两个倒排索引：

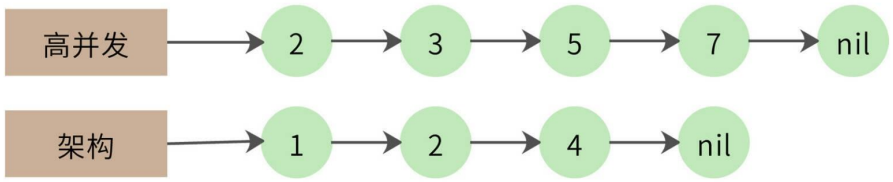
高并发->2,3,5,7

架构->1,2,4

需要对这两个倒排索引求交集，也就是同时包含“高并发”和“架构”的网页才是符合搜索要求的结果，最终的交集结果应该是只有一篇网页，即 docID 为 2 的满足要求。

列表求交集最简单的实现就是双层 for 循环，但是这种算法的时间复杂度是 O(n^2)，我们的网页列表长度（n）可能有千万级甚至更高，这样的计算效率太低。

一个改进的算法是**拉链法**，我们将网页列表先按照 docID 的编号进行排序，得到的就是这样两个有序链表：

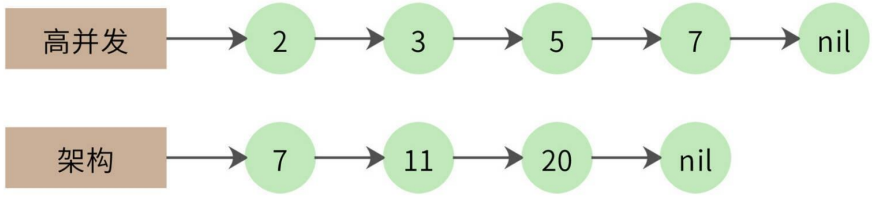


同时遍历两个链表，如果其中一个链表当前指向的元素小于另一个链表当前指向的元素，那么这个链表就继续向前遍历；如果两个链表当前指向的元素相同，该元素就是交集元素，记录在结果列表中；依此继续向前遍历，直到其中一个链表指向自己的尾部 nil。

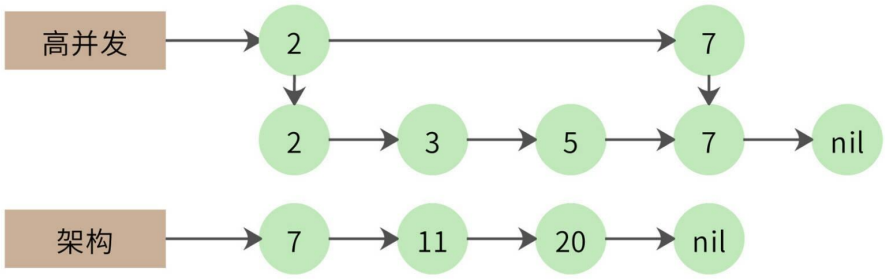
拉链法的时间复杂度是 O(2n)，远优于双层循环。但是对于千万级的数据而言，还是太慢。我们还可以采用**数据分片**的方式进行并行计算，以实现性能优化。

比如，我们的 docID 分布在[0, 1 万亿)区间，而每个倒排索引链表平均包含 1 千万个 docID。我们把所有的 docID 按照 1 千亿进行数据分片，就会得到 10 个区间[0, 1 千亿)[1 千亿，2 千亿)……[9 千亿，1 万亿)。每个倒排索引链表大致均匀分布在这 10 个区间，我们就可以依照这 10 个区间范围，将每个要遍历的链表切分为 10 片，每片大约包含 1 百万个 docID。两个链表只在自己对应的分片内求交集即可，因此我们可以启动 10 个线程对 10 个分片进行并行计算，速度可提高 10 倍。

事实上，两个 1 千万长度的链表求交集，最终的结果可能不过几万，也就是说，大部分的比较都是不相等的。比如下面的例子。



第一个链表遍历到自己的最后一个元素，才和第二个链表的第一个元素相同。那么第一个链表能不能跳过前面那些元素呢？很自然，我们想到可以用**跳表**来实现，如下图。



跳表实际上是在链表上构建多级索引，在索引上遍历可以跳过底层的部分数据，我们可以利用这个特性实现链表的跳跃式比较，加快计算速度。使用跳表的交集计算时间复杂度大约是 O(log(n))。

此外，虽然搜索引擎利用倒排索引已经能很快得到搜索结果了，但搜索引擎应用还会使用缓存对搜索进行加速，将整个搜索词对应的搜索结果直接放入缓存，以减少倒排索引的访问压力，以及不必要的集合计算。

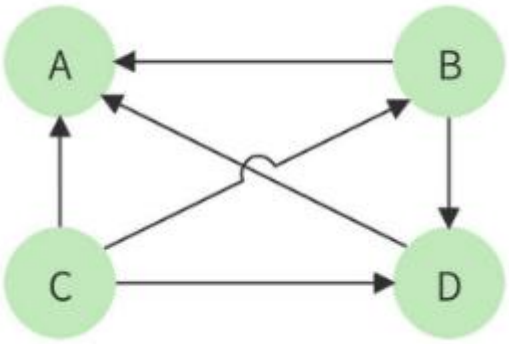
12.2.2 PageRank 排名算法

Bingoo 使用 PageRank 算法进行网页结果排名，以保证搜索结果更符合用户期待。

PageRank 算法会根据网页的链接关系给网页打分。如果一个网页 A 包含另一个网页 B 的超链接，那么就认为 A 网页给 B 网页投了一票。一个网页得到的投票越多，说明自己越重要；越重要的网页给自己投票，自己也越重要。

PageRank 算法就是计算每个网页的 PageRank 值，最终的搜索结果也是以网页的 PageRank 值排序，展示给用户。事实证明，这种排名方法非常有效，PageRank 值更高的网页，确实更满足用户的搜索期望。

以下面四个网页 A、B、C、D 举例，带箭头的线条表示链接。



B 网页包含了 A、D 两个页面的超链接，相当于 B 网页给 A、D 每个页面投了一票，如果初始的时候，所有页面都是 1 分，那么经过这次投票后，B 给了 A 和 D 每个页面 1/2 分（B 包含了 A、D 两个超链接，所以每个投票值 1/2 分），自己从 C 页面得到 1/3 分（C 包含了 A、B、D 三个页面的超链接，每个投票值 1/3 分）。

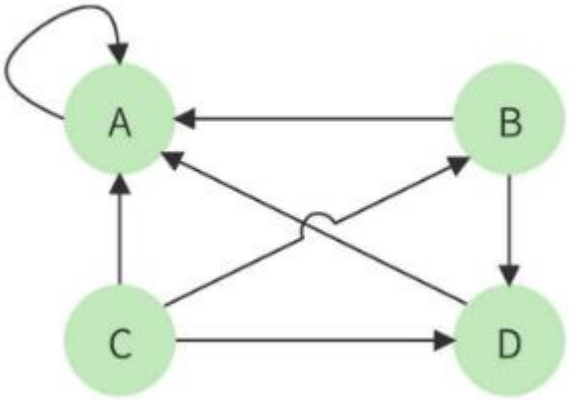
而 A 页面则从 B、C、D 分别得到 1/2，1/3，1 分。用公式表示就是

\(\\small PR（A） = \\frac{PR（B）}{2}+\\frac{PR（C）}{3}+\\frac{PR（D）}{1}\)

等号左边是经过一次投票后，A 页面的 PageRank 分值；等号右边每一项的分子是包含 A 页面超链接的页面的 PageRank 分值，分母是该页面包含的超链接数目。

这样经过一次计算后，每个页面的 PageRank 分值就会重新分配，重复同样的算法过程，经过几次计算后，根据每个页面 PageRank 分值进行排序，就得到一个页面重要程度的排名表。根据这个排名表，将用户搜索出来的网页结果排序，排在前面的通常也正是用户期待的结果。

但是这个算法还有个问题，如果某个页面只包含指向自己的超链接，其他页面不断给它送分，而自己一分不出，随着计算执行次数越多，它的分值也就越高，这显然是不合理的。这种情况就像下图所示的，A 页面只包含指向自己的超链接。



解决方案是，设想浏览一个页面的时候，有一定概率不是点击超链接，而是在地址栏输入一个 URL 访问其他页面，表示在公式上，就是

\(\\small PR（A） = \\alpha(\\frac{PR（B）}{2}+\\frac{PR（C）}{3}+\\frac{PR（D）}{1})+\\frac{（1-\\alpha）}{4}\)

上面\(\\small （1-\\alpha）\)就是跳转到其他任何页面的概率，通常取经验值 0.15(即\(\\small \\alpha\) 为 0.85)，因为有一定概率输入的 URL 是自己的，所以加上上面公式最后一项，其中分母 4 表示所有网页的总数。

那么对于 N 个网页，任何一个页面\(\\small P_{i}\)的 PageRank 计算公式如下：

\(\\small PageRank（P_{i}）=\\alpha \\sum_{P_{j}\\in M(P_{i})}^{}{\\frac{PageRank(P_{j})}{L(P_{j})}} + \\frac{1-\\alpha}{N}\)

公式中，\(\\small P_{j}\\in M(P_{i})\) 表示所有包含有\(\\small P_{i}\)超链接的\(\\small P_{j}\)，\(\\small L(P_{j})\)表示\(\\small P_{j}\)页面包含的超链接数，N 表示所有的网页总和。由于 Bingoo 要对全世界的网页进行排名，所以这里的 N 是一个万亿级的数字。

计算开始的时候，将所有页面的 PageRank 值设为 1，带入上面公式计算，每个页面都得到一个新的 PageRank 值。再把这些新的 PageRank 值带入上面的公式，继续得到更新的 PageRank 值，如此迭代计算，直到所有页面的 PageRank 值几乎不再有大的变化才停止。

13. 微博系统设计：怎么应对热点事件的突发访问压力？

微博（microblog）是一种允许用户即时更新简短文本（比如 140 个字符），并可以公开发布的微型博客形式。今天我们就来开发一个面向全球用户、可以支持 10 亿级用户体量的微博系统，系统名称为“Weitter”。

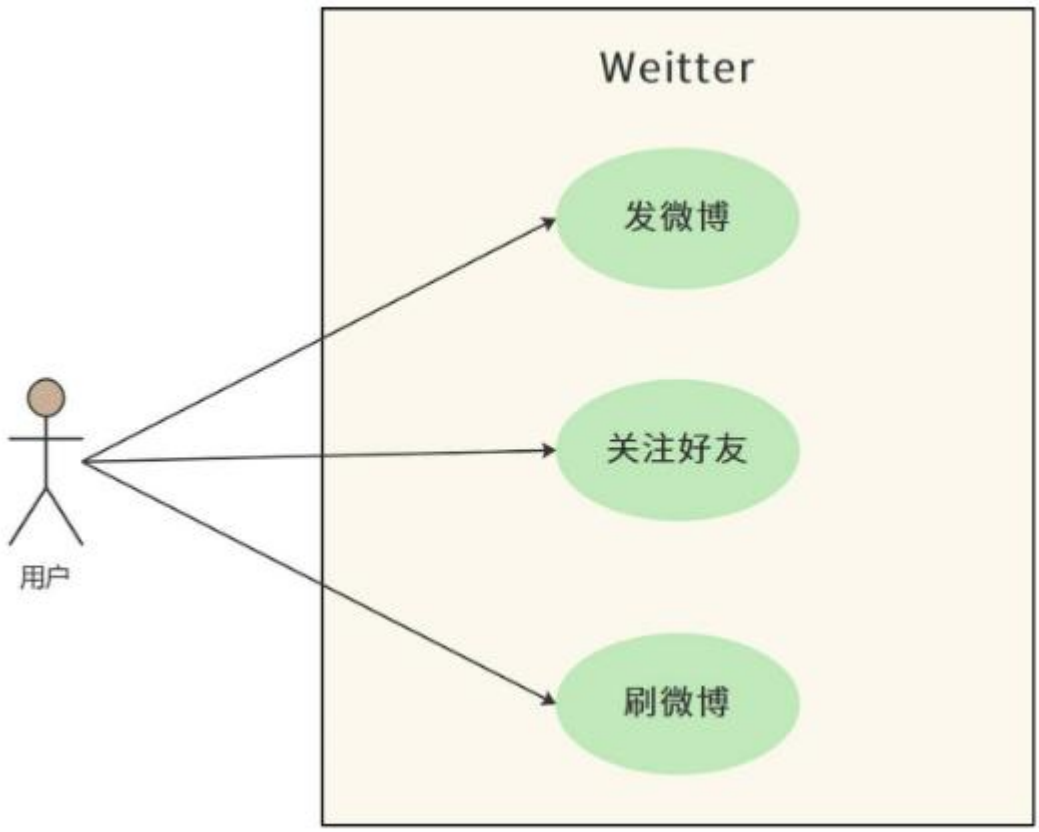
我们知道，微博有一个重要特点就是部分明星大 V 拥有大量的粉丝。如果明星们发布一条比较有话题性的个人花边新闻，比如宣布结婚或者离婚，就会引起粉丝们大量的转发和评论，进而引起更大规模的用户阅读和传播。

这种突发的单一热点事件导致的高并发访问会给系统带来极大的负载压力，处理不当甚至会导致系统崩溃。而这种崩溃又会成为事件热点的一部分，进而引来更多的围观和传播。

因此，Weitter 的技术挑战，一方面是微博这样类似的信息流系统架构是如何设计的，另一方面就是如何解决大 V 们的热点消息产生的突发高并发访问压力，保障系统的可用性。今天我们就来看看这样的系统架构该怎么设计。

13.1 需求分析

Weitter 的核心功能只有三个：发微博，关注好友，刷微博。



1. 发微博：用户可以发表微博，内容包含不超过 140 个字的文本，可以包含图片和视频。
2. 关注好友：用户可以关注其他用户。
3. 刷微博：用户打开自己的微博主页，主页显示用户关注的好友最近发表的微博；用户向下滑动页面（或者点刷新按钮），主页将更新关注好友的最新微博，且最新的微博显示在最上方；主页一次显示 20 条微博，当用户滑动到主页底部后，继续向上滑动，会按照时间顺序，显示当前页面后续的 20 条微博。
4. 此外，用户还可以收藏、转发、评论微博。

13.1. 性能指标估算

系统按 10 亿用户设计，按 20%日活估计，大约有 2 亿日活用户（DAU），其中每个日活用户每天发表一条微博，并且平均有 500 个关注者。

而对于**发微博所需的存储空间**，我们做如下估算。

- **文本内容存储空间**

遵循惯例，每条微博 140 个字，如果以 UTF8 编码存储汉字计算，则每条微博需要 \(\\small 140\\times3=420\)个字节的存储空间。除了汉字内容以外，每条微博还需要存储微博 ID、用户 ID、时间戳、经纬度等数据，按 80 个字节计算。那么每天新发表微博文本内容需要的存储空间为 100GB。

\(\\small 2 亿 \\times (420B +80B) = 100GB/天\)

- **多媒体文件存储空间**

除了 140 字文本内容，微博还可以包含图片和视频，按每 5 条微博包含一张图片，每 10 条微博包含一个视频估算，每张图片 500KB，每个视频 2MB，每天还需要 60TB 的多媒体文件存储空间。

\(\\small 2 亿\\div5\\times500KB+2 亿\\div10\\times2MB=60TB/天\)

对于**刷微博的访问并发量**，我们做如下估算。

- **QPS**

假设两亿日活用户每天浏览两次微博，每次向上滑动或者进入某个人的主页 10 次，每次显示 20 条微博，每天刷新微博次数 40 亿次，即 40 亿次微博查询接口调用，平均 QPS 大约 5 万。

\(\\small 40 亿\\div（24\\times60\\times60）=46296/秒\)

高峰期 QPS 按平均值 2 倍计算，所以系统需要满足 10 万 QPS。

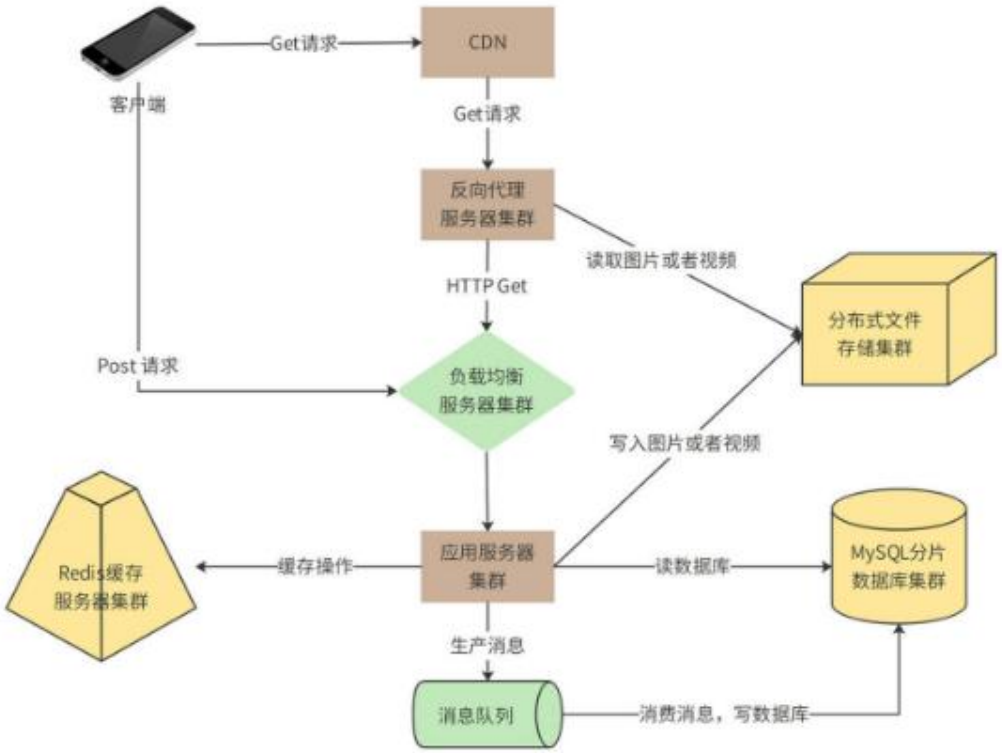
- **网络带宽**

10 万 QPS 刷新请求，每次返回微博 20 条，那么每秒需访问 200 万条微博。按此前估计，每 5 条微博包含一张图片，每 10 条微博包含一个视频，需要的**网络总带宽**为 4.8Tb/s。

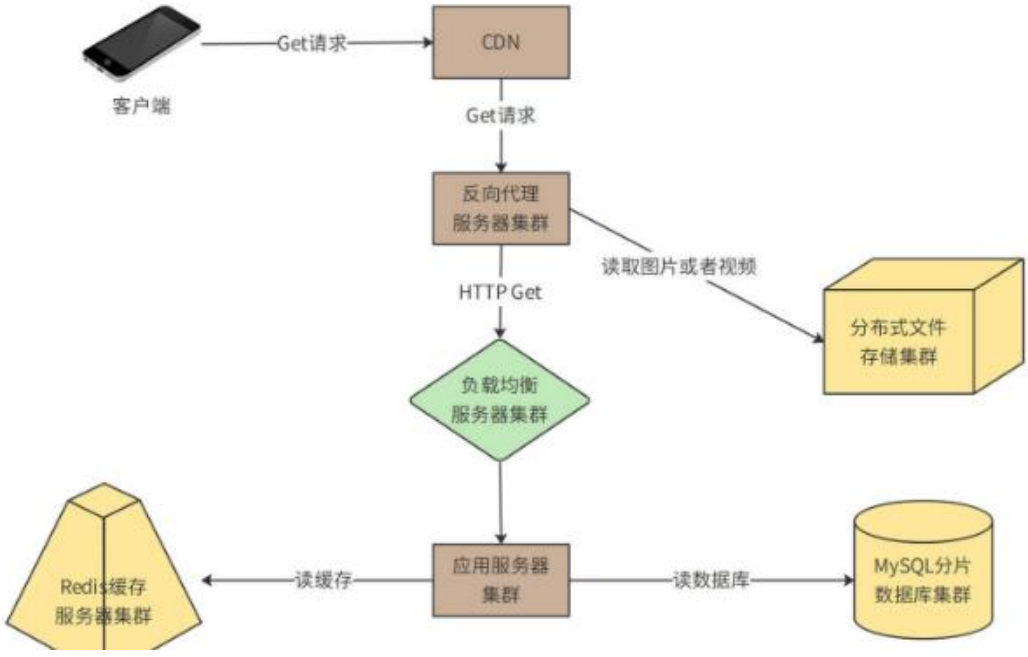
\(\\small （200 万\\div5\\times500KB+200 万\\div10\\times2MB）
\\times8bit=4.8Tb/s\)

13.2 概要设计

在需求分析中我们可以看到，Weitter 的业务逻辑比较简单，但是**并发量**和**数据量**都比较大，所以，**系统架构的核心就是解决高并发的问题**，系统整体部署模型如下。



这里包含了“Get 请求”和“Post 请求”两条链路，Get 请求主要处理刷微博的操作，Post 请求主要处理发微博的请求，这两种请求处理也有重合的部分，我们拆分着来看。我们先来看看 **Get 请求**的部分。

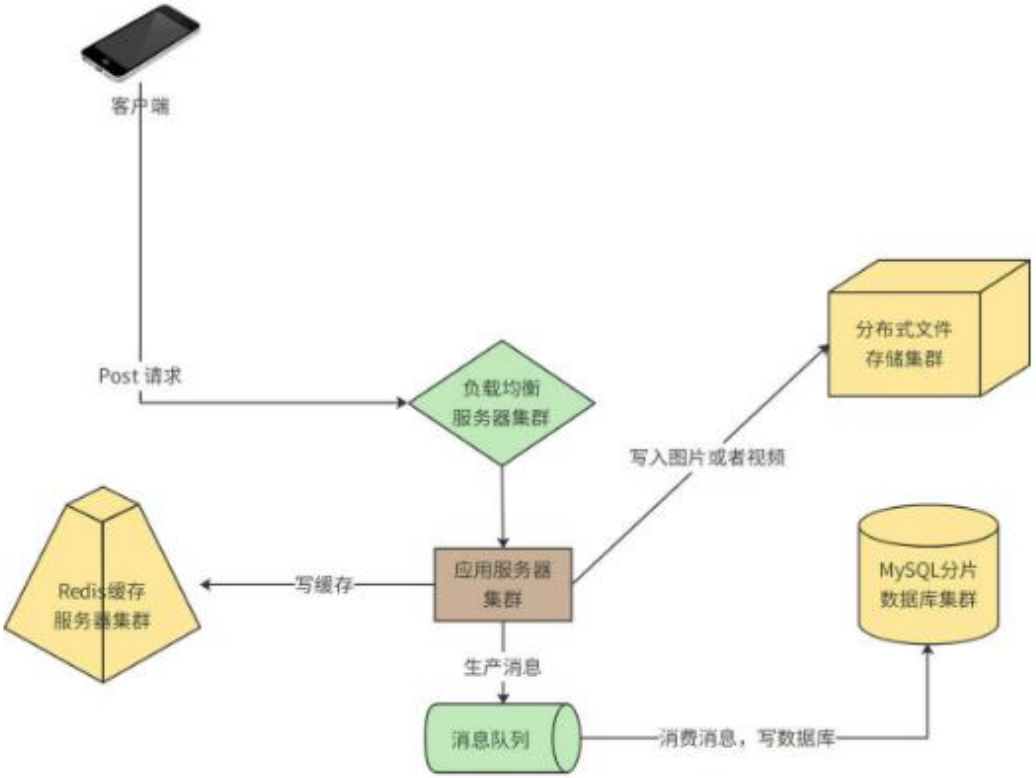


用户通过 CDN 访问 Weitter 的数据中心、图片以及视频等极耗带宽的请求，绝大部分可以被 CDN 缓存命中，也就是说，4.8Tb/s 的带宽压力，90%以上可以通过 CDN 消化掉。

没有被 CDN 命中的请求，一部分是图片和视频请求，其余主要是用户刷新微博请求、查看用户信息请求等，这些请求到达数据中心的反向代理服务器。反向代理服务器检查本地缓存是否有请求需要的内容。如果有，就直接返回；如果没有，对于图片和视频文件，会通过分布式文件存储集群获取相关内容并返回。分布式文件存储集群中的图片和视频是用户发表微博的时候，上传上来的。

对于用户微博内容等请求，如果反向代理服务器没有缓存，就会通过负载均衡服务器到达应用服务器处理。应用服务器首先会从 Redis 缓存服务器中，检索当前用户关注的好友发表的最新微博，并构建一个结果页面返回。如果 Redis 中缓存的微博数据量不足，构造不出一个结果页面需要的 20 条微博，应用服务器会继续从 MySQL 分片数据库中查找数据。

以上处理流程主要是针对读（http get）请求，那如果是发表微博这样的写（http post）请求呢？我们再来看一下**写请求**部分的图。



你会看到，客户端不需要通过 CDN 和反向代理，而是直接通过负载均衡服务器到达应用服务器。应用服务器一方面会将发表的微博写入 Redis 缓存集群，一方面写入分片数据库中。

在写入数据库的时候，如果直接写数据库，当有高并发的写请求突然到来，可能会导致数据库过载，进而引发系统崩溃。所以，数据库写操作，包括发表微博、关注好友、评论微博等，都写入到消息队列服务器，由消息队列的消费者程序从消息队列中按照一定的速度消费消息，并写入数据库中，保证数据库的负载压力不会突然增加。

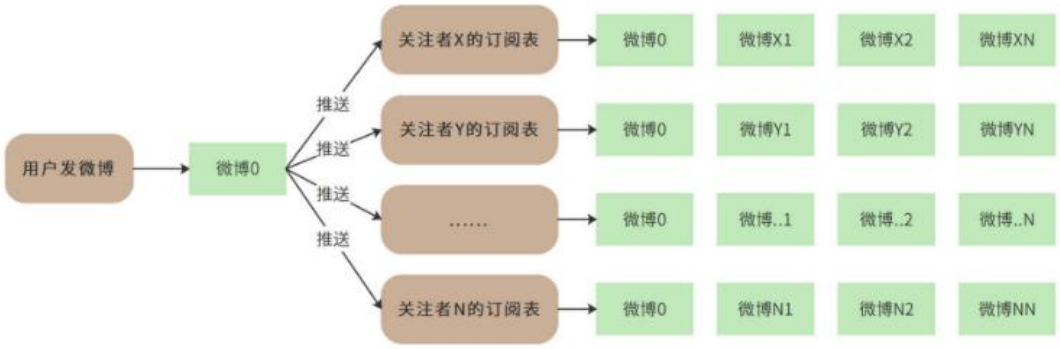
13.3 详细设计

用户刷新微博的时候，如何能快速得到自己关注的好友的最新微博列表？10 万 QPS 的并发量如何应对？如何避免数据库负载压力太大以及如何快速响应用户请求？详细设计将基于功能需求和概要设计，主要讨论这些问题。

13.3.1 微博的发表/订阅问题

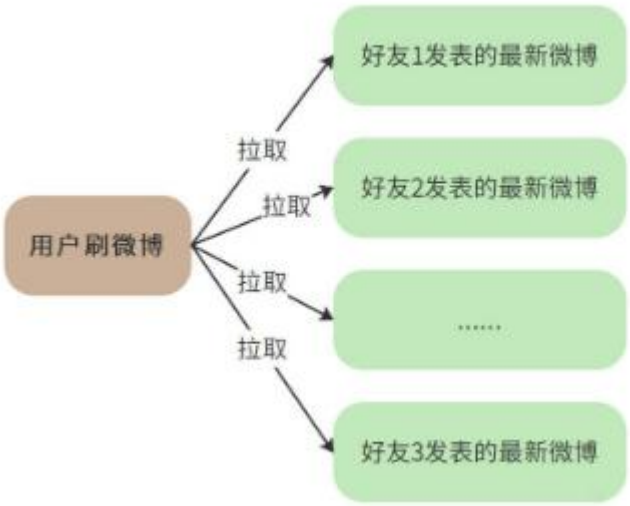
Weitter 用户关注好友后，如何快速得到所有好友的最新发表的微博内容，即发表/订阅问题，是微博的核心业务问题。

一种简单的办法就是“推模式”，即建一张用户订阅表，用户关注的好友发表微博后，立即在用户订阅中为该用户插入一条记录，记录用户 id 和好友发表的微博 id。这样当用户刷新微博的时候，只需要从用户订阅表中按用户 id 查询所有订阅的微博，然后按时间顺序构建一个列表即可。也就是说，**推模式是在用户发**微博**的时候推送给所有的关注者**，如下图，用户发表了微博 0，他的所有关注者的订阅表都插入微博 0。



推模式实现起来比较简单，但是推模式意味着，如果一个用户有大量的关注者，那么该用户每发表一条微博，就需要在订阅表中为每个关注者插入一条记录。而对于明星用户而言，可能会有几千万的关注者，明星用户发表一条微博，就会导致上千万次的数据库插入操作，直接导致系统崩溃。

所以，对于 10 亿级用户的微博系统而言，我们需要使用“拉模式”解决发表/订阅问题。也就是说，用户刷新微博的时候，根据其关注的好友列表，查询每个好友近期发表的微博，然后将所有微博按照时间顺序排序后构建一个列表。也就是说，**拉模式是在用户刷微博的时候拉取他关注的所有好友的最新微博**，如下图：



拉模式极大降低了发表微博时写入数据的负载压力，但是却又急剧增加了刷微博时候读数据库的压力。因为对于用户关注的每个好友，都需要进行一次数据库查询。如果一个用户关注了大量好友，查询压力也是非常巨大的。

所以，首先需要限制用户关注的好友数，在 Weitter 中，普通用户关注上限是 2000 人，VIP 用户关注上限是 5000 人。其次，需要尽量减少刷新时查询数据库的次数，也就是说，微博要尽量通过缓存读取。

但即使如此，你会发现每次刷新的查询压力还是太大，所以 Weitter 最终采用“推拉结合”的模式。也就是说，如果用户当前在线，那么就会使用推模式，系统会在缓存中为其创建一个好友最新发表微博列表，关注的好友如果有新发表微博，就立即将该微博插入列表的头部，当该用户刷新微博的时候，只需要将这个列表返回即可。

如果用户当前不在线，那么系统就会将该列表删除。当用户登录刷新的时候，用拉模式为其重新构建列表。

那么如何确定一个用户是否在线？一方面可以通过用户操作时间间隔来判断，另一方面也可以通过机器学习，预测用户的上线时间，利用系统空闲时间，提前为其构建最新微博列表。

13.3.2 缓存使用策略

通过前面的分析我们已经看到，Weitter 是一个典型的高并发读操作的场景。10 万 QPS 刷新请求，每个请求需要返回 20 条微博，如果全部到数据库中查询的话，数据库的 QPS 将达到 200 万，即使是使用分片的分布式数据库，这种压力也依然是无法承受的。所以，我们需要大量使用缓存以改善性能，提高吞吐能力。

但是缓存的空间是有限的，我们必定不能将所有数据都缓存起来。一般缓存使用的是 LRU 淘汰算法，即当缓存空间不足时，将最近最少使用的缓存数据删除，空出缓存空间存储新数据。

但是 LRU 算法并不适合微博的场景，因为在拉模式的情况下，当用户刷新微博的时候，我们需要确保其关注的好友最新发表的微博都能展示出来，如果其关注的某个好友较少有其他关注者，那么这个好友发表的微博就很可能会被 LRU 算法淘汰删除出缓存。对于这种情况，系统就不得不去数据库中进行查询。

而最关键的是，系统并不能知道哪些好友的数据通过读缓存就可以得到全部最新的微博，而哪些好友需要到数据库中查找。因此不得不全部到数据库中查找，这就失去了使用缓存的意义。

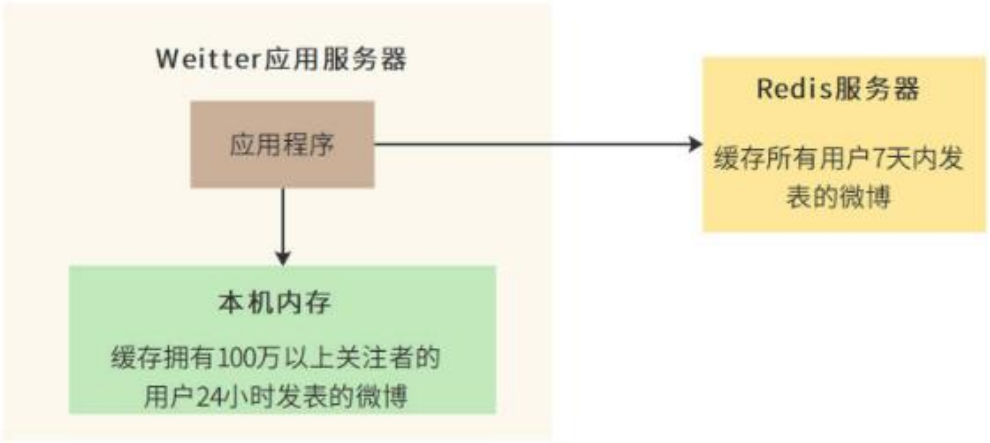
基于此，我们在 Weitter 中使用**时间淘汰算法**，**也就是将最近一定天数内发布的微博全部缓存起来，用户刷新微博的时候，只需要在缓存中进行查找。如果查找到的微博数满足一次返回的条数（20 条），就直接返回给用户；如果缓存中的微博数不足，就再到数据库中查找。

最终，Weitter 决定缓存 7 天内发表的全部微博，需要的缓存空间约 700G。缓存的 key 为用户 ID，value 为用户最近 7 天发表的微博 ID 列表。而微博 ID 和微博内容分别作为 key 和 value 也缓存起来。

此外，对于特别热门的微博内容，比如某个明星的离婚微博，这种针对单个微博内容的高并发访问，由于访问压力都集中一个缓存 key 上，会给单台 Redis 服务器造成极大的负载压力。因此，微博还会启用**本地缓存模式**，即应用服务器在内存中缓存特别热门的微博内容，应用构建微博刷新页的时候，会优先检查微博 ID 对应的微博内容是否在本地缓存中。

Weitter 最后确定的本地缓存策略是：针对拥有 100 万以上关注者的大 V 用户，缓存其 48 小时内发表的全部微博。

现在，我们来看一下 Weitter 整体的缓存架构。



13.3.3 数据库分片策略

前面我们分析过，Weitter 每天新增 2 亿条微博。也就是说，平均每秒钟需要写入 2400 条微博，高峰期每秒写入 4600 条微博。这样的写入压力，对于单机数据库而言是无法承受的。而且，每年新增 700 亿条微博记录，这也超出了单机数据库的存储能力。因此，Weitter 的数据库需要采用分片部署的分布式数据库。分片的规则可以采用用户 ID 分片或者微博 ID 分片。

如果按用户 ID（的 hash 值）分片，那么一个用户发表的全部微博都会保存到一台数据库服务器上。这样做的好处是，当系统需要按用户查找其发表的微博的时候，只需要访问一台服务器就可以完成。

但是这样做也有缺点，对于一个明星大 V 用户，其数据访问会成热点，进而导致这台服务器负载压力太大。同样地，如果某个用户频繁发表微博，也会导致这台服务器数据增长过快。

要是按微博 ID（的 hash 值）分片，虽然可以避免上述按用户 ID 分片的热点聚集问题，但是当查找一个用户的所有微博时，需要访问所有的分片数据库服务器才能得到所需的数据，对数据库服务器集群的整体压力太大。

综合考虑，用户 ID 分片带来的热点问题，可以通过优化缓存来改善；而某个用户频繁发表微博的问题，可以通过设置每天发表微博数上限（每个用户每天最多发表 50 条微博）来解决。最终，Weitter 采用按用户 ID 分片的策略。

14. 如何避免超预期的高并发压力压垮系统?

在互联网高可用架构设计中，限流是一种经典的高可用架构模式。因为某些原因，大量用户突然访问我们的系统时，或者有黑客恶意用 DoS（Denial of Service，拒绝服务）方式攻击我们的系统时，这种未曾预期的高并发访问对系统产生的负载压力可能会导致系统崩溃。

解决这种问题的一个主要手段就是限流，即拒绝部分访问请求，使访问负载压力降低到一个系统可以承受的程度。这样虽然有部分用户访问失败，但是整个系统依然是可用的，依然能对外提供服务，而不是因为负载压力太大而崩溃，导致所有用户都不能访问。

为此，我们准备开发一个限流器，产品名称为“Diana”。

14.1 需求分析

我们将 Diana 定位为一个限流器组件，即 Diana 的主要应用场景是部署在微服务网关或者其他 HTTP 服务器入口，以过滤器的方式对请求进行过滤，对超过限流规则的请求返回“服务不可用”HTTP 响应。

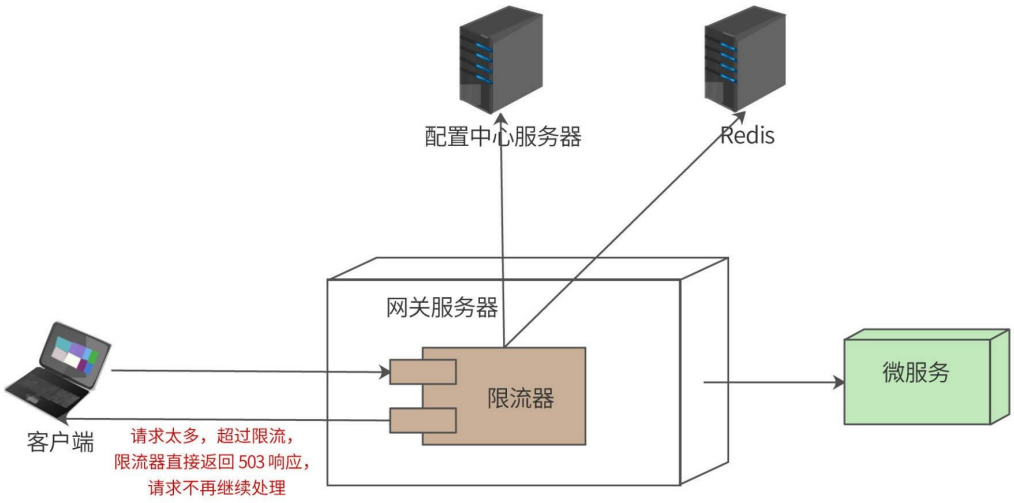
Diana 的限流规则可通过配置文件获取，并需要支持本地配置和远程配置两种方式，远程配置优先于本地配置。限流方式包括：

- 全局限流：针对所有请求进行限流，即保证整个系统处理的请求总数满足限流配置。
- 账号限流：针对账号进行限流，即对单个账号发送的请求进行限流。
- 设备限流：针对设备进行限流，即对单个客户端设备发送的请求进行限流。
- 资源限流：针对某个资源（即某个 URL）进行限流，即保证访问该资源的请求总数满足限流配置。

并且 Diana 设计应遵循开闭原则，能够支持灵活的限流规则功能扩展，即未来在不修改现有代码和兼容现有配置文件的情况下，支持新的配置规则。

14.2 概要设计

Diana 的设计目标是一个限流器组件，即 Diana 并不是一个独立的系统，不可以独立部署进行限流，而是部署在系统网关（或者其他 HTTP 服务器上），作为网关的一个组件进行限流，部署模型如下：



用户请求（通过负载均衡服务器）到达网关服务器。网关服务器本质也是一个 HTTP 服务器，限流器是部署在网关中的一个过滤器（filter）组件，和网关中的签名校验过滤器、用户权限过滤器等配置在同一个过滤器责任链（Chain of Responsibility）上。限流器应该配置在整个过滤器责任链的前端，也就是说，如果请求超过了限流，请求不需要再进入其他过滤器，直接被限流器拒绝。

用户请求进入限流器后，根据限流策略，判断该请求是否已经超过限流，如果超过，限流器直接返回状态码为 503（Too Many Requests）的响应；如果没有超过限流，请求继续向下处理（经过其他网关过滤器），并最终调用微服务完成处理。

限流器的策略可以在本地配置，也可以通过远程的配置中心服务器加载，即远程配置。远程配置优先于本地配置。

14.2.1 限流模式设计

请求是否超过限流，主要就是判断单位时间请求数量是否超过配置的请求限流数量。单位时间请求数量，可以本地记录，也可以远程记录。方便起见，本地记录称作本地限流，远程记录称作远程限流（也叫分布式限流）。

本地限流意味着，每个网关服务器需要根据本地记录的单位时间请求数量进行限流。假设限流配置为每秒限流 50 请求，如果该网关服务器本地记录的当前一秒内接受请求数量达到 50，那么这一秒内的后续请求都返回 503 响应。如果整个系统部署了 100 台网

关服务器，每个网关配置本地限流为每秒 50，那么，整个系统每秒最多可以处理 5000 个请求。

远程限流意味着，所有网关共享同一个限流数量，每个网关服务器收到请求后，从远程服务器中获取单位时间内已处理请求数，如果超过限流，就返回 503 响应。也就是说，可能某个网关服务器一段时间内根本就没有请求到达，但是远程的已处理请求数已经达到了限流上限，那么这台网关服务器也必须拒绝请求。我们使用 Redis 作为记录单位时间请求数量的远程服务器。

14.2.2 高可用设计

为了保证配置中心服务器和 Redis 服务器宕机时，限流器组件的高可用。限流器应具有自动降级功能，即配置中心不可用，则使用本地配置；Redis 服务器不可用，则降级为本地限流。

14.3 详细设计

常用的限流算法有 4 种，固定窗口（Window）限流算法，滑动窗口（Sliding Window）限流算法，漏桶（Leaky Bucket）限流算法，令牌桶（Token Bucket）限流算法。我们将详细讨论这四种算法的实现。

此外，限流器运行期需要通过配置文件获取对哪些 URL 路径进行限流；本地限流还是分布式限流；对用户限流还是对设备限流，还是对所有请求限流；限流的阈值是多少；阈值的时间单位是什么；具体使用哪种限流算法。因此，我们需要先看下配置文件的设计。

14.3.1 配置文件设计

Diana 限流器使用 YAML 进行配置，配置文件举例如下：

```
Url:/
rules:
  - actor:device
    unit:second
    rpu:10
    algo:TB
    scope:global- actor:all
```

```
unit:second
rpu:50
algo:W
scope:local
```

配置文件的配置项有 7 种，分别说明如下：

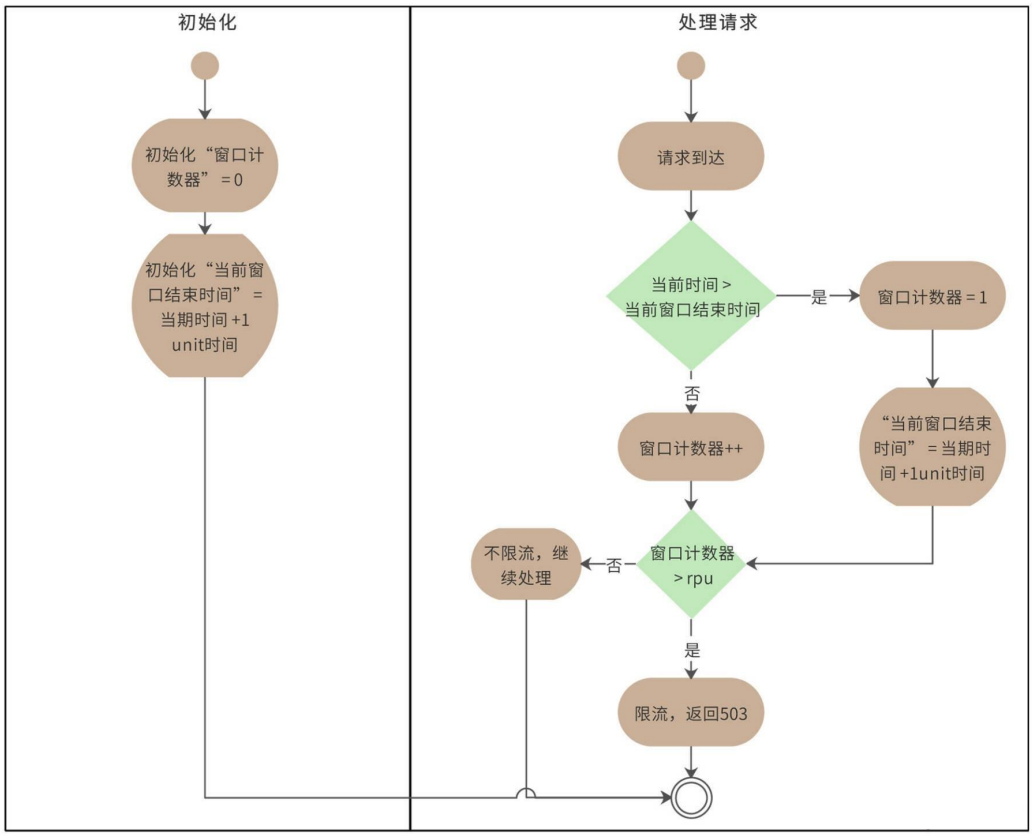
1. Url 记录限流的资源地址，”/“表示所有请求，配置文件中的路径可以互相包含，比如“/”包含“/sample”，限流器要先匹配“/”的限流规则，如果“/”的限流规则还没有触发（即访问”/“的流量，也就是单位时间所有的请求总和没有达到限流规则），则再匹配“/sample”。
2. 每个 Url 可以配置多个规则 rules，每个规则包括 actor，unit，rpu，algo，scope
3. actor 为限流对象，可以是账号（actor），设备（device），全部（all）
4. unit 为限流时间单位，可以是秒（second），分（minute），时（hour），天（day）
5. rpu 为单位时间限流请求数（request per unit），即上面 unit 定义的单位时间内允许通过的请求数目，如 unit 为 second，rpu 为 100，表示每秒允许通过 100 个请求，每秒超过 100 个请求就进行限流，返回 503 响应
6. scope 为 rpu 生效范围，可以是本地（local），也可以是全局（global），scope 也决定了单位时间请求数量是记录在本地还是远程，local 记录在本地，global 记录在远程。
7. algo 限流算法，可以是 window，sliding window，leaky bucket，token bucket 。

Diana 支持配置 4 种限流算法，使用者可以根据自己的需求场景，为不同资源地址配置不同的限流算法，下面详细描述这四种算法实现。

14.3.2 固定窗口（Window）限流算法

固定窗口限流算法就是将配置文件中的时间单位 unit 作为一个时间窗口，每个窗口仅允许限制流量内的请求通过，如图。

我们将时间轴切分成一个一个的限流窗口，每个限流窗口有一个窗口开始时间和一个窗口结束时间，窗口开始时，计数器清零，每进入一个请求，计数器就记录+1。如果请求数目超过 rpu 配置的限流请求数，就拒绝服务，返回 503 响应。当前限流窗口结束后，就进入下个限流窗口，计数器再次清零，重新开始。处理流程活动图如下。



上图包括“初始化”和“处理流程”两个泳道。初始化的时候，设置“窗口计数器”和“当前窗口结束时间”两个变量。处理请求的时候，判断当前时间是否大于“当前窗口结束时间”，如果大于，那么重置“窗口计数器”和“当前窗口结束时间”两个变量；如果没有，窗口计数器+1，并判断计数器是否大于配置的限流请求数 rpu，根据结果决定是否进行限流。

这里的“窗口计数器”可以本地记录，也可以远程记录，也就是配置中的 local 和 global。固定窗口算法在配置文件中 algo 项可配置“window”或者缩写“W”。

固定窗口实现比较容易，但是如果使用这种限流算法，在一个限流时间单位内，通过的请求数可能是 rpu 的两倍，无法达到限流的目的，如下图。

假设单位时间请求限流数 rpu 为 100，在第一个限流窗口快要到结束时间的时候，突然进来 100 个请求，因为这个请求量在限流范围内，所以没有触发限流，请求全部通过。然后进入第二个限流窗口，限流计数器清零。这时又忽然进入 100 个请求，因为已经进入第二个限流窗口，所以也没触发限流。在短时间内，通过了 200 个请求，这样可能会给系统造成巨大的负载压力。

14.3.3 滑动窗口（Sliding Window）限流算法

改进固定窗口缺陷的方法是采用滑动窗口限流算法，如下图。

滑动窗口就是将限流窗口内部切分成一些更小的时间片，然后在时间轴上滑动，每次滑动，滑过一个小时间片，就形成一个新的限流窗口，即滑动窗口。然后在这个滑动窗口内执行固定窗口算法即可。

滑动窗口可以避免固定窗口出现的放过两倍请求的问题，因为一个短时间内出现的所有请求必然在一个滑动窗口内，所以一定会被滑动窗口限流。

滑动窗口的算法实现基本和固定窗口一致，只要改动重置“窗口计数器”和“当前窗口结束时间”的逻辑就可以。固定窗口算法重置为窗口结束时间+1 unit 时间，滑动窗口算法重置为窗口结束时间+1 个时间片。但是固定窗口算法重置后，窗口计数器为 0，而滑动窗口需要将窗口计数器设置为当前窗口已经经过的时间片的请求总数，比如上图

里，一个滑动窗口被分为 5 个时间片，滑动窗口 2 的浅蓝色部分就是已经经过了 4 个时间片。

滑动窗口算法在配置文件中 algo 项可配置“sliding window”或者缩写“SW”。

14.3.4 漏桶（Leaky Bucket）限流算法

漏桶限流算法是模拟水流过一个有漏洞的桶进而限流的思路，如图。

水龙头的水先流入漏桶，再通过漏桶底部的孔流出。如果流入的水量太大，底部的孔来不及流出，就会导致水桶太满溢出去。

限流器利用漏桶的这个原理设计漏桶限流算法，用户请求先流入到一个特定大小的漏桶中，系统以特定的速率从漏桶中获取请求并处理。如果用户请求超过限流，就会导致漏桶被请求数据填满，请求溢出，返回 503 响应。

所以漏桶算法不仅可以限流，当流量超过限制的时候会拒绝处理，直接返回 503 响应，还能控制请求的处理速度。

实践中，可以采用队列当做漏桶。如图。

构建一个特定长度的队列 queue 作为漏桶，开始的时候，队列为空，用户请求到达后从队列尾部写入队列，而应用程序从队列头部以特定速率读取请求。当读取速度低于写入速度的时候，一段时间后，队列会被写满，这时候写入队列操作失败。写入失败的请求直接构造 503 响应返回。

但是使用队列这种方式，实际上是把请求处理异步化了（写入请求的线程和获取请求的线程不是同一个线程），并不适合我们目前同步网关的场景（如果使用前面设计过的 Flower 框架开发的异步网关就可以用这种队列方式）。

因此 Diana 实现漏桶限流算法并不使用消息队列，而是阻塞等待。根据限流配置文件计算每个请求之间的间隔时间，例如：限流每秒 10 个请求，那么每两个请求的间隔时间就必须>=100ms。用户请求到达限流器后，根据当前最近一个请求处理的时间和阻塞的请求线程数目，计算当前请求线程的 sleep 时间。每个请求线程的 sleep 时间不同，最后就可以实现每隔 100ms 唤醒一个请求线程去处理，从而达到漏桶限流的效果。

计算请求线程 sleep 时间的伪代码如下：

```
初始化 ：
间隔时间 = 100ms;
阻塞线程数 = 0;
最近请求处理时间戳 = 0;

long sleep 时间(){
  //最近没有请求，不阻塞 if((now - 最近请求处理时间戳) >= 间隔时间 and 阻塞线程数 <= 0) {
    最近请求处理时间戳 = now;
    return0; //不阻塞
  }
  //排队请求太多，漏桶溢出 if(阻塞线程数 > 最大溢出线程数) {
    return MAX_TIME;//MAX_TIME 表示阻塞时间无穷大，当前请求被限流
  }
  //请求在排队，阻塞等待
    阻塞线程数++;
    return 间隔时间 * 阻塞线程数 - (now - 最近请求处理时间戳) ;
}
```

请求线程 sleep 时间结束，继续执行的时候，修改阻塞线程数：

```
最近请求处理时间戳 = now;
阻塞线程数--;
```

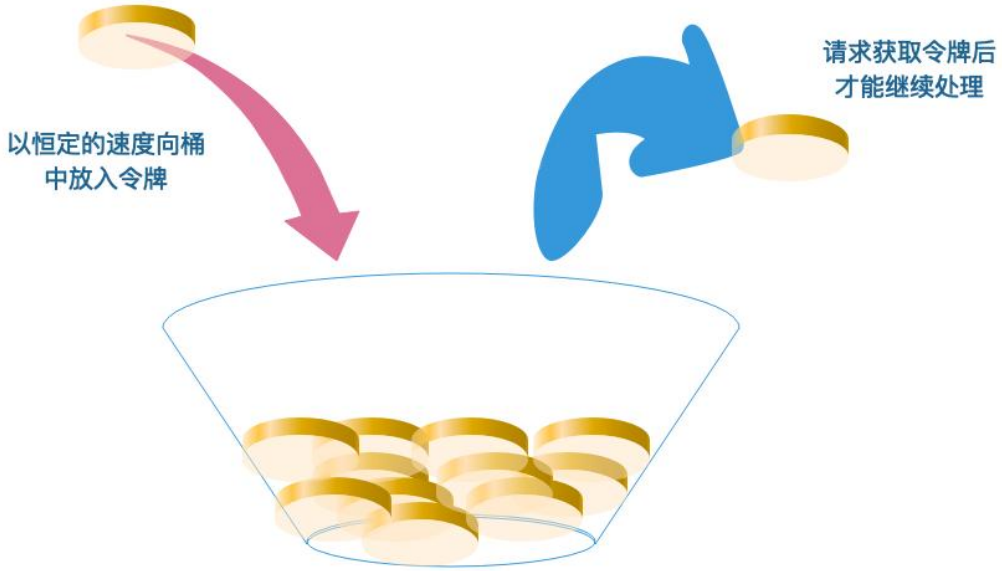
注意，以上代码多线程并发执行，需要进行加锁操作。

使用漏桶限流算法，即使系统资源很空闲，多个请求同时到达时，漏桶也是慢慢地一个接一个地去处理请求，这其实并不符合人们的期望，因为这样就是在浪费计算资源。因此除非有特别的场景需求，否则不推荐使用该算法。

漏桶算法的 algo 配置项名称为“leaky bucket”或者“LB”。

令牌桶（Token Bucket）限流算法

令牌桶是另一种桶限流算法，模拟一个特定大小的桶，然后向桶中以特定的速度放入令牌（token），请求到达后，必须从桶中取出一个令牌才能继续处理。如果桶中已经没有令牌了，那么当前请求就被限流，返回 503 响应。如果桶中的令牌放满了，令牌桶也会溢出。



上面的算法描述似乎需要有一个专门线程生成令牌，还需要一个数据结构模拟桶。实际上，令牌桶的实现，只需要在请求获取令牌的时候，通过时间计算，就可以算出令牌桶中的总令牌数。伪代码如下：

```
初始化 ：
最近生成令牌时间戳 = 0;
总令牌数 = 0;
令牌生成时间间隔 = 100ms;

boolean 获取令牌(){
  //令牌桶中有令牌，直接取令牌即可 if(总令牌数 >= 1){
    总令牌数--;
```

```
        returntrue;
    }
    //令牌桶中没有令牌了，重算现在令牌桶中的总令牌数，可能算出的总令牌数依然为 0
    总令牌数 = min(令牌数上限值，总令牌数 +
    (now - 最近生成令牌时间戳) / 令牌生成时间间隔);
    if(总令牌数 >= 1){
        总令牌数--;
        最近生成令牌时间戳 = now; //有令牌了，才能重设时间 returntrue;
    }
    returnfalse;
}
```

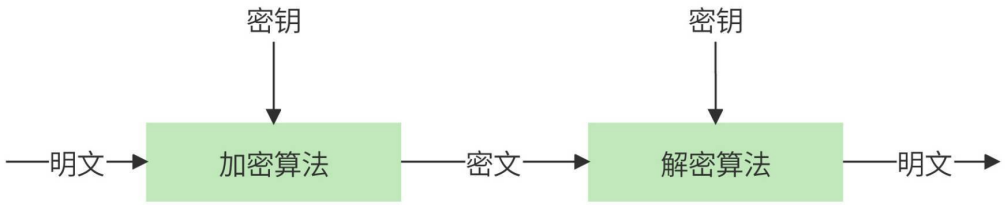
令牌桶限流算法综合效果比较好，能在最大程度利用系统资源处理请求的基础上，实现限流的目标，建议通常场景中优先使用该算法，Diana 的缺省配置算法也是令牌桶。令牌桶算法的 algo 配置项名称为“token bucket”或“TB”。

15. 设计一个支持敏感数据存储和传输安全的加解密平台

在一个应用系统运行过程中，需要记录、传输很多数据，这些数据有的是非常敏感的，比如用户姓名、手机号码、密码、甚至信用卡号等等。这些数据如果直接存储在数据库，记录在日志中，或者在公网上传输的话，一旦发生数据泄露，不但可能会产生重大的经济损失，还可能会使公司陷入重大的公关与法律危机。公司上下辛苦十几年，一夜回到解放前。

所以，敏感信息必须进行加密处理，也就是把敏感数据以密文的形式存储、传输。这样即使被黑客攻击，发生数据泄露，被窃取的数据也是密文，获取数据的人无法得到真实的明文内容，敏感数据依然被保护着。而当应用程序需要访问这些密文的时候，只需要进行数据解密，即可还原得到原始明文数据。加解密处理既保证了数据的安全，又保证了数据的正常访问。

但是，这一切的前提是**加密和解密过程的安全**。加密、解密过程由加密算法、加密密钥、解密算法、解密密钥组成。下图是一个对称加密、解密过程。对称加密密钥和解密密钥是同一个密钥，调用加密算法可将明文加密为密文，调用解密算法可将密文还原为明文。



所以，如果窃取数据的人知道了解密算法和密钥，即使数据是加密的，也可以轻松对密文进行还原，得到原始的明文数据。而很多时候，解密算法和密钥都以源代码的方式保存在代码仓库里，黑客如果窃取了源代码，或者内部人泄露了源代码，那么所有的秘密就都不是秘密了。

此外，在某些情况下，我们的系统需要和外部系统进行对称加密数据传输，比如和银行加密传输信用卡卡号，这时候涉及到密钥交换，即我方人员和银行人员对接，直接传递密钥。如果因密钥泄露导致重大经济损失，那么持有密钥的人员将无法自证清白，这又会导致没有人愿意保管密钥。

因此，我们设计了一个加解密服务系统，系统名称为“Venus”，统一管理所有的加解密算法和密钥。应用程序只需要依赖加解密服务 SDK，调用接口进行加解密即可，而真正的算法和密钥在系统服务端进行管理，保证算法和密钥的安全。

15.1 需求分析

一般说来，日常开发中的加解密程序存在如下问题：

1. 密钥（包括非对称加解密证书）保存在源文件或者配置文件中，存储分散而不安全。
2. 密钥没有分片交换机制，不能满足高安全级密钥管理和交换的要求。
3. 密钥缺乏版本管理，不能灵活升级，一旦修改密钥，此前加密的数据就可能无法解密。
4. 加密解密算法程序不统一，同样算法不同实现，内部系统之间密文不能正确解析。
5. 部分加解密算法程序使用了弱加解密算法和弱密钥，存在安全隐患。

为此，我们需要设计开发一个专门的加解密服务及密钥管理系统，以解决以上问题。

Venus 是一个加解密服务系统，核心功能是加解密服务，辅助功能是密钥与算法管理。

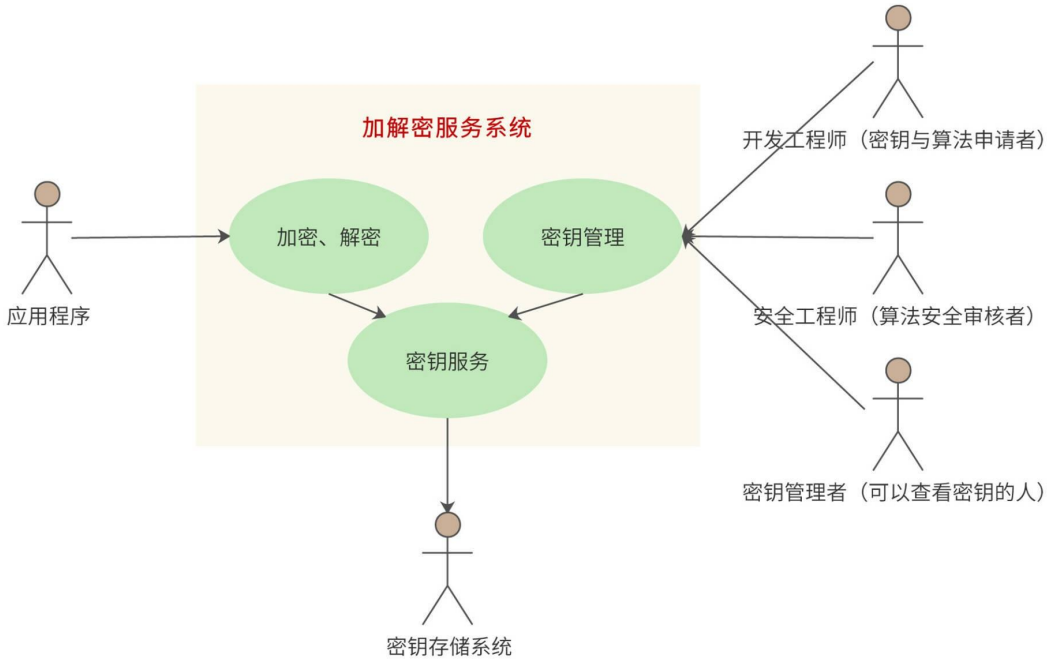
此外，Venus 还需要满足以下非功能需求：

- **安全性需求**- 必须保证密钥的安全性，保证没有人能够有机会看到完整的密钥。因此一个密钥至少要拆分成两片，分别存储在两个异构的、物理隔离的存储服务器

中 。在需要进行密钥交换的场景中，将密钥至少拆分成两个片段，每个管理密钥的人只能看到一个密钥片段，需要双方所有人分别交接才能完成一次密钥交换。

- **可靠性需求**- 加解密服务必须可靠，即保证高可用。无论在加解密服务系统服务器宕机、还是网络中断等各种情况下，数据正常加解密都需要得到保障。
- **性能需求**- 加解密计算的时间延迟主要花费在加解密算法上，也就是说，加载加解密算法程序、获取加解密密钥的时间必须短到可以忽略不计。

根据以上加解密服务系统功能和非功能需求，系统用例图设计如下：



系统主要参与者（Actor）包括：

应用程序	使用加密、解密服务的程序
开发工程师	应用程序开发者
安全工程师	审核算法和密钥安全的运维工程师
密钥管理者	和第三方系统进行密钥交换的时候，可以直接获得密钥内容的人员
密钥存储系统	存储密钥分片的文件系统、数据库等

系统主要用例过程和功能包括：

6. 开发工程师使用密钥管理功能为自己开发的应用申请加解密算法和密钥；

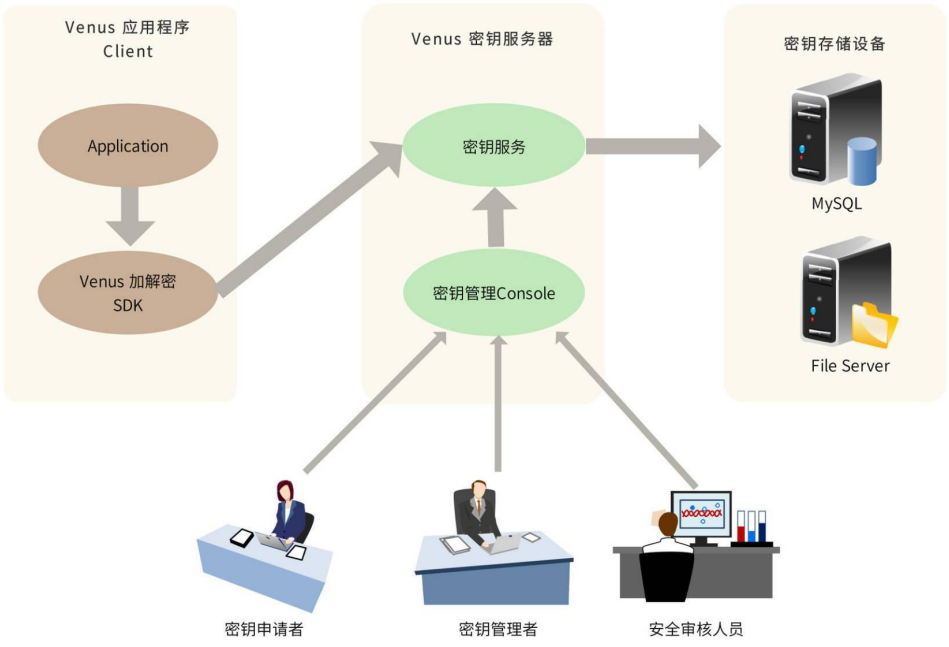
7. 安全工程师使用密钥管理功能审核算法和密钥的强度是否满足数据安全要求；
8. （经过授权的）密钥管理者使用密钥管理功能可以查看密钥（的一个分片）；
9. 应用程序调用加解密功能完成数据的加密、解密；
10. 加密解密功能和密钥管理功能调用密钥服务功能完成密钥的存储和读取；
11. 密钥服务功能访问一个安全、可靠的密钥存储系统读写密钥。

总地说来，Venus 应满足如下需求：

12. 集中、分片密钥存储与管理，多存储备份，保证密钥安全易管理。
13. 密钥申请者、密钥管理者、密钥访问者，多角色多权限管理，保证密钥管理与传递的安全。
14. 通过密钥管理控制台完成密钥申请、密钥管理、密钥访问控制等一系列密钥管理操作，实现便捷的密钥管理。
15. 统一加解密服务 API，简单接口，统一算法，为内部系统提供一致的加解密算法实现。

15.2 概要设计

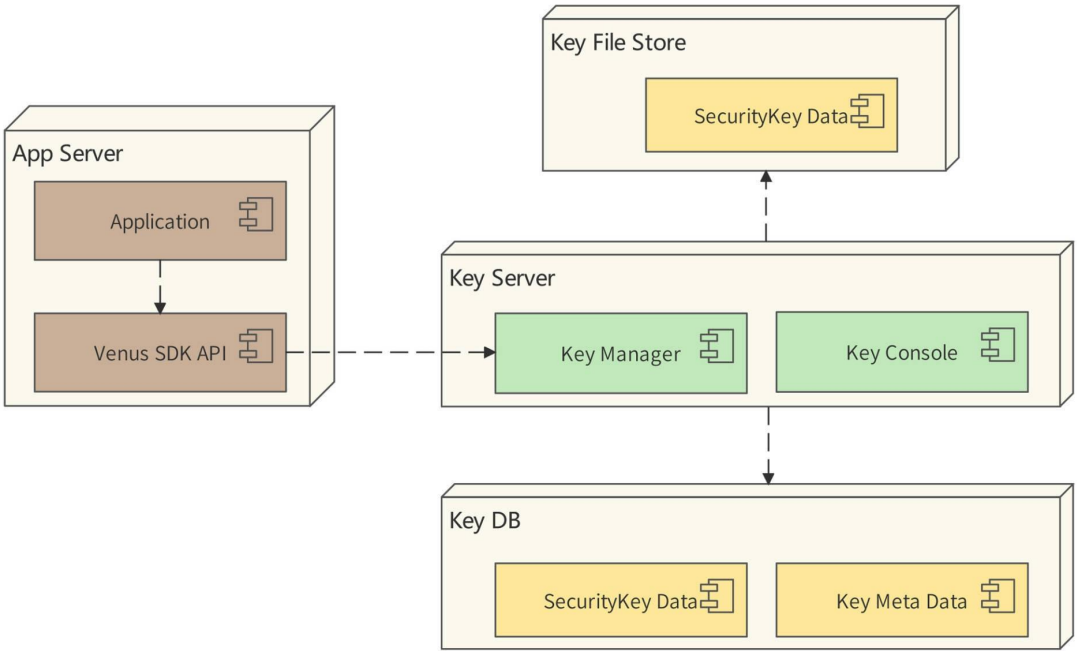
针对上述加解密服务及密钥安全管理的需求，设计加解密服务系统 Venus 整体结构如下：



应用程序调用 Venus 提供的加解密 SDK 服务接口，对信息进行加解密，该 SDK 接口提供了常用的加密解密算法并可根据需求任意扩展。SDK 加解密服务接口调用 Venus 密钥服务器的密钥服务，以取得加解密密钥，并缓存在本地。而密钥服务器中的密钥则来自多个密钥存储服务器，一个密钥分片后存储在多个存储服务器中，每个服务器都由不同的人负责管理。密钥申请者、密钥管理者、安全审核人员通过密钥管理控制台管理更新密钥，每个人各司其事，没有人能查看完整的密钥信息。

15.2.1 部署模型

Venus 部署模型如图：



Venus 系统的核心服务器是 Key Server 服务器，提供密钥管理服务。密钥分片存储在文件服务器 File Store 和数据库 DB 中。

使用 Venus 加解密服务的应用程序（Application）部署在应用程序服务器（App Server）中，依赖 Venus 提供的 SDK API 进行数据加解密。而 Venus SDK 则是访问密钥服务器（Key Server）来获取加解密算法代码和密钥。

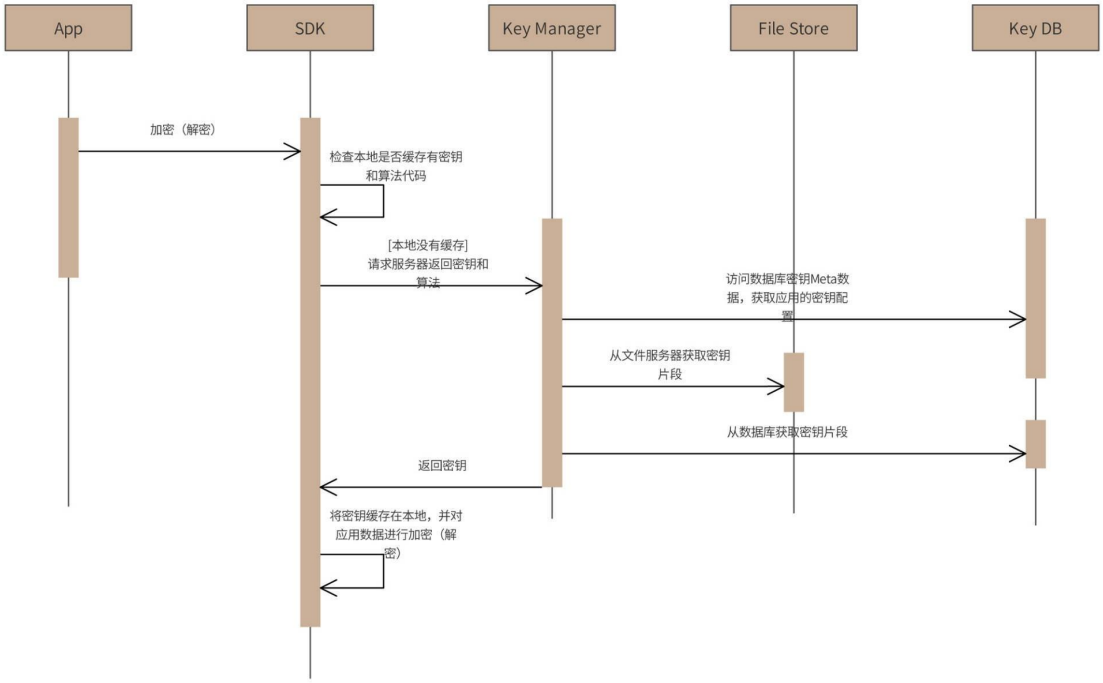
安全起见，密钥将被分片存储在文件服务器（Key File Store）和数据库服务器（Key DB）中。所以 Key Server 服务器中部署了密钥管理组件（Key Manager），用于访问数据库中的应用程序密钥元信息（Key Meta Data），以此获取密钥分片存储信息。Key

Server 服务器根据这些信息访问 File Store 和 DB，获取密钥分片，并把分片拼接为完整密钥，最终返回给 SDK。

此外，密钥管理控制台（Key Console）提供一个 web 页面，供开发工程师、安全工程师、密钥管理者进行密钥申请、更新、审核、查看等操作。

15.2.2 加解密调用时序图

加解密调用过程如下时序图所示。



1. 应用程序 App 调用 Venus SDK 对数据进行加密（解密）。
2. SDK 检查在本地是否有缓存加解密需要的密钥和加解密算法代码，如果有缓存，就直接使用该算法和密钥进行加解密。
3. 如果本地没有缓存密钥和算法，请求远程服务器返回密钥和算法。
4. 部署在 Venus 服务器的 Key Manager 收到请求后，访问数据库，检查该应用配置的密钥和算法 Meta 信息。
5. 数据库返回的 Mata 信息中包括了密钥的分片信息和存储位置，Key Manager 访问文件服务器和数据库，获取密钥分片，并将多个分片合并成一个完整密钥，返回给客户端 SDK。
6. SDK 收到密钥后，缓存在本地进程内存中，并完成对 App 加解密调用的处理。

通过该设计，我们可以看到，Venus 对密钥进行分片存储，不同存储服务器由不同运维人员管理。就算需要进行密钥交换，那么参与交换的人员，每个人也只能获得一个密钥分片，无法得到完整的密钥，这样就保证了密钥的安全性。

密钥缓存在 SDK 所在的进程（也就是应用程序 App 所在的进程）中，只有第一次调用时会访问远程的 Venus 服务器，其他调用只访问本进程缓存。因此加解密的性能只受加解密的数据大小和算法的影响，不受 Venus 服务的性能影响，满足了性能要求。

同时，由于密钥在缓存中，如果 Venus 服务器临时宕机，或者网络通信中断，也不会影响到应用程序的正常使用，保证了 Venus 的可靠性。但是如果 Venus 服务器长时间宕机，那么应用重新启动，本地缓存被清空，就需要重新请求密钥，这时候应用就不可用了。那么 Venus 如何在这种情况下仍然保证高可用呢？

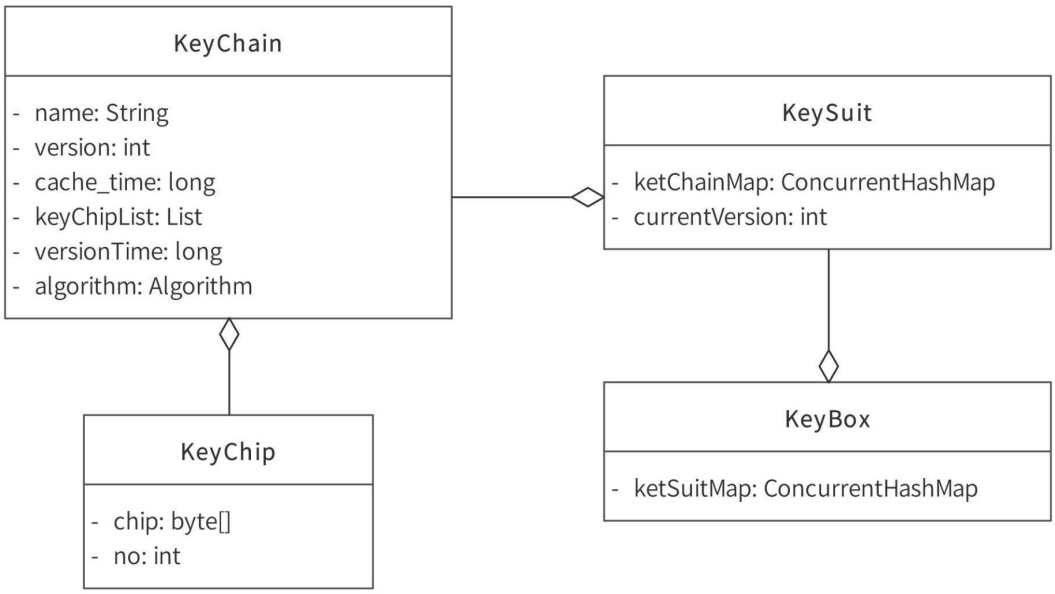
解决方案就是对 Venus 服务器、数据库和文件服务器做高可用备份。Venus 服务器部署 2-3 台服务器，构建一个小型集群，SDK 通过软负载均衡访问 Venus 服务器集群，若发现某台 Venus 服务器宕机，就进行失效转移。同样，数据库和文件服务器也需要做主从备份。

15.3 详细设计

Venus 详细设计主要关注 SDK 核心类设计。其他的例如数据库结构设计、服务器密钥管理 Console 设计等，这里不做展开。

15.3.1 密钥领域模型

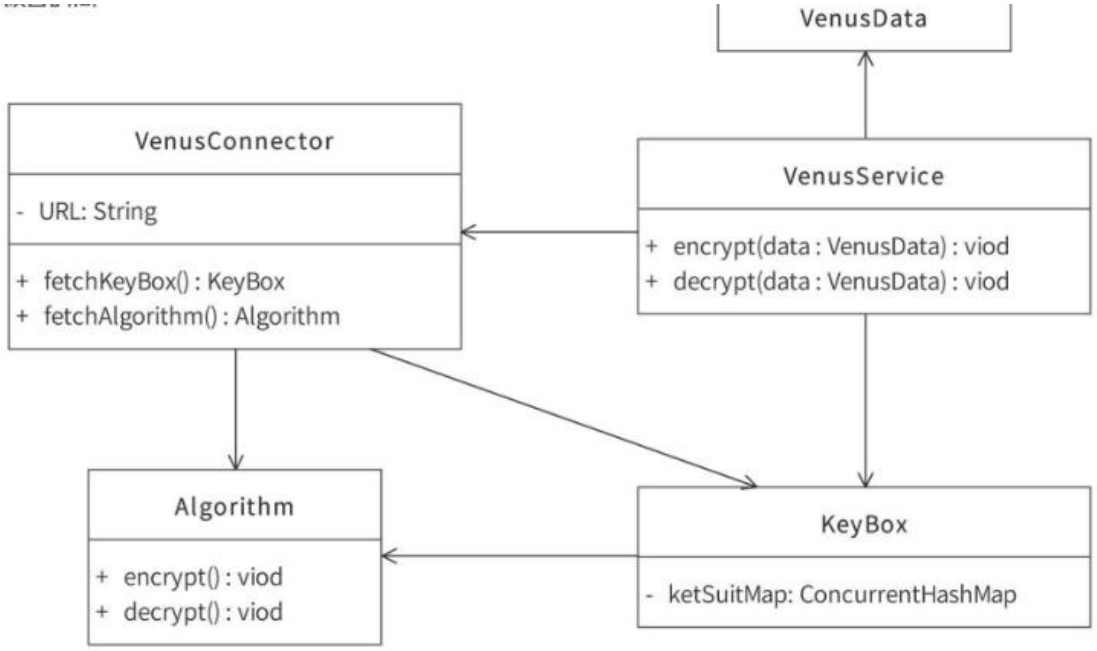
为了便于 SDK 缓存、管理密钥信息以及 SDK 与 Venus 服务端传输密钥信息，我们设计了一个密钥领域模型，如下图：



1. 一个应用程序使用的所有密钥信息都记录在 KeyBox 对象中，KeyBox 对象中有一个 keySuitMap 成员变量，这个 map 的 key 是密钥名称，value 是一个 KeySuit 对象。
2. KeySuit 类中有一个 keyChainMap 成员变量，这个 map 类的 key 是版本号，value 是一个 KeyChain 对象。Venus 因为安全性需求，需要支持多版本的密钥。也就是说，对同一类数据的加密密钥过一段时间就会进行版本升级，这样即使密钥泄露，也只会影响一段时间的数据，不会导致所有的数据都被解密。
3. KeySuit 类的另一个成员变量 currentVersion 记录当前最新的密钥版本号，也就是当前用来进行数据加密的密钥版本号。而解密的时候，则需要从密文数据中提取出加密密钥版本号（或者由应用程序自己记录密钥版本号，在解密的时候提供给 Venus SDK API），根据这个版本号获取对应的解密密钥。
4. 具体每个版本的密钥信息记录在 KeyChain 中，包含了密钥名称 name、密钥版本号 version、加入本地缓存的时间 cache_time、该版本密钥创建的时间 versionTime、对应的加解密算法 algorithm，当然，还有最重要的密钥分片列表 keyChipList，里面按序记录着这个密钥的分片信息。
5. KeyChip 记录每个密钥分片，包括分片编号 no，以及分片密钥内容 chip。

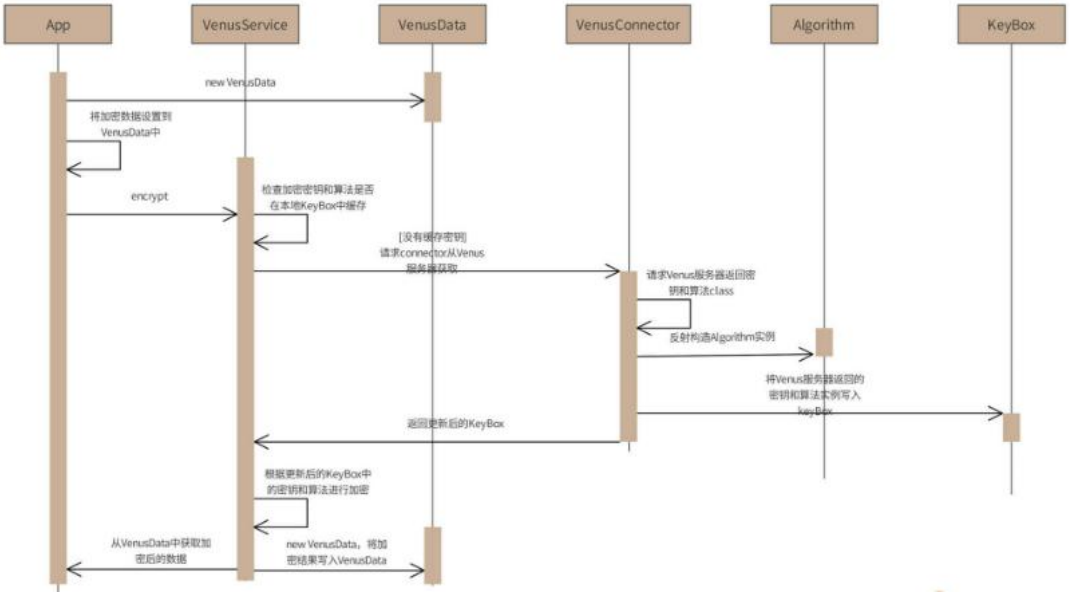
15.3.2 核心服务类设计

应用程序通过调用加解密 API VenusService 完成数据加解密。如下图：



1. Venus SDK 的核心类是 VenusService，应用程序调用该对象的 encrypt 方法进行加密，decrypt 方法进行解密。应用程序需要构造 VenusData 对象，将加解密数据传给 VenusService，VenusService 加解密完成后创建一个新的 VenusData 对象，将加解密的结果写入该对象并返回。VenusData 成员变量在后面详细讲解。
2. VenusService 通过 VenusConnector 类连接 Venus 服务器获取密钥 KeyBox 和算法 Algorithm，并调用 Algorithm 的对应方法完成加解密。

以加密为例，具体处理过程时序图如下：



首先，应用程序 App 创建 VenusData 对象，并将待加密数据写入该对象。接着，App 调用 VenusService 的 encrypt 方法进行加密，VenusService 检查加密需要的密钥和算法是否已经有缓存，如果没有，就调用 VenusConnector 请求服务器，返回密钥和算法。VenusConnector 将根据返回的算法字节码来构造加密算法的实例对象，同时根据返回的密钥构造相关密钥对象，并写入 KeyBox，完成更新。

下一步，VenusService 会根据更新后的 KeyBox 中的密钥和算法进行加密，并将加密结果写入 VenusData。最后，应用程序 App 从返回的 VenusData 中获取加密后的数据即可。

15.3.3 加解密数据接口 VenusData 设计

VenusData 用于表示 Venus 加解密操作输入和输出的数据，也就是说，加解密的时候构造 VenusData 对象调用 Service 对应的方法，加解密完成后返回值还是一个 VenusData 对象。

VenusData 包含的属性如下图：

keyName	加解密操作要使用的密钥名
version	加解密操作要使用的密钥版本
bytes	数据的byte数组格式
text	数据的String格式
outputWithText	VenusData输出时，是否输出String格式的数据，缺省是true
dataWithVersion	VenusData输出时，输出结果是否包含版本信息，缺省是false

VenusData 用作输入时：

1. 属性 bytes 和 text 只要设置一个，即要么处理的是二进制 bytes 数据，要么是 Striing 数据，如果两个都设置了，Venus 会抛出异常。
2. 属性 version 可以不设置（即 null），表示 Venus 操作使用的密钥版本是当前版本。
3. 属性 outputWithText 表示输出的 VenusData 是否处理为 text 类型，缺省值是 true。
4. 属性 dataWithVersion 表示加密后的 VenusData 的 bytes 和 text 中是否包含使用密钥的版本信息，这样在解密的时候可以不指定版本，缺省值是 false。

如果 dataWithVersion 设置为 true，即表示加密后密文内包含版本号，这种情况下，VenusService 需要在密文头部增加 3 个字节的版本号信息，其中头两个字节为固定的 magic code：0x5E、0x23，第三个字节为版本号（也就是说，密钥版本号只占用一个字节，最多支持 256 个版本）。

VenusData 用作输出时，Venus 会设置属性 keyName (和输入时的值一样)、version、bytes、 outputWithText、dataWithVersion（和输入时的值一样），并根据输入的 outputWithText 决定是否设置 text 属性。

15.3.4 测试用例代码 demo

```
1 publicstaticvoidtestVenusService()throws Exception {
2             // 准备数据 VenusDatadata1=newVenusData();
```

```
        data1.setKeyName("aeskey1");
        data1.setText("PlainText");
        // 加密操作 VenusDataencrypt= VenusService.encrypt(data1);
        System.out.printf("Key Name: %s, Secret Text: %s, Version: %d.\n",
encrypt.getKeyName(),
                encrypt.getText(), encrypt.getVersion());
        // 准备数据 VenusDatadata2=newVenusData();
        data2.setKeyName("aeskey1");
        data2.setBytes(encrypt.getBytes());
        data2.setVersion(encrypt.getVersion());
        // 解密操作 VenusDatadecrypt= VenusService.decrypt(data2);
        System.out.printf("Key Name: %s, Plain Text: %s, Version: %d.\n",
decrypt.getKeyName(),
                decrypt.getText(), decrypt.getVersion());
    }
```

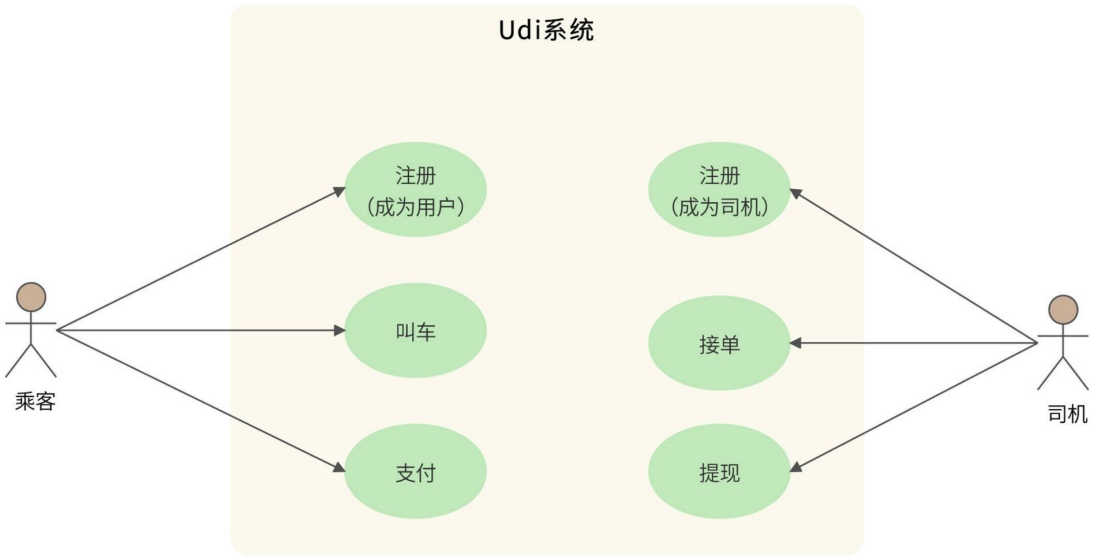
16. 如何设计一个支持 5 亿用户规模的网约车系统?

网约车的官方定义是：“以互联网技术为依托，构建服务平台，整合供需信息，使用符合条件的车辆和驾驶员，提供非巡游的预约出租汽车服务的经营活动。”通俗地说就是：利用互联网技术平台，将乘客的乘车信息发送给合适的司机，由司机完成接送乘客的服务。网约车包含专车、快车、拼车等多种形式。

中国目前网约车用户规模约 5 亿，我们准备开发一个可支撑目前全部中国用户使用的网约车平台，应用名称为“Udi”。

16.1 需求分析

Udi 是一个网约车平台，核心功能是将乘客的叫车订单发送给附近的网约车司机，司机接单后，到上车点接乘客并送往目的地，到达后，乘客支付订单。根据平台的分成比例，司机提取一部分金额作为收益，用例图如下：

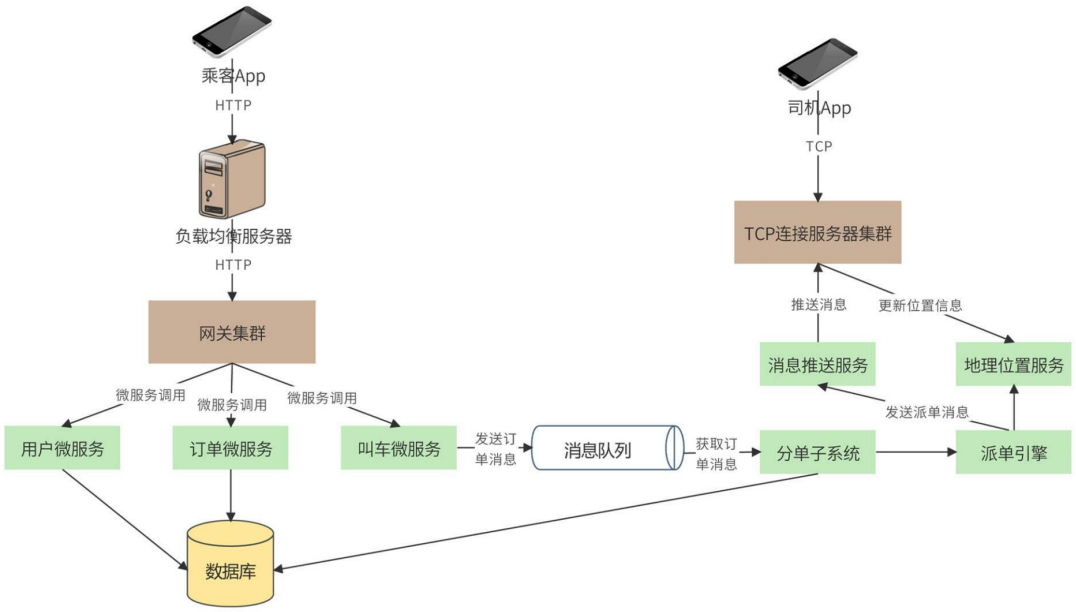


Udi 平台预计注册乘客 5 亿，日活用户 5 千万，平均每个乘客 1.2 个订单，日订单量 6 千万。平均客单价 30 元，平台每日总营收 18 亿元。平台和司机按 3：7 的比例进行分成，那么平台每天可赚 5.4 亿元。

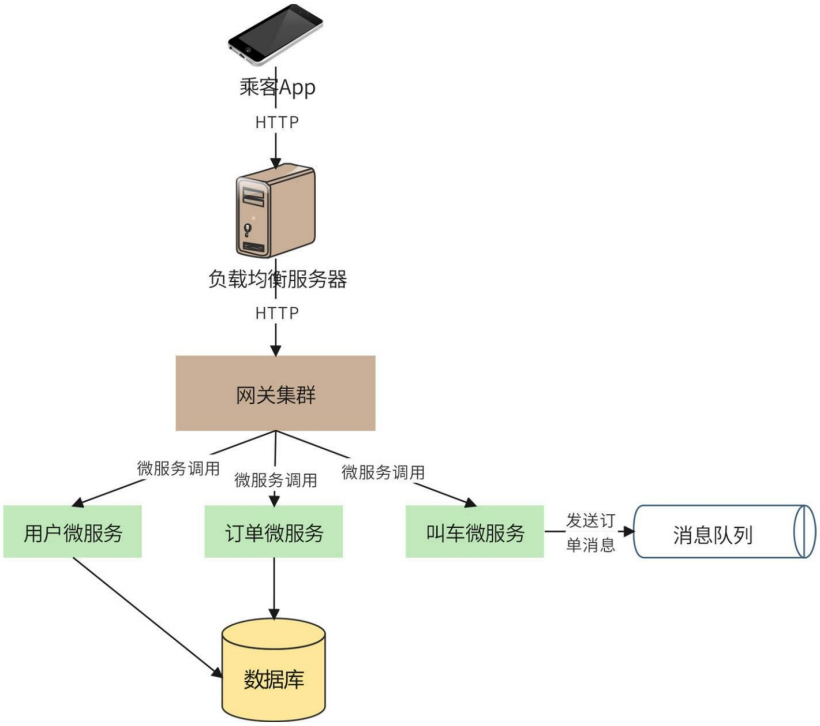
另外，平台预计注册司机 5 千万，日活司机 2 千万。

16.2 概要设计

网约车平台是共享经济的一种，目的就是要将乘客和司机撮合起来，所以需要开发两个 App 应用，一个是给乘客的，用来叫车；一个是给司机的，用来接单。Udi 整体架构如下图：

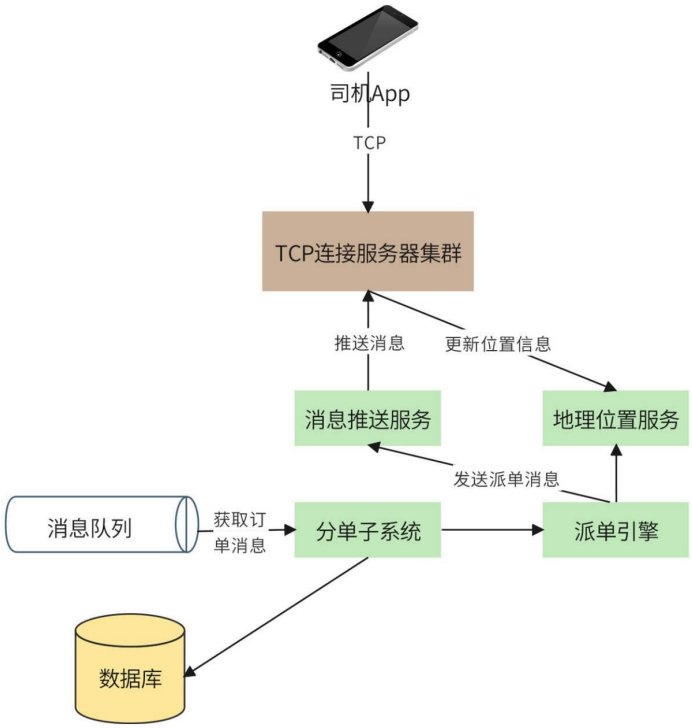


相应的，Udi 的系统也可以分成两个部分，一个部分是面向乘客的。乘客通过手机 App 注册成为用户，然后就可以在手机上选择出发地和目的地，进行叫车了。乘客叫车的 HTTP 请求首先通过一个负载均衡服务器集群，到达网关集群，再由网关集群调用相关的微服务，完成请求处理，如下图：



网关处理叫车请求的过程是：网关首先调用订单微服务，为用户的叫车请求创建一个订单，订单微服务将订单记录到数据库中，并将订单状态设置为“创建”。然后网关调用叫车微服务，叫车微服务将用户信息、出发地、目的地等数据封装成一个消息，发送到消息队列，等待系统为订单分配司机。

Udi 系统的另一部分是面向司机的，司机需要不停将自己的位置信息发送给平台，同时，还需要随时接收来自平台的指令。因此，不同于用户通过 HTTP 发送请求给平台，司机 App 需要通过 TCP 长连接和平台服务器保持通信，如下图：



Udi 司机 App 每 3 秒向平台发送一次当前的位置信息，包括当前车辆经纬度，车头朝向等。位置信息通过 TCP 连接到达平台的 TCP 连接服务器集群，TCP 连接服务器集群的作用类似网关，只不过是以 TCP 长连接的方式向 App 端提供接入服务。TCP 连接服务器将司机的位置信息更新到地理位置服务。

对于前面已经写入到消息队列的乘客叫车订单信息，分单子系统作为消息消费者，从消息队列中获取并处理。分单子系统首先将数据库中的订单状态修改为“派单中”，然后调用派单引擎进行派单。派单引擎根据用户的上车出发地点，以及司机上传的地理位置信息进行匹配，选择最合适的司机进行派单。派单消息通过一个专门的消息推送服务进

行发送，消息推送服务利用 TCP 长连接服务器，将消息发送给匹配到的司机，同时分单子系统更新数据库订单状态为“已派单”。

16.3 详细设计

关于 Udi 的详细设计，我们将关注网约车平台一些独有的技术特点：长连接管理、派单算法、距离计算。此外，因为订单状态模型是所有交易类应用都非常重要的一个模型，所以我们也会在这里讨论 Udi 的订单状态模型。

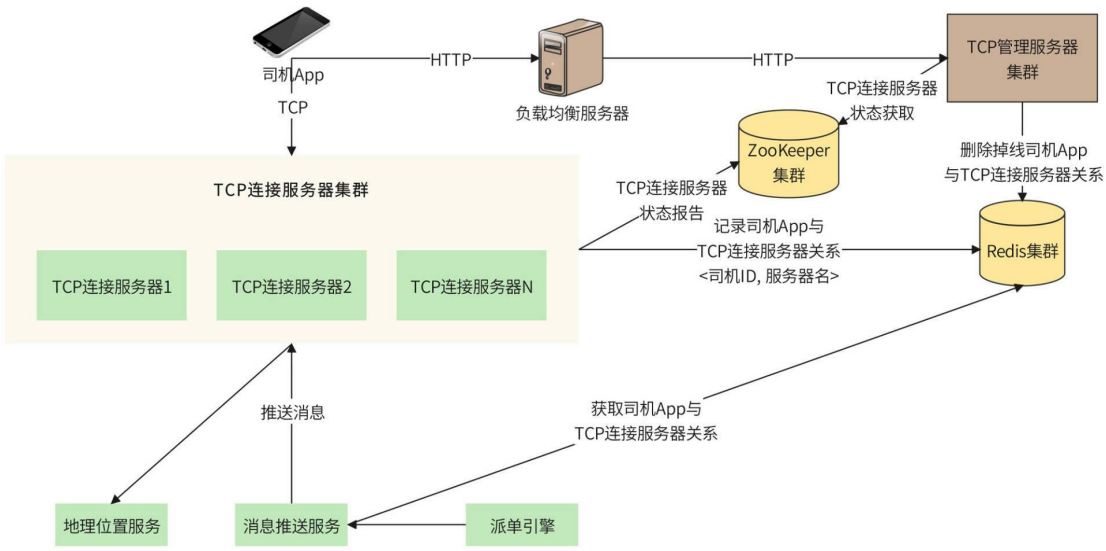
16.3.1 长连接管理

因为司机 App 需要不断向 Udi 系统发送当前位置信息，以及实时接收 Udi 推送的派单请求，所以司机 App 需要和 Udi 系统保持长连接。因此，我们选择让司机 App 和 Udi 系统直接通过 TCP 协议进行长连接。

TCP 连接和 HTTP 连接不同。HTTP 是无状态的，每次 HTTP 请求都可以通过负载均衡服务器，被分发到不同的网关服务器进行处理，正如乘客 App 和服务器的连接那样。也就是说，HTTP 在发起请求的时候，无需知道自己要连接的服务器是哪一台。

而 TCP 是长连接，一旦建立了连接，连接通道就需要长期保持，**不管是司机 App 发送位置信息给服务器，还是服务器推送派单信息给司机 App，都需要使用这个特定的连接通道。**也就是说，司机 App 和服务器的连接是特定的，司机 App 需要知道自己连接的服务器是哪一台，而 Udi 给司机 App 推送消息的时候，也需要知道要通过哪一台服务器才能完成推送。

所以，司机端的 TCP 长连接需要进行专门管理，处理司机 App 和服务器的连接信息，具体架构如下图。



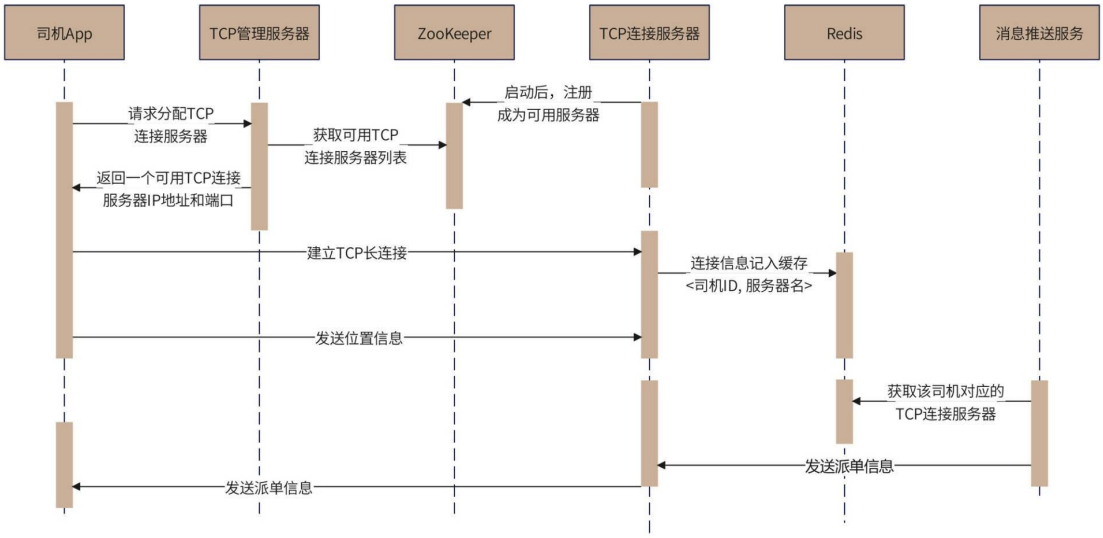
处理长连接的核心是 TCP 管理服务器集群。司机 App 会在启动时通过负载均衡服务器，与 TCP 管理服务器集群通信，请求分配一个 TCP 长连接服务器。

TCP 管理服务器检查 ZooKeeper 服务器，获取当前可以服务的 TCP 连接服务器列表，然后从这些服务器中选择一个，返回其 IP 地址和通信端口给司机 App。这样，司机 App 就可以直接和这台 TCP 连接服务器建立长连接，并发送位置信息了。

TCP 连接服务器启动的时候，会和 ZooKeeper 集群通信，报告自己的状态，便于 TCP 管理服务器为其分配连接。司机 App 和 TCP 连接服务器建立长连接后，TCP 连接服务器需要向 Redis 集群记录这个长连接关系，记录的键值对是<司机 ID，服务器名>。

当 Udi 系统收到用户订单，派单引擎选择了合适的司机进行派单时，系统就可以通过消息推送服务给该司机发送派单消息。消息推送服务器通过 Redis 获取该司机 App 长连接对应的 TCP 服务器，然后消息推送服务器就可以通过该 TCP 服务器的长连接，将派单消息推送给司机 App 了。

长连接管理的主要时序图如下：



如果 TCP 服务器宕机，那么司机 App 和它的长连接也就丢失了。司机 App 需要重新通过 HTTP 来请求 TCP 管理服务器为它分配新的 TCP 服务器。TCP 管理服务器收到请求后，一方面返回新的 TCP 服务器的 IP 地址和通信端口，一方面需要从 Redis 中删除原有的<司机 ID，服务器名>键值对，保证消息推送服务不会使用一个错误的连接线路推送消息。

16.3.2 距离计算

乘客发起一个叫车请求时，Udi 需要为其寻找合适的司机并进行派单，所谓合适的司机，最主要的因素就是距离。在[第 9 讲]的交友系统设计中，我们已经讨论过 GeoHash 算法，Udi 就是直接使用 Redis 的 GeoHash 进行邻近计算。司机的位置信息实时更新到 Redis 中，并直接调用 Redis 的 GeoHash 命令 georadius 计算乘客的邻近司机。

但是 Redis 使用跳表存储 GeoHash，Udi 日活司机两千万，每 3 秒更新一次位置信息，平均每秒就需要对跳表做将近 7 百万次的更新，如此高并发地在一个跳表上更新，是系统不能承受的。所以，我们需要将司机以及跳表的粒度拆得更小。

Udi 以城市作为地理位置的基本单位，也就是说，每个城市在 Redis 中建立一个 GeoHash 的 key，这样，一个城市范围内的司机存储在一个跳表中。对于北京这样的超级城市，还可以更进一步，以城区作为 key，进一步降低跳表的大小和单个跳表上的并发量。

16.3.3 派单算法

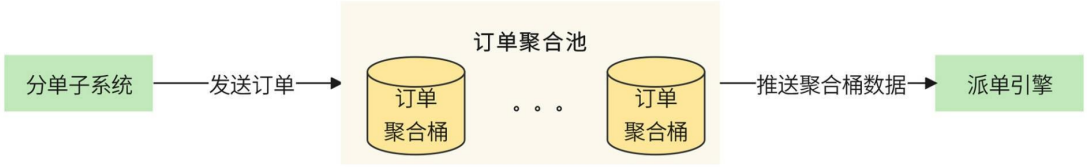
前面说过，派单就是寻找合适的司机，而合适的主要因素就是距离，所以最简单的派单算法就是直接通过 Redis 获取距离乘客上车点最近的空闲网约车即可。

但是这种算法效果非常差，因为 Redis 计算的是两个点之间的空间距离，但是司机必须沿道路行驶过来，在复杂的城市路况下，也许几十米的空间距离行驶十几分钟也未可知。因此，我们必须用行驶距离代替空间距离，即 Udi 必须要依赖一个地理系统，对司机当前位置和上车点进行路径规划，计算司机到达上车点的距离和时间。事实上，我们主要关注的是时间，也就是说，派单算法需要从 Redis 中获取多个邻近用户上车点的空闲司机，然后通过地理系统来计算每个司机到达乘客上车点的时间，最后将订单分配给花费时间最少的司机。

如果附近只有一个乘客，那么为其分配到达时间最快的司机就可以了。但如果附近有多个乘客，那么就需要考虑所有人的等待时间了。比如附近有乘客 1 和乘客 2，以及司机 X 和司机 Y。司机 X 接乘客 1 的时间是 2 分钟，接乘客 2 的时间是 3 分钟；司机 Y 接乘客 1 的时间是 3 分钟，接乘客 2 的时间是 5 分钟。

如果按照单个乘客最短时间选择，给乘客 1 分配司机 X，那么乘客 2 只能分配司机 Y 了，乘客总的等待时间就是 7 分钟。如果给乘客 1 分配司机 Y，乘客 2 分配司机 X，乘客总等待时间就是 6 分钟。司机的时间就是平台的金钱，显然，后者这样的派单更节约所有司机的整体时间，也能为公司带来更多营收，同时也为整体用户带来更好的体验。

这样，我们就不能一个订单一个订单地分别分配司机，我们需要将一批订单聚合在一起，统一进行派单，如下图：



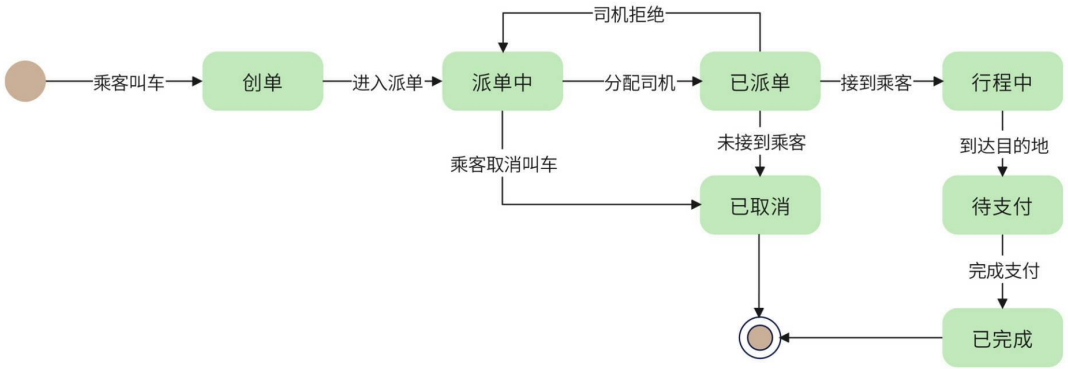
分单子系统收到用户的叫车订单后，不是直接发送给派单引擎进行派单，而是发给一个订单聚合池，订单聚合池里有一些订单聚合桶。订单写完一个聚合桶，就把这个聚合桶内的全部订单推送给派单引擎，由派单引擎根据整体时间最小化原则进行派单。

这里的“写完一个聚合桶”，有两种实现方式，一种是间隔一段时间算写完一个桶，一种是达到一定数量算写完一个桶。最后 Udi 选择间隔 3 秒写一个桶。

这里需要关注的是，派单的时候需要依赖地理系统进行路径规划。事实上，乘客到达时间和金额预估、行驶过程导航、订单结算与投诉处理，都需要依赖地理系统。Udi 初期会使用第三方地理系统进行路径规划，但是将来必须要建设自己的地理系统。

16.3.4 订单状态模型

对于交易型系统而言，订单是其最核心的数据，主要业务逻辑也是围绕订单展开。在订单的生命周期里，订单状态会多次变化，每次变化都是由于核心的业务状态发生了改变，也因此在前面设计的多个地方都提到订单状态。但是这种散乱的订单状态变化无法统一描述订单的完整生命周期，因此我们设计了订单状态模型，如下图：



用户叫车后，系统即为其创建一个订单，订单进入“创单”状态。然后该订单通过消息队列进入分单子系统，分单子系统调用派单引擎为其派单，订单状态进入“派单中”。派单引擎分配到司机，一方面发送消息给司机，一方面修改订单状态为“已派单”。如果司机去接到乘客，订单状态就改为“行程中”；如果司机拒绝接单，就需要为乘客重新派单，订单重新进入消息队列，同时订单状态也改回为“派单中”；如果司机到达上车点，但是联系不到乘客，没有接到乘客，那么订单就会标记为“已取消”。如果在派单中，乘客自己选择取消叫车，订单也进入“已取消”状态。“已取消”是订单的一种最终状态，订单无法再转变为其他状态。

司机到达目的地后，通过 App 确认送达，订单进入“待支付”状态，等待用户支付订单金额。用户支付后，完成订单生命周期，订单状态为“已完成”。

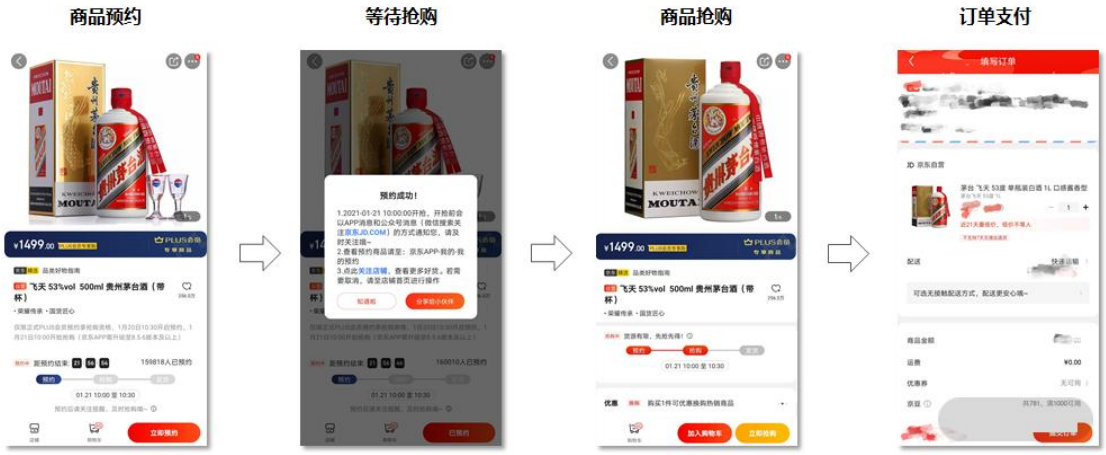
订单状态模型可以帮助我们总览核心业务流程，在设计阶段，可以通过状态图发现业务流程不完备的地方，在开发阶段，可以帮助开发者确认流程实现是否有遗漏。

17. 如何让系统抗住双十一的预约抢购活动？

在大促活动期间，“预约抢购”已经是各大电商平台的主要促销手段，京东自然也会和一些大的供应商合作，推出一些低价的爆款产品，比如 2019 年的 “1499 元抢购飞天茅台”活动，就让很多人每天准时准点拿着手机拼人品。

那这类电商领域的大促抢购场景涉及专栏的哪些内容呢？它们是怎么通过架构设计的方式组合在一起，实现一个完整的需求流程呢？这就是今天要讨论的话题。

我们先把需求梳理一下，总的来说，实现一个抢购系统大概可以分为四个阶段。



- **商品预约**：用户进入商品详情页面，获取购买资格，并等待商品抢购倒计时。
- **等待抢购**：等待商品抢购倒计时，直到商品开放抢购。
- **商品抢购**：商品抢购倒计时结束，用户提交抢购订单，排队等待抢购结果，抢购成功后，扣减系统库存，生成抢购订单。
- **订单支付**：等待用户支付成功后，系统更新订单状态，通知用户购买成功。

接下来，我们就针对各阶段容易出现的问题，来分析其中的技术考点和解决方案。

17.1 商品预约阶段

这几年，很多电商平台为了方便流量运营，改造了传统秒杀场景，通过先预约再抢购的方式预热商品，并根据预约量调整运营策略。而且在预约抢购的活动中，为了增加商品售卖量，会允许抢购前，预约资格超过实际的库存数量。

那么问题来了：如何在高并发量的情况下，让每个用户都能得到抢购资格呢？**这是预约抢购场景第一个技术考察点。** 那你可以基于“06 | 分布式系统中，如何回答锁的实现原理？”来控制抢购资格的发放。

我们基于 Redis 实现分布式锁（这是最常用的方式），在加锁的过程中，实际上是给 Key 键设置一个值，为避免死锁，还要给 Key 键设置一个过期时间。

```
SET lock_key unique_value NX PX 10000
```

- lock_key 就是 key 键；
- unique_value 是客户端生成的唯一的标识；
- NX 代表只在 lock_key 不存在时，才对 lock_key 进行设置操作；
- PX 10000 表示设置 lock_key 的过期时间为 10s，这是为了避免客户端发生异常而无法释放锁。

而解锁的过程就是将 lock_key 键删除，但不能乱删，要保证执行操作的客户端就是加锁的客户端。而这个时候， unique_value 的作用就体现出来，你可以通过 Lua 脚本判断 unique_value 是否为加锁客户端。

选用 Lua 脚本是为了保证解锁操作的原子性。因为 Redis 在执行 Lua 脚本时，可以以原子性的方式执行，保证了锁释放操作的原子性。

```
// 释放锁时，先比较 unique_value 是否相等，避免锁的误释放 if redis.call("get",KEYS[1]) == ARGV[1] then

    return redis.call("del",KEYS[1])

elsereturn0

end
```

这样一来，就通过使用 SET 命令和 Lua 脚本在 Redis 单节点上完成了分布式锁的加锁和解锁。但你要注意，此方案是基于单节点的 Redis 实例实现的，如果此时 Redis 实例发生故障宕机，那么锁变量就没有了，客户端也就无法进行锁操作，就会影响到业务的正常执行。 所以，基于 Redis 实现分布式锁时，你还要掌握如何保证锁的可靠性，也就是怎么基于多个 Redis 节点实现分布式锁 (这部分也可以参考 06 讲中的内容) 。

17.2 等待抢购阶段

用户预约成功之后，在商品详情页面中，会存在一个抢购倒计时，这个倒计时的初始时间是从服务端获取的，用户点击购买按钮时，系统还会去服务端验证是否已经到了抢购时间。

在等待抢购阶段，流量突增，因为在抢购商品之前（尤其是临近开始抢购之前的一分钟内），大部分用户会频繁刷新商品详情页，**商品详情页面的读请求量剧增，** 如果商品详情页面没有做好流量控制，就容易成为整个预约抢购系统中的性能瓶颈点。

那么问题来了：如何解决等待抢购时间内的流量突增问题呢？有两个解决思路。

- **页面静态化**：提前对抢购商品的详情页面做静态化，生成一个静态页面，再把页面放到距离用户最近的 CDN 节点中，这样一来，当浏览器访问页面时，就会自动缓存该页面的静态资源文件（对于静态化技术，很多页面端的模板引擎都支持这样的功能，我就不展开讲了）。
- **服务端限流**：对商品详情页中的动态请求接口设置最大并发访问数量（具体的数量根据上线前的性能压测为准），防止超出预期的请求集中访问系统，造成系统压力过载。操作上，你可以在商品详情页的后端系统入口层（如网关系统）中进行接口限流，如果使用 Nginx 来做反向代理，可以直接基于 Nginx 配置限流算法，比如 Nginx 的 ngx_http_limit_req_module（限制单位时间内所有 IP 的请求数量）和 ngx_stream_limit_conn_module（限制单位时间内单个 IP 的请求数量）两个模块就提供了限流控制的功能，所以你还要提前掌握限流策略的原理，如令牌桶算法的原理。

17.3 商品抢购阶段

在商品抢购阶段，用户会点击提交订单，这时，抢购系统会先校验库存，当库存足够时，系统会先扣减库存，然后再生成订单。在这个过程中，短时间之内提交订单的写流量非常高，所以为了做流量削峰，会将提单请求暂存在消息队列中，并提示用户“抢购排队中……”然后再由后端服务异步处理用户的请求。

而你可以基于数据库和缓存两种方式，来实现校验库存和扣减库存的操作。

但因为抢购场景的瞬时流量极高，一般不会直接基于数据库来实现（因为每次操作数据库，即使通过消息队列做了流量削峰，对数据库来说压力也很大，会产生性能瓶颈）。

如果非要基于数据库的话，你要通过分布式锁来优化扣减库存的并发操作，但此阶段的

分布式锁对可靠性的要求会极高（因为在大促抢购阶段，小的可用性故障，都可能造成大的线上事故），所以基于单节点 Redis 实现的分布式锁不合适，你要选择多节点 Redis 实现分布式锁，或者选型 ZooKeeper。

为了避免上述问题，我们一般基于缓存来存储库存，实现扣减库存的操作。这样在提交订单时，库存的查询和锁定就不会给数据库带来性能瓶颈。不过你仍要注意，基于缓存（如 Redis）的库存扣减操作，仍要考虑缓存系统的单点问题，就算是多节点存储库存，也要引入锁的策略，保证 Redis 实现库存的一致性。

实现了校验库存和扣减库存之后，最后一步是生成抢购订单。由于数据库表会承载订单数据，一旦出现瞬时流量，磁盘 I/O、数据库请求连接数等资源都会出现性能瓶颈，你可以考虑对订单表分库分表，通过对用户 ID 字段进行 Hash 取模，实现分库分表，提高系统的并发能力。

从“商品抢购阶段的架构设计”中我们可以总结出三个技术考点：**流量削峰、扣减库存、分库分表。**

- **“流量削峰”的面试考点**

流量削峰是由于正式抢购场景下，短时间内的提单请求非常高，所以引入消息队列做异步化，然后在抢购系统的后端服务中，启动若干个队列处理消息队列中的提单请求，再执行校验库存、下单等逻辑。

那么如何快速处理消息队列中的提单请求，避免出现大量的消息积压，就是本阶段的考点之一了，方案可以参考“08 | MQ：如何回答消息队列的丢失、重复与积压问题？”

- **“扣减库存”的面试考点**

我刚刚提到，当基于 Redis 实现库存的扣减时，要考虑怎么解决 Redis 的单点问题。而如果基于 Redis 集群来实现扣减库存，还要解决 Redis 在哨兵模式部署的情况下，因为主从切换带来的数据不一致的问题。这就涉及“06 | 分布式系统中，如何回答锁的实现原理？”中的内容。

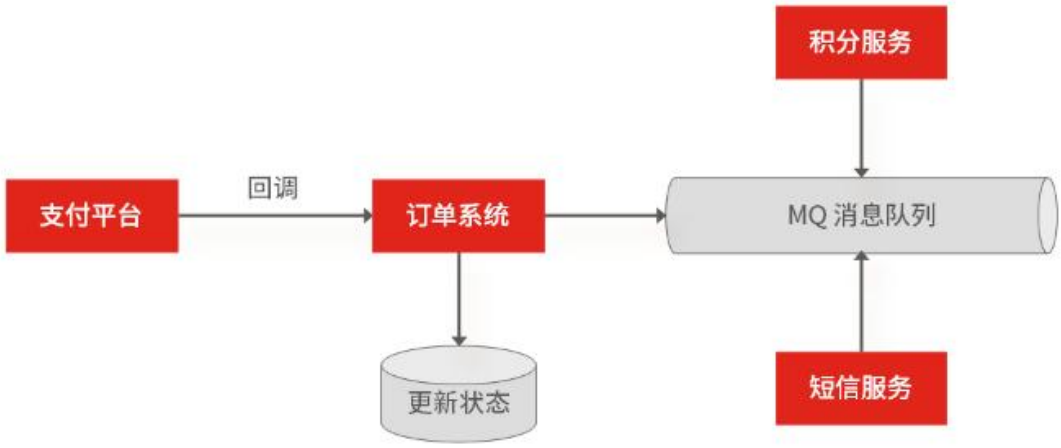
- **“分库分表”的面试考点**

生成订单后如何实现分库分表？你可以参考“04 | 亿级商品存储下，如何深度回答分布式系统的原理性问题？”中的解决方案。

当然还有一个容易忽略的问题：带宽的影响。由于抢购入口的请求量会非常大，可能会占用大量带宽，为了不影响提交订单的请求，有时会从网络工程的角度解决，通过单独的子域名绑定独立的网络服务器，这里就会涉及 DNS 的设计与优化手段。

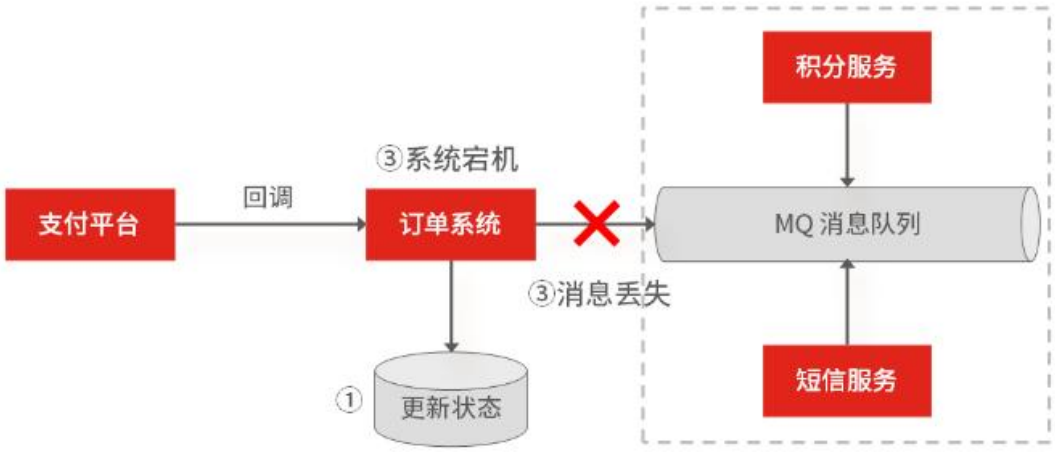
17.4 订单支付阶段

在用户支付订单完成之后，一般会由支付平台回调系统接口，更新订单状态。在支付回调成功之后，抢购系统还会通过异步通知的方式，实现订单更新之外的非核心业务处理，比如积分累计、短信通知等，此阶段可以基于 MQ 实现业务的异步操作。



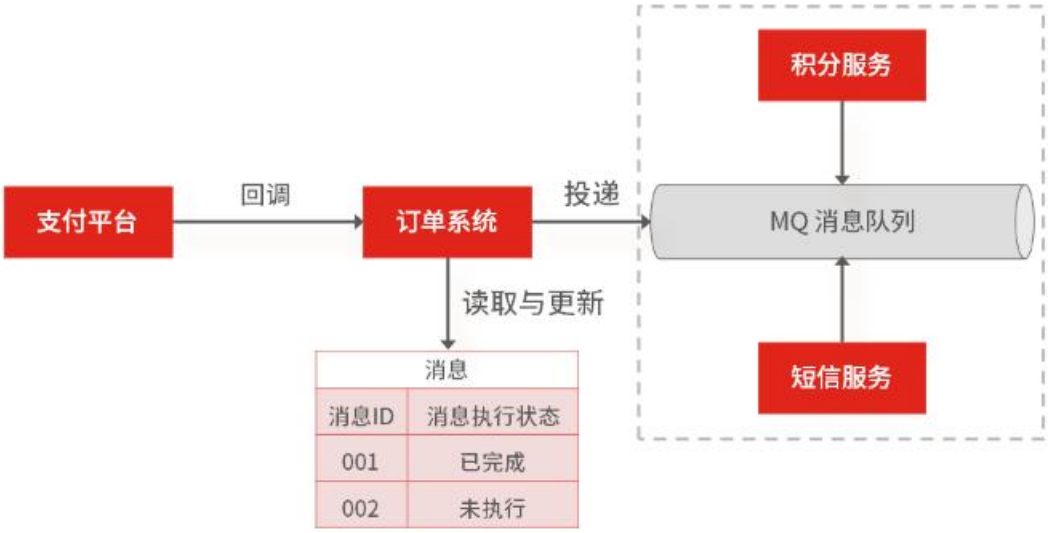
订单支付后操作

不过针对服务的异常（如宕机），会存在请求数据丢失的可能，比如当支付回调系统后，修改订单状态成功了，但是在异步通知积分系统，更新用户累计积分时，订单系统挂掉了，此时 MQ 还没有收到这条消息，那么这条消息数据就无法还原了。



订单支付后操作（异常）

所以你还要考虑“05 | 海量并发场景下，如何回答分布式事务一致性问题？”中，可靠消息投递机制：先做消息的本地存储，再通过异步重试机制，来实现消息的补偿。比如当支付平台回调订单系统，然后在更新状态的同时，插入一个消息，之后再返回第三方支付操作成功的结果。最后，通过数据库中的这条消息，再异步推送其他系统，完成后续的工作。



18. 如果让你设计一个支持千万级流量的架构，你会怎么设计？

对于这个问题，需要从系统全链路的视角，从前端请求到后端服务评估各环节的性能指标,要从全链路的视角上进行高性能的设计。

18.1 前端优化

前端的优化主要有三个环节：**减少请求次数、页面静态化、边缘计算。**

减少请求次数：减少前端脚本与后端服务的请求次数，有三种方案。

- **增加缓存控制**：前端研发同学经常会设置 HTML 的缓存控制头部（Cache-Control 头），这样浏览器在请求同一个文件时，只访问本地保存的资源副本，从而加速文件的访问速度。

- **减少图像的请求次数**：你可能经常会发现，大部分网站会将所有用到的多张图片拼成一张，这样多张图片只需要下载一次，然后再通过 CSS 中的 background-image 和 background-position 来定位目标位置选择显示哪一张图片。
- **减少脚本的请求次数**：通用的做法就是 CSS 压缩和 JavaScript 压缩，将多个文件压缩成一个，目的是减少传输文件的大小，而更重要的是减少请求数。

而页面静态化就是缓存的一种方式，相当于把整个页面元素缓存起来，那么缓存在哪里呢?

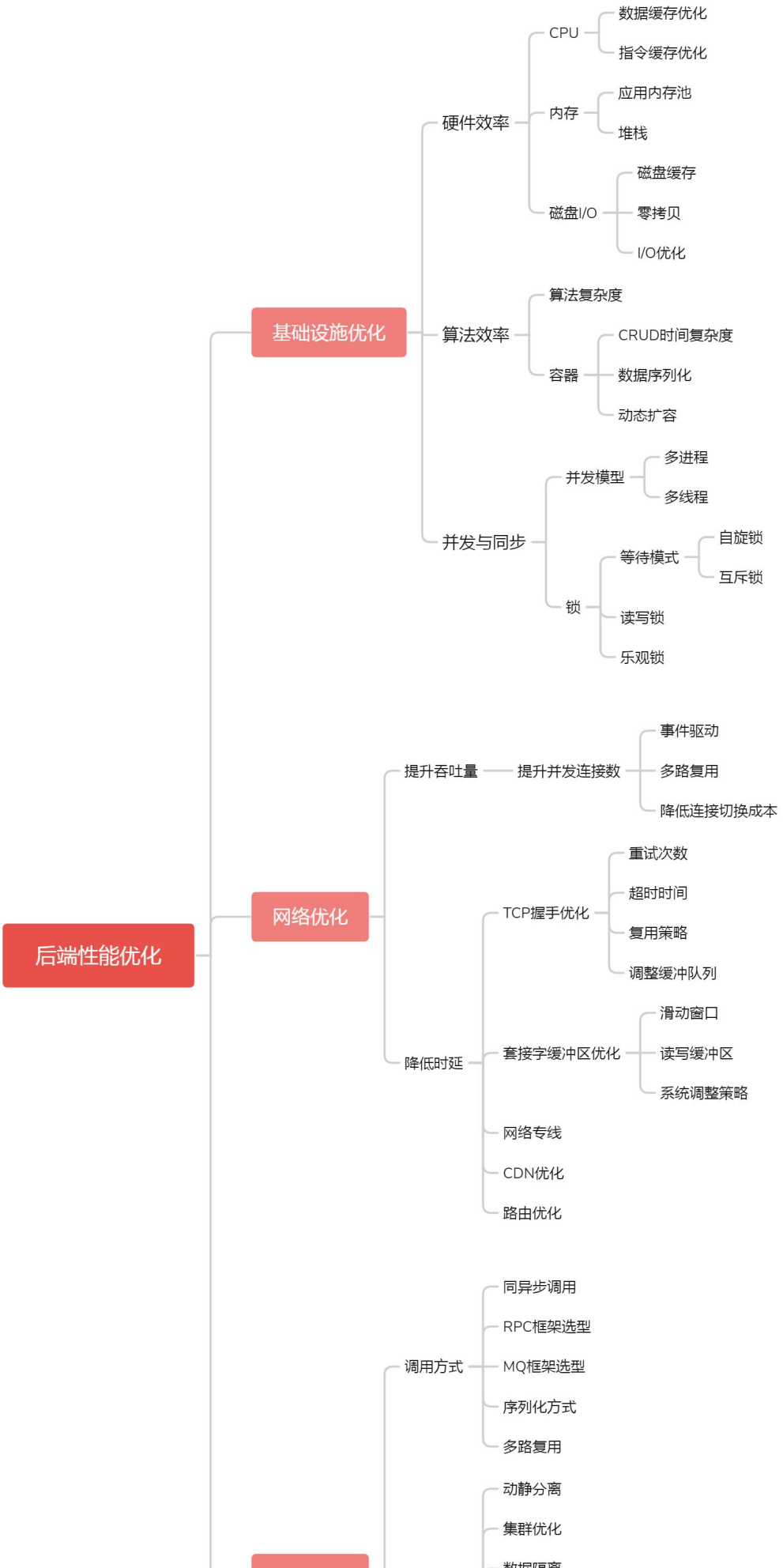
通常是将页面文件事先存储在 CDN 节点中，比如将商品详情页做静态化，就是将商品详情页的页面元素存储在 CDN 节点，然后所有请求就可以直接以由 CDN 来提供服务，就不会到达后端服务了，就减少了对后端服务的压力。

边缘计算，被很多人提及，原因是大数据处理的实时性越来越高，由集中式的服务系统提供实时性的计算能力捉襟见肘，所以很多大厂开始将计算能力放到距离用户最近的 CDN 节点中，这就要求原有的 CDN 节点不只作为静态资源文件的缓存，而是要提供可以定制化的计算能力。

这部分内容会涉及一些新的概念，比如无服务架构 Serverless、BaaS、FaaS，在面试中不要求候选人必须掌握，但它会是你的加分项。

18.2 后端优化

后端环节的性能问题，可以从**基础设施层面、网络层面、架构层面**三个角度进行考量，为了帮助你记忆，我总结了一张脑图给你参考。


后端性能优化
基础设施优化
硬件效率
CPU
数据缓存优化
指令缓存优化
内存
应用内存池
堆栈
磁盘I/O
磁盘缓存
零拷贝
I/O优化
算法效率
算法复杂度
容器
CRUD时间复杂度
数据序列化
动态扩容
并发与同步
并发模型
多进程
多线程
锁
等待模式
自旋锁
互斥锁
读写锁
乐观锁
网络优化
提升吞吐量
提升并发连接数
事件驱动
多路复用
降低连接切换成本
降低时延
TCP握手优化
重试次数
超时时间
复用策略
调整缓冲队列
套接字缓冲区优化
滑动窗口
读写缓冲区
系统调整策略
网络专线
CDN优化
路由优化
调用方式
同异步调用
RPC框架选型
MQ框架选型
序列化方式
多路复用
动静分离
集群优化

比如，网络层面可以考虑网络专线、CDN 优化；架构层面可以考虑动静分离、集群优化、数据隔离、服务拆分、异步驱动、负载均衡等方案。

以上就是高性能架构设计中的技术点，初中级研发工程师要能知道系统的性能瓶颈在哪儿，以及如何优化，但高级研发工程师，不能只停留掌握技术点上，而是要有自己对技术的理解（例如下面的例子）。接下来，我就通过讲解自己对高性能的认知，帮你了解并培养自己对于技术的思考。

18.3 高性能设计中的“道”

你在设计高性能系统架构时，首先是清楚认知系统的硬性性能指标，举个例子。

- **指标需求**：我们要保证系统的 TP 99 为 2s；
- **表面意思**：系统要保证 99% 的请求的响应时间都在 2s 内；
- **深层意思**：对有经验的架构师来说，这样的需求其实是不明确的，任何系统都有自己的承载能力范围，换句话说就是在并发用户数限定范围之内，一定要保证系统的 TP 99 = 2s，例如“我们要保证系统并发数在 100 万用户内的时候，TP 99 为 2s”，对于系统设计者而言，要清楚系统有所能，有所不能。

所以，对一个架构师来说，要设计一个高性能的架构，至少要有以下四个系统设计的思考步骤。

- **明确指标：** 比如当系统小于 100 万并发用户时，要保证系统的 TP 99 = 2s 。
- **保护系统：**当系统的并发用户数量超过 100 万，要做到保证有 100 万用户的 TP 99 = 2s ，然后保护系统，并拒绝其他用户的连接请求。
- **用户体验：** 为了用户体验，要给系统承载容量外的用户提供优雅的体验，比如服务器排队机制，并附加具体、明确的信息提示。
- **快速扩容：** 这一步很容易被一些同学忽略，如今系统的性能指标还有一点就是贵在快速反应，比如能承诺出现流量压力时，可以在 3 分钟内完成扩容，并保证扩容后能承载的并发用户数量的 TP 99 = 2s。

在明确了性能指标之后，高性能架构的落地工作，可以分为以下三个关键技术点。

- **做好系统限流：** 通过流量控制来保证系统的稳定性。当实际并发压力超过系统性能设计指标的时候，就拒绝新的请求的连接，让用户进行排队。

- **做好快速扩容：** 对于扩容能力，一般要储备额外的计算资源，用于不时之需，也就是事先通过预估流出一部分资源池。

有的同学可能会疑惑，既然有多余的资源为什么不提前扩容呢？这是出于对 IT 成本的考虑，作为系统设计者也要把成本作为系统的设计原则之一。

另一个关键因素是系统的扩容速度。这是在当今互联网软件中非常重要的系统能力之一了，就算架构设计的不够优雅，但如果响应够快，也是能解决问题。

- **做好系统优化：** 就是我在上面讲的前后端优化的技术点，我要再补充一点，对系统设计者来说，性能设计要贯穿于系统建设的始终。以一个系统的研发管理过程为例，内容大致包括需求阶段、设计阶段、研发阶段、测试阶段、上线阶段、运行阶段

对于性能设计（不仅仅是性能设计，所有非功能性的设计）要在项目的各阶段都进行考虑，以便根据项目过程的推进随时进行调整和优化。

19. 如何设计 RPC 框架在 10 万 QPS 下实现毫秒级的服务调用?

来思考这样一个场景：你的垂直电商系统的 QPS 已经达到了每秒 2 万次，在做了服务化拆分之后，由于我们把业务逻辑都拆分到了单独部署的服务中，那么假设你在完成一次完整的请求时需要调用 4～5 次服务，计算下来，RPC 服务需要承载大概每秒 10 万次的请求。而你该如何设计 RPC 框架承载如此大的请求量呢？我建议你：

选择合适的网络模型，有针对性地调整网络参数优化网络传输性能；

选择合适的序列化方式，以提升封包、解包的性能。

接下来我从原理出发，让你对于 RPC 有一个理性的认识，这样你在设计 RPC 框架时就可以清晰地知道自己的设计目标是什么了。

19.1 你所知道的 RPC

说到 RPC（Remote Procedure Call，远程过程调用），你不会陌生，它指的是通过网络调用另一台计算机上部署服务的技术。

而 RPC 框架就封装了网络调用的细节，让你像调用本地服务一样调用远程部署的服务。你也许觉得只有像 Dubbo、Grpc、Thrift 这些新兴的框架才算是 RPC 框架，其实严格来说，你很早之前就接触到与 RPC 相关的技术了。

比如，Java 原生就有一套远程调用框架叫做 RMI（Remote Method Invocation），它可以让 Java 程序通过网络调用另一台机器上的 Java 对象的方法。它是一种远程调用的方法，也是 J2EE 时代大名鼎鼎的 EJB 的实现基础。

时至今日，你仍然可以通过 Spring 的“RmiServiceExporter”将 Spring 管理的 bean 暴露成一个 RMI 的服务，从而继续使用 RMI 来实现跨进程的方法调用。之所以 RMI 没有像 Dubbo、Grpc 一样大火，**是因为它存在着一些缺陷**：

RMI 使用专为 Java 远程对象定制的协议 JRMP（Java Remote Messaging Protocol）进行通信，这限制了它的通信双方只能是 Java 语言的程序，无法实现跨语言通信；

RMI 使用 Java 原生的对象序列化方式，生成的字节数组空间较大，效率很差。

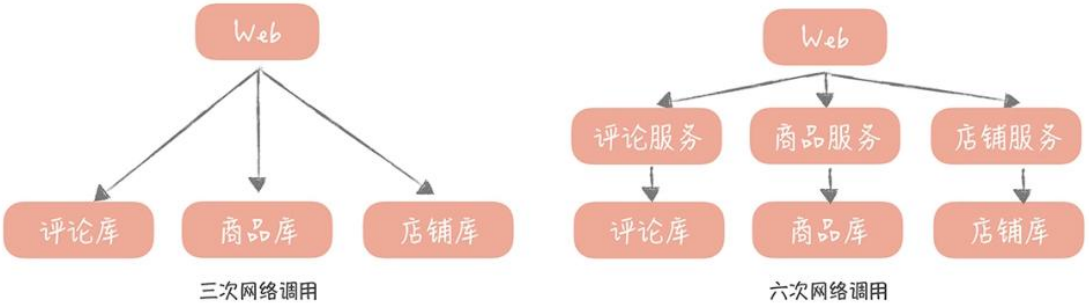
另一个你可能听过的技术是 Web Service，它也可以认为是 RPC 的一种实现方式。它的优势是使用 HTTP+SOAP 协议，保证了调用可以跨语言、跨平台。只要你支持 HTTP 协议，可以解析 XML，那么就能够使用 Web Service。在我看来，由于它使用 XML 封装数据，数据包大，性能还是比较差。

借上面几个例子我主要是想告诉你，RPC 并不是互联网时代的产物，也不是服务化之后才衍生出来的技术，而是一种规范，只要是封装了网络调用的细节能够实现远程调用其他服务，就可以算作是一种 RPC 技术了。

那么你的垂直电商项目在使用 RPC 框架之后会产生什么变化呢?

在性能上的变化是不可忽视的，举个例子。 比方说，你的电商系统中商品详情页面需要商品数据、评论数据还有店铺数据，如果在一体化的架构中，你只需要从商品库、评论库和店铺库获取数据就可以了，不考虑缓存的情况下有三次网络请求。

但是如果独立出商品服务、评论服务和店铺服务之后，那么就需要分别调用这三个服务，而这三个服务又会分别调用各自的数据库，这就是六次网络请求。如果你服务拆分得更细粒度，那么多出的网络调用就会越多，请求的延迟就会更长，而这就是你为了提升系统的扩展性在性能上所付出的代价。



那么我们要如何优化 RPC 的性能，从而尽量减少网络调用对于性能的影响呢？在这里，你首先需要了解一次 RPC 的调用都经过了哪些步骤，因为这样你才可以针对这些步骤中可能存在的性能瓶颈点提出优化方案。步骤如下：

在一次 RPC 调用过程中，客户端首先会将调用的类名、方法名、参数名、参数值等信息，序列化成二进制流；

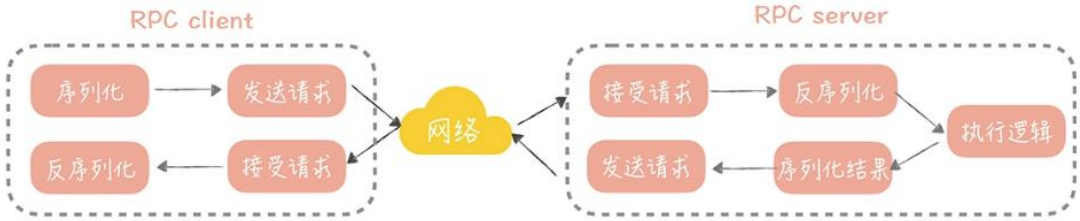
然后客户端将二进制流通过网络发送给服务端；

服务端接收到二进制流之后将它反序列化，得到需要调用的类名、方法名、参数名和参数值，再通过动态代理的方式调用对应的方法得到返回值；

服务端将返回值序列化，再通过网络发送给客户端；

客户端对结果反序列化之后，就可以得到调用的结果了。

过程图如下：



从这张图中你可以看到网络传输的过程，将请求序列化和反序列化的过程， 所以如果要提升 RPC 框架的性能，需要从**网络传输和序列化**两方面来优化。

19.2 如何提升网络传输性能

在网络传输优化中，你首先要做的是选择一种高性能的 I/O 模型。所谓 I/O 模型，就是我们处理 I/O 的方式。而一般单次 I/O 请求会分为两个阶段，每个阶段对于 I/O 的处理方式是不同的。

首先，I/O 会经历一个等待资源的阶段，比方说，等待网络传输数据可用，在这个过程中我们对 I/O 会有两种处理方式：

阻塞。指的是在数据不可用时 I/O 请求一直阻塞，直到数据返回；

非阻塞。指的是数据不可用时 I/O 请求立即返回，直到被通知资源可用为止。

然后是使用资源的阶段，比如说从网络上接收到数据，并且拷贝到应用程序的缓冲区里面。在这个阶段我们也会有两种处理方式：

同步处理。指的是 I/O 请求在读取或者写入数据时会阻塞，直到读取或者写入数据完成；

异步处理。指的是 I/O 请求在读取或者写入数据时立即返回，当操作系统处理完成 I/O 请求并且将数据拷贝到用户提供的缓冲区后，再通知应用 I/O 请求执行完成。

将这两个阶段的四种处理方式做一些排列组合，再做一些补充，就得到了我们常见的五种 I/O 模型：

同步阻塞 I/O；

同步非阻塞 I/O；

同步多路 I/O 复用；

信号驱动 I/O；

异步 I/O。

你需要理解这五种 I/O 模型的区别和特点，不过在理解上你可能会有些难度，所以我来做个比喻方便你理解。

我们把 I/O 过程比喻成烧水倒水的过程，等待资源（就是烧水的过程），使用资源（就是倒水的过程）：

如果你站在灶台边上一直等着（等待资源）水烧开，然后倒水（使用资源），那么就是同步阻塞 I/O；

如果你偷点儿懒，在烧水的时候躺在沙发上看会儿电视（不再时时刻刻等待资源），但是还是要时不时地去看看水开了没有，一旦水开了，马上去倒水（使用资源），那么这就是同步非阻塞 I/O；

如果你想要洗澡，需要同时烧好多壶水，那你就在看电视的间隙去看看哪壶水开了（等待多个资源），哪一壶开了就先倒哪一壶，这样就加快了烧水的速度，这就是同步多路 I/O 复用；

不过你发现自己总是跑厨房去看水开了没，太累了，于是你考虑给你的水壶加一个报警器（信号），只要水开了就马上去倒水，这就是信号驱动 I/O；

最后一种就高级了，你发明了一个智能水壶，在水烧好后自动就可以把水倒好，这就是异步 I/O。

这五种 I/O 模型中最被广泛使用的是**多路 I/O 复用**，Linux 系统中的 select、epoll 等系统调用都是支持多路 I/O 复用模型的，Java 中的高性能网络框架 Netty 默认也是使用这种模型。你可以选择它。

选择好了一种高性能的 I/O 模型，是不是就能实现数据在网络上的高效传输呢？其实并没有那么简单，网络性能的调优涉及很多方面，**其中不可忽视的一项就是网络参数的调优**，接下来我带你了解其中一个典型例子。当然，你可以结合网络基础知识以及成熟 RPC 框架（比如 Dubbo）的源码来深入了解网络参数调优的方方面面。

在之前的项目中，我的团队曾经写过一个简单的 RPC 通信框架。在进行测试的时候发现远程调用一个空业务逻辑的方法时，平均响应时间居然可以到几十毫秒，这明显不符合我们的预期，在我们看来，运行一个空的方法应该在 1 毫秒之内可以返回。于是，我先在测试的时候使用 tcpdump 抓了包，发现一次请求的 ACK 包竟然要经过 40ms 才返回。在网上 google 了一下原因，发现和一个叫做 tcp_nodelay 的参数有关。**这个参数是什么作用呢**？

TCP 协议的包头有 20 字节，IP 协议的包头也有 20 字节，如果仅仅传输 1 字节的数据在网络上传输的就有 20 + 20 + 1 = 41 字节，其中真正有用的数据只有 1 个字节，这对效率和带宽是极大的浪费。所以在 1984 年的时候，John Nagle 提出了以他的名字命名的 Nagle`s 算法，他希望：

> 如果是连续的小数据包，大小没有一个 MSS（Maximum SegmentSize，最大分段大小），并且还没有收到之前发送的数据包的 Ack 信息，那么这些小数据包就会在发送端暂存起来，直到小数据包累积到一个 MSS，或者收到一个 Ack 为止。

这原本是为了减少不必要的网络传输，但是如果接收端开启了 DelayedACK（延迟 ACK 的发送，这样可以合并多个 ACK，提升网络传输效率），那就会发生发送端发送

第一个数据包后接收端没有返回 ACK，这时发送端发送了第二个数据包，因为 Nagle`s 算法的存在，并且第一个发送包的 ACK 还没有返回，所以第二个包会暂存起来。而 DelayedACK 的超时时间默认是 40ms，所以一旦到了 40ms，接收端回给发送端 ACK，那么发送端才会发送第二个包，这样就增加了延迟。

解决的方式非常简单：只要在 Socket 上开启 tcp_nodelay 就好了，这个参数关闭了 Nagle`s 算法，这样发送端就不需要等到上一个发送包的 ACK 返回直接发送新的数据包就好了。这对于强网络交互的场景来说非常的适用，基本上，如果你要自己实现一套网络框架，tcp_nodelay 这个参数最好是要开启的。

19.3 选择合适的序列化方式

在对网络数据传输完成调优之后，另外一个需要关注的点就是数据的序列化和反序列化。通常所说的序列化是将传输对象转换成二进制串的过程，而反序列化则是相反的动作，是将二进制串转换成对象的过程。

从上面的 RPC 调用过程中你可以看到，一次 RPC 调用需要经历两次数据序列化的过程和两次数据反序列化的过程，可见它们对于 RPC 的性能影响是很大的，**那么我们在选择序列化方式的时候需要考虑哪些因素呢?**

首先需要考虑的肯定是性能嘛，性能包括时间上的开销和空间上的开销，时间上的开销就是序列化和反序列化的速度，这是显而易见需要重点考虑的，而空间上的开销则是序列化后的二进制串的大小，过大的二进制串也会占据传输带宽影响传输效率。

除去性能之外，我们需要考虑的是它是否可以跨语言、跨平台，这一点也非常重要，因为一般的公司的技术体系都不是单一的，使用的语言也不是单一的，那么如果你的 RPC 框架中传输的数据只能被一种语言解析，这无疑限制了框架的使用。

另外，扩展性也是一个需要考虑的重点问题。你想想，如果对象增加了一个字段就会造成传输协议的不兼容，导致服务调用失败，这会是多么可怕的事情。

综合上面的几个考虑点，在我看来，我们的序列化备选方案主要有以下几种：

首先是大家熟知的 JSON，它起源于 JavaScript 是一种最广泛使用的序列化协议，它的优势简单易用，同时在性能上相比 XML 有比较大的优势。

另外的 Thrift 和 Protobuf 都是需要引入 IDL（Interface description language）的，也就是需要按照约定的语法写一个 IDL 文件，然后通过特定的编译器将它转换成各语言对应的代码，从而实现跨语言的特点。

Thrift 是 Facebook 开源的高性能的序列化协议，也是一个轻量级的 RPC 框架；Protobuf 是谷歌开源的序列化协议。它们的共同特点是无论在空间上还是时间上都有着很高的性能，缺点就是由于 IDL 存在带来一些使用上的不方便。

那么你要如何选择这几种序列化协议呢？这里我给你几点建议：

如果对于性能要求不高，在传输数据占用带宽不大的场景下可以使用 JSON 作为序列化协议；

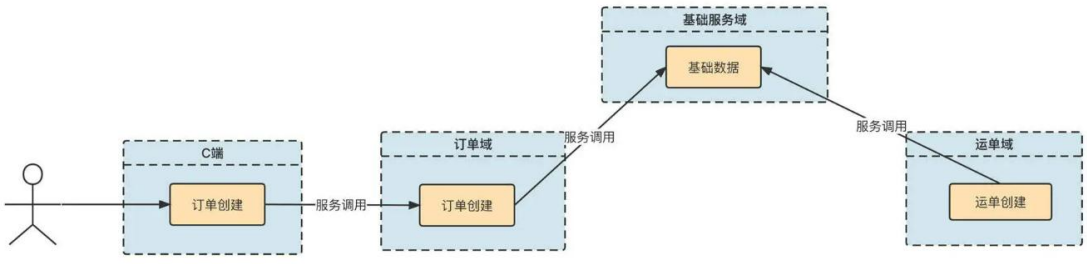
如果对于性能要求比较高，那么使用 Thrift 或者 Protobuf 都可以。而 Thrift 提供了配套的 RPC 框架，所以想要一体化的解决方案，你可以优先考虑 Thrift；

在一些存储的场景下，比如说你的缓存中存储的数据占用空间较大，那么你可以考虑使用 Protobuf 替换 JSON 作为存储数据的序列化方式。

20. 在你上一家公司中是如何如何实现蓝绿发布的?

随着微服务应用的不断增加，各个微服务之间的依赖关系也变得比较复杂，各个微服务的更新、升级部署给整个服务域的稳定性带来很大挑战。**怎么以不停机的方式部署升级微服务呢?**

在进行技术方案的设计之前，我们先来了解一下生产环境的基本部署情况，如下图所示：



用户在面向用户端（下文通称 C 端）下单后，C 端订单系统需要远程调用订单域中的“创建订单“接口。同时，订单域、运单域相关服务都需要调用基础服务域，进行基础数据的查询服务。

从这里也可以看出，基础服务的稳定运行对整个微服务体系至关重要。那如何确保基础服务域不受版本的影响，始终能够提供稳定可控的服务呢?

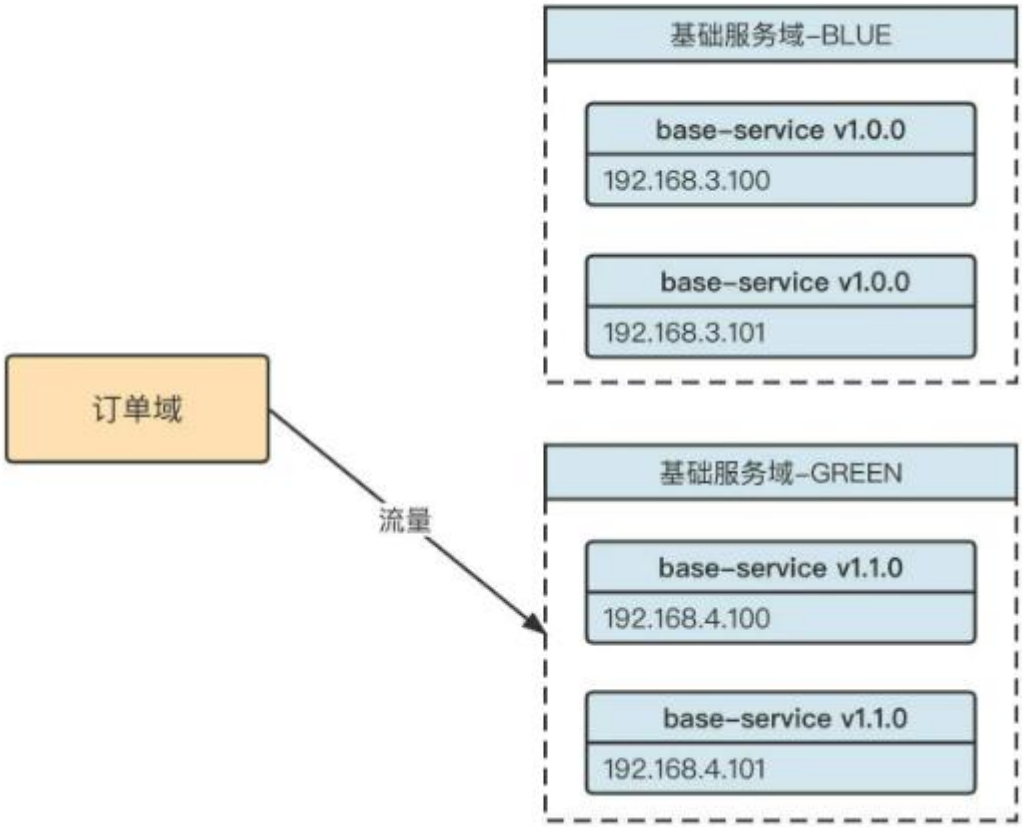
20.1 设计方案

我们公司为了解决这个问题实现了**蓝绿发布**。那什么是蓝绿发布呢?

蓝绿发布指的是在蓝、绿两套环境中分别运行项目的两个版本的代码。但是在进行版本发布时只更新其中一个环境，这样方便另一个环境快速回滚。

接下来我们看一下蓝绿发布的基本流程。

如果系统采取蓝绿发布，在下一个版本（base-service v1.2.0）发布之前，会这样部署架构:

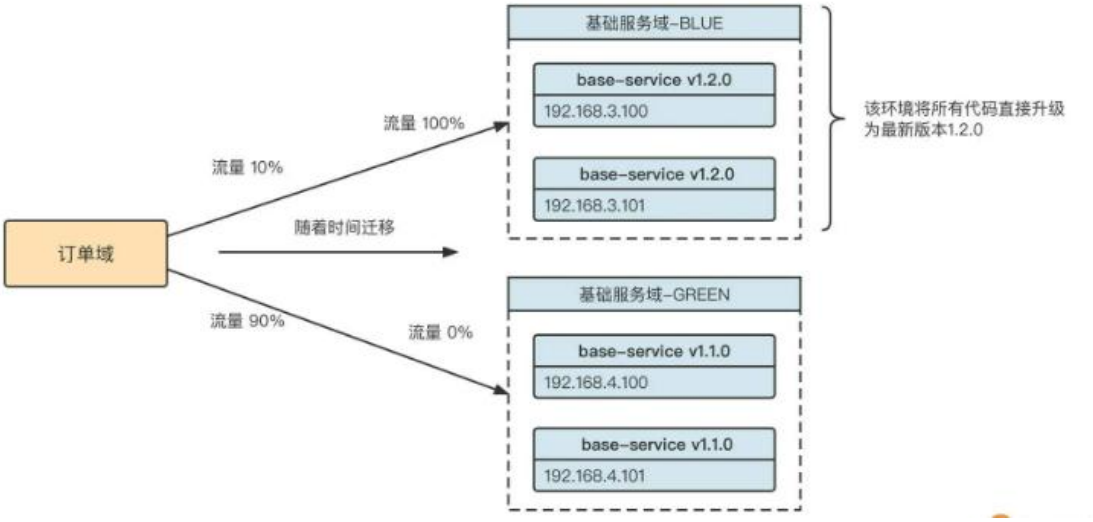


当前订单域调用流量进入基础服务域 GREEN 环境。团队计划在 12:00 发布新版本（base-service v1.2.0），这时我们通常会执行下面几个操作。

- 将新版本 1.2.0 全部发布在 BLUE 环境上。因为此时 BLUE 环境没有任何流量，对运行中的系统无任何影响。

- 在请求入口对**流量进行切分**。通常可以按照百分比分配流量，待系统运行良好后，再逐步将流量全部切换到新版本。
- 如果发现新版本存在严重问题，可以将流量全部切换到原来的环境，实现版本**快速回滚**。

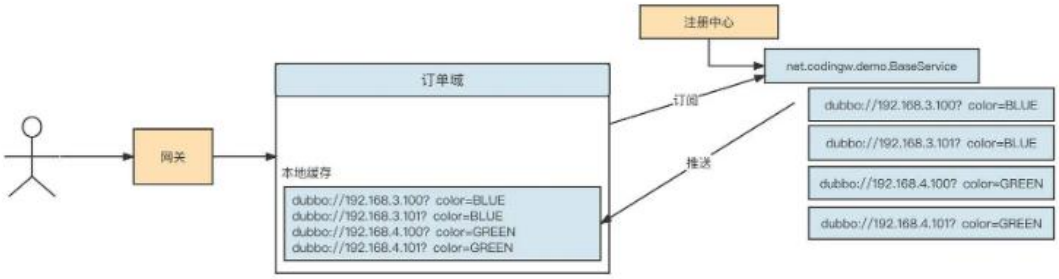
这个过程可以用下面这张图表示：



这个思路听起来很简单，但是怎么实现呢？

这就不得不提到上节课专门提到的**路由选择（Router）**了，它是 Dubbo 服务调用中非常重要的一步。路由选择的核心思想是在客户端进行负载均衡之前，通过一定的过滤规则，只在服务提供者列表中选择符合条件的提供者。

我们再看上面的实例图，从订单域消费者的视角，服务提供者列表大概是下面这个样子：



然后呢，我们按照比例对入口流量进行分流。例如，80% 的请求颜色为 BLUE，20% 的请求颜色为 GREEN。那些颜色为 BLUE 的请求，在真正执行 RPC 服务调用时，只从服务提供者列表中选择“color=BLUE”的服务提供者。同样，颜色为 GREEN 的请求只选择“color=GREEN”的服务提供者，这就实现了流量切分。

具体的操作是，在 Dubbo 中为这个场景引入 Tag 路由机制。

首先，服务提供者在启动时需要通过“-Dubbo.provider.tag”系统参数来设置服务提供者所属的标签。

例如，在 192.168.3.100 和 192.168.3.101 这两台机器上启动 base-service 程序时，需要添加“-Dubbo.provider.tag=BLUE”系统参数；而在 192.168.4.100 和 192.168.4.101 这两台机器上启动 base-service 程序时，则要添加“-Dubbo.provider.tag=GREEN”系统参数，通过这个操作完成对服务提供者的打标。服务提供者启动后，生成的服务提供者 URL 连接如下所示：

```
dubbo://192.168.3.100:20880/net.codingw.demo.BaseUser?dubbo.tag=BLUE
```

下一步，在服务入口对流量进行染色，从而实现流量切分。

蓝绿发布的流量通常是在流量入口处进行染色的。例如，我们可以使用随机加权来实现流量切分算法，用它对流量进行染色，具体示范代码如下：

```
public static String selectColor(String[] colorArr, int[] weightArr) {
        int length = colorArr.length;
        boolean sameWeight = true;
        int totalWeight = 0;
        for (int i = 0; i < length; i++) {
            int weight = weightArr[i];
            totalWeight += weight;
            if (sameWeight && totalWeight != weight * (i + 1)) {
                sameWeight = false;
            }
        }
        if (totalWeight > 0 && !sameWeight) {
            int offset = ThreadLocalRandom.current().nextInt(totalWeight);
            System.out.println("offset:" + offset);
            for (int i = 0; i < length; i++) {
                if (offset < weightArr[i]) {
                    return colorArr[i];
                }
            }
        }
        return colorArr[ThreadLocalRandom.current().nextInt(length)];
    }

    //测试代码
    public static void main(String[] args) {
        String[] colorArr = new String[]{"GREEN","BLUE"};
```

```
        int[] weightArr = new int[] {100,50};
        for(int i = 0; i < 20; i ++) {
            System.out.println(selectColor(colorArr, weightArr));
        }
    }
```

根据流量切分算法计算得到流量标识后，怎么在消费端跟进流量标识从而进行路由选择呢？我们通常会将染色标记放在 ThreadLocal 中，然后再编写 Filter，获取或者传递路由标签。

但这个只是一个流量的切分算法，那如何动态设置蓝绿的比例或者说权重呢？其实，我们可以为发布系统提供一个设置权重的页面，用户设置完权重后写入到配置中心(ZooKeeper、Apollo)，然后应用程序动态感知到变化，利用最新的权重进行流量切分。

通过流量切分算法计算出一个请求的流量标识后，通常会存储在 ThreadLocal 中，实现代码如下：

```
publicclassThreadLocalContext {
    privatestaticfinal ThreadLocal<String> tagContext   = newThreadLocal<>();

    publicstaticvoidsetTag(String tag) {
        tagContext.set(tag);
    }

    publicstatic String getTag() {
        return tagContext.get();
    }

    publicstaticvoidresetTag() {
        tagContext.remove();
    }
}


//在整个请求的入口 Stringcolor= selectColor(colorArr, weightArr);
try {
  ThreadLocalContext.setTag(color);

  //执行第一个远程调用
  invokeRpc1();

  //执行另外一个远程调用
  invokeRpc2();
```

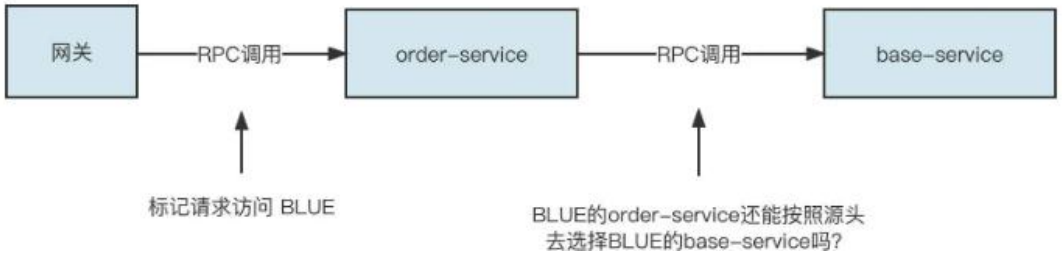
```
27
28 } finally {
29     ThreadLocalContext.reset();
30 }
```

将请求的流量标识存储到线程本地变量之后，还需要将流量标识附加到 RPC 请求调用中，这样才能触发正确的路由选择，具体代码示例如下：

```
1  import org.apache.commons.lang3.StringUtils;
2  import org.apache.dubbo.common.extension.Activate;
3  import org.apache.dubbo.common.logger.Logger;
4  import org.apache.dubbo.common.logger.LoggerFactory;
5  import org.apache.dubbo.rpc.*;
6  import org.apache.dubbo.rpc.cluster.router.tag.TagRouter;
7
8  importstatic org.apache.dubbo.common.constants.CommonConstants.CONSUMER;
9  importstatic org.apache.dubbo.rpc.Constants.ACCESS_LOG_KEY;
10
11 @Activate(group = CONSUMER, value =
"tagConsumerFilter")publicclassTagConsumerContextFilterimplementsFilter {
12
13     privatestaticfinalLoggerlogger=
LoggerFactory.getLogger(TagConsumerContextFilter.class);
14
15     @Overridepublic Result invoke(Invoker<?> invoker, Invocation invocation)throws
RpcException {
16         try {
17             Stringtag= ThreadLocalContext.getTag();
18             if(StringUtils.isNotEmpty(tag)) {
19                 invocation.setAttachment(TagRouter.NAME, tag );
20             }
21         } catch (Throwable t) {
22             logger.warn("Exception in TagConsumerContextFilter of service(" + invoker +
" -> " + invocation + ")", t);
23         }
24
25         // 调用链传递 return invoker.invoke(invocation);
26     }
27 }
```

这样在 RPC 调用的过程中，服务调用者就能根据本地线程变量中存储的流量标记，选择不同机房的服务提供者，从而实现蓝绿发布了。

同时，在实际生产环境中，一个调用链条中往往会存在多个 RPC 调用，那第一个 RPC 中的路由标签能自动传递到第二个 RPC 调用吗？



答案是不可以，我们需要再写一个服务端生效的 Filter，示例代码如下：

```
import org.apache.commons.lang3.StringUtils;
import org.apache.dubbo.common.logger.Logger;
import org.apache.dubbo.common.logger.LoggerFactory;
import org.apache.dubbo.common.extension.Activate;
import org.apache.dubbo.rpc.*;
import org.apache.dubbo.rpc.cluster.router.tag.TagRouter;

importstatic org.apache.dubbo.common.constants.CommonConstants.PROVIDER;

@Activate(group = PROVIDER, value =
"tagProviderFilter")publicclassTagProviderContextFilterimplementsFilter {

    privatestaticfinalLoggerlogger=
LoggerFactory.getLogger(TagProviderContextFilter.class);

    @Overridepublic Result invoke(Invoker<?> invoker, Invocation invocation)throws
RpcException {
        try {
            Stringtag= invocation.getAttachment(TagRouter.NAME);
            if(StringUtils.isNotEmpty(tag)) {
                ThreadLocalContext.setTag(tag);
            }
        } catch (Throwable t) {
            logger.warn("Exception in TagProviderContextFilter of service(" + invoker + "
-> " + invocation + ")", t);
        }
        // 调用链传递 return invoker.invoke(invocation);
    }
}
```

也就是将调用链中的 tag 存储到服务端的线程本地上下文环境中，当服务端调用其他服务时，可以继续将 tag 传递到下一个 RPC 调用链中。

这样，我们的蓝绿发布就基本完成了。但这里还有一个问题。规模较大的公司的生产环境往往会运行很多微服务，我们无法将蓝绿机制一下引入到所有微服务当中，**必然会存在一部分应用使用蓝绿发布，但其他应用没有使用蓝绿的情况。怎么做到兼容呢?**

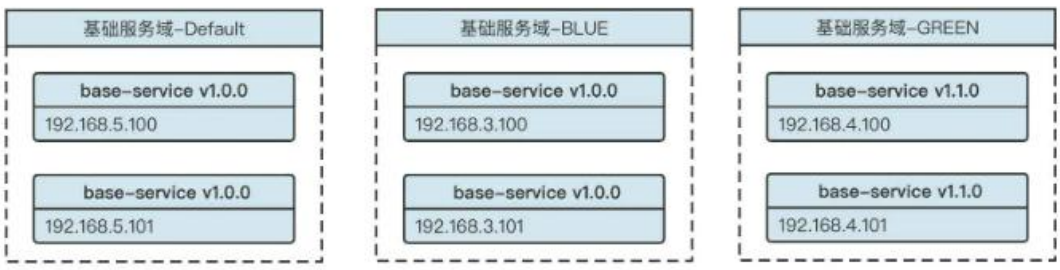
比方说，我们公司目前核心业务域的蓝绿部署情况如下：

这里，订单域接入了蓝绿发布；C 端应用需要调用订单域相关接口，因此也接入了蓝绿发布；但运单中心并未接入蓝绿发布。这时候，运单中心能调用订单域的服务吗?

要回答这个问题，我们要先看看 Dubbo 官方的降级策略。

- 如果消费者侧设置了标签，那么如果集群中没有对应标签的服务提供者，默认可以选择不带任何标签的服务提供者进行服务调用。该行为可以通过设置 request.tag.force=true 来禁止，这就是说如果 request.tag.force 为 true，一旦没有对应标签的服务提供者，就会跑出“No Provider”异常。
- 如果消费者侧没有设置标签，那就只能向集群中没有设置标签的服务提供者发起请求，如果不存在没有标签的服务提供者，则报“No Provider”异常。

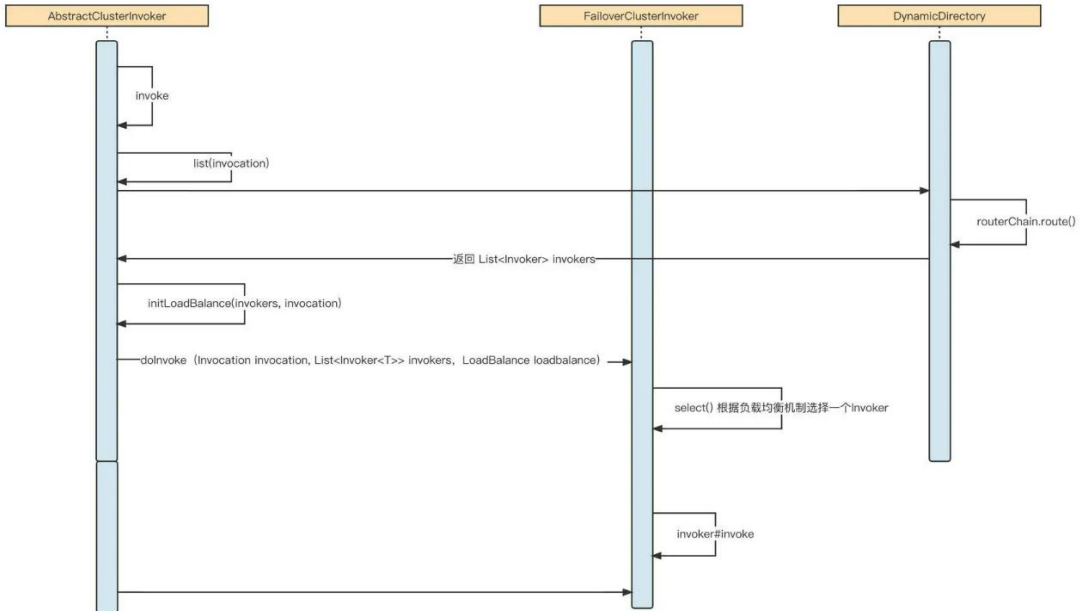
回到上面的问题，运单中心由于未接入蓝绿发布，所以不带任何标签，它无法调用订单域的服务。为了解决这个问题，订单域还需要部署一些不带标签的服务。订单域最终的部署大概如下图所示：



也就是说，订单域为了兼容那些还没接入蓝绿发布的应用需要部署 3 套环境，一套为不设置标签的服务提供者，一套为蓝颜色的服务提供者，另一套为绿颜色的服务提供者。蓝绿发布实践就介绍到这里了，在这节课的最后，我们再来学习一下蓝绿发布底层依托的原理。

20.2 实现原理

先来看一下 Dubbo 服务调用的基本时序图：



我建议你按照这张时序图跟踪一下源码，更加详细地了解 Dubbo 服务调用的核心流程与实现关键点，我在这里总结了几个要点：

- Dubbo 的服务调用支持容错，对应的抽象类为 AbstractClusterInvoker，它封装了服务调用的基本流程。Dubbo 内置了 failover、failfast、failsafe、failback、forking 等失败容错策略，每一个策略对应 AbstractClusterInvoker 的一个实现；
- 在调用 AbstractClusterInvoker 服务的时候，首先需要获取所有的服务提供者列表，这个过程我们称之为服务动态发现（具体实现类为 DynamicDirectory）。在获取路由信息之前，需要调用 RouterChain 的 route 方法，执行路由选择策略，筛选出服务动态发现的服务提供者列表。我们这一课的重点，标签路由的具体实现类 TagRouter 就是在这里发挥作用的。

我们也详细拆解一下 TagRouter 的 route 方法。因为这个方法的实现代码比较多，我们还是分步讲解。

第一步，执行静态路由过滤机制，代码如下：

```
finalTagRouterRuletagRouterRuleCopy= tagRouterRule;
if (tagRouterRuleCopy == null || !tagRouterRuleCopy.isValid()
|| !tagRouterRuleCopy.isEnabled()) {
    return filterUsingStaticTag(invokers, url, invocation);
}
```

如果路由规则为空，则根据 tag 进行过滤。我们顺便也看一下基于 tag 的静态过滤机制是如何实现的：

```
private <T> List<Invoker<T>> filterUsingStaticTag(List<Invoker<T>> invokers, URL url,
Invocation invocation) {
    List<Invoker<T>> result;
    Stringtag= StringUtils.isEmpty(invocation.getAttachment(TAG_KEY)) ?
url.getParameter(TAG_KEY)
                :invocation.getAttachment(TAG_KEY)
    if (!StringUtils.isEmpty(tag)) {
        result = filterInvoker(invokers, invoker ->
tag.equals(invoker.getUrl().getParameter(TAG_KEY)));
        if (CollectionUtils.isEmpty(result) && !isForceUseTag(invocation)) {
            result = filterInvoker(invokers, invoker ->

StringUtils.isEmpty(invoker.getUrl().getParameter(TAG_KEY)));
        }
    } else {
        result = filterInvoker(invokers, invoker ->

StringUtils.isEmpty(invoker.getUrl().getParameter(TAG_KEY)));
    }
    return result;
}
```

尝试从 Invocation (服务调用上下文) 中或者从 URL 中获取 tag 的值，根据 tag 是否为空，执行两种不同的策略：

- 如果 tag 不为空，首先按照 tag 找到服务提供者列表中打了同样标签的服务提供者列表，如果 dubbo.force.tag 的设置为 false，则查找服务提供者列表，筛查出没有打标签的服务提供者列表。
- 如果 tag 为空，则直接查找没有打标签的服务提供者列表。

我们继续回到 TagRouter 的 route 方法。第二步操作是，按照路由规则进行筛选，具体代码如下：

```
// if we are requesting for a Provider with a specific tagif (StringUtils.isNotEmpty(tag)) {
    List<String> addresses = tagRouterRuleCopy.getTagnameToAddresses().get(tag);
    if (CollectionUtils.isNotEmpty(addresses)) {
        result = filterInvoker(invokers, invoker -> addressMatches(invoker.getUrl(), addresses));
        if (CollectionUtils.isNotEmpty(result) || tagRouterRuleCopy.isForce()) {
            return result;
        }
    } else {
      result = filterInvoker(invokers, invoker -> tag.equals(invoker.getUrl().getParameter(TAG_KEY)));
    }
    if (CollectionUtils.isNotEmpty(result) || isForceUseTag(invocation)) {
        return result;
    } else {
        List<Invoker<T>> tmp = filterInvoker(invokers, invoker -> addressNotMatches(invoker.getUrl(),
                                    tagRouterRuleCopy.getAddresses()));
        return filterInvoker(tmp, invoker -> StringUtils.isEmpty(invoker.getUrl().getParameter(TAG_KEY)));
    }
}
```

上面这段代码比较简单，它的过滤思路和静态 tag 过滤是相似的。不同点是，这里可以通过 YAML 格式配置单个服务的路由规则。具体的配置格式如下：

```
 force:trueenabled:truepriority:1key:demo-provider(服务名称)tags:-name:tag1addresses: [ip1, ip2]
    -name:tag2addresses: [ip3, ip4]
```

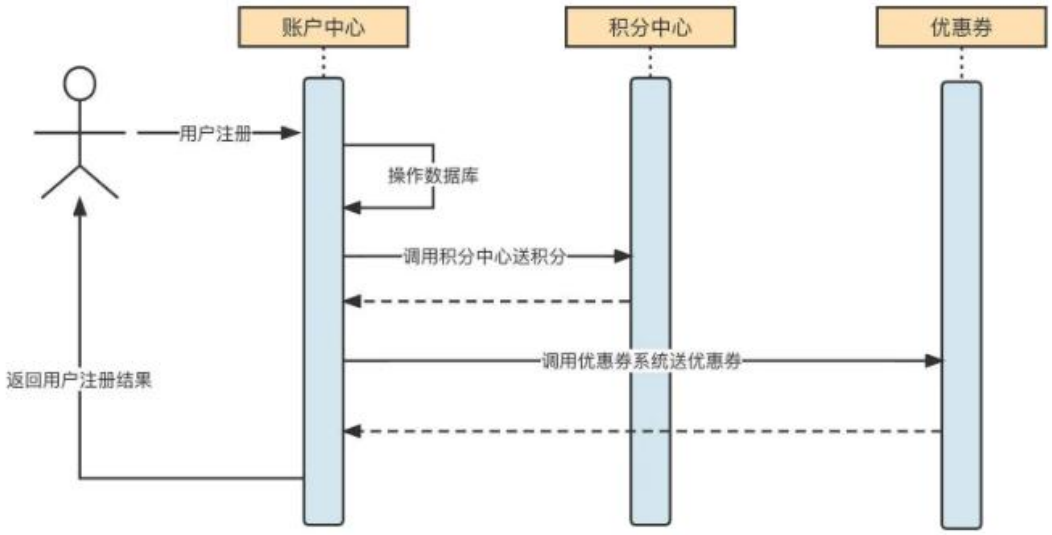
这些数据都会记录在注册中心，并在发生变化后实时通知 TagRouter，从而实现路由规则的动态配置。

21. 如何根据应用场景选择合适的消息中间件?

21.1 消息中间件的应用场景

消息中间件的应用场景主要有两个：异步解耦与削峰填谷。

我们首先通过电商平台用户注册送积分、送优惠券这个场景来理解异步解耦合。如果不使用消息中间件，电商平台送积分的实现也许是下图这个样子：



我们简单看一下这个流程。

1. 用户在网站前端注册页面填写相关信息，然后调用账号中心服务，注册账号。
2. 账户中心首先执行用户注册逻辑处理（例如判断用户是否已注册、是否符合注册条件等），然后写入到数据库。
3. 注册成功后，需要调用积分中心（赠送积分接口）给用户送积分。
4. 送完积分后，再调用优惠券相关接口，为用户赠送优惠券。
5. 成功送完积分、优惠券后，向用户返回“注册成功”

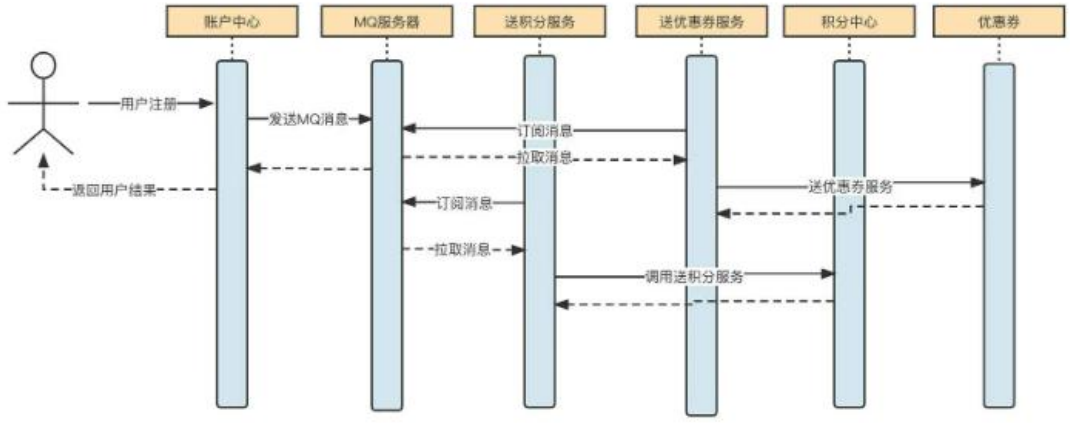
从架构角度看，上面这个实现方法有一个非常严重的问题，那就是可扩展性低。

例如，如果要在春节期间调整活动策略，在发送积分的同时，还需要额外发送新春大礼包，开发人员为了实现这一功能，就不得不修改用户注册流程，并重新部署用户注册模块。

从功能维度来看，这次需求的变更集中在活动相关的内容。用户注册本身的逻辑并未发生变化，但由于用户注册逻辑与活动模块存在耦合，两个模块必须一起调整和发布，这就对系统稳定性造成了影响。

另外，调用积分、优惠券两个远程 RPC 请求让用户注册主流程变长，在高并发场景下，用户注册这一环容易成为系统瓶颈。

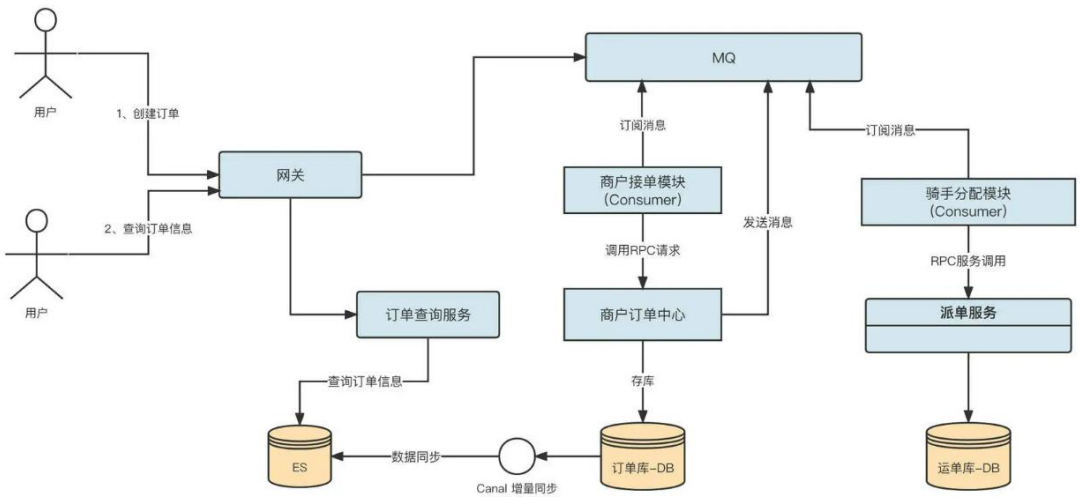
要解决上面这两个明显的设计缺陷，常用的方案是引入消息中间件，让用户注册主流程和商家活动异步解耦合。改造后的时序图如下：



账户中心完成用户注册相关逻辑后，会向 MQ 发送一条消息到 MQ 服务器，然后就直接给用户返回“注册成功”。赠送优惠券、积分等与活动相关的需求我们可以异步执行，这样，无论后续互动逻辑发生什么变化，账户中心都不需要发布新版本。

引入送积分服务（MQ 消费者应用）和送优惠券服务（MQ 消费者应用）会订阅消息，并根据消息调用积分中心、优惠券中心的服务。如果后续活动发生变化，例如取消送积分活动但开始赠送新春大礼包，那我们只需停止送积分服务应用，增加送新春大礼包的消费者应用，**就可以真正做到对新增开放，对修改关闭。**

消息中间件的另外一个常用场景是削峰填谷。我们来看一个外卖骑手送餐的场景。它的设计架构图如下：



我们分别说明一下“创建订单流程”和“查询订单信息”两个流程，探究一下这个方案的精髓。

先来看创建订单流程。

6. 用户在 App 中下单，App 会调用网关相关接口创建订单，网关接收到请求后，并不是直接调用内部商户订单中心来创建订单接口，而是先发送一条消息到 MQ。
7. 商户接单模块（Consumer）订阅 MQ 中的消息，处理消息的时候调用内部商户订单中心创建订单接口，创建一条真正的订单数据到数据库。
8. 创建订单后，商户订单中心将再发送一条消息到 MQ 服务器。然后骑手分配模块（Consumer）订阅消息，调用派单服务相关接口，引导骑手进行外卖配送。
9. 同时，数据同步组件（Canal）将数据库中的数据准实时同步到 Es 服务器。

为什么网关不直接调用外部的创建订单接口，而是将数据先写入到 MQ 中呢?

我们不妨设想一下，商户订单中心支持的最大并发为 1w/tps。如果某一个业务高峰期，从网关进入的流量突然飙升到 1.5w/tps，而且持续了 10 分钟，商户订单系统会直接崩溃，造成服务不可用等严重故障!

那该如何解决呢?

有人可能会说，我们可以使用限流机制保护商户订单系统。例如，我们只允许 9000TPS 的流量从网关进入到商户订单中心，直接拒绝多余的流量，让客户端重试。这确实可以解决问题，但会带来经济损失和糟糕的用户体验。

这个时候我们有一个更加友好的解决方案：引入消息中间件。

引入消息中间件的目的是让它来扛住海量流量，流量先进入到消息队列中，然后消费端下游系统可以慢慢消费消息中间件中的数据，这样能有效保护下游系统不被瞬时的流量击破。这种方案可能带来的最坏结果就是，消费这些消息会存在延迟。但这些订单都可以成功创建，真正的交易行为已经产生了。接下来要做的就是根据实际情况扩容或者缩容，尽快将积压的数据处理掉。

不过我们这个时候引入消息中间件，其实潜台词是它们的性能必须满足下面几个基本要求：高吞吐量、低延迟，还要具体消息堆积能力。

我们再看一下订单查询流程：

10. 用户在 App 端发起订单查询，App 会调用网关的订单查询接口，网关再将请求转发到内部的订单查询服务；
11. 订单查询服务不是在 MySQL 数据库，而是直接查询 Es 中的数据。

这里一个设计的亮点是，引入了数据同步组件 Canal，将 MySQL 数据库中的数据实时同步到了 Es。这样查询订单时只查 Es 就可以了，实现了订单写入与订单查询在**异构数据源的读写分离**。

21.2 消息中间件的技术选型

在这节课的最后，我们来看看如何选择消息中间件。

目前消息中间件领域主要的中间件包括 RocketMQ、Kafka 和 RabbitMQ，我们先来看一下这张功能对比图：

维度	对比项	Kafka	RocketMQ	RabbitMQ
功能维度	延迟消息	不支持	支持	支持
	优先级队列	不支持	不支持	支持
	事务消息	支持	支持	支持
	消息重试	不支持	支持	不支持
	消息堆积能力	强	强	弱（性能会受影响）
	消息回溯	支持	支持	不支持
	消息过滤	不支持	支持	不支持
	消息轨迹	不支持	支持	支持（需要插件）
	多语言	支持多语言客户端	支持多语言客户端	支持多语言客户端
	ACL	支持	支持	支持
性能维度	单机吞吐量	百万级	十万级	万级
	消息发送时延	ms级	us级	us级
	可用性	非常高（分布式架构）	非常高（分布式架构）	高（主从架构）
	水平伸缩能力	支持（伴随大量数据复制）	支持（轻量级）	支持
其他维度	技术栈	Java、Scala	Java	Elang

结合上面这张图，我们再对比分析一下。

首先，我认为功能级别不具备一票否决权。

例如，RabbitMQ 支持优先级队列，而 RocketMQ、Kafka 不支持，那么如果我们的项目中有优先级队列的使用诉求，我们就必须将 Kafka、RocketMQ 排除掉，选择使用 RabbitMQ 吗？我是不建议这样做的，任何涉及到功能的短板，都可以通过其他方式实现。

但我也并不是说功能特性就一点都不重要。这一点我在后面讨论 RocketMQ 与 Kafka 的选型时会再次谈到。

其次，我认为在选型时要特别注意中间件的性能和扩展性。

因为随着业务不断地发展，性能问题会越来越突出，而且性能问题都具有**隐蔽性**，一旦发生，破坏性大，影响程度深，让人防不胜防。

例如，RabbitMQ 的消息堆积能力不强，一旦消费端无法及时将消息处理掉，会极大影响消息服务器发送消息的性能。这一点是非常致命的，因为引入消息中间件的目的就是抵挡住洪峰流量，如果消息中间件因为积压问题影响了消息的发送，那是万万不可取的。

因此，从性能的角度来看，RocketMQ 和 Kafka 比 RabbitMQ 的表现更好。

另外一个重要的因素也不得不加以考虑，那就是中间件使用的编程语言。

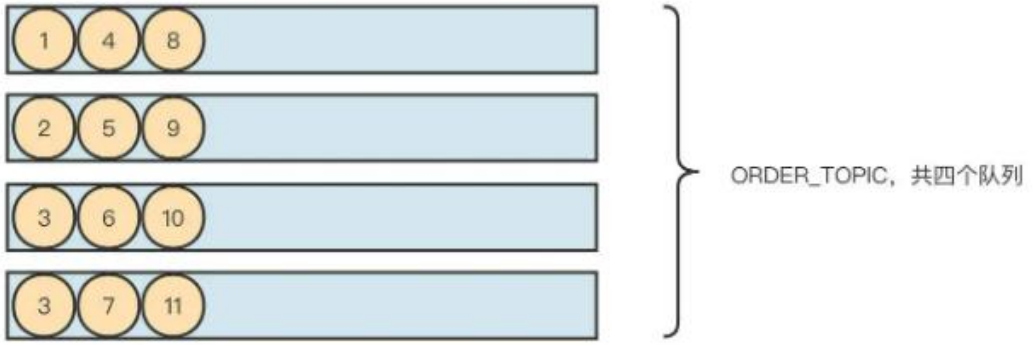
在使用中间件时一般都会遇到很多问题，一个非常行之有效的方法就是深入研究源码。这时候，如果中间件的编写语言和团队技术栈不匹配，将会极大地增加深入研究这款中间件的难度。如果团队对中间件的掌控能力很弱，自然很难保持中间件的稳定运行。

在进行具体的选型时，我们可以结合自己团队的实际情况。

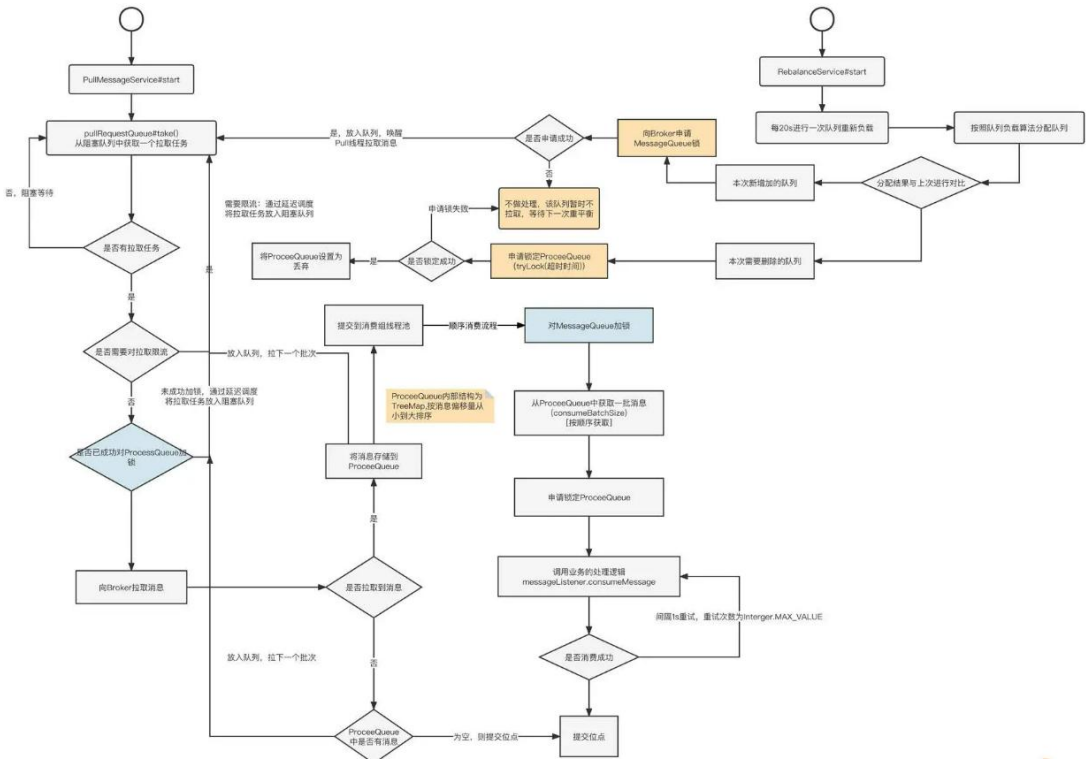
- 如果公司或团队的技术栈以 Golang 为主，建议选择 RabbitMQ，RabbitMQ 在性能上的缺陷可以通过搭建多套集群加以规避。
- 如果公司或团队的技术栈以 Java 为主，我建议使用 Kafka 或 RocketMQ。RocketMQ 和 Kafka 都是性能优秀的中间件，在这两者之间进行选择时可以更多地关注功能特性。RocketMQ 提供了消息重试、消息过滤、消息轨迹、消息检索等功能特性，特别是 RocketMQ 的消息检索功能，因此 RocketMQ 很适合核心业务场景。而 kafka 更加擅长于日志、大数据计算、流式计算等场景。

22. 如何提升 RocketMQ 顺序消费性能?

我们先来了解一下 RocketMQ 顺序消费的实现原理。RocketMQ 支持局部顺序消息消费，可以保证同一个消费队列上的消息顺序消费。例如，消息发送者向主题为 ORDER_TOPIC 的 4 个队列共发送 12 条消息， RocketMQ 可以保证 1、4、8 这三条按顺序消费，但无法保证消息 4 和消息 2 的先后顺序。



那 RocketMQ 是怎么做到分区顺序消费的呢？我们可以看一下它的工作机制：



顺序消费实现的核心要点可以细分为三个阶段。

第一阶段：消费队列负载。

RebalanceService 线程启动后，会以 20s 的频率计算每一个消费组的队列负载、当前消费者的消费队列集合（用 newAssignQueueSet 表），然后与上一次分配结果（用 oldAssignQueueSet 表示）进行对比。这时候会出现两种情况。

- 如果一个队列在 newAssignQueueSet 中，但并不在 oldAssignQueueSet 中，表示这是新分配的队列。这时候我们可以尝试向 **Broker 申请锁**：
  - 如果成功获取锁，则为该队列创建拉取任务并放入到 PullMessageService 的 pullRequestQueue 中，以此唤醒 Pull 线程，触发消息拉取流程；

- 如果未获取锁，说明该队列当前被其他消费者锁定，放弃本次拉取，等下次重平衡时再尝试申请锁。

这种情况下，消费者能够拉取消息的前提条件是，在 Broker 上加锁成功。

- 如果一个队列在 newAssignQueueSet 中不存在，但存在于 oldAssignQueueSet 中，表示该队列应该分配给其他消费者，需要将该队列丢弃。但在丢弃之前，要**尝试申请 ProceeQueue 的锁**：
  - 如果成功锁定 ProceeQueue，说明 ProceeQueue 中的消息已消费，可以将该 ProceeQueue 丢弃，并释放锁；
  - 如果未能成功锁定 ProceeQueue，说明该队列中的消息还在消费，暂时不丢弃 ProceeQueue，这时消费者并不会释放 Broker 中申请的锁，其他消费者也就暂时无法消费该队列中的消息。

这样，消费者在经历队列重平衡之后，就会创建拉取任务，并驱动 Pull 线程进入到消息拉取流程。

第二阶段：消息拉取。

PullMessageService 线程启动，从 pullRequestQueue 中获取拉取任务。如果该队列中没有待拉取任务，则 Pull 线程会阻塞，等待 RebalanceImpl 线程创建拉取任务，并向 Broker 发起消息拉取请求：

- 如果未拉取到消息。可能是 Tag 过滤的原因，被过滤的消息其实也可以算成被成功消费了。所以如果此时处理队列中没有待消费的消息，就提交位点（当前已拉取到最大位点 +1），同时再将拉取请求放到待拉取任务的末尾，反复拉取，实现 Push 模式。
- 如果拉取到一批消息。首先要将拉取到的消息放入 ProceeQueue(TreeMap)，同时将消息提交到消费线程池，进入消息消费流程。再将拉取请求放到待拉取任务的末尾，反复拉取，实现 Push 模式。

第三阶段：顺序消费。

RocketMQ 一次只会拉取一个队列中的消息，然后将其提交到线程池。为了保证顺序消费，RocketMQ 在消费过程中有下面几个关键点：

- 申请 MessageQueue 锁，确保在同一时间，一个队列中只有一个线程能处理队列中的消息，未获取锁的线程阻塞等待。

- 获取 MessageQueue 锁后，从处理队列中依次拉取一批消息（消息偏移量从小到大），保证消费时严格遵循消息存储顺序。
- 申请 MessageQueue 对应的 ProcessQueue，申请成功后调用业务监听器，执行相应的业务逻辑。

经过上面三个关键步骤，RocketMQ 就可以实现队列（Kafka 中称为分区）级别的顺序消费了。

22.1 RocketMQ 顺序消费设计缺陷

回顾上面 RocketMQ 实现顺序消费的核心关键词，我们发现其实就是加锁、加锁、加锁。没错，为了实现顺序消费，RocketMQ 需要进行三次加锁：

- 进行队列负载平衡后，对新分配的队列，并不能立即进行消息拉取，必须先在 Broker 端获取队列的锁；
- 消费端在正式消费数据之前，需要锁定 MessageQueue 和 ProceeQueue。

上述三把锁的控制，让并发度受到了队列数量的限制。在互联网、高并发编程领域，通常是“**谈锁色变**”，锁几乎成为了性能低下的代名词。试图减少锁的使用、缩小锁的范围几乎是性能优化的主要手段。

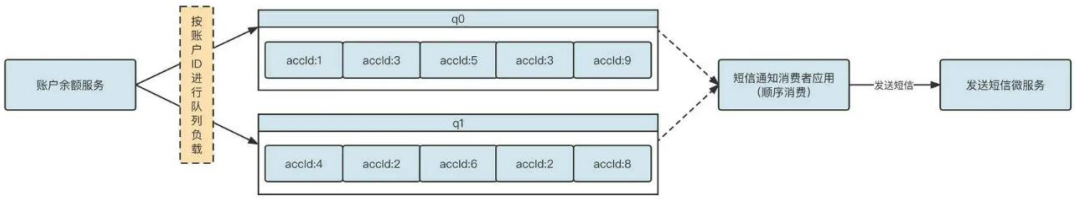
22.2 RocketMQ 顺序消费优化方案

而 RocketMQ 为了实现顺序消费引入了三把锁，极大地降低了并发性能。那如何对其进行优化呢？

22.2.1 破局思路：关联顺序性

我们不妨来看一个金融行业的真实业务场景：**银行账户余额变更短信通知。**

当用户的账户余额发生变更时，金融机构需要发送一条短信，告知用户余额变更情况。为了实现余额变更和发送短信的解耦，架构设计时通常会引入消息中间件，它的基本实现思路你可以参考这张图：



基于 RocketMQ 的顺序消费机制，我们可以实现基于队列的顺序消费，在消息发送时只需要确保同一个账号的多条消息（多次余额变更通知）发送到同一个队列，消费端使用顺序消费，就可以保证同一个账号的多次余额变更短信不会顺序错乱。

q0 队列中依次发送了账号 ID 为 1、3、5、3、9 的 5 条消息，这些消息将严格按照顺序执行。但是，我们为账号 1 和账号 3 发送余额变更短信，时间顺序必须和实际的时间顺序保持一致吗?

答案是显而易见的，没有这个必要。

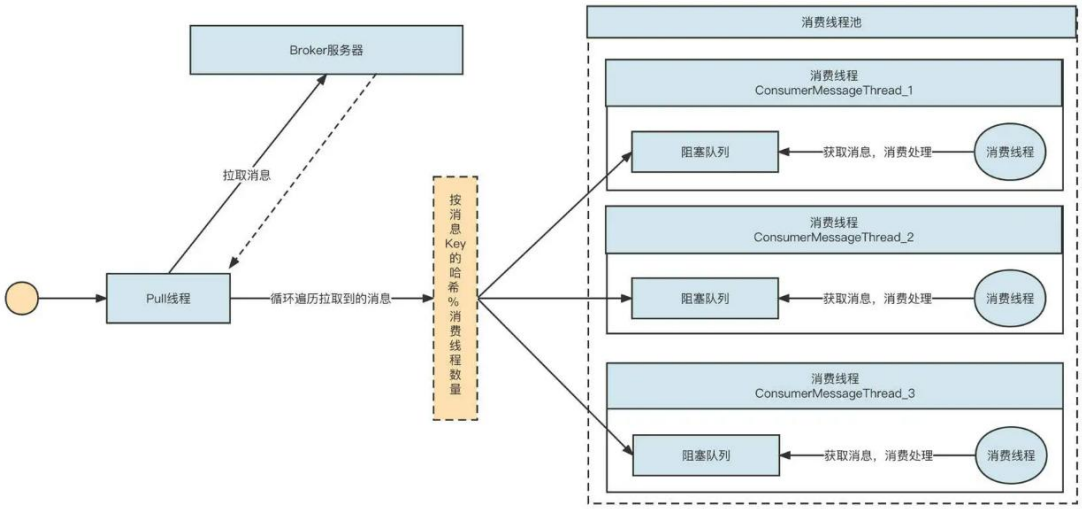
例如，用户 1 在 10:00:01 发生了一笔电商订单扣款，而用户 2 在 10:00:02 同样发生了一笔电商订单扣款，那银行先发短信告知用户 2 余额发生变更，然后再通知用户 1，并没有破坏业务规则。

不过要注意的是，同一个用户的两次余额变更，必须按照发生顺序来通知，这就是所谓的**关联顺序性。**

显然，RocketMQ 顺序消费模型并没有做到关联顺序性。针对这个问题，我们可以看到一条清晰的优化路线：**并发执行同一个队列中不同账号的消息，串行执行同一个队列中相同账号的消息。**

22.2.2 RocketMQ 顺序模型优化

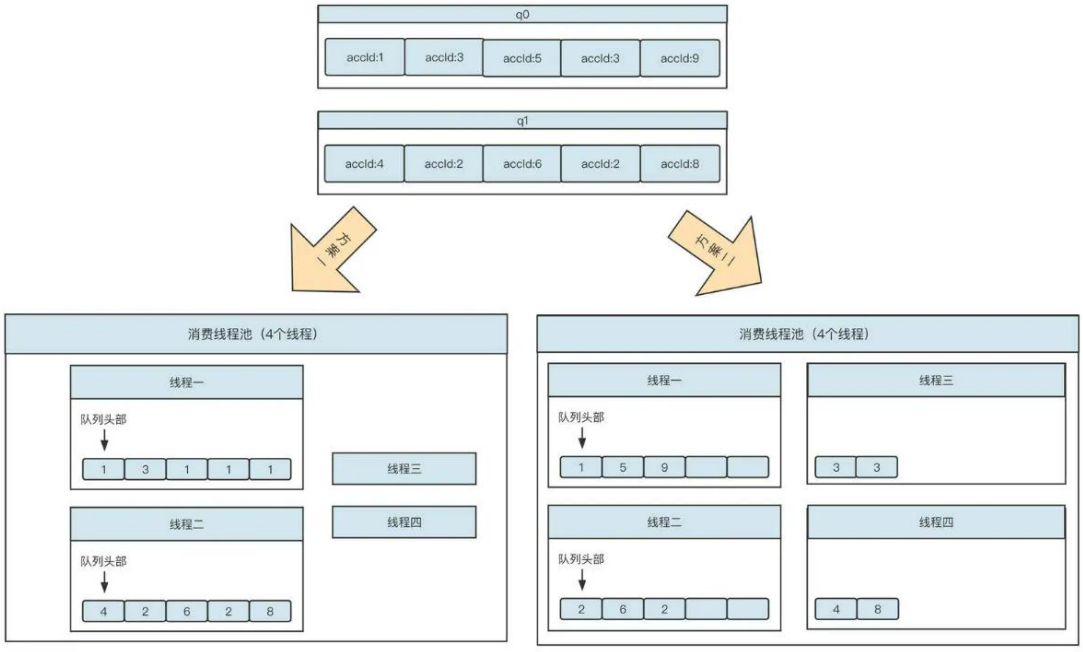
基于关联顺序性的整体指导思路，我设计出了一种**顺序消费改进模型**:



详细说明一下。

1. 消息拉取线程（PullMeessageService）从 Broker 端拉取一批消息。
2. 遍历消息，获取消息的 Key（消息发送者在发送消息时根据 Key 选择队列，同一个 Key 的消息进入同一个队列）的 HashCode 和线程数量，将消息投递到对应的线程。
3. 消息进入到某一个消费线程中，排队单线程执行消费，遵循严格的消费顺序。

为了让你更加直观地体会两种设计的优劣，我们来看一下两种模式针对一批消息的消费行为对比：



在这里，方案一是 RocketMQ 内置的顺序消费模型。实际执行过程中，线程三、线程四也会处理消息，但内部线程在处理消息之前必须获取队列锁，所以说同一时刻一个队列只会有一个线程真正存在消费动作。

方案二是优化后的顺序消费模型，它和方案一相比最大的优势是并发度更高。

方案一的并发度取决于消费者分配的队列数，单个消费者的消费并发度并不会随着线程数的增加而升高，而方案二的并发度与消息队列数无关，消费者线程池的线程数量越高，并发度也就越高。

22.3 代码实现

在实际生产过程中，再好看的架构方案如果不能以较为简单的方式落地，那就等于零，相当于什么都没干。

所以我们就尝试落地这个方案。接下来我们基于 RocketMQ4.6 版本的 DefaultLitePullConsumer 类，引入新的线程模型，实现新的 Push 模式。

为了方便你阅读代码，我们先详细看看各个类的职责（类图）与运转主流程（时序图）。

22.3.1 类图设计

1. DefaultMQLitePushConsumer

基于 DefaultMQLitePullCOnsumer 实现的 Push 模式，它的内部对线程模型进行了优化，对标 DefaultMQPushConsumer。

2. ConsumeMessageQueueService

消息消费队列消费服务类接口，只定义了 void execute(List< MessageExt > msg) 方法，是基于 MessageQueue 消费的抽象。

3. AbstractConsumeMessageService

消息消费队列服务抽象类，定义一个抽象方法 selectTaskQueue 来进行**消息的路由策略**，同时实现最小位点机制，拥有两个实现类：

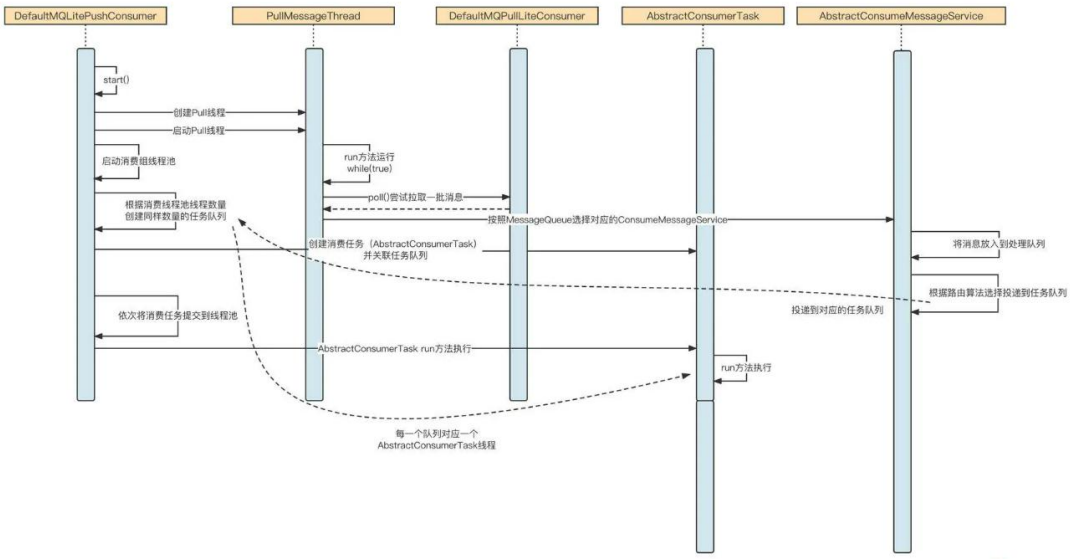
- 顺序消费模型（ConsumeMessageQueueOrderlyService），消息路由时按照 Key 的哈希与线程数取模；
- 并发消费模型（ConsumerMessageQueueConcurrentlyService），消息路由时使用默认的轮循机制选择线程。

4. AbstractConsumerTask 定义消息消费的流程，同样有两个实现类，分别是并发消费模型（ConcurrentlyConsumerTask）和顺序消费模型（OrderlyConsumerTask）。

定义消息消费的流程，同样有两个实现类，分别是并发消费模型（ConcurrentlyConsumerTask）和顺序消费模型（OrderlyConsumerTask）。

22.3.2 时序图

类图只能简单介绍各个类的职责，接下来，我们用时序图勾画出核心的设计要点：



这里，我主要解读一下与顺序消费优化模型相关的核心流程：

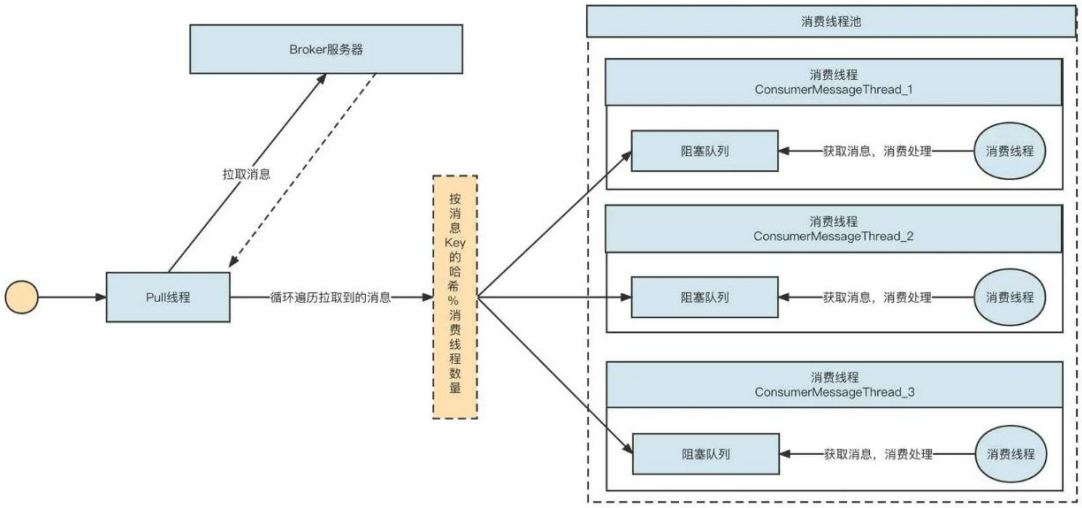
6. 调用 DefaultMQLitePushConsumer 的 start 方法后，会依次启动 Pull 线程（消息拉取线程）、消费组线程池、消息处理队列与消费处理任务。这里的重点是，一个 AbstractConsumerTask 代表一个消费线程，一个 AbstractConsumerTask 关联一个任务队列，消息在按照 Key 路由后会放入指定的任务队列，从而被指定线程处理。
7. Pull 线程每拉取一批消息，就按照 MessageQueue 提交到对应的 AbstractConsumeMessageService。
8. AbstractConsumeMessageService 会根据顺序消费、并发消费模式选择不同的路由算法。其中，顺序消费模型会将消息 Key 的哈希值与任务队列的总个数取模，将消息放入到对应的任务队列中。
9. 每一个任务队列对应一个消费线程，执行 AbstractConsumerTask 的 run 方法，将从对应的任务队列中按消息的到达顺序执行业务消费逻辑。
10. AbstractConsumerTask 每消费一条或一批消息，都会提交消费位点，提交处理队列中最小的位点。

22.3.3 关键代码解读

类图与时序图已经强调了顺序消费模型的几个关键点，接下来我们结合代码看看具体的实现技巧。

22.3.3.1 创建消费线程池

创建消费线程池部分是我们这个方案的点睛之笔，它对应的是第三小节顺序消费改进模型图中用虚线勾画出的线程池。为了方便你回顾，我把这个图粘贴在下面。



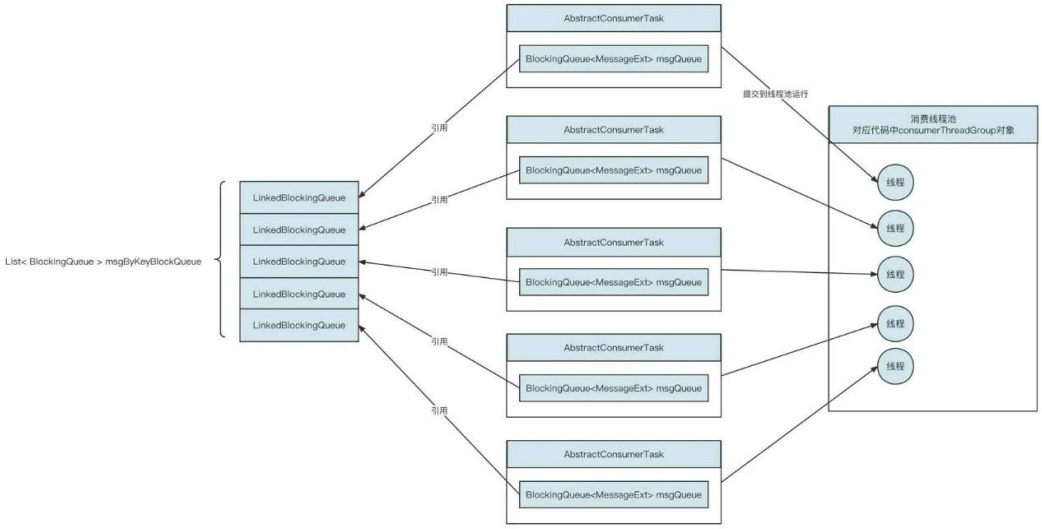
代码实现如下所示：

```
// 启动消费组线程池 privatevoidstartConsumerThreads() {
    //设置线程的名称 StringthreadPrefix= isOrderConsumerModel ? "OrderlyConsumerThreadMessage_" : "ConcurrentlyConsumerThreadMessage_";
    AtomicIntegerthreadNumIndex=newAtomicInteger(0);
    //创建消费线程池
    consumerThreadGroup = newThreadPoolExecutor(consumerThreadCount, consumerThreadCount, 0, TimeUnit.MILLISECONDS, newLinkedBlockingQueue<>(), r -> {
        Threadt=newThread(r);
        t.setName(threadPrefix + threadNumIndex.incrementAndGet() );
        return t;
    });
    //创建任务阻塞线程数组
    msgByKeyBlockQueue = newArrayList(consumerThreadCount);
    consumerRunningTasks = newArrayList<>(consumerThreadCount);
    for(inti=0; i < consumerThreadCount; i ++ ) {
        msgByKeyBlockQueue.add(newLinkedBlockingQueue());
        AbstractConsumerTasktask=null;
        //根据是否是顺序消费，创建对应的消费实现类 if(isOrderConsumerModel) {
            task = newOrderlyConsumerTask(this, msgByKeyBlockQueue.get(i), this.messageListener);
        } else {
            task = newConcurrentlyConsumerTask(this, msgByKeyBlockQueue.get(i), this.messageListener);
        }
        consumerRunningTasks.add(task);
        //启动消费线程
        consumerThreadGroup.submit(task);
    }
}
```

这段代码有三个实现要点。

- 第 7 行：创建一个指定线程数量的线程池，消费线程数可以由 consumerThreadCont 指定。
- 第 12 行：创建一个 ArrayList < LinkedBlockingQueue > taskQueues 的任务队列集合，其中 taskQueues 中包含 consumerThreadCont 个队列。
- 第 13 行：创建 consumerThreadCont 个 AbstractConsumerTask 任务，每一个 task 关联一个 LinkedBlockingQueue 任务队列，然后将 AbstractConsumerTask 提交到线程池中执行。

以 5 个消费线程池为例，从运行视角来看，它对应的效果如下：



22.3.3.2 消费线程内部执行流程

将任务提交到提交到线程池后，异步运行任务，具体代码由 AbstractConsumerTask 的 run 方法来实现，其 run 方法定义如下：

```
1  publicvoidrun() {
2      try {
3          while (isRunning) {
4              try {
5                  //判断是否是批量消费
6                  List<MessageExt> msgs =
   newArrayList<>(this.consumer.getConsumeBatchSize());
7                  //这里是批消费的核心，一次从队列中提前多条数据，一次提交到用户消费者线
   程 while(msgQueue.drainTo(msgs, this.consumer.getConsumeBatchSize()) <= 0 ) {
8                      Thread.sleep(20);
9                  }
```

```
                    //执行具体到消费代码，就是调用用户定义的消费逻辑，位点提交
                    doTask(msgs);
                } catch (InterruptedException e) {
                    LOGGER.info(Thread.currentThread().getName() + "is Interrupt");
                    break;
                } catch (Throwable e) {
                    LOGGER.error("consume message error", e);
                }
            }
        } catch (Throwable e) {
            LOGGER.error("consume message error", e);
        }
}
```

在这段代码中，消费线程从阻塞队列中抽取数据进行消费。顺序消费、并发消费模型具体的重试策略不一样，根据对应的子类实现即可。

22.3.3.3 Pull 线程

这段代码对标的是改进方案中的 Pull 线程，它负责拉取消息，并提交到消费线程。Pull 线程的核心代码如下：

```
privatevoidstartPullThread() {
        {
            //设置线程的名称，方便我们在分析线程栈中准确找到 PULL 线程
StringthreadName="Lite-Push-Pull-Service-" + this.consumer + "-" +
LocalDateTime.now();
            ThreadlitePushPullService=newThread(() -> {
                try {
                    while (isRunning) {
                        //待超时时间的消息拉取
                        List<MessageExt> records =
consumer.poll(consumerPollTimeoutMs);
                        //将拉取到的消息提交到线程池，从而触发消费
                        submitRecords(records);
                        //为需要限流的队列开启限流
                        consumerLimitController.pause();
                        //为需要解除限流的队列解除限流
                        consumerLimitController.resume();
                    }
                } catch (Throwable ex) {
                    LOGGER.error("consume poll error", ex);
                } finally {
                    stopPullThread();
```

```
                }
            }, threadName);
            litePushPullService.start();
            LOGGER.info("Lite Push Consumer started at {}, consumer group name:{}",
System.currentTimeMillis(), this.consumerGroup);
        }
    }

    privatevoidsubmitRecords(List<MessageExt> records) {
        if (records == null || records.isEmpty()) {
            return;
        }
        MessageExtfirstMsg= records.get(0);
        MessageQueuemessageQueue=newMessageQueue(firstMsg.getTopic(),
firstMsg.getBrokerName(), firstMsg.getQueueId());
        // 根据队列获取队列级别消费服务类
ConsumeMessageQueueServicetempConsumeMessageService=
ConsumeMessageQueueServiceFactory.getOrCreateConsumeMessageService(this,
messageQueue, isOrderConsumerModel, lastAssignSet);
        // 提交具体的线程池
        tempConsumeMessageService.execute(records);
    }
```

Pull 线程做的事情比较简单，就是反复拉取消息，然后按照 MessageQueue 提交到对应的 ConsumeMessageQueueService 去处理，进入到消息转发流程中。

22.3.3.4 消息路由机制

此外，优化后的线程模型还有一个重点，那就是消息的派发，它的实现过程如下：

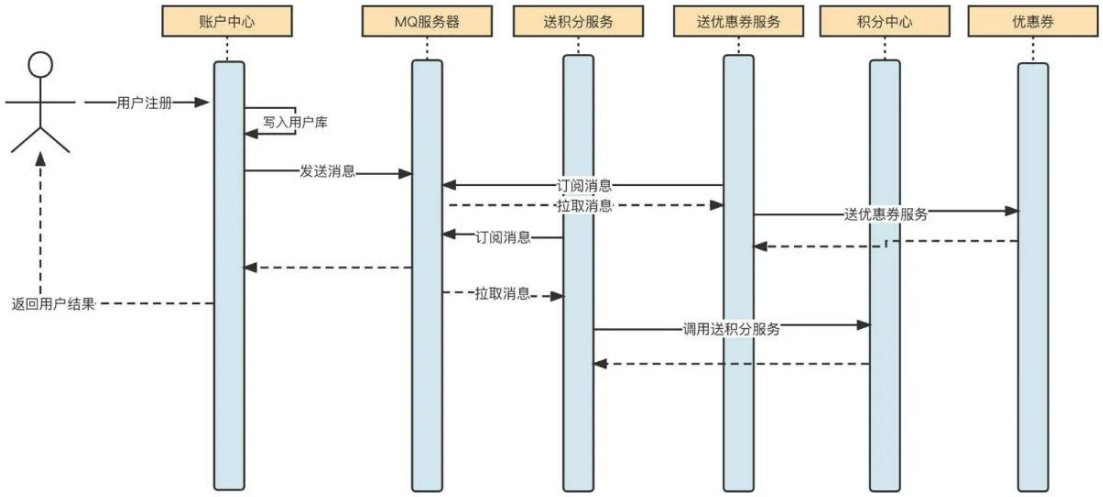
```
publicvoidexecute(List<MessageExt> consumerRecords) {
    if (consumerRecords == null || consumerRecords.isEmpty()) {
        return;
    }

    // 将消息放入到待消费队列中，这里实际是一个 TreeMap 结构，用于进行最小位点计算
    putMessage(consumerRecords);

    if (isNeedPause()) {
        consumer.getConsumerLimitController().addPausePartition(messageQueue);
    }

    for (MessageExt msg : consumerRecords) {
        inttaskIndex= selectTaskQueue(msg, consumer.getTaskQueueSize());
        try {
```

```
                consumer.submitMessage(taskIndex, msg);
            } catch (Throwable e) {
                // ignore e
                e.printStackTrace();
            }
        }

    }

    publicclassConsumeMessageQueueOrderlyServiceextendsAbstractConsumeMessageServ
    ice{
        privatefinalStringNO_KEY_HASH="__nokey";
        publicConsumeMessageQueueOrderlyService(DefaultMQLitePushConsumer
    consumer, MessageQueue messageQueue) {
            super(consumer, messageQueue);
        }

        @OverrideprotectedintselectTaskQueue(MessageExt msg, int taskQueueTotal) {
            Stringkeys= msg.getKeys();
            if(StringUtils.isEmpty(keys)) {
                keys = NO_KEY_HASH;
            }
            return    Math.abs(    keys.hashCode()     ) %    taskQueueTotal;
        }
    }
```

这里，顺序消费模型按照消息的 Key 选择不同的队列，而每一个队列对应一个线程，即实现了按照 Key 来选择线程，消费并发度与队列个数无关。

23. 使用分布式调度框架该考虑哪些问题?

下面是一个用户注册优惠活动的场景，为了实现用户注册主流程与活动的解耦合，我们引入了消息中间件，它的时序图如下所示：



这里的核心指导思想是让账户中心完成用户的注册逻辑，将用户写入到账户中心数据库，然后发送一条消息到 MQ 服务器，再给返回用户“注册成功”。之后，引入两个消费者，分别对消息进行对应的处理，异步赠送优惠券或者积分。

这个方法的架构思路非常不错，但是我们还不得不思考一个问题：如何保证写入数据库与消息发送这两个步骤的一致性呢？我们希望这两个步骤要么一起成功，要么一起失败，绝不能出现用户数据成功写入数据库，但消息发送失败的情况。因为这样用户无法收到优惠券，容易产生一系列的投诉和纠纷。

这其实是一个分布式事务的问题，也就是要保证数据库写入和消息发送这两个分布式操作的一致性。

一种比较常见的解决方案就是：“本地消息表 + 定时任务”。

具体而言，我们首先需要在数据库创建一张本地消息表，表的结构大致如下：

创建了消息发送本地记录表之后，用户注册的流程将变成：

1. 开启数据库本地事务；

2. insert into user 表（用户注册表）；
3. insert into msg_send_record，并且存储账户的唯一编号、状态，其中状态的初始值为 0；
4. 提交本地事务。

这样做的目的是，保证 user 表和 msg_send_record 的事务一致性，如果用户信息成功存入 user 表，msg_send_record 表中必然存在一条对应的记录，后续我们只需要根据 msg_send_record 表中的记录发送一条对应的 MQ 消息即可。

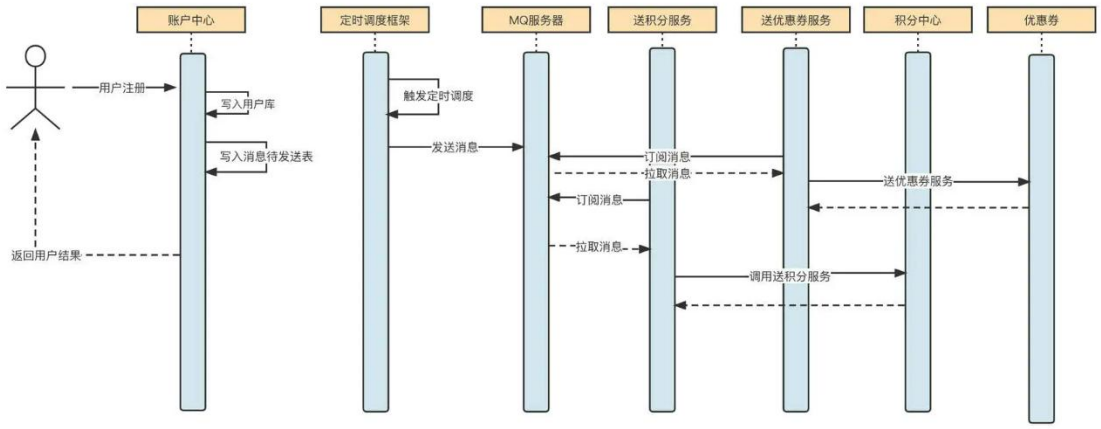
当然，为了保证 msg_send_record 的写入不至于带来太大的性能损耗，通常我们会采取下面几个措施。

- 如果在分库分表环境中，msg_send_record 采取的分库策略与 user 表一致，我们要保证这个过程是一个本地事务，不至于出现跨库 Join 的情况。
- 为 account_no、创建时间这两个字段添加索引。
- 定时清除 msg_send_record 表中的数据，这个表不需要保留太长时间，尽量控制单表数据量。

数据成功写入消息待发送表后，接下来我们需要引入定时调度程序，定时扫描 msg_send_record 中的记录，将消息发送到 MQ 中。

定时调度程序的数据处理策略主要有三步。首先，按照分页机制从数据库中拉取一批数据；然后，根据用户账户查询用户表，构建消息体（用户账户、用户注册时间）；最后，将消息发送到消息服务器（这里必须提供重试机制）。

引入定时调度程序后，用户注册送积分的时序图变成了下面这样：



在计划执行这个方案时，还有一个非常重要的事情，就是要明确定时调度任务的执行频率。

因为定时调度任务的调度频率直接决定了消息发送的实时性，随着需要调度的任务越来越多，大部分定时调度框架对秒级别的定时调度都不太友好。这时的调度通常都是分钟级的，但分钟级的调度会给任务带来较大的延迟，这是大部分业务无法容忍的，怎么办呢？

ElasticJob 可以通过支持流式任务解决这个问题。具体的思路是：将任务配置为按照分钟级进行调度，例如每分钟执行一次调度。每次调度按照分页去查找数据，处理完一批数据，再查询下一批，如果查到待处理数据，就继续处理数据，直到没有待处理数据时，才结束本次业务处理。如果本次处理时间超过了一个调度周期，那么利用 ElasticJob 的任务错过补偿执行机制会再触发一次调度。

在业务高峰期，这种方式基本上提供了准实时的处理效果。只有在业务量较少时，如果处理完一批数据后没有其他待处理的数据，这时新到的数据才会延迟 1 分钟执行。

综合来看，通过支持流式任务，我们可以极大地提高数据的处理时效。

消息领域定时调度框架的设计方案就介绍到这里了，我想你也许会问，RocketMQ 不是已经提供了事务消息机制吗？这里能不能直接使用 RocketMQ 的事务消息呢？

当然可以。但是很多公司的内部都采用了多种类型的消息中间件，有的中间件并不支持事务消息这个功能。考虑到架构设计方案的普适性，我们通常不会依赖单个中间件的特性。

了解了设计方案，我们就可以实现消息发送和数据库操作的分布式事务一致性了。光说不练假把式，接下来，我们就尝试落地这个方案。我会给出一些关键代码，方便你在生产环境中落地实战。

我会基于 ElasticJob 框架简单梳理一下关键代码。通过 ElasticJob 实现一个定时调度任务通常包含两个重要步骤。

首先，我们要实现 ElasticJob 的流式任务接口 DataflowJob，这个接口主要完成定时调度任务的具体业务逻辑：

```
1 publicclassUserSendMqJobimplementsDataflowJob<UserMsgSendRecord> {
2     privatestaticintPAGE_SIZE=100;
3     privatestaticStringUSER_SEND_MQ_TOPIC="user_register_topic";
4     private IUserMsgSendRecordDao userMsgSendRecordDao;
```

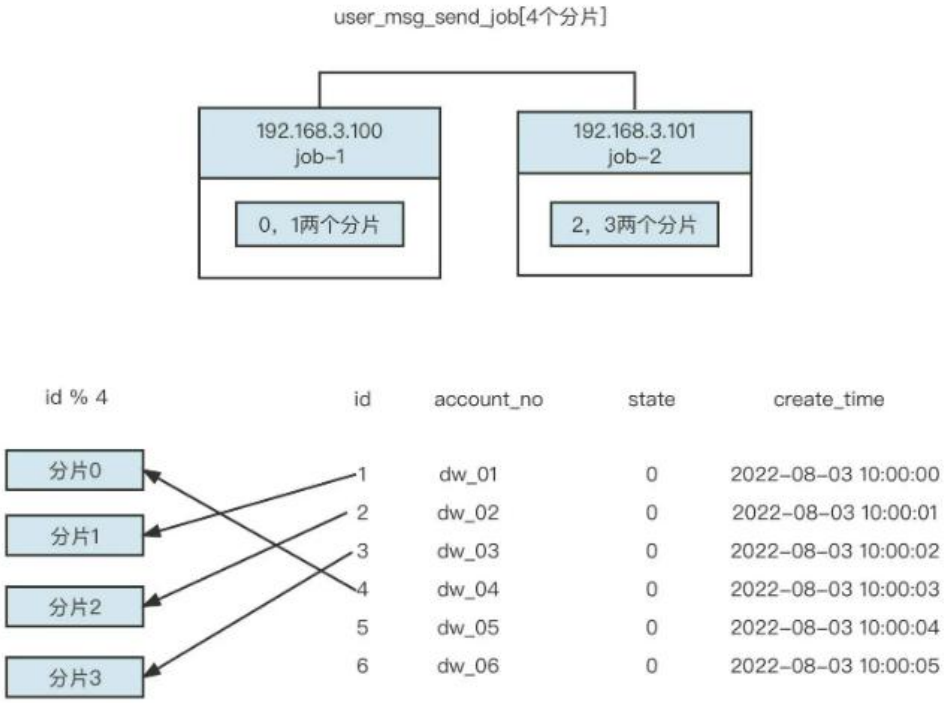
```
    private DefaultMQProducer defaultMQProducer;
    @Overridepublic List<UserMsgSendRecord> fetchData(ShardingContext shardingContext) {
        // 分片总数 intshardingTotalCount= shardingContext.getShardingTotalCount();
        //当前任务所处的分片序号 intshardingItem= shardingContext.getShardingItem();
        intmod= shardingItem % shardingTotalCount;
        // 每次从数据库中取出一批数据 return userMsgSendRecordDao.selectWaitSendRecordPage(mod, 0, PAGE_SIZE);
    }

    @OverridepublicvoidprocessData(ShardingContext shardingContext, List<UserMsgSendRecord> datas) {
        if(datas == null || datas.isEmpty()) {
            return;
        }
        for(UserMsgSendRecord record : datas) {
            Stringbody= toJsonMsgBody(record);
            Stringkey= record.getAccountNo();
            try {
                SendResultresult= defaultMQProducer.send(newMessage(USER_SEND_MQ_TOPIC, null, key, body.getBytes(StandardCharsets.UTF_8)));
                record.setMsgId( record.getMsgId());
                record.setSendStatus(1);
                userMsgSendRecordDao.update(record);
            } catch (Throwable e) {
                e.printStackTrace();
                // 等待下一次调度
            }
        }
    }

    private String toJsonMsgBody(UserMsgSendRecord record) {
        // 其实就是构建 JSON 消息内容，例如包含 注册用户编号
        Map<String, Object> body = newHashMap<>();
        body.put("accountNo", record.getAccountNo());
        body.put("registerTime", record.getCreateTime());
        return JSON.toJSONString(body);
    }
}
```

DataflowJob 接口主要定义 fetchData 和 processData 接口，我们分别解读一下这两个接口的实现要点。

先说一下 fetchData。

它主要用于拉取待处理数据，ElasticJob 每触发一次任务调度，都会首先调用 fetchData 方法，尝试获取数据。如果该方法返回数据，ElasticJob 将调用 processData 来完成具体的业务逻辑。处理完一批数据后，还会循环调用 fetchData，看有没有待处理的数据。如果有，则继续调用 processData，直到查询不到待处理数据时，结束本次业务调度。

这里有一个非常关键的点：我们可以通过 ShardingContext 来获取任务的分片信息。其中，shardingTotalCount 是本次任务的总分片数量，shardingItem 是当前任务所处理数据的分片序号。通常我们可以用这两个数和 id 取模，实现数据分片。你可以看看下面这张示意图：



processData 方法，顾名思义，就是用来处理业务逻辑的方法。通过 fetchData 方法查询到的数据会传入 processData 方法中执行，我们这个实例主要是根据待发送记录组装 MQ 消息，然后将消息发送到 MQ 服务器，更新待发送记录，最后将状态从待发送变更为已发送的过程。

不过在实际执行过程中，通常还会遇到另外一个问题。假设我们的业务逻辑要根据不同的类型发送不到不同的 MQ 集群中，部分主题可能一直发送失败，最后影响到其他主题的正常发送。具体的示意图如下：

id	account_no	state	create_time	topic_name	cluster_name
1	dw_01	0	2022-08-03 10:00:00	topic_a	cluster_a
2	dw_02	0	2022-08-03 10:00:01	topic_a	cluster_a
3	dw_03	0	2022-08-03 10:00:02	topic_a	cluster_a
4	dw_04	0	2022-08-03 10:00:03	topic_a	cluster_a
5	dw_05	0	2022-08-03 10:00:04	topic_b	cluster_b
6	dw_06	0	2022-08-03 10:00:05	topic_b	cluster_b

如果 fetchData 在获取数据时,每一次只拉取 3 条消息,那么它会一次取出 id 为 1，2，3 的三条消息，然后将这些消息发送到 cluster_a 集群的 topic_a 主题。如果某一时刻集群 cluster_a 发生故障，一段时间内无法发送消息，数据仍然被传入 proccessData 方法，就会发现 fetchData 每次拉取出的 id 都是 1，2，3。因为这些消息在 proccessData 中没有处理成功，state 的状态不会更新，需要发往集群 b 的消息也无法正常发送，这会导致严重的业务故障。

那我们如何解决这个问题呢?

第一步，我们要在消息待发送表（msg_send_record）中增加两个字段，一个是当前重试次数（retry_count），另外一个是下一次调度的最小时间（next_select_time）。增加了这两个字段的表数据是下面这样:

id	account_no	state	create_time	topic_name	cluster_name	retry_count	next_select_time
1	dw_01	0	2022-08-03 10:00:00	topic_a	cluster_a	1	2022-08-03 11:01:00
2	dw_02	0	2022-08-03 10:00:01	topic_a	cluster_a	1	2022-08-03 11:01:00
3	dw_03	0	2022-08-03 10:00:02	topic_a	cluster_a	1	2022-08-03 11:01:00
4	dw_04	0	2022-08-03 10:00:03	topic_a	cluster_a	0	
5	dw_05	0	2022-08-03 10:00:04	topic_b	cluster_b	0	
6	dw_06	0	2022-08-03 10:00:05	topic_b	cluster_b		

在处理数据时，如果第一次处理数据失败了，我们需要将重试次数加一，并设置下一次调度的最小时间。例如，用当前时间加一分钟，意味着一分钟内流式处理任务将不再拉取该数据，这就给了其他数据执行机会。

第二步，通过配置文件或者其他整合方式声明一个任务，我们这个实例是用 Spirng 方式整合了 ElasticJob，所以我们需要在 XML 文件中配置任务，具体的配置代码如下:

```
1  <job:dataflowid="UserSendMqJob"class="net.codingw.mq.task.UserSendMqJob"registry
   -center-ref="zkRegistryCenter"cron="0 0/2 * *
   * ?"sharding-total-count="4"sharding-item-parameters="0=0,1=1,2=2,3=3"failover="true
   "streaming-process="true">
```

我们简要说明一下这些配置参数的含义。

- id：任务 id，它是全局唯一的。
- class：调度任务逻辑具体实现类。
- registry-center-ref：ElasticJob 调度器依赖的 ZooKeeper Bean。
- cron：定时调度 cron 表达式。
- sharding-total-count：总分片个数。
- sharding-item-parameters：分片参数，用于定义各个分片的参数。在进入到 fetchData 方法时，可以原封不动地获取该值，方便地实现一些定制化数据切分策略。
- failover：是否支持故障转移。设置为“true”表示支持，设置为“false”表示禁用故障转移机制。
- streaming-process：是否启用流式任务，true 表示启用流式任务。#

24. 在做同城多活方案中如何实现机房之间的数据同步？

在业务初期，考虑到投入成本，很多公司通常只用一个机房提供服务。但随着业务发展，流量不断增加，我们对服务的响应速度和可用性有了更高的要求，这时候我们就要开始考虑将服务分布在不同的地区来提供更好的服务，这是互联网公司在流量增长阶段的必经之路。

之前我所在的公司，流量连续三年不断增长。一次，机房对外网络突然断开，线上服务全部离线，网络供应商失联。因为没有备用机房，我们经过三天紧急协调，拉起新的线路才恢复了服务。这次事故影响很大，公司损失达千万元。

经过这次惨痛的教训，我们将服务迁移到了大机房，并决定在同城建设双机房提高可用性。这样当一个机房出现问题无法访问时，用户端可以通过 HttpDNS 接口快速切换到无故障机房。

为了保证在一个机房损坏的情况下，另外一个机房能直接接手流量，这两个机房的设备必须是 1:1 采购。但让其中一个机房长时间冷备不工作过于浪费，因此我们期望两个机房能同时对外提供服务，也就是实现同城双机房双活。

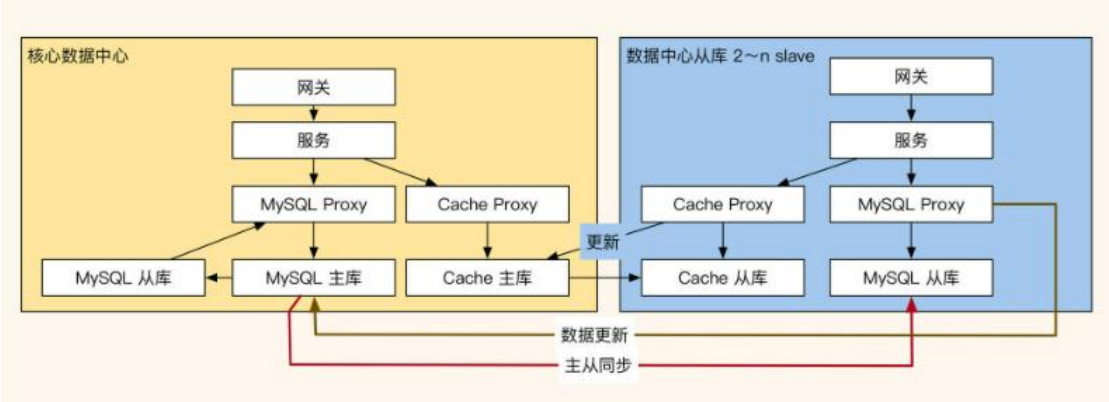
对此，我们碰到的一个关键问题就是，**如何实现同城双活的机房数据库同步？**

24.1 核心数据中心设计

因为数据库的主从架构，全网必须只能有一个主库，所以我们只能有一个机房存放更新数据的主库，再由这个机房同步给其他备份机房。虽然机房之间有专线连接，但并不能保证网络完全稳定。如果网络出现故障，我们要想办法确保机房之间能在网络修复后快速恢复数据同步。

有人可能会说，直接采用分布式数据库不就得了。要知道改变现有服务体系，投入到分布式数据库的大流中需要相当长的时间，成本也非常高昂，对大部分公司来说是不切实际的。所以我们要看看怎么对现有系统进行改造，实现同城双活的机房数据库同步，这也是我们这节课的目标。

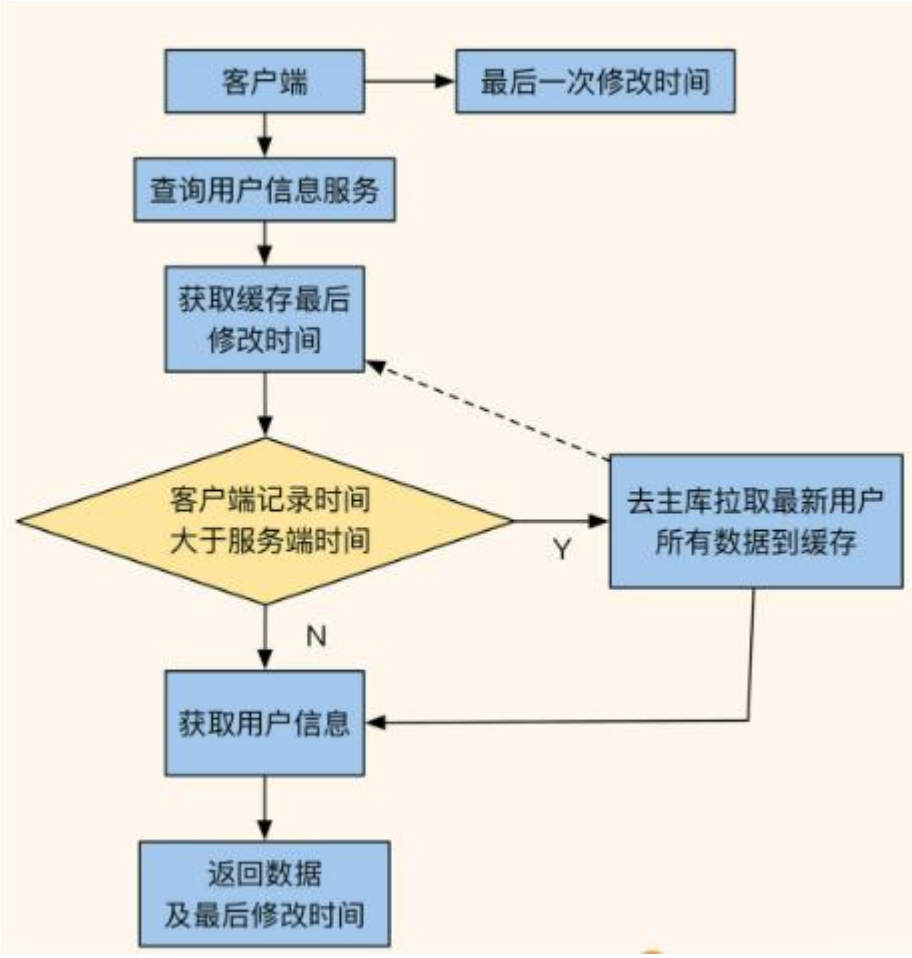
核心数据库中心方案是常见的实现方式，这种方案只适合相距不超过 50 公里的机房。



在这个方案中，数据库主库集中在一个机房，其他机房的数据库都是从库。当有数据修改请求时，核心机房的主库先完成修改，然后通过**数据库主从同步**把修改后的数据传给备份机房的从库。由于用户平时访问的信息都是从缓存中获取的，为了降低主从延迟，备份机房会把修改后的数据先更新到本地缓存。

与此同时，客户端会在本地记录下数据修改的最后时间戳（如果没有就取当前时间）。当客户端请求服务端时，服务端会自动对比缓存中对应数据的更新时间，是否小于客户端本地记录的修改时间。

如果缓存更新时间小于客户端内的修改时间，服务端会触发同步指令尝试在从库中查找最新数据；如果没有找到，就把从主库获取的最新数据放到被访问机房的缓存中。这种方式可以避免机房之间用户数据更新不及时的问题。



除此之外，客户端还会通过请求调度接口，让一个用户在短期内只访问一个机房，防止用户在多机房间来回切换的过程中，数据在两个机房同时修改引发更新合并冲突。

总体来看，这是一个相对简单的设计，但缺点也很多。比如如果核心机房离线，其他机房就无法更新，故障期间需要人工切换各个 proxy 内的主从库配置才能恢复服务，并且在故障过后还需要人工介入恢复主从同步。

此外，因为主从同步延迟较大，业务中刚更新的数据要延迟一段时间，才能在备用机房查到，这会导致我们业务需要人工兼顾这种情况，整体实现十分不便。

这里我给你一个常见的网络延迟参考：

- 同机房服务器：0.1 ms
- 同城服务器（100 公里以内）：1ms（10 倍 同机房）
- 北京到上海：38ms（380 倍 同机房）
- 北京到广州：53ms（530 倍 同机房）

注意，上面只是一次 RTT 请求，而机房间的同步是多次顺序地叠加请求。如果要大规模更新数据，主从库的同步延迟还会加大，所以这种双活机房的数据量不能太大，并且业务不能频繁更新数据。

此外还要注意，如果服务有强一致性的要求，所有操作都必须在主库“远程执行”，那么这些操作也会加大主从同步延迟。

除了以上问题外，双机房之间的专线还会偶发故障。我碰到过机房之间专线断开两小时的情况，期间只能临时用公网保持同步，但公网同步十分不稳定，网络延迟一直在 10ms ~ 500ms 之间波动，主从延迟达到了 1 分钟以上。好在用户中心服务主要以**长期缓存**的方式存储数据，业务的主要流程没有出现太大问题，只是用户修改信息太慢了。

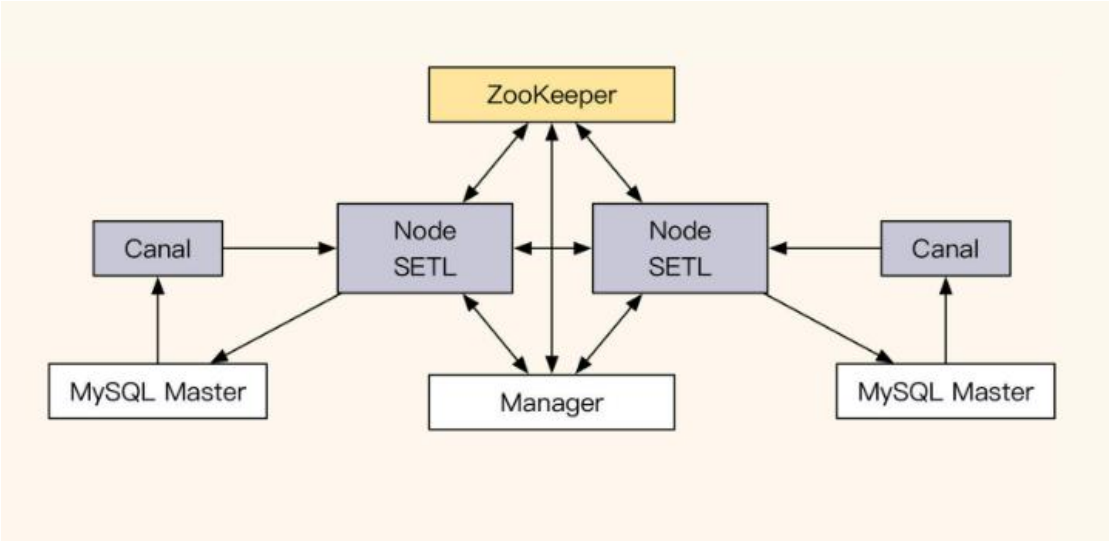
有时候，双机房还会偶发主从同步断开，对此建议做告警处理。一旦出现这种情况，就发送通知到故障警报群，由 DBA 人工修复处理。

另外，我还碰到过主从不同步期间，有用户注册自增 ID 出现重复，导致主键冲突这种情况。这里我推荐将自增 ID 更换为“由 SnowFlake 算法计算出的 ID”，这样可以减少机房不同步导致的主键冲突问题。

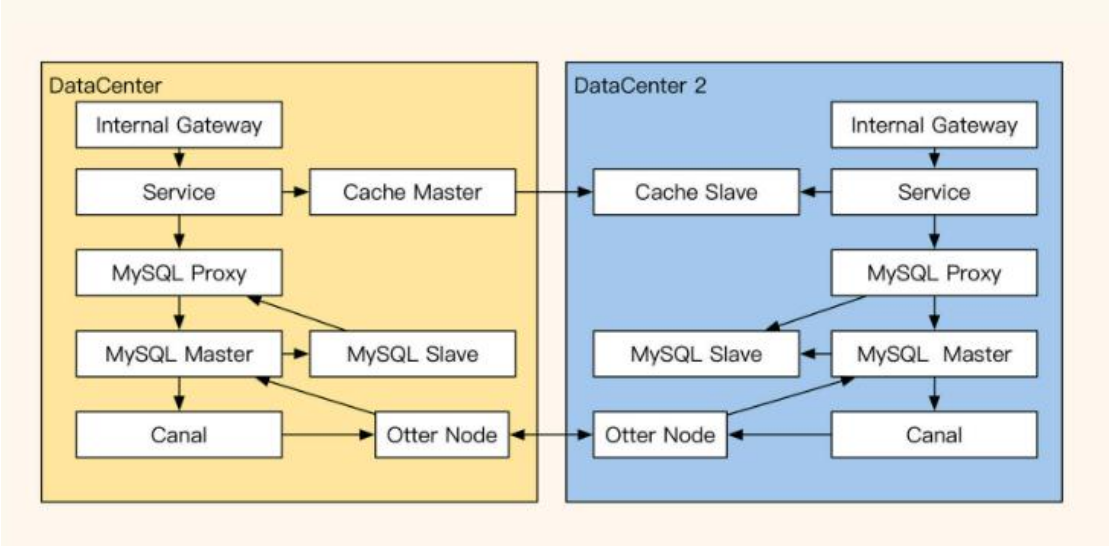
可以看到，核心数据库的中心方案虽然实现了同城双机房双活，但是人力投入很大。DBA 需要手动维护同步，主从同步断开后恢复起来也十分麻烦，耗时耗力，而且研发人员需要时刻关注主从不同步的情况，整体维护起来十分不便，所以我在这里推荐另外一个解决方案：数据库同步工具 Otter。

24.2 跨机房同步神器：Otter

Otter 是阿里开发的数据库同步工具，它可以快速实现跨机房、跨城市、跨国家的数据同步。如下图所示，其核心实现是通过 Canal 监控主库 MySQL 的 Row binlog，将数据更新并行同步给其他机房的 MySQL。



因为我们要实现同城双机房双活，所以这里我们用 Otter 来实现同城双主（注意：双主不通用，不推荐一致要求高的业务使用），这样双活机房可以双向同步：



如上图，每个机房内都有自己的主库和从库，缓存可以是跨机房主从，也可以是本地主从，这取决于业务形态。Otter 通过 Canal 将机房内主库的数据变更同步到 Otter Node 内，然后经由 Otter 的 SETL 整理后，再同步到对面机房的 Node 节点中，从而实现双机房之间的数据同步。

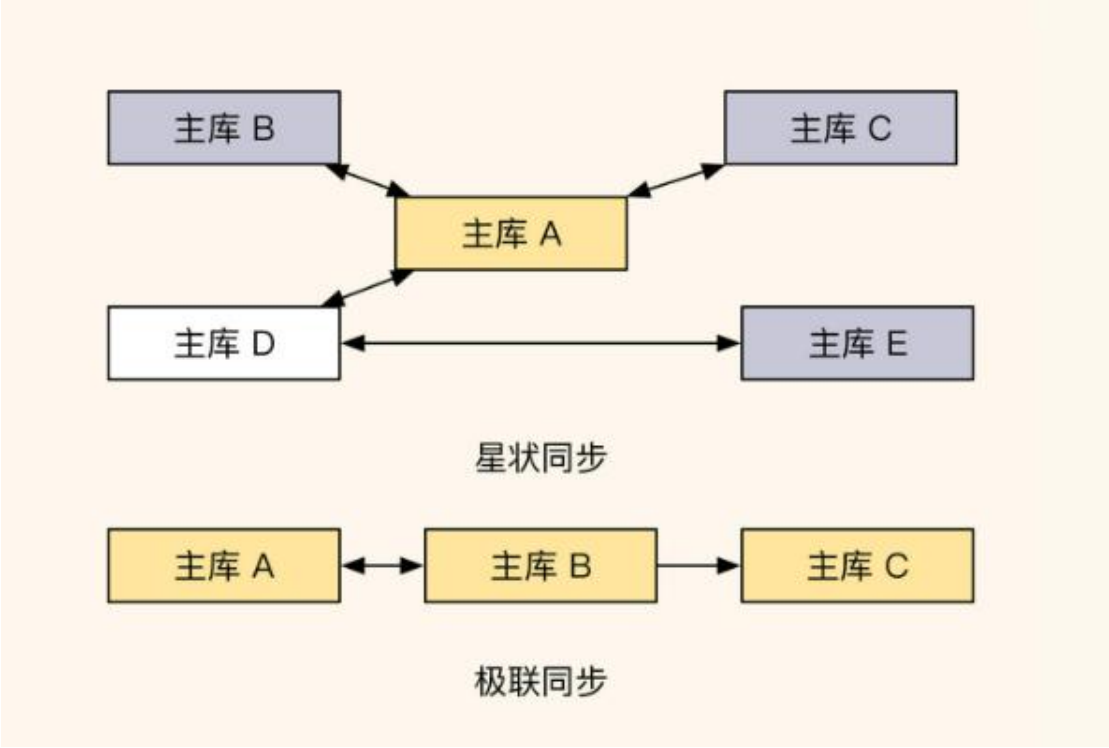
讲到这里不得不说一下，Otter 是怎么解决两个机房同时修改同一条数据所造成的冲突的。

在 Otter 中数据冲突有两种：一种是行冲突，另一种是字段冲突。**行冲突**可以通过对比数据修改时间来解决，或者是在冲突时回源查询覆盖目标库；对于**字段冲突**，我们可以

根据修改时间覆盖或把多个修改动作合并，比如 a 机房-1，b 机房-1，合并后就是-2，以此来实现数据的最终一致性。

但是请注意，**这种合并方式并不适合库存一类的数据管理，因为这样会出现超卖现象。如果有类似需求，建议用长期缓存解决。**

Otter 不仅能支持双主机房，还可以支持多机房同步，比如星形双向同步、级联同步（如下图）等。但是这几种方式并不实用，因为排查问题比较困难，而且当唯一决策库出现问题时，恢复起来很麻烦。所以若非必要，不推荐用这类复杂的结构。



另外，我还要强调一点，我们讲的**双活双向同步方案只适合同城。**一般来说，50～100 公里以内的机房同步都属于同城内。

超过这个距离的话，建议只做数据同步备份，因为**同步延迟过高**，业务需要在每一步关注延迟的代价过大。如果我们的业务对一致性的要求极高，那么建议在设计时，把这种一致性要求限制在同一个机房内，其他数据库只用于保存结果状态。

那为什么机房间的距离必须是 100 公里以内呢？你看看 Otter 对于不同距离的同步性能和延迟参考，应该就能理解了。

具体表格如下所示：

同步距离	操作数据量	Otter 主从延迟
同城机房同步 1 ms RTT 50km以内	100tps	延迟 0.1 s
	5000tps	延迟 1 s
中美异地机房同步 (200 ms RTT)	100tps	延迟 2 s
	5000tps	延迟 10 s

为了提高跨机房数据同步的效率，Otter 对用于主从同步的操作日志做了合并，把同一条数据的多次修改合并成了一条日志，同时对网络传输和同步策略做了滑窗并行优化。对比 MySQL 的同步，Otter 有 5 倍的性能提升。通过上面的表格可以看到，通过 Otter 实现的数据同步并发性能好、延迟低，只要我们将用户一段时间内的请求都控制在一个机房内不频繁切换，那么相同数据的修改冲突就会少很多。

用 Otter 实现双向同步时，我们的业务不需要做太多改造就能适应双主双活机房。具体来说，业务只需要操作本地主库，把“自增主键”换成“snowflake 算法生成的主键”、“唯一索引互斥”换成“分布式互斥锁”，即可满足大部分需求。

但是要注意，采用同城双活双向同步方案时，数据更新不能过于频繁，否则会出现更大的同步延迟。当业务操作的数据量不大时，才会有更好的效果。

说到这里，我们再讲一讲 Otter 的故障切换。目前 Otter 提供了简单的主从故障切换功能，在 Manager 中点击“切换”，即可实现 Canal 和数据库的主从同步方式切换。如果是同城双活，那关于数据库操作的原有代码我们不需要做更改，因为这个同步是双向的。

当一个机房出现故障时，先将故障机房的用户流量引到正常运转的机房，待故障修复后再恢复数据同步即可，不用切换业务代码的 MySQL 主从库 IP。切记，**如果双活机房有一个出现故障了，其他城市的机房只能用于备份或临时独立运行，不要跨城市做双活，因为同步延迟过高会导致业务数据损坏的后果。**

最后，我再啰嗦一下使用 Otter 的注意事项：第一，为了保证数据的完整性，变更表结构时，我们一般会先从从库修改表结构，因此在设置 Otter 同步时，建议将 pipeline

同步设置为忽略 DDL 同步错误；第二，数据库表新增字段时，只能在表结尾新增，不能删除老字段，并且建议先把新增字段同步到目标库，然后再同步到主库，因为只有这样才不会丢数据；第三，双向同步的表在新增字段时不要有默认值，同时 Otter 不支持没有主键的表同步。

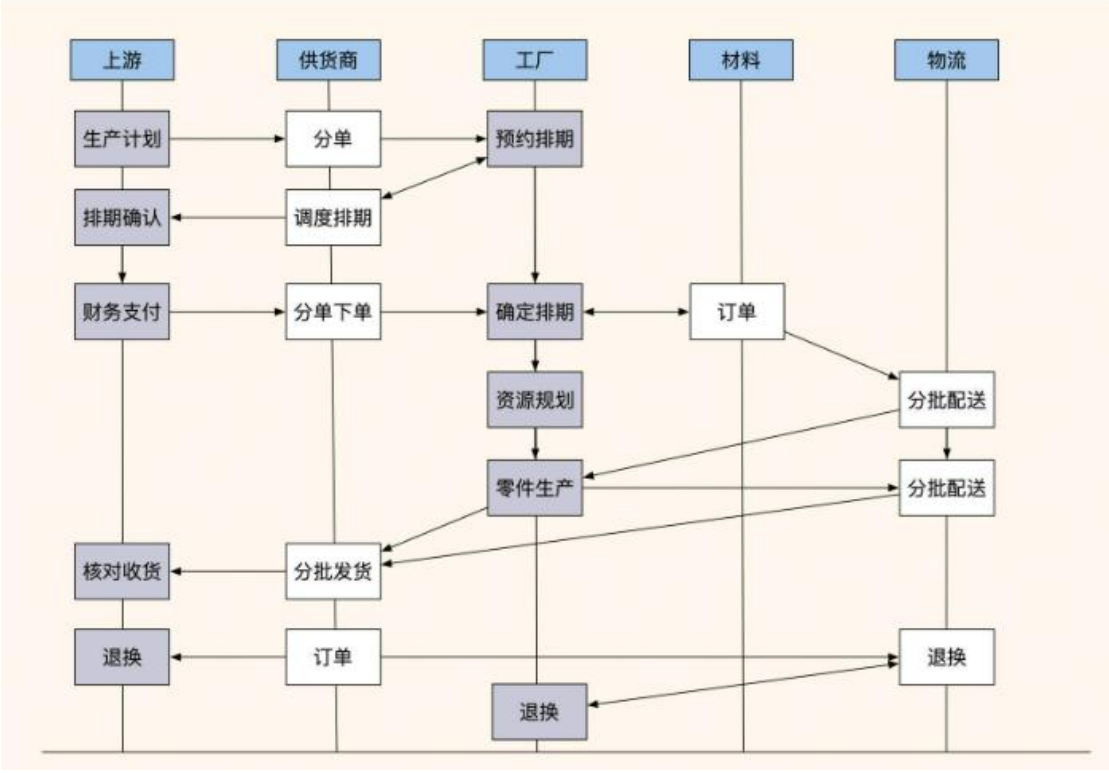
25. 你们采用微服务架构，请问你是什么做系统拆分的？

下面基于一个案例来说明微服务架构拆分的方法。

假设你现在是某行业知名电商的供货商，供应链比较长，而且供应品类和规格复杂。为确保生产计划平滑运转，系统还需要调配多个子工厂和材料商的生产排期。

原本调配订单需要电话沟通，但这样太过随机。为了保证生产链稳定供货，同时提高协调效率，朋友基于订单预订系统增加了排期协商功能，具体就是将 “排期” 作为下订单主流程里的一个步骤，并将协商出的排期按照日历样式来展示，方便上游供应商和各个工厂以此协调生产周期。

整个供货协商流程如下图所示：



如图，上游项目会先发布生产计划（或采购计划），供货商根据计划拆分采购列表（分单），并联系不同的工厂协调做预排期（预约排期）。之后，上游采购方对工厂产品进行质量审核，然后下单支付、确认排期。

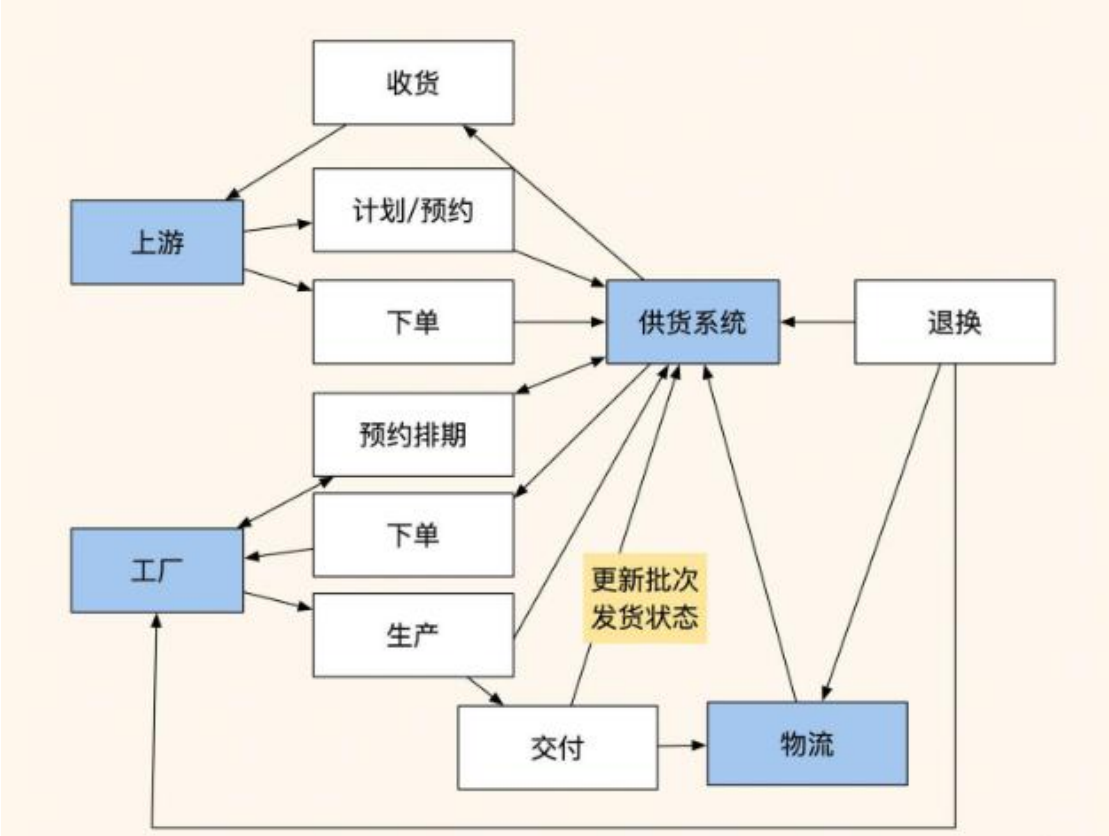
工厂根据确认好的排期制定采购材料计划，并通知材料供货商分批供货，开始分批生产制造产品。每当制造好一批产品后，工厂就会通知物流按批次发货到采购方（即供货商），同时更新供货商系统内的分批订单信息。接着，上游对产品进行验收，将不合格的产品走退换流程。

但系统运行了一段时间后朋友发现，由于之前系统是以订单为主体的，增加排期功能后还是以主订单作为聚合根（即主要实体），这就导致上游在发布计划时需要创建主订单。而主订单一直处于开启状态，随着排期不断调整和新排期的不断加入，订单数据就会持续增加，一年内订单数据量达到了一亿多条。因为数据过多、合作周期长，并且包含了售后环节，所以这些数据无法根据时间做归档，导致整个系统变得越来越慢。

考虑到这是核心业务，如果持续存在问题影响巨大，因此朋友找我取经，请教如何对数据进行分表拆分。但根据我的理解，这不是分表分库维护的问题，而是系统功能设计不合理导致了系统臃肿。于是经过沟通，我们决定对系统订单系统做一次领域拆分。

25.1 流程分析整理

我先梳理了主订单的 API 和流程，从上到下简单绘制了流程和订单系统的关系，如下图所示：



可以看到，有多个角色在使用这个“订单排期系统”。通过这张图与产品、研发团队进行沟通，来确认我理解的**主要流程的数据走向和系统数据依赖关系**都没有问题。

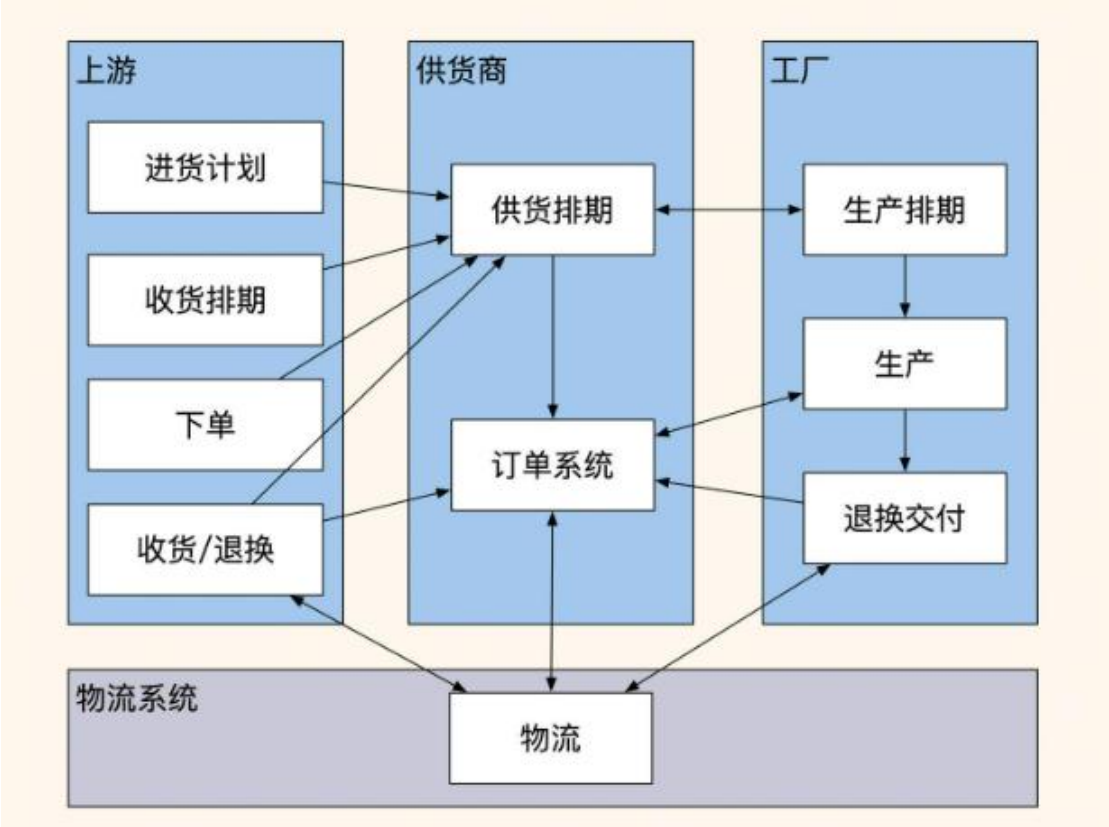
接着我们将目光放在了订单表上，订单表承载的职能过多，导致多个流程依赖订单表无法做数据维护，而且订单存在多个和订单业务无关的状态，比如排期周期很长，导致订单一直不能关闭。我们在第 1 节课讲过，**一个数据实体不要承担太多职能，否则很难管理**，所以我们需要对订单和排期的主要实体职能进行拆分。

经过分析我们还发现了另一个问题，**现在系统的核心并不是订单，而是计划排期。**原订单系统在改造前是通过自动匹配功能实现上下游订单分单的，所以系统的主要模块都是围绕订单来流转的。而增加排期功能后，系统的核心就从围绕订单实现匹配分单，转变成了围绕排期产生订单的模式，这更符合业务需要。

排期和订单有关联关系，但职能上有不同的方向用途，排期只是计划，而订单只为工厂后续生产运输和上游核对结果使用。**这意味着系统的模块和表的设计核心已经发生了偏移**，我们需要拆分模块才能拥有更好的灵活性。

综上所述，我们总体的拆分思路是：要将排期流程和订单交付流程完全拆分开。要知道在创业公司，我们做的项目一开始的设计初衷常常会因为市场需求变化，逐渐偏离原有设计，这就需要我们不断重新审视我们的系统，持续改进，才能保证系统的完善。

因为担心研发团队摆脱不了原有系统的思维定势，拆分做得不彻底，导致改版失败，所以我对角色和流程做了一次梳理，明确了各个角色的职责和流程之间的关系。我**按角色及其所需动作**画出多个框，将他们需要做的动作和数据流穿插起来，如下图所示：



基于这个图，我再次与研发、产品沟通，找出了订单与排期在功能和数据上的拆分点。具体来讲，就是将上游的职能拆分为：发布进货计划、收货排期、下单、收货/退换；而供货商主要做协调排期分单，同时提供订单相关服务；工厂则主要负责生产排期、生产和售后。这样一来，**系统的流程就可以归类成几个阶段：**

1.计划排期协调阶段- 2.按排期生产供货+周期物流交付阶段- 3.售后服务调换阶段

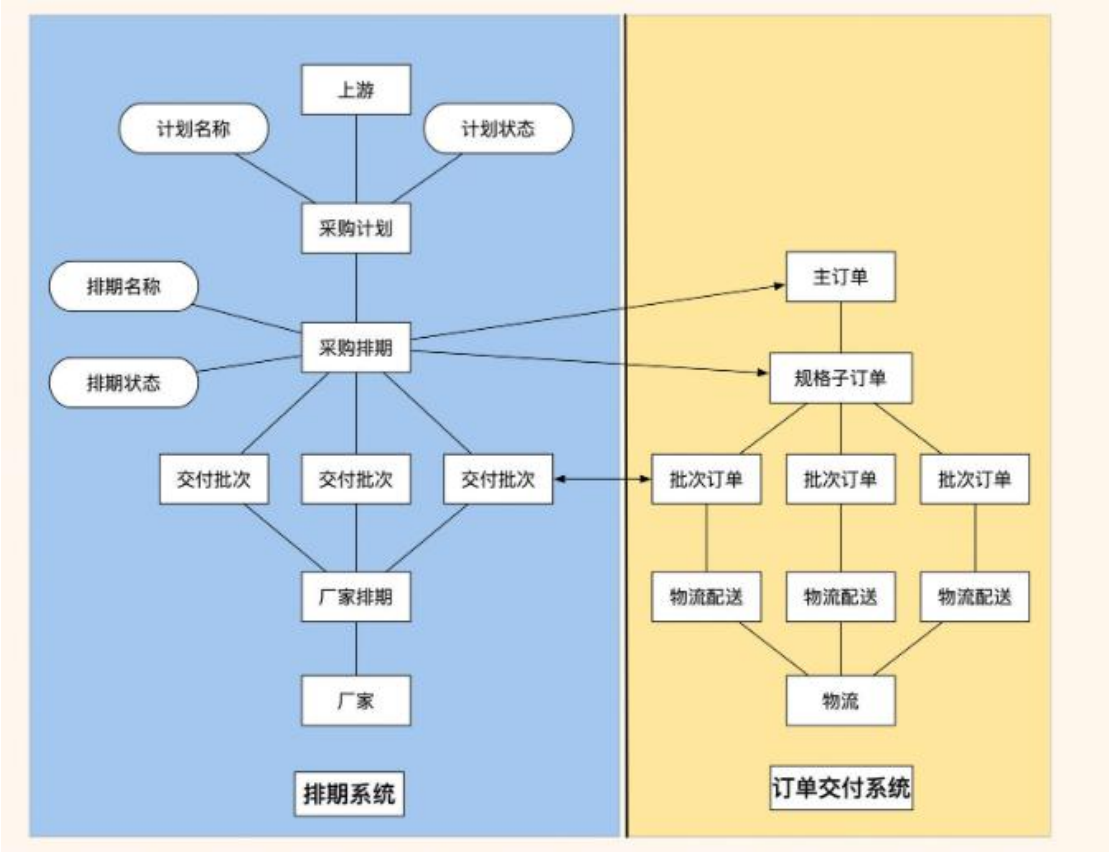
可以看到，第一个阶段不牵扯订单，主要是上游和多个工厂的**排期协调；**第二、三阶段是工厂生产供货和售后，这些服务需要和**订单**交互，而上游、工厂和物流的视角是完全不同的。

基于这个结论，我们完全可以**根据数据的主要实体和主要业务流程**（订单 ID 做聚合根，将流程分为订单和排期两个领域）将系统拆分成两个子系统：排期调度系统、订单交付系统。

在计划排期协调阶段，上游先在排期调度系统内提交进货计划和收货排期，然后供货商根据上游的排期情况和进货需求，与多家合作工厂协调分单和议价。多方达成一致后，上游对计划排期和工厂生产排期进行预占。

待上游正式签署协议、支付生产批次定金后，排期系统会根据排期和工厂下单在订单系统中产生对应的订单。同时，**上游、供货商和工厂一旦达成合作，后续可以持续追加下单排期**，而不是将合作周期限制在订单内。

在排期生产供货阶段，排期系统在调用订单系统的同时，会传递具体的主订单号和订单明细。订单明细内包含着计划生产的品类、个数以及每期的交付量，工厂可以根据自己的情况调整生产排期。产品生产完毕后，工厂分批次发送物流进行派送，并在订单系统内记录交付时间、货物量和物流信息。同时，订单系统会生成财务信息，与上游财务和仓库分批次地对账。



这么拆分后，两个系统把采购排期和交付批次作为聚合根，进行了数据关联，这样一来，整体的订单流程就简单了很多。

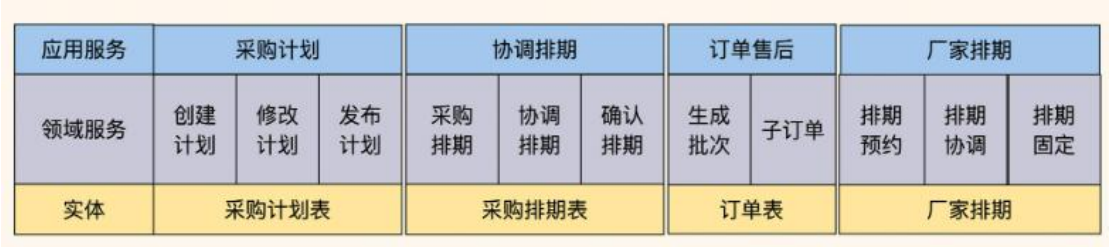
总体来讲，前面对业务的梳理都以流程、角色和关键动作这三个元素作为分析的切入点，然后将不同流程划分出不同阶段来归类分析，根据不同阶段拆分出两个业务领域：排期和订单，同时找出两个业务领域的聚合根。经过这样大胆的拆分后，再与产品和研发论证可行性。

25.2 系统拆分从表开始

经历了上面的过程，相信你对按流程和阶段拆分实体职责的方法，已经有了一定的感觉，这里我们再用代码和数据库表的视角复盘一下该过程。

一般来说，系统功能从表开始拆分，这是最容易实现的路径，因为我们的业务流程往往都会围绕一个主要的实体表运转，并关联多个实体进行交互。在这个案例中，我们将订单表内关于排期的数据和状态做了剥离，拆分之前的代码分层如下图所示：

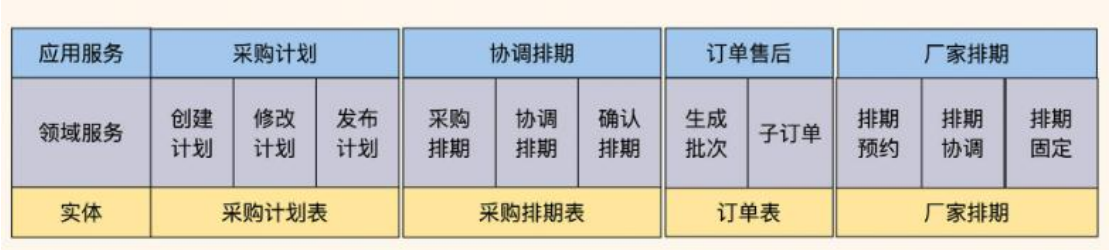
拆分之后，代码分层变成了这样：



可以看到，最大的变化就是订单实体表的职责被拆分了，我们的系统代码随之变得更加简单，而且同一个订单实体被多个角色交叉调用的情况也完全消失了。在拆分过程中，我们的依据有三个：

1. **数据实体职能只做最核心的一件事**，比如订单只管订单的生老病死（包括创建、流程状态更改、退货、订单结束）；
2. **业务流程归类按涉及实体进行归类**，看能否分为多个阶段，比如“协调排期流程进行中”、“生产流程”、“售后服务阶段”；

3. **由数据依赖交叉的频率决定把订单划分成几个模块**，如果两个模块业务流程上交互紧密，并且有数据关联关系，比如 Join、调用 A 必然调用 B 这种，就把这两个模块合并，同时保证短期内不会再做更进一步的拆分。



一个核心的系统，如果按实体表职责进行拆分整理，那么它的流程和修改难度都会大大降低。

而模块的拆分，也可以通过图 6，从下往上去看。如果它们之间的数据交互不是特别频繁，比如没有出现频繁的 Join，我们就将系统分成四个模块。如图 7 所示，可以看到这四个模块之间相对独立，各自承担一个核心的职责。同时，两个实体之间交互没有太大的数据关联，每个模块都维护着某个阶段所需的全部数据，这么划分比较清晰，也易于统一管理。

到这里，我们只需要将数据和流程关系都梳理一遍，确保它们之间的数据在后续的统计分析中没有频繁数据 Join，即可完成对表的拆分。

但如果要按业务划分模块，我还是建议从上到下去看业务流程，来决定数据实体拆分（领域模型设计 DDD）的领域范围，以及各个模块的职责范围。

25.3 越是底层服务越要抽象

除了系统的拆分外，我们还要注意一下服务的抽象问题。很多服务经常因业务细节变更需要经常修改，而越是底层服务，越要减少变更。如果服务的抽象程度不够，一旦底层服务变更，我们很难确认该变更对上游系统的影响范围。

所以，我们要搞清楚哪些服务可以抽象为底层服务，以及如何对这些服务做更好的抽象。因为电商类系统经常对服务做拆分和抽象，所以我就以这类系统为例为你进行讲解。你可能感到疑惑：电商系统为什么要经常做系统拆分和服务抽象呢？

这是因为电商系统最核心且最复杂的地方就是订单系统，电商商品有多种品类（sku+spu），不同品类的筛选维度、服务、计量单位都不同，这就导致系统要记录大

量的冗余品类字段，才能保存好用户下单时的交易快照。所以我们需要频繁拆分整理系统，避免这些独有特性影响到其他商品。

此外，电商系统不同业务的服务流程是不同的。比如下单购买食品，与下单定制一个柜子完全不同。在用户购买食品时，电商只需要通知仓库打包、打物流单、发货、签收即可；而用户定制柜子则需要厂家上门量尺寸、复尺、定做、运输、后续调整等。所以，我们需要做服务抽象，让业务流程更标准、更通用，避免变更过于频繁。

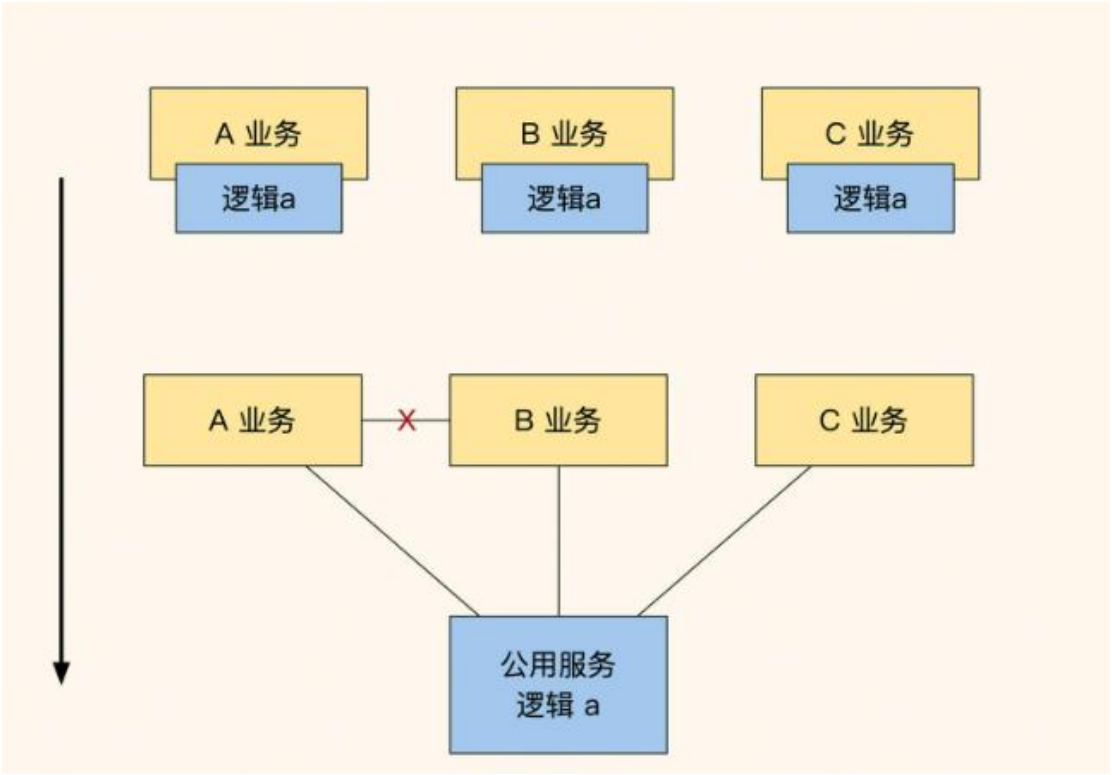
正是由于业务服务形态存在不同的差异，订单系统需要将自己的职能控制在“一定范围”内。对此，我们应该考虑如何在满足业务需求的情况下，让订单表的数据职能最小。事实上，这没有绝对的答案，因为不同行业、不同公司的业务形态都是不同的，这里我举几个常见的抽象思路供你参考。

25.3.1 被动抽象法

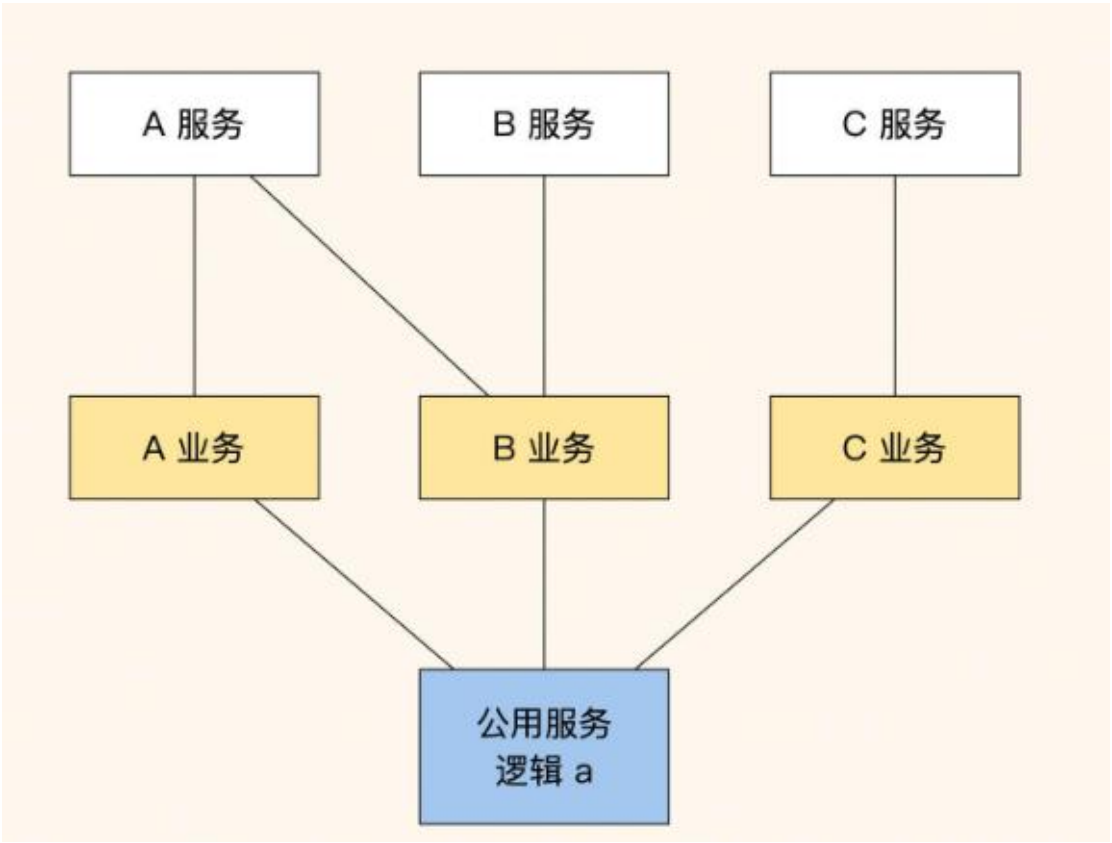
如果两个或多个服务使用同一个业务逻辑，就把这个业务逻辑抽象成公共服务。比如业务 A 更新了逻辑 a，业务 B 也会同步使用新的逻辑 a，那么就将这个逻辑 a 放到底层抽象成一个公共服务供两个服务调用。这种属于比较被动的抽象方式，很常见，适合代码量不大、维护人员很少的系统。

对于创业初期主脉络不清晰的系统，利用被动抽象法很容易做抽象。不过，它的缺点是抽象程度不高，当业务需要大量变更时，需要一定规模的重构。

总的来说，虽然这种方式的代码结构很贴近业务，但是很麻烦，而且代码分层没有规律。所以，被动抽象法适用于新项目的探索阶段。



这里说一个题外话，同层级之间的模块是禁止相互调用的。如果调用了，就需要将两个服务抽象成公共服务，让上层对两个服务进行聚合，如上图中的红 X，拆分后如下图所示：

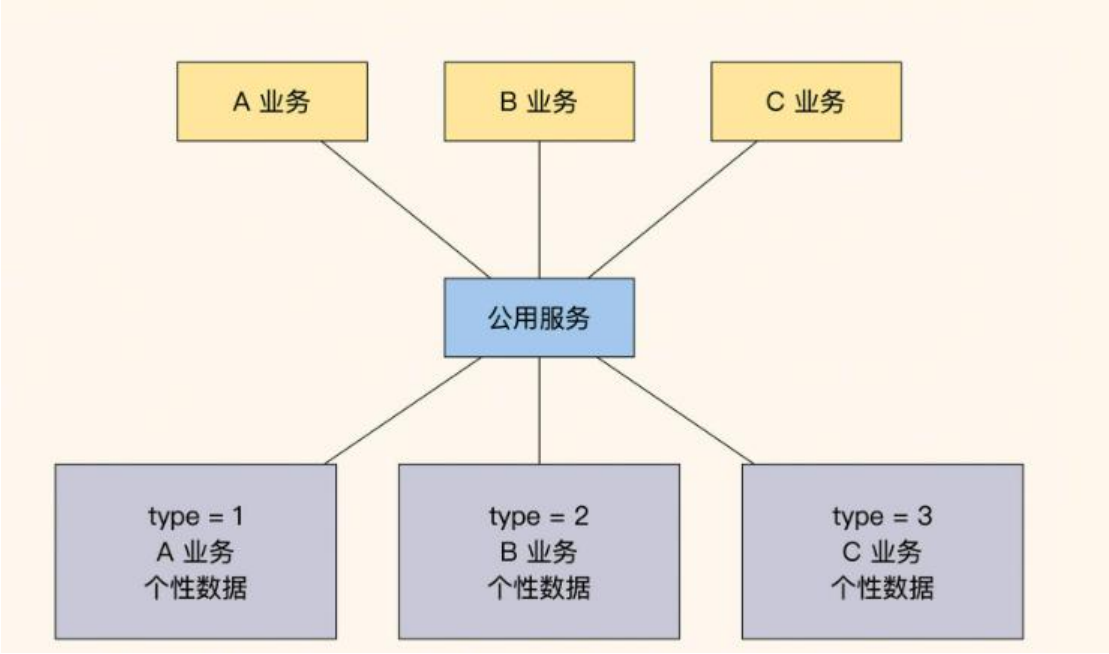


这么做是为了让系统结构从上到下是一个倒置的树形，保证不会出现引用交叉循环的情况，否则会让项目难以排查问题，难以迭代维护，如果前期有大量这样的调用，当我们做系统改造优化时只能投入大量资源才能解决这个问题。

25.3.2 动态辅助表方式

这个方式适用于规模稍微大一点的团队或系统，它的具体实现是这样的：当订单系统被几个开发小组共同使用，而不同业务创建的主订单有不同的 type，不同的 type 会将业务特性数据存储在不同的辅助表内，比如普通商品保存在表 order 和表 order_product_extra 中，定制类商品的定制流程状态保存在 order_customize_extra 中。

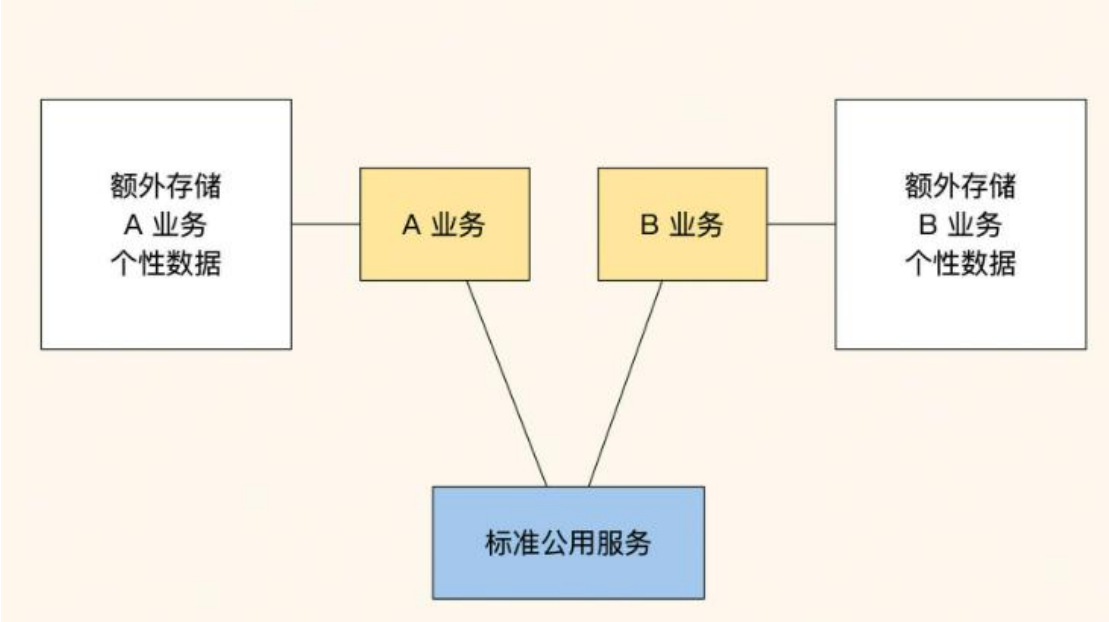
这样处理的好处是更贴近业务，方便查询。但由于辅助表有其他业务数据，业务的隔离性比较差，所有依赖订单服务的业务常会受到影响，而且订单需要时刻跟着业务改版。所以，通过这种方式抽象出来的订单服务已经形同虚设，一般只有企业的核心业务才会做类似的定制。



25.3.4 强制标准接口方式

这种方式在大型企业比较常见，其核心点在于：底层服务只做标准的服务，业务的个性部分都由业务自己完成，比如订单系统只有下单、等待支付、支付成功、发货和收货功能，展示的时候用前端对个性数据和标准订单做聚合。

用这种方式抽象出的公共服务订单对业务的耦合性是最小的，业务改版时不需要订单跟随改版，订单服务维护起来更容易。只是上层业务交互起来会很难受，因为需要在本地保存很多附加的信息，并且一些流转要自行实现。不过，从整体来看，对于使用业务多的系统来说，因为业务导致的修改会很少。



通过上面三种方式可以看出，业务的稳定性取决于服务的抽象程度。如果底层经常更改，那么整个业务就需要不断修改，最终会导致业务混乱。所以，我个人还是推荐你使用**强制标准接口方式**，这也是很多公司的常见做法。虽然很难用，但比起经常重构整个系统总要好一些。

你可能很奇怪，**为什么不把第一种方式一口气设计好呢？**这是因为大部分的初创业务都不稳定，提前设计虽然能让代码结构保持统一，但是等两年后再回头看，你会发现当初的设计已经面目全非，我们最初信心满满的设计，最后会成为业务的绊脚石。

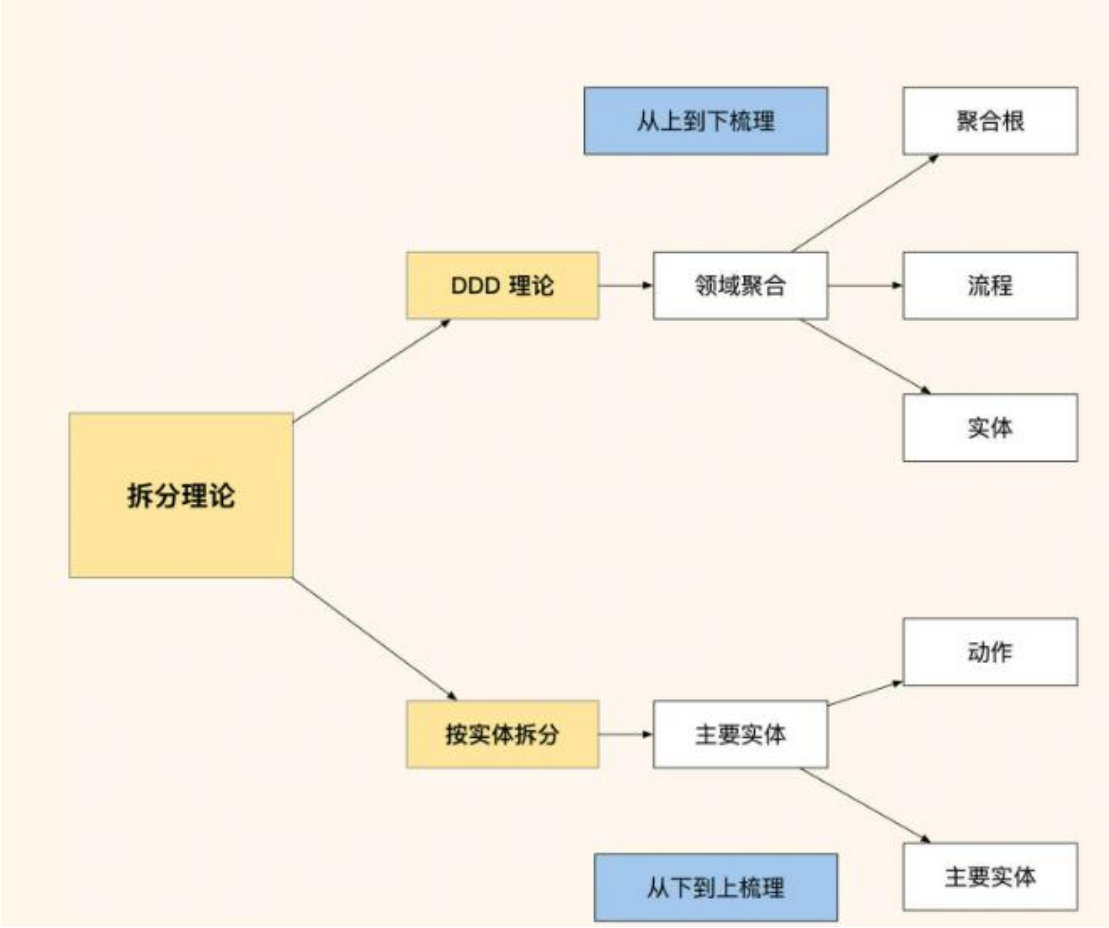
所以，这种拆分和架构设计需要我们不定期回看、自省、不断调整。毕竟技术是为业务服务的，业务更重要，没有人可以保证项目初期设计的个人中心不会被改成交友的个人门户。

总之，每一种方法并非绝对正确，我们需要根据业务需求来决策用哪一种方式。

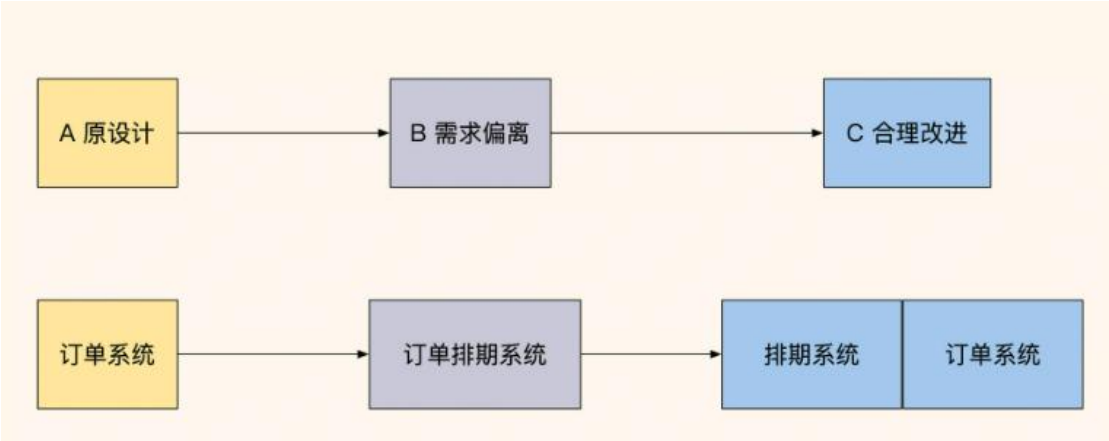
25.4 总结

业务拆分的方法有很多，最简单便捷的方式是：先**从上到下**做业务流程梳理，将流程归类聚合；然后从不同的**领域聚合**中找出交互所需**主要实体**，根据流程中主要实体之间的数据依赖程度决定是否拆分（**从下到上看**）；把不同的实体和动作拆分成多个模块后，再根据业务流程归类，划分出最终的模块（**最终汇总**）。

这个拆分过程用一句话总结就是：从上往下看流程，从下往上看模块，最后综合考虑流程和模块的产出结果。用这种方式能快速拆出模块范围，拆分出的业务也会十分清晰。



除了拆分业务外，我们还要关注如何抽象服务。如果底层业务变更频繁，就会导致上层业务频繁修改，甚至出现变更遗漏的情况。所以，我们要确保底层服务足够抽象，具体有很多种办法，比如被动拆分法、动态辅助表方式、标准抽象方式。这几种方式各有千秋，需要我们根据业务来决策。



通常，我们的业务系统在初期都会按照一个特定的目标来设计，但是随着市场需求的变化，业务系统经过不断改版，往往会偏离原有的设计。

虽然我们每次改版都实现了既定需求，但也很容易带来许多不合理的问题。所以，在需求稳定后，一般都会做更合理的改造，保证系统的完整性，提高可维护性。很多时候，第一版本不用做得太过精细，待市场验证后明确了接下来的方向，再利用留出足够的空间改进，这样设计的系统才会有更好的扩展性。

26. 如何解决高并发下的库存抢购超卖少买问题?

我相信很多人都看到过相关资料，但是在实践过程中，仍然会碰到具体的实现无法满足需求的情况，比如说有的实现无法秒杀多个库存，有的实现新增库存操作缓慢，有的实现库存耗尽时会变慢等等。

这是因为对于不同的需求，库存争抢的具体实现是不一样的，我们需要详细深挖，理解各个锁的特性和适用场景，才能针对不同的业务需要做出灵活调整。

由于秒杀场景是库存争抢非常经典的一个应用场景，接下来我会结合秒杀需求，带你看看如何实现高并发下的库存争抢，相信在这一过程中你会对锁有更深入的认识。

26.1 锁争抢的错误做法

在开始介绍库存争抢的具体方案之前，我们先来了解一个小知识——并发库存锁。还记得在我学计算机的时候，老师曾演示过一段代码：

```
1 public classThreadCounter {
```

```
    private staticint count = 0;

    public staticvoidmain(String[] args) throws Exception {
        Runnable task = new Runnable() {
            public void run() {
                for (int i = 0; i < 1000; ++i) {
                    count += 1;
                }
            }
        };

        Thread t1 = new Thread(task);
        t1.start();

        Thread t2 = new Thread(task);
        t2.start();

        t1.join();
        t2.join();

        cout << "count = " << count << endl;
    }
}
```

从代码来看，我们运行后结果预期是 2000，但是实际运行后并不是。为什么会这样呢？当多线程并行对同一个公共变量读写时，由于没有互斥，多线程的 set 会相互覆盖或读取时容易读到其他线程刚写一半的数据，这就导致变量数据被损坏。反过来说，我们要想保证一个变量在多线程并发情况下的准确性，就需要这个变量在修改期间不会被其他线程更改或读取。

对于这个情况，我们一般都会用到锁或原子操作来保护库存变量：

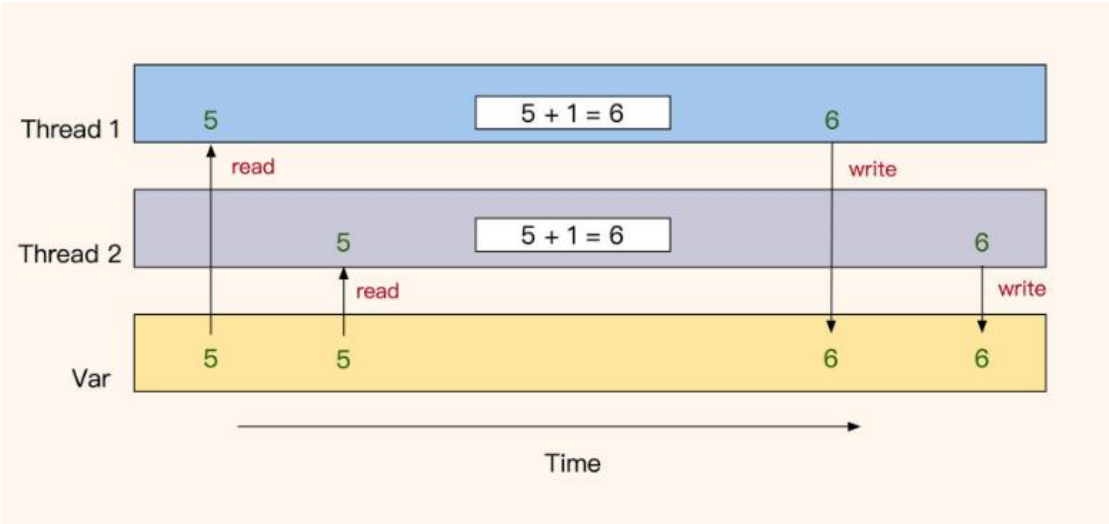
- 如果是简单 int 类型数据，可以使用原子操作保证数据准确；
- 如果是复杂的数据结构或多步操作，可以加锁来保证数据完整性。

考虑到我们之前的习惯会有一定惯性，为了让你更好地理解争抢，这里我再举一个我们常会犯错的例子。因为扣库存的操作需要注意原子性，我们实践的时候常常碰到后面这种方式：

```
redis> get prod_1475_stock_1
15
redis> set prod_1475_stock_1 14
```

```
4 OK
```

也就是先将变量从缓存中取出，对其做-1 操作，再放回到缓存当中，**这是个错误做法。**



如上图，原因是多个线程一起读取的时候，多个线程同时读到的是 5，set 回去时都是 6，实际每个线程都拿到了库存，但是库存的实际数值并没有累计改变，这会导致库存超卖。如果你需要用这种方式去做，一般建议加一个自旋互斥锁，互斥其他线程做类似的操作。

不过锁操作是很影响性能的，在讲锁方式之前，我先给你介绍几个相对轻量的方式。

26.2 原子操作

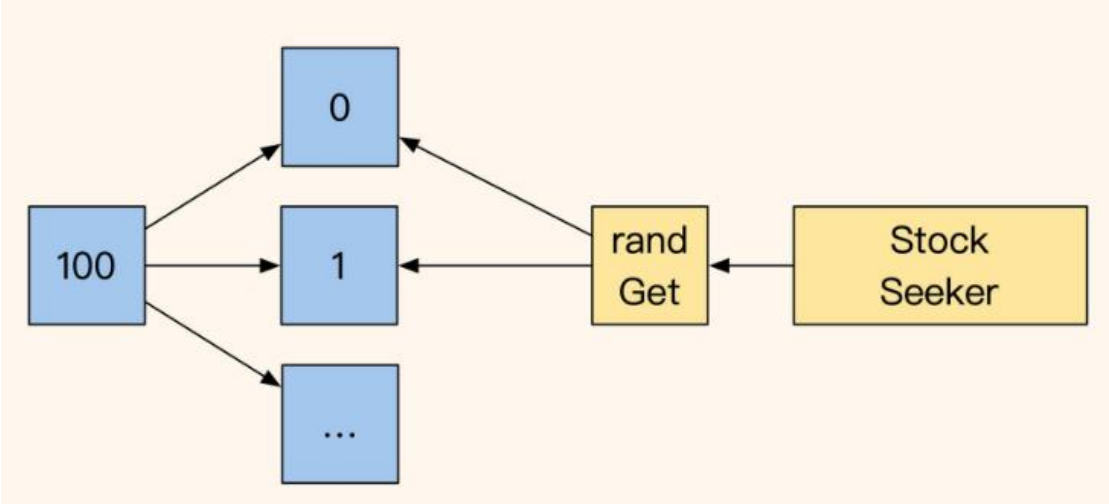
在高并发修改的场景下，用互斥锁保证变量不被错误覆盖性能很差。让一万个用户抢锁，排队修改一台服务器的某个进程保存的变量，这是个很糟糕的设计。

因为锁在获取期间需要自旋循环等待，这需要不断地循环尝试多次才能抢到。而且参与争抢的线程越多，这种情况就越糟糕，这期间的通讯过程和循环等待很容易因为资源消耗造成系统不稳定。

对此，我会把库存放在一个独立的且性能很好的内存缓存服务 Redis 中集中管理，这样可以减少用户争抢库存导致其他服务的抖动，并且拥有更好的响应速度，这也是目前互联网行业保护库存量的普遍做法。

同时，我不建议通过数据库的行锁来保证库存的修改，因为数据库资源很珍贵，使用数据库行锁去管理库存，性能会很差且不稳定。

前面我们提到当有大量用户去并行修改一个变量时，只有用锁才能保证修改的正确性，但锁争抢性能很差，那怎么降低锁的粒度、减少锁的争枪呢?



如上图，其实我们可以将一个热门商品的库存做拆分，放在多个 key 中去保存，这样可以大幅度减少锁争抢。

举个例子，当前商品库存有 100 个，我们可以把它放在 10 个 key 中用不同的 Redis 实例保存，每个 key 里面保存 10 个商品库存，当用户下单的时候可以随机找一个 key 进行扣库存操作。如果没库存，就记录好当前 key 再随机找剩下的 9 个 key，直到成功扣除 1 个库存。

除了这种方法以外，我个人更推荐的做法是使用 Redis 的原子操作，因为原子操作的粒度更小，并且是高性能单线程实现，可以做到全局唯一决策。而且很多原子操作的底层实现都是通过硬件实现的，性能很好，比如文稿后面这个例子:

```
redis> decr prod_1475_stock_1
14
```

incr、decr 这类操作就是原子的，我们可以根据返回值是否大于 0 来判断是否扣库存成功。但是这里你要注意，如果当前值已经为负数，我们需要考虑一下是否将之前扣除的补偿回来。并且为了减少修改操作，我们可以在扣减之前做一次值检测，整体操作如下:

```
//读取当前库存，确认是否大于零
//如大于零则继续操作，小于等于拒绝后续
redis> get prod_1475_stock_1
1

//开始扣减库存、如返回值大于或等于 0 那么代表扣减成功，小于 0 代表当前已经没有库存
//可以看到返回-2，这可以理解成同时两个线程都在操作扣库存，并且都没拿到库存
```

```
redis> decr prod_1475_stock_1
-2

//扣减失败、补偿多扣的库存
//这里返回 0 是因为同时两个线程都在做补偿，最终恢复 0 库存
redis> incr prod_1475_stock
0
```

这看起来是个不错的保护库存量方案，不过它也有缺点，相信你已经猜到了，这个库存的数值准确性取决于我们的业务是否能够返还恢复之前扣除的值。如果在服务运行过程中，“返还”这个操作被打断，人工修复会很难，因为你不知道当前有多少库存还在路上狂奔，只能等活动结束后所有过程都落地，再来看剩余库存量。

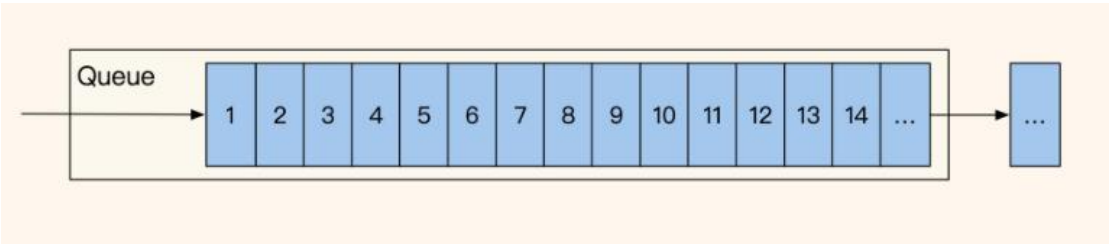
而要想完全保证**库存不会丢失**，我们习惯性通过事务和回滚来保障。但是外置的库存服务 Redis 不属于数据库的缓存范围，这一切需要通过人工代码去保障，这就要求我们在处理业务的每一处故障时都能处理好库存问题。

所以，很多常见秒杀系统的库存在**出现故障时是不返还的**，并不是不想返还，而是很多意外场景做不到。

提到锁，也许你会想到使用 Setnx 指令或数据库 CAS 的方式实现互斥排他锁，以此来解决库存问题。但是这个锁有自旋阻塞等待，并发高的时候用户服务需要循环多次做尝试才能够获取成功，这样很浪费系统资源，对数据服务压力较大，不推荐这样去做。

26.3 令牌库存

除了这种用数值记录库存的方式外，还有一种比较科学的方式就是“发令牌”方式，通过这个方式可以避免出现之前因为抢库存而让库存出现负数的情况。



具体是使用 Redis 中的 list 保存多张令牌来代表库存，一张令牌就是一个库存，用户抢库存时拿到令牌的用户可以继续支付：

```
//放入三个库存
redis> lpush prod_1475_stock_queue_1 stock_1
```

```
redis> lpush prod_1475_stock_queue_1 stock_2
redis> lpush prod_1475_stock_queue_1 stock_3

//取出一个，超过 0.5 秒没有返回，那么抢库存失败
redis> brpop prod_1475_stock_queue_1 0.5
```

在没有库存后，用户只会拿到 nil。当然这个实现方式只是解决抢库存失败后不用再补偿库存的问题，在我们对业务代码异常处理不完善时仍会出现丢库存情况。

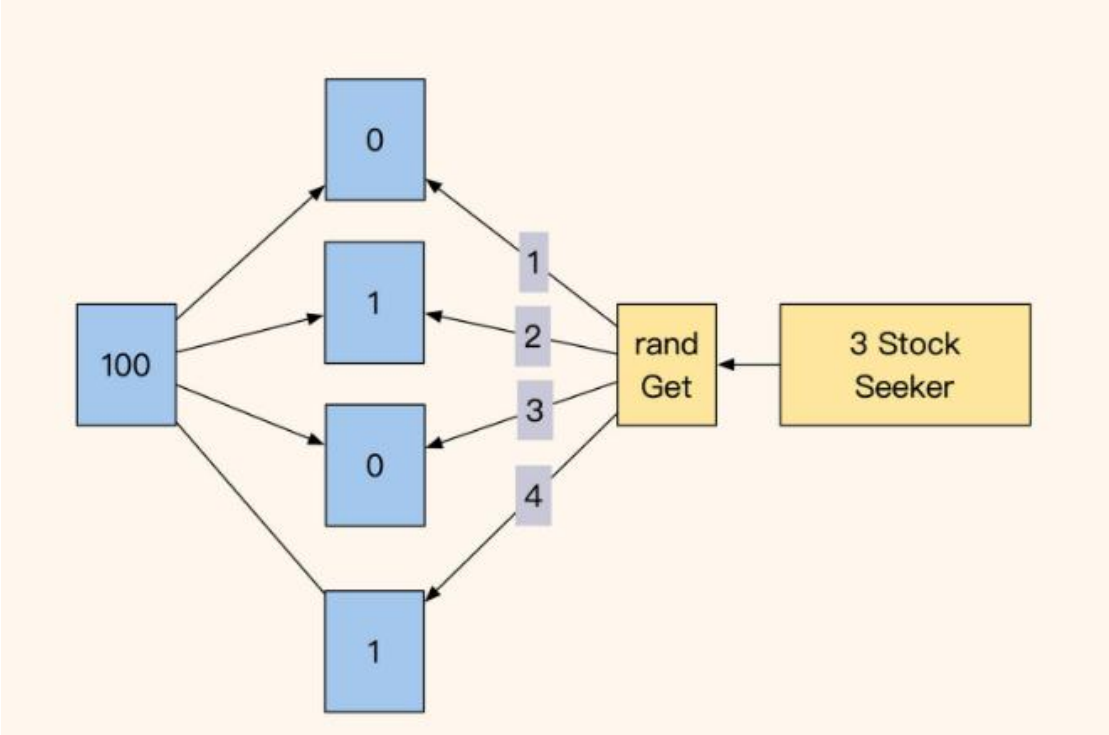
同时，我们要注意 brpop 可以从 list 队列“右侧”中拿出一个令牌，如果不需要阻塞等待的话，使用 rpop 压测性能会更好一些。

不过，当我们的库存成千上万的时候，可能不太适合使用令牌方式去做，因为我们需要往 list 中推送 1 万个令牌才能正常工作来表示库存。如果有 10 万个库存就需要连续插入 10 万个字符串到 list 当中，入库期间会让 Redis 出现大量卡顿。

到这里，关于库存的设计看起来已经很完美了，不过请你想一想，如果产品侧提出“一个商品可以抢多个库存”这样的要求，也就是一次秒杀多个同种商品（比如一次秒杀两袋大米），我们利用多个锁降低锁争抢的方案还能满足吗？

26.4 多库存秒杀

其实这种情况经常出现，这让我们对之前的优化有了更多的想法。对于一次秒杀多个库存，我们的设计需要做一些调整。



之前我们为了减少锁冲突把库存拆成 10 个 key 随机获取，我们设想一下，当库存剩余最后几个商品时，极端情况下要想秒杀三件商品（如上图），我们需要尝试所有的库存 key，然后在尝试 10 个 key 后最终只拿到了两个商品库存，那么这时候我们是拒绝用户下单，还是返还库存呢？

这其实就要看产品的设计了，同时我们也需要加一个检测：如果商品卖完了就不要再尝试拿 10 个库存 key 了，毕竟没库存后一次请求刷 10 次 Redis，对 Redis 的服务压力很大（Redis O(1)指令性能理论可以达到 10w OPS，一次请求刷 10 次，那么理想情况下抢库存接口性能为 1W QPS，压测后建议按实测性能 70%漏斗式限流）。

这时候你应该发现了，在“一个商品可以抢多个库存”这个场景下，拆分并没有减少锁争抢次数，同时还加大了维护难度。当库存越来越少的时候，抢购越往后性能表现越差，这个设计已经不符合我们设计的初衷（由业务需求造成我们底层设计不合适的情况经常会碰到，这需要我们在设计之初，多挖一挖产品具体的需求）。

那该怎么办呢？我们不妨将 10 个 key 合并成 1 个，改用 rpop 实现多个库存扣减，但库存不够三个只有两个的情况，仍需要让产品给个建议看看是否继续交易，同时在开始的时候用 LLEN（O(1)）指令检查一下我们的 List 里面是否有足够的库存供我们 rpop，以下是这次讨论的最终设计：

```
1 //取之前看一眼库存是否空了，空了不继续了(llen O(1))
```

```
redis> llen prod_1475_stock_queue
3

//取出库存 3 个，实际抢到俩
redis> rpop prod_1475_stock_queue 3
"stock_1""stock_2"

//产品说数量不够，不允许继续交易，将库存返还
redis> lpush prod_1475_stock_queue stock_1
redis> lpush prod_1475_stock_queue stock_2

```

通过这个设计，我们已经大大降低了下单系统锁争抢压力。要知道，Redis 是一个性能很好的缓存服务，其 O(1)类复杂度的指令在使用长链接的情况下多线程压测，5.0 版本的 Redis 就能够跑到 10w OPS，而 6.0 版本的网络性能会更好。

这种利用 Redis 原子操作减少锁冲突的方式，对各个语言来说是通用且简单的。不过你要注意，不要把 Redis 服务和复杂业务逻辑混用，否则会影响我们的库存接口效率。

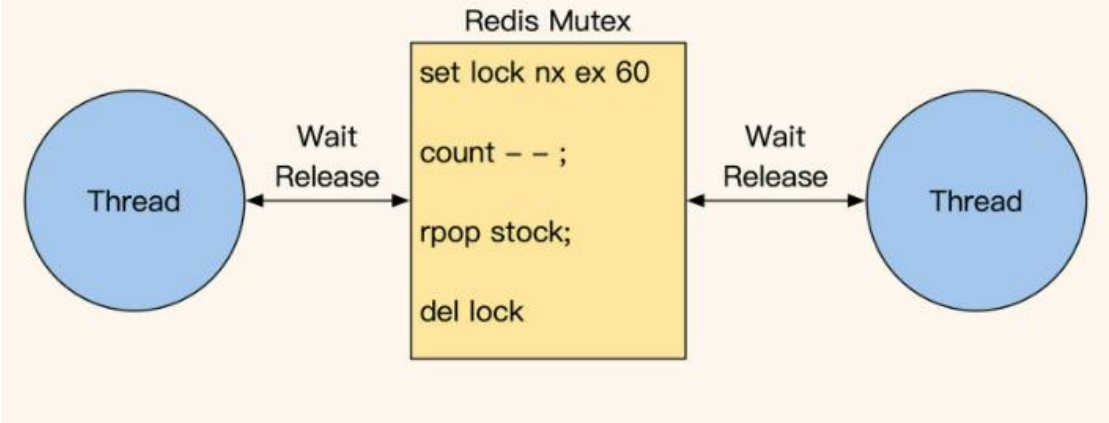
26.5 自旋互斥超时锁

如果我们在库存争抢时需要操作**多个决策 key 才能够完成争抢**，那么原子这种方式是不适合的。因为原子操作的粒度过小，无法做到事务性地维持多个数据的 ACID。

这种多步操作，适合用自旋互斥锁的方式去实现，但流量大的时候不推荐这个方式，因为它的核心在于如果我们要保证用户的体验，我们需要逻辑代码多次循环抢锁，直到拿到锁为止，如下：

```
//业务逻辑需要循环抢锁，如循环 10 次，每次 sleep 10ms，10 次失败后返回失败给用户
//获取锁后设置超时时间，防止进程崩溃后没有释放锁导致问题
//如果获取锁失败会返回 nil
redis> set prod_1475_stock_lock EX 60 NX
OK

//抢锁成功，扣减库存
redis> rpop prod_1475_stock_queue 1
"stock_1"

//扣减数字库存，用于展示
redis> decr prod_1475_stock_1
3

```

```
// 释放锁
redis> del prod_1475_stock_lock
```

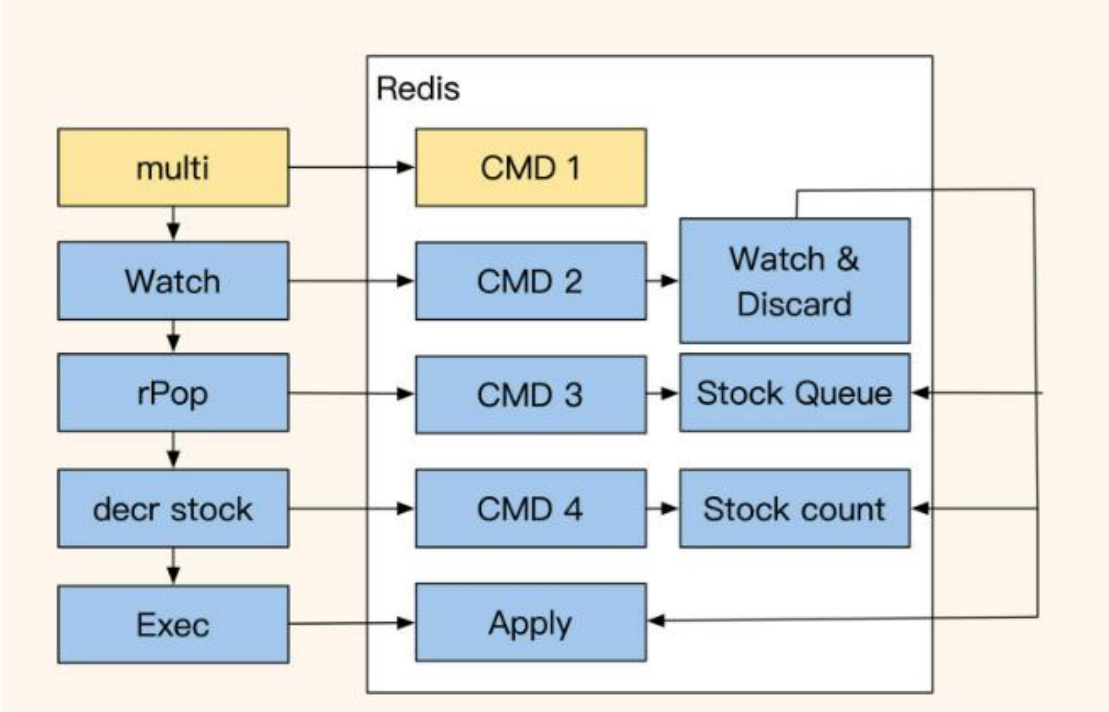


这种方式的缺点在于，在抢锁阶段如果排队抢的线程越多，等待时间就越长，并且由于多线程一起循环 check 的缘故，在高并发期间 Redis 的压力会非常大，如果有 100 人下单，那么有 100 个线程每隔 10ms 就会 check 一次，此时 Redis 的操作次数就是：\[100 线程\\times（1000ms\\div10ms）次 = 10000 ops\]

26.6 CAS 乐观锁：锁操作后置

除此之外我再推荐一个实现方式：CAS 乐观锁。**相对于自旋互斥锁来说，它在并发争抢库存线程少的时候效率会更好。**通常，我们用锁的实现方式是先抢锁，然后，再对数据进行操作。这个方式需要先抢到锁才能继续，而抢锁是有性能损耗的，即使没有其他线程抢锁，这个消耗仍旧存在。

CAS 乐观锁的核心实现为：记录或监控当前库存信息或版本号，对数据进行预操作。



如上图，在操作期间如果发现监控的数值有变化，那么就回滚之前操作；如果期间没有变化，就提交事务的完成操作，操作期间的所有动作都是事务的。

```
//开启事务
redis> multi
OK

// watch 修改值
// 在 exec 期间如果出现其他线程修改，那么会自动失败回滚执行 discard
redis> watch prod_1475_stock_queue prod_1475_stock_1

//事务内对数据进行操作
redis> rpop prod_1475_stock_queue 1
QUEUED

//操作步骤 2
redis> decr prod_1475_stock_1
QUEUED

//执行之前所有操作步骤
//multi 期间 watch 有数值有变化则会回滚
redis> exec
3
```

可以看到，通过这个方式我们可以批量地快速实现库存扣减，并且能大幅减少锁争抢时间。它的好处我们刚才说过，就是争抢线程少时效率特别好，但争抢线程多时会需要大量重试，不过即便如此，CAS 乐观锁也会比用自旋锁实现的性能要好。

当采用这个方式的时候，我建议内部的操作步骤尽量少一些。同时要注意，**如果 Redis 是 Cluster 模式，使用 multi 时必须在一个 slot 内才能保证原子性。**

26.7 Redis Lua 方式实现 Redis 锁

与“事务+乐观锁”类似的实现方式还有一种，就是使用 Redis 的 Lua 脚本实现多步骤库存操作。因为 **Lua 脚本内所有操作都是连续的**，这个操作不会被其他操作打断，所以不存在锁争抢问题。

而且、可以根据不同的情况对 Lua 脚本做不同的操作，业务只需要执行指定的 Lua 脚本传递参数即可实现高性能扣减库存，这样可以大幅度减少业务多次请求等待的 RTT。

为了方便演示怎么执行 Lua 脚本，我使用了 PHP 实现：

```
<?php$script = <<<EOF
// 获取当前库存个数
local stock=tonumber(redis.call('GET',KEYS[1]));
//没找到返回-1
if stock==nil
then
    return -1;
end
//找到了扣减库存个数
local result=stock-ARGV[1];
//如扣减后少于指定个数，那么返回 0
if result<0
then
    return 0;
else
    //如果扣减后仍旧大于 0，那么将结果放回 Redis 内，并返回 1
    redis.call('SET',KEYS[1],result);
    return 1;
end
EOF;

$redis = new\Redis();
$redis->connect('127.0.0.1', 6379);
$result = $redis->eval($script, array("prod_stock", 3), 1);
```

```
25 echo$result;
```

通过这个方式，我们可以远程注入各种连贯带逻辑的操作，并且可以实现一些补库存的操作。

27. 为什么高并发下数据写入不推荐关系数据库?

说到高并发写，就不得不提及新分布式数据库 HTAP，它实现了 OLAP 和 OLTP 的融合，可以同时提供数据分析挖掘和关系查询。

事实上，HTAP 的 OLAP 并不是大数据，或者说它并不是我们印象中每天拿几 T 的日志过来用于离线分析计算的那个大数据。这里更多的是指数据挖掘的最后一环，也就是数据挖掘结果对外查询使用的场景。

对于这个范围的服务，在行业中比较出名的实时数据统计分析的服务有 ElasticSearch、ClickHouse，虽然它们的 QPS 不高，但是能够充分利用系统资源，对大量数据做统计、过滤、查询。但是，相对地，为什么 MySQL 这种关系数据库不适合做类似的事情呢?

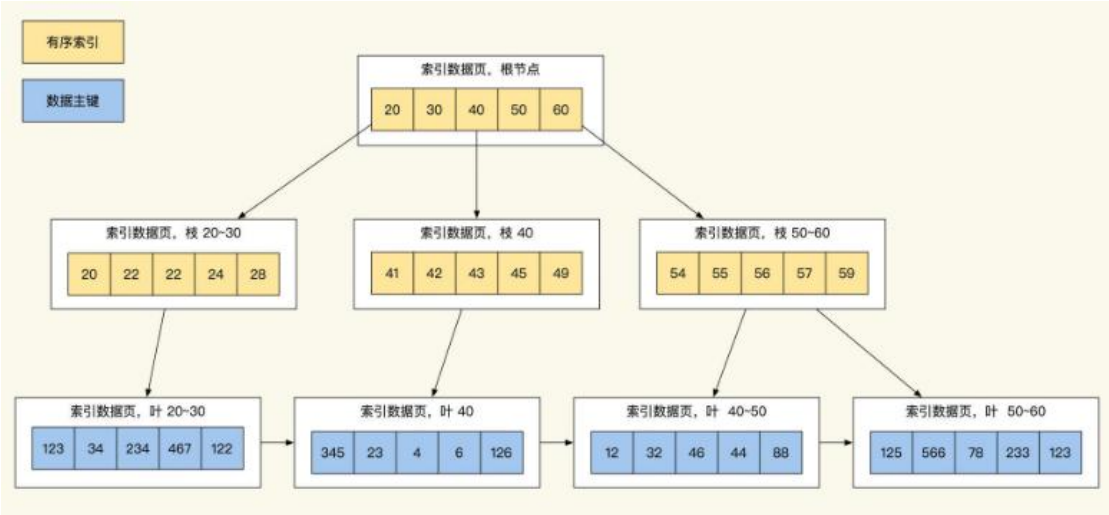
27.1 B+Tree 索引与数据量

MySQL 我们已经很熟悉了，我们常常用它做业务数据存储查询以及信息管理的工作。相信你也听过“一张表不要超过 2000 万行数据”这句话，为什么会有这样的说法呢?核心在于 MySQL 数据库的索引，实现上和我们的需求上有些冲突。具体点说，我们对外的服务基本都要求实时处理，在保证高并发查询的同时，还需要在一秒内找出数据并返回给用户，这意味着对数据大小以及数据量的要求都非常高高。

MySQL 为了达到这个效果，几乎所有**查询**都是通过索引去**缩小扫描数据的范围**，然后再回到表中对范围内数据进行**遍历加工、过滤**，最终拿到我们的业务需要的数据。

事实上，**并不是 MySQL 不能存储更多的数据，而限制我们的多数是数据查询效率问题。**

那么 MySQL 限制查询效率的地方有哪些？请看下图：



众所周知，MySQL 的 InnoDB 数据库的索引是 B+Tree，**B+Tree 的特点在于只有在最底层才会存储真正的数据 ID，通过这个 ID 就可以提取到数据的具体内容**，同时 B+Tree 索引最底层的数据是按索引字段顺序进行存储的。

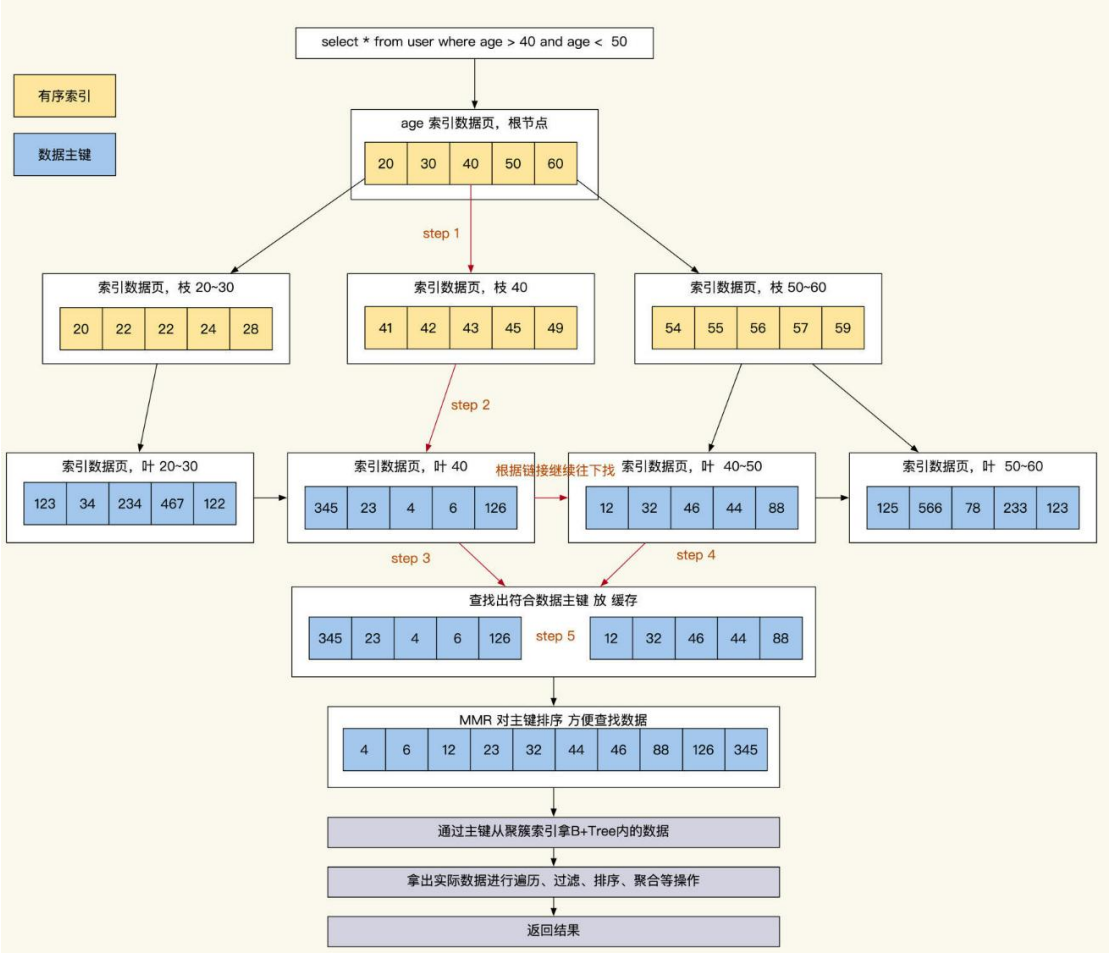
通过**这种设计方式，我们只需进行 1～3 次 IO（树深度决定了 IO 次数）就能找到所查范围内排序好的数据**，而树形的索引最影响查询效率的是**树的深度以及数据量**（数据越独特，筛选的数据范围就越少）。

数据量我么很好理解，只要我们的索引字段足够独特，筛选出来的数据量就是可控的。但是什么会影响到索引树的深度个数呢？这是因为 MySQL 的索引是使用 Page 作为单位进行存储的，而每页只能存储 16KB（innodb_page_size）数据。如果我们每行数据的索引是 1KB，那么除去 Page 页的一些固定结构占用外，一页只能放 16 条数据，这导致树的一些分支装不下更多数据时，我么就需要对索引的深度再加一层。

我们从这个 Page 就可以推导出：索引第一层放 16 条，树第二层大概能放 2 万条，树第三层大概能放 2400 万条，三层的深度 B+Tree 按主键查找数据每次查询需要 **3 次 IO**（一层索引在内存，IO 两次索引，最后一次是拿数据）。

不过这个 2000 万并不是绝对的，如果我们的每行数据是 0.5KB，那么大概在 4000 万以后才会出现第四层深度。而对于辅助索引，一页 Page 能存放 1170 个索引节点（主键 bigint8 字节+数据指针 6 字节），三层深度的辅助索引大概能记录 10 亿条索引记录。

可以看到，我们的数据存储数量超过三层时，每次数据操作**需要更多的 IO 操作**来进行查询，这样做的后果就是查询数据返回的速度变慢。所以，很多互联网系统为了保持服务的高效，会定期整理数据。



通过上面的讲解，相信你已经对整个查询有画面感了：当我们查询时，通过 1～3 次 IO 查找辅助索引，从而找到一批数据主键 ID。然后，通过 MySQL 的 MMR 算法将这些 ID 做排序，再回表去**聚簇索引**按取值范围提取在子叶上的业务数据，将这些数据边取边算或一起取出再进行聚合排序后，之后再返回结果。

可以看到，我们常用的数据库之所以快，核心在于索引用得好。由于加工数据光用索引是无法完成的，我们还需要找到具体的数据进行再次加工，才能得到我们业务所需的数据，这也是为什么**我们的字段数据长度和数据量会直接影响我们对外服务的响应速度。**

同时请你注意，我们一个表不能增加过多的索引，因为索引太多会影响到表插入的性能。并且我们的查询要遵循左前缀原则来逐步缩小查找的数据范围，而不能利用多个 CPU 并行去查询索引数据。这些大大限制了我们对大数据的处理能力。

另外，如果有数据持续高并发插入数据库会导致 MySQL 集群工作异常、主库响应缓慢、主从同步延迟加大等问题。**从部署结构上来说，MySQL 只有主从模式**，大批量的数据写操作只能由主库承受，当我们数据写入缓慢时客户端只能等待服务端响应，严重影响数据写入效率。

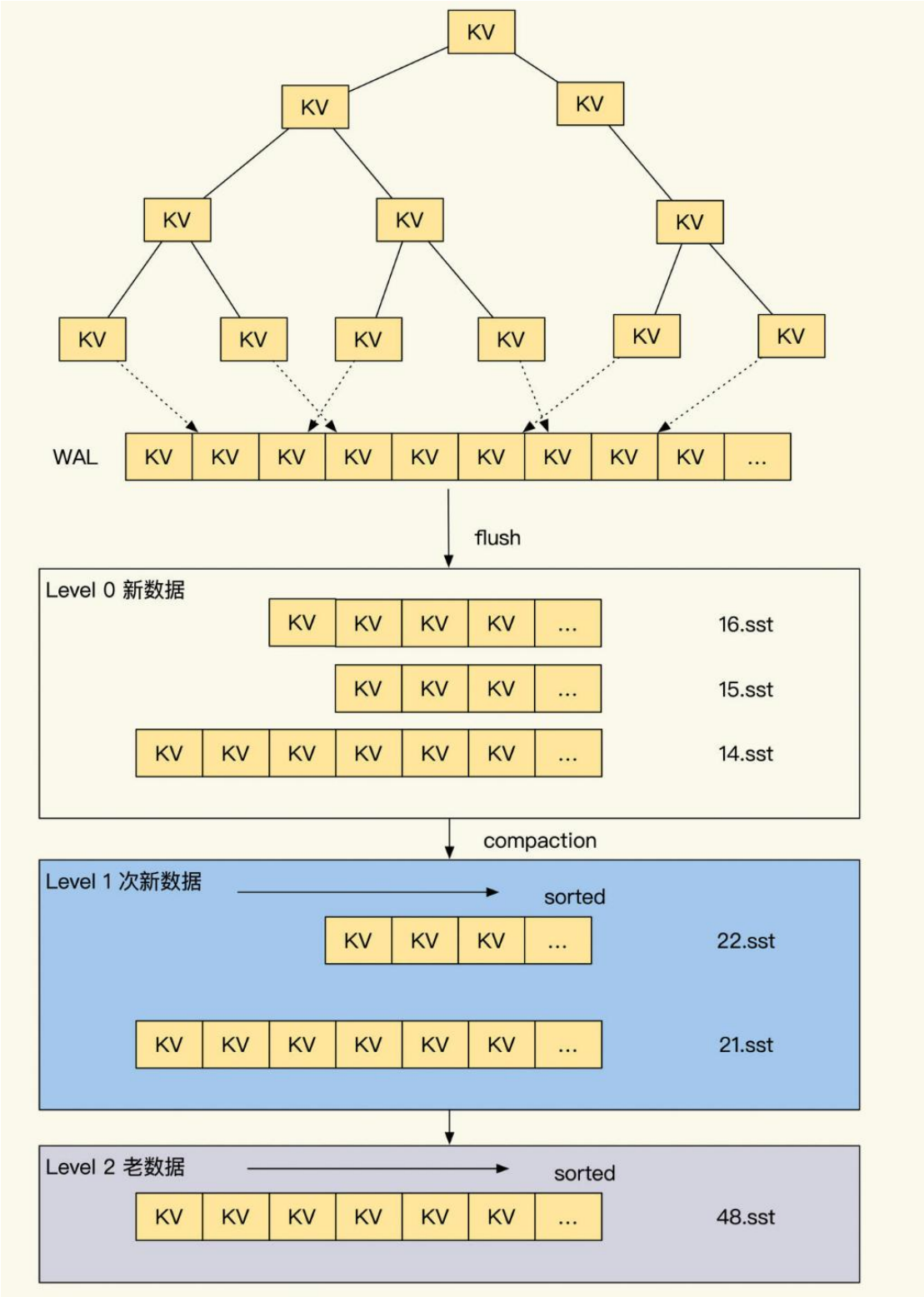
看到这里，相信你已经理解为什么关系型数据库并不适合太多的数据，其实 OLAP 的数据库也不一定适合大量的数据，正如我提到的 OLAP 提供的服务很多也需要实时响应，所以很多时候这类数据库对外提供服务的时候，计算用的数据也是做过深加工的。但即使如此，OLAP 和 OLTP 底层实现仍旧有很多不同。

我们先来分析索引的不同。OLTP 常用的是 B+Tree，我们知道，B+tree 索引是一个整体的树，当我们的数据量大时会影响索引树的深度，如果深度过高就会严重影响其工作效率。对于大量数据，OLAP 服务会用什么类型的索引呢？

27.2 稀疏索引 LSM Tree 与存储

这里重点介绍一下 LSM 索引。我第一次见到 LSM Tree 还是从 RocksDB（以及 LevelDB）上看到的，RocksDB 之所以能够得到快速推广并受到欢迎，主要是因为它利用了**磁盘顺序写性能超绝**的特性，并以较小的性能查询代价提供了写多读少的 KV 数据存储查询服务，这和关系数据库的存储有很大的不同。

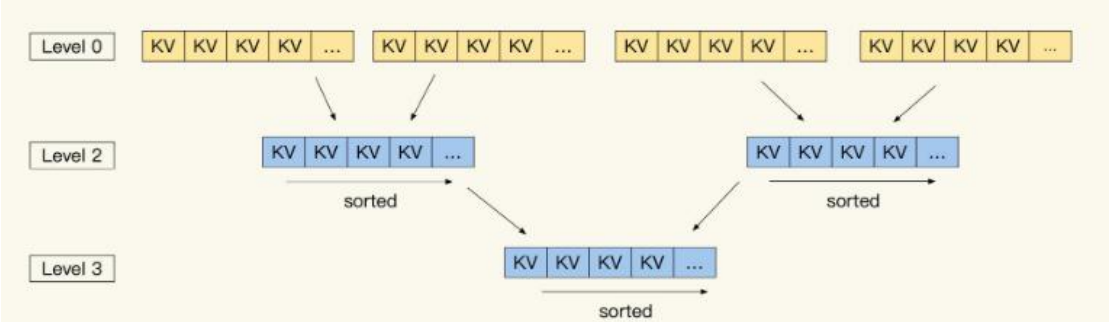
为了更好理解，我们详细讲讲 Rocksdb 稀疏索引是如何实现的，如下图所示：



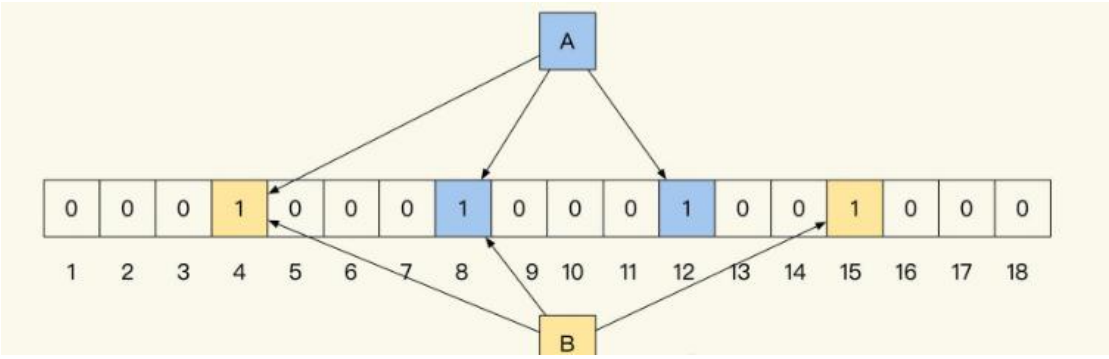
我们前面讲过，B+Tree 是一个大树，它是一个聚合的完整整体，任何数据的增删改都是在这个整体内进行操作，这就导致了大量的随机读写 IO。

RocksDB LSM 则不同，它是由一棵棵小树组成，当我们新数据写入时会在内存中暂存，这样能够获得非常大的写并发处理能力。而当内存中数据积累到一定程度后，会将内存中数据和索引做**顺序写**，落地形成一个数据块。

这个数据块内保存着一棵小树和具体的数据，新生成的数据块会保存在 Level 0 层（最大有几层可配置），Level 0 层会有多个类似的数据块文件。结构如下图所示：



每一层的数据块和数据量超过一定程度时，RocksDB 合并不同 Level 的数据，将多个数据块内的数据和索引合并在一起，并推送到 Level 的下一层。通过这个方式，每一层的数据块个数和数据量就能保持一定的数量，合并后的数据会更紧密、更容易被找到。这样的设计，可以让一个 Key 存在于多个 Level 或者数据块中，但是最新的常用的数据肯定是在 Level 最顶部或内存（0～4 层，0 为顶部）中最新的数据块内。



而当我们查询一个 key 的时候，RocksDB 会先查内存。如果没找到，会从 Level 0 层到下层，每层按生成最新到最老的顺序去查询每层的数据块。同时为了减少 IO 次数，每个数据块都会有一个 BloomFIlter 辅助索引，来辅助确认这个数据块中是否**可能有**对应的 Key；如果当前数据块没有，那么可以快速去找下一个数据块，直到找到为止。当然，最惨的情况是遍历所有数据块。

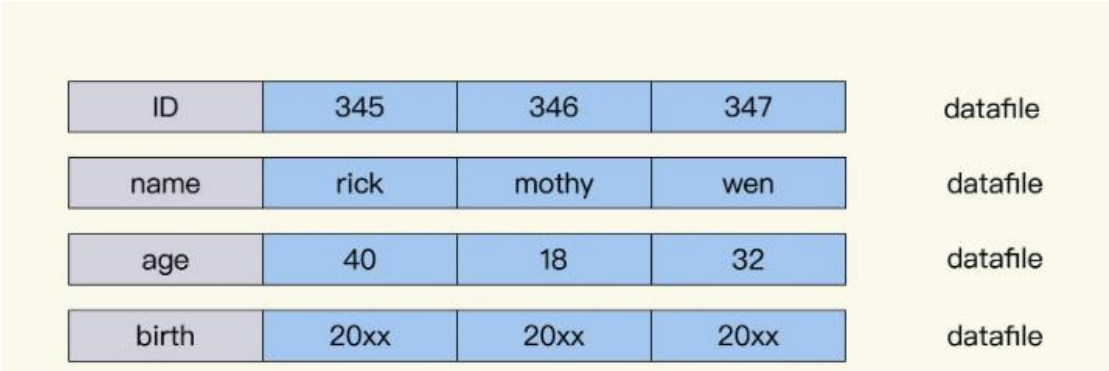
可以看到，这个方式虽然放弃了整体索引的一致性，却换来了更高效的写性能。在读取时通过**遍历所有子树来**查找，减少了写入时对树的合并代价。

LSM 这种方式的数据存储在 OLAP 数据库中很常用，因为 OLAP 多数属于**写多读少**，而当我们使用 OLAP 对外提供数据服务的时候，多数会通过缓存来帮助数据库承受更大的读取压力。

27.3 列存储数据库

说到这里，不得不提 OLAP 数据库和 OLTP 数据之间的另一个区别。我们常用的关系型数据库，属于行式存储数据库 Row-based，表数据结构是什么样，它就会按表结构的字段顺序进行存储；而大数据挖掘使用的数据库普遍使用列式存储（Column-based），原因在于我们用关系数据库保存的多数是实体属性和实体关系，很多查询每一列都是不可或缺的。

实体表–行存储



画像行为记录表 – 列存储

但是，实时数据分析则相反，很多情况下常用一行表示一个用户或主要实体（聚合根），而列保存这个用户或主要实体是否买过某物、使用过什么 App、去过哪里、开什么车、点过什么食品、哪里人等等。

这样组织出来的数据，做数据挖掘、分析对比很方便，不过也会导致一个表有成百上千个字段，如果用行存储的数据引擎，我们对数据的筛选是一行行进行读取的，会浪费大量的 IO 读取。

而列存储引擎可以指定用什么字段读取所需字段的数据，并且这个方式能够充分利用到磁盘顺序读写的性能，大大提高这种列筛选式的查询，并且列方式更好进行数据压缩，在实时计算领域做数据统计分析的时候，表现会更好。

ID	name	age	birth	car	wine	country	...
345	rick	40	20xx	bmw	white	China	...
346	mothy	18	20xx	tokoyo	beer	China	...
347	wen	32	20xx	jeep	n/a	n/a	...

用户画像行为记录表–列存储常用效果

到了这里相信你已经发现，使用场景不同，数据底层的实现也需要不同的方式才能换来更好的性能和性价比。随着行业变得更加成熟，这些需求和特点会不断挖掘、总结、合并到我们的底层服务当中，逐渐降低我们的工作难度和工作量。

27.4 HTAP

通过前面的讲解，我么可以看到 OLAP 和 OLTP 数据库各有特点，并且有不同的发展方向，事实上它们对外提供的数据查询服务都是期望实时快速的，而不同在于如何存储和查找索引。

最近几年流行将两者结合成一套数据库集群服务，同时提供 OLAP 以及 OLTP 服务，并且相互不影响，实现行数据库与列数据库的互补。

2022 年国产数据库行业内 OceanBase、PolarDB 等云厂商提供的分布式数据库都在紧锣密鼓地开始支持 HTAP。这让我们可以保存同一份数据，根据不同查询的范围触发不同的引擎，共同对外提供数据服务。

可以看到，未来的某一天，我们的数据库既能快速地实时分析，又能快速提供业务数据服务。逐渐地，数据服务底层会出现多套存储、索引结构来帮助我们更方便地实现数据库。

而目前常见的 HTAP 实现方式，普遍采用一个服务集群内同一套数据支持多种数据存储方式（行存储、列存储），通过对数据提供不同的索引来实现 OLAP 及 OLTP 需求，而用户在查询时，可以指定或由数据库查询引擎根据 SQL 和数据情况，自动选择使用哪个引擎来优化查询。

28. 如果让你设计一个分布式链路跟踪系统？你怎么做？

分布式链路跟踪服务属于写多读少的服务，是我们线上排查问题的重要支撑。我经历过的一个系统，同时支持着多条业务线，实际用上的服务器有两百台左右，这种量级的系统想排查故障，难度可想而知。

因此，我结合 ELK 特性设计了一套十分简单的全量日志分布式链路跟踪，把日志串了起来，大大降低了系统排查难度。

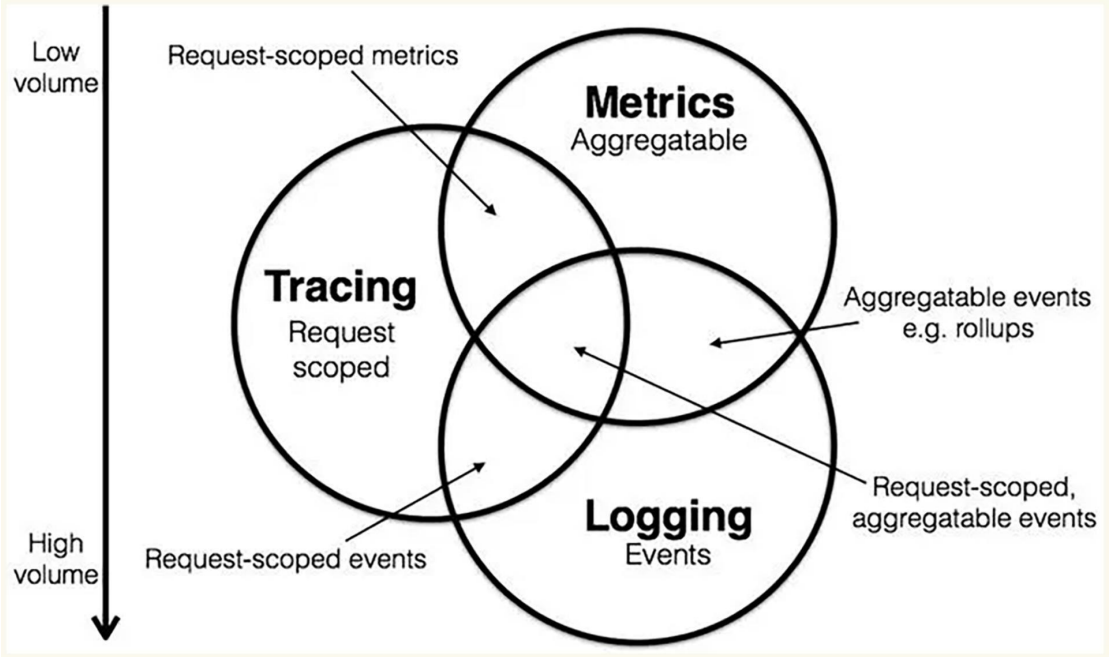
目前市面上开源提供的分布式链路跟踪都很抽象，当业务复杂到一定程度的时候，为核心系统定制一个符合自己业务需要的链路跟踪，还是很有必要的。

事实上，实现一个分布式链路跟踪并不难，而是难在埋点、数据传输、存储、分析上，如果你的团队拥有这些能力，也可以很快制作出一个链路跟踪系统。所以下面我们一起看看，如何实现一个简单的定制化分布式链路跟踪。

28.1 监控行业发展现状

在学习如何制作一个简单的分布式链路跟踪之前，为了更好了解这个链路跟踪的设计特点，我们先简单了解一下监控行业的现状。

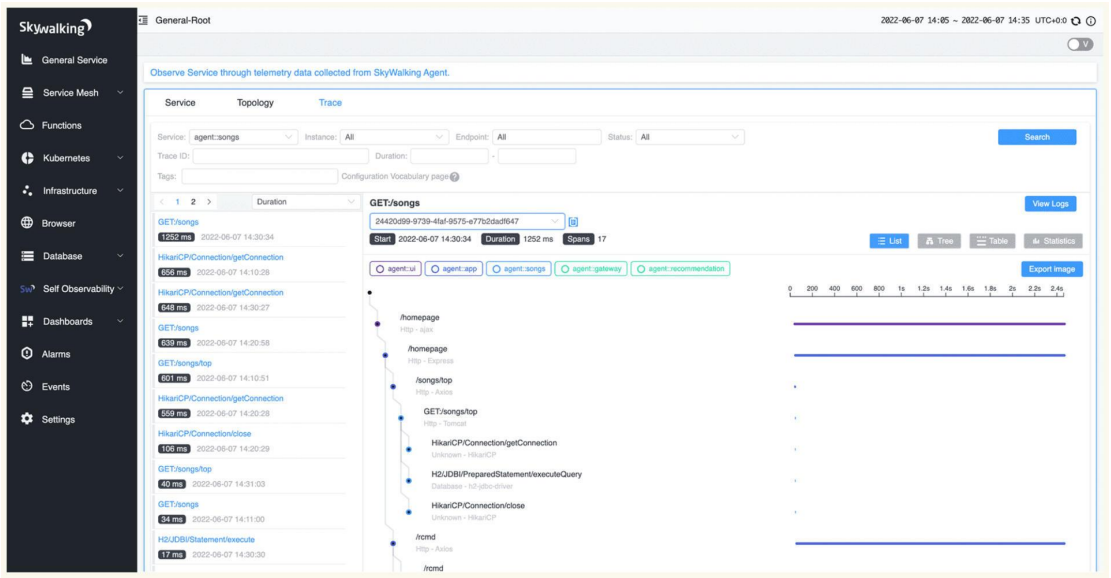
最近监控行业有一次大革新，现代的链路跟踪标准已经不拘泥于请求的链路跟踪，目前已经开始进行融合，新的标准和我们定制化的分布式链路跟踪的设计思路很相似，即 Trace、Metrics、日志合并成一套系统进行建设。

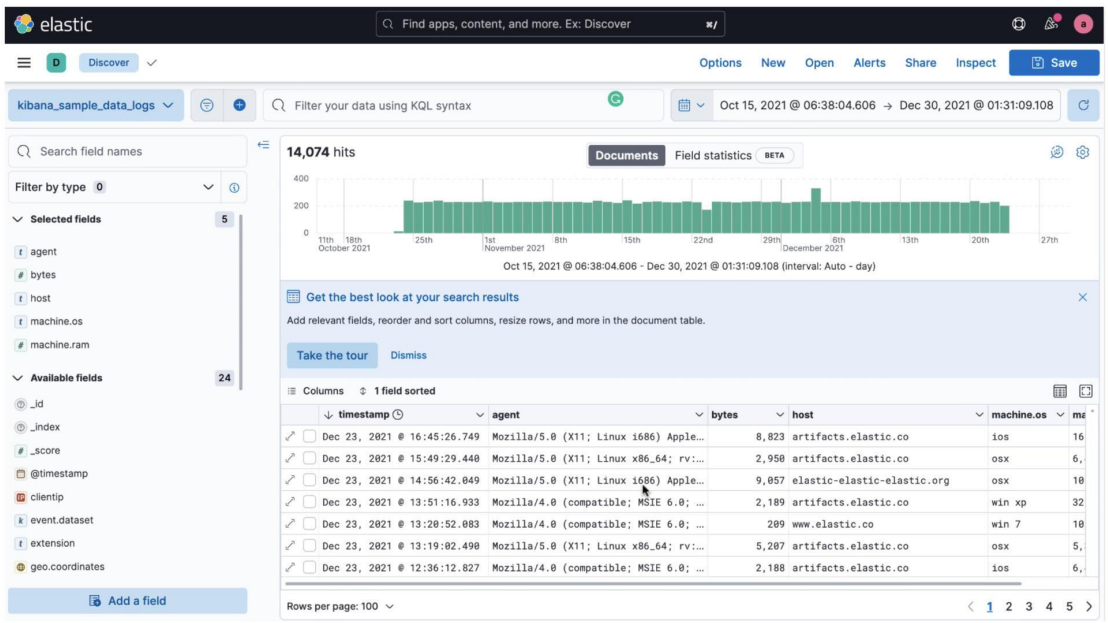


在此之前，常见监控系统主要有三种类型：Metrics、Tracing 和 Logging。

常见的开源 Metrics 有 Zabbix、Nagios、Prometheus、InfluxDb、OpenFalcon，主要做各种量化指标汇总统计，比如监控系统的容量剩余、每秒请求量、平均响应速度、某个时段请求量多少。

常见的开源链路跟踪有 Jaeger、Zipkin、Pinpoint、Skywalking，主要是通过分析每次请求链路监控分析的系统，我么可以通过 TraceID 查找一次请求的依赖及调用链路，分析故障点和传导过程的耗时。



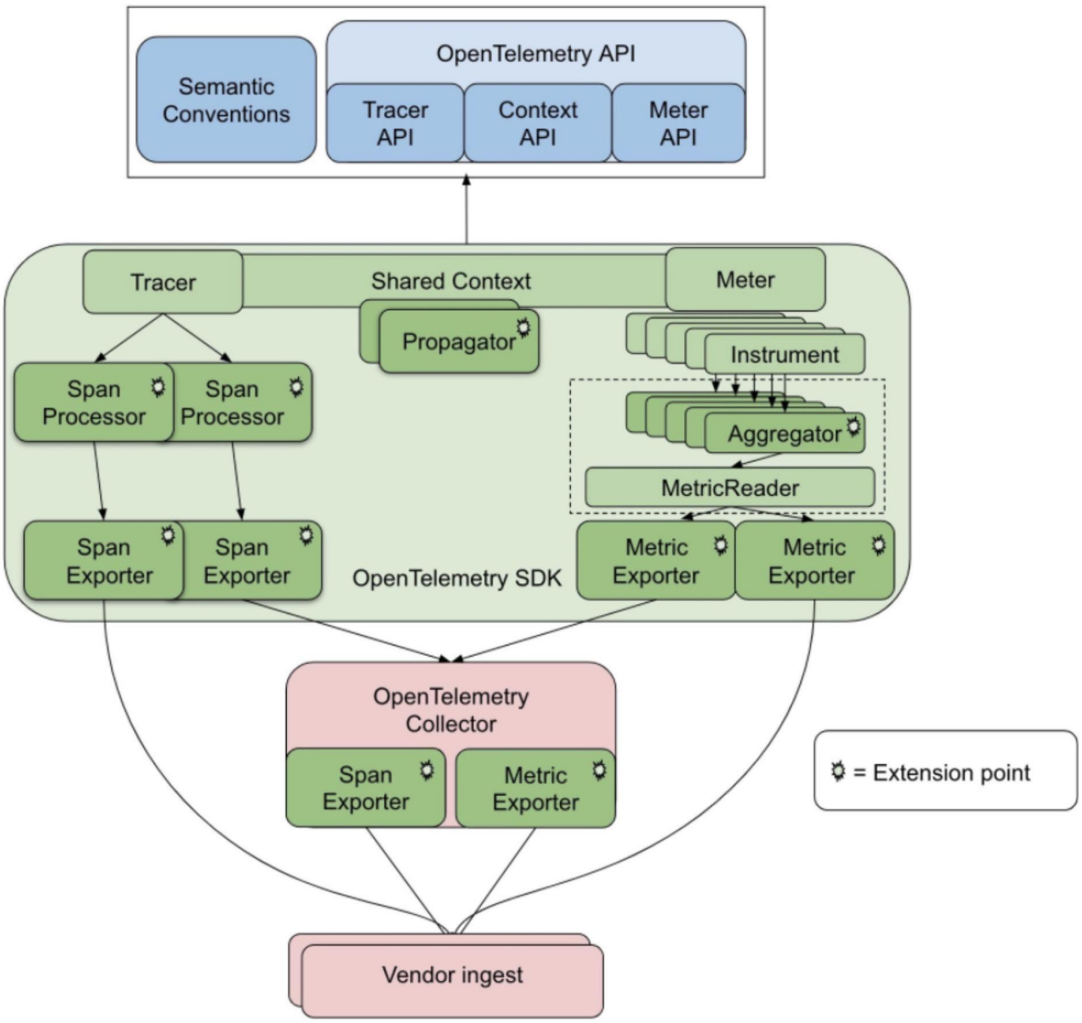


而常见的开源 Logging 有 ELK、Loki、Loggly，主要是对文本日志的收集归类整理，可以对错误日志进行汇总、警告，并分析系统错误异常等情况。

这三种监控系统可以说是大服务集群监控的主要支柱，它们各有优点，但一直是分别建设的。这让我们的系统监控存在一些割裂和功能重复，而且每一个标准都需要独立建设一个系统，然后在不同界面对同一个故障进行分析，排查问题时十分不便。

随着行业发展，三位一体的标准应运而生，这就是 OpenTelemetry 标准（集成了 OpenCensus、OpenTracing 标准）。这个标准将 Metrics+Tracing+Logging 集成一体，这样我们监控系统的时候就可以通过三个维度综合观测系统运转情况。

常见 OpenTelemetry 开源项目中的 Prometheus、Jaeger 正在遵循这个标准逐步改进实现 OpenTelemetry 实现的结构如下图所示：



事实上，分布式链路跟踪系统及监控主要提供了以下支撑服务：

- 监控日志标准
- 埋点 SDK（AOP 或侵入式）
- 日志收集
- 分布式日志传输
- 分布式日志存储
- 分布式检索计算
- 分布式实时分析
- 个性化定制指标盘
- 系统警告

我建议使用 ELK 提供的功能去实现分布式链路跟踪系统，因为它已经完整提供了如下功能：

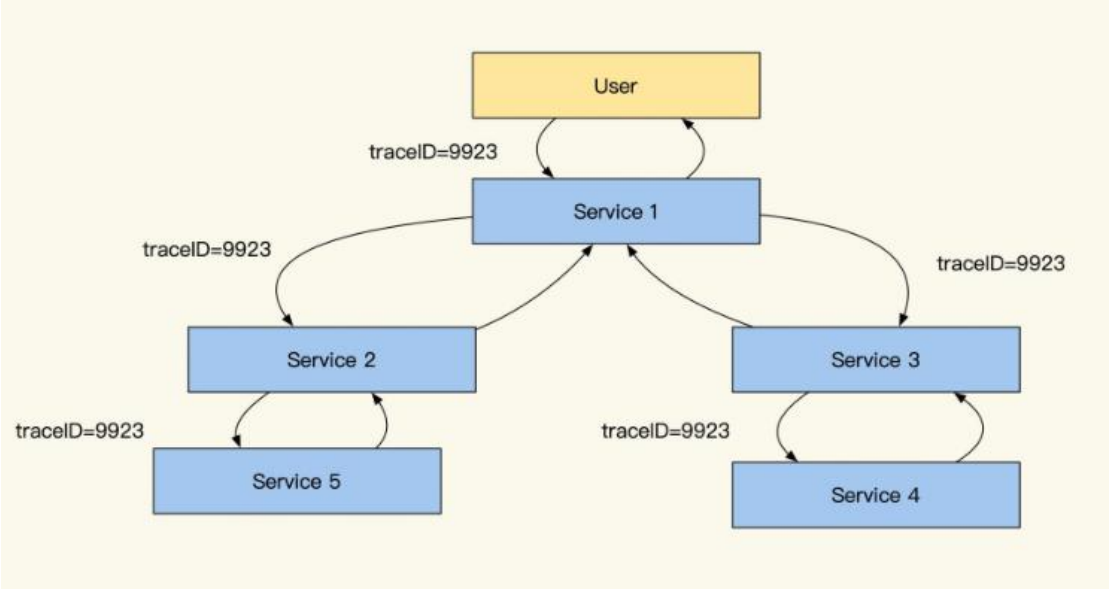
- 日志收集（Filebeat）
- 日志传输（Kafka+Logstash）
- 日志存储（Elasticsearch）
- 检索计算（Elasticsearch + Kibana）
- 实时分析（Kibana）
- 个性定制表格查询（Kibana）

这样一来，我只需要制定日志格式、埋点 SDK，即可实现一个具有分布式链路跟踪、Metrics、日志分析系统。

事实上，Log、Metrics、trace 三种监控体系最大的区别就是日志格式标准，底层实现其实是很相似的。既然 ELK 已提供我们需要的分布式相关服务，下面我简单讲讲日志格式和 SDK 埋点，通过这两个点我们就可以窥见分布式链路跟踪的全貌。

28.2 TraceID 单次请求标识

可以说，要想构建一个简单的 Trace 系统，我们首先要做的就是生成并传递 TraceID。



分布式链路跟踪的原理其实很简单，就是在请求发起方发送请求时或服务被请求时生成一个 UUID，被请求期间的业务产生的任何日志（Warning、Info、Debug、Error）、任何**依赖资源请求**（MySQL、Kafka、Redis）、任何内部接口调用（Restful、Http、RPC）都会带上这个 UUID。

这样，当我们把所有拥有同样 UUID 的日志收集起来时，就可以根据时间（有误差）、RPCID（后续会介绍 RPCID）或 SpanID，将它们按依赖请求顺序串起来。

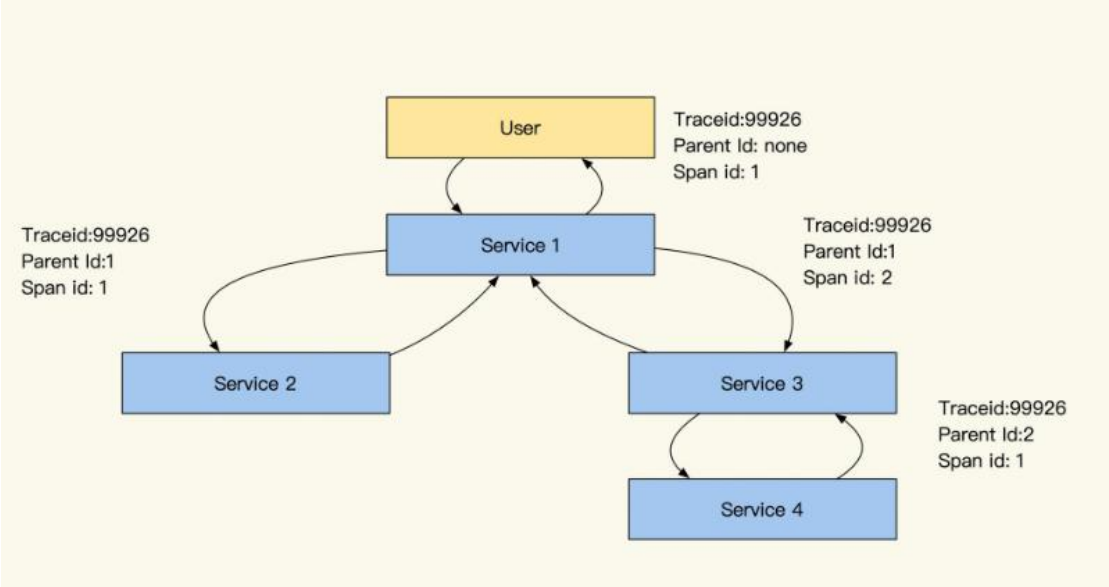
只要日志足够详细，我们就能监控到系统大部分的工作状态，比如用户请求一个服务会调用多少个接口，每个数据查询的 SQL 以及具体耗时调用的内网请求参数是什么、调用的内网请求返回是什么、内网被请求的接口又做了哪些操作、产生了哪些异常信息等等。

同时，我们可以通过对这些日志做归类分析，分析项目之间的调用关系、项目整体健康程度、对链路深挖自动识别出故障点等，帮助我们主动、快速地查找问题。

28.3 “RPCID” VS “SpanID 链路标识”

那么如何将汇总起来的日志串联起来呢？有两种方式：span（链式记录依赖）和 RPCID（层级计数器）。我们在记录日志带上 UUID 的同时，也带上 RPCID 这个信息，通过它帮我们把日志关联关系串联起来，那么这两种方式有什么区别呢？

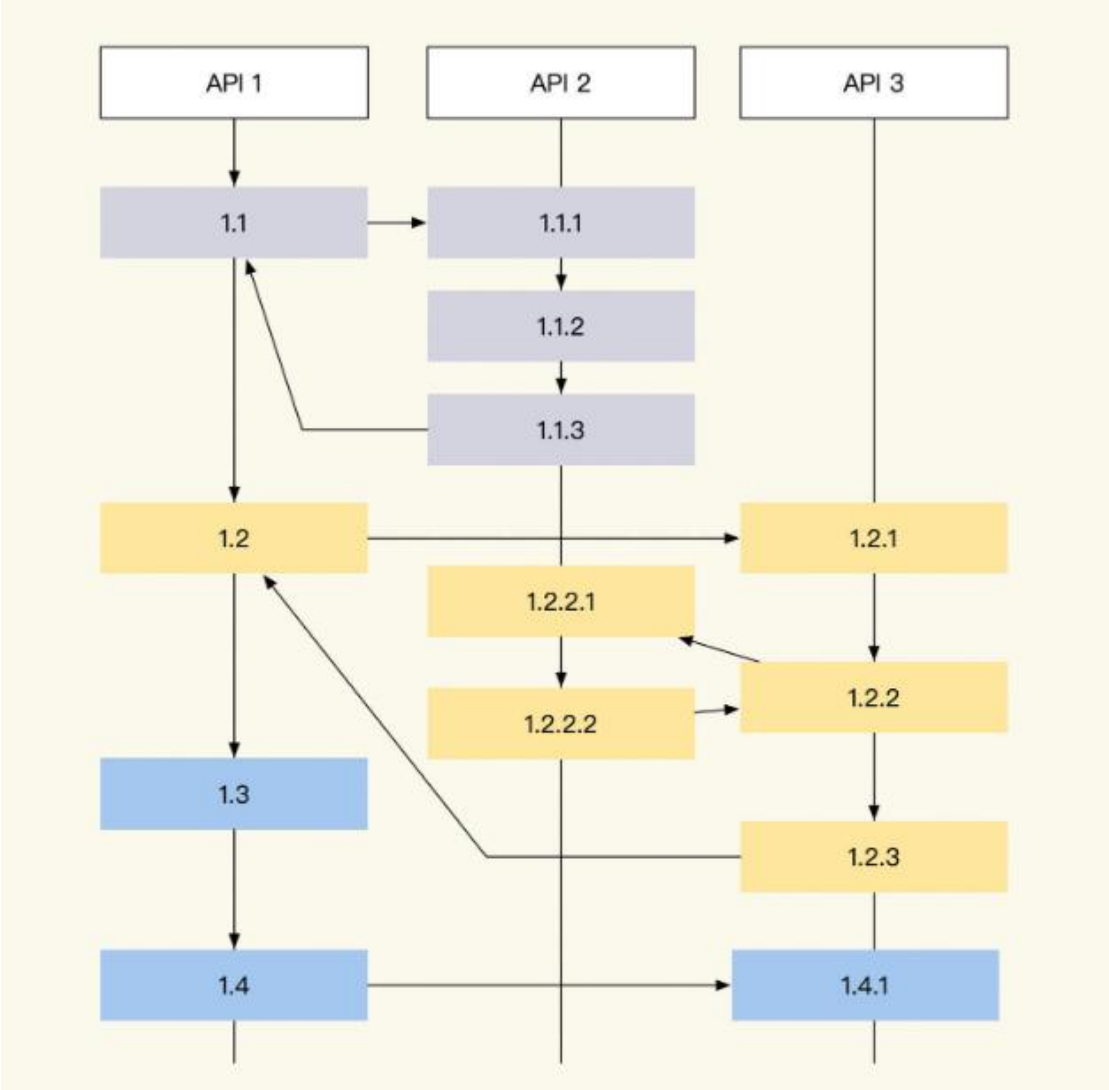
我们先看看 span 实现，具体如下图：



结合上图，我们分析一下 span 的链式依赖记录方式。对于代码来说，写的很多功能会被封装成功能模块（Service、Model），我们通过组合不同的模块实现业务功能，并且记录这两个模块、两个服务间或是资源的调用依赖关系。

span 这个设计会通过记录自己上游依赖服务的 SpanID 实现上下游关系关联（放在 Parent ID 中），通过整理 span 之间的依赖关系就能组合成一个调用链路树。

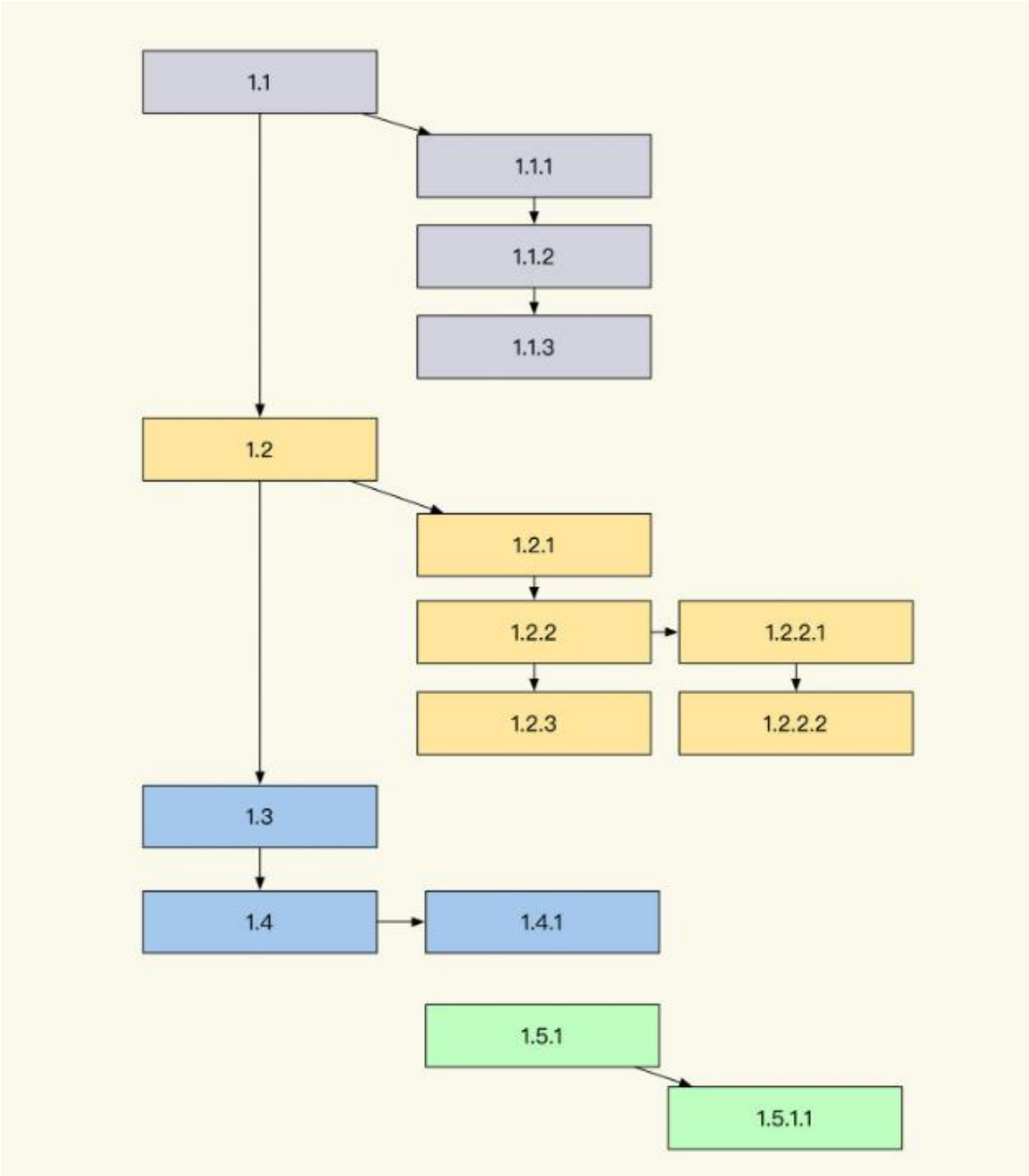
那 RPCID 方式是什么样的呢？RPCID 也叫层级计数器，我在微博和好未来时都用过，为了方便理解，我们来看下面这张图：



你看，RPCID 的层级计数器实现很简单，第一个接口生成 RPCID 为 **1.1** ，RPCID 的**前缀**是 1，**计数器**是 1（日志记录为 1.1）。

当所在接口请求其他接口或数据服务（MySQL、Redis、API、Kafka）时，**计数器**+1，并在请求当中带上 1.2 这个数值（因为当前的**前缀** + “.” + **计数器值 = 1.2)** ，等到返回结果后，继续请求下一个资源时继续+1，期间产生的任何日志都会记录当前 **前缀**+ “.” +**计数器值。**

每一层收到了前缀后，都在后面加了一个累加的计数器，实际效果如下图所示：



而被请求的接口收到请求时，如果请求传递了 TraceID，那么被请求的服务会继续使用传递过来的 TraceID，如果请求没有 TraceID 则自己生成一个。同样地，如果传递了 RPCID，那么被请求的服务会将传递来的 RPCID 当作前缀，计数器从 1 开始计数。

相对于 span，通过这个层级计数器做出来的 RPCID 有两个优点。

第一个优点是我们可以记录请求方日志，如果被请求方没有记录日志，那么还可以通过请求方日志观测分析被调用方性能（MySQL、Redis）。

另一个优点是哪怕日志收集得不全，丢失了一些，我们还可以通过前缀有几个分隔符，判断出日志所在层级进行渲染。举个例子，假设我们不知道上图的 1.5.1 是谁调用的，但是根据它的 UUID 和层级 1.5.1 这些信息，渲染的时候，我们仍旧可以渲染它大概的链路位置。

除此之外，我们可以利用 AOP 顺便将各个模块做一个 Metrics 性能统计分析，分析各个模块的耗时、调用次数做周期统计。

同时，通过这个维度采样统计数据，能够帮助我们分析这个模块的性能和错误率。由于 Metrics 这个方式产生的日志量很小，有些统计是每 10 秒才会产生一条 Metrics 统计日志，统计的数值很方便对比，很有参考价值。

但是你要注意，对于一个模块内有多个分支逻辑时，Metrics 很多时候取的是平均数，偶发的超时在平均数上看不出来，所以我们需要另外记录一下最大最小的延迟，才可以更好地展现。同时，这种统计只是让我们知道这个模块是否有性能问题，但是无法帮助我们分析具体的原因。

回到之前的话题，我们前面提到，请求和被请求方通过传递 TraceID 和 RPCID（或 SpanID）来实现链路的跟踪，我列举几个常见的方式供你参考：

- HTTP 协议放在 Header；
- RPC 协议放在 meta 中传递；
- 队列可以放在消息体的 Header 中，或直接在消息体中传递；
- 其他特殊情况下可以通过网址请求参数传递。

那么应用内多线程和多协程之间如何传递 TraceID 呢？一般来说，我们会通过复制一份 Context 传递进入线程或协程，并且如果它们之前是并行关系，我们复制之后需要对下发之前的 RPCID 计数器加 1，并把前缀和计数器合并成新的前缀，以此区分并行的链路。

除此之外，我们还做了一些特殊设计，当我们的请求中带一个特殊的密语，并且设置类似 X-DEBUG Header 等于 1 时，我们可以开启在线 debug 模式，在被调用接口及所有依赖的服务都会输出 debug 级别的日志，这样我们临时排查线上问题会更方便。

28.4 日志类型定义

可以说，只要让日志输出当前的 TraceId 和 RPCID（SpanID），并在请求所有依赖资源时把计数传递给它们，就完成了大部分的分布式链路跟踪。下面是我定制的一些日志类型和日志格式，供你参考：

```
## 日志类型

* request.info 当前被请求接口的相关信息，如被请求接口，耗时，参数，返回值，客户端信息
* mysql.connect mysql 连接时长
* mysql.connect.error mysql 链接错误信息
* mysql.request mysql 执行查询命令时长及相关信息
* mysql.request.error mysql 操作时报错的相关信息
* redis.connect redis 链接时长
* redis.connect.error redis 链接错误信息
* redis.request redis 执行命令
* redis.request.error redis 操作时错误
* memcache.connect
* memcache.connect.error
* memcache.request.error
* http.get 另外可以支持 restful 操作 get put delete
* http.post
* http.*.error

## Metric 日志类型

* metric.counter
...略

## 分级日志类型
* log.debug: debug log
* log.trace: trace log
* log.notice: notice log
* log.info: info log
* log.error: application error log
* log.alarm: alarm log
* log.exception: exception log
```

你会发现，所有对依赖资源的请求都有相关日志，这样可以帮助我们分析所有依赖资源的耗时及返回内容。此外，我们的分级日志也在 trace 跟踪范围内，通过日志信息可以更好地分析问题。而且，如果我们监控的是静态语言，还可以像之前说的那样，对一些模块做 Metrics，定期产生日志。

28.5 日志格式样例

日志建议使用 JSON 格式，所有字段除了标注为 string 的都建议保存为字符串类型，每个字段必须是固定数据类型，选填内容如果没有内容就直接不输出。

这样设计其实是为了适配 Elasticsearch+Kibana，Kibana 提供了日志的聚合、检索、条件检索和数值聚合，但是对字段格式很敏感，不是数值类型就无法聚合对比。

下面我给你举一个例子用于链路跟踪和监控，你主要关注它的类型和字段用途。

```
{
    "name": "string:全量字段介绍,必填，用于区分日志类型，上面的日志列表内容写这里",
    "trace_id": "string:traceid，必填",
    "rpc_id": "string:RPCID，服务端链路必填，客户端非必填",
    "department":"部门缩写如 client_frontend 必填",
    "version": "string:当前服务版本 cpp-client-1.1 php-baseserver-1.4 java-rti-1.9，建议都填",
    "timestamp": "int:日志记录时间，单位秒，必填",

    "duration": "float:消耗时间，浮点数 单位秒，能填就填",
    "module": "string:模块路径，建议格式应用名称_模块名称_函数名称_动作，必填",
    "source": "string:请求来源 如果是网页可以记录 ref page，选填",
    "uid": "string:当前用户 uid，如果没有则填写为 0 长度字符串，可选填，能够帮助分析用户一段时间行为",
    "pid": "string:进程 pid，如果没有填写为 0 长度字符串，如果有线程可以为 pid-tid 格式，可选填",
    "server_ip": "string 当前服务器 ip，必填",
    "client_ip": "string 客户端 ip，选填",
    "user_agent": "string curl/7.29.0 选填",
    "host": "string 链接目标的 ip 及端口号，用于区分环境 12.123.23.1:3306，选填",
    "instance_name": "string 数据库连接配置的标识，比如 rti 的数据库连接，选填",
    "db": "string 数据库名称如：peiyou_stastic，选填",
    "code": "string:各种驱动或错误或服务的错误码，选填，报错误必填",
    "msg": "string 错误信息或其他提示信息，选填，报错误必填",
    "backtrace": "string 错误的 backtrace 信息，选填，报错误必填",
    "action": "string 可以是 url、sql、redis 命令、所有让远程执行的命令，必填",
    "param": "string 通用参数模板，用于和 script 配合，记录所有请求参数，必填",
    "file": "string userinfo.php，选填",
    "line": "string 232，选填",
    "response": "string:请求返回的结果，可以是本接口或其他资源返回的数据，如果数据太长会影响性能，选填",
    "response_length": "int:相应内容结果的长度，选填",
    "dns_duration": "float dns 解析时间，一般 http mysql 请求域名的时候会出现此选项，选填",
    "extra": "json 放什么都可以,用户所有附加数据都扔这里"
}

```

样例
被请求日志
```
{
      "x_name": "request.info",
      "x_trace_id": "123jiojfdsao",
      "x_rpc_id": "0.1",
      "x_version": "php-baseserver-4.0",
      "x_department":"tal_client_frontend",
      "x_timestamp": 1506480162,
      "x_duration": 0.021,
      "x_uid": "9527",
      "x_pid": "123",
      "x_module": "js_game1_start",
      "x_user_agent": "string curl/7.29.0",
      "x_action": "http://testapi.speiyou.com/v3/user/getinfo?id=9527",
      "x_server_ip": "192.168.1.1:80",
      "x_client_ip": "192.168.1.123",
       "x_param": "json string",
      "x_source": "www.baidu.com",
      "x_code": "200",
      "x_response": "json:api result",
      "x_response_len": 12324
}
```

mysql 链接性能日志
```
{
      "x_name": "mysql.connect",
      "x_trace_id": "123jiojfdsao",
      "x_rpc_id": "0.2",
      "x_version": "php-baseserver-4",
      "x_department":"tal_client_frontend",
      "x_timestamp": 1506480162,
      "x_duration": 0.024,
      "x_uid": "9527",
      "x_pid": "123",
      "x_module": "js_mysql_connect",
      "x_instance_name": "default",
      "x_host": "12.123.23.1:3306",
      "x_db": "tal_game_round",
      "x_msg": "ok",
      "x_code": "1",
      "x_response": "json:****"
}
```

Mysql 请求日志

```
{
    "x_name": "mysql.request",
    "x_trace_id": "123jiojfdsao",
    "x_rpc_id": "0.2",
    "x_version": "php-4",
    "x_department":"tal_client_frontend",
    "x_timestamp": 1506480162,
    "x_duration": 0.024,
    "x_uid": "9527",
    "x_pid": "123",
    "x_module": "js_game1_round_sigup",
    "x_instance_name": "default",
    "x_host": "12.123.23.1:3306",
    "x_db": "tal_game_round",
    "x_action": "select * from xxx where xxxx",
    "x_param": "json string",
    "x_code": "1",
    "x_msg": "ok",
    "x_response": "json:****"
}

### http 请求日志
{
    "x_name": "http.post",
    "x_trace_id": "123jiojfdsao",
    "x_department":"tal_client_frontend",
    "x_rpc_id": "0.3",
    "x_version": "php-4",
    "x_timestamp": 1506480162,
    "x_duration": 0.214,
    "x_uid": "9527",
    "x_pid": "123",
    "x_module": "js_game1_round_win_report",
    "x_action": "http://testapi.speiyou.com/v3/game/report",
    "x_param": "json:",
    "x_server_ip": "192.168.1.1",
    "x_msg": "ok",
    "x_code": "200",
    "x_response_len": 12324,
    "x_response": "json:responsexxxx",
    "x_dns_duration": 0.001
}

### level log info 日志
{
```

```
    "x_name": "log.info",
    "x_trace_id": "123jiojfdsao",
    "x_department":"tal_client_frontend",
    "x_rpc_id": "0.3",
    "x_version": "php-4",
    "x_timestamp": 1506480162,
    "x_duration": 0.214,
    "x_uid": "9527",
    "x_pid": "123",
    "x_module": "game1_round_win_round_end",
    "x_file": "userinfo.php",
    "x_line": "232",
    "x_msg": "ok",
    "x_code": "201",
    "extra": "json game_id lesson_num    xxxxx"
}

### exception  异常日志
{
    "x_name": "log.exception",
    "x_trace_id": "123jiojfdsao",
    "x_department":"tal_client_frontend",
    "x_rpc_id": "0.3",
    "x_version": "php-4",
    "x_timestamp": 1506480162,
    "x_duration": 0.214,
    "x_uid": "9527",
    "x_pid": "123",
    "x_module": "game1_round_win",
    "x_file": "userinfo.php",
    "x_line": "232",
    "x_msg": "exception:xxxxx call stack",
    "x_code": "hy20001",
    "x_backtrace": "xxxxx.php(123) gotError:..."
}

### 业务自发告警日志
{
    "x_name": "log.alarm",
    "x_trace_id": "123jiojfdsao",
    "x_department":"tal_client_frontend",
    "x_rpc_id": "0.3",
    "x_version": "php-4",
    "x_timestamp": 1506480162,
    "x_duration": 0.214,
```

```
    "x_uid": "9527",
    "x_pid": "123",
    "x_module": "game1_round_win_round_report",
    "x_file": "game_win_notify.php",
    "x_line": "123",
    "x_msg": "game report request fail! retryed three time..",
    "x_code": "201",
    "x_extra": "json game_id lesson_num   xxxxx"
}

### matrics 计数器

{
    "x_name": "metrix.count",
    "x_trace_id": "123jiojfdsao",
    "x_department":"tal_client_frontend",
    "x_rpc_id": "0.3",
    "x_version": "php-4",
    "x_timestamp": 1506480162,
    "x_uid": "9527",
    "x_pid": "123",
    "x_module": "game1_round_win_click",
    "x_extra": "json curl invoke count"
}
```

这个日志不仅可以用在服务端，还可以用在客户端。客户端每次被点击或被触发时，都可以自行生成一个新的 TraceID，在请求服务端时就会带上它。通过这个日志，我们可以分析不同地域访问服务的性能，也可以用作用户行为日志，仅仅需添加我们的日志类型即可。

上面的日志例子基本把我们依赖的资源情况描述得很清楚了。另外，我补充一个技巧，性能记录日志可以将被请求的接口也记录成一个日志，记录自己的耗时等信息，方便之后跟请求方的请求日志对照，这样可分析出两者之间是否有网络延迟等问题。

除此之外，这个设计还有一个核心要点：研发并不一定完全遵守如上字段规则生成日志，业务只要保证项目范围内输出的日志输出所有必填项目（TraceID，RPCID/SpanID，TimeStamp），同时保证数值型字段功能及类型稳定，即可实现 trace。

我们完全可以汇总日志后，再对不同的日志字段做自行解释，定制出不同业务所需的统计分析，这正是 ELK 最强大的地方。

为什么大部分设计都是记录依赖资源的日志呢？原因在于在没有 IO 的情况下，程序大部分都是可控的（侧重计算的服务除外）。只有 IO 类操作容易出现不稳定因素，并且日志记录过多也会影响系统性能，通过记录对数据源的操作能帮助我们排查业务逻辑的错误。

我们刚才提到日志如果过多会影响接口性能，那如何提高日志的写吞吐能力呢？这里我为你归纳了几个注意事项和技巧：

1.提高写线程的个数，一个线程写一个日志，也可以每个日志文件单独放一个磁盘，但是你要注意控制系统的 IOPS 不要超过 100；

2.当写入日志长度超过 1kb 时，不要使用多个线程高并发写同一个文件，简单来说就是文件的 append 操作对于写入长度超过缓冲区长度的操作不是原子性的，多线程并发写长内容到同一个文件，会导致日志乱序；

3.日志可以通过内存暂存，汇总达到一定数据量或缓存超过 2 秒后再落盘，这样可以减少过小日志写磁盘系统的调用次数，但是代价是被强杀时会丢日志；

4.日志缓存要提前 malloc 使用固定长度缓存，不要频繁分配回收，否则会导致系统整体缓慢；

5.服务被 kill 时，记得拦截信号，快速 fsync 内存中日志到磁盘，以此减少日志丢失的可能。

28.6 “侵入式埋点 SDK”VS“AOP 方式埋点”

最后，我们再说说 SDK。事实上，使用“ELK+自定义的标准”基本上已经能实现大多数的分布式链路跟踪系统，使用 Kibana 可以很快速地对各种日志进行聚合分析统计。虽然行业中出现过很多链路跟踪系统服务公司，做了很多 APM 等类似产品，但是能真正推广开的服务实际占少数，究其原因，我认为是以下几点：

- 分布式链路跟踪的日志吞吐很大，需要耗费大量的资源，成本高昂；
- 通用分布式链路跟踪服务很难做贴近业务的个性化，不能定制的第三方服务不如用开源；
- 分布式链路跟踪的埋点库对代码的侵入性大，需要研发手动植入到业务代码里，操作很麻烦，而且不够灵活。

- 另外，这种做法对语言也有相关的限制，因为目前只有 Java 通过动态启动注入 agent，才实现了静态语言 AOP 注入。我之前推广时，也是统一了内网项目的开源框架，才实现了统一的链路跟踪。

那么如果底层代码不能更新，如何简单暴力地实现链路跟踪呢?

这时候我们可以改造分级日志，让它每次在落地的时候都把 TraceId 和 RPCID（或 SpanID）带上，就会有很好的效果。如果数据底层做了良好的封装，我们可以在发起请求部分中写一些符合标准性能的日志，在框架的统一异常处理中也注入我们的标准跟踪，即可实现关键点的监控。

当然如果条件允许，我们最好提供一个标准的 SDK，让业务研发伙伴按需调用，这能帮助我们统一日志结构。毕竟手写很容易格式错乱，需要人工梳理，不过即使混乱，也仍旧有规律可言，这是 ELK 架构的强大之处，它的全文检索功能其实不在乎你的输入格式，但是数据统计类却需要我们确保各个字段用途固定。

最后再讲点其他日志的注意事项，可能你已经注意到了，这个设计日志是全量的。很多链路跟踪其实都是做的采样方式，比如 Jaeger 在应用本地会部署一个 Agent，对数据暂存汇总，统计出每个接口的平均响应时间，对具有同样特征的请求进行归类汇总，这样可以大大降低服务端压力。

但这么做也有缺点，当我们有一些小概率的业务逻辑错误，在采样中会被遗漏。所以很多核心系统会记录全量日志，周边业务记录采样日志。

由于我们日志结构很简单，如有需要可以自行实现一个类似 Agent 的功能，降低我们存储计算压力。甚至我们可以在服务端本地保存原始日志 7 天，当我们查找某个 Trace 日志的时候，直接请求所有服务器在本地查找。事实上，在写多读少的情况下，为了追一个 Trace 详细过程而去请求 200 个服务器，这时候即使等十秒钟都是可以接受的。

29. 如何优化系统架构设计来缓解流量压力提升并发性能?

我会以直播互动为例，带你看看读多写多的情况下如何应对流量压力。- 一般来说，这种服务多数属于实时互动服务，因为时效性要求很高，导致很多场景下，我们无法用读

缓存的方式来降低核心数据的压力。所以，为了降低这类互动服务器的压力，我们可以从架构入手，做一些灵活拆分的设计改造。

事实上这些设计是混合实现对外提供服务的，为了让你更好地理解，我会针对直播互动里的特定的场景进行讲解。一般来说，直播场景可以分为可预估用户量和不可预估用户量的场景，两者的设计有很大的不同，我们分别来看看。

29.1 可预估用户量的服务：游戏创建房间

相信很多玩对战游戏的伙伴都有类似经历，就是联网玩游戏要先创建房间。这种设计主要是通过设置一台服务器可以开启的房间数量上限，来限制一台服务器能同时服务多少用户。

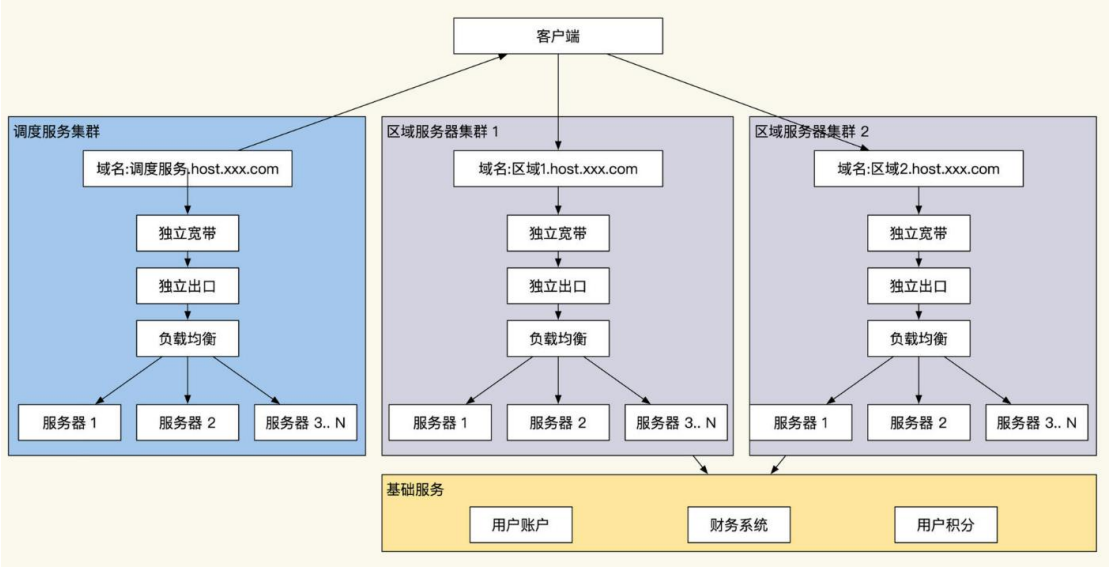
我们从服务器端的资源分配角度分析一下，创建房间这个设计是如何做资源调配的。创建房间后，用户通过房间号就可以邀请其他伙伴加入游戏进行对战，房主和加入的伙伴，都会通过房间的标识由调度服务统一分配到同一服务集群上进行互动。

这里我提示一下，开房间这个动作不一定需要游戏用户主动完成，可以设置成用户开启游戏就自动分配房间，这样做不但能提前预估用户量，还能很好地规划和掌控我们的服务资源。

如何评估一个服务器支持多少人同时在线呢？

我们可以通过压测测出单台服务器的服务在线人数，以此精确地预估带宽和服务器资源，算出一个集群（集群里包括若干服务器）需要多少资源、可以承担多少人在线进行互动，再通过调度服务分配资源，将新来的房主分配到空闲的服务集群。

最后的实现效果如下所示：



如上图所示，在创建房间阶段，我们的客户端在进入区域服务器集群之前，都是通过请求调度服务来进行调度的。调度服务器会定期接收各组服务器的服务用户在线情况，以此来评估需要调配多少用户进入到不同区域集群；同时客户端收到调度后，会拿着调度服务给的 token 去不同区域申请创建房间。

房间创建后，调度服务会在本地集群内维护这个房间的列表和信息，提供给其他要加入游戏的玩家展示。而加入的玩家同样会接入对应房间的区域服务器，与房主及同房间玩家进行实时互动。

这种通过配额房间个数来做服务器资源调度的设计，不光是对战游戏里，很多场景都用了类似设计，比如在线小课堂这类教学互动的。我们可以预见，通过这个设计能够对资源做到精准把控，用户不会超过我们服务器的设计容量。

29.2 不可预估用户量的服务

但是，有很多场景是随机的，我们无法把控有多少用户会进入这个服务器进行互动。全国直播就无法确认会有多少用户访问，为此，很多直播服务首先按主播过往预测用户量。通过预估量，提前将他们的直播安排到相对空闲的服务器群组里，同时提前准备一些调度工具，比如通过控制曝光度来延缓用户进入直播，通过这些为服务器调度争取更多时间来动态扩容。

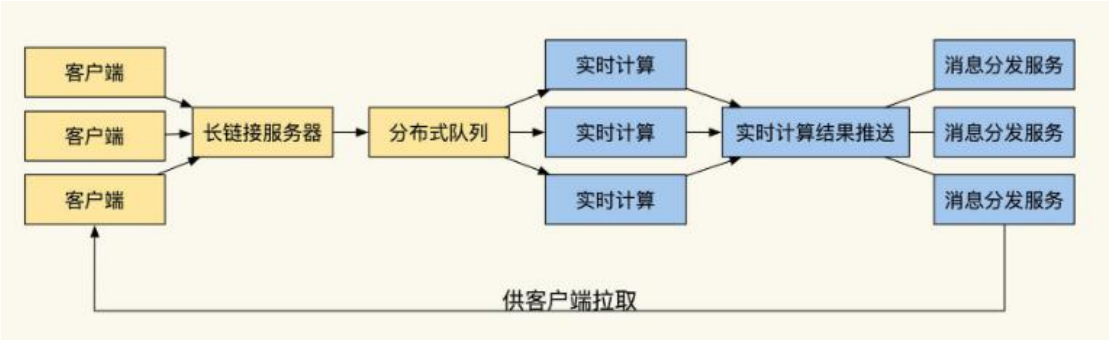
由于这一类的服务无法预估会有多少用户，所以之前的服务器小组模式并不适用于这种方式，需要更高一个级别的调度。

我们分析一下场景，对于直播来说，用户常见的交互形式包括聊天、答题、点赞、打赏和购物，考虑到这些形式的特点不同，我们针对不同的关键点依次做分析。

29.2.1 聊天：信息合并

聊天的内容普遍比较短，为了提高吞吐能力，通常会把用户的聊天内容放入分布式队列做传输，这样能延缓写入压力。

另外，在点赞或大量用户输入同样内容的刷屏情境下，我们可以通过大数据实时计算分析用户的输入，并压缩整理大量重复的内容，过滤掉一些无用信息。



压缩整理后的聊天内容会被分发到多个聊天内容分发服务器上，直播间内用户的聊天长连接会收到消息更新的推送通知，接着客户端会到指定的**内容分发服务器群组**里**批量拉取数据**，拿到数据后会根据时间顺序来回放。请注意，这个方式只适合用在疯狂刷屏的情况，如果用户量很少可以通过长链接进行实时互动。

29.2.2 答题：瞬时信息拉取高峰

除了交互流量极大的聊天互动信息之外，还有一些特殊的互动，如做题互动。直播间老师发送一个题目，题目消息会广播给所有用户，客户端收到消息后会从服务端拉取题目的数据。

如果有 10w 用户在线，很有可能导致瞬间有 10w 人在线同时请求服务端拉取题目。这样的数据请求量，需要我们投入大量的服务器和带宽才能承受，不过这么做这个性价比并不高。

理论上我们可以将数据静态化，并通过 CDN 阻挡这个流量，但是为了避免出现瞬时的高峰，推荐客户端拉取时加入随机延迟几秒，再发送请求，这样可以大大延缓服务器压力，获得更好的用户体验。

切记对于客户端来说，这种服务如果失败了，就不要频繁地请求重试，不然会将服务端打沉。如果必须这样做，那么建议你对重试的时间做退火算法，以此保证服务端不会因为一时故障收到大量的请求，导致服务器崩溃。

如果是教学场景的直播，有两个缓解服务器压力的技巧。第一个技巧是在上课当天，把抢答题目提前交给客户端做预加载下载，这样可以减少实时拉取的压力。

第二个方式是题目抢答的情况，老师发布题目的时候，提前设定发送动作生效后 5 秒再弹出题目，这样能让所有直播用户的接收端“准时”地收到题目信息，而不至于出现用户题目接收时间不一致的情况。

至于非抢答类型的题目，用户回答完题目后，我们可以先在客户端本地先做预判卷，把正确答案和解析展示给用户，然后在直播期间异步缓慢地提交用户答题结果到服务端，以此保证服务器不会因用户瞬时的流量被冲垮。

29.2.3 点赞：客户端互动合并

对于点赞的场景，我会分成客户端和服务端两个角度带你了解。

先看客户端，很多时候，客户端无需实时提交用户的所有交互，因为有很多机械的重复动作对实时性要求没那么高。

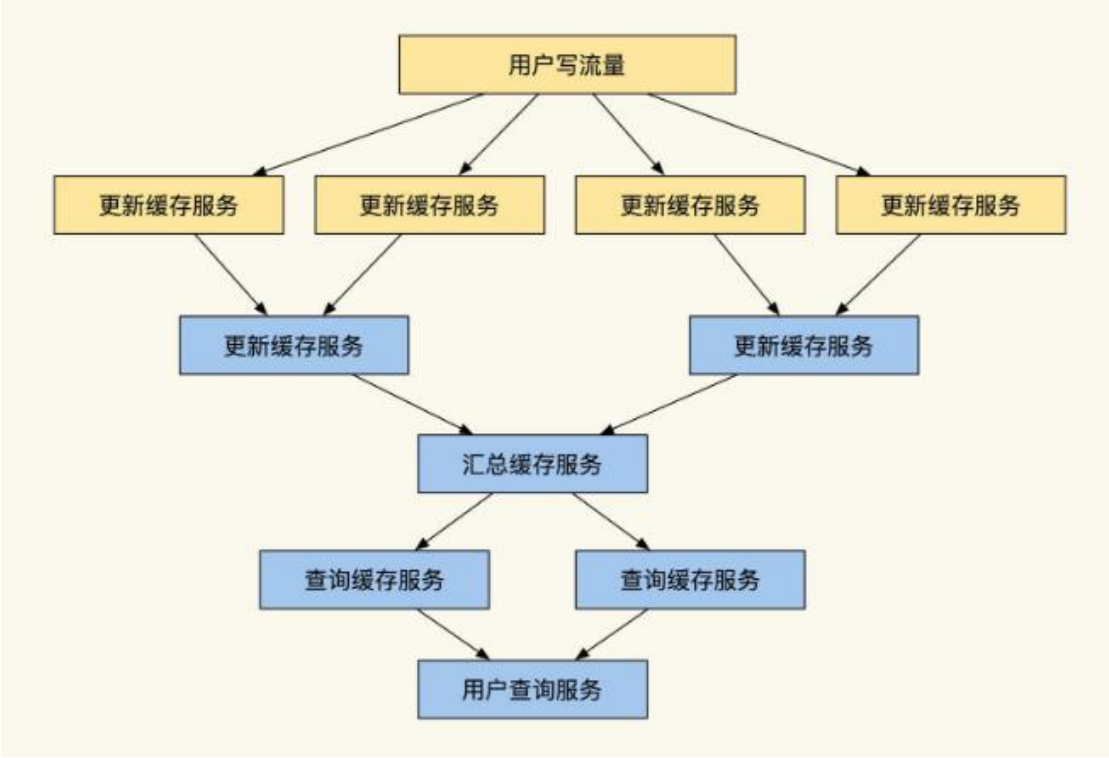
举个例子，用户在本地狂点了 100 下赞，客户端就可以合并这些操作为一条消息（例如用户 3 秒内点赞 10 次）。相信聪明如你，可以把互动动作合并这一招用在更多情景，比如用户连续打赏 100 个礼物。

通过这个方式可以大幅度降低服务器压力，既可以保证直播间的火爆依旧，还节省了大量的流量资源，何乐而不为。

29.2.4 点赞：服务端树形多层汇总架构

我们回头再看看点赞的场景下，如何设计服务端才能缓解请求压力。

如果我们的集群 QPS 超过十万，服务端数据层已经无法承受这样的压力时，如何应对高并发写、高并发读呢？微博做过一个类似的案例，用途是缓解用户的点赞请求流量，这种方式适合一致性要求不高的计数器，如下图所示：



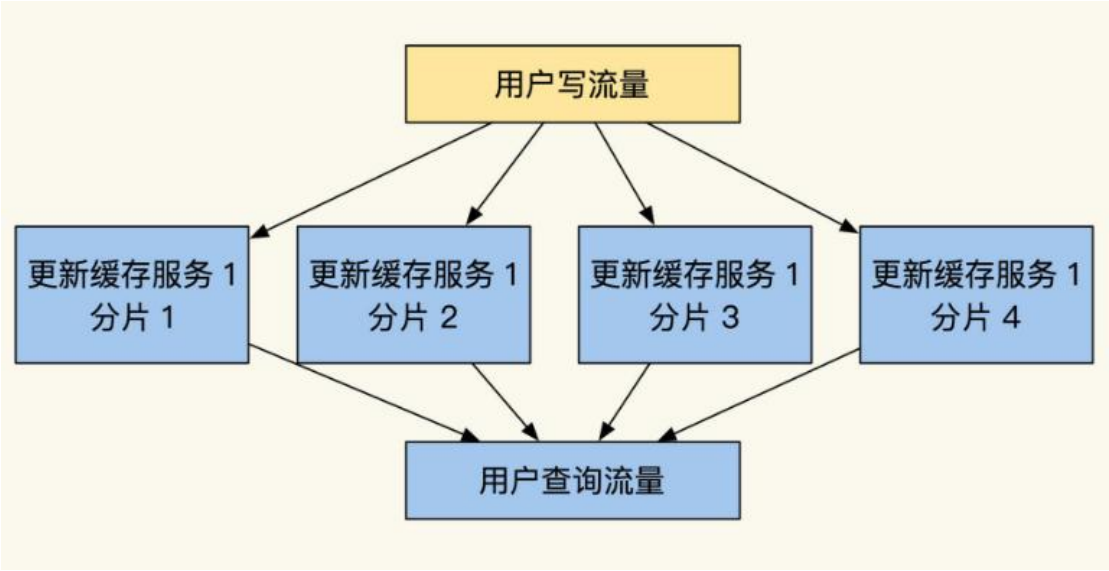
这个方式可以将用户点赞流量随机压到不同的写缓存服务上，通过第一层写缓存本地的实时汇总来缓解大量用户的请求，将更新数据周期性地汇总后，提交到二级写缓存。之后，二级汇总所在分片的所有上层服务数值后，最终汇总同步给核心缓存服务。接着，通过核心缓存把最终结果汇总累加起来。最后通过主从复制到多个子查询节点服务，供用户查询汇总结果。

另外，说个题外话，微博是 Redis 重度用户，后来因为点赞数据量太大，在 Redis 中缓存点赞数内存浪费严重，改为自行实现点赞服务来节省内存。

29.2.5 打赏&购物：服务端分片及分片实时扩容

前面的互动只要保证最终一致性就可以，但打赏和购物的场景下，库存和金额需要提供事务一致性的服务。

因为事务一致性的要求，这种服务我们不能做成多层缓冲方式提供服务，而且这种服务的数据特征是读多写多，所以我们可以通过数据分片方式实现这一类服务，如下图：



看了图是不是很好理解？我们可以按用户 id 做了 **hash 拆分**，通过网关将不同用户 uid 取模后，根据范围分配到不同分片服务上，然后分片内的服务对类似的请求进行内存实时计算更新。

通过这个方式，可以快速方便地实现负载切分，但缺点是 hash 分配容易出现个别热点，当我们流量扛不住的时候需要扩容。

但是 hash 这个方式如果出现个别服务器故障的话，会导致 hash 映射错误，从而请求到错误的分片。类似的解决方案有很多，如**一致性 hash 算法**，这种算法可以对局部的区域扩容，不会影响整个集群的分片，但是这个方法很多时候因为算法不通用，无法人为控制，使用起来很麻烦，需要开发配套工具。

除此之外，我给你推荐另外一个方式——**树形热迁移切片法**，这是一种类似虚拟桶的方式。

比如我们将全量数据拆分成 256 份，一份代表一个桶，16 个服务器每个分 16 个桶，当我们个别服务器压力过大的时候，可以给这个服务器增加两个订阅服务器去做主从同步，迁移这个服务器的 16 个桶的数据。

待同步迁移成功后，将这个服务器的请求流量拆分转发到两个 8 桶服务器，分别请求这两个订阅服务器继续对外服务，原服务器摘除回收即可。

服务切换成功后，由于是全量迁移，这两个服务同时同步了不属于自己的 8 个桶数据，这时新服务器遍历自己存储的数据，删除掉不属于自己的数据即可。当然也可以在同步 16 桶服务的数据时，过滤掉这些数据，这个方法适用于 Redis、MySQL 等所有**有状态分片数据服务**。

这个服务的难点在于请求的客户端不直接请求分片，而是通过代理服务去请求数据服务，只有通过代理服务才能够动态更新调度流量，实现平滑无损地转发流量。

最后，如何让客户端知道请求哪个分片才能找到数据呢？我给你分享两个常见的方式：

第一种方式是，客户端通过算法找到分片，比如：用户 hash(uid) % 100 = 桶 id，在配置中通过桶 id 找到对应分片。

第二种方式是，数据服务端收到请求后，将请求转发到有数据的分片。比如客户端请求 A 分片，再根据数据算法对应的分片配置找到数据在 B 分片，这时 A 分片会转发这个请求到 B，待 B 处理后返回给客户端数据（A 返回或 B 返回，取决于客户端跳转还是服务端转发）。

29.3 服务降级：分布式队列汇总缓冲

即使通过这么多技术来优化架构，我们的服务仍旧无法完全承受过高的瞬发流量。

对于这种情况，我们可以做一些服务降级的操作，通过队列将修改合并或做网关限流。

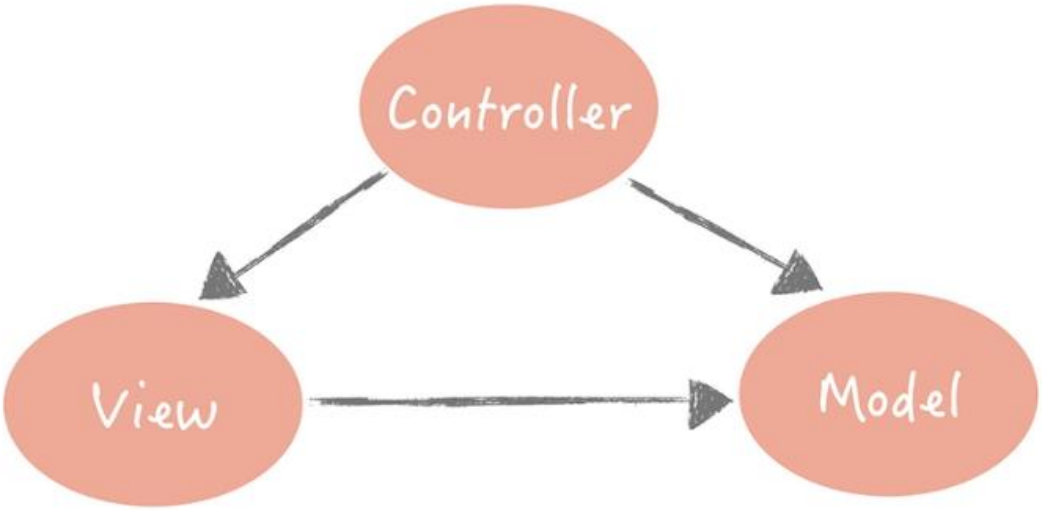
虽然这会牺牲一些实时性，但是实际上，很多数字可能没有我们想象中那么重要。像微博的点赞统计数据，如果客户端点赞无法请求到服务器，那么这些数据会在客户端暂存一段时间，在用户看数据时看到的只是短期历史数字，不是实时数字。

十万零五的点赞数跟十万零三千的点赞数，差异并不大，等之后服务器有空闲了，结果追上来最终是一致的。但作为降级方案，这么做能节省大量的服务器资源，也算是个好方法。

30. 为什么复杂的架构一定要做分层设计?

软件架构分层在软件工程中是一种常见的设计方式，它是将整体系统拆分成 N 个层次，每个层次有独立的职责，多个层次协同提供完整的功能。

我们在刚刚成为程序员的时候，会被“教育”说系统的设计要是“MVC”（Model-View-Controller）架构。它将整体的系统分成了 Model（模型），View（视图）和 Controller（控制器）三个层次，也就是将用户视图和业务处理隔离开，并且通过控制器连接起来，很好地实现了表现和逻辑的解耦，是一种标准的软件分层架构。



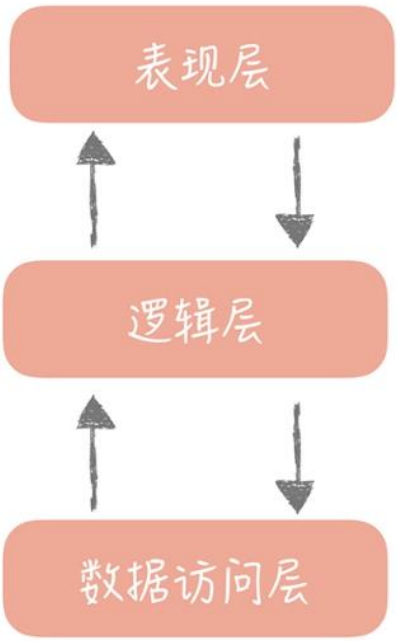
另外一种常见的分层方式是将整体架构分为表现层、逻辑层和数据访问层：

表现层，顾名思义嘛，就是展示数据结果和接受用户指令的，是最靠近用户的一层；

逻辑层里面有复杂业务的具体实现；

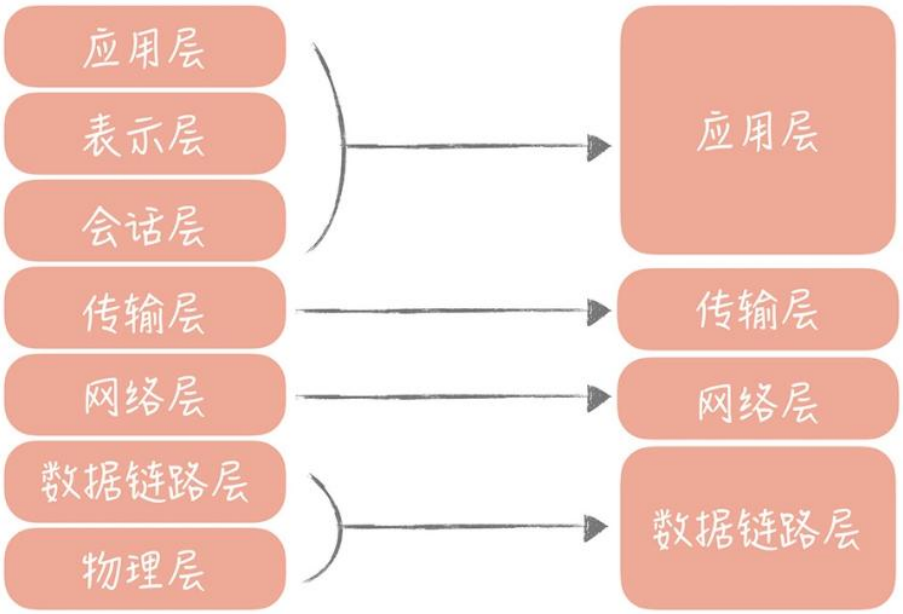
数据访问层则是主要处理和存储之间的交互。

这是在架构上最简单的一种分层方式。其实，我们在不经意间已经按照三层架构来做系统分层设计了，比如在构建项目的时候，我们通常会建立三个目录：Web、Service 和 Dao，它们分别对应了表现层、逻辑层还有数据访问层。



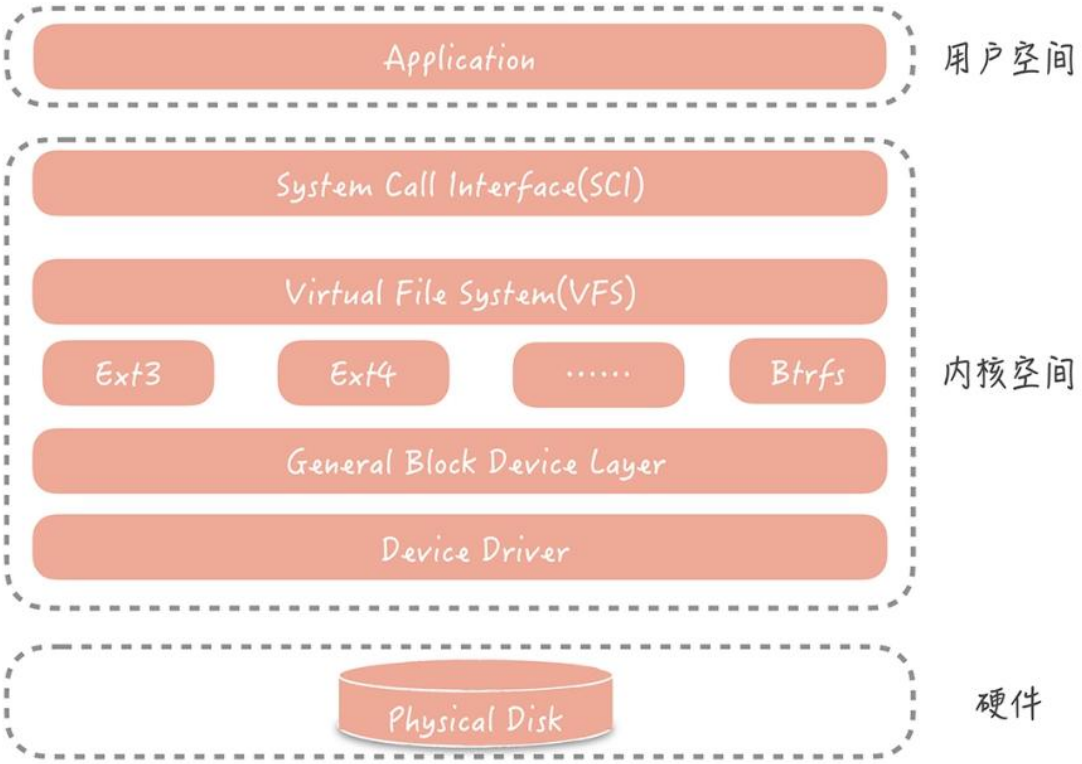
除此之外，如果我们稍加留意，就可以发现很多的分层的例子。比如我们在大学中学到的 OSI 网络模型，它把整个网络分成了七层，自下而上分别是物理层、数据链路层、网络层、传输层、会话层、表示层和应用层。

工作中经常能用到 TCP/IP 协议，它把网络简化成了四层，即链路层、网络层、传输层和应用层。每一层各司其职又互相帮助，网络层负责端到端的寻址和建立连接，传输层负责端到端的数据传输等，同时相邻两层还会有数据的交互。这样可以隔离关注点，让不同的层专注做不同的事情。



Linux 文件系统也是分层设计的，从下图你可以清晰地看出文件系统的层次。在文件系统的最上层是虚拟文件系统（VFS），用来屏蔽不同的文件系统之间的差异，提供统一的系统调用接口。虚拟文件系统的下层是 Ext3、Ext4 等各种文件系统，再向下是为了屏蔽不同硬件设备的实现细节，我们抽象出来的单独的一层——通用块设备层，然后就是不同类型的磁盘了。

我们可以看到，某些层次负责的是对下层不同实现的抽象，从而对上层屏蔽实现细节。比方说 VFS 对上层（系统调用层）来说提供了统一的调用接口，同时对下层中不同的文件系统规约了实现模型，当新增一种文件系统实现的时候，只需要按照这种模型来设计，就可以无缝插入到 Linux 文件系统中。



那么，为什么这么多系统一定要做分层的设计呢？答案是分层设计存在一定的优势。

30.1 分层有什么好处

分层的设计可以简化系统设计，让不同的人专注做某一层次的事情。想象一下，如果你要设计一款网络程序却没有分层，该是一件多么痛苦的事情。

因为你必须是一个通晓网络的全才，要知道各种网络设备的接口是什么样的，以便可以将数据包发送给它。你还要关注数据传输的细节，并且需要处理类似网络拥塞，数据超时重传这样的复杂问题。当然了，你更需要关注数据如何在网络上安全传输，不会被别人窥探和篡改。

而有了分层的设计，你只需要专注设计应用层的程序就可以了，其他都可以交给下面几层来完成。

再有，分层之后可以做到很高的复用。比如，我们在设计系统 A 的时候，发现某一层具有一定的通用性，那么我们可以把它抽取独立出来，在设计系统 B 的时候使用起来，这样可以减少研发周期，提升研发的效率。

最后一点，分层架构可以让我们更容易做横向扩展。如果系统没有分层，当流量增加时我们需要针对整体系统来做扩展。但是，如果我们按照上面提到的三层架构将系统分层后，就可以针对具体的问题来做细致的扩展。

比如说，业务逻辑里面包含有比较复杂的计算，导致 CPU 成为性能的瓶颈，那这样就可以把逻辑层单独抽取出来独立部署，然后只对逻辑层来做扩展，这相比于针对整体系统扩展所付出的代价就要小得多了。

这一点也可以解释我们课程开始时提出的问题：架构分层究竟和高并发设计的关系是怎样的？从 01 节中我们了解到，横向扩展是高并发系统设计的常用方法之一，既然分层的架构可以为横向扩展提供便捷， 那么支撑高并发的系统一定是分层的系统。

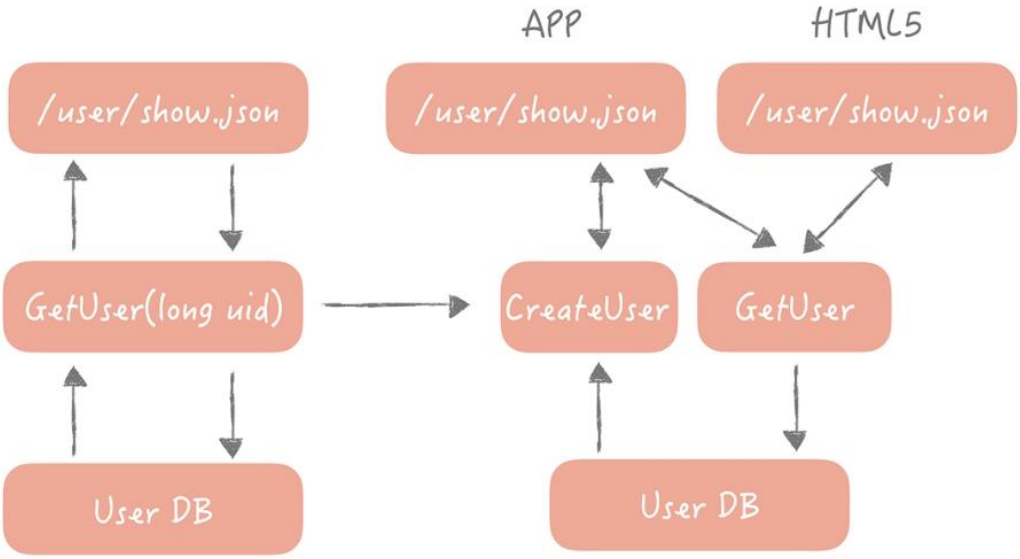
30.2 如何来做系统分层

说了这么多分层的优点，那么当我们要做分层设计的时候，需要考虑哪些关键因素呢？在我看来，最主要的一点就是你需要理清楚每个层次的边界是什么。你也许会问：“如果按照三层架构来分层的话，每一层的边界不是很容易就界定吗？”

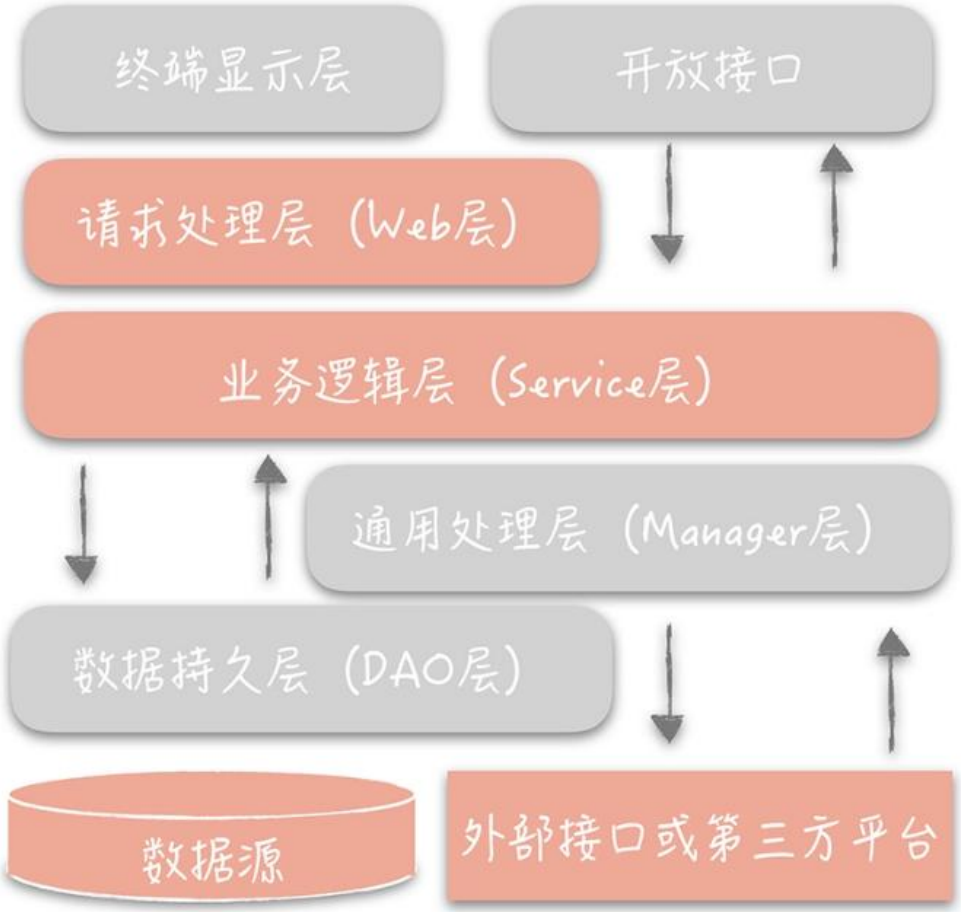
没错，当业务逻辑简单时，层次之间的边界的确清晰，开发新的功能时也知道哪些代码要往哪儿写。但是当业务逻辑变得越来越复杂时，边界就会变得越来越模糊，给你举个例子。

任何一个系统中都有用户系统，最基本的接口是返回用户信息的接口，它调用逻辑层的 GetUser 方法，GetUser 方法又和 User DB 交互获取数据，就像下图左边展示的样子。

这时，产品提出一个需求，在 APP 中展示用户信息的时候，如果用户不存在，那么要自动给用户创建一个用户。同时，要做一个 HTML5 的页面，HTML5 页面要保留之前的逻辑，也就是不需要创建用户。这时逻辑层的边界就变得不清晰，表现层也承担了一部分的业务逻辑（将获取用户和创建用户接口编排起来）。



那我们要如何做呢？参照阿里发布的《阿里巴巴 Java 开发手册 v1.4.0（详尽版）》，我们可以将原先的三层架构细化成下面的样子：



解释一下这个分层架构中的每一层的作用。

终端显示层：各端模板渲染并执行显示的层。当前主要是 Velocity 渲染，JS 渲染，JSP 渲染，移动端展示等。

开放接口层：将 Service 层方法封装成开放接口，同时进行网关安全控制和流量控制等。

Web 层：主要是对访问控制进行转发，各类基本参数校验，或者不复用的业务简单处理等。

Service 层：业务逻辑层。

Manager 层：通用业务处理层。这一层主要有两个作用，其一，你可以将原先 Service 层的一些通用能力下沉到这一层，比如与缓存和存储交互策略，中间件的接入；其二，你也可以在这一层封装对第三方接口的调用，比如调用支付服务，调用审核服务等。

DAO 层：数据访问层，与底层 MySQL、Oracle、HBase 等进行数据交互。

外部接口或第三方平台：包括其它部门 RPC 开放接口，基础平台，其它公司的 HTTP 接口。在这个分层架构中主要增加了 Manager 层，它与 Service 层的关系是：Manager 层提供原子的服务接口，Service 层负责依据业务逻辑来编排原子接口。以上面的例子来说，Manager 层提供创建用户和获取用户信息的接口，而 Service 层负责将这两个接口组装起来。这样就把原先散布在表现层的业务逻辑都统一到了 Service 层，每一层的边界就非常清晰了。

除此之外，分层架构需要考虑层次之间一定是相邻层互相依赖，数据的流转也只能在相邻的两层之间流转。

我们还是以三层架构为例，数据从表示层进入之后一定要流转到逻辑层，做业务逻辑处理，然后流转到数据访问层来和数据库交互。那么你可能会问：“如果业务逻辑很简单的话可不可以从表示层直接到数据访问层，甚至直接读数据库呢？”

其实从功能上是可以的，但是从长远的架构设计考虑，这样会造成层级调用的混乱，比方说如果表示层或者业务层可以直接操作数据库，那么一旦数据库地址发生变更，你就需要在多个层次做更改，这样就失去了分层的意义，并且对于后面的维护或者重构都会是灾难性的。

30.3 分层架构的不足

任何事物都不可能是尽善尽美的，分层架构虽有优势也会有缺陷，它最主要的一个缺陷就是增加了代码的复杂度。

这是显而易见的嘛，明明可以在接收到请求后就可以直接查询数据库获得结果，却偏偏要在中间插入多个层次，并且有可能每个层次只是简单地做数据的传递。有时增加一个小小的需求也需要更改所有层次上的代码，看起来增加了开发的成本，并且从调试上来看也增加了复杂度，原本如果直接访问数据库我只需要调试一个方法，现在我却要调试多个层次的多个方法。

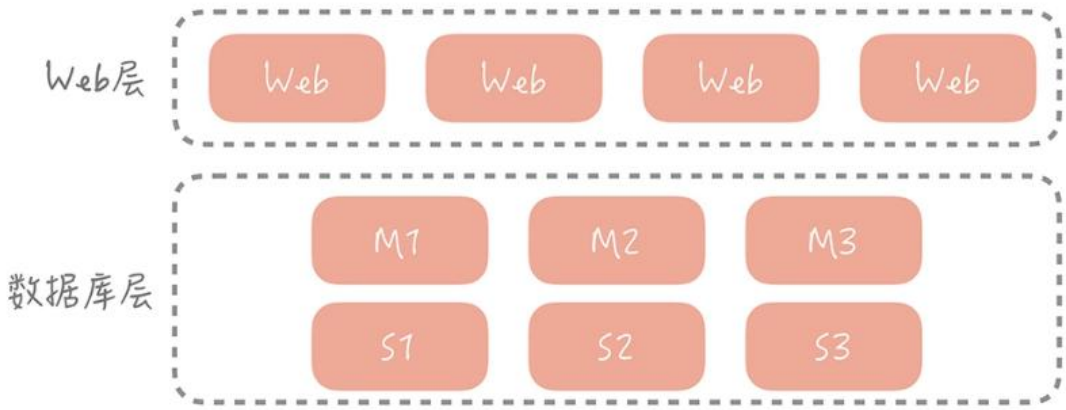
另外一个可能的缺陷是，如果我们把每个层次独立部署，层次间通过网络来交互，那么多层的架构在性能上会有损耗。这也是为什么服务化架构性能要比单体架构略差的原因，也就是所谓的“多一跳”问题。

那我们是否要选择分层的架构呢？答案当然是肯定的。

你要知道，任何的方案架构都是有优势有缺陷的，天地尚且不全何况我们的架构呢？分层架构固然会增加系统复杂度，也可能会有性能的损耗，但是相比于它能带给我们的好处来说，这些都是可以接受的，或者可以通过其它的方案解决的。我们在做决策的时候切不可以偏概全，因噎废食。

31. 假设数据库成为了性能瓶颈点，动态数据查询如何提升效率

假设我们有一个这样的电商系统，并且这个系统完成了对数据库的主从分离和分库分表之后，已经可以支撑十几万 DAU 了，整体系统的架构如下：



从整体上看，数据库分成了主库和从库，数据也被切分到多个数据库节点上。但随着并发的增加，存储数据量的增多，数据库的磁盘 IO 逐渐成了系统的瓶颈，我们需要一种访问更快的组件来降低请求响应时间，提升整体系统性能。这时我们就会使用缓存。那么什么是缓存，我们又该如何将它的优势最大化呢？

31.1 什么是缓存

缓存，是一种存储数据的组件，它的作用是让对数据的请求更快地返回。

我们经常会把缓存放在内存中来存储， 所以有人就把内存和缓存画上了等号，这完全是外行人的见解。作为业内人士，你要知道在某些场景下我们可能还会使用 SSD 作为冷数据的缓存。比如说 360 开源的 Pika 就是使用 SSD 存储数据解决 Redis 的容量瓶颈的。

实际上，凡是位于速度相差较大的两种硬件之间，用于协调两者数据传输速度差异的结构，均可称之为缓存。那么说到这儿我们就需要知道常见硬件组件的延时情况是什么样的了，这样在做方案的时候可以对延迟有更直观的印象。幸运的是，业内已经有人帮我们总结出这些数据了，我将这些数据整理了一下，你可以看一下。

```
Latency Comparison Numbers
--------------------------
L1 cache reference                           0.5 ns
Branch mispredict                            5   ns
L2 cache reference                           7   ns                      14x
Mutex lock/unlock                          100   ns
Main memory reference                      100   ns                      20x
Compress 1K bytes with Zippy            10,000   ns       10 us
Send 1 KB bytes over 1 Gbps network     10,000   ns       10 us
Read 4 KB randomly from SSD*           150,000   ns      150 us          ~1GB
Read 1 MB sequentially from memory     250,000   ns      250 us
Round trip within same datacenter      500,000   ns      500 us
Read 1 MB sequentially from SSD*     1,000,000   ns    1,000 us    1 ms  ~1GB
Disk seek                           10,000,000   ns   10,000 us   10 ms  20x
Read 1 MB sequentially from 1 Gbps  10,000,000   ns   10,000 us   10 ms  40x
Read 1 MB sequentially from disk    30,000,000   ns   30,000 us   30 ms 120x
Send packet CA->Netherlands->CA    150,000,000   ns  150,000 us  150 ms

Notes
-----
1 ns = 10^-9 seconds
1 us = 10^-6 seconds = 1,000 ns
1 ms = 10^-3 seconds = 1,000 us = 1,000,000 ns
```

从这些数据中，你可以看到，做一次内存寻址大概需要 100ns，而做一次磁盘的查找则需要 10ms。如果我们将做一次内存寻址的时间类比为一个课间，那么做一次磁盘查找相当于度过了大学的一个学期。可见，我们使用内存作为缓存的存储介质相比于以磁盘作为主要存储介质的数据库来说，性能上会提高多个数量级，同时也能够支撑更高的并发量。所以，内存是最常见的一种缓存数据的介质。

缓存作为一种常见的空间换时间的性能优化手段，在很多地方都有应用，我们先来看几个例子，相信你一定不会陌生。

31.2 缓存案例

Linux 内存管理是通过一个叫做 MMU（Memory Management Unit）的硬件，来实现从虚拟地址到物理地址的转换的，但是如果每次转换都要做这么复杂计算的话，无疑会造成性能的损耗，所以我们会借助一个叫做 TLB（Translation Lookaside Buffer）的组件来缓存最近转换过的虚拟地址，和物理地址的映射。TLB 就是一种缓存组件，缓存复杂运算的结果，就好比你做一碗色香味俱全的面条可能比较复杂，那么我们把做好的面条油炸处理一下做成方便面，你做方便面的话就简单多了，也快速多了。这个缓存组件比较底层，这里你只需要了解一下就可以了。

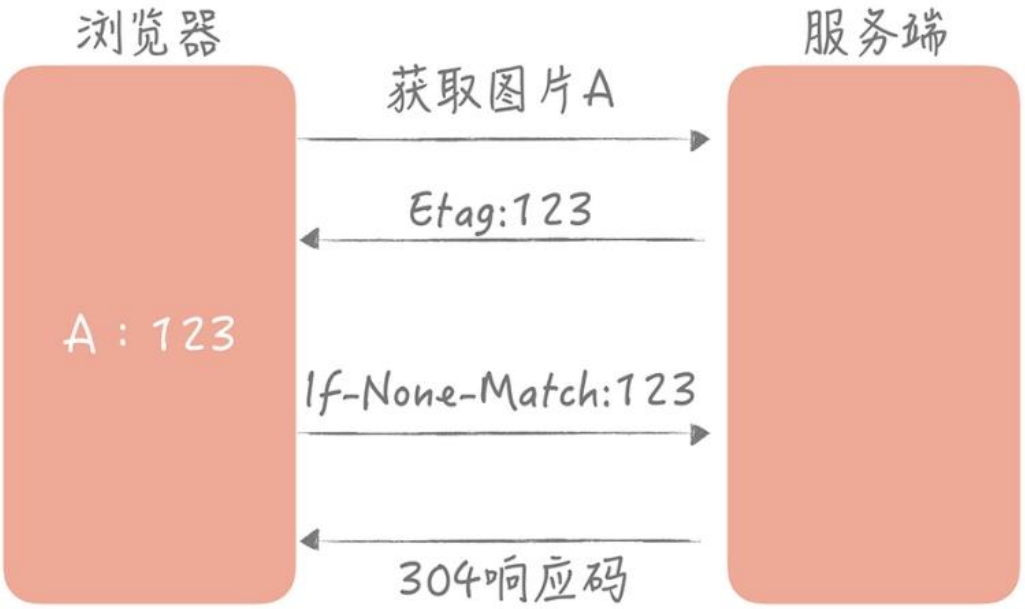
在大部分的笔记本，桌面电脑和服务器上都会有一个或者多个 TLB 组件，在不经意间帮助我们加快地址转换的速度。

再想一下你平时经常刷的抖音。平台上的短视频实际上是使用内置的网络播放器来完成的。网络播放器接收的是数据流，将数据下载下来之后经过分离音视频流，解码等流程后输出到外设设备上播放。

如果我们在打开一个视频的时候才开始下载数据的话，无疑会增加视频的打开速度（我们叫首播时间），并且播放过程中会有卡顿。所以我们的播放器中通常会设计一些缓存的组件，在未打开视频时缓存一部分视频数据，比如我们打开抖音，服务端可能一次会返回三个视频信息，我们在播放第一个视频的时候，播放器已经帮我们缓存了第二、三个视频的部分数据，这样在看第二个视频的时候就可以给用户“秒开”的感觉。

除此之外，我们熟知的 HTTP 协议也是有缓存机制的。当我们第一次请求静态的资源时，比如一张图片，服务端除了返回图片信息，在响应头里面还有一个“Etag”的字段。浏览器会缓存图片信息以及这个字段的值。当下一次再请求这个图片的时候，浏览器发

起的请求头里面会有一个“If-None-Match”的字段，并且把缓存的“Etag”的值写进去发给服务端。服务端比对图片信息是否有变化，如果没有，则返回浏览器一个 304 的状态码，浏览器会继续使用缓存的图片信息。通过这种缓存协商的方式，可以减少网络传输的数据大小，从而提升页面展示的性能。

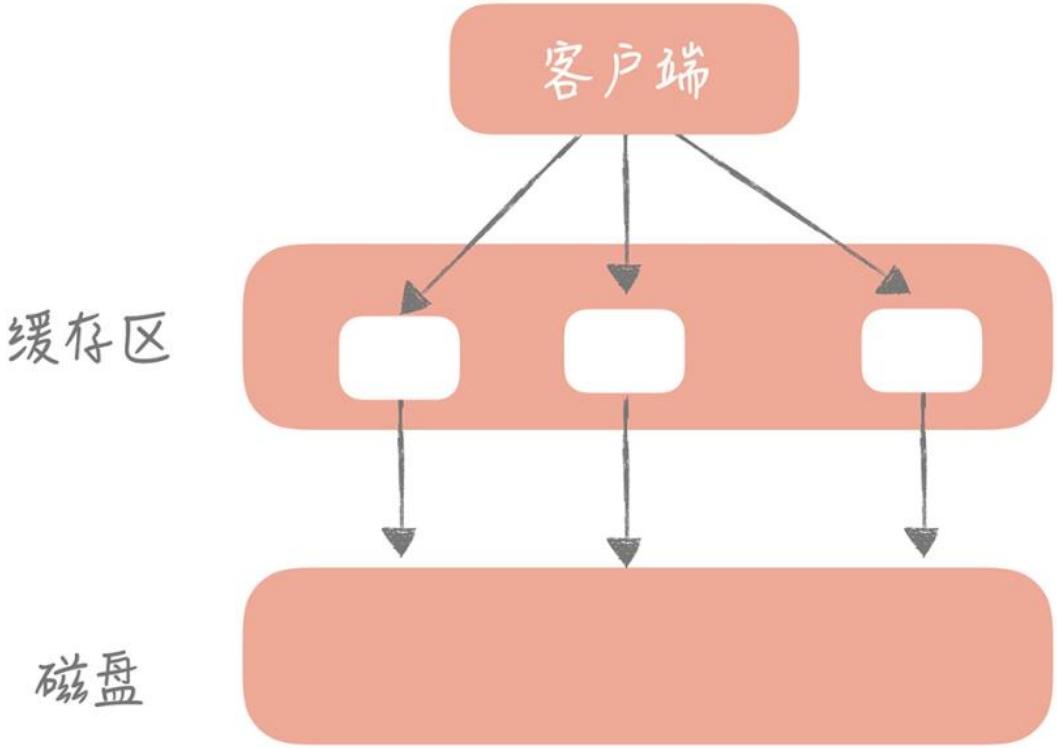


31.3 缓存与缓冲区

讲了这么多缓存案例，想必你对缓存已经有了一个直观并且形象的了解了。除了缓存，我们在日常开发过程中还会经常听见一个相似的名词——缓冲区，那么，什么是缓冲区呢？缓冲和缓存只有一字之差，它们有什么区别呢？

我们知道，缓存可以提高低速设备的访问速度，或者减少复杂耗时的计算带来的性能问题。理论上说，我们可以通过缓存解决所有关于“慢”的问题，比如从磁盘随机读取数据慢，从数据库查询数据慢，只是不同的场景消耗的存储成本不同。

缓冲区则是一块临时存储数据的区域，这些数据后面会被传输到其他设备上。缓冲区更像“消息队列篇”中即将提到的消息队列，用以弥补高速设备和低速设备通信时的速度差。比如，我们将数据写入磁盘时并不是直接刷盘，而是写到一块缓冲区里面，内核会标识这个缓冲区为脏。当经过一定时间或者脏缓冲区比例到达一定阈值时，由单独的线程把脏块刷新到硬盘上。这样避免了每次写数据都要刷盘带来的性能问题。



以上就是缓冲区和缓存的区别，从这个区别来看，上面提到的 TLB 的命名是有问题的，它应该是缓存而不是缓冲区。

现在你已经了解了缓存的含义，那么我们经常使用的缓存都有哪些？我们又该如何使用缓存，将它的优势最大化呢？

31.4 缓存分类

在我们日常开发中，常见的缓存主要就是静态缓存、分布式缓存和热点本地缓存这三种。静态缓存在 Web 1.0 时期是非常著名的，它一般通过生成 Velocity 模板或者静态 HTML 文件来实现静态缓存，在 Nginx 上部署静态缓存可以减少对于后台应用服务器的压力。例如，我们在做一些内容管理系统的时候，后台会录入很多的文章，前台在网站上展示文章内容，就像新浪，网易这种门户网站一样。

当然，我们也可以把文章录入到数据库里面，然后前端展示的时候穿透查询数据库来获取数据，但是这样会对数据库造成很大的压力。虽然我们使用分布式缓存来挡读请求，但是对于像日均 PV 几十亿的大型门户网站来说，基于成本考虑仍然是不划算的。

所以解决思路是每篇文章在录入的时候渲染成静态页面，放置在所有的前端 Nginx 或者 Squid 等 Web 服务器上，这样用户在访问的时候会优先访问 Web 服务器上的静态页面，在对旧的文章执行一定的清理策略后，依然可以保证 99% 以上的缓存命中率。这种缓存只能针对静态数据来缓存，对于动态请求就无能为力了。那么我们如何针对动态请求做缓存呢？这时就需要分布式缓存了。

分布式缓存的大名可谓是如雷贯耳了，我们平时耳熟能详的 Memcached、Redis 就是分布式缓存的典型例子。它们性能强劲，通过一些分布式的方案组成集群可以突破单机的限制。所以在整体架构中，分布式缓存承担着非常重要的角色（接下来的课程会专门针对分布式缓存，了解分布式缓存的使用技巧以及高可用的方案，让你能在工作中对分布式缓存运用自如）。

对于静态的资源的缓存你可以选择静态缓存，对于动态的请求你可以选择分布式缓存，那么什么时候要考虑热点本地缓存呢？

答案是当我们遇到极端的热点数据查询的时候。热点本地缓存主要部署在应用服务器的代码中，用于阻挡热点查询对于分布式缓存节点或者数据库的压力。

比如某一位明星在微博上有了热点话题，“吃瓜群众”会到他（她）的微博首页围观，这就会引发这个用户信息的热点查询。这些查询通常会命中某一个缓存节点或者某一个数据库分区，短时间内会形成极高的热点查询。

那么我们会在代码中使用一些本地缓存方案，如 HashMap，Guava Cache 或者是 Ehcache 等，它们和应用程序部署在同一个进程中，优势是不需要跨网络调度，速度极快，所以可以用来阻挡短时间内的热点查询。来看个例子。

比方说你的垂直电商系统的首页有一些推荐的商品，这些商品信息是由编辑在后台录入和变更。你分析编辑录入新的商品或者变更某个商品的信息后，在页面的展示是允许有一些延迟的，比如说 30 秒的延迟，并且首页请求量最大，即使使用分布式缓存也很难抗住，所以你决定使用 Guava Cache 来将所有的推荐商品的信息缓存起来，并且设置每隔 30 秒重新从数据库中加载最新的所有商品。

首先，我们初始化 Guava 的 Loading Cache：

```
1 CacheBuilder<String, List<Product>> cacheBuilder =
  CacheBuilder.newBuilder().maximumSize(maxSize).recordStats(); //设置缓存最大值
2 cacheBuilder = cacheBuilder.refreshAfterWrite(30, TimeUnit.Seconds); //设置刷新间隔
3
```

```
LoadingCache<String, List<Product>> cache = cacheBuilder.build(new
CacheLoader<String, List<Product>>() {
    @Override
    public List<Product> load(String k) throws Exception {
        return productService.loadAll(); // 获取所有商品
    }
});
```

这样，你在获取所有商品信息的时候可以调用 Loading Cache 的 get 方法，就可以优先从本地缓存中获取商品信息，如果本地缓存不存在，会使用 CacheLoader 中的逻辑从数据库中加载所有的商品。

由于本地缓存是部署在应用服务器中，而我们应用服务器通常会部署多台，当数据更新时，我们不能确定哪台服务器本地中了缓存，更新或者删除所有服务器的缓存不是一个好的选择，所以我们通常会等待缓存过期。因此，这种缓存的有效期很短，通常为分钟或者秒级别，以避免返回前端脏数据。

31.5 缓存的不足

通过了解上面的内容，你不难发现，缓存的主要作用是提升访问速度，从而能够抗住更高的并发。那么，缓存是不是能够解决一切问题？显然不是。事物都是具有两面性的，缓存也不例外，我们要了解它的优势的同时也需要了解它有哪些不足，从而扬长避短，将它的作用发挥到最大。

首先，缓存比较适合于读多写少的业务场景，并且数据最好带有一定的热点属性。这是因为缓存毕竟会受限于存储介质不可能缓存所有数据，那么当数据有热点属性的时候才能保证一定的缓存命中率。比如说类似微博、朋友圈这种 20% 的内容会占到 80% 的流量。所以，一旦当业务场景读少写多时或者没有明显热点时，比如在搜索的场景下，每个人搜索的词都会不同，没有明显的热点，那么这时缓存的作用就不明显了。

其次，缓存会给整体系统带来复杂度，并且会有数据不一致的风险。当更新数据库成功，更新缓存失败的场景下，缓存中就会存在脏数据。对于这种场景，我们可以考虑使用较短的过期时间或者手动清理的方式来解决。

再次，之前提到缓存通常使用内存作为存储介质，但是内存并不是无限的。因此，我们在使用缓存的时候要做数据存储量级的评估，对于可预见的需要消耗极大存储成本的数

据，要慎用缓存方案。同时，缓存一定要设置过期时间，这样可以保证缓存中的会是热点数据。

最后，缓存会给运维也带来一定的成本，运维需要对缓存组件有一定的了解，在排查问题的时候也多了一个组件需要考虑在内。

虽然有这么多的不足，但是缓存对于性能的提升是毋庸置疑的，我们在做架构设计的时候也需要把它考虑在内，只是在做具体方案的时候需要对缓存的设计有更细致的思考，才能最大化地发挥缓存的优势。

32. 如何设计支持 50 万 QPS 的站内未读消息系统?

当有人 @你、评论你、给你的博文点赞或者给你发送私信的时候，你会收到相应的未读提醒；

在早期的微博版本中有系统通知的功能，也就是系统会给全部用户发送消息，通知用户有新的版本或者有一些好玩的运营活动，如果用户没有看，系统就会给他展示有多少条未读的提醒。

我们在浏览信息流的时候，如果长时间没有刷新页面，那么信息流上方就会提示你在这段时间有多少条信息没有看。

那当你遇到第一个需求时，要如何记录未读数呢？其实，这个需求可以用上节课提到的通用计数系统来实现，因为二者的场景非常相似。

你可以在计数系统中增加一块儿内存区域，以用户 ID 为 Key 存储多个未读数，当有人 @ 你时，增加你的未读 @的计数；当有人评论你时，增加你的未读评论的计数，以此类推。当你点击了未读数字进入通知页面，查看 @ 你或者评论你的消息时，重置这些未读计数为零。相信通过上一节课的学习，你已经非常熟悉这一类系统的设计了，所以我不再赘述。

那么系统通知的未读数是如何实现的呢？我们能用通用计数系统实现吗？答案是不能的，因为会出现一些问题

32.1 系统通知的未读数要如何设计

来看具体的例子。假如你的系统中只有 A、B、C 三个用户，那么你可以在通用计数系统中增加一块儿内存区域，并且以用户 ID 为 Key 来存储这三个用户的未读通知数据，当系统发送一个新的通知时，我们会循环给每一个用户的未读数加 1，这个处理逻辑的伪代码就像下面这样：

```
List<Long> userIds = getAllUserIds();
for(Long id : userIds) {
   incrUnreadCount(id);
}
```

这样看来，似乎简单可行，但随着系统中的用户越来越多，这个方案存在两个致命的问题。

首先，获取全量用户就是一个比较耗时的操作，相当于对用户库做一次全表的扫描，这不仅会对数据库造成很大的压力，而且查询全量用户数据的响应时间是很长的，对于在线业务来说是难以接受的。如果你的用户库已经做了分库分表，那么就要扫描所有的库表，响应时间就更长了。不过有一个折中的方法， 那就是在发送系统通知之前，先从线下的数据仓库中获取全量的用户 ID，并且存储在一个本地的文件中，然后再轮询所有的用户 ID，给这些用户增加未读计数。

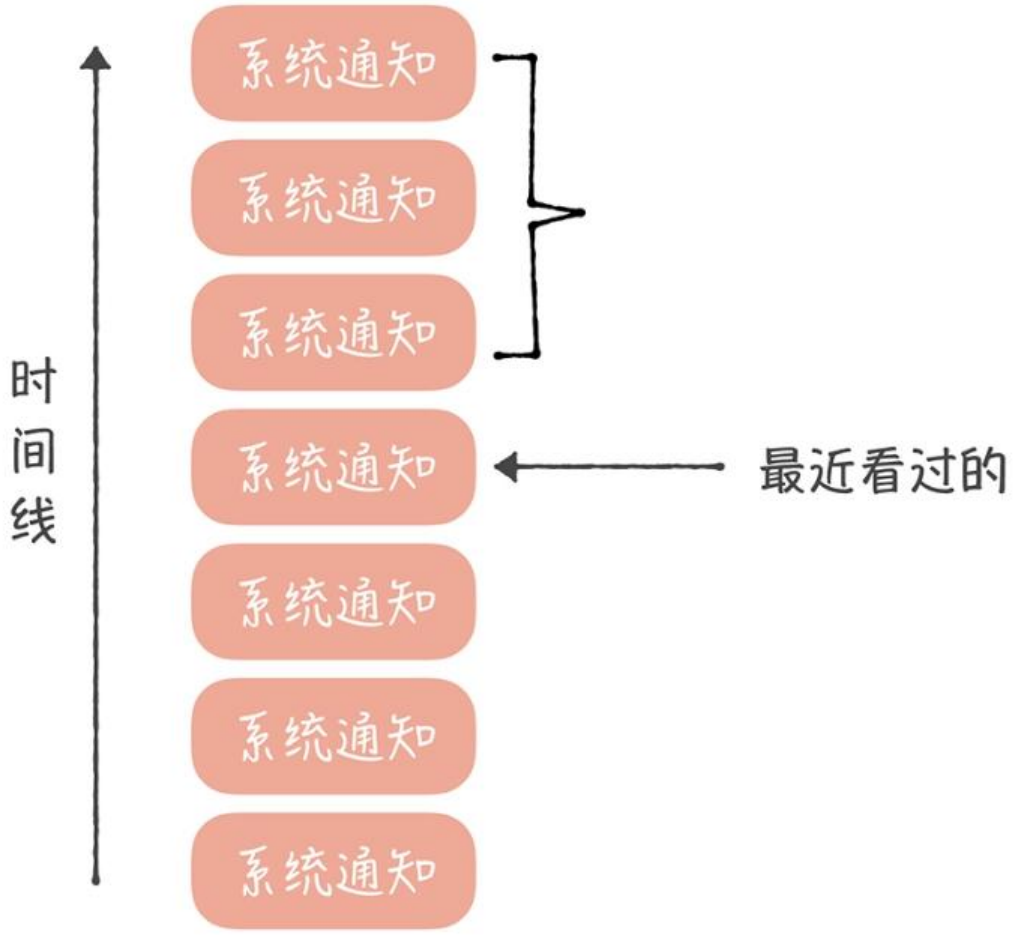
这似乎是一个可行的技术方案，然而它给所有人增加未读计数，会消耗非常长的时间。你计算一下，假如你的系统中有一个亿的用户，给一个用户增加未读数需要消耗 1ms，那么给所有人都增加未读计数就需要 100000000 * 1 /1000 = 100000 秒，也就是超过一天的时间；即使你启动 100 个线程并发的设置，也需要十几分钟的时间才能完成，而用户很难接受这么长的延迟时间。

另外，使用这种方式需要给系统中的每一个用户都记一个未读数的值，而在系统中，活跃用户只是很少的一部分，大部分的用户是不活跃的，甚至从来没有打开过系统通知，为这些用户记录未读数显然是一种浪费。

通过上面的内容，你可以知道为什么我们不能用通用计数系统实现系统通知未读数了吧？那正确的做法是什么呢？

要知道，系统通知实际上是存储在一个大的列表中的，这个列表对所有用户共享，也就是所有人看到的都是同一份系统通知的数据。不过不同的人最近看到的消息不同，所以

每个人会有不同的未读数。因此，你可以记录一下在这个列表中每个人看过最后一条消息的 ID，然后统计这个 ID 之后有多少条消息，这就是未读数了。



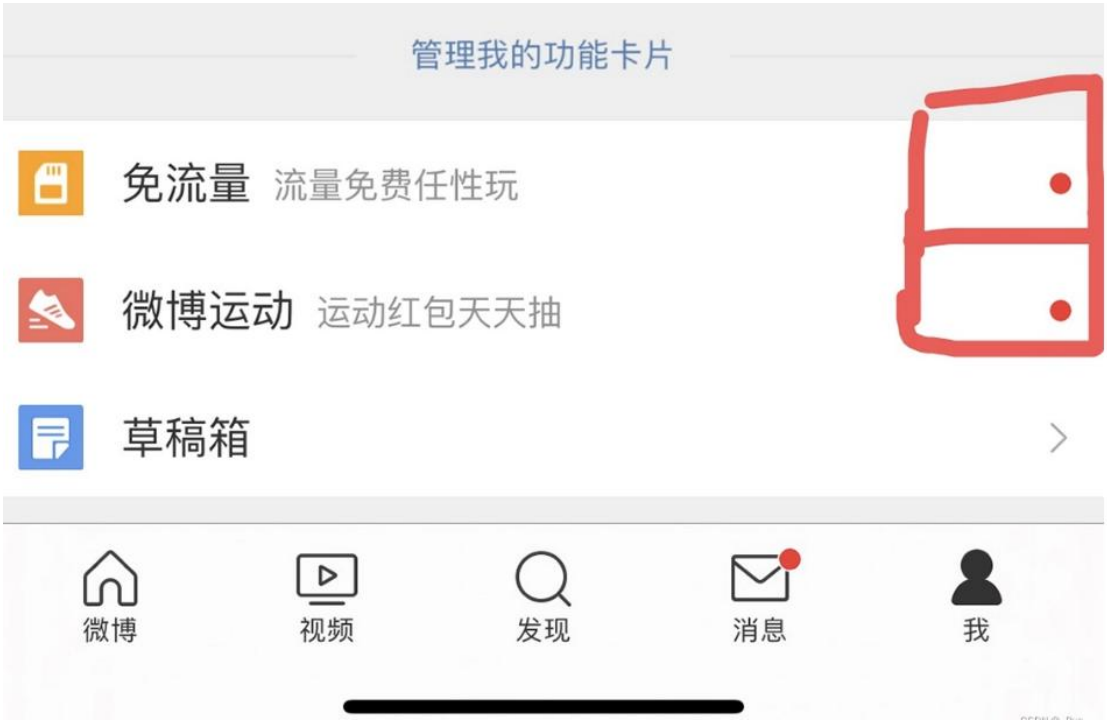
这个方案在实现时有这样几个关键点：

用户访问系统通知页面需要设置未读数为 0，我们需要将用户最近看过的通知 ID 设置为最新的一条系统通知 ID；

如果最近看过的通知 ID 为空，则认为是一个新的用户，返回未读数为 0；

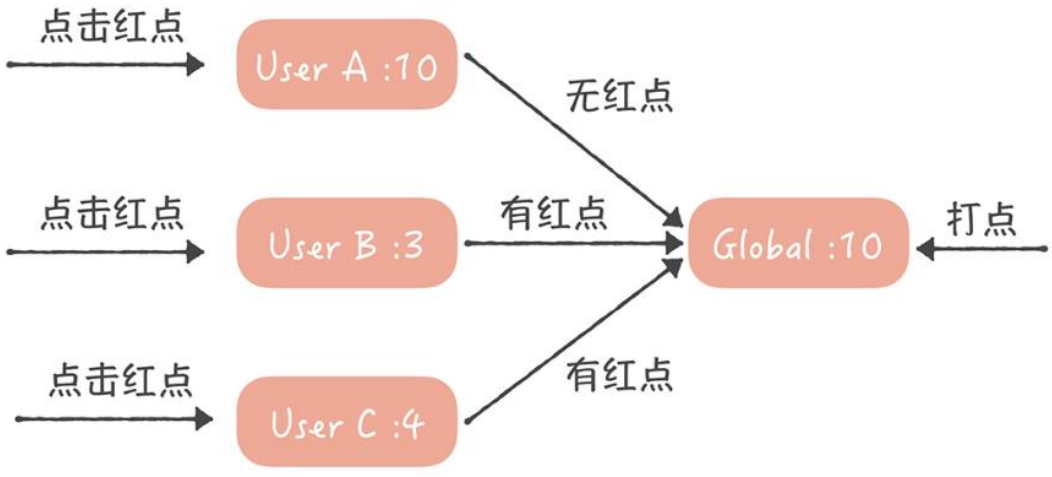
对于非活跃用户，比如最近一个月都没有登录和使用过系统的用户，可以把用户最近看过的通知 ID 清空，节省内存空间。

这是一种比较通用的方案，既节省内存，又能尽量减少获取未读数的延迟。 这个方案适用的另一个业务场景是全量用户打点的场景，比如像下面这张微博截图中的红点。



这个红点和系统通知类似，也是一种通知全量用户的手段，如果逐个通知用户，延迟也是无法接受的。因此你可以采用和系统通知类似的方案。

首先，我们为每一个用户存储一个时间戳，代表最近点过这个红点的时间，用户点了红点，就把这个时间戳设置为当前时间；然后，我们也记录一个全局的时间戳，这个时间戳标识最新的一次打点时间，如果你在后台操作给全体用户打点，就更新这个时间戳为当前时间。而我们在判断是否需要展示红点时，只需要判断用户的时间戳和全局时间戳的大小，如果用户时间戳小于全局时间戳，代表在用户最后一次点击红点之后又有新的红点推送，那么就要展示红点，反之，就不展示红点了。



这两个场景的共性是全部用户共享一份有限的存储数据，每个人只记录自己在这份存储中的偏移量，就可以得到未读数了。

你可以看到，系统消息未读的实现方案不是很复杂，它通过设计避免了操作全量数据未读数，如果你的系统中有这种打红点的需求，那我建议你可以结合实际工作灵活使用上述方案。

最后一个需求关注的是微博信息流的未读数，在现在的社交系统中，关注关系已经成为标配的功能，而基于关注关系的信息流也是一种非常重要的信息聚合方式，因此，如何设计信息流的未读数系统就成了你必须面对的一个问题。

32.2 如何为信息流的未读数设计方案

信息流的未读数之所以复杂主要有这样几点原因。

首先，微博的信息流是基于关注关系的，未读数也是基于关注关系的，就是说，你关注的人发布了新的微博，那么你作为粉丝未读数就要增加 1。如果微博用户都是像我这样只有几百粉丝的“小透明”就简单了，你发微博的时候系统给你粉丝的未读数增加 1 不是什么难事儿。但是对于一些动辄几千万甚至上亿粉丝的微博大 V 就麻烦了，增加未读数可能需要几个小时。假设你是杨幂的粉丝，想了解她实时发布的博文，那么如果当她发布博文几个小时之后，你才收到提醒，这显然是不能接受的。所以未读数的延迟是你在设计方案时首先要考虑的内容。

其次，信息流未读数请求量极大、并发极高，这是因为接口是客户端轮询请求的，不是用户触发的。也就是说，用户即使打开微博客户端什么都不做，这个接口也会被请求到。在几年前，请求未读数接口的量级就已经接近每秒 50 万次，这几年随着微博量级的增长，请求量也变得更高。而作为微博的非核心接口，我们不太可能使用大量的机器来抗未读数请求，因此，如何使用有限的资源来支撑如此高的流量是这个方案的难点。

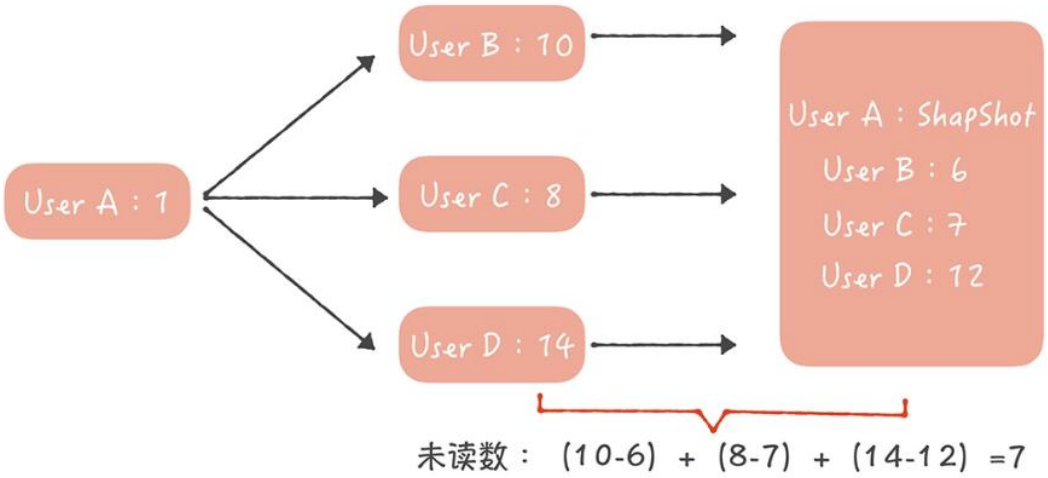
最后，它不像系统通知那样有共享的存储，因为每个人关注的人不同，信息流的列表也就不同，所以也就没办法采用系统通知未读数的方案。

那要如何设计能够承接每秒几十万次请求的信息流未读数系统呢？你可以这样做：

首先，在通用计数器中记录每一个用户发布的博文数；

然后在 Redis 或者 Memcached 中记录一个人所有关注人的博文数快照，当用户点击未读消息重置未读数为 0 时，将他关注所有人的博文数刷新到快照中；

这样，他关注所有人的博文总数减去快照中的博文总数就是他的信息流未读数。



假如用户 A，像上图这样关注了用户 B、C、D，其中 B 发布的博文数是 10，C 发布的博文数是 8，D 发布的博文数是 14，而在用户 A 最近一次查看未读消息时，记录在快照中的这三个用户的博文数分别是 6、7、12，因此用户 A 的未读数就是（10-6）+（8-7）+（14-12）=7。

这个方案设计简单，并且是全内存操作，性能足够好，能够支撑比较高的并发，事实上微博团队仅仅用 16 台普通的服务器就支撑了每秒接近 50 万次的请求，这就足以证明这个方案的性能有多出色，因此，它完全能够满足信息流未读数的需求。

当然了这个方案也有一些缺陷，比如说快照中需要存储关注关系，如果关注关系变更的时候更新不及时，那么就会造成未读数不准确；快照采用的是全缓存存储，如果缓存满了就会剔除一些数据，那么被剔除用户的未读数就变为 0 了。但是好在用户对于未读数的准确度要求不高（未读 10 条还是 11 条，其实用户有时候看不出来），因此，这些缺陷也是可以接受的。

通过分享未读数系统设计这个案例，我想给你一些建议：

缓存是提升系统性能和抵抗大并发量的神器，像是微博信息流未读数这么大的量级我们仅仅使用十几台服务器就可以支撑，这全都是缓存的功劳；

要围绕系统设计的关键困难点想解决办法，就像我们解决系统通知未读数的延迟问题一样；

合理分析业务场景，明确哪些是可以权衡的，哪些是不行的，会对你的系统设计增益良多，比如对于长久不登录用户，我们就会记录未读数为 0，通过这样的权衡，可以极大地减少内存的占用，减少成本。

33. 面试必问之 Java 常见线上故障排查方案详解

在软件开发过程中，排查和修复产线问题是每一位工程师都需要掌握的基本技能。但是在生产环境中，

程序代码、硬件、网络、协作软件等任一因素，都会引发意想不到的问题，所以排查产线问题比较困

难，所以问题的定位体现了一名工程师的基础能力，问题的解决则体现了工程师的技能素养。

以下从 5 个方面分享产线常见问题的排查手段。

1. Java 常见线上问题总结
2. 如何定位问题
3. APM 链路跟踪分析
4. 常用Linux 分析命令
5. Arthas（阿尔萨斯）诊断命令
6. JVM 问题定位命令
7. GC 分析

33.1 Java 常见线上问题总结

绝大多数 Java 线上问题从表象来看通常可以归纳为 4 个方面：**CPU**、**内存**、**磁盘**、网**络**。比如，应用上线

后突然 CPU 使用率 99%、内存泄漏、STW 时间过长，这些问题通常可以分为两大类：

系统异常 （CPU 占用率过高、磁盘使用率 100%、系统可用内存低等）

业务异常 （服务运行一段时间自动退出、服务间调用时间过长、多线程并发异常、死锁等）

33.2 如何定位问题

解决问题的第一步是定位问题，因为只有定位到了问题产生的原因，才能准确的抉择出解决方案，排查

手段一般包括以下几项，也可以将此理解为排查顺序：

1. 业务日志分析排查
2. APM 分析排查
3. 外部环境排查
4. 应用服务排查
5. 云厂商或运营商问题排查

33.2.1 业务日志分析排查

通常情况下，大部分错误信息都会在日志上有所体现，比如看看下面这段代码

```
68      issueDOList.parallelStream().filter(v -> StringUtils.isNotBlank(v.getProjectCategoryName())).distinct()
69              .forEach(v -> {
70                  String key = v.getProjectCategoryName();
71                  if (key.length() > 3) {
72                      key = key.substring(0, 3);
73                  }
74                  List<Long> idList = departIssueIdMap.getOrDefault(key, Lists.newArrayList());
75                  idList.add(v.getId());
76                  departIssueIdMap.put(key, idList);
77              });
```

这段代码使用了 并发流 将数据加入到一个 List 中，再运行过程中，抛出了 ArrayIndexOutOfBoundsException ，具体异常栈信息如下：

```
[2020-07-29 17:41:45.347+0800] [http-nio-8090-exec-4] ERROR c.c.b.d.w.e.GlobalExceptionHandler [80] - 163
java.lang.ArrayIndexOutOfBoundsException: 163
    at java.util.ArrayList.add(ArrayList.java:463)
    at [redacted].service.impl.TaskServiceImpl.lambda$queryDeliveryCycle$1(TaskServiceImpl.java:75)
    at java.util.stream.ForEachOps$ForEachOp$OfRef.accept(ForEachOps.java:184) <2 internal calls>
    at java.util.stream.ForEachOps$ForEachTask.compute(ForEachOps.java:291) <4 internal calls>
    at java.util.stream.ForEachOps$ForEachOp.evaluateParallel(ForEachOps.java:160)
    at java.util.stream.ForEachOps$ForEachOp$OfRef.evaluateParallel(ForEachOps.java:174) <1 internal call>
    at java.util.stream.ReferencePipeline.forEach(ReferencePipeline.java:418)
    at [redacted]service.impl.TaskServiceImpl.queryDeliveryCycle(TaskServiceImpl.java:69)
    at [redacted].web.controller.TaskController.deliveryCycle(TaskController.java:36) <14 internal calls>
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:634) <1 internal call>
    at javax.servlet.http.HttpServlet.service(HttpServlet.java:741)
    at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:231)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:166)
    at org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:53)
```

通过日志可以看到出错的位置在 TaskServiceImpl.java 文件的第 75行代码上，错误信息

是 java.lang.ArrayIndexOutOfBoundsException:163 ，这就是问题定位，整体描述出来就是： 在

TaskServiceImpl.java文件中第 75行代码上出现了数组越界异常，引起异常的原因是在调用
java.util.ArrayList#add()方法时因为并发请求导致没有动态扩展数组容量 。
为什么我们能够直接描述出来是在调用 ArrayList#add() 方法时因为并发请求导致的没有动态扩展数
组容量呢？因为我们结合 ArrayList 的内部实现原理及动态扩容的特性，可以知道在单线程的情况下，代
码是串行执行，ArrayList 内的数组都会是先扩容再插入，所以不会出现数组越界的错误，既然单线程不
会引起该错误，那一定就是多线程并发造成的了，解决方案就是给代码加锁或者由并发改串行，就是这
么简单，这就是技能素养的体现。
通过上面的分析，教育了我们：一定要在关键代码逻辑位置输出相关日志，尤其是在代码发生异常的时
候，定要将日志输出到文件中，只有这样，才更利于我们的排查。

33.2.2 APM 分析排查

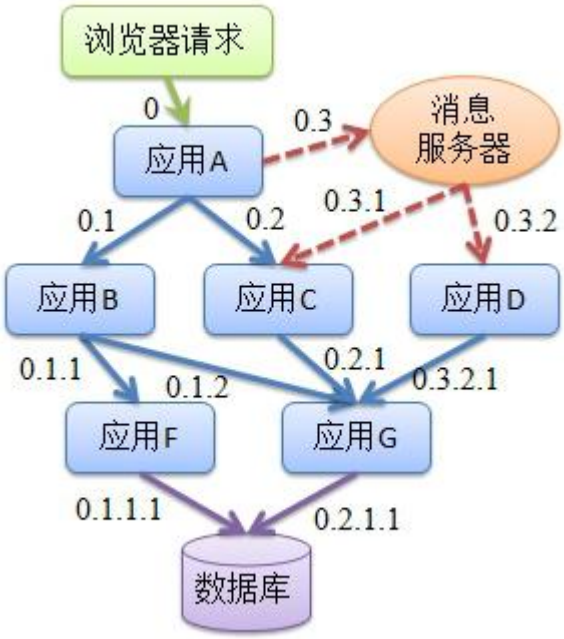
APM，全称 Application Performance Management，应用性能管理，目的是通过各种探针采集数据，
收集关键指标，同时搭配数据呈现以实现对应用程序性能管理和故障管理的系统化解决方案。通过分布
式链路调用跟踪系统，通过在系统请求中透传 trace-id，将所有相关日志进行聚合，然后日志统一采集
和分析后，以图形化的形式展示给工程师们，而他们在排查问题的时候，可以简单粗暴且直观的调度出
问题最根本的原因。
业务日志较优于解决单体服务异常，但现在的应用通常使用的是分布式架构，而在分布式系统中，仅通

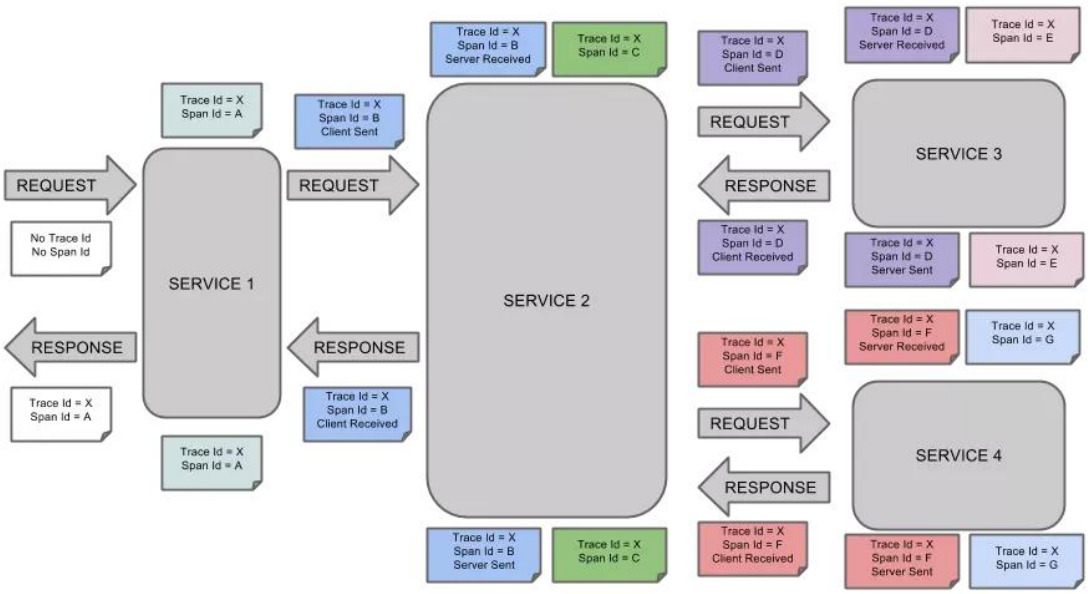
过单服务节点的日志，很难将错误信息与请求链路关联在一起，也就是通过系统中某个服务的异常日志

信息很难定位到问题的根本原因。并且我们还需要关注各服务执行过程中的耗时情况，及时的发现异常

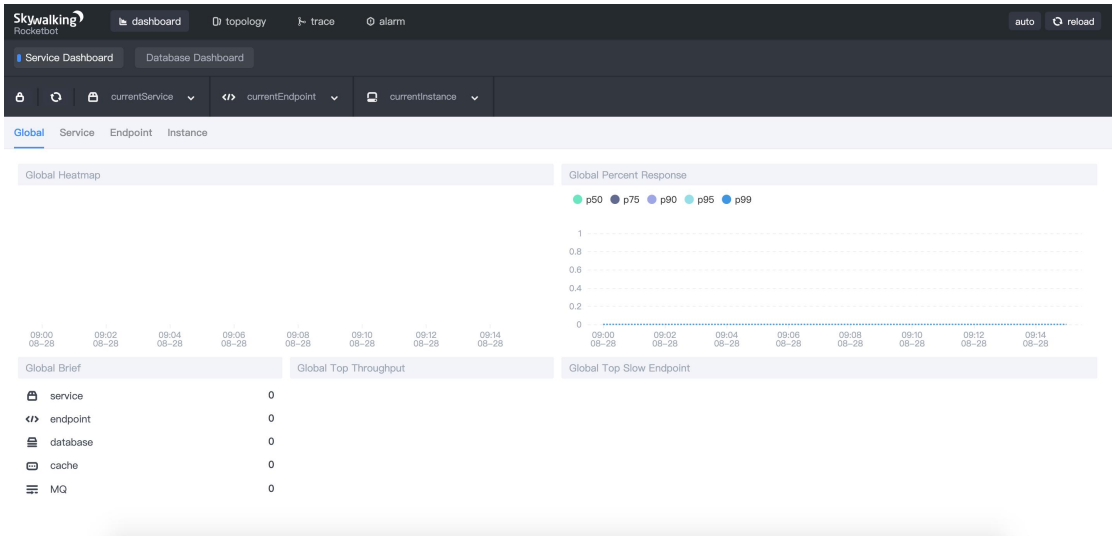
服务，并根据异常信息进行修复。要满足这种需求，仅通过分析单个服务的日志信息是不够的，此时则

需要 APM 进行全链路分析，通过请求链路监控，实时的发现链路中相关服务的异常情况。





比如我们使用Skywalking，进入到 Skywalking 的控制台查看的链路信息就是这样的：



目前市场上使用较多的链路跟踪工具有如下几个：

Apache Skywalking：https://skywalking.apache.org/

Pinpoint：https://pinpoint.com/product/for-engineers

SpringCloud Zipkin：

https://docs.spring.io/spring-cloud-sleuth/docs/current-SNAPSHOT/referenc

e/html/#sending-spans-to-zipkin

33.2.3 物理环境排查

物理环境是指工程所依附的物理环境，比如服务器、宿主机、容器等，细分为服务器负载、CPU、内

存、磁盘、网络几个方面。

33.2.3.1 CPU 分析

排查 CPU 的目的主要是查看服务器 CPU 的使用率， 使用top 命令分析 CPU 使用情况

33.2.3.2 内存分析

使用free -m 命令查看内存使用情况

33.2.3.3 磁盘分析

使用df -h、iostat、lsof 等命令查看磁盘 IO 情况，找到读写异常的进程

33.2.3.4 网络分析

使用dstat、vmstat 等命令查看网络流量、TCP 连接等情况，分析异常流量

33.2.4 应用服务排查

应用排查，排查应用本身最有可能引发的问题，针对各种场景进行对应分析

33.2.4.1 CPU 分析

使用jstack 等命令进行JVM 分析

33.2.4.2 内存分析

使用jmap 等命令分析内存使用情况

33.2.4 云厂商或运营商问题排查

排查到了这一步的话，只需关注云厂商或运营商官方公告即可。

假设我们从业务日志、APM 分析中都没分析出问题所在，那么此时基本只能连接到生产环境中进行排查

了。根据上面的外部排查方案，这里简单复习下常用的 Linux 分析命令以便从系统层面上分析。

33.3 常用Linux 分析命令

针对外部环境排查，需要使用的常用Linux 分析命令包括 ：top、free、df、lsof、dstat、vmstat 等，简

单介绍如下：

33.3.1 CPU

CPU 使用率是衡量系统繁忙程度的重要指标。但是 **CPU 使用率的安全阈值是相对的，取决于你的系统的**

IO 密集型还是计算密集型。一般计算密集型应用CPU 使用率偏高load 偏低，IO 密集型相反。

top 命令是 Linux 下常用的 CPU 性能分析工具，能够实时显示系统中各个进程的资源占用状况，常用于服
务端性能分析。

```
1 top
```

```
top - 22:29:37 up 462 days, 11:19,  5 users,  load average: 3.40, 3.31, 3.46
Tasks: 1163 total,   2 running, 1161 sleeping,   0 stopped,   0 zombie
%Cpu(s):  7.8 us,  0.8 sy,  0.0 ni, 90.1 id,  0.7 wa,  0.1 hi,  0.5 si,  0.0 st
KiB Mem : 26375555+total,  2018780 free, 81015696 used, 18072107+buff/cache
KiB Swap:        0 total,        0 free,        0 used. 17856694+avail Mem

   PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
437968 root      20   0 11.076g 1.467g  18696 S 102.3  0.6  19046:43 java
431187 root      20   0 7329660 321964  60740 S  38.0  0.1  14824:43 kubelet
570909 root      20   0 6799696 1.628g  19088 S  31.7  0.6   0:25.11 java
294774 root      20   0 14.858g 4.240g  21680 S  31.0  1.7  23:43.82 java
407680 root      20   0 6341384 263112  41428 S  22.8  0.1  23145:08 dockerd
146395 root      20   0  272456  45448   7052 D   9.9  0.0   1520:52 fluent-bit
511982 root      20   0 4049640 289960  79340 S   5.0  0.1 858:09.74 kube-controller
463265 root      20   0       0      0      0 S   3.0  0.0   1:20.88 kworker/9:2
574073 root      20   0 4904136 865396  87288 S   2.6  0.3   1371:55 kube-apiserver
294750 root      20   0   11788   5588   4312 S   2.3  0.0   1:14.30 containerd-shim
384969 root      20   0 10.464g 1.867g  20092 S   1.7  0.7  34:28.46 java
578763 root      20   0  180200  90128  38836 S   1.7  0.0 371:42.43 ks-apiserver
408716 root      20   0 10.153g 138260  25048 S   1.3  0.1   1706:51 etcd
     1 root      20   0  197064  11708   5540 S   1.0  0.0   2599:00 systemd
169908 root      20   0 38.605g 1.084g  16308 S   1.0  0.4 469:20.59 java
438145 root      20   0  149008  52612  25920 S   1.0  0.0   1564:54 calico-node
512012 root      20   0 2856080 130544  77872 S   1.0  0.0 163:22.09 kube-scheduler
572780 root      20   0  158808   5436   3628 R   1.0  0.0   0:00.40 top
580735 root      20   0  835660 325504  35812 S   1.0  0.1 373:55.35 prometheus
 25141 telegraf  20   0   27816   4792   1996 S   0.7  0.0  40:12.16 redis-server
200683 root      20   0  9.854g 1.182g  16744 S   0.7  0.5  52:11.91 java
```

top 命令显示了各个进程 CPU 使用情况，一般 CPU 使用率从高到低排序展示输出。其中 Load
Average 显示最近 1 分钟、5 分钟和 15 分钟的系统平均负载，上图各值为 3.4、3.31、3.46。
我们一般会关注 CPU 使用率最高的进程，正常情况下就是我们的应用主进程。第七行以下：各进
程的状态监控，参数说明：
PID：进程 id
USER：进程所有者的用户名
PR：进程优先级
NI：nice 值。负值表示高优先级，正值表示低优先级
VIRT：进程使用的虚拟内存总量，单位 kb

SHR：共享内存大小

%CPU：上次更新到现在的 CPU 时间占用百分比

%MEM：进程使用的物理内存百分比

TIME+：进程使用的 CPU 时间总计，单位 1/100 秒

COMMAND：命令名称、命令行

33.3.2 内存

内存是排查线上问题的重要参考依据，free 是显示的当前内存的使用，-h 表示人类可读性

```
1 free -h
```

```
[root@ydy-bcop02 console-openstack-compute]# free -h
              total        used        free      shared  buff/cache   available
Mem:           251G         77G        2.4G        170M        171G        170G
Swap:            0B          0B          0B
```

参数说明：

total ：内存总数

used：已经使用的内存数

free：空闲的内存数

shared：被共享使用的物理内存大小

buffers/buffer：被 buffer 和 cache 使用的物理内存大小

available： 还可以被应用程序使用的物理内存大小

33.3.3 磁盘

磁盘满了很多时候会引发系统服务不可用等问题

```
1 df -h
```

```
[root@ydy-bcop02 console-openstack-compute]# df -h
Filesystem               Size  Used Avail Use% Mounted on
devtmpfs                 126G     0  126G   0% /dev
tmpfs                    126G     0  126G   0% /dev/shm
tmpfs                    126G  170M  126G   1% /run
tmpfs                    126G     0  126G   0% /sys/fs/cgroup
/dev/mapper/bfd-root     323G  314G  8.9G  98% /
/dev/sda1                488M  117M  336M  26% /boot
/dev/mapper/bfd-var      400G   20G  381G   5% /var
/dev/mapper/bfd-home      50G  915M   50G   2% /home
tmpfs                     26G     0   26G   0% /run/user/0
/dev/mapper/centos-dcos  2.0T   44G  2.0T   3% /dcos
```

33.3.4 网络

dstat 是一个可以取代 vmstat，iostat，netstat 和 ifstat 这些命令的多功能产品。

```
1 dstat
```

```
[root@ydy-bcop02 console-openstack-compute]# dstat
You did not select any stats, using -cdngy by default.
----total-cpu-usage---- -dsk/total- -net/total- ---paging-- ---system--
usr sys idl wai hiq siq| read  writ| recv  send|  in   out | int   csw
  3   1  96   0   0   0|  45k  479k|   0     0 |   0     0 |4735    29k
 18   1  79   1   0   1|   0  2563k|1759k 1777k|   0     0 |  32k   44k
  9   1  88   1   0   1|   0   504k| 419k  528k|   0     0 |  27k   42k
  6   1  92   0   0   0|   0   588k|1026k 1656k|   0     0 |  28k   48k
  4   1  93   1   0   0|   0   606k| 148k  218k|   0     0 |  21k   35k
  6   1  92   0   0   0|   0  1045k| 195k  232k|   0     0 |  27k   45k
  9   2  88   1   0   1|   0  1091k| 467k  514k|   0     0 |  32k   50k
 11   1  88   0   0   0|   0   306k| 667k  886k|   0     0 |  26k   41k
  4   0  95   0   0   0|   0   362k| 405k  460k|   0     0 |  21k   36k
  4   0  95   0   0   0|   0   463k| 131k  141k|   0     0 |  21k   38k
  6   1  92   1   0   0|   0  1297k| 253k  363k|   0     0 |  27k   46k
  5   1  93   1   0   0|   0  1388k| 280k  300k|   0     0 |  23k   38k
  6   1  92   0   0   1|   0   278k| 327k 1207k|   0     0 |  25k   42k
  6   1  92   0   0   1|   0   225k| 456k  504k|   0     0 |  26k   43k
  9   0  89   1   0   0|   0   653k| 316k  440k|   0     0 |  23k   36k
```

默认情况下，dstat 每秒都会刷新数据。需要注意的是报告的第一行，由于 dstat 会通过上一次的报

告来给出一个总结，所以第一次运行时是没有平均值和总值的相关数据。

默认输出解释：

CPU 状态：显示了用户，系统和空闲部分，便于分析 CPU 当前的使用状况

磁盘统计：磁盘的读写操作，显示磁盘的读、写总数。

网络统计：网络设备发送和接受的数据，显示了网络收、发数据总数。

分页统计：系统的分页活动。

系统统计：这一项显示的是中断（int）和上下文切换（csw）。

到了这一步，如果 CPU、内存、磁盘、网络这四个地方都没有问题的话，那就进入了应用本身的问题排

查阶段。下一步就该使用jvm 命令进行问题定位了，但很多研发可能由于自身工作经验不足、对 Java 内存

模型理解不深、尚未掌握 Jvm 排查命令等原因对 Jvm 相关排查畏手畏脚，不够自信，进而影响排查进度。

针对这种情况，本文推荐一款开源的 Java 诊断工具，对 Jvm 不熟的研发可以尝试学习使用下。相对 Jvm 命

令来说简单多了

33.4 Arthas 诊断命令

Arthas 是 Alibaba 开源的 Java 诊断工具，深受开发者喜爱。

当你遇到以下类似问题而束手无策时，Arthas 可以帮助你解决

- 这个类从哪个 jar 包加载的？为什么会报各种类相关的 Exception？
- 我改的代码为什么没有执行到？难道是我没 commit？分支搞错了？
- 遇到问题无法在线上 debug，难道只能通过加日志再重新发布吗？
- 线上遇到某个用户的数据处理有问题，但线上同样无法 debug，线下无法重现！
- 是否有一个全局视角来查看系统的运行状况？
- 有什么办法可以监控到 JVM 的实时运行状态？
- 怎么快速定位应用的热点，生成火焰图？

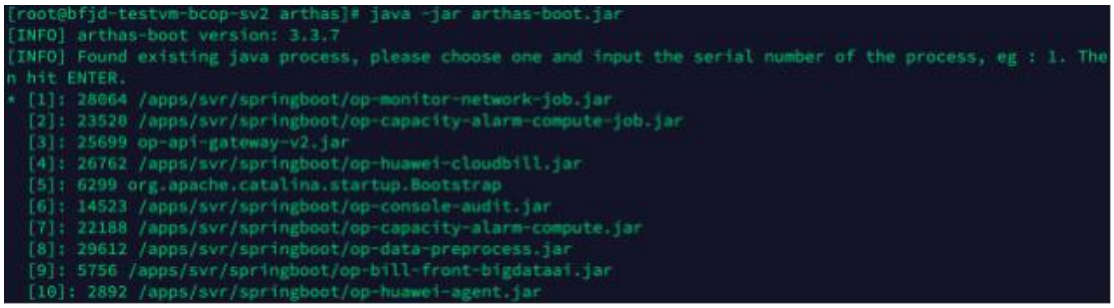
Arthas支持 JDK 6+，支持 Linux、Mac、Winodws，采用命令行交互模式，同时提供丰富的 Tab 自动补全功能，进一步方便进行问题的定位和诊断。

33.4.1 下载安装

```
curl -O https://alibaba.github.io/arthas/arthas-boot.jar
```

33.4.2 启动 Arthas

```
java -jar arthas-boot.jar
```



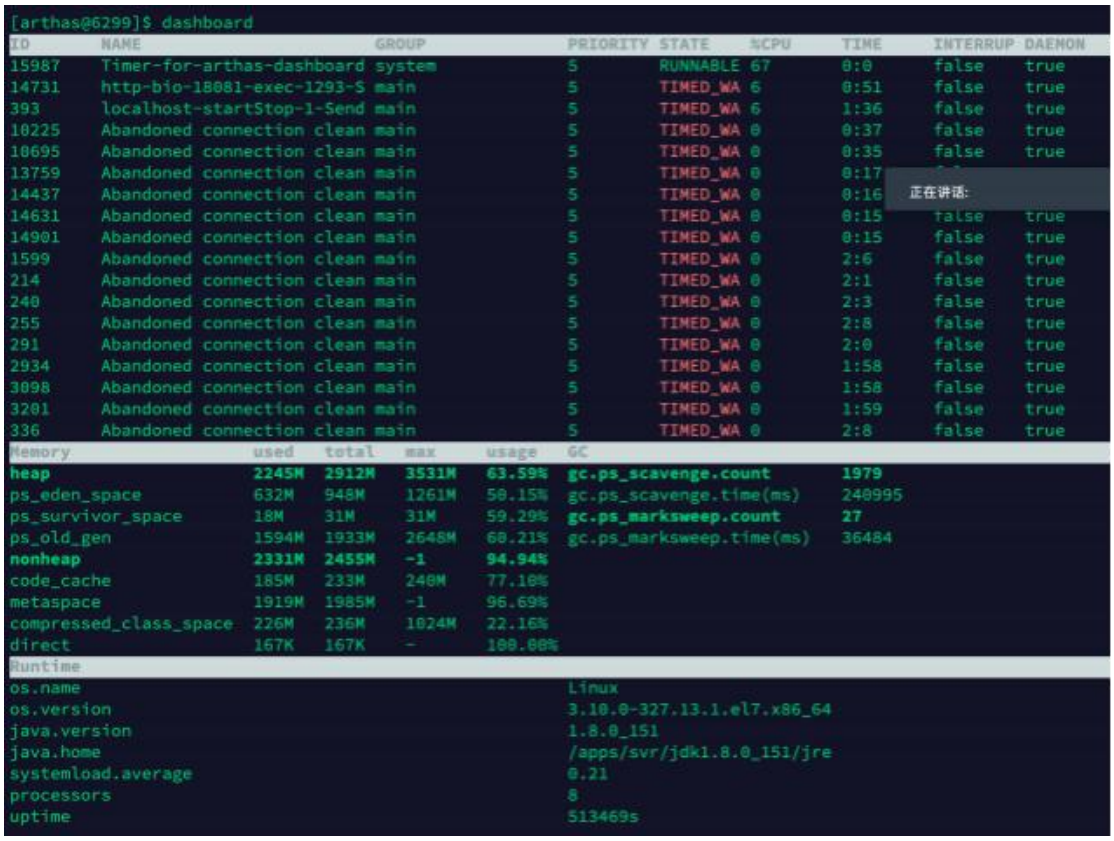
arthas 会列出已存在的 Java 进程，并提醒输入序号，键入回车，进入 arthas 诊断界面。

33.4.3 开始诊断

按照提醒，这里输入需要诊断的序号，输入回车，响应界面如下：

33.4.4 查看 dashboard

输入 dashboard【支持 Tab 补全命令】，输入回车，会展示当前系统的实时面板，按 ctrl+c 可以中断执 行。



上图线上了当前系统对应的**线程** 、**内存（分代）** 、GC 、Runtime 的各项信息。

除了 dashboard 命令外， arthas 还有许多常用命令，这里挑一些做简单说明。

33.4.5 arthas 常见命令介绍

- jvm 查看当前 JVM 的信息
- thread 查看当前 JVM 的线程堆栈信息， -b 选项可以一键检测死锁
- trace 方法内部调用路径，并输出方法路径上的每个节点上耗时，服务间调用时间过长时使用
- stack 输出当前方法被调用的调用路径
- Jad 反编译指定已加载类的源码，反编译便于理解业务
- logger 查看和修改 logger，可以动态更新日志级别。

> 这里只列出常用命令，完整列表参考命令列表：
>
> https://alibaba.github.io/arthas/commands.ht ml。

arthas 使用上相对 JVM 命令简单很多，即便是工作年限不多的小伙伴学起来也很快。

熟练使用 arthas 应 该能诊断出大部分线上应用问题，但生产环境通常不允许擅自拷

贝 jar 包，而且 arthas 会拖慢应用本身， 如果条件不允许，又该如何诊断呢？这边简单介绍下 jdk 自带的命令行工具。

33.5 JVM 问题定位命令

在 JDK 安装目录的 bin 目录下默认提供了很多有价值的命令行工具。每个小工具体积基本都比较小，因 为这些工具只是 jdk\lib\tools.jar 的简单封装。

```
[root@bfjd-testvm-bcop-sv2 bin]# ls
appletviewer  java            java-rmi.cgi  jhat     jrunscript  native2ascii  schemagen   xjc
ControlPanel  javac           javaws        jinfo    jsadebugd   orbd          serialver
idisconf      javadoc         jcmd          jjs      jstack      pack200       servertool
extcheck      javafxpackager  jconsole      jmap     jstat       policytool    tnameserv
idlj          javah           jcontrol      jmc      jstatd      rmic          unpack200
jar           javap           jdb           jmc.ini  jvisualvm   rmid          wsgen
jarsigner     javapackager    jdeps         jps      keytool     rmiregistry   wsimport
[root@bfjd-testvm-bcop-sv2 bin]#
```

其中，定位排查问题时最为常用命令包括： jps（进程）、jmap（内存）、jstack（线程）、jinfo（参 数）等。

- jps：查询当前机器所有 Java 进程信息
- jmap：输出某个 Java 进程内存情况
- jstack：打印某个 Java 线程的线程栈信息
- jinfo：用于查看 jvm 的配置参数

33.5.1 jps

jps 用于输出当前用户启动的所有进程 ID，当线上发现故障或者问题时，利用 jps 快速定位对应的 Java 进程 ID。

> 参数解释：
>
> -m ：输出传入 main 方法的参数
>
> -l：输出完全的包名，应用主类名， jar 的完全路径名

```
1 jps -m
```

```
[root@bfjd-testvm-bcop-sv2 ~]# jps -ml
28864 /apps/svr/springboot/op-monitor-network-job.jar --spring.profiles.active=release-ecloudHuaBei1
23520 /apps/svr/springboot/op-capacity-alarm-compute-job.jar --spring.profiles.active=release-ecloudHuaBei1
25699 op-api-gateway-v2.jar --spring.profiles.active=release-ecloudHuaBei2
1462 org.apache.catalina.startup.Bootstrap start
2200 sun.tools.jps.Jps -ml
10474 /apps/svr/springboot/op-huawei-agent.jar --spring.profiles.active=release-ecloudHuaBei1
26762 /apps/svr/springboot/op-huawei-cloudbill.jar --spring.profiles.active=release-ecloudHuaBei1
14523 /apps/svr/springboot/op-console-audit.jar --spring.profiles.active=dev-ecloudHuaBei1
22188 /apps/svr/springboot/op-capacity-alarm-compute.jar --spring.profiles.active=release-ecloudHuaBei1
29612 /apps/svr/springboot/op-data-preprocess.jar --spring.profiles.active=release-ecloudHuabei1
5756 /apps/svr/springboot/op-bill-front-bigdataai.jar --spring.profiles.active=release-ecloudHuaBei1
```

当然，我们也可以使用 Linux 提供的查询进程状态命令也能快速获取 Tomcat 服务的进程 id。比如：

```
ps -ef | grep tomcat
```

33.5.2 jmap

jmap（Java Memory Map）可以输出所有内存中对象的工具，甚至可以将 VM 中的 heap，以二进制输

出成文本，使用方式如下：　　jmap -heap：

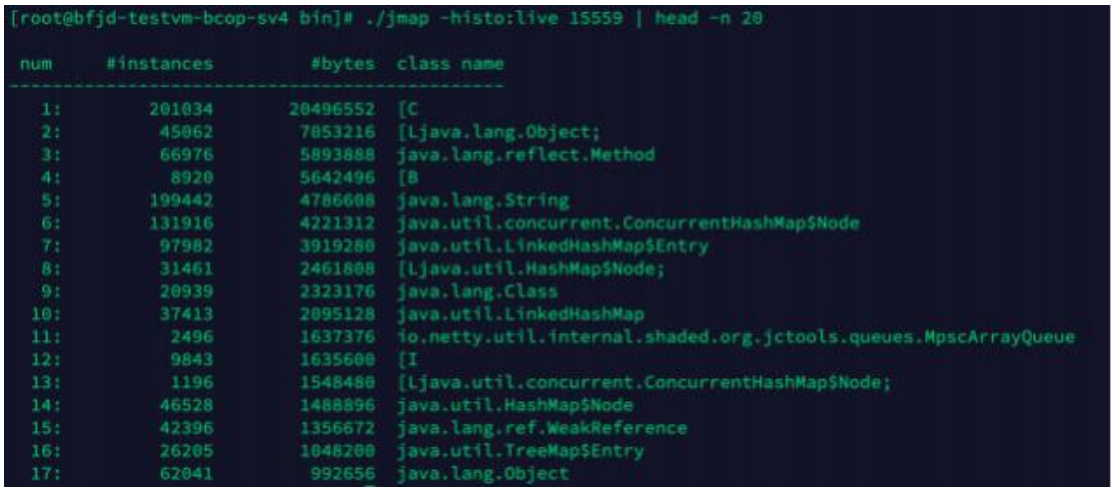
```
jmap -heap pid    输出当前进程 JVM 堆内存新生代、老年代、持久代、   GC 算法等信息
```

> **注意：** pid 通过 jps 命令得知

```
[root@bfjd-testvm-bcop-sv4 bin]# ./jmap -heap 15559
Attaching to process ID 15559, please wait...
Debugger attached successfully.
Server compiler detected.
JVM version is 25.172-b11

using thread-local object allocation.
Parallel GC with 8 thread(s)

Heap Configuration:
   MinHeapFreeRatio         = 0
   MaxHeapFreeRatio         = 100
   MaxHeapSize              = 536870912 (512.0MB)
   NewSize                  = 178782208 (170.5MB)
   MaxNewSize               = 178782208 (170.5MB)
   OldSize                  = 358088704 (341.5MB)
   NewRatio                 = 2
   SurvivorRatio            = 8
   MetaspaceSize            = 21807104 (20.796875MB)
   CompressedClassSpaceSize = 1073741824 (1024.0MB)
   MaxMetaspaceSize         = 17592186044415 MB
   G1HeapRegionSize         = 0 (0.0MB)

Heap Usage:
PS Young Generation
Eden Space:
   capacity = 173539328 (165.5MB)
   used     = 112729016 (107.50676727294922MB)
   free     = 60810312 (57.99323272705078MB)
   64.95877176613246% used
From Space:
   capacity = 2621440 (2.5MB)
   used     = 549856 (0.524383544921875MB)
   free     = 2071584 (1.975616455078125MB)
   20.975341796875% used
To Space:
   capacity = 2621440 (2.5MB)
   used     = 0 (0.0MB)
   free     = 2621440 (2.5MB)
   0.0% used
PS Old Generation
   capacity = 358088704 (341.5MB)
   used     = 100589528 (95.92964935302734MB)
   free     = 257499176 (245.57035064697266MB)
   28.09067330981767% used

55761 interned Strings occupying 5926040 bytes.
```

jmap -histo：

```
1 jmap -histo:live pid | head -n 20  输出当前进程内存中所有对象包含的大小
```



```
[root@bfjd-testvm-bcop-sv4 bin]# ./jmap -histo:live 15559 | head -n 20

 num     #instances         #bytes  class name
----------------------------------------------
   1:        201034       20496552  [C
   2:         45062        7053216  [Ljava.lang.Object;
   3:         66976        5893888  java.lang.reflect.Method
   4:          8920        5642496  [B
   5:        199442        4786608  java.lang.String
   6:        131916        4221312  java.util.concurrent.ConcurrentHashMap$Node
   7:         97982        3919280  java.util.LinkedHashMap$Entry
   8:         31461        2461808  [Ljava.util.HashMap$Node;
   9:         20939        2323176  java.lang.Class
  10:         37413        2095128  java.util.LinkedHashMap
  11:          2496        1637376  io.netty.util.internal.shaded.org.jctools.queues.MpscArrayQueue
  12:          9843        1635600  [I
  13:          1196        1548480  [Ljava.util.concurrent.ConcurrentHashMap$Node;
  14:         46528        1488896  java.util.HashMap$Node
  15:         42396        1356672  java.lang.ref.WeakReference
  16:         26205        1048200  java.util.TreeMap$Entry
  17:         62041         992656  java.lang.Object
```

输出当前进程内存中所有对象实例数（instances）和大小（bytes），如果某个业务对象实例数和大小 存在异常情况，可能存在内存泄露或者业务设计方面存在不合理之处。

jmap -dump：

```
1 jmap -dump:format=b,file=/apps/logs/gc/dump.hprof {pid}
2 以二进制形式输出当前内存的堆情况，便于导入 MAT 等工具进行分析情况
```

一般我们要求给 JVM 添加参数-XX:+Heap Dump On Out Of Memory Error OOM 确保应用发生

OOM 时 JVM 能够保存并 dump 出当前的内存镜像。当然如果你决定手动 dump 内存时， dump 操作占 据一定 CPU 时间片、内存资源、磁盘资源等，因此会带来一定的负面影响。

此外， dump 的文件可能比较大,一般我们可以考虑使用 zip 命令对文件进行压缩处理，这样在下载文件 时能减少带宽的开销。在下载 dump 文件完成之后，由于 dump 文件较大可将 dump 文件备份至制定 位置或者直接删除，以释放磁盘在这块的空间占用。

33.5.3 jstack

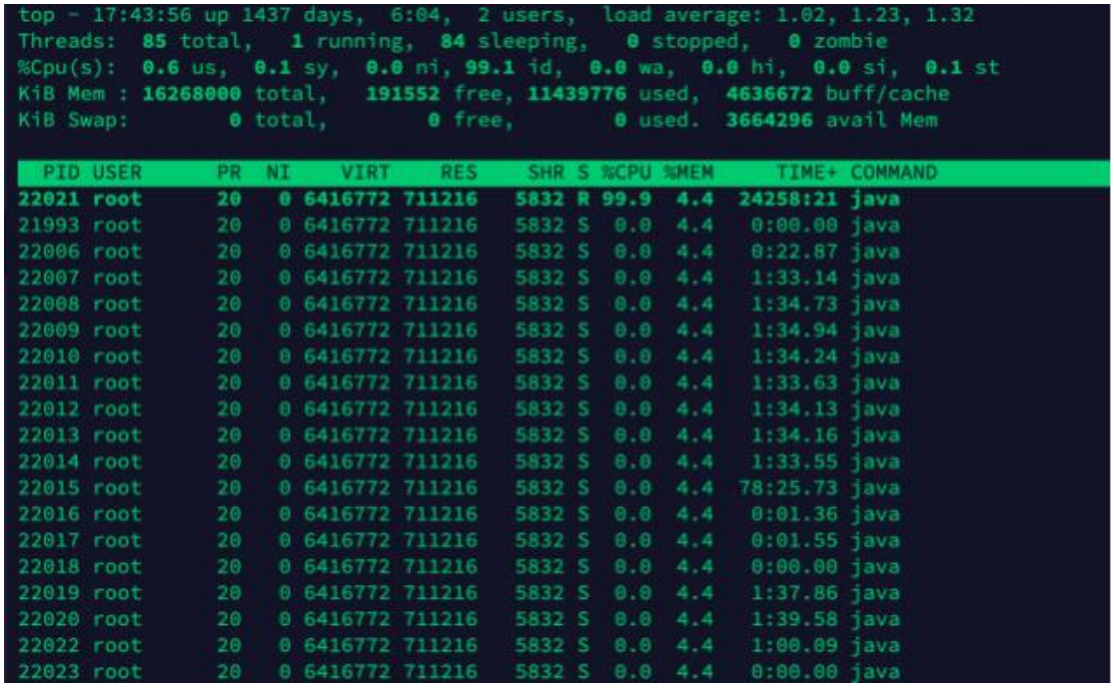
jstack 用于打印某个 Java 线程的线程栈信息，通常用以下三步分析：

```
1 printf '%x\n' tid      10 进制至 16 进制线程转换
2 jstack pid | grep tid -C 30 --color
3 ps -mp 8278 -o THREAD,tid,time | head -n 40
```

举个栗子，某 Java 进程 CPU 占用率高，我们想要定位到其中 CPU 占用率最高的线程，如何定位?

1、利用 top 命令可以查出占 CPU 最高的线程 pid

```
top -Hp pid
```

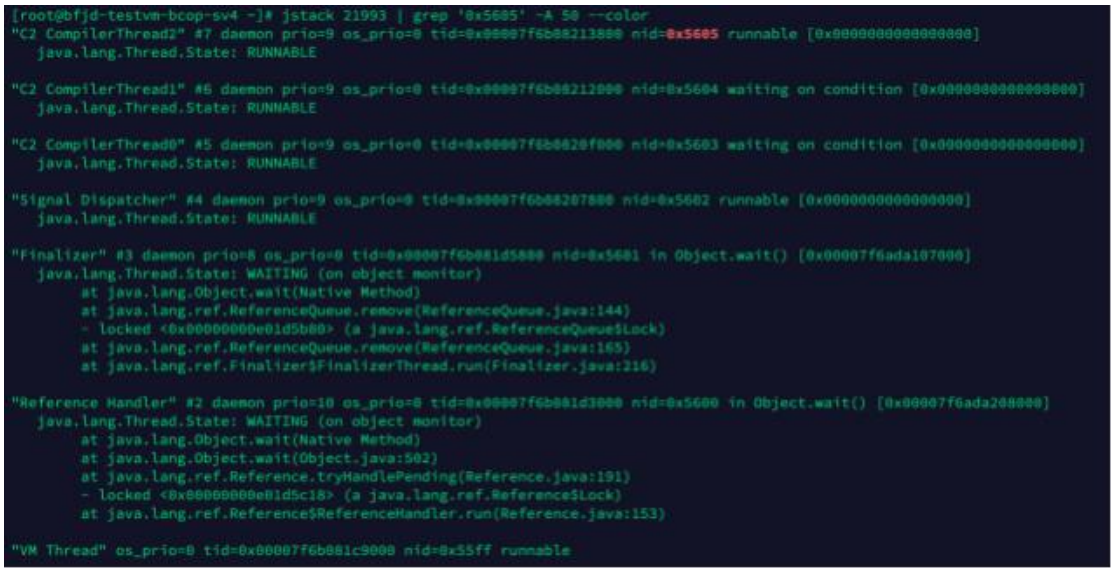


2 、 占用率最高的线程 ID 为 22021，将其转换为 16 进制形式（因为 java native 线程以 16 进制形式输 出）

```
printf '%x\n' 22021
```

3 、利用 jstack 打印出 Java 线程调用栈信息

```
jstack 21993 | grep '0x5605' -A 50 --color

```



33.5.4 jinfo

jinfo 可以用来查看正在运行的 java 应用程序的扩展参数，包括 Java System 属性和 JVM 命令行参数；也 可以动态的修改正在运行的 JVM 一些参数。

```
1 jinfo pid
```

```
[root@bfjd-testvm-bcop-sv4 bin]# ./jinfo 15559
Attaching to process ID 15559, please wait...
Debugger attached successfully.
Server compiler detected.
JVM version is 25.172-b11
Java System Properties:

rocketmq.client.logFileMaxSize = 67108864
java.runtime.name = Java(TM) SE Runtime Environment
java.vm.version = 25.172-b11
sun.boot.library.path = /apps/svr/jdk1.8.0_172/jre/lib/amd64
java.protocol.handler.pkgs = org.springframework.boot.loader
rocketmq.remoting.version = 275
java.vendor.url = http://java.oracle.com/
java.vm.vendor = Oracle Corporation
path.separator = :
file.encoding.pkg = sun.io
java.vm.name = Java HotSpot(TM) 64-Bit Server VM
sun.os.patch.level = unknown
sun.java.launcher = SUN_STANDARD
user.country = US
user.dir = /root
java.vm.specification.name = Java Virtual Machine Specification
PID = 15559
java.runtime.version = 1.8.0_172-b11
java.awt.graphicsenv = sun.awt.X11GraphicsEnvironment
rocketmq.client.logLevel = INFO
os.arch = amd64
java.endorsed.dirs = /apps/svr/jdk1.8.0_172/jre/lib/endorsed
org.jboss.logging.provider = slf4j
line.separator =
```

33.5.5 jstat

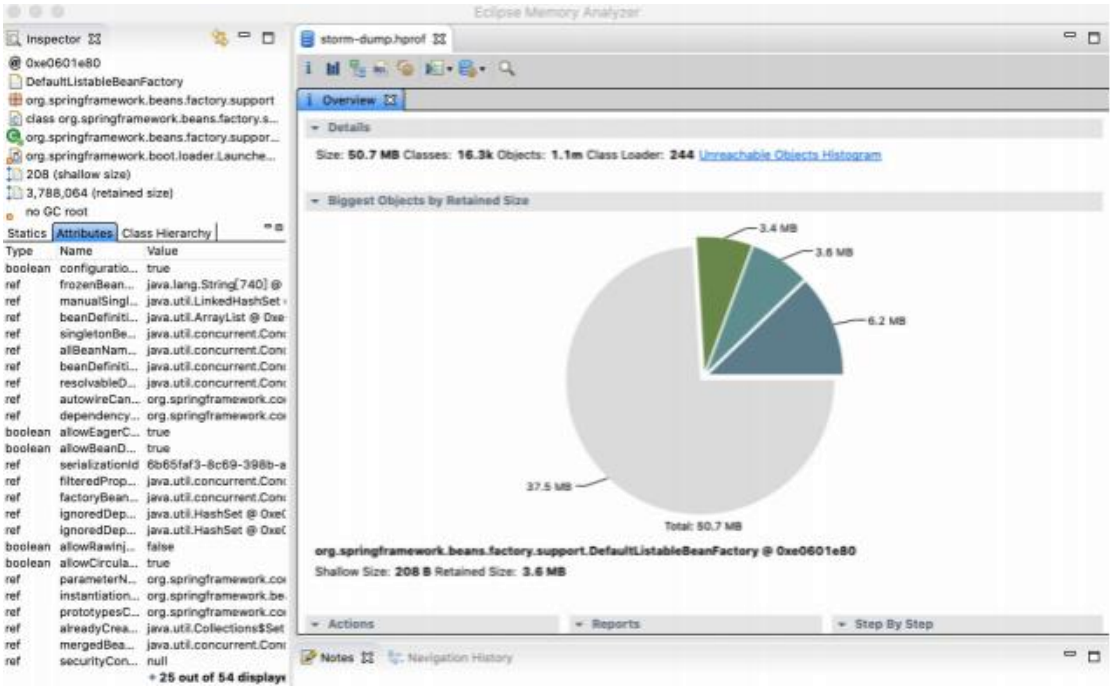
jstat 命令可以查看堆内存各部分的使用量，以及加载类的数量。

```
1 jstat -gc pid
```

```
[root@bfjd-testvm-bcop-sv4 bin]# jstat -gc 15559
Warning: Unresolved Symbol: sun.gc.generation.2.space.0.capacity substituted NaN
Warning: Unresolved Symbol: sun.gc.generation.2.space.0.used substituted NaN
 S0C    S1C    S0U    S1U      EC       EU        OC         OU       PC     PU    YGC     YGCT    FGC    FGCT     GCT
2560.0 2560.0 880.0   0.0   169472.0 116569.2  349696.0   91086.4     6      6      1088   31.590   7      3.402   34.992
```

33.5.6 内存分析工具 MAT

MAT（Memory Analyzer Tool），一个基于 Eclipse 的内存分析工具，是一个快速、功能丰富的 JAVA heap 分析工具，它可以帮助我们查找内存泄漏和减少内存消耗。使用内存分析工具从众多的对象中进行 分析，快速的计算出在内存中对象的占用大小，看看是谁阻止了垃圾收集器的回收工作，并可以通过报 表直观的查看到可能造成这种结果的对象。



右侧的饼图显示当前快照中最大的对象。单击工具栏上的柱状图，可以查看当前堆的类信息，包括类的 对象数量、浅堆（Shallow heap）、深堆（Retained Heap）。

浅堆表示一个对象结构所占用内存的大小。 **深堆**表示一个对象被回收后，可以真实释放的内存大小。

- 支配树（The Dominator Tree）：

  列出了堆中最大的对象，第二层级的节点表示当被第一层级的节点所引用到的对象，当第一层级对 象被回收时，这些对象也将被回收。这个工具可以帮助我们定位对象间的引用情况，垃圾回收时候 的引用依赖关系

- Path to GC Roots

  被 JVM 持有的对象，如当前运行的线程对象，被 systemclass loader 加载的对象被称为 GC Roots，从一个对象到 GC Roots 的引用链被称为 Path to GC Roots， 通过分析 Path to GC Roots 可以找出 JAVA 的内存泄露问题，当程序不在访问该对象时仍存在到该对象的引用路径。

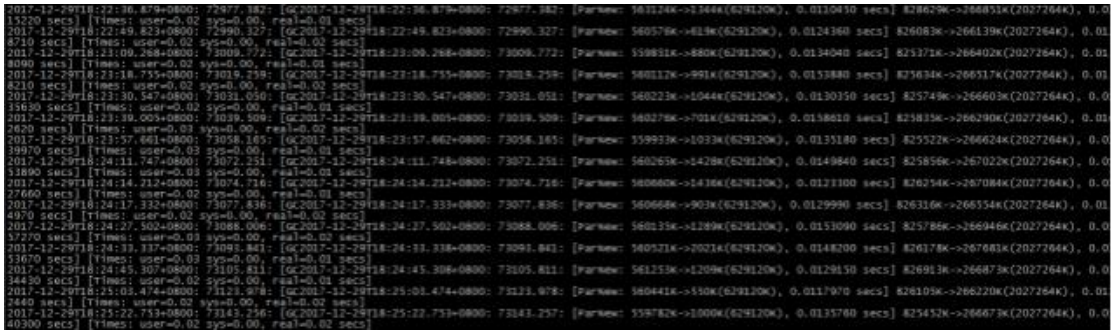
33.6 GC 分析

33.6.1 GC 日志分析

Java 虚拟机 GC 日志是用于定位问题重要的日志信息，频繁的 GC 将导致应用吞吐量下降、响应时间增 加，甚至导致服务不可用。

```
-XX:+PrintGCDetails -XX:+PrintGCDateStamps -Xloggc:/apps/logs/gc/gc.log -
XX:+UseConcMarkSweepGC
```

我们可以在 Java 应用的启动参数中增加-XX:+PrintGCDetails 可以输出 GC 的详细日志，例外还可以增 加其他的辅助参数，如-Xloggc 制定 GC 日志文件地址。如果你的应用还没有开启该参数,下次重启时请 加入该参数。



上图为线上某应用在平稳运行状态下的 GC 日志截图。

```
2017-12-29T18:25:22.753+0800: 73143.256: [GC2017-12-29T18:25:22.753+0800:
73143.257: [ParNew: 559782K->1000K(629120K), 0.0135760 secs] 825452K-
>266673K(2027264K), 0.0140300 secs] [Times: user=0.02 sys=0.00, real=0.02 secs]
```

[2017-12-29T18:25:22.753+0800: 73143.256] ： 自 JVM 启动 73143.256 秒时发生本次 GC.

[ParNew: 559782K->1000K(629120K), 0.0135760 secs]：对新生代进行的 GC，使用 ParNew 收 集器，559782K 是新生代回收前的大小,1000K 是新生代回收后大小,629120K 是当前新生代分配的内存总大 小 , 0.0135760 secs 表示本次新生代回收耗时 0.0135760 秒

[825452K->266673K(2027264K), 0.0140300 secs]:825452K 是回收堆内存大小,266673K 是回收 堆之后内存大小， 2027264K 是当前堆内存总大小,0.0140300 secs 表示本次回收共耗时 0.0140300 秒

[Times: user=0.02 sys=0.00, real=0.02 secs]：用户态耗时 0.02 秒 ,系统态耗时 0.00,实际耗 时 0.02 秒

无论是 minor GC 或者是 Full GC，我们主要关注 GC 回收实时耗时， 如 real=0.02secs，即 stop the world 时间，如果该时间过长，则严重影响应用性能。

33.6.2 CMS GC 日志分析

Concurrent Mark Sweep（CMS）是老年代垃圾收集器，从名字（Mark Sweep）可以看出， CMS 收集 器就是“标记-清除”算法实现的，分为六个步骤：

- 初始标记（STW initial mark）

- 并发标记（Concurrent marking）
- 并发预清理（Concurrent precleaning）
- 重新标记（STW remark）
- 并发清理（Concurrent sweeping）
- 并发重置（Concurrent reset）

其中初始标记（STW initial mark） 和重新标记（STW remark）需要“Stop the World”。

初始标记：在这个阶段，需要虚拟机停顿正在执行的任务，官方的叫法 STW（Stop The Word）。这个过程从垃圾回收的"根对象"开始，只扫描到能够和"根对象"直接关联的对象，并作标记。所以这个过程虽 然暂停了整个 JVM，但是很快就完成了。

并发标记：这个阶段紧随初始标记阶段，在初始标记的基础上继续向下追溯标记。并发标记阶段，应用 程序的线程和并发标记的线程并发执行，所以用户不会感受到停顿。

并发预清理：并发预清理阶段仍然是并发的。在这个阶段，虚拟机查找在执行并发标记阶段新进入老年 代的对象（可能会有一些对象从新生代晋升到老年代， 或者有一些对象被分配到老年代）。通过重新扫 描，减少下一个阶段"重新标记"的工作，因为下一个阶段会 Stop The World。

重新标记：这个阶段会暂停虚拟机，收集器线程扫描在 CMS 堆中剩余的对象。扫描从"跟对象"开始向下 追溯，并处理对象关联。

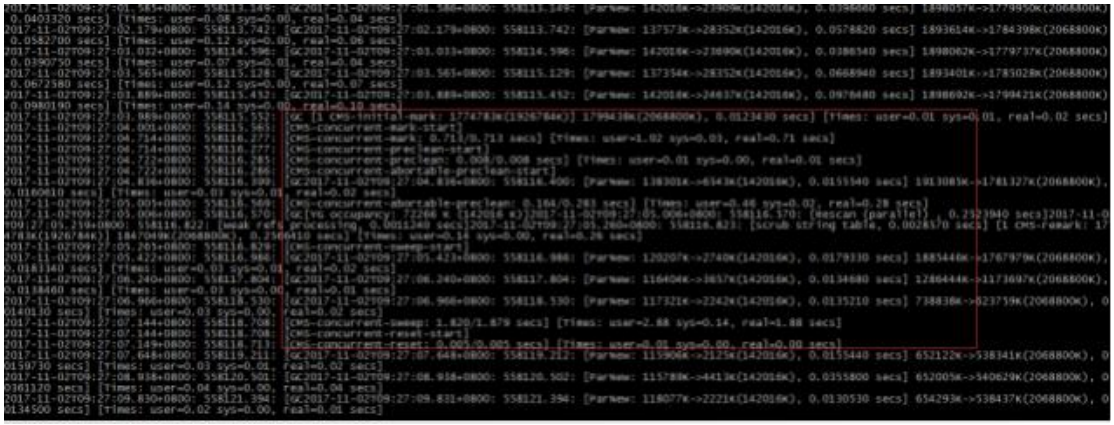
并发清理：清理垃圾对象，这个阶段收集器线程和应用程序线程并发执行。

并发重置：这个阶段，重置 CMS 收集器的数据结构，等待下一次垃圾回收。

CMS 使得在整个收集的过程中只是很短的暂停应用的执行，可通过在 JVM 参数中设置-

XX:UseConcMarkSweepGC 来使用此收集器，不过此收集器仅用于 old 和 Perm（永生）的对象收集。 CMS 减少了 stop the world 的次数，不可避免地让整体 GC 的时间拉长了。

Full GC 的次数说的是 stop the world 的次数，所以一次 CMS 至少会让 Full GC 的次数+2，因为 CMS Initial mark 和 remark 都会 stop the world，记做 2 次。而 CMS 可能失败再引发一次 Full GC。



上图为线上某应用在进行 CMS GC 状态下的 GC 日志截图。

```
2017-11-02T09:27:03.989+0800: 558115.552: [GC [1 CMS-initial-mark:
1774783K(1926784K)] 1799438K(2068800K), 0.0123430 secs] [Times: user=0.01
sys=0.01, real=0.02 secs]
2017-11-02T09:27:04.001+0800: 558115.565: [CMS-concurrent-mark-start]
2017-11-02T09:27:04.714+0800: 558116.277: [CMS-concurrent-mark: 0.713/0.713
secs] [Times: user=1.02 sys=0.03, real=0.71 secs]
2017-11-02T09:27:04.714+0800: 558116.277: [CMS-concurrent-preclean-start]
2017-11-02T09:27:04.722+0800: 558116.285: [CMS-concurrent-preclean: 0.008/0.008
secs] [Times: user=0.01 sys=0.00, real=0.01 secs]
2017-11-02T09:27:04.722+0800: 558116.286: [CMS-concurrent-abortable-preclean-
start]
2017-11-02T09:27:04.836+0800: 558116.399: [GC2017-11-02T09:27:04.836+0800:
558116.400: [ParNew: 138301K->6543K(142016K), 0.0155540 secs] 1913085K-
>1781327K(2068800K), 0.0160610 secs] [Times: user=0.03 sys=0.01, real=0.02
secs]
2017-11-02T09:27:05.005+0800: 558116.569: [CMS-concurrent-abortable-preclean:
0.164/0.283 secs] [Times: user=0.46 sys=0.02, real=0.28 secs]
2017-11-02T09:27:05.006+0800: 558116.570: [GC[YG occupancy: 72266 K (142016
K)]2017-11-02T09:27:05.006+0800: 558116.570: [Rescan (parallel) , 0.2523940
secs]2017-11-02T09:27:05.259+0800: 558116.822: [weak refs processing, 0.0011240
secs]2017-11-02T09:27:05.260+0800: 558116.823: [scrub string table, 0.0028570
secs] [1 CMS-remark: 1774783K(1926784K)] 1847049K(2068800K), 0.2566410 secs]
[Times: user=0.14 sys=0.00, real=0.26 secs]
2017-11-02T09:27:05.265+0800: 558116.829: [CMS-concurrent-sweep-start]
2017-11-02T09:27:05.422+0800: 558116.986: [GC2017-11-02T09:27:05.423+0800:
558116.986: [ParNew: 120207K->2740K(142016K), 0.0179330 secs] 1885446K-
>1767979K(2068800K), 0.0183340 secs] [Times: user=0.03 sys=0.01, real=0.02
secs]
2017-11-02T09:27:06.240+0800: 558117.804: [GC2017-11-02T09:27:06.240+0800:
558117.804: [ParNew: 116404K->3657K(142016K), 0.0134680 secs] 1286444K-
>1173697K(2068800K), 0.0138460 secs] [Times: user=0.03 sys=0.00, real=0.01
secs]
2017-11-02T09:27:06.966+0800: 558118.530: [GC2017-11-02T09:27:06.966+0800:
```

```
558118.530: [ParNew: 117321K->2242K(142016K), 0.0135210 secs] 738838K-
>623759K(2068800K), 0.0140130 secs] [Times: user=0.03 sys=0.00, real=0.02 secs]
2017-11-02T09:27:07.144+0800: 558118.708: [CMS-concurrent-sweep: 1.820/1.879
secs] [Times: user=2.88 sys=0.14, real=1.88 secs]
2017-11-02T09:27:07.144+0800: 558118.708: [CMS-concurrent-reset-start]
2017-11-02T09:27:07.149+0800: 558118.713: [CMS-concurrent-reset: 0.005/0.005
secs] [Times: user=0.01 sys=0.00, real=0.00 secs]
```

如果你已掌握 CMS 的垃圾收集过程，那么上面的 GC 日志你应该很容易就能看的懂，这里我就不详细展 开解释说明了。

此外 CMS 进行垃圾回收时也有可能会发生失败的情况。

异常情况有：

伴随 prommotion failed，然后 Full GC：

[prommotion failed：存活区内存不足，对象进入老年代，而此时老年代也仍然没有内存容纳对象，将 导致一次 Full GC]

伴随 concurrent mode failed，然后 Full GC：

[concurrent mode failed ：CMS 回收速度慢， CMS 完成前，老年代已被占满，将导致一次 Full GC]

频繁 CMS GC：

[内存吃紧，老年代长时间处于较满的状态]

33.7 常见问题分析、解答

问： 生产环境里某个节点里的容器里面服务挂了，但是在预生产环境无法重现，而生产环境监控里面显 示 cpu、内存、堆栈都正常。而那个节点重启之后就正常，但是过段时间又出现问题。这种情况有什么 好的方法排查问题?

答： 这种情况首先需要排查内核版本，排查 tomcat 版本，可能自身应用存在 bug，真的没办法可以考虑 降低日志等级，打印出更为详细有价值的启动/业务日志信息。

问：线上排查问题 dump 或 jstack 导出文件分析经常无响应导致无法导出查看内存问题和线程问题，有 什么办法吗?

答： 增加-F 强制，在 jmap-dump 使用，如果 pid 没有相应的回复。当然-F 也可能没办法 dump 下 来，那就只能去分析日志了。当然如果你决定手动 dump 内存时，dump 操作占据一定 CPU 时间片、 内存资源、磁盘资源等，对系统的正常的运行可能会有一些影响。

33.8 案例分析

某工程消费 ActiveMQ 消息，该工程整合了 Apache Camel，经过运维排查定位到 Camel 拉取到了消息， 但收到消息后打印日志时却延迟了十几分钟， Camel 配置了 MySQL 用于校验幂等，一些列初步排查下 来，未找到原因，该如何排查?
线上问题出错，而且只有线上环境能复现问题，在运维人员同意的情况下，将 Alibaba Arthas 拷贝进容器中，启动 Arthas，开始跟踪，由于是 Springboot 工程，并且 Camel 作为消息入口，故先从配置开始跟 踪，详细排查问题如下：

33.8.1 延迟场景，首先排查有无死锁产生

```
1 thread -b
```

```
[arthas@7]$ thread -b
No most blocking thread found!
Affect(row-cnt:0) cost in 49 ms.
[arthas@7]$ thread -b
No most blocking thread found!
Affect(row-cnt:0) cost in 45 ms.
[arthas@7]$ thread -b
"ActiveMQ Task-1" Id=1395 RUNNABLE
    at java.net.PlainSocketImpl.socketConnect(Native Method)
    at java.net.AbstractPlainSocketImpl.doConnect(AbstractPlainSocketImpl.java:350)
    -  locked java.net.SocksSocketImpl@45c6ddc5
    at java.net.AbstractPlainSocketImpl.connectToAddress(AbstractPlainSocketImpl.java:2
06)
    at java.net.AbstractPlainSocketImpl.connect(AbstractPlainSocketImpl.java:188)
    at java.net.SocksSocketImpl.connect(SocksSocketImpl.java:392)
    at java.net.Socket.connect(Socket.java:589)
    at org.apache.activemq.transport.tcp.TcpTransport.connect(TcpTransport.java:525)
    at org.apache.activemq.transport.tcp.TcpTransport.doStart(TcpTransport.java:488)
    at org.apache.activemq.util.ServiceSupport.start(ServiceSupport.java:55)
    at org.apache.activemq.transport.AbstractInactivityMonitor.start(AbstractInactivity
Monitor.java:169)
    at org.apache.activemq.transport.InactivityMonitor.start(InactivityMonitor.java:52)
    at org.apache.activemq.transport.TransportFilter.start(TransportFilter.java:64)
    at org.apache.activemq.transport.WireFormatNegotiator.start(WireFormatNegotiator.ja
va:72)
    at org.apache.activemq.transport.failover.FailoverTransport.doReconnect(FailoverTra
nsport.java:1017)
    -  locked java.lang.Object@468713cb <---- but blocks 1 other threads!
    at org.apache.activemq.transport.failover.FailoverTransport$2.iterate(FailoverTrans
port.java:148)
    -  locked java.lang.Object@688e271b
    at org.apache.activemq.thread.PooledTaskRunner.runTask(PooledTaskRunner.java:133)
    at org.apache.activemq.thread.PooledTaskRunner$1.run(PooledTaskRunner.java:48)
    at com.alibaba.ttl.TtlRunnable.run(TtlRunnable.java:51)
    at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1149)
    at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:624)
    at java.lang.Thread.run(Thread.java:748)

    Number of locked synchronizers = 1
    - java.util.concurrent.ThreadPoolExecutor$Worker@7e1b410f

Affect(row-cnt:0) cost in 55 ms.
[arthas@7]$
```

仔细分析下来并不是死锁，那就从入口开始。

33.8.2 从 Camel 配置开始跟踪

```
1 trace --skipJDKMethod true org.apache.camel.builder.RouteBuilder configure
2 '#cost > 20' -n 10
```

> **注意：** skipJDKMethod：跟踪结果不包含 JDK 方法。

开始构造延迟场景，但未能跟踪到相关信息。

```
[arthas@7]$ trace --skipJDKMethod true org.apache.camel.builder.RouteBuilder configure '#cost > 20' -n 10
Press Q or Ctrl+C to abort.
Affect(class count: 7 , method count: 6) cost in 640 ms, listenerId: 4
```

于是调整策略，从消息派发器开始跟踪。

33.8.3 跟踪统一消息派发器 MessageDispatcher

```
trace com.test.messagedispatcher.MessageDispatcher onMessage -n 10
```

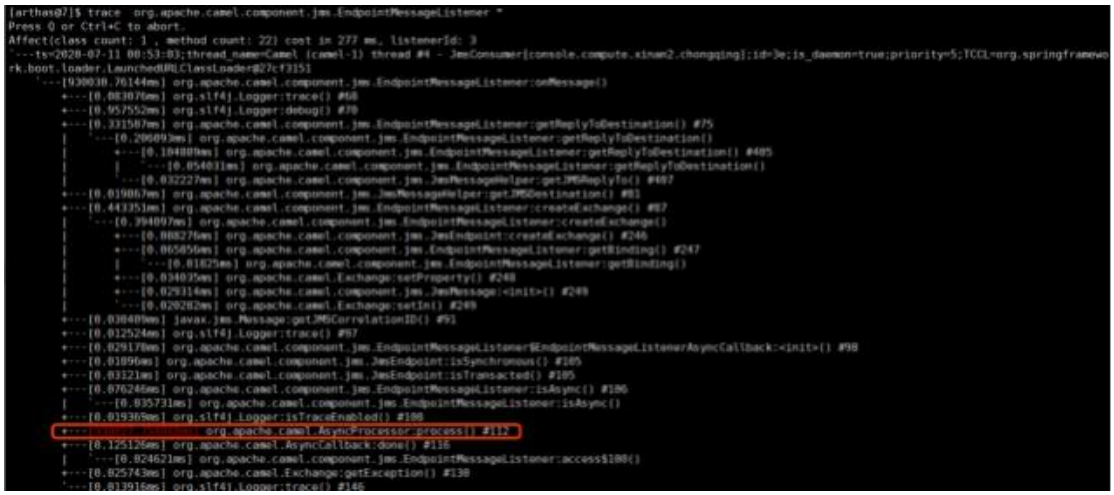
开始构造延迟场景，也没能跟踪到与延迟相关的信息，继续调整方向，下载了 Apache Camel 源代码，开 始分析从 JMS 消费者开始分析 Camel 核心入口方法，定位到创建监听器代码：

```
protected void createMessageListener(JmsEndpoint endpoint, Processor processor)
{
messageListener = new EndpointMessageListener(endpoint, processor);

getEndpoint().getConfiguration().configureMessageListener(messageListener);
messageListener.setBinding(endpoint.getBinding());

messageListener.setAsync(endpoint.getConfiguration().isAsyncConsumer());
}
```

33.8.4 分析 Camel 创建监听器方法

```
trace org.apache.camel.component.jms.EndpointMessageListener *
```

继续构造延迟场景，结果如下：

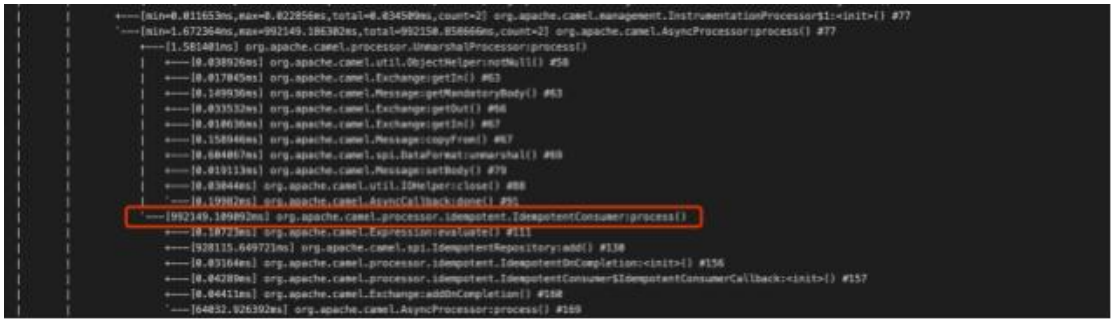


上图红色部分出现了 930027ms，初步定位到了延迟发生在 Camel 中间件，继续跟踪

33.8.5 跟踪 AsyncProcessor:process 方法

```
trace org.apache.camel.AsyncProcessor process
```

继续构造延迟场景，跟踪结果如下、



原来是幂等消费阻塞了几分钟，最后的关键信息是，继续跟踪。

33.8.6 跟踪 IdempotentRepository:add 方法

```
trace org.apache.camel.spi.IdempotentRepository add
```

原来是数据库事务的问题，继续跟踪。

33.8.7 直至定位到原因

原来是抓取 JDBC 连接的时候阻塞了

34. 线上的 API 接口响应比较慢，该如何快速排查和定位问题?

34.1 定位问题

34.1.1 慢查询日志

通常情况下，为了定位 sql 的性能瓶颈，我们需要开启 mysql 的慢查询日志。把超过指定时间的 sql 语句，单独记录下来，方面以后分析和定位问题。

开启慢查询日志需要重点关注三个参数：

- `slow_query_log` 慢查询开关
- `slow_query_log_file` 慢查询日志存放的路径
- `long_query_time` 超过多少秒才会记录日志

通过 mysql 的 `set` 命令可以设置：

```
set global slow_query_log='ON';
set global slow_query_log_file='/usr/local/mysql/data/slow.log';
set global long_query_time=2;
```

设置完之后，如果某条 sql 的执行时间超过了 2 秒，会被自动记录到 slow.log 文件中。

当然也可以直接修改配置文件 `my.cnf`

```
[mysqld]
slow_query_log = ON
slow_query_log_file = /usr/local/mysql/data/slow.log
long_query_time = 2
```

但这种方式需要重启 mysql 服务。

34.1.2 监控

可观测性帮助企业在复杂的分布式系统中更加快速的排查、定位问题，已经是分布式系统中必不可少的运维工具。

可观测性从传统监控场景不断延伸，逐渐覆盖 Metrics、Traces、Logs 三个维度并将之相互融合。在性能压测领域中，可观测性更为重要，除了有助于定位性能问题，其中 Metrics 性能指标更直接决定了压测是否通过，系统最终是否可以上线，具体如下：

- Metrics - 监控指标

系统性能指标，包括请求成功率、系统吞吐量、响应时长

资源性能指标，衡量系统软硬件资源使用情况，配合系统性能指标，观察系统资源水位

- Logs - 日志

施压引擎日志，观察施压引擎是否健康，压测脚本执行是否有报错

采样日志，采样记录 API 的请求和响应详情，辅助排查压测过程中的一些出错请求的参数是否正常，并通过响应详情，查看完整的错误信息

- Traces - 分布式链路追踪

用于性能问题诊断阶段，通过追踪请求在系统中的调用链路，定位报错 API 的报错系统和报错堆栈，快速定位性能问题点。

34.2 压测监控核心指标

压测过程中，对系统硬件、中间件、数据库资源的监控也很重要，包括系统性能指标、资源指标、中间件指标、数据库指标、前端指标、稳定性指标、批量处理指标、可扩展性指标、可靠性指标等。

34.2.1 系统性能指标

34.2.1.1 交易响应时间

定义 ：响应时间指用户从客户端发起一个请求开始，到客户端接收到从服务器端返回的响应结束，整个过程所耗费的时间。在性能检测中一般以压力发起端至被压测服务器返回处理结果的时间为计量，单位一般为秒或毫秒。平均响应时间指系统稳定运行时间段内，同一交易的平均响应时间。一般而言，交易响应时间均指平均响应时间。

简称 ：Response Time: RT

参考标准不同行业不同业务可接受的响应时间是不同的

- 互联网企业：500 毫秒以下，例如淘宝业务 10 毫秒左右
- 金融企业：1 秒以下为佳，部分复杂业务 3 秒以下
- 保险企业：3 秒以下为佳
- 制造业：5 秒以下为佳
- 时间窗口：即整个压测过程的时间，不同数据量则时间不一样，例如双 11 和 99 大促，数据量级不一样则时间窗口不同。大数据量的情况下，2 小时内可完成压测

34.2.1.2 系统处理能力

定义 ：系统处理能力是指系统在利用系统硬件平台和软件平台进行信息处理的能力。系统处理能力通过系统每秒钟能够处理的交易数量来评价，是技术测试活动中重要指标简称一般情况下，用以下指标来度量：

- HPS（Hits Per Second） ：每秒点击次数，单位是次/秒
- TPS（Transaction per Second）：系统每秒处理交易数，单位是笔/秒。

- QPS（Query per Second）：系统每秒处理查询次数，单位是次/秒。对于互联网业务中，如果某些业务有且仅有一个请求连接，那么 TPS=QPS=HPS，一般情况下用 **TPS 来衡量整个业务流程**，用 **QPS 来衡量接口查询次数**，用 HPS 来表示对服务器单击请求

标准无论 TPS、QPS、HPS，**此指标是衡量系统处理能力非常重要的指标，越大越好，根据经验**，一般情况下：

- 金融行业：1000 TPS~50000 TPS，不包括互联网化的活动
- 保险行业：100 TPS~100000 TPS，不包括互联网化的活动
- 制造行业：10 TPS~5000 TPS
- 互联网电子商务：10000 TPS~1000000 TPS
- 互联网中型网站：1000 TPS~50000 TPS
- 互联网小型网站：500 TPS~10000 TPS

34.2.1.3 错误率

定义 ：错误率指系统在负载情况下，失败交易的概率。错误率 = （失败交易数/交易总数）×100%。稳定性较好的系统，其错误率应该由超时引起，即为超时率。

标准不同系统对错误率的要求不同，但一般不超出千分之六，即成功率不低于 99.4%

34.2.2 资源指标

34.2.2.1 CPU

定义 ：中央处理器是一块超大规模的集成电路，是一台计算机的运算核心（Core）和控制核心（ Control Unit）。它的功能主要是解释计算机指令以及处理计算机软件中的数据。

34.2.2.2 Memory

定义 内存是计算机中重要的部件之一，它是与 CPU 进行沟通的桥梁。计算机中所有程序的运行都是在内存中进行的，因此内存的性能对计算机的影响非常大

34.2.2.3 磁盘吞吐量

定义 磁盘吞吐量是指在无磁盘故障的情况下单位时间内通过磁盘的数据量

34.2.2.4 网络吞吐量

定义 网络吞吐量是指在无网络故障的情况下单位时间内通过的网络的数据数量。单位为 Byte/s。网络吞吐量指标用于衡量系统对于网络设备或链路传输能力的需求。当网络吞吐量指标接近网络设备或链路最大传输能力时，则需要考虑升级网络设备

34.2.2.5 中间件指标

一级指标	二级指标	单位	解释
GC	GC 频率	每秒多少次	Java 虚拟机垃圾部分回收频率
	Full GC 频率	每小时多少次	Java 虚拟机垃圾完全回收频率
	Full GC 平均时长	秒	用于垃圾完全回收的平均时长
	Full GC 最大时长	秒	用于垃圾完全回收的最大时长
	堆使用率	百分比	堆使用率
ThreadPool	Active Thread Count	个	活动的线程数
	Pending User Request	个	处于排队的用户请求个数
JDBC	JDBC Active Connection	个	JDBC 活动连接数

@稀土掘金技术社区

34.2.2.6 数据库指标

一级指标	二级指标	单位	解释
SQL	耗时	微秒	执行 SQL 耗时
吞吐量	QPS	个	每秒查询次数
	TPS	个	每秒事务次数
命中率	Key Buffer 命中率	百分之	索引缓冲区命中率
	InnoDB Buffer 命中率	百分之	InnoDB 缓冲区命中率
	Query Cache 命中率	百分之	查询缓存命中率
	Table Cache 命中率	百分之	表缓存命中率
	Thread Cache 命中率	百分之	线程缓存命中率
锁	等待次数	次	锁等待次数
	等待时间	微秒	锁等待时间

@稀土掘金技术社区

数据库监控中的命中率通常指的是缓存命中率，它表示在数据库访问中，请求能够从缓存中获取所需数据的比例。换句话说，命中率越高，就意味着数据库查询所需的数据越多地可以从缓存中获取，而不需要去访问磁盘或进行其他昂贵的操作。这通常被认为是一个性能指标，因为高命中率通常意味着更快的响应时间和更好的系统性能。

34.2.2.7 稳定性指标

定义 ：最短稳定时间：系统按照最大容量的 80% 或标准压力（系统的预期日常压力）情况下运行，能够稳定运行的最短时间。一般来说，对于正常工作日（8 小时）运行的系统，至少应该能保证系统稳定运行 8 小时以上。对于 7×24 运行的系统，至少应该能够保证系统稳定运行 24 小时以上。如果系统不能稳定的运行，上线后，随着业务量的增长和长时间运行，将会出现性能下降甚至崩溃的风险

标准

- TPS 曲线稳定，没有大幅度的波动
- 各项资源指标没有泄露或异常情况

34.2.2.8 可扩展性指标

定义 ：指应用软件或操作系统以集群方式部署，增加的硬件资源与增加的处理能力之间的关系。计算公式为：（增加性能/原始性能）/（增加资源/原始资源）×100%。扩展能力应通过多轮测试获得扩展指标的变化趋势。一般扩展能力非常好的应用系统，扩展指标应是线性或接近线性的，现在很多大规模的分布式系统的扩展能力非常好。

标准

- 理想的扩展能力是资源增加几倍，性能就提升几倍。
- 扩展能力至少在 70%以上。

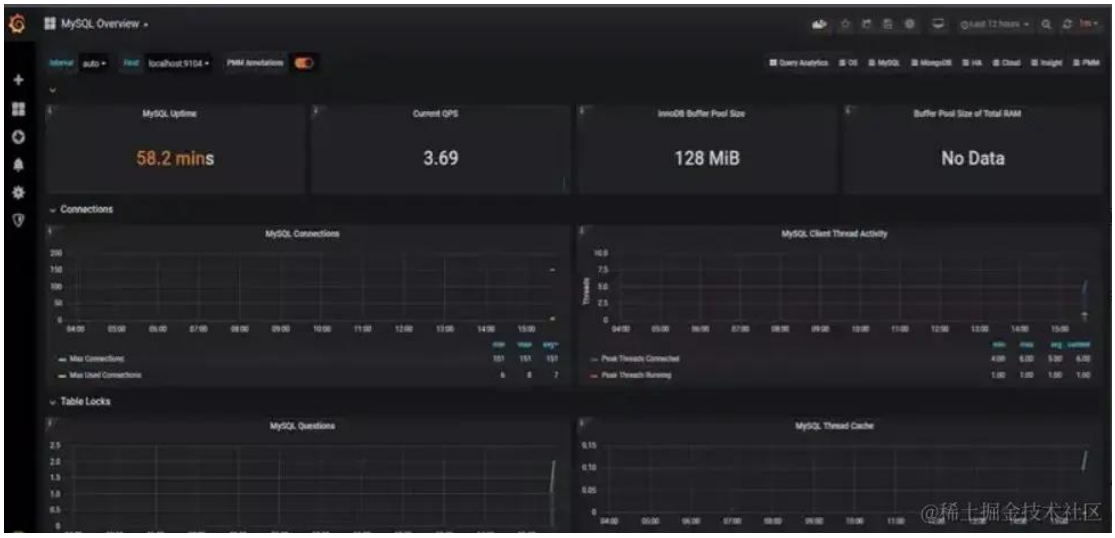
目前业界使用比较多的开源监控系统是：Prometheus。

它提供了 监控 和 预警 的功能。

我们可以用它监控如下信息：

- 接口响应时间
- 调用第三方服务耗时
- 慢查询 sql 耗时
- cpu 使用情况
- 内存使用情况
- 磁盘使用情况
- 数据库使用情况

它的界面大概长这样子：



可以看到 mysql 当前 qps，活跃线程数，连接数，缓存池的大小等信息。

如果发现数据量连接池占用太多，对接口的性能肯定会有影响。

这时可能是代码中开启了连接忘了关，或者并发量太大了导致的，需要做进一步排查和系统优化。

34.3 链路追踪

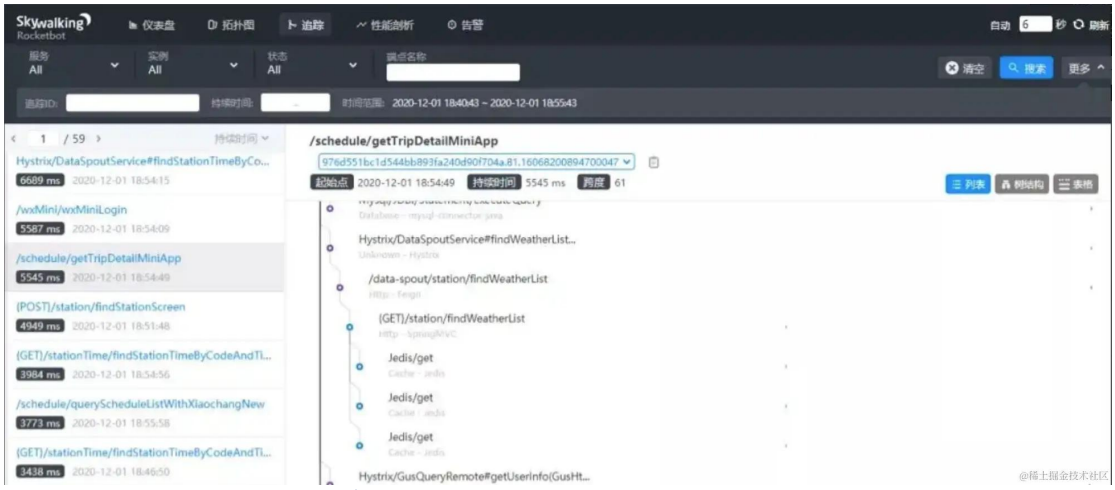
有时候某个接口涉及的逻辑很多，比如：查数据库、查 redis、远程调用接口，发 mq 消息，执行业务代码等等。

该接口一次请求的链路很长，如果逐一排查，需要花费大量的时间，这时候，我们已经没法用传统的办法定位问题了。

有没有办法解决这问题呢？

用分布式链路跟踪系统：`skywalking`。

通过 skywalking 定位性能问题：



在 skywalking 中可以通过 `traceId`（全局唯一的 id），串联一个接口请求的完整链路。

可以看到整个接口的耗时，调用的远程服务的耗时，访问数据库或者 redis 的耗时等等，功能非常强大。

之前没有这个功能的时候，为了定位线上接口性能问题，我们还需要在代码中加日志，手动打印出链路中各个环节的耗时情况，然后再逐一排查。

34.4 问题排查

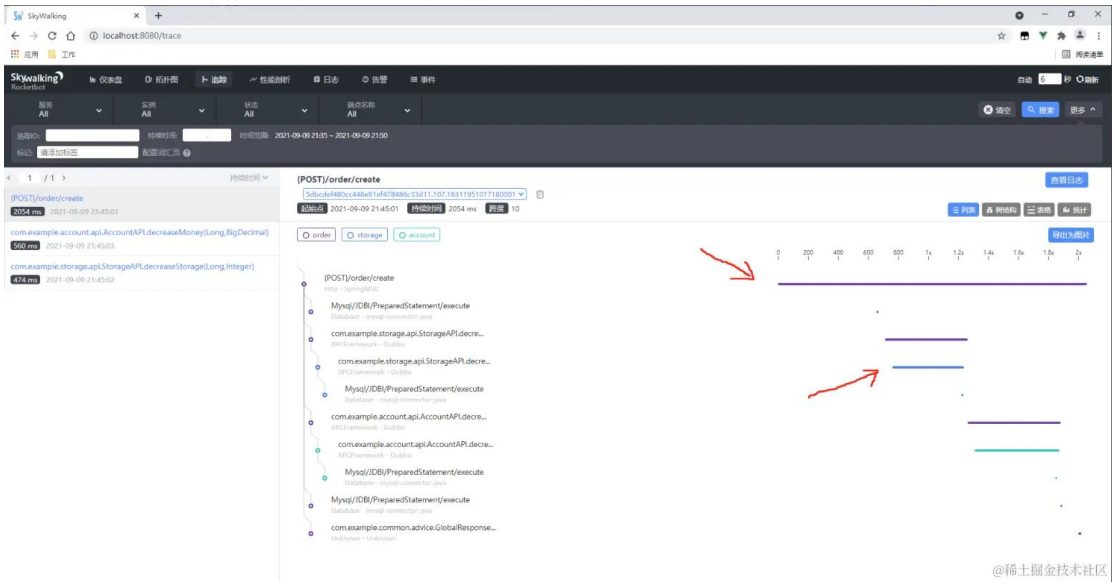
34.4.1 个别接口响应慢

34.4.1.1 链路追踪

使用 SkyWalking，展示出每一个与网络有关的耗时。比如：读写数据库、读写 Redis、SpringCloud 调用、Dubbo 调用等。这样就能立马定位是哪次操作耗时了。

同时，SkyWalking 可以记录每一个 SQL 语句，可以帮助定位。

例如：（如箭头所指处，最上边一个是总耗时，下边的线段是单个操作的耗时）



34.4.1.2 在链路上打印日志

在相应的链路上打印日志，然后查看日志，看是哪个地方耗时。

34.4.1.3 死锁问题

在这里重点讲一下出现死锁问题的排查经验吧

有这样的情况：一个线程需要同时获取多把锁，这时就容易发生死锁

t1 线程 获得 A 对象 锁，接下来想获取 B 对象 的锁 t2 线程 获得 B 对象 锁，接下来想获取 A 对象 的锁

```
Object A = new Object();
    Object B = new Object();
    Thread t1 = new Thread(() -> {
        synchronized (A) {
            log.debug("lock A");
            sleep(1);
            synchronized (B) {
                log.debug("lock B");
                log.debug("操作...");
```

```
                }
            }
        }, "t1");
        Thread t2 = new Thread(() -> {
            synchronized (B) {
                log.debug("lock B");
                sleep(0.5);
                synchronized (A) {
                    log.debug("lock A");
                    log.debug("操作...");
                }
            }
        }, "t2");
        t1.start();
        t2.start();


```

```
12:22:06.962 [t2] c.TestDeadLock - lock B
12:22:06.962 [t1] c.TestDeadLock - lock A

```

34.4.1.4 定位死锁

检测死锁可以使用 jconsole 工具，或者使用 jps 定位进程 id，再用 jstack 定位死锁：

jps

```
cmd > jps
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
12320 Jps
22816 KotlinCompileDaemon
33200 TestDeadLock // JVM 进程
11508 Main
28468 Launcher

```

```
cmd > jstack 33200
            Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF-8
            2018-12-29 05:51:40
            Full thread dump Java HotSpot(TM) 64-Bit Server VM (25.91-b14 mixed mode):
            "DestroyJavaVM" #13 prio=5 os_prio=0 tid=0x0000000003525000 nid=0x2f60
waiting on condition
```

```
        [0x0000000000000000]
        java.lang.Thread.State: RUNNABLE
        "Thread-1" #12 prio=5 os_prio=0 tid=0x000000001eb69000 nid=0xd40 waiting for monitor entry
        [0x000000001f54f000]
        java.lang.Thread.State: BLOCKED (on object monitor)
        at thread.TestDeadLock.lambda$main$1(TestDeadLock.java:28)
        - waiting to lock <0x000000076b5bf1c0> (a java.lang.Object)
        - locked <0x000000076b5bf1d0> (a java.lang.Object)
        at thread.TestDeadLock$$Lambda$2/883049899.run(Unknown Source)
        at java.lang.Thread.run(Thread.java:745)
        "Thread-0" #11 prio=5 os_prio=0 tid=0x000000001eb68800 nid=0x1b28 waiting for monitor entry
        [0x000000001f44f000]
        java.lang.Thread.State: BLOCKED (on object monitor)
        at thread.TestDeadLock.lambda$main$0(TestDeadLock.java:15)
        - waiting to lock <0x000000076b5bf1d0> (a java.lang.Object)
        - locked <0x000000076b5bf1c0> (a java.lang.Object)
        at thread.TestDeadLock$$Lambda$1/495053715.run(Unknown Source)
        at java.lang.Thread.run(Thread.java:745)

 // 略去部分输出
        Found one Java-level deadlock:
        =============================
        "Thread-1":
        waiting to lock monitor 0x000000000361d378 (object 0x000000076b5bf1c0, a java.lang.Object),
        which is held by "Thread-0"
        "Thread-0":
        waiting to lock monitor 0x000000000361e768 (object 0x000000076b5bf1d0, a java.lang.Object),
        which is held by "Thread-1"
        Java stack information for the threads listed above:
        ===================================================
        "Thread-1":
        at thread.TestDeadLock.lambda$main$1(TestDeadLock.java:28)
        - waiting to lock <0x000000076b5bf1c0> (a java.lang.Object)
        - locked <0x000000076b5bf1d0> (a java.lang.Object)
        at thread.TestDeadLock$$Lambda$2/883049899.run(Unknown Source)
        at java.lang.Thread.run(Thread.java:745)
        "Thread-0":
        at thread.TestDeadLock.lambda$main$0(TestDeadLock.java:15)
        - waiting to lock <0x000000076b5bf1d0> (a java.lang.Object)
        - locked <0x000000076b5bf1c0> (a java.lang.Object)
        at thread.TestDeadLock$$Lambda$1/495053715.run(Unknown Source)
```

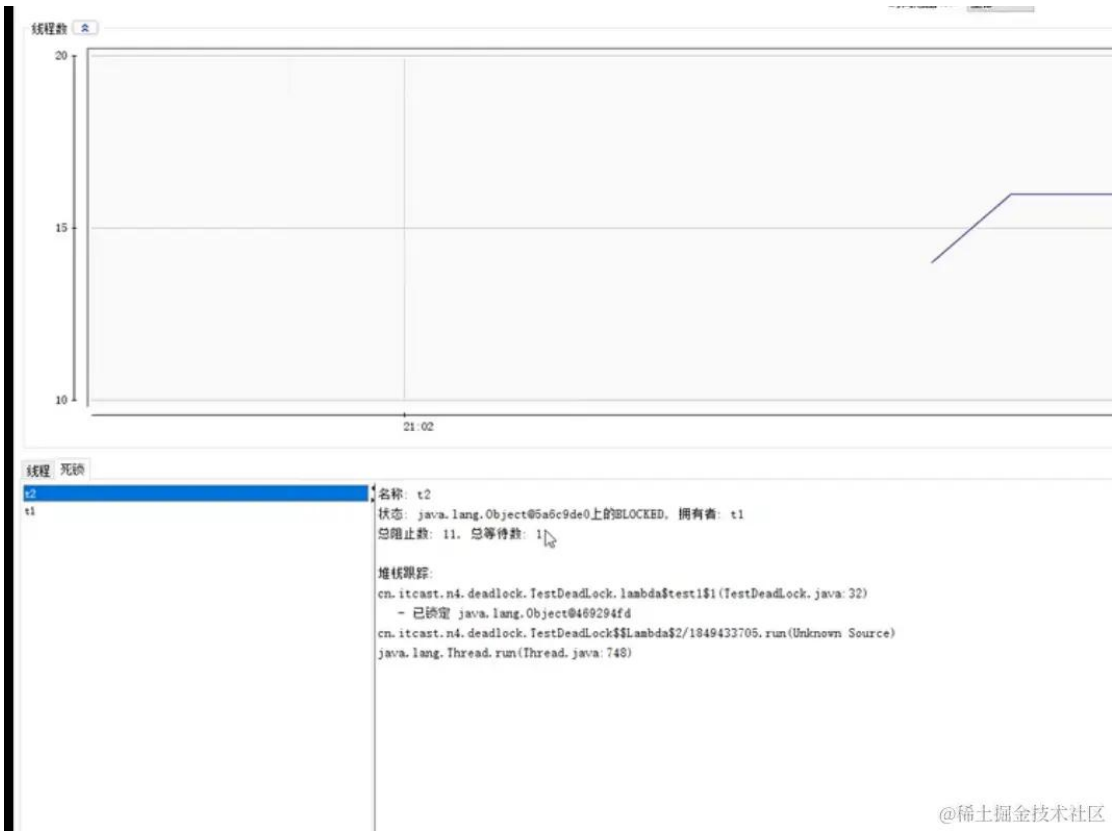
```
47          at java.lang.Thread.run(Thread.java:745)
48          Found 1 deadlock
49
50
```

这里是两个线程 “Thread-0” 和 “Thread-1” 在互相等待对方释放锁，从而导致了死锁。

线程 “Thread-1” 试图获取对象 0x000000076b5bf1c0 的监视器锁（即 synchronized 关键字保护的那个对象），但是这个锁已经被线程 “Thread-0” 持有。因此，“Thread-1” 进入了等待状态，等待 “Thread-0” 释放这个锁。

同时，线程 “Thread-0” 试图获取对象 0x000000076b5bf1d0 的监视器锁，但是这个锁已经被线程 “Thread-1” 持有。因此，“Thread-0” 进入了等待状态，等待 “Thread-1” 释放这个锁。

jconsole



34.4.2 所有接口响应慢

如果线上出了问题，首先判断是业务问题还是整个系统的问题。如果是业务问题，就去看应用的日志等进行排查。如果出现了如下问题，就可能是整个系统的问题

1. 大量接口都很慢
2. 页面打不开

如何得知系统出问题了?

系统出问题时，我们需要进行详细排查，一般情况下，有以下场景我们可以得知线上出问题了:

3. 用户反馈功能不能正常使用
4. 监控系统的邮件或者短信提醒

系统问题排查步骤

以下按顺序进行

5. 是否 CPU 占用过高
6. 是否内存占用过高
7. 是否磁盘占用过高

8. 是否网络故障
9. 查看后台日志
10. 是否是数据库问题（比如：索引失效、死锁）
11. 是否是垃圾回收导致

34.4.2.1 CPU 占用过高

什么场景需要排查 CPU 占用?

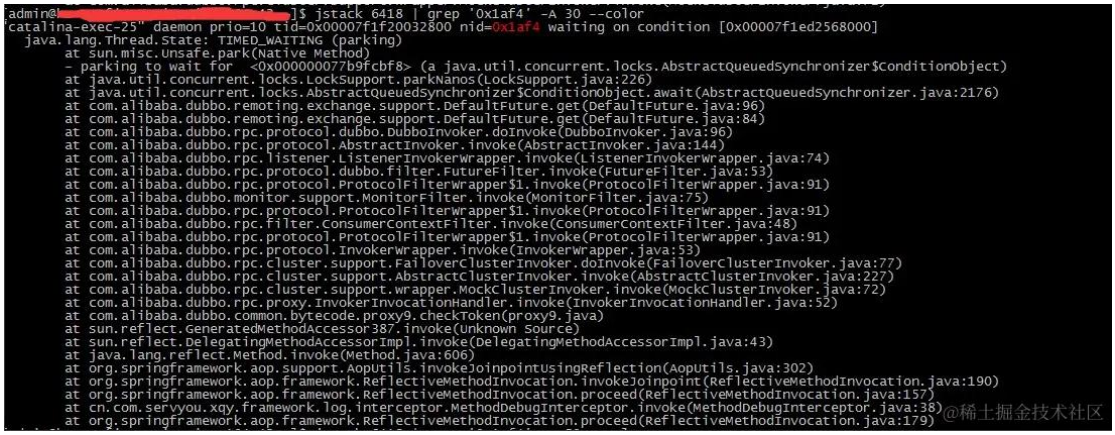
1. 访问接口的响应速度很慢。
2. 系统崩溃无响应
3. 压测时要查看 CPU、内存、load、rt、qps 等指标

步骤简述

4. 定位进程 （命令：top）
5. 定位线程 （命令：top -Hp 进程号）
6. 定位代码位置 （命令：jstack）

排查方法详述

7. 找到占 CPU 最高的进程。
    a. top 命令，记下进程号（PID）。假设最高的是：1893
8. 通过进程找到对应的线程。
    a. top -Hp 1893。假设最高的是：4519
    b. （因为 Java 是单进程多线程的）
9. 通过线程定位到代码大概的位置信息。
    a. printf %x 4519。结果为：11a7
    b. jstack 1893 | grep 11a7 -A 30 –color 结果大概是这样的：



@稀土掘金技术社区

34.4.2.2 可能导致 CPU 使用率飙升的操作

- 无限循环的 while
- 经常使用 Young GC
- 频繁的 GC

如果访问量很高，可能会导致频繁的 GC 甚至 Full GC。当调用量很大时，内存分配将如此之快以至于 GC 线程将连续执行，这将导致 CPU 飙升。

- 序列化和反序列化

序列化和反序列化本身不会导致 CPU 使用率飙升。序列化是将对象转换为字节流的过程，而反序列化则是将字节流转换回对象的过程。这两个过程通常在内存中完成，并不直接导致 CPU 使用率的飙升。

然而，在某些情况下，序列化和反序列化可能会导致 CPU 使用率增加。例如，当需要大量的数据进行序列化或反序列化时，可能会消耗较多的 CPU 资源。此外，如果序列化和反序列化的操作频繁且耗时较长，也可能对 CPU 使用率产生一定影响。

- 正则表达式

正则表达式本身不会导致 CPU 飙升，但是使用不当或者复杂的正则表达式可能会导致 CPU 负载增加。

正则表达式在处理大量文本时，特别是复杂的表达式或者需要进行大量回溯的情况下，可能会消耗大量的 CPU 资源。这是因为正则表达式引擎需要在文本中进行搜索和匹配，并且有些复杂的规则可能需要大量的计算才能找到匹配的结果。

因此，在编写正则表达式时，需要注意避免过于复杂或低效的表达式，尤其是在处理大量文本的场景下。另外，对于一些需要频繁执行的操作，可以考虑在代码中进行优化，以减少对 CPU 的负载影响。

- 线程上下文切换

34.4.2.3 内存占用过高

步骤简述

定位 Java 程序内存使用过高或者内存泄漏的问题跟 CPU 也类似，可分为以下步骤：

1. 定位进程 （命令：top）
2. 定位线程 （命令：top -Hp 进程号）
3. 定位代码位置 （命令：jmap 生成快照，**MAT / jprofiler** / VisualVM / jhat 查看快照数据）

1.定位进程

4. top
5. 按下 M（按内存占用由大到小排序）
    a. // 假设定位到的进程 ID 为 14279。

2.定位线程

top -Hp 14279;

按下 M（按内存占用由大到小排序）

```
top - 18:33:07 up 25 days,  7:48,  1 user,  load average: 0.17, 0.27, 0.23
Threads:  54 total,   1 running,  53 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.5 us,  0.7 sy,  0.0 ni, 98.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :  8168236 total,   231696 free,  3660496 used,  4276044 buff/cache
KiB Swap:   969964 total,   969964 free,        0 used.  4197860 avail Mem
  PID USER      PR  NI    VIRT    RES    SHR S %CPU %MEM     TIME+
COMMAND
14293 weiping   20   0 4508772  97036  18112 S  10     12 152:35.42 java
14279 weiping   20   0 4508772  97036  18112 S  5.0  1.2   0:00.00 java
14282 weiping   20   0 4508772  97036  18112 S  0.0  1.2   0:00.37 java

```

注意观察两点：

6. 线程的数量

a. 可以看到，它线程数是 54（左上角的 Threads 项），属于正常。
b. 若比较大（比如大于 1000），要考虑是不是代码有问题：
    i. 是不是代码里手动起了多个线程。比如：使用 OkHttpClient 时每次都创建了连接池（ConnectionPool）；应该是只创建一个连接池的。
    ii. 是不是自己创建的线程池最大个数太大了。

7. 占内存大的线程的 PID
    a. 如果线程数量正常，就要 dump 内存的快照信息来查看。

3.定位代码位置

如果是线上环境，注意 dump 之前必须先将流量切走，否则大内存 dump 是直接卡死服务。

dump 当前快照：jmap -dump:format=b,file=dump.hprof 14279

查找实例数比较多的业务相关的实例，然后找到相应代码查看。（使用工具查看 dump.hprof。比如：MAT、VisualVM、jhat）

34.4.2.4 磁盘问题

步骤简介

1. 是否磁盘快用完了（命令：**df**、du）
2. TPS 是否正常（命令：**iostat**、vmstat、lsof）

这个命令是用来检查系统的磁盘性能是否正常。具体来说：

- iostat 命令用于报告 CPU 使用情况和磁盘 I/O 统计信息。
- vmstat 命令用于报告虚拟内存统计信息。
- lsof 命令用于列出打开文件。

通过使用这些命令，可以获得关于磁盘 I/O 活动、CPU 使用率、内存使用等方面的统计数据。其中，磁盘 I/O 统计信息中的 TPS（每秒传输的 I/O 操作数）可以用来评估磁盘的性能是否正常。如果 TPS 过高或者过低，可能意味着磁盘存在性能问题，需要进一步分析和解决。

34.4.2.5 网络问题

34.5 垃圾回收

34.5.1 什么情况下，GC 会对程序产生影响?

不管 Minor GC 还是 FGC，都会造成一定程度的程序卡顿（即 Stop The World：GC 线程开始工作，其他工作线程被挂起），即使采用 ParNew、CMS 或者 G1 这些更先进的垃圾回收算法，也只是在减少卡顿时间，而并不能完全消除卡顿。

那到底什么情况下，GC 会对程序产生影响呢？根据严重程度从高到底，包括以下 4 种情况：

1. FGC 过于频繁
    a. FGC 通常比较慢，少则几百毫秒，多则几秒，正常情况 FGC 每隔几个小时甚至几天才执行一次，对系统的影响还能接受。
    b. 一旦出现 FGC 频繁（比如几十分钟执行一次），是存在问题的，会导致工作线程频繁被停止，让系统看起来一直有卡顿现象，也会使得程序的整体性能变差。
2. YGC 耗时过长
    a. 一般来说，YGC 的总耗时在几十或者上百毫秒是比较正常的，虽然会引起系统卡顿几毫秒或者几十毫秒，这种情况几乎对用户无感知，对程序的影响可以忽略不计。
    b. 如果 YGC 耗时达到了 1 秒甚至几秒（都快赶上 FGC 的耗时了），那卡顿时间就会增大，加上 YGC 本身比较频繁，就会导致比较多的服务超时问题。
3. FGC 耗时过长
    a. FGC 耗时增加，卡顿时间也会随之增加，尤其对于高并发服务，可能导致 FGC 期间比较多的超时问题，可用性降低，这种也需要关注。
4. YGC 过于频繁
    a. 即使 YGC 不会引起服务超时，但是 YGC 过于频繁也会降低服务的整体性能，对于高并发服务也是需要关注的。

34.5.2 如何排查

1. 公司的监控系统：大部分公司都会有，可全方位监控 JVM 的各项指标。
2. JDK 自带工具：jmap、jstat

a. 查看堆内存各区域的使用率以及 GC 情况：jstat -gcutil pid 1000 （重点关注结果中的 YGC、YGCT、FGC、FGCT、GCT）
b. 查看堆内存中的存活对象，并按空间排序：jmap -histo pid | head -n20
c. dump 堆内存文件：jmap -dump:format=b,file=heap pid

35. 百万级别数据的 Excel 如何快速导入到数据库中

35.1 需求说明

项目中有一个 Excel 导入的需求：缴费记录导入

由实施 / 用户 将别的系统的数据填入我们系统中的 Excel 模板，应用将文件内容读取、校对、转换之后产生欠费数据、票据、票据详情并存储到数据库中。

在接手之前可能由于之前导入的数据量并不多没有对效率有过高的追求。但是到了 4.0 版本，预估导入时 Excel 行数会是 10w+ 级别，而往数据库插入的数据量是大于 3n 的，也就是说 10w 行的 Excel，则至少向数据库插入 30w 行数据。因此优化原来的导入代码是势在必行的。逐步分析和优化了导入的代码，使之在百秒内完成(最终性能瓶颈在数据库的处理速度上，测试服务器 4g 内存不仅放了数据库，还放了很多微服务应用。处理能力不太行)。具体的过程如下，每一步都有列出影响性能的问题和解决的办法。

导入 Excel 的需求在系统中还是很常见的，优化办法可能不是最优的，欢迎读者在评论区留言交流提供更优的思路

一些细节

1.数据导入：导入使用的模板由系统提供，格式是 xlsx (支持 65535+行数据) ，用户按照表头在对应列写入相应的数据

2.数据校验：数据校验有两种：

2.1 字段长度、字段正则表达式校验等，内存内校验不存在外部数据交互。对性能影响较小

2.2 数据重复性校验，如票据号是否和系统已存在的票据号重复(需要查询数据库，十分影响性能)

3.数据插入：测试环境数据库使用 MySQL 5.7，未分库分表，连接池使用 Druid

迭代记录

35.2 A. 第一版：POI + 逐行查询校对 + 逐行插入

这个版本是最古老的版本，采用原生 POI，手动将 Excel 中的行映射成 ArrayList 对象，然后存储到 List，代码执行的步骤如下：

1.手动读取 Excel 成 List

2.循环遍历，在循环中进行以下步骤

.检验字段长度

.一些查询数据库的校验，比如校验当前行欠费对应的房屋是否在系统中存在，需要查询房屋表

3.写入当前行数据

4.返回执行结果，如果出错 / 校验不合格。则返回提示信息并回滚数据

显而易见的，这样实现一定是赶工赶出来的，后续可能用的少也没有察觉到性能问题，但是它最多适用于个位数/十位数级别的数据。存在以下明显的问题：

1.查询数据库的校验对每一行数据都要查询一次数据库，应用访问数据库来回的网络 IO 次数被放大了 n 倍，时间也就放大了 n 倍

2.写入数据也是逐行写入的，问题和上面的一样

3.数据读取使用原生 POI，代码十分冗余，可维护性差。

35.3 B. 第二版：EasyPOI + 缓存数据库查询操作 + 批量插入

针对第一版分析的三个问题，分别采用以下三个方法优化

1.缓存数据，以空间换时间

逐行查询数据库校验的时间成本主要在来回的网络 IO 中，优化方法也很简单。将参加校验的数据全部缓存到 HashMap 中。直接到 HashMap 去命中。

例如：校验行中的房屋是否存在，原本是要用 区域 + 楼宇 + 单元 + 房号 去查询房屋表匹配房屋 ID，查到则校验通过，生成的欠单中存储房屋 ID，校验不通过则返回错误信息给用户。而房屋信息在导入欠费的时候是不会更新的。并且一个小区的房屋信息也不会很多(5000 以内)因此我采用一条 SQL，将该小区下所有的房屋以 区域/楼宇/单元/房号 作为 key，以 房屋 ID 作为 value，存储到 HashMap 中，后续校验只需要在 HashMap 中命中

自定义 SessionMapper

Mybatis 原生是不支持将查询到的结果直接写人一个 HashMap 中的，需要自定义 SessionMapper

SessionMapper 中指定使用 MapResultHandler 处理 SQL 查询的结果集

```
@Repository
public class SessionMapper extends SqlSessionDaoSupport {
    @Resource
    public void setSqlSessionFactory(SqlSessionFactory sqlSessionFactory) {
        super.setSqlSessionFactory(sqlSessionFactory);
    }
    // 堆代码 duidaima.com
    // 区域楼宇单元房号 - 房屋 ID
    @SuppressWarnings("unchecked")
    public Map<String, Long> getHouseMapByAreaId(Long areaId) {
        MapResultHandler handler = new MapResultHandler();
  this.getSqlSession().select(BaseUnitMapper.class.getName()+".getHouseMapByAreaId", areaId, handler);
        Map<String, Long> map = handler.getMappedResults();
        return map;
    }
}
```

MapResultHandler 处理程序，将结果集放入 HashMap

```
public class MapResultHandler implements ResultHandler {
    private final Map mappedResults = new HashMap();
    @Override
    public void handleResult(ResultContext context) {
        @SuppressWarnings("rawtypes")
```

```
            Map map = (Map)context.getResultObject();
            mappedResults.put(map.get("key"), map.get("value"));
        }
        public Map getMappedResults() {
            return mappedResults;
        }
}
```

示例 Mapper

```
@Mapper
@Repository
public interface BaseUnitMapper {
    // 收费标准绑定 区域楼宇单元房号 - 房屋 ID
    Map<String, Long> getHouseMapByAreaId(@Param("areaId") Long areaId);
}
```

示例 Mapper.xml

```
<select id="getHouseMapByAreaId" resultMap="mapResultLong">
    SELECT
        CONCAT( h.bulid_area_name, h.build_name, h.unit_name, h.house_num ) k,
        h.house_id v
    FROM
        base_house h
    WHERE
        h.area_id = ##{areaId}
    GROUP BY
        h.house_id
</select>

<resultMap id="mapResultLong" type="java.util.HashMap">
    <result property="key" column="k" javaType="string" jdbcType="VARCHAR"/>
    <result property="value" column="v" javaType="long" jdbcType="INTEGER"/>
</resultMap>
```

之后在代码中调用 SessionMapper 类对应的方法即可。

2.使用 values 批量插入

MySQL insert 语句支持使用 values (),(),() 的方式一次插入多行数据，通过 mybatis foreach 结合 java 集合可以实现批量插入，代码写法如下：

```
<insert id="insertList">
    insert into table(colom1, colom2)
```

```
        values
        <foreach collection="list" item="item" index="index" separator=",">
         ( ##{item.colom1}, ##{item.colom2})
        </foreach>
</insert>
```

35.4 C. 第三版：优化数据插入速度

在第二版插入的时候，我使用了 values 批量插入代替逐行插入。每 30000 行拼接一个长 SQL、顺序插入。整个导入方法这块耗时最多，非常拉跨。后来我将每次拼接的行数减少到 10000、5000、3000、1000、500 发现执行最快的是 1000。结合网上一些对 innodb_buffer_pool_size 描述我猜是因为过长的 SQL 在写操作的时候由于超过内存阈值，发生了磁盘交换。关注公号：码猿技术专栏，回复关键词：1111 获取阿里内部 Java 调优手册；限制了速度，另外测试服务器的数据库性能也不怎么样，过多的插入他也处理不过来。所以最终采用每次 1000 条插入。

每次 1000 条插入后，为了榨干数据库的 CPU，那么网络 IO 的等待时间就需要利用起来，这个需要多线程来解决，而最简单的多线程可以使用 并行流 来实现，接着我将代码用并行流来测试了一下：

10w 行的 excel、42w 欠单、42w 记录详情、2w 记录、16 线程并行插入数据库、每次 1000 行。插入时间 72s，导入总时间 95 s。

并行插入工具类

并行插入的代码我封装了一个函数式编程的工具类，也提供给大家

```
/**
 * 功能：利用并行流快速插入数据
 */
public class InsertConsumer {
    /**
     * 每个长 SQL 插入的行数，可以根据数据库性能调整
     */
    private final static int SIZE = 1000;
    /**
```

```
     * 如果需要调整并发数目，修改下面方法的第二个参数即可
     */
    static {
        System.setProperty("java.util.concurrent.ForkJoinPool.common.parallelism", "4");
    }
    /**
     * 插入方法
     *
     * @param list        插入数据集合
     * @param consumer 消费型方法，直接使用 mapper::method 方法引用的方式
     * @param <T>          插入的数据类型
     */
    public static <T> void insertData(List<T> list, Consumer<List<T>> consumer) {
        if (list == null || list.size() < 1) {
            return;
        }
        List<List<T>> streamList = new ArrayList<>();
        for (int i = 0; i < list.size(); i += SIZE) {
            int j = Math.min((i + SIZE), list.size());
            List<T> subList = list.subList(i, j);
            streamList.add(subList);
        }
        // 并行流使用的并发数是 CPU 核心数，不能局部更改。全局更改影响较大，斟酌
        streamList.parallelStream().forEach(consumer);
    }
}
```

这里多数使用到很多 Java8 的 API，不了解的朋友可以翻看我之前关于 Java 的博客。

方法使用起来很简单

```
InsertConsumer.insertData(feeList, arrearageMapper::insertList);
```

35.5 其他影响性能的内容

日志

避免在 for 循环中打印过多的 info 日志

在优化的过程中，我还发现了一个特别影响性能的东西：info 日志，还是使用 41w 行、25 列、45.5m 数据，在 开始-数据读取完毕 之间每 1000 行打印一条 info 日志，缓存校验数据-校验完毕 之间每行打印 3+ 条 info 日志，日志框架使用 Slf4j 。打印并持久化到磁盘。下面是打印日志和不打印日志效率的差别

打印日志

```
[2020-06-28 14:33:39,555] [http-nio-8083-exec-6] [INFO] [FastBeginFeeService.java:296] 开始-数据读取完毕，耗时：126069 毫秒
[2020-06-28 14:33:39,555] [http-nio-8083-exec-6] [INFO] [FastBeginFeeService.java:297] 数据读取完毕-缓存校验数据，耗时：178 毫秒
[2020-06-28 14:33:39,555] [http-nio-8083-exec-6] [INFO] [FastBeginFeeService.java:298] 缓存校验数据-校验完毕，耗时：519352 毫秒
```

不打印日志

```
[2020-06-28 14:58:13,550] [http-nio-8083-exec-1] [INFO] [FastBeginFeeService.java:295] 开始-数据读取完毕，耗时：54203 毫秒
[2020-06-28 14:58:13,551] [http-nio-8083-exec-1] [INFO] [FastBeginFeeService.java:296] 数据读取完毕-缓存校验数据，耗时：208 毫秒
[2020-06-28 14:58:13,551] [http-nio-8083-exec-1] [INFO] [FastBeginFeeService.java:297] 缓存校验数据-校验完毕，耗时：53922 毫秒
```

我以为是我选错 Excel 文件了，又重新选了一次，结果依旧

```
[2020-06-28 15:37:57,729] [http-nio-8083-exec-10] [INFO] [FastBeginFeeService.java:295] 开始-数据读取完毕，耗时：51508 毫秒
[2020-06-28 15:37:57,729] [http-nio-8083-exec-10] [INFO] [FastBeginFeeService.java:296] 数据读取完毕-缓存校验数据，耗时：187 毫秒
[2020-06-28 15:37:57,730] [http-nio-8083-exec-10] [INFO] [FastBeginFeeService.java:297] 缓存校验数据-校验完毕，耗时：52459 毫秒
```

缓存校验数据-校验完毕 不打印日志耗时仅仅是打印日志耗时的 1/10 ！

35.6 总结

提升 Excel 导入速度的方法：

1.使用更快的 Excel 读取框架(推荐使用阿里 EasyExcel)

2.对于需要与数据库交互的校验、按照业务逻辑适当的使用缓存。用空间换时间

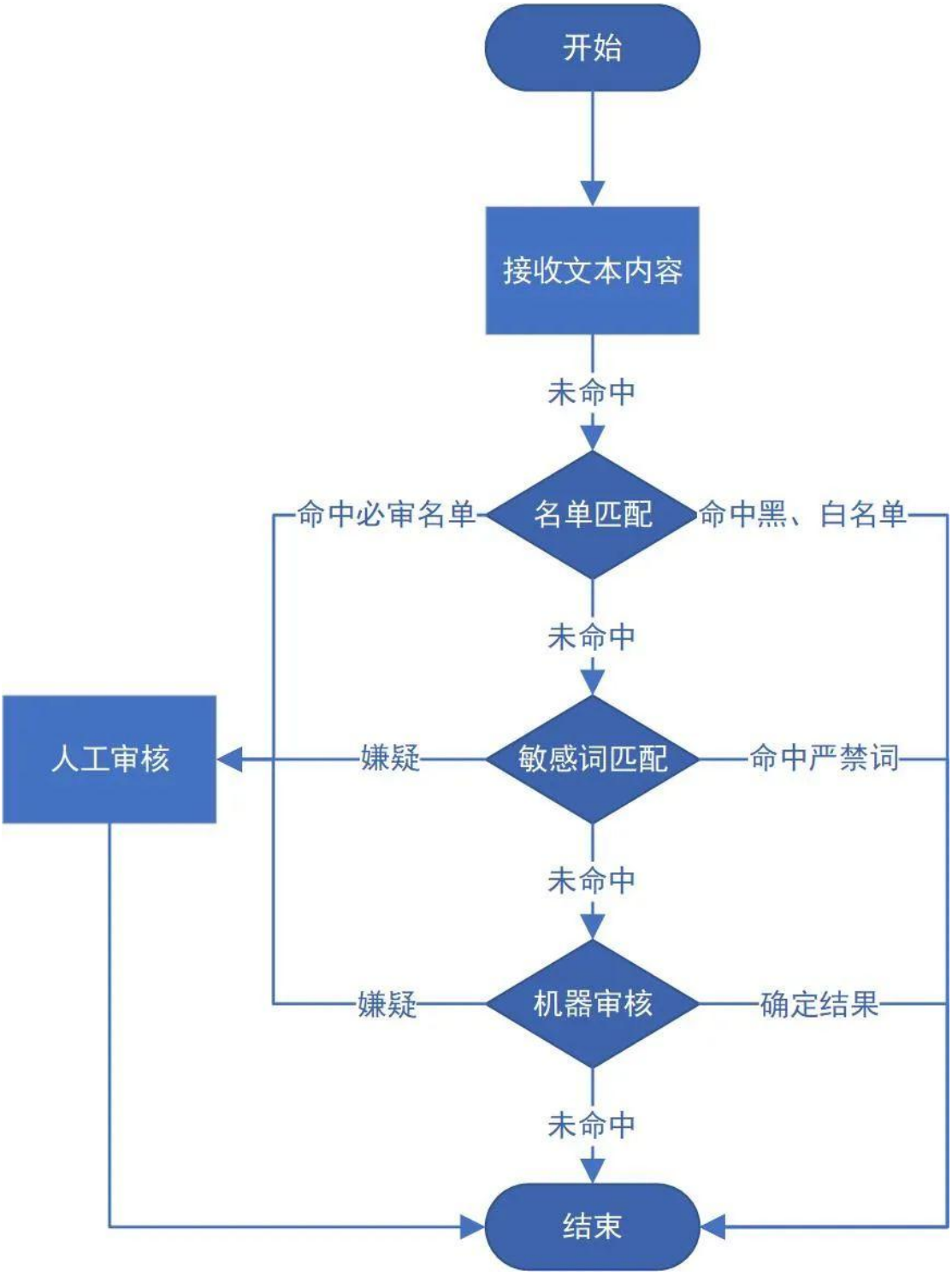
3.使用 values(),(),() 拼接长 SQL 一次插入多行数据

4.使用多线程插入数据，利用掉网络 IO 等待时间(推荐使用并行流，简单易用)

5.避免在循环中打印无用的日志

36. 如何设计一个敏感词过滤系统?

下图是一个完整的文本审核流程，包括名单匹配、敏感词匹配、AI 机器审核、人工审核四个环节。待审核文本需要顺次通过名单匹配、敏感词匹配、AI 机器审核三个流程，若结果为嫌疑则需要人工审核，否则将直接给出确定的结果。



敏感词匹配功能可以迅速地匹配文本中的敏感词汇，算法平均耗时为 50ms，因其简单、快速、直接、灵活的特点，成为了审核人员对抗垃圾文本的利器。然而身处信息爆炸时代的网民们非常“优秀”，他们源源不断的发明各种新词、谐音词来绕过敏感词检测。

例如某些用户会使用“啋票”、“采漂”等词汇来规避敏感词“彩票”，其中“啋票”不仅是谐音词，还包含多音字，常规的模式匹配算法很难保证完全命中。这不仅给运营规则带来挑战，也对匹配算法的精准度、漏杀率提出了要求。

本文将从算法选型入手，结合两个实际场景来介绍谛听系统的敏感词匹配算法。其中第一节介绍了底层算法选型思路，第二节介绍了两种实际场景下提升敏感词精准度、降低漏杀率的方案。

36.1 算法选型

敏感词匹配功能依托于模式匹配算法。模式匹配的定义是，给定一个子串，在某个字符串中找出与该子串相同的所有子串。其中给定的子串被称为模式串，被匹配的字符串被称为目标串。基于多个模式串进行匹配的算法被称为多模式匹配算法，目前成熟的多模式匹配算法有 AC 自动机和 WM。

多模式匹配算法对比

	AC 自动机	**WM**
优势	匹配效率高模式串对匹配效率无影响	匹配效率高内存占用量少模 处理流程简单，速度快
劣势	内存占用量大模式串加载流程复杂，耗时较长	对模式串长度有要求（长度 同，有最小长度限制）模式 在大量重复时，算法性能显 要根据实际情况调整算法参

谛听的敏感词匹配业务有如下特点：

- 词库量大，需要维护和加载百万级别的词库（模式串）；
- 敏感词与业务特性、国家政策相关性强，无法统一约定长度、前缀等特征。

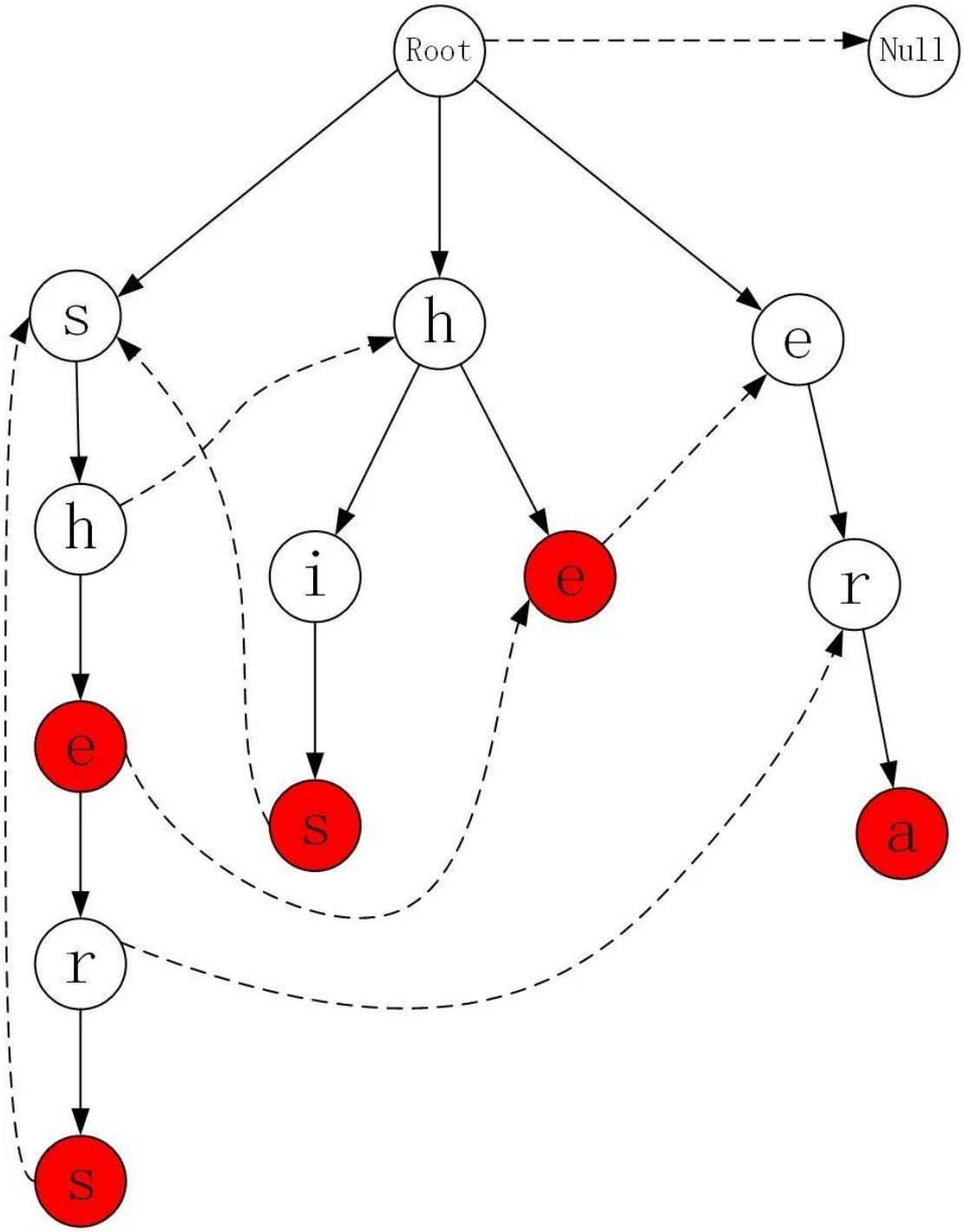
WM 算法对模式串的长度和前缀存在一定的要求，可能会影响业务的使用。虽然 AC 自动机加载耗时长，内存占用大，但敏感词加载并不频繁，且服务器内存资源充足，所以我们最终选用 AC 自动机作为底层算法。

36.1.1 AC 自动机算法介绍

AC 自动机算法（Aho–Corasick 算法）是一种字符串搜索算法，可以同时将目标串与所有模式串进行匹配，算法均摊情况下具有近似于线性的时间复杂度。AC 自动机的匹配原理有两个核心概念：Trie 字典树、Fail 指针。下面以模式串集合{“she”，“he”，“shers”，“his”，“era”}为例，来介绍这两个概念。

AC 自动机算法启动时，需要将所有的模式串加载到内存中，构建成一个 Trie 字典树。例如下图是以模式串集合{“she”，“he”，“shers”，“his”，“era”}构建的字典树，树中每个节点代表一个字符，从根节点到某一个节点的路径，即可表示一个模式串，且红色节点表示一个字符串的终结，例如图中最右边子树上的三个节点，可代表模式串“era”。

从字典树的根节点出发，可以快速的查找到某个模式串。此外，拥有相同前缀的模式串会合并到同一个子树中，例如中间子树表示模式串“he”、 “his”，这两个字符串分别是“h”节点的一个分支。AC 自动机在搜索这类字符串时，可以节省匹配的次数。

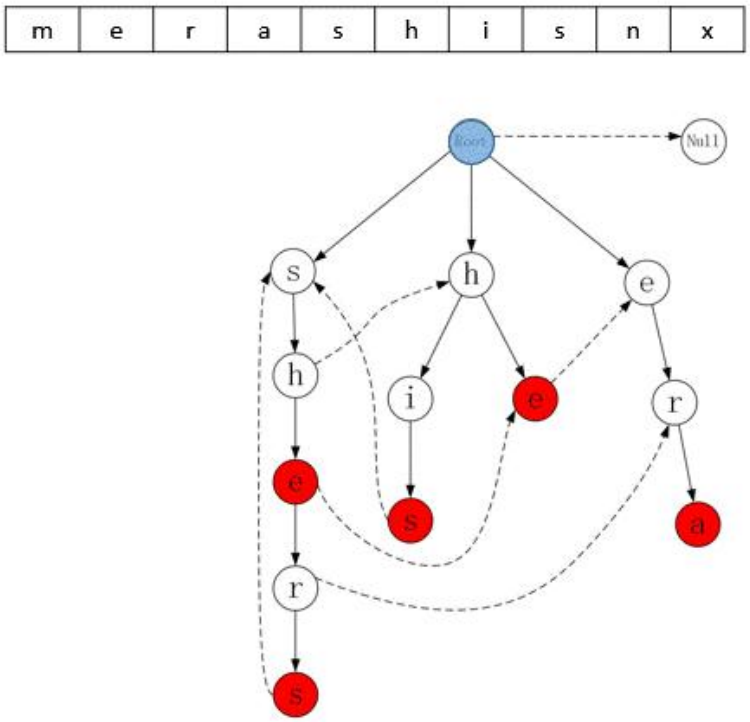


AC 自动机在 Trie 树的基础上，为每个节点加入了 Fail 指针，上图使用虚线画出了部分节点的 Fail 指针，未画出虚线的节点，其 Fail 指针指向根节点。算法在某个节点匹配失败时，可以通过该指针转移到其他包含相同前缀的分支上继续匹配。

例如匹配目标串“shis”时，对于前两个字符“sh”，Trie 字典树匹配到左边字数的“h”节点上，由于该节点的子节点是字符“e”，与目标串的下一个字符“i”不匹配，

因此算法通过 Fail 指针转移到中间子树的“h”节点上继续匹配，最终命中字符串“his”。

上述的 Trie 字典树与 Fail 指针组成了 AC 自动机的数据模型。AC 自动机匹配目标串时，会按顺序从目标串中取出字符，从 Trie 字典树的根节点出发，在子结点中寻找与该字符匹配的结点，若能找到，则转移到该节点，若找不到，则转移到 Fail 指针指向的节点。当状态转移到图中的红色节点时，就是命中了一个模式串。下图展示了 AC 自动机对目标串“merashisnx”进行匹配的过程。



36.1.2 谛听系统实践

谛听系统基于 AC 自动机算法构建了一套敏感词匹配服务，将敏感词作为模式串，文本内容作为目标串，可以实现常用的中、英文敏感词匹配。但是实际的业务有很多细分的场景，普通的 AC 自动机算法已不能满足业务使用需求，因此我们探索了组合敏感词匹配和拼音敏感词匹配两种匹配方式，下面分别介绍。

36.2 组合敏感词

常规的敏感词匹配算法通常匹配单个词或者短句，但某些词单独出现时并不违规，只有在与几个特定的词同时出现时，才能判定为违规。例如“澳门”、“博彩”、“网站”，这几个词，单独出现都是没问题的，但是在这句话中：“欢迎登录澳门 XX 博彩官方网站”，就是违规的赌博广告。针对这样的场景，我们实现了“组合敏感词”匹配方案，运营人员可以将这几个词配置成一个组合“澳门+博彩+网站”，只有这三个词同时出现时，敏感词服务才会判定这个组合命中。

组合敏感词的匹配算法依然基于 AC 自动机算法。由于 AC 自动机只能判断单个词的命中情况，因此我们将组合敏感词分割成单个敏感词，并维护各敏感词与组合间的映射关系，在 AC 自动机算法运行结束后，只有某个组合对应的敏感词全部命中时，才能判断该组合敏感词命中。为此我们需要给 AC 自动机添加一些前置和后置的处理步骤，具体步骤如下：

1. 将组合敏感词分割为单个敏感词，并记录敏感词与组合的映射关系；
2. 将分割后的组合敏感词添加到 AC 自动机的 Tire 树中；
3. 运行 AC 自动机，匹配文本；
4. 遍历匹配结果，将匹配的结果根据映射关系映射到相应的组合上；
5. 记录组合的命中情况，得到最终匹配结果。

假设目前存在组合敏感词“澳门+博彩+网站”、“博彩+广告”、“华人圈+赌博”、“赌博+广告”，以及普通敏感词“暴政”，则在步骤 1 中，我们将组合敏感词分割为单个敏感词：“澳门”、“网站”、“博彩”、“广告”、“赌博”、“华人圈”、“暴政”，并建立图中所示的映射关系。

将这些词添加到 AC 自动机后，对文本“欢迎登录澳门 XX 博彩官方网站”进行匹配时，会命中单个敏感词“澳门”、“网站”、“博彩”。在步骤 4 中，算法将匹配的词映射到组合中，并标记对应的词命中。例如根据“博彩”，可取到组合“澳门+博彩+网站”、“博彩+广告”，则分别标记这两个组合中的“博彩”已命中。步骤 5 会根据各组合中单词的命中情况，来判断该组合是否命中，由于在步骤 4 中组合敏感词“澳门+博彩+网站”的三个词均被标记为命中，因此可判断该组合命中。

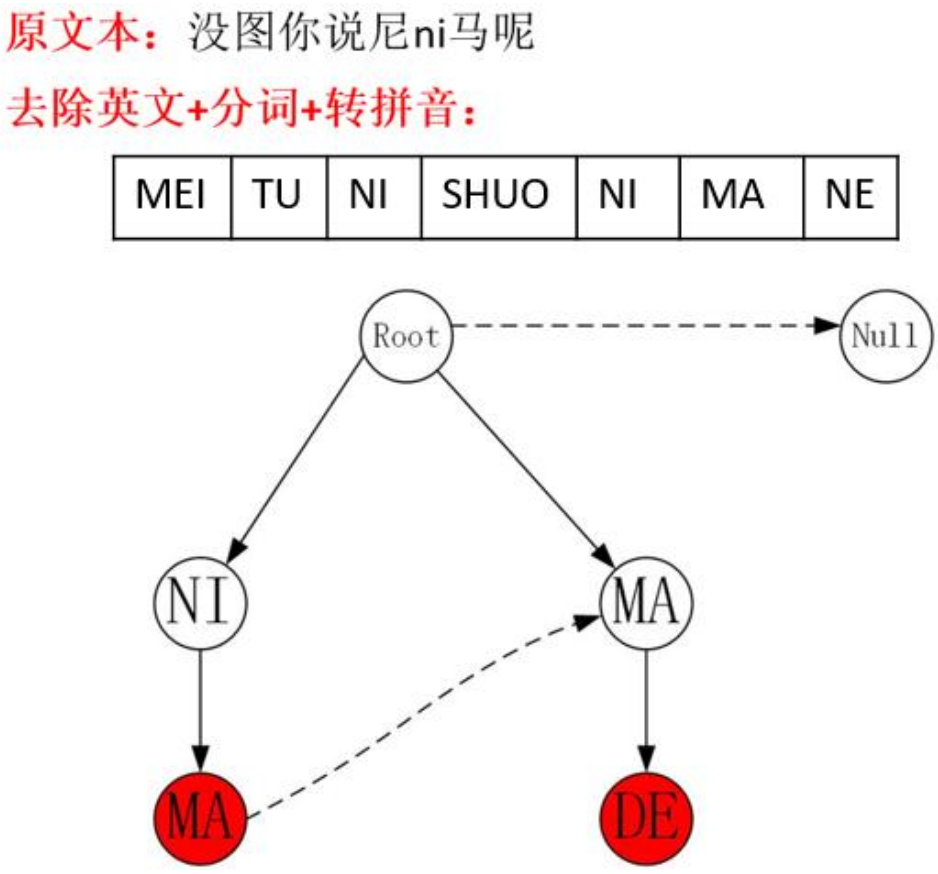
36.3 拼音敏感词

在评论、弹幕等创作自由度很高的场景中，某些用户为了规避机器审核，会使用一些多音字、同音字来表达敏感词汇，例如用“啋票”来代表“彩票”等。由于多音字、同音字变化较多，将一个词的所有谐音词都穷举出来是很困难的。

因此我们实现了拼音敏感词的匹配方案，将中文文本转换为拼音再匹配，通过读音匹配敏感词，即可保证命中所有的同音字，运营直接配置敏感词的拼音，例如“CAI PIAO”，即可命中“啋票”、“彩票”、“采漂”等词汇。

1、匹配流程

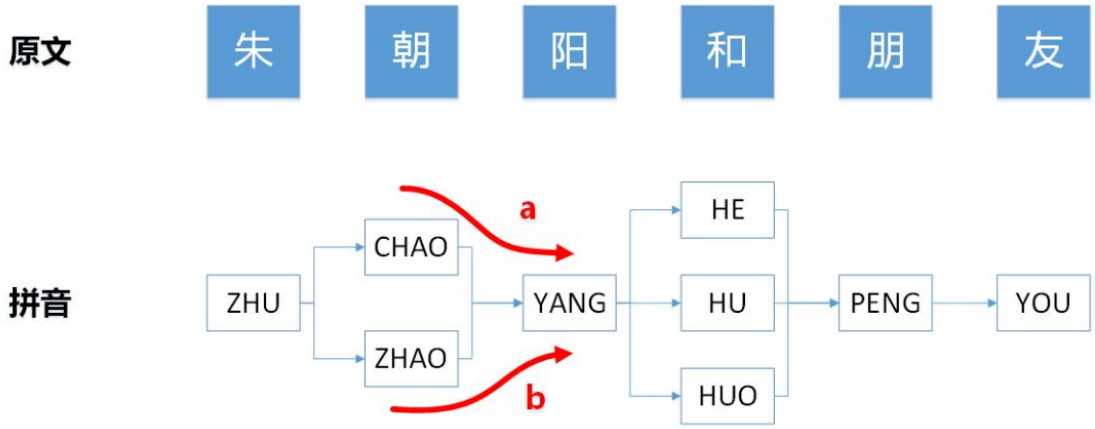
常规的 AC 自动机算法是逐字符匹配的，因此 Trie 树上每个节点存储一个字符，但拼音敏感词需要按照音节匹配，因此我们将 Trie 树的节点数据类型由 char 改为 String，示例：



拼音敏感词的匹配关键在于将汉字准确的转换为拼音，这一点在多音字的场景下尤为重要。由于多音字的读音是受语境影响的，现有的技术条件很难确保能将多音字准确的转换为拼音，而上文提到同音词“啋票”是用户自行造的词，算法无法准确的识别语境，可能转换得到“CAI PIAO”、“XIAO PIAO”两种不同的结果。如果拼音转换不精准，则拼音敏感词也无法准确命中。

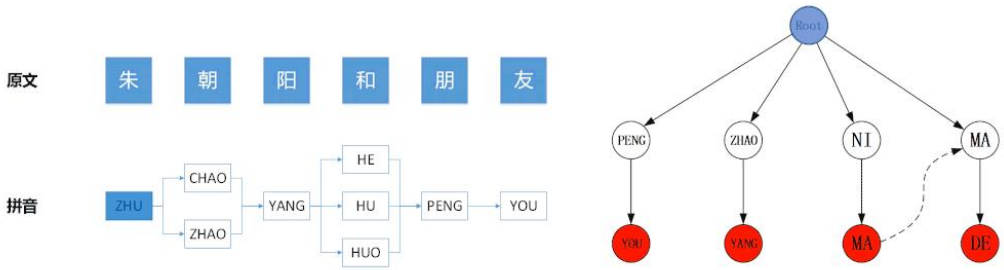
因此我们不依赖算法识别多音字的读音，而是将文本内容的所有读音都列出来匹配一遍，就可以避免避免拼音转换不精准的问题。

下图展示了文本内容与拼音的对应关系，由于存在多音字，因此存储拼音的数组从一维扩展到了二维，更像是“图”的数据结构，下文将其称为拼音图。拼音图的起始节点和终止节点之间存在多条路径，这些路径对应了多音字的所有排列组合情况，为了避免漏杀，我们需要使用 AC 自动机将这些路径都匹配一遍。



从第二节的匹配流程可以看出，目标串是一维数组，因此 AC 自动机在匹配文本时，通常采用顺序遍历的方式。而在拼音敏感词中，由于目标串采用二维数组存储，是一种类似于“图”的数据结构，不再适合使用顺序遍历的方式，因此需要采用图的遍历算法。图的遍历算法中，最常用的就是深度优先遍历（DFS）和广度优先遍历（BFS）。由于词语是前后关联的，为了使算法更符合人类思考习惯，我们选择了 DFS。

DFS 算法使用栈存储节点信息，在当前分支遍历完成后，通过栈中的信息回溯到上一个分支处继续遍历。由于 Trie 树的状态位与拼音图的节点是相关的，在 DFS 回溯时，Trie 树也需要同步回溯，因此需要将 Trie 树状态位与拼音图的节点信息一起保存到 DFS 栈中。下图展示了拼音敏感词的匹配流程。



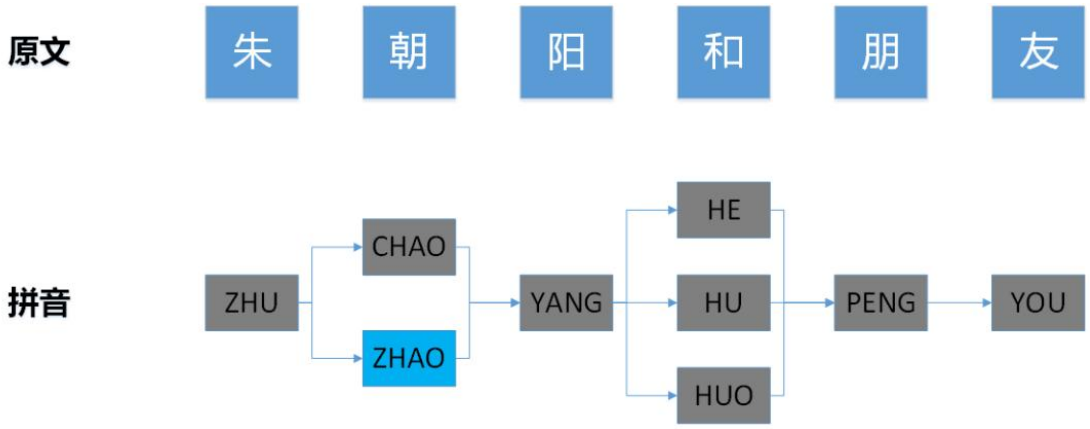
匹配成功：

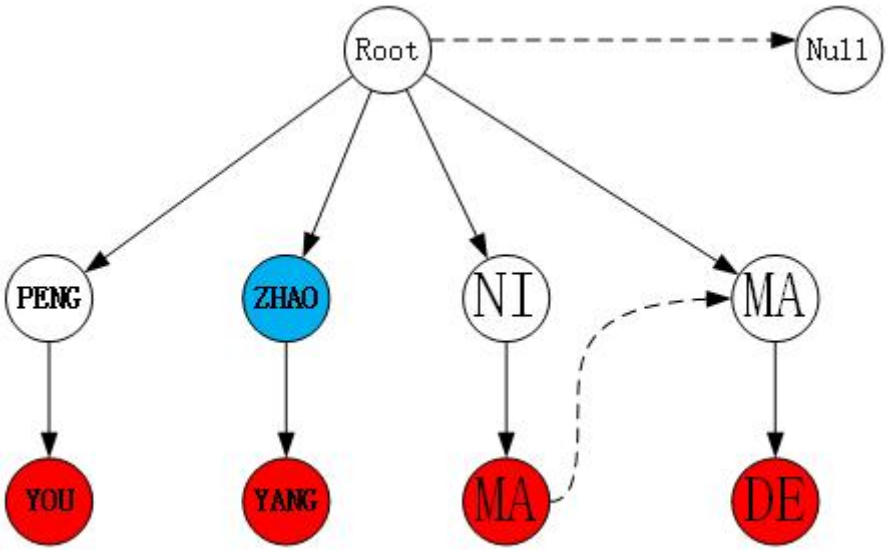
2、终止条件与剪枝策略

DFS 的终止条件是当所有节点都被遍历过，且算法会确保每个节点只会被遍历一次。但是 DFS 遍历时会在分支处回溯，所以往往终止的节点并不是待匹配文本的终点，很有可能 AC 自动机的匹配并未完成。

例如在下图所示的匹配流程中，左图是基于待匹配文本“朱朝阳和朋友”构建的拼音图，右图是基于拼音敏感词“PENG YOU”、“ZHAO YANG”、“NI MA”、“MA DE”构建的 Trie 树。左图的拼音图采用 DFS 算法遍历，算法最后访问的节点是蓝色节点“ZHAO”，此时拼音图中所有节点均被遍历了一次，已经达到了 DFS 的终止条件。但右图中 Trie 树上的状态位处于蓝色节点“ZHAO”的位置，并没有达到终止状态，若此时停止匹配，则会导致无法命中敏感词“ZHAO YANG”，故算法应继续运行，直到 AC 自动机匹配失败为止。

因此合适的终止条件是：拼音图所有节点均被遍历 且 AC 自动机匹配失败。





由于算法需要结合 DFS 和 AC 自动机的状态来判断终止条件，因此会出现拼音图中一个节点和路径被遍历多次的情况，当待匹配文本中多音字数量增多时，DFS 遍历的路径数量会以笛卡尔积的形式增加。而这些路径中会存在一部分重复的情况，因此在遍历的过程中需要采取合适的剪枝策略，避免搜索一些重复的路径。

例如下面左图所示的遍历情况，路径②上“PENG”、“YOU”两个节点已被路径①遍历过，且对应的 AC 自动机状态位（参考右图）前缀并不包含当前遍历节点“HU”，所以“PENG”对应的敏感词与路径②无关，不必再搜索一次。我们可以针对这种场景设计了剪枝策略，需要剪枝的路径需要满足两个条件：

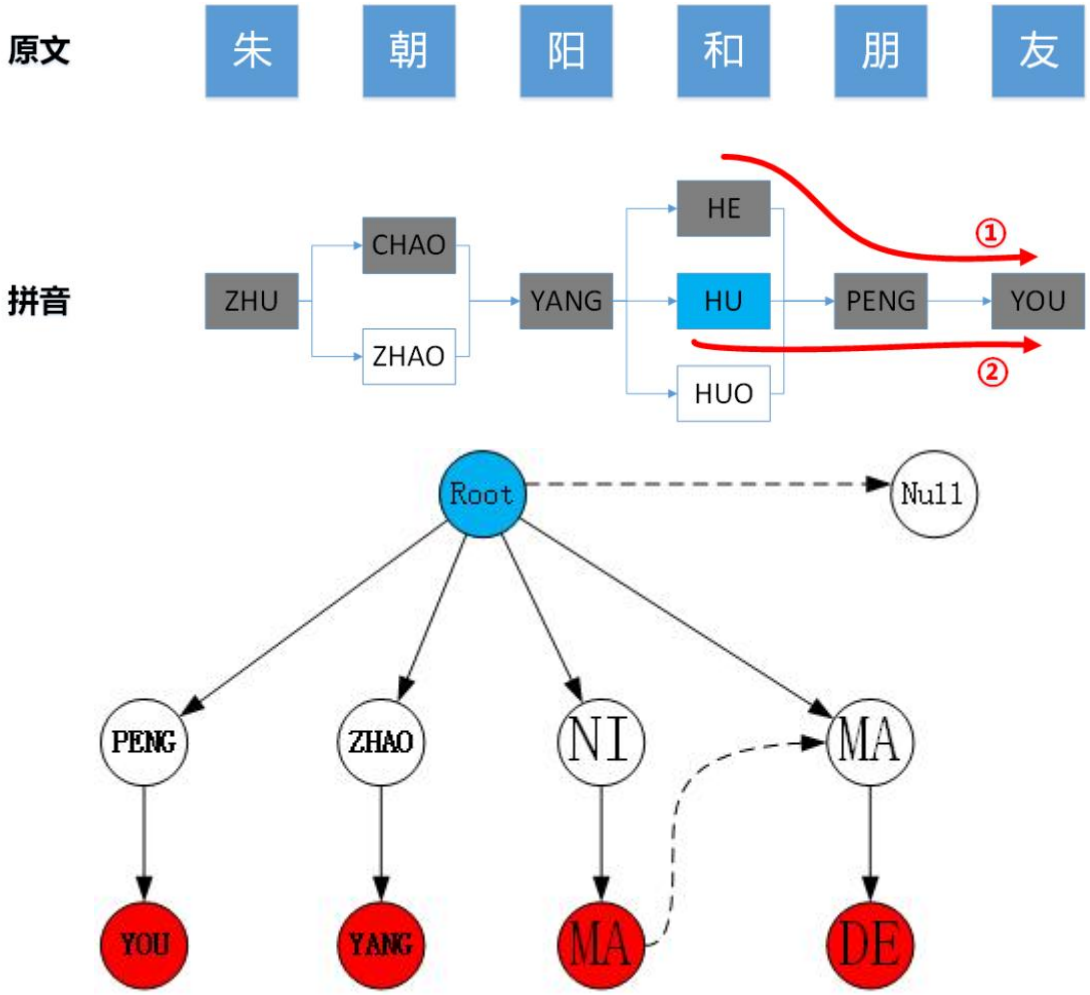
1. 首先当前节点的下一节点已被遍历过；
2. 下一节点对应的 AC 自动机状态与当前节点无关。

我们为拼音图中的每个节点标记一个分支路径长度 B，表示当前节点与上一个最近的分支节点间的路径长度，例如节点“PENG”对应的分支路径长度为 B = 2（从“HU”到“PENG”），节点“YOU”的分支路径长度为 B = 3（“HU”、“PENG”、“YOU”）。在 AC 自动机的状态机中，记录 Trie 树上每个节点的深度 D。例如状态机中“PENG”节点的深度 D=1，“YOU”节点的深度 D=2。从 Trie 树根节点到某一个节点的路径上经过的字符连接起来，为该节点对应的模式串，因此节点的深度 D 即为模式串的长度。当 D < B 时，表明当前正在匹配的模式串长度短于拼音图中当前节点的分支路径长度，所以当前的模式串与当前的路径无关。

总结一下，剪枝所需的条件为：

3. 拼音图中下一节点已被遍历；

4. 拼音图的分支路径长度 B > Trie 树节点的深度 D。



37. 微信抢红包的功能是如何实现的，如果让你来做你怎么设计?

实现拼手气红包算法，有以下几个需要注意的地方：

- 抢红包的期望收益应与先后顺序无关
- 保证每个用户至少能抢到一个预设的最小金额，人民币红包设置的最小金额一般是 0.01 元，如果需要发其他货币类型的红包，比如区块链货币或者积分，需要自定义一个最小金额。
- 所有抢红包的人领取的子红包的金额之和加起来，等于发红包的人发出的总红包的金额。 下面实现的方式是一次生成所有的子红包，让用户按顺序领取。也可以每领取一个生成一个，两种方式性能上各有优劣。

37.1 抢红包设计常见问题

37.1.1 微信的金额什么时候算?

答：微信金额是拆的时候实时算出来，不是预先分配的，采用的是纯内存计算，不需要预算空间存储。 采取实时计算金额的考虑：预算需要占存储，实时效率很高，预算才效率低。

37.1.2 实时性：为什么明明抢到红包，点开后发现没有?

答：2014 年的红包一点开就知道金额，分两次操作，先抢到金额，然后再转账。 2015 年的红包的拆和抢是分离的，需要点两次，因此会出现抢到红包了，但点开后告知红包已经被领完的状况。进入到第一个页面不代表抢到，只表示当时红包还有。

37.1.3 分配：红包里的金额怎么算？为什么出现各个红包金额相差很大?

答：随机，额度在 0.01 和(剩余平均值 *2)之间。 例如：发 100 块钱，总共 10 个红包，那么平均值是 10 块钱一个，那么发出来的红包的额度在 0.01 元~20 元之间波动。 当前面 3 个红包总共被领了 40 块钱时，剩下 60 块钱，总共 7 个红包，那么这 7 个红包的额度在：0.01 ~ （60/7*2）=17.14 之间。* 注意：这里的算法是每被抢一个后，剩下的会再次执行上面的这样的算法（Tim 老师也觉得上述算法太复杂，不知基于什么样的考虑）。

这样算下去，会超过最开始的全部金额，因此到了最后面如果不够这么算，那么会采取如下算法：保证剩余用户能拿到最低 1 分钱即可。

如果前面的人手气不好，那么后面的余额越多，红包额度也就越多，因此实际概率一样的。

37.1.4 红包的设计

答：微信从财付通拉取金额数据郭莱，生成个数/红包类型/金额放到 redis 集群里，app 端将红包 ID 的请求放入请求队列中，如果发现超过红包的个数，直接返回。根据红包的裸祭(逻辑)处理成功得到令牌请求，则由财付通进行一致性调用，通过像比特币一样，

两边保存交易记录，交易后交给第三方服务审计，如果交易过程中出现不一致就强制回归。

37.1.5 发性处理：红包如何计算被抢完?

答：cache 会抵抗无效请求，将无效的请求过滤掉，实际进入到后台的量不大。cache 记录红包个数，原子操作进行个数递减，到 0 表示被抢光。财付通按照 20 万笔每秒入账准备，但实际还不到 8 万每秒。

37.1.6 通如何保持 8w 每秒的写入?

答：多主 sharding，水平扩展机器。

37.1.7 据容量多少?

答：一个红包只占一条记录，有效期只有几天，因此不需要太多空间。

37.1.8 询红包分配，压力大不?

答：抢到红包的人数和红包都在一条 cache 记录上，没有太大的查询压力。

37.1.9 一个红包一个队列?

答：没有队列，一个红包一条数据，数据上有一个计数器字段。

有没有从数据上证明每个红包的概率是不是均等？ 答：不是绝对均等，就是一个简单的拍脑袋算法。

37.1.10 拍脑袋算法，会不会出现两个最佳?

答：会出现金额一样的，但是手气最佳只有一个，先抢到的那个最佳。

37.1.11 每领一个红包就更新数据么?

答：每抢到一个红包，就 cas 更新剩余金额和红包个数。

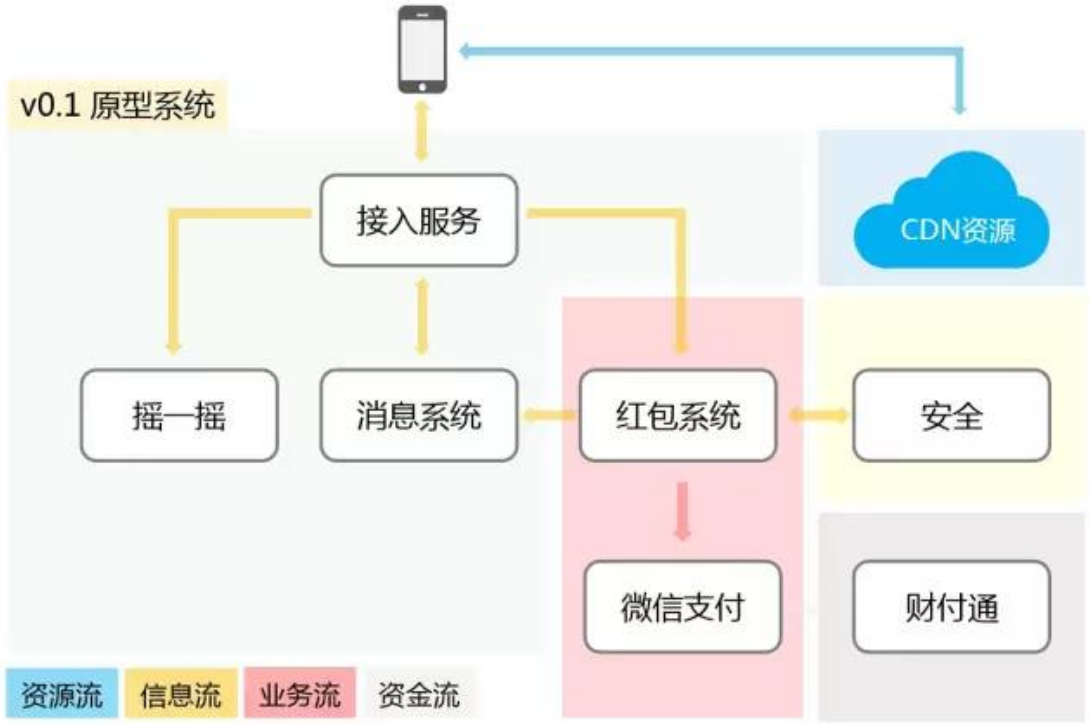
37.1.12 红包如何入库入账?

数据库会累加已经领取的个数与金额，插入一条领取记录。入账则是后台异步操作。

37.1.13 入帐出错怎么办？比如红包个数没了，但余额还有？

答：最后会有一个 take all 操作。另外还有一个对账来保障。

37.2 V0.1 原型系统

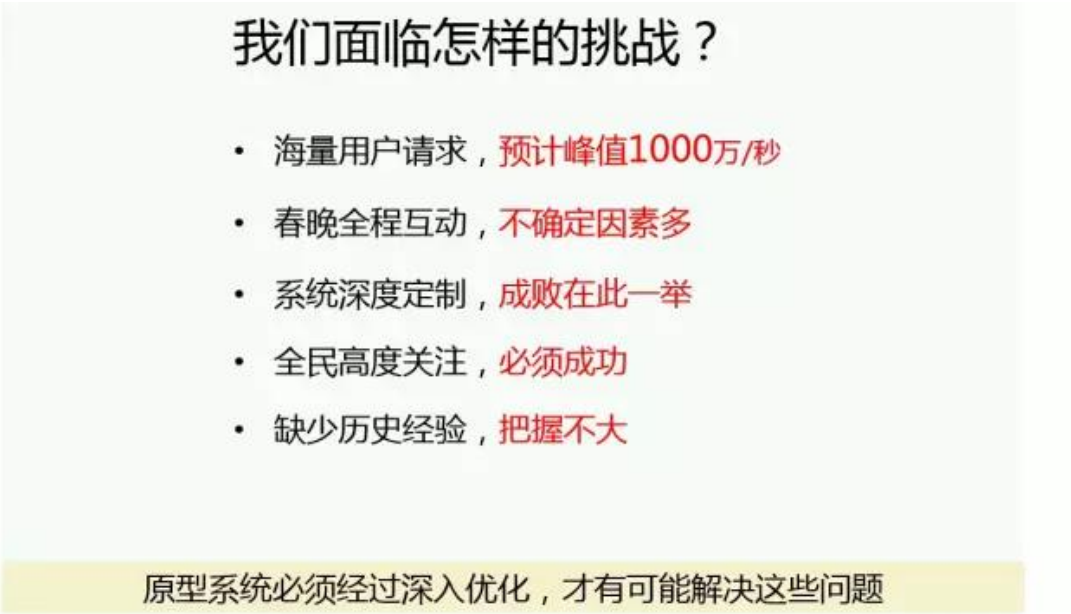


我们看一下这个系统，我们当时做了一个原型系统，比较简单，它已经实现了所有的功能，摇那个手机的时候会通过客户端发出一个请求，接入服务器，然后摇一摇服务，进行等级判断，判断以后把结果给到后端，可能摇到拜年或红包，假设摇到红包，上面有 LOGO 和背景图，客户端把这个 LOGO 和背景图拉回去，用户及时拆开红包，拆的请求会来到红包系统，红包系统进行处理之后会到支付系统，到财富通的转帐系统，最终用户拿到红包。拿到钱以后，只是其中一份，还有好几份是可以分享出去，我们称之为“分裂红包”，通过信息系统转发给好友或群里，好友跟群里的人可以再抢一轮。

整个过程归一下类，叫资源流、信息流、业务流、资金流，今天讲的主要是资源流跟信息流。

原始系统看起来比较简单，是不是修改一下直接拿到春晚上用就可以了？肯定不行的。到底它有什么样的问题呢，为什么我们不能用，在回答这个问题之前想请大家看一下我们面临的挑战。

1、我们面临怎样的挑战?



第一个挑战是比较容易想到的，用户请求量很大，当时预计 7 亿观众，微信用户也挺多的，当时预估一下当时峰值达到一千万每秒，通过图对比一下，左边是春运抢火车票，一秒钟请求的峰值是 12 万，第二个是微信系统，微信系统发消息有个小高峰，那时候峰值每秒钟是 33 万，比较高一点的是预估值一千万每秒，右边是春晚时达到的请求峰值是 1400 万每秒。

这个活动跟春晚是紧密互动的，有很多不确定因素，体现在几个方面。一个是在开发过程中，我们的活动怎么配合春晚，一直没有定下来，很可能持续到春晚开始前，显然我们的客户端跟我们的系统到那时候才发布出去，这时候我们的开发就会碰到比较多的问题了，这是第一个。

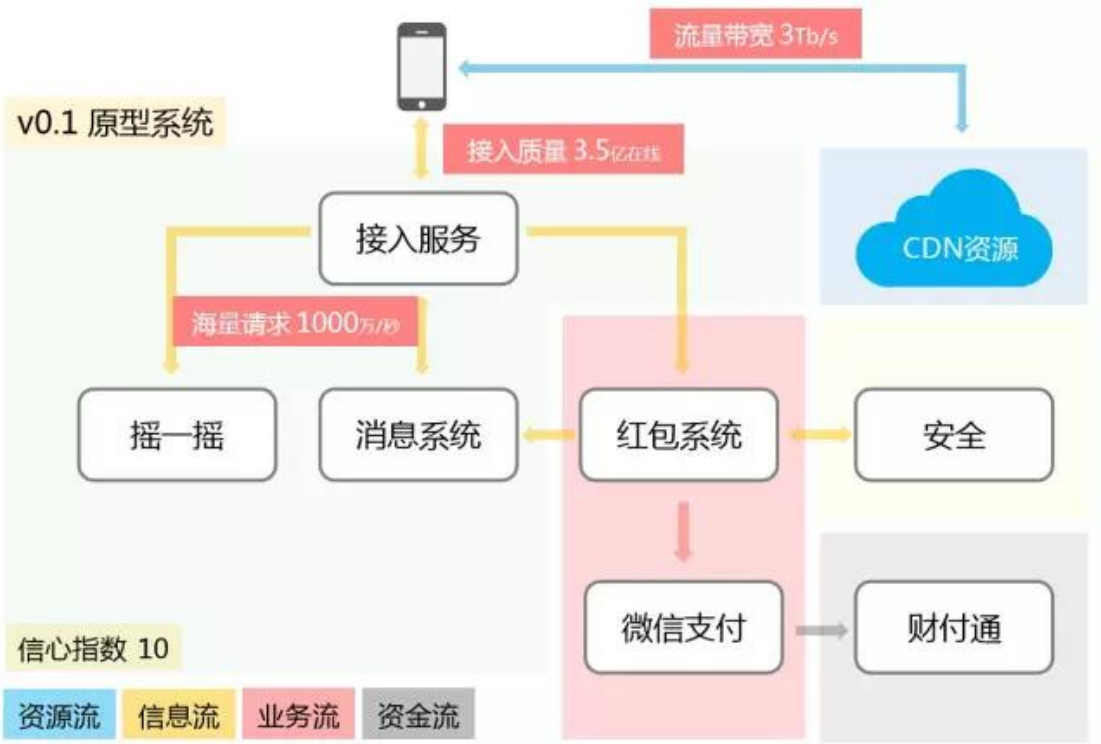
第二个，在春晚过程中，因为春晚是直播型节目，节目有可能会变，时长会变，顺序会变，活动过程跟春晚节目紧密衔接在一起，自己也是会有挑战的，这也是不确定的因素。再就是我们系统是定制的，专门为春晚定制，只能运行这么一次，这是挺大的挑战，运

行一次的系统不能通过很长的时间，检查它其中有什么问题很难，发出去了以后那一次要么就成功了，要么就失败了。

第三个，因为春晚观众很多，全国人民都在看，高度关注，我们必须保证成功，万一搞砸了就搞砸在全国人民面前了。这么大型的活动在业界少见，缺少经验，没有参考的东西。还有就是我们需要做怎样的准备才能保证万无一失或者万有一失，保证绝大部分用的体验是 OK 的，有很多问题需要我们不断地摸索思考。原型系统不能再用的，再用可能就挂了。

2、原型系统存在哪些问题?

原型系统有哪些问题呢？第一个是在流量带宽上，大量的用户请求会产生大量的带宽，预估带宽峰值是 3000pb 每秒，假设我们资源是无限的能够满足带宽需求，也会碰到一个问题，用户摇到以后有一个等待下载的过程。第二个问题，在接入质量这一块，我们预估同时在线 3.5 亿左右，特别是在外网一旦产生波动的时候怎么保证用户体验不受损而系统正常运作。第三个挑战，请求量很大，1000 万每秒，如何转到摇一摇服务，摇一摇服务也面临一千万请求量，我们系统要同时面对两个一千万请求量，这不是靠机器的，大家都有分布式的经验，这么大请求量的时候任何一点波动都会带来问题，这是一个很大的挑战。

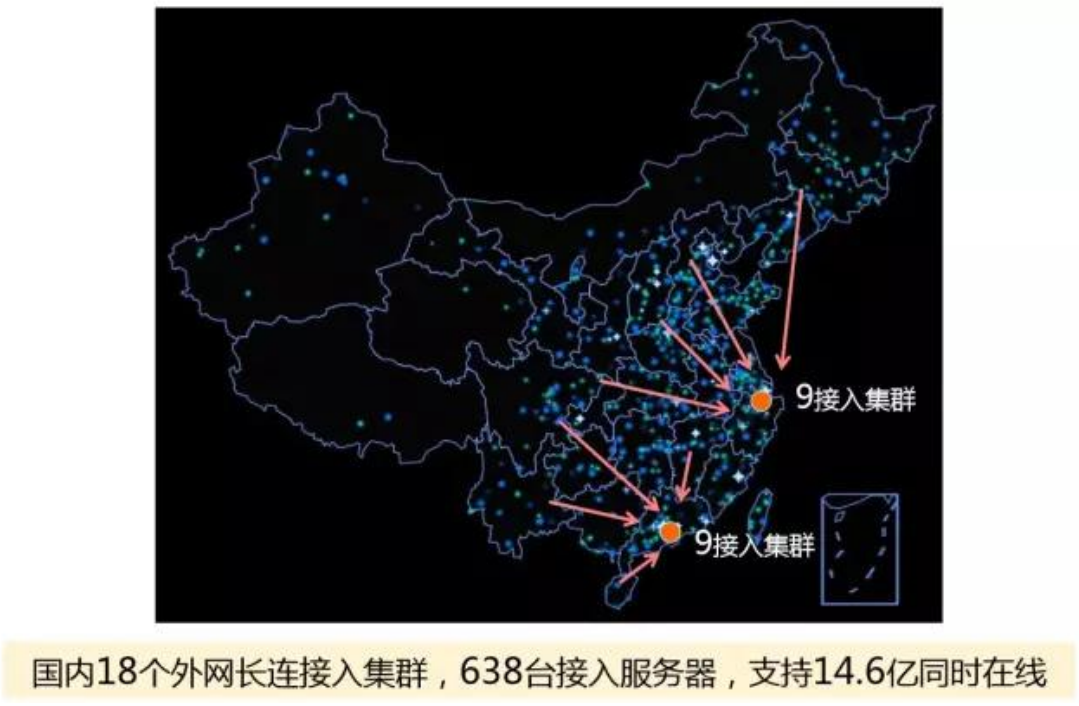


3、我们是如何解决这些问题的?

针对以上几点，我们详细看一下每一点我们是怎么做到的。我们首先看一下信心指数，把这个系统拿到春晚去跑有多少信心，这里的指数是 10，如果这个系统拿到春晚去用，而且还成功了，这个概率是 10%。当然我们的系统不能建立在运气的基础上，应该怎么做？第一个，在带宽这一块客户端可以摇到多种多样的结果，结果大部分都是静态资源，静态资源我们可以提前制作出来下发到客户端，在后台做了资源推送的服务，客户端拿到列表以后可以先行下载，客户端利用闲时把资源拉过去。碰到几个问题，资源交付情况的问题，需要增量的发下去；二是资源更新；三是资源下载失败，失败的话怎么办呢；四是资源覆盖率，依靠这个系统下载资源的用户，比如覆盖率只有 20%、30%，两个东西就没有意义了，覆盖率要达到 90%左右；五是离线资源下载，万一有些人把里面的东西修改了，可能会产生意想不到的结果，怎么保证离线资源的安全。

这里有个数据，2 月 9 号到 2 月 18 号下发资源 65 个，累积流量 3.7PB，峰值流量 1Tb/s。通过这种方式解决了下载资源的问题。

再就是外网接入质量，在上海跟深圳两地建立了十八个接入集群，每个城市有三网的介入，总共部署了 638 台接入服务器，可以支持同时 14.6 亿的在线。

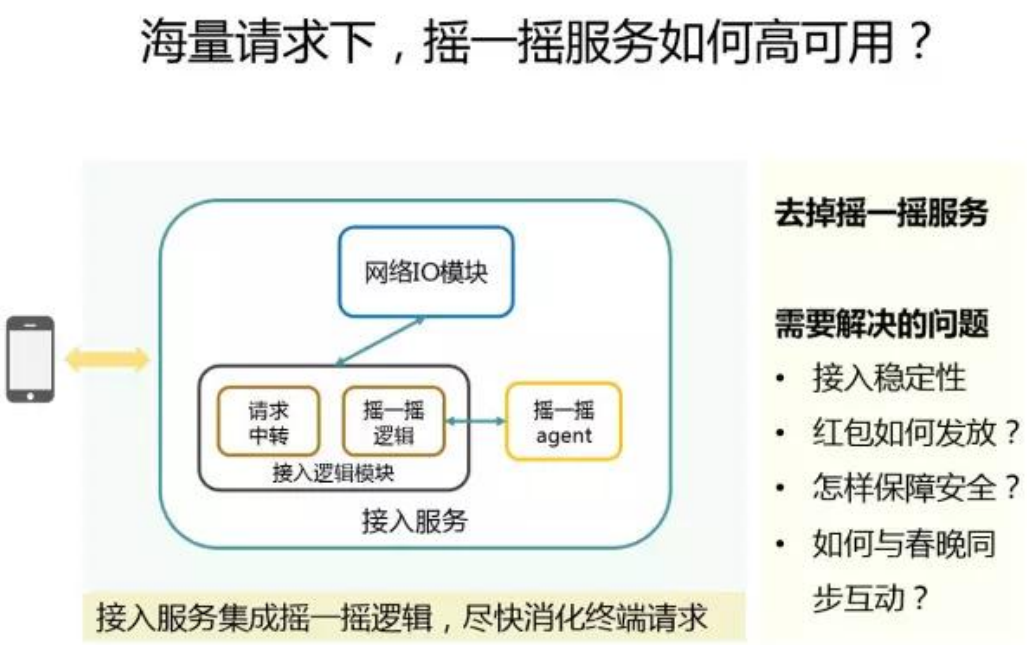


所有用户的请求都会进入到接入服务器，我们建立了 18 个接入集群，保证如果一个出现问题的时候用户可以通过其它的接入，但是在我们内部怎么把请求转给摇一摇服务，摇一摇处理完还要转到后端，怎么解决呢？解决这个问题代价非常大，需要很多资源，

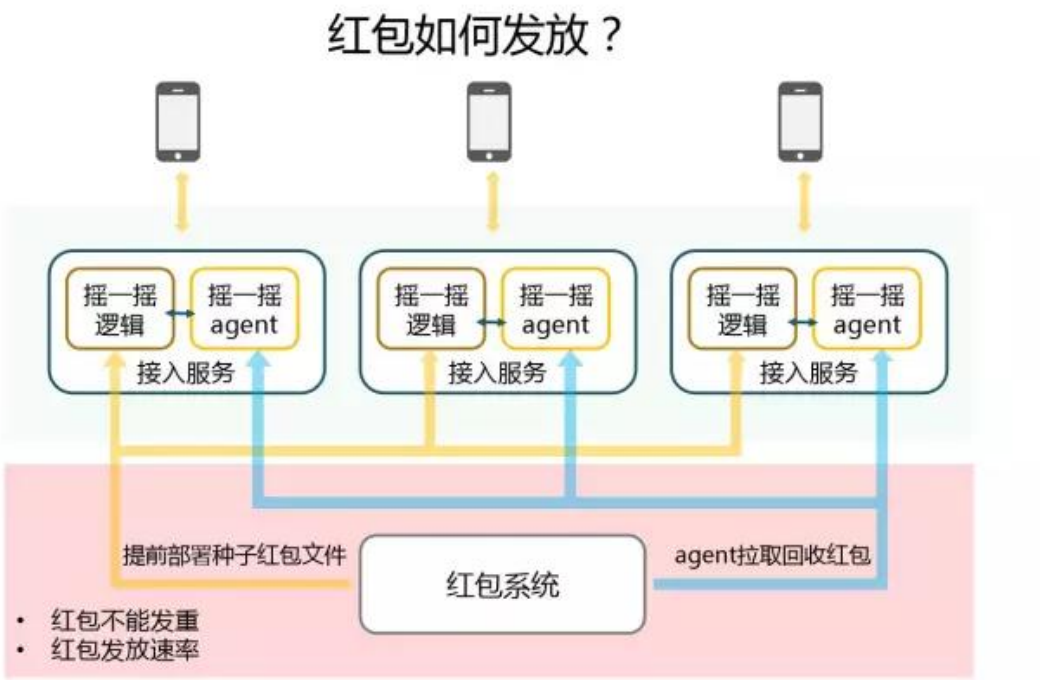
最终我们选择把摇一摇服务去掉，把一千万每秒的请求干掉了，把这个服务挪入到接入服务。除了处理摇一摇请求之外，所有微信收消息和发消息都需要中转，因为这个接入服务本身，摇一摇的逻辑，因为时间比较短，如果发消息也受影响就得不偿失了。

恰好有一个好处，我们的接入服务的架构是有利于我们解决这个问题的，在这个接入节点里分为几个部分，一个是负责网络 IO 的，提供长链接，用户可以通过长链接发消息，回头可以把请求中转到另外一个模块，就是接入到逻辑模块，平时提供转发这样的功能，现在可以把接入逻辑插入。这样做还不够，比方说现在做一点修改，还需要上线更新，摇一摇的活动形式没有怎么确定下来，中间还需要修改，但是上线这个模块也不大对，我们就把接入的逻辑这一块再做一次拆分，把逻辑比较固定、比较轻量可以在本地完成的东西，不需要做网络交互的东西放到了接入服务里。

另外一个涉及到网络交互的，需要经常变更的，处理起来也比较复杂的，做了个 Agent，通过这种方式基本上实现了让接入能够内置摇一摇的逻辑，而且接入服务本身的逻辑性不会受到太大的损伤。解决这个问题之后就解决了接入的稳定性问题，后面的问题是摇一摇怎么玩，摇一摇怎么玩是红包怎么玩，在红包过程中怎么保证红包是安全的呢，红包涉及到钱，钱是不能开玩笑的。第三个问题是怎样跟春晚保持互动，春晚现场直播，我们怎么跟现场直播挂钩衔接起来。



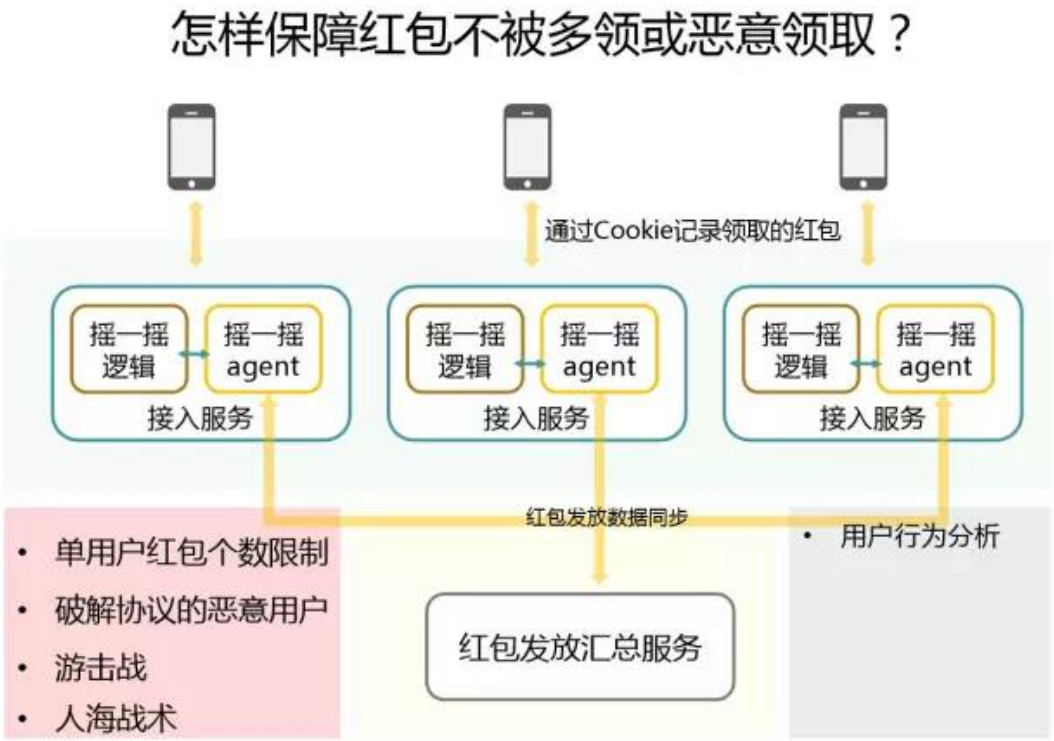
先看红包如何发放。



前面说道摇一摇请求，其实是在接入服务做的，红包也是在接入服务里发出去的，为了在发红包过程中不依赖这个系统，我们把红包的种子文件在红包系统里生成出来，切分，分到每个接入服务器里，每个接入服务器里都部署了专门的红包文件。一个红包不能发两次，红包的发放速率需要考虑，发放红包一定有用户拆，拆了还要再抢，我们需要精确控制，确保所有请求量都是在红包系统能够接受的范围内。在这个过程中还会有另外一个风险，用户摇到红包之后还可以有一些分裂红包分出去，他也可以不分享，不分享

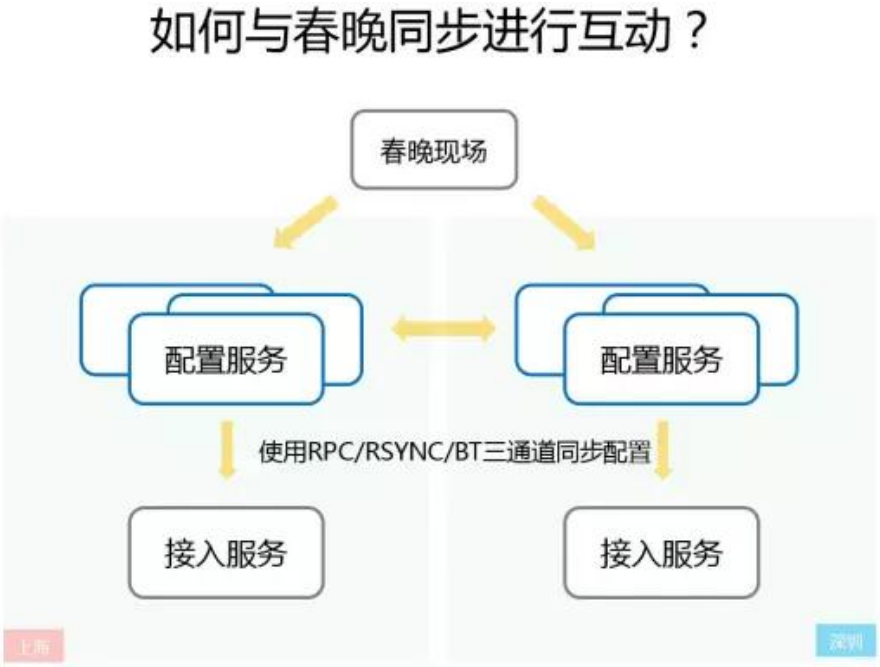
的也不会浪费，可以回收过来，会通过本地拉回去。这部分因为是比较少量的，问题不大，因为所有红包已经发出去了，只是补充的。这里我们就搞定了红包发放。

二是怎么样保证红包不被多领或恶意领取，每个客户领三个红包，这是要做限制的，但这是有代价的，就是存储的代价。



我们在我们的协议里后台服务接入的摇一摇文件里下发红包的时候写一个用户领取的情况，客户端发再次摇一摇请求的时候带上来，我们检查就行了，这是一个小技巧，这种方式解决用户最多只能领三个、企业只能领一个限制的问题。这个只能解决正版客户端的问题，恶意用户可能不用正版，绕过你的限制，这是有可能的。怎么办呢？一个办法是在 Agent 里面，通过检查本机的数据能够达到一个目的，摇一摇接入服务例有 638 台，如果迫到不同的机器，我们是长连，还可以短连，还可以连到另一台服务器，可以连到不同的地方去。还有一个问题是人海战术，有些人拿着几万、几十万的号抢，抢到都是你的，那怎么办呢？这个没有太好的办法，用大数据分析看用户的行为，你平时养号的吗，正常养号吗，都会登记出来。

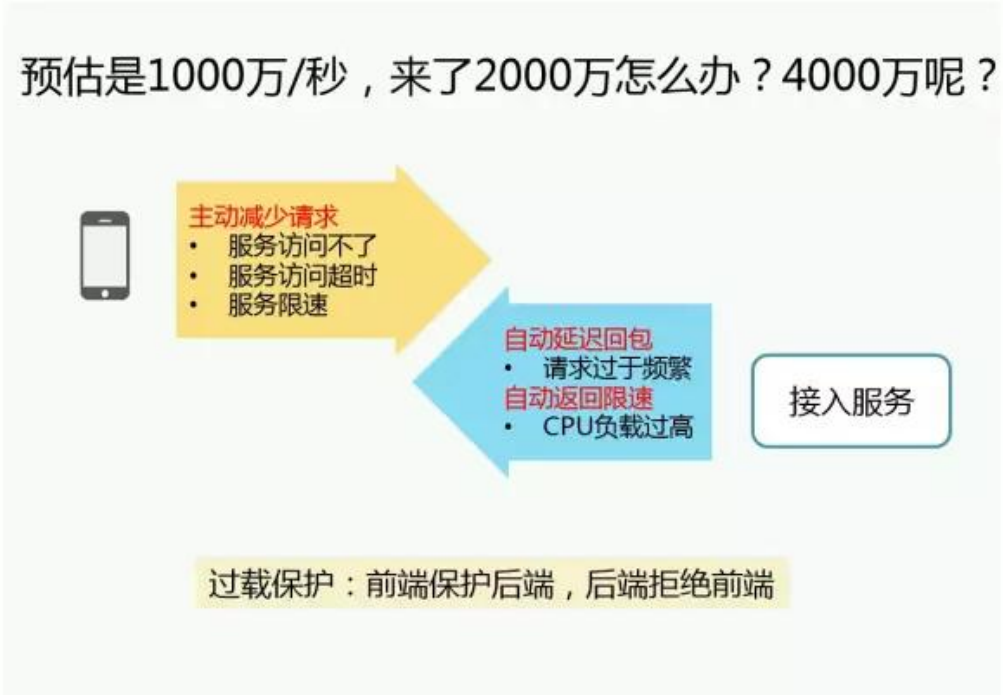
怎样跟春晚现场保持互动？需要解决的问题有两个，一个是要迅速，不能拖太长时间，比如现在是刘德华唱歌，如果给出的明星摇一摇还是上一个节目不太合适，要求我们配置变更需要迅速，二是可靠。我们怎么做的呢？



春晚现场我们是专门有同学过去的，在他们电脑装了系统，可以跟我们后台进行交互的，节目变了节切一下，变动的请求会发到后台，我们部署两套，一套在深圳、一套在上海，在这个配置里还准备了三步服务，哪一步都可以，同时还可以同步这个数据，这个数据还可以下发到所有的接入机器，会把它同步过去，不是用一种方式，而是用三种方式，通过这种方式可以迅速的在一千台服务器成功，是不是能够达到配置一定能够用？不一定，春晚现场是不可控的，万一指令没有发出怎么办？如果六个配置服务都挂了怎么办，从上一个节目切到下一个节目的时候发生这种问题不是太大，但是主持人在十点三十的时候如果摇红包一摇啥都没有，这就怒了，口播出不来也就挂了。

怎么做的呢？主持人肯定有口播，口播的时间点我们大致知道，虽然不知道精确的时间点，比如彩排的时候告诉我们一个时间，后来变了，我们大致知道时间范围，可以做倒计时的配置，比如十点半不管你有没有口播我们都要发红包了。如果节目延时太长了，你的红包十分钟发完了，之后摇不到，那也不行，这种情况下我们做了校正，在节目过程中我们不断校正倒计时的时间，设了一个策略，定了一个流程，半小时的时候要通知一下我这个时间，因为它是预估的，节目越到后面能定下来的时间范围越精确，提前告诉我们，我们就可以调整。那时候现场是在春晚的小会议室，在小会议室看不到现场什么情况，也是通过电视看，结果电视没信号了，就蒙了，校准就不知道现在怎么回事，进行到哪一步了，当时很着急，还好后来没事，后续的几个节目还是校正回来了，最终我们是精确的在那个时间点出现了抢红包。

前面讲了怎么在系统解决流量的问题、请求量的问题，最重要的一点是我们预估是一千万每秒，但如果春晚现场出来的是两千万、三千万或四千万怎么办，是不是整个系统就挂掉了。

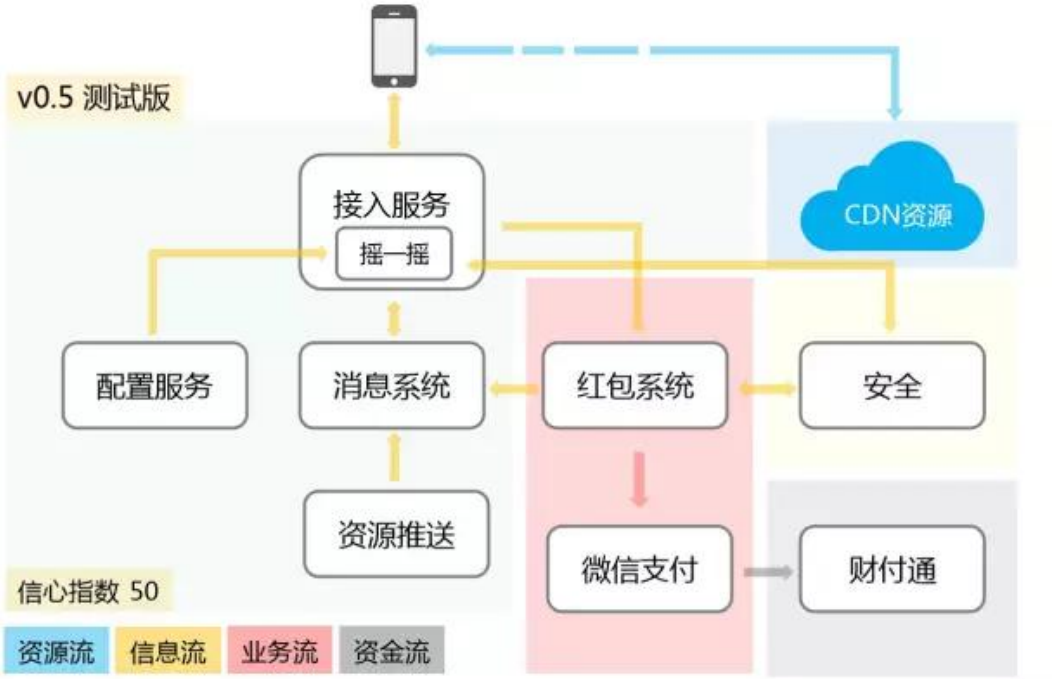


我们就是采用过载保护，过载保护中心点是两点，前端保护后端，后端拒绝前端。一个是在客户端埋入一个逻辑，每次摇变成一个请求，摇每十秒钟或五秒钟发送一个请求，

这样可以大幅度降低服务器的压力，这只会发生到几个点，一个是服务访问不了、服务访问超时和服务限速。实时计算接入负载，看 CPU 的负载，在衔接点给这台服务器的用户返回一个东西，就是你要限速了，你使用哪一档的限速，通过这种方式，当时有四千万用户在摇我们也能扛得住。

37.3 V0.5 测试版

这是我们的 0.5 测试版，对这个我们的信心指数是 50，为什么只有 50%的把握?

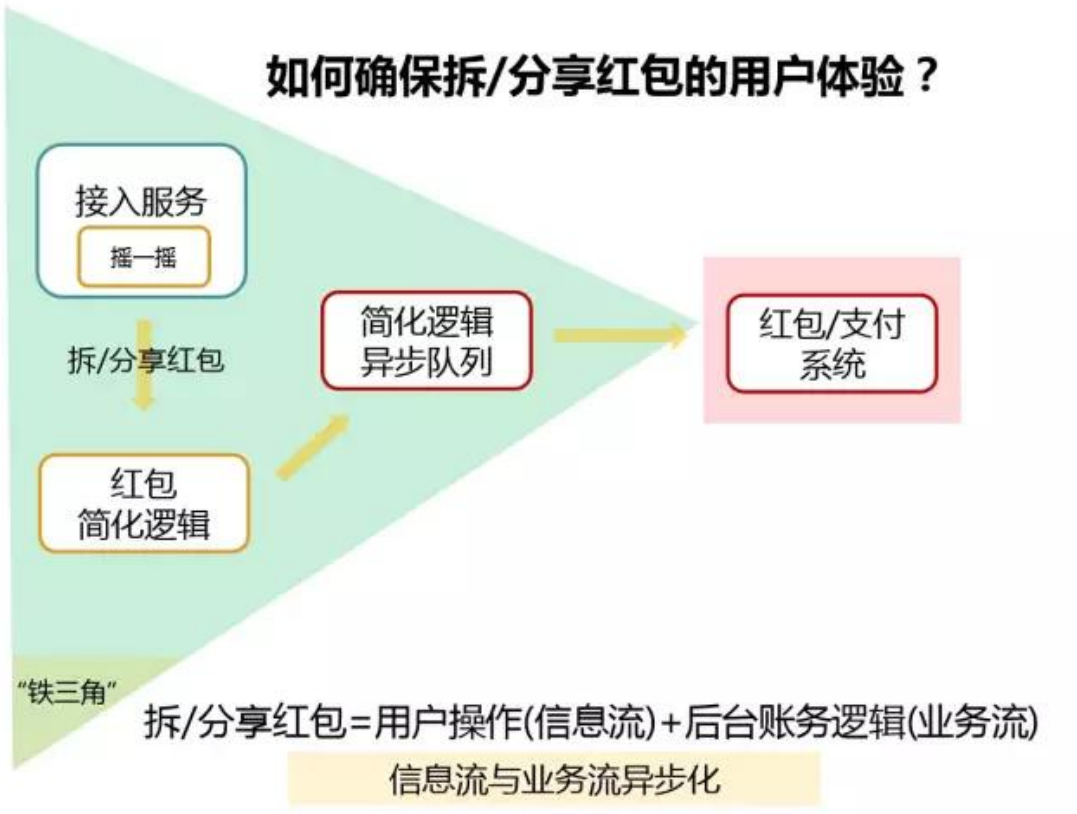


我们前面解决的问题都是解决用户能摇到红包，服务器还不会坏掉，但是对摇红包来说那是第一步，后面还有好几步，还要把红包拆出来，还要分享，分享完以后其它人可以抢，这个体验是要保证的，简单分析一下可以发现前面是本人操作，后面是好友操作，这里就存在一个契机，你可以做一些服务，一旦出现问题是可以利用的点，可以做延时。剩下的问题是保证本人操作比较好，后面出点问题可以延迟，有延迟表示有时间差处理那个东西。

1、核心体验是什么?

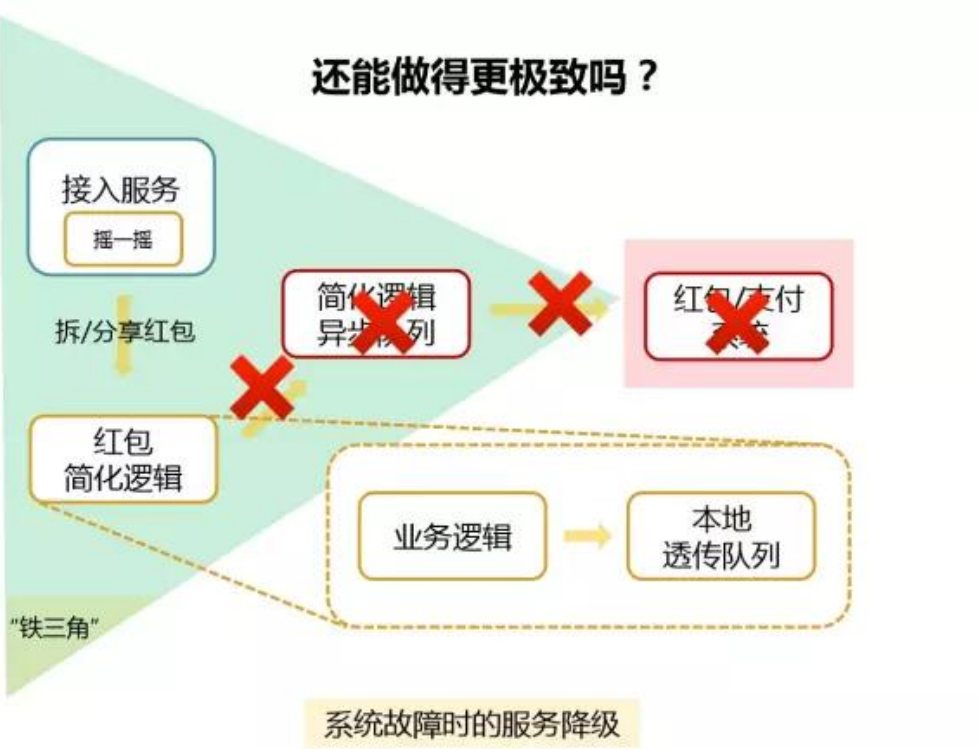
这里面我们需要确保成功，确保体验是完全 OK 的，确保成功的时候前面提到原型的系统里解决了摇到红包的问题，剩下的就是拆红包和分享红包。怎么样确保拆红包和分享红包的用户体验？

2、如何确保拆/分享红包的用户体验？



拆红包和分享红包可以做一下切割，可以切割成两个部分，一个是用户的操作，点了分享红包按钮，之后是转帐，对我们来说确保前面一点就可以了，核心体验设计的东西再次缩小范围，确保用户操作这一步能够成功。怎么确保呢？我们称之为“铁三角”的东西，拆/分享红包=用户操作+后台彰武逻辑。这是我们能做到的最高程度了。

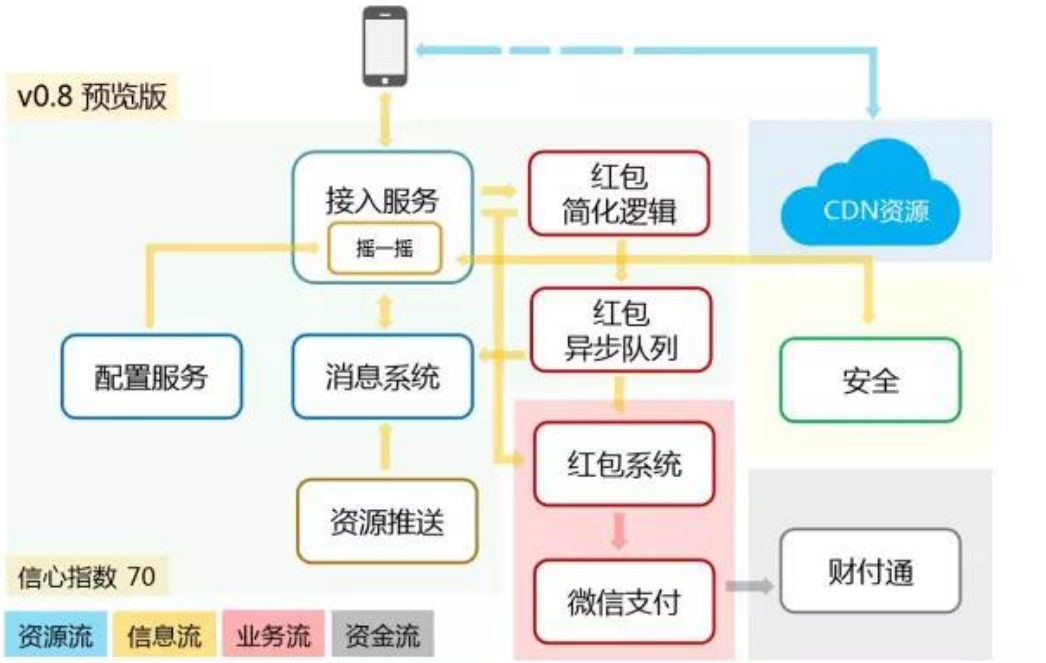
3、还能做得更极致吗?



但我们还可以做的更好一点，前面这个用户看起来还是成功的，只是入帐入的稍微迟一点点，用户感觉不到。如果我们异步队列这里挂了，或者网络不可用了，概率比较低，我们有三个数据中心，挂掉一个问题不大，万一真的不能用呢，我们又做了一次异步，分两部分：一个是业务逻辑，校验这个红包是不是这个用户的，还有一个透传队列，把这个数据再丢到后边，其实可以相信本机的处理一般是可以成功的，只要做好性能测试基本上是靠谱的。在后面出现问题的时候我们用户的体验基本不受损，保证绝大多数用户的体验是 OK 的。

37.4 V0.8 预览版

我们又做了 0.8 的版本，预览版，信心指数 70，我们认为这个东西有七成把握是可以成功的。



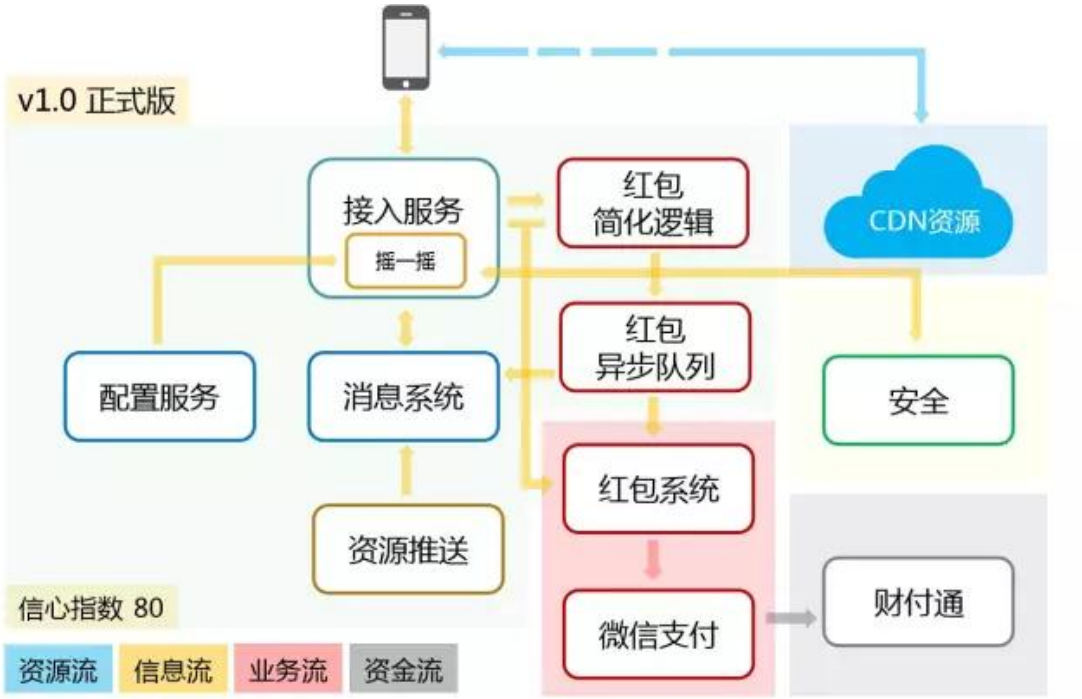
大家知道设计并不等于实现，设计的再好实践有问题也很崩溃，要保证设计一是全程压测，二是专题 CODE REVIEW，三是内部演练，四是线上预热，五是复盘与调整。

复盘包括两部分，有问题的时候可以把异常问题看出来，二是很正常，跑的时候是不是跟想象的一样，需要对正常情况下的数据做预估的重新评估，看看是不是符合预期。两次预热，一次是摇了 3.1 亿次，峰值 5000 万一分钟，100 万每秒，跟我们估算的一千万每秒差很远，当时只是针对 iPhone 用户，放开一个小红点，你看到的时候可以抢，

发放红包 5 万每秒，春晚当晚也是五万每秒。后面又发了一次，针对前面几个问题再做一次。

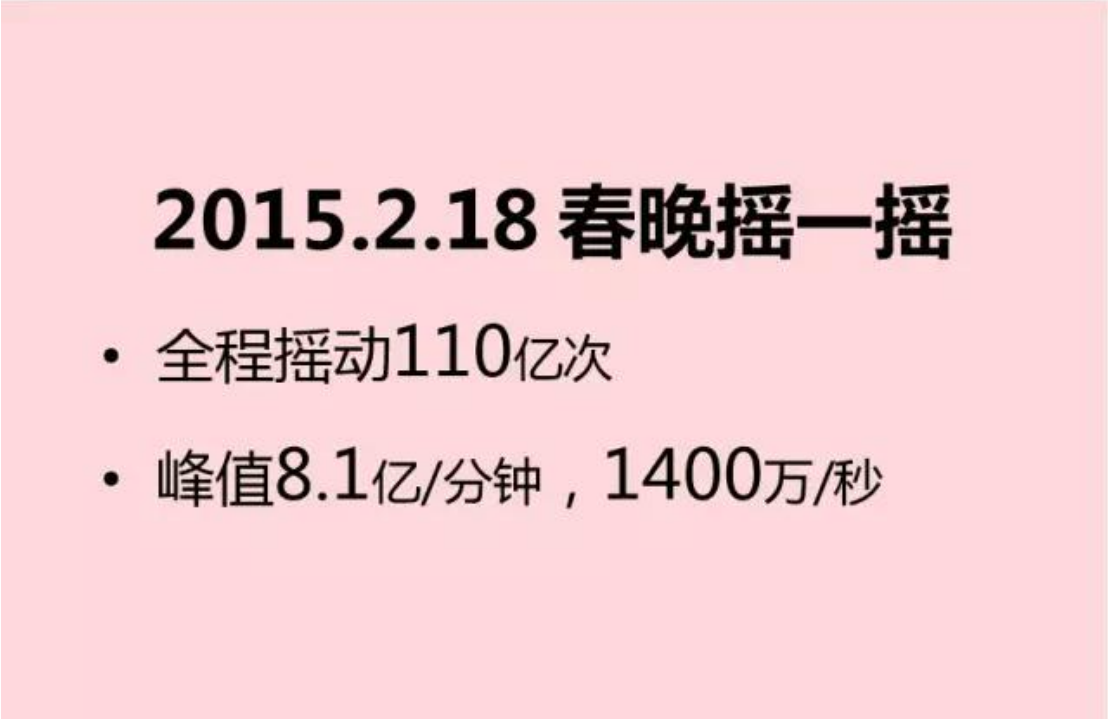
37.5 V1.0 正式版

做完这两次连接后面就迎接 2 月 18 号春晚的真正考验。这是 1.0 正式版，信心指数达到 80，我们认为 80%是可以搞定的。



剩下 20%在哪里？有 10%是在现场，现场不可能一帆风顺，有可能现场摇的很 High，但后面到处灭火，10%是在现场处置的时候有比较好的预案和方案能够解决，另外 10%是人算不如天算，一个很小的点出现问题导致被放大所有用户受影响也是有可能的，我们很难控制了。要做出完美无缺的基本不太可能，剩下 10%就是留运气。

2 月 18 号跑出来的结果是这样的，当时摇了 110 亿次，峰值是 8.1 亿每分钟，1400 万每秒。



38. 系统升级过程中如何实现数据的平滑迁移

在实际工作中，不只是对数据库拆分时会做数据迁移，**很多场景都需要你给出数据迁移的方案，**比如说某一天，你的老板想要将应用从自建机房迁移到云上，那么你就要考虑将所有自建机房中的数据，包括 MySQL，Redis，消息队列等组件中的数据，全部迁移到云上，这无论对哪种规模的公司来说都是一项浩瀚的工程，所以你需要在迁移之前，准备完善的迁移方案。

38.1 如何平滑地迁移数据库中的数据

你可能会认为：数据迁移无非是将数据从一个数据库拷贝到另一个数据库，可以通过 MySQL 主从同步的方式做到准实时的数据拷贝；也可以通过 mysqldump 工具将源库的数据导出，再导入到新库，**这有什么复杂的呢?**

其实，这两种方式只能支持单库到单库的迁移，无法支持单库到多库多表的场景。而且即便是单库到单库的迁移，迁移过程也需要满足以下几个目标：

迁移应该是在线的迁移，也就是在迁移的同时还会有数据的写入；

数据应该保证完整性，也就是说在迁移之后需要保证新的库和旧的库的数据是一致的；

迁移的过程需要做到可以回滚，这样一旦迁移的过程中出现问题，可以立刻回滚到源库，不会对系统的可用性造成影响。

如果你使用 Binlog 同步的方式，在同步完成后再修改代码，将主库修改为新的数据库，这样就不满足可回滚的要求，一旦迁移后发现问题，由于已经有增量的数据写入了新库而没有写入旧库，不可能再将数据库改成旧库。

一般来说，我们有两种方案可以做数据库的迁移。

38.1.1 “双写”方案

第一种方案我称之为双写，其实说起来也很简单，它可以分为以下几个步骤：

\1. 将新的库配置为源库的从库，用来同步数据；如果需要将数据同步到多库多表，那么可以使用一些第三方工具获取 Binlog 的增量日志（比如开源工具 Canal），在获取增量日志之后就可以按照分库分表的逻辑写入到新的库表中了。

\2. 同时，我们需要改造业务代码，在数据写入的时候，不仅要写入旧库，也要写入新库。当然，基于性能的考虑，我们可以异步地写入新库，只要保证旧库写入成功即可。**但是，我们需要注意的是，**需要将写入新库失败的数据记录在单独的日志中，这样方便后续对这些数据补写，保证新库和旧库的数据一致性。

\3. 然后，我们就可以开始校验数据了。由于数据库中数据量很大，做全量的数据校验不太现实。你可以抽取部分数据，具体数据量依据总体数据量而定，只要保证这些数据是一致的就可以。

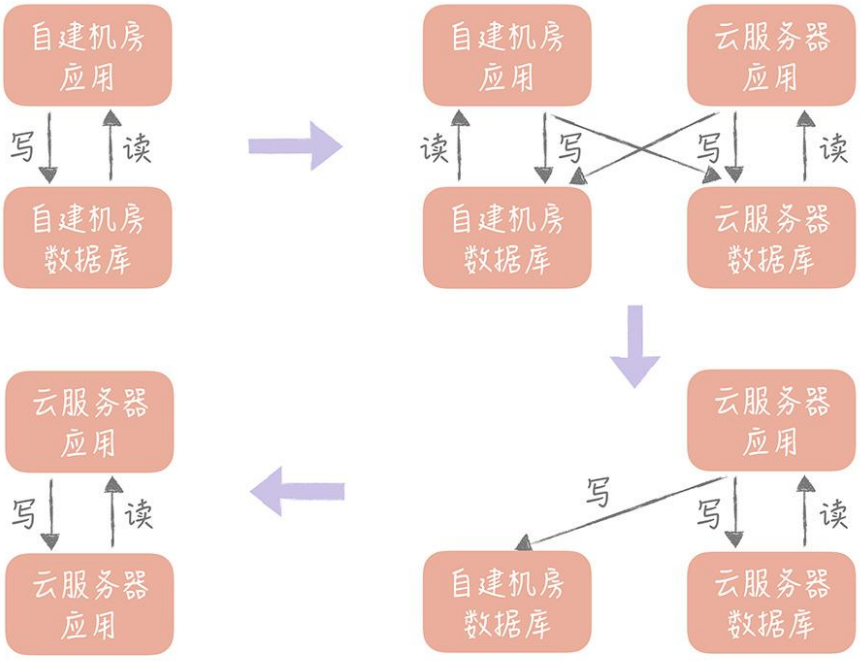
\4. 如果一切顺利，我们就可以将读流量切换到新库了。由于担心一次切换全量读流量可能会对系统产生未知的影响，所以这里**最好采用灰度的方式来切换，**比如开始切换 10% 的流量，如果没有问题再切换到 50% 的流量，最后再切换到 100%。

\5. 由于有双写的存在，所以在切换的过程中出现任何的问题，都可以将读写流量随时切换到旧库去，保障系统的性能。

\6. 在观察了几天发现数据的迁移没有问题之后，就可以将数据库的双写改造成只写新库，数据的迁移也就完成了。

其中，最容易出问题的步骤就是数据校验的工作，所以，我建议你在未开始迁移数据之前先写好数据校验的工具或者脚本，在测试环境上测试充分之后，再开始正式的数据迁移。

如果是将数据从自建机房迁移到云上，你也可以使用这个方案，**只是你需要考虑的一个重要的因素是：**自建机房到云上的专线的带宽和延迟，你需要尽量减少跨专线的读操作，所以在切换读流量的时候，你需要保证自建机房的应用服务器读取本机房的数据库，云上的应用服务器读取云上的数据库。这样在完成迁移之前，只要将自建机房的应用服务器停掉，并且将写入流量都切到新库就可以了。



数据从自建机房迁移到云上的方案

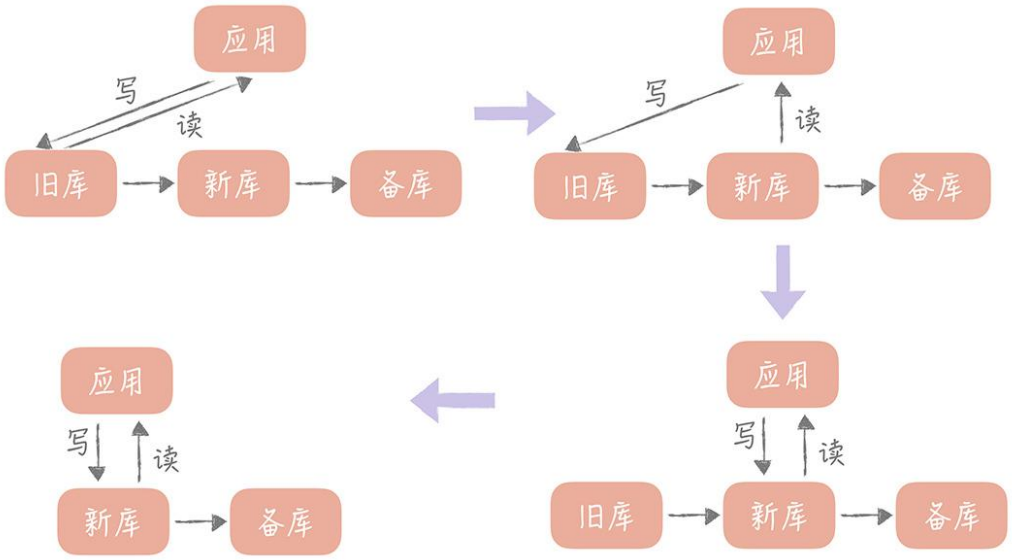
这种方案是一种比较通用的方案，无论是迁移 MySQL 中的数据，还是迁移 Redis 中的数据，甚至迁移消息队列都可以使用这种方式，**你在实际的工作中可以直接拿来使用。**这种方式的**好处是：**迁移的过程可以随时回滚，将迁移的风险降到了最低。**劣势是：**时间周期比较长，应用有改造的成本。

38.1.2 级联同步方案

这种方案也比较简单，比较适合数据从自建机房向云上迁移的场景。因为迁移上云，最担心云上的环境和自建机房的环境不一致，会导致数据库在云上运行时，因为参数配置或者硬件环境不同出现问题。

所以，我们会在自建机房准备一个备库，在云上环境上准备一个新库，通过级联同步的方式在自建机房留下一个可回滚的数据库，具体的步骤如下：

1. 先将新库配置为旧库的从库，用作数据同步；
2. 再将一个备库配置为新库的从库，用作数据的备份；
3. 等到三个库的写入一致后，将数据库的读流量切换到新库；
4. 然后暂停应用的写入，将业务的写入流量切换到新库（由于这里需要暂停应用的写入，所以需要安排在业务的低峰期）。



级联迁移方案示意图

这种方案的回滚方案也比较简单，可以先将读流量切换到备库，再暂停应用的写入，将写流量切换到备库，这样所有的流量都切换到了备库，也就是又回到了自建机房的环境，就可以认为已经回滚了。

上面的级联迁移方案可以应用在，将 MySQL 从自建机房迁移到云上的场景，也可以应用在将 Redis 从自建机房迁移到云上的场景，**如果你有类似的需求可以直接拿来应用。**

这种方案**优势是**简单易实施，在业务上基本没有改造的成本；**缺点是**在切写的时候需要短暂的停止写入，对于业务来说是有损的，不过如果在业务低峰期来执行切写，可以将对业务的影响降至最低。

38.2 数据迁移时如何预热缓存

另外，在从自建机房向云上迁移数据时，我们也需要考虑缓存的迁移方案是怎样的。那么你可能会说：缓存本来就是作为一个中间的存储而存在的，我只需要在云上部署一个空的缓存节点，云上的请求也会穿透到云上的数据库，然后回种缓存，对于业务是没有影响的。

你说的没错，但是你还需要考虑的是缓存的命中率。

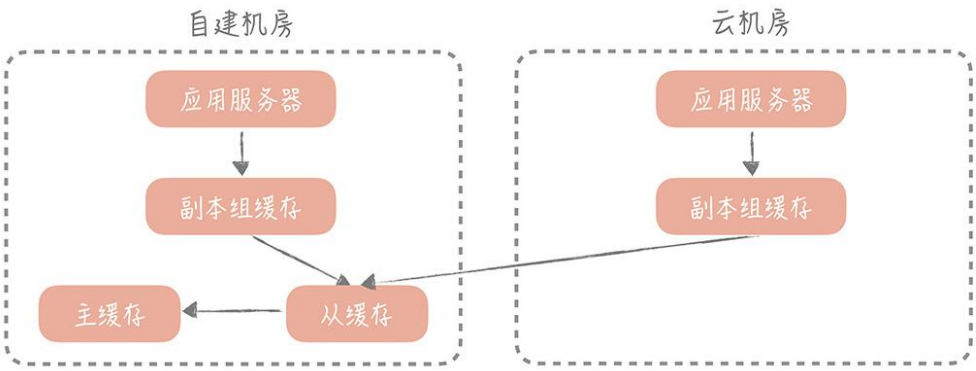
如果你部署一个空的缓存，那么所有的请求就都穿透到数据库，数据库可能因为承受不了这么大的压力而宕机，这样你的服务就会不可用了。**所以，缓存迁移的重点是保持缓存的热度。**

刚刚我提到，Redis 的数据迁移可以使用双写的方案或者级联同步的方案，所以在这里我就不考虑 Redis 缓存的同步了，而是以 Memcached 为例来说明。

38.2.1 使用副本组预热缓存

在“缓存的使用姿势（二）：缓存如何做到高可用？”中，我曾经提到，为了保证缓存的可用性，我们可以部署多个副本组来尽量将请求阻挡在数据库层之上。

数据的写入流程是写入 Master、Slave 和所有的副本组，而在读取数据的时候，会先读副本组的数据，如果读取不到再到 Master 和 Slave 里面加载数据，再写入到副本组中。**那么，我们就可以在云上部署一个副本组，**这样，云上的应用服务器读取云上的副本组，如果副本组没有查询到数据，就可以从自建机房部署的主从缓存上加载数据，回种到云上的副本组上。



副本组迁移方式示意图

当云上部署的副本组足够热之后，也就是缓存的命中率达到至少 90%，就可以将云机房上的缓存服务器的主从都指向这个副本组，这时迁移也就完成了。

这种方式足够简单，不过有一个致命的问题是：如果云上的请求穿透云上的副本组，到达自建机房的主从缓存时，这个过程是需要跨越专线的。

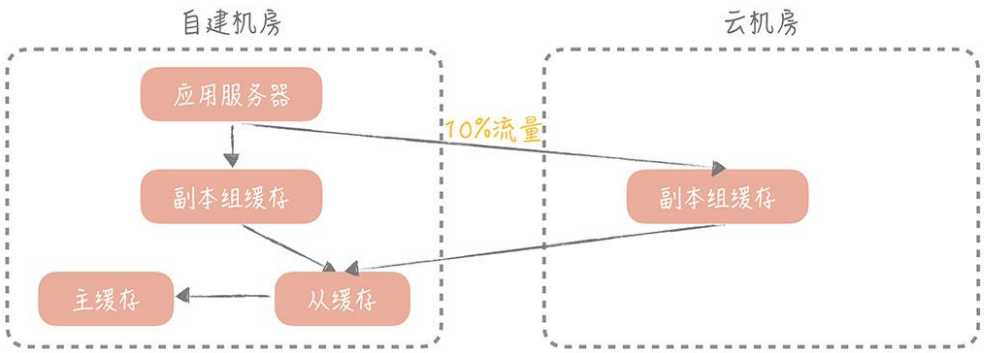
这不仅会占用较多专线的带宽，同时专线的延迟相比于缓存的读取时间是比较大的，一般，即使是本地的不同机房之间的延迟也会达到 2ms～3ms，那么，一次前端请求可能会访问十几次甚至几十次的缓存，一次请求就会平白增加几十毫秒甚至过百毫秒的延迟，会极大地影响接口的响应时间，因此在实际项目中我们很少使用这种方案。

但是，这种方案给了我们思路，让我们可以通过方案的设计在系统运行中自动完成缓存的预热，所以，我们对副本组的方案做了一些改造，以尽量减少对专线带宽的占用。

38.2.2 改造副本组方案预热缓存

改造后的方案对读写缓存的方式进行改造，步骤是这样的：

1. 在云上部署多组 mc 的副本组，自建机房在接收到写入请求时，会优先写入自建机房的缓存节点，异步写入云上部署的 mc 节点；
2. 在处理自建机房的读请求时，会指定一定的流量，比如 10%，优先走云上的缓存节点，这样虽然也会走专线穿透回自建机房的缓存节点，但是流量是可控的；
3. 当云上缓存节点的命中率达到 90% 以上时，就可以在云上部署应用服务器，让云上的应用服务器完全走云上的缓存节点就可以了。

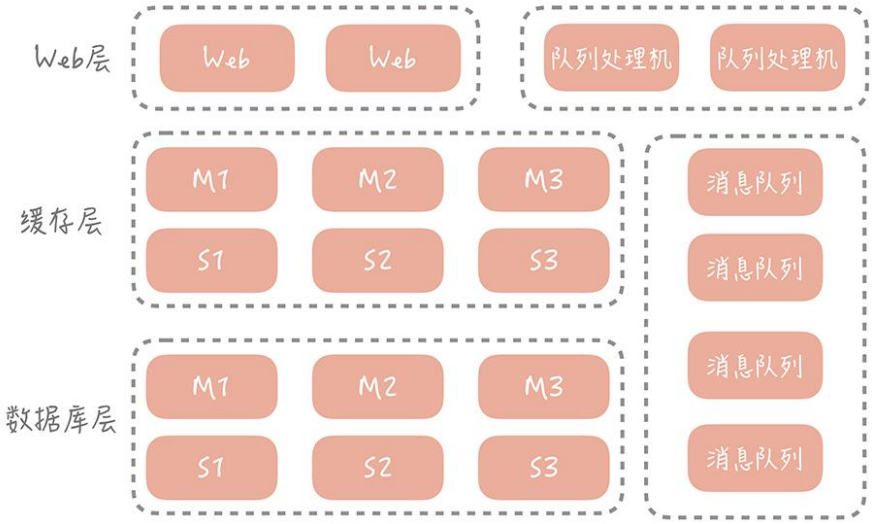


改进后的副本组迁移方式示意图

使用了这种方式，我们可以实现缓存数据的迁移，又可以尽量控制专线的带宽和请求的延迟情况，**你也可以直接在项目中使用。**

39. 如何确保你的消息只被消费一次?

假设存在如下这样一个架构，里面用到了很多消息队列来处理消息、业务逻辑。



系统架构图

这是一个简化版的架构图，实际上，随着业务逻辑越来越复杂，会引入更多的外部系统和服务来解决业务上的问题。比如说，我们会引入 Elasticsearch 来解决商品和店铺搜索的问题，也会引入审核系统，来对售卖的商品、用户的评论做自动的和人工的审核，你会越来越多地使用消息队列与外部系统解耦合，以及提升系统性能。

比如说，你的电商系统需要上一个新的红包功能：用户在购买一定数量的商品之后，由你的系统给用户发一个现金的红包，鼓励用户消费。由于发放红包的过程不应该在购买

商品的主流程之内，所以你考虑使用消息队列来异步处理。**这时，你发现了一个问题：**如果消息在投递的过程中发生丢失，那么用户就会因为没有得到红包而投诉。相反，如果消息在投递的过程中出现了重复，那么你的系统就会因为发送两个红包而损失。

那么我们如何保证，产生的消息一定会被消费到，并且只被消费一次呢？这个问题虽然听起来很浅显，很好理解，但是实际上却藏着很多玄机

39.1 消息为什么会丢失

如果要保证消息只被消费一次，首先就要保证消息不会丢失。那么消息从被写入到消息队列，到被消费者消费完成，这个链路上会有哪些地方存在丢失消息的可能呢？其实，主要存在三个场景：

消息从生产者写入到消息队列的过程。

消息在消息队列中的存储场景。

消息被消费者消费的过程。

消息丢失示意图

接下来，我就针对每一个场景，详细地剖析一下，这样你可以针对不同的场景选择合适的，减少消息丢失的解决方案。

39.1.1 在消息生产的过程中丢失消息

在这个环节中主要有两种情况。

首先，消息的生产者一般是我们的业务服务器，消息队列是独立部署在单独的服务器上的。两者之间的网络虽然是内网，但是也会存在抖动的可能，而一旦发生抖动，消息就有可能因为网络的错误而丢失。

针对这种情况，我建议你采用的方案是消息重传：也就是当你发现发送超时后你就将消息重新发一次，但是你也不能无限制地重传消息。一般来说，如果不是消息队列发生故障，或者是到消息队列的网络断开了，重试 2～3 次就可以了。

不过，这种方案可能会造成消息的重复，从而导致在消费的时候会重复消费同样的消息。比方说，消息生产时由于消息队列处理慢或者网络的抖动，导致虽然最终写入消息队列成功，但在生产端却超时了，生产者重传这条消息就会形成重复的消息，那么针对上面的例子，直观显示在你面前的就会是你收到了两个现金红包。

那么消息发送到了消息队列之后是否就万无一失了呢？当然不是，**在消息队列中消息仍然有丢失的风险。**

39.1.2 在消息队列中丢失消息

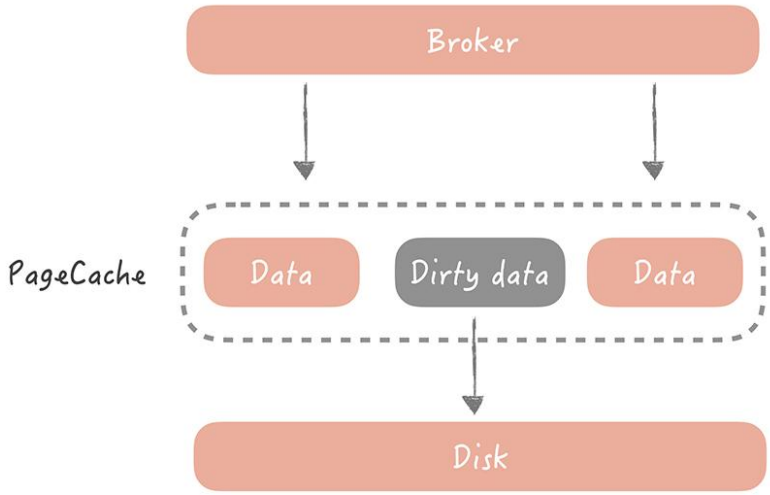
拿 Kafka 举例，消息在 Kafka 中是存储在本地磁盘上的，而为了减少消息存储时对磁盘的随机 I/O，我们一般会将消息先写入到操作系统的 Page Cache 中，然后再找合适的时机刷新到磁盘上。

比如，Kafka 可以配置当达到某一时间间隔，或者累积一定的消息数量的时候再刷盘，**也就是所说的异步刷盘。**

来看一个形象的比喻：假如你经营一个图书馆，读者每还一本书你都要去把图书归位，不仅工作量大而且效率低下，但是如果你可以选择每隔 3 小时，或者图书达到一定数量的时候再把图书归位，这样可以把同一类型的书一起归位，节省了查找图书位置的时间，这样就可以提高效率了。

不过，如果发生机器掉电或者机器异常重启，那么 Page Cache 中还没有来得及刷盘的消息就会丢失了。**那么怎么解决呢?**

你可能会把刷盘的间隔设置很短，或者设置累积一条消息就就刷盘，但这样频繁刷盘会对性能有比较大的影响，而且从经验来看，出现机器宕机或者掉电的几率也不高，**所以我不建议你这样做。**



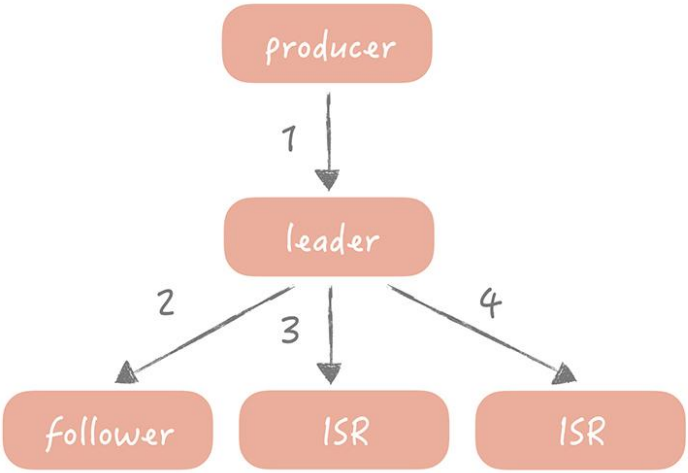
消息刷盘示意图

如果你的电商系统对消息丢失的容忍度很低，**那么你可以考虑以集群方式部署 Kafka 服务，通过部署多个副本备份数据，保证消息尽量不丢失。**

那么它是怎么实现的呢？

Kafka 集群中有一个 Leader 负责消息的写入和消费，可以有多个 Follower 负责数据的备份。Follower 中有一个特殊的集合叫做 ISR (in-sync replicas)，当 Leader 故障时，新选举出来的 Leader 会从 ISR 中选择，默认 Leader 的数据会异步地复制给 Follower，这样在 Leader 发生掉电或者宕机时，Kafka 会从 Follower 中消费消息，减少消息丢失的可能。

由于默认消息是异步地从 Leader 复制到 Follower 的，所以一旦 Leader 宕机，那些还没有来得及复制到 Follower 的消息还是会丢失。为了解决这个问题，Kafka 为生产者提供一个选项叫做“acks”，当这个选项被设置为“all”时，生产者发送的每一条消息除了发给 Leader 外还会发给所有的 ISR，并且必须得到 Leader 和所有 ISR 的确认后才被认为发送成功。这样，只有 Leader 和所有的 ISR 都挂了，消息才会丢失。



Kafka复制示意图

从上面这张图来看，当设置“acks=all”时，需要同步执行 1，3，4 三个步骤，对于消息生产的性能来说也是有比较大的影响的，所以你在实际应用中需要仔细地权衡考量。**我给你的建议是：**

\1. 如果你需要确保消息一条都不能丢失，那么建议不要开启消息队列的同步刷盘，而是需要使用集群的方式来解决，可以配置当所有 ISR Follower 都接收到消息才返回成功。

\2. 如果对消息的丢失有一定的容忍度，那么建议不部署集群，即使以集群方式部署，也建议配置只发送给一个 Follower 就可以返回成功了。

\3. 我们的业务系统一般对于消息的丢失有一定的容忍度，比如说以上面的红包系统为例，如果红包消息丢失了，我们只要后续给没有发送红包的用户补发红包就好了。

39.1.3 在消费的过程中存在消息丢失的可能

我还是以 Kafka 为例来说明。一个消费者消费消息的进度是记录在消息队列集群中的，而消费的过程分为三步：接收消息、处理消息、更新消费进度。

这里面接收消息和处理消息的过程都可能会发生异常或者失败，比如说，消息接收时网络发生抖动，导致消息并没有被正确的接收到；处理消息时可能发生一些业务的异常导致处理流程未执行完成，这时如果更新消费进度，那么这条失败的消息就永远不会被处理了，也可以认为是丢失了。

所以，在这里你需要注意的是，一定要等到消息接收和处理完成后才能更新消费进度，但是这也会造成消息重复的问题，比方说某一条消息在处理之后，消费者恰好宕机了，那么因为没有更新消费进度，所以当这个消费者重启之后，还会重复地消费这条消息。

39.2 如何保证消息只被消费一次

从上面的分析中，你能发现，为了避免消息丢失，我们需要付出两方面的代价：一方面是性能的损耗；一方面可能造成消息重复消费。

性能的损耗我们还可以接受，因为一般业务系统只有在写请求时才会有发送消息队列的操作，而一般系统的写请求的量级并不高，但是消息一旦被重复消费，就会造成业务逻辑处理的错误。那么我们要如何避免消息的重复呢？

想要完全的避免消息重复的发生是很难做到的，因为网络的抖动、机器的宕机和处理的异常都是比较难以避免的，在工业上并没有成熟的方法，因此我们会把要求放宽，只要保证即使消费到了重复的消息，从消费的最终结果来看和只消费一次是等同的就好了，也就是保证在消息的生产和消费的过程是“幂等”的。

39.2.1 什么是幂等

幂等是一个数学上的概念，它的含义是多次执行同一个操作和执行一次操作，最终得到的结果是相同的，说起来可能有些抽象，我给你举个例子：

比如，男生和女生吵架，女生抓住一个点不放，传递“你不在乎我了吗？”（生产消息）的信息。那么当多次埋怨“你不在乎我了吗？”的时候（多次生产相同消息），她不知道的是，男生的耳朵（消息处理）会自动把 N 多次的信息屏蔽，就像只听到一次一样，这就是幂等性。

如果我们消费一条消息的时候，要给现有的库存数量减 1，那么如果消费两条相同的消息就会给库存数量减 2，这就不是幂等的。而如果消费一条消息后，处理逻辑是将库存的数量设置为 0，或者是如果当前库存数量是 10 时则减 1，这样在消费多条消息时，所得到的结果就是相同的，**这就是幂等的。**

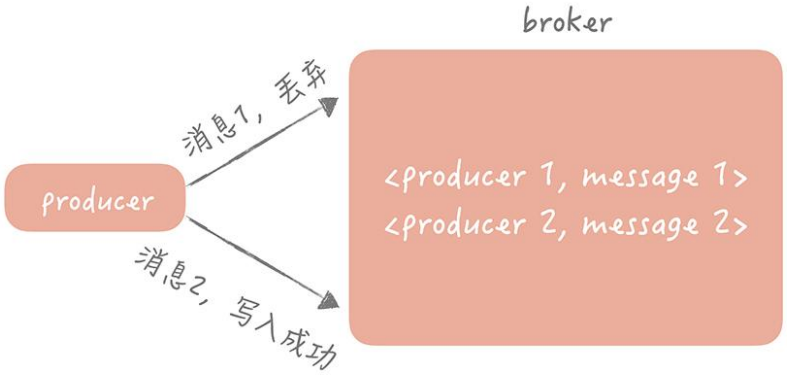
说白了，你可以这么理解“幂等”：一件事儿无论做多少次都和做一次产生的结果是一样的，那么这件事儿就具有幂等性。

39.2.2 在生产、消费过程中增加消息幂等性的保证

消息在生产和消费的过程中都可能会产生重复，所以你要做的是，在生产过程和消费过程中增加消息幂等性的保证，这样就可以认为从“最终结果上来看”，消息实际上是只被消费了一次的。

在消息生产过程中，在 Kafka0.11 版本和 Pulsar 中都支持“producer idempotency”的特性，翻译过来就是生产过程的幂等性，这种特性保证消息虽然可能在生产端产生重复，但是最终在消息队列存储时只会存储一份。

它的做法是给每一个生产者一个唯一的 ID，并且为生产的每一条消息赋予一个唯一 ID，消息队列的服务端会存储 < 生产者 ID，最后一条消息 ID> 的映射。当某一个生产者产生新的消息时，消息队列服务端会比对消息 ID 是否与存储的最后一条 ID 一致，如果一致，就认为是重复的消息，服务端会自动丢弃。



Producer idempotency示意图

而在消费端，幂等性的保证会稍微复杂一些，你可以从通用层和业务层两个层面来考虑。

在通用层面，你可以在消息被生产的时候，使用发号器给它生成一个全局唯一的消息 ID，消息被处理之后，把这个 ID 存储在数据库中，在处理下一条消息之前，先从数据库里面查询这个全局 ID 是否被消费过，如果被消费过就放弃消费。

你可以看到，无论是生产端的幂等性保证方式，还是消费端通用的幂等性保证方式，它们的共同特点都是为每一个消息生成一个唯一的 ID，然后在使用这个消息的时候，先比对这个 ID 是否已经存在，如果存在，则认为消息已经被使用过。所以这种方式是一种标准的实现幂等的方式，**你在项目之中可以拿来直接使用，**它在逻辑上的伪代码就像下面这样：

```
booleanisIDExisted= selectByID(ID); // 判断 ID 是否存在 if(isIDExisted) {
    return; // 存在则直接返回
} else {
    process(message); // 不存在，则处理消息
    saveID(ID);    // 存储 ID
}
```

不过这样会有一个问题：如果消息在处理之后，还没有来得及写入数据库，消费者宕机了重启之后发现数据库中并没有这条消息，还是会重复执行两次消费逻辑，这时你就需要引入事务机制，保证消息处理和写入数据库必须同时成功或者同时失败，但是这样消息处理的成本就更高了，所以，如果对于消息重复没有特别严格的要求，可以直接使用这种通用的方案，而不考虑引入事务。

在业务层面怎么处理呢？这里有很多种处理方式，其中有一种是增加乐观锁的方式。比如，你的消息处理程序需要给一个人的账号加钱，那么你可以通过乐观锁的方式来解决。

具体的操作方式是这样的：你给每个人的账号数据中增加一个版本号的字段，在生产消息时先查询这个账户的版本号，并且将版本号连同消息一起发送给消息队列。消费端在拿到消息和版本号后，在执行更新账户金额 SQL 的时候带上版本号，类似于执行：

```
update user set amount = amount + 20, version=version+1 where userId=1 and version=1;
```

你看，我们在更新数据时给数据加了乐观锁，这样在消费第一条消息时，version 值为 1，SQL 可以执行成功，并且同时把 version 值改为了 2；在执行第二条相同的消息时，由于 version 值不再是 1，所以这条 SQL 不能执行成功，也就保证了消息的幂等性。

40. 如何设计一个在生产环境下做全链路压测的平台

一旦你的系统流量有大的增长，比如类似“双十一”的流量，那么你在面临性能问题时就可能会手足无措。为了解决后顾之忧，你需要了解在流量增长若干倍的时候，系统的哪些组件或者服务会成为整体系统的瓶颈点，这时你就需要做一次全链路的压力测试。那么，什么是压力测试呢？要如何来做全链路的压测呢？

40.1 什么是压力测试

压力测试（简称为压测）这个名词儿，你在业界的分享中一定听过很多次，当然了，你也可能在项目的研发过程中做过压力测试，所以，对于你来说，压力测试并不陌生。

不过，我想让你回想一下，自己是怎么做压力测试的？是不是像很多同学一样：先搭建一套与正式环境功能相同的测试环境，并且导入或者生成一批测试数据，然后在另一台服务器，启动多个线程并发地调用需要压测的接口（接口的参数一般也会设置成相同的，比如，想要压测获取商品信息的接口，那么压测时会使用同一个商品 ID）。最后，通过统计访问日志，或者查看测试环境的监控系统，来记录最终压测 QPS 是多少之后，直接交差？

这么做压力测试其实是不正确的，**错误之处主要有以下几点：**

1. 首先，做压力测试时，最好使用线上的数据和线上的环境，因为，你无法确定自己搭建的测试环境与正式环境的差异，是否会影响到压力测试的结果；
2. 其次，压力测试时不能使用模拟的请求，而是要使用线上的流量。你可以通过拷贝流量的方式，把线上流量拷贝一份到压力测试环境。因为模拟流量的访问模型，和线上流量相差很大，会对压力测试的结果产生比较大的影响。 比如，你在获取商品信息的时候，线上的流量会获取不同商品的数据，这些商品的数据有的命中了缓存，有的没有命中缓存。如果使用同一个商品 ID 来做压力测试，那么只有第一次请求没有命中缓存，而在请求之后会将数据库中的数据回种到缓存，后续的请求就一定会命中缓存了，这种压力测试的数据就不具备参考性了。
3. 不要从一台服务器发起流量，这样很容易达到这台服务器性能瓶颈，从而导致压力测试的 QPS 上不去，最终影响压力测试的结果。而且，为了尽量真实地模拟用户请求，我们倾向于把流量产生的机器，放在离用户更近的位置，比如放在 CDN 节点上。如果没有这个条件，那么可以放在不同的机房中，这样可以尽量保证压力测试结果的真实性。

之所以有很多同学出现这个问题，主要是对压力测试的概念没有完全理解，以为只要是使用多个线程并发的请求服务接口，就算是对接口进行压力测试了。

那么究竟什么是压力测试呢？压力测试指的是，在高并发大流量下，进行的测试，测试人员可以通过观察系统在峰值负载下的表现，从而找到系统中存在的性能隐患。

与监控一样，压力测试是一种常见的，发现系统中存在问题的方式，也是保障系统可用性和稳定性的重要手段。而在压力测试的过程中，我们不能只针对某一个核心模块来做压测，而需要将接入层、所有后端服务、数据库、缓存、消息队列、中间件以及依赖的第三方服务系统及其资源，都纳入压力测试的目标之中。因为，一旦用户的访问行为增加，包含上述组件服务的整个链路都会受到不确定的大流量的冲击，因此，它们都需要依赖压力测试来发现可能存在的性能瓶颈，**这种针对整个调用链路执行的压力测试也称为“全链路压测”。**

由于在互联网项目中，功能迭代的速度很快，系统的复杂性也变得越来越高，新增加的功能和代码很可能会成为新的性能瓶颈点。也许半年前做压力测试时，单台机器可以承担每秒 1000 次请求，现在很可能就承担每秒 800 次请求了。所以，压力测试应该作为系统稳定性保障的常规手段，周期性地进行。

但是，通常做一次全链路压力测试，需要联合 DBA、运维、依赖服务方、中间件架构等多个团队，一起协调进行，无论是人力成本还是沟通协调的成本都比较高。同时，在压力测试的过程中，如果没有很好的监控机制，那么还会对线上系统造成不利的影响。**为了解决这些问题，我们需要搭建一套自动化的全链路压测平台，来降低成本和风险。**

40.2 如何搭建全链路压测平台

搭建全链路压测平台，主要有两个关键点。

一点是流量的隔离。由于压力测试是在正式环境进行，所以需要区分压力测试流量和正式流量，这样可以针对压力测试的流量做单独的处理。

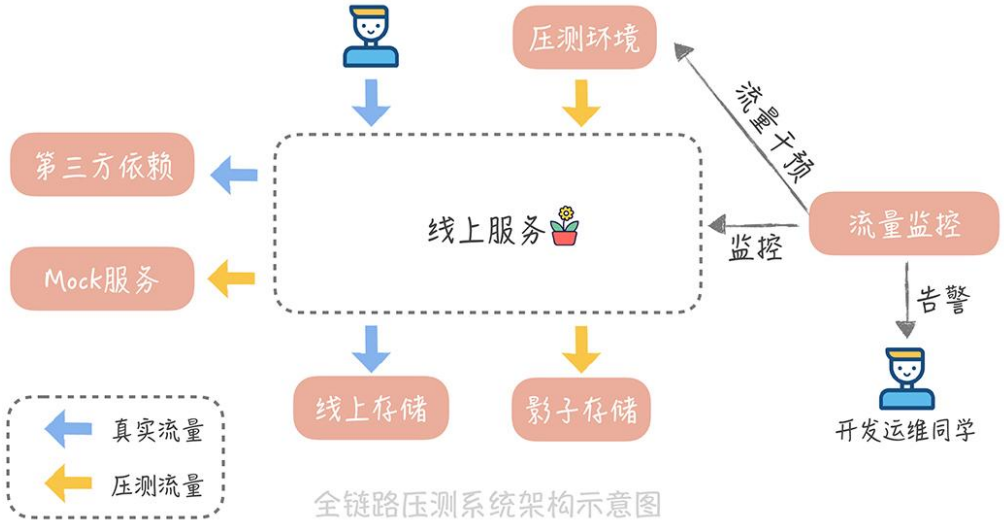
另一点是风险的控制。也就是，尽量避免压力测试对于正常访问用户的影响，因此，一般来说全链路压测平台需要包含以下几个模块：

流量构造和产生模块；

压测数据隔离模块；

系统健康度检查和压测流量干预模块。

整体压测平台的架构图可以是下面这样的：



为了让你能够更清晰地了解每一个模块是如何实现的，方便你来设计适合自身业务的全链路压测平台，我会对压测平台的每一个模块做更细致地介绍。先来看看压力测试的流量是如何产生的。

40.2.1 压测数据的产生

一般来说，我们系统的入口流量是来自于客户端的 HTTP 请求，所以，我们会考虑在系统高峰期时，将这些入口流量拷贝一份，在经过一些流量清洗的工作之后（比如过滤一些无效的请求），将数据存储在像是 HBase、MongoDB 这些 NoSQL 存储组件，或者亚马逊 S3 这些云存储服务中，我们称之为流量数据工厂。

这样，当我们要压测的时候，就可以从这个工厂中获取数据，将数据切分多份后下发到多个压测节点上了，**在这里，我想强调几个，你需要特殊注意的点。**

首先，我们可以使用多种方式来实现流量的拷贝。最简单的一种方式：直接拷贝负载均衡服务器的访问日志，数据就以文本的方式写入到流量数据工厂中，但是这样产生的数据在发起压测时，需要自己写解析的脚本来解析访问日志，会增加压测时候的成本，不太建议使用。

另一种方式：通过一些开源的工具来实现流量的拷贝。这里，我推荐一款轻型的流量拷贝工具 GoReplay，它可以劫持本机某一个端口的流量，将它们记录在文件中，传送到

流量数据工厂中。在压测时，你也可以使用这个工具进行加速的流量回放，这样就可以实现对正式环境的压力测试了。

其次，如上文中提到的，我们在下发压测流量时，需要保证下发流量的节点与用户更加接近，起码不能和服务部署节点在同一个机房中，这样可以尽量保证压测数据的真实性。另外，我们还需要对压测流量染色，也就是增加压测标记。在实际项目中，我会在 HTTP 的请求头中增加一个标记项，比如说叫做 is stress test，在流量拷贝之后，批量在请求中增加这个标记项，再写入到数据流量工厂中。

40.2.2 数据如何隔离

将压测流量拷贝下来的同时，我们也需要考虑对系统做改造，以实现压测流量和正式流量的隔离，这样一来就会尽量避免压测对线上系统的影响，一般来说，我们需要做两方面的事情。

一方面，针对读取数据的请求（一般称之为下行流量），我们会针对某些不能压测的服务或者组件，做 Mock 或者特殊的处理。举个例子。

在业务开发中，我们一般会依据请求，记录用户的行为，比如，用户请求了某个商品的页面，我们会记录这个商品多了一次浏览的行为，这些行为数据会写入一份单独的大数据日志中，再传输给数据分析部门，形成业务报表给到产品或者老板做业务的分析决策。在压测的时候，肯定会增加这些行为数据，比如原本一天商品页面的浏览行为是一亿次，而压测之后变成了十亿次，这样就会对业务报表产生影响，影响后续的产品方向的决策。因此，我们对于这些压测产生的用户行为做特殊处理，不再记录到大数据日志中。

再比如，我们系统会依赖一些推荐服务，推荐一些你可能感兴趣的商品，但是这些数据的展示有一个特点就是，展示过的商品就不再会被推荐出来。如果你的压测流量经过这些推荐服务，大量的商品就会被压测流量请求到，线上的用户就不会再看到这些商品了，也就会影响推荐的效果。

所以，我们需要 Mock 这些推荐服务，让不带有压测标记的请求经过推荐服务，而让带有压测标记的请求经过 Mock 服务。搭建 Mock 服务，你需要注意一点：这些 Mock 服务最好部署在真实服务所在的机房，这样可以尽量模拟真实的服务部署结构，提高压测结果的真实性。

另一方面，针对写入数据的请求（一般称之为上行流量），我们会把压测流量产生的数据，写入到影子库，也就是和线上数据存储，完全隔离的一份存储系统中。针对不同的存储类型，我们会使用不同的影子库的搭建方式：

如果数据存储在 MySQL 中，我们可以在同一个 MySQL 实例，不同的 Schema 中创建一套和线上相同的库表结构，并且把线上的数据也导入进来。

而如果数据是放在 Redis 中，我们对压测流量产生的数据，增加一个统一的前缀，存储在同一份存储中。

还有一些数据会存储在 Elasticsearch 中，针对这部分数据，我们可以放在另外一个单独的索引表中。

通过对下行流量的特殊处理以及对上行流量增加影子库的方式，我们就可以实现压测流量的隔离了。

40.2.3 压力测试如何实施

在拷贝了线上流量和完成了对线上系统的改造之后，我们就可以进行压力测试的实施了。在此之前，一般会设立一个压力测试的目标，比如说，整体系统的 QPS 需要达到每秒 20 万。

不过，在压测时，不会一下子把请求量增加到每秒 20 万次，而是按照一定的步长（比如每次压测增加一万 QPS），逐渐地增加流量。在增加一次流量之后，让系统稳定运行一段时间，观察系统在性能上的表现。如果发现依赖的服务或者组件出现了瓶颈，可以先减少压测流量，比如，回退到上一次压测的 QPS，保证服务的稳定，再针对此服务或者组件进行扩容，然后再继续增加流量压测。

为了能够减少压力测试过程中，人力投入成本，可以开发一个流量监控的组件，在这个组件中，预先设定一些性能阈值。比如，容器的 CPU 使用率的阈值可以设定为 60%～70%；系统的平均响应时间的上限可以设定为 1 秒；系统慢请求的比例设置为 1% 等等。

当系统性能达到这个阈值之后，流量监控组件可以及时发现，并且通知压测流量下发组件减少压测流量，并且发送报警给到开发和运维的同学，开发和运维同学就迅速地排查性能瓶颈，在解决问题或者扩容之后再继续执行压测。

业界关于全链路压测平台的探索有很多，一些大厂比如阿里、京东、美团和微博都有了适合自身业务的全链路压测平台。在我看来，这些压测平台万变不离其宗，都无非是经过流量拷贝、流量染色隔离、打压、监控熔断等步骤，与本课程中介绍的核心思想都是相通的。因此，你在考虑自研适合自己项目的全链路压测平台时，也可以遵循这个成熟的套路。

41. [线上问题排查]MQ 消息队列消息堆积问题排查和解决思路

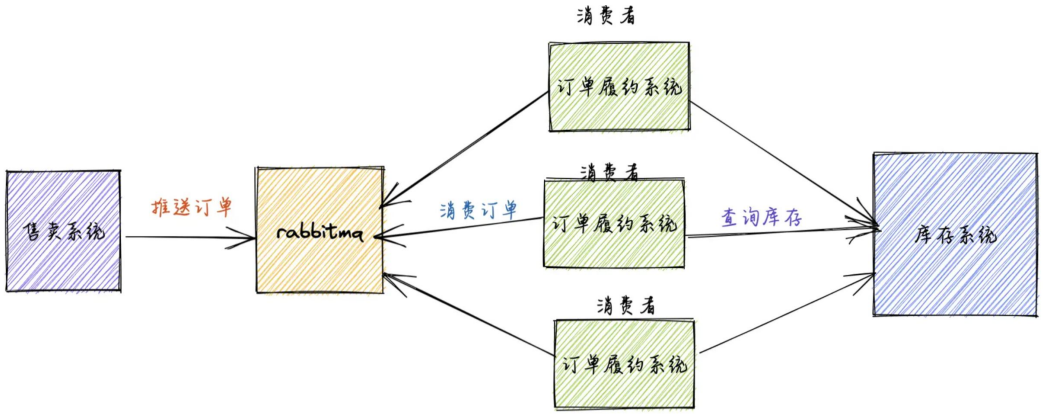
MQ 执行有三大阶段：

- 消息生产阶段。
- 消息存储阶段。
- 消息消费阶段。

很显然，消息堆积是出现在第三个消息消费阶段的。

消息积压问题的处理取决于消息积压的类型，例如，消息积压是突发性消息积压问题？还是缓慢持续增长的消息积压问题？不同的问题的解决方案略有不同，接下来我们一起来看。

41.1 系统架构描述



我们使用 rabbitmq 主要是为了系统解耦、异步提高系统的性能

前端售卖系统，生成订单后，推送订单消息到 rabbitmq，订单履约系统作为消费者，消费订单消息落库，做后续操作

41.2 排查以及解决

41.2.1 方案一 增加消费者

第一我们想到的原因，流量激增，生成的订单速度远远大于消费者消费消息的速度，目前我们只部署了三个节点，那我们是否增加消费者，就可以解决这个问题，让**消费者消费消息的速度远远大于生成者生成消息的速度**，那消息就不存在堆积的问题，自然服务器压力也就下来了

通知运维，再部署三个点，也是就增加三个消费者，由原来的三个消费者变为 6 个消费者，信心满满的部署完成后，等待一段时间，不出意外还是出了意外，**消息还是在持续堆积**，没有任何改善，我心里那个急啊，为什么增加了消费者？一点改善没有呢

41.2.2 方案二 优化消费者的处理逻辑

持续分析，是不是消费者的逻辑有问题，处理速度还是慢？在消费逻辑分析中，发现在处理订单消息的逻辑里，调用了库存系统的一个接口，有可能是这个接口响应慢，导致消费的速度慢，跟不上生产消息的速度。

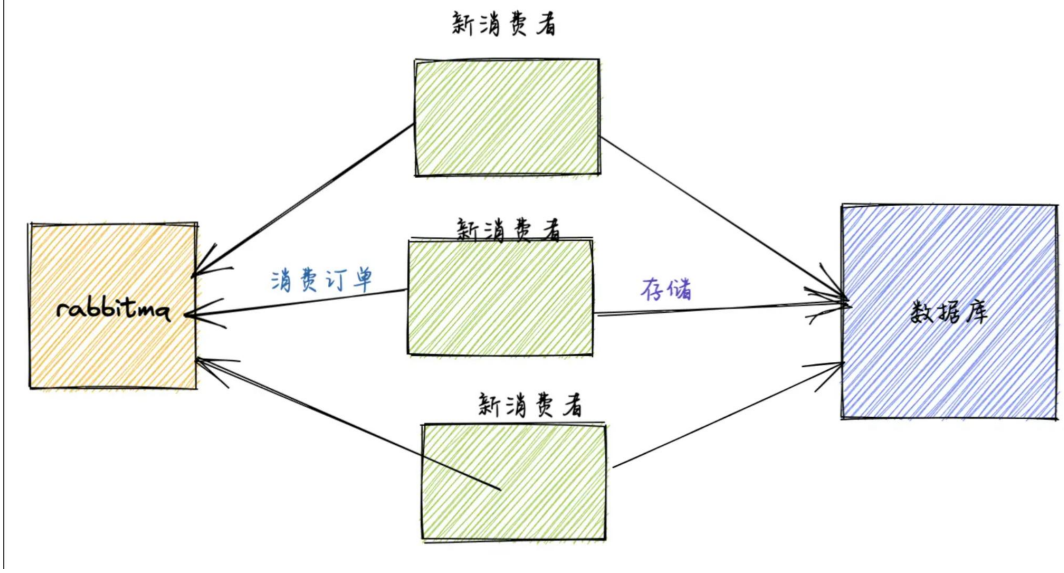
查看库存系统的运行情况，发现系统压力非常大，接口请求存在大量超时的情况，系统也在崩溃的边缘，因为我们上面的解决方案，增加了三个节点，间接的增大了并发。告知负责库存系统的同学，进行处理排查解决，但一时解决不了，如果持续这样，整体链路有可能全部崩掉，这怎么办呢?

消费者逻辑优化，屏蔽掉调用库存的接口，直接处理消息，但这种我们的逻辑是不完成，虽然能减少服务器的压力，后续处理起来也非常的麻烦，这种方式不可取

41.2.3 方案三 清空堆积的消息

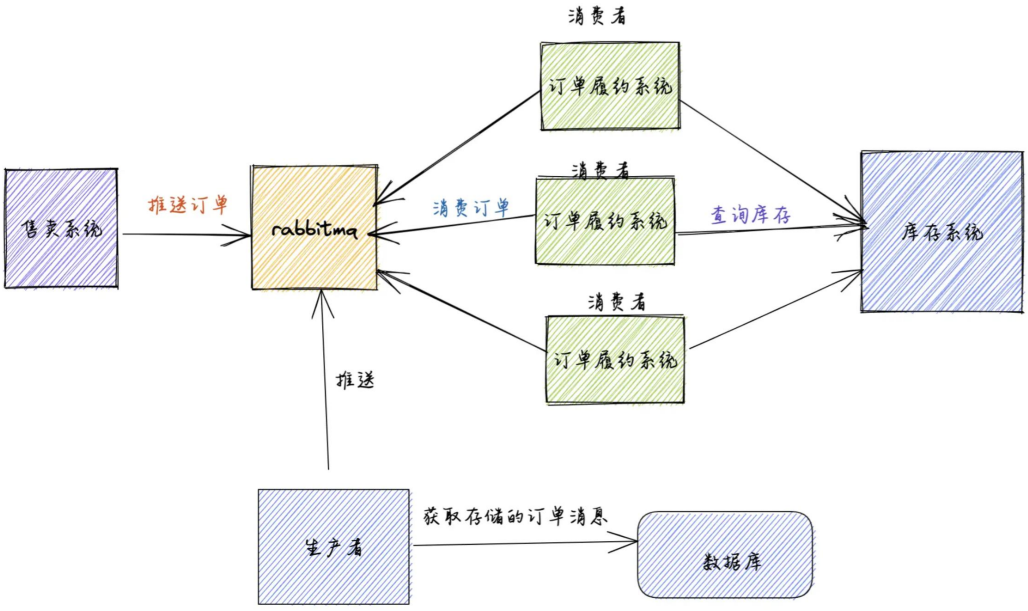
为了减少消息的堆积，减轻服务器的压力，我们是否可以把 mq 里面的消息拿出来，先存储，等服务恢复后，再把存储的消息推送到 mq，再处理呢?

新建消费者，消费 rabbitmq 的消息，不做任何业务逻辑处理，直接快速消费消息，把消息存在一张表里，这样就没消息的堆积，服务器压力自然就下来了。



这方案上线后，过了一段时间观察，消息不再堆积，服务器的负载也下来了，我内心也不再慌了，那存储的那些消息，还处理吗？当然处理，怎么处理呢？

后续等库存服务问题解决后，停掉新的消费者，新建一个生产者，再把表里的订单数据推送到 rabbitmq，进行业务逻辑的处理



至此，问题就完美的解决了，悬着的心也放下了

41.2.4 问题产生的原因分析

整个链路服务一直都是很稳定的，因为流量的激增，库存服务的服务能力跟不上，导致整个链路出了问题，如果平台要搞促销这种活动，我们还是要提前评估下系统的性能，对整个链路做一次压测，找出瓶颈，该优化的要优化，资源不足的加资源

41.3 消息堆积为什么会导致 cpu 飙升呢?

问题虽然解决了，但我很好奇，消息堆积为什么会导致 cpu 飙升呢?

RabbitMQ 是一种消息中间件，用于在应用程序之间传递消息。当消息堆积过多时，可能会导致 CPU 飙升的原因有以下几点：

1. 消息过多导致消息队列堆积：当消息的产生速度大于消费者的处理速度时，消息会积累在消息队列中。如果消息堆积过多，RabbitMQ 需要不断地进行消息的存储、检索和传递操作，这会导致 CPU 使用率升高。
2. 消费者无法及时处理消息：消费者处理消息的速度不足以追赶消息的产生速度，导致消息不断积累在队列中。这可能是由于消费者出现瓶颈，无法处理足够多的消息，或者消费者的处理逻辑复杂，导致消费过程耗费过多的 CPU 资源。
3. 消息重试导致额外的 CPU 开销：当消息处理失败时，消费者可能会进行消息的重试操作，尝试再次处理消息。如果重试频率较高，会导致消息在队列中频繁流转、被重复消费，这会增加额外的 CPU 开销。
4. 过多的连接以及网络 IO：当消息堆积过多时，可能会引发大量的连接请求和网络数据传输。这会增加网络 IO 的负载，并占用 CPU 资源。

41.4 通用的解决方案

- 增加消费者：通过增加消费者的数量来提升消息的处理能力。增加消费者可以分担消息消费的负载，缓解消息队列的堆积问题。
- 优化消费者的处理逻辑：检查消费者的代码是否存在性能瓶颈或是复杂的处理逻辑。可以通过优化算法、减少消费过程的计算量或是提高代码的效率来减少消费者的 CPU 开销。
- 避免频繁的消息重试：当消息无法处理时，可以根据错误类型进行不同的处理方式，如将无法处理的消息转移到死信队列中或进行日志记录。避免频繁地对同一消息进行重试，以减少额外的 CPU 开销。

- 调整 RabbitMQ 配置：可以调整 RabbitMQ 的参数来适应系统的需求，如增加内存、调整消息堆积的阈值和策略，调整网络连接等配置。
- 扩展硬件资源：如果以上措施无法解决问题，可能需要考虑增加 RabbitMQ 的集群节点或者扩容服务器的硬件资源，以提升整个系统的处理能力。

42. [线上问题排查]如果 JVM 出现频繁 FullGC 该如何解决

我们知道 Full GC 的触发条件大致情况有以下几种情况：

1. 程序执行了 System.gc() //建议 jvm 执行 fullgc，并不一定会执行
2. 执行了 jmap -histo:live pid 命令 //这个会立即触发 fullgc
3. 在执行 minor gc 的时候进行的一系列检查

> 执行 Minor GC 的时候，JVM 会检查老年代中最大连续可用空间是否大于了当前新生代所有对象的总大小。
> 如果大于，则直接执行 Minor GC（这个时候执行是没有风险的）。
> 如果小于了，JVM 会检查是否开启了空间分配担保机制，如果没有开启则直接改为执行 Full GC。
> 如果开启了，则 JVM 会检查老年代中最大连续可用空间是否大于了历次晋升到老年代中的平均大小，如果小于则执行改为执行 Full GC。
> 如果大于则会执行 Minor GC，如果 Minor GC 执行失败则会执行 Full GC

4. 使用了大对象 //大对象会直接进入老年代
5. 在程序中长期持有了对象的引用 //对象年龄达到指定阈值也会进入老年代

对于我们的情况，可以初步排除 1，2 两种情况，最有可能是 4 和 5 这两种情况。为了进一步排查原因，我们在线上开启了 -XX:+HeapDumpBeforeFullGC。

> 注意：
>
> JVM 在执行 dump 操作的时候是会发生 stop the word 事件的，也就是说此时所有的用户线程都会暂停运行。
>
> 为了在此期间也能对外正常提供服务，建议采用分布式部署，并采用合适的负载均衡算法

42.1 JVM 参数的设置

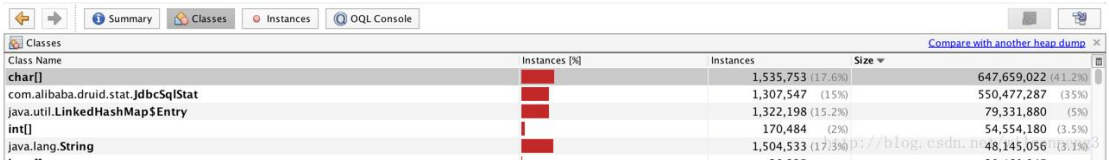
线上这个 dubbo 服务是分布式部署，在其中一台机子上开启了

-XX:HeapDumpBeforeFullGC，总体 JVM 参数如下：

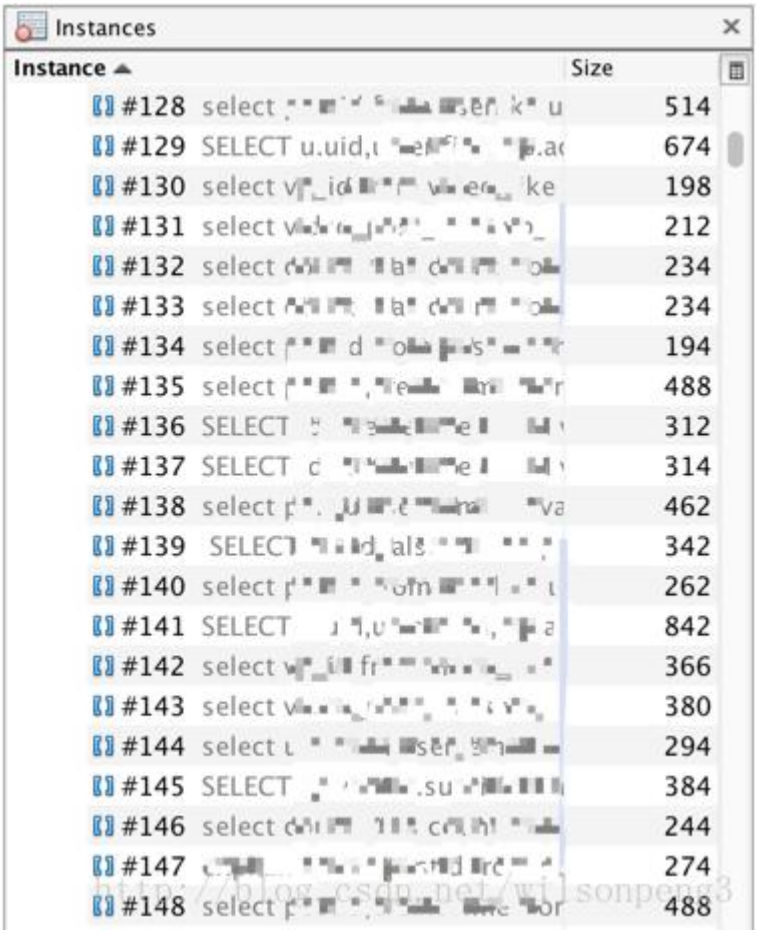
```
-Xmx2g
-XX:+HeapDumpBeforeFullGC
-XX:HeapDumpPath=.
-Xloggc:gc.log
-XX:+PrintGC
-XX:+PrintGCDetails
-XX:+PrintGCDateStamps
-XX:+UseGCLogFileRotation
-XX:NumberOfGCLogFiles=10
-XX:GCLogFileSize=100m
-XX:HeapDumpOnOutOfMemoryError
```

42.2 Dump 文件分析

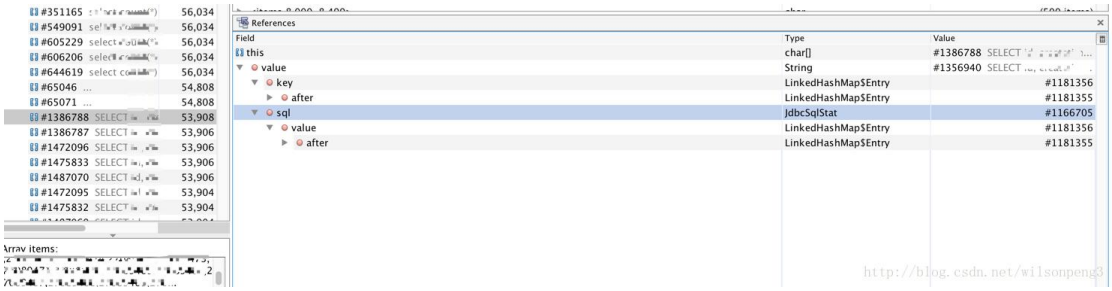
dump 下来的文件大约 1.8g，用 jvisualvm 查看，发现用 char[]类型的数据占用了 41%内存，同时另外一个 com.alibaba.druid.stat.JdbcSqlStat 类型的数据占用了 35%的内存，也就是说整个堆中几乎全是这两类数据。如下图：



查看 char[]类型数据，发现几乎全是 sql 语句



接下来查看 char[]的引用情况：



找到了 JdbcSqlStat 类，在代码中查看这个类的代码，关键代码如下：

```
1  //构造函数只有这一个
2  public JdbcSqlStat(String sql){
3          this.sql = sql;
4          this.id = DruidDriver.createSqlStatId();
5  }
6
7  //查看这个函数的调用情况，找到 com.alibaba.druid.stat.JdbcDataSourceStat#createSqlStat
   方法：
8
9  public JdbcSqlStat createSqlStat(String sql) {
10         lock.writeLock().lock();
11         try {
```

```
            JdbcSqlStat sqlStat = sqlStatMap.get(sql);
            if (sqlStat == null) {
                sqlStat = new JdbcSqlStat(sql);
                sqlStat.setDbType(this.dbType);
                sqlStat.setName(this.name);
                sqlStatMap.put(sql, sqlStat);
            }

            return sqlStat;
        } finally {
            lock.writeLock().unlock();
        }
    }

    //这里用了一个 map 来存放所有的 sql 语句。

```

其实到这里也就知道什么原因造成了这个问题，因为我们使用的数据源是阿里巴巴的 druid，这个 druid 提供了一个 sql 语句监控功能，同时我们也开启了这个功能。只需要在配置文件中把这个功能关掉应该就能消除这个问题，事实也的确如此，关掉这个功能后到目前为止线上没再触发 FullGC

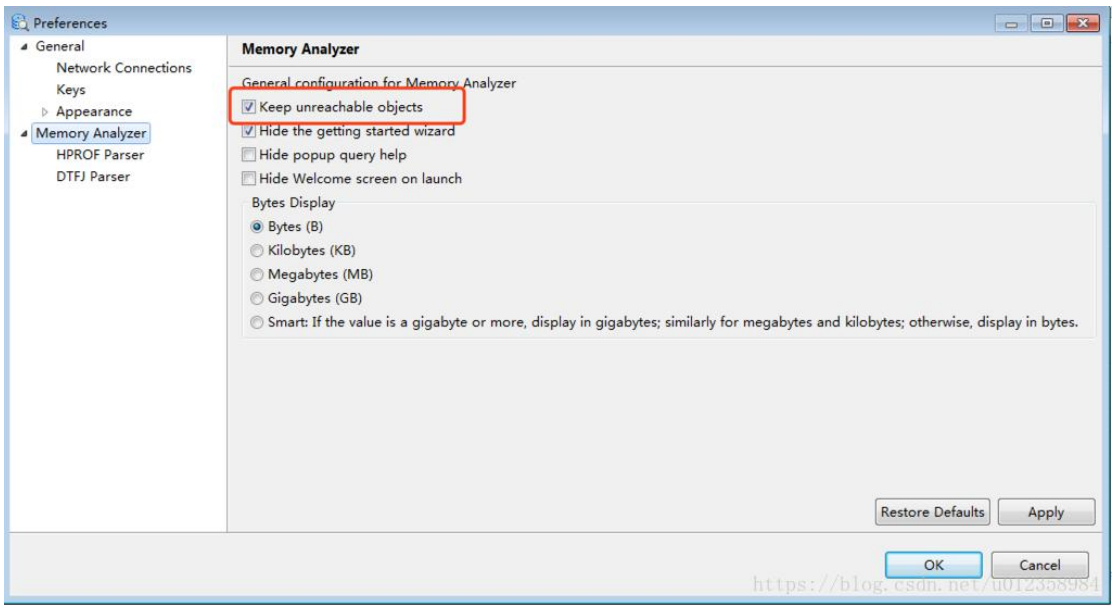
```
<bean id="readDataSource" class="com.alibaba.druid.pool.DruidDataSource" init-method="init" destroy-method="close">
    <!-- 数据源驱动类可不写，Druid默认会自动根据URL识别DriverClass -->
    <property name="driverClassName" value="#{app.driver}" />

    <!-- 基本属性 url、user、password -->
    <property name="url" value="#{app.slaveUrl}" />
    <property name="username" value="#{app.username}" />
    <property name="password" value="#{app.password}" />

    <!-- 配置初始化大小、最小、最大 -->
    <property name="initialSize" value="#{app.initialSize}" />
    <property name="minIdle" value="#{app.minIdle}" />
    <property name="maxActive" value="#{app.maxActive}" />

    <!-- 配置获取连接等待超时的时间 -->
    <property name="maxWait" value="60000" />

    <!-- 配置间隔多久才进行一次检测，检测需要关闭的空闲连接，单位是毫秒 -->
    <property name="timeBetweenEvictionRunsMillis" value="60000" />

    <!-- 配置一个连接在池中最小生存的时间，单位是毫秒 -->
    <property name="minEvictableIdleTimeMillis" value="300000" />

    <property name="validationQuery" value="SELECT 'x'" />
    <property name="testWhileIdle" value="true" />
    <property name="testOnBorrow" value="false" />
    <property name="testOnReturn" value="false" />

    <!-- 配置监控统计拦截的filters -->
    <!--<property name="filters" value="stat" />-->
</bean>
```

http://blog.csdn.net/wilsonpeng3

其他

如果用 mat 工具查看，建议把 “Keep unreachable objects” 勾上，否则 mat 会把堆中不可达的对象去除掉，这样我们的分析也许会变得没有意义。如下图：

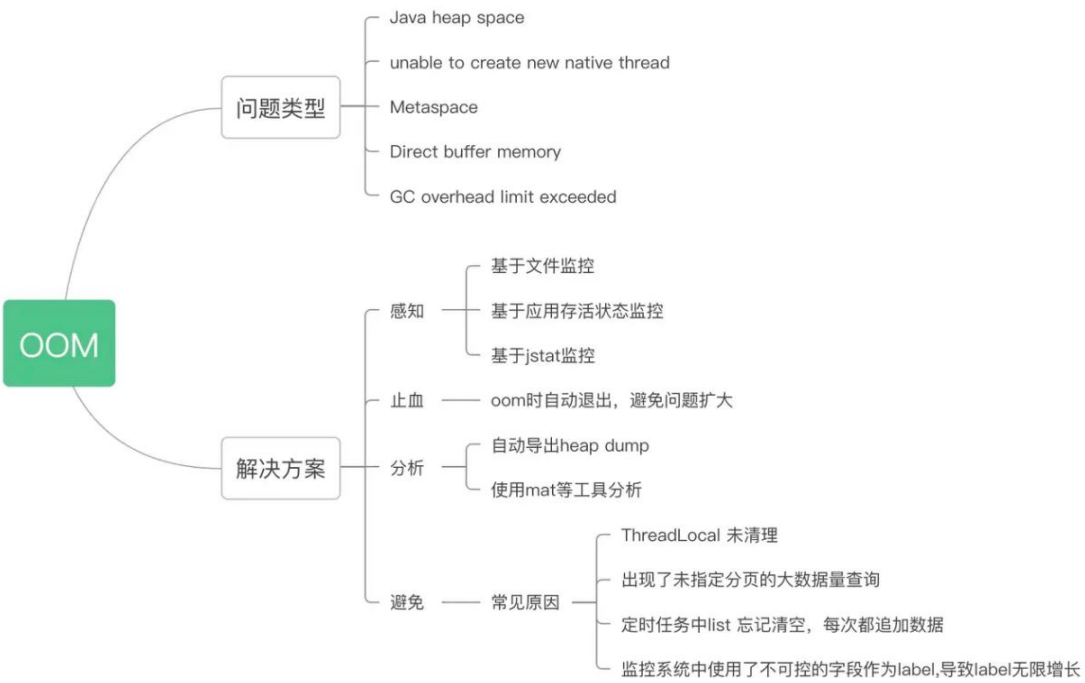
Window–>References 。另外 jvisualvm 对 ool 的支持不是很好，如果需要 oql 建议使用 mat。



43. [线上问题排查]JVM OOM 问题如何排查和解决

OutOfMemoryError (以下缩写为 oom)是 java 中最常见的内存问题，也是一旦影响就非常大的问题。oom 出现的原因就是内存不够用了，GC 虽然在回收，然后回收的速度赶不上新对象分配了或者根本就没有对象可以被回收，就会抛出 OutOfMemoryError 错误。

排查思路：问题发现-问题止血-问题分析-问题规避的闭环思路



五种常见的 oom 错误

- java.lang.OutOfMemoryError: Java heap space
- java.lang.OutOfMemoryError: unable to create new native thread
- java.lang.OutOfMemoryError: Metaspace
- java.lang.OutOfMemoryError: Direct buffer memory
- java.lang.OutOfMemoryError: GC overhead limit exceeded

43.1 通用手段

43.1.1 发现

通过以下手段可以做到感知应该出现了 oom.

1. 日志监控

通过监控日志中关键字 java.lang.OutOfMemoryError，就可以知道应用是否出现 oom.对于文件的监控，可以使用 filebeat 或者 flume 等采集.

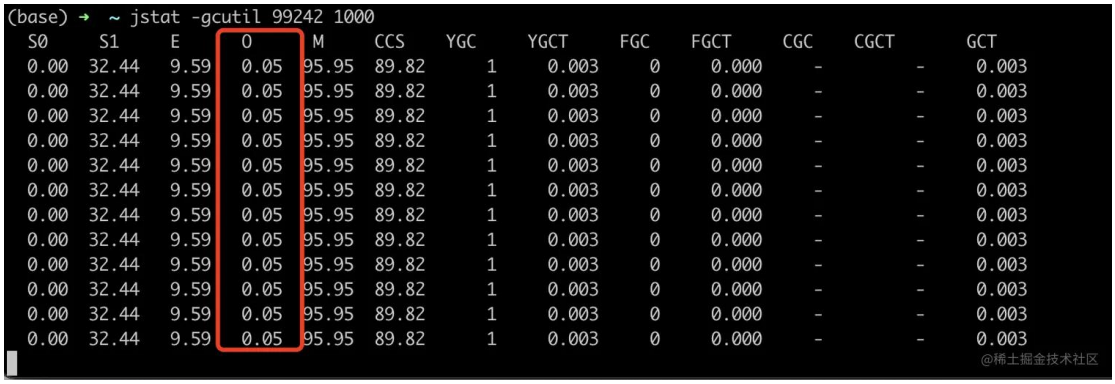
2. 配置-XX:+ExitOnOutOfMemoryError

当 jvm 启用该参数时，如果出现了 oom，就会自动退出程序，咱们的健康检测自然能发现应用不存在了，从而能发出告警

3. 使用 jstat 监控 jvm 内存使用率

通过 jstat 工具可以查看 jvm 的内存使用情况，如果老年代使用率一直是 100%，并且期间还在一直不断 GC，这也是发生了 oom 的一种现象。

`jstat -gcutil $pid 1000` 使用该命令即可实现每秒打印一次 jvm 的内存信息。



O 列代表老年代内存使用比例，YGC 和 FGC 分别代表新生代 GC 次数和老年代 GC 次数。

43.1.2 止血

当应用出现 oom 时，意味着应用无法申请到内存来分配对象，那么咱们的业务代码就会处于混沌状态，不清楚能进行到哪一步，不清楚状态是否完备，业务的请求可能在系统无限阻塞。这种情况对于用户体验的伤害会非常大。

所以这里有一个简单的方法就是，配置`-XX:+HeapDumpOnOutOfMemoryError`参数，当应用发生 oom 后，`自动`关停，借助 k8s 的健康检测实现`自动`重启能力再次拉起来新的容器。注意，这里需要实现业务系统能够实现故障转移或者请求幂等。这样才能保证用户请求不会出问题。

> 画外音:笔者把上面`自动`的地方都特殊标注了，因为自动化是效率之源，没有自动化，就得手工操作忙来忙去。

43.2 解决方案

对于 `unable to create new native thread` 、`Metaspace`、`Direct buffer memory` 和 `GC overhead limit exceeded`,它们的原因相对简单，可以直接给出结论。

- unable to create new native thread： 无法创建更多的操作系统线程，以下 3 种情况:

- 说明当前系统的线程数过多，超过了系统线程数的上限，减少当前应用创建的线程即可。
- 没有 native 内存了
- Metaspace：
- 说明 Metaspace 不够用，修改 `-XX:MaxMetaspaceSize` 启动参数，调大永久代空间即可。
- Direct buffer memory：
- Direct ByteBuffer 的默认大小为 64 MB，一旦使用超出限制，就会抛出 Directbuffer memory 错误。
- GC overhead limit exceeded：
- 当 jvm 98%的时间都在 GC 时，就会出现该异常。这种情况需要增大堆或者结合下面的方法继续排查。

以下是针对 `Java heap space` 类型的深度分析方案

43.2.1 分析

当应用发生 oom 后，最佳的分析载体就是 heap dump。通过添加 `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/tmp/app.dump` 参数，可以实现当应用发生 oom 时，全自动导出 heap dump。

有了 heap dump，我们就可以借助 Eclipse MAT 这款工具来分析 heap dump 了

43.2.2 规避

oom 发生的原因有两种:

1. 应用正常，但是堆设置过小，无法支持正常的对象分配
2. 应用发生了内存泄漏，导致应该被清理的对象没有清理，无限增长

通过分析 heap dump，我们基本可以区分出两种情况，第一种情况，对象的分布都正确，没有那种以下占用 30%*-，甚至 50%的对象。第二种情况就是某一种类型的对象，占据了大量的内存空间。

第一种情况的解决方案就是：

3. 增大堆内存
4. 优化应用内存使用效率

第二种情况，就需要针对性分析了，这里笔者给出常见的 oom 原因，大家在问题排查时，可以逐个排除，直到找到正确答案：

5. ThreadLocal 未清理
6. 出现了未指定分页的大数据量查询
7. 定时任务中 list 忘记清空，每次都追加数据
8. 监控系统中使用了不可控的字段作为 label,导致 label 无限增长

44. [线上问题排查]CPU 使用率较高排查和解决思路

解决问题的能力以超高比例排名第一，这也是为什么很多面试过程中，面试官都喜欢问如下问题：

> 1、你这个项目遇到的最大挑战是什么？如何解决的？
> 2、如果线上发生了报警你回如何排查呢？
> 3、你有解决过什么线上问题吗？
> 4、能列举几个你知道的排查 Linux 服务器线上问题的命令吗？

这些，都是比较常见的问题，还有一些比较具体的问题也是建议很多开发者都需要掌握的，如：

> 1、线上服务器 Load 飙高如何排查？
> 2、线上服务器 CPU 占用率高如何排查？
> 3、线上服务器频繁发生 Full GC 如何排查？
> 4、线上服务器发生死锁如何排查？

这些问题的回答，一方面考察了面试者是否具有很强的实战经验，另外一方面也能体现出其解决问题的能力。

毋庸置疑，作为开发人员来说，定位并解决问题的能力是至关重要的。因为一旦线上发生了问题，如 CPU 占用率高，如果不及时解决，很容易导致网站响应慢、服务器宕机等问题。

那么，书归正传，本文我们就来简单介绍一下，如果线上服务器发生 CPU 占用率过高的问题时，应该如何排查并定位问题。

44.1 问题发现

本文整理自一个真实的案例，是楼主负责的业务，在一次大促之前的压测时发现了这个问题。

在每次大促之前，我们的测试人员都会对网站进行压力测试，这个时候会查看服务的 cpu、内存、load、rt、qps 等指标。

在一次压测过程中，测试人员发现我们的某一个接口，在 qps 上升到 500 以后，CPU 使用率急剧升高。

> CPU 利用率，又称 CPU 使用率。顾名思义，CPU 利用率是来描述 CPU 的使用情况的，表明了一段时间内 CPU 被占用的情况。使用率越高，说明你的机器在这个时间上运行了很多程序，反之较少。

44.2 问题定位

遇到这种问题，首先是登录到服务器，看一下具体情况。

44.2.1 定位进程

登录服务器，执行 top 命令，查看 CPU 占用情况：

```
$top
  PID  USER      PR  NI    VIRT    RES    SHR S %CPU %MEM     TIME+
COMMAND
 1893 admin     20   0 7127m 2.6g   38m S 181.7 32.6   10:20.26 java
```

> top 命令是 Linux 下常用的性能分析工具，能够实时显示系统中各个进程的资源占用状况，类似于 Windows 的任务管理器。

通过以上命令，我们可以看到，进程 ID 为 1893 的 Java 进程的 CPU 占用率达到了 181%，基本可以定位到是我们的 Java 应用导致整个服务器的 CPU 占用率飙升。

44.2.2 定位线程

我们知道，Java 是单进程多线程的，那么，我们接下来看看 PID=1893 的这个 Java 进程中的各个线程的 CPU 使用情况，同样是用 top 命令：

```
$top -Hp 1893
  PID USER      PR  NI  VIRT  RES  SHR S %CPU %MEM    TIME+  COMMAND
 4519 admin     20   0 7127m 2.6g  38m R 18.6 32.6   0:40.11 java
```

通过 top -Hp 1893 命令，我们可以发现，当前 1893 这个进程中，ID 为 4519 的线程占用 CPU 最高。

44.2.3 定位代码

通过 top 命令，我们目前已经定位到导致 CPU 使用率较高的具体线程， 那么我么接下来就定位下到底是哪一行代码存在问题。

首先，我们需要把 4519 这个线程转成 16 进制：、

$printf %x 4519

11a7

接下来，通过 jstack 命令，查看栈信息：

```
 $sudo -u admin  jstack 1893 |grep -A 200 11a7
 "HSFBizProcessor-DEFAULT-8-thread-5" #500 daemon prio=10 os_prio=0
tid=0x00007f632314a800 nid=0x11a2 runnable [0x000000005442a000]
    java.lang.Thread.State: RUNNABLE
   at sun.misc.URLClassPath$Loader.findResource(URLClassPath.java:684)
   at sun.misc.URLClassPath.findResource(URLClassPath.java:188)
   at java.net.URLClassLoader$2.run(URLClassLoader.java:569)
   at java.net.URLClassLoader$2.run(URLClassLoader.java:567)
   at java.security.AccessController.doPrivileged(Native Method)
   at java.net.URLClassLoader.findResource(URLClassLoader.java:566)
   at java.lang.ClassLoader.getResource(ClassLoader.java:1093)
   at java.net.URLClassLoader.getResourceAsStream(URLClassLoader.java:232)
   at
org.hibernate.validator.internal.xml.ValidationXmlParser.getInputStreamForPath(ValidationXmlParser.java:248)
```

```
13      at
org.hibernate.validator.internal.xml.ValidationXmlParser.getValidationConfig(ValidationX
mlParser.java:191)
14      at
org.hibernate.validator.internal.xml.ValidationXmlParser.parseValidationXml(ValidationXm
lParser.java:65)
15      at
org.hibernate.validator.internal.engine.ConfigurationImpl.parseValidationXml(Configurati
onImpl.java:287)
16      at
org.hibernate.validator.internal.engine.ConfigurationImpl.buildValidatorFactory(Configura
tionImpl.java:174)
17      at javax.validation.Validation.buildDefaultValidatorFactory(Validation.java:111)
18      at com.test.common.util.BeanValidator.validate(BeanValidator.java:30)
```

通过以上代码，我们可以清楚的看到，BeanValidator.java 的第 30 行是有可能存在问题的。

44.3 问题解决

接下来就是通过查看代码来解决问题了，我们发现，我们自定义了一个 BeanValidator，封装了 Hibernate 的 Validator，然后在 validate 方法中，通过 Validation.buildDefaultValidatorFactory().getValidator()初始化一个 Validator 实例，通过分析发现这个实例化的过程比较耗时。

我们重构了一下代码，把 Validator 实例的初始化提到方法外，在类初始化的时候创建一次就解决了问题。

44.4 总结

以上，展示了一次比较完成的线上问题定位过程。**主要用到的命令有:top 、printf 和 jstack**

另外，线上问题排查还可以使用 Alibaba 开源的工具 Arthas 进行排查，以上问题，可以使用一下命令定位：

thread -n 3 //查看 cpu 占比前三的线程

以上，本文介绍了如何排查线上服务器 CPU 使用率过高的问题，如果大家感兴趣，后面可以再介绍一些关于 LOAD 飙高、频繁 GC 等问题的排查手段。

45. [合集]海量数据下的数据统计高频面试题系列

45.1 [百度二面]内存 100M 下统计不同号码的个数

题目来自百度二面。

> **已知某个文件内包含大量电话号码，每个号码为 8 位数字，如何统计不同号码的个数？内存限制 100M**

这类题目其实是求解数据重复的问题。对于这类问题，可以使用**位图法**处理

8 位电话号码可以表示的范围为 00000000～99999999。如果用 bit 表示一个号码，那么总共需要 1 亿个 bit，总共需要大约 **10MB** 的内存。

申请一个位图并初始化为 0，然后遍历所有电话号码，**把遍历到的电话号码对应的位图中的 bit 设置为 1**。当遍历完成后，如果 bit 值为 1，则表示这个电话号码在文件中存在，否则这个 bit 对应的电话号码在文件中不存在。

最后这个**位图中 bit 值为 1 的数量**就是不同电话号码的个数了。

那么如何确定电话号码对应的是位图中的哪一位呢？

可以使用下面的方法来做**电话号码和位图的映射**。

```
00000000 对应位图最后一位：0×0000...000001。
00000001 对应位图倒数第二位：0×0000...0000010（1 向左移 1 位）。
00000002 对应位图倒数第三位：0×0000...0000100（1 向左移 2 位）。
.....
00000012 对应位图的倒数第十三位：0×0000...0001000000000000（1 向左移 12 位）。
```

也就是说，电话号码就是 1 这个数字左移的次数。

首先位图可以使用一个 **int 数组**来实现（在 Java 中 int 占用 **4byte**）。

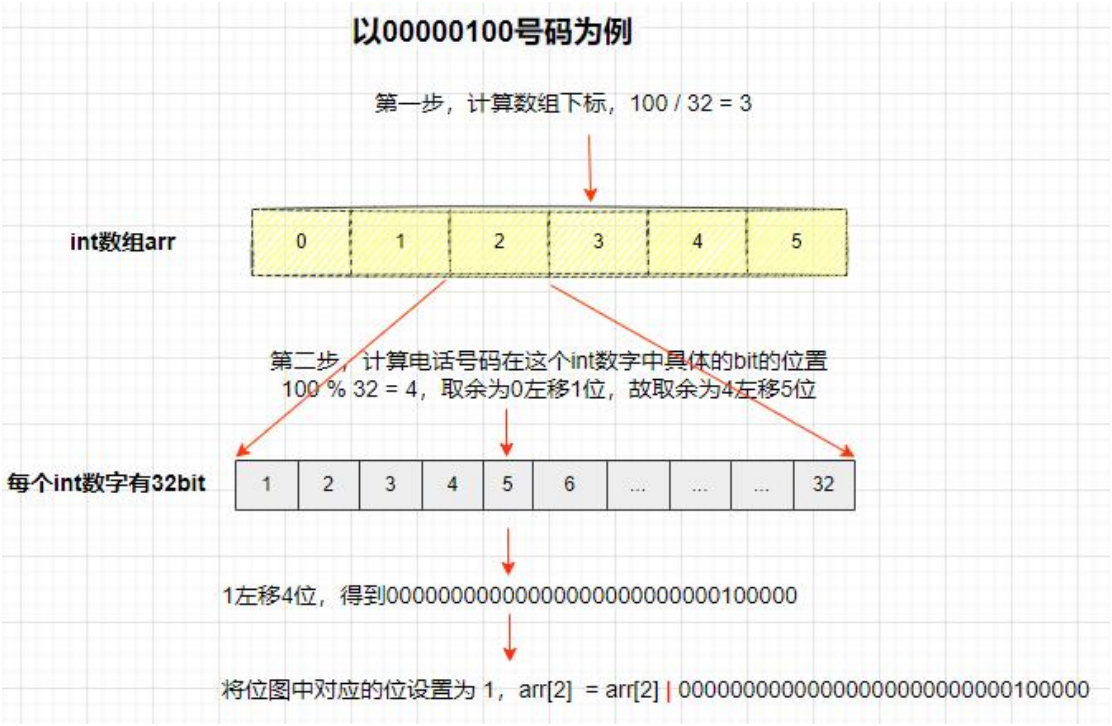
假设电话号码为 P，而通过电话号码获取位图中对应位置的方法为：

第一步，因为 int 整数占用 4*8=32bit，通过 **P/32** 就可以计算出该电话号码在 bitmap 数组中的下标，从而可以确定它对应的 bit 在数组中的位置。

第二步，通过 **P%32** 就可以计算出这个电话号码在这个 int 数字中具体的 bit 的位置。只要把 1 向左移 **P%32** 位，然后把得到的值与这个数组中的值做或运算，就可以把这个电话号码在位图中对应的位设置为 1。

以 00000100 号码为例。

8. 首先计算数组下标，100 / 32 = 3，得到数组下标位 3。
9. 然后计算电话号码在这个 int 数字中具体的 bit 的位置，100 % 32 = 4。取余为 0 左移 1 位，故取余为 4 左移 5 位，得到 000...000010000
10. 将位图中对应的位设置为 1，即 arr[2] = arr[2] | 000..00010000。
11. 这就将电话号码映射到了位图的某一位了。



最后，统计位图中 bit 值为 1 的数量，便能得到不同电话号码的个数了。

https://arcstack.top/md/interview/advance/mass-data/1-count-phone-num.html

45.2 [美团一面]出现频率最高的 100 个词

> 假如有一个 **1G** 大小的文件，文件里每一行是一个词，每个词的大小不超过 **16byte**，要求返回出现频率最高的 100 个词。内存大小限制是 **10M**

解法 1

由于内存限制，我们无法直接将大文件的所有词一次性读到内存中。

可以采用**分治策略**，把一个大文件分解成多个小文件，保证每个文件的大小小于 10M，进而直接将单个小文件读取到内存中进行处理。

第一步，首先遍历大文件，对遍历到的每个词 x，执行 `hash(x) % 500`，将结果为 i 的词存放到文件 f(i)中，遍历结束后，可以得到 500 个小文件，每个小文件的大小为 2M 左右；

第二步，接着统计每个小文件中出现频数最高的 100 个词。可以使用 HashMap 来实现，其中 key 为词，value 为该词出现的频率。

对于遍历到的词 x，如果在 map 中不存在，则执行 `map.put(x, 1)`。

若存在，则执行 `map.put(x, map.get(x)+1)`，将该词出现的次数加 1。

第三步，在第二步中找出了每个文件出现频率最高的 100 个词之后，通过维护一个**小顶堆**来找出所有小文件中出现频率最高的 100 个词。

具体方法是，遍历第一个文件，把第一个文件中出现频率最高的 100 个词构建成一个小顶堆。

如果第一个文件中词的个数小于 100，可以继续遍历第二个文件，直到构建好有 100 个结点的小顶堆为止。

继续遍历其他小文件，如果遍历到的词的出现次数大于堆顶上词的出现次数，可以用新遍历到的词替换堆顶的词，然后重新调整这个堆为小顶堆。

当遍历完所有小文件后，这个小顶堆中的词就是出现频率最高的 100 个词。

总结一下，这种解法的主要思路如下：

1. 采用**分治**的思想，进行哈希取余
2. 使用 **HashMap** 统计每个小文件单词出现的次数
3. 使用**小顶堆**，遍历步骤 2 中的小文件，找出词频 top100 的单词

但是很容易可以发现问题，在第二步中，如果这个 1G 的大文件中有某个词词频过高，可能导致小文件大小超过 10m。这种情况下该怎么处理呢?

接下来看另外一种解法。

解法 2

第一步：使用多路归并排序对大文件进行排序，这样相同的单词肯定是紧挨着的

多路归并排序对大文件进行排序的步骤如下：

① 将文件按照顺序切分成大小不超过 2m 的小文件，总共 500 个小文件

② 使用 10MB 内存**分别**对 500 个小文件中的单词进行**排序**

③ 使用一个大小为 500 大小的堆，对 500 个小文件进行**多路排序**，结果写到一个大文件中

其中第三步，对 500 个小文件进行多路排序的思路如下：

- 初始化一个最小堆，大小就是有序小文件的个数 500。堆中的每个节点存放每个有序小文件对应的输入流。
- 按照每个有序文件中的下一行数据对所有文件输入流进行排序，单词小的输入文件流放在堆顶。
- 拿出堆顶的输入流，并其下一行数据写入到最终排序的文件中，如果拿出来的输入流中还有数据的话，那么将这个输入流再一次添加到栈中。否则说明该文件输入流中没有数据了，那么可以关闭这个流。
- 循环这个过程，直到所有文件输入流都没有数据为止。

第二步：

① 初始化一个 100 个节点的**小顶堆**，用于保存 100 个出现频率最多的单词

② 遍历整个文件，一个单词一个单词的从文件中取出来，并计数

③ 等到遍历的单词和上一个单词不同的话，那么上一个单词及其频率如果大于堆顶的词的频率，那么放在堆中，否则不放

最终，小顶堆中就是出现频率前 100 的单词了。

解法 2 相对解法 1，更加严谨，如果某个词词频过高或者整个文件都是同一个词的话，解法 1 不适用。

45.3 [字节一面]查找两个大文件共同的 URL

> 给定 a、b 两个文件，各存放 50 亿个 URL，每个 URL 各占 64B，找出 a、b 两个文件共同的 URL。内存限制是 4G

分析

每个 URL 占 64B，那么 50 亿个 URL 占用的空间大小约为 320GB。

5,000,000,000 * 64B ≈ 320GB

由于内存大小只有 4G，因此，不可能一次性把所有 URL 加载到内存中处理。

可以采用分治策略，也就是把一个文件中的 URL 按照某个特征划分为多个小文件，使得每个小文件大小不超过 4G，这样就可以把这个小文件读到内存中进行处理了。

首先遍历文件 a，对遍历到的 URL 进行哈希取余 `hash(URL) % 1000`，根据计算结果把遍历到的 URL 存储到 a0, a1,a2, ..., a999，这样每个大小约为 300MB。使用同样的方法遍历文件 b，把文件 b 中的 URL 分别存储到文件 b0, b1, b2, ..., b999 中。这样处理过后，所有可能相同的 URL 都在对应的小文件中，即 a0 对应 b0, ..., a999 对应 b999，不对应的小文件不可能有相同的 URL。那么接下来，我们只需要求出这 1000 对小文件中相同的 URL 就好了。

接着遍历 ai(`i∈[0,999]`)，把 URL 存储到一个 HashSet 集合中。然后遍历 bi 中每个 URL，看在 HashSet 集合中是否存在，若存在，说明这就是共同的 URL，可以把这个 URL 保存到一个单独的文件中。

总结

最后总结一下：

1. 分而治之，进行哈希取余；
2. 对每个子文件进行 HashSet 统计。

45.4 [腾讯一面]如何在 100 亿数据中找到中位数?

> 给定 100 亿个无符号的乱序的整数序列，如何求出这 100 亿个数的中位数（中位数指的是排序后最中间那个数），内存只有 **512M**。

中位数问题可以看做一个统计问题，而不是排序问题，无符号整数大小为 4B，则能表示的数的范围为为 0 ~ 2^32 - 1 (40 亿) , 如果没有限制内存大小, 则可以用一个 2^32（4GB）大小的数组（也叫做桶）来保存 100 亿个数中每个无符号数出现的次数。遍历这 100 亿个数，当元素值等于桶元素索引时，桶元素的值加 1。当统计完 100 亿个数以后，则从索引为 0 的值开始累加桶的元素值，当累加值等于 50 亿时，这个值对应的索引为中位数。时间复杂度为 O(n)。

因为题目要求内存限制 512M，所以上述解法不合适。

下面分享另一种解法（**分治法**的思想）

如果 100 亿个数字保存在一个大文件中，可以依次读一部分文件到内存(不超过内存限制)，将每个数字用二进制表示，比较二进制的最高位(第 32 位，符号位，0 是正，1 是负)。

如果数字的最高位为 0，则将这个数字写入 file_0 文件中；如果最高位为 1，则将该数字写入 file_1 文件中。 从而将 100 亿个数字分成了两个文件。

假设 file_0 文件中有 60 亿 个数字，file_1 文件中有 40 亿 个数字。那么中位数就在 file_0 文件中，并且是 file_0 文件中所有数字排序之后的第 10 亿 个数字。因为 file_1 中的数都是负数，file_0 中的数都是正数，也即这里一共只有 40 亿个负数，那么排序之后的第 50 亿个数一定位于 file_0 中。

现在，我们只需要处理 file_0 文件了，不需要再考虑 file_1 文件。对于 file_0 文件，同样采取上面的措施处理：将 file_0 文件依次读一部分到内存(不超过内存限制)，将每个数字用二进制表示，比较二进制的 次高位（第 31 位），如果数字的次高位为 0，写入 file_0_0 文件中；如果次高位为 1，写入 file_0_1 文件 中。

现假设 file_0_0 文件中有 30 亿个数字，file_0_1 中也有 30 亿个数字，则中位数就是：file_0_0 文件中的数字从小到大排序之后的第 10 亿个数字。

抛弃 file_0_1 文件，继续对 file_0_0 文件 根据 次次高位(第 30 位) 划分，假设此次划分的两个文件为：file_0_0_0 中有 5 亿个数字，file_0_0_1 中有 25 亿个数字，那么中位数就是 file_0_0_1 文件中的所有数字排序之后的 第 5 亿 个数。

按照上述思路，直到划分的文件可直接加载进内存时，就可以直接对数字进行快速排序，找出中位数了。

45.5 [阿里一面]1000 万数据中如何查询最热门的查询串?

搜索引擎会通过日志文件把用户每次检索使用的所有查询串都记录下来，每个查询床的长度不超过 255 字节。

假设目前有 1000w 个记录（这些查询串的重复度比较高，虽然总数是 1000w，但如果除去重复后，则不超过 300w 个）。请统计最热门的 10 个查询串，要求使用的内存不能超过 1G。 （一个查询串的重复度越高，说明查询它的用户越多，也就越热门。）

每个查询串最长为 255B，1000w 个串需要占用 约 2.55G 内存，因此，我们无法将所有字符串全部读入到内存中处理。

方法一：分治法

分治法依然是一个非常实用的方法。

划分为多个小文件，保证单个小文件中的字符串能被直接加载到内存中处理，然后求出每个文件中出现次数最多的 10 个字符串；最后通过一个小顶堆统计出所有文件中出现最多的 10 个字符串。

方法可行，但不是最好，下面介绍其他方法。

方法二：HashMap 法

虽然字符串总数比较多，但去重后不超过 300w，因此，可以考虑把所有字符串及出现次数保存在一个 HashMap 中，所占用的空间为 300w*(255+4)≈777M（其中，4 表示整数占用的 4 个字节）。由此可见，1G 的内存空间完全够用。

思路如下：

首先，遍历字符串，若不在 map 中，直接存入 map，value 记为 1；若在 map 中，则把对应的 value 加 1，这一步时间复杂度 `O(N)`。

接着遍历 map，构建一个 10 个元素的小顶堆，若遍历到的字符串的出现次数大于堆顶字符串的出现次数，则进行替换，并将堆调整为小顶堆。

遍历结束后，堆中 10 个字符串就是出现次数最多的字符串。这一步时间复杂度 `O(Nlog10)`。

方法三：前缀树法

方法二使用了 HashMap 来统计次数，当这些字符串有大量相同前缀时，可以考虑使用前缀树来统计字符串出现的次数，树的结点保存字符串出现次数，0 表示没有出现。

思路如下：

在遍历字符串时，在前缀树中查找，如果找到，则把结点中保存的字符串次数加 1，否则为这个字符串构建新结点，构建完成后把叶子结点中字符串的出现次数置为 1。

最后依然使用小顶堆来对字符串的出现次数进行排序。

45.6 [大众点评]如何找出排名前 500 的数?

> 有 1w 个数组，每个数组有 500 个元素，并且有序排列。如何在这 10000*500 个数中找出前 500 的数?

方法 1

题目中每个数组是排好序的，可以使用**归并**的方法。

先将第 1 个和第 2 个归并，得到 500 个数据。然后再将结果和第 3 个数组归并，得到 500 个数据，以此类推，直到最后找出前 500 个的数。

方法 2

对于这种 topk 问题，更常用的方法是使用**堆排序。**

对本题而言，假设数组降序排列，可以采用以下方法：

首先建立大顶堆，堆的大小为数组的个数，即为 10000 ，把每个数组最大的值存到堆中。

接着删除堆顶元素，保存到另一个大小为 500 的数组中，然后向大顶堆插入删除的元素所在数组的下一个元素。

重复上面的步骤，直到删除完第 500 个元素，也即找出了最大的前 500 个数。

示例代码如下：

```
import lombok.Data;

import java.util.Arrays;
import java.util.PriorityQueue;

@Data
public class DataWithSource implements Comparable<DataWithSource> {
    /**
     * 数值
     */
    private int value;

    /**
     * 记录数值来源的数组
     */
    private int source;
```

```
17
18      /**
19       * 记录数值在数组中的索引
20       */
21      private int index;
22
23      public DataWithSource(int value, int source, int index) {
24          this.value = value;
25          this.source = source;
26          this.index = index;
27      }
28
29      /**
30       *
31       * 由于 PriorityQueue 使用小顶堆来实现，这里通过修改
32       * 两个整数的比较逻辑来让 PriorityQueue 变成大顶堆
33       */
34      @Override
35      public int compareTo(DataWithSource o) {
36          return Integer.compare(o.getValue(), this.value);
37      }
38  }
39
40
41  class Test {
42      public static int[] getTop(int[][] data) {
43          int rowSize = data.length;
44          int columnSize = data[0].length;
45
46          // 创建一个 columnSize 大小的数组，存放结果
47          int[] result = new int[columnSize];
48
49          PriorityQueue<DataWithSource> maxHeap = new PriorityQueue<>();
50          for (int i = 0; i < rowSize; ++i) {
51              // 将每个数组的最大一个元素放入堆中
52              DataWithSource d = new DataWithSource(data[i][0], i, 0);
53              maxHeap.add(d);
54          }
55
56          int num = 0;
57          while (num < columnSize) {
58              // 删除堆顶元素
59              DataWithSource d = maxHeap.poll();
60              result[num++] = d.getValue();
61              if (num >= columnSize) {
```

```
                    break;
                }

                d.setValue(data[d.getSource()][d.getIndex() + 1]);
                d.setIndex(d.getIndex() + 1);
                maxHeap.add(d);
            }
            return result;

        }

        public static void main(String[] args) {
            int[][] data = {
                    {29, 17, 14, 2, 1},
                    {19, 17, 16, 15, 6},
                    {30, 25, 20, 14, 5},
            };

            int[] top = getTop(data);
            System.out.println(Arrays.toString(top)); // [30, 29, 25, 20, 19]
        }
    }

```

45.7 [京东一面]如何按照 query 的频度排序?

有 10 个文件，每个文件大小为 1G，每个文件的每一行存放的都是用户的 query，每个文件的 query 都可能重复。要求按照 query 的频度排序。

如果 query 的重复度比较大，可以考虑一次性把所有 query 读入内存中处理；如果 query 的重复率不高，那么可用内存不足以容纳所有的 query，这时候就需要采用分治法或其他的方法来解决。

方法一：HashMap 法

如果 query 重复率高，说明不同 query 总数比较小，可以考虑把所有的 query 都加载到内存中的 HashMap 中。接着就可以按照 query 出现的次数进行排序。

方法二：分治法

分治法需要根据数据量大小以及可用内存的大小来确定问题划分的规模。对于这道题，可以顺序遍历 10 个文件中的 query，通过 Hash 函数 hash(query) % 10 把这些 query 划分到 10 个小文件中。之后对每个小文件使用 HashMap 统计 query 出现次数，根据次数排序并写入到零外一个单独文件中。

接着对所有文件按照 query 的次数进行排序，这里可以使用归并排序（由于无法把所有 query 都读入内存，因此需要使用外排序）。

45.8 [面试热点]数据中 TopK 问题的常用套路

今天想跟大家聊一些**常见的 topK 问题。**

对于海量数据到处理经常会涉及到 topK 问题。在设计数据结构和算法的时候，主要需要考虑的应该是当前算法（包括数据结构）跟给定情境（比如数据量级、数据类型）的适配程度，和当前问题最核心的瓶颈（如降低时间复杂度，还是降低空间复杂度）是什么。

首先，我们来举几个常见的 topK 问题的例子：

1. 给定 100 个 int 数字，在其中找出最大的 10 个；
2. 给定 10 亿个 int 数字，在其中找出最大的 10 个（这 10 个数字可以无序）；
3. 给定 10 亿个 int 数字，在其中找出最大的 10 个（这 10 个数字依次排序）；
4. 给定 10 亿个不重复的 int 数字，在其中找出最大的 10 个；
5. 给定 10 个数组，每个数组中有 1 亿个 int 数字，在其中找出最大的 10 个；
6. 给定 10 亿个 string 类型的数字，在其中找出最大的 10 个（仅需要查 1 次）；
7. 给定 10 亿个 string 类型的数字，在其中找出最大的 k 个（需要反复多次查询，其中 k 是一个随机数字）。

上面这些问题看起来很相似，但是解决的方式却千差万别。稍有不慎，就可能使得 topK 问题成为系统的瓶颈。不过也不用太担心，接下来我会总结几种常见的解决思路，遇到问题的时候，大家把这些基础思路融会贯通并且杂糅组合，即可做到见招拆招。

45.8.1 堆排序法

这里说的是堆排序法，而不是快排或者希尔排序。虽然理论时间复杂度都是 O(nlogn)，但是堆排在做 topK 的时候有一个优势，就是可以维护一个仅包含 k 个数字的小顶堆

（想清楚，为啥是小顶堆哦），当新加入的数字大于堆顶数字的时候，将堆顶元素剔除，并加入新的数字。

用 C++ 来说明，堆在 stl 中是 priority_queue（不是 set）。

```
int main() {
    const int topK = 3;
    vector<int> vec = {4,1,5,8,7,2,3,0,6,9};
    priority_queue<int, vector<int>, greater<>> pq;    // 小顶堆
    for (const auto& x : vec) {
        pq.push(x);
        if (pq.size() > topK) {
            // 如果超出个数，则弹出堆顶（最小的）数据
            pq.pop();
        }
    }

    while (!pq.empty()) {
        cout << pq.top() << endl;    // 输出依次为 7,8,9
        pq.pop();
    }

    return 0;
}

```

45.8.2 类似快排法

快排大家都知道，针对 topK 问题，可以对快排进行改进。仅对部分数据进行递归计算。比如，在 100 个数字中，找最大的 10 个，第一次循环的时候，povit 被移动到了 80 的位置，则接下来仅需要在后面的 20 个数字中找最大的 10 个即可。

这样做的优势是，理论最优时间复杂度可以达到 O(n)，不过平均时间复杂度还是 O(nlogn)。需要说明的是，通过这种方式，找出来的最大的 k 个数字之间，是无序的。

```
int partition(vector<int>& arr, int begin, int end) {
    int left = begin;
    int right = end;
    int povit = arr[begin];

    while (left < right) {
        while (left < right && arr[right] >= povit) {right--;}
        while (left < right && arr[left] <= povit) {left++;}
```

```
            if (left < right) {swap(arr[left], arr[right]);}
        }

        swap(arr[begin], arr[left]);
        return left;
}

void partSort(vector<int>& arr, int begin, int end, int target) {
        if (begin >= end) {
            return;
        }

        int povit = partition(arr, begin, end);
        if (target < povit) {
            partSort(arr, begin, povit - 1, target);
        } else if (target > povit) {
            partSort(arr, povit + 1, end, target);
        }
}

vector<int> getMaxNumbers(vector<int>& arr, int k) {
        int size = (int)arr.size();
        // 把求最大的 k 个数，转换成求最小的 size-k 个数字
        int target = size - k;
        partSort(arr, 0, size - 1, target);
        vector<int> ret(arr.end() - k, arr.end());
        return ret;
}

int main() {
        vector<int> vec = {4,1,5,8,7,2,3,0,6,9};
        auto ret = getMaxNumbers(vec, 3);

        for (auto x : ret) {
            cout << x << endl;      // 输出 7，8，9（理论上无序）
        }

        return 0;
}

```

45.8.3 使用 bitmap

有时候 topK 问题会遇到数据量过大，内存无法全部加载。这个时候，可以考虑将数据存放至 bitmap 中，方便查询。

比如，给出 10 个 int 类型的数据，分别是【13，12，11，1，2，3，4，5，6，7】，int 类型的数据每个占据 4 个字节，那这个数组就占据了 40 个字节。现在，把它们放到一个 16 个长度 bool 的 bitmap 中，结果就是【0，1，1，1，1，1，1，1，0，0，0，1，1，1，0，0】，在将空间占用降低至 4 字节的同时，也可以很方便的看出，最大的 3 个数字，分别是 11，12 和 13。

需要说明的是，bitmap 结合跳表一起使用往往有奇效。比如以上数据还可以记录成：从第 1 位开始，有连续 7 个 1；从第 11 位开始，有连续 3 个 1。这样做，空间复杂度又得到了进一步的降低。

这种做法的优势，当然是降低了空间复杂度。不过需要注意一点，bitmap 比较适合不重复且有范围（比如，数据均在 0 ~ 10 亿之间）的数据的查询。至于有重复数据的情况，可以考虑与 hash 等结构的混用。

45.8.4 使用 hash

如果遇到了查询 string 类型数据的大小，可以考虑 hash 方法。

举个例子，10 个 string 数字【"1001"，"23"，"1002"，"3003"，"2001"，"1111"，"65"，"834"，"5"，"987"】找最大的 3 个。我们先通过长度进行 hash，得到长度最大为 4，且有 5 个长度为 4 的 string。接下来再通过最高位值做 hash，发现有 1 个最高位为"3"的，1 个为"2"的，3 个为"1"的。接下来，可以通过再设计 hash 函数，或者是循环的方式，在 3 个最高位为"1"的 string 中找到最大的一个，即可找到 3 个最值大的数据。

这种方法比较适合网址或者电话号码的查询。缺点就是如果需要多次查询的话，需要多次计算 hash，并且需要根据实际情况设计多个 hash 函数。

45.8.5 字典树

字典树（trie）的具体结构和查询方式，不在这里赘述了，自行百度一下就有很多。这里主要说一下优缺点。

字典树的思想，还是通过前期建立索引信息，后期可以反复多次查询，并且后期增删数据也很方便。比较适合于需要反复多次查询的情况。

比如，反复多次查询字符序（例如：z>y>...>b>a）最大的 k 个 url 这种，使用字典树把数据存储一遍，就非常适合。既减少了空间复杂度，也加速了查询效率。

45.8.6 混合查询

以上几种方法，都是比较独立的方法。其实，在实际工作中，遇到更多的问题还是混合问题，这就需要我们对相关的内容，融会贯通并且做到活学活用。

我举个例子：我们的分布式服务跑在 10 台不同机器上，每台机器上部署的服务均被请求 10000 次，并且记录了个这 10000 次请求的耗时（耗时值为 int 数据），找出这 10*10000 次请求中，从高到低的找出耗时最大的 50 个。看看这个问题，很现实吧。我们试着用上面介绍的方法，组合一下来求解。

方法一

首先，对每台机器上的 10000 个做类似快排，找出每台机器上 top50 的耗时信息。此时，单机上的这 50 条数据是无序的。

然后，再将 10 台机器上的 50 条数据（共 500 条）放到一起，再做一次类似快排，找到最大的 50 个（此时应该这 50 个应该是无序的）。

最后，对这 50 个数据做快排，从而得到最终结果。

方法二

首先通过堆排，分别找出 10 台机器上耗时最高的 50 个数据，此时的这 50 个数据，已经是从大到小有序的了。

然后，我们依次取出 10 台机器中，耗时最高的 5 条放入小顶堆中。

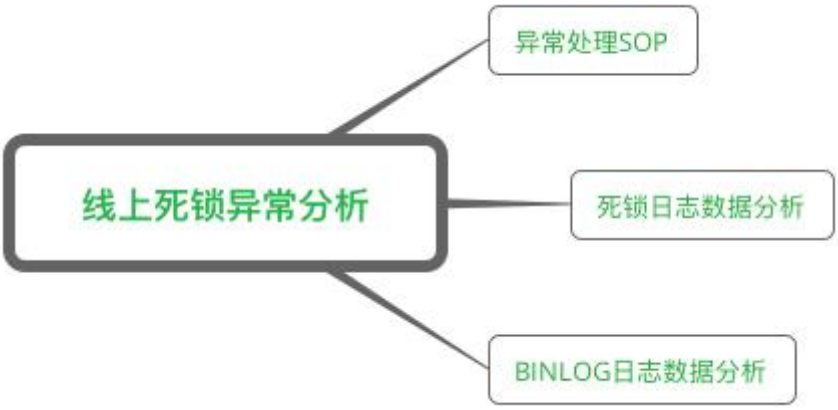
最后，遍历 10 台机器上的数据，每台机器从第 6 个数据开始往下循环，如果这个值比堆顶的数据大，则抛掉堆顶数据并且把它加入，继续用下一个值进行同样比较。如果这个值比堆顶的值小，则结束当前循环，并且在下一台机器上做同样操作。

以上我介绍了两种方法，并不是为了说明哪种方法更好，或者时间复杂度更低。而是想说同样的事情有多种不同的解决方法，而且随着数据量的增加，可能会需要更多组合形式。在这个领域，数据决定了数据结构，数据结构决定了算法。

46. [线上问题排查]数据库出现死锁如何排查

MySQL 死锁异常是我们经常会遇到的线上异常类别，一旦线上业务日间复杂，各种业务操作之间往往会产生锁冲突，有些会导致死锁异常。这种死锁异常一般要在特定时间特定数据和特定业务操作才会复现，并且分析解决时还需要了解 MySQL 锁冲突相关知识，所以一般遇到这些偶尔出现的死锁异常，往往一时没有头绪，不好处理。

本篇文章会讲解一下如果线上发生了死锁异常，如何去排查和处理。除了系列前文讲解的有关加锁和锁冲突的原理还，还需要对 MySQl 死锁日志和 binlog 日志进行分析。



线上死锁异常分析

日常工作中，应对各类线上异常都要有我们自己的 SOP (标准作业流程) ，这样不仅能够提高自己的处理问题效率，也有助于将好的处理流程推广到团队，提高团队的整体处理异常能力。

所以，面对线上偶发的 MySQL 死锁问题，我的排查处理过程如下：

1. 线上错误日志报警发现死锁异常
2. 查看错误日志的堆栈信息
3. 查看 MySQL 死锁相关的日志
4. 根据 binlog 查看死锁相关事务的执行内容

5. 根据上述信息找出两个相互死锁的事务执行的 SQL 操作，根据本系列介绍的锁相关理论知识，进行分析推断死锁原因
6. 修改业务代码

根据 1，2 步骤可以找到死锁异常时进行回滚事务的具体业务，也就能够找到该事务执行的 SQL 语句。然后我们需要通过 3，4 步骤找到死锁异常时另外一个事务，也就是最终获得锁的事务所执行的 SQL 语句，然后再进行锁冲突相关的分析。

第一二步的线上错误日志和堆栈信息一般比较容易获得，第五步的分析 SQL 锁冲突原因中涉及的锁相关的理论在系列文章中都有介绍，没有了解的同学可以自行去阅读以下。

下面我们就来重点说一下其中的第三四步骤，也就是如何查看死锁日志和 binlog 日志来找到死锁相关的 SQL 操作。

45.1 死锁日志的获取

发生死锁异常后，我们可以直接使用 `show engine innodb status` 命令获取死锁信息，但是该命令只能获取最近一次的死锁信息。所以，我们可以通过开启 InnoDB 的监控机制来获取实时的死锁信息，它会周期性（每隔 15 秒）打印 InnoDb 的运行状态到 mysqld 服务的错误日志文件中。

InnoDb 的监控较为重要的有标准监控（Standard InnoDB Monitor）和 锁监控（InnoDB Lock Monitor），通过对应的系统参数可以将其开启。

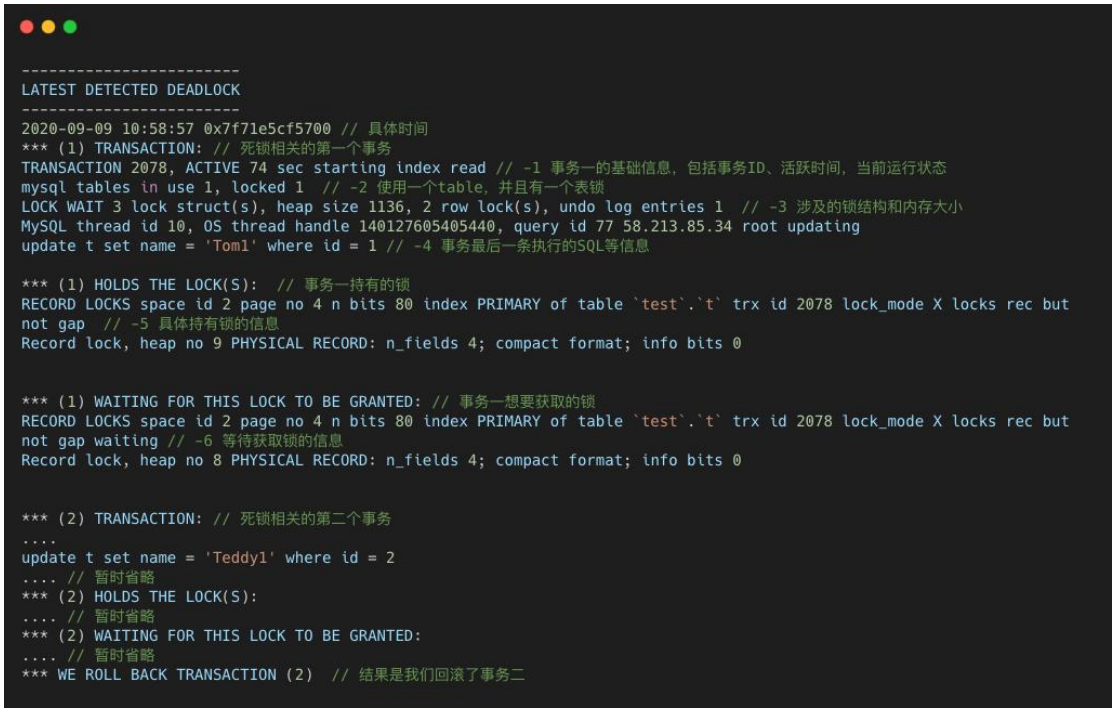
```
-- 开启标准监控
set GLOBAL innodb_status_output=ON;
-- 关闭标准监控
set GLOBAL innodb_status_output=OFF;
-- 开启锁监控
set GLOBAL innodb_status_output_locks=ON;
-- 关闭锁监控
set GLOBAL innodb_status_output_locks=OFF;
```

另外，MySQL 提供了一个系统参数 `innodb_print_all_deadlocks` 专门用于记录死锁日志，当发生死锁时，死锁日志会记录到 MySQL 的错误日志文件中。

```
set GLOBAL innodb_print_all_deadlocks=ON;
```

45.2 死锁日志的分析

通过上述手段，我们可以拿到死锁日志，下图是我做实验触发死锁异常时获取的日志(省略的部分信息)。



该日志会列出死锁发生的时间，死锁相关的事务，并显示出两个事务(可惜，多事务发生死锁时，也只显示两个事务)在**发生死锁时执行的 SQL 语句、持有或等待的锁信息和最终回滚的事务。**

下面，我们来一段一段的解读该日志中给出的信息，我们按照图中标注的顺序来介绍：

```
1  TRANSACTION 2078, ACTIVE 74 sec starting index read // -1 事务一的基础信息，包括事务ID、活跃时间，当前运行状态
```

表示的是 ACTIVE 74 sec 表示事务活动时间，starting index read 为事务当前正在运行的状态，可能的事务状态有：fetching rows，updating，deleting，inserting，starting index read 等状态。

```
1  mysql tables in use 1, locked 1  // -2 使用一个 table，并且有一个表锁
2  LOCK WAIT 3 lock struct(s), heap size 1136, 2 row lock(s), undo log entries 1  // -3 涉及的锁结构和内存大小
```

tables in use 1 表示有一个表被使用，locked 1 表示有一个表锁。LOCK WAIT 表示事务正在等待锁，3 lock struct(s) 表示该事务的锁链表的长度为 3，每个链表节点代

表该事务持有的一个锁结构，包括表锁，记录锁或 autoinc 锁等。heap size 1136 为事务分配的锁堆内存大小。

2 row lock(s) 表示当前事务持有的行锁个数，通过遍历上面提到的 11 个锁结构，找出其中类型为 LOCK_REC 的记录数。undo log entries 1 表示当前事务有 1 个 undo log 记录，说明该事务已经更新了 1 条记录。

下面就是死锁日志中最为重要的持有或者待获取锁信息，如图中-5 和-6 行所示，**通过它可以分析锁的具体类型和涉及的表，这些信息能辅助你按照系列文章的锁相关的知识来分析 SQL 的锁冲突。**

```
1  RECORD LOCKS space id 2 page no 4 n bits 80 index PRIMARY of table `test`.`t` trx id 2078 lock_mode X locks rec but not gap    // -5 具体持有锁的信息
2  RECORD LOCKS space id 2 page no 4 n bits 80 index PRIMARY of table `test`.`t` trx id 2078 lock_mode X locks rec but not gap waiting // -6 等待获取锁的信息
```

一共有四种类型的行锁：记录锁，间隙锁，Next-key 锁和插入意向锁。这四种锁对应的死锁日志各不相同，如下：

记录锁（LOCK_REC_NOT_GAP）: lock_mode X locks rec but not gap

间隙锁（LOCK_GAP）: lock_mode X locks gap before rec

Next-key 锁（LOCK_ORNIDARY）: lock_mode X

插入意向锁（LOCK_INSERT_INTENTION）: lock_mode X locks gap before rec insert intention

所以，按照死锁日志，我们发现事务一持有了 test.t 表上的记录锁，并且等待另一个记录锁。

通过死锁日志，我们可以找到最终获得锁事务最后执行的 SQL，但是如果该事务执行了多条 SQL，这些信息就可能不够用的啦，我们需要完整的了解该事务所有执行的 SQL 语句。这时，我们就需要从 binlog 日志中获取。

45.3 binlog 的获取和分析

binlog 日志会完整记录事务执行的所有 SQL，借助它，我们就能找到最终获取锁事务所执行的全部 SQL。然后再进行具体的锁冲突分析。

我们可以使用 MySQL 的命令行工具 Mysqlbinlog 远程获取线上数据库的 binlog 日志。具体命令如下所示：

```
Mysqlbinlog -h127.0.0.1 -u root -p --read-from-remote-server binlog.000001
--base64-output=decode-rows -v
```

其中 --base64-output=decode-rows 表示 row 模式 binlog 日志，所以该方法只适用于 row 模式的 binlog 日志，但是目前主流 MySQL 运维也都是把 binlog 日志设置为 row 模式，所以这点限制也就无伤大雅。-v 则表示将行事件重构成被注释掉的伪 SQL 语句。

我们可以**通过死锁日志中死锁发生的具体事件和最终获取锁事务正在执行的 SQL 的参数信息找到 binlog 中该事务的对应信息**，比如我们可以直接通过死锁日志截图中的具体的时间 10 点 57 分和 Tom1、Teddy2 等 SQL 的具体数据信息在 binlog 找到对应的位置，具体如下图所示。

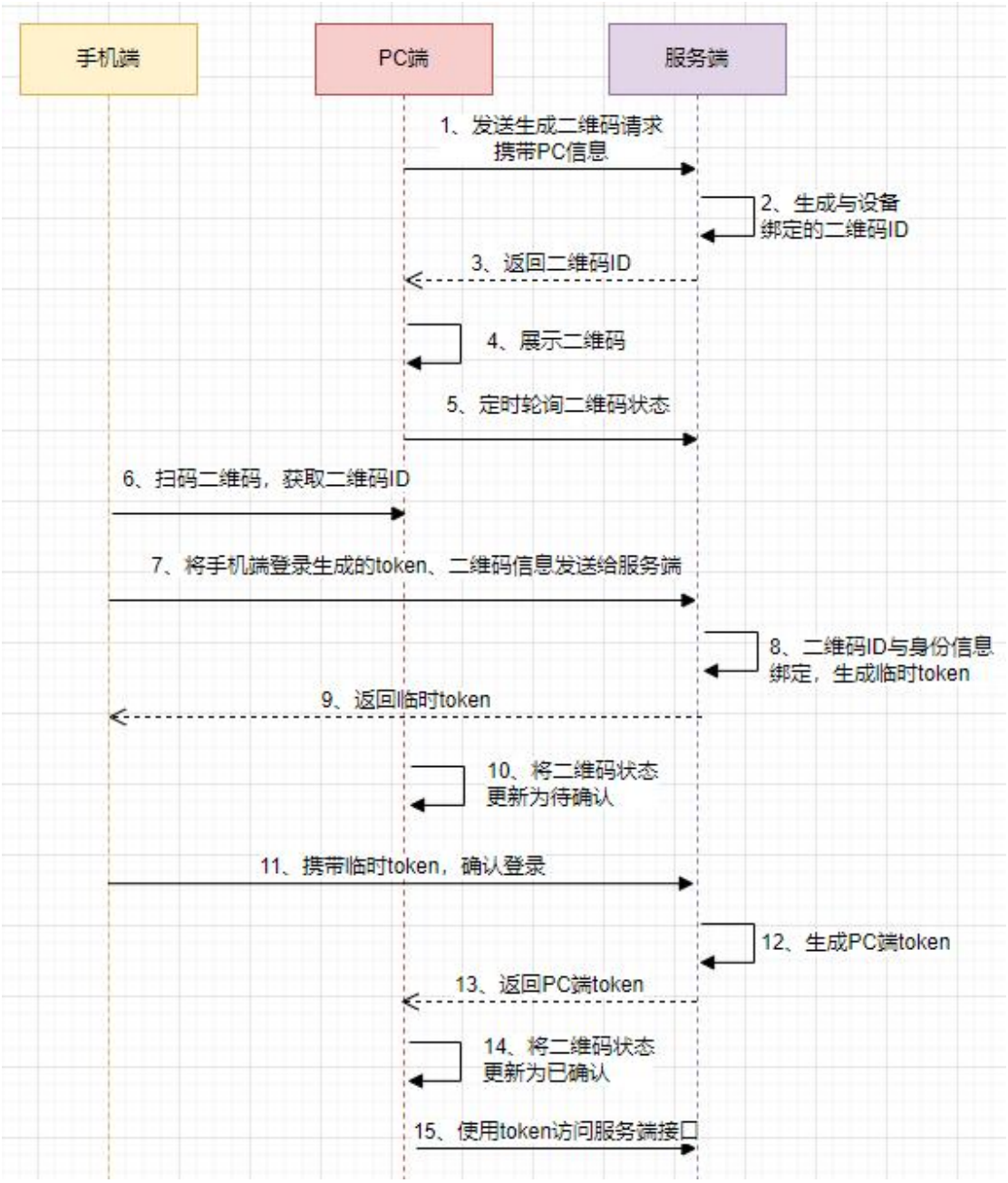
```
# at 1289  //文件offset或者行事件在 binlog 中的位置
#200915 10:58:10 server id 1  end_log_pos 1373 CRC32 0x090dfedd     Query   thread_id=370   exec_time=0 error_code=0
// 具体的日期date时间和服务器id
SET TIMESTAMP=1600171702/*!*/;
BEGIN  // 开启事务
....
# at 1428 // 事务一执行的第一条sql update t set name = 'Tom1' where id = 1;
#200915 10:58:32 server id 1  end_log_pos 1491 CRC32 0xfce91453     Update_rows: table id 96 flags: STMT_END_F
### UPDATE `test`.`t`
### WHERE
###   @1=1
###   @2='Tom'
### SET
###   @1=1
###   @2='Tom1'
# at 1491 // 事务一直线的第二条sql  update t set name = 'Teddy2' where id = 2;
#200915 20:08:54 server id 1  end_log_pos 1546 CRC32 0x3e3ae019     Table_map: `test`.`t` mapped to number 96
# at 1546
#200915 20:08:57 server id 1  end_log_pos 1613 CRC32 0x0dd10ed8     Update_rows: table id 96 flags: STMT_END_F
### UPDATE `test`.`t`
### WHERE
###   @1=2
###   @2='Teddy'
### SET
###   @1=2
###   @2='Teddy2'
# at 1613
#200915 20:09:00 server id 1  end_log_pos 1644 CRC32 0xf4f6e54b     Xid = 205
COMMIT/*!*/; // 提交事务
```

根据 binlog 的具体信息，我们可以清晰的找到最终获取锁事务所执行的所有 SQL 语句，也就能找到其对应的业务代码，接下来我们就能进行具体的锁冲突分析。

47. 请你详细介绍一下扫码登录的实现原理？

扫码登录功能主要分为三个阶段：**待扫描、已扫描待确认、已确认。**

整体流程图如图。



下面分阶段来看看设计原理。

1、待扫描阶段

首先是待扫描阶段，这个阶段是 PC 端跟服务端的交互过程。

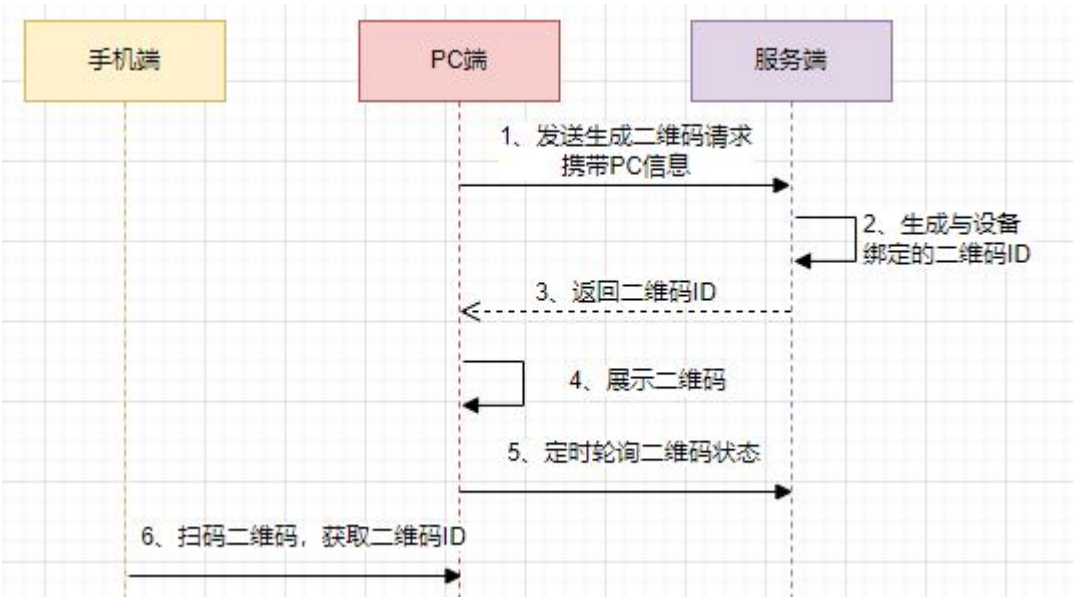
每次用户打开 PC 端登陆请求，系统返回一个**唯一的二维码 ID**，并将二维码 ID 的信息绘制成二维码返回给用户。

这里的二维码 ID 一定是唯一的，后续流程会将二维码 ID 跟身份信息绑定，不唯一的话就会造成你登陆了其他用户的账号或者其他用户登陆你的账号。

此时在 PC 端会启动一个定时器，**轮询查询二维码是否被扫描。**

如果移动端未扫描的话，那么一段时间后二维码将会失效。

这个阶段的交互过程如下图所示。



2、已扫描待确认阶段

第二个阶段是已扫描待确认阶段，主要是移动端跟服务端交互的过程。

首先移动端扫描二维码，获取二维码 ID，然后**将手机端登录的凭证（token）和 二维码 ID 作为参数发送给服务端**

此时的手机在之前已经是登录的，不存在没登录的情况。

服务端接受请求后，会将 token 与二维码 ID 关联，然后会生成一个临时 token，这个 token 会返回给移动端，临时 token 用作确认登录的凭证。

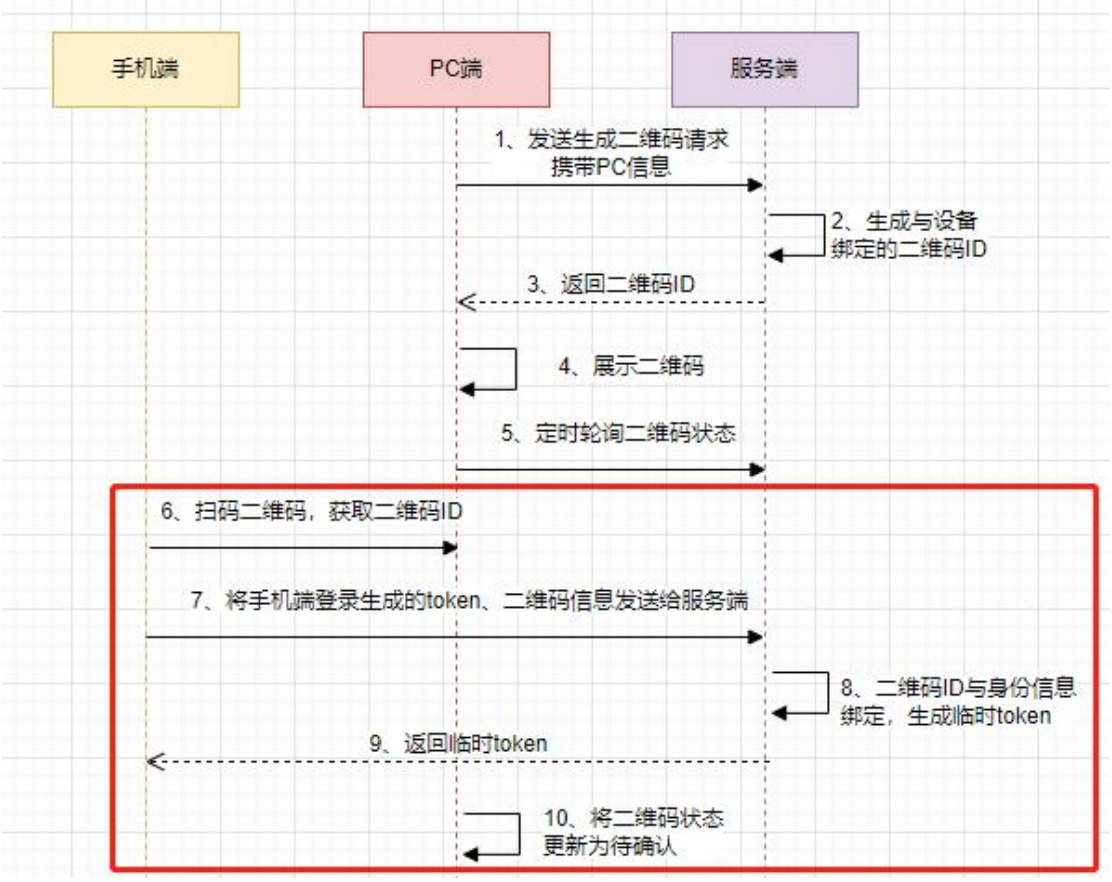
PC 端的定时器，会轮询到二维码的状态已经发生变化，会将 PC 端的二维码更新为已扫描，请在手机端确认。

这里为什么要有手机端确认的操作?

假设没有确认这个环节，很容易就会被坏人拦截 token 去冒充登录。所以二维码扫描一定要有这个确认的页面，让用户去确认是否进行登录。

另外，二维码扫描确认之后，再往用户 app 或手机等发送登录提醒的通知，告知如果不是本人登录的，则建议用户立即修改密码。

这个阶段是交互过程如下图所示。



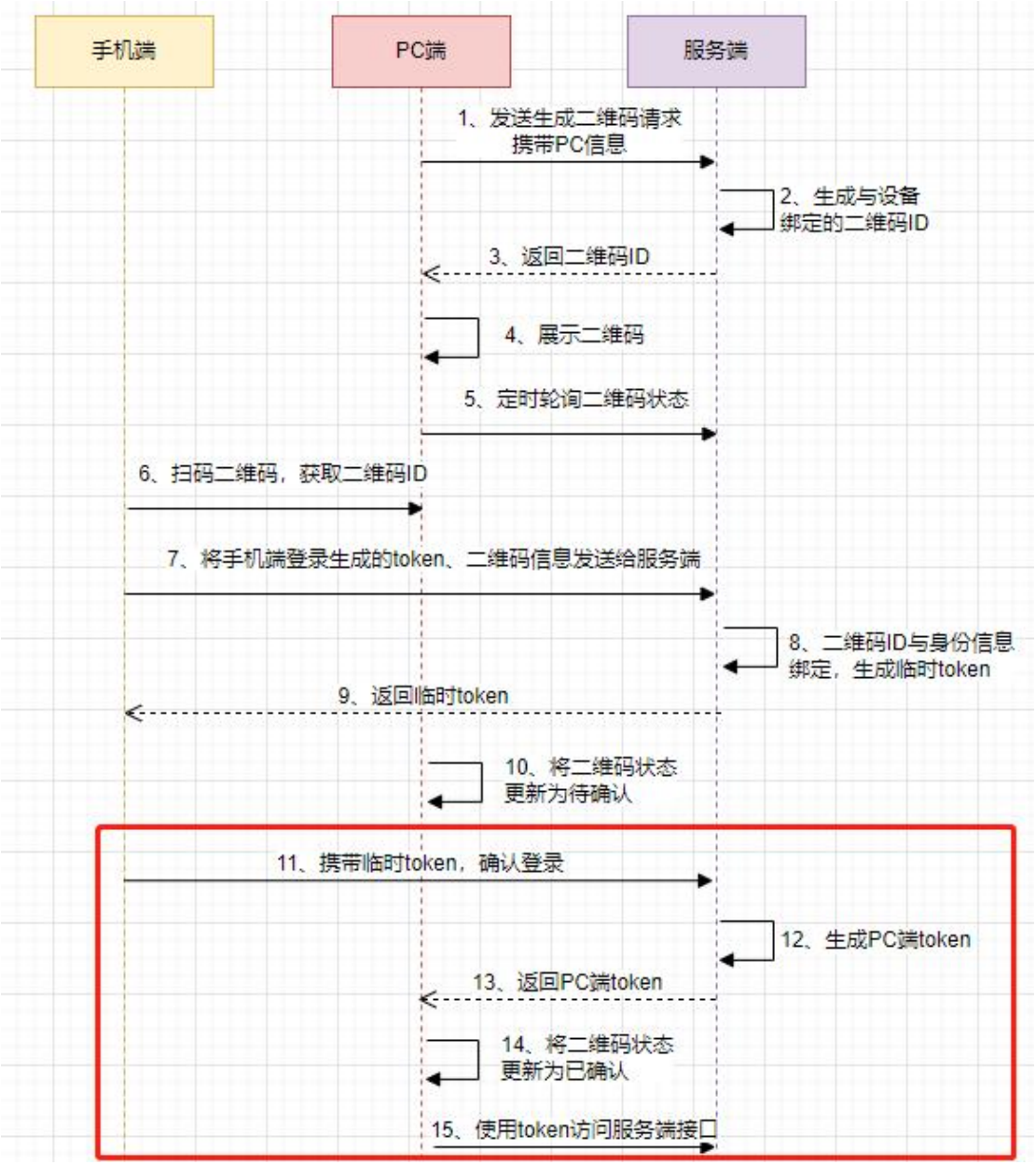
3、已确认

扫码登录的最后阶段，用户点击确认登录，移动端携带上一步骤中获取的临时 token 访问服务端。

服务端校对完成后，会更新二维码状态，并且给 PC 端生成一个正式的 token。

后续 PC 端就是持有这个 token 访问服务端。

这个阶段是交互过程如下图所示。



48. 单点登录（SSO）的设计与实现

48.1 功能背景介绍

1、SSO 说明

SSO 英文全称 Single Sign On，单点登录。SSO 是在多个应用系统中，用户只需要登录一次就可以访问所有相互信任的应用系统。

https://baike.baidu.com/item/SSO/3451380

例如访问在网易账号中心（http://reg.163.com/ ）登录之后 访问以下站点都是登录状态

- 网易直播 **http://v.163.comopen in new window**
- 网易博客 **http://blog.163.comopen in new window**
- 网易花田 **http://love.163.comopen in new window**
- 网易考拉 **https://www.kaola.comopen in new window**
- 网易 Lofter **http://www.lofter.comopen in new window**

2、设计目标

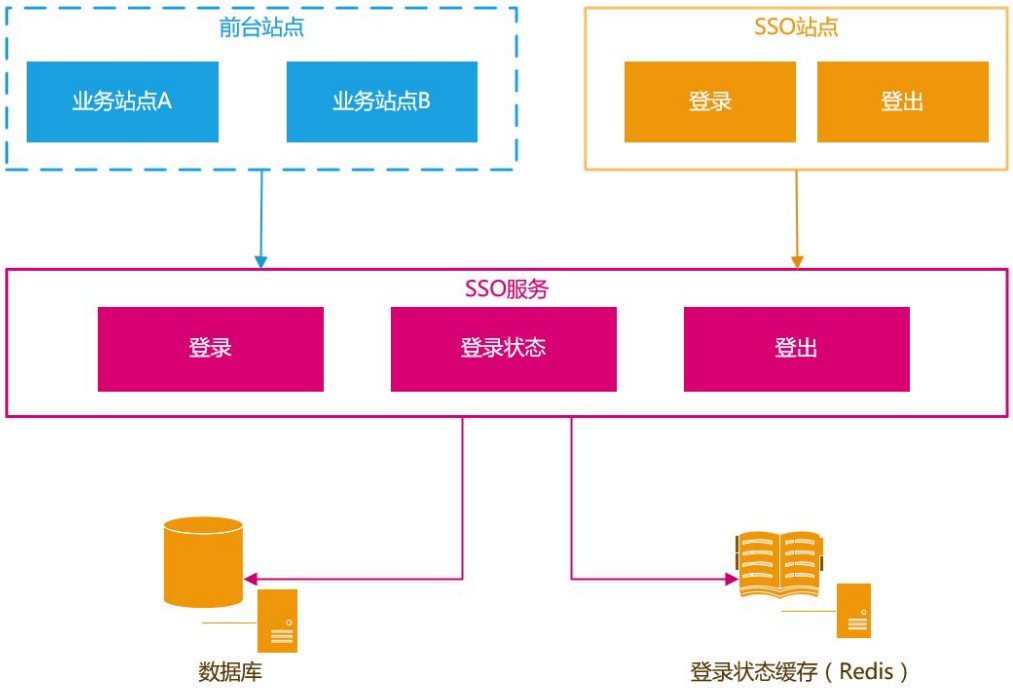
本篇文章也主要是为了探讨如何设计&实现一个 SSO 系统

以下为需要实现的核心功能：

- 单点登录
- 单点登出
- 支持跨域单点登录
- 支持跨域单点登出

48.2 SSO 设计与实现

#1、核心应用与依赖



应用/模块/对象	**说明**
前台站点	需要登录的站点
SSO 站点-登录	提供登录的页面
SSO 站点-登出	提供注销登录的入口
SSO 服务-登录	提供登录服务
SSO 服务-登录状态	提供登录状态校验/登录信息查询的服务
SSO 服务-登出	提供用户注销登录的服务
数据库	存储用户账户信息
缓存	存储用户的登录信息，通常使用 Redis

#2、用户登录状态的存储与校验

常见的 Web 框架对于 Session 的实现都是生成一个 SessionId 存储在浏览器 Cookie 中。然后将 Session 内容存储在服务器端内存中。

用户登录成功之后，生成 AuthToken 交给客户端保存。如果是浏览器，就保存在 Cookie 中。如果是手机 App 就保存在 App 本地缓存中。本篇主要探讨基于 Web 站点的 SSO。

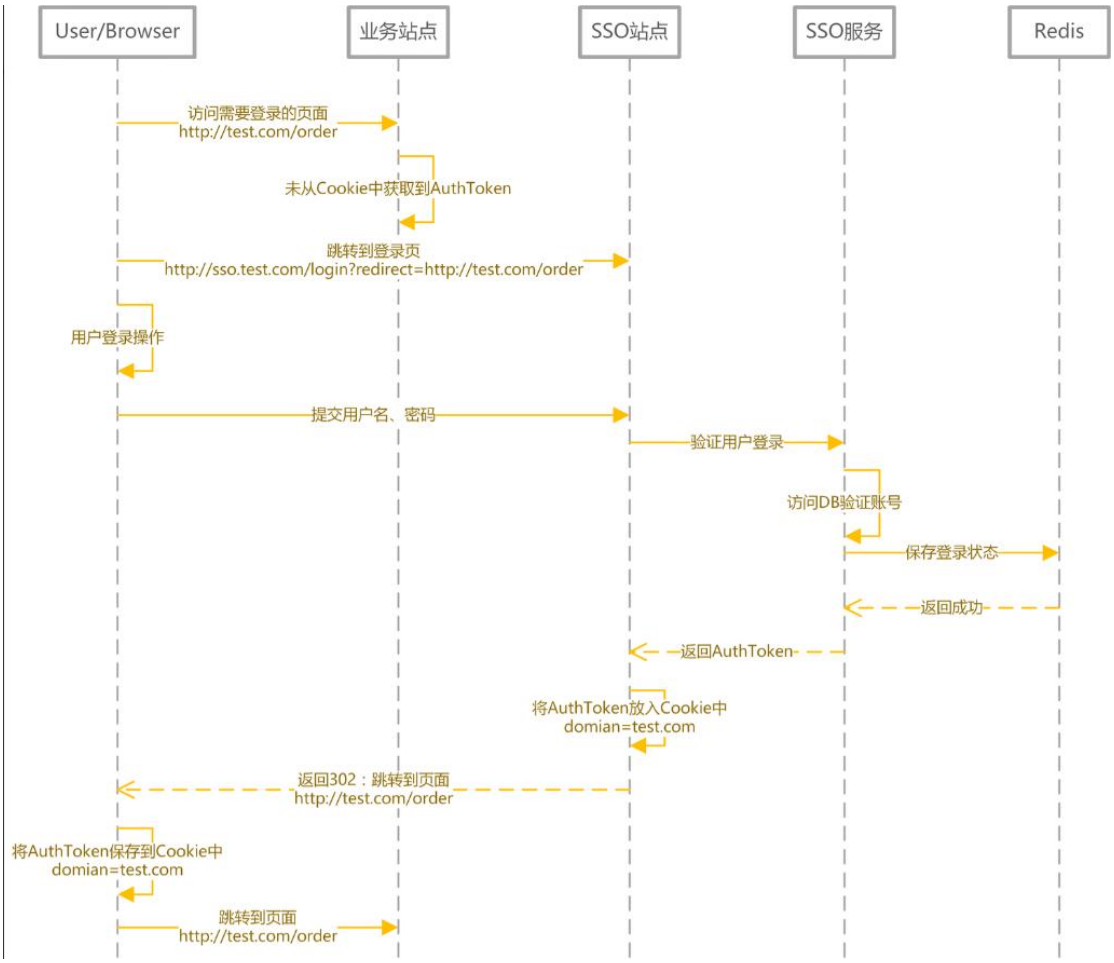
用户在浏览需要登录的页面时，客户端将 AuthToken 提交给 SSO 服务校验登录状态/获取用户登录信息

对于登录信息的存储，建议采用 Redis，使用 Redis 集群来存储登录信息，既可以保证高可用，又可以线性扩充。同时也可以让 SSO 服务满足负载均衡/可伸缩的需求。

对象	**说明**
AuthToken	直接使用 UUID/GUID 即可，如果有验证 Auth 合法性需求，可以将 UserName+时间戳加密生 务端解密之后验证合法性
登录信息	通常是将 UserId，UserName 缓存起来

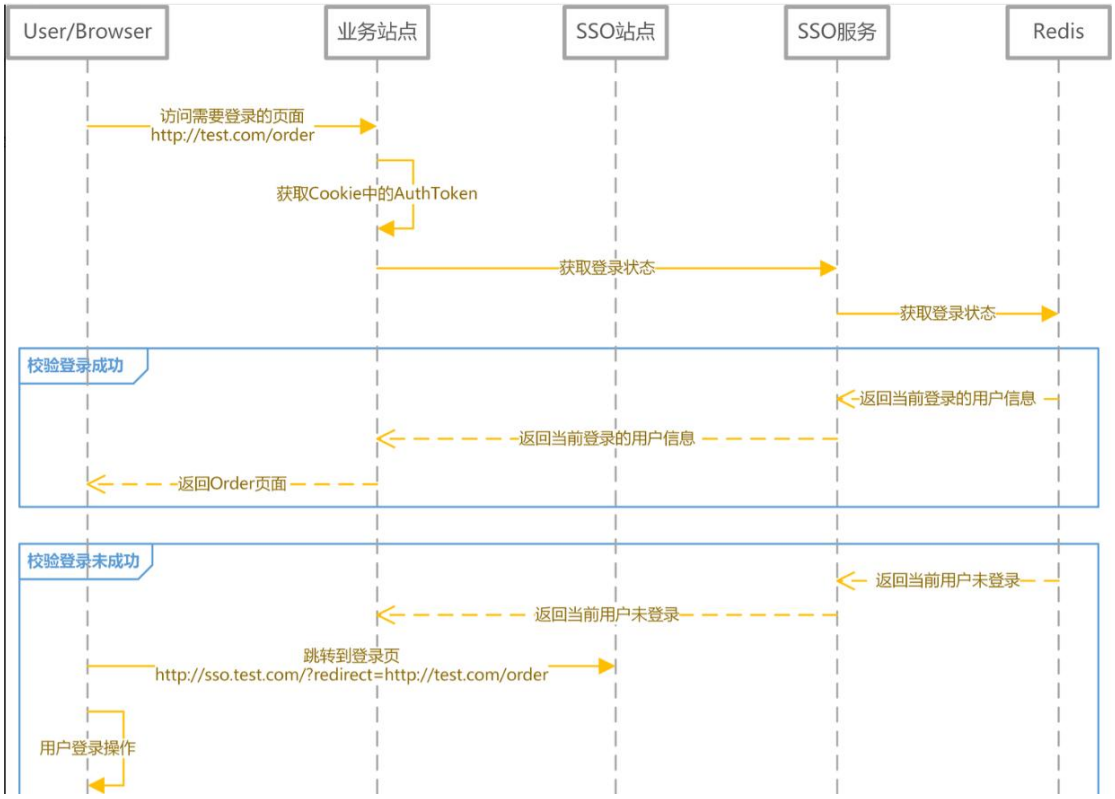
#3、用户登录/登录校验

- 登录时序图



按照上图，用户登录后 Authtoken 保存在 Cookie 中。 domian= test. com 浏览器会将 domain 设置成 .test.com， 这样访问所有*.test.com 的 web 站点，都会将 Authtoken 携带到服务器端。 然后通过 SSO 服务，完成对用户状态的校验/用户登录信息的获取

- 登录信息获取/登录状态校验

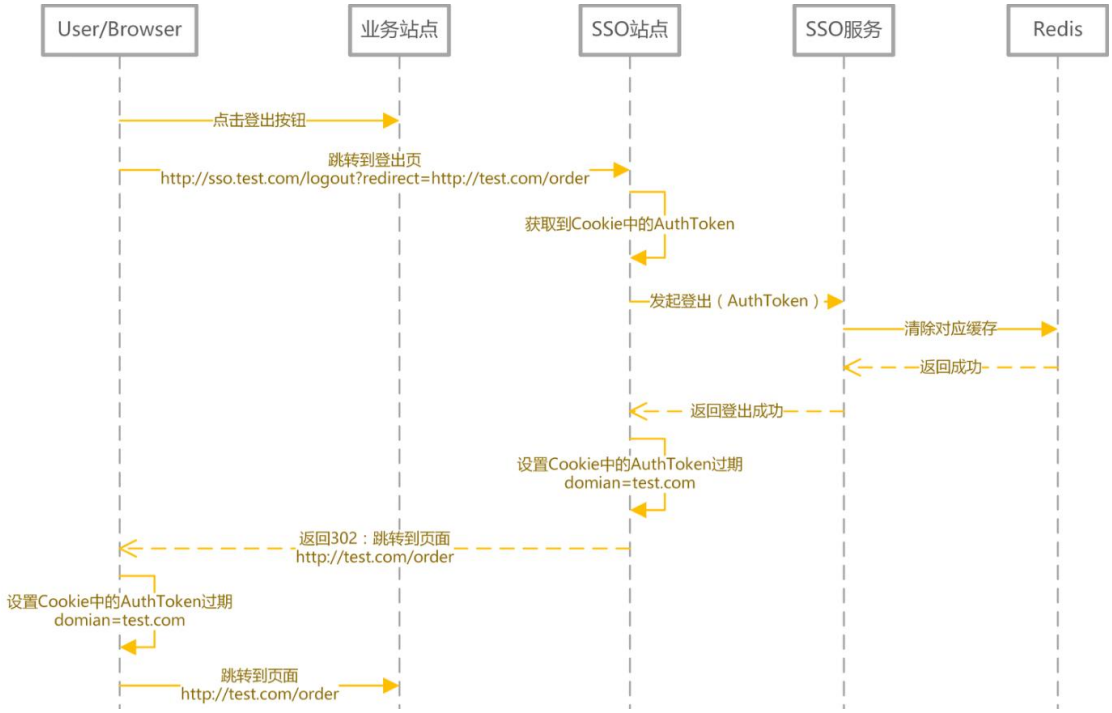


#4、用户登出

用户登出时要做的事情很简单：

1. 服务端清除缓存（Redis）中的登录状态
2. 客户端清除存储的 AuthToken

- 登出时序图

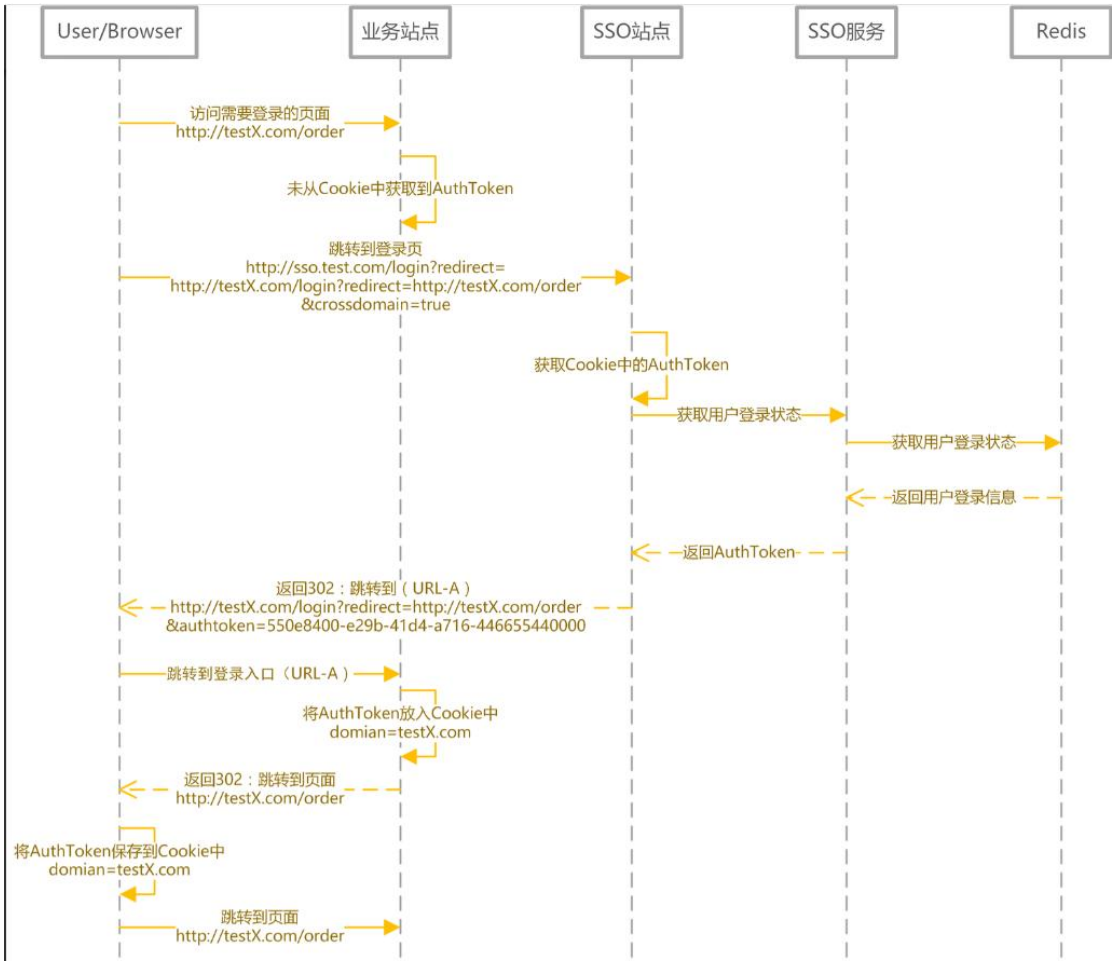


#5、跨域登录、登出

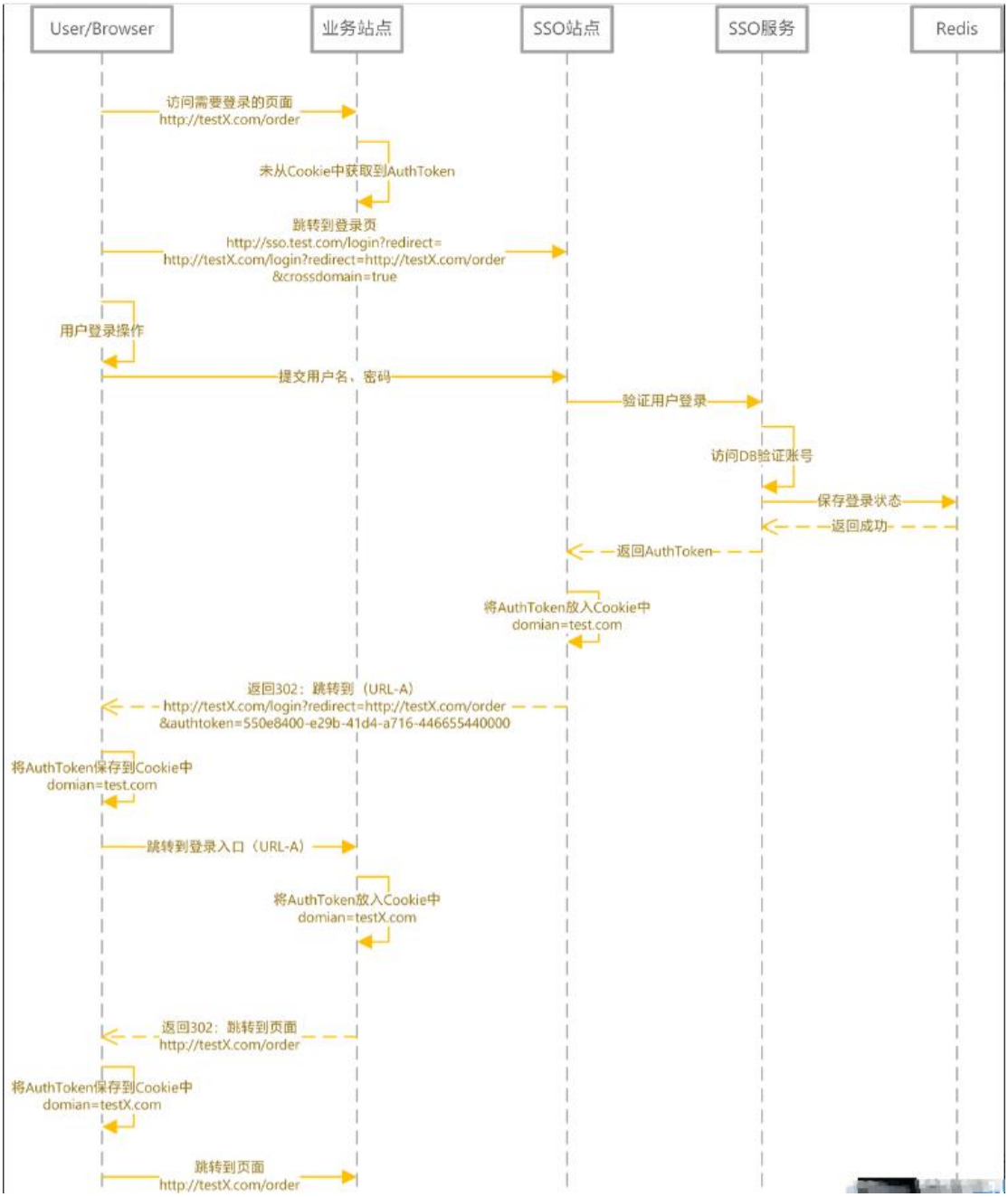
前面提到过，核心思路是客户端存储 AuthToken，服务器端通过 Redis 存储登录信息。由于客户端是将 AuthToken 存储在 Cookie 中的。所以跨域要解决的问题，就是如何解决 Cookie 的跨域读写问题。

解决跨域的核心思路就是：

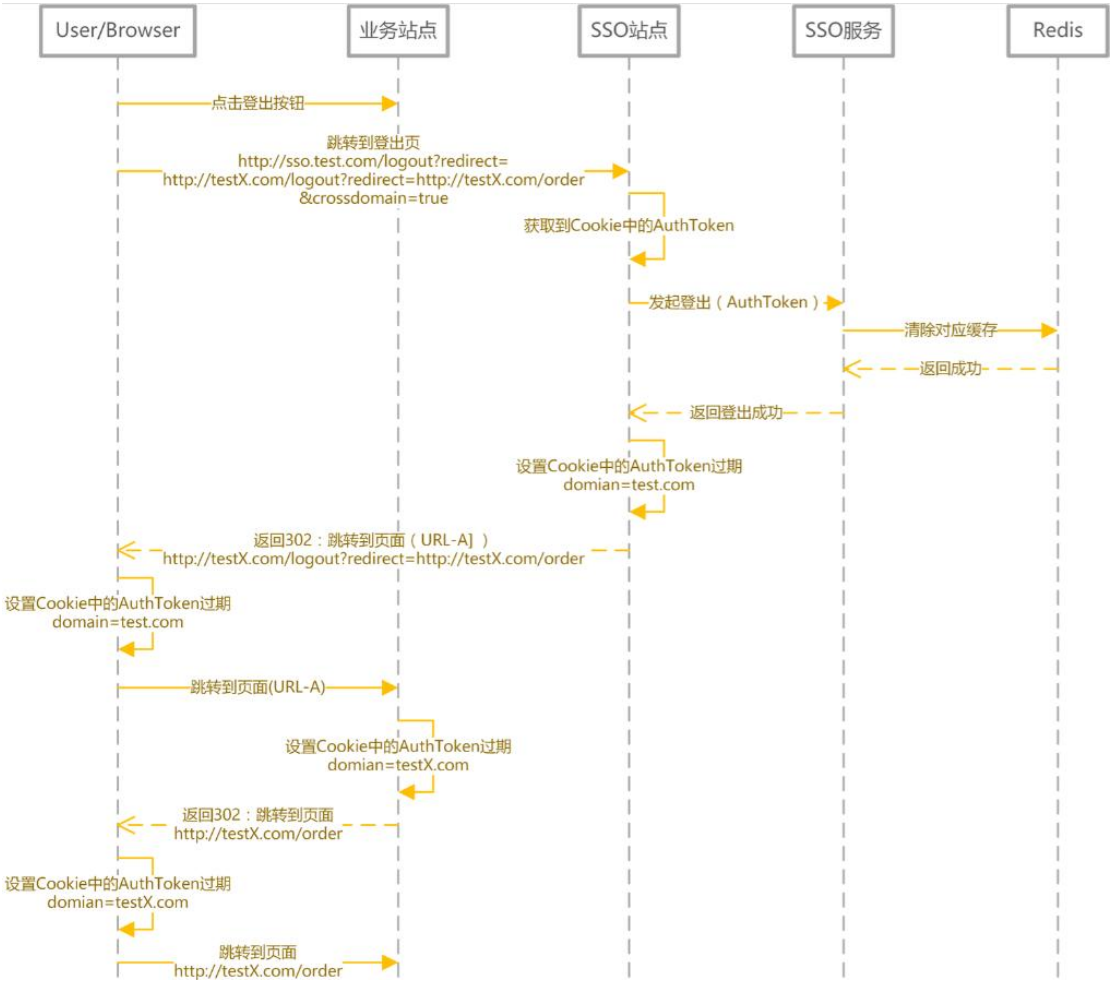
- 登录完成之后通过回调的方式，将 AuthToken 传递给主域名之外的站点，该站点自行将 AuthToken 保存在当前域下的 Cookie 中。
- 登出完成之后通过回调的方式，调用非主域名站点的登出页面，完成设置 Cookie 中的 AuthToken 过期的操作。
- 跨域登录（主域名已登录）


User/Browser
业务站点
SSO站点
SSO服务
Redis
访问需要登录的页面
http://testX.com/order
未从Cookie中获取到AuthToken
跳转到登录页
http://sso.test.com/login?redirect=
http://testX.com/login?redirect=http://testX.com/order
&crossdomain=true
获取Cookie中的AuthToken
获取用户登录状态
获取用户登录状态
返回用户登录信息
返回AuthToken
返回302：跳转到（URL-A）
http://testX.com/login?redirect=http://testX.com/order
&authtoken=550e8400-e29b-41d4-a716-446655440000
跳转到登录入口（URL-A）
将AuthToken放入Cookie中
domian=testX.com
返回302：跳转到页面
http://testX.com/order
将AuthToken保存到Cookie中
domian=testX.com
跳转到页面
http://testX.com/order

- 跨域登录（主域名未登录）


User/Browser
业务站点
SSO站点
SSO服务
Redis
访问需要登录的页面
http://testX.com/order
未从Cookie中获取到AuthToken
跳转到登录页
http://sso.test.com/login?redirect=
http://testX.com/login?redirect=http://testX.com/order
&crossdomain=true
用户登录操作
提交用户名、密码
验证用户登录
访问DB验证账号
保存登录状态
返回成功
返回AuthToken
将AuthToken放入Cookie中
domian=test.com
返回302：跳转到（URL-A）
http://testX.com/login?redirect=http://testX.com/order
&authtoken=550e8400-e29b-41d4-a716-446655440000
将AuthToken保存到Cookie中
domian=test.com
跳转到登录入口（URL-A）
将AuthToken放入Cookie中
domian=testX.com
返回302：跳转到页面
http://testX.com/order
将AuthToken保存到Cookie中
domian=testX.com
跳转到页面
http://testX.com/order

- 跨域登出



49. [京东一面]如何用 Redis 统计用户访问量?

拼多多有数亿的用户，那么对于某个网页，怎么使用 Redis 来统计一个网站的用户访问数呢?

1、**Hash**

哈希是 Redis 的一种基础数据结构，Redis 底层维护的是一个开散列，会把不同的 key 映射到哈希表上，如果是遇到关键字冲突，那么就会拉出一个链表出来。

当一个用户访问的时候，如果用户登陆过，那么我们就使用用户的 id，如果用户没有登陆过，那么我们也能够前端页面随机生成一个 key 用来标识用户

当用户访问的时候，我们可以使用 HSET 命令，key 可以选择 URI 与对应的日期进行拼凑，field 可以使用用户的 id 或者随机标识，value 可以简单设置为 1。

当我们要统计某一个网站某一天的访问量的时候，就可以直接使用 HLEN 来得到最终的结果了。

```
[root@VM_0_2_centos ~]# redis-c
redis-check-aof  redis-check-rdb  redis-cli
[root@VM_0_2_centos ~]# redis-cli
127.0.0.1:6379> hset index.html0527 11110 1
(integer) 1
127.0.0.1:6379> hset index.html0527 11115 2
(integer) 1
127.0.0.1:6379> hlen index.html0527
(integer) 2
127.0.0.1:6379>
```

优点：简单，容易实现，查询也是非常方便，数据准确性非常高。

缺点：占用内存过大。随着 key 的增多，性能也会下降。网站访问量不高还行，拼多多这种数亿 PV 的网站肯定顶不住。

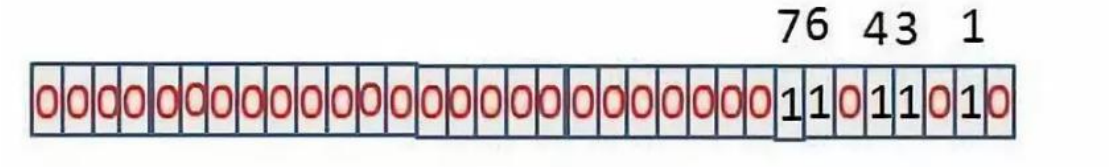
2、**Bitset**

我们知道，对于一个 32 位的 int，如果我们只用来记录 id，那么只能够记录一个用户，但如果我们转成 2 进制，每位用来表示一个用户，那么我们就能够一口气表示 32 个用户，空间节省了 32 倍！

对于有大量数据的场景，如果我们使用 bitset，那么可以节省非常多的内存。

对于没有登陆的用户，我们也可以使用哈希算法，把对应的用户标识哈希成一个数字 id。

bitset 非常的节省内存，假设有 1 亿个用户，也只需要 100000000/8/1024/1024 约等于 12 兆内存。



Redis 已经为我们提供了 SETBIT 的方法，使用起来非常的方便，我们可以看看下面的例子。

我们在 item 页面可以不停地使用 SETBIT 命令，设置用户已经访问了该页面，也可以使用 GETBIT 的方法查询某个用户是否访问。最后我们通过 BITCOUNT 可以统计该网页每天的访问数量。

```
127.0.0.1:6379>
127.0.0.1:6379> setbit item.html0528 11111 1
(integer) 0
127.0.0.1:6379> setbit item.html0528 11 1
(integer) 0
127.0.0.1:6379> getbit item.html0528 33
(integer) 0
127.0.0.1:6379> getbit item.html0528 11
(integer) 1
127.0.0.1:6379> bitcount item.html0528
(integer) 2
127.0.0.1:6379>
```

优点： 占用内存更小，查询方便，可以指定查询某个用户，数据可能略有瑕疵，对于非登陆的用户，可能不同的 key 映射到同一个 id，否则需要维护一个非登陆用户的映射，有额外的开销。

缺点： 如果用户非常的稀疏，那么占用的内存可能比方法一更大。

3、**概率算法**

对于拼多多这种多个页面都可能非常多访问量的网站，如果所需要的数量不用那么准确，可以使用概率算法。

事实上，我们对一个网站的 UV 的统计，1 亿跟 1 亿零 30 万其实是差不多的。

在 Redis 中，已经封装了 HyperLogLog 算法，他是一种基数评估算法。这种算法的特征，一般都是数据不存具体的值，而是存用来计算概率的一些相关数据。

```
127.0.0.1:6379>
127.0.0.1:6379>
127.0.0.1:6379> pfadd pay.html0528 111 222 123 asb
(integer) 1
127.0.0.1:6379> pfcount pay.html0528
(integer) 4
127.0.0.1:6379>
```

当用户访问网站的时候，我们可以使用 PFADD 命令，设置对应的命令，最后我们只要通过 PFCOUNT 就能顺利计算出最终的结果，因为这个只是一个概率算法，所以可能存在 0.81%的误差。

优点： 占用内存极小，对于一个 key，只需要 12kb。对于拼多多这种超多用户的特别适用。

缺点： 查询指定用户的时候，可能会出错，毕竟存的不是具体的数据。总数也存在一定的误差。

上面就是常见的 3 种适用 Redis 统计网站用户访问数的方法了。

50. shoppe 面试题实时订阅推送设计与实现

什么是订阅推送？就是用户订阅了优惠券的推送，在可领取前的一分钟就要把提醒信息推送到用户的 app 中。具体方案就是到具体的推送时间点了，coupon 系统调用消息中心的推送接口，把信息推送出去。

下面我们分析一下这个功能的业务情景。假设公司目前注册用户 6000W+，比如有一张无门槛的优惠券下单立减 20 元，那么抢这张券的人就会比较多，我们保守估计 10W+，百万级别不好说。我们初定为 20W 万人，那么这 20W 条推送信息要在一分钟推送完成！并且一个用户是可以订阅多张券的。所以我们知道了这个订阅功能的有两个突出的难点：

1、**推送的实效性**：推送慢了，用户会抱怨没有及时通知他们错过了开抢时机。

2、**推送的体量大**：爆款的神券，人人都想抢！

然而推送体量又会影响到推送的实效性。这真是一个让人头疼的问题！

那就让我们把问题一个个解决掉吧！

推送的实效性的问题：当用户在领券中心订阅了某个券的领取提醒后，在后台就会生成一条用户的订阅提醒记录，里面记录了在哪个时间点给用户发送推送信息。所以问题就变成了系统如何快速实时选出哪些要推送的记录！

- 方案 1：**MQ 的延迟投递。**MQ 虽然支持消息的延迟投递，但是不适合用来做精确时间点投递！并且用户执行订阅之后又取消订阅的话，要把发出去的 MQ 消息 delete 掉这个操作有点头大，短时间内难以落地！并且用户可以取消之后再订阅，这又涉及到去重的问题。所以 MQ 的方案否掉。
- 方案 2：**传统定时任务。**这个相对来说就简单一点，用定时任务是去 db 里面 load 用户的订阅提醒记录，从中选出当前可以推送的记录。但有句话说得好任何脱离实际业务的设计都是耍流氓。下面我们就分析一下传统的定时任务到底适不适合我们的这个业务！

能否支持多机同时跑	**一般不能，同一时刻只能单机跑。**
存储数据源	一般是 mysql 或者其它传统数据库，并且是单
频率	支持秒、分、时、天，一般不能太快

如上表格所示，可以看到一般传统的定时任务存在以下缺点：

1、**性能瓶颈。**只有一台机在处理，在大体量数据面前力不从心！

2、**实效性差。**定时任务的频率不能太高，太高会业务数据库造成很大的压力！

3、**单点故障。**万一跑的那台机挂了，那整个业务不可用了。这是一个很可怕的事情！

所以传统定时任务也不太适合这个业务。

那有其他解决方案吗？其实可以对传统的定时任务做一个简单的改造即可！把它变成可以同时多机跑，并且实效性可以精确到秒级，并且拒绝单点故障的定时任务集群！这其中就要借助我们的强大的 redis 了。

- 方案 3：**定时任务集群**

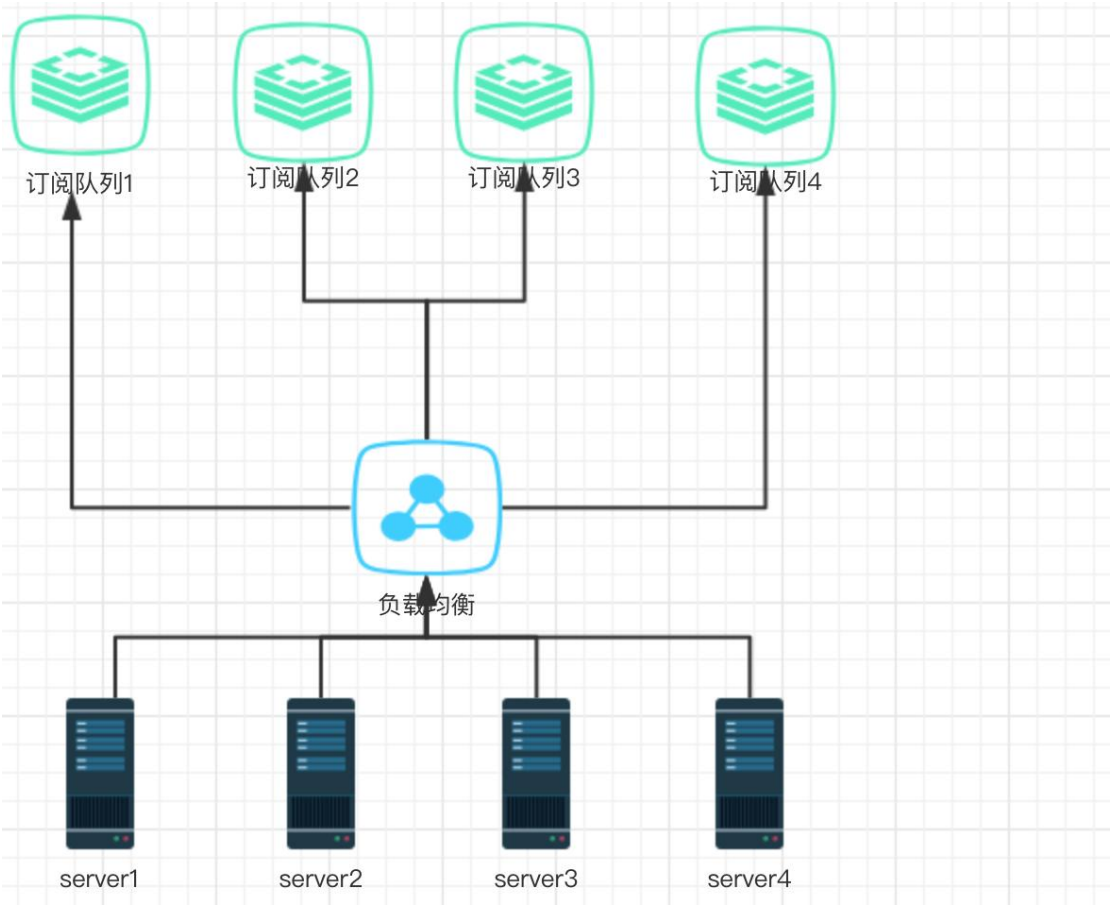
首先我们要定义定时任务集群要解决的三个问题！

1、**实效性要高**

2、**吞吐量要大**

3、**服务要稳定**，不能有单点故障

下面是整个定时任务集群的架构图。



架构很简单：我们把用户的订阅推送记录存储到 redis 集群的 sortedSet 队列里面，且以提醒用户提醒时间戳作为 score 值，然后在我们个每业务 server 里面起一个定时器频率是秒级，我的设定就是 1s，然后经过负载均衡之后从某个队列里面获取要推送的用户记录进行推送。下面我们分析以下这个架构。

1、性能：除去带宽等其它因素，基本与机器数成线性相关。机器数量越多吞吐量越大，机器数量少时相对的吞吐量就减少。

2、实效性：提高到了秒级，效果还可以接受。

3、单点故障？不存在的！除非 redis 集群或者所有 server 全挂了。

这里解释一下为什么用 redis?

1. redis 可以作为一个高性能的存储 db，性能要比 MySQL 好很多，并且支持持久化，稳定性好。

2. redis SortedSet 队列天然支持以时间作为条件排序，完美满足我们选出要推送的记录。

既然方案已经有了，怎么实现呢?

首先我们以 user_id 作为 key，然后 mod 队列数 hash 到 redis SortedSet 队列里面。为什么要这样呢，因为如果用户同时订阅了两张券并且推送时间很近，这样的两条推送就可以合并成一条，并且这样 hash 也相对均匀。下面是部分代码的截图:

```
String subscribeContent = generateSubscribeContent(userId, code);
Long add = redisManager.zadd(getSubscribeQueue(userId), grantTime.getTime(), subscribeContent);
```

然后要决定队列的数量，一般正常来说我们有多少台处理的服务器就定义多少条队列。因为队列太少，会造成队列竞争，太多可能会导致记录得不到及时处理。

然而最佳实践是队列数量应该是可动态配置化的，因为线上的集群机器数是会经常变的。大促的时候我们会加机器是不是，并且业务量增长了，机器数也是会增加是不是~。所以我是借用了淘宝的 diamond 进行队列数的动态配置。

```
@DiamondValue(dataId ="coupon.subscribe.queue.size")
public static Integer COUPON_SUBSCRIBE_QUEUE_SIZE = 10;
```

我们每次从队列里面取多少条记录也是可以动态配置的

```
@DiamondValue(dataId ="coupon.subscribe.queue.poll.size")
public static Integer COUPON_SUBSCRIBE_QUEUE_POOL_SIZE = 2000;
```

这样就可以随时根据实际的生产情况调整整个集群的吞吐量。 所以我们的定时任务集群还是具有一个特性就是支持动态调整。

最后一个关键组件就是负载均衡了。这个是非常重要的！因为这个做得不好就会可能导致多台机竞争同时处理一个队列，影响整个集群的效率！在时间很紧的情况下我就用了

一个简单的方案，利用 redis 一个自增 key，然后 mod 队列数量算法。这样就很大程度上就保证不会有两台机器同时去竞争一条队列。

```
private String getSubscribeQueue(){
    return "coupon_subscribe_" + (redisManager.incr(COUPON_SUBSCRIBE_QUEUE_INCR_KEY) % COUPON_SUBSCRIBE_QUEUE_SIZE);
}
```

最后我们算一下整个集群的吞吐量

10（机器数） * 2000（一次拉取数） = 20000。然后以 MQ 的形式把消息推送到消息中心，发 MQ 是异步的，算上其它处理 0.5s。

其实发送 20W 的推送也就是 10 几 s 的事情。

到这里我们整个定时任务集群就差不多基本落地好了。反过来仔细思考了一下，有以下可以**优化的点**：

1、加监控， 集群怎么可以木有监控呢，万一出问题有任务堆积怎么办。

2、加上可视化界面。

3、最好有智能调度，增加任务优先级。确保优先级高的任务先运行。

4、资源调度，万一机器数量不够，力不从心，优先保证重要任务执行。

51. [阿里一面]购物车系统怎么设计?

1 主要功能

在用户选购商品时，下单前，暂存用户想购买的商品。

购物车对数据可靠性要求不高，性能也无特别要求，在整个电商系统是相对容易设计和实现的一个子系统。

购物车系统的主要功能：

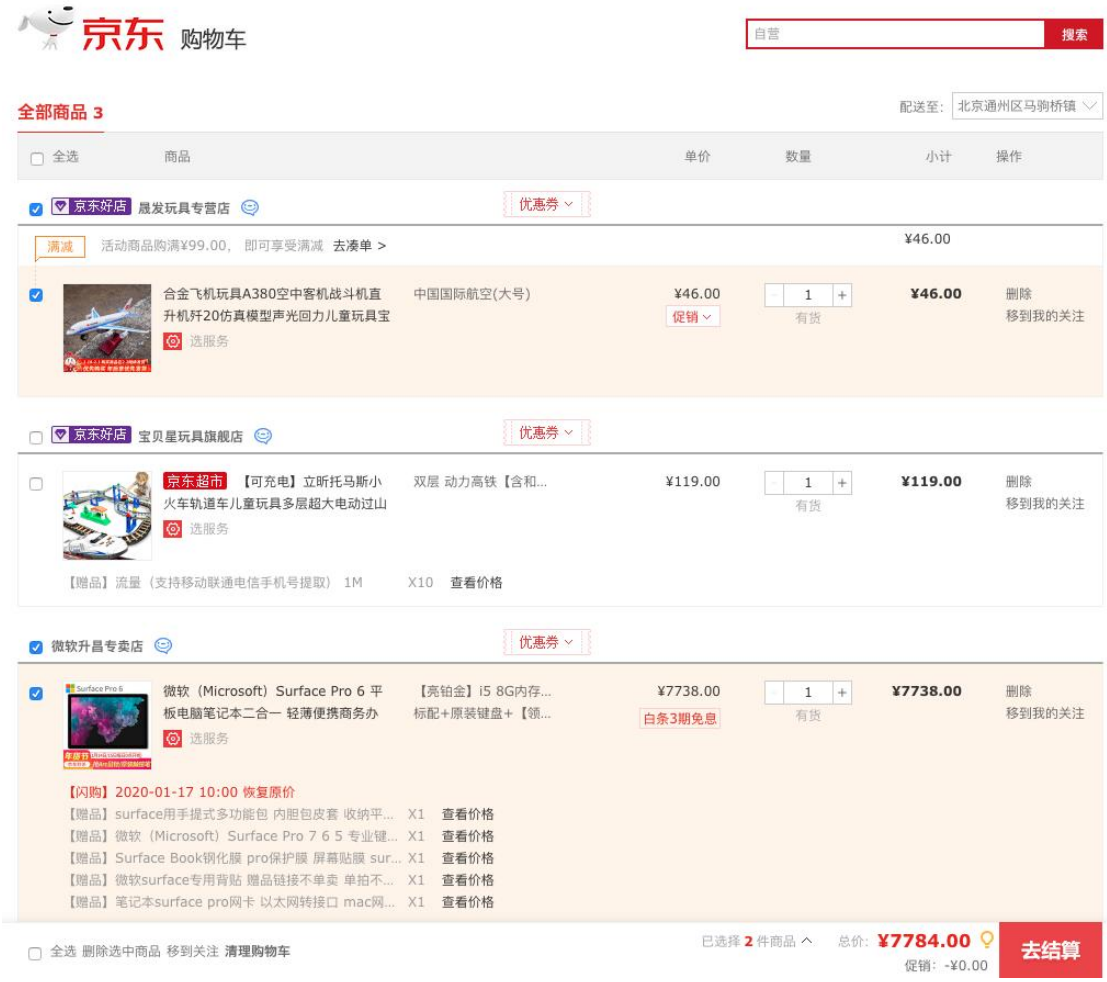
- 把商品加入购物车（后文称“加购”）
- 购物车列表页
- 发起结算下单
- 在所有界面都要显示的购物车小图标

支撑这些功能，存储模型如何设计?

只要一个“购物车”实体。

2 主要属性

打开京东购物车页面：SKUID（商品 ID）、数量、加购时间和勾选状态



“勾选状态”属性，即在购物车界面，每件商品前面的那个小对号，表示在结算下单时，是否要包含这件商品。至于商品价格和总价、商品介绍等都能实时从其他系统获取，无需购物车系统保存。

购物车功能简单，但设计购物车系统的存储时，仍有一些问题需考虑。

3 原则

3.1 思考

3.1.1 用户未登录，在浏览器中加购，关闭浏览器再打开，刚才加购的商品还在吗?

存在。

若用户未登录，加购的商品也会被保存在用户的电脑。即使关闭浏览器再打开，购物车的商品仍存在。

3.1.2 用户未登录，在浏览器中加购，然后登录，刚才加购的商品还在吗?

存在。

若用户先加购，再登录。登录前加购的商品就会被自动合并到用户名下，所以登录后购物车中仍有登录前加购的商品。

3.1.3 关闭浏览器再打开，上一步加购的商品还在吗?

不存在。

关闭浏览器再打开，这时又变为未登录状态，但是之前未登录时加购的商品已经被合并到刚刚登录的用户名下了，所以购物车是空的。

3.1.4 再打开手机，用相同的用户登录，第二步加购的商品还在吗?

存在。使用手机登录相同的用户，看到的就是该用户的购物车，这时无论你在手机 App、电脑还是微信中登录，只要相同用户，看到就是同一购物车，所以第 2 步加购的商品是存在的。

若不仔细把这些问题考虑清楚，用户使用购物车时，就会感觉不好用，不是加购的商品莫名其妙丢了，就是购物车莫名其妙多出一些商品。

要解决上面这些问题，只要在存储设计时，把握如下

3.2 原则

- • 若未登录，需临时暂存购物车的商品
- • 用户登录时，把暂存购物车的商品合并到用户购物车，并清除暂存购物车
- • 用户登录后，购物车中的商品，需在浏览器、手机 APP 和微信等等这些终端保持同步

购物车系统需保存两类购物车：

- • 未登录情况下的“暂存购物车”
- • 登录后的“用户购物车”

4 “暂存购物车”存储设计

4.1 保存在客户端 or 服务端?

若存在服务端，则每个暂存购物车都得有个全局唯一标识，这不易设计。保存在服务端，还要浪费服务端资源。所以，肯定保存在客户端：

- • 节约服务器存储资源
- • 无购物车标识问题每个客户端就保存它自己唯一一个购物车即可，无需标识。

客户端存储可选择不多：

- • Session 不太合适。SESSION 保留时间短，且 SESSION 的数据实际上还是保存在服务端
- • Cookie
- • LocalStorage 浏览器的 LocalStorage 和 App 的本地存储类似，都以 LocalStorage 代表。Cookie、LocalStorage 都可用来保存购物车数据。

选择哪种更好？各有优劣。这场景中，使用 Cookie 和 LocalStorage 最关键区别：

- • 客户端、服务端的每次交互，都会自动带着 Cookie 数据往返，这样服务端可读写客户端 Cookie 中的数据
- • LocalStorage 里的数据，只能由客户端访问

使用 Cookie 存储，实现简单，加减购物车、合并购物车过程，由于服务端可读写 Cookie，这样全部逻辑都可在服务端实现，并且客户端和服务端请求的次数也相对少。

使用 LocalStorage 存储，实现相对复杂，客户端和服务端都要实现业务逻辑，但 LocalStorage 好在其存储容量比 Cookie 的 4KB 上限大得多，而且不用像 Cookie 那样，无论用不用，每次请求都要带着，可节省带宽。

所以，选择 Cookie 或 LocalStorage 存储“暂存购物车”都行，根据优劣势选型即可：

- • 设计的是个小型电商，Cookie 存储实现起来更简单
- • 你的电商是面那种批发的行业用户，用户需加购大量商品，Cookie 可能容量不够用，选择 LocalStorage 更合适

不管哪种存储，暂存购物车保存的

4.2 数据格式

都一样。参照实体模型设计，JSON 表示：

```
{
    "cart": [
        {
            "SKUID": 8888,
            "timestamp": 1578721136,
            "count": 1,
            "selected": true
        },
        {
            "SKUID": 6666,
            "timestamp": 1578721138,
            "count": 2,
            "selected": false
        }
    ]
}
```

5 用户购物车 存储设计

用户购物车须保证多端数据同步，数据须保存在服务端。常规思路：设计一张购物车表，把数据存在 MySQL。表结构同样参照实体模型：

列名	数据类型	主键	非空	说明
id	BIGINT	是	是	自增主键
user_id	BIGINT		是	用户ID
sku_id	BIGINT		是	商品ID
count	INT		是	商品数量
timestamp	DATE		是	加购时间
selected	TINYINT (1)			购选状态

需在 user_id 建索引，因为查询购物车表，都以 user_id 作为查询条件。

也可选择更快的 Redis 保存购物车数据：

- • 用户 ID=Key
- • Redis 的 HASH=Value，保存购物车中的商品

如：

```
{
    "KEY": 6666,
    "VALUE": [
        {
            "FIELD": 8888,
            "FIELD_VALUE": {
                "timestamp": 1578721136,
                "count": 1,
                "selected": true
            }
        },
        {
            "FIELD": 6666,
            "FIELD_VALUE": {
                "timestamp": 1578721138,
                "count": 2,
                "selected": false
            }
        }
    ]
}
```

为便理解，用 JSON 表示 Redis 中 HASH 的数据结构:

- • KEY 中的值 6666 是用户 ID
- • FIELD 存放商品 ID
- • FIELD_VALUE 是个 JSON 字符串，保存加购时间、商品数量和勾选状态

读写性能，Redis 比 MySQL 快得多，Redis 就一定比 MySQL 好吗?

5.1 MySQL V.S Redis 存储

- • Redis 性能比 MySQL 高出至少一个量级，响应时间更短，支撑更多并发请求
- • MySQL 数据可靠性好于 Redis，因为 Redis 异步刷盘，若服务器掉电，Redis 有可能丢数据。但考虑到购物车里的数据，对可靠性要求不高，丢少量数据的后果也

就是，个别用户的购物车少了几件商品，问题不大。所以，购物车场景，Redis 数据可靠性不高这个缺点，不是不能接受

- • MySQL 另一优势：支持丰富的查询方式和事务机制，但对购物车核心功能无用。但每个电商系统都有它个性化需求，若需以其他方式访问购物车数据，如统计今天加购的商品总数，这时，使用 MySQL 存储数据，易实现，而使用 Redis 存储，查询麻烦且低效

综合比较下来，考虑到需求变化，推荐 MySQL 存储购物车数据。若追求性能或高并发，也可选择使用 Redis。

设计存储架构过程就是不断抉择过程。很多情况下，可选择方案不止一套，选择时需考虑实现复杂度、性能、系统可用性、数据可靠性、可扩展性等。这些条件每一个都不是绝对不可以牺牲的，不要让一些“所谓的常识”禁锢思维。

比如，一般认为数据绝不可丢，即不能牺牲数据可靠性。但用户购物车存储，使用 Redis 替代 MySQL，就是牺牲数据可靠性换取高性能。很低概率的丢失少量数据可接受。性能提升带来的收益远大于丢失少量数据而付出的代价，这选择就值得。

如果说不考虑需求变化这个因素，牺牲一点点数据可靠性，换取大幅性能提升，Redis 是最优解。

6 总结

- • 购物车系统的主要功能包括：加购、购物车列表页和结算下单
- • 核心实体：只有一个“购物车”实体
- • 至少包括：SKUID、数量、加购时间和勾选状态属性

在给购物车设计存储时，为确保：

- • 购物车内的数据在多端一致
- • 用户登录前后购物车内商品能无缝衔接

除了每个用户的“用户购物车”，还要实现一个“暂存购物车”保存用户未登录时加购的商品，并在用户登录后自动合并“暂存购物车”和“用户购物车”。

暂存购物车存储在客户端浏览器或 App，可存放到 Cookie 或 LocalStorage。用户购物车保存在服务端，可以选择使用：

- • Redis 存储会有更高的性能，可以支撑更多的并发请求

- • MySQL 是更常规通用的方式，便于应对变化，系统扩展性更好

思考

既然用户的购物车数据存放在 MySQL 或 Redis 各有优劣。那能否把购物车数据存在 MySQL，并用 Redis 缓存？不就兼顾二者优势？若可行，如何保证 Redis 中的数据和 MySQL 数据一致性?

用 Redis 给购物车库做缓存，技术可行。但考虑：

- 值得吗？每个人的购物车都不一样，所以这个缓存它的读写比差距不会很大，缓存命中率不会太高，缓存收益有限，为维护缓存，还会增加系统复杂度。所以我们就要自行权衡一下，是不是值得的问题。除非超大规模系统，否则没必要设置这缓存
- 若非要做这样一个缓存，用什么缓存更新策略?

52. [合集]2024 年一线大厂面试真题

46.1 [美团]CMS 的垃圾回收过程。为啥要分成 4 步

46.2 [美团]为啥初始标记和重新标记需要 STW?

46.3 [美团]什么东西可以当做 GC Root，跨代引用怎么办?

46.4 [美团]安全点了解吗？如果有一段代码进不了安全点怎么办?

46.5 [美团]除了 GC 还有其他场景用安全点吗?

46.6 [美团]如果外部接口的 RT 无法保证，如何处理?

46.7 [美团]completableFuture 和 executorService 区别是啥?

46.8 [美团]分库分表方案中出现数据倾斜问题怎么解决?

46.9 [美团]Redis 实现分布锁的时候，哪些问题需要考虑?

46.10 [美团]公平锁和非公平锁区别是啥？公平锁的缺点? reentrantLock 是如何实现公平锁和非公平锁的?

46.11 [美团]Mysql 的行级锁到底锁的是什么东西?

46.12 [阿里]Kafka 的重平衡问题介绍下？重平衡的过程是怎么样？重平衡过程的问题有哪些?

46.13 [阿里]Kafka 的消费者可能有哪些状态？kafka 的高水位是什么

46.14 [阿里]Pull 对 MQ 造成压力怎么办？拉的很慢消息堆积了怎么办？堆积之后如何解决?

46.15 [阿里]假如让你实现一个消息队列，会如何实现？考虑哪些问题呢?

46.16 [阿里]Synchronized 的自旋是怎么实现的？代码上是如何实现的?

46.17 [阿里]如何实现一个类加载器？为什么是 LoadClass 而不是 findClass?

46.18 [阿里]如何实现登录用户可以存 1000 sku,未登录用户只能存 200 sku

46.19 [阿里]如果需要实现一个凑单的功能，你会考虑怎么设计?

46.20 [阿里]近端缓存是什么？相比于本地缓存和分布式缓存有什么优缺点？一致性问题如何解决?

46.21 [阿里]如果让你实现一个 dubbo，会考虑用那些技术解决哪些问题?

46.22 [腾讯]DUBBO 的负载均衡是如何实现的？服务端挂了怎么避免被调用到?

46.23 [腾讯]设计一个订单号的生成服务，该如何设计?

46.24 [腾讯]热点商家交易订单的写入如何处理?

46.25 [腾讯]和外部机构的 API 交互如何防止外部机构服务不可用拖垮调用服务

46.26 [腾讯]微信运动排行榜如何设计

46.27 [腾讯]如何降低软件复杂度?

46.28 [腾讯]如何利用 AOP 切面实现多数据源切换?

46.29 [腾讯]为啥有了基本类型还需要包装类?

46.30 [腾讯]分布式锁加锁失败后的等待逻辑是如何实现的?

46.31 [腾讯]Netty 相比原生的 IO 模型有啥优势?

46.32 [腾讯]Innodb 的索引结构和 myisam 有区别吗?

46.33 [腾讯]创建索引的时候需要考虑哪些问题?

46.34 [腾讯]什么时候会考虑用联合索引?如果只有一个条件查就没有建联合索引的必要了么?

46.35 [腾讯]如果查询优化器选错了索引怎么办?

46.36 [腾讯]两个动作，下订单和扣钱，怎么保证只能扣一次钱

46.37 [腾讯]nacos 是怎么推送配置到本地的? 新增和修改有区别吗?

46.38 [腾讯]redis 里面 lru，lfu 缺点是什么? 有什么解决的办法?

46.39 [小米]JVM 中一次完整的 GC 流程（从 ygc 到 fgc）是怎样的

46.40 [小米]如果要你实现 Spring IOC，你会注意哪些问题?

46.41 [小米]Spring 是如何管理事务的，事务管理机制?

46.42 [小米]threadlocal 参数如何传递，线程池如何传递

46.43 [小米]Java 中的对象一定在堆上分配吗

46.44 [小米]线程同步和线程协作是怎么理解的

46.45[小米]为什么 k8s 的设计里有 pod，而不是直接用容器

46.46 k8s 中 pod 和 node 是什么关系

46.47 [小米]网关的作用；网关如何挂了还能正常运行吗

46.48 [小米]Mysql 为什么一定要有一个主键?

46.49 [小米]如果让你设计一个推荐系统，你会从哪些纬度来考虑?

46.50 [小米]Redis 中多大的 Key 算热 key，该如何解决?

46.51 [小米]Spring AOP 在什么场景下会失效?

46.52 [小米]如何用 Redis 统计亿级网站的 UV

46.53 [小米]接口防刷怎么实现?

46.54 [小米]应用启动后前面几分钟，RT 较长、Load 和 CPU 都比较高，该如何定位？可能有哪些原因?

46.55 [快手]Dubbo 和 feign 有啥区别？rpc vs http，为啥 rpc 快？他们的负载均衡是如何做的？ribbon，ribbon

是客户端还是服务端的。后面 ribbon 不支持咋办？限流做过吗?

46.56 [快手]当给第三方提供接口调用，需要注意哪些事情?

46.57 [快手]设计 redis 的 key 和 value 上，有哪些原则?

53. [原创合计]2024 年 Java 程序员最新面试"八股文"攻略

53.1 [快手]谈谈你对 AQS 的理解

AQS 是 AbstractQueuedSynchronizer 的简称，是并发编程中比较核心的组件。

在很多大厂的面试中，面试官对于并发编程的考核要求相对较高，简单来说，如果你不懂并发编程，那么你很难通过大厂高薪岗位的面试。

我们来看一下，关于“谈谈你对 AQS 的理解 “，高手是如何回答的!

> AQS 是多线程同步器，它是 J.U.C 包中多个组件的底层实现，如 Lock、CountDownLatch、Semaphore 等都用到了 AQS.
>
> 从本质上来说，AQS 提供了两种锁机制，分别是排它锁，和 共享锁。
>
> 排它锁，就是存在多线程竞争同一共享资源时，同一时刻只允许一个线程访问该共享资源，也就是多个线程中只能有一个线程获得锁资源，比如 Lock 中的 ReentrantLock 重入锁实现就是用到了 AQS 中的排它锁功能。
>
> 共享锁也称为读锁，就是在同一时刻允许多个线程同时获得锁资源，比如 CountDownLatch 和 Semaphore 都是用到了 AQS 中的共享锁功能。

53.2 [快手]fail-safe 机制与 fail-fast 机制分别有什么作用

前段时间一个小伙伴去面试，遇到这样一个问题。

” fail-safe 机制与 fail-fast 机制分别有什么作用 “

他说他听到这个问题的时候，脑子里满脸问号。那么今天我们来看一下，关于这个问题，普通人和高手应该如何回答吧。

以下为面试回答思路

fail-safe 和 fail-fast ，是多线程并发操作集合时的一种失败处理机制。

Fail-fast ： 表示快速失败，在集合遍历过程中，一旦发现容器中的数据被修改了，会立刻抛出 ConcurrentModificationException 异常，从而导致遍历失败，像这种情况（贴下面这个图）。

定义一个 Map 集合，使用 Iterator 迭代器进行数据遍历，在遍历过程中，对集合数据做变更时，就会发生 fail-fast。

java.util 包下的集合类都是快速失败机制的，常见的的使用 fail-fast 方式遍历的容器有 HashMap 和 ArrayList 等。

```
    public static void main(String[] args) {
        Map<String, String> empName = new HashMap<String, String>();
        empName.put("name", "mic");
        empName.put("sex", "male");
        empName.put("age", "18");
        Iterator iterator = empName.keySet().iterator();
        while (iterator.hasNext()) {
            System.out.println(empName.get(iterator.next()));
            empName.put("work", "Java");
        }
    }

>上述程序运行结果如下：
  male
  Exception in thread "main" java.util.ConcurrentModificationException
     at java.util.HashMap$HashIterator.nextNode(HashMap.java:1445)
     at java.util.HashMap$KeyIterator.next(HashMap.java:1469)
     at org.example.cl09.ThreadExample.main(ThreadExample.java:14)
```

Fail-safe，表示失败安全，也就是在这种机制下，出现集合元素的修改，不会抛出 ConcurrentModificationException。

原因是采用安全失败机制的集合容器，在遍历时不是直接在集合内容上访问的，而是先复制原有集合内容，

在拷贝的集合上进行遍历。由于迭代时是对原集合的拷贝进行遍历，所以在遍历过程中对原集合所作的修改并不能被迭代器检测到

比如这种情况（贴下面这个图），定义了一个 CopyOnWriteArrayList，在对这个集合遍历过程中，对集合元素做修改后，不会抛出异常，但同时也不会打印出增加的元素。

java.util.concurrent 包下的容器都是安全失败的,可以在多线程下并发使用,并发修改。

常见的的使用 fail-safe 方式遍历的容器有 ConcerrentHashMap 和 CopyOnWriteArrayList 等。

```
public static void main(String[] args) {
    CopyOnWriteArrayList<Integer> list
            = new CopyOnWriteArrayList<>(new Integer[] { 1, 7, 9, 11 });
    Iterator itr = list.iterator();
    while (itr.hasNext()) {
        Integer i = (Integer)itr.next();
        System.out.println(i);
        if (i == 7)
            list.add(15); // 在fail-safe模式下，这里不会被打印
    }
}
```

53.3 [快手]谈谈你对 Seata 的理解

很多面试官都喜欢问一些“谈谈你对 xxx 技术的理解”，

那么我们来看一下，高手是如何回答这个问题的?

以下为高手的回答思路

1. 在微服务架构下，由于数据库和应用服务的拆分，导致原本一个事务单元中的多个 DML 操作，

变成了跨进程或者跨数据库的多个事务单元的多个 DML 操作，

而传统的数据库事务无法解决这类的问题，所以就引出了分布式事务的概念。

2. 分布式事务本质上要解决的就是跨网络节点的多个事务的数据一致性问题，业内常见的解决方法有两种
    a. 强一致性，就是所有的事务参与者要么全部成功，要么全部失败，全局事务协调者需要知道每个事务参与者的执行状态，再根据状态来决定数据的提交或者回滚！
    b. 最终一致性，也叫弱一致性，也就是多个网络节点的数据允许出现不一致的情况，但是在最终的某个时间点会达成数据一致。

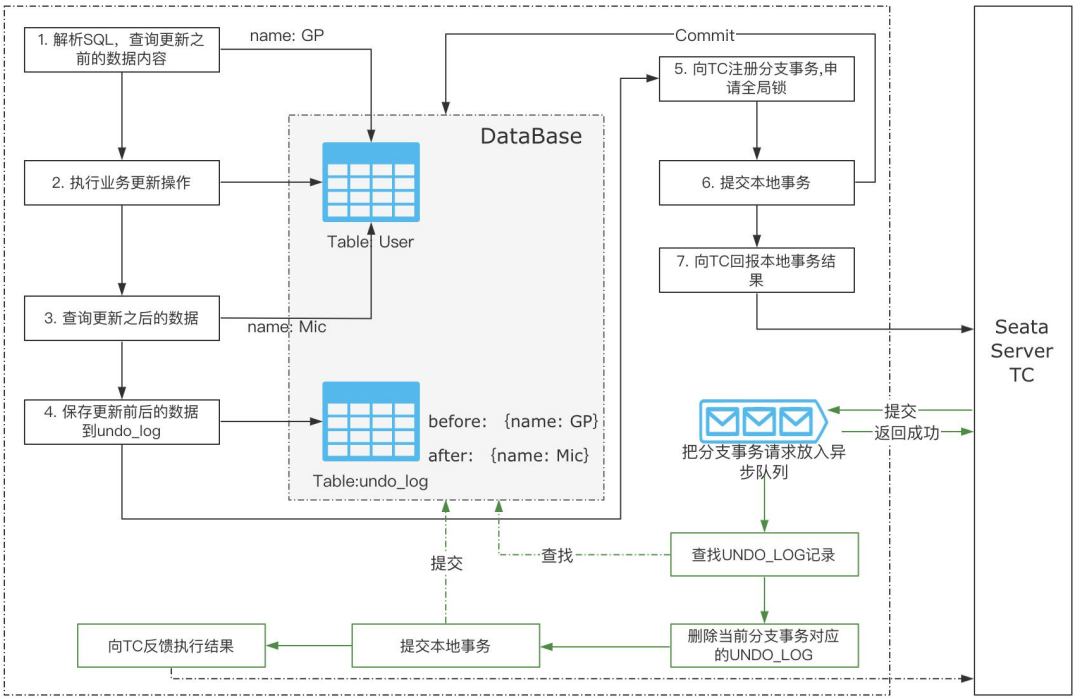
基于 CAP 定理我们可以知道，强一致性方案对于应用的性能和可用性会有影响，所以对于数据一致性要求不高的场景，就会采用最终一致性算法。

3. 在分布式事务的实现上，对于强一致性，我们可以通过基于 XA 协议下的二阶段提交来实现，对于弱一致性，可以基于 TCC 事务模型、可靠性消息模型等方案来实现。
4. 市面上有很多针对这些理论模型实现的分布式事务框架，我们可以在应用中集成这些框架来实现分布式事务。

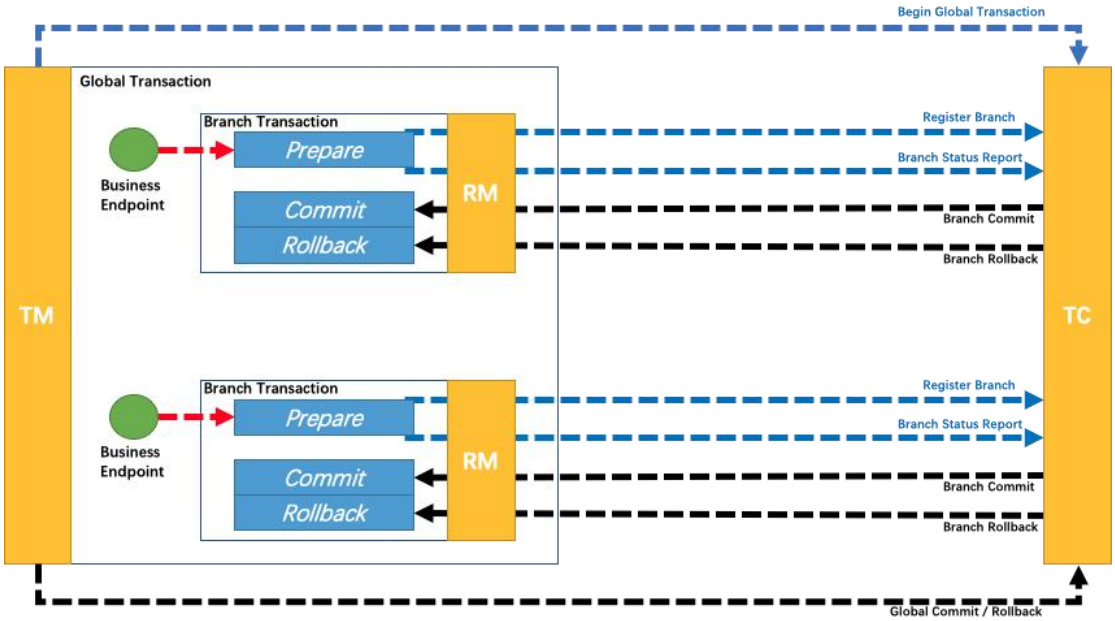
而 Seata 就是其中一种，它是阿里开源的分布式事务解决方案，提供了高性能且简单易用的分布式事务服务。

Seata 中封装了四种分布式事务模式，分别是：

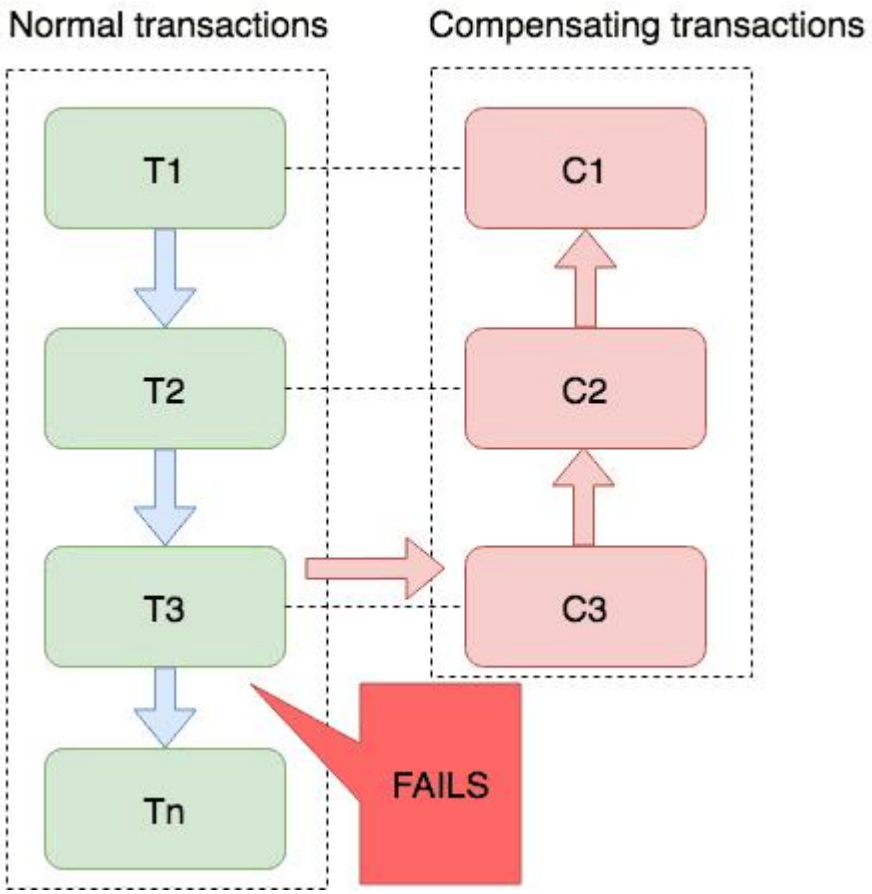
- AT 模式，是一种基于本地事务+二阶段协议来实现的最终数据一致性方案，也是 Seata 默认的解决方案



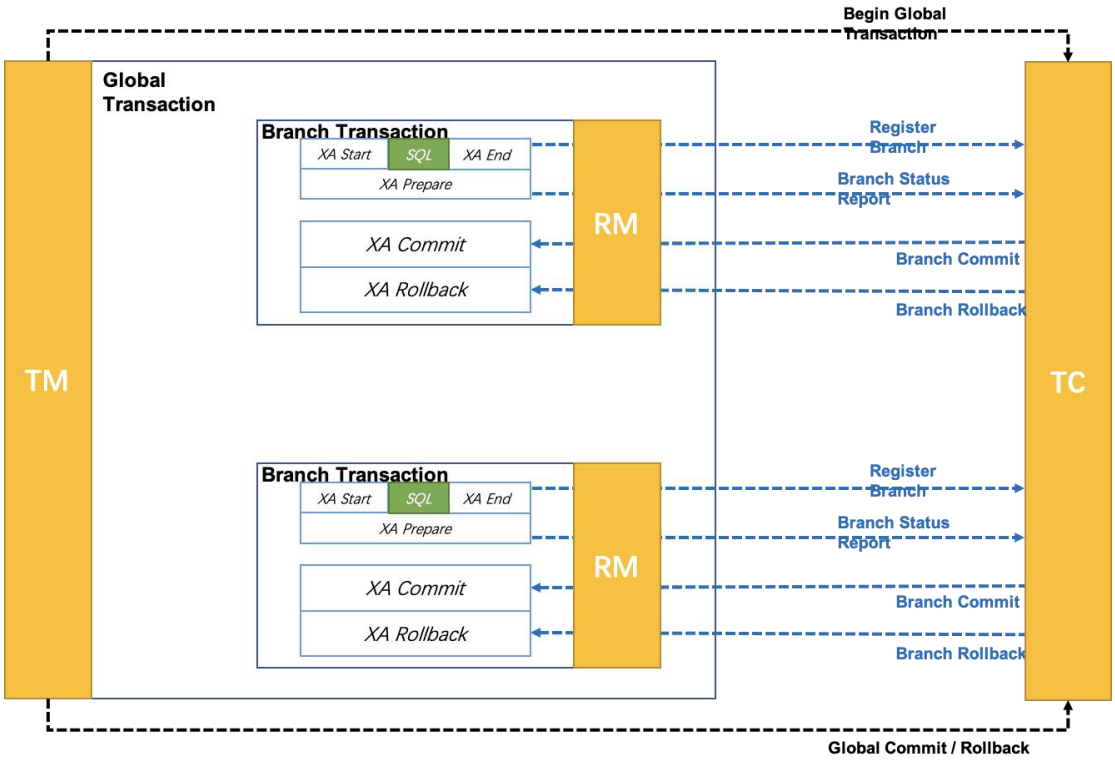
- TCC 模式，TCC 事务是 Try、Confirm、Cancel 三个词语的缩写，简单理解就是把一个完整的业务逻辑拆分成三个阶段，然后通过事务管理器在业务逻辑层面根据每个分支事务的执行情况分别调用该业务的 Confirm 或者 Cacel 方法。



- Saga 模式，Saga 模式是 SEATA 提供的长事务解决方案，在 Saga 模式中，业务流程中每个参与者都提交本地事务，当出现某一个参与者失败则补偿前面已经成功的参与者。



- XA 模式，XA 可以认为是一种强一致性的事务解决方法，它利用事务资源 (数据库、消息服务等) 对 XA 协议的支持，以 XA 协议的机制来管理分支事务的一种 事务模式。



从这四种模型中不难看出，在不同的业务场景中，我们可以使用 Seata 的不同事务模型来解决不同业务场景中的分布式事务问题，因此我们可以认为 Seata 是一个一站式的分布式事务解决方案。

53.4 [快手]Spring Boot 的约定优于配置，你的理解是什么?

对于 Spring Boot 约定优于配置这个问题，看看高手是如何回答的?

以下为高手的回答思路

我从 4 个点方面来回答。

1. 首先， 约定优于配置是一种软件设计的范式，它的核心思想是减少软件开发人员对于配置项的维护，从而让开发人员更加聚焦在业务逻辑上。
2. Spring Boot 就是约定优于配置这一理念下的产物，它类似于 Spring 框架下的一个脚手架，通过 Spring Boot，我们可以快速开发基于 Spring 生态下的应用程序。
3. 基于传统的 Spring 框架开发 web 应用，我们需要做很多和业务开发无关并且只需要做一次的配置，比如

a. 管理 jar 包依赖
b. web.xml 维护
c. Dispatch-Servlet.xml 配置项维护
d. 应用部署到 Web 容器
e. 第三方组件集成到 Spring IOC 容器中的配置项维护

而在 Spring Boot 中，我们不需要再去做这些繁琐的配置，Spring Boot 已经自动帮我们完成了，这就是约定由于配置思想的体现。

4. Spring Boot 约定由于配置的体现有很多，比如
   a. Spring Boot Starter 启动依赖，它能帮我们管理所有 jar 包版本
   b. 如果当前应用依赖了 spring mvc 相关的 jar，那么 Spring Boot 会自动内置 Tomcat 容器来运行 web 应用，我们不需要再去单独做应用部署。
   c. Spring Boot 的自动装配机制的实现中，通过扫描约定路径下的 spring.factories 文件来识别配置类，实现 Bean 的自动装配。
   d. 默认加载的配置文件 application.properties 等等。

总的来说，约定优于配置是一个比较常见的软件设计思想，它的核心本质都是为了更高效以及更便捷的实现软件系统的开发和维护。

以上就是我对这个问题的理解。

53.5 滴滴二面：kafka 的零拷贝原理?

最近一个学员去滴滴面试，在第二面的时候遇到了这个问题：

”请你简单说一下 Kafka 的零拷贝原理“

然后那个学员努力在大脑里检索了很久，没有回答上来。

那么今天，我们基于这个问题来看看，高手是如何回答的！

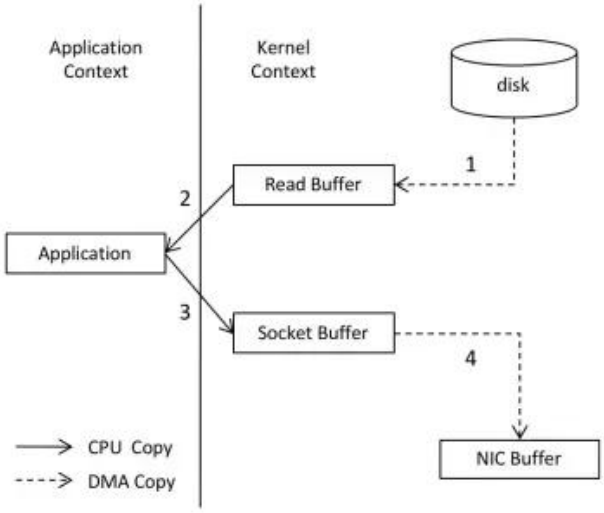
以下为高手的回答思路

在实际应用中，如果我们需要把磁盘中的某个文件内容发送到远程服务器上，如图

那么它必须要经过几个拷贝的过程，如图（贴图）。

1. 从磁盘中读取目标文件内容拷贝到内核缓冲区
2. CPU 控制器再把内核缓冲区的数据赋值到用户空间的缓冲区中

3. 接着在应用程序中，调用 write()方法，把用户空间缓冲区中的数据拷贝到内核下的 Socket Buffer 中。
4. 最后，把在内核模式下的 SocketBuffer 中的数据赋值到网卡缓冲区（NIC Buffer)
5. 网卡缓冲区再把数据传输到目标服务器上。



在这个过程中我们可以发现，数据从磁盘到最终发送出去，要经历 4 次拷贝，而在这四次拷贝过程中，有两次拷贝是浪费的，分别是：

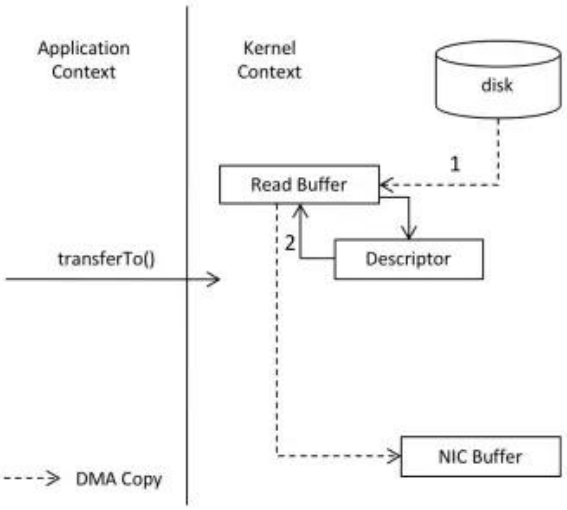
6. 从内核空间赋值到用户空间
7. 从用户空间再次复制到内核空间

除此之外，由于用户空间和内核空间的切换会带来 CPU 的上线文切换，对于 CPU 性能也会造成性能影响。

而零拷贝，就是把这两次多于的拷贝省略掉，应用程序可以直接把磁盘中的数据从内核中直接传输给 Socket，而不需要再经过应用程序所在的用户空间，如下图所示。

零拷贝通过 DMA(Direct Memory Access)技术把文件内容复制到内核空间中的 Read Buffer，接着把包含数据位置和长度信息的文件描述符加载到 Socket Buffer 中，DMA 引擎直接可以把数据从内核空间中传递给网卡设备。

在这个流程中，数据只经历了两次拷贝就发送到了网卡中，并且减少了 2 次 cpu 的上下文切换，对于效率有非常大的提高。



所以，所谓零拷贝，并不是完全没有数据赋值，只是相对于用户空间来说，不再需要进行数据拷贝。对于前面说的整个流程来说，零拷贝只是减少了不必要的拷贝次数而已。

在程序中如何实现零拷贝呢?

- 在 Linux 中，零拷贝技术依赖于底层的 sendfile()方法实现
- 在 Java 中，FileChannal.transferTo() 方法的底层实现就是 sendfile() 方法。

除此之外，还有一个 mmap 的文件映射机制

它的原理是：将磁盘文件映射到内存，用户通过修改内存就能修改磁盘文件。使用这种方式可以获取很大的 I/O 提升，省去了用户空间到内核空间复制的开销。

以上就是我对于 Kafka 中零拷贝原理的理解

53.6 [快手]innoDB 如何解决幻读

前天有个去快手面试的小伙伴私信我，他遇到了这样一个问题： “InnoDB 如何解决幻读”？

这个问题确实不是很好回答，在实际应用中，很多同学几乎都不关注数据库的事务隔离性。

所有问题基本就是 CRUD，一把梭~

那么今天，我们看一下 关于“InnoDB 如何解决幻读”这个问题，高手的回答！

以下为高手的回答

我会从三个方面来回答：

1、 Mysql 的事务隔离级别

Mysql 有四种事务隔离级别，这四种隔离级别代表当存在多个事务并发冲突时，可能出现的脏读、不可重复读、幻读的问题。

其中 InnoDB 在 RR 的隔离级别下，解决了幻读的问题。

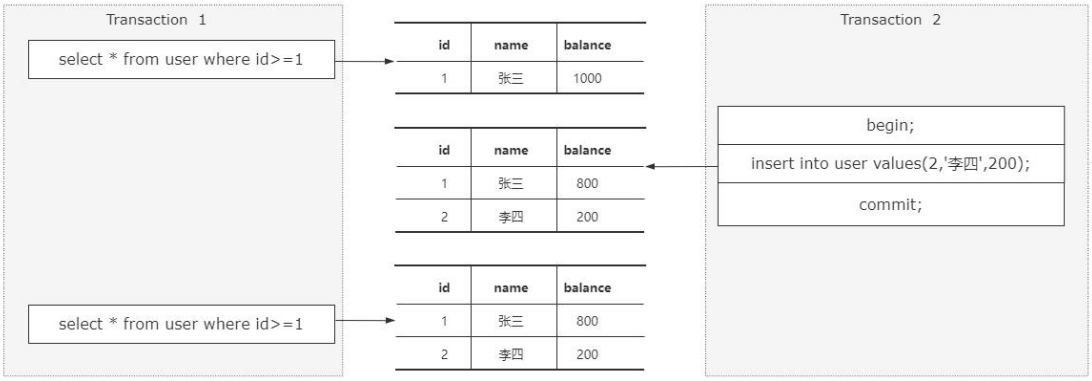
事务隔离级别	脏读	不可重复读	幻读
未提交读（Read Uncommitted）	可能	可能	可能
已提交读（Read Committed）	不可能	可能	可能
可重复读（Repeatable Read）	不可能	不可能	对InnoDB不可能
串行化（Serializable）	不可能	不可能	不可能

2、 **什么是幻读?**

那么， 什么是幻读呢?

幻读是指在同一个事务中，前后两次查询相同的范围时，得到的结果不一致（我们来看这个图）

- 第一个事务里面我们执行了一个范围查询，这个时候满足条件的数据只有一条
- 第二个事务里面，它插入了一行数据，并且提交了
- 接着第一个事务再去查询的时候，得到的结果比第一查询的结果多出来了一条数据。

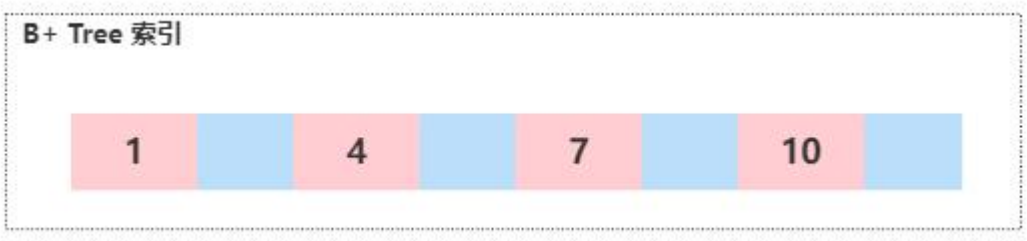


所以，幻读会带来数据一致性问题。

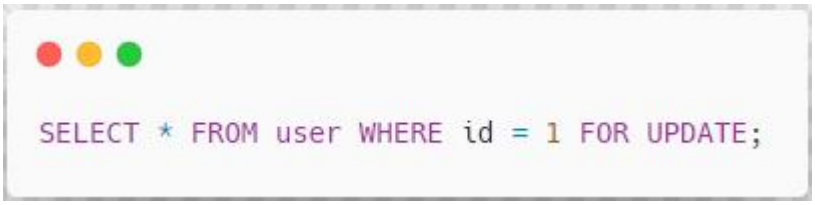
3、 InnoDB 如何解决幻读的问题

InnoDB 引入了间隙锁和 next-key Lock 机制来解决幻读问题，为了更清晰的说明这两种锁，我举一个例子：

假设现在存在这样（图片）这样一个 B+ Tree 的索引结构，这个结构中有四个索引元素分别是：1、4、7、10。

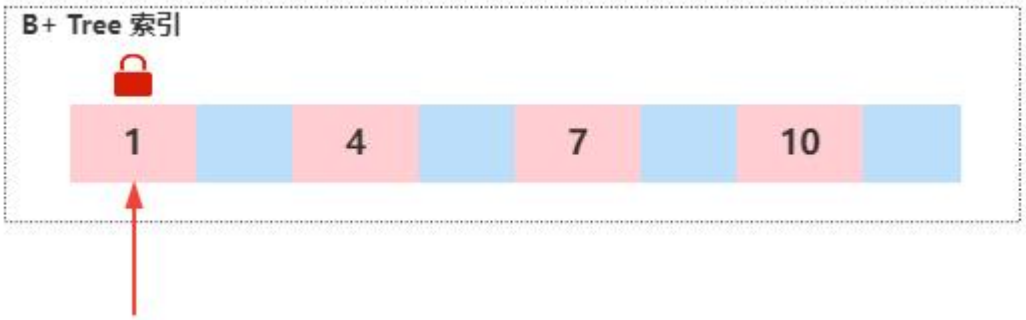


当我们通过主键索引查询一条记录，并且对这条记录通过 `for update` 加锁（请看这个图片）



```
SELECT * FROM user WHERE id = 1 FOR UPDATE;
```

这个时候，会产生一个记录锁，也就是行锁，锁定 `id=1` 这个索引（请看这个图片）。



被锁定的记录在锁释放之前，其他事务无法对这条记录做任何操作。

前面我说过对幻读的定义： 幻读是指在同一个事务中，前后两次查询相同的范围时，得到的结果不一致！

注意，这里强调的是范围查询，

也就是说，InnoDB 引擎要解决幻读问题，必须要保证一个点，就是如果一个事务通过这样一条语句（如图）进行锁定时。

```
SELECT * FROM user WHERE id >4 and id<7 FOR UPDATE;
```

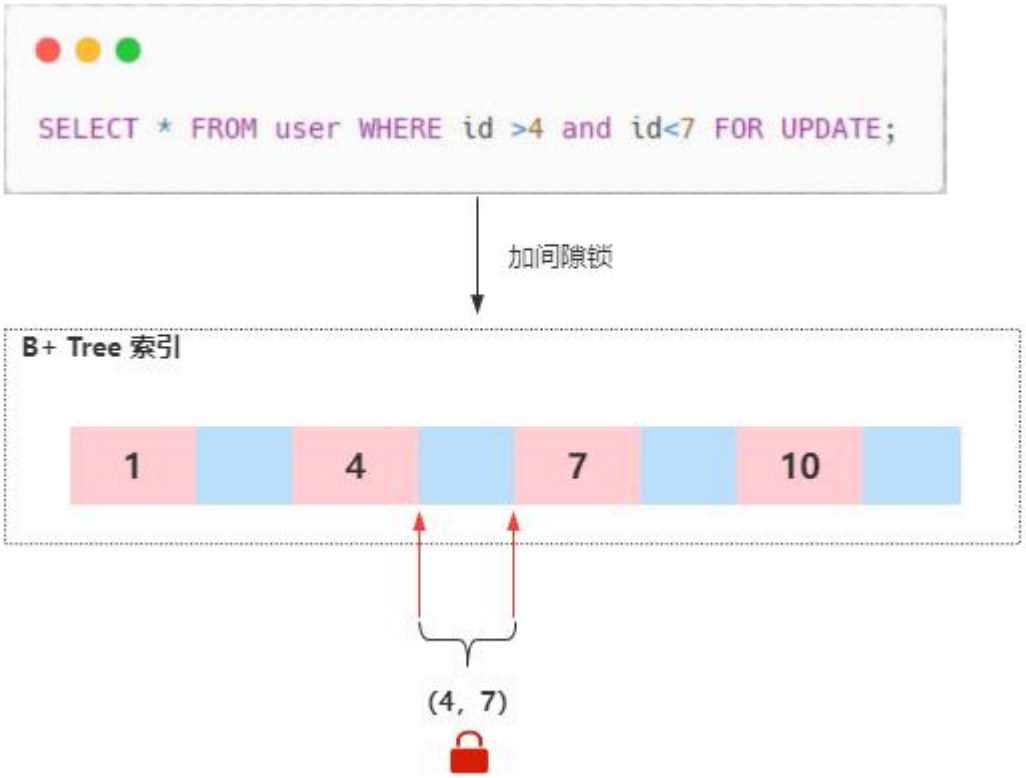
另外一个事务再执行这样一条（显示图片）insert 语句，需要被阻塞，直到前面获得锁的事务释放。

```
INSERT INTO user(id,name) VALUES(5,'mimi');
```

所以，在 InnoDB 中设计了一种间隙锁，它的主要功能是锁定一段范围内的索引记录（如图）

当对查询范围 id>4 and id <7 加锁的时候，会针对 B+树中（4，7）这个开区间范围的索引加间隙锁。

意味着在这种情况下，其他事务对这个区间的数据进行插入、更新、删除都会被锁住。

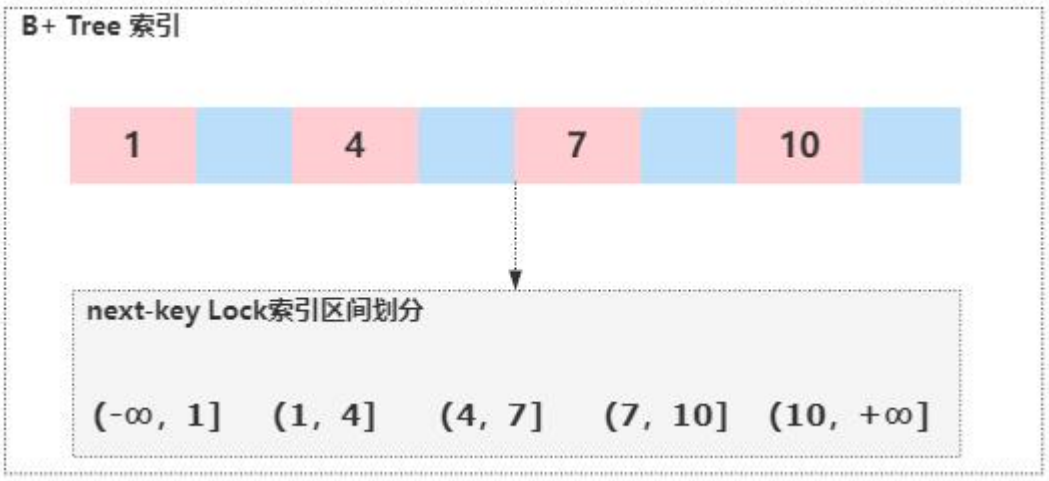


但是，还有另外一种情况，比如像这样（图片）

```
SELECT * FROM user WHERE id>4 FOR UPDATE;
```

这条查询语句是针对 id>4 这个条件加锁，那么它需要锁定多个索引区间，所以在这种情况下 InnoDB 引入了 next-key Lock 机制。

next-key Lock 相当于间隙锁和记录锁的合集，记录锁锁定存在的记录行，间隙锁锁住记录行之间的间隙，而 next-key Lock 锁住的是两者之和。（如图所示）



每个数据行上的非唯一索引列上都会存在一把 **next-key lock**，当某个事务持有该数据行的 **next-key lock** 时，会锁住一段**左开右闭区间**的数据。

因此，当通过 id>4 这样一种范围查询加锁时，会加 next-key Lock，锁定的区间范围是：(4, 7] , (7,10],(10,+∞]

间隙锁和 next-key Lock 的区别在于加锁的范围，间隙锁只锁定两个索引之间的引用间隙，而 next-key Lock 会锁定多个索引区间，它包含记录锁和间隙锁。

当我们使用了范围查询，不仅仅命中了 Record 记录，还包含了 Gap 间隙，在这种情况下我们使用的就是临键锁，它是 MySQL 里面默认的行锁算法。

4 、总结

虽然 InnoDB 中通过间隙锁的方式解决了幻读问题，但是加锁之后一定会影响到并发性能，因此，如果对性能要求较高的业务场景中，可以把隔离级别设置成 RC，这个级别中不存在间隙锁。

以上就是我对于 innoDB 如何解决幻读问题的理解！

53.7 [京东]CPU 飙高系统反应慢怎么排查?

面试过程中，场景类的问题更容易检测出一个开发人员的基本能力。

这不，一个小伙伴去阿里面试，第一面就遇到了关于“CPU 飙高系统反应慢怎么排查”的问题?

对于这个问题，我们来看看高手的回答！

以下为高手的回答

好的，关于这个问题，我从四个方面来回答。

1. CPU 是整个电脑的核心计算资源，对于一个应用进程来说，CPU 的最小执行单元是线程。
2. 导致 CPU 飙高的原因有几个方面
    a. CPU 上下文切换过多，对于 CPU 来说，同一时刻下每个 CPU 核心只能运行一个线程，如果有多个线程要执行，CPU 只能通过上下文切换的方式来执行不同的线程。上下文切换需要做两个事情
        i. 保存运行线程的执行状态
        ii. 让处于等待中的线程执行

这两个过程需要 CPU 执行内核相关指令实现状态保存，如果较多的上下文切换会占据大量 CPU 资源，从而使得 cpu 无法去执行用户进程中的指令，导致响应速度下降。

在 Java 中，文件 IO、网络 IO、锁等待、线程阻塞等操作都会造成线程阻塞从而触发上下文切换

    b. CPU 资源过度消耗，也就是在程序中创建了大量的线程，或者有线程一直占用 CPU 资源无法被释放，比如死循环！

CPU 利用率过高之后，导致应用中的线程无法获得 CPU 的调度，从而影响程序的执行效率！

3. 既然是这两个问题导致的 CPU 利用率较高，于是我们可以通过 `top` 命令，找到 CPU 利用率较高的进程，在通过 `Shift+H` 找到进程中 CPU 消耗过高的线程，这里有两种情况。
    a. CPU 利用率过高的线程一直是同一个，说明程序中存在线程长期占用 CPU 没有释放的情况，这种情况直接通过 jstack 获得线程的 Dump 日志，定位到线程日志后就可以找到问题的代码。
    b. CPU 利用率过高的线程 id 不断变化，说明线程创建过多，需要挑选几个线程 id，通过 jstack 去线程 dump 日志中排查。
4. 最后有可能定位的结果是程序正常，只是在 CPU 飙高的那一刻，用户访问量较大，导致系统资源不够。

以上就是我对这个问题的理解！

53.8 [京东]lock 和 synchronized 区别

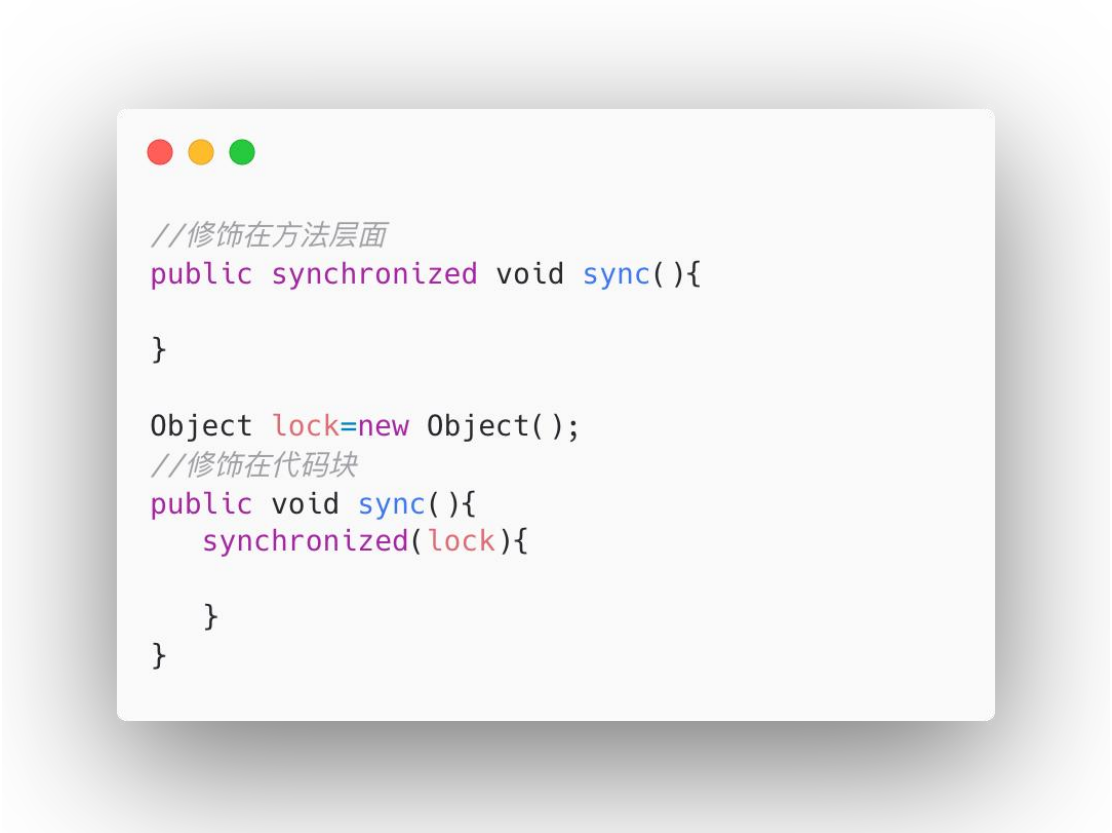
今天来分享一道阿里一面的面试题，“lock 和 synchronized 的区别”。

对于这个问题，看看高手的回答！

以下是高手的回答思路

下面我从 4 个方面来回答

1. 从功能角度来看，Lock 和 Synchronized 都是 Java 中用来解决线程安全问题的工具。
2. 从特性来看，
   a. Synchronized 是 Java 中的同步关键字，Lock 是 J.U.C 包中提供的接口，这个接口有很多实现类，其中就包括 ReentrantLock 重入锁
   b. Synchronized 可以通过两种方式来控制锁的粒度，（贴图）



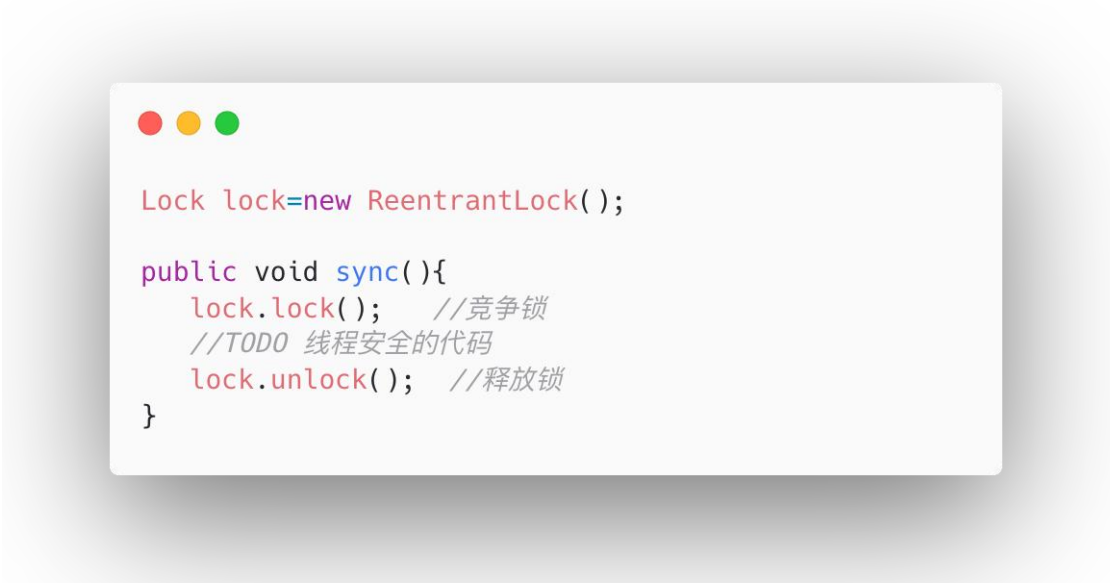
```
//修饰在方法层面
public synchronized void sync(){

}

Object lock=new Object();
//修饰在代码块
public void sync(){
   synchronized(lock){

   }
}
```

一种是把 synchronized 关键字修饰在方法层面，

另一种是修饰在代码块上，并且我们可以通过 Synchronized 加锁对象的声明周期来控制锁的作用范围，比如锁对象是静态对象或者类对象，那么这个锁就是全局锁。

如果锁对象是普通实例对象，那这个锁的范围取决于这个实例的声明周期。

Lock 锁的粒度是通过它里面提供的 lock()和 unlock()方法决定的（贴图），包裹在这两个方法之间的代码能够保证线程安全性。而锁的作用域取决于 Lock 实例的生命周期。



```
Lock lock=new ReentrantLock();

public void sync(){
    lock.lock();    //竞争锁
    //TODO 线程安全的代码
    lock.unlock();  //释放锁
}
```

c. Lock 比 Synchronized 的灵活性更高，Lock 可以自主决定什么时候加锁，什么时候释放锁，只需要调用 lock()和 unlock()这两个方法就行，同时 Lock 还提供了非阻塞的竞争锁方法 tryLock()方法，这个方法通过返回 true/false 来告诉当前线程是否已经有其他线程正在使用锁。

Synchronized 由于是关键字，所以它无法实现非阻塞竞争锁的方法，另外，Synchronized 锁的释放是被动的，就是当 Synchronized 同步代码块执行完以后或者代码出现异常时才会释放。

d. Lock 提供了公平锁和非公平锁的机制，公平锁是指线程竞争锁资源时，如果已经有其他线程正在排队等待锁释放，那么当前竞争锁资源的线程无法插队。而非公平锁，就是不管是否有线程在排队等待锁，它都会尝试去竞争一次锁。Synchronized 只提供了一种非公平锁的实现。

3. 从性能方面来看，Synchronized 和 Lock 在性能方面相差不大，在实现上会有一些区别，Synchronized 引入了偏向锁、轻量级锁、重量级锁以及锁升级的方式来优化加锁的性能，而 Lock 中则用到了自旋锁的方式来实现性能优化。

以上就是我对于这个问题的理解。

53.9 [京东]线程池如何知道一个线程的任务已经执行完成

一个小伙伴私信了一个小米的面试题，问题是： “线程池如何知道一个线程的任务已经执行完成”？

说实话，这个问题确实很刁钻，毕竟像很多工作 5 年多的小伙伴，连线程池都没用过，怎么可能回答出来这个问题呢?

下面我们来看看高手遇到这个问题的回答思路。

以下是高手的回答思路

好的，我会从两个方面来回答。

1. 在线程池内部，当我们把一个任务丢给线程池去执行，线程池会调度工作线程来执行这个任务的 run 方法，run 方法正常结束，也就意味着任务完成了。

所以线程池中的工作线程是通过同步调用任务的 `run()`方法并且等待 `run` 方法返回后，再去统计任务的完成数量。

2. 如果想在线程池外部去获得线程池内部任务的执行状态，有几种方法可以实现。
    a. 线程池提供了一个 `isTerminated()`方法，可以判断线程池的运行状态，我们可以循环判断 `isTerminated()`方法的返回结果来了解线程池的运行状态，一旦线程池的运行状态是 `Terminated`，意味着线程池中的所有任务都已经执行完了。想要通过这个方法获取状态的前提是，程序中主动调用了线程池的 `shutdown()`方法。在实际业务中，一般不会主动去关闭线程池，因此这个方法在实用性和灵活性方面都不是很好。
    b. 在线程池中，有一个 `submit()`方法，它提供了一个 Future 的返回值，我们通过 `Future.get()`方法来获得任务的执行结果，当线程池中的任务没执行完之前，`future.get()`方法会一直阻塞，直到任务执行结束。因此，只要 `future.get()`方法正常返回，也就意味着传入到线程池中的任务已经执行完成了!
    c. 可以引入一个 CountDownLatch 计数器，它可以通过初始化指定一个计数器进行倒计时，其中有两个方法分别是 `await()`阻塞线程，以及 `countDown()`进行倒计时，一旦倒计时归零，所以被阻塞在 `await()`方法的线程都会被释放。

基于这样的原理，我们可以定义一个 CountDownLatch 对象并且计数器为 1，接着在线程池代码块后面调用 `await()`方法阻塞主线程，然后，当传入到线程池中的任务执行完成后，调用 `countDown()`方法表示任务执行结束。

最后，计数器归零 0，唤醒阻塞在 `await()`方法的线程。

```java
public static void main(String[] args) throws InterruptedException {
        ExecutorService executorService= Executors.newFixedThreadPool(10);
        CountDownLatch countDownLatch=new CountDownLatch(1);
        executorService.execute(new Runnable() {
            @Override
            public void run() {
                //开始执行任务
                try {
                    Thread.sleep(3000); //模拟任务执行时间
                    countDownLatch.countDown();  //任务执行结束后，计数器减1
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });
        //阻塞main线程| 当任务执行结束调用countDown()方法使得计数器归零后，唤醒主线程。
        countDownLatch.await();
        executorService.shutdown();
    }
```

3. 基于这个问题，我简单总结一下，不管是线程池内部还是外部，要想知道线程是否执行结束，我们必须要获取线程执行结束后的状态，而线程本身没有返回值，所以只能通过阻塞-唤醒的方式来实现，future.get 和 CountDownLatch 都是这样一个原理。

以上就是我对于这个问题的回答！

53.10 [京东]HashMap 是怎么解决哈希冲突的?

常用数据结构基本上是面试必问的问题，比如 HashMap、LinkList、ConcurrentHashMap 等。

关于 HashMap，有个学员私信了我一个面试题说： “HashMap 是怎么解决哈希冲突的？”

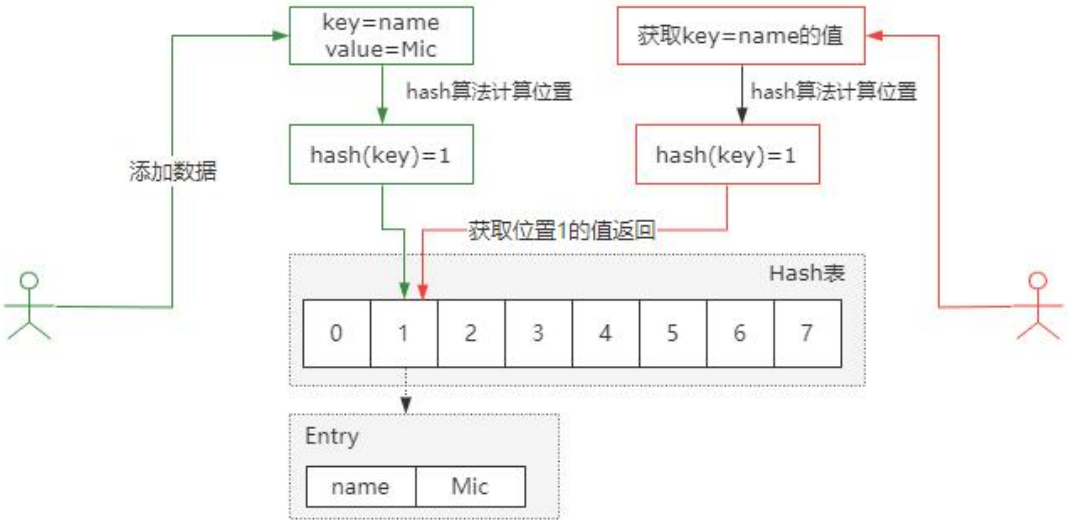
关于这个问题，我们来模拟一下普通人和高手对于这个问题的回答。

> 以下是高手的回答思路

嗯，这个问题我从三个方面来回答。

1. 要了解 Hash 冲突，那首先我们要先了解 Hash 算法和 Hash 表。（如图）
    a. Hash 算法，就是把任意长度的输入，通过散列算法，变成固定长度的输出，这个输出结果是散列值。

b. Hash 表又叫做“散列表”，它是通过 key 直接访问在内存存储位置的数据结构，在具体实现上，我们通过 hash 函数把 key 映射到表中的某个位置，来获取这个位置的数据，从而加快查找速度。

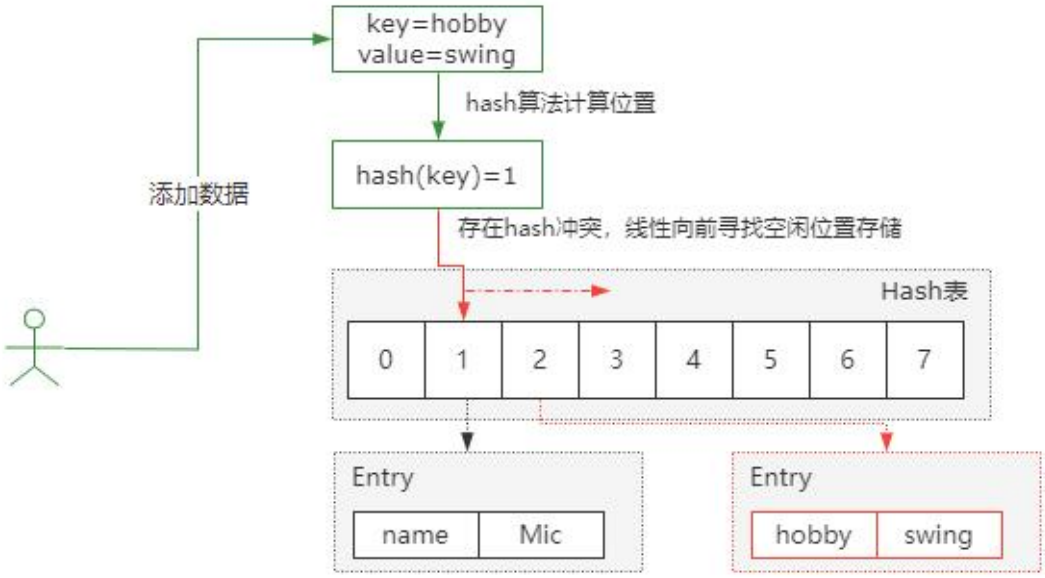


c. 所谓 hash 冲突，是由于哈希算法被计算的数据是无限的，而计算后的结果范围有限，所以总会存在不同的数据经过计算后得到的值相同，这就是哈希冲突。

d. 通常解决 hash 冲突的方法有 4 种。

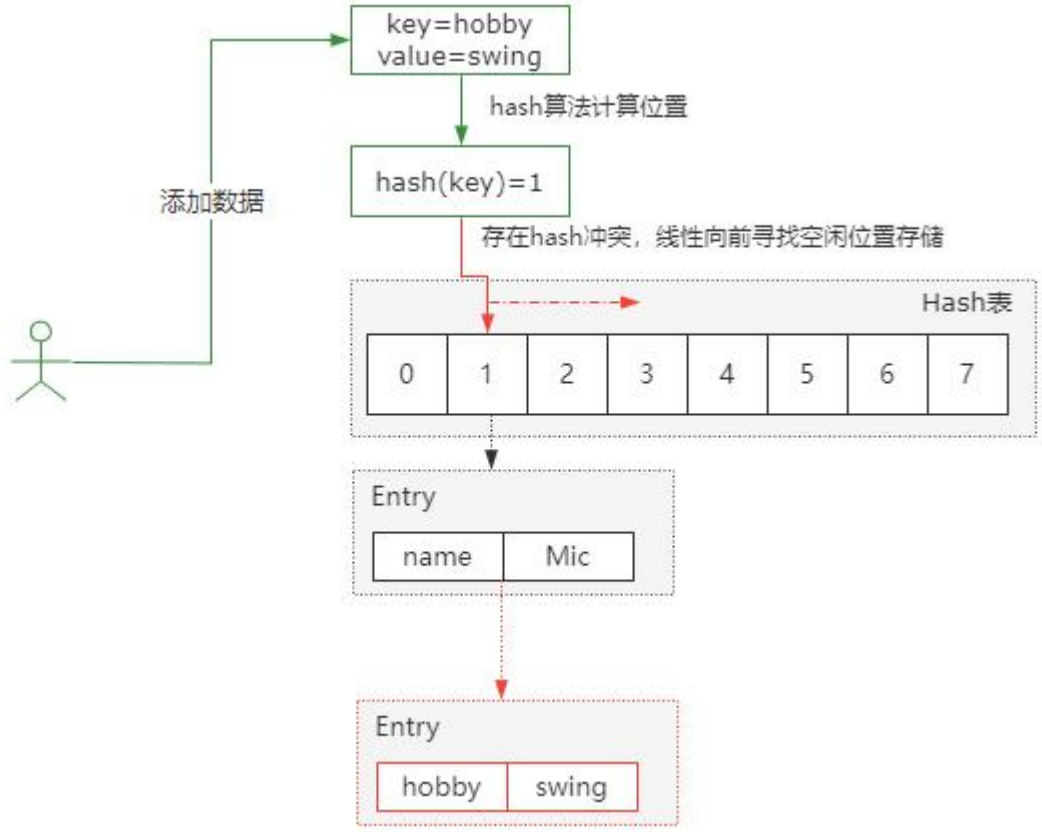
   i. 开放定址法，也称为线性探测法，就是从发生冲突的那个位置开始，按照一定的次序从 hash 表中找到一个空闲的位置，然后把发生冲突的元素存入到这个空闲位置中。ThreadLocal 就用到了线性探测法来解决 hash 冲突的。

向这样一种情况（如图），在 hash 表索引 1 的位置存了一个 key=name，当再次添加 key=hobby 时，hash 计算得到的索引也是 1，这个就是 hash 冲突。而开放定址法，就是按顺序向前找到一个空闲的位置来存储冲突的 key。



ii. 链式寻址法，这是一种非常常见的方法，简单理解就是把存在 hash 冲突的 key，以单向链表的方式来存储，比如 HashMap 就是采用链式寻址法来实现的。

向这样一种情况（如图），存在冲突的 key 直接以单向链表的方式进行存储。



iii. 再 hash 法，就是当通过某个 hash 函数计算的 key 存在冲突时，再用另外一个 hash 函数对这个 key 做 hash，一直运算直到不再产生冲突。这种方式会增加计算时间，性能影响较大。

iv. 建立公共溢出区， 就是把 hash 表分为基本表和溢出表两个部分，凡事存在冲突的元素，一律放入到溢出表中。

e. HashMap 在 JDK1.8 版本中，通过链式寻址法+红黑树的方式来解决 hash 冲突问题，其中红黑树是为了优化 Hash 表链表过长导致时间复杂度增加的问题。当链表长度大于 8 并且 hash 表的容量大于 64 的时候，再向链表中添加元素就会触发转化。

以上就是我对这个问题的理解！

53.11 [京东]什么叫做阻塞队列的有界和无界

昨天一个 3 年 Java 经验的小伙伴私信我，他说现在面试怎么这么难啊！

我只是面试一个业务开发，他们竟然问我： 什么叫阻塞队列的有界和无界。现在面试也太卷了吧!

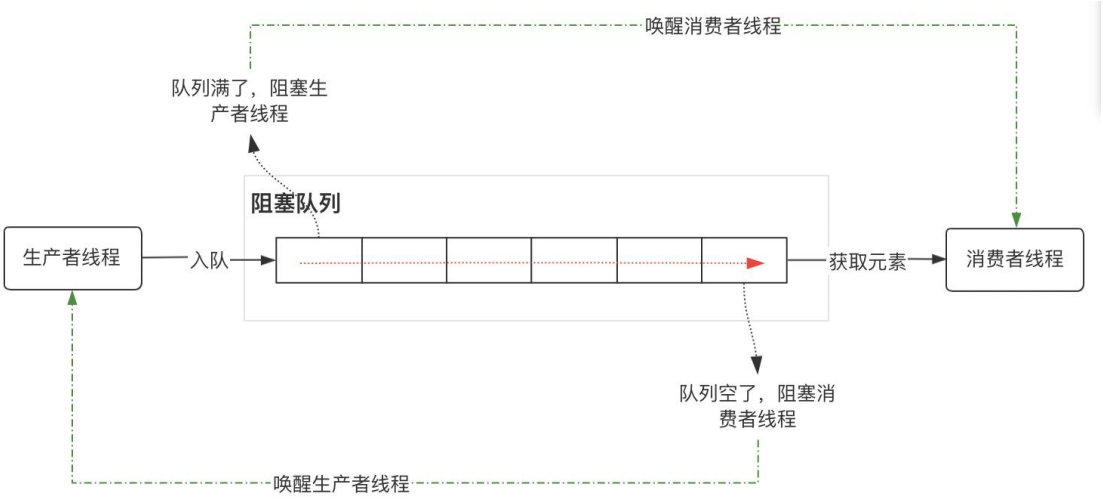
如果你也遇到过类似问题，那我们来看看高手的回答吧！

以下是高手的回答思路

1. （如图），阻塞队列，是一种特殊的队列，它在普通队列的基础上提供了两个附加功能

   a. 当队列为空的时候，获取队列中元素的消费者线程会被阻塞，同时唤醒生产者线程。

   b. 当队列满了的时候，向队列中添加元素的生产者线程被阻塞，同时唤醒消费者线程。



2. 其中，阻塞队列中能够容纳的元素个数，通常情况下是有界的，比如我们实例化一个 ArrayBlockingList，可以在构造方法中传入一个整形的数字，表示这个基于数组的阻塞队列中能够容纳的元素个数。这种就是有界队列。
3. 而无界队列，就是没有设置固定大小的队列，不过它并不是像我们理解的那种元素没有任何限制，而是它的元素存储量很大，像 LinkedBlockingQueue，它的默认队列长度是 Integer.Max_Value，所以我们感知不到它的长度限制。
4. 无界队列存在比较大的潜在风险，如果在并发量较大的情况下，线程池中可以几乎无限制的添加任务，容易导致内存溢出的问题！

以上就是我对这个问题的理解!

53.12 [京东]Dubbo 的服务请求失败怎么处理?

今天分享的面试题，几乎是 90%以上的互联网公司都会问到的问题。

“Dubbo 的服务请求失败怎么处理”？

对于这个问题，我们来看一下高手的回答。

以下是高手的回答思路

Dubbo 是一个 RPC 框架，它为我们的应用提供了远程通信能力的封装，同时，Dubbo 在 RPC 通信的基础上，逐步在向一个生态在演进，它涵盖了服务注册、动态路由、容错、服务降级、负载均衡等能力，基本上在微服务架构下面临的问题，Dubbo 都可以解决。

而对于 Dubbo 服务请求失败的场景，默认提供了重试的容错机制，也就是说，如果基于 Dubbo 进行服务间通信出现异常，服务消费者会对服务提供者集群中其他的节点发起重试，确保这次请求成功，默认的额外重试次数是 2 次。

除此之外，Dubbo 还提供了更多的容错策略，我们可以根据不同的业务场景来进行选择。

1. 快速失败策略，服务消费者只发起一次请求，如果请求失败，就直接把错误抛出去。这种比较适合在非幂等性场景中使用
2. 失败安全策略，如果出现服务通信异常，直接把这个异常吞掉不做任何处理
3. 失败自动恢复策略，后台记录失败请求，然后通过定时任务来对这个失败的请求进行重发。
4. 并行调用多个服务策略，就是把这个消息广播给服务提供者集群，只要有任何一个节点返回，就表示请求执行成功。
5. 广播调用策略，逐个调用服务提供者集群，只要集群中任何一个节点出现异常，就表示本次请求失败

要注意的是，默认基于重试策略的容错机制中，需要注意幂等性的处理，否则在事务型的操作中，容易出现多次数据变更的问题。

以上就是我对这个问题的理解！

53.13 [京东]ConcurrentHashMap 底层具体实现知道吗？实现原理是什么？

之前分享过一期 HashMap 的面试题，然后有个小伙伴私信我说，他遇到了一个 ConcurrentHashMap 的问题不知道怎么回答。

于是，就有了这一期的内容！！

我是 Mic，一个工作了 14 年的 Java 程序员，今天我来分享关

于 " ConcurrentHashMap 底层实现原理 " 这个问题，

看看高手是如何回答的！

> 以下是高手的回答思路

这个问题我从这三个方面来回答：（下面这三个点，打印在屏幕上）

3. ConcurrentHashMap 的整体架构
4. ConcurrentHashMap 的基本功能
5. ConcurrentHashMap 在性能方面的优化

- ConcurrentHashMap 的整体架构

（如图所示），这个是 ConcurrentHashMap 在 JDK1.8 中的存储结构，它是由数组、单向链表、红黑树组成。

当我们初始化一个 ConcurrentHashMap 实例时，默认会初始化一个长度为 16 的数组。

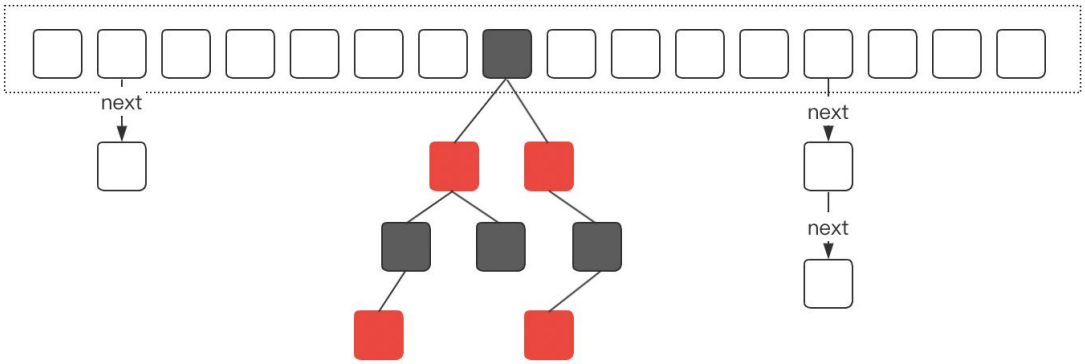
由于 ConcurrentHashMap 它的核心仍然是 hash 表，所以必然会存在 hash 冲突问题。

ConcurrentHashMap 采用链式寻址法来解决 hash 冲突。

当 hash 冲突比较多的时候，会造成链表长度较长，这种情况会使得 ConcurrentHashMap 中数据元素的查询复杂度变成 $O(_n)$。因此在 JDK1.8 中，引入了红黑树的机制。

当数组长度大于 64 并且链表长度大于等于 8 的时候，单项链表就会转换为红黑树。

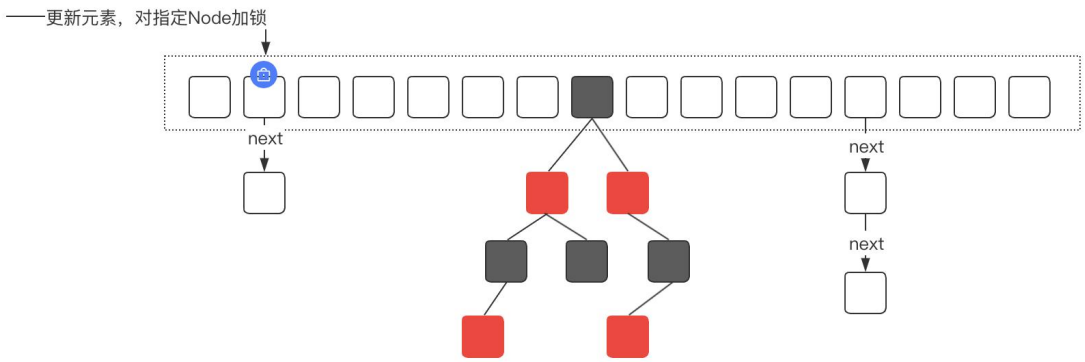
另外，随着 ConcurrentHashMap 的动态扩容，一旦链表长度小于 8，红黑树会退化成单向链表。



- ConcurrentHashMap 的基本功能

ConcurrentHashMap 本质上是一个 HashMap，因此功能和 HashMap 一样，但是 ConcurrentHashMap 在 HashMap 的基础上，提供了并发安全的实现。

并发安全的主要实现是通过对指定的 Node 节点加锁，来保证数据更新的安全性（如图所示）。

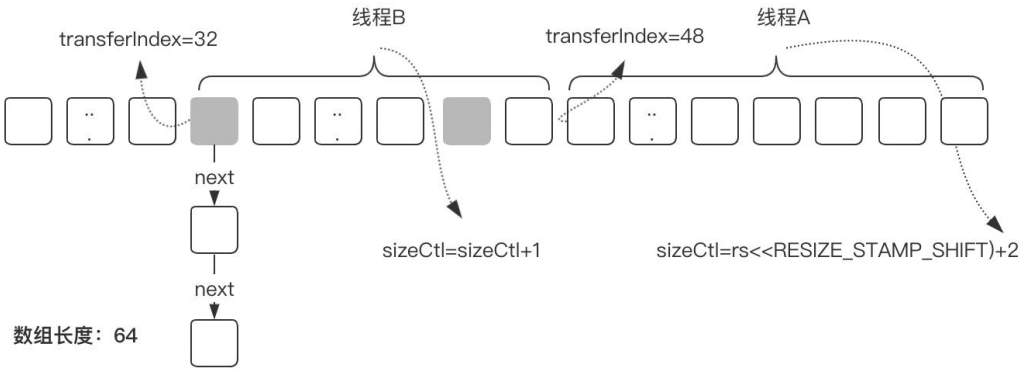


- ConcurrentHashMap 在性能方面做的优化（字幕提示）

如果在并发性能和数据安全性之间做好平衡，在很多地方都有类似的设计，比如 cpu 的三级缓存、mysql 的 buffer_pool、Synchronized 的锁升级等等。

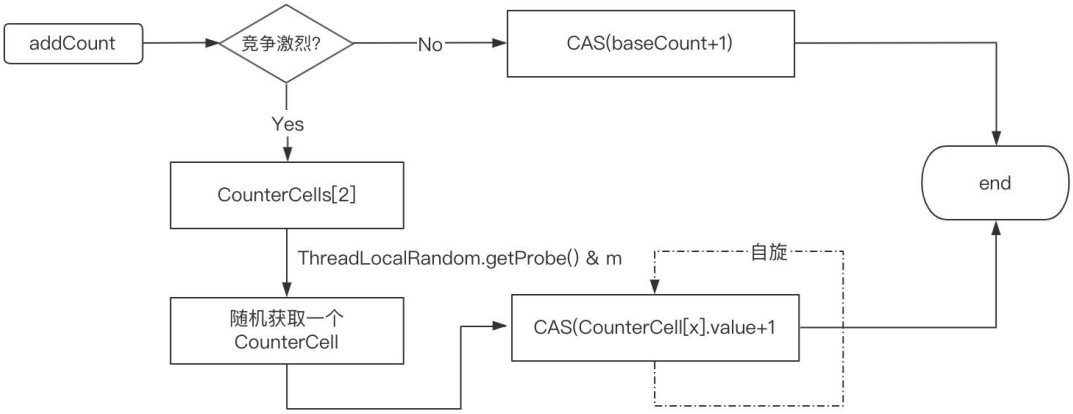
ConcurrentHashMap 也做了类似的优化，主要体现在以下几个方面：

- 在 JDK1.8 中，ConcurrentHashMap 锁的粒度是数组中的某一个节点，而在 JDK1.7，锁定的是 Segment，锁的范围要更大，因此性能上会更低。
- 引入红黑树，降低了数据查询的时间复杂度，红黑树的时间复杂度是 $O(\log n)$。
- （如图所示），当数组长度不够时，ConcurrentHashMap 需要对数组进行扩容，在扩容的实现上，ConcurrentHashMap 引入了多线程并发扩容的机制，简单来说就是多个线程对原始数组进行分片后，每个线程负责一个分片的数据迁移，从而提升了扩容过程中数据迁移的效率。



- ConcurrentHashMap 中有一个 size()方法来获取总的元素个数，而在多线程并发场景中，在保证原子性的前提下来实现元素个数的累加，性能是非常低的。ConcurrentHashMap 在这个方面的优化主要体现在两个点：
  - 当线程竞争不激烈时，直接采用 CAS 来实现元素个数的原子递增。

- 如果线程竞争激烈，使用一个数组来维护元素个数，如果要增加总的元素个数，则直接从数组中随机选择一个，再通过 CAS 实现原子递增。它的核心思想是引入了数组来实现对并发更新的负载。



以上就是我对这个问题的理解！

53.14 [京东]b 树和 b+树的理解

数据结构与算法问题，困扰了无数的小伙伴。

很多小伙伴对数据结构与算法的认知有一个误区，认为工作中没有用到，为什么面试要问，问了能解决实际问题？

图灵奖获得者： Niklaus Wirth 说过： 程序=数据结构+算法， 也就说我们无时无刻都在和数据结构打交道。

只是作为 Java 开发，由于技术体系的成熟度较高，使得大部分人认为：程序应该等于 框架 + SQL 呀？

今天我们就来分析一道数据结构的题目：” B 树和 B+树 “。

关于这个问题，我们来看看高手的回答！

以下是高手的回答思路

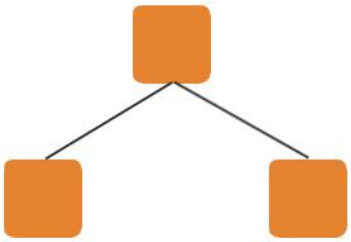
为了更清晰的解答这个问题，我打算从三个方面来回答：

- 了解二叉树、AVL 树、B 树的概念
- B 树和 B+树的应用场景
-

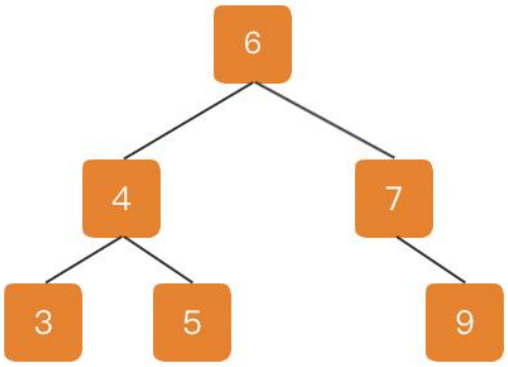
1. B 树是一种多路平衡查找树，为了更形象的理解，（我们来看这张图）。

二叉树，每个节点支持两个分支的树结构，相比于单向链表，多了一个分支。

二叉查找树，在二叉树的基础上增加了一个规则，左子树的所有节点的值都小于它的根节点，右子树的所有子节点都大于它的根节点。

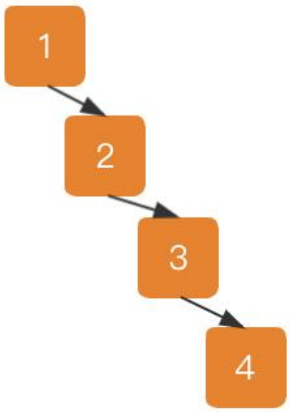


二叉树

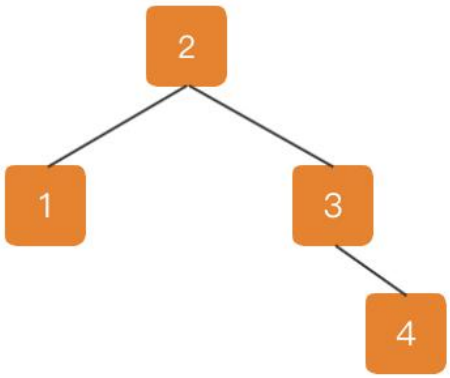


二叉查找树

（如图），二叉查找树会出现斜树问题，导致时间复杂度增加，因此又引入了一种平衡二叉树，它具有二叉查找树的所有特点，同时增加了一个规则：”它的左右两个子树的高度差的绝对值不超过 1 “。平衡二叉树会采用左旋、右旋的方式来实现平衡。



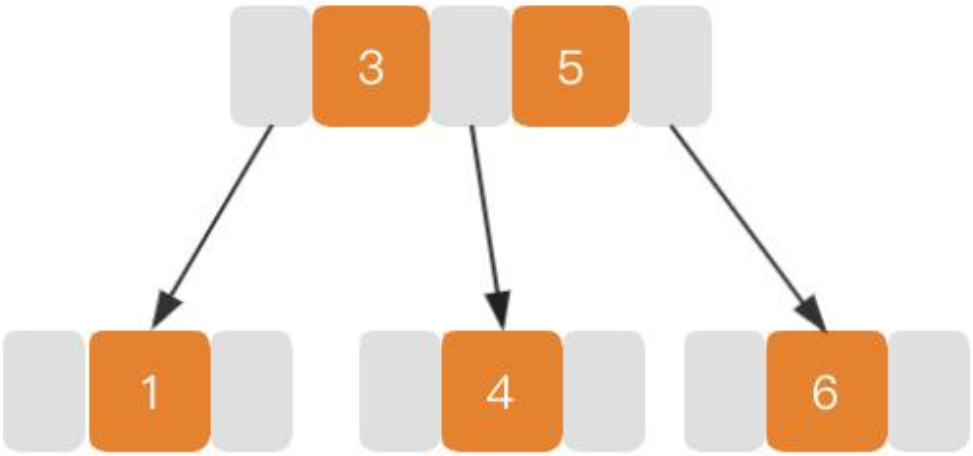
斜树



平衡二叉树

（如图），而 B 树是一种多路平衡查找树，它满足平衡二叉树的规则，但是它可以有多个子树，子树的数量取决于关键字的数量，比如这个图中根节点有两个关键字 3 和 5，那么它能够拥有的子路数量=关键字数+1。

因此从这个特征来看，在存储同样数据量的情况下，平衡二叉树的高度要大于 B 树。



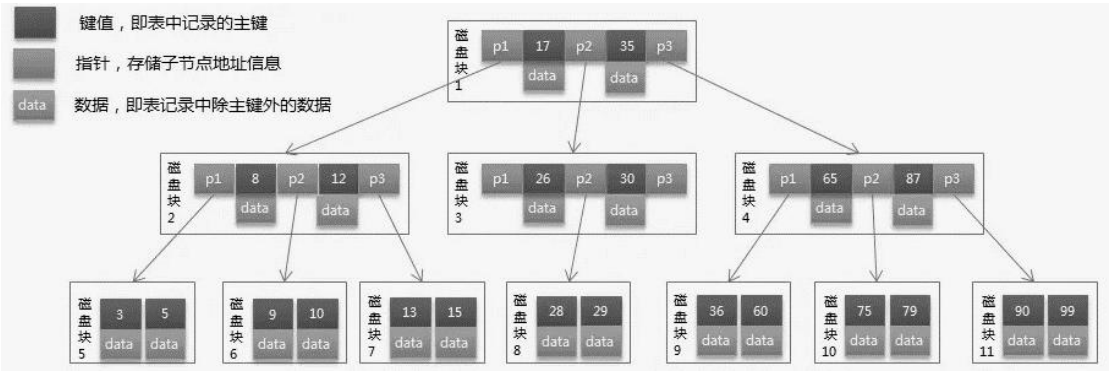
B树

B+树，其实是在 B 树的基础上做的增强，最大的区别有两个：

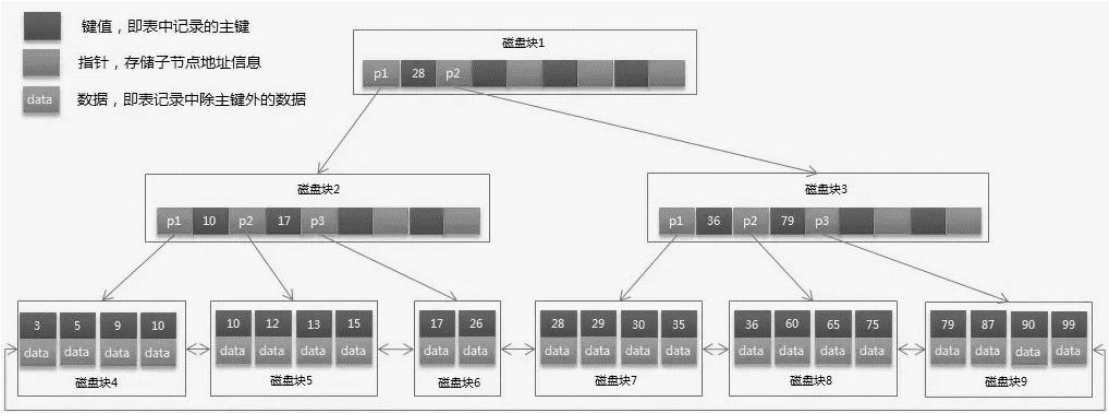
a. B 树的数据存储在每个节点上，而 B+树中的数据是存储在叶子节点，并且通过链表的方式把叶子节点中的数据进行连接。

b. B+树的子路数量等于关键字数

（如图所示）这个是 B 树的存储结构，从 B 树上可以看到每个节点会存储数据。



（如图所示）这个是 B+树，B+树的所有数据是存储在叶子节点，并且叶子节点的数据是用双向链表关联的。



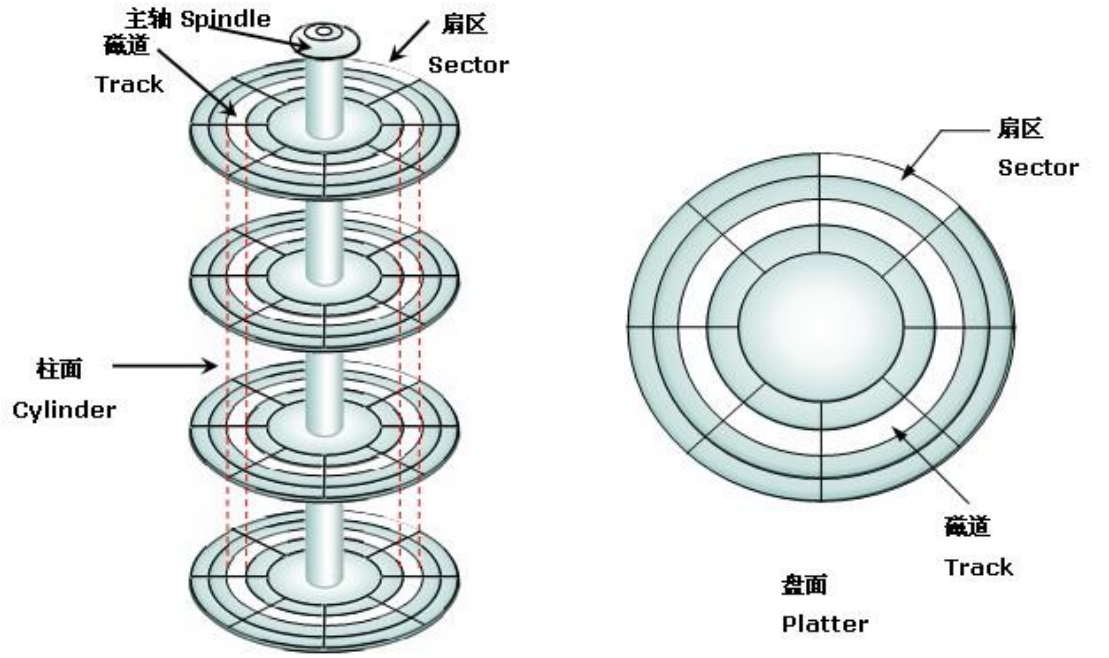
2. B 树和 B+树，一般都是应用在文件系统和数据库系统中，用来减少磁盘 IO 带来的性能损耗。

以 Mysql 中的 InnoDB 为例，当我们通过 select 语句去查询一条数据时，InnoDB 需要从磁盘上去读取数据，这个过程会涉及到磁盘 IO 以及磁盘的随机 IO（如图所示）

我们知道磁盘 IO 的性能是特别低的，特别是随机磁盘 IO。

因为，磁盘 IO 的工作原理是，首先系统会把数据逻辑地址传给磁盘，磁盘控制电路按照寻址逻辑把逻辑地址翻译成物理地址，也就是确定要读取的数据在哪个磁道，哪个扇区。

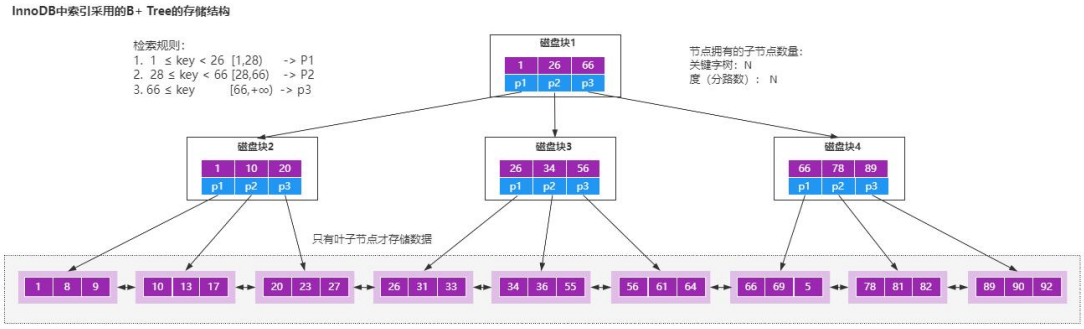
为了读取这个扇区的数据，需要把磁头放在这个扇区的上面，为了实现这一个点，磁盘会不断旋转，把目标扇区旋转到磁头下面，使得磁头找到对应的磁道，这里涉及到寻道事件以及旋转时间。



很明显，磁盘 IO 这个过程的性能开销是非常大的，特别是查询的数据量比较多的情况下。

所以在 InnoDB 中，干脆对存储在磁盘块上的数据建立一个索引，然后把索引数据以及索引列对应的磁盘地址，以 B+树的方式来存储。

如图所示，当我们需要查询目标数据的时候，根据索引从 B+树中查找目标数据即可，由于 B+树分路较多，所以只需要较少次数的磁盘 IO 就能查找到。



3. 为什么用 B 树或者 B+树来做索引结构？原因是 AVL 树的高度要比 B 树的高度要高，而高度就意味着磁盘 IO 的数量。所以为了减少磁盘 IO 的次数，文件系统或者数据库才会采用 B 树或者 B+树。

以上就是我对 B 树和 B+树的理解！

53.15 [京东]能谈一下 CAS 机制吗?

一个小伙伴私信我，他说遇到了一个关于 CAS 机制的问题，他以为面试官问的是 CAS 实现单点登录。

心想，这个问题我熟啊，然后就按照单点登录的思路去回答，结果面试官一直摇头。

他来和我说，到了面试结束都没明想白自己回答这么好，怎么就没有当场给我发 offer 呢?

实际上，面试官问的是并发编程中的 CAS 机制。

下面我们来看看高手对于 CAS 机制的回答吧

以下是高手的回答思路

CAS 是 Java 中 Unsafe 类里面的方法，它的全称是 CompareAndSwap，比较并交换的意思。它的主要功能是能够保证在多线程环境下，对于共享变量的修改的原子性。

我来举个例子，比如说有这样一个场景（如图），有一个成员变量 state，默认值是 0，定义了一个方法 doSomething()，这个方法的逻辑是，判断 state 是否为 0 ，如果为 0，就修改成 1。

这个逻辑看起来没有任何问题，但是在多线程环境下，会存在原子性的问题，因为这里是一个典型的，Read - Write 的操作。

一般情况下，我们会在 doSomething()这个方法上加同步锁来解决原子性问题。

```
public class Example {

    private int state=0;

    public void doSomething(){
        if(state==0){ //多线程环境中，存在原子性问题
            state=1;
            //TODO
        }
    }
}
```

但是，加同步锁，会带来性能上的损耗，所以，对于这类场景，我们就可以使用 CAS 机制来进行优化

这个是优化之后的代码（如图）

在 doSomething()方法中，我们调用了 unsafe 类中的 compareAndSwapInt()方法来达到同样的目的，这个方法有四个参数，

分别是：当前对象实例、成员变量 state 在内存地址中的偏移量、预期值 0、期望更改之后的值 1。

CAS 机制会比较 state 内存地址偏移量对应的值和传入的预期值 0 是否相等，如果相等，就直接修改内存地址中 state 的值为 1.

否则，返回 false，表示修改失败，而这个过程是原子的，不会存在线程安全问题。

```java
public class Example {

    private volatile int state=0;

    private static final Unsafe unsafe = Unsafe.getUnsafe();
    private static final long stateOffset;

    static {
        try {
            stateOffset = unsafe.objectFieldOffset
                    (Example.class.getDeclaredField("state"));
        } catch (Exception ex) { throw new Error(ex); }
    }

    public void doSomething(){
        if(unsafe.compareAndSwapInt(this,stateOffset,0,1)){
            //TODO
        }
    }
}
```

CompareAndSwap 是一个 native 方法，实际上它最终还是会面临同样的问题，就是先从内存地址中读取 state 的值，然后去比较，最后再修改。

这个过程不管是在什么层面上实现，都会存在原子性问题。

所以呢，CompareAndSwap 的底层实现中，在多核 CPU 环境下，会增加一个 Lock 指令对缓存或者总线加锁，从而保证比较并替换这两个指令的原子性。

CAS 主要用在并发场景中，比较典型的使用场景有两个。

1. 第一个是 J.U.C 里面 Atomic 的原子实现，比如 AtomicInteger，AtomicLong。
2. 第二个是实现多线程对共享资源竞争的互斥性质，比如在 AQS、ConcurrentHashMap、ConcurrentLinkedQueue 等都有用到。

以上就是我对这个问题的理解。

53.16 [京东]请说一下网络四元组

之前我一直在向大家征集一些刁钻的面试题，然后今天就收到了这样的一个问题。

“请你说一下网络四元组的理解”，他说他听到这个问题的时候，完全就懵了。

好吧，关于这个问题，我们来看看高手的回答。

以下是高手的回答思路

四元组，简单理解就是在 TCP 协议中，去确定一个客户端连接的组成要素，它包括源 IP 地址、目标 IP 地址、源端口号、目标端口号。

正常情况下，我们对于网络通信的认识可能是这样（如图）。

服务端通过 ServerSocket 建立一个对指定端口号的监听，比如 8080。 客户端通过目标 ip 和端口就可以和服务端建立一个连接，然后进行数据传输。

但是我们知道的是，一个 Server 端可以接收多个客户端的连接，比如像这种情况（如图）。

那，当多个客户端连接到服务端的时候，服务端需要去识别每一个连接。

并且(如图），TCP 是全双工协议，也就是说数据允许在连接的两个方向上同时传输，因此这里的客户端，如果是反向通信，它又变成了服务端。

所以基于这两个原因，就引入了四元组的设计，也就是说，当一个客户端和服务端建立一个 TCP 连接的时候，通过源 IP 地址、目标 IP 地址、源端口号、目标端口号来确定一

个唯一的 TCP 连接。因为服务器的 IP 和端口是不变的，只要客户端的 IP 和端口彼此不同就 OK 了。

比如像这种情况（如图），同一个客户端主机上有三个连接连到 Server 端，那么这个时候源 IP 相同，源端口号不同。此时建立的四元组就是（10.23.15.3，59461 ，192.168.8.135，8080）

其中，源端口号是每次建立连接的时候系统自动分配的。

以上就是我对于四元组的理解。

53.17 [京东]什么是服务网格?

今天继续来分享一个有趣的面试题，“什么是服务网格”？

服务网格这个概念出来很久了，从 2017 年被提出来，到 2018 年正式爆发，很多云厂商和互联网企业都在纷纷向服务网格靠拢。像蚂蚁集团、美团、百度、网易等一线互联网公司，都有服务网格的落地应用。

在我看来呢，服务网格是微服务架构的更进一步升级，它的核心目的是实现网络通信与业务逻辑的分离，使得开发人员更加专注在业务的实现上。

那么基于这个问题，我们来看看高手的回答。

以下是高手的回答思路

服务网格，也就是 Service Mesh，它是专门用来处理服务通讯的基础设施层。它的主要功能是处理服务之间的通信，并且负责实现请求的可靠性传递。

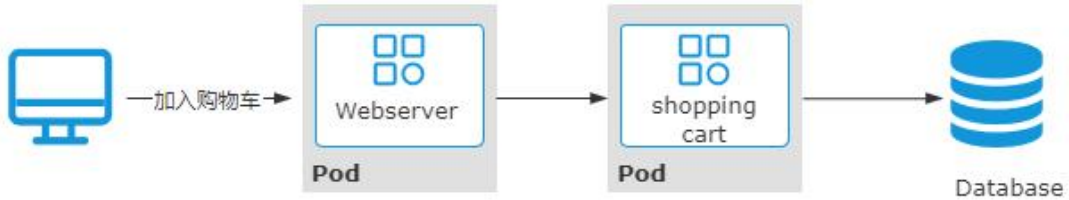
Service Mesh，我们通常把他称为第三代微服务架构，既然是第三代，那么意味着他是在原来的微服务架构下做的升级。

为了更好的说明 Service Mesh，那我就不得不说一下微服务架构部分的东西。

首先，当我们把一个电商系统以微服务化架构进行拆分后，会的到这样的一个架构（如图），其中包括 Webserver、payment、inventory 等等。

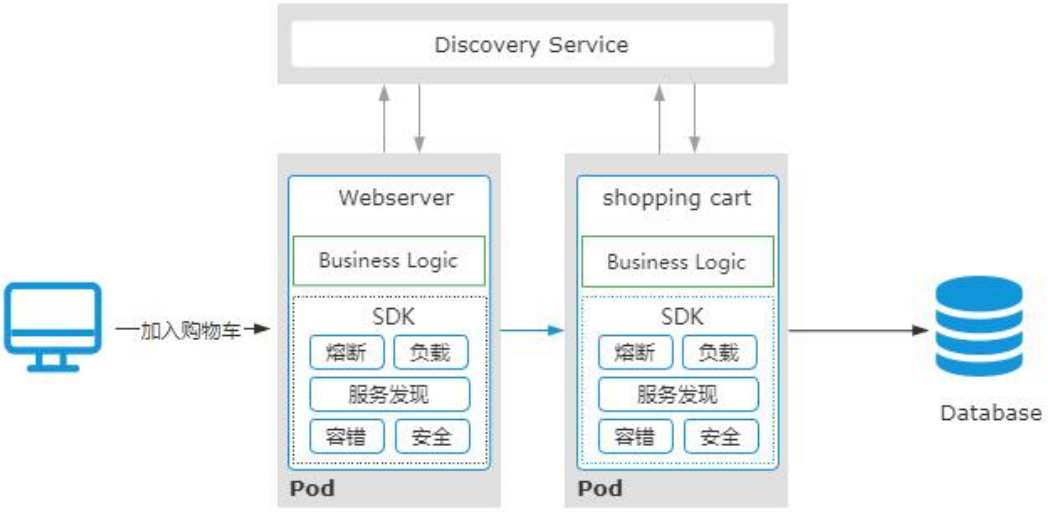
（如图）这些微服务应用，会被部署到 Docker 容器、或者 Kubernetes 集群。由于每个服务的业务逻辑是独立的，比如 payment 会实现支付的业务逻辑、order 实现订单的处理、Webserver 实现客户端请求的响应等。

（如图）所以，服务之间必须要相互通信，才能实现功能的完整性。比如用户把一个商品加入购物车，请求会进入到 Webserver，然后转发到 shopping cart 进行处理，并存到数据库。



而在这个过程中，每个服务之间必须要知道对方的通信地址，并且当有新的节点加入进来的时候，还需要对这些通信地址进行动态维护。所以，在第一代微服务架构中，每个微服务除了要实现业务逻辑以外，还需要解决上下游寻址、通讯、以及容错等问题。

（如图）于是，在第二代微服务架构下，引入了服务注册中心来实现服务之间的寻址，并且服务之间的容错机制、负载均衡也逐步形成了独立的服务框架，比如主流的 Spring Cloud、或者 Spring Cloud Alibaba。



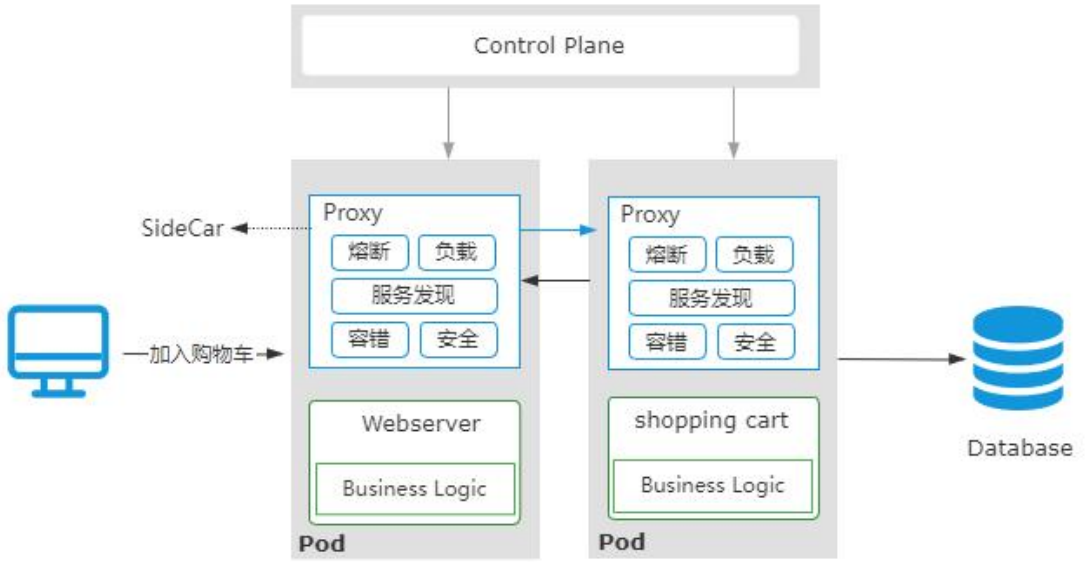
在第二代微服务架构中，负责业务开发的小伙伴不仅仅需要关注业务逻辑，还需要花大量精力去处理微服务中的一些基础性配置工作，虽然 Spring Cloud 已经尽可能去完成了这些事情，但对于开发人员来说，学习 Spring Cloud，以及针对 Spring Cloud 的配置和维护，仍然存在较大的挑战。另外呢，也增加了整个微服务的复杂性。

实际上，在我看来，“微服务中所有的这些服务注册、容错、重试、安全等工作，都是为了保证服务之间通信的可靠性”。

于是，就有了第三代微服务架构，Service Mesh。

（如图）原本模块化到微服务框架里的微服务基础能力，被进一步的从一个 SDK 中演进成了一个独立的代理进程-SideCar

SideCar 的主要职责就是负责各个微服务之间的通信，承载了原本第二代微服务架构中的服务发现、调用容错、服务治理等功能。使得微服务基础能力和业务逻辑迭代彻底解耦。



之所以我们称 Service Mesh 为服务网格，是因为在大规模微服务架构中，每个服务的通信都是由 SideCar 来代理的，各个服务之间的通信拓扑图，看起来就像一个网格形状（如图）。

Istio 是目前主流的 Service Mesh 开源框架。

以上就是我对服务网格的理解。

53.18 [京东]Redis 和 Mysql 如何保证数据一致性

今天分享一道一线互联网公司高频面试题。

“Redis 和 Mysql 如何保证数据一致性”。

这个问题难倒了不少工作 5 年以上的程序员，难的不是问题本身，而是解决这个问题的思维模式。

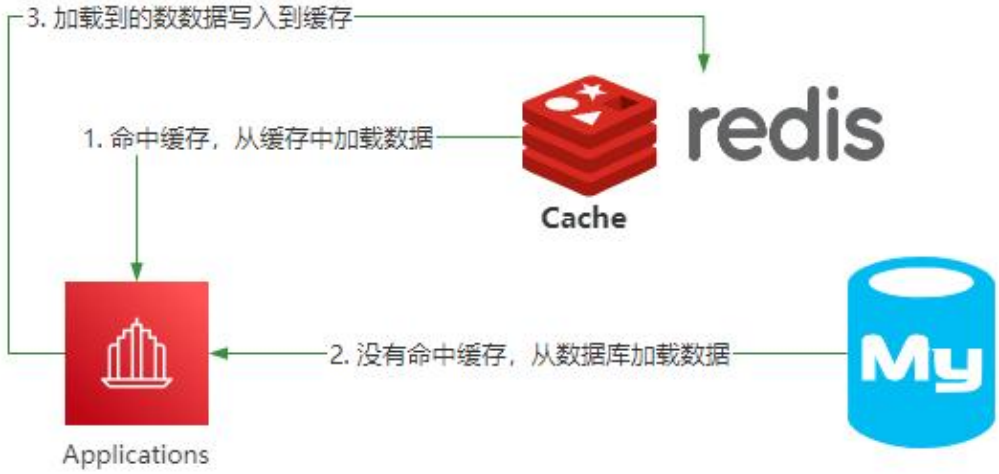
下面来看看高手对于这个问题的回答。

以下是高手的回答思路

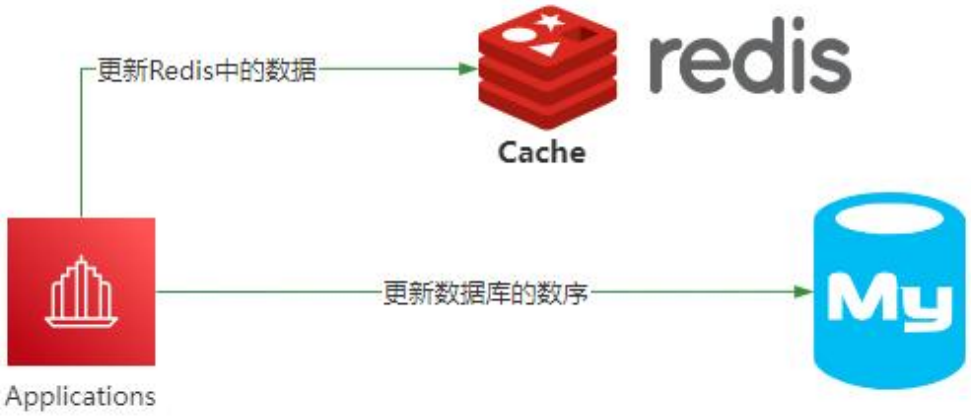
一般情况下，Redis 用来实现应用和数据库之间读操作的缓存层，主要目的是减少数据库 IO，还可以提升数据的 IO 性能。

这是它的整体架构。

当应用程序需要去读取某个数据的时候，首先会先尝试去 Redis 里面加载，如果命中就直接返回。如果没有命中，就从数据库查询，查询到数据后再把这个数据缓存到 Redis 里面。



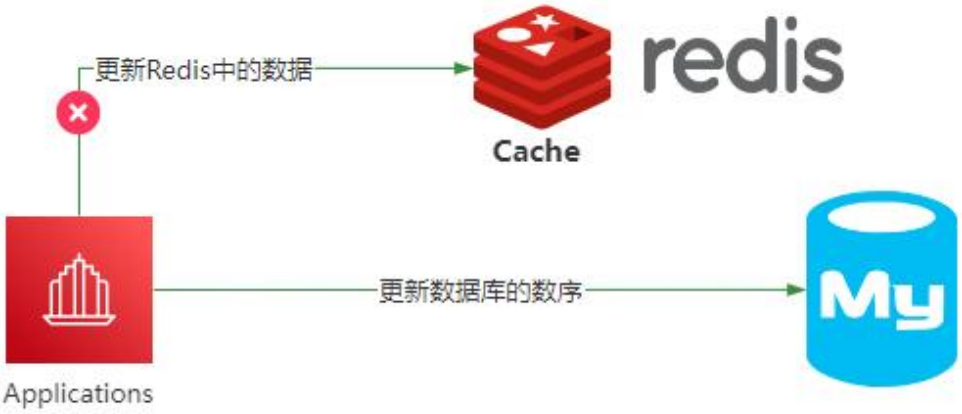
（如图）在这样一个架构中，会出现一个问题，就是一份数据，同时保存在数据库和 Redis 里面，当数据发生变化的时候，需要同时更新 Redis 和 Mysql，由于更新是有先后顺序的，并且它不像 Mysql 中的多表事务操作，可以满足 ACID 特性。所以就会出现数据一致性问题。



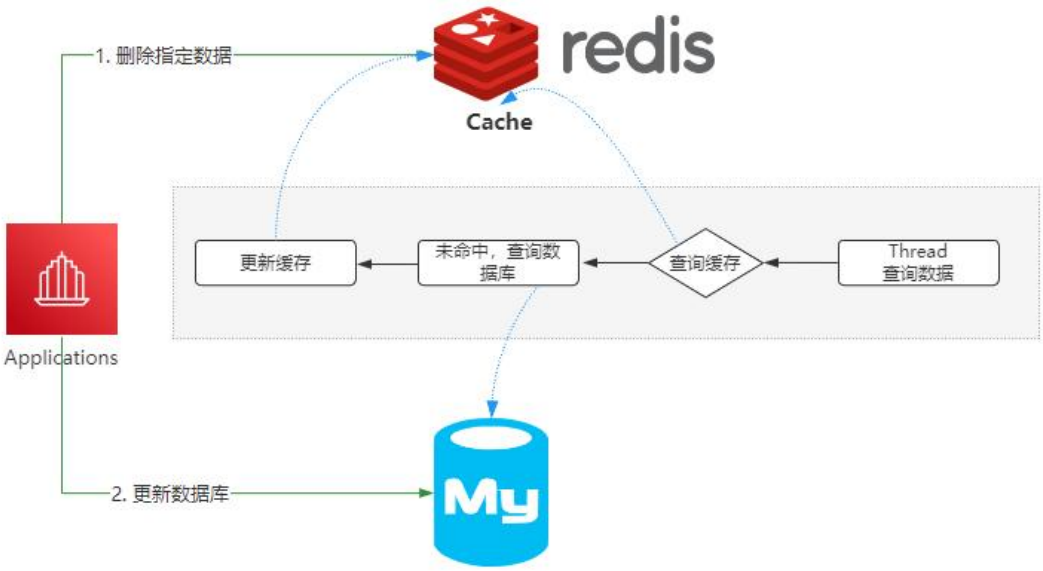
在这种情况下，能够选择的方法只有几种。

1. 先更新数据库，再更新缓存
2. 先删除缓存，再更新数据库

如果先更新数据库，再更新缓存，如果缓存更新失败，就会导致数据库和 Redis 中的数据不一致。

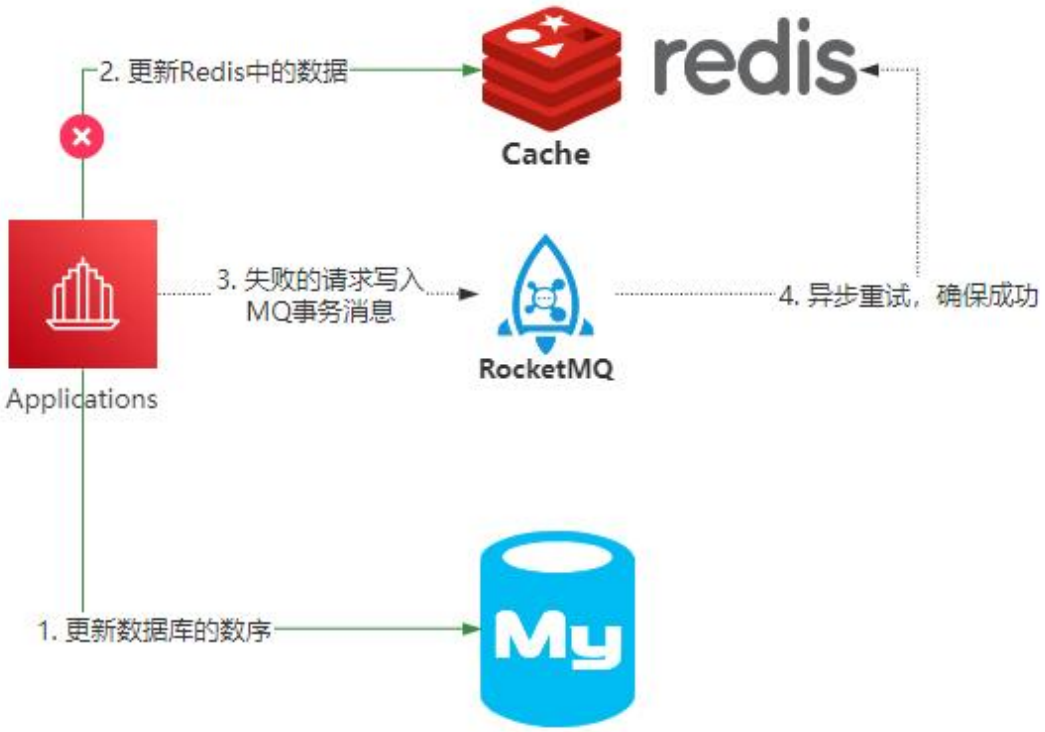


如果是先删除缓存，再更新数据库，理想情况是应用下次访问 Redis 的时候，发现 Redis 里面的数据是空的，就从数据库加载保存到 Redis 里面，那么数据是一致的。但是在极端情况下，由于删除 Redis 和更新数据库这两个操作并不是原子的，所以这个过程如果有其他线程来访问，还是会存在数据不一致问题。

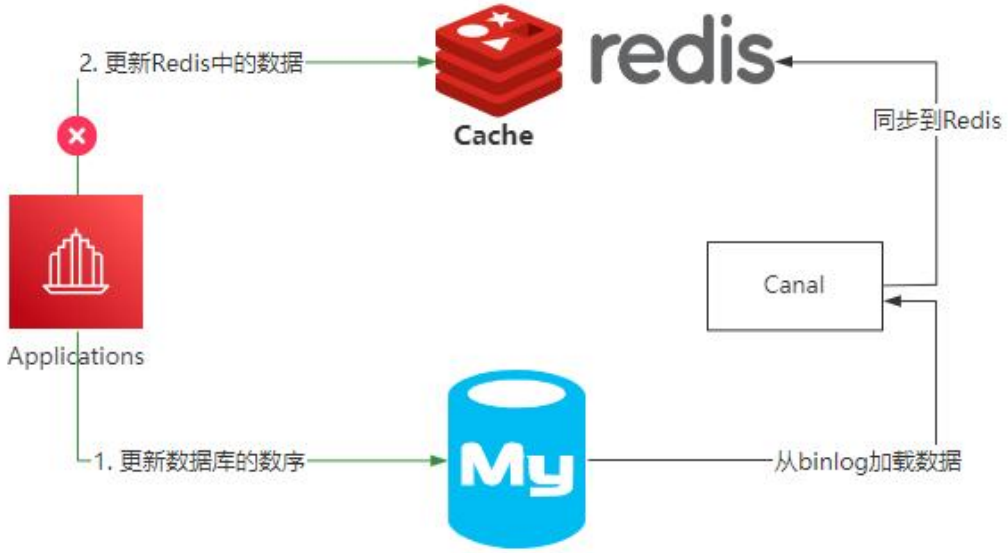


所以，如果需要在极端情况下仍然保证 Redis 和 Mysql 的数据一致性，就只能采用最终一致性方案。

（如图）比如基于 RocketMQ 的可靠性消息通信，来实现最终一致性。



（如图）还可以直接通过 Canal 组件，监控 Mysql 中 binlog 的日志，把更新后的数据同步到 Redis 里面。



因为这里是基于最终一致性来实现的，如果业务场景不能接受数据的短期不一致性，那就不能使用这个方案来做。

以上就是我对这个问题的理解。

53.19 [京东]Spring Boot 中自动装配机制的原理

最近一个粉丝说，他面试了 4 个公司，有三个公司问他：“Spring Boot 中自动装配机制的原理”

他回答了，感觉没回答错误，但是怎么就没给 offer 呢?

对于这个问题，看看高手该如何回答。

以下是高手的回答思路

自动装配，简单来说就是自动把第三方组件的 Bean 装载到 Spring IOC 器里面，不需要开发人员再去写 Bean 的装配配置。

在 Spring Boot 应用里面，只需要在启动类加上@SpringBootApplication 注解就可以实现自动装配。

@SpringBootApplication 是一个复合注解，真正实现自动装配的注解是@EnableAutoConfiguration。

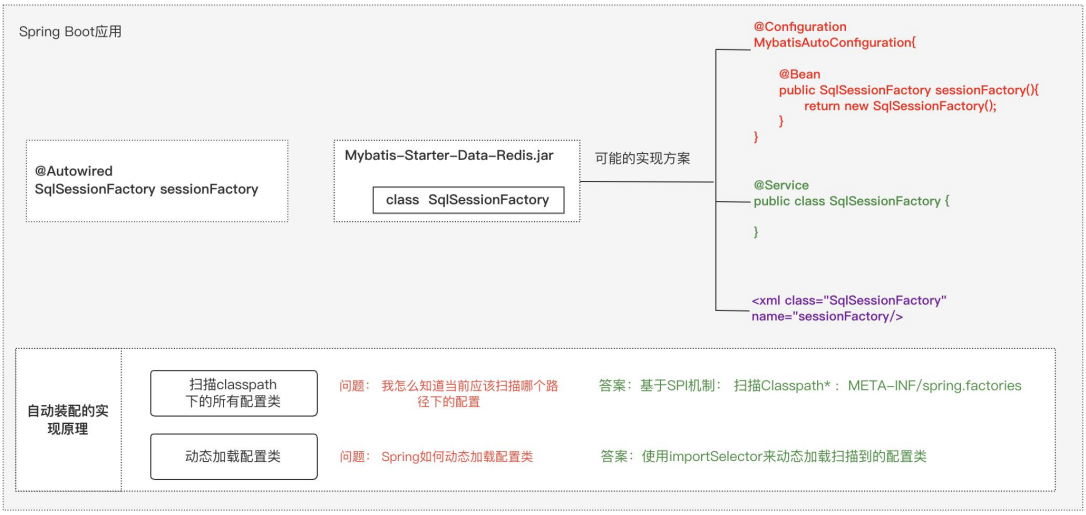
（如图）自动装配的实现主要依靠三个核心关键技术。

1. 引入 Starter 启动依赖组件的时候，这个组件里面必须要包含@Configuration 配置类，在这个配置类里面通过@Bean 注解声明需要装配到 IOC 容器的 Bean 对象。
2. 这个配置类是放在第三方的 jar 包里面，然后通过 SpringBoot 中的约定优于配置思想，把这个配置类的全路径放在 classpath:/META-INF/spring.factories 文件中。这样 SpringBoot 就可以知道第三方 jar 包里面的配置类的位置，这个步骤主要是用到了 Spring 里面的 SpringFactoriesLoader 来完成的。
3. SpringBoot 拿到所第三方 jar 包里面声明的配置类以后，再通过 Spring 提供的 ImportSelector 接口，实现对这些配置类的动态加载。

在我看来，SpringBoot 是约定优于配置这一理念下的产物，所以在很多的地方，都会看到这类的思想。它的出现，让开发人员更加聚焦在了业务代码的编写上，而不需要去关心和业务无关的配置。

其实，自动装配的思想，在 SpringFramework3.x 版本里面的@Enable 注解，就有了实现的雏形。@Enable 注解是模块驱动的意思，我们只需要增加某个@Enable 注解，就自动打开某个功能，而不需要针对这个功能去做 Bean 的配置，@Enable 底层也是帮我们去自动完成这个模块相关 Bean 的注入。

以上，就是我对 Spring Boot 自动装配机制的理解。



53.20 [京东]死锁的发生原因和怎么避免

一个去阿里面试的小伙伴私信我说：今天被一个死锁的问题难到了。

平常我都特意看了死锁这块的内容，但是回答的时候就想不起来。

这里可能存在一个误区，认为技术是要靠记的。

大家可以想想，平时写代码的时候，这些代码是背下来的吗?

遇到一个需求的时候，能够立刻提供解决思路，这个也是记下来的吗?

所有的技术问题，都可以用一个问题来解决： “如果让你遇到这个问题，你会怎么设计”？

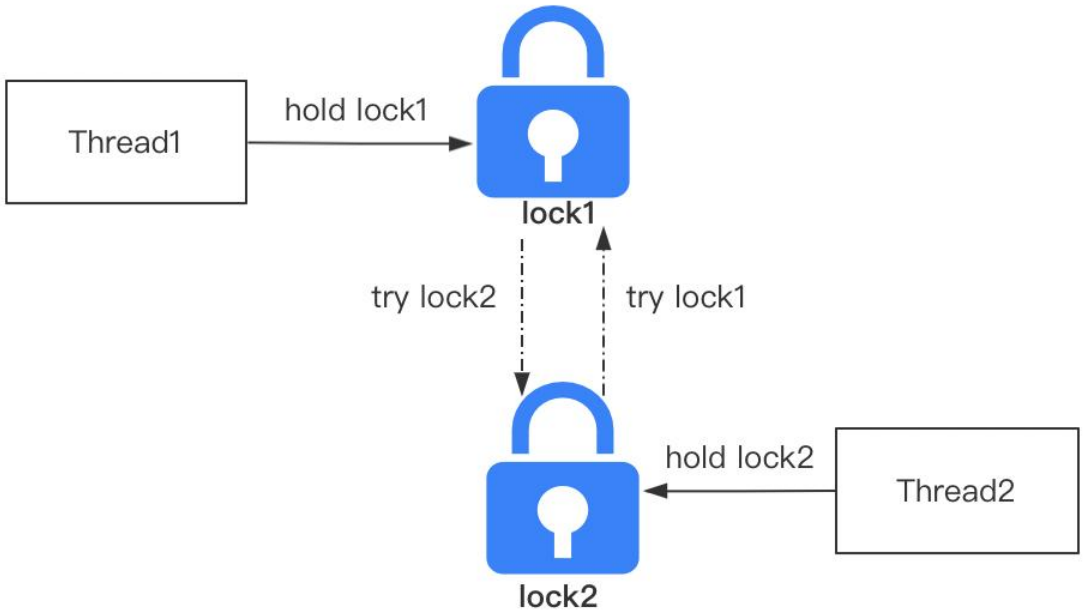
当你大脑一篇空白时，说明你目前掌握的技术只能足够支撑你写 CURD 的能力。

好了，下面来看看高手是如何回答这个问题的。

以下是高手的回答思路

(如图)，死锁，简单来说就是两个或者两个以上的线程在执行的过程中，争夺同一个共享资源造成的相互等待的现象。

如果没有外部干预，线程会一直阻塞无法往下执行，这些一直处于相互等待资源的线程就称为死锁线程。



导致死锁的条件有四个，也就是这四个条件同时满足就会产生死锁。

- 互斥条件，共享资源 X 和 Y 只能被一个线程占用；
- 请求和保持条件，线程 T1 已经取得共享资源 X，在等待共享资源 Y 的时候，不释放共享资源 X；
- 不可抢占条件，其他线程不能强行抢占线程 T1 占有的资源；
- 循环等待条件，线程 T1 等待线程 T2 占有的资源，线程 T2 等待线程 T1 占有的资源，就是循环等待。

导致死锁之后，只能通过人工干预来解决，比如重启服务，或者杀掉某个线程。

所以，只能在写代码的时候，去规避可能出现的死锁问题。

按照死锁发生的四个条件，只需要破坏其中的任何一个，就可以解决，但是，互斥条件是没办法破坏的，因为这是互斥锁的基本约束，其他三方条件都有办法来破坏：

- 对于“请求和保持”这个条件，我们可以一次性申请所有的资源，这样就不存在等待了。
- 对于“不可抢占”这个条件，占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源，这样不可抢占这个条件就破坏掉了。
- 对于“循环等待”这个条件，可以靠按序申请资源来预防。所谓按序申请，是指资源是有线性顺序的，申请的时候可以先申请资源序号小的，再申请资源序号大的，这样线性化后自然就不存在循环了。

以上就是我对这个问题的理解。

53.21 [拼多多]什么是 Dubbo？它有哪些核心功能？

Dubbo 是以高性能 RPC 框架，它提供了分布式架构下的服务之间通信方案，使得开发者可以不需要关心网络通信的细节。通过该框架可以使得远程服务调用方式和本地服务调用方式一样简单。

Dubbo 是一款高性能、轻量级的开源 RPC 框架。由 10 层模式构成，整个分层依赖由上至下。

通过这张图我们也可以将 Dubbo 理解为三层模式：

第一层的 Business 业务逻辑层由我们自己来提供接口和实现还有一些配置信息。

第二层的 RPC 调用的核心层负责封装和实现整个 RPC 的调用过程、负载均衡、集群容错、代理等核心功能。

Remoting 则是对网络传输协议和数据转换的封装。

根据 Dubbo 官方文档的介绍，Dubbo 提供了六大核心能力

- 面向接口代理的高性能 RPC 调用。
- 智能容错和负载均衡。
- 服务自动注册和发现。

- 高度可扩展能力。
- 运行期流量调度。
- 可视化的服务治理与运维。

53.22 [拼多多]既然说到 Dubbo 的功能，请详细说说 Dubbo 负载均衡的几种策略

Dubbo 有五种负载策略：

第一种是加权随机：假设我们有一组服务器 servers = [A, B, C]，他们对应的权重为 weights = [5, 3, 2]，权重总和为 10。现在把这些权重值平铺在一维坐标值上，[0, 5）区间属于服务器 A，[5, 8）区间属于服务器 B，[8, 10）区间属于服务器 C。接下来通过随机数生成器生成一个范围在 [0, 10）之间的随机数，然后计算这个随机数会落到哪个区间上就可以了。

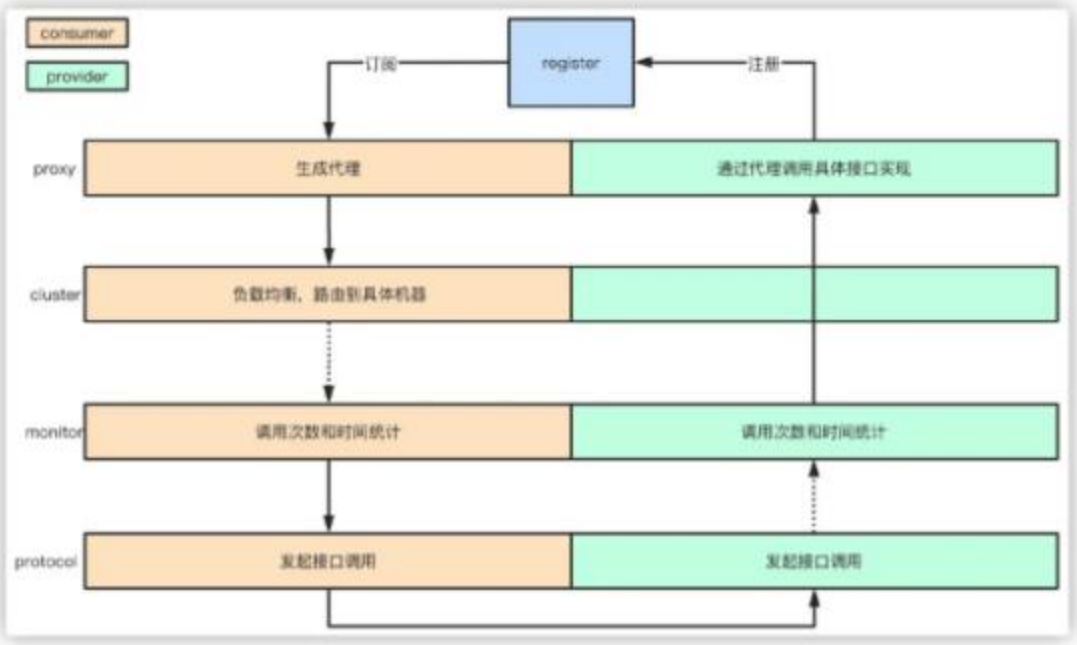
第二种是最小活跃数：每个服务提供者对应一个活跃数 active，初始情况下，所有服务提供者活跃数均为 0。每收到一个请求，活跃数加 1，完成请求后则将活跃数减 1。在服务运行一段时间后，性能好的服务提供者处理请求的速度更快，因此活跃数下降的也越快，此时这样的服务提供者能够优先获取到新的服务请求。

第三种是一致性 hash：通过 hash 算法，把 provider 的 invoke 和随机节点生成 hash，并将这个 hash 投射到 [0, 2^32 - 1] 的圆环上，查询的时候根据 key 进行 md5 然后进行 hash，得到第一个节点的值大于等于当前 hash 的 invoker。

第四种是加权轮询：比如服务器 A、B、C 权重比为 5:2:1，那么在 8 次请求中，服务器 A 将收到其中的 5 次请求，服务器 B 会收到其中的 2 次请求，服务器 C 则收到其中的 1 次请求。

第五种是最短响应时间权重随机：计算目标服务的请求的响应时间，根据响应时间最短的服务，配置更高的权重进行随机访问。

53.23 [拼多多]Dubbo 的工作原理是什么样的?



1.服务启动的时候，provider 和 consumer 根据配置信息，连接到注册中心 register，分别向注册中心注册和订阅服务

2.register 根据服务订阅关系，返回 provider 信息到 consumer，同时 consumer 会把 provider 信息缓存到本地。如果信息有变更，consumer 会收到来自 register 的推送

3.consumer 生成代理对象，同时根据负载均衡策略，选择一台 provider，同时定时向 monitor 记录接口的调用次数和时间信息

4.拿到代理对象之后，consumer 通过代理对象发起接口调用

5.provider 收到请求后对数据进行反序列化，然后通过代理调用具体的接口实现

53.24 [拼多多]最后在说说 Dubbo 与 Spring Cloud 的区别吧!

Dubbo 是 SOA 时代的产物，它的关注点主要在于服务的调用，流量分发、流量监控和熔断。而 Spring Cloud 诞生于微服务架构时代，考虑的是微服务治理的方方面面，另外由于依托了 Spirng、Spirng Boot 的优势之上，两个框架在开始目标就不一致，Dubbo 定位服务治理、Spirng Cloud 是一个生态。

两者最大的区别是 Dubbo 底层是使用 Netty 这样的 NIO 框架，是基于 TCP 协议传输的，配合以 Hession 序列化完成 RPC 通信。而 SpringCloud 是基于 Http 协议+Rest 接口调用远程过程的通信，相对来说，Http 请求会有更大的报文，占的带宽也会更多。但是 REST 相比 RPC 更为灵活，服务提供方和调用方的依赖只依靠一纸契约，不存在代码级别的强依赖。

53.25 [拼多多]你是否在面试中也被过 MySQL 优化相关的问题呢?

大家好，我是 MIC，一个工作了 13 年的 Java 程序员。

MySQL 作为一种免费开源的关系型数据库，深受着互联网公司的喜爱。

因此，它也成为了技术面试官最常问的问题之一。

相信很多小伙伴，也曾被 MySQL 相关的一些面试题弄的手足无措。

今天，我们就通过高手的形式，带大家深入了解 MySQL 的性能优化技巧。

> 以下是高手的回答思路

MySQL 的性能优化我认为可以分为 4 大部分

- 硬件和操作系统层面的优化
- 架构设计层面的优化
- MySQL 程序配置优化
- SQL 优化

53.25.1 硬件及操作系统层面优化

从硬件层面来说，影响 Mysql 性能的因素有，CPU、可用内存大小、磁盘读写速度、网络带宽

从操作系层面来说，应用文件句柄数、操作系统网络的配置都会影响到 Mysql 性能。

这部分的优化一般由 DBA 或者运维工程师去完成。

在硬件基础资源的优化中，我们重点应该关注服务本身承载的体量，然后提出合理的指标要求，避免出现资源浪费！

53.25.2 架构设计层面的优化

MySQL 是一个磁盘 IO 访问量非常频繁的关系型数据库

在高并发和高性能的场景中.MySQL 数据库必然会承受巨大的并发压力，而此时，我们的优化方式可以分为几个部分。

1. 搭建 Mysql 主从集群，单个 Mysql 服务容易单点故障，一旦服务器宕机，将会导致依赖 Mysql 数据库的应用全部无法响应。 主从集群或者主主集群可以保证服务的高可用性。
2. 读写分离设计，在读多写少的场景中，通过读写分离的方案，可以避免读写冲突导致的性能影响
3. 引入分库分表机制，通过分库可以降低单个服务器节点的 IO 压力，通过分表的方式可以降低单表数据量，从而提升 sql 查询的效率。
4. 针对热点数据，可以引入更为高效的分布式数据库，比如 Redis、MongoDB 等，他们可以很好的缓解 Mysql 的访问压力，同时还能提升数据检索性能。

53.25.3 MySQL 程序配置优化

MySQL 是一个经过互联网大厂验证过的生产级别的成熟数据库，对于 Mysql 数据库本身的优化，一般是通过 Mysql 中的配置文件 my.cnf 来完成的，比如。

Mysql5.7 版本默认的最大连接数是 151 个，这个值可以在 my.cnf 中修改。

binlog 日志，默认是不开启

缓存池 bufferpoll 的默认大小配置等。

由于这些配置一般都和用户安装的硬件环境以及使用场景有关系，因此这些配置官方只会提供一个默认值，具体情况还得由使用者来修改。

关于配置项的修改，需要关注两个方面。

- 配置的作用域，分为会话级别和全局
- 是否支持热加载

因此，针对这两个点，我们需要注意的是：

- 全局参数的设定对于已经存在的会话无法生效
- 会话参数的设定随着会话的销毁而失效
- 全局类的统一配置建议配置在默认配置文件中，否则重启服务会导致配置失效

53.25.4 SQL 优化

SQL 优化又能分为三步曲

- 第一、慢 SQL 的定位和排查

我们可以通过慢查询日志和慢查询日志分析工具得到有问题的 SQL 列表。

- 第二、执行计划分析

针对慢 SQL，我们可以使用关键字 explain 来查看当前 sql 的执行计划.可以重点关注 type key rows filterd 等字段 ，从而定位该 SQL 执行慢的根本原因。再有的放矢的进行优化

- 第三、使用 show profile 工具

Show Profile 是 MySQL 提供的可以用来分析当前会话中，SQL 语句资源消耗情况的工具，可用于 SQL 调优的测量。在当前会话中.默认情况下处于 show profile 是关闭状态，打开之后保存最近 15 次的运行结果

针对运行慢的 SQL，通过 profile 工具进行详细分析.可以得到 SQL 执行过程中所有的资源开销情况.

如 IO 开销,CPU 开销,内存开销等.

以上就是我对 MySQL 性能优化的理解。

好的，看完高手的回答后，相信各位对 MySQL 性能优化有了一定的理解了，最后我在给各位总结一下常见的 SQL 优化规则：

- SQL 的查询一定要基于索引来进行数据扫描
- 避免索引列上使用函数或者运算,这样会导致索引失效
- where 字句中 like %号,尽量放置在右边
- 使用索引扫描,联合索引中的列从左往右,命中越多越好.
- 尽可能使用 SQL 语句用到的索引完成排序,避免使用文件排序的方式
- 查询有效的列信息即可.少用 * 代替列信息
- 永远用小结果集驱动大结果集。

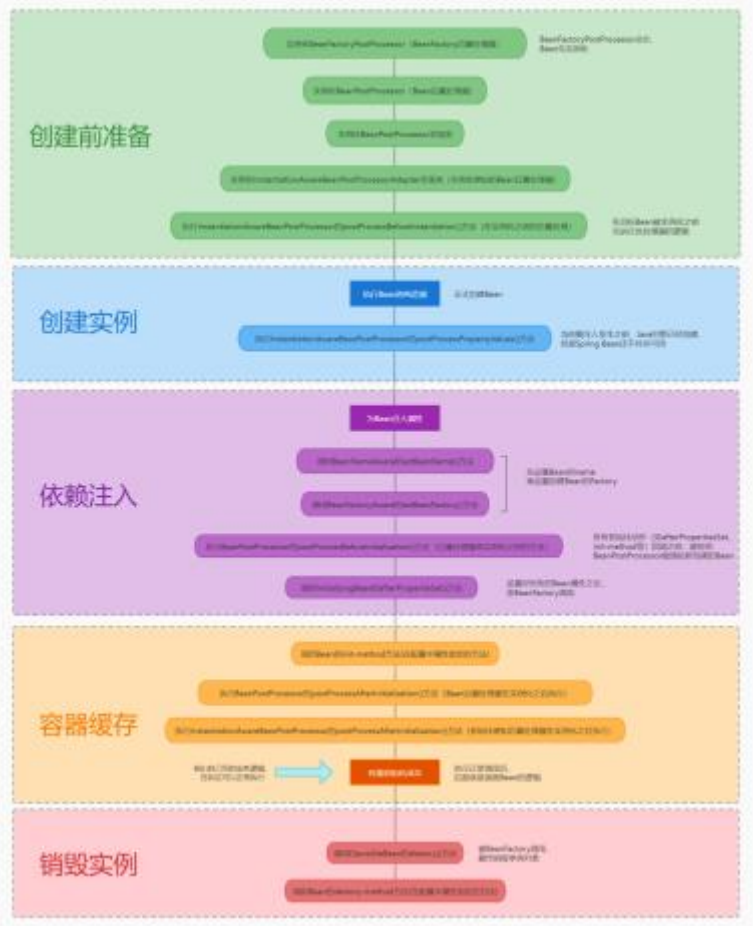
以上就是常见的 SQL 优化规则了

53.26 [拼多多]能回答一下 Spring Bean 生命周期的执行流程吗

Spring 生命周期全过程大致分为五个阶段：创建前准备阶段、创建实例阶段、依赖注入阶段、

容器缓存阶段和销毁实例阶段。

这张是 Spring Bean 生命周期完整流程图，其中对每个阶段的具体操作做了详细介绍：



（把这张图按照阶段截断，当我说第一个阶段时，就显示第一个阶段的图片，依次往后..）

53.26.1 创建前准备阶段【干什么、作用】

这个阶段主要的作用是，Bean 在开始加载之前，需要从上下文和相关配置中解析并查找 Bean 有关的扩展实现，

比如像`init-method`-容器在初始化 bean 时调用的方法、`destory-method`，容器在销毁 bean 时调用的方法。

以及，BeanFactoryPostProcessor 这类的 bean 加载过程中的前置和后置处理。
这些类或者配置其实是 Spring 提供给开发者，用来实现 Bean 加载过程中的扩展机制，在很多和 Spring 集成的中间件中比较常见，比如 Dubbo。

53.26.2 创建实例阶段

这个阶段主要是通过反射来创建 Bean 的实例对象，并且扫描和解析 Bean 声明的一些属性。

53.26.3 依赖注入阶段

如果被实例化的 Bean 存在依赖其他 Bean 对象的情况，则需要对这些依赖 bean 进行对象注入。比如常见的`@Autowired`、setter 注入等依赖注入的配置形式。
同时，在这个阶段会触发一些扩展的调用，比如常见的扩展类：BeanPostProcessors（用来实现 bean 初始化前后的扩展回调）、
InitializingBean（这个类有一个 afterPropertiesSet()，这个在工作中也比较常见）、BeanFactoryAware 等等。

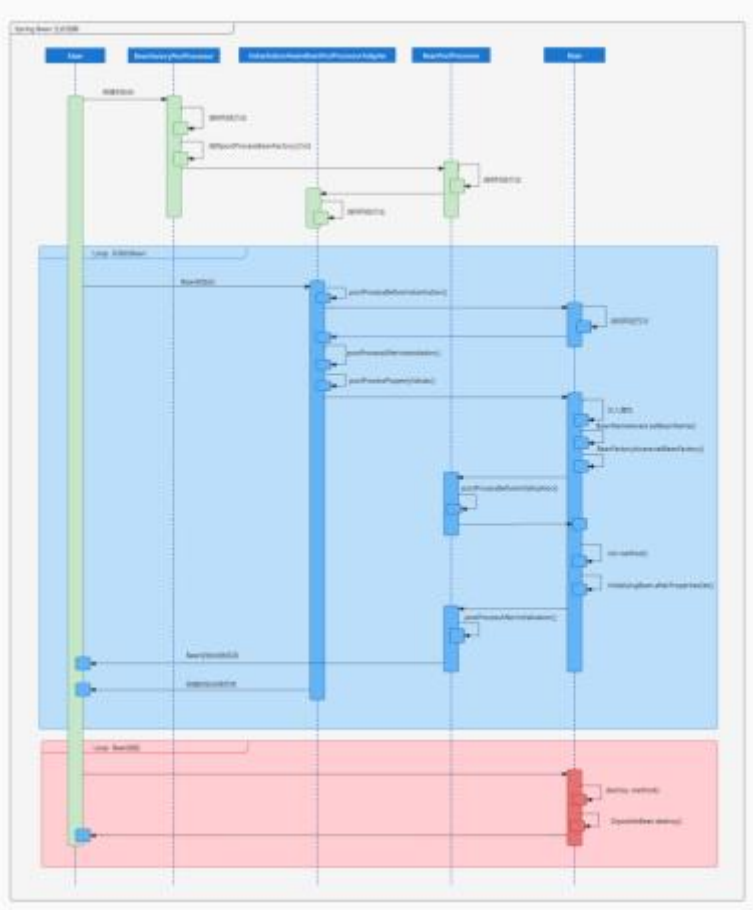
53.26.4 容器缓存阶段

容器缓存阶段主要是把 bean 保存到容器以及 Spring 的缓存中，到了这个阶段，Bean 就可以被开发者使用了。
这个阶段涉及到的操作，常见的有，`init-method`这个属性配置的方法， 会在这个阶段调用。
以及像 BeanPostProcessors 方法中的后置处理器方法如：
postProcessAfterInitialization，也会在这个阶段触发。

53.26.5 销毁实例阶段

当 Spring 应用上下文关闭时，该上下文中的所有 bean 都会被销毁。
如果存在 Bean 实现了 DisposableBean 接口，或者配置了`destory-method`属性，会在这个阶段被调用。

MIC：嗯，看完高手的回答后，相信大家对 Spring Bean 的生命周期有了深刻的印象了，需要视频中 Spring Bean 生命周期的高清流程图，可以私信我。在附赠一张高清的时序图给大家！



好的，本期普通人 VS 高手系列的视频就到这里就结束了，喜欢的朋友一键三连，加个关注。

我是 MIC，一个工作了十三年的 Java 程序员，咱们下期见！

53.27 [拼多多]请简述你对 Spring MVC 的理解，看看普通人和高手是如何回答的？

在 Java 面试中，有很多高频面试题，Spring MVC 就是其中之一。

面试官经常会问你对 Spring MVC 是怎么理解的。

下面我们来看看高手是如何回答这个问题的！

> 以下是高手的回答思路

SpringMVC 是一种基于 Java 语言开发，实现了 Web MVC 设计模式，请求驱动类型的轻量级 Web 框架。

采用了 MVC 架构模式的思想，通过把 Model，View，Controller 分离，将 Web 层进行职责解耦，从而把复杂的 Web 应用分成逻辑清晰的几个组件，在 Spring MVC 中有 9 大重要的组件。

下面详细说明一下这些组件的作用和初始化方法：

1、MultipartResolver 文件处理器

对应的初始化方法是 initMultipartResolver(context)，用于处理上传请求。

2、LocaleResolver 当前环境处理器

其对应的初始化方法是 initLocaleResolver(context)

SpringMVC 主要有两个地方用到了 Locale：

一是 ViewResolver 视图解析的时候；

二是用到国际化资源或者主题的时候。

3、ThemeResolver 主题处理器

其对应的初始化方法是 initThemeResolver(context)，

用于解析主题。 也就是解析样式、图片及它们所形成的显示效果的集合。

4、HandlerMapping 处理器映射器

其对应的初始化方法是 initHandlerMappings(context) ，

在 SpringMVC 中会有很多请求，每个请求都需要一个 Handler 处理。

HandlerMapping 的作用便是找到请求相应的处理器 Handler 和 Interceptor。

5、HandlerAdapter 处理器适配器

其对应的初始化方法是 initHandlerAdapters(context)

从名字上看，它就是一个适配器。HandlerAdapters 要做的事情就是如何让固定的 Servlet 处理方法调用灵活的 Handler 来进行处理

6、HandlerExceptionResolver 异常处理器

对应的初始化方法是 initHandlerExceptionResolvers(context)

它的主要作用是处理其他组件产生的异常情况。

7、RequestToViewNameTranslator 视图名称翻译器

其对应的初始化方法是 initRequestToViewNameTranslator(context)

它的作用是从请求中获取 ViewName。

有的 Handler 处理完后并没有设置 View 也没有设置 ViewName，这时就需要从 request 中获取，而 RequestToViewNameTranslator 就是为 request 提供获取 ViewName 的实现。

8、ViewResolvers 页面渲染处理器

其对应的初始化方法是 initViewResolvers(context)

ViewResolvers 的主要作用是将 String 类型的视图名和 Locale 解析为 View 类型的视图。

9、FlashMapManager 参数传递管理器

其对应的初始化方法是 initFlashMapManager(context)

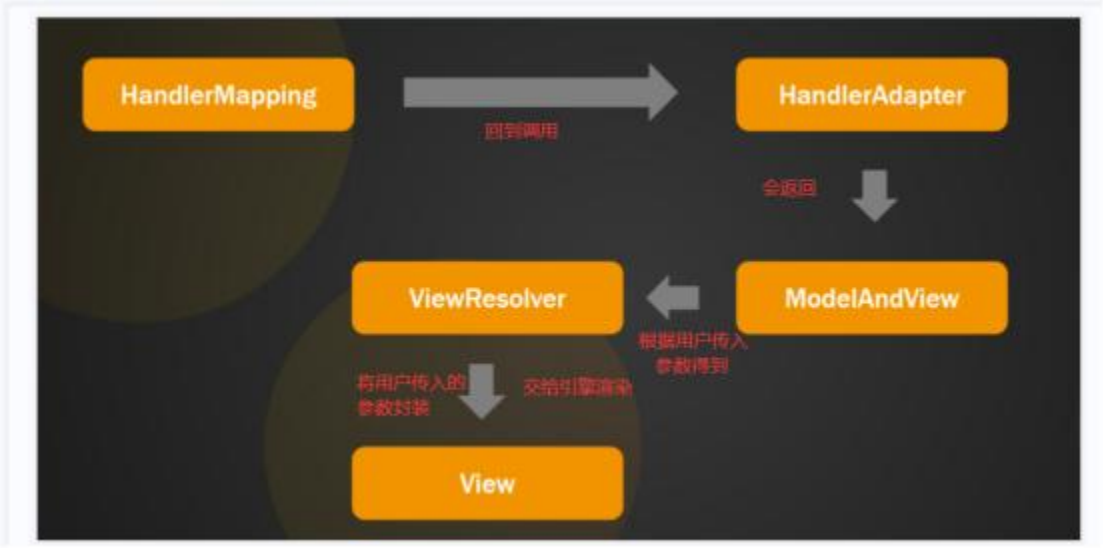
在实际应用中，为了避免重复提交，我们可以在处理完 post 请求后重定向到另外一个 get 请求，这个 get 请求可以用来返回页面渲染需要的信息。

FlashMap 就是用于这种请求重定向场景中的参数传递。

在 Spring MVC 的九大组件中，涉及到请求处理响应的核心组件分别是：

1. HandlerMapping、
2. HandlerAdapter、
3. ViewResolver

这张图表示这三个组件的整体执行流程，具体调用分为以下几个步骤：【插入图片】



1、HandlerMapping 回到调用 HandlerAdapter

2、HandlerAdapter 会返回 ModelAndView

3、ModelAndView 根据用户传入参数得到 ViewResolvers

4、ViewResolvers 会将用户传入的参数封装为 View，交给引擎进行渲染。

注意：有大家最熟悉的两个类：ModelAndView 和 View 类并不属于 Spring MVC 九大组件之列。

以上，就是我对 Spring MVC 的理解！

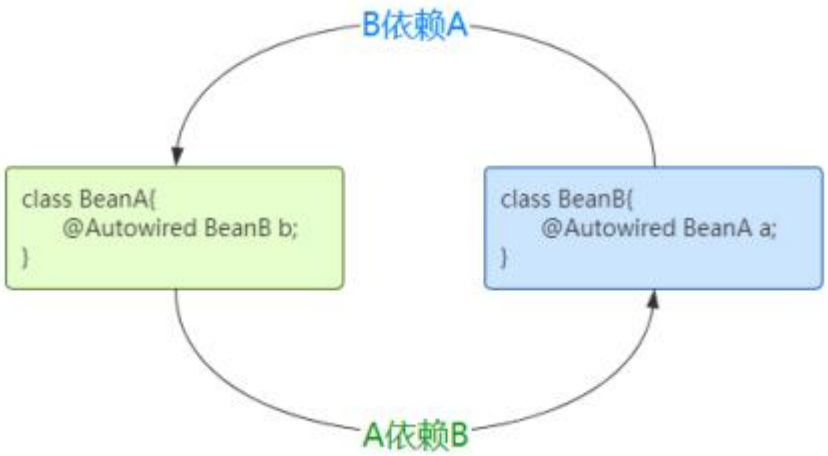
53.28 [拼多多]Spring 是如何解决循环依赖问题的?

以下是高手的回答思路

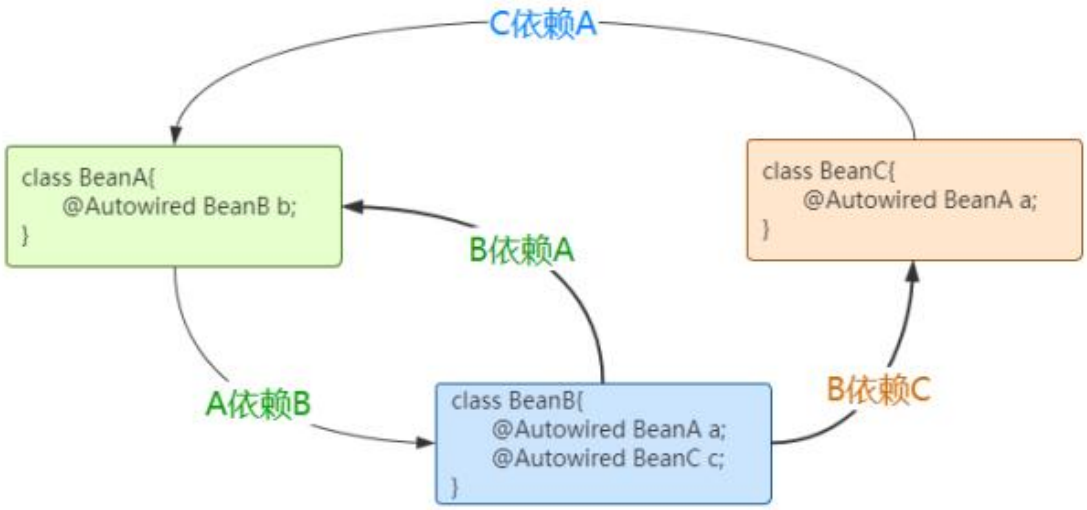
我们都知道，如果在代码中，将两个或多个 Bean 互相之间持有对方的引用就会发生循环依赖。循环的依赖将会导致注入死循环。这是 Spring 发生循环依赖的原因。

循环依赖有三种形态：

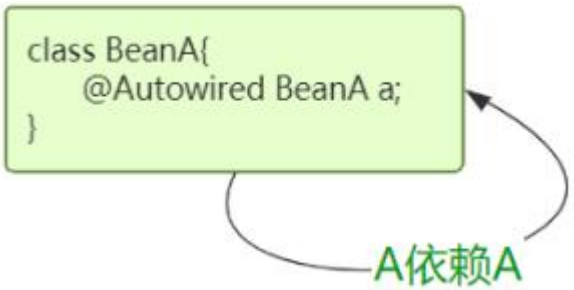
第一种互相依赖：A 依赖 B，B 又依赖 A，它们之间形成了循环依赖。【插入图片】



第二种三者间依赖：A 依赖 B，B 依赖 C，C 又依赖 A，形成了循环依赖。【 展示图片】



第三种是自我依赖：A 依赖 A 形成了循环依赖。【图片展示】

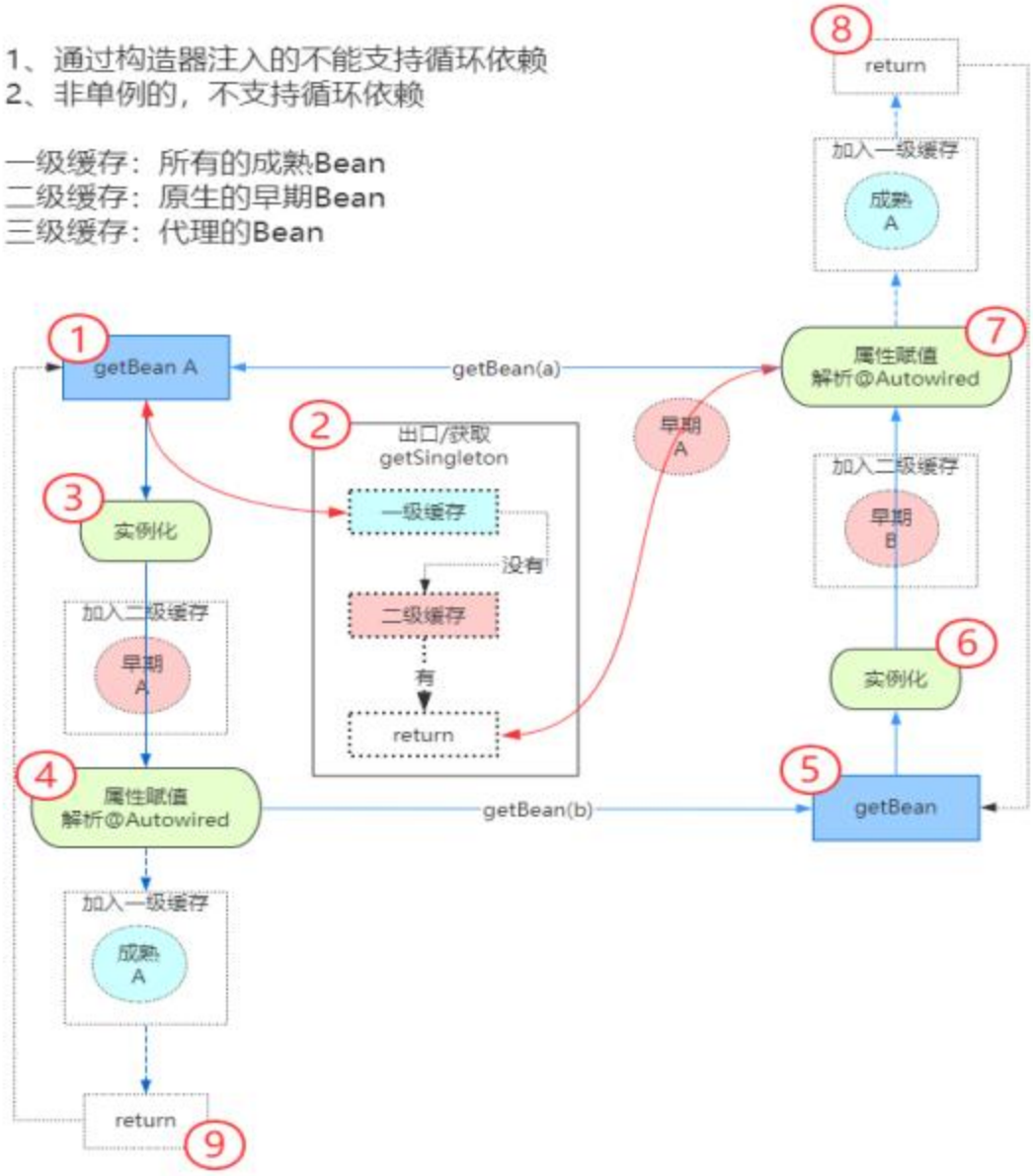


而 Spring 中设计了三级缓存来解决循环依赖问题，当我们去调用 getBean()方法的时候，Spring 会先从一级缓存中去找到目标 Bean，如果发现一级缓存中没有便会去二级缓存中去找，而如果一、二级缓存中都没有找到，意味着该目标 Bean 还没有实例化。于是，Spring 容器会实例化目标 Bean（PS：刚初始化的 Bean 称为早期 Bean） 。然后，将目标 Bean 放入到二级缓存中，同时，加上标记是否存在循环依赖。如果不存在循环依赖便会将目标 Bean 存入到二级缓存，否则，便会标记该 Bean 存在循环依赖，然后将等待下一次轮询赋值，也就是解析@Autowired 注解。等@Autowired 注解赋值完成后（PS：完成赋值的 Bean 称为成熟 Bean） ，会将目标 Bean 存入到一级缓存。

这里我可以做个总结，我们来看这张图

Spring 一级缓存中存放所有的成熟 Bean，

二级缓存中存放所有的早期 Bean，先取一级缓存，再去二级缓存。



53.29.1 面试官：那么，前面有提到三级缓存，三级缓存的作用是什么?

三级缓存是用来存储代理 Bean，当调用 getBean()方法时，发现目标 Bean 需要通过代理工厂来创建，此时会将创建好的实例保存到三级缓存，最终也会将赋值好的 Bean 同步到一级缓存中。

53.29.2 面试官：Spring 中哪些情况下，不能解决循环依赖问题?

1.多例 Bean 通过 setter 注入的情况，不能解决循环依赖问题

2.构造器注入的 Bean 的情况，不能解决循环依赖问题

3.单例的代理 Bean 通过 Setter 注入的情况，不能解决循环依赖问题

4.设置了@DependsOn 的 Bean 的情况，不能解决循环依赖问题

好的，看完高手的回答后，相信每位看完视频的小伙伴循环依赖有了更深刻的理解，

53.29 [拼多多]Zookeeper 和 Redis 哪种更好?

以下是高手的回答思路

关于这个问题，我想从 3 个方面来说：

1. **为什么使用分布式锁?**

使用分布式锁的目的，是为了保证同一时间只有一个 JVM 进程可以对共享资源进行操作。

根据锁的用途可以细分为以下两类：

- 允许多个客户端操作共享资源，我们称为共享锁

这种锁的一般是对共享资源具有幂等性操作的场景，主要是为了避免重复操作共享资源频繁加锁带来的性能开销。

- 只允许一个客户端操作共享资源，我们成为排他锁

这种锁一般是用在对共享资源操作具有非幂等性操作的场景，也就是需要保证在同一时刻只有一个进程或者线程能够访问这个共享资源。

2. **目前实现分布式锁最常用的中间件是 Redis 和 Zookeeper**
    a. **第一种，Redis 可以通过两种方式来实现**
        i. 利用 Redis 提供的`SET key value NX PX milliseconds `指令，这个指令是设置一个 key-value，如果 key 已经存在，则返回 0，否则返回 1，我们基于这个返回值来判断锁的占用情况从而实现分布式锁。
        ii. 基于 Redission 客户端来实现分布式锁，Redisson 提供了分布式锁的封装方法，我们只需要调用 api 中的`lock（）`和`unlock()`方法。它帮我们封装锁实现的细节和复杂度
            1. redisson 所有指令都通过 lua 脚本执行并支持 lua 脚本原子性执行
            2. redisson 中有一个 watchdog 的概念，翻译过来就是看门狗，它会在你获取锁之后，每隔 10 秒帮你把 key 的超时时间设为 30s，就算一

直持有锁也不会出现 key 过期了。“看门狗”的逻辑保证了没有死锁发生。

b. **第二种，基于 ZK 实现分布式锁的落地方案**

Zookeeper 实现分布式锁的方法比较多，我们可以使用有序节点来实现，

i. 每个线程或进程在 Zookeeper 上的`/lock`目录下创建一个临时有序的节点表示去抢占锁，所有创建的节点会按照先后顺序生成一个带有序编号的节点。

ii. 线程创建节点后，获取/lock 节点下的所有子节点，判断当前线程创建的节点是否是所有的节点的序号最小的。

iii. 如果当前线程创建的节点是所有节点序号最小的节点，则认为获取锁成功。

iv. 如果当前线程创建的节点不是所有节点序号最小的节点，则对节点序号的前一个节点添加一个事件监听，当前一个被监听的节点释放锁之后，触发回调通知，从而再次去尝试抢占锁。

3. **两种方案都有各自的优缺点**

对于 redis 的分布式锁而言，它有以下缺点：

- 它获取锁的方式简单粗暴，如果获取不到锁，会不断尝试获取锁，比较消耗性能。
- Redis 是 AP 模型，在集群模式中由于数据的一致性会导致锁出现问题，即便使用 Redlock 算法来实现，在某些复杂场景下，也无法保证其实现 100%的可靠性。

不过在实际开发中使用 Redis 实现分布式锁还是比较常见，而且大部分场情况下不会遇到”极端复杂的场景“，更重要的是 Redis 性能很高，在高并发场景中比较合适。

对于 zk 分布式锁而言:

- zookeeper 天生设计定位就是分布式协调，强一致性。锁的模型健壮、简单易用、适合做分布式锁。
- 如果获取不到锁，只需要添加一个监听器就可以了，不用一直轮询，性能消耗较小。

如果要在两者之间做选择，就我个人而言的话，比较推崇 ZK 实现的锁，因为对于分布式锁而言，它应该符合 CP 模型，但是 Redis 是 AP 模型，所以在这个点上，Zookeeper 会更加合适。

53.30 [拼多多]5 分钟带你快速弄懂 RPC

大家好，我是 MIC，一个工作了 13 年的 Java 程序员

在分布式微服务架构中，远程通信是最基本的需求。

常见的远程通信方式，有基于 REST 架构的 HTTP 协议、以及 RPC 框架。

最近，很多小伙伴私信我说，

在面试的时候，面试官经常会问：

什么是 RPC?

这一期内容，我们从三个维度了解一下 RPC。

1、什么是远程调用

2、什么是 RPC

3、RPC 的运用场景和优势

相信看完这个内容，你再也不用担心被问到 RPC 的问题了！

53.30.1 什么是远程调用?

有的小伙伴误以为，远程调用，是指跨域物理距离的远。

实际上，远程调用是指跨进程的功能调用，跨进程可以理解成一个计算机节点的多个进程，或者多个计算机节点的多个进程。

有的小伙伴误认为，远程就是距离远。

其实，远程并不是指距离上的远程，而是指由于进程和进程之间彼跨越进程的

53.30.2 什么是 RPC?

RPC 的概念与技术其实是比较早的，40 年前，也就是 1981 年由 Nelson 提出。1984 年，Birrell 和把它用于分布式系统间的通讯。Java 在 1.1 版本提供了 Java 版本的 RPC 框架（RMI）。

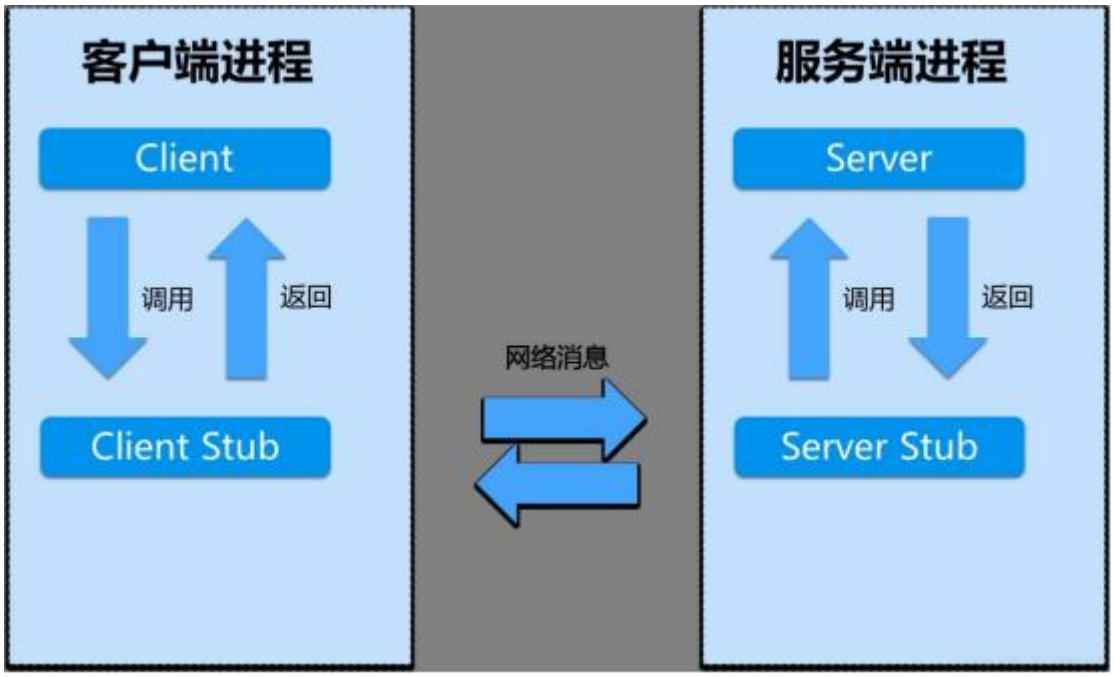
全称为 Remote Procedure Call，翻译过来就是远程过程调用，它是一种通过网络从远程计算机程序上请求服务，而不需要了解底层网络技术的协议，凡是符合该协议的框架，我们都可以称它为 RPC 框架。

关于 RPC 协议，通俗来讲就是，A 计算机提供一个服务，B 计算机可以像调用本地服务那样调用 A 计算机的服务。

要实现 RPC，需要通过网络传输数据，并对调用的过程进行封装。

现在比较流行的 RPC 框架，都会采用 TCP 作为底层传输协议。

RPC 强调的是过程调用，调用的过程对用户而言是是透明的，用户不需要关心调用的细节，可以像调用本地服务一样调用远程服务。



如图所示，一个完整的 RPC 架构里面包含了四个核心的组件，分别是 Client ,Server,Client Stub 以及 Server Stub：

1)客户端（Client），服务的调用方。

2)服务端（Server），真正的服务提供者。

3)客户端存根（Client Stub），存放服务端的地址消息，再将客户端的请求参数打包成网络消息，然后通过网络远程发送给服务方。

4)服务端存根（Server Stub），接收客户端发送过来的消息，将消息解包，并调用本地的方法。

目前比较流行的开源 RPC 框架有 Google 的 gRPC，Facebook 的 Thrift，阿里巴巴的 Dubbo。

53.30.3 RPC 的运用场景和优势

在开发电商系统时，随着业务越来越复杂，走分布式架构这个方向是绝大部分互联网企业必然的选择。

分布式架构落地的里程碑，应该是阿里的去 IOE 运动的成功，它让互联网企业看到了，如何利用更少的软硬件成本来支撑海量的用户。

分布式架构的核心，就是利用多个普通计算机节点，组成一个庞大而复杂的计算网络，提供高性能以及高并发的能力支撑。

在分布式架构中，原本的单体应用被拆分成多个独立部署的服务分部在计算及网络上，这些服务必然需要通过网络进行数据交互。

而 RPC 框架就是解决在分布式架构中的各个业务服务彼此的网络通信问题。

一般来说，RPC 服务主要是针对大型的企业，也就说当我们业务复杂度以及用户量都比较高时，需要解耦服务，扩展性强、部署灵活

一般市面上开源的 PRC 框架，除了提供基础的远程通信功能以外，还会在性能消耗、传输效率、服务治理等方面做很多的设计，比如阿里开源的 RPC 框架 Dubbo。

53.31 [拼多多]分布式事务的原理

大家好，我是 MIC，一个工作了 13 年的 Java 程序员

在互联网企业的面试中，经常会问到分布式、高并发下的技术问题，其中分布式事务问题就是其中之一。

下面我们来看一下，关于分布式事务的解决方案，高手是如何回答这个问题的！

> 以下是高手的回答思路

分布式事务是指事务的参与者、支持事务的服务器、资源服务器以及事务管理器分别位于分布式系统的不同节点之上。

比如大型的电商系统中的下单场景，会涉及到扣库存、优惠促销计算、订单 ID 生成。

通常情况下，库存、促销、主键生成策略都位于不同的服务器和数据库表中。

下单接口的成功与否，不仅取决于本地节点的数据库操作，而且还依赖第三方系统的结果，这时候分布式事务就保证这些操作要么全部成功，要么全部失败。

因此，本质上来说，分布式事务就是为了保证不同数据库的数据一致性。

基于 CAP 定理可以知道，对于上述情况产生的分布式事务问题，我们要么采用强一致性方案、要么采用弱一致性方案。

- 所谓强一致性，就是指通过第三方的事务管理器来协调多个节点的事务，保证每个节点的事务达到同时成功和同时失败，为了实现这样一个需求，我们会引入

Xopen/DTP 模型提供的 XA 协议，基于 2pc 或者 3pc 的方式来实现。但是，在如果全局事务管理器中的多个节点中，如果任意一个节点再进行事务提交确认时，由于网络通信延迟导致阻塞，就会影响到所有节点事务的提交，而这个阻塞过程也会影响阻塞用户的请求线程，这对于用户体验以及整体性能的影响较大。

- 而弱一致性方案，就是针对强一致性方案所衍生出来性能和数据一致性平衡的一个方案，简单来说就是损失掉强一致性，数据在某一个时刻会存在不一致的状态，但是最终这些数据会达成一致。这样的好处是提升了系统的性能。

在弱一致性方案中，常见的解决方案：

1. 使用分布式消息队列，来实现最终一致性
2. 基于 TCC 事务，通过演进版本的二阶段提交实现最终一致性
3. 使用 Seata 事务框架，它提供了多种事务模式，如 AT、XA、Saga、TCC 等

53.32 [拼多多]实现分布式锁的解决方案中，你认为 Zookeeper 和 Redis 哪种更好?

以下是高手的回答思路

关于这个问题，我想从 3 个方面来说：

53.32.1 为什么使用分布式锁?

使用分布式锁的目的，是为了保证同一时间只有一个 JVM 进程可以对共享资源进行操作。

53.32.2 根据锁的用途可以细分为以下两类：

- **允许多个客户端操作共享资源，我们称为共享锁**

这种锁的一般是对共享资源具有幂等性操作的场景，主要是为了避免重复操作共享资源频繁加锁带来的性能开销。

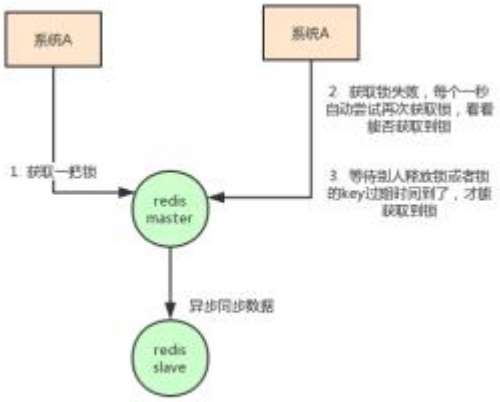
- **只允许一个客户端操作共享资源，我们成为排他锁**

这种锁一般是用在对共享资源操作具有非幂等性操作的场景，也就是需要保证在同一时刻只有一个进程或者线程能够访问这个共享资源。

53.32.3 目前实现分布式锁最常用的中间件是 Redis 和 Zookeeper

第一种，Redis 可以通过两种方式来实现

1. 利用 Redis 提供的`**SET key value NX PX milliseconds (打印在屏幕上)** `指令，这个指令是设置一个 key-value，如果 key 已经存在，则返回 0，否则返 回 1，我们基于这个返回值来判断锁的占用情况从而实现分布式锁。



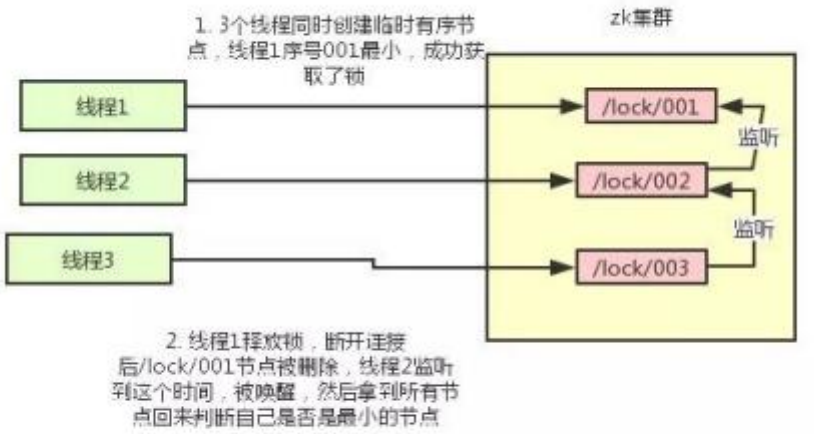
2. 基于 Redission 客户端来实现分布式锁，Redisson 提供了分布式锁的封装方法，我们只需要调用 api 中的`lock（）`和`unlock()`方法。它帮我们封装锁实现的细节和复杂度

- redisson 所有指令都通过 lua 脚本执行并支持 lua 脚本原子性执行
- redisson 中有一个 watchdog 的概念，翻译过来就是看门狗，它会在你获取锁之后，每隔 10 秒帮你把 key 的超时时间设为 30s，就算一直持有锁也不会出现 key 过期了。“看门狗”的逻辑保证了没有死锁发生。

第二种，基于 ZK 实现分布式锁的落地方案

画外音字幕：关于 Zookeeper 是什么，大家可以看看我前几天上传 B 站的视频《谈谈你对 Zookeeper 的理解》

Zookeeper 实现分布式锁的方法比较多，我们可以使用有序节点来实现，



53.32.4 两种方案都有各自的优缺点

> 对于 redis 的分布式锁而言，它有以下缺点：

- 它获取锁的方式简单粗暴，如果获取不到锁，会不断尝试获取锁，比较消耗性能。
- Redis 是 AP 模型，在集群模式中由于数据的一致性会导致锁出现问题，即便使用 Redlock 算法来实现，在某些复杂场景下，也无法保证其实现 100%的可靠性。

不过在实际开发中使用 Redis 实现分布式锁还是比较常见，而且大部分场情况下不会遇到”极端复杂的场景“，更重要的是 Redis 性能很高，在高并发场景中比较合适。

> 对于 zk 分布式锁而言：

- zookeeper 天生设计定位就是分布式协调，强一致性。锁的模型健壮、简单易用、适合做分布式锁。
- 如果获取不到锁，只需要添加一个监听器就可以了，不用一直轮询，性能消耗较小。

如果要在两者之间做选择，就我个人而言的话，**比较推崇 ZK 实现的锁，因为对于分布式锁而言，它应该符合 CP 模型，但是 Redis 是 AP 模型，所以在这个点上，Zookeeper 会更加合适。**

好的，看完高手的回答后，相信每位看完视频的小伙伴对分布式锁有了更深刻的理解 ，本期普通人 VS 高手系列的视频就到这里就结束了，喜欢的朋友一键三连，加个关注，我是头发很多的程序员 Mic，咱们下期见！

53.33 [拼多多]关于“你对 Spring Cloud 的理解”

看看普通人和高手是如何回答这个问题的?

普通人

Spring Cloud 是一套微服务解决方案，它包括配置中心、RPC 通信、服务注册、服务熔断等组件

高手

Spring Cloud 是一套分布式微服务的技术解决方案，它提供了快速构建分布式系统的常用的一些组件

比如说配置管理、服务的注册与发现、服务调用的负载均衡、资源隔离、熔断降级等等

不过 Spring Cloud 只是 Spring 官方提供的一套微服务标准定义，而真正的实现目前有两套体系用的比较多。

- 一个是 Spring Cloud Netflix
- 一个是 Spring Cloud Alibaba

Spring Cloud Netflix 是基于 Netflix 这个公司的开源组件集成的一套微服务解决方案，其中的组件有

1. Ribbon——负载均衡
2. Hystrix——服务熔断
3. Zuul——网关
4. Eureka——服务注册与发现
5. Feign——服务调用

Spring Cloud Alibaba 是基于阿里巴巴开源组件集成的一套微服务解决方案，其中包括

1. Dubbo——消息通讯
2. Nacos——服务注册与发现
3. Seata——事务隔离
4. Sentinel——熔断降级

有了 Spring Cloud 这样的技术生态，使得我们在落地微服务架构时。不用去考虑第三方技术集成带来额外成本，只要通过配置组件来完成架构下的技术问题，从而可以让我们更加侧重性能方面

以上这些就是我对 Spring Cloud 的个人理解！

53.34 关于“谈谈你对 ES 的理解，看看普通人和高手是如何回答这个问题的？”

> 普通人

它是一个基于 Apache Lucene 开源的一个分布式搜索引擎框架。

一般用它来做

- 日志记录和分析
- 公共数据采集
- 全文检索
- 数据可视化分析等等

> 高手

Elasticsearch ，简称 ES 。它是建立在全文搜索引擎库 Apache Lucene 基础之上的一个开源的搜索引擎，也可以作为 NoSQL 数据库，存储任意格式的文档和数据。也可以做大数据的分析，是一个跨界开源产品。

它最主要的应用场景是 ELK 的日志分析系统。

另外它还有以下特点：

1. 第一、采用 Master-slave 架构，实现数据的分片和备份
2. 第二、使用 Java 编写，并对 Lucene 进行封装，隐藏了 Lucene 的复杂性
3. 第三、能胜任上百个服务节点的扩展，并支持 PB 级别的结构化或者非结构化数据
4. 第四、ES 提供的 Restful API，不仅简化了 ES 的操作，还支持任何语言的客户端提供 API 接口，另外 Restful API 的风格还实现了 CURD 操作、创建索引，删除索引等功能。

在 ES 的使用上我也有一些经验心得，比如：

1. es 里面复杂的关联查询尽量别用，一旦用了性能都不太好。最好是先在 Java 系统里就完成关联，将关联好的数据直接写入 ES 中
2. 避免一些太复杂的操作，比如 join/nested/parent-child 搜索，不然容易出现性能问题。
3. 避免深分页查询，ES 集群的分页查询支持 from 和 size 参数，查询的时候，每个分片必须构造一个长度为 from+size 的优先队列，然后回传到网关节点，网关节点再对这些优先队列进行排序找到正确的 size 文档。当 from 足够大的时候，就算不发生 OOM，也会影响到 CPU 和带宽等，从而影响到整个集群的性能。

以上就是我对 ES 的认识和使用心得。

相信每位看完视频的小伙伴对会对 ElasticSearch 有了初步的理解了。关于 ES 的内容体系非常庞大，你还想听哪些 ES 的相关内容，请在下方的评论区留言！

本期的普通人 VS 高手系列就到这里就结束了，喜欢的朋友一键三连，加个关注，我是头发很多的程序员 Mic，咱们下期见！

53.35 关于“你对 MVCC 的理解，看看普通人和高手是如何回答这个问题的?

对于 MVCC 的理解，我觉得可以先从数据库的三种并发场景说起：

53.35.1 第一种：读读

就是线程 A 与线程 B 同时在进行读操作，这种情况下不会出现任何并发问题。

53.35.2 第二种：读写

就是线程 A 与线程 B 在同一时刻分别进行读和写操作。

这种情况下，可能会对数据库中的数据造成以下问题：

- 事物隔离性问题，
- 出现脏读，幻读，不可重复读的问题

53.35.3 第三种：写写

就是线程 A 与线程 B 同时进行写操作

这种情况下可能会存在数据更新丢失的问题。

而 MVCC 就是为了解决事务操作中并发安全性问题的无锁并发控制技术全称为 Multi-Version Concurrency Control ，也就是多版本并发控制。它是通过数据库记录中的隐式字段，undo 日志 ，Read View 来实现的。

MVCC 主要解决了三个问题

- 第一个是：通过 MVCC 可以解决读写并发阻塞问题从而提升数据并发处理能力
- 第二个是：MVCC 采用了乐观锁的方式实现，降低了死锁的概率
- 第三个是：解决了一致性读的问题也就是事务启动时根据某个条件读取到的数据，直到事务结束时，再次执行相同条件，还是读到同一份数据，不会发生变化。

而我们在使用 MVCC 时一般会根据业务场景来选择组合搭配乐观锁或悲观锁。

这两个组合中，MVCC 用来解决读写冲突，乐观锁或者悲观锁解决写写冲突从而最大程度的提高数据库并发性能。

以上就是我的对 MVCC 的理解。

53.36 [拼多多]说一下你对 Redis 的理解

1. Redis 是一个高性能的基于 Key-Value 结构存储的 Nosql 开源数据库。
2. 目前市面上绝大部分公司都采用 Redis 来实现分布式缓存，从而提高数据的检索效率。
3. Redis 之所以这么流行，主要有几个特点：
    a. 它是基于内存存储，在进行数据 IO 操作时，能够 10WQPS
    b. 提供了非常丰富的数据存储结构，如 String、List、Hash、Set、ZSet 等。
    c. Redis 底层采用单线程实现数据的 IO，所以在数据算法层面并不需要要考虑并发安全性，所以底层算法上的时间复杂度基本上都是常量。
4. Redis 虽然是内存存储，但是它也可以支持持久化，避免因为服务器故障导致数据丢失的问题

基于这些特点，Redis 一般用来实现分布式缓存，从而降低应用程序对关系型数据库检索带来的性能影响。除此之外，Redis 还可以实现分布式锁、分布式队列、排行榜、查找附近的人等功能，为复杂应用提供非常方便和成熟的解决方案

53.37 关于“你对 Zookeeper 的理解，看看普通人和高手是如何回答这个问题的?

以下是高手的回答思路

对于 Zookeeper 的理解，我觉得可以从分布式系统中的三种典型应用场景说起：

- **第一种：集群管理**

在多个节点组成的集群中，为了保证集群的 HA 特性，每个节点都会冗余一份数据副本。这种情况下需要保证客户端访问集群中的任意一个节点都是最新的数据。

- **第二种：分布式锁**

如何保证跨进程的共享资源的并发安全性，对于分布式系统来说也是一个比较大的挑战，而为了达到这样一个目的，必须要使用跨进程的锁也就是分布式锁来实现。

- **第三种：Master 选举**

在多个节点组成的集群中，为了降低集群数据同步的复杂度，一般会存在 Master 和 Slave 两种角色的节点，Master 负责事务和非事务请求处理，Slave 负责非事务请求处理。但是在分布式系统中如何确定某个节点是 Master 还是 Slave，也成了一个难度不小的挑战。

> 基于这三类常见场景的需求，所以产生了 Zookeeper 这样一个中间件。

它是一个分布式开源协调组件，简单来说，就是类似于一个裁判员的角色，专门负责协调和解决分布式系统中的各类问题。

比如，针对上述描述的问题，Zookeeper 都可以解决。（这个地方每次讲解一个点，都需要突出标题）

1. 集群管理（凸显）

Zookeeper 提供了 CP 的模型，来保证集群中的每个节点的数据一致性，当然 Zk 本身的集群并不是 CP 模型，而是顺序一致性模型，如果要保证 CP 特性，需要调用`sync`同步方法。

2. 分布式锁（凸显）

Zookeeper 提供了多种不同的节点类型，如持久化节点、临时节点、有序节点、容器节点等，其中对于分布式锁这个场景来说，Zookeeper 可以利用有序节点（插入图片）的特性来实现。除此之外，还可以利用同一级节点的唯一性特性来实现分布式锁。

3. Master 选举（凸显）

Zookeeper 可以利用持久化节点来存储和管理其他集群节点的信息，从而进行 Master 选举机制。或者还可以利用集群中的有序节点特性，来实现 Master 选举。

目前主流的 Kafka、Hbase、Hadoop 都是通过 Zookeeper 来实现集群节点的主从选举。

总的来说，Zookeeper 就是经典的分布式数据一致性解决方案，致力于为分布式应用提供高性能、高可用，并且具有严格顺序访问控制能力的分布式协调服务。它底层通过基于 Paxos 算法演化而来的 ZAB 协议实现。

以上就是我对于 Zookeeper 的理解。

53.38 [字节跳动]关于索引的底层实现,为什么选择 B+Tree 而不是红黑树?

以下是高手的回答思路

我主要从两个点来回答

- 第一点

对于一个数据库来说 存储的数据量会比较多，导致索引也很大 因此需要将索引存储在磁盘，但是磁盘的 IO 操作又非常耗，所以提高索引效率的关键在于减少磁盘 IO 的次数。

> 举个例子 对于 31 个节点的树来说 ， 一个 5 阶 B+Tree 的高度是 3 一个红黑树的最小高度是 5，树的高度基本决定了磁盘的 IO 次数 ，所以使用 B+Tree 性能要高很多

- 第二点

B+Tree 有个特点是相邻的数据在物理上也是相邻的，因为 B+Tree 的 node 的大小设为一个页，而一个节点上存有多个相邻的关键字和分支信息，每个节点只需要一次 IO 就能完全载入，相当于一次 IO 载入了多个相邻的关键字和分支，而红黑树不具有这个特性，红黑树中大小相邻的数据，在物理结构上可能距离相差很大。由于程序的局部性原理，如果我们在索引中采用了预加载的技术，每次磁盘访问的时候除了将访问到的页

加载到磁盘，我们还可以基于局部性原理加载，几页相邻的数据到内存中，而这个加载是不需要消耗多余磁盘 IO 时间的。

因此 基于局部性原理，以及 B+Tree 存储结构物理上的特性，所以 B+Tree 的索引性能比红黑树要好很多。

好的关于普通人和高手的回答谁的回答较好呢，大家心中自有定论如果有更好的回答欢迎在评论区补充留言，最后呢 ，本期的普通人 VS 高手系列视频就到这里结束了喜欢的朋友不妨点个关注一键三连，咱们下期不见不撒！

53.39 [字节跳动]还原面试现场，带大家理解何为 Docker

大家好，我是 MIC，一个工作了 13 年的 Java 程序员

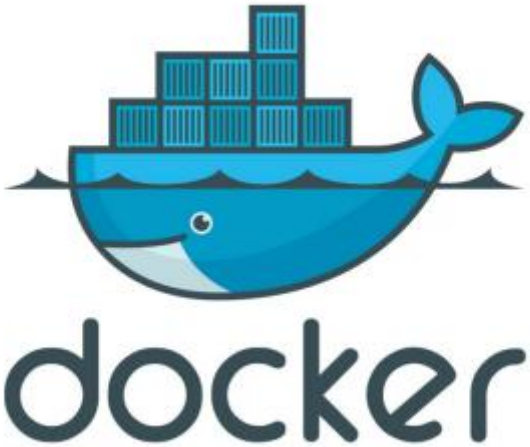
今天，我们通过一场面试，带大家理解什么是 Docker，什么是容器！

53.39.1 面试官：什么是 Docker

MIC：我个人理解，Docker 是一个超轻量级的虚拟机，也是实现容器化技术的一种应用工具。

Docker 是使用时下很火的 Golang 语言进行开发的，基于 Linux 内核的 cgroup，namespace，以及 AUFS 类的 UnionFS 等技术，对进程进行封装隔离，属于操作系统层面的虚拟化技术。 由于隔离的进程独立于宿主和其它的隔离进程，因此也被称为容器。

我们来看一下Docker的LOGO，一头鲸鱼驮着一堆集装箱，什么意思呢？【展示Docker的 LOGO】



集装箱表现的含义是：把所有不规整的货物，按照统一标准进行包装，在 Docker 中，它代表各种不同的应用程序使用 Docker 的标准来实现构建。

而运输集装箱的鲸鱼，代表操作系统！

53.39.2 面试官：那你再说说为什么要用 Dcoker，它具有哪些优势?

MIC：基于我的理解，Docker 有六大优势。

资源占用小

由于容器不需要进行硬件虚拟，也不需要运行完整操作系统等额外的资源开销，使得 Docker 对系统资源的利用率更高，无论是应用执行速度还是文件存储速度,都要比传统虚拟机技术更高效,内存消 耗更少 。

启动速度快

传统的虚拟机技术启动应用服务往往需要较长时间，而 Docker 容器应用，由于直接运行于宿主内核，无需启动完整的操作系统，因此可以做到秒级，甚至毫秒级的启动时间，大大的节约了开发，测试，部署的时间。

迁移更轻松

由于 Docker 确保了执行环境的一致性，使得应用的迁移更加容易，
Docker 可以在很多平台上运行，无论是物理机，虚拟机，公有云，私有云，它们的运行结果是一致的，
因此用户可以很轻易的将一个平台上运行的应用，迁移到另一个平台上，而不用担心运行环境的变化导致应用无法正常运行这类的问题。

维护和拓展更轻松

docker 使用的分层存储和镜像技术，让应用重复部分的复用更容易，也让应用的维护更新更简单，基于基础镜像进一步扩展镜像也变得十分简单。
另外，docker 团队和各个开源项目团队一起维护了一大批高质量的官网镜像，既可以直接在生产环境使用，又可以作为基础进一步定制，大大降低了应用服务的镜像制作成本。

运行环境一致

开发过程中一个常见的问题是环境一致性问题，由于开发环境，测试环境，生产环境不一致，导致有些 bug 并未在开发过程中被发现，
而 Docker 的镜像提供了除内核外完整的运行时环境，确保了应用运行环境一致性。

持续交付和部署

使用 Docker 可以通过定制应用镜像来实现持续集成，持续交付，部署。开发人员可以通过 Dockerfile 来进行镜像构建，
并结合持续集成系统进行集成测试，而运维人员则可以在生产环境中快速部署该镜像，甚至结合持续部署系统进行自动部署

53.39.3 面试官：再说说 Docker 核心组件吧！

MIC：

Docker 包含三个基本组件：

- 镜像（Image）
- 容器（Contaier）

- 仓库（Repository）

镜像（Image）——一个特殊的文件系统

简单地理解，Docker 镜像就是一个 Linux 的文件系统（Root FileSystem），这个文件系统里面包含可以运行在 Linux 内核的程序以及相应的数据。

一个镜像可以包含一个完整的操作系统环境，里面仅安装了 Apache 或用户需要的其它应用程序。镜像可以用来创建 Docker 容器。Docker 提供了一个很简单的机制来创建镜像或者更新现有的镜像，用户甚至可以直接从其他人那里下载一个已经做好的镜像来直接使用。

容器（Container）——镜像运行的实体

Docker 利用容器来运行应用。容器是从镜像创建的运行实例。它可以被启动、开始、停止、删除。每个容器都是相互隔离的、保证安全的平台。

可以把容器看做是一个简易版的 Linux 环境（包括 root 用户权限、进程空间、用户空间和网络空间等）和运行在其中的应用程序。

仓库（Repository）——集中存放镜像文件的地方

仓库是集中存放镜像文件的场所。很多人会把仓库和仓库注册服务器（Registry）混为一谈。实际上，仓库注册服务器上往往存放着多个仓库，每个仓库中又包含了多个镜像，每个镜像有不同的标签（tag）。

仓库分为公开仓库（Public）和私有仓库（Private）两种形式。最大的公开仓库是 Docker Hub，存放了数量庞大的镜像供用户下载。

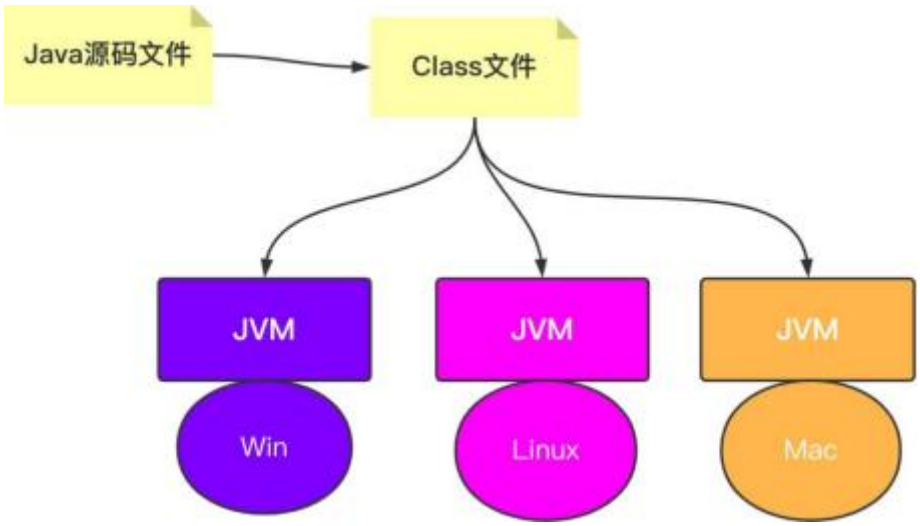
以上这些就是我对 Docker 的了解.

面试官：好的，看完本期视频，相信大家对 Docker 有了一定的了解，当然关于 Docker 这个话题远不止今天聊的这些，例如：Docker 的基本架构，运行原理，数据卷等等，有兴趣的同学可以在评论区留言，我会在后续的视频中更新。

53.40 [字节跳动]说一下对于 JVM 底层的深度理解

以下是高手的回答思路

JVM 全称是 Java 虚拟机，在聊什么是 JVM 之前，我们不妨看一下这张图。



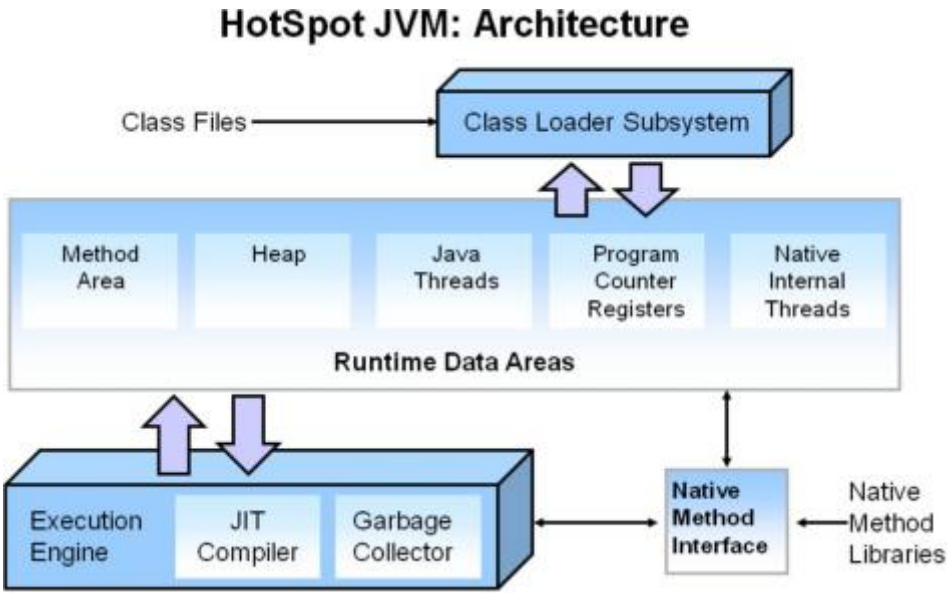
从这张图中可以看出 JVM 所处的位置，同时也能看出它两个作用：

- 运行并管理 Java 源码文件所生成的 Class文件，
- 在不同的操作系统上安装不同的 JVM，从而实现了跨平台的保证。

一般情况下，对于开发者而言，即使不熟悉 JVM 的运行机制并不影响业务代码的开发，因为在安装完 JDK 或者 JRE 之后，其中就已经内置了 JVM，所以只需要将 Class文件交给 JVM 运行即可。

但当程序运行的过程中出现了问题，而这个问题发生在 JVM 层面的，那我们就需要熟悉 JVM 的运行机制，才能迅速排查并解决 JVM 的性能问题。

我们先看下目前主流的 JVM HotSpot 的架构图，通过这张架构图，我们可以看出 JVM 的大致流程是把一个 class 文件通过类加载器加载进系统，然后放到不同的区域，通过编译器编译。



第一个部分 Class Files

在 Java 中，Class文件是由源码文件生成的，至于源码文件的内容，是每个 Java 开发者在 JavaSE 阶段的必备知识，这里就不再赘述了，我们可以关注一下 Class文件的格式，比如其中的常量池、成员变量、方法等，这样就能知道 Java 源码内容在 Class文件中的表示方式

第二个部分 Class Loader Subsystem 即类加载机制

Class文件加载到内存中，需要借助 Java 中的类加载机制。类加载机制分为装载、链接和初始化，其主要就是对类进行查找、验证以及分配相关的内存空间和赋值

第三个部分 Runtime Data Areas 也就是通常所说的运行时数据区

其解决的问题就是 Class文件进入内存之后，该如何进行存储不同的数据以及数据该如何进行扭转。比如：Method Area 通常会储存由 Class文件常量池所对应的运行时常量池、字段和方法的元数据信息、类的模板信息等；Heap 是存储各种 Java 中的对象实例；Java Threads 通过线程以栈的方式运行加载各个方法；Native Internal Thread 可以理解为是加载运行native 类型的方法；PC Register 则是保存每个线程执行方法的实时地址。

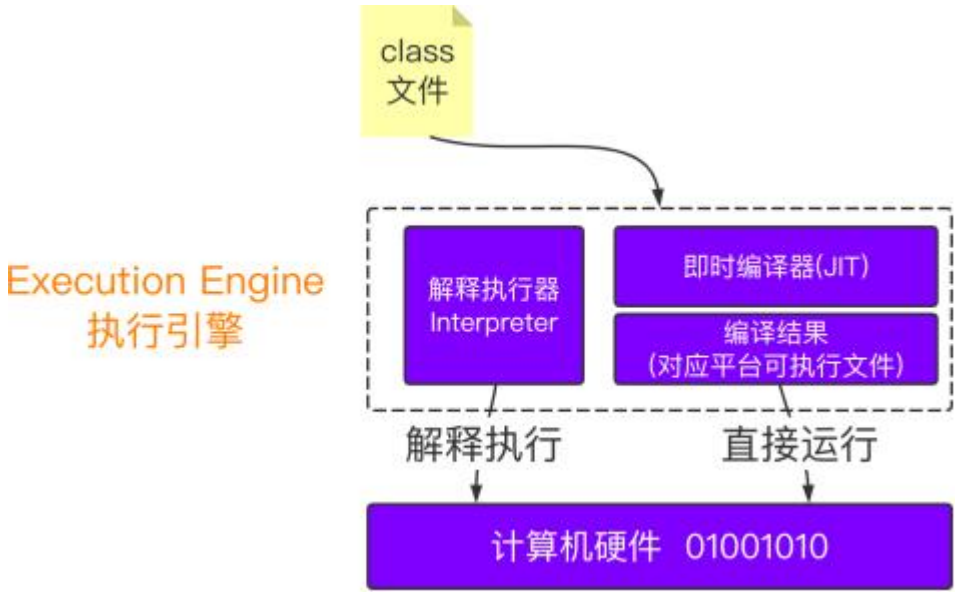
这样通过运行时数据区的 5 个部分就能很好地把数据存储和运行起来了

第四个部分 Garbage Collector 也就是通常所说的垃圾回收

就是对运行时数据区中的数据进行管理和回收。回收机制可以基于不同的垃圾收集器，比如 Serial、Parallel、CMS、G1、ZGC 等，可以针对不同的业务场景选择不同的收集器，只需要通过 JVM 参数设置 即可。如果我们打开 hotspot 的源码，可以发现这些收集器其实就是对于不同垃圾收集算法的实现，核心的算法有 3 个：标记-清除、标记-整理、复制

第五个部分是 JIT Compiler 和 Interpreter

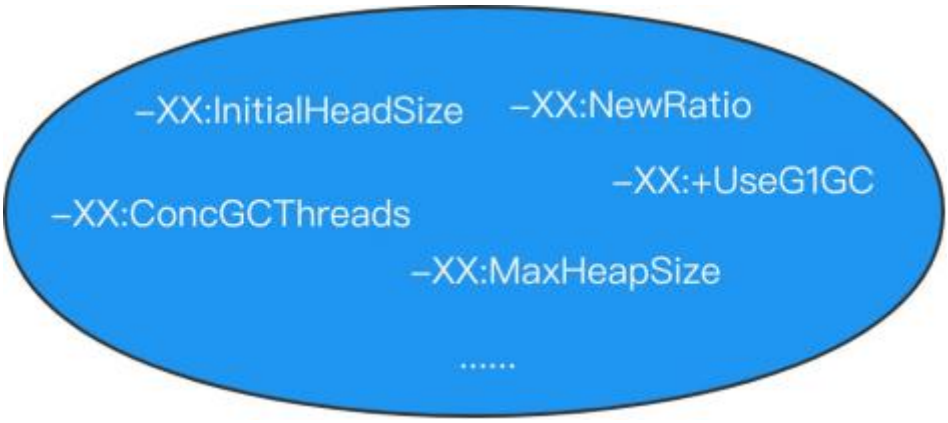
通俗理解就是翻译器，Class 的字节码指令通过 JIT Compiler 和 Interpreter 翻译成对应操作系统的 CPU 指令，只不过可以选择解释执行或者编译执行，在 HotSpot JVM 默认采用的是这两种方式的混合。



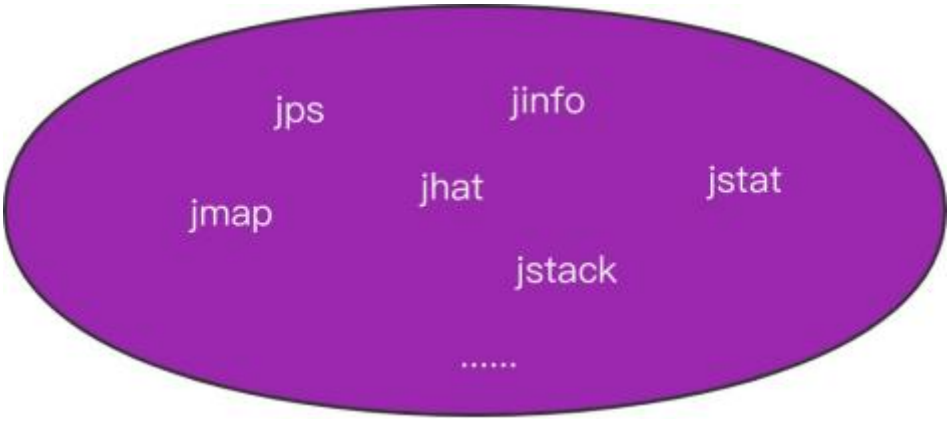
第六就是 JNI 的技术

如果我们想要找 Java 中的某个 native方法是如何通过 C 或者 C++实现的，那么可以通过 Native Method Interface 来进行查找，也就是所谓的 JNI 技术。

通过官网上给出的 HotSpot 架构图，我们就能够知道 JVM 到底是如何运行的了，当然在实际操作的过程中我们可以借助一些 JVM 参数：例如：【MIC 停顿 2S，展示图片】



和一些常见的 JDK 常见命令【MIC 停顿 2S，展示图片】



再结合 JDK 常见工具以及第三方的一些工具，【MIC 停顿 2S，展示图片】

我们就可以优雅地分析 JVM 出现的常见问题并对其进行调优。

以上就是我对 JVM 的理解。

好的，看完高手的回答后，相信每位看完视频的小伙伴对 JVM 有了更深刻的理解了 ，本期普通人 VS 高手系列的视频就到这里就结束了，喜欢的朋友一键三连，加个关注，我是头发很多的程序员 Mic，咱们下期见！

53.40 [字节跳动]5 分钟带你快速了解什么是 Kubernetes

近些年，随着 Docker 与微服务的普及，K8s 也乘着这两股东风，迅速蹿红起来。

作为最火的容器编排工具之一，它的很多思想设计都契合了微服务和云原生应用的设计法则。

也正因如此，越来越多的公司开始使用起 k8s。

那么，称霸容器界的 k8s，究竟有什么魔力呢？

大家好，我是 MIC，一个工作了 13 年的 Java 程序员。

今天，我们通过这一期内容，带大家了解一下，什么是 Kubernetes？

K8s 全称 Kubernetes，8 是中间 8 个字母的简称。

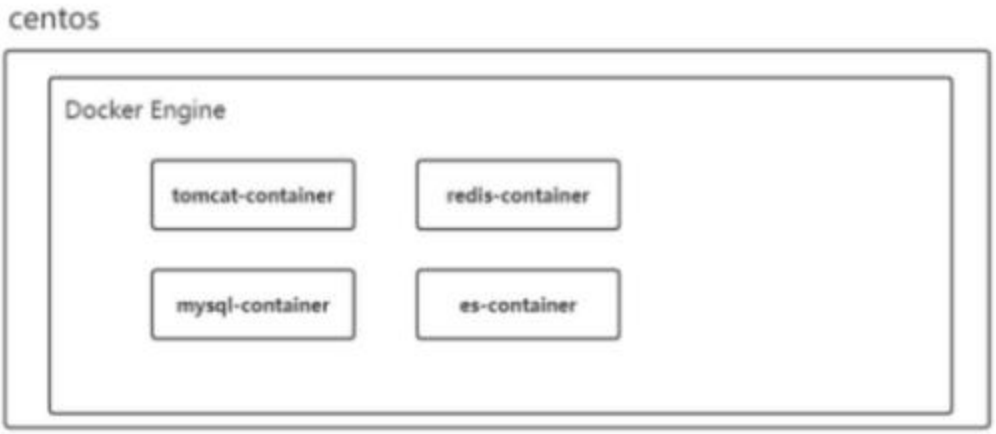
作为一种容器自动部署、扩容以及管理的技术，我们可以简单理解其是一种容器编排技术。前身是 Borg 系统，在谷歌内部已经有了十多年的使用经验。

53.40.1 那什么是容器编排技术？

我们先以 Docker 容器为例，

如果大家不熟悉什么是 Docker 的话，

可以翻阅一下 Docker 相关的内容。

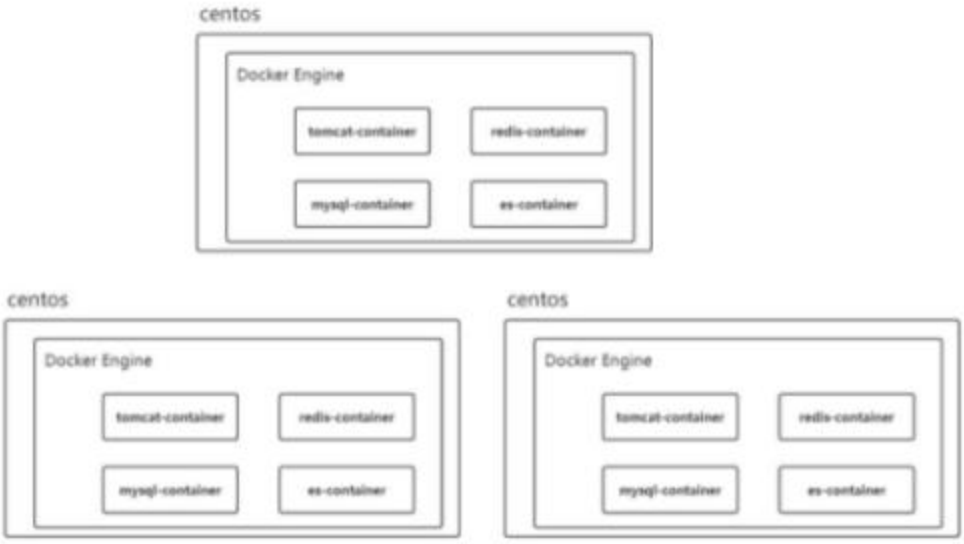


如图所示，在一台 centos 物理机上安装了 Docker 镜像，然后在镜像上跑几个 container 容器。

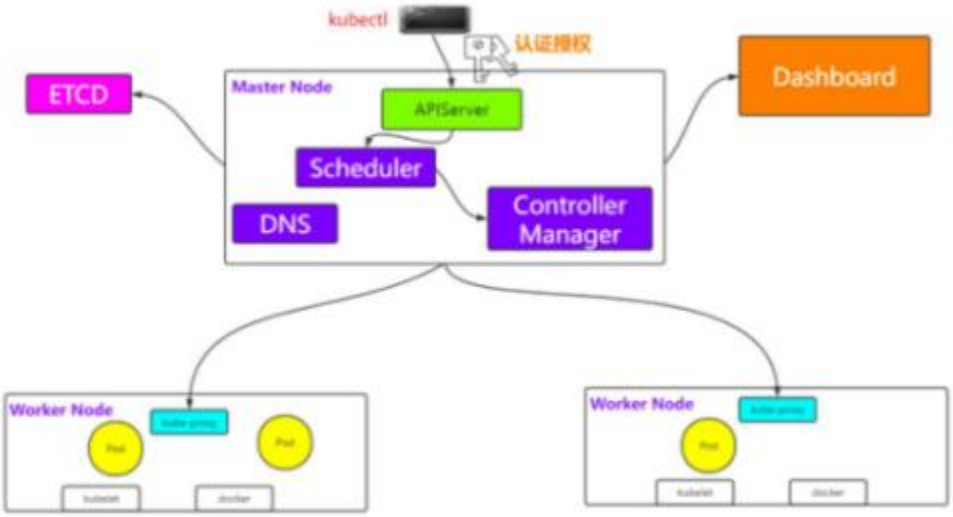
那最明显的一个问题是，当物理机 centos 资源不够使用的时候，就没办法再创建新的 container 了，

另外一个问题就是单点故障，也就是当安装了 Docker Engine 的这台 centos 机器挂了之后，所有的 container 都会挂掉，这样很多中间件和服务都变成不可用了，

所以最直接的解决方案，就是对 centos 机器做集群化的部署，如图所示。



但问题是怎样通过一个统一的入口，管理安装了 Docker Engine 的 3 台 centos 机器，不然每台 centos 就变成孤立的了，这里就需要用到容器编排的技术，我们就直接以 K8s 中的组件和大家来分析一下，它究竟是如何做到统一管理的。



可以看到这是由 3 台 centos 机器组成的 Docker 集群或者 K8s 集群，其中 1 台是 master，另外 2 台是 worker。

每台机器上都安装了 Docker Engine，当需要对容器进行操作时，会通过 kubectl 来接收用户的操作命令。

API Server 会做客户端的命令进行认证授权，接下来会通过 Scheduler 组件进行调度，就可以知道要创建的资源到底会落到哪个机器上。

然后会通过不同类型的 Controller Manager 进行管理要创建的资源，比如要将资源创建在其中一个 worker 节点上，就会把请求转发给 worker 节点的 kube-proxy 组件。

看到这里，要跟大家强调一点，在 K8s 中最小的操作单元是 Pod，它可以理解为是 container 的逻辑单位，不会直接操作 docker 中的 container，

而是会以 Pod 的形式来操作，因为 Pod 里面包含了各个类型的 container。

53.40.2 那在 worker 中是如何操作 Pod 的呢?

就是通过 kubelet 组件完成的。当然可以发现，整个 K8s 集群中还可以持久化一些数据到 ETCD 组件里，

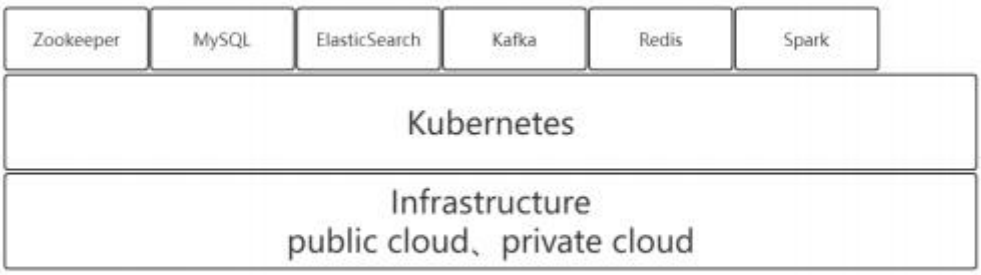
可以通过 Dashboard 查看整个集群资源的状态，可以通过 DNS 组件进行相应的域名解析等， 这些组件和功能都是可选的、可扩展的。

到这里我们不妨再想一个问题，K8s 集群中的物理机可以是不同的硬件和操作系统吗?

显然是没问题的，这样一来我们就可以通过 Docker 和 K8s 来屏蔽基础设施的差异性，在上层进行容器创建以及管理就变得方便多了，所以 K8s 也是云原生中很重要的基础保障。

如图所示，最下面的基础设施可以是各种公有云、私有云、混合云，在此基础上构建 K8s 的集群支撑，

然后就可以非常方便调度管理不同的容器了。



看到这里，你明白容器编排技术的工作原理了吗?

容器作为一种先进的虚拟化技术，是云原生时代软件开发和运维不可或缺的部分。

而 K8s 作为当前最知名的容器编排工具，称得上是云原生时代的“操作系统”，熟悉和使用它是研发、运维、产品等的必备技能。

53.41 [连环问]介绍一下自己对 Netty 的认识吧

以下是高手的回答思路

好的，我用三点来简单的介绍下 Netty

- 第一：Netty 是一个 **基于 NIO** 模型的高性能网络通信框架，其实可以认为它是对 NIO 网络模。型的封装，提供了简单易用的 API，我们可以利用这些封装好的 API 快速开发自己的网络程序。
- 第二：Netty 在 NIO 的基础上做了很多优化，比如零拷贝机制、高性能无锁队列、内存池等，因此性能会比 NIO 更高。
- 第三：Netty 可以支持多种通信协议，如 Http、WebSocket 等，并且针对数据通信的拆包黏包问题，Netty 内置了拆包策略。

53.41.1 面试官：哦，还不错，那你在说说为什么要用 Netty ？

Nety 相比于直接使用 JDK 自带的 NIO 相关的 API 来说更加易用。同时，它还具有以下特点：

1. 统一的 API，支持多种传输类型，如阻塞、非阻塞，以及 epoll、poll 等模型。
2. 我们可以使用非常少的代码来实现，多线程 Reactor 模型以及主从多线程 Reactor 模型
3. 自带编解码器解决 TCP 粘包/拆包问题。
4. 自带各种协议栈。
5. 比直接使用 Java 库中的 NIO API 有更高的吞吐量、更低的延迟、更低的资源消耗和更少的内存复制。
6. 安全性不错，有完整的 SSL/TLS 以及 StartTLS 支持。
7. 社区活跃成熟稳定，经历了大型项目的使用和考验，而且很多开源项目都使用到了 Netty， 比如我们经常接触的 Dubbo、RocketMQ 等等。

53.41.2 面试官：那你在通俗地说一下 Netty 可以做什么事情?

我们之所以要用 Netty，核心点还是在于解决服务器如何承载更多的用户同时访问的问题。

传统的 BIO 模型，由于阻塞的特性，使得在高并发场景中，很难获得更高的吞吐量。

而后来基于 NIO 的多路复用模型虽然在阻塞方面进行了优化，但是它的 API 使用比较复杂，对于初学者来说使用不是很友好。而 Netty 是基于 NIO 的封装，提供了成熟且简单易用的 API，降低了使用成本和学习成本。

本质上来说，Netty 和 NIO 所扮演的角色是相同的，都是为了提升服务端的吞吐量，让用户获得更好的产品体验。

另外，Netty 这个中间件经过很多年的验证，在目前主流的中间件如 Zookeeper、Dubbo、RocketMQ 中都有应用。

53.41.3 面试官：Netty 核心组件了解吗？分别有什么作用？

高手：Netty 由三层结构构成：网络通信层、事件调度器与服务编排层

在网络通信层有三个核心组件：Bootstrap、ServerBootStrap、Channel

- Bootstrap 负责客户端启动并用来链接远程 netty server
- ServerBootStrap 负责服务端监听，用来监听指定端口，
- Channel 是负责网络通信的载体

事件调度器有两个核心组件：EventLoopGroup 与 EventLoop

- EventLoopGroup 本质上是一个线程池，主要负责接收 I/O 请求，并分配线程执行处理请求。
- EventLoop。相当于线程池中的线程

在服务编排层有三个核心组件 ChannelPipeline、ChannelHandler、ChannelHandlerContext

- Channel**Pipeline 负责将**多个 Channelhandler 链接在一起
- ChannelHandler 针对 IO 数据的处理器，数据接收后，通过指定的 Handler 进行处理。
- ChannelHandlerContext 用来保存 ChannelHandler 的上下文信息

面试官：那，在说说 Netty 有几种线程模型吧?

高手 Netty 提供了三种 Reactor 模型的支持

- 单线程单 Reactor 模型
- 多线程单 Reactor 模型
- 多线程多 Reactor 模型

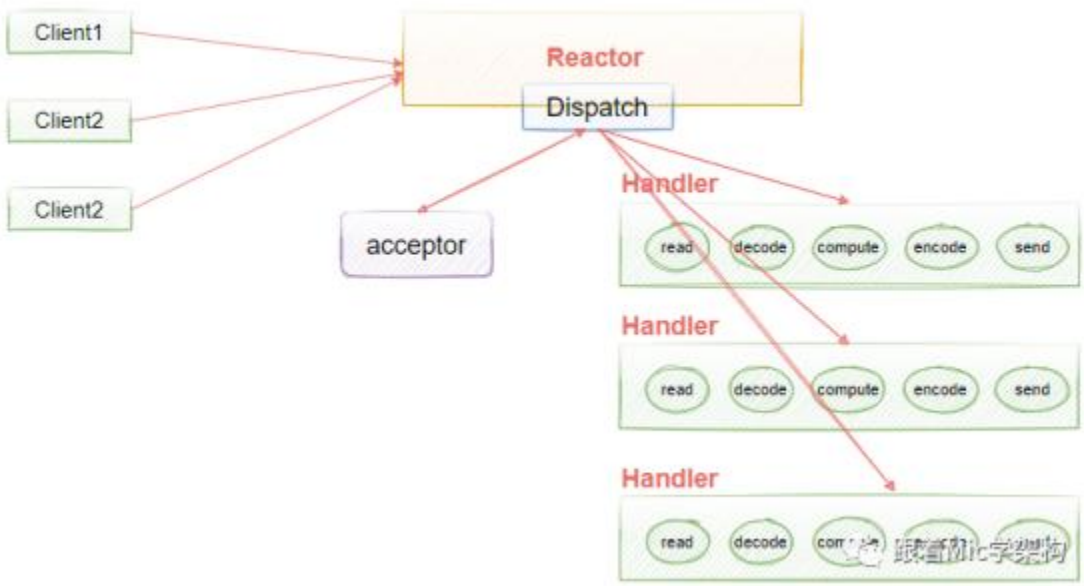
53.41.4 面试官：你说一下对于这三种线程 Reactor 模型的理解?

高手：Reactor 模型有三个重要的组件：

1. **Reactor ：**将 I/O 事件发派给对应的 Handler
2. **Acceptor ：**处理客户端连接请求
3. **Handlers ：**执行非阻塞读/写

> 这是最基本的单 Reactor 单线程模型

我们来看这张图：



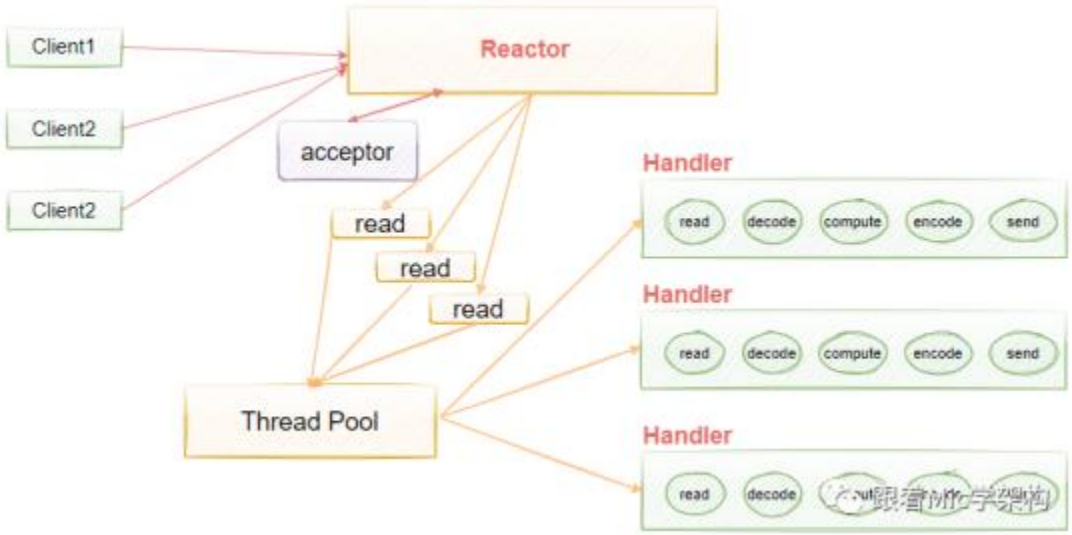
其中 Reactor 线程，负责多路分离套接字，有新连接到来触发 connect 事件之后，交由 Acceptor 进行处理，有 IO 读写事件之后交给 hanlder 处理。

Acceptor 主要任务就是构建 handler ，在获取到和 client 相关的 SocketChannel 之后 ，绑定到相应的 hanlder 上，对应的 SocketChannel 有读写事件之后，基于 racotor 分发,hanlder 就可以处理了（所有的 IO 事件都绑定到 selector 上，有 Reactor 分发）

> **多线程单 Reactor 模型**

单线程 Reactor 这种实现方式有存在着缺点，从实例代码中可以看出，handler 的执行是串行的，如果其中一个 handler 处理线程阻塞将导致其他的业务处理阻塞。由于 handler 和 reactor 在同一个线程中的执行，这也将导致无法接收新的请求。

为了解决这种问题，有人提出使用多线程的方式来处理业务，也就是在业务处理的地方加入线程池异步处理，将 reactor 和 handler 在不同的线程来执行，这就是多线程单 Reactor 模型【插入图片】

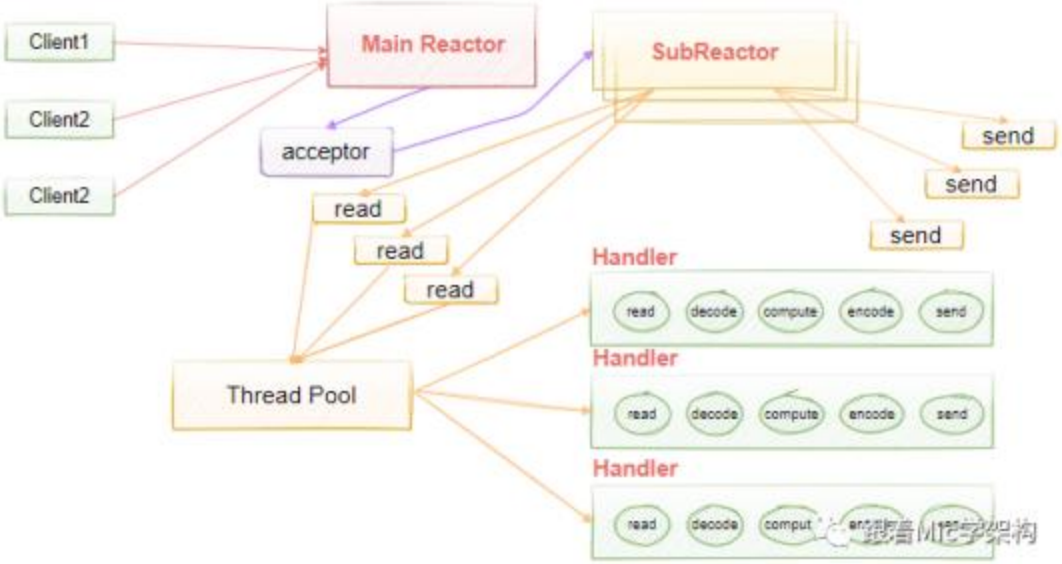


多线程多 Reactor 模型

在多线程单 Reactor 模型中，所有的 I/O 操作是由一个 Reactor 来完成，而 Reactor 运行在单个线程中，它需要处理包括 Accept()/read()/write/connect 操作，对于小容量的场景，影响不大。但是对于高负载、大并发或大数据量的应用场景时，容易成为瓶颈，主要原因如下：

- 一个 NIO 线程同时处理成百上千的链路，性能上无法支撑，即便 NIO 线程的 CPU 负荷达到 100%，也无法满足海量消息的读取和发送；
- 当 NIO 线程负载过重之后，处理速度将变慢，这会导致大量客户端连接超时，超时之后往往会进行重发，这更加重了 NIO 线程的负载，最终会导致大量消息积压和处理超时，成为系统的性能瓶颈；

所以，我们还可以更进一步优化，引入多 Reactor 多线程模式，



- Main Reactor 负责接收客户端的连接请求，然后把接收到的请求传递给 SubReactor (其中 subReactor 可以有多个)，具体的业务 IO 处理由 SubReactor 完成。
- **Acceptor**，请求接收者，在实践时其职责类似服务器，并不真正负责连接请求的建立，而只将其请求委托 Main Reactor 线程池来实现，起到一个转发的作用。
- **Main Reactor**，主 Reactor 线程组，主要**负责连接事件**，并将 **IO 读写请求转发到 SubReactor 线程池。**
- **Sub Reactor**，Main Reactor 通常监听客户端连接后会将通道的读写转发到 Sub Reactor 线程池中一个线程(负载均衡)，负责数据的读写。在 NIO 中 通常注册通道的读(OP_READ)、写事件(OP_WRITE)。

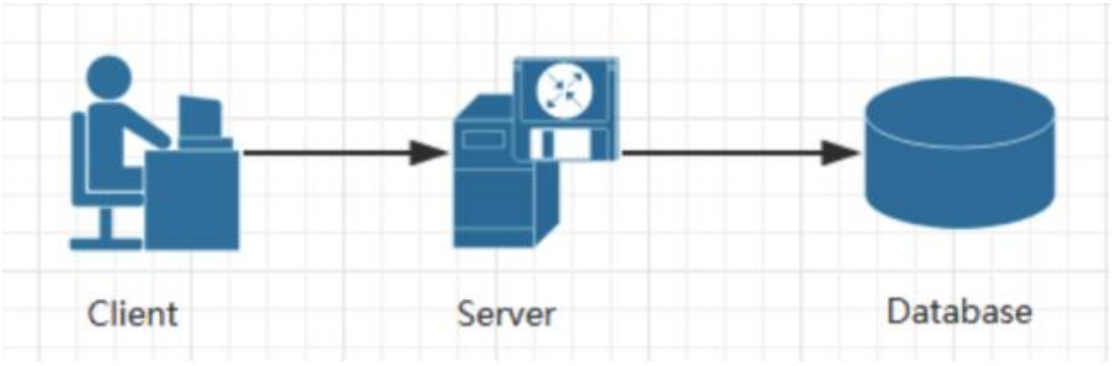
53.42 [携程]什么是负载均衡

关于负载均衡，我会从四个方面去说。

1. 负载均衡产生的背景
2. 负载均衡的实现技术
3. 负载均衡的作用范围
4. 负载均衡的常用算法

53.42.1 负载均衡的诞生背景

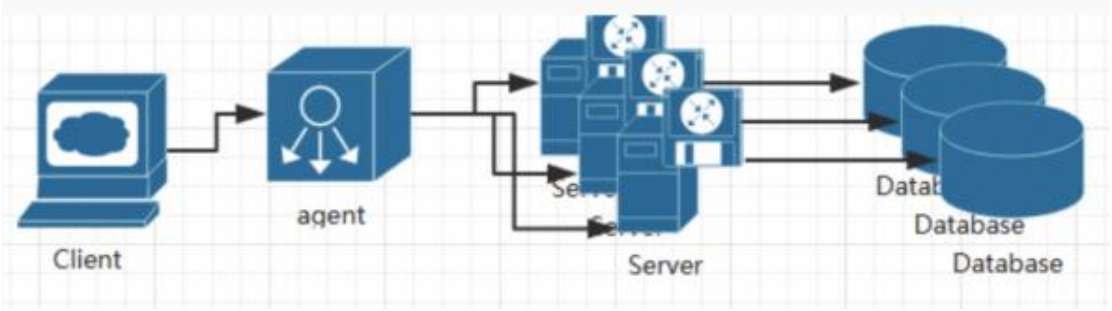
在互联网发展早期，由于用户量较少、业务需求也比较简单。对于软件应用，我们只需要一台高配的服务器即可完成业务的支撑，这样的软件架构称为单体架构**（展示单体架构的图片）**



随着用户量的增加，服务器的请流量也随之增加，在这个过程中单体架构会产生两个问题。

1. 软件的性能逐步下降，访问延迟越来越高
2. 容易出现单点故障

为了解决这个问题，我们引入了集群化部署的架构，也就是把一个软件应用同时部署在多个服务器上



架构的变化带来了两个问题：

1. 客户端请求如何均匀的分发到多台目标服务器上?
2. 如何检测目标服务器的健康状态，使得客户端请求不向已经宕机的服务器发送请求。

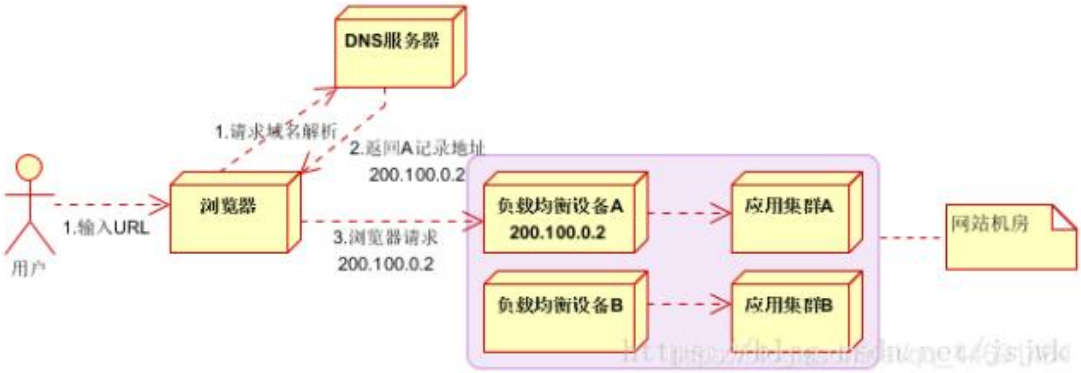
为了解决这两个问题，引入了负载均衡的设计，简单来说，负载均衡机制的核心目的是让客户端的请求合理均匀的分发到多台目标服务器，由于请求被多个节点分发，使得服务端的性能得到有效的提升。

53.42.2 如何实现负载均衡呢?

常见的实现方案有三种！

- 基于 DNS 实现负载均衡
- 基于硬件实现负载均衡
- 基于软件实现负载均衡

先来说一下基于 DNS 实现负载均衡的方式，它的实现方式比较简单，只需要在 DNS 服务器上针对某个域名做多个 IP 映射即可。



它的工作原理是：当用户通过域名访问某个网站时，会先通过 DNS 服务器进行域名解析得到一个 IP 地址，DNS 服务器可以随机分配一个 IP 地址进行访问，这样就可以实现目标服务集群的请求分发。

除此之外，DNS 还可以根据不同的地域分配就近机房的 IP，比如长沙的小伙伴，可能会得到在湖南范围内最近的一个机房的 IP，在这个模式下可以实现「就近原则」实现请求处理，缩短了通信距离从而提升网站访问效率。

DNS 实现负载均衡的优点是： 配置简单，实现成本低，无需额外的开发和维护。

不过缺点也很明显：

由于 DNS 多级缓存的特性，当我们修改 DNS 配置之后，会因为缓存导致 IP 变更不及时，从而影响负载均衡的效果。

第二种，基于硬件实现负载均衡

硬件负载设备，我们可以简单把它理解成一个网络设备，类似于网络交换机，它

3. 它的性能很好，每秒能够处理百万级别的请求，
4. 支持多种负载均衡算法，我们可以非常灵活的配置不同的负载策略
5. 它还具备防火墙等安全功能。
6. 硬件负载是商业产品，有专门的售后来支持，所以企业不需要花精力去做维护。

F5 是比较常见的硬件负载设备，由于硬件负载设备价格比较贵，一般应用在大型银行、政府、电信等领域。

第三种，基于软件实现负载均衡

所谓软件负载，就是通过一些开源软件或者商业软件来完成负载均衡的功能。常见的软件负载技术有：Nginx、LVS、HAProxy 等。

目前互联网企业绝大部分采用的都是软件负载，主要原因是：

1. 免费，企业不需要投入较高的成本。
2. 开源，不同企业对于负载均衡的要求有差异，所以可以基于开源软件上做二次开发。
3. 灵活性较高

这三种方式，没有好坏之分，只有是否合适，因此大家可以根据实际情况选择。

53.42.3 负载均衡的作用范围

负载均衡是作用在网络通信上，来实现请求的分发。

而在网络架构中，基于 OSI 模型，又分为 7 层网络模型

<table>
<tr><th colspan="5">OSI vs TCP/IP</th></tr>
<tr><th>OSI七层网络模型</th><th>TCP/IP四层概念模型</th><th>对应网络协议</th><th>对应的典型设备</th><th>区域</th></tr>
<tr><td>应用层（Application）</td><td rowspan="3">应用层</td><td>TFTP、FTP、NFS、WAIS</td><td>应用程序，如FTP，SMTP，HTTP</td><td rowspan="4">计算机</td></tr>
<tr><td>表示层(Presentation)</td><td>Telnet、Rlogin、SNMP、Gopher</td><td>编码方式，图像编解码、URL字段传输编码</td></tr>
<tr><td>会话层(Session)</td><td>SMTP、DNS</td><td>建立会话，SESSION认证、断点续传</td></tr>
<tr><td>传输层(Transport)</td><td>传输层</td><td>TCP、UDP</td><td>进程和端口</td></tr>
<tr><td>网络层（Network）</td><td>网际层</td><td>IP、ICMP、ARP、RARP、AKP、UUCP</td><td>路由器，防火墙、多层交换机</td><td rowspan="3">网络</td></tr>
<tr><td>数据链路层（Data Link）</td><td rowspan="2">网络接口</td><td>FDDI、Ethernet、Arpanet、PDN、SLIP、PPP</td><td>网卡，网桥，交换机</td></tr>
<tr><td>物理层（Physical）</td><td>IEEE 802.1A、IEEE802.2到IEEE802.11</td><td>中继器，集线器、网线、HUB</td></tr>
</table>

也就是意味着我们可以在网络的某些分层上做请求分发处理，因此根据这样一个特性，对于负载均衡的作用范围又可以分为：

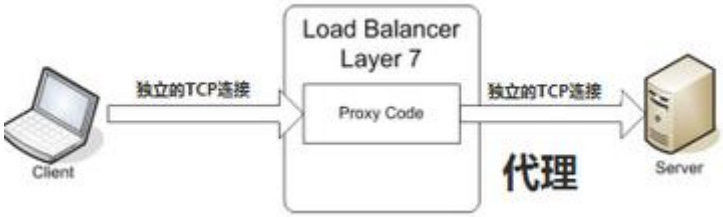
4. 二层负载
5. 三层负载
6. 四层负载
7. 七层负载

二层负载：基于 Mac 地址来实现请求分发，一般采用虚拟 Mac 的方式实现，服务器收到请求后，通过动态分配后端服务的实际 Mac 地址进行响应从而实现负载均衡

三层负载：基于 IP 层负载，一般通过虚拟 IP 的方式实现，外部请求访问虚拟 IP，服务器收到请求后根据后端实际 IP 地址进行转发。

四层负载：通过请求报文中的目标地址和端口进行负载，Nginx、F5、LVS 等都可以实现四层负载。**（展示图片）**

七层负载：七层负载是基于应用层负载，也就是服务器端可以根据 http 协议中请求的报文信息来决定把请求分发到哪个目标服务器上，比如 Cookie、消息体、RequestHeader 等。**（展示图片）**



最后一个，就是负载均衡的常用算法

所谓负载均衡算法，就是决定当前客户端请求匹配到目标服务器集群中的具体哪个节点。

常见的负载均衡算法有：

1. 轮训，也就是多个服务器按照顺序轮训返回，这样每个服务器都能获得相同的请求次数

图1 轮询调度实现逻辑图示

2. 随机，根据随机算法获得一个目标服务地址（就像古时候皇帝翻牌子），由于该算法具备随机性，因此每个服务器获得的请求数量不一定均等。
3. 一致性 hash，也就是对于具有相同 hash 码的请求，永远发送到同一个节点上。
4. 最小连接数，根据目标服务器的请求数量来决定请求分发的权重，也就是目标服务集群中，请求更少的节点将会获得更多的请求。这是负载均衡中比较好的策略，真正能够实现目标服务器的请求均衡。

以上就是关于负载均衡相关的内容，当然，负载均衡还有很多值得去挖掘的，比如负载算法如何实现？网络分层模型的原理等。

好的，本期的视频就到这里结束了，喜欢的朋友记得一键三连，加个关注，我是头发很多的程序员 Mic，咱们下期再见。

53.43 [携程]什么是消息队列?

以下是高手的回答思路

消息队列 Message Queue，简称 MQ。

是一种应用间的通信方式，主要由三个部分组成。

生产者：Producer

消息的产生者与调用端

主要负责消息所承载的业务信息的实例化

是一个队列的发起方

代理：Broker

主要的处理单元

负责消息的存储、投递、及各种队列附加功能的实现

是消息队列最核心的组成部分

消费者：Consumer

一个消息队列的终端

也是消息的调用端

具体是根据消息承载的信息，处理各种业务逻辑。

消息队列的应用场景较多，常用的可以分为三种：

53.43.1 异步处理

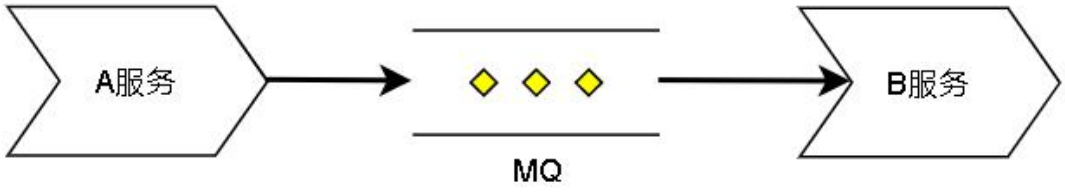
主要应用于对实时性要求不严格的场景，

比如：用户注册发送验证码、下单通知、发送优惠券等等。

服务方只需要把协商好的消息发送到消息队列，

剩下的由消费消息的服务去处理，

不用等待消费服务返回结果。

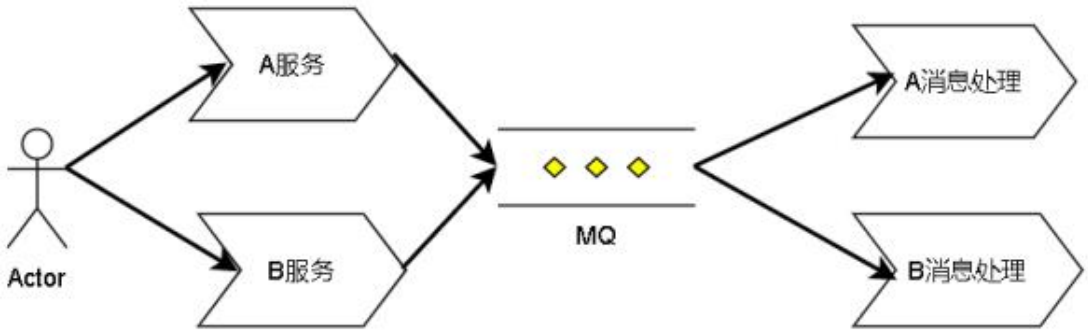


应用解耦

应用解耦可以看作是把相关但耦合度不高的系统联系起来。

比如订单系统与 WMS、EHR 系统，有关联但不哪么紧密，每个系统之间只需要把约定的消息发送到 MQ，另外的系统去消费即可。

解决了各个系统可以采用不同的架构、语言来实现，从而大大增加了系统的灵活性。



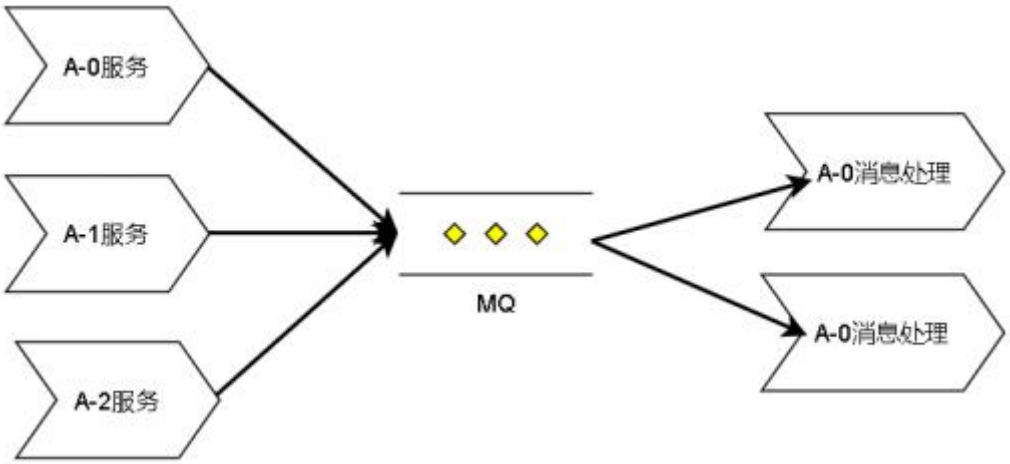
53.43.2 流量削峰

流量削峰一般应用在大流量入口且短时间内业务需求处理不完的服务中心，

为了权衡高可用，把大量的并行任务发送到 MQ 中，

依据 MQ 的存储及分发功能，平稳的处理后续的业务，

起到一个大流量缓冲的作用。



目前市面上常见的消息队列中间件主要有

ActiveMQ、RabbitMQ、Kafka、RocketMQ 这几种，

在架构技术选型的时候一般根据业务的需求选择合适的中间件：

比如中小型公司，低吞吐量的一般用 ActiveMQ、RabbitMQ 较为合适，

大数据高吞吐量的大型公司一般选用 Kafka 和 RocketMQ。

以上就是我的 MQ 的理解。

好的，看完高手的回答后，

相信每位看完视频的小伙伴对消息队列有了更深刻的理解 ，当然本期视频还有很多内容未涉及到，比如中间件产品的介绍、消息队列的实现原理等等，如果你还想听请在下方的评论区留言，我会逐步安排。

本期的菜鸟 VS 高手系列就到这里就结束了，喜欢的朋友一键三连，加个关注，我是头发很多的程序员 Mic，咱们下期见！

53.44 [携程]面试官：你能说一下什么是受检异常和非受检异常吗?

以下是高手的回答思路

我觉得可以从三个方面回答这个问题

1. **一、首先是异常的本质**

受检异常和非受检异常，都是继承自 Throwable 这个类中，分别是 Error 和 Exception，Error 是程序报错，系统收到无法处理的错误消息，它和程序本身无关。

Excetpion 是指程序运行时抛出需要处理的异常信息如果不主动捕获，则会被 jvm 处理。

2. **二、然后是对受检异常和非受检异常的定义**

前面说过受检异常和非受检异常均派生自 Exception 这个类。

1. 受检异常的定义是程序在编译阶段必须要主动捕获的异常，遇到该异常有两种处理方法

通过 try/catch 捕获该异常或者通过 throw 把异常抛出去

2. 非受检异常的定义是程序不需要主动捕获该异常，一般发生在程序运行期间，比如 NullPointException

3. **三、最后我还可以说下他们优点和缺点**

受检异常优点有两个：

- 第一，它可以响应一个明确的错误机制，这些错误在写代码的时候可以随时捕获并且能很好的提高代码的健壮性。
- 第二，在一些连接操作中，它能很好的提醒我们关注异常信息，并做好预防工作。

不过受检异常的缺点是：抛出受检异常的时候需要上声明，而这个做法会直接破坏方法签名导致版本不兼容。这个恶心特性导致我会经常使用 RuntimeException 包装。

非受检异常的好处是可以去掉一些不需要的异常处理代码，而不好之处是开发人员可能忽略某些应该处理的异常，导致带来一些隐藏很深的 Bug，比如流忘记关闭？连接忘记释放等。

这些就是我对这个问题的回答！

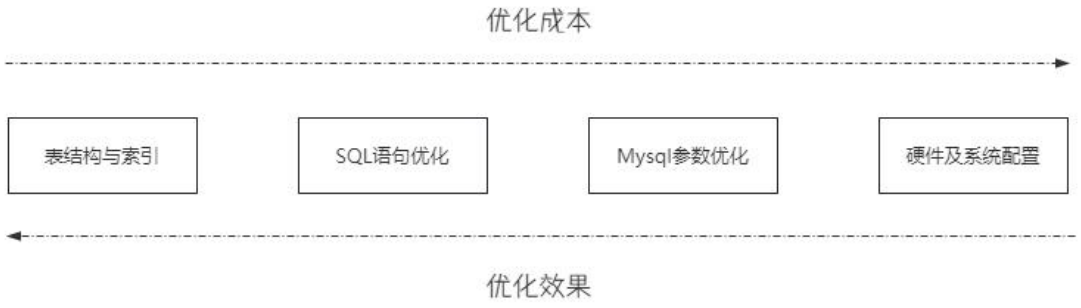
各位观众姥爷，对受检异常和非受检异常还有更完整的答案吗？在评论区你写下你对两种异常处理的回答哟~本期的菜鸟 VS 高手系列就到这里就结束了，喜欢的朋友一键三连，加个关注，咱们下期见！

53.45 [携程]请你说一下 Mysql 中的性能调优方法

Mysql 性能调优方法可以从四个方面来说，分别是

4. 表结构与索引
5. SQL 语句优化
6. Mysql 参数优化
7. 硬件及系统配置

这四个方面的优化成本和优化效果是成反比的。



8. 表结构和索引的优化，主要可以下面这些方面去优化

分库分表、读写分离、为字段选择合适的数据类型、适当的反范式设计，适当冗余设计、为查询操作创建必要的索引但是要避免索引滥用、尽可能使用 Not Null。

9. SQL 语句优化可以从几个方面来做

通过慢查询分析需要优化的 SQL 进行合理优化、利用 explain、profile 等工具分析 SQL 执行计划、避免使用 SELECT *查询。

尽可能使用索引扫描来排序。

10. 参数调优方面，主要可以设置 Buffer_pool 的大小，建议占总内存的 70%左右。设置刷盘策略，平衡好数据安全性和性能的关系等
11. 硬件方面、主要是 CPU 核数、磁盘的读写性能（减小寻道时间、旋转时间、传输时间），可以选择 SSD、网卡、内存等方面。

由于短视频时长限制，我就没办法逐一去展开分析，如果需要 Mysql 相关面试资料，可以在评论区置顶中获取。

另外，大家可以注意我回答问题的结构？这是一种典型的结构化表述方式，这种回答方式能够有效的避免大家表述的过程中逻辑混乱不清晰的问题。

好了，今天的分享就到这里了，感谢大家的关注，我们下期再见。

53.46 [携程]RPC 和 HTTP 协议有什么区别?

我已经不记得有多少个同学跑过来问我：“RPC 和 HTTP 协议的区别”这个问题了。

面试官本来认为这个问题没什么意义，就顺口问了一句，你随便回答到点上就会让你过。

结果你还真回答不出来，真实给足了面试官面子啊~

RPC 全称（Remote Procedure Call），它是一种针对跨进程或者跨网络节点的应用之间的远程过程调用协议。

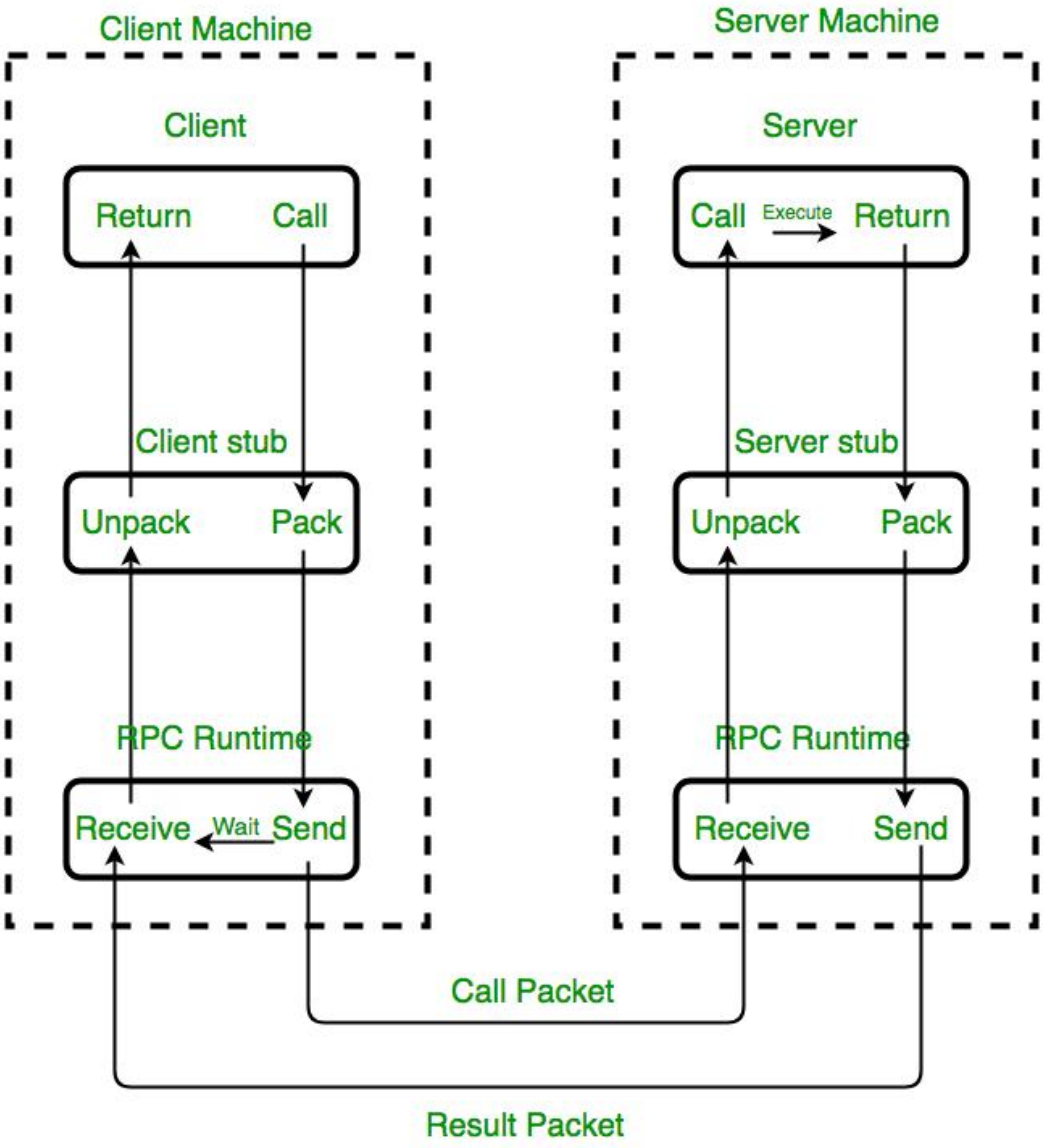
它的核心目标是，让开发人员在进行远程方法调用的时候，就像调用本地方法一样，不需要额外为了完成这个交互做过的编码。

为了达到这样一个目标，就有了这样一个 RPC 协议模型[图]。

Client stub 作为客户端存根，代理客户端请求并把请求参数进行组装，再把组装的参数通过 Pack 也就是序列化。

最后通过远程通信协议把序列化的数据传输到目标服务器上。

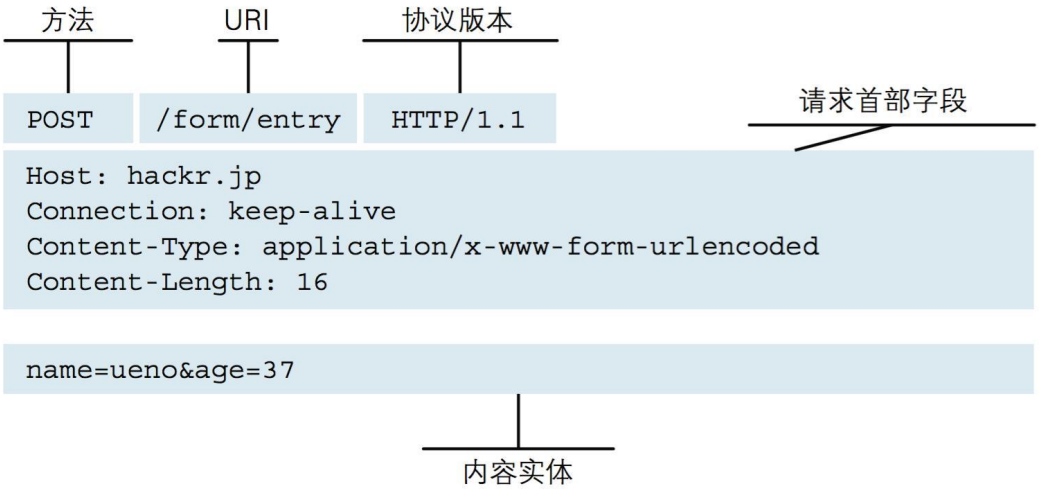
服务端收到数据以后，通过 Unpack 也就是反序列化，再通过 Server stub 服务端存根来调用目标方法获取执行结果。



Implementation of RPC mechanism

所以，通过这样一个模型，就实现了 RPC 的目标，让开发者在没有任何感知的情况下实现了远程方法的调用。

而 Http 协议是为 Web 浏览器与 Web 服务器之间的通信而设计的远程通信协议，它定义了通信协议的报文规范（如图），我们可以使用 http 协议来实现跨网络节点的数据传输。



图：请求报文的构成

基于这样的特点，在 RPC 协议底层的数据传输，即可以直接使用 TCP 协议，也可以使用 http 协议。

因此，Rpc 协议和 Http 协议完全不是同一个纬度的东西，这两者并没有什么可比性。

好了，今天的分享就到这里了，另外，我把 2022 年最热门的 1000 道面试题整理成了面试手册，有需要的小伙伴可以在评论区置顶中领取。

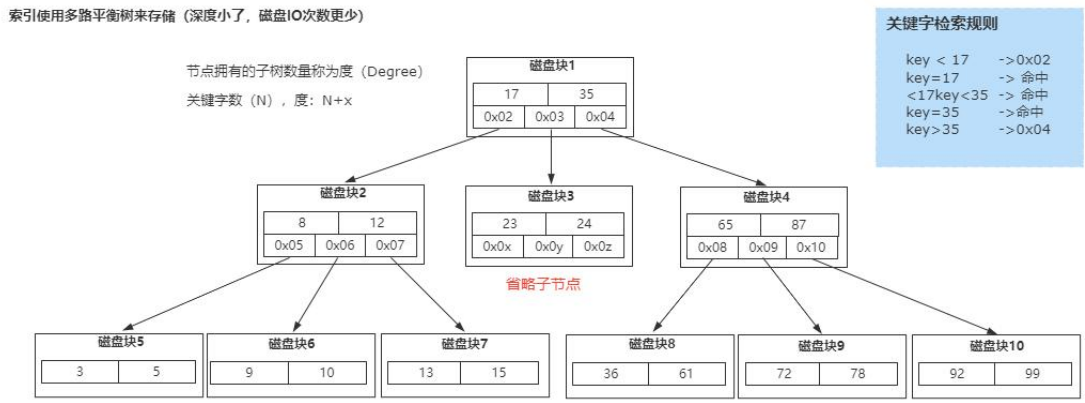
53.47 [携程]为什么索引要用 B+树来实现呢，而不是 B 树?

首先，常规的数据库存储引擎，一般都是采用 B 树或者 B+树来实现索引的存储。

（如图）因为 B 树是一种多路平衡树，用这种存储结构来存储大量数据，它的整个高度会相比二叉树来说，会矮很多。

而对于数据库来说，所有的数据必然都是存储在磁盘上的，而磁盘 IO 的效率实际上是很低的，特别是在随机磁盘 IO 的情况下效率更低。

所以树的高度能够决定磁盘 IO 的次数，磁盘 IO 次数越少，对于性能的提升就越大，这也是为什么采用 B 树作为索引存储结构的原因。

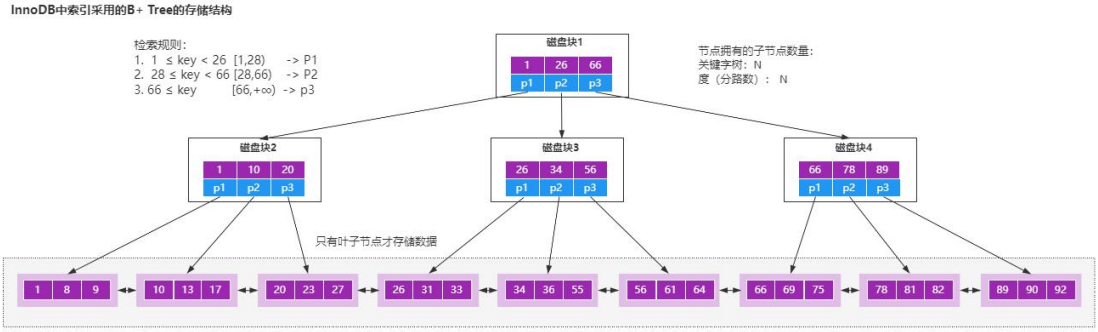


（如图）但是在 Mysql 的 InnoDB 存储引擎里面，它用了一种增强的 B 树结构，也就是 B+树来作为索引和数据的存储结构。

相比较于 B 树结构，B+树做了几个方面的优化。

12. B+树的所有数据都存储在叶子节点，非叶子节点只存储索引。
13. 叶子节点中的数据使用双向链表的方式进行关联。

对了，关于 Mysql 相关的面试题，我给大家准备了 1000 道带标准答案的面试题，只有 10 份，大家可以在评论区置顶中去领取。



使用 B+树来实现索引的原因，我认为有几个方面。

14. B+树非叶子节点不存储数据，所以每一层能够存储的索引数量会增加，意味着 B+树在层高相同的情况下存储的数据量要比 B 树要多，使得磁盘 IO 次数更少。
15. 在 Mysql 里面，范围查询是一个比较常用的操作，而 B+树的所有存储在叶子节点的数据使用了双向链表来关联，所以在查询的时候只需查两个节点进行遍历就行，而 B 树需要获取所有节点，所以 B+树在范围查询上效率更高。
16. 在数据检索方面，由于所有的数据都存储在叶子节点，所以 B+树的 IO 次数会更加稳定一些。
17. 因为叶子节点存储所有数据，所以 B+树的全局扫描能力更强一些，因为它只需要扫描叶子节点。但是 B 树需要遍历整个树。

另外，基于 B+树这样一种结构，如果采用自增的整型数据作为主键，还能更好的避免增加数据的时候，带来叶子节点分裂导致的大量运算的问题。

总的来说，我认为技术方案的选型，更多的是去解决当前场景下的特定问题，并不一定是说 B+树就是最好的选择，就像 MongoDB 里面采用 B 树结构，本质上来说，其实是关系型数据库和非关系型数据库的差异。

好的，今天的分享就到这里了。

53.48 [携程]布隆过滤器到底是什么东西？它有什么用

昨天，一个工作了 6 年的粉丝私聊我，说最近面试被问到布隆过滤器没回答出来。

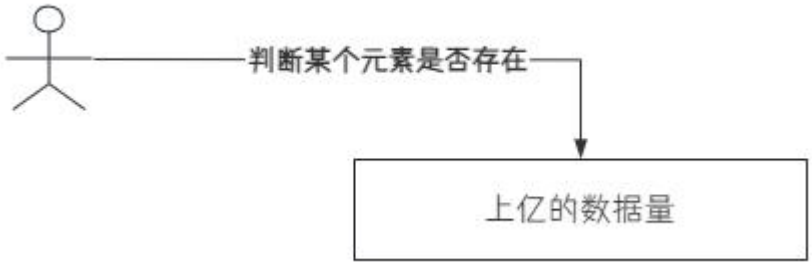
然后在网上找了一堆资料也没有说清楚，想让我帮他讲解一下，[今天正好有空，给大家分享一下布隆过滤器]。

在解释布隆过滤器之前，我先问大家一个问题，如果我想判断一个元素是否存在某个集合里面怎么做?

一般的解决方案是先把所有元素保存起来，然后通过循环比较来确定。

但是如果我们有几千万甚至上亿的数据的时候｛如图｝，虽然可以通过不同的数据结构来优化数据检索的时间复杂度，但是整体的效率依然很慢，

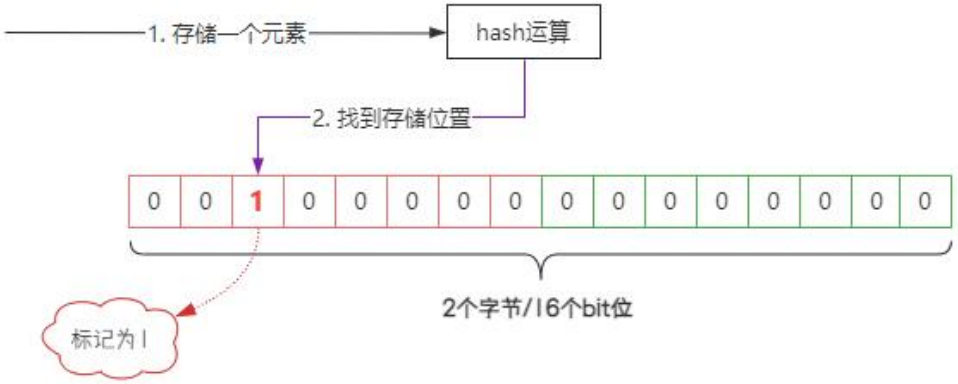
而且会占用非常多的内存空间，这个问题该怎么解决呢?



这个时候，位图就派上了用场｛如图｝

BitMap 的基本原理就是用一个 bit 位来存储当前数据是否存在的状态值，也就是把一个数据通过 hash 运算取模后落在 bit 位组成的数组中，通过 1 对该位置进行标记。

这种方式适用于大规模数据，但数据状态又不是很多的情况，通常是用来判断某个数据存不存在的。



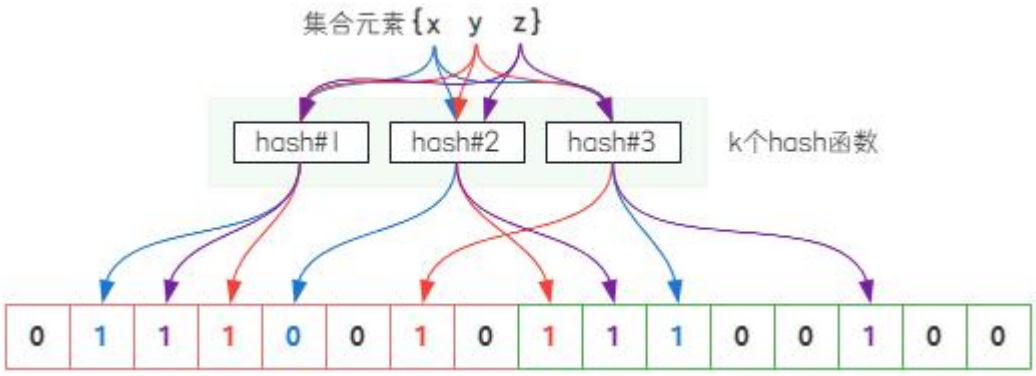
对了，我最近整理了一份 20W 字的面试文档，马上就要金三银四了，趁现在可以多花点时间准备一下，找份好工作。

布隆过滤器就是在位图的基础上做的一个优化设计｛如图｝。

它的原理是，当一个元素被加入集合时，通过 K 个散列函数将这个元素映射成一个位数组中的 K 个点，把它们置为 1。

检索的时候，使用同样的方式去映射，只要看到每个映射的位置的值是不是 1，就可以大概知道该元素是否存在集合中了。

如果这些点有任何一个 0，则被检查的元素一定不在；如果都是 1，则被检查的元素很可能存在。



好了，这就是我今天的分享。

如果你有更多技术问题，欢迎在评论区给我留言，我们下期再见。

53.49 [携程]Mysql 是如何解决幻读问题的?

1、 Mysql 的事务隔离级别

Mysql 有四种事务隔离级别，这四种隔离级别代表当存在多个事务并发冲突时，可能出现的脏读、不可重复读、幻读的问题。

其中 InnoDB 在 RR 的隔离级别下，解决了幻读的问题。

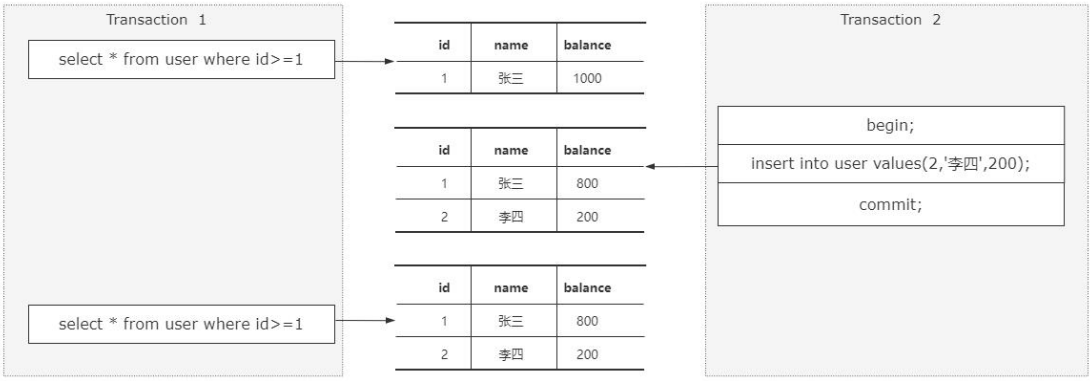
事务隔离级别	脏读	不可重复读	幻读
未提交读（Read Uncommitted）	可能	可能	可能
已提交读（Read Committed）	不可能	可能	可能
可重复读（Repeatable Read）	不可能	不可能	对InnoDB不可能
串行化（Serializable）	不可能	不可能	不可能

2、 什么是幻读?

那么， 什么是幻读呢?

幻读是指在同一个事务中，前后两次查询相同的范围时，得到的结果不一致（我们来看这个图）

- 第一个事务里面我们执行了一个范围查询，这个时候满足条件的数据只有一条
- 第二个事务里面，它插入了一行数据，并且提交了
- 接着第一个事务再去查询的时候，得到的结果比第一查询的结果多出来了一条数据。

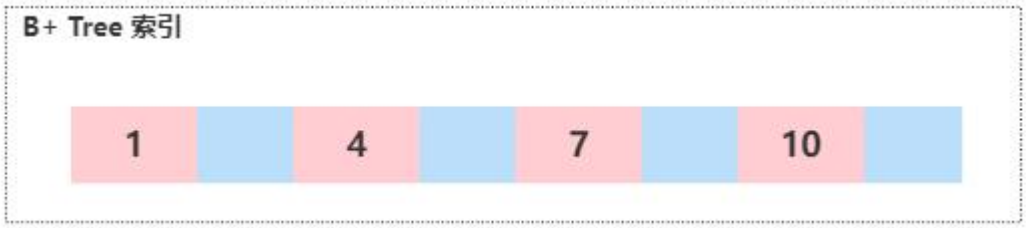


所以，幻读会带来数据一致性问题。

3、 InnoDB 如何解决幻读的问题

InnoDB 引入了间隙锁和 next-key Lock 机制来解决幻读问题，为了更清晰的说明这两种锁，我举一个例子：

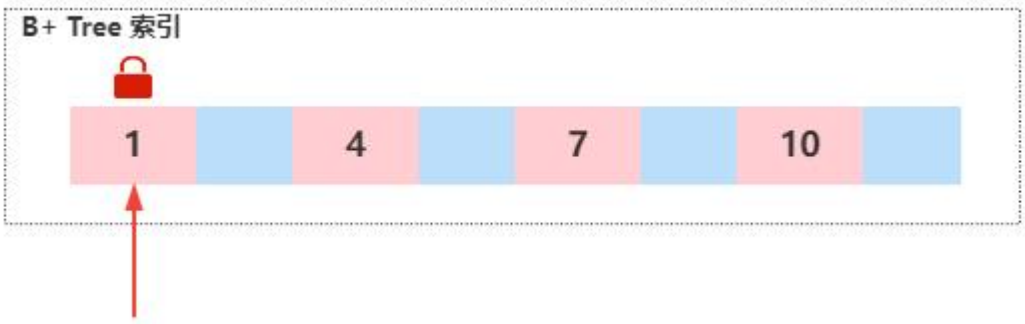
假设现在存在这样（图片）这样一个 B+ Tree 的索引结构，这个结构中有四个索引元素分别是：1、4、7、10。



当我们通过主键索引查询一条记录，并且对这条记录通过 for update 加锁（请看这个图片）

```
SELECT * FROM user WHERE id = 1 FOR UPDATE;
```

这个时候，会产生一个记录锁，也就是行锁，锁定 id=1 这个索引（请看这个图片）。



被锁定的记录在锁释放之前，其他事务无法对这条记录做任何操作。

前面我说过对幻读的定义： 幻读是指在同一个事务中，前后两次查询相同的范围时，得到的结果不一致！

注意，这里强调的是范围查询，

也就是说，InnoDB 引擎要解决幻读问题，必须要保证一个点，就是如果一个事务通过这样一条语句（如图）进行锁定时。

```
SELECT * FROM user WHERE id >4 and id<7 FOR UPDATE;
```

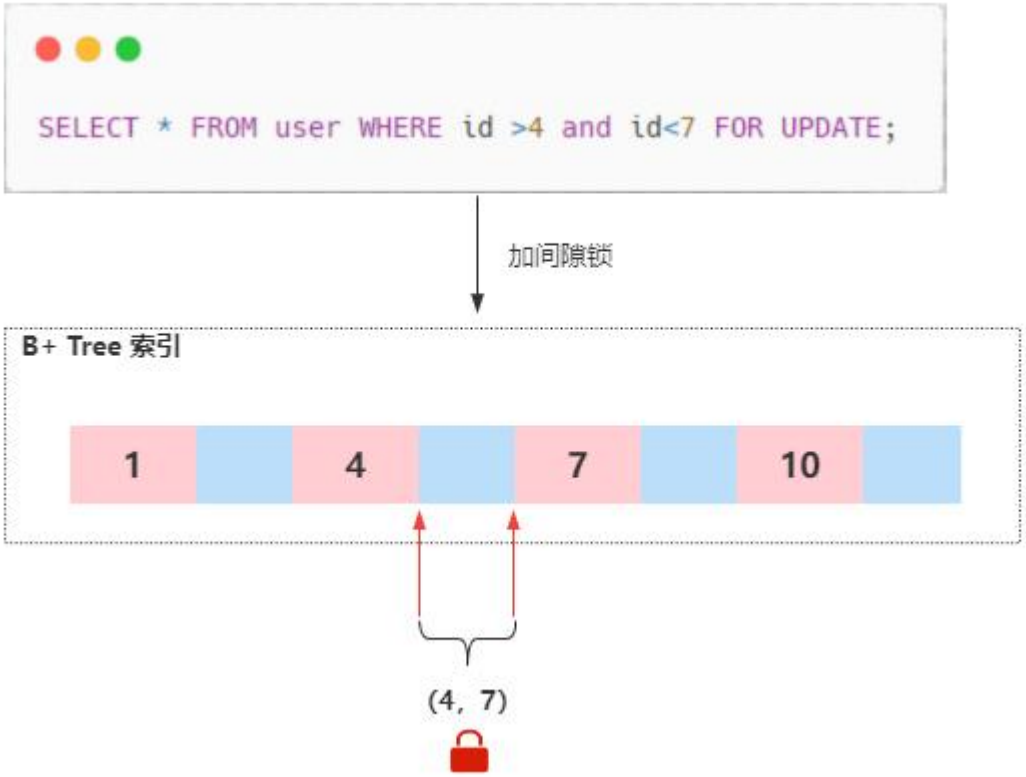
另外一个事务再执行这样一条（显示图片）insert 语句，需要被阻塞，直到前面获得锁的事务释放。

```
INSERT INTO user(id,name) VALUES(5,'mimi');
```

所以，在 InnoDB 中设计了一种间隙锁，它的主要功能是锁定一段范围内的索引记录（如图）

当对查询范围 id>4 and id <7 加锁的时候，会针对 B+树中（4，7）这个开区间范围的索引加间隙锁。

意味着在这种情况下，其他事务对这个区间的数据进行插入、更新、删除都会被锁住。

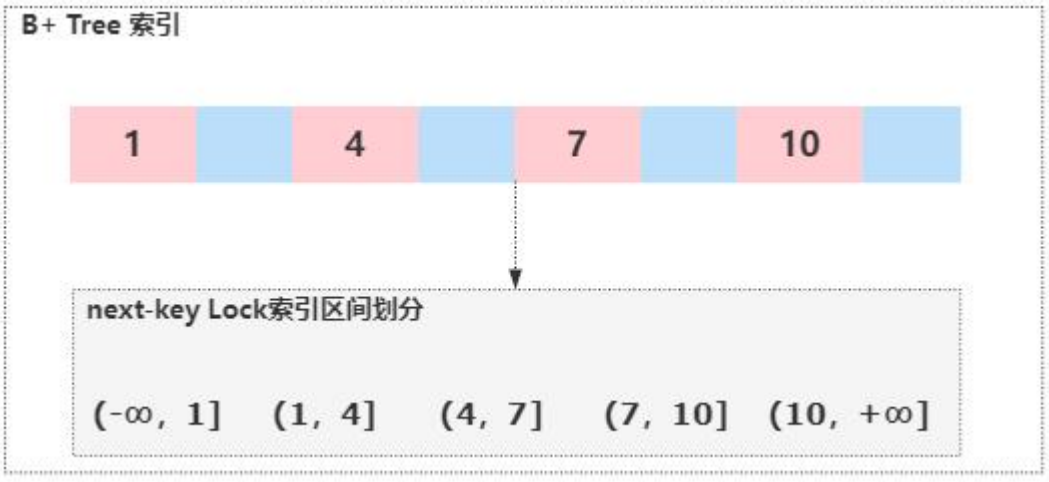


但是，还有另外一种情况，比如像这样（图片）

```
SELECT * FROM user WHERE id>4 FOR UPDATE;
```

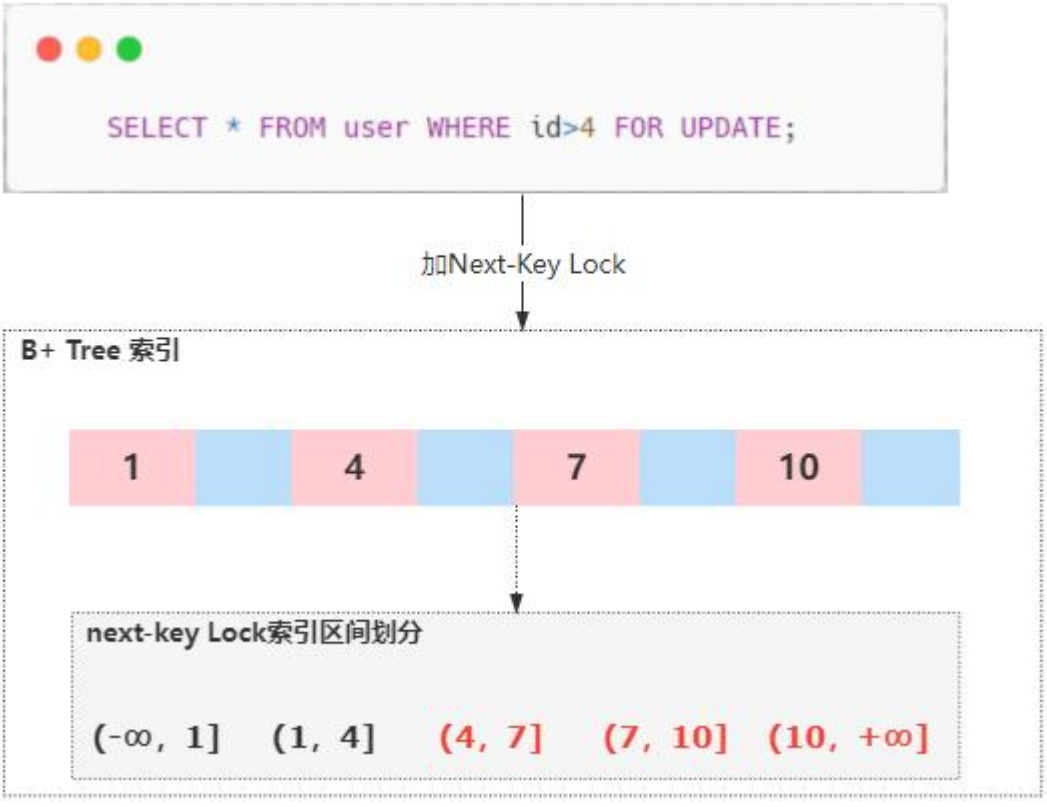
这条查询语句是针对 id>4 这个条件加锁，那么它需要锁定多个索引区间，所以在这种情况下 InnoDB 引入了 next-key Lock 机制。

next-key Lock 相当于间隙锁和记录锁的合集，记录锁锁定存在的记录行，间隙锁锁住记录行之间的间隙，而 next-key Lock 锁住的是两者之和。（如图所示）



每个数据行上的非唯一索引列上都会存在一把 **next-key lock**，当某个事务持有该数据行的 **next-key lock** 时，会锁住一段**左开右闭区间**的数据。

因此，当通过 id>4 这样一种范围查询加锁时，会加 next-key Lock，锁定的区间范围是：(4, 7] , (7,10],(10,+∞]



间隙锁和 next-key Lock 的区别在于加锁的范围，间隙锁只锁定两个索引之间的引用间隙，而 next-key Lock 会锁定多个索引区间，它包含记录锁和间隙锁。

当我们使用了范围查询，不仅仅命中了 Record 记录，还包含了 Gap 间隙，在这种情况下我们使用的就是临键锁，它是 MySQL 里面默认的行锁算法。

4 、总结

虽然 InnoDB 中通过间隙锁的方式解决了幻读问题，但是加锁之后一定会影响到并发性能，因此，如果对性能要求较高的业务场景中，可以把隔离级别设置成 RC，这个级别中不存在间隙锁。

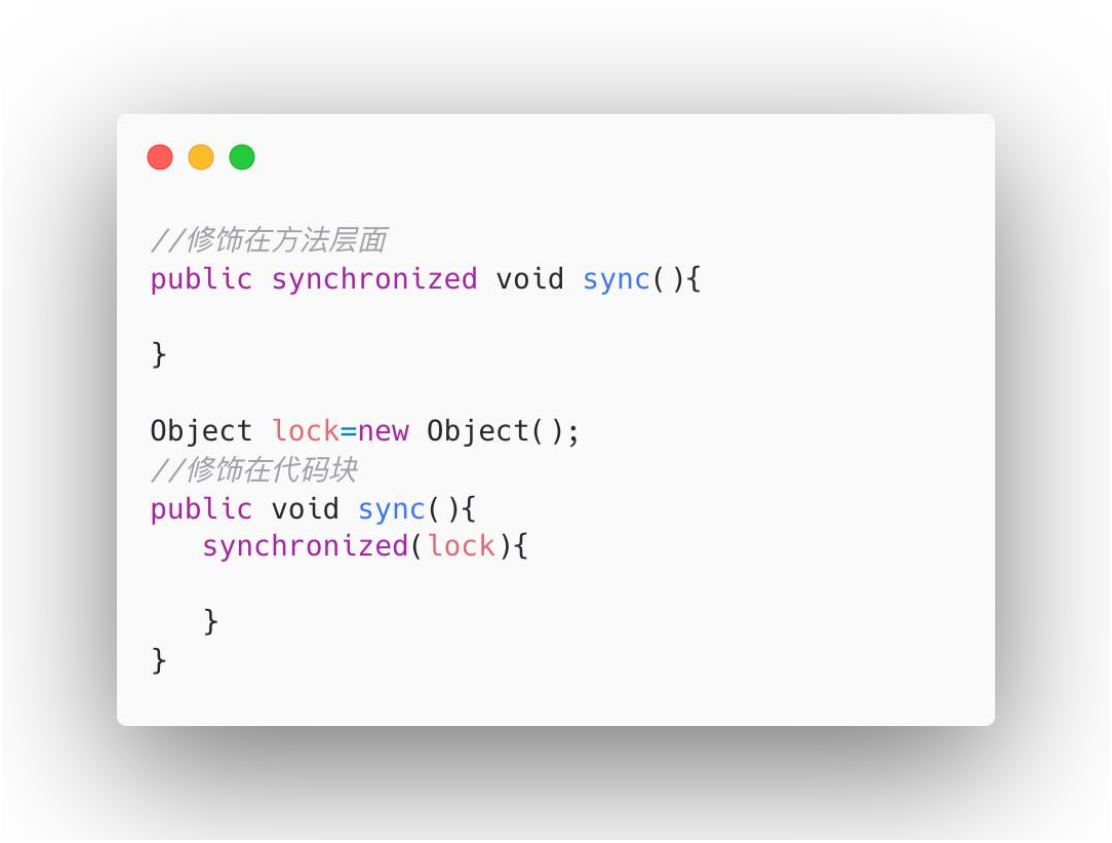
好了，今天的分享就到这里，如果你在面试中遇到了比较奇葩的问题，欢迎在评论区留言。

53.50 [携程]lock 和 Synchronized 区别

昨天一个工作 4 年的粉丝去面试，被问到 lock 和 Synchronized 的区别这个问题，竟然没回答出来。

今天正好有空给大家来分享一下。对于这个问题，其实可以通过结构化的方式来回答。

18. 从功能角度来看，Lock 和 Synchronized 都是 Java 中用来解决线程安全问题的工具。
19. 从特性来看，
    a. Synchronized 是 Java 中的同步关键字，Lock 是 J.U.C 包中提供的接口，这个接口有很多实现类，其中就包括 ReentrantLock 重入锁
    b. Synchronized 可以通过两种方式来控制锁的粒度，（贴图）



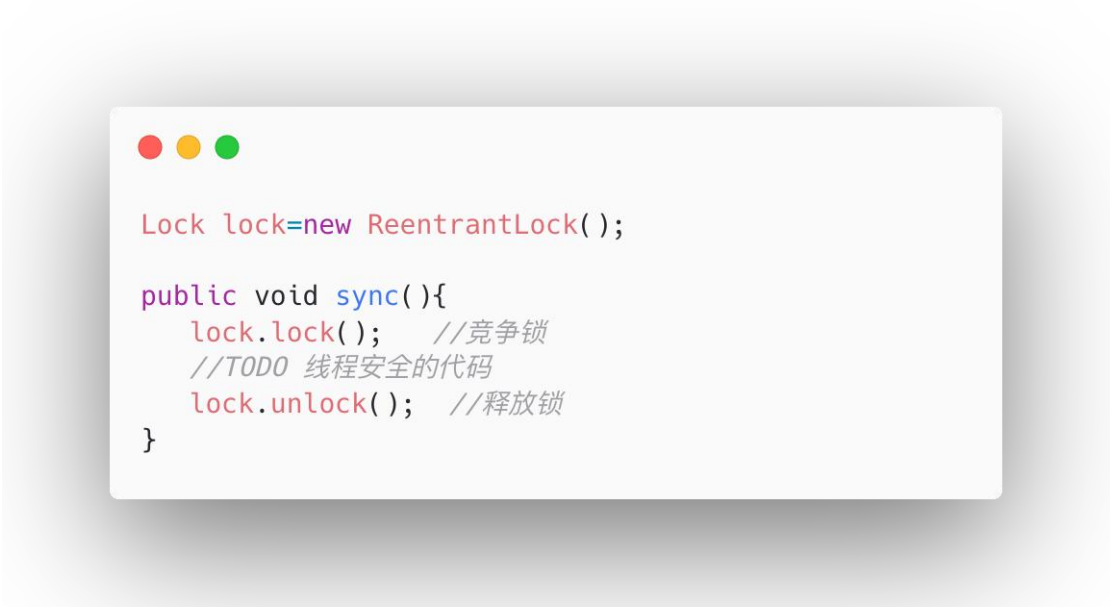
```java
//修饰在方法层面
public synchronized void sync(){

}

Object lock=new Object();
//修饰在代码块
public void sync(){
    synchronized(lock){

    }
}
```

一种是把 synchronized 关键字修饰在方法层面，

另一种是修饰在代码块上，并且我们可以通过 Synchronized 加锁对象的声明周期来控制锁的作用范围，比如锁对象是静态对象或者类对象，那么这个锁就是全局锁。

如果锁对象是普通实例对象，那这个锁的范围取决于这个实例的声明周期。

Lock 锁的粒度是通过它里面提供的 lock()和 unlock()方法决定的（贴图），包裹在这两个方法之间的代码能够保证线程安全性。而锁的作用域取决于 Lock 实例的生命周期。



```
Lock lock=new ReentrantLock();

public void sync(){
    lock.lock();    //竞争锁
    //TODO 线程安全的代码
    lock.unlock();  //释放锁
}
```

c. Lock 比 Synchronized 的灵活性更高，Lock 可以自主决定什么时候加锁，什么时候释放锁，只需要调用 lock()和 unlock()这两个方法就行，同时 Lock 还提供了非阻塞的竞争锁方法 tryLock()方法，这个方法通过返回 true/false 来告诉当前线程是否已经有其他线程正在使用锁。

Synchronized 由于是关键字，所以它无法实现非阻塞竞争锁的方法，另外，Synchronized 锁的释放是被动的，就是当 Synchronized 同步代码块执行完以后或者代码出现异常时才会释放。

d. Lock 提供了公平锁和非公平锁的机制，公平锁是指线程竞争锁资源时，如果已经有其他线程正在排队等待锁释放，那么当前竞争锁资源的线程无法插队。而非公平锁，就是不管是否有线程在排队等待锁，它都会尝试去竞争一次锁。Synchronized 只提供了一种非公平锁的实现。

20. 从性能方面来看，Synchronized 和 Lock 在性能方面相差不大，在实现上会有一些区别，Synchronized 引入了偏向锁、轻量级锁、重量级锁以及锁升级的方式来优化加锁的性能，而 Lock 中则用到了自旋锁的方式来实现性能优化。

好了，今天的分享就到这里了。

53.51 [携程]Dubbo 的服务请求失败怎么处理?

昨天一个工作了 8 年的粉丝去阿里面试，被问到：Dubbo 的服务请求失败的处理方式，没有回答上来没通过面试。

今天正好有空，给大家分享一下这个问题的回答。

Dubbo 是一个 RPC 框架，它为我们的应用提供了远程通信能力的封装，同时，Dubbo 在 RPC 通信的基础上，逐步在向一个生态在演进，它涵盖了服务注册、动态路由、容错、服务降级、负载均衡等能力，基本上在微服务架构下面临的问题，Dubbo 都可以解决。

而对于 Dubbo 服务请求失败的场景，默认提供了重试的容错机制，也就是说，如果基于 Dubbo 进行服务间通信出现异常，服务消费者会对服务提供者集群中其他的节点发起重试，确保这次请求成功，默认的额外重试次数是 2 次。

除此之外，Dubbo 还提供了更多的容错策略，我们可以根据不同的业务场景来进行选择。

21. 快速失败策略，服务消费者只发起一次请求，如果请求失败，就直接把错误抛出去。这种比较适合在非幂等性场景中使用
22. 失败安全策略，如果出现服务通信异常，直接把这个异常吞掉不做任何处理
23. 失败自动恢复策略，后台记录失败请求，然后通过定时任务来对这个失败的请求进行重发。
24. 并行调用多个服务策略，就是把这个消息广播给服务提供者集群，只要有任何一个节点返回，就表示请求执行成功。
25. 广播调用策略，逐个调用服务提供者集群，只要集群中任何一个节点出现异常，就表示本次请求失败

要注意的是，默认基于重试策略的容错机制中，需要注意幂等性的处理，否则在事务型的操作中，容易出现多次数据变更的问题。

53.52 [携程]Redis 和 Mysql 如何保证数据一致性

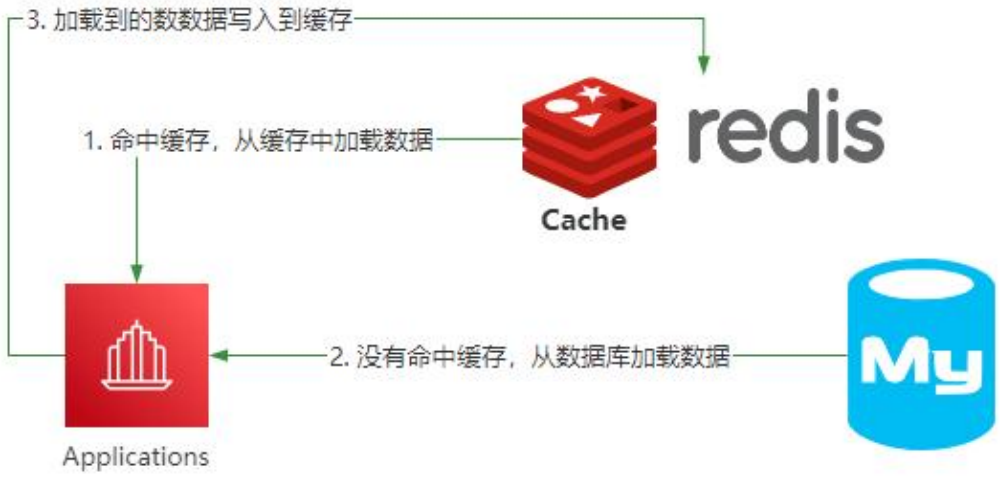
Redis 和 Mysql 的数据一致性问题，几乎每个公司面试都会问，但还是有很多小伙伴回答不上来

今天正好有空，给大家分享一下这个问题的回答。

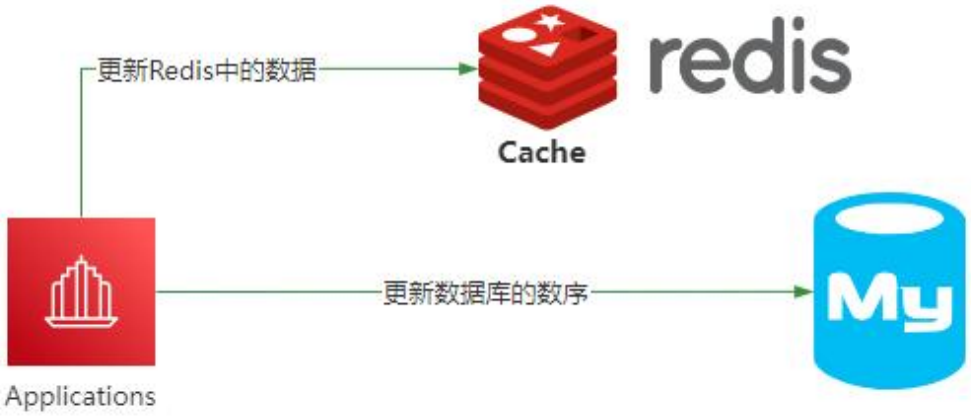
一般情况下，Redis 用来实现应用和数据库之间读操作的缓存层，主要目的是减少数据库 IO，还可以提升数据的 IO 性能。

这是它的整体架构。

当应用程序需要去读取某个数据的时候，首先会先尝试去 Redis 里面加载，如果命中就直接返回。如果没有命中，就从数据库查询，查询到数据后再把这个数据缓存到 Redis 里面。



（如图）在这样一个架构中，会出现一个问题，就是一份数据，同时保存在数据库和 Redis 里面，当数据发生变化的时候，需要同时更新 Redis 和 Mysql，由于更新是有先后顺序的，并且它不像 Mysql 中的多表事务操作，可以满足 ACID 特性。所以就会出现数据一致性问题。

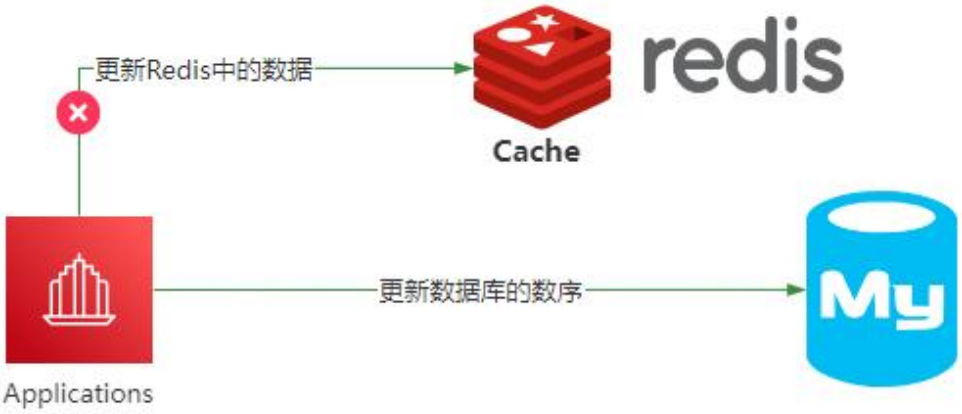


在这种情况下，能够选择的方法只有几种。

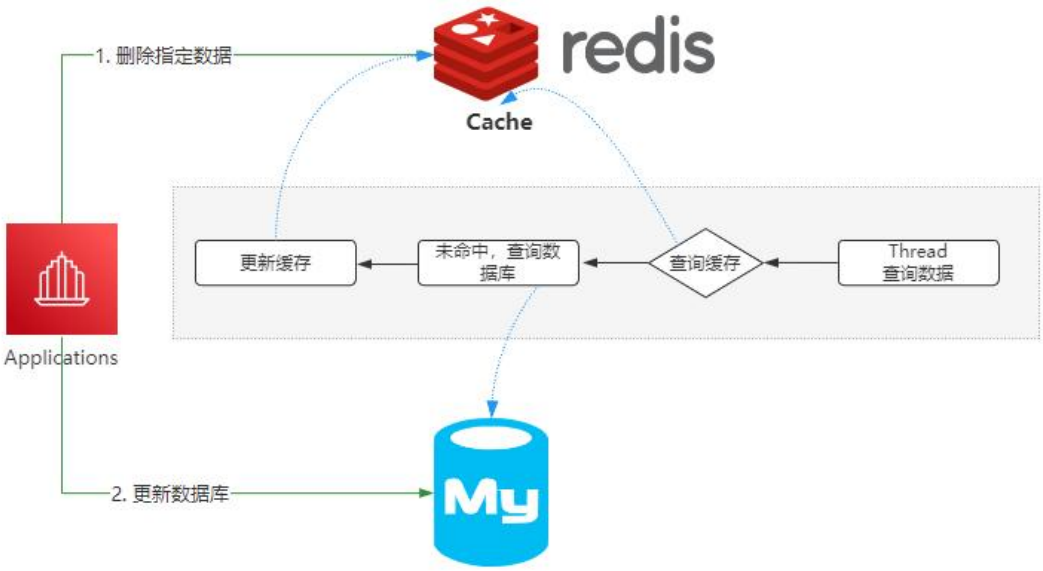
26. 先更新数据库，再更新缓存

27. 先删除缓存，再更新数据库

如果先更新数据库，再更新缓存，如果缓存更新失败，就会导致数据库和 Redis 中的数据不一致。

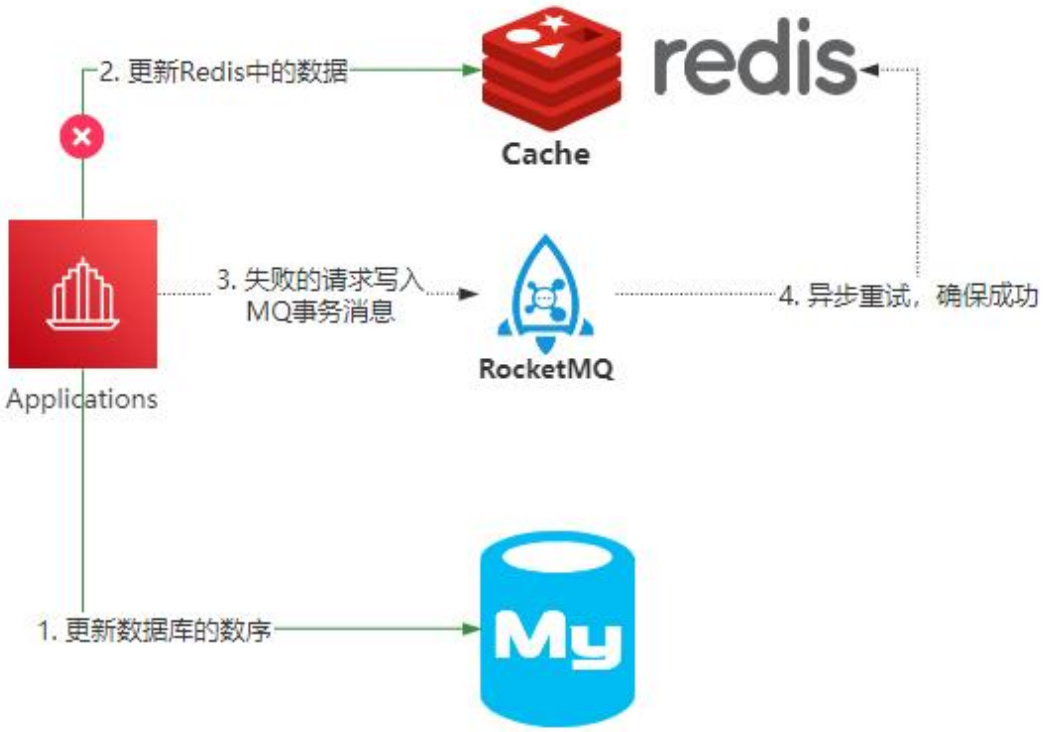


如果是先删除缓存，再更新数据库，理想情况是应用下次访问 Redis 的时候，发现 Redis 里面的数据是空的，就从数据库加载保存到 Redis 里面，那么数据是一致的。但是在极端情况下，由于删除 Redis 和更新数据库这两个操作并不是原子的，所以这个过程如果有其他线程来访问，还是会存在数据不一致问题。

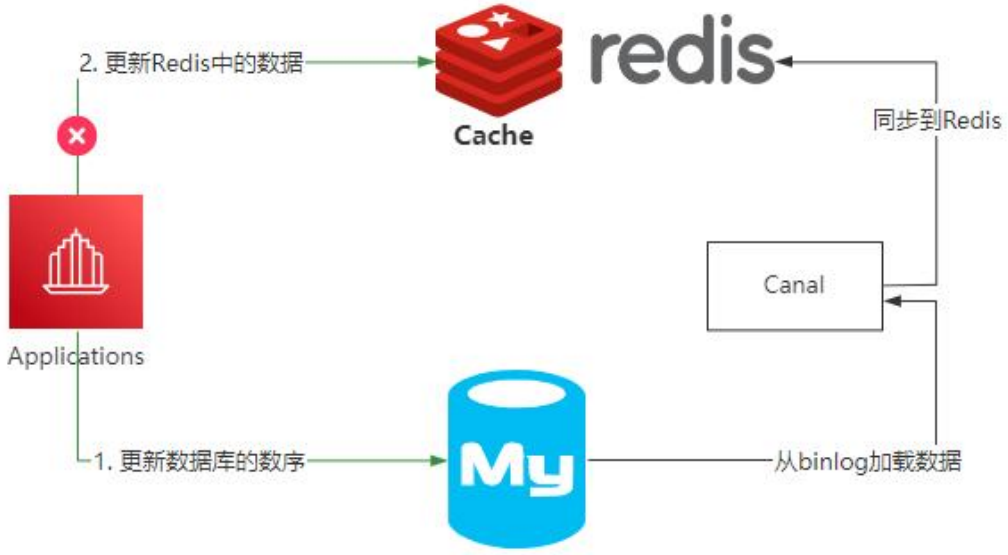


所以，如果需要在极端情况下仍然保证 Redis 和 Mysql 的数据一致性，就只能采用最终一致性方案。

（如图）比如基于 RocketMQ 的可靠性消息通信，来实现最终一致性。



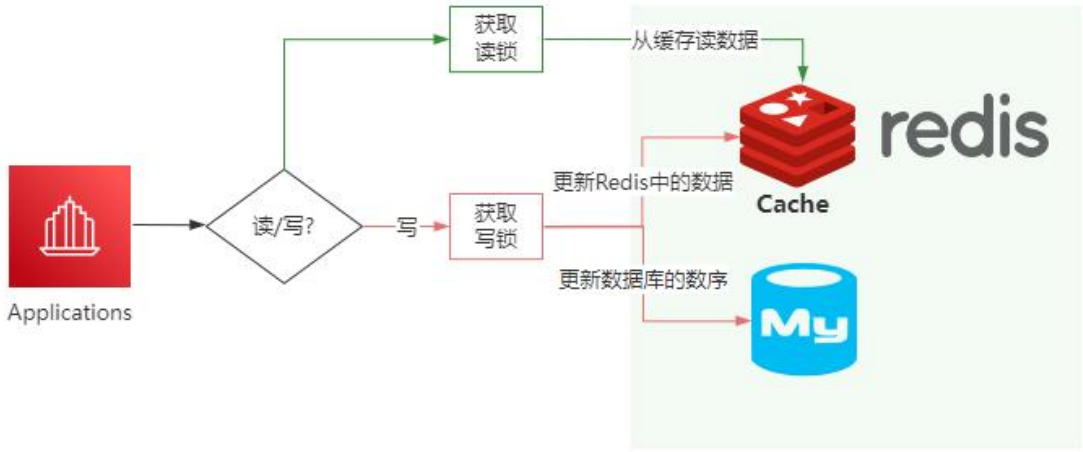
（如图）还可以直接通过 Canal 组件，监控 Mysql 中 binlog 的日志，把更新后的数据同步到 Redis 里面。



因为这里是基于最终一致性来实现的，如果业务场景不能接受数据的短期不一致性，那我们可以通过读写锁的方式来保证强一致性（如图）

在数据更新的时候，其他任何请求都无法访问缓存中的数据，直到数据更新完毕，从而保证了数据的强一致性

但是这种方式由于增加了锁的操作，所以在性能上会存在一定的影响。



好了，今天的分享就到这里。

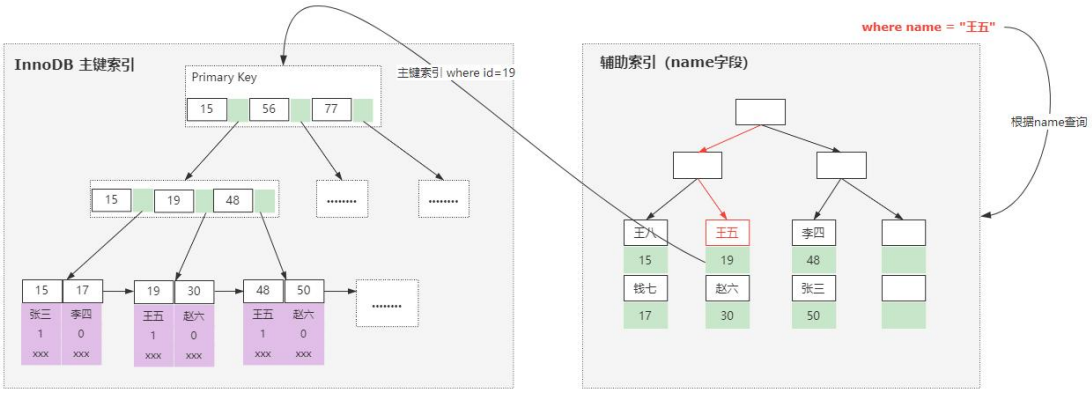
53.53 [携程]什么是聚集索引和非聚集索引

一个去阿里面试并且第一面就挂了的粉丝私信我，被数据库里面几个问题难倒了，他说面试官问了事务隔离级别、MVCC、聚集索引/非聚集索引、B 树、B+树这些，没回答好。

大厂面试基本上是这样，由点到面去展开，如果你对这个技术理解不够全面，很容易就会被看出来。

28. 简单来说，聚集索引就是基于主键创建的索引，除了主键索引以外的其他索引，称为非聚集索引，也叫做二级索引。
29. 由于在 InnoDB 引擎里面，一张表的数据对应的物理文件本身就是按照 B+树来组织的一种索引结构，而聚集索引就是按照每张表的主键来构建一颗 B+树，然后叶子节点里面存储了这个表的每一行数据记录。
30. 所以基于 InnoDB 这样的特性，聚集索引并不仅仅是一种索引类型，还代表着一种数据的存储方式。
31. 同时也意味着每个表里面必须要有一个主键，如果没有主键，InnoDB 会默认选择或者添加一个隐藏列作为主键索引来存储这个表的数据行。一般情况是建议使用自增 id 作为主键，这样的话 id 本身具有连续性使得对应的数据也会按照顺序存储在磁盘上，写入性能和检索性能都很高。否则，如果使用 uuid 这种随机 id，那么在频繁插入数据的时候，就会导致随机磁盘 IO，从而导致性能较低。

32. 需要注意的是，InnoDB 里面只能存在一个聚集索引，原因很简单，如果存在多个聚集索引，那么意味着这个表里面的数据存在多个副本，造成磁盘空间的浪费，以及数据维护的困难。
33. （如图）由于在 InnoDB 里面，主键索引表示的是一种数据存储结构，所以如果是基于非聚集索引来查询一条完整的记录，最终还是需要访问主键索引来检索。



53.54 [携程]说一下你对双亲委派的理解

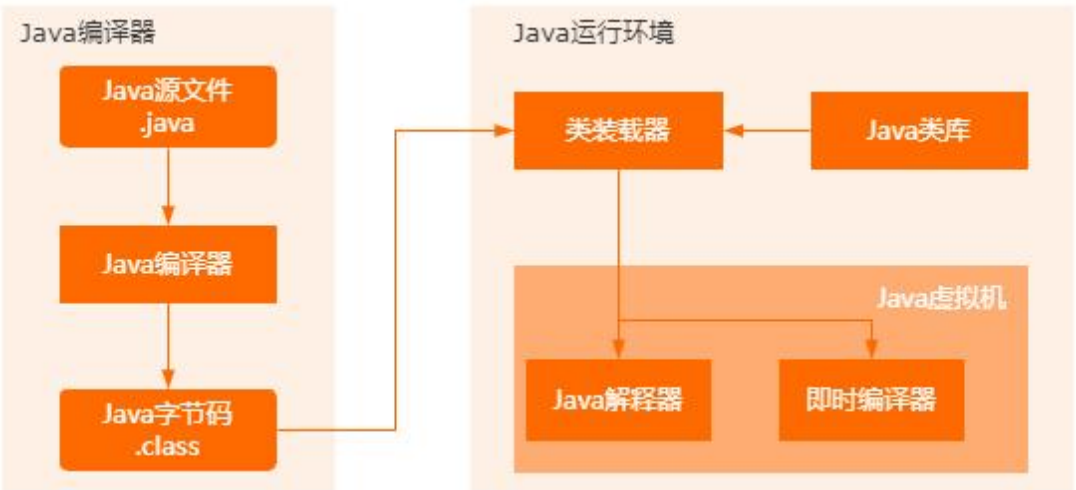
昨天一个工作了 3 年的粉丝，去蚂蚁金服面试，遇到双亲委派的问题没回答出来。

今天有时间，给大家分享一下这个问题。

首先，我简单说一下类的加载机制（如图），就是我们自己写的 java 源文件到最终运行，必须要经过编译和类加载两个阶段。

编译的过程就是把.java 文件编译成.class 文件。

类加载的过程，就是把 class 文件装载到 JVM 内存中，装载完成以后就会得到一个 Class 对象，我们就可以使用 new 关键字来实例化这个对象。



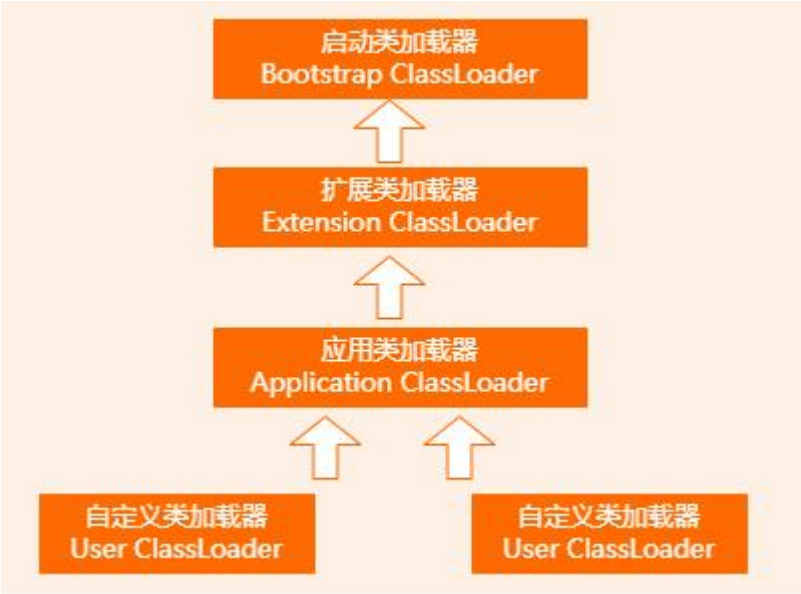
（如图）而类的加载过程，需要涉及到类加载器。

JVM 在运行的时候，会产生 3 个类加载器，这三个类加载器组成了一个层级关系

每个类加载器分别去加载不同作用范围的 jar 包，比如

- Bootstrap ClassLoader，主要是负责 Java 核心类库的加载，也就是 %{JDK_HOME}\lib 下的 rt.jar、resources.jar 等
- Extension ClassLoader，主要负责%{JDK_HOME}\lib\ext 目录下的 jar 包和 class 文件
- Application ClassLoader，主要负责当前应用里面的 classpath 下的所有 jar 包和类文件

除了系统自己提供的类加载器以外，还可以通过 ClassLoader 类实现自定义加载器，去满足一些特殊场景的需求。



（如图）所谓的父委托模型，就是按照类加载器的层级关系，逐层进行委派。

比如当需要加载一个 class 文件的时候，首先会把这个 class 的查询和加载委派给父加载器去执行，如果父加载器都无法加载，再尝试自己来加载这个 class。

这样设计的好处，我认为有几个。

34. 安全性，因为这种层级关系实际上代表的是一种优先级，也就是所有的类的加载，优先给 Bootstrap ClassLoader。那对于核心类库中的类，就没办法去破坏，比如自己写一个 java.lang.String，最终还是会交给启动类加载器。再加上每个类加载器的作用范围，那么自己写的 java.lang.String 就没办法去覆盖类库中类。
35. 我认为这种层级关系的设计，可以避免重复加载导致程序混乱的问题，因为如果父加载器已经加载过了，那么子类就没必要去加载了。

好了，今天的分享就到这里。

53.55 [携程]讲一下 wait 和 notify 这个为什么要在 synchronized 代码块中?

昨天，一个工作了 6 年的粉丝，去蚂蚁金服面试，第一面就挂了。

我顺便问了一下是哪个问题没回答出来，他说基本上都没回答上来，我让他提供一下面试题。

然后发现其中一道题目还挺有意思的，今天正好有空，给大家做一个分享。

对了，如果大家准备去面试，我之前整理过一个 20W 字的面试文档，基本上涵盖了市面上 99%的面试题，里面还有各种

场景问题的回答，大家可以在评论区置顶中去领取。

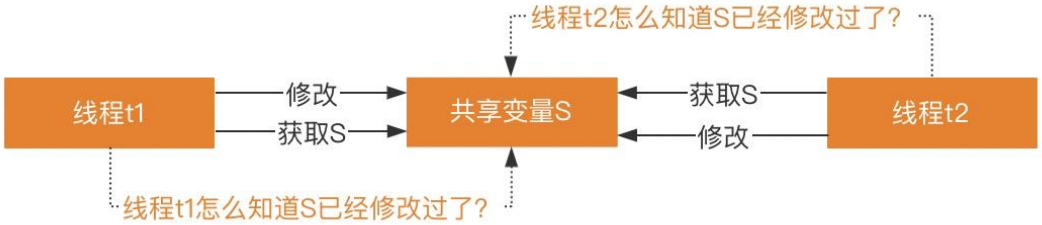
ok，下面我来回答一下这个问题：

36. wait 和 notify 用来实现多线程之间的协调，wait 表示让线程进入到阻塞状态，notify 表示让阻塞的线程唤醒。
37. wait 和 notify 必然是成对出现的，如果一个线程被 wait()方法阻塞，那么必然需要另外一个线程通过 notify()方法来唤醒这个被阻塞的线程，从而实现多线程之间的通信。
38. （如图）在多线程里面，要实现多个线程之间的通信，除了管道流以外，只能通过共享变量的方法来实现，也就是线程 t1 修改共享变量 s，线程 t2 获取修改后的共享变量 s，从而完成数据通信。

但是多线程本身具有并行执行的特性，也就是在同一时刻，多个线程可以同时执行。在这种情况下，线程 t2 在访问共享变量 s 之前，必须要知道线程 t1 已经修改过了共享变量 s，否则就需要等待。

同时，线程 t1 修改过了共享变量 S 之后，还需要通知在等待中的线程 t2。

所以要在这种特性下要去实现线程之间的通信，就必须要有一个竞争条件控制线程在什么条件下等待，什么条件下唤醒。



39. 而 Synchronized 同步关键字就可以实现这样一个互斥条件，也就是在通过共享变量来实现多个线程通信的场景里面，参与通信的线程必须要竞争到这个共享变量的锁资源，才有资格对共享变量做修改，修改完成后就释放锁，那么其他的线程就可以再次来竞争同一个共享变量的锁来获取修改后的数据，从而完成线程之前的通信。
40. 所以这也是为什么 wait/notify 需要放在 Synchronized 同步代码块中的原因，有了 Synchronized 同步锁，就可以实现对多个通信线程之间的互斥，实现条件等待和条件唤醒。
41. 另外，为了避免 wait/notify 的错误使用，jdk 强制要求把 wait/notify 写在同步代码块里面，否则会抛出 IllegalMonitorStateException

42. 最后，基于 wait/notify 的特性，非常适合实现生产者消费者的模型，比如说用 wait/notify 来实现连接池就绪前的等待与就绪后的唤醒。

53.56 [携程]从 B 站崩溃的故障排查和恢复过程中学到什么?

2021 年 7 月 13 日晚上 22：52，B 站崩了。整个事件的罪魁祸首竟然只是，这么短短的几行代码；（手势）

```
17  local _gcd
18  _gcd = function (a, b)
19      if b == 0 then
20          return a
21      end
22
23      return _gcd(b, a % b)
24  end
25
```

回到 B 站崩溃那天，仅不到半个小时，这个消息就冲上了微博的热搜头条。

从 B 站出来的用户甚至带崩了 A 站、豆瓣、知乎等多个网站。

有网友调侃说，B 站服务器一蹦，紧张加班的除了 B 站的程序员，同时让 A 站知乎、豆瓣、微博的程序员默默打开了电脑。

B 站崩溃的背后，背后的根本原因到底是什么呢?

时隔一年以后，B 站对外公布了导致这个问题的底层原因，实际上这个问题并不复杂，

> 了解 B 栈的公网架构

但是为了让大家彻底搞懂这个问题，我们先来了解一下 B 站的公网架构（如图）。

- CDN 是内容分发网络的一个简称，它提供了地域的就近访问功能，为用户获取服务器信息提供了加速的功能。

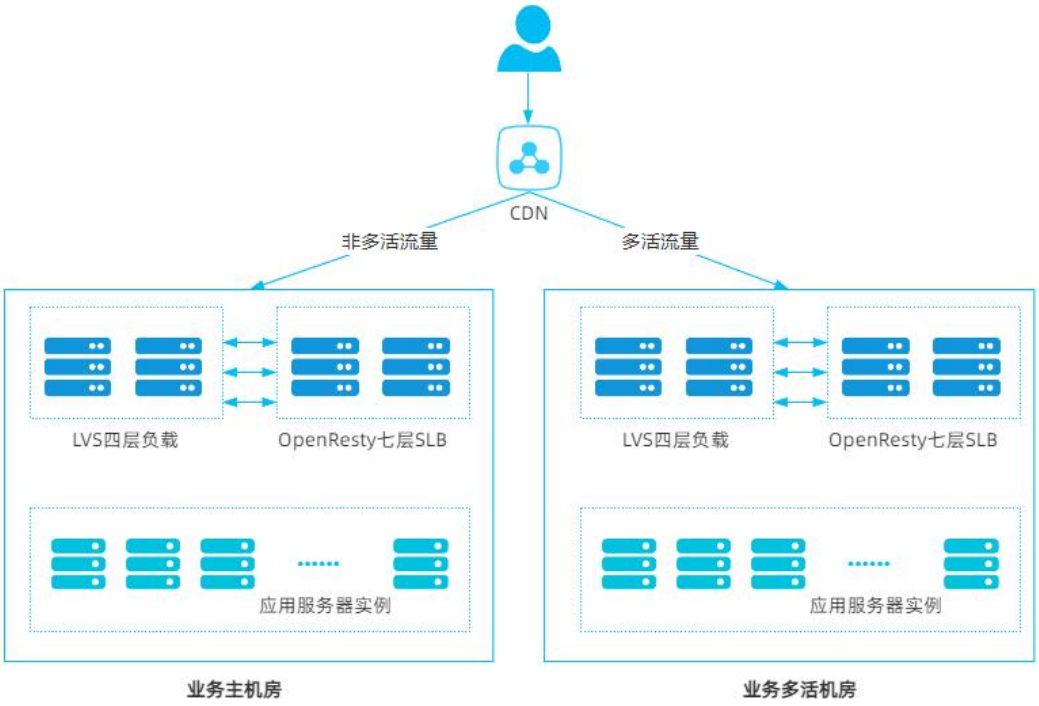
- lvs 是一个四层负载均衡器，也就是基于 ip+端口的负载，为 OpenResty 提供高可用集群。
- OpenResty，是一个基于 Nginx+Lua 的高性能 Web 平台，简单来说，

就是我们可以直接使用 Lua 脚本来扩展 Nginx 的能力，从而构建动态的 Web 应用。

- 最后，使用了多机房部署实现异地多活的架构保证高可用性

当用户发起请求后，经过 CDN 分发到业务主机房，然后通过 LVS 四层负载路由到 OpenResty 服务器上，

最后再转发到对应的应用服务器实例获取相关数据并返回。



> 故障解决过程分析

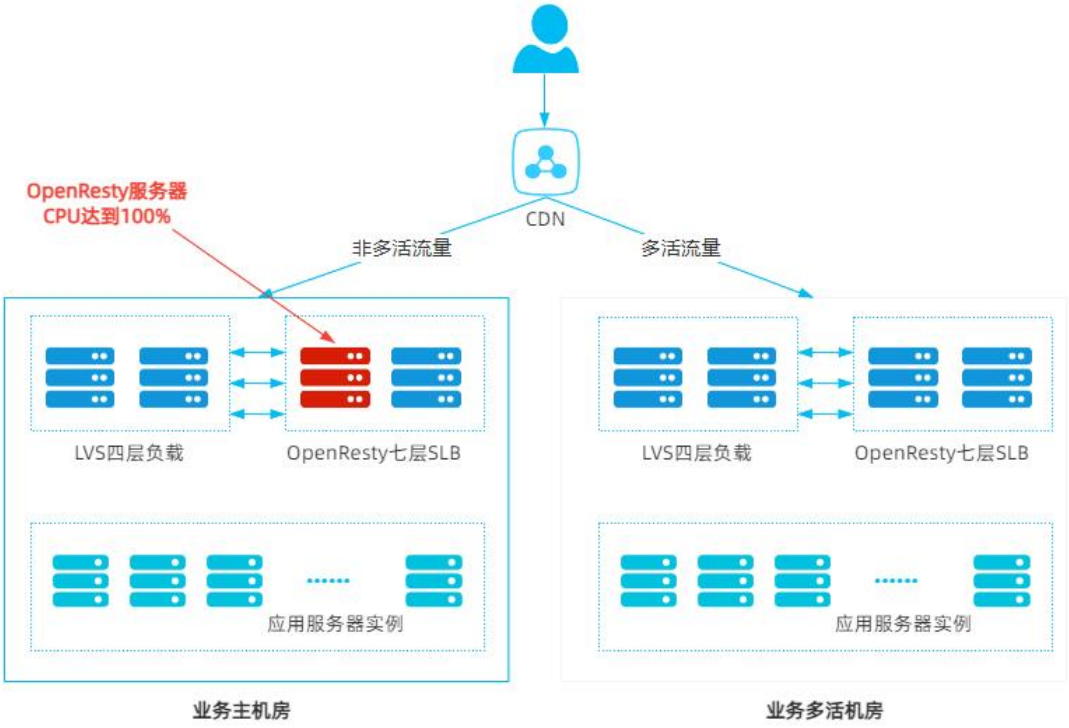
针对这样的一种架构，（如图）在 B 站崩溃的时候，运维人员采取了一些常规的措施去定位和解决问题。

- 梳理整个请求链路，查看存在异常的服务器节点，SLB 运维人员发现七层负载服务器的 CPU 达到 100%，

于是采取了重新加载以及冷重启 SLB 的方式都没有解决这个问题。

- 接着发现多活机房在 CPU 使用率正常的情况下，SLB 存在大量超时请求，于是重启多活机房 SLB，

恢复了部分业务的使用。



大家发现没有，遇到生产事故的时候，在不清楚具体问题的情况下，

优先重启相关服务节点来达到故障止损是效率最高的手段，

因此 B 站的运维人员通过重启 SLB 以及多活节点后。

使得多活机房恢复正常后部分业务可以正常访问。

但是业务主机房还没有恢复，这个时候，就需要慢慢去排查了。

于是运维人员使用了 Linux 提供的 Perf 系统分析工具，

定位到主机房 SLB 服务器的 CPU 热点集中在一个 Lua 函数上。

于是，也采取了常规的解决思路- 版本回滚。

一般一个问题的出现，在宏观层面没有太多变化的情况下，

很有可能是因为最近发布了新版本导致的。

从 B 站的问题复盘来看，回滚并没有解决问题，最后是通过重新构建新的 SLB 集群解决的。

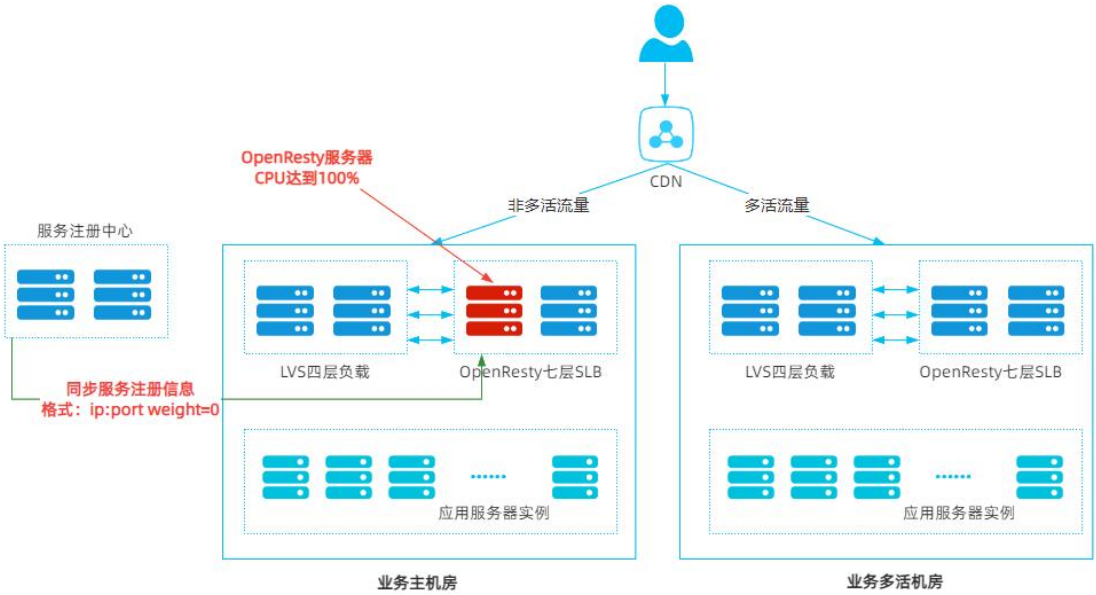
整个解决过程耗时 3 小时，对于互联网公司来说，这是一个非常大的故障。

虽然问题解决了，但是这个问题的根源以及让谁来背这个锅，还没搞清楚，所以需要继续分析原因。

经过一段时间的排查发现，CPU 跑满的原因是 OpenResty 里面的一个 lua 函数导致的（如图）。

这个函数的作用是从注册中心同步服务注册地址以及该服务节点的访问权重保存到 Nginx 的共享内存里面。

然后使用 lua-resty-balancer 模块对服务地址进行动态路由。



其中，在进行目标服务器的动态选择的时候，用到了加权轮询算法，

并使用了下面（如图）这个方法计算所有实例权重的最大公约数。

这个方法本身没有问题，但是当节点的权重 weight=“0”的时候，_gcd 函数收到的入参 b 可能是字符串“0”。

而 Lua 又是弱类型语言，允许传入字符串“0”，

这个时候 `if` 条件字符串 `0` 不等于数字 0，会执行_gcd 递归调用，其中在执行 `a%b` 的时候，用字符串和数字取模，

得到一个 NaN 的结果。

于是再次执行的时候，就变成了_gcd(NaN，NaN)的递归调用导致死循环问题。

```
17  local _gcd
18  _gcd = function (a, b)
19      if b == 0 then
20          return a
21      end
22
23      return _gcd(b, a % b)
24  end
25
```

可能大家会有疑惑，为什么回滚代码没有解决这个问题。

官方的声明说了，权重这个功能是 2 个月之前上线的，也就是这个问题的潜在风险存在了 2 个月。

在某种发布模式下，应用实例的权重会短暂调整为 0，导致注册中心返回给 SLB 的权重是一个字符串的“0”。

这个问题之所以一直没暴露出来，是因为这种发布模式的使用频率极低。

你们看。

一个影响生产环境 3 个小时，造成巨大影响和损失的生成事故，竟然是一个数据类型导致的

当得到这个答案的时候，大家可能会觉得难以接受，这就是千里之堤毁于蚁穴的真实写照啊。

在这个事故发生后，公司内部必然要做的几件事情

- 出一份详细的事故报告，明确事故的责任人和事故的级别
- 针对事故进行复盘
- 从技术层面、以及管理层面提出优化和改进的措施，避免后续再出现类似问题。

总的来说，越是偏向底层的开发，对于技术能力的要求以及工作的严谨性就越高。

通过 B 站的这次案例，屏幕前的你们也可以学习一些经验，从容应对未来具有挑战的工作。

我是 Mic，关注我，轻松学习技术架构！

53.57 [携程]利用 ChatGPT 去面试，轻松搞定年薪 50W。

最近 ChatGpt 这个工具火得一塌糊涂

抖音上随时都能刷到有博主在讲 ChatGpt

那它到底是什么呢？我们让它自己回答一下。

- ChatGPT 是啥?

它能够理解人类语言，并且根据语言描述做出对应的反馈。那其实我们可以简单的把 ChatGPT 理解成钢铁侠里面的贾维斯。

意味着利用 ChatGPT 可以帮我们解决很多问题，比如写文章、写代码、生成文案、写论文、修 Bug 等等

那么接下来，我们先先来问几个简单的问题，去体验一下 ChatGPT 的强大！

什么是微服务 Spring Boot 有哪些特性?

如何保证 Mysql 和 Redis 的数据一致性

接下来再来提升一下挑战难度

我们在 LeeCode 上找到一个困难模式的算法题

让 ChatGpt 帮我们回答一下。

给定两个大小分别为 m 和 n 的正序（从小到大）数组 nums1 和 nums2。请你找出并返回这两个正序数组的 中位数 。

算法的时间复杂度应该为 O(log (m+n)) 。

然后它很快就给了我们一个最优解，这个是在 Leecode 上运行后生成的报告。

我突然明白这款工具对于程序员来说最大的价值

就是用它来辅助面试。

用 ChatGpt 来回答面试过程中各种技术原理和技术场景问题，完全可以秒杀 99%以上的面试官。

当然，大家也不用担心 ChatGPT 会让程序员失业这个问题。

目前来看，ChatGpt 还是没办法真正做到理解人类语言和复杂性对话的能力，这意味着，它产生的任何反馈都可能是肤浅的，缺乏深度和洞察力。

好的，今天的分享就到这里了，对了，马上就要情人节了，给女朋友送什么礼物？ 我去问问 ChatGpt!

53.58 [携程]你是怎么理解线程安全问题的?

前两天，一个工作了 4 年的粉丝去面试，结果被这个问题卡住了。

这个问题其实在我整理的 30W 多万字的面试文档里面有，有需要的小伙伴可以在评论区置顶中去领取。

所谓线程安全问题，简单来说，就是在多个线程访问某个方法或者对象的时候，不管通过任何的方式调用以及线程如何去交替执行。

在程序中不做任何同步干预操作的情况下，这个方法或者对象的执行/修改都能按照预期的结果来反馈。

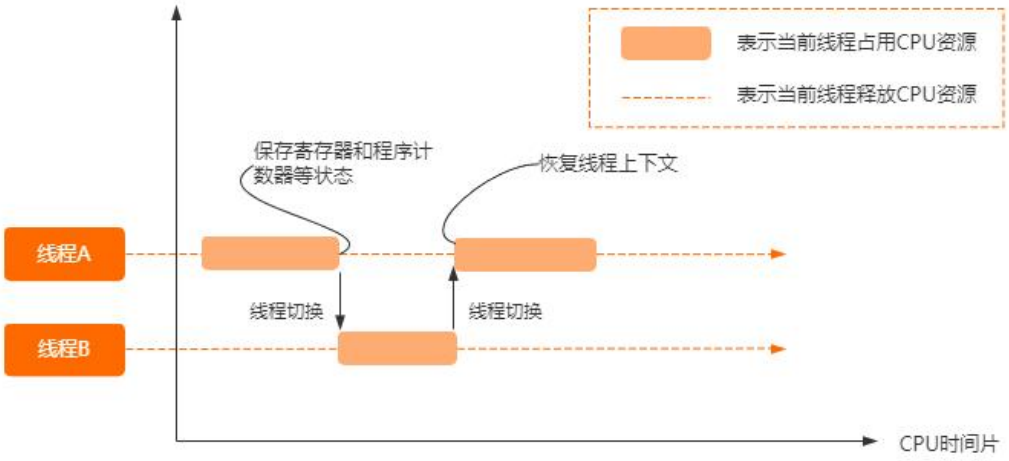
我这样去解释，大家可能会有点懵逼。

实际上，线程安全问题的具体表现在三个方面，原子性、有序性、可见性。

原子性呢，是指当一个线程执行一系列程序指令操作的时候，它应该是不可中断的，因为一旦出现中断，站在多线程的视角来看，这一系列的程序指令会出现前后执行结果不一致的问题。

这个和数据库里面的原子性是一样的，就是一段程序只能由一个线程完整的执行完成，而不能存在多个线程干扰。

（如图）CPU 的上下文切换，是导致原子性问题的核心，而 JVM 里面提供了 Synchronized 关键字来解决原子性问题。



可见性，就是说在多线程环境下，由于读和写是发生在不同的线程里面，有可能出现某个线程对共享变量的修改，对其他线程不是实时可见的。

导致可见性问题的原因有很多，比如 CPU 的高速缓存、CPU 的指令重排序、编译器的指令重排序。

有序性，指的是程序编写的指令顺序和最终 CPU 运行的指令顺序可能出现不一致的现象，这种现象也可以称为指令重排序，所以有序性也会导致可见性问题。

可见性和有序性可以通过 JVM 里面提供了一个 Volatile 关键字来解决。

在我看来，导致有序性、原子性、可见性问题的本质，是计算机工程师为了最大化提升 CPU 利用率导致的。比如为了提升 CPU 利用率，设计了三级缓存、设计了 StoreBuffer、设计了缓存行这种预读机制、在操作系统里面，设计了线程模型、在编译器里面，设计了编译器的深度优化机制。

今天的分享就到这里，在面试的时候大家还有遇到哪些比较难的问题，欢迎在评论区留言。

53.59 [携程]Spring 中事务的传播行为有哪些?

我们现在不是正在招聘 Java 高级开发嘛，然后昨天有个工作 5 年的程序员来面试。

简历写得非常漂亮，什么精通微服务、高并发、jvm、并发编程等技术。

我怕一上来就问他比较难的问题他扛不住，就先问了一个简单的问题：Spring 中的事务传播行为有哪些?

估计他在刷面试题的时候没想到我会问这么简单的问题，由于准备不充分导致没回答上来。

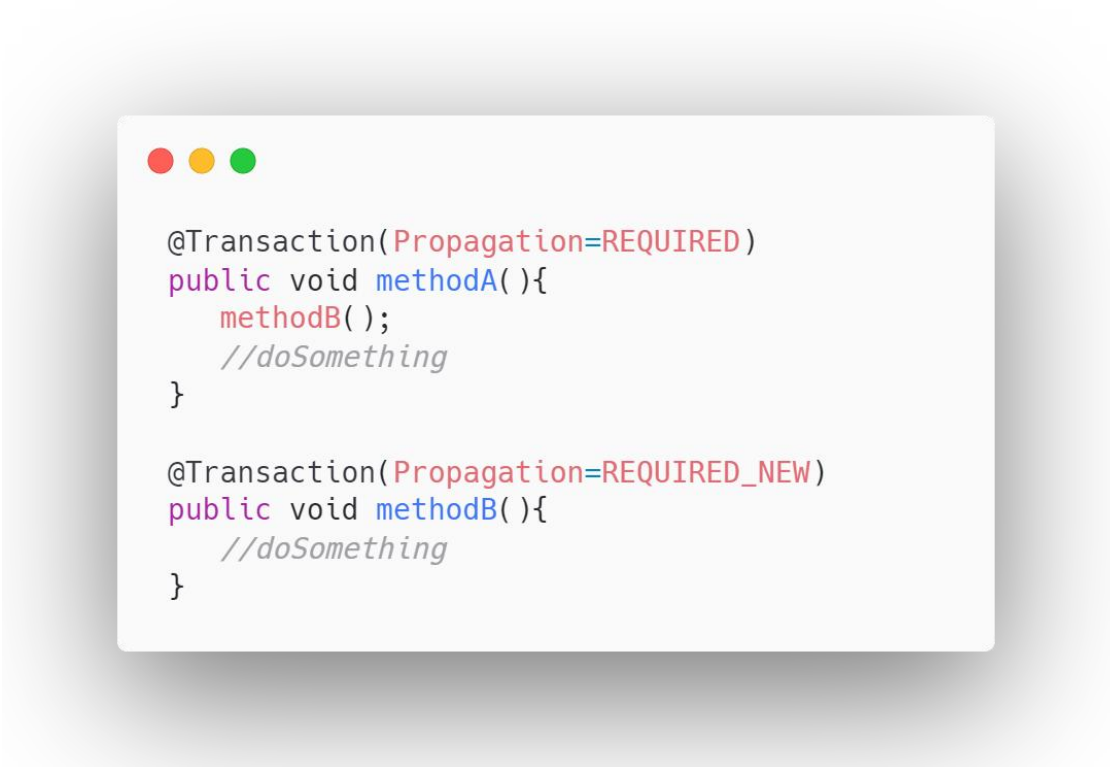
于是我把自己花了一年时间沉淀下来的 20 多万字的面试问题分享给他，有需要的小伙伴可以在评论区置顶中去领取。

当然，从长期职业发展角度来看，我还是建议大家系统的去学习一些技术。

就拿事务传播行为来说，它解决的核心问题是，多个声明了事务的方法相互调用的时候存在事务嵌套问题，那么这个事务的行为应该如何进行传递？

（如图）比如说，methodA()调用 methodB()，两个方法都显示的开启了事务。

那么 methodB()是开启一个新事务，还是继续在 methodA()这个事务中执行？就取决于事务的传播行为。



```
@Transaction(Propagation=REQUIRED)
public void methodA(){
    methodB();
    //doSomething
}

@Transaction(Propagation=REQUIRED_NEW)
public void methodB(){
    //doSomething
}
```

所以，Spring 为了解决这个问题，定义了 7 种事务传播行为。

1. REQUIRED：默认的 Spring 事物传播级别，如果当前存在事务，则加入这个事务，如果不存在事务，就新建一个事务。
2. REQUIRE_NEW：不管是否存在事务，都会新开一个事务，新老事务相互独立。外部事务抛出异常回滚不会影响内部事务的正常提交。
3. NESTED：如果当前存在事务，则嵌套在当前事务中执行。如果当前没有事务，则新建一个事务，类似于 REQUIRE_NEW。

4. SUPPORTS：表示支持当前事务，如果当前不存在事务，以非事务的方式执行。
5. NOT_SUPPORTED：表示以非事务的方式来运行，如果当前存在事务，则把当前事务挂起。
6. MANDATORY：强制事务执行，若当前不存在事务，则抛出异常.
7. NEVER：以非事务的方式执行，如果当前存在事务，则抛出异常。

很多同学会尝试用背诵的方法去记下这些行为，但其实如果大家站在代码设计的角度去思考，是可以直接推导出来至少 5 种的吧。

好了，今天的分享就到这里，在面试的时候大家还有遇到哪些比较难的问题，欢迎在评论区留言。

53.60 [携程]说一说 Mybatis 里面的缓存机制

昨天一个工作了 4 年的粉丝去面试，被问到 Mybatis 里面的缓存机制时，哆嗦了半天没回答上来。

然后回来就在群里面抱怨面试太卷了，实际上，Mybatis 多级缓存在开发中是会用到的一个功能，如果这个部分没弄清楚。

可能会出现一些莫名其妙的问题，最后又得通过百度帮你解决，个人能力还是没有提升。

我之前给大家分享的 20W 字的面试文档里面也有这个问题的标准回答，短期面试突击的话还是可以去看一下，具体可以在评论区的置顶中领取。

回答正题，

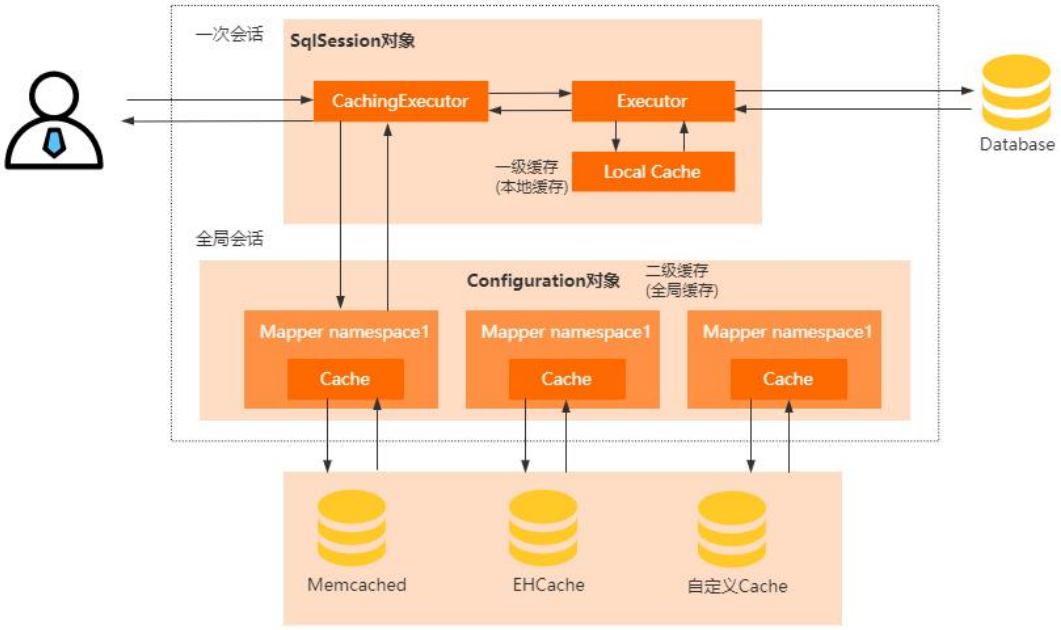
（如图）Mybatis 里面设计的二级缓存是用来提升数据的检索效率，避免每次数据的访问都需要去查询数据库。

一级缓存，是 SqlSession 级别的缓存，也叫本地缓存，因为每个用户在执行查询的时候都需要使用 SqlSession 来执行，

为了避免每次都去查数据库，Mybatis 把查询出来的数据保存到 SqlSession 的本地缓存中，后续的 SQL 如果命中缓存，就可以直接从本地缓存读取了。

如果想要实现跨 SqlSession 级别的缓存？那么一级缓存就无法实现了，因此在 Mybatis 里面引入了二级缓存，就是当多个用户

在查询数据的时候，只有有任何一个 SqlSession 拿到了数据就会放入到二级缓存里面，其他的 SqlSession 就可以从二级缓存加载数据。

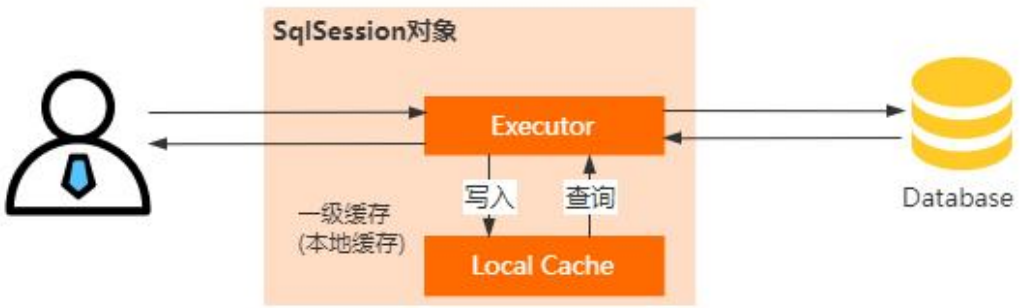


每个

(如图）一级缓存的具体实现原理是：

在 SqlSession 里面持有一个 Executor，每个 Executor 中有一个 LocalCache 对象。当用户发起查询的时候，Mybatis 会根据执行语句在 Local Cache 里面查询，如果没命中，再去查询数据库并写入到 LocalCache，否则直接返回。

所以，以及缓存的生命周期是 SqlSessiion，而且在多个 Sqlsession 或者分布式环境下，可能会导致数据库写操作出现脏数据。

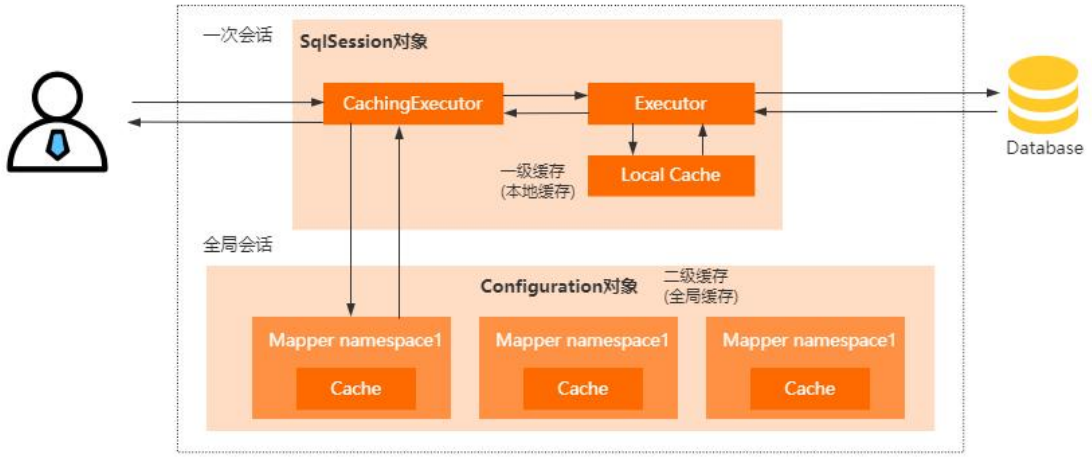


（如图）二级缓存的具体实现原理是：

使用 CachingExecutor 装饰了 Executor，所以在进入一级缓存的查询流程之前，会先通过 CachingExecutor 进行二级缓存的查询。

开启二级缓存以后，会被多个 SqlSession 共享，所以它是一个全局缓存。因此它的查询流程是先查二级缓存，再查一级缓存，最后再查数据库。

另外，MyBatis 的二级缓存相对于一级缓存来说，实现了 SqlSession 之间缓存数据的共享，同时缓存粒度也能够到 namespace 级别，并且还可以通过 Cache 接口实现类不同的组合，对 Cache 的可控性也更强。



二级缓存的设计思想非常常见，比如 Nacos、Eureka 都用到了，学习这块的思想有助于提升解决问题的能力。

好了，今天的分享就到这里，在面试的时候大家还有遇到哪些比较难的问题，欢迎在评论区留言。

53.61 [携程]Redis 存在线程安全问题吗？为什么？

一个工作了 6 年的粉丝和我说，最近面试的机会确实多了，但是面试难度反而更大了。

昨天刚面试的时候被问到：Redis 里面的线程安全性问题 ， 没回答上来。

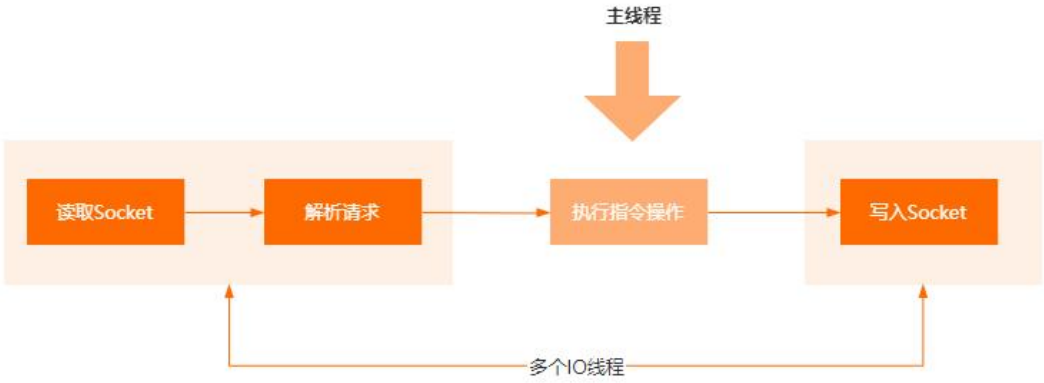
如果是急着面试，又没有太多时间去系统性的学习，我建议还是要通过刷一些面试题突击一下。

我花了一年多时间整理的 40W 字的面试文档，涵盖了几乎 99%的面试题，有需要的同学可以在评论区置顶中去领取一下。

其实关于 Redis 线程安全性的问题并不难理解。

Redis Server 本身是一个线程安全的 K-V 数据库，也就是说在 Redis Server 上执行的指令，不需要任何同步机制，不会存在线程安全问题。

（如图）虽然 Redis 6.0 里面，增加了多线程的模型，但是增加的多线程只是用来处理网络 IO 事件，对于指令的执行过程，仍然是由主线程来处理，所以不会存在多个线程通知执行操作指令的情况。



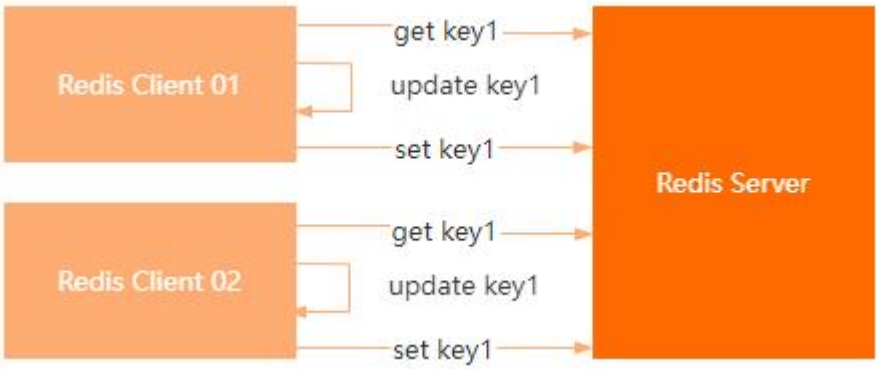
至于为什么 Redis 没有采用多线程来执行指令，我认为有几个方面的原因。

- Redis Server 本身可能出现的性能瓶颈点无非就是网络 IO、CPU、内存。但是 CPU 不是 Redis 的瓶颈点，所以没必要使用多线程来执行指令。
- 如果采用多线程，意味着对于 redis 的所有指令操作，都必须要考虑到线程安全问题，也就是说需要加锁来解决，这种方式带来的性能影响反而更大。

第二个，从 Redis 客户端层面。

（如图）虽然 Redis Server 中的指令执行是原子的，但是如果有多个 Redis 客户端同时执行多个指令的时候，就无法保证原子性。

假设两个 redis client 同时获取 Redis Server 上的 key1， 同时进行修改和写入，因为多线程环境下的原子性无法被保障，以及多进程情况下的共享资源访问的竞争问题，使得数据的安全性无法得到保障。



当然，对于客户端层面的线程安全性问题，解决方法有很多，比如尽可能的使用 Redis 里面的原子指令，或者对多个客户端的资源访问加锁，或者通过 Lua 脚本来实现多个指令的操作等等。

线程安全性确实是开发中非常重要的一个知识点，所以面试的频率会特别高，大家可以重点关注一下。

好了，今天的分享就到这里，在面试的时候大家还有遇到哪些比较难的问题，欢迎在评论区留言。

53.62 [携程]请你简单说一下 Mysql 的事务隔离级别

什么情况，写了 5 年的 CRUD，还搞不清楚 Mysql 的事务隔离级别，难怪第一面就被刷下来。

一个 5 年经验的粉丝，在一个公司干了 5 年，觉得自己特厉害，什么都能搞定，结果每次一到技术面就被刷。

问我该怎么办，原本我是想让他系统学习一下，但是他已经找了 2 个月工作了，急需要快速找到一个工作稳定下来。

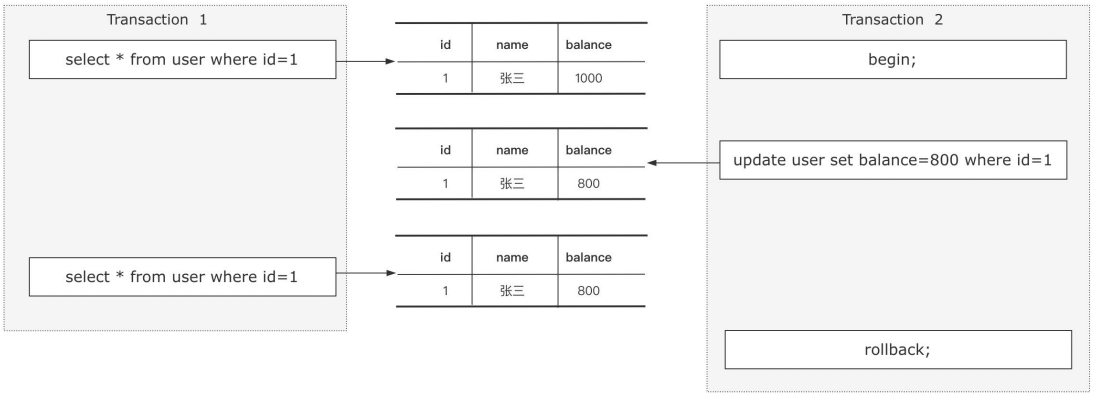
所以我只能把之前整理的 40W 字的面试文档先发给他，让他先突击一下。这个面试文档覆盖面特别广，而且每个问题都有很清晰完整的回答，有需要的小伙伴可以在评论区置顶中去领取。

回到正题。

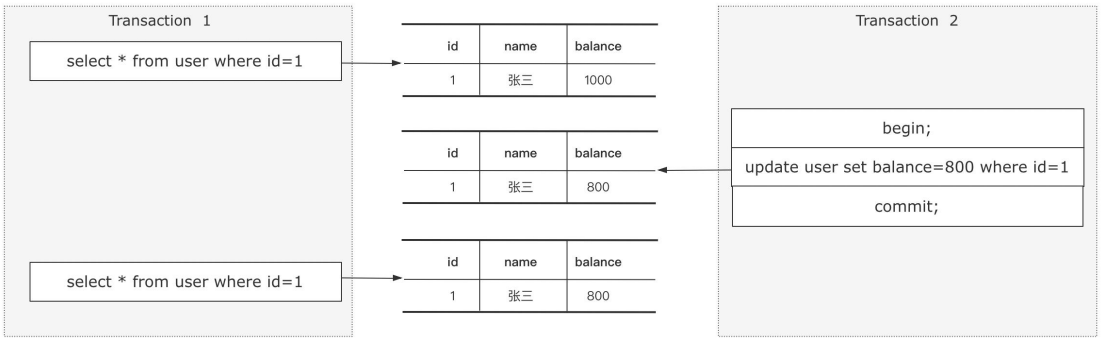
事务隔离级别，是为了解决多个并行事务竞争导致的数据安全问题的一种规范。

具体来说，多个事务竞争可能会产生三种不同的现象。

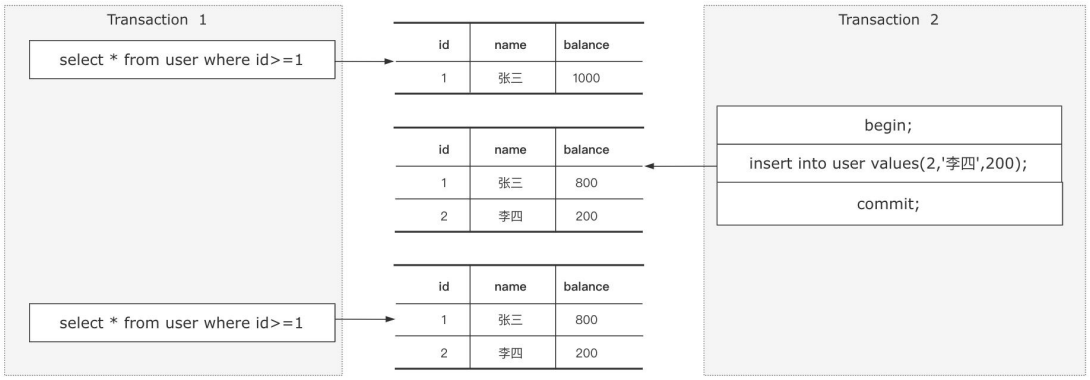
1. （如图）假设有两个事务 T1/T2 同时在执行，T1 事务有可能会读取到 T2 事务未提交的数据，但是未提交的事务 T2 可能会回滚，也就导致了 T1 事务读取到最终不一定存在的数据产生脏读的现象。



2. （如图）假设有两个事务 T1/T2 同时执行，事务 T1 在不同的时刻读取同一行数据的时候结果可能不一样，从而导致不可重复读的问题。



3. （如图），假设有两个事务 T1/T2 同时执行，事务 T1 执行范围查询或者范围修改的过程中，事务 T2 插入了一条属于事务 T1 范围内的数据并且提交了，这时候在事务 T1 查询发现多出来了一条数据，或者在 T1 事务发现这条数据没有被修改，看起来像是产生了幻觉，这种现象称为幻读。



而这三种现象在实际应用中，可能有些场景不能接受某些现象的存在，所以在 SQL 标准中定义了四种隔离级别，分别是：

4. 读未提交，在这种隔离级别下，可能会产生脏读、不可重复读、幻读。
5. 读已提交（RC），在这种隔离级别下，可能会产生不可重复读和幻读。

6. 可重复读（RR），在这种隔离级别下，可能会产生幻读

7. 串行化，在这种隔离级别下，多个并行事务串行化执行，不会产生安全性问题。

这四种隔离级别里面，只有串行化解决了全部的问题，但也意味着这种隔离级别的性能是最低的。

在 Mysql 里面，InnoDB 引擎默认的隔离级别是 RR（可重复读），因为它需要保证事务 ACID 特性中的隔离性特征。

好了，今天的分享就到这里，在面试的时候大家还有遇到哪些比较难的问题，欢迎在评论区留言。

53.63 [携程]说一下你对 SPI 机制的理解以及 Dubbo 中有哪些 SPI 机制的实现

昨天一个工作了 4 年的粉丝问了一个比较小众的面试题。

“说一下你对 SPI 机制的理解以及 Dubbo 中提供的 SPI 机制”。

SPI 这个思想在很多地方都有实现，比如 Spring 里面的 SpringFactoriesLoader

这个问题的答案，在我整理的 30W 字的面试文档里面有详细的解释

有需要的可以在评论区置顶中去领取。

SPI 全称为 Service Provider Interface，是 Java 中一种服务发现机制。

在 SPI 机制中，服务提供者为某个接口实现具体的类，而在运行时，通过 SPI 机制，查找到对应的实现类，并将其加载进行使用。

比较典型的一个例子就是 java.jdbc.Driver 这个数据库驱动接口，在 Java 中只提供了接口的定义，

具体的实现类由各个数据库厂商提供的驱动包来完成，程序在运行的时候会根据当前导入的驱动包来完成对应数据库的连接

在 Dubbo 中，根据 Java SPI 的思想，衍生了不同的 SPI 实现。

- 指定名称的扩展点实现，也就是根据指定名称获取并加载对应的扩展点实现类
- @Adaptive 注解定义自适应扩展点，它会根据上下文参数动态适配一个具体的实现类
- @Activate 注解定义的激活扩展点，它会根据上下文的相关参数配置，来决定哪些扩展点会被动态激活，主要体现在 Filter 中。

SPI 思想可以为程序提供比较强大的的可扩展性能力，在企业级开发中，可以尝试去借鉴和使用。

好了，今天的分享就到这里，bye bye!

53.64 [滴滴]RDB 和 AOF 的实现原理以及优缺点

一个工作了 5 年的粉丝私信我，关于 RDB 和 AOF 的实现原理这个问题

在面试的时候，应该怎么回答?

于是我把之前整理过的一个高手回答整理成文档发给了他，后来他参考这个回复在面试的时候顺利拿到了 offer

今天我把这个文档分享给大家，可以在评论区置顶中获取。

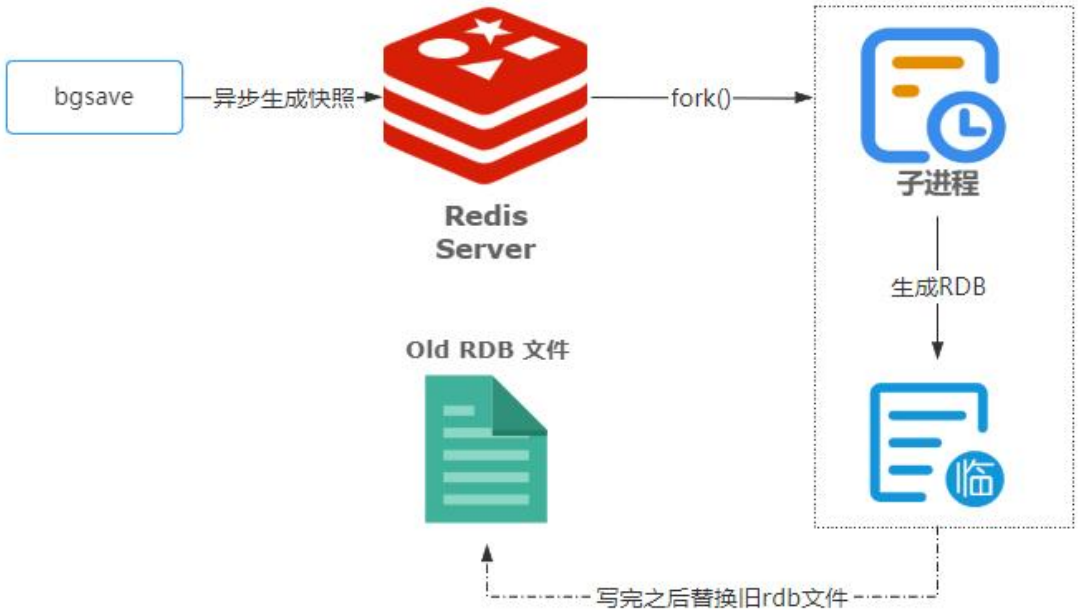
关于 RDB 和 AOF 的实现原理以及优缺点，我觉得可以分成三个部分去回答。

第一部分，先说明这两种持久化机制的特性。

RDB 和 AOF 都是 Redis 里面提供的持久化机制，RDB 是通过快照方式实现持久化、AOF 是通过命令追加的方式实现持久化。

第二部分，说明这两种机制的工作原理。

【如图】RDB 持久化机制会根据快照触发条件，把内存里面的数据快照写入到磁盘，以二进制的压缩文件进行存储。

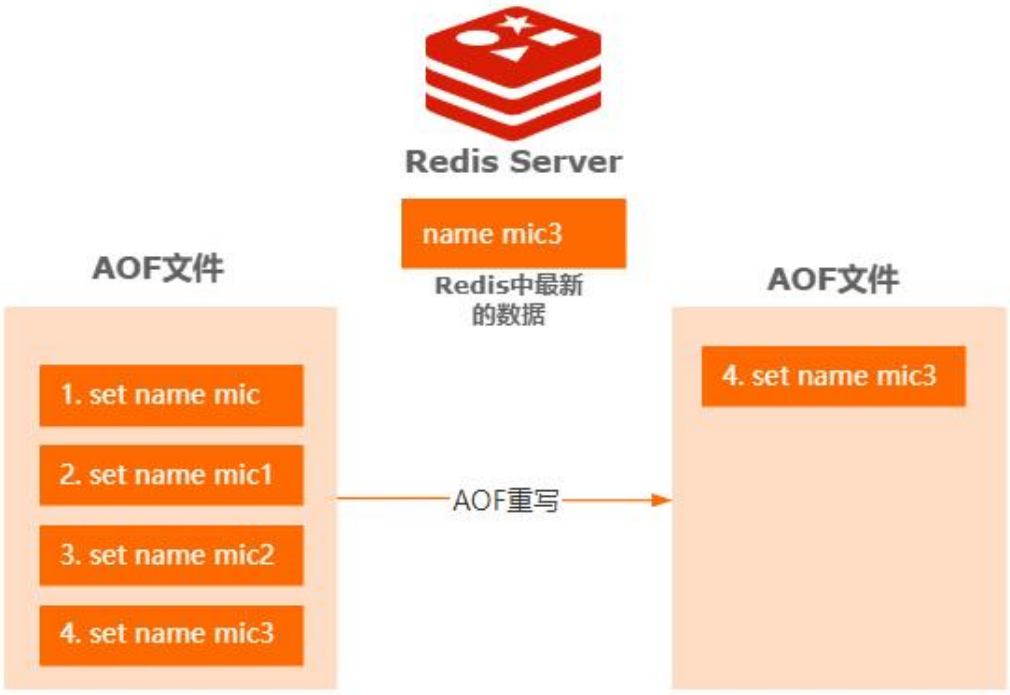


RDB 快照的触发方式有很多，比如

- 执行 bgsave 命令触发异步快照，执行 save 命令触发同步快照，同步快照会阻塞客户端的执行指令。
- 根据 redis.conf 文件里面的配置，自动触发 bgsave
- 主从复制的时候触发

【如图】，AOF 持久化机制是近乎实时的方式来完成持久化的，就是客户端执行一个数据变更的操作，Redis Server 就会把这个命令追加到 aof 缓冲区的末尾，
然后再把缓冲区的数据写入到磁盘的 AOF 文件里面，至于最终什么时候真正持久化到磁盘，是根据刷盘的策略来决定的。

为了避免追加的方式导致 AOF 文件过大的问题，Redis 提供了 AOF 重写机制(如图），也就是说当 AOF 文件的大小达到某个阈值的时候，就会把这个文件里面相同的指令进行压缩。



第三部分，AOF 和 RDB 的优缺点分析

我认为 RDB 和 AOF 的优缺点有两个。

- RDB 是每隔一段时间触发持久化，因此数据安全性低，AOF 可以做到实时持久化，数据安全性较高
- RDB 文件默认采用压缩的方式持久化，AOF 存储的是执行指令，所以 RDB 在数据恢复的时候性能比 AOF 要好

你看，通过这样的回答，逻辑清晰，表达完整，会给面试官一个很好的感受。

好了，今天的分享就到这里，bye bye。

53.65 [滴滴]什么是守护线程，它有什么特点

今天有个粉丝朋友问了一个问题，我看到问题的时候还是有点惊讶

竟然还有人不知道什么是守护线程?

我之前分享给大家的那个 30W 字的面试文档里面，是有详细解释守护线程的。

没有领到的可以在评论区置顶中去获取。

下面我用最简单的方式让大家彻底搞懂守护线程。

简单来说，守护线程就是一种后台服务线程，他和我们在 Java 里面创建的用户线程是一模一样的。

守护线程和用户线程的区别有几个点，这几个点也是守护线程本身的特性：

1. 在线程创建方面，对于守护线程，我们需要主动调用 setDaemon()并且设置成 true。
2. 我们知道，一个 Java 进程中，只要有任何一个用户线程还在运行，那么这个 java 进程就不会结束，否则，这个程序才会终止。

注意，Java 进程的终止与否，只和用户线程有关。如果当前还有守护线程正在运行，也不会阻止 Java 程序的终止。

因此，守护线程的生命周期依赖于用户线程。

举个例子，JVM 垃圾回收线程就是一个典型的守护线程，它存在的意义是不断的处理用户线程运行过程中产生的内存垃圾。

一旦用户线程全部结束了，那垃圾回收线程也就没有存在的意义了。

由于守护线程的特性，所以它它适合用在一些后台的通用服务场景里面。

但是守护线程不能用在线程池或者一些 IO 任务的场景里面，因为一旦 JVM 退出之后，守护线程也会直接退出。

就会可能导致任务没有执行完或者资源没有正确释放的问题。

好了，今天的分享就到这里，bye bye。

53.66 [滴滴]AbstractQueuedSynchronized为什么采用双向链表

Hi，大家好，我是 Mic。

昨天一个工作 7 年的粉丝去参加小米的一面，被问到“AbstractQueuedSynchronized 为什么采用双向链表”？这个问题没回答出来被拒了。

然后他去找 ChatGPT，也没有得到答案。

其实他找错地方了，我之前给大家整理的 30W 字的面试文档里面，是有一个比较完整和清晰的回答的。

没领到的同学可以在评论区置顶中去领取。

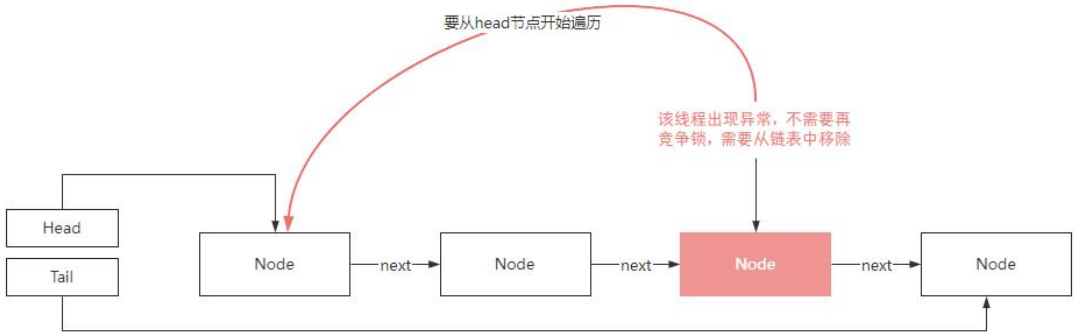
下面我从两个方面给大家解释一下这个问题：

第一个方面，双向链表的优势：

- 双向链表提供了双向指针，可以在任何一个节点方便向前或向后进行遍历，这种对于有反向遍历需求的场景来说非常有用。
- 双向链表可以在任意节点位置实现数据的插入和删除，并且这些操作的时间复杂度都是 O(1)，不受链表长度的影响。这对于需要频繁对链表进行增删操作的场景非常有用。

第二个方面，说一下 AQS 采用双向链表的原因

- 存储在双向链表中的线程，有可能这个线程出现异常不再需要竞争锁，所以需要把这些异常节点从链表中删除，而删除操作需要找到这个节点的前驱结点，如果不采用双向链表，就必须要从头节点开始遍历，时间复杂度就变成了 O(n)。



- 新加入到链表中的线程，在进入到阻塞状态之前，需要判断前驱节点的状态，只有前驱节点是 Sign 状态的时候才会让当前线程阻塞，所以这里也会涉及到前驱节点的查找，采用双向链表能够更好的提升查找效率

```
if (shouldParkAfterFailedAcquire(p, node) &&
    parkAndCheckInterrupt())
    interrupted = true;
```

- 线程在加入到链表中后，会通过自旋的方式去尝试竞争锁来提升性能，在自旋竞争锁的时候为了保证锁竞争的公平性，需要先判断当前线程所在节点的前驱节点是否是头节点。这个判断也需要获取当前节点的前驱节点，同样采用双向链表能提高查找效率。

```
final boolean acquireQueued(final Node node, int arg) {
    boolean failed = true;
    try {
        boolean interrupted = false;
        for (;;) {
            final Node p = node.predecessor();
            if (p == head && tryAcquire(arg)) {
```

总而言之，采用单向链表不支持双向遍历，而 AQS 中存在很多需要双向遍历的场景来提升线程阻塞和唤醒的效率。

大家理解了吗?

好了，今天的分享就到这里，bye bye。

53.67 [滴滴]说一下你日常工作中是怎么优化 SQL 的

昨天一个工作 2 年的粉丝去面试，被问到日常工作中的 SQL 优化问题。

这个问题在面试过程中的频率会特别高，SQL 编写和 SQL 优化是开发人员的基础技能。但是要回答好这个问题还是有难度，因为很多时候大家第一反应是加索引，但这个回答并不完整。

我之前整理的一个 30W 字的面试文档里面，有这个问题的详细回答，需要的小伙伴可以在评论区置顶中领取。

关于这个问题，我们其实可以从下面几个纬度来回答：

1. 加索引，增加索引是一种简单高效的手段，但是需要选择合适的列，同时避免导致索引失效的操作，比如 like、函数等。
2. 避免返回不必要的数据列，减少返回的数据列可以增加查询的效率。
3. 根据查询分析器适当优化 SQL 的结构，比如是否走全表扫描、避免子查询等
4. 分库分表，在单表数据量较大或者并发连接数过高的情况下，通过这种方式可以有效提升查询效率
5. 读写分离，针对读多写少的场景，这样可以保证写操作的数据库承受更小的压力，也可以缓解独占锁和共享锁的竞争。

基本上回答到这个程度得到面试官的认可是没问题，不过要注意，面试官还会基于你回答的内容再做更进一步的深度考察，

比如为什么增加索引能够提升查询效率等，这些在我整理的面试文档中都有。

好了，今天的分享就到这里，bye

53.68 [滴滴]limit 1000000,10 加载很慢该怎么优化

昨天一个工作 5 年的粉丝去面试，被问到这样一个场景问题：

`limit 1000000 10` 查询比较慢该如何优化，他当时没回答出来，于是向我求助。

这个问题，在我整理的 30W 字的《大厂后端面试指南》里面是有的，可以在评论区置顶中去领取

关于这个问题，有多种解决方案，大家可以在回答的时候尽可能的考虑全面一点。

1. 如果 id 是连续的，可以直接使用这样的方式。

select id，name fromuserwhere id>1000000limit10.

这种方式其实就是先对数据做过滤，然后再 limit，可以有效提升查询效率

2. 通过 order by+索引来解决

select id，name fromuserorderby id limit1000000，10

需要注意 id 是索引列，通过索引排序后再 limit，同样减少了计算次数

3. 从业务层面来考虑，限制页数，一般情况下用户去翻 100W 页来找数据，如果让你们老板去翻 100w 页，估计第二天就把你开除了。我们通常会通过搜索来优化查找过程。

以上就是这个问题的回答思路，面试的时候不一定要完全陷入到面试官的逻辑中，也可以跳出来思考。

好的，今天的分享就到这里，byebye

53.69 [滴滴]Mysql 主从集群同步延迟问题怎么解决

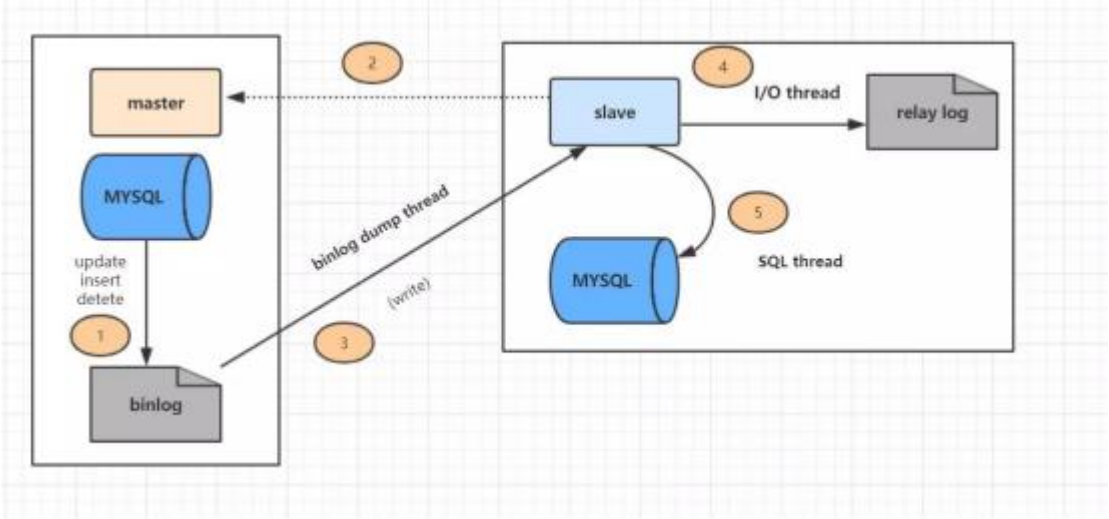
一个工作了 8 年的粉丝，去京东面试在第一面被问到这个问题。

Mysql 主从集群同步延迟问题怎么解决。

我发现这个问题面试命中率还挺高的，如果大家准备去面试的话，我建议可以去评论区置顶中领取一下

之前整理的 30W 字的面试文档。

下面我先来个大家复习一下主从复制的工作原理。



复制过程分为几个步骤：

1. 主库的更新事件(update、insert、delete)被写到 binlog
2. 从库发起连接，连接到主库。
3. 此时主库创建一个 binlog dump thread，把 binlog 的内容发送到从库。
4. 从库启动之后，创建一个 I/O 线程，读取主库传过来的 binlog 内容并写入到 relay log
5. 从库还会创建一个 SQL 线程，从 relay log 里面读取内容，从 Exec_Master_Log_Pos 位置开始执行读取到的更新事件，将更新内容写入到 slave 的 db

主从数据同步涉及到网络数据传输，由于网络通信的延迟以及从库数据处理的效率问题，就会导致主从数据同步延迟的情况。

一般可以通过以下几个方法来解决

- 设计一主多从来分担从库压力，减少主从同步延迟问题
- 如果对数据一致性要求高，在从库存在延迟的情况下，可以强制走主库查询数据
- 可以在从库上执行 `show slave status` 命令，获取 `seconds_behind_master` 字段的延迟时间，然后通过 sleep 阻塞等待固定时间后再次查询
- 通过并行复制解决从库复制延迟的问题

实际上，主从复制的场景无法避免同步延迟的问题，如果一定要用强一致方案，那就应该考虑其他能够实现一致性场景的技术方案。

好的，几天的分享就到这里，byebye

53.70 [滴滴]Mysql 的 binlog 有几种格式？分别有什么区别

一个工作 2 年的粉丝去面试，被问到 binlog 的格式问题。

这个问题其实一般情况下开发人员关注会比较少，只有在涉及到数据库主从同步的时候才会关注。

不过这个问题在我整理的《大厂后端面试指南》中有，大家可以在评论区置顶中去领取

准确来说，Binlog 有三种格式：statement，row 和 mixed。

- statement，记录的是 SQL 的原文。好处是，不需要记录每一行的变化，减少了 binlog 日志量，节约了 IO，提高性能。由于 sql 的执行是有上下文的，因此在保存的时候需要保存相关的信息，同时还有一些使用了函数之类的语句无法被记录复制。
- row，不记录 sql 语句上下文相关信息，仅保存哪条记录被修改。记录单元为每一行的改动，基本是可以全部记下来但是由于很多操作，会导致大量行的改动(比如 alter table)，因此这种模式的文件保存的信息太多，日志量太大。
- mixed，一种折中的方案，普通操作使用 statement 记录，当无法使用 statement 的时候使用 row。

如果大家想加深印象更好的应对面试，可以自己搭建 Mysql 的主从环境，然后通过配置不同方式来查看日志内容。

好的，今天的分享就到这里。byebye

53.71 [滴滴]索引有哪些缺点以及具体有哪些索引类型

昨天一个工作 4 年的粉丝，被问到：“Mysql 索引有哪些缺点，以及具体有哪些索引类型”这个问题。

这个问题比较基础，但凡有 Java 经验的同学都能回答一二，不过在目前整个就业环境并不是特别好的情况下。

回答出来和回答好是两个概念，所以我建议大家可以去评论区置顶中领取那本 30W 字的《大厂面试手册》，

上面记录了一个比较清晰和完整的回答，大家只需要按照这个模版回答就很好了。

下面我来说一下我的回答：

第一个，索引的优缺点

优点：

- 合理的增加索引，可以提高数据查询的效率，减少查询时间
- 有一些特殊的索引，可以保证数据的完整性，比如唯一索引

缺点：

- 创建索引和维护索引需要消耗时间
- 索引需要额外占用物理空间
- 对创建了索引的表进行数据的增加、修改、删除时，会同步动态维护索引，这个部分会造成性能的影响

第二个，索引的类型

- 主键索引： 数据列不允许重复，不允许为 NULL，一个表只能有一个主键。
- 唯一索引： 数据列不允许重复，允许为 NULL 值，一个表允许多个列创建唯一索引。
- 普通索引： 基本的索引类型，没有唯一性的限制，允许为 NULL 值。
- 全文索引：是目前搜索引擎使用的一种关键技术，对文本的内容进行分词、搜索。
- 覆盖索引：查询列要被所建的索引覆盖，不必读取数据行
- 组合索引：多列值组成一个索引，用于组合搜索，效率大于索引合并

以上就是我的回答，大家如果觉得这个回答比较完整，可以点赞收藏一下，后续面试的时候可以复习

好了，今天的分享就到这里， byebye

53.72 [滴滴]MySQL 数据库 cpu 飙升的话，要怎么处理呢?

昨天一个工作 5 年的粉丝，被问到一个 Mysql 数据库 CPU 飙升的实战场景问题，一般情况下如果没有这方面的操作经验，可能就会当场被拒绝。

目前的面试除了八股文以外，还会增加一些场景提考察求职者的真实能力，所以我也把这个问题的回答整理到了《大厂面试指南》中，有需要的小伙伴可以在评论区置顶中去领取。

关于这个问题，我的回答思路如下，大家可以参考。

第一步，排查问题

- 使用 top 命令，找到 cpu 占用过高的进程是否是 mysqld
- 如果是，可以在 mysql 中通过 `show processlist` 查看当前的会话情况，确定是否有消耗资源的 SQL 正在运行
- 找到消耗过高的 SQL，通过执行计划进行具体的分析

第二步，处理方式

- 如果确定是 SQL 问题，可以通过 SQL 的优化手段进行调整
- 重新执行 SQL 分析确认是否有达到优化的目的

第三步，其他情况

如果不是 SQL 的问题导致，那就需要分析 CPU 飙高的这个时间段，Mysql 的整体并发连接数。

如果有大量的请求连接进来，那我们就需要分析这个时间段业务的情况，再做出相应的调整。

最后，如果是 Mysql 本身的参数并不是最优状态，那我们可以对 Mysql 服务节点的配置进行调整，比如缓存大小、线程池大小等

综上所述，处理 MySQL 数据库 CPU 飙升问题需要综合考虑多个方面，包括查询优化、索引优化、MySQL 配置优化、硬件升级、负载均衡等。大家在回答的时候尽可能全面一点去考虑。

好的，今天的分享就到这里，byebye

53.73 [滴滴]会员批量过期的方案怎么实现?

昨天，一个工作 6 年的同学去某互联网大厂面试，遇到这样一个场景问题。

“有一张 200W 数据量的会员表，每个会员会有长短不一的到期时间，现在想在快到期之前发送邮件通知提醒续费”

该怎么实现。

这个问题解决思路有很多，我把这个问题的回答整理到了 40W 字的面试文档里面，大家可以去评论区置顶中领取。

53.73.1 问题解析

对于这类的场景问题，我建议大家不要急着去回答，而是先冷静下来，想清楚面试官希望通过这个问题去了解哪方面的能力，以及这个问题里面有哪些陷阱。

特别注意一个点，一定要理解清楚面试官的问题，如果没听懂，可以确认一遍。

很显然，这个问题里面有几个关键词：

- 200W 数据意味着数据量比较大
- 每个会员都有过期时间，需要能够筛选出快过期的会员

很显然，如果直接去通过 select 语句做筛选，就掉入坑里了，因为这里会存在性能问题，那接下来看一下一些相对比较合理的回答。

53.73.2 问题答案

关于这个问题，可以有三种解决方案：

第一种，系统不主动轮询，而是等用户登录到系统以后，触发一次检查。

如果发现会员的过期时间小于设定的阈值，就触发一次弹窗和邮件提醒。

这种方式规避了轮询问题，不会对数据库和后端应用程序造成任何压力。

缺点是，如果用户一直不登陆，就一直无法实现会员过期，并且也无法提前去根据运营策略发送续期的提醒消息。

第二种，我们可以使用搜索引擎，比如 Solr、或者 Elasticsearch。

把会员表里面的会员 id 和会员到期时间存储一份到搜索引擎中。

搜索引擎的优势在于大数据量的快速检索，并且具有高可扩展性和高可靠性，非常适合大规模数据的处理。

第三种，可以使用 Redis 来实现。

用户开通会员以后，在 Redis 里面存储这个会员 id，以及设置这个 id 的过期时间。

然后可以使用 redis 的过期提醒功能，

把配置项 notify-keyspace-events 改为 notify-keyspace-events "Ex"

当 Redis 里面的 key 过期以后，会触发一个 key 过期事件，我们可以在应用程序中监听这个事件来处理。

第四种，可以直接使用 MQ 提供的延迟队列，当用户开通会员以后，直接计算这个会员的过期时间，然后发送一个延迟消息到 MQ 上，一旦消息达到过期时间，消费者就可以消费这个消息来触发会员过期的提醒。

以上就是我的一些回答思路，如果大家还有更好的思路，欢迎在评论区留言。

53.74 [滴滴]binlog 和 redolog 有什么区别?

昨天，一个工作 3 年的同学好不容易约到一个面试，却被这个问题难倒了。

“binlog 和 redolog 有什么区别？”

关于这个问题的回答，我把它整理到了 40W 字的面试文档里面，大家可以去评论区置顶中领取。

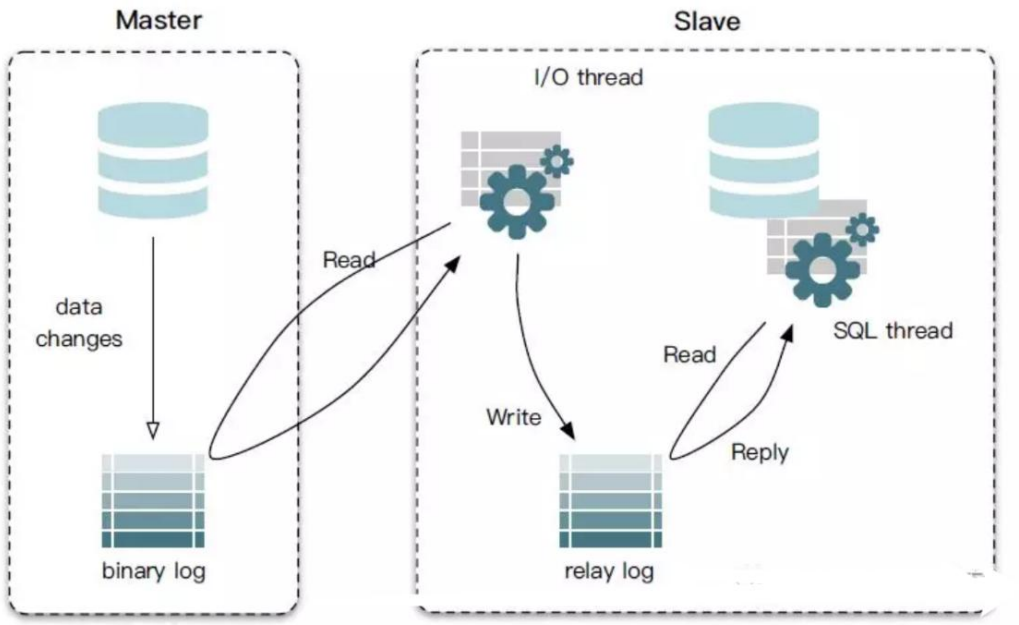
53.74.1 问题解析

这个问题其实比较简单，但凡稍微了解过 Mysql 数据库，都很容易回答出来。

一般考察 1~3 年的程序员，所以对于这个工作年限的小伙伴要注意了解它们的原理。

首先，binlog 和 redolog 都是 Mysql 里面用来记录数据库数据变更操作的日志。

{如图}其中 binlog 主要用来做数据备份、数据恢复和数据同步，大家初步接触这个概念 ，应该是在 Mysql 的主从数据同步的场景中，master 节点的数据变更，会写入到 binlog 中，然后再把 binlog 中的数据通过网络传输给 slave 节点，实现数据同步。



而 redolog，主要是在 Mysql 数据库事务的 ACID 特性里面，用来保证数据的持久化特性。但是其实它还有很多的作用。

比如数据库崩溃时，可以通过 Redo Log 来恢复未完成的数据，保证数据的完整性。

通过合理的配置 Redo Log 的大小和数量，还可以优化 Mysql 的性能。

那下面来看下这个面试题的回答吧。

53.74.2 问题答案

binlog 和 redolog 的区别有很多，我可以简单总结三个点

8. 使用场景不同，binlog 主要用来做数据备份、数据恢复、以及主从集群的数据同步；Redo Log 主要用来实现 Mysql 数据库的事务恢复，保证事务的 ACID 特性。当数据库出现崩溃的时候，Redo Log 可以把未提交的事务回滚，把已提交的事务进行持久化，从而保证数据的一致性和持久性。
9. 记录的信息不同，binlog 是记录数据库的逻辑变化，它提供了三种日志格式分别是 **statement，row 以及 mixed**；

redo log 记录的是物理变化，也就是数据页的变化结果。

10. 记录的时机不同， binlog 是在执行 SQL 语句的时候，在主线程中生成逻辑变化写入到磁盘中，所以它是语句级别的记录方式； RedoLog 是在 InnoDB 存储引擎层面的操作，它是在 Mysql 后台线程中生成并写入到磁盘中的，所以它是事务级别的记录方式，一个事务操作完成以后才会被写入到 redo log 中。

以上就是我的理解！

53.75 [滴滴]说说 Spring MVC 的执行流程?

昨天，一个工作 2 年的粉丝在面试的时候，面试官要求他说 Spring MVC 的执行流程。他没回答上来，错过了这个 offer。

这个问题在我之前整理的大厂面试指南里面，有标准的回答，大家可以去评论区置顶中领取。

53.75.1 问题解析

Spring MVC 的执行流程，一个面试频率超级高的问题，但是缺难倒了无数的程序员。这个问题的考察范围主要是 3~5 年，甚至 5 年以上都会问到。和它同类型的问题还有 Bean 的加载过程、IOC 的原理等。

由于 Spring MVC 是基于 Jsp/Servlet 之上做的封装，并为开发人员提供了 MVC 的分层设计以及更加友好的开发模型。

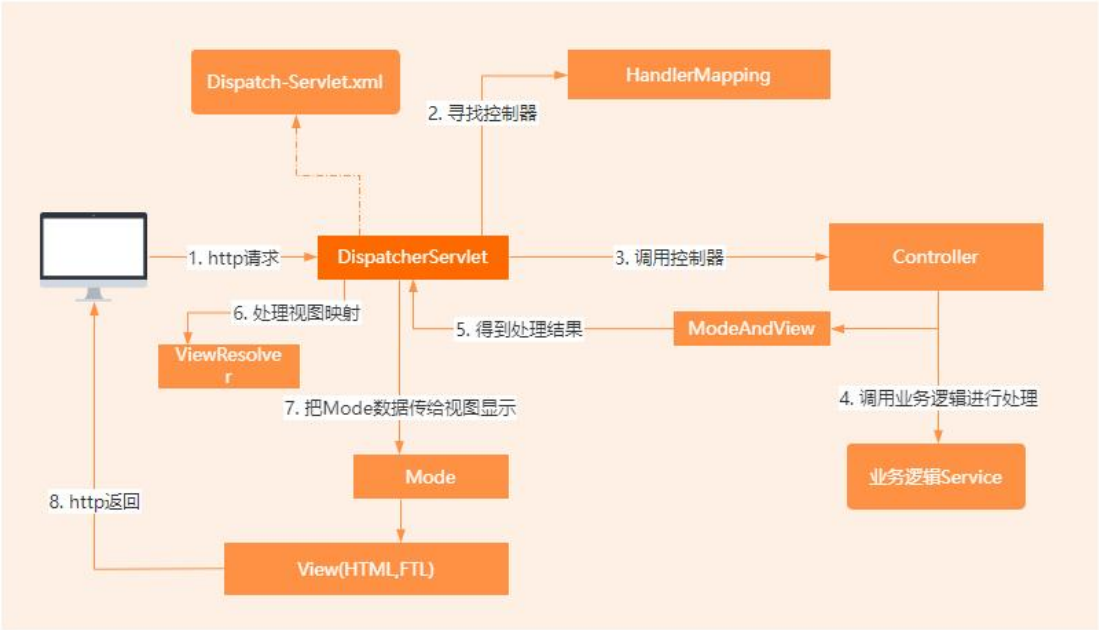
所以我认为大家都应该去把 Spring MVC 的整个执行流程梳理一遍。

53.75.2 问题答案

Spring MVC 的工作流程可以分为几个步骤

1. 用户发起请求，请求先被 Servlet 拦截转发给 Spring MVC 框架
2. Spring MVC 里面的 DispatcherSerlvet 核心控制器，会接收到请求并转发给 HandlerMapping
3. HandlerMapping 负责解析请求，根据请求信息和配置信息找到匹配的 Controller 类，不过这里如果有配置拦截器，就会按照顺序执行拦截器里面的 preHandle 方法
4. 找到匹配的 Controller 以后，把请求参数传递给 Controller 里面的方法
5. Controller 中的方法执行完以后，会返回一个 ModeAndView，这里面会包括视图名称和需要传递给视图的模型数据
6. 视图解析器根据名称找到视图，然后把数据模型填充到视图里面再渲染成 Html 内容返回给客户端

以上就是我的理解！



53.76 入职一家新公司，如何快速上手开发

跳槽到一家新公司后，

如何以最快的速度融入团队协作

上手项目开发?

今天给大家分享的干货，

每个人都用得上

还是老规矩

点赞收藏，然后反复观看

第一点，了解业务背景

做开发千万不能眼里只有技术细节，

技术只能告诉你怎么做，

而产品、业务逻辑则是告诉你做什么

进入新公司以后，

首先要搞清楚

新公司的产品功能以及业务逻辑

公司产品的用户是谁、

提供的核心功能是什么、

系统在上下游的地位。

带着这些问题，

去熟悉产品的核心功能，

对项目有一个全局性把控。

而了解业务逻辑，

本质上是了解这个业务背后的运转模式，

基于运转模式进行深入。

只有在理解业务逻辑的基础上做出的技术方案，

才有价值和针对性。

在了解产品和业务逻辑后，

我们还可以站在行业的高度对竞品有一定的了解

通过阅读一些行业报告来加深对公司业务未来发展趋势的认知

第二点，了解技术栈

刚入职，

不用太着急下沉到技术细节里去。

可以先从宏观上了解技术。

比如当前的技术栈、使用的语言、明确使用的工具和框架，

然后对照业务模型，

对系统功能进行拆解。

系统对外提供的接口有哪些?

依赖的内部或者外部系统的接口有哪些?

采用的是通信方式、通信协议分别是什么?

其次，

项目内部分哪几个模块，

每个模块的职责，

最后是熟悉代码，

目录结构，配置文件的位置，业务功能在源码里面的体现；

有了宏观的了解之后，

可以针对接口的重要程度，

从头到尾绘制一下接口的时序图，顺序图，

加深对核心业务的理解。

第三点，从小需求开始，尝试编码

很多小伙伴加入新公司后，

觉得自己对项目还不了解，

不着急做需求，

一直处于准备状态。

但是，

站在岸上，

永远学不会游泳。

只有带着具体问题去实践，

逐步踩坑，才能快速上手；

例如在本地完成工程的搭建后，

针对代码进行 debug，
了解每步实现的处理逻辑和数据变化，
从而完全掌握这一个小模块或功能点的实现。

接下来， 就可以接一些小的需求进行开发了
多做几个小需求后，
对应用的实现细节就会越来越熟悉

刚开始做的小需求不用求快，
而是以规范为主。
做完之后有了成就感，
对新入职的你也是一种正向激励。

第四点，也是最重要的一点，具备 owner 的心态

在新公司接手一个系统后，
就需要以 owner 的心态来对待。
有些小伙伴习惯性的事无巨细都请教老员工，
缺少了一份独立思考，
成长的速度就会慢一些。

遇到疑问首先有自己的判断，
不论判断的是否正确，
经过一次思考后，
对系统的理解将会上一个台阶。
在熟悉系统的过程中，
可以多问一下，
如果是我来设计这个项目，
或者由我来实现这个功能，

应该怎么做。

而且现有的项目可能由于历史原因，
并不是以最优方式实现，
对比自己期望的做法，
可以快速了解到系统这样做的深层原因。

通过一次次对自己的追问中，
更快的理解系统的深层背景，
同时增加自身的设计能力。
而自己原有的技能，
也能更好的反馈在新的项目当中。
我是 Mic
一个工作了 14 年的程序员
我们下期再见

53.77 [滴滴]谈谈你对 AQS 的理解

AQS 是 AbstractQueuedSynchronizer 的简称，是并发编程中比较核心的组件。
在很多大厂的面试中，面试官对于并发编程的考核要求相对较高，简单来说，如果你不懂并发编程，那么你很难通过大厂高薪岗位的面试。

Hello，大家好， 我是 Mic， 一个工作了 14 年的程序员，今天来和大家聊聊并发编程中的 AQS 组件。
我们来看一下，关于“谈谈你对 AQS 的理解 “，看看普通人和高手是如何回答的！

> 以下是高手的回答

AQS 是多线程同步器，它是 J.U.C 包中多个组件的底层实现，如 Lock、CountDownLatch、Semaphore 等都用到了 AQS.

从本质上来说，AQS 提供了两种锁机制，分别是排它锁，和 共享锁。

排它锁，就是存在多线程竞争同一共享资源时，同一时刻只允许一个线程访问该共享资源，也就是多个线程中只能有一个线程获得锁资源，比如 Lock 中的 ReentrantLock 重入锁实现就是用到了 AQS 中的排它锁功能。

共享锁也称为读锁，就是在同一时刻允许多个线程同时获得锁资源，比如 CountDownLatch 和 Semaphore 都是用到了 AQS 中的共享锁功能。

53.78 [滴滴]fail-safe 机制与 fail-fast 机制分别有什么作用

前段时间一个小伙伴去面试，遇到这样一个问题。

” fail-safe 机制与 fail-fast 机制分别有什么作用 “

他说他听到这个问题的时候，脑子里满脸问号。那么今天我们来看一下，关于这个问题，普通人和高手应该如何回答吧。

> 以下是高手的回答

fail-safe 和 fail-fast ，是多线程并发操作集合时的一种失败处理机制。

Fail-fast ： 表示快速失败，在集合遍历过程中，一旦发现容器中的数据被修改了，会立刻抛出 ConcurrentModificationException 异常，从而导致遍历失败，像这种情况（贴下面这个图）。

定义一个 Map 集合，使用 Iterator 迭代器进行数据遍历，在遍历过程中，对集合数据做变更时，就会发生 `fail-fast`。

java.util 包下的集合类都是快速失败机制的，常见的的使用 fail-fast 方式遍历的容器有 HashMap 和 ArrayList 等。

```java
    public static void main(String[] args) {
        Map<String, String> empName = new HashMap<String, String>();
        empName.put("name", "mic");
        empName.put("sex", "male");
        empName.put("age", "18");
        Iterator iterator = empName.keySet().iterator();
        while (iterator.hasNext()) {
            System.out.println(empName.get(iterator.next()));
            empName.put("work", "Java");
        }
    }

>上述程序运行结果如下:
  male
  Exception in thread "main" java.util.ConcurrentModificationException
     at java.util.HashMap$HashIterator.nextNode(HashMap.java:1445)
     at java.util.HashMap$KeyIterator.next(HashMap.java:1469)
     at org.example.cl09.ThreadExample.main(ThreadExample.java:14)
```

Fail-safe，表示失败安全，也就是在这种机制下，出现集合元素的修改，不会抛出ConcurrentModificationException。

原因是采用安全失败机制的集合容器，在遍历时不是直接在集合内容上访问的，而是先复制原有集合内容，

在拷贝的集合上进行遍历。由于迭代时是对原集合的拷贝进行遍历，所以在遍历过程中对原集合所作的修改并不能被迭代器检测到

比如这种情况（贴下面这个图），定义了一个 CopyOnWriteArrayList，在对这个集合遍历过程中，对集合元素做修改后，不会抛出异常，但同时也不会打印出增加的元素。

java.util.concurrent 包下的容器都是安全失败的,可以在多线程下并发使用,并发修改。

常见的的使用 fail-safe 方式遍历的容器有 ConcerrentHashMap 和 CopyOnWriteArrayList 等。

```java
public static void main(String[] args) {
    CopyOnWriteArrayList<Integer> list
            = new CopyOnWriteArrayList<>(new Integer[] { 1, 7, 9, 11 });
    Iterator itr = list.iterator();
    while (itr.hasNext()) {
        Integer i = (Integer)itr.next();
        System.out.println(i);
        if (i == 7)
            list.add(15); // 在fail-safe模式下，这里不会被打印
    }
}
```

53.79 [滴滴]谈谈你对 Seata 的理解

很多面试官都喜欢问一些“谈谈你对 xxx 技术的理解”，

大家遇到这种问题时，是不是完全不知道从何说起，有同感小伙伴的 call 1.

那么我们来看一下，普通人和高手是如何回答这个问题的?

以下是高手的回答

1. 在微服务架构下，由于数据库和应用服务的拆分，导致原本一个事务单元中的多个 DML 操作，

变成了跨进程或者跨数据库的多个事务单元的多个 DML 操作，

而传统的数据库事务无法解决这类的问题，所以就引出了分布式事务的概念。

2. 分布式事务本质上要解决的就是跨网络节点的多个事务的数据一致性问题，业内常见的解决方法有两种
    a. 强一致性，就是所有的事务参与者要么全部成功，要么全部失败，全局事务协调者需要知道每个事务参与者的执行状态，再根据状态来决定数据的提交或者回滚!
    b. 最终一致性，也叫弱一致性，也就是多个网络节点的数据允许出现不一致的情况，但是在最终的某个时间点会达成数据一致。

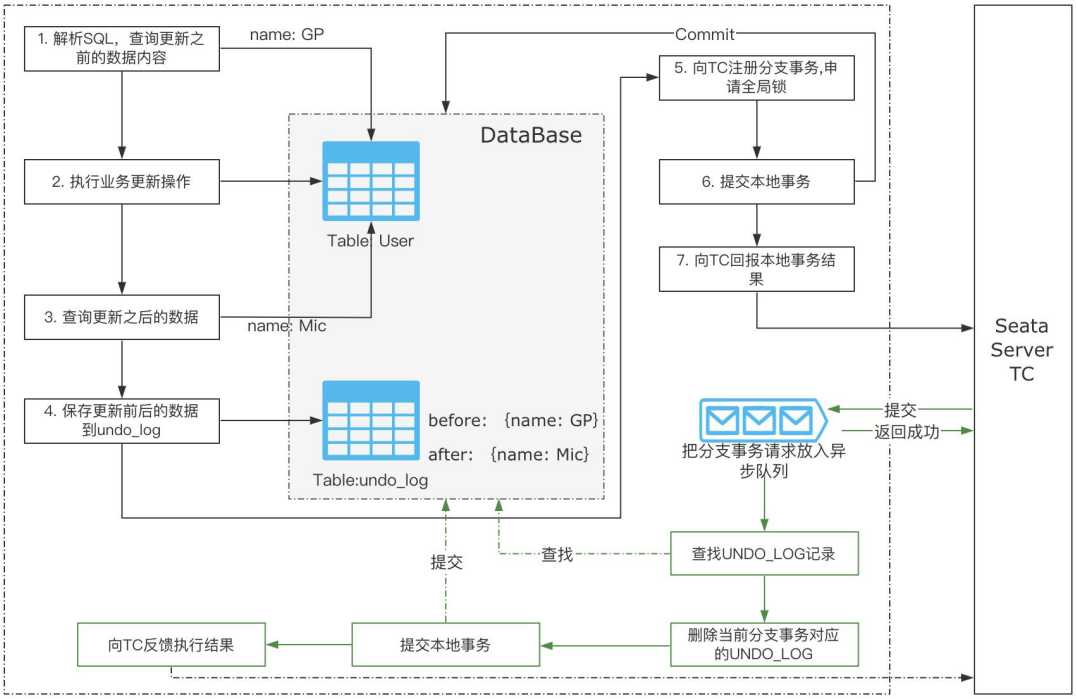
基于 CAP 定理我们可以知道，强一致性方案对于应用的性能和可用性会有影响，所以对于数据一致性要求不高的场景，就会采用最终一致性算法。

3. 在分布式事务的实现上，对于强一致性，我们可以通过基于 XA 协议下的二阶段提交来实现，对于弱一致性，可以基于 TCC 事务模型、可靠性消息模型等方案来实现。
4. 市面上有很多针对这些理论模型实现的分布式事务框架，我们可以在应用中集成这些框架来实现分布式事务。

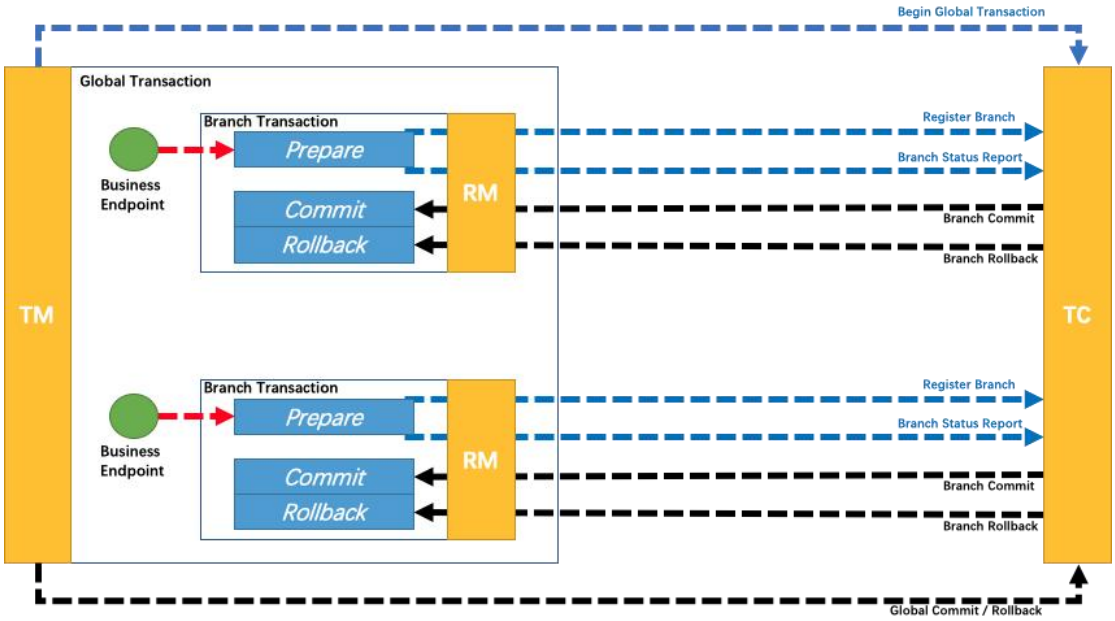
而 Seata 就是其中一种，它是阿里开源的分布式事务解决方案，提供了高性能且简单易用的分布式事务服务。

Seata 中封装了四种分布式事务模式，分别是：

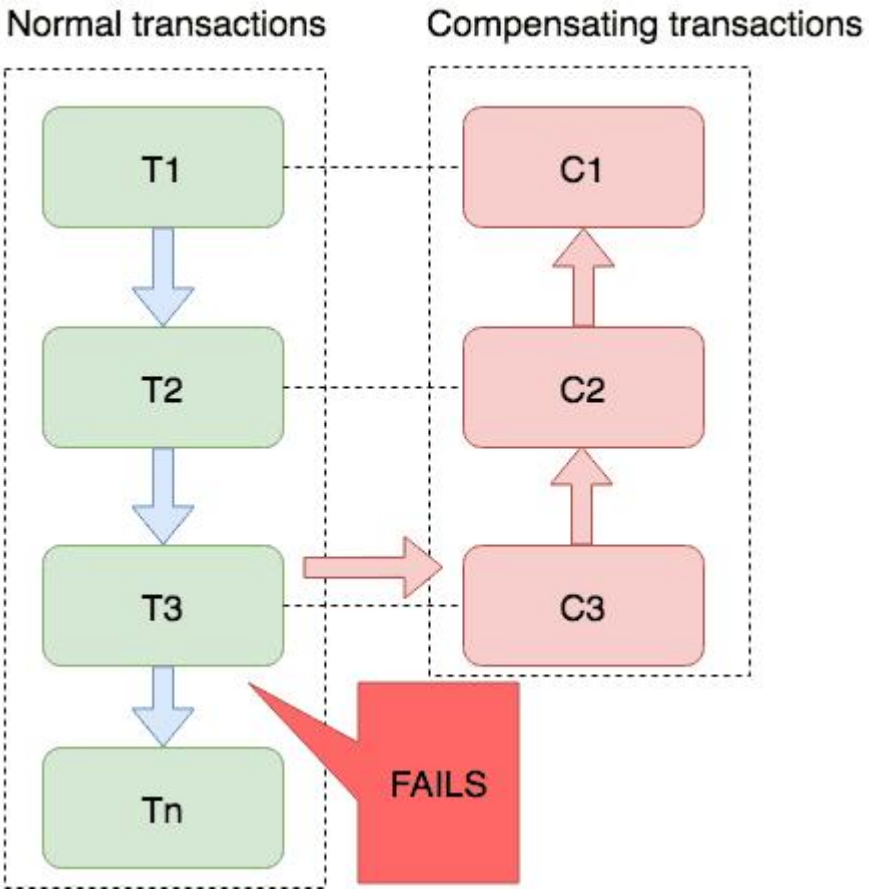
- AT 模式，是一种基于本地事务+二阶段协议来实现的最终数据一致性方案，也是 Seata 默认的解决方案



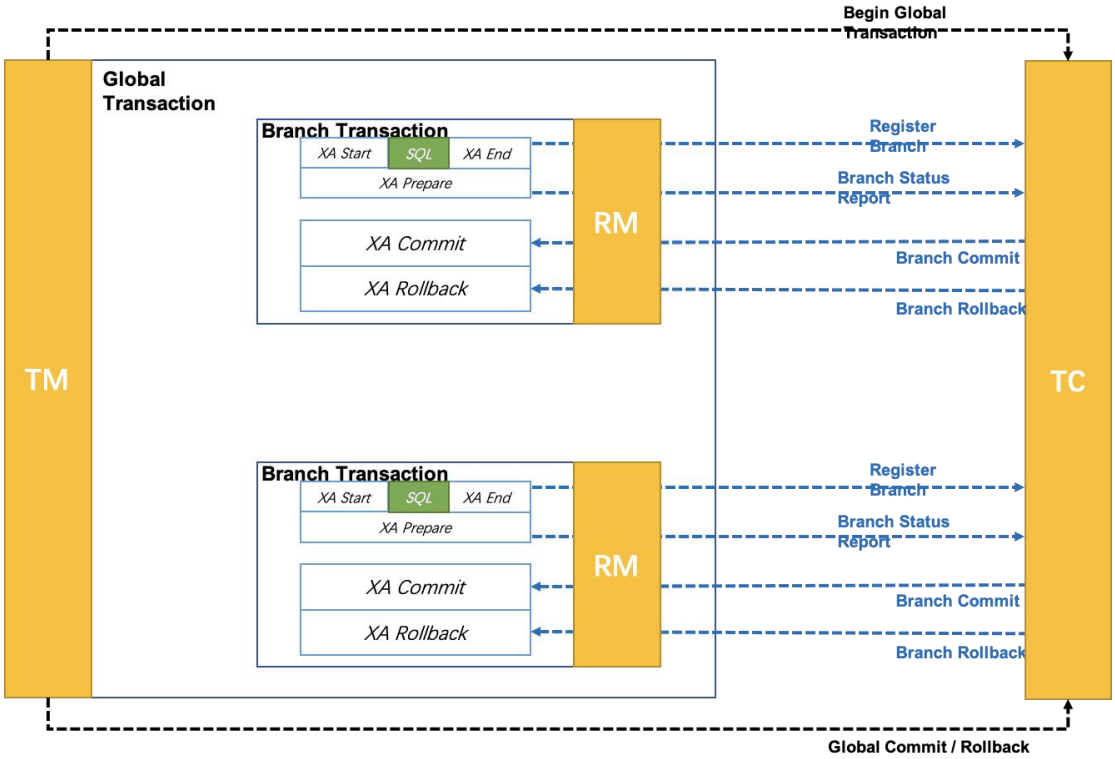
- TCC 模式，TCC 事务是 Try、Confirm、Cancel 三个词语的缩写，简单理解就是把一个完整的业务逻辑拆分成三个阶段，然后通过事务管理器在业务逻辑层面根据每个分支事务的执行情况分别调用该业务的 Confirm 或者 Cacel 方法。



- Saga 模式，Saga 模式是 SEATA 提供的长事务解决方案，在 Saga 模式中，业务流程中每个参与者都提交本地事务，当出现某一个参与者失败则补偿前面已经成功的参与者。



- XA 模式，XA 可以认为是一种强一致性的事务解决方法，它利用事务资源（数据库、消息服务等）对 XA 协议的支持，以 XA 协议的机制来管理分支事务的一种 事务模式。



从这四种模型中不难看出，在不同的业务场景中，我们可以使用 Seata 的不同事务模型来解决不同业务场景中的分布式事务问题，因此我们可以认为 Seata 是一个一站式的分布式事务解决方案。

53.80 [滴滴]Spring Boot 的约定优于配置，你的理解是什么?

对于 Spring Boot 约定优于配置这个问题，看看高手是如何回答的?

以下是高手的回答

我从 4 个点方面来回答。

1. 首先， 约定优于配置是一种软件设计的范式，它的核心思想是减少软件开发人员对于配置项的维护，从而让开发人员更加聚焦在业务逻辑上。

2. Spring Boot 就是约定优于配置这一理念下的产物，它类似于 Spring 框架下的一个脚手架，通过 Spring Boot，我们可以快速开发基于 Spring 生态下的应用程序。
3. 基于传统的 Spring 框架开发 web 应用，我们需要做很多和业务开发无关并且只需要做一次的配置，比如
    a. 管理 jar 包依赖
    b. web.xml 维护
    c. Dispatch-Servlet.xml 配置项维护
    d. 应用部署到 Web 容器
    e. 第三方组件集成到 Spring IOC 容器中的配置项维护

而在 Spring Boot 中，我们不需要再去做这些繁琐的配置，Spring Boot 已经自动帮我们完成了，这就是约定由于配置思想的体现。

4. Spring Boot 约定由于配置的体现有很多，比如
    a. Spring Boot Starter 启动依赖，它能帮我们管理所有 jar 包版本
    b. 如果当前应用依赖了 spring mvc 相关的 jar，那么 Spring Boot 会自动内置 Tomcat 容器来运行 web 应用，我们不需要再去单独做应用部署。
    c. Spring Boot 的自动装配机制的实现中，通过扫描约定路径下的 spring.factories 文件来识别配置类，实现 Bean 的自动装配。
    d. 默认加载的配置文件 application.properties 等等。

总的来说，约定优于配置是一个比较常见的软件设计思想，它的核心本质都是为了更高效以及更便捷的实现软件系统的开发和维护。

以上就是我对这个问题的理解。

53.81 滴滴二面：kafka 的零拷贝原理?

最近一个学员去滴滴面试，在第二面的时候遇到了这个问题：

”请你简单说一下 Kafka 的零拷贝原理“

然后那个学员努力在大脑里检索了很久，没有回答上来。

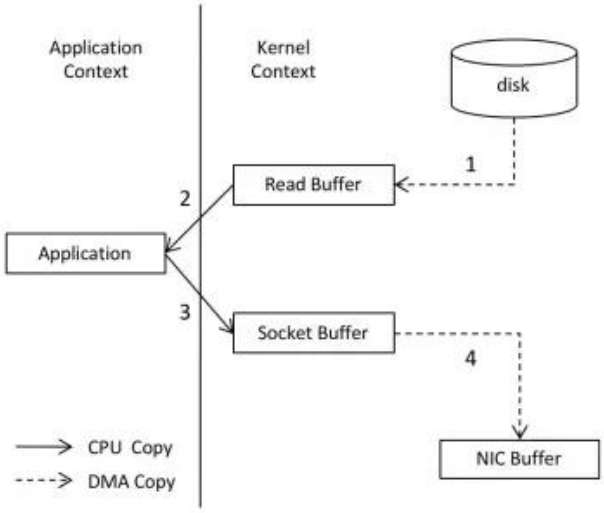
那么今天，我们基于这个问题来看看，高手是如何回答的！

> 以下是高手的回答

在实际应用中，如果我们需要把磁盘中的某个文件内容发送到远程服务器上，如图

那么它必须要经过几个拷贝的过程，如图（贴图）。

1. 从磁盘中读取目标文件内容拷贝到内核缓冲区
2. CPU 控制器再把内核缓冲区的数据赋值到用户空间的缓冲区中
3. 接着在应用程序中，调用 write()方法，把用户空间缓冲区中的数据拷贝到内核下的 Socket Buffer 中。
4. 最后，把在内核模式下的 SocketBuffer 中的数据赋值到网卡缓冲区（NIC Buffer)
5. 网卡缓冲区再把数据传输到目标服务器上。



在这个过程中我们可以发现，数据从磁盘到最终发送出去，要经历 4 次拷贝，而在这四次拷贝过程中，有两次拷贝是浪费的，分别是：

6. 从内核空间赋值到用户空间
7. 从用户空间再次复制到内核空间

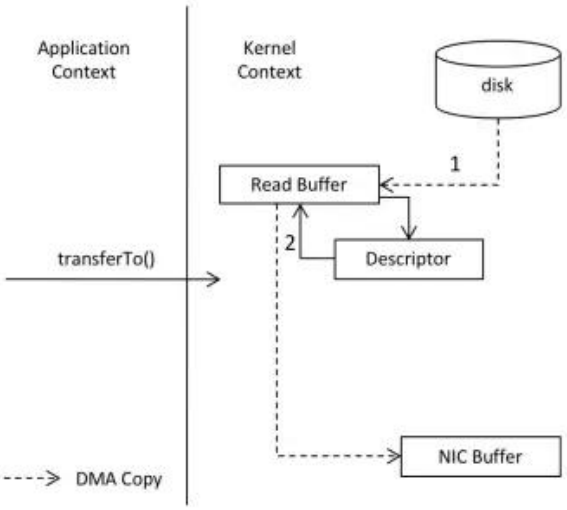
除此之外，由于用户空间和内核空间的切换会带来 CPU 的上线文切换，对于 CPU 性能也会造成性能影响。

而零拷贝，就是把这两次多于的拷贝省略掉，应用程序可以直接把磁盘中的数据从内核中直接传输给 Socket，而不需要再经过应用程序所在的用户空间，如下图所示。

零拷贝通过 DMA(Direct Memory Access)技术把文件内容复制到内核空间中的 Read Buffer，

接着把包含数据位置和长度信息的文件描述符加载到 Socket Buffer 中，DMA 引擎直接可以把数据从内核空间中传递给网卡设备。

在这个流程中，数据只经历了两次拷贝就发送到了网卡中，并且减少了 2 次 cpu 的上下文切换，对于效率有非常大的提高。



所以，所谓零拷贝，并不是完全没有数据赋值，只是相对于用户空间来说，不再需要进行数据拷贝。对于前面说的整个流程来说，零拷贝只是减少了不必要的拷贝次数而已。

在程序中如何实现零拷贝呢?

- 在 Linux 中，零拷贝技术依赖于底层的 sendfile()方法实现
- 在 Java 中，FileChannal.transferTo() 方法的底层实现就是 sendfile() 方法。

除此之外，还有一个 mmap 的文件映射机制

它的原理是：将磁盘文件映射到内存，用户通过修改内存就能修改磁盘文件。使用这种方式可以获取很大的 I/O 提升，省去了用户空间到内核空间复制的开销。

以上就是我对于 Kafka 中零拷贝原理的理解

53.82 [滴滴]innoDB 如何解决幻读

前天有个去快手面试的小伙伴私信我，他遇到了这样一个问题： “InnoDB 如何解决幻读”？

这个问题确实不是很好回答，在实际应用中，很多同学几乎都不关注数据库的事务隔离性。

所有问题基本就是 CRUD，一把梭~

那么今天，我们看一下 关于“InnoDB 如何解决幻读”这个问题，普通人和高手的回答!

以下是高手的回答

我会从三个方面来回答:

1、 Mysql 的事务隔离级别

Mysql 有四种事务隔离级别，这四种隔离级别代表当存在多个事务并发冲突时，可能出现的脏读、不可重复读、幻读的问题。

其中 InnoDB 在 RR 的隔离级别下，解决了幻读的问题。

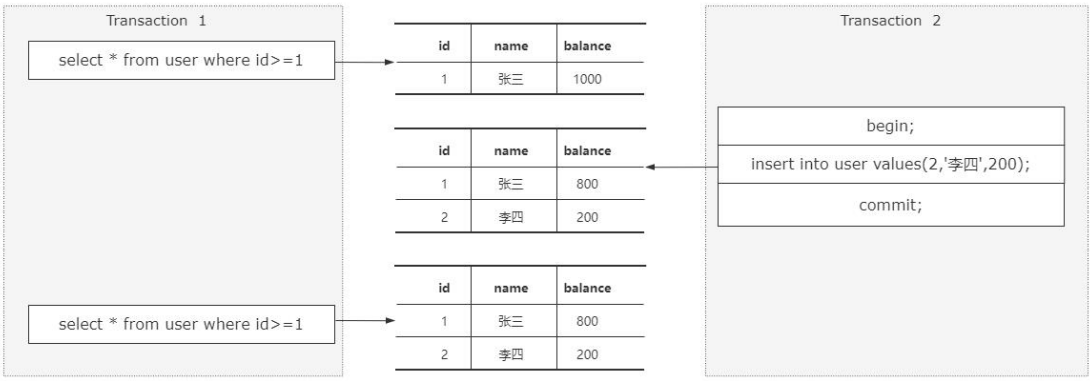
事务隔离级别	脏读	不可重复读	幻读
未提交读（Read Uncommitted）	可能	可能	可能
已提交读（Read Committed）	不可能	可能	可能
可重复读（Repeatable Read）	不可能	不可能	对InnoDB不可能
串行化（Serializable）	不可能	不可能	不可能

2、 **什么是幻读?**

那么， 什么是幻读呢?

幻读是指在同一个事务中，前后两次查询相同的范围时，得到的结果不一致（我们来看这个图）

- 第一个事务里面我们执行了一个范围查询，这个时候满足条件的数据只有一条
- 第二个事务里面，它插入了一行数据，并且提交了
- 接着第一个事务再去查询的时候，得到的结果比第一查询的结果多出来了一条数据。

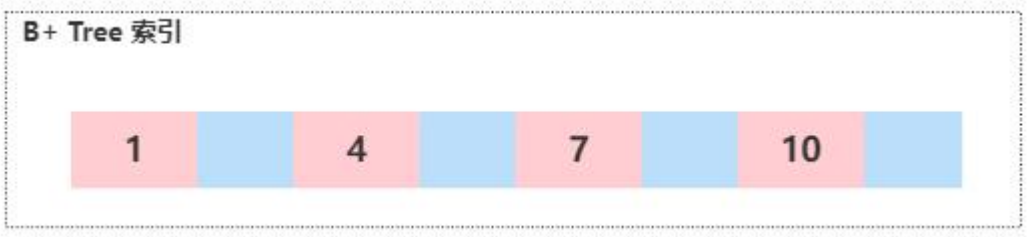


所以，幻读会带来数据一致性问题。

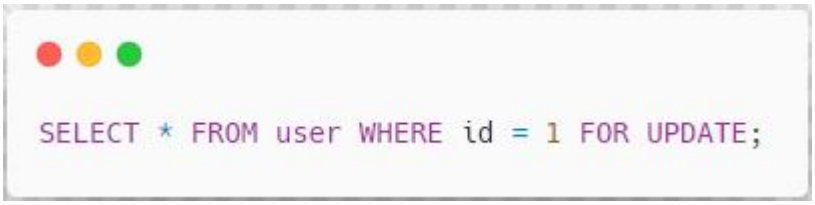
3、 InnoDB 如何解决幻读的问题

InnoDB 引入了间隙锁和 next-key Lock 机制来解决幻读问题，为了更清晰的说明这两种锁，我举一个例子：

假设现在存在这样（图片）这样一个 B+ Tree 的索引结构，这个结构中有四个索引元素分别是：1、4、7、10。

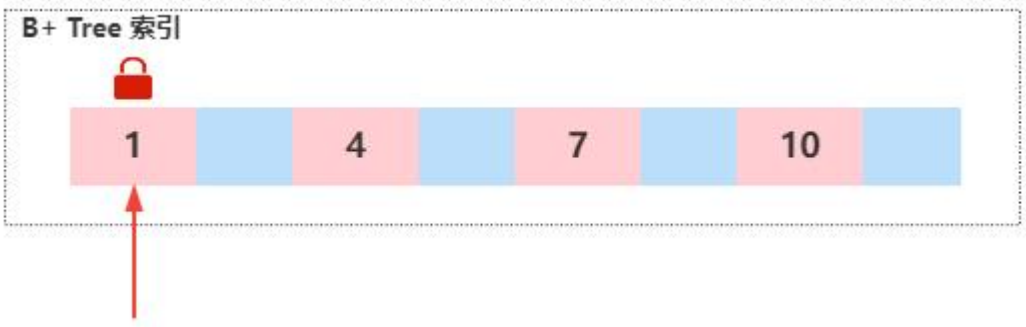


当我们通过主键索引查询一条记录，并且对这条记录通过 `for update` 加锁（请看这个图片）



```
SELECT * FROM user WHERE id = 1 FOR UPDATE;
```

这个时候，会产生一个记录锁，也就是行锁，锁定 `id=1` 这个索引（请看这个图片）。



被锁定的记录在锁释放之前，其他事务无法对这条记录做任何操作。

前面我说过对幻读的定义： 幻读是指在同一个事务中，前后两次查询相同的范围时，得到的结果不一致！

注意，这里强调的是范围查询，

也就是说，InnoDB 引擎要解决幻读问题，必须要保证一个点，就是如果一个事务通过这样一条语句（如图）进行锁定时。

```
SELECT * FROM user WHERE id >4 and id<7 FOR UPDATE;
```

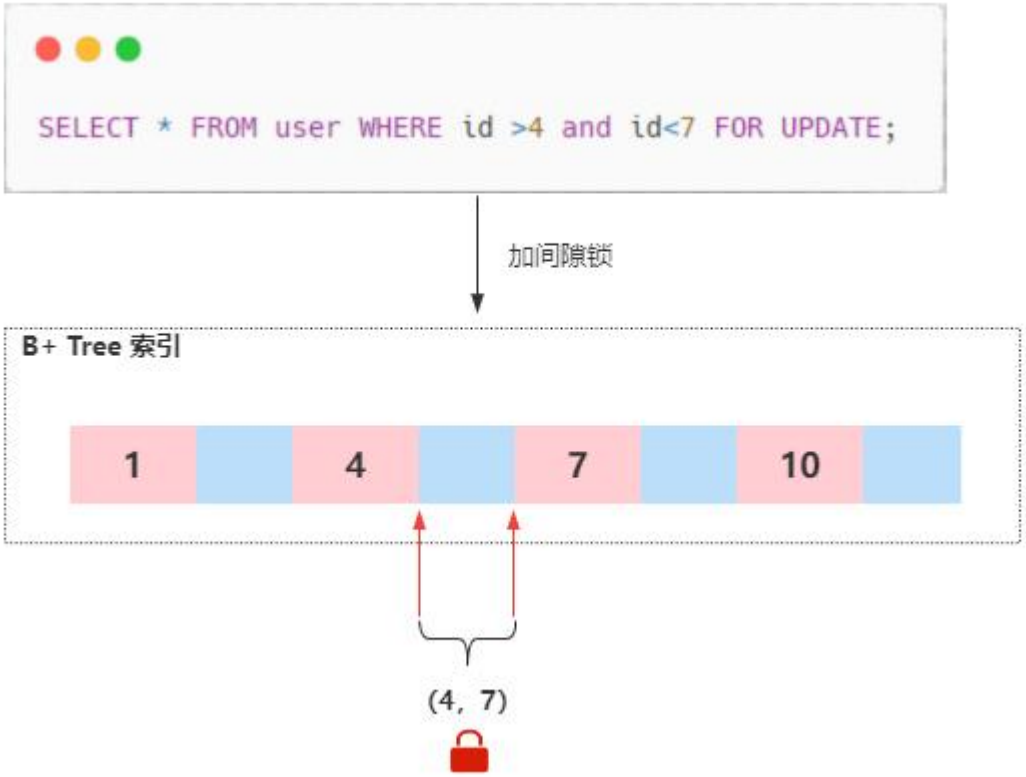
另外一个事务再执行这样一条（显示图片）insert 语句，需要被阻塞，直到前面获得锁的事务释放。

```
INSERT INTO user(id,name) VALUES(5,'mimi');
```

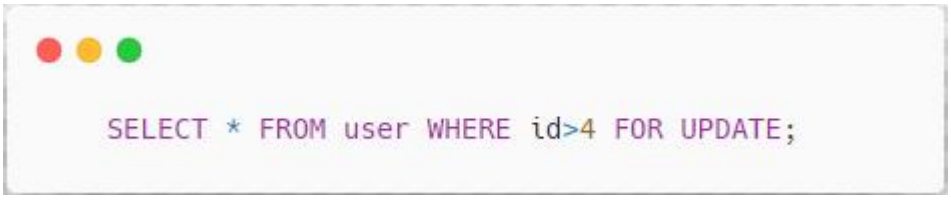
所以，在 InnoDB 中设计了一种间隙锁，它的主要功能是锁定一段范围内的索引记录（如图）

当对查询范围 id>4 and id <7 加锁的时候，会针对 B+树中（4，7）这个开区间范围的索引加间隙锁。

意味着在这种情况下，其他事务对这个区间的数据进行插入、更新、删除都会被锁住。

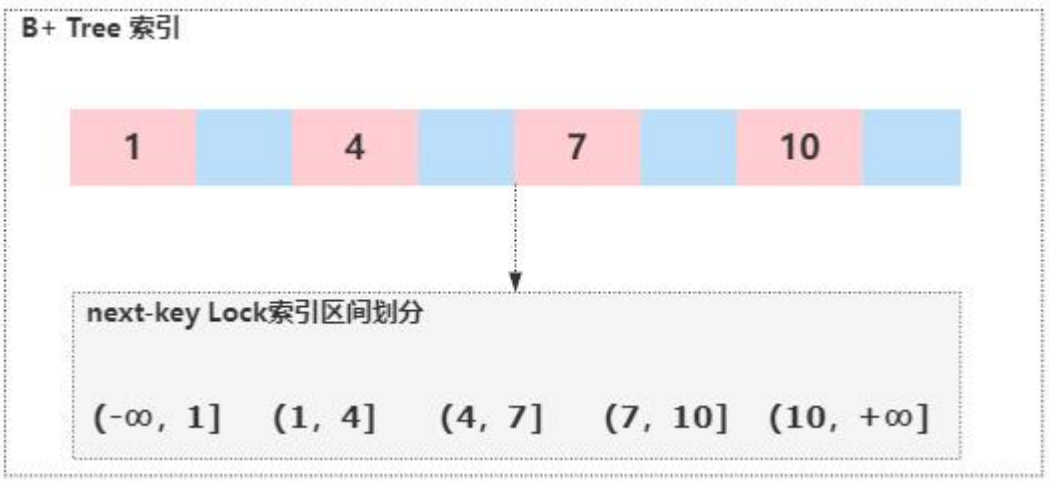


但是，还有另外一种情况，比如像这样（图片）



这条查询语句是针对 id>4 这个条件加锁，那么它需要锁定多个索引区间，所以在这种情况下 InnoDB 引入了 next-key Lock 机制。

next-key Lock 相当于间隙锁和记录锁的合集，记录锁锁定存在的记录行，间隙锁锁住记录行之间的间隙，而 next-key Lock 锁住的是两者之和。（如图所示）



每个数据行上的非唯一索引列上都会存在一把 **next-key lock**，当某个事务持有该数据行的 **next-key lock** 时，会锁住一段**左开右闭区间**的数据。

因此，当通过 id>4 这样一种范围查询加锁时，会加 next-key Lock，锁定的区间范围是：(4, 7] , (7,10],(10,+∞]

间隙锁和 next-key Lock 的区别在于加锁的范围，间隙锁只锁定两个索引之间的引用间隙，而 next-key Lock 会锁定多个索引区间，它包含记录锁和间隙锁。

当我们使用了范围查询，不仅仅命中了 Record 记录，还包含了 Gap 间隙，在这种情况下我们使用的就是临键锁，它是 MySQL 里面默认的行锁算法。

4 、总结

虽然 InnoDB 中通过间隙锁的方式解决了幻读问题，但是加锁之后一定会影响到并发性能，因此，如果对性能要求较高的业务场景中，可以把隔离级别设置成 RC，这个级别中不存在间隙锁。

以上就是我对于 innoDB 如何解决幻读问题的理解！

53.83 [滴滴]CPU 飙高系统反应慢怎么排查?

面试过程中，场景类的问题更容易检测出一个开发人员的基本能力。

这不，一个小伙伴去阿里面试，第一面就遇到了关于“CPU 飙高系统反应慢怎么排查”的问题？

对于这个问题，我们来看看高手的回答！

以下是高手的回答

好的，关于这个问题，我从四个方面来回答。

1. CPU 是整个电脑的核心计算资源，对于一个应用进程来说，CPU 的最小执行单元是线程。
2. 导致 CPU 飙高的原因有几个方面
    a. CPU 上下文切换过多，对于 CPU 来说，同一时刻下每个 CPU 核心只能运行一个线程，如果有多个线程要执行，CPU 只能通过上下文切换的方式来执行不同的线程。上下文切换需要做两个事情
        i. 保存运行线程的执行状态
        ii. 让处于等待中的线程执行

这两个过程需要 CPU 执行内核相关指令实现状态保存，如果较多的上下文切换会占据大量 CPU 资源，从而使得 cpu 无法去执行用户进程中的指令，导致响应速度下降。

在 Java 中，文件 IO、网络 IO、锁等待、线程阻塞等操作都会造成线程阻塞从而触发上下文切换

    b. CPU 资源过度消耗，也就是在程序中创建了大量的线程，或者有线程一直占用 CPU 资源无法被释放，比如死循环！

CPU 利用率过高之后，导致应用中的线程无法获得 CPU 的调度，从而影响程序的执行效率！

3. 既然是这两个问题导致的 CPU 利用率较高，于是我们可以通过 `top` 命令，找到 CPU 利用率较高的进程，在通过 `Shift+H` 找到进程中 CPU 消耗过高的线程，这里有两种情况。
    a. CPU 利用率过高的线程一直是同一个，说明程序中存在线程长期占用 CPU 没有释放的情况，这种情况直接通过 jstack 获得线程的 Dump 日志，定位到线程日志后就可以找到问题的代码。

b. CPU 利用率过高的线程 id 不断变化，说明线程创建过多，需要挑选几个线程 id，通过 jstack 去线程 dump 日志中排查。

4. 最后有可能定位的结果是程序正常，只是在 CPU 飙高的那一刻，用户访问量较大，导致系统资源不够。

以上就是我对这个问题的理解！

53.84 [滴滴]lock 和 synchronized 区别

今天来分享一道阿里一面的面试题，“lock 和 synchronized 的区别”。

对于这个问题，看看高手的回答！

> 以下是高手的回答

下面我从 4 个方面来回答

1. 从功能角度来看，Lock 和 Synchronized 都是 Java 中用来解决线程安全问题的工具。
2. 从特性来看，
   a. Synchronized 是 Java 中的同步关键字，Lock 是 J.U.C 包中提供的接口，这个接口有很多实现类，其中就包括 ReentrantLock 重入锁
   b. Synchronized 可以通过两种方式来控制锁的粒度，（贴图）

```java
//修饰在方法层面
public synchronized void sync(){

}

Object lock=new Object();
//修饰在代码块
public void sync(){
    synchronized(lock){

    }
}
```

一种是把 synchronized 关键字修饰在方法层面，

另一种是修饰在代码块上，并且我们可以通过 Synchronized 加锁对象的声明周期来控制锁的作用范围，比如锁对象是静态对象或者类对象，那么这个锁就是全局锁。

如果锁对象是普通实例对象，那这个锁的范围取决于这个实例的声明周期。

Lock 锁的粒度是通过它里面提供的 lock()和 unlock()方法决定的（贴图），包裹在这两个方法之间的代码能够保证线程安全性。而锁的作用域取决于 Lock 实例的生命周期。

```java
Lock lock=new ReentrantLock();

public void sync(){
    lock.lock();    //竞争锁
    //TODO 线程安全的代码
    lock.unlock();  //释放锁
}
```

c. Lock 比 Synchronized 的灵活性更高，Lock 可以自主决定什么时候加锁，什么时候释放锁，只需要调用 lock()和 unlock()这两个方法就行，同时 Lock 还提供了非阻塞的竞争锁方法 tryLock()方法，这个方法通过返回 true/false 来告诉当前线程是否已经有其他线程正在使用锁。

Synchronized 由于是关键字，所以它无法实现非阻塞竞争锁的方法，另外，Synchronized 锁的释放是被动的，就是当 Synchronized 同步代码块执行完以后或者代码出现异常时才会释放。

d. Lock 提供了公平锁和非公平锁的机制，公平锁是指线程竞争锁资源时，如果已经有其他线程正在排队等待锁释放，那么当前竞争锁资源的线程无法插队。而非公平锁，就是不管是否有线程在排队等待锁，它都会尝试去竞争一次锁。Synchronized 只提供了一种非公平锁的实现。

3. 从性能方面来看，Synchronized 和 Lock 在性能方面相差不大，在实现上会有一些区别，Synchronized 引入了偏向锁、轻量级锁、重量级锁以及锁升级的方式来优化加锁的性能，而 Lock 中则用到了自旋锁的方式来实现性能优化。

以上就是我对于这个问题的理解。

53.85 线程池如何知道一个线程的任务已经执行完成

一个小伙伴私信了一个小米的面试题，问题是： “线程池如何知道一个线程的任务已经执行完成”？

说实话，这个问题确实很刁钻，毕竟像很多工作 5 年多的小伙伴，连线程池都没用过，怎么可能回答出来这个问题呢?

下面我们来看看普通人和高手遇到这个问题的回答思路。

以下是高手的回答

好的，我会从两个方面来回答。

1. 在线程池内部，当我们把一个任务丢给线程池去执行，线程池会调度工作线程来执行这个任务的 run 方法，run 方法正常结束，也就意味着任务完成了。

所以线程池中的工作线程是通过同步调用任务的 run()方法并且等待 run 方法返回后，再去统计任务的完成数量。

2. 如果想在线程池外部去获得线程池内部任务的执行状态，有几种方法可以实现。

a. 线程池提供了一个 isTerminated()方法，可以判断线程池的运行状态，我们可以循环判断 isTerminated()方法的返回结果来了解线程池的运行状态，一旦线程池的运行状态是 Terminated，意味着线程池中的所有任务都已经执行完了。想要通过这个方法获取状态的前提是，程序中主动调用了线程池的 shutdown()方法。在实际业务中，一般不会主动去关闭线程池，因此这个方法在实用性和灵活性方面都不是很好。

b. 在线程池中，有一个 submit()方法，它提供了一个 Future 的返回值，我们通过 Future.get()方法来获得任务的执行结果，当线程池中的任务没执行完之前，future.get()方法会一直阻塞，直到任务执行结束。因此，只要 future.get()方法正常返回，也就意味着传入到线程池中的任务已经执行完成了！

c. 可以引入一个 CountDownLatch 计数器，它可以通过初始化指定一个计数器进行倒计时，其中有两个方法分别是 await()阻塞线程，以及 countDown()进行倒计时，一旦倒计时归零，所以被阻塞在 await()方法的线程都会被释放。

基于这样的原理，我们可以定义一个 CountDownLatch 对象并且计数器为 1，接着在线程池代码块后面调用 await()方法阻塞主线程，然后，当传入到线程池中的任务执行完成后，调用 countDown()方法表示任务执行结束。

最后，计数器归零 0，唤醒阻塞在 await()方法的线程。

```java
public static void main(String[] args) throws InterruptedException {
        ExecutorService executorService= Executors.newFixedThreadPool(10);
        CountDownLatch countDownLatch=new CountDownLatch(1);
        executorService.execute(new Runnable() {
            @Override
            public void run() {
                //开始执行任务
                try {
                    Thread.sleep(3000); //模拟任务执行时间
                    countDownLatch.countDown();  //任务执行结束后，计数器减1
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
            }
        });
        //阻塞main线程| 当任务执行结束调用countDown()方法使得计数器归零后，唤醒主线程。
        countDownLatch.await();
        executorService.shutdown();
    }
```

3. 基于这个问题，我简单总结一下，不管是线程池内部还是外部，要想知道线程是否执行结束，我们必须要获取线程执行结束后的状态，而线程本身没有返回值，所以只能通过阻塞-唤醒的方式来实现，future.get 和 CountDownLatch 都是这样一个原理。

以上就是我对于这个问题的回答！

53.86 [滴滴]HashMap 是怎么解决哈希冲突的?

常用数据结构基本上是面试必问的问题，比如 HashMap、LinkList、ConcurrentHashMap 等。

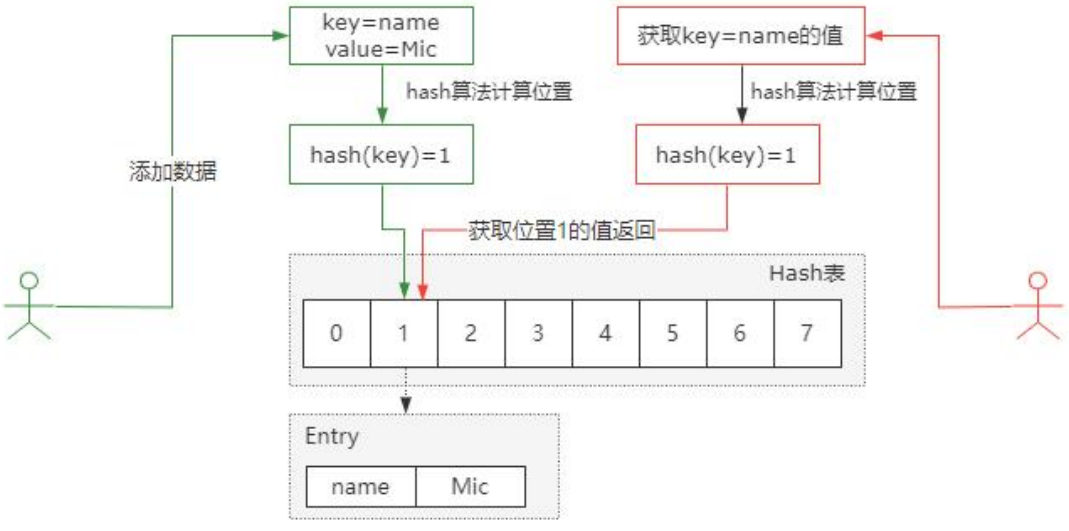
关于 HashMap，有个学员私信了我一个面试题说： “HashMap 是怎么解决哈希冲突的？”

关于这个问题，我们来模拟一下高手对于这个问题的回答。

以下是高手的回答

嗯，这个问题我从三个方面来回答。

1. 要了解 Hash 冲突，那首先我们要先了解 Hash 算法和 Hash 表。（如图）
    a. Hash 算法，就是把任意长度的输入，通过散列算法，变成固定长度的输出，这个输出结果是散列值。
    b. Hash 表又叫做“散列表”，它是通过 key 直接访问在内存存储位置的数据结构，在具体实现上，我们通过 hash 函数把 key 映射到表中的某个位置，来获取这个位置的数据，从而加快查找速度。

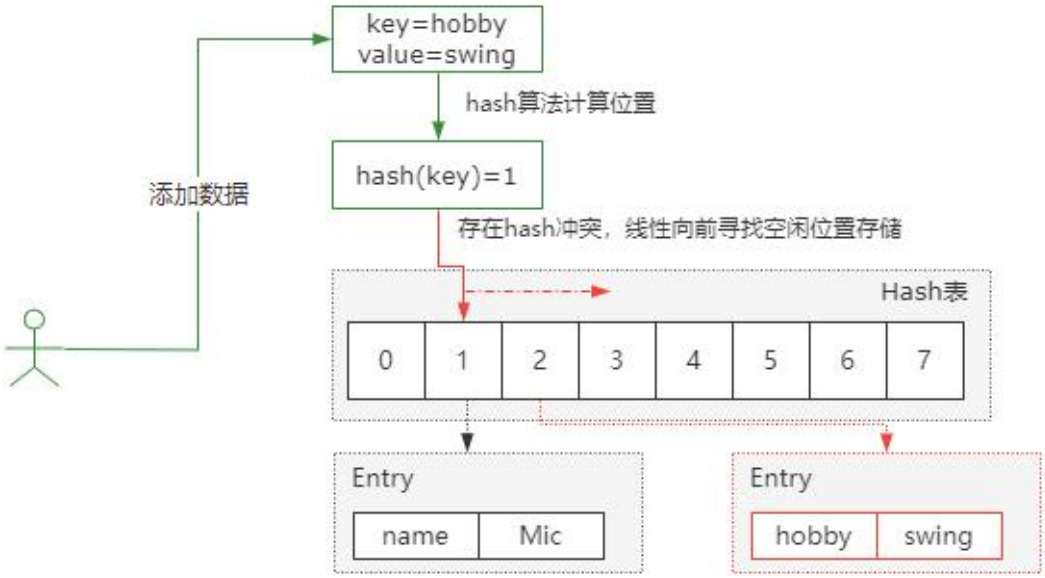


c. 所谓 hash 冲突，是由于哈希算法被计算的数据是无限的，而计算后的结果范围有限，所以总会存在不同的数据经过计算后得到的值相同，这就是哈希冲突。

d. 通常解决 hash 冲突的方法有 4 种。

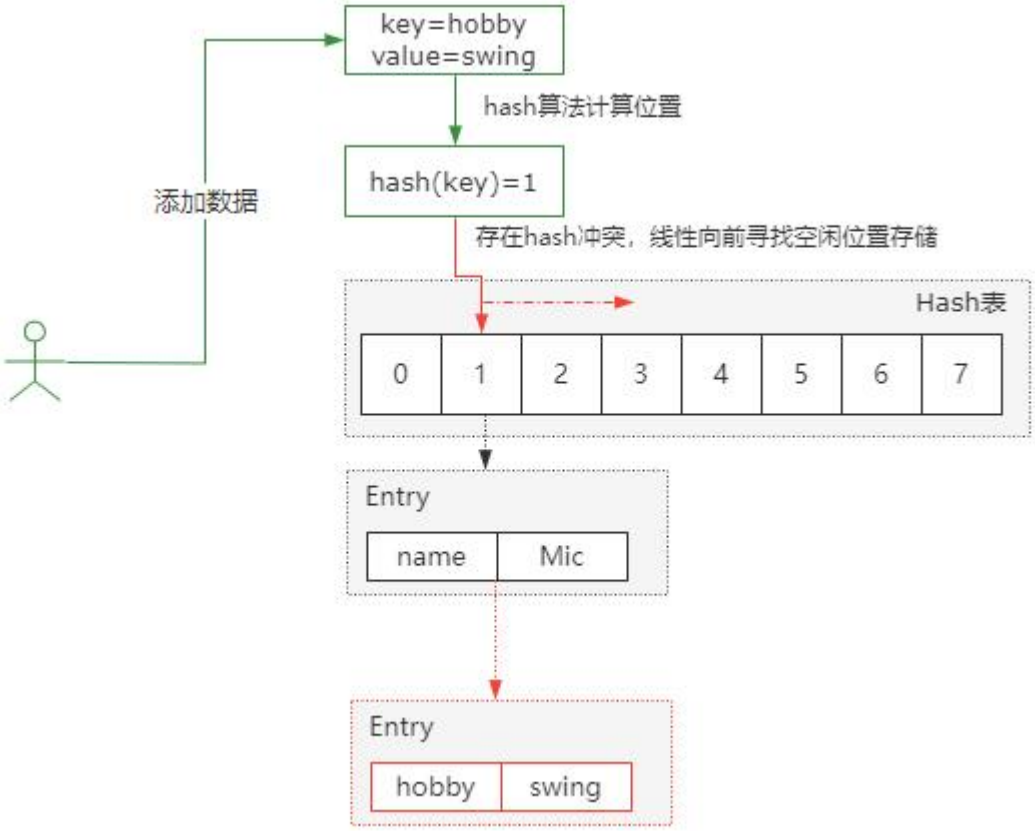
i. 开放定址法，也称为线性探测法，就是从发生冲突的那个位置开始，按照一定的次序从 hash 表中找到一个空闲的位置，然后把发生冲突的元素存入到这个空闲位置中。ThreadLocal 就用到了线性探测法来解决 hash 冲突的。

向这样一种情况（如图），在 hash 表索引 1 的位置存了一个 key=name，当再次添加 key=hobby 时，hash 计算得到的索引也是 1，这个就是 hash 冲突。而开放定址法，就是按顺序向前找到一个空闲的位置来存储冲突的 key。



ii. 链式寻址法，这是一种非常常见的方法，简单理解就是把存在 hash 冲突的 key，以单向链表的方式来存储，比如 HashMap 就是采用链式寻址法来实现的。

向这样一种情况（如图），存在冲突的 key 直接以单向链表的方式进行存储。



iii. 再 hash 法，就是当通过某个 hash 函数计算的 key 存在冲突时，再用另外一个 hash 函数对这个 key 做 hash，一直运算直到不再产生冲突。这种方式会增加计算时间，性能影响较大。

iv. 建立公共溢出区， 就是把 hash 表分为基本表和溢出表两个部分，凡事存在冲突的元素，一律放入到溢出表中。

e. HashMap 在 JDK1.8 版本中，通过链式寻址法+红黑树的方式来解决 hash 冲突问题,其中红黑树是为了优化 Hash 表链表过长导致时间复杂度增加的问题。当链表长度大于 8 并且 hash 表的容量大于 64 的时候，再向链表中添加元素就会触发转化。

以上就是我对这个问题的理解！

53.87 [滴滴]什么叫做阻塞队列的有界和无界

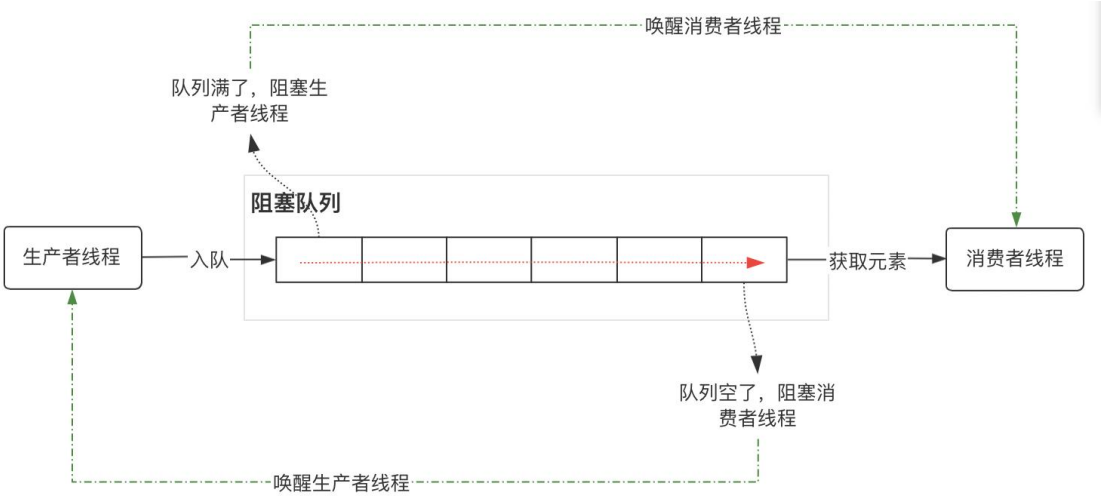
昨天一个 3 年 Java 经验的小伙伴私信我，他说现在面试怎么这么难啊！

我只是面试一个业务开发，他们竟然问我： 什么叫阻塞队列的有界和无界。现在面试也太卷了吧!

如果你也遇到过类似问题，那我们来看看高手的回答吧！

以下是高手的回答

1. （如图），阻塞队列，是一种特殊的队列，它在普通队列的基础上提供了两个附加功能
    a. 当队列为空的时候，获取队列中元素的消费者线程会被阻塞，同时唤醒生产者线程。
    b. 当队列满了的时候，向队列中添加元素的生产者线程被阻塞，同时唤醒消费者线程。



2. 其中，阻塞队列中能够容纳的元素个数，通常情况下是有界的，比如我们实例化一个 ArrayBlockingList，可以在构造方法中传入一个整形的数字，表示这个基于数组的阻塞队列中能够容纳的元素个数。这种就是有界队列。
3. 而无界队列，就是没有设置固定大小的队列，不过它并不是像我们理解的那种元素没有任何限制，而是它的元素存储量很大，像 LinkedBlockingQueue，它的默认队列长度是 Integer.Max_Value，所以我们感知不到它的长度限制。
4. 无界队列存在比较大的潜在风险，如果在并发量较大的情况下，线程池中可以几乎无限制的添加任务，容易导致内存溢出的问题！

以上就是我对这个问题的理解！

53.88 [滴滴]Dubbo 的服务请求失败怎么处理？

今天分享的面试题，几乎是 90%以上的互联网公司都会问到的问题。

“Dubbo 的服务请求失败怎么处理”？

对于这个问题，我们来看一下高手的回答。

以下是高手的回答

Dubbo 是一个 RPC 框架，它为我们的应用提供了远程通信能力的封装，同时，Dubbo 在 RPC 通信的基础上，逐步在向一个生态在演进，它涵盖了服务注册、动态路由、容错、服务降级、负载均衡等能力，基本上在微服务架构下面临的问题，Dubbo 都可以解决。

而对于 Dubbo 服务请求失败的场景，默认提供了重试的容错机制，也就是说，如果基于 Dubbo 进行服务间通信出现异常，服务消费者会对服务提供者集群中其他的节点发起重试，确保这次请求成功，默认的额外重试次数是 2 次。

除此之外，Dubbo 还提供了更多的容错策略，我们可以根据不同的业务场景来进行选择。

1. 快速失败策略，服务消费者只发起一次请求，如果请求失败，就直接把错误抛出去。这种比较适合在非幂等性场景中使用
2. 失败安全策略，如果出现服务通信异常，直接把这个异常吞掉不做任何处理
3. 失败自动恢复策略，后台记录失败请求，然后通过定时任务来对这个失败的请求进行重发。
4. 并行调用多个服务策略，就是把这个消息广播给服务提供者集群，只要有任何一个节点返回，就表示请求执行成功。
5. 广播调用策略，逐个调用服务提供者集群，只要集群中任何一个节点出现异常，就表示本次请求失败

要注意的是，默认基于重试策略的容错机制中，需要注意幂等性的处理，否则在事务型的操作中，容易出现多次数据变更的问题。

以上就是我对这个问题的理解！

53.89 [滴滴]ConcurrentHashMap 底层具体实现知道吗？实现原理是什么？

之前分享过一期 HashMap 的面试题，然后有个小伙伴私信我说，他遇到了一个 ConcurrentHashMap 的问题不知道怎么回答。

于是，就有了这一期的内容！！

我是 Mic，一个工作了 14 年的 Java 程序员，今天我来分享关

于 " ConcurrentHashMap 底层实现原理 " 这个问题，

看看高手是如何回答的！

以下是高手的回答

这个问题我从这三个方面来回答：（下面这三个点，打印在屏幕上）

1. ConcurrentHashMap 的整体架构
2. ConcurrentHashMap 的基本功能
3. ConcurrentHashMap 在性能方面的优化

- ConcurrentHashMap 的整体架构（字幕提示）

（如图所示），这个是 ConcurrentHashMap 在 JDK1.8 中的存储结构，它是由数组、单向链表、红黑树组成。

当我们初始化一个 ConcurrentHashMap 实例时，默认会初始化一个长度为 16 的数组。

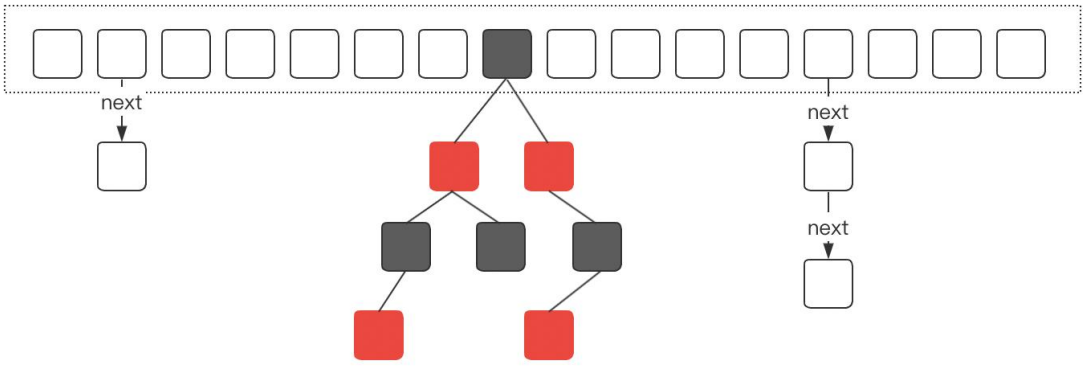
由于 ConcurrentHashMap 它的核心仍然是 hash 表，所以必然会存在 hash 冲突问题。

ConcurrentHashMap 采用链式寻址法来解决 hash 冲突。

当 hash 冲突比较多的时候，会造成链表长度较长，这种情况会使得 ConcurrentHashMap 中数据元素的查询复杂度变成 $O(_n)$。因此在 JDK1.8 中，引入了红黑树的机制。

当数组长度大于 64 并且链表长度大于等于 8 的时候，单项链表就会转换为红黑树。

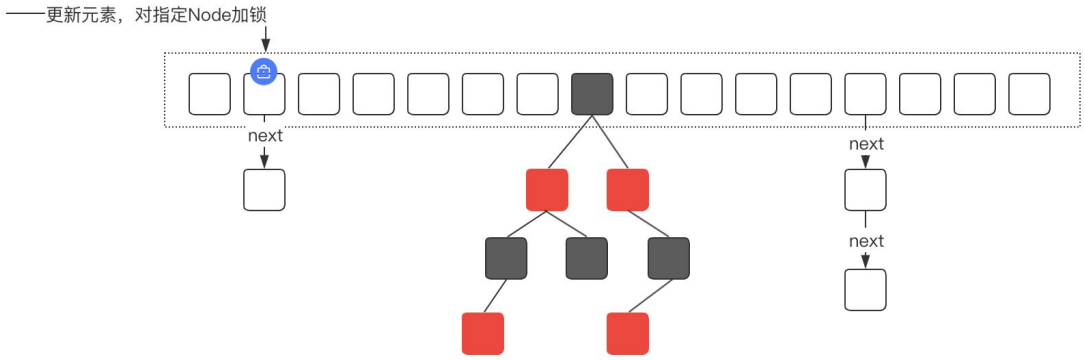
另外，随着 ConcurrentHashMap 的动态扩容，一旦链表长度小于 8，红黑树会退化成单向链表。



- ConcurrentHashMap 的基本功能（字幕提示）

ConcurrentHashMap 本质上是一个 HashMap，因此功能和 HashMap 一样，但是 ConcurrentHashMap 在 HashMap 的基础上，提供了并发安全的实现。

并发安全的主要实现是通过对指定的 Node 节点加锁，来保证数据更新的安全性（如图所示）。

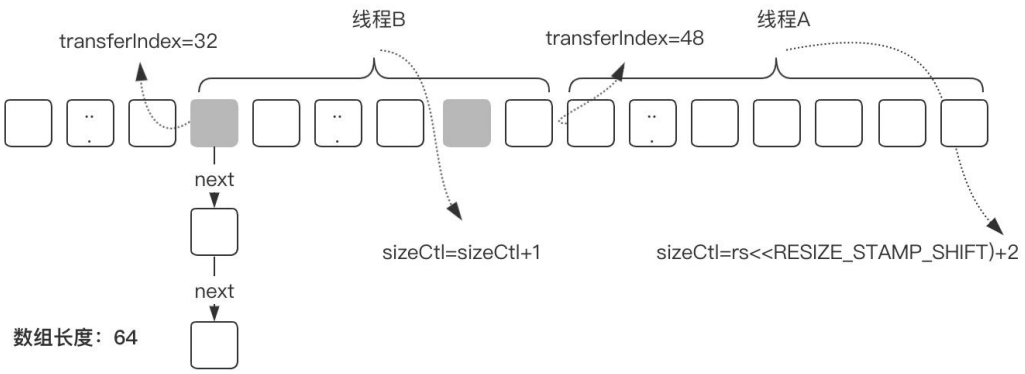


- ConcurrentHashMap 在性能方面做的优化（字幕提示）

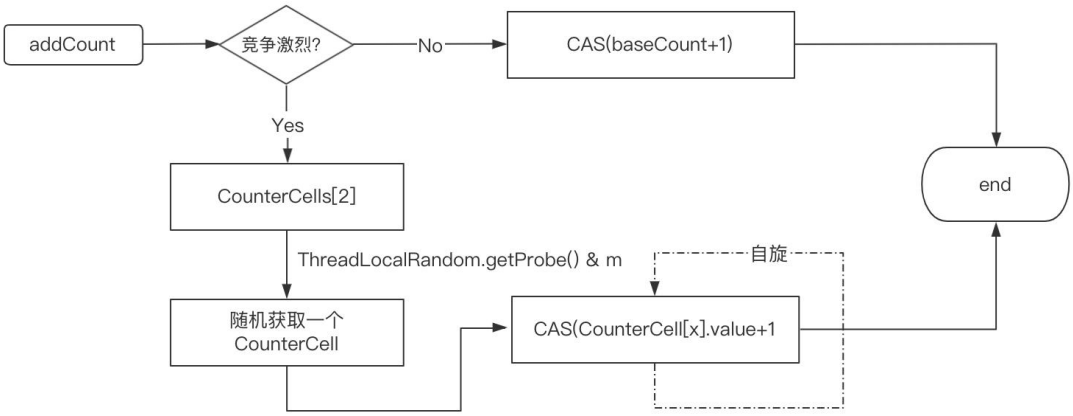
如果在并发性能和数据安全性之间做好平衡，在很多地方都有类似的设计，比如 cpu 的三级缓存、mysql 的 buffer_pool、Synchronized 的锁升级等等。

ConcurrentHashMap 也做了类似的优化，主要体现在以下几个方面：

- 在 JDK1.8 中，ConcurrentHashMap 锁的粒度是数组中的某一个节点，而在 JDK1.7，锁定的是 Segment，锁的范围要更大，因此性能上会更低。
- 引入红黑树，降低了数据查询的时间复杂度，红黑树的时间复杂度是 $O(_{logn})$。
- （如图所示），当数组长度不够时，ConcurrentHashMap 需要对数组进行扩容，在扩容的实现上，ConcurrentHashMap 引入了多线程并发扩容的机制，简单来说就是多个线程对原始数组进行分片后，每个线程负责一个分片的数据迁移，从而提升了扩容过程中数据迁移的效率。



- ConcurrentHashMap 中有一个 size()方法来获取总的元素个数，而在多线程并发场景中，在保证原子性的前提下来实现元素个数的累加，性能是非常低的。ConcurrentHashMap 在这个方面的优化主要体现在两个点：
  - 当线程竞争不激烈时，直接采用 CAS 来实现元素个数的原子递增。
  - 如果线程竞争激烈，使用一个数组来维护元素个数，如果要增加总的元素个数，则直接从数组中随机选择一个，再通过 CAS 实现原子递增。它的核心思想是引入了数组来实现对并发更新的负载。



以上就是我对这个问题的理解！

53.90 b 树和 b+树的理解

数据结构与算法问题，困扰了无数的小伙伴。

很多小伙伴对数据结构与算法的认知有一个误区，认为工作中没有用到，为什么面试要问，问了能解决实际问题？

图灵奖获得者： Niklaus Wirth 说过： 程序=数据结构+算法， 也就说我们无时无刻都在和数据结构打交道。

只是作为 Java 开发，由于技术体系的成熟度较高，使得大部分人认为：程序应该等于 框架 + SQL 呀?

今天我们就来分析一道数据结构的题目：” B 树和 B+树 “。

关于这个问题，我们来看看高手的回答!

以下是高手的回答

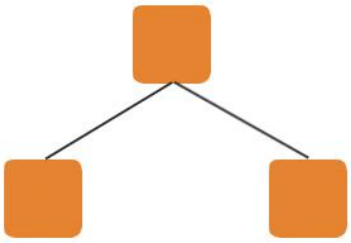
为了更清晰的解答这个问题，我打算从三个方面来回答：

- 了解二叉树、AVL 树、B 树的概念
- B 树和 B+树的应用场景
-

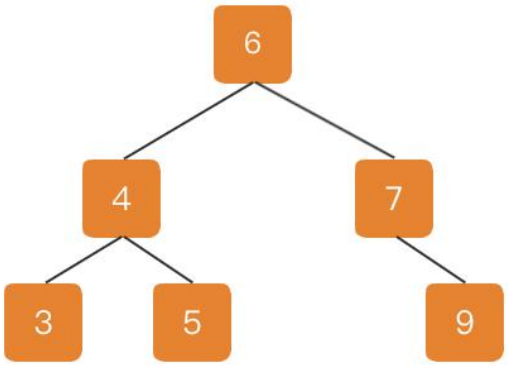
1. B 树是一种多路平衡查找树，为了更形象的理解，（我们来看这张图）。

二叉树，每个节点支持两个分支的树结构，相比于单向链表，多了一个分支。

二叉查找树，在二叉树的基础上增加了一个规则，左子树的所有节点的值都小于它的根节点，右子树的所有子节点都大于它的根节点。

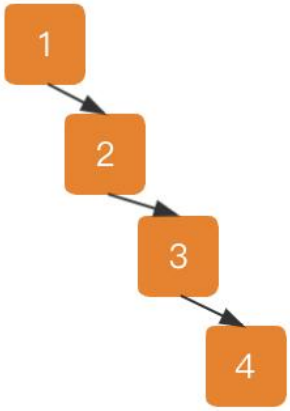


二叉树

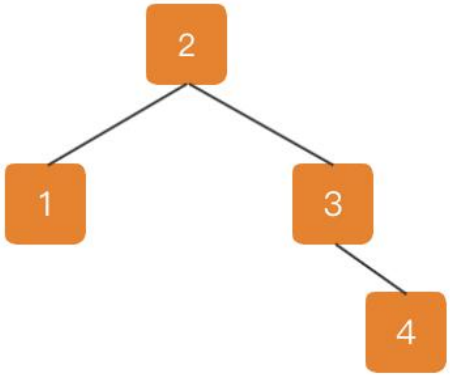


二叉查找树

（如图），二叉查找树会出现斜树问题，导致时间复杂度增加，因此又引入了一种平衡二叉树，它具有二叉查找树的所有特点，同时增加了一个规则：” 它的左右两个子树的高度差的绝对值不超过 1 “。平衡二叉树会采用左旋、右旋的方式来实现平衡。



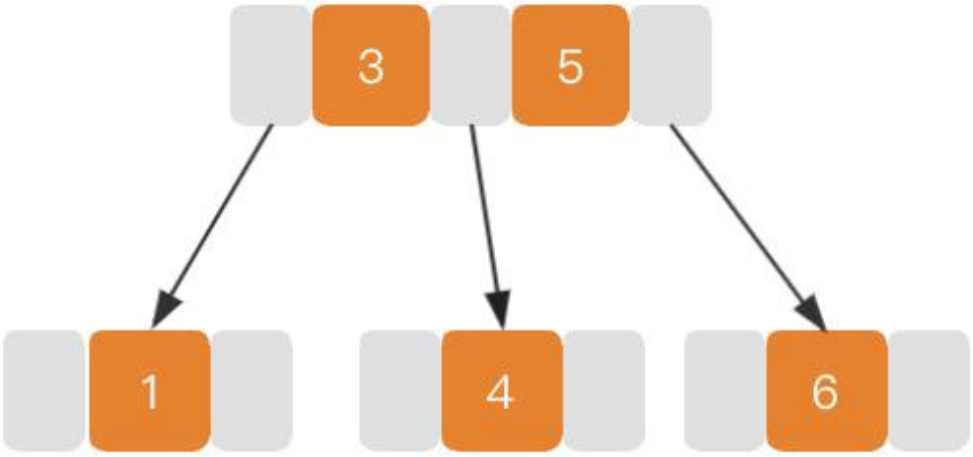
斜树



平衡二叉树

（如图），而 B 树是一种多路平衡查找树，它满足平衡二叉树的规则，但是它可以有多个子树，子树的数量取决于关键字的数量，比如这个图中根节点有两个关键字 3 和 5，那么它能够拥有的子路数量=关键字数+1。

因此从这个特征来看，在存储同样数据量的情况下，平衡二叉树的高度要大于 B 树。



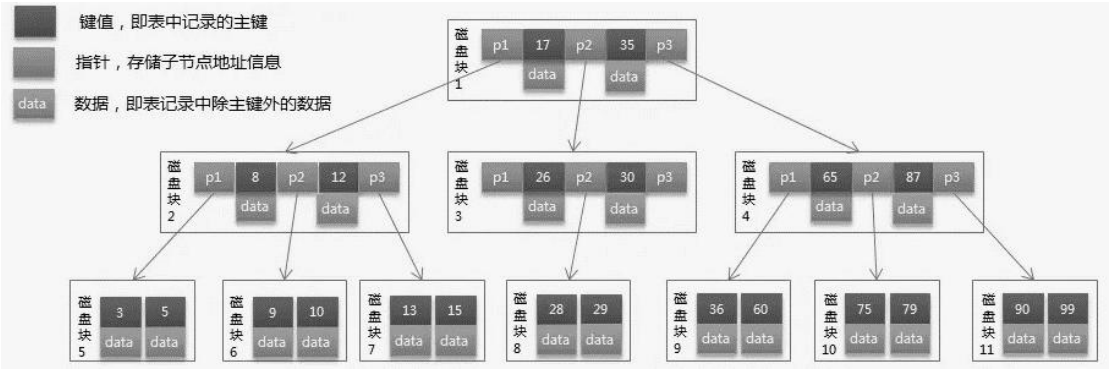
B树

B+树，其实是在 B 树的基础上做的增强，最大的区别有两个：

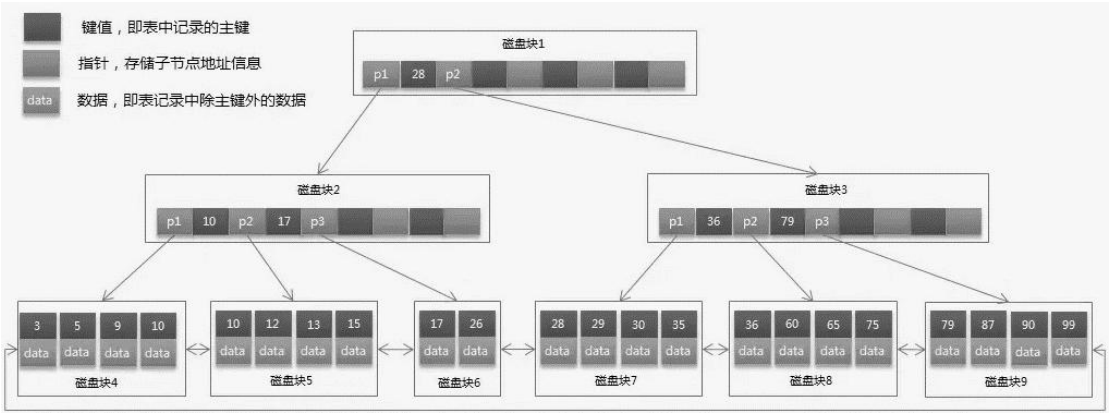
a. B 树的数据存储在每个节点上，而 B+树中的数据是存储在叶子节点，并且通过链表的方式把叶子节点中的数据进行连接。

b. B+树的子路数量等于关键字数

（如图所示）这个是 B 树的存储结构，从 B 树上可以看到每个节点会存储数据。



（如图所示）这个是 B+树，B+树的所有数据是存储在叶子节点，并且叶子节点的数据是用双向链表关联的。



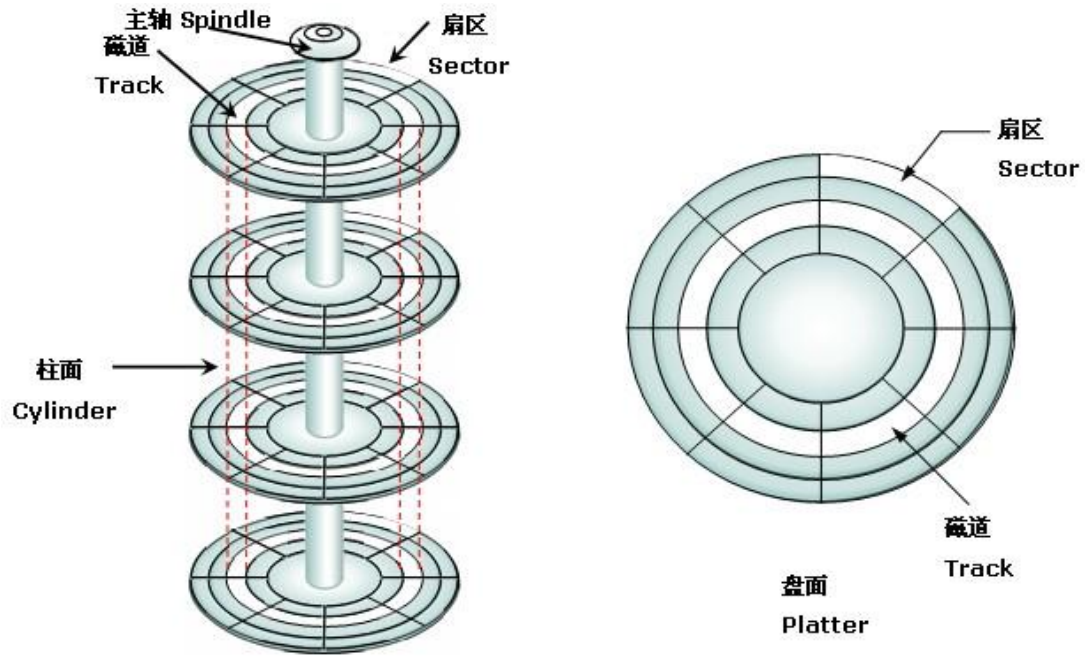
2. B 树和 B+树，一般都是应用在文件系统和数据库系统中，用来减少磁盘 IO 带来的性能损耗。

以 Mysql 中的 InnoDB 为例，当我们通过 select 语句去查询一条数据时，InnoDB 需要从磁盘上去读取数据，这个过程会涉及到磁盘 IO 以及磁盘的随机 IO（如图所示）

我们知道磁盘 IO 的性能是特别低的，特别是随机磁盘 IO。

因为，磁盘 IO 的工作原理是，首先系统会把数据逻辑地址传给磁盘，磁盘控制电路按照寻址逻辑把逻辑地址翻译成物理地址，也就是确定要读取的数据在哪个磁道，哪个扇区。

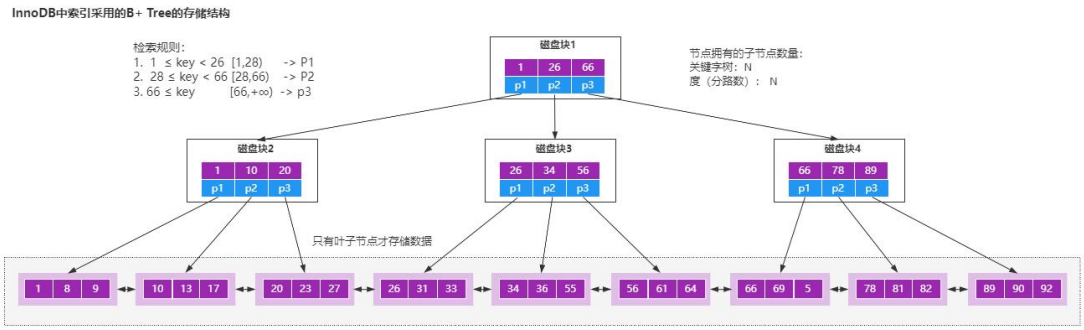
为了读取这个扇区的数据，需要把磁头放在这个扇区的上面，为了实现这一个点，磁盘会不断旋转，把目标扇区旋转到磁头下面，使得磁头找到对应的磁道，这里涉及到寻道事件以及旋转时间。



很明显，磁盘 IO 这个过程的性能开销是非常大的，特别是查询的数据量比较多的情况下。

所以在 InnoDB 中，干脆对存储在磁盘块上的数据建立一个索引，然后把索引数据以及索引列对应的磁盘地址，以 B+树的方式来存储。

如图所示，当我们需要查询目标数据的时候，根据索引从 B+树中查找目标数据即可，由于 B+树分路较多，所以只需要较少次数的磁盘 IO 就能查找到。



3. 为什么用 B 树或者 B+树来做索引结构？原因是 AVL 树的高度要比 B 树的高度要高，而高度就意味着磁盘 IO 的数量。所以为了减少磁盘 IO 的次数，文件系统或者数据库才会采用 B 树或者 B+树。

以上就是我对 B 树和 B+树的理解！

53.91 能谈一下 CAS 机制吗?

一个小伙伴私信我，他说遇到了一个关于 CAS 机制的问题，他以为面试官问的是 CAS 实现单点登录。

心想，这个问题我熟啊，然后就按照单点登录的思路去回答，结果面试官一直摇头。

他来和我说，到了面试结束都没明想白自己回答这么好，怎么就没有当场给我发 offer 呢?

实际上，面试官问的是并发编程中的 CAS 机制。

下面我们来看看高手对于 CAS 机制的回答吧

以下是高手的回答

CAS 是 Java 中 Unsafe 类里面的方法，它的全称是 CompareAndSwap，比较并交换的意思。它的主要功能是能够保证在多线程环境下，对于共享变量的修改的原子性。

我来举个例子，比如说有这样一个场景（如图），有一个成员变量 state，默认值是 0，定义了一个方法 `doSomething()`，这个方法的逻辑是，判断 state 是否为 0 ，如果为 0，就修改成 1。

这个逻辑看起来没有任何问题，但是在多线程环境下，会存在原子性的问题，因为这里是一个典型的，Read - Write 的操作。

一般情况下，我们会在 doSomething()这个方法上加同步锁来解决原子性问题。

```
public class Example {

    private int state=0;

    public void doSomething(){
        if(state==0){ //多线程环境中，存在原子性问题
            state=1;
            //TODO
        }
    }
}
```

但是，加同步锁，会带来性能上的损耗，所以，对于这类场景，我们就可以使用 CAS 机制来进行优化

这个是优化之后的代码（如图）

在 doSomething()方法中，我们调用了 unsafe 类中的 compareAndSwapInt()方法来达到同样的目的，这个方法有四个参数，

分别是：当前对象实例、成员变量 state 在内存地址中的偏移量、预期值 0、期望更改之后的值 1。

CAS 机制会比较 state 内存地址偏移量对应的值和传入的预期值 0 是否相等，如果相等，就直接修改内存地址中 state 的值为 1.

否则，返回 false，表示修改失败，而这个过程是原子的，不会存在线程安全问题。

```
public class Example {

    private volatile int state=0;

    private static final Unsafe unsafe = Unsafe.getUnsafe();
    private static final long stateOffset;

    static {
        try {
            stateOffset = unsafe.objectFieldOffset
                    (Example.class.getDeclaredField("state"));
        } catch (Exception ex) { throw new Error(ex); }
    }

    public void doSomething(){
        if(unsafe.compareAndSwapInt(this,stateOffset,0,1)){
            //TODO
        }
    }
}
```

CompareAndSwap 是一个 native 方法，实际上它最终还是会面临同样的问题，就是先从内存地址中读取 state 的值，然后去比较，最后再修改。

这个过程不管是在什么层面上实现，都会存在原子性问题。

所以呢，CompareAndSwap 的底层实现中，在多核 CPU 环境下，会增加一个 Lock 指令对缓存或者总线加锁，从而保证比较并替换这两个指令的原子性。

CAS 主要用在并发场景中，比较典型的使用场景有两个。

1. 第一个是 J.U.C 里面 Atomic 的原子实现，比如 AtomicInteger，AtomicLong。

2. 第二个是实现多线程对共享资源竞争的互斥性质，比如在 AQS、ConcurrentHashMap、ConcurrentLinkedQueue 等都有用到。

以上就是我对这个问题的理解。

53.92 请说一下网络四元组

之前我一直在向大家征集一些刁钻的面试题，然后今天就收到了这样的一个问题。

“请你说一下网络四元组的理解”，他说他听到这个问题的时候，完全就懵了。

"这个是程序员应该懂的吗？ 你是让我去做啥？造火箭吗？"(声音的起伏)

好吧，关于这个问题，我们来看看普通人和高手的回答。

以下是高手的回答

四元组，简单理解就是在 TCP 协议中，去确定一个客户端连接的组成要素，它包括源 IP 地址、目标 IP 地址、源端口号、目标端口号。

正常情况下，我们对于网络通信的认识可能是这样（如图）。

服务端通过 ServerSocket 建立一个对指定端口号的监听，比如 8080。 客户端通过目标 ip 和端口就可以和服务端建立一个连接，然后进行数据传输。

但是我们知道的是，一个 Server 端可以接收多个客户端的连接，比如像这种情况（如图）。

那，当多个客户端连接到服务端的时候，服务端需要去识别每一个连接。

并且(如图），TCP 是全双工协议，也就是说数据允许在连接的两个方向上同时传输，因此这里的客户端，如果是反向通信，它又变成了服务端。

所以基于这两个原因，就引入了四元组的设计，也就是说，当一个客户端和服务端建立一个 TCP 连接的时候，通过源 IP 地址、目标 IP 地址、源端口号、目标端口号来确定一个唯一的 TCP 连接。因为服务器的 IP 和端口是不变的，只要客户端的 IP 和端口彼此不同就 OK 了。

比如像这种情况（如图），同一个客户端主机上有三个连接连到 Server 端，那么这个时候源 IP 相同，源端口号不同。此时建立的四元组就是（10.23.15.3，59461 ，192.168.8.135，8080）

其中，源端口号是每次建立连接的时候系统自动分配的。

以上就是我对于四元组的理解。

53.93 什么是服务网格?

今天继续来分享一个有趣的面试题，“什么是服务网格”？

服务网格这个概念出来很久了，从 2017 年被提出来，到 2018 年正式爆发，很多云厂商和互联网企业都在纷纷向服务网格靠拢。像蚂蚁集团、美团、百度、网易等一线互联网公司，都有服务网格的落地应用。

在我看来呢，服务网格是微服务架构的更进一步升级，它的核心目的是实现网络通信与业务逻辑的分离，使得开发人员更加专注在业务的实现上。

那么基于这个问题，我们来看看普通人和高手的回答。

以下是高手的回答

服务网格，也就是 Service Mesh，它是专门用来处理服务通讯的基础设施层。它的主要功能是处理服务之间的通信，并且负责实现请求的可靠性传递。

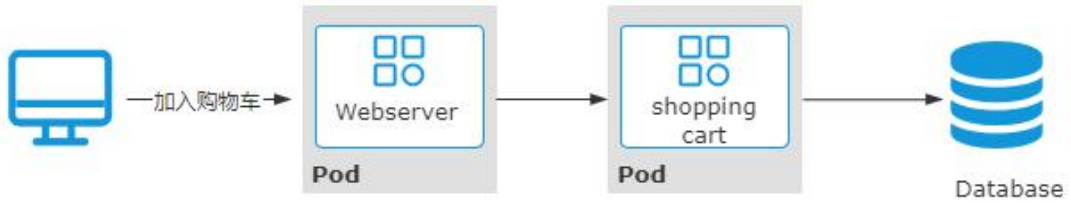
Service Mesh，我们通常把他称为第三代微服务架构，既然是第三代，那么意味着他是在原来的微服务架构下做的升级。

为了更好的说明 Service Mesh，那我就不得不说一下微服务架构部分的东西。

首先，当我们把一个电商系统以微服务化架构进行拆分后，会的到这样的一个架构（如图），其中包括 Webserver、payment、inventory 等等。

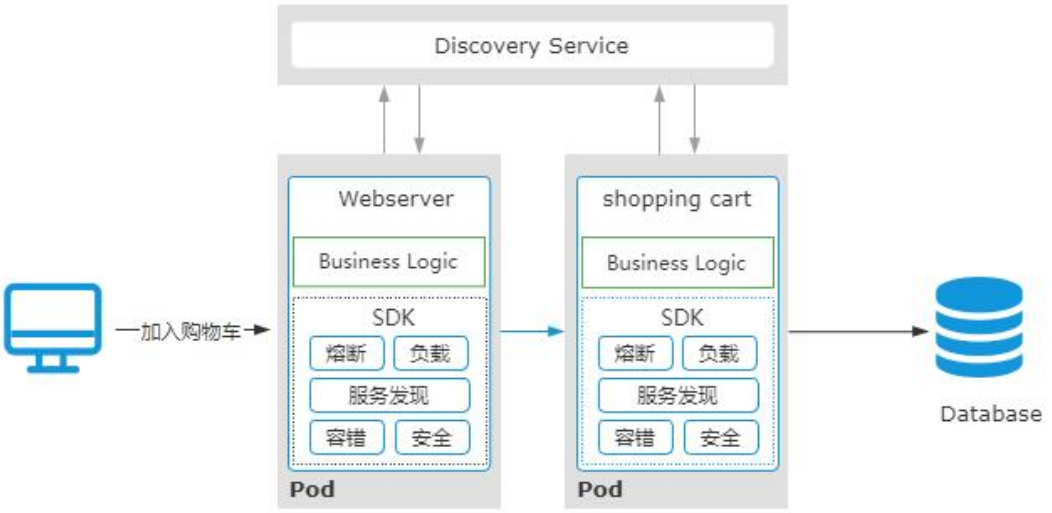
（如图）这些微服务应用，会被部署到 Docker 容器、或者 Kubernetes 集群。由于每个服务的业务逻辑是独立的，比如 payment 会实现支付的业务逻辑、order 实现订单的处理、Webserver 实现客户端请求的响应等。

（如图）所以，服务之间必须要相互通信，才能实现功能的完整性。比如用户把一个商品加入购物车，请求会进入到 Webserver，然后转发到 shopping cart 进行处理，并存到数据库。



而在这个过程中，每个服务之间必须要知道对方的通信地址，并且当有新的节点加入进来的时候，还需要对这些通信地址进行动态维护。所以，在第一代微服务架构中，每个微服务除了要实现业务逻辑以外，还需要解决上下游寻址、通讯、以及容错等问题。

（如图）于是，在第二代微服务架构下，引入了服务注册中心来实现服务之间的寻址，并且服务之间的容错机制、负载均衡也逐步形成了独立的服务框架，比如主流的 Spring Cloud、或者 Spring Cloud Alibaba。



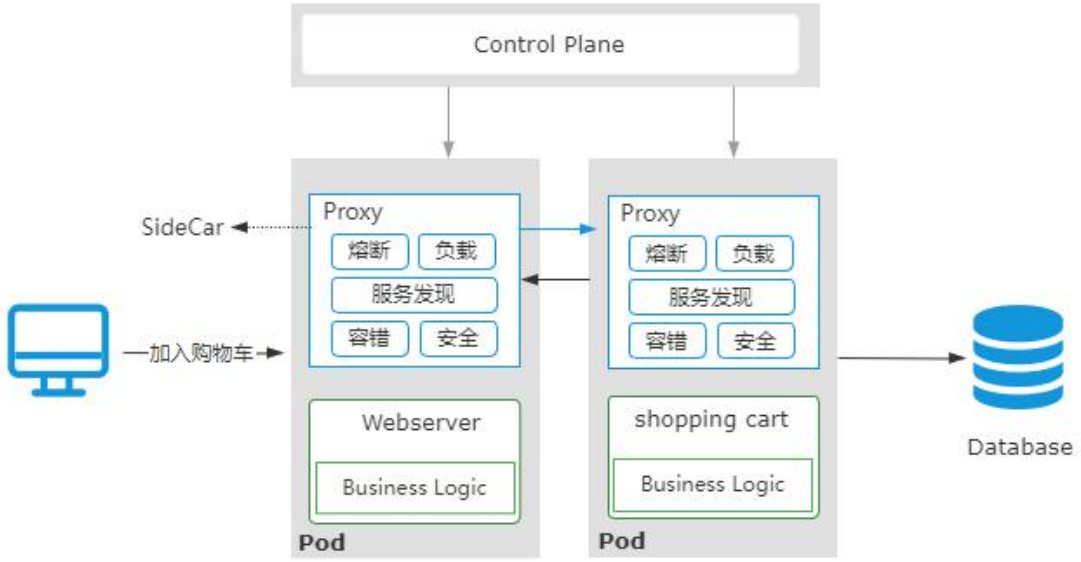
在第二代微服务架构中，负责业务开发的小伙伴不仅仅需要关注业务逻辑，还需要花大量精力去处理微服务中的一些基础性配置工作，虽然 Spring Cloud 已经尽可能去完成了这些事情，但对于开发人员来说，学习 Spring Cloud，以及针对 Spring Cloud 的配置和维护，仍然存在较大的挑战。另外呢，也增加了整个微服务的复杂性。

实际上，在我看来，“微服务中所有的这些服务注册、容错、重试、安全等工作，都是为了保证服务之间通信的可靠性”。

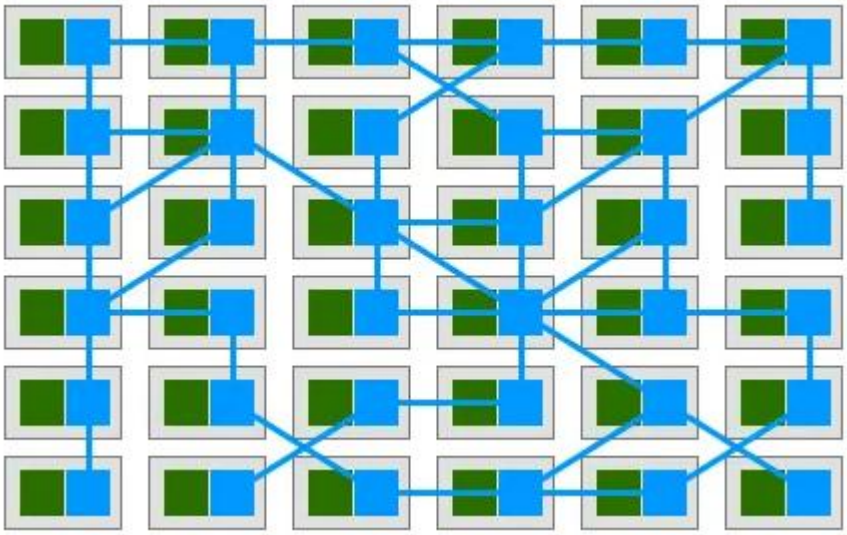
于是，就有了第三代微服务架构，Service Mesh。

（如图）原本模块化到微服务框架里的微服务基础能力，被进一步的从一个 SDK 中演进成了一个独立的代理进程-SideCar

SideCar 的主要职责就是负责各个微服务之间的通信，承载了原本第二代微服务架构中的服务发现、调用容错、服务治理等功能。使得微服务基础能力和业务逻辑迭代彻底解耦。



之所以我们称 Service Mesh 为服务网格，是因为在大规模微服务架构中，每个服务的通信都是由 SideCar 来代理的，各个服务之间的通信拓扑图，看起来就像一个网格形状（如图）。



Istio 是目前主流的 Service Mesh 开源框架。

以上就是我对服务网格的理解。

53.94 Redis 和 Mysql 如何保证数据一致性

今天分享一道一线互联网公司高频面试题。

“Redis 和 Mysql 如何保证数据一致性”。

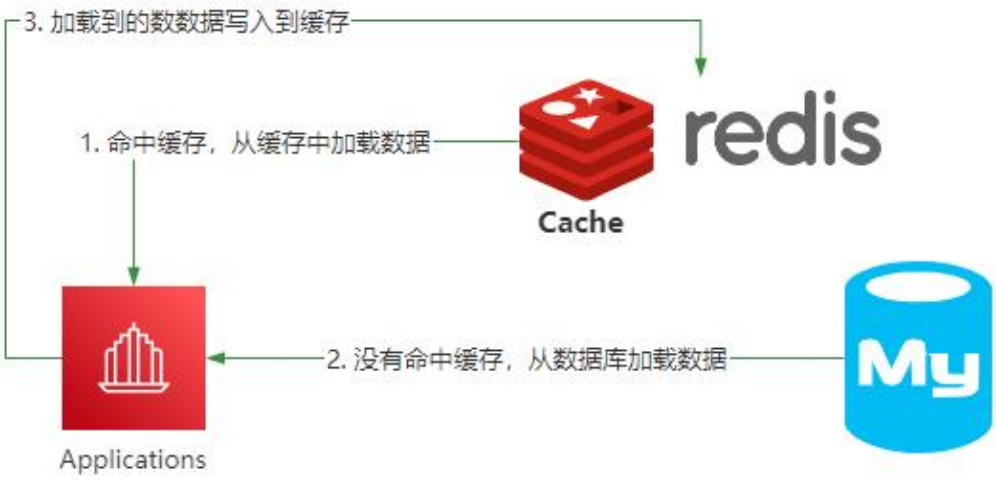
这个问题难倒了不少工作 5 年以上的程序员，难的不是问题本身，而是解决这个问题的思维模式。

下面来看看高手对于这个问题的回答。

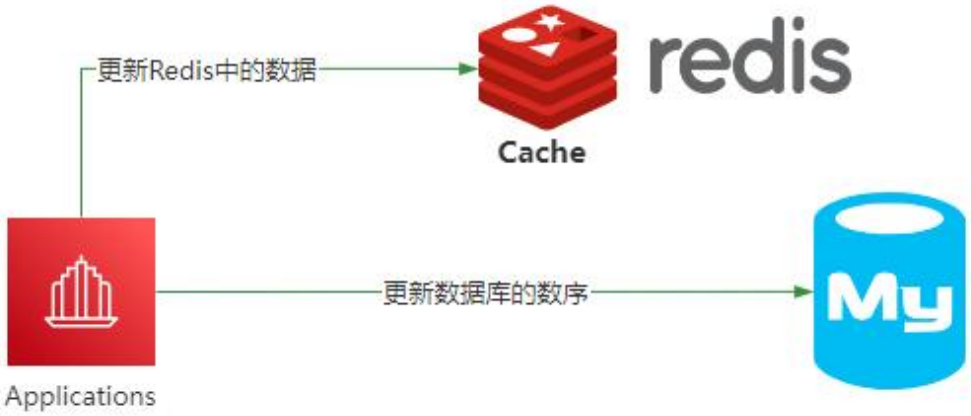
一般情况下，Redis 用来实现应用和数据库之间读操作的缓存层，主要目的是减少数据库 IO，还可以提升数据的 IO 性能。

这是它的整体架构。

当应用程序需要去读取某个数据的时候，首先会先尝试去 Redis 里面加载，如果命中就直接返回。如果没有命中，就从数据库查询，查询到数据后再把这个数据缓存到 Redis 里面。



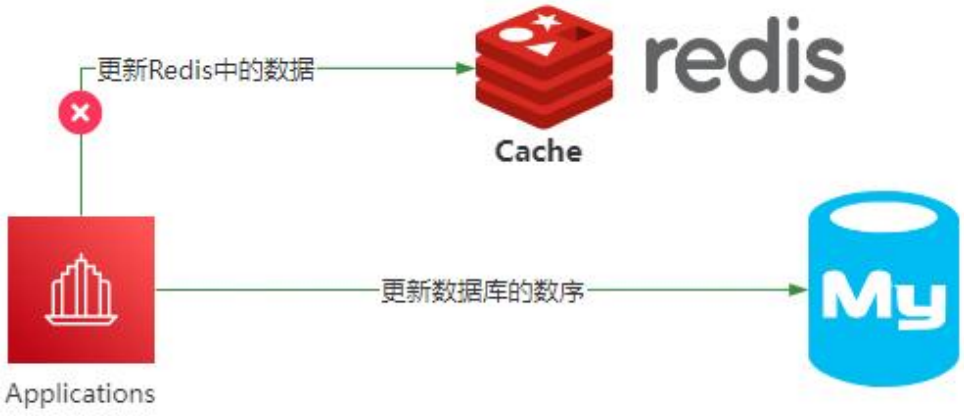
（如图）在这样一个架构中，会出现一个问题，就是一份数据，同时保存在数据库和 Redis 里面，当数据发生变化的时候，需要同时更新 Redis 和 Mysql，由于更新是有先后顺序的，并且它不像 Mysql 中的多表事务操作，可以满足 ACID 特性。所以就会出现数据一致性问题。



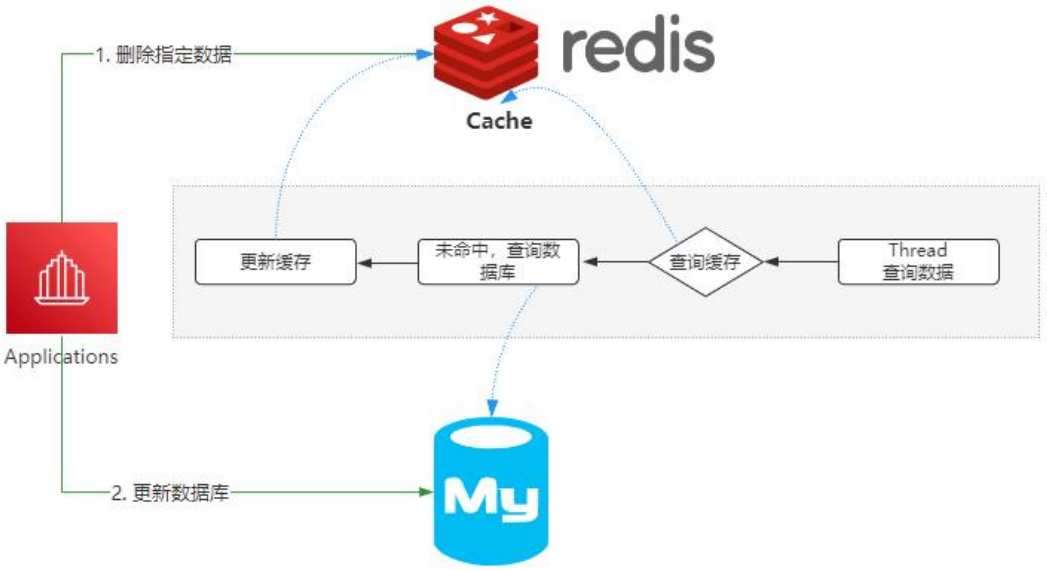
在这种情况下，能够选择的方法只有几种。

1. 先更新数据库，再更新缓存
2. 先删除缓存，再更新数据库

如果先更新数据库，再更新缓存，如果缓存更新失败，就会导致数据库和 Redis 中的数据不一致。

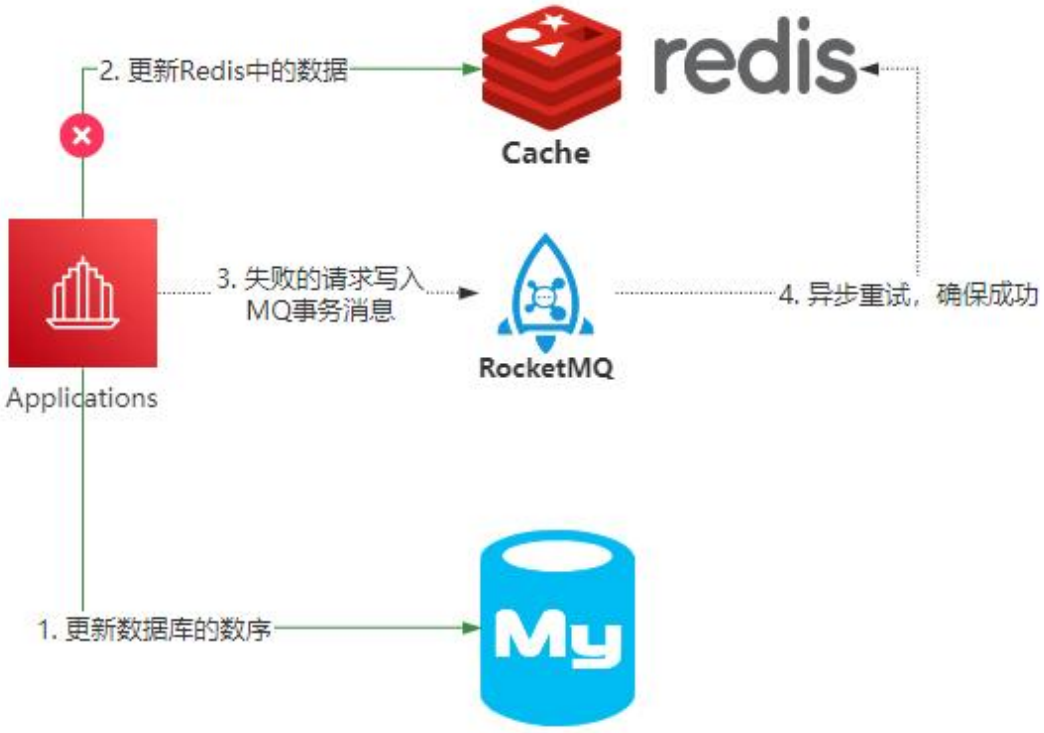


如果是先删除缓存，再更新数据库，理想情况是应用下次访问 Redis 的时候，发现 Redis 里面的数据是空的，就从数据库加载保存到 Redis 里面，那么数据是一致的。但是在极端情况下，由于删除 Redis 和更新数据库这两个操作并不是原子的，所以这个过程如果有其他线程来访问，还是会存在数据不一致问题。

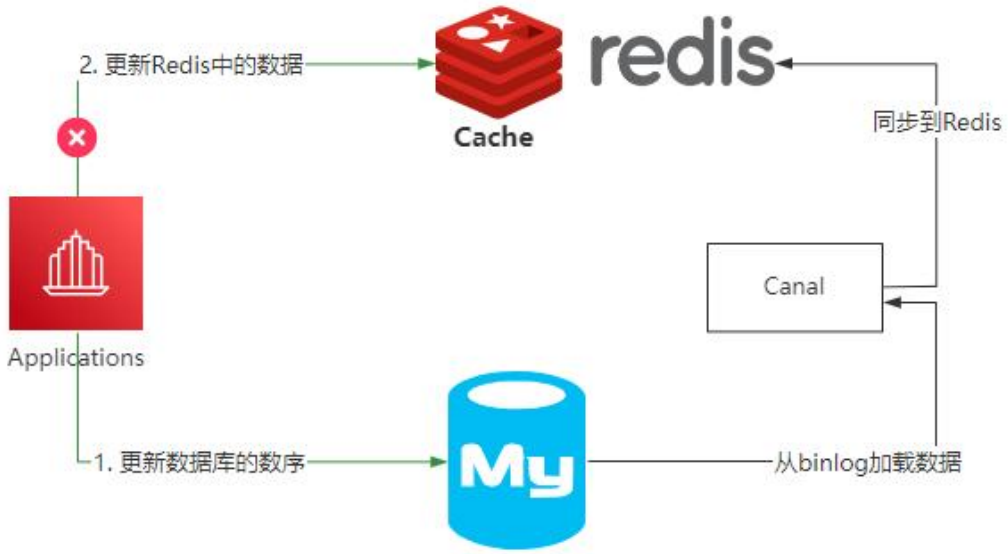


所以，如果需要在极端情况下仍然保证 Redis 和 Mysql 的数据一致性，就只能采用最终一致性方案。

（如图）比如基于 RocketMQ 的可靠性消息通信，来实现最终一致性。



（如图）还可以直接通过 Canal 组件，监控 Mysql 中 binlog 的日志，把更新后的数据同步到 Redis 里面。



因为这里是基于最终一致性来实现的，如果业务场景不能接受数据的短期不一致性，那就不能使用这个方案来做。

以上就是我对这个问题的理解。

53.95 Spring Boot 中自动装配机制的原理

最近一个粉丝说，他面试了 4 个公司，有三个公司问他：“Spring Boot 中自动装配机制的原理”

他回答了，感觉没回答错误，但是怎么就没给 offer 呢?

对于这个问题，看看高手该如何回答。

以下是高手的回答

自动装配，简单来说就是自动把第三方组件的 Bean 装载到 Spring IOC 器里面，不需要开发人员再去写 Bean 的装配配置。

在 Spring Boot 应用里面，只需要在启动类加上@SpringBootApplication 注解就可以实现自动装配。

@SpringBootApplication 是一个复合注解，真正实现自动装配的注解是 @EnableAutoConfiguration。

（如图）自动装配的实现主要依靠三个核心关键技术。

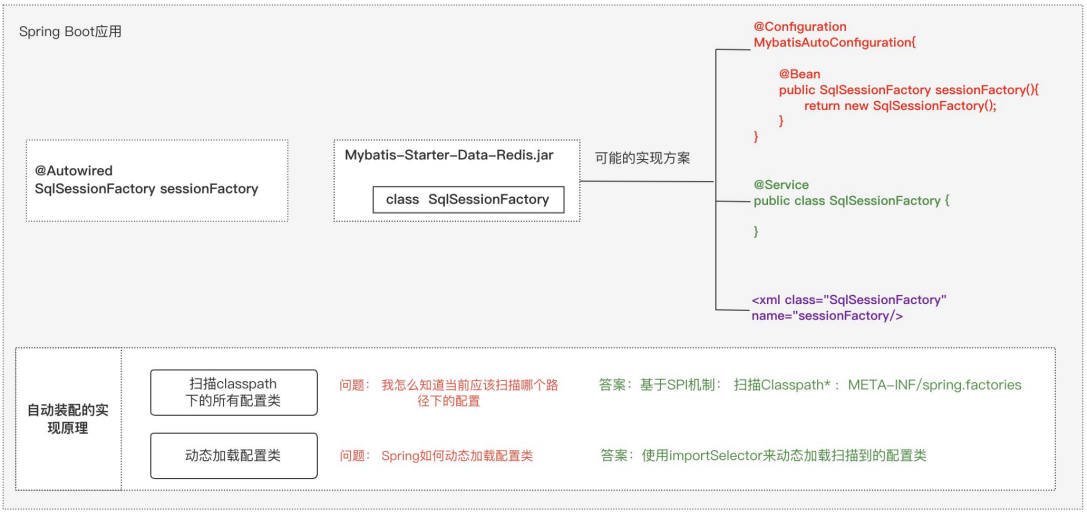
1. 引入 Starter 启动依赖组件的时候，这个组件里面必须要包含@Configuration 配置类，在这个配置类里面通过@Bean 注解声明需要装配到 IOC 容器的 Bean 对象。

2. 这个配置类是放在第三方的 jar 包里面，然后通过 SpringBoot 中的约定优于配置思想，把这个配置类的全路径放在 classpath:/META-INF/spring.factories 文件中。这样 SpringBoot 就可以知道第三方 jar 包里面的配置类的位置，这个步骤主要是用到了 Spring 里面的 SpringFactoriesLoader 来完成的。
3. SpringBoot 拿到所第三方 jar 包里面声明的配置类以后，再通过 Spring 提供的 ImportSelector 接口，实现对这些配置类的动态加载。

在我看来，SpringBoot 是约定优于配置这一理念下的产物，所以在很多的地方，都会看到这类的思想。它的出现，让开发人员更加聚焦在了业务代码的编写上，而不需要去关心和业务无关的配置。

其实，自动装配的思想，在 SpringFramework3.x 版本里面的@Enable 注解，就有了实现的雏形。@Enable 注解是模块驱动的意思，我们只需要增加某个@Enable 注解，就自动打开某个功能，而不需要针对这个功能去做 Bean 的配置，@Enable 底层也是帮我们去自动完成这个模块相关 Bean 的注入。

以上，就是我对 Spring Boot 自动装配机制的理解。



53.96 死锁的发生原因和怎么避免

一个去阿里面试的小伙伴私信我说：今天被一个死锁的问题难到了。

平常我都特意看了死锁这块的内容，但是回答的时候就想不起来。

这里可能存在一个误区，认为技术是要靠记的。

大家可以想想，平时写代码的时候，这些代码是背下来的吗？

遇到一个需求的时候，能够立刻提供解决思路，这个也是记下来的吗？

所有的技术问题，都可以用一个问题来解决： “如果让你遇到这个问题，你会怎么设计”？

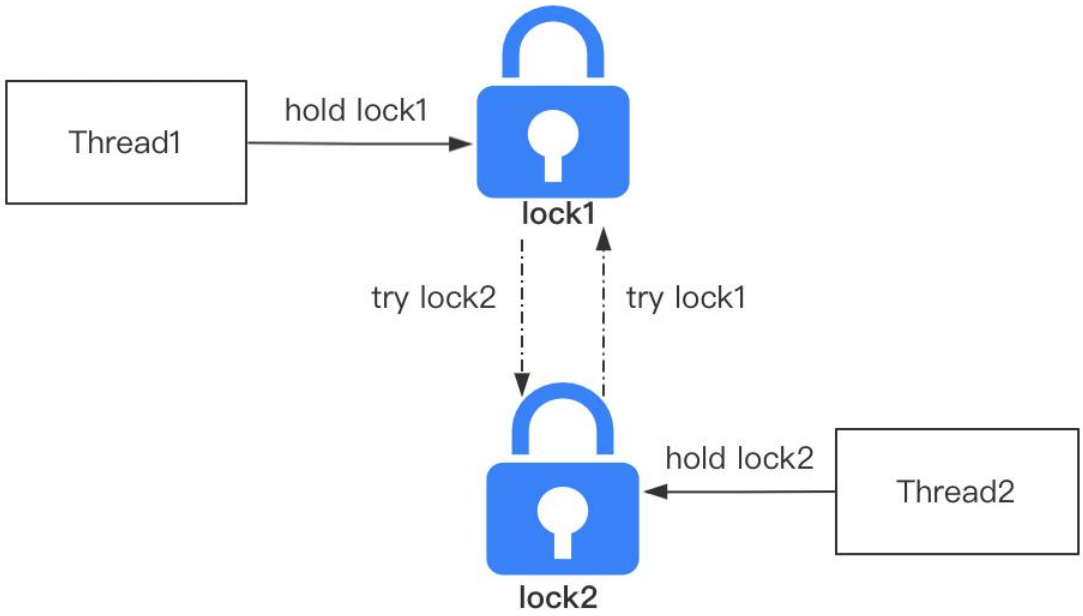
当你大脑一篇空白时，说明你目前掌握的技术只能足够支撑你写 CURD 的能力。

好了，下面来看看普通人和高手是如何回答这个问题的。

以下是高手的回答

(如图)，死锁，简单来说就是两个或者两个以上的线程在执行的过程中，争夺同一个共享资源造成的相互等待的现象。

如果没有外部干预，线程会一直阻塞无法往下执行，这些一直处于相互等待资源的线程就称为死锁线程。



导致死锁的条件有四个，也就是这四个条件同时满足就会产生死锁。

- 互斥条件，共享资源 X 和 Y 只能被一个线程占用；
- 请求和保持条件，线程 T1 已经取得共享资源 X，在等待共享资源 Y 的时候，不释放共享资源 X；
- 不可抢占条件，其他线程不能强行抢占线程 T1 占有的资源；
- 循环等待条件，线程 T1 等待线程 T2 占有的资源，线程 T2 等待线程 T1 占有的资源，就是循环等待。

导致死锁之后，只能通过人工干预来解决，比如重启服务，或者杀掉某个线程。

所以，只能在写代码的时候，去规避可能出现的死锁问题。

按照死锁发生的四个条件，只需要破坏其中的任何一个，就可以解决，但是，互斥条件是没办法破坏的，因为这是互斥锁的基本约束，其他三方条件都有办法来破坏：

- 对于“请求和保持”这个条件，我们可以一次性申请所有的资源，这样就不存在等待了。
- 对于“不可抢占”这个条件，占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源，这样不可抢占这个条件就破坏掉了。
- 对于“循环等待”这个条件，可以靠按序申请资源来预防。所谓按序申请，是指资源是有线性顺序的，申请的时候可以先申请资源序号小的，再申请资源序号大的，这样线性化后自然就不存在循环了。

以上就是我对这个问题的理解。

53.97 JDK 动态代理为什么只能代理有接口的类?

一个工作 5 年的 Java 程序员，去应聘得 物 APP 月薪 30k 的 Java 开发岗位。

被一个基础问题问得措手不及，这个问题是： JDK 动态代理为什么只能代理有接口的类。

这个问题在他的认知里面，是一个很正常的事情，竟然还有问什么?

好吧，对于这个问题，来看看普通人和高手的回答吧。

> 以下是高手的回答

嗯，这个问题的核心本质，是 JDK 动态代理本身的机制来决定的(如图)。

首先，在 Java 里面，动态代理是通过 Proxy.newProxyInstance()方法来实现的，它需要传入被动态代理的接口类。

```java
public class ProxyExample {
    public static void main(String[] args) {
        // 让代理对象的class文件写入到磁盘
        System.getProperties().put("sun.misc.ProxyGenerator.saveGeneratedFiles", "true");
        IHelloService helloService = (IHelloService) Proxy.newProxyInstance
                (IHelloService.class.getClassLoader(),
                        new Class[]{IHelloService.class}, (proxy, method, args1) -> {
                            System.out.println("动态代理执行的逻辑");
                            return "Hello World";
                        });
        System.out.println(helloService.say());
    }
}
```

之所以要传入接口，不能传入类，还是取决于 JDK 动态代理的底层实现（如图）。

JDK 动态代理会在程序运行期间动态生成一个代理类$Proxy0，这个动态生成的代理类会继承 java.lang.reflect.Proxy 类，同时还会实现被代理类的接口 IHelloService。

在 Java 中，是不支持多重继承的。而每个动态代理类都会继承 Proxy 类（这也是 JDK 动态代理的实现规范），所以就导致 JDK 里面的动态代理只能代理接口，而不能代理实现类。

(注意，下面这张图片展示的时候，上面的图片仍然保存在一个画面里面)

```java
public final class $Proxy0 extends Proxy implements IHelloService {
    //省略....
    public $Proxy0(InvocationHandler var1) throws  {
        super(var1);
    }

    public final String say() throws  {
        return (String)super.h.invoke(this, m3, (Object[])null);
    }
   //省略....
 }
```

我分析过动态代理的源码，发现 Proxy 这个类只是保存了动态代理的处理器 InvocationHandler，如果不抽出来，直接设置到$Proxy0 动态代理类里面，也是可以的。

如果这么做，就可以针对实现类来做动态代理了。作者为什么这么设计，我认为有几个方面的原因。

23. 动态代理本身的使用场景或者需求，只是对原始实现的一个拦截，然后去做一些功能的增强或者扩展。而实际的开发模式也都是基于面向接口来开发，所以基于接口来实现动态代理，从需求和场景都是吻合的。当然确实可能存在有些类没有实现接口的，那这个时候，JDK 动态代理确实无法满足。
24. 在 Java 里面，类的继承关系的设计，更多的是考虑到共性能力的抽象，从而提高代码的重用性和扩展性，而动态代理也是在做这样一个事情，它封装了动态代理类生成的抽象逻辑、判断一个类是否是动态代理类、InvocationHandler 的持有等等，那么把这些抽象的公共逻辑放在 Proxy 这个父类里面，很显然是一个比较正常的设计思路。

总的来说，我认为这个设计上并没有什么特别值得讨论的地方，因为我认为技术方案的设计是解决特定场景问题的。

如果一定要针对普通类来做动态代理，可以选择 cglib 这个组件，它会动态生成一个被代理类的子类，子类重写了父类中所有非 final 修饰的方法，在子类中拦截父类的所有方法调用从而实现动态代理。

以上就是我对于这个问题的理解！

53.98 请说一下对象的创建过程（阿里）

一个工作 5 年的 Java 程序员，去阿里面试，第一面就遇到了一个比较有深度的问题。

“请说一下对象的创建过程？”

唉，Java 生态的技术这么多，你永远不知道面试官会从哪个角度来刁难你。

好吧，对于这个问题，我们来看看高手的回答吧。

以下是高手的回答

（如图）在实例化一个对象的时候，JVM 首先会去检查目标对象是否已经被加载并初始化了。

如果没有，JVM 需要立刻去加载目标类，然后调用目标类的构造器完成初始化。 目标类的加载是通过类加载器来实现的，主要就是把一个类加载到内存里面。

然后初始化的过程，主要是对目标类里面的静态变量、成员变量、静态代码块进行初始化。

当目标类被初始化以后，就可以从常量池里面找到对应的类元信息，并且目标对象的大小在类加载之后就已经确定了，所以这个时候就需要为新创建的对象，根据目标对象的大小在堆内存里面分配内存空间。
内存分配的方式一般有两种，一种指针碰撞，另一种是空闲列表，JVM 会根据 Java 堆内存是否规整来决定内存分配方式。

接下来，JVM 会把目标对象里面的普通成员变量初始化为零值，比如 int 类型初始化为 0，对象类型初始化为 null，（类变量在类加载的准备阶段就已经初始化过了）。
这一步操作主要是保证对象里面的实例字段，不用初始化就可以直接使用，也就是程序能够获得这些字段对应数据类型的零值。
然后，JVM 还需要对目标对象的对象头做一些设置，比如对象所属的类元信息、对象的 GC 分代年龄、hashcode、锁标记等等。

完成这些步骤以后，对于 JVM 来说，新对象的创建工作已经完成。但是基于 Java 语言来说，对象创建才算是开始。
接下来要做的，就是执行目标对象内部生成的 init 方法，初始化成员变量的值、执行构造块、最后执行目标对象的构造方法，完成对象的创建。
其中，init 方法是 Java 文件编译之后在字节码文件中生成的，它是一个实例构造器，这个构造器会把语句块、变量初始化、调用父类构造器等操作组织在一起。所以调用 init 方法能够完成一系列的初始化动作。

以上就是我对这个问题的理解。

53.99 Redis 的内存淘汰算法和原理是什么?

一个工作 5 年的小伙伴私信我，他去京东面试，前面项目部分和应用技术回答得都挺好的。

然后，面试官看我这么牛逼，就开始在问题上加码，不断问我技术原理，前面我都能一一应对。

直到，遇到一个绝杀题：“Redis 的内存淘汰算法和原理是什么？”

虽然最后拿到了月薪 30k 的 offer，但是比较遗憾这个问题没回答上来。

好吧，对于这个问题，我们来看看普通人和高手的回答！

以下是高手的回答

Redis 里面的内存淘汰策略，是指内存的使用率达到 maxmemory 上限的时候的一种内存释放的行为。

Redis 里面提供了很多中内存淘汰算法，归纳起来主要就四种

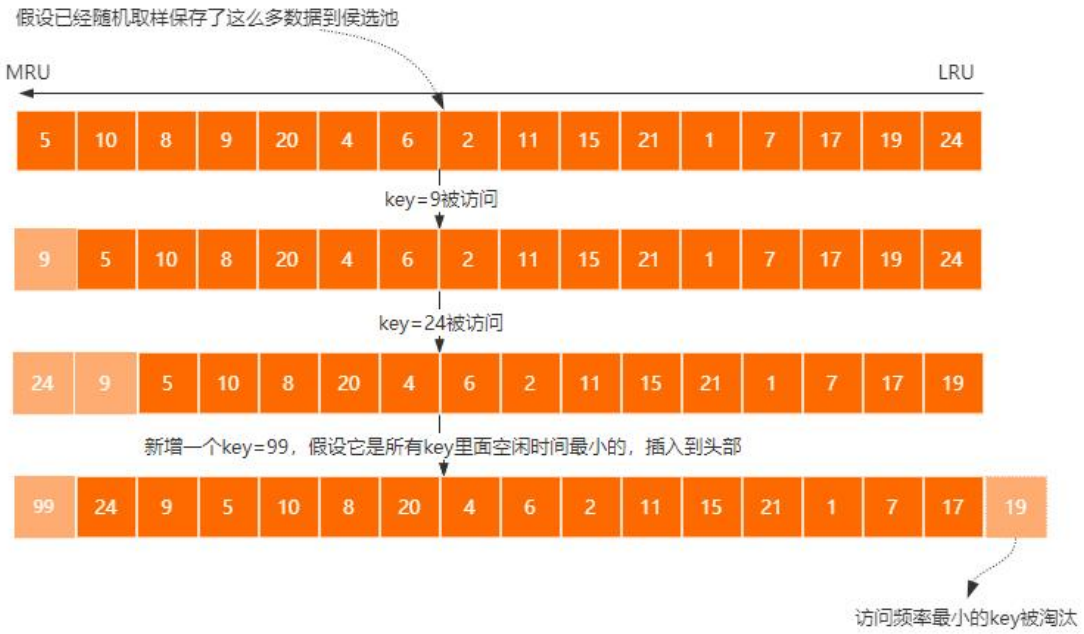
1. Random 算法，随机移除某个 key

2. TTL 算法 ，在设置了过期时间的键中，把更早过期时间的 key 有限移除
3. LRU 算法，移除最近很少使用的 key
4. LFU 算法，移除最近很少使用的 key

（如图）LRU 是比较常见的一种内存淘汰算法，在 Redis 里面会维护一个大小为 16 的侯选池，这个侯选池里面的数据会根据时间进行排序，然后每一次随机取出 5 个 key 放入到这个侯选池里面，当侯选池满了以后，访问的时间间隔最大的 key 就会从侯选池里面取出来淘汰掉。

通过这样的一个设计，就可以把真实的最少访问的 key 从内存中淘汰掉。

但是这样的一种 LRU 算法还是存在一个问题，假如一个 key 的访问频率很低，但是最近一次偶尔被访问到，那么 LRU 就会认为这是一个热点 Key，不会被淘汰。



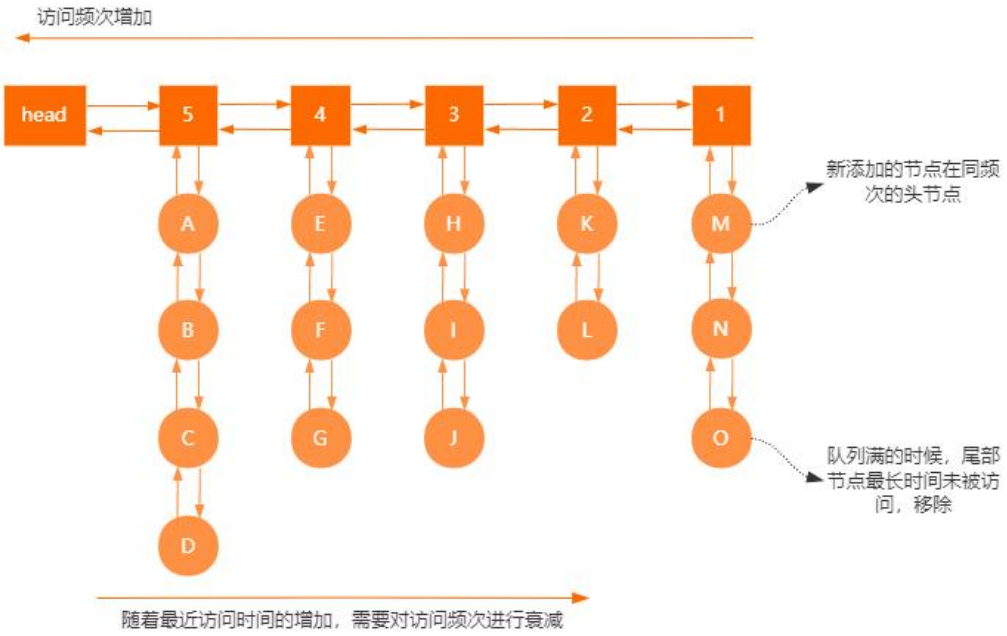
所以在 Redis4 里面，增加了一个 LFU 的算法，相比于 LRU，LFU 增加了访问频率这个维度来统计数据的热点情况。

（如图）LFU 的主要设计是，使用了两个双向链表形成一个二维双向链表，一个用来保存访问频率，另一个用来保存访问频率相同的所有元素。

当添加元素的时候，访问次数默认为 1，于是找到相同访问频次的节点，然后添加到相同频率节点对应的双向链表头部。

当元素被访问的时候，就会增加对应 key 的访问频次，并且把当前访问的节点移动到下一个频次节点。

有可能出现某个数据前期访问次数很多，然后后续就一直不用了，如果单纯按照访问频率，这个 key 就很难被淘汰，所以在 LFU 中通过使用频率和上次访问时间来标记数据的热度，如果有有读写，就增加访问频率，如果一段时间内没有读写，就减少访问频率。



所以，通过 LFU 算法改进之后，就可以真正达到非热点数据的淘汰了。当然，LFU 也有缺点，相比 LRU 算法，LFU 增加了访问频次的维护，以及实现的复杂度要比 LRU 更高。

以上就是我对这个问题的理解。

53.100 面试点评

LRU 算法的使用场景还挺多的，不过主要还是用在内存管理。比如 Mysql 里面的 Buffer_Pool 也用到了 LRU，不过它的实现有点不一样，设计了冷热数据分离的一个结构。

在业务场景中，可以用来解决 top10 的问题， 或者在 IM 的场景中，缓存最近的聊天记录等等。

不难发现，我们去面对一个业务需求来提出技术解决方案的时候，实际上可以从很多技术的底层设计思想和各种中间件里面的设计思想去借鉴，这也是我认为，为什么技术基础会决定技术人的职业高度的原因。

好的，本期的普通人 VS 高手面试系列的视频就到这里结束了，喜欢的朋友记得点赞和收藏。

另外，有任何技术上的问题，职业发展有关的问题，都可以私信我，我会在第一时间回复。

53.101 Kafka 如何保证消息不丢失?

今天分享的这道面试题，是一个工作 2 年的小伙伴私信给我的。

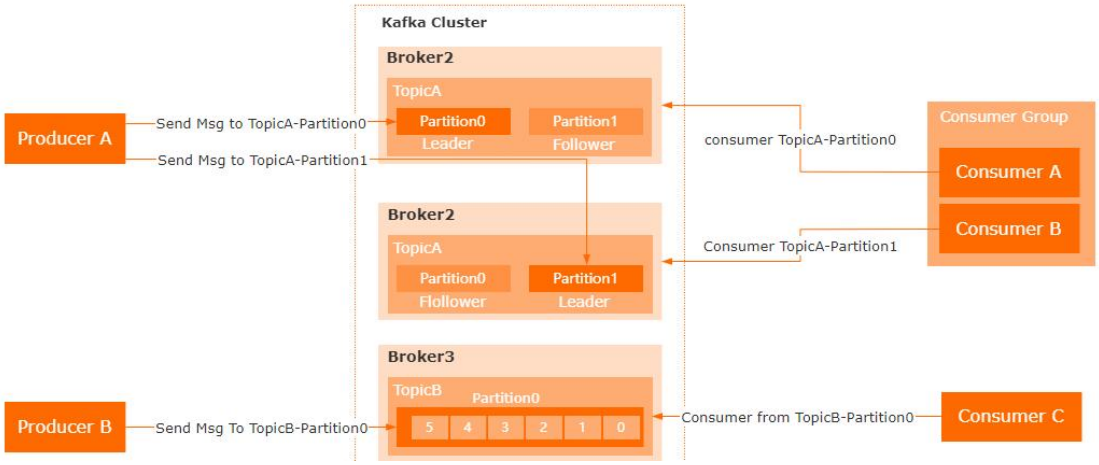
我觉得这个问题比较简单，本来不打算说，但是，唉~ 作为新的 UP 主

满足粉丝的基本要求，才能获得更多的点赞呀~是吧。

关于“Kafka 如何保证消息不丢失”这个问题，看看高手的回答。

以下是高手的回答

（如图）kafka 是一个用来实现异步消息通信的中间件，它的整个架构由 Producer、Consumer、Broker 组成。



所以，对于 kafka 如何保证消息不丢失这个问题，可以从三个方面来考虑和实现。

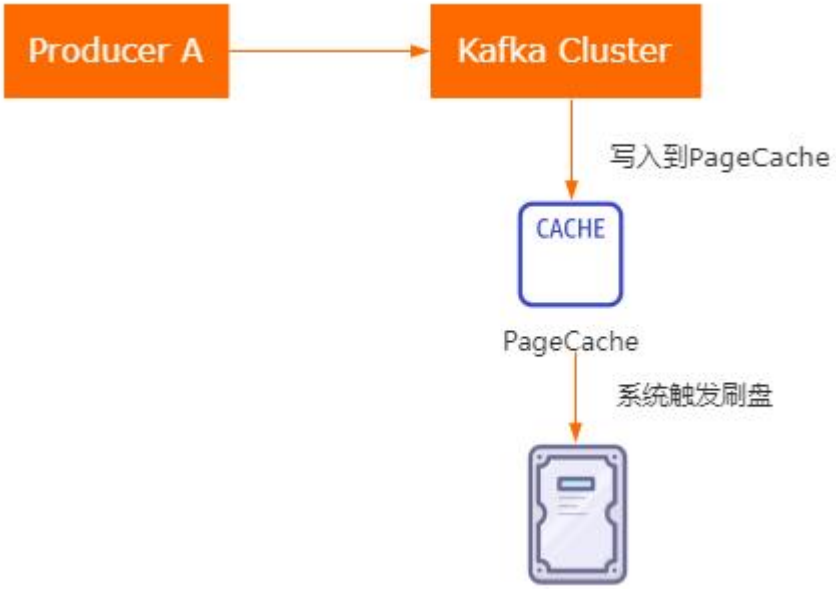
首先是 Producer 端，需要确保消息能够到达 Broker 并实现消息存储，在这个层面，有可能出现网络问题，导致消息发送失败，所以，针对 Producer 端，可以通过 2 种方式来避免消息丢失

1. Producer 默认是异步发送消息，这种情况下要确保消息发送成功，有两个方法
    a. 把异步发送改成同步发送，这样 producer 就能实时知道消息发送的结果。
    b. 添加异步回调函数来监听消息发送的结果，如果发送失败，可以在回调中重试。

2. Producer 本身提供了一个重试参数 retries，如果因为网络问题或者 Broker 故障导致发送失败，Producer 会自动重试。

然后是 Broker 端，Broker 需要确保 Producer 发送过来的消息不会丢失，也就是只需要把消息持久化到磁盘就可以了。

（如图）但是，Kafka 为了提升性能，采用了异步批量刷盘的实现机制，也就是说按照一定的消息量和时间间隔来刷盘，而最终刷新到磁盘的这个动作，是由操作系统来调度的，所以如果在刷盘之前系统崩溃，就会导致数据丢失。



（如图）Kafka 并没有提供同步刷盘的实现，所以针对这个问题，需要通过 Partition 的副本机制和 acks 机制来一起解决。

“我简单说一下 Partition 副本机制，它是针对每个数据分区的高可用策略，每个 partition 副本集包含唯一的一个 Leader 和多个 Follower，Leader 专门处理事务类的请求，Follower 负责同步 Leader 的数据”。

在这样的一种机制的基础上，kafka 提供了一个 acks 的参数，Producer 可以设置 acks 参数再结合 Broker 的副本机制来个共同保障数据的可靠性。

acks 有几个值的选择。

3. acks=0， 表示 producer 不需要等 Broker 的响应，就认为消息发送成功，这种情况会存在消息丢失。

4. acks=1，表示 Broker 中的 Leader Partition 收到消息以后，不等待其他 Follower Partition 同步完，就给 Producer 返回确认，这种情况下 Leader Partition 挂了，会存在数据丢失。
5. acks=-1，表示 Broker 中的 Leader Parititon 收到消息后，并且等待 ISR 列表中的 follower 同步完成，再给 Producer 返回确认，这个配置可以保证数据的可靠性。

最后，就是 Consumer 必须要能消费到这个消息，实际上，我认为，只要 producer 和 broker 的消息可靠的到了保障，那么消费端是不太可能出现消息无法消费的问题，除非是 Consumer 没有消费完这个消息就直接提交了，但是即便是这个情况，也可以通过调整 offset 的值来重新消费。

以上就是我对这个问题的理解。

53.102 请说一下你对分布式锁的理解，以及分布式锁的实现

一个工作了 7 年的 Java 程序员，私信我关于分布式锁的问题。

一上来就两个灵魂拷问：

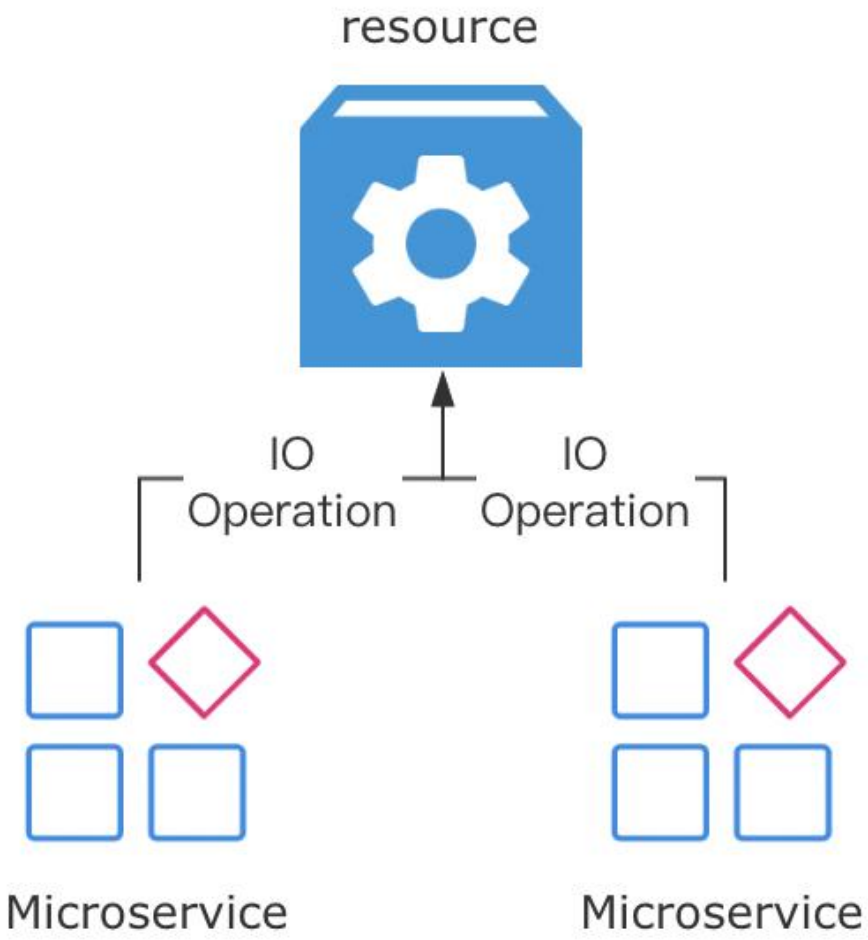
- Redis 锁超时怎么办？
- Redis 主从切换导致锁失效怎么办？

我说，别着急，这些都是小问题。

那么，关于“分布式锁的理解和实现”这个问题，我们看看普通人高手的回答。

以下是高手的回答

（如图）分布式锁，是一种跨进程的跨机器节点的互斥锁，它可以用来保证多机器节点对于共享资源访问的排他性。



我觉得分布式锁和线程锁本质上是一样的，线程锁的生命周期是单进程多线程，分布式锁的声明周期是多进程多机器节点。

在本质上，他们都需要满足锁的几个重要特性：

- 排他性，也就是说，同一时刻只能有一个节点去访问共享资源。
- 可重入性，允许一个已经获得锁的进程，在没有释放锁之前再次重新获得锁。
- 锁的获取、释放的方法
- 锁的失效机制，避免死锁的问题

所以，我认为，只要能够满足这些特性的技术组件都能够实现分布式锁。

1. 关系型数据库，可以使用唯一约束来实现锁的排他性，

如果要针对某个方法加锁，就可以创建一个表包含方法名称字段，

并且把方法名设置成唯一的约束。

那抢占锁的逻辑就是：往表里面插入一条数据，如果已经有其他的线程获得了某个方法的锁，那这个时候插入数据会失败，从而保证了互斥性。

这种方式虽然简单啊，但是要实现比较完整的分布式锁，还需要考虑重入性、锁失效机制、没抢占到锁的线程要实现阻塞等，就会比较麻烦。

2. Redis，它里面提供了 SETNX 命令可以实现锁的排他性，当 key 不存在就返回 1，存在就返回 0。然后还可以用 expire 命令设置锁的失效时间，从而避免死锁问题。

当然有可能存在锁过期了，但是业务逻辑还没执行完的情况。 所以这种情况，可以写一个定时任务对指定的 key 进行续期。

Redisson 这个开源组件，就提供了分布式锁的封装实现，并且也内置了一个 Watch Dog 机制来对 key 做续期。

我认为 Redis 里面这种分布式锁设计已经能够解决 99%的问题了，当然如果在 Redis 搭建了高可用集群的情况下出现主从切换导致 key 失效，这个问题也有可能造成多个线程抢占到同一个锁资源的情况，所以 Redis 官方也提供了一个 RedLock 的解决办法，但是实现会相对复杂一些。

3. 在我看来，分布式锁应该是一个 CP 模型，而 Redis 是一个 AP 模型，所以在集群架构下由于数据的一致性问题导致极端情况下出现多个线程抢占到锁的情况很难避免。

那么基于 CP 模型又能实现分布式锁特性的组件，我认为可以选择 Zookeeper 或者 etcd，

- 在数据一致性方面，zookeeper 用到了 zab 协议来保证数据的一致性，etcd 用到了 raft 算法来保证数据一致性。
- 在锁的互斥方面，zookeeper 可以基于有序节点再结合 Watch 机制实现互斥和唤醒，etcd 可以基于 Prefix 机制和 Watch 实现互斥和唤醒。

以上就是我对于分布式锁的理解！

53.103 volatile 关键字有什么用？它的实现原理是什么？

一个工作了 6 年的 Java 程序员，在阿里二面，被问到“volatile”关键字。

然后，就没有然后了...

同样，另外一个去美团面试的工作 4 年的小伙伴，也被“volatile 关键字“。

然后，也没有然后了...

这个问题说实话，是有点偏底层，但也的确是并发编程里面比较重要的一个关键字。

下面，我们来看看高手对于这个问题的回答吧。

以下是高手的回答

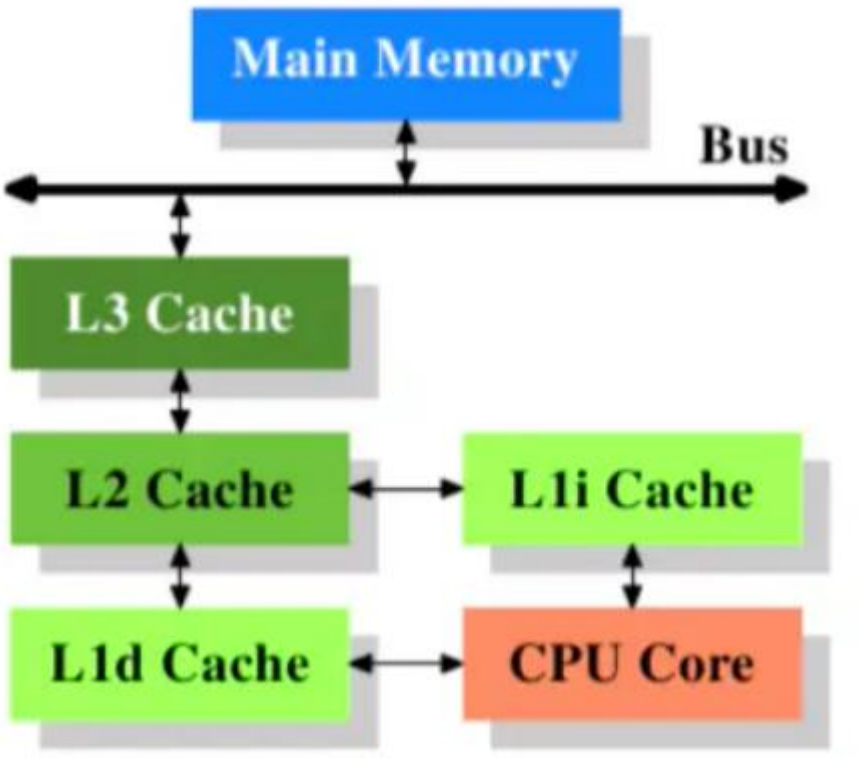
volatile 关键字有两个作用。

1. 可以保证在多线程环境下共享变量的可见性。
2. 通过增加内存屏障防止多个指令之间的重排序。

我理解的可见性，是指当某一个线程对共享变量的修改，其他线程可以立刻看到修改之后的值。

其实这个可见性问题，我认为本质上是由几个方面造成的。

3. （如图）CPU 层面的高速缓存，在 CPU 里面设计了三级缓存去解决 CPU 运算效率和内存 IO 效率问题，但是带来的就是缓存的一致性问题，而在多线程并行执行的情况下，缓存一致性就会导致可见性问题。



所以，对于增加了 volatile 关键字修饰的共享变量，JVM 虚拟机会自动增加一个#Lock 汇编指令，这个指令会根据 CPU 型号自动添加总线锁或/缓存锁

我简单说一下这两种锁，

- 总线锁是锁定了 CPU 的前端总线，从而导致在同一时刻只能有一个线程去和内存通信，这样就避免了多线程并发造成的可见性。
- 缓存锁是对总线锁的优化，因为总线锁导致了 CPU 的使用效率大幅度下降，所以缓存锁只针对 CPU 三级缓存中的目标数据加锁，缓存锁是使用 MESI 缓存一致性来实现的。

4. 指令重排序，所谓重排序，就是指令的编写顺序和执行顺序不一致，在多线程环境下导致可见性问题。指令重排序本质上是一种性能优化的手段，它来自于几个方面。
   - CPU 层面，针对 MESI 协议的更进一步优化去提升 CPU 的利用率，引入了 StoreBuffer 机制，而这一种优化机制会导致 CPU 的乱序执行。当然为了避免这样的问题，CPU 提供了内存屏障指令，上层应用可以在合适的地方插入内存屏障来避免 CPU 指令重排序问题。
   - 编译器的优化，编译器在编译的过程中，在不改变单线程语义和程序正确性的前提下，对指令进行合理的重排序优化来提升性能。

所以，如果对共享变量增加了 volatile 关键字，那么在编译器层面，就不会去触发编译器优化，同时再 JVM 里面，会插入内存屏障指令来避免重排序问题。

当然，除了 volatile 以外，从 JDK5 开始，JMM 就使用了一种 Happens-Before 模型去描述多线程之间的内存可见性问题。

如果两个操作之间具备 Happens-Before 关系，那么意味着这两个操作具备可见性关系，不需要再额外去考虑增加 volatile 关键字来提供可见性保障。

以上就是我对这个问题的理解。

53.104 说说缓存雪崩和缓存穿透的理解，以及如何避免？

听说 10 个人去互联网公司面试，有 9 个人会被问到缓存雪崩和缓存穿透的问题。

听说，这 9 个人里面，至少有 8 个人回答得不完整。

而这 8 个人里面，全都是在网上找的各种面试资料去应付的，并没有真正理解。

当然，也很正常，只有大规模应用缓存的架构才会重点关注这两个问题。

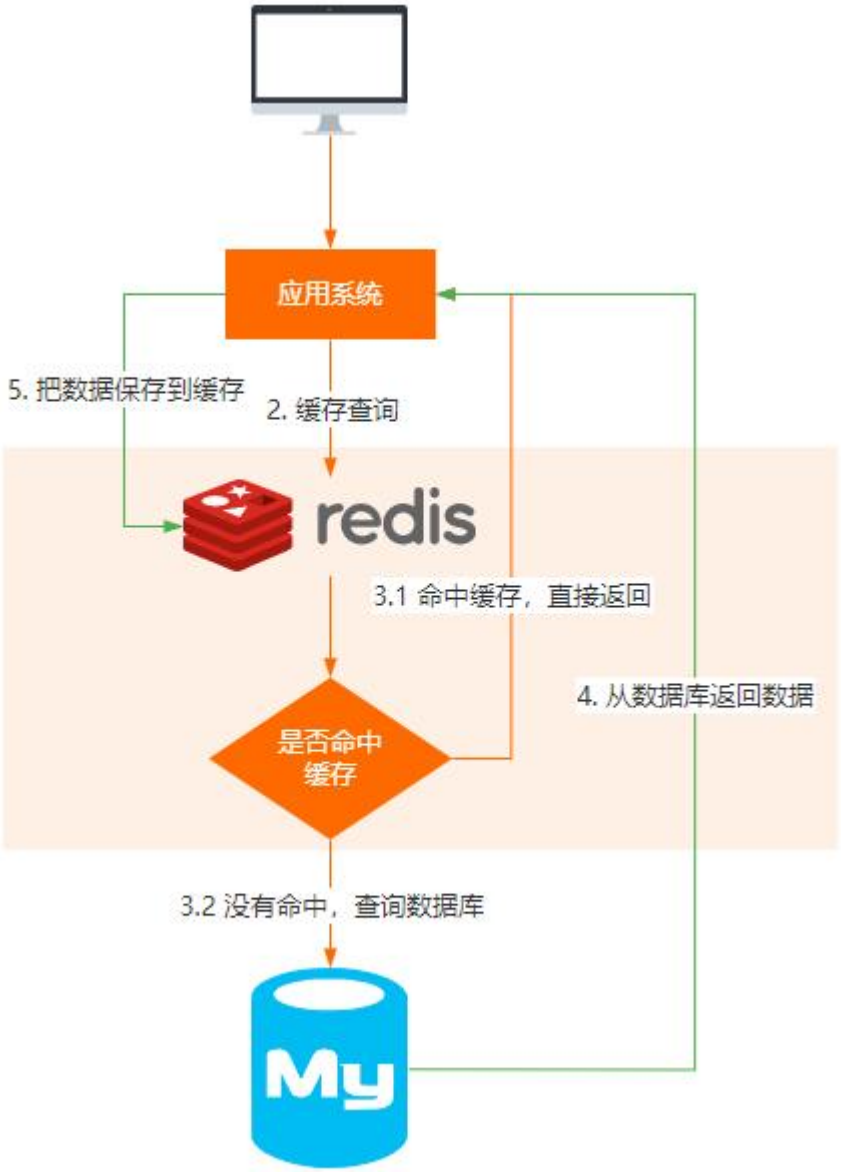
那么如何真正理解这两个问题的底层逻辑，我们来看高手的回答。

> 以下是高手的回答

（如图）缓存雪崩，就是存储在缓存里面的大量数据，在同一个时刻全部过期，

原本缓存组件抗住的大部分流量全部请求到了数据库。

导致数据库压力增加造成数据库服务器崩溃的现象。



导致缓存雪崩的主要原因，我认为有两个：

1. 缓存中间件宕机，当然可以对缓存中间件做高可用集群来避免。
2. 缓存中大部分 key 都设置了相同的过期时间，导致同一时刻这些 key 都过期了。对于这样的情况，可以在失效时间上增加一个 1 到 5 分钟的随机值。

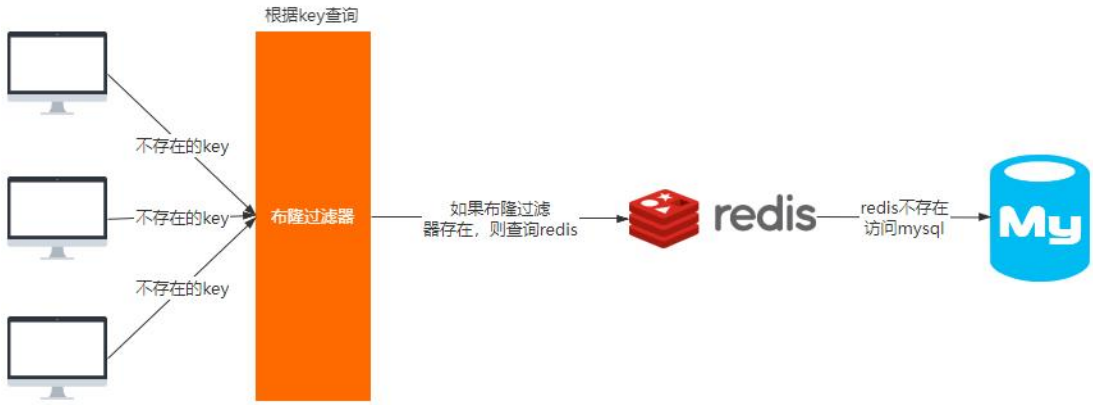
缓存穿透问题，表示是短时间内有大量的不存在的 key 请求到应用里面，而这些不存在的 key 在缓存里面又找不到，从而全部穿透到了数据库，造成数据库压力。

我认为这个场景的核心问题是针对缓存的一种攻击行为，因为在正常的业务里面，即便是出现了这样的情况，由于缓存的不断预热，影响不会很大。

而攻击行为就需要具备时间是的持续性，而只有 key 确实在数据库里面也不存在的情况下，才能达到这个目的，所以，我认为有两个方法可以解决：

3. 把无效的 key 也保存到 Redis 里面，并且设置一个特殊的值，比如“null”，这样的话下次再来访问，就不会去查数据库了。
4. （如图）但是如果攻击者不断用随机的不存在的 key 来访问，也还是会存在问题，所以可以用布隆过滤器来实现，在系统启动的时候把目标数据全部缓存到布隆过滤器里面，当攻击者用不存在的 key 来请求的时候，先到布隆过滤器里面查询，如果不存在，那意味着这个 key 在数据库里面也不存在。

布隆过滤器还有一个好处，就是它采用了 bitmap 来进行数据存储，占用的内存空间很少。



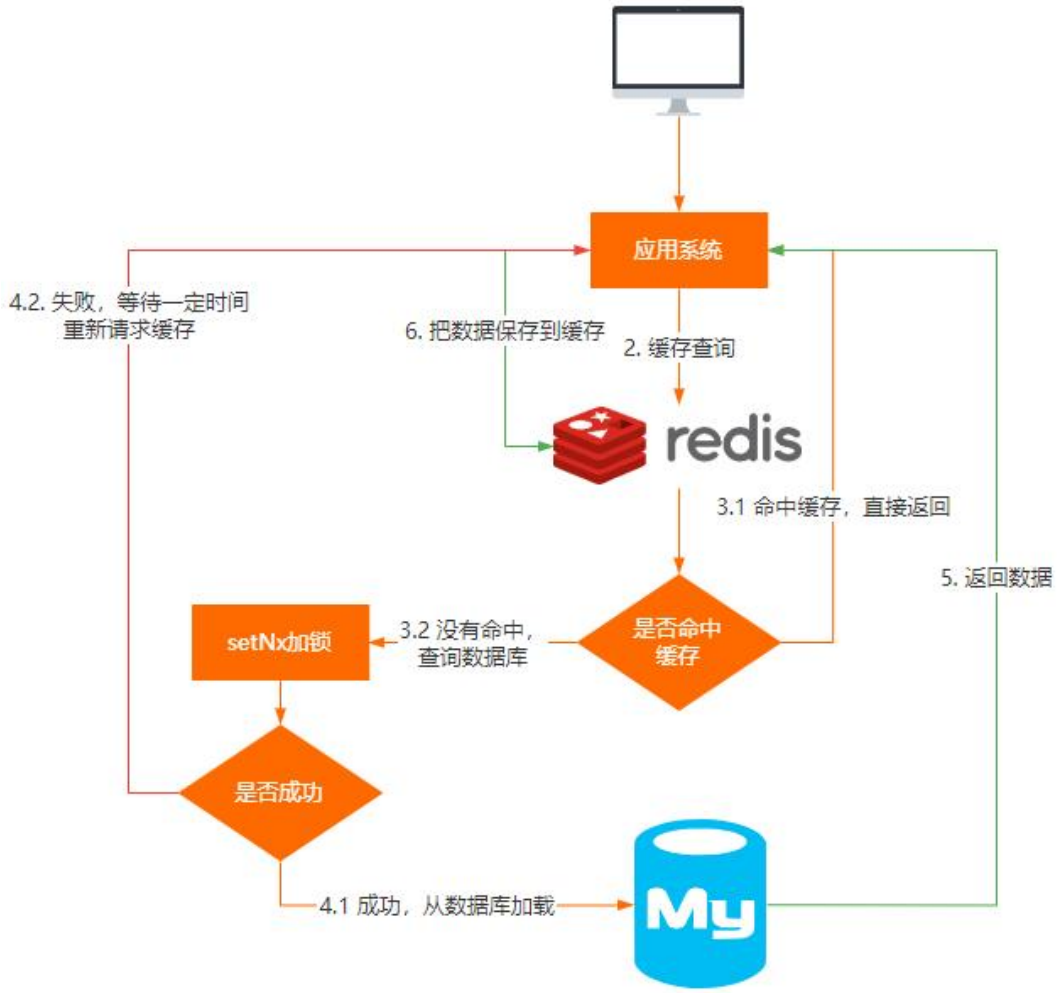
不过，在我看来，您提出来的这个问题，有点过于放大了它带来的影响。

首先，在一个成熟的系统里面，对于比较重要的热点数据，必然会有一个专门缓存系统来维护，同时它的过期时间的维护必然和其他业务的 key 会有一定的差别。而且非常重要的场景，我们还会设计多级缓存系统。

其次，即便是触发了缓存雪崩，数据库本身的容灾能力也并没有那么脆弱，数据库的主从、双主、读写分离这些策略都能够很好的缓解并发流量。

最后，数据库本身也有最大连接数的限制，超过限制的请求会被拒绝，再结合熔断机制，也能够很好的保护数据库系统，最多就是造成部分用户体验不好。

（如图）另外，在程序设计上，为了避免缓存未命中导致大量请求穿透到数据库的问题，还可以在访问数据库这个环节加锁。虽然影响了性能，但是对系统是安全的。



总而言之，我认为解决的办法很多，具体选择哪种方式，还是看具体的业务场景。

以上就是我对这个问题的理解。

53.105 请说一下你对分布式锁的理解，以及分布式锁的实现

一个工作了 7 年的 Java 程序员，私信我关于分布式锁的问题。

一上来就两个灵魂拷问：

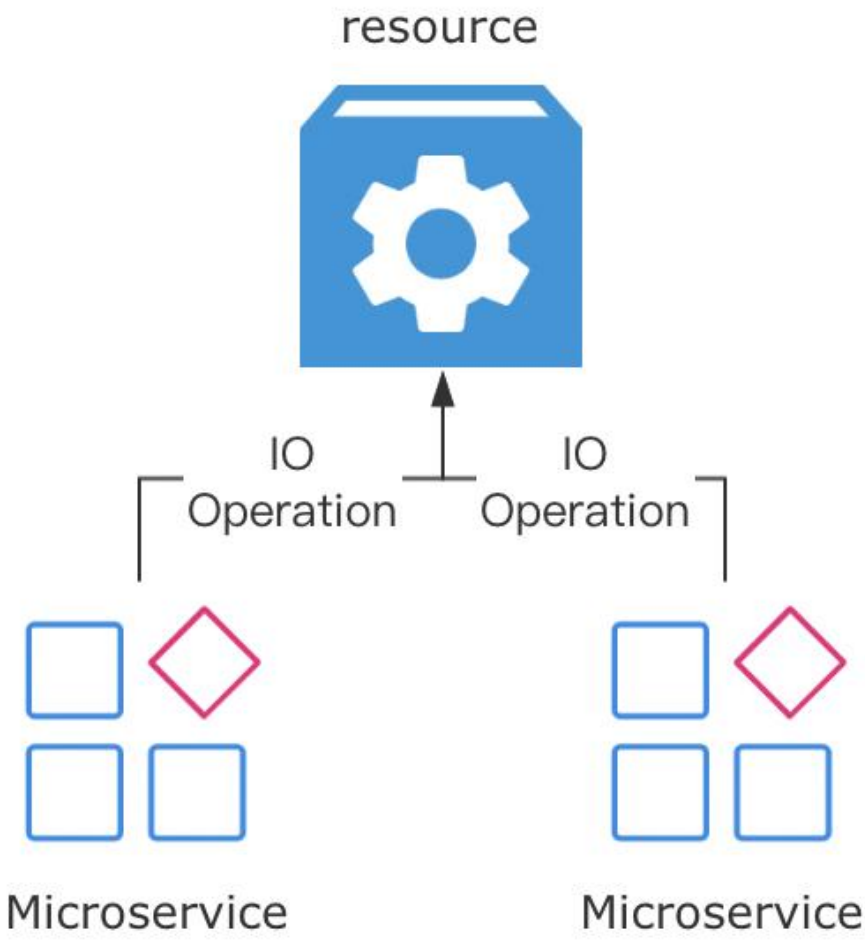
- Redis 锁超时怎么办？
- Redis 主从切换导致锁失效怎么办？

我说，别着急，这些都是小问题。

那么，关于“分布式锁的理解和实现”这个问题，我们看看高手的回答。

以下是高手的回答

（如图）分布式锁，是一种跨进程的跨机器节点的互斥锁，它可以用来保证多机器节点对于共享资源访问的排他性。



我觉得分布式锁和线程锁本质上是一样的，线程锁的生命周期是单进程多线程，分布式锁的声明周期是多进程多机器节点。

在本质上，他们都需要满足锁的几个重要特性：

- 排他性，也就是说，同一时刻只能有一个节点去访问共享资源。
- 可重入性，允许一个已经获得锁的进程，在没有释放锁之前再次重新获得锁。
- 锁的获取、释放的方法
- 锁的失效机制，避免死锁的问题

所以，我认为，只要能够满足这些特性的技术组件都能够实现分布式锁。

1. 关系型数据库，可以使用唯一约束来实现锁的排他性，

如果要针对某个方法加锁，就可以创建一个表包含方法名称字段，

并且把方法名设置成唯一的约束。

那抢占锁的逻辑就是：往表里面插入一条数据，如果已经有其他的线程获得了某个方法的锁，那这个时候插入数据会失败，从而保证了互斥性。

这种方式虽然简单啊，但是要实现比较完整的分布式锁，还需要考虑重入性、锁失效机制、没抢占到锁的线程要实现阻塞等，就会比较麻烦。

2. Redis，它里面提供了 SETNX 命令可以实现锁的排他性，当 key 不存在就返回 1，存在就返回 0。然后还可以用 expire 命令设置锁的失效时间，从而避免死锁问题。

当然有可能存在锁过期了，但是业务逻辑还没执行完的情况。 所以这种情况，可以写一个定时任务对指定的 key 进行续期。

Redisson 这个开源组件，就提供了分布式锁的封装实现，并且也内置了一个 Watch Dog 机制来对 key 做续期。

我认为 Redis 里面这种分布式锁设计已经能够解决 99%的问题了，当然如果在 Redis 搭建了高可用集群的情况下出现主从切换导致 key 失效，这个问题也有可能造成多个线程抢占到同一个锁资源的情况，所以 Redis 官方也提供了一个 RedLock 的解决办法，但是实现会相对复杂一些。

3. 在我看来，分布式锁应该是一个 CP 模型，而 Redis 是一个 AP 模型，所以在集群架构下由于数据的一致性问题导致极端情况下出现多个线程抢占到锁的情况很难避免。

那么基于 CP 模型又能实现分布式锁特性的组件，我认为可以选择 Zookeeper 或者 etcd，

- 在数据一致性方面，zookeeper 用到了 zab 协议来保证数据的一致性，etcd 用到了 raft 算法来保证数据一致性。
- 在锁的互斥方面，zookeeper 可以基于有序节点再结合 Watch 机制实现互斥和唤醒，etcd 可以基于 Prefix 机制和 Watch 实现互斥和唤醒。

以上就是我对于分布式锁的理解！

53.106 volatile 关键字有什么用？它的实现原理是什么？

一个工作了 6 年的 Java 程序员，在阿里二面，被问到“volatile”关键字。

然后，就没有然后了...

同样，另外一个去美团面试的工作 4 年的小伙伴，也被“volatile 关键字 “。

然后，也没有然后了...

这个问题说实话，是有点偏底层，但也的确是并发编程里面比较重要的一个关键字。

下面，我们来看看普通人和高手对于这个问题的回答吧。

以下是高手的回答

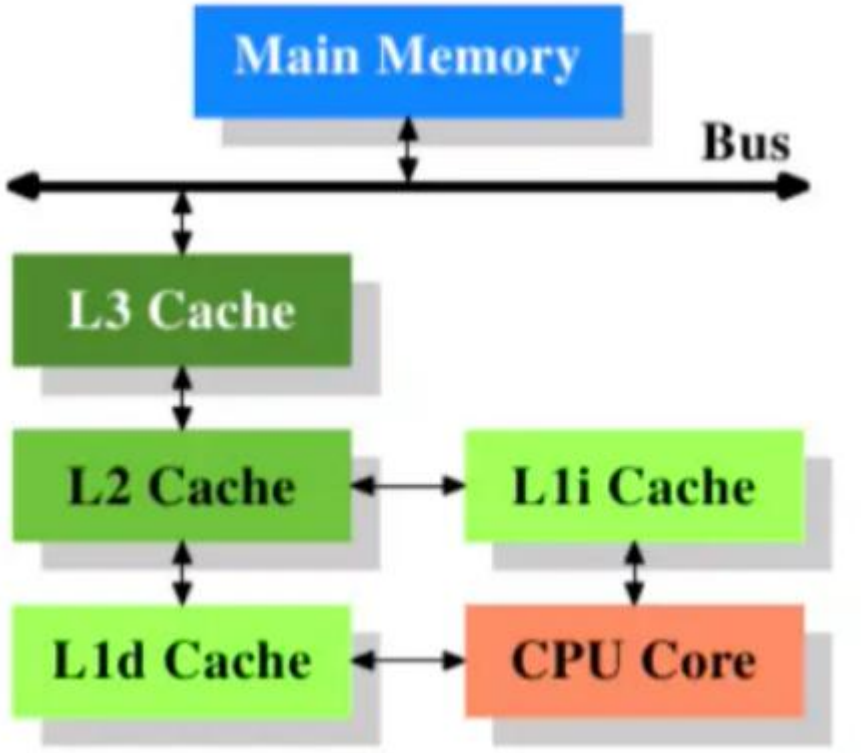
volatile 关键字有两个作用。

1. 可以保证在多线程环境下共享变量的可见性。
2. 通过增加内存屏障防止多个指令之间的重排序。

我理解的可见性，是指当某一个线程对共享变量的修改，其他线程可以立刻看到修改之后的值。

其实这个可见性问题，我认为本质上是由几个方面造成的。

3. （如图）CPU 层面的高速缓存，在 CPU 里面设计了三级缓存去解决 CPU 运算效率和内存 IO 效率问题，但是带来的就是缓存的一致性问题，而在多线程并行执行的情况下，缓存一致性就会导致可见性问题。



所以，对于增加了 volatile 关键字修饰的共享变量，JVM 虚拟机会自动增加一个#Lock 汇编指令，这个指令会根据 CPU 型号自动添加总线锁或/缓存锁

我简单说一下这两种锁，

- 总线锁是锁定了 CPU 的前端总线，从而导致在同一时刻只能有一个线程去和内存通信，这样就避免了多线程并发造成的可见性。
- 缓存锁是对总线锁的优化，因为总线锁导致了 CPU 的使用效率大幅度下降，所以缓存锁只针对 CPU 三级缓存中的目标数据加锁，缓存锁是使用 MESI 缓存一致性来实现的。

4. 指令重排序，所谓重排序，就是指令的编写顺序和执行顺序不一致，在多线程环境下导致可见性问题。指令重排序本质上是一种性能优化的手段，它来自于几个方面。
   - CPU 层面，针对 MESI 协议的更进一步优化去提升 CPU 的利用率，引入了 StoreBuffer 机制，而这一种优化机制会导致 CPU 的乱序执行。当然为了避免这样的问题，CPU 提供了内存屏障指令，上层应用可以在合适的地方插入内存屏障来避免 CPU 指令重排序问题。
   - 编译器的优化，编译器在编译的过程中，在不改变单线程语义和程序正确性的前提下，对指令进行合理的重排序优化来提升性能。

所以，如果对共享变量增加了 volatile 关键字，那么在编译器层面，就不会去触发编译器优化，同时再 JVM 里面，会插入内存屏障指令来避免重排序问题。

当然，除了 volatile 以外，从 JDK5 开始，JMM 就使用了一种 Happens-Before 模型去描述多线程之间的内存可见性问题。

如果两个操作之间具备 Happens-Before 关系，那么意味着这两个操作具备可见性关系，不需要再额外去考虑增加 volatile 关键字来提供可见性保障。

以上就是我对这个问题的理解。

53.107 说说缓存雪崩和缓存穿透的理解，以及如何避免？

听说 10 个人去互联网公司面试，有 9 个人会被问到缓存雪崩和缓存穿透的问题。

听说，这 9 个人里面，至少有 8 个人回答得不完整。

而这 8 个人里面，全都是在网上找的各种面试资料去应付的，并没有真正理解。

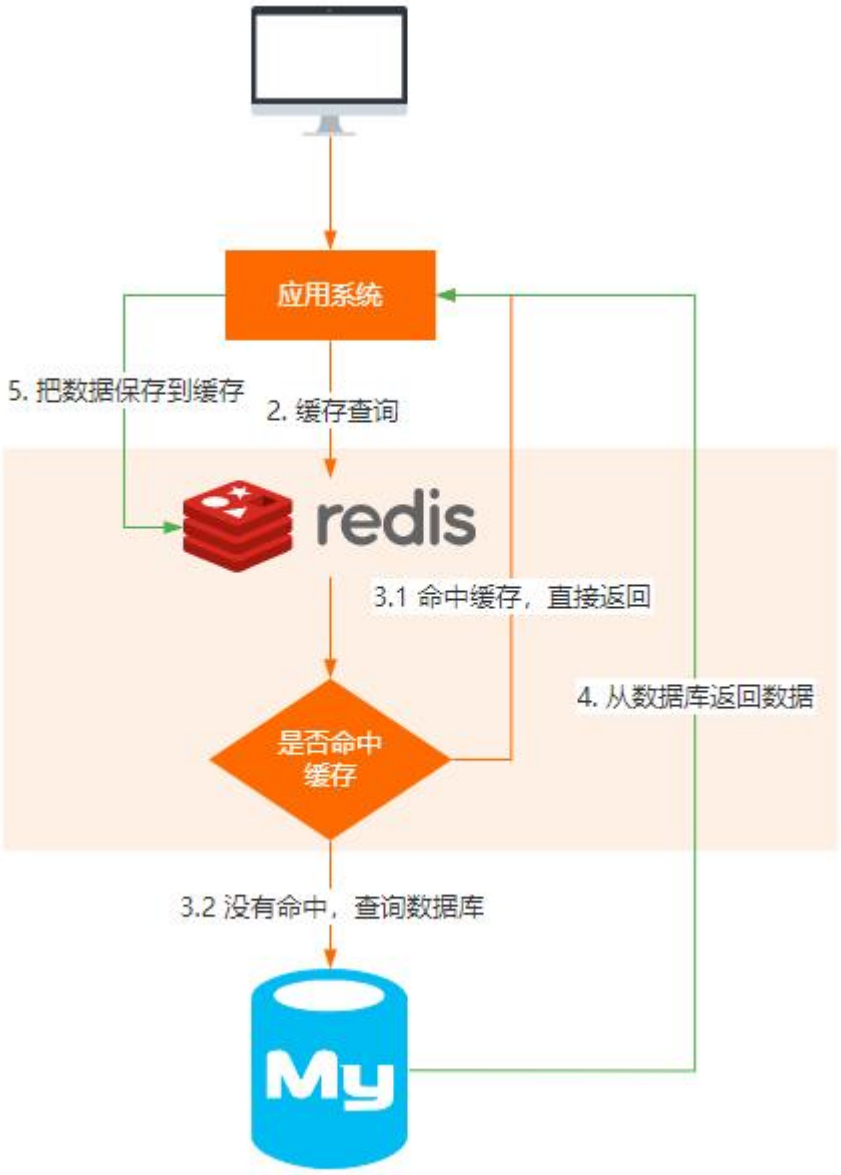
当然，也很正常，只有大规模应用缓存的架构才会重点关注这两个问题。

那么如何真正理解这两个问题的底层逻辑，我们来看普通人和高手的回答。

以下是高手的回答

（如图）缓存雪崩，就是存储在缓存里面的大量数据，在同一个时刻全部过期，原本缓存组件抗住的大部分流量全部请求到了数据库。

导致数据库压力增加造成数据库服务器崩溃的现象。



导致缓存雪崩的主要原因，我认为有两个：

1. 缓存中间件宕机，当然可以对缓存中间件做高可用集群来避免。
2. 缓存中大部分 key 都设置了相同的过期时间，导致同一时刻这些 key 都过期了。对于这样的情况，可以在失效时间上增加一个 1 到 5 分钟的随机值。

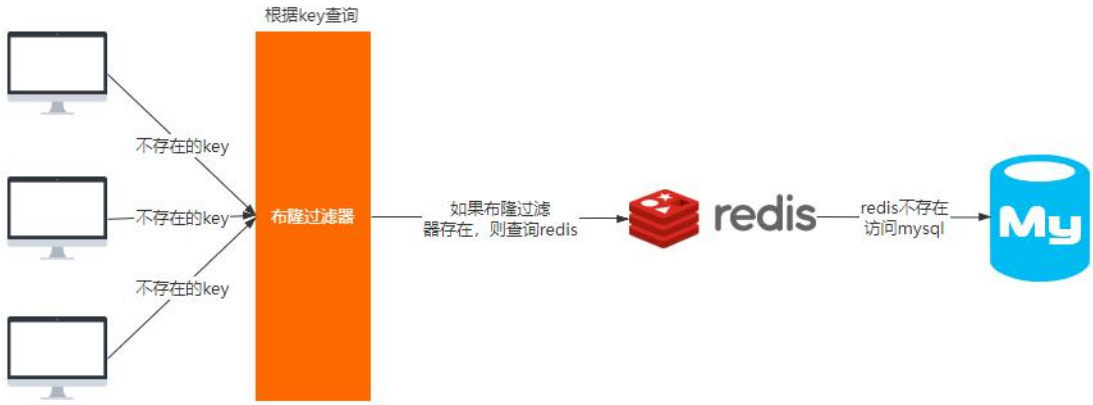
缓存穿透问题，表示是短时间内有大量的不存在的 key 请求到应用里面，而这些不存在的 key 在缓存里面又找不到，从而全部穿透到了数据库，造成数据库压力。

我认为这个场景的核心问题是针对缓存的一种攻击行为，因为在正常的业务里面，即便是出现了这样的情况，由于缓存的不断预热，影响不会很大。

而攻击行为就需要具备时间是的持续性，而只有 key 确实在数据库里面也不存在的情况下，才能达到这个目的，所以，我认为有两个方法可以解决：

3. 把无效的 key 也保存到 Redis 里面，并且设置一个特殊的值，比如“null”，这样的话下次再来访问，就不会去查数据库了。
4. （如图）但是如果攻击者不断用随机的不存在的 key 来访问，也还是会存在问题，所以可以用布隆过滤器来实现，在系统启动的时候把目标数据全部缓存到布隆过滤器里面，当攻击者用不存在的 key 来请求的时候，先到布隆过滤器里面查询，如果不存在，那意味着这个 key 在数据库里面也不存在。

布隆过滤器还有一个好处，就是它采用了 bitmap 来进行数据存储，占用的内存空间很少。



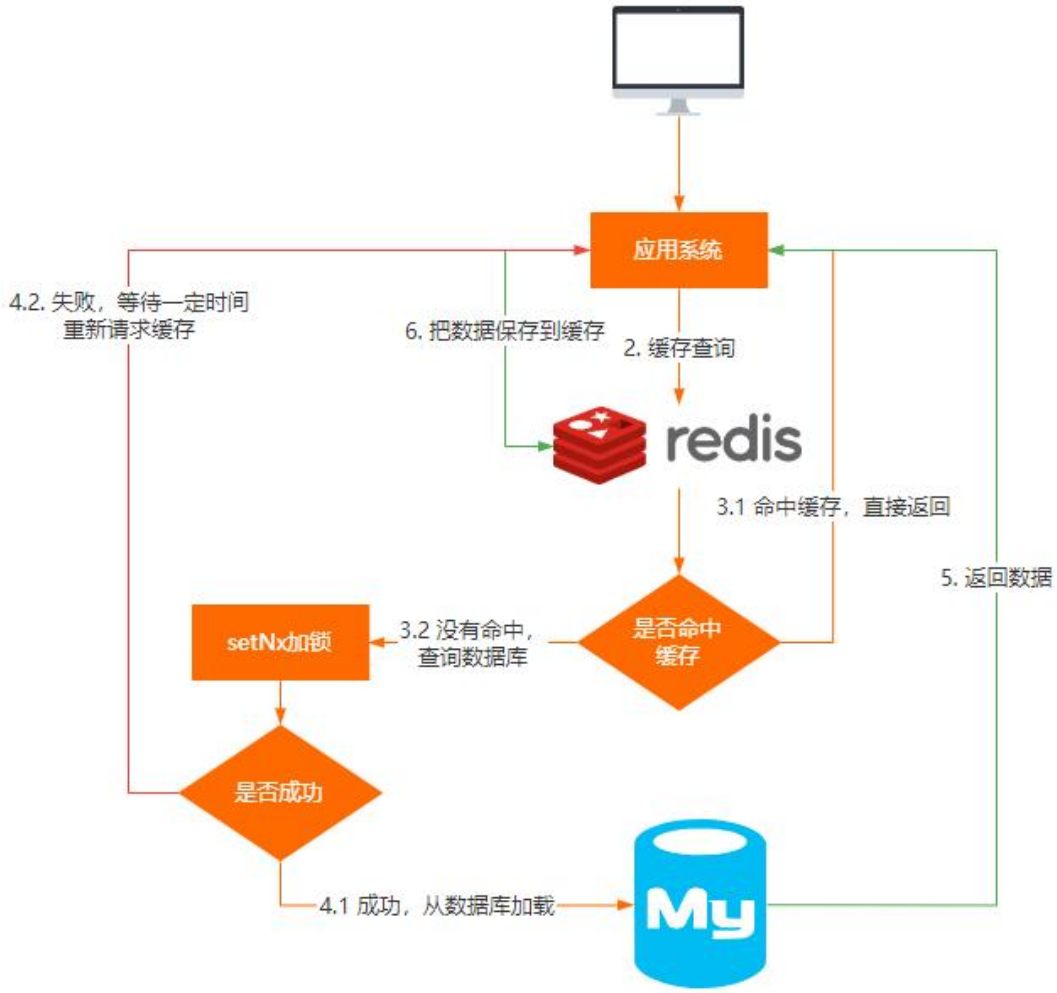
不过，在我看来，您提出来的这个问题，有点过于放大了它带来的影响。

首先，在一个成熟的系统里面，对于比较重要的热点数据，必然会有一个专门缓存系统来维护，同时它的过期时间的维护必然和其他业务的 key 会有一定的差别。而且非常重要的场景，我们还会设计多级缓存系统。

其次，即便是触发了缓存雪崩，数据库本身的容灾能力也并没有那么脆弱，数据库的主从、双主、读写分离这些策略都能够很好的缓解并发流量。

最后，数据库本身也有最大连接数的限制，超过限制的请求会被拒绝，再结合熔断机制，也能够很好的保护数据库系统，最多就是造成部分用户体验不好。

（如图）另外，在程序设计上，为了避免缓存未命中导致大量请求穿透到数据库的问题，还可以在访问数据库这个环节加锁。虽然影响了性能，但是对系统是安全的。



总而言之，我认为解决的办法很多，具体选择哪种方式，还是看具体的业务场景。

以上就是我对这个问题的理解。

53.108 讲一下 wait 和 notify 这个为什么要在 synchronized 代码块中?

一个工作七年的小伙伴，竟然不知道”wait”和“notify”为什么要在 Synchronized 代码块里面。

好吧，如果屏幕前的你也不知道，请在公屏上刷”不知道“。

对于这个问题，我们来看看高手的回答。

以下是高手的回答

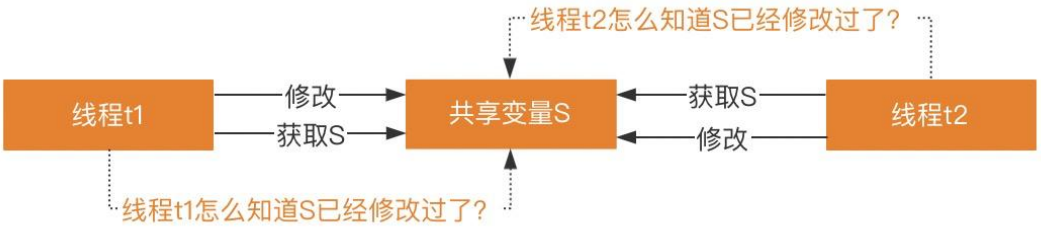
1. wait 和 notify 用来实现多线程之间的协调，wait 表示让线程进入到阻塞状态，notify 表示让阻塞的线程唤醒。

2. wait 和 notify 必然是成对出现的，如果一个线程被 wait()方法阻塞，那么必然需要另外一个线程通过 notify()方法来唤醒这个被阻塞的线程，从而实现多线程之间的通信。
3. （如图）在多线程里面，要实现多个线程之间的通信，除了管道流以外，只能通过共享变量的方法来实现，也就是线程 t1 修改共享变量 s，线程 t2 获取修改后的共享变量 s，从而完成数据通信。

但是多线程本身具有并行执行的特性，也就是在同一时刻，多个线程可以同时执行。在这种情况下，线程 t2 在访问共享变量 s 之前，必须要知道线程 t1 已经修改过了共享变量 s，否则就需要等待。

同时，线程 t1 修改过了共享变量 S 之后，还需要通知在等待中的线程 t2。

所以要在这种特性下要去实现线程之间的通信，就必须要有一个竞争条件控制线程在什么条件下等待，什么条件下唤醒。



4. 而 Synchronized 同步关键字就可以实现这样一个互斥条件，也就是在通过共享变量来实现多个线程通信的场景里面，参与通信的线程必须要竞争到这个共享变量的锁资源，才有资格对共享变量做修改，修改完成后就释放锁，那么其他的线程就可以再次来竞争同一个共享变量的锁来获取修改后的数据，从而完成线程之前的通信。
5. 所以这也是为什么 wait/notify 需要放在 Synchronized 同步代码块中的原因，有了 Synchronized 同步锁，就可以实现对多个通信线程之间的互斥，实现条件等待和条件唤醒。
6. 另外，为了避免 wait/notify 的错误使用，jdk 强制要求把 wait/notify 写在同步代码块里面，否则会抛出 IllegalMonitorStateException
7. 最后，基于 wait/notify 的特性，非常适合实现生产者消费者的模型，比如说用 wait/notify 来实现连接池就绪前的等待与就绪后的唤醒。

以上就是我对 wait/notify 这个问题的理解。

53.109 ThreadLocal 是什么？它的实现原理呢？

一个工作了 4 年的小伙伴，又私信了我一个并发编程里面的问题。

他说他要抓狂了，每天 CRUD，也没用到过 ThreadLocal 啊，怎么就不能问我怎么写 CRUD 呢？

我反问他，如果只问你项目和业务，那有些 4 年的小伙伴他要求月薪 30K，有些只要求月薪 15K，

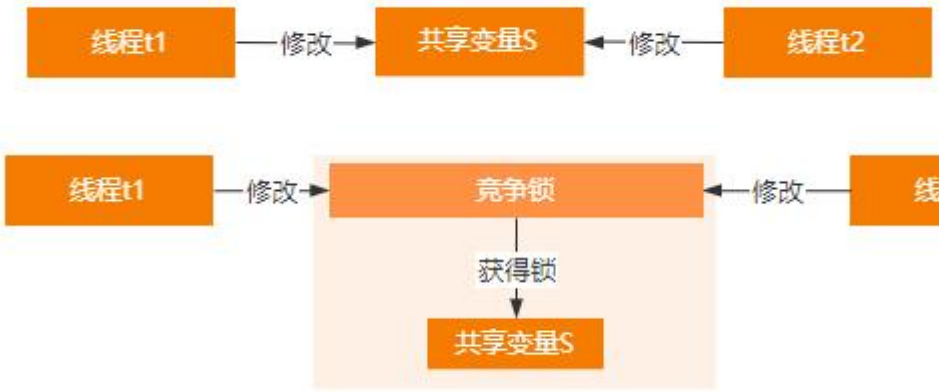
那请问，凭什么每个月要多出 15k 给你？我花 30k 招两个 15k 的，不能写 CRUD 吗？

好吧，我们来看看 ThreadLocal 这个问题，普通人和高手的回答。

> 以下是高手的回答

好的，这个问题我从三个方面来回答。

1. ThreadLocal 是一种线程隔离机制，它提供了多线程环境下对于共享变量访问的安全性。
2. 在多线程访问共享变量的场景中（出现下面第一个图），一般的解决办法是对共享变量加锁（出现下面第二个图），从而保证在同一时刻只有一个线程能够对共享变量进行更新，并且基于 Happens-Before 规则里面的监视器锁规则，又保证了数据修改后对其他线程的可见性。



3. 但是加锁会带来性能的下降，所以 ThreadLocal 用了一种空间换时间的设计思想，也就是说在每个线程里面，都有一个容器来存储共享变量的副本，然后每个线程只对自己的变量副本来做更新操作，这样既解决了线程安全问题，又避免了多线程竞争加锁的开销。

4. ThreadLocal 的具体实现原理是，在 Thread 类里面有一个成员变量 ThreadLocalMap，它专门来存储当前线程的共享变量副本，后续这个线程对于共享变量的操作，都是从这个 ThreadLocalMap 里面进行变更，不会影响全局共享变量的值。

以上就是我对这个问题的理解。

53.110 基于数组的阻塞队列 ArrayBlockingQueue 原理

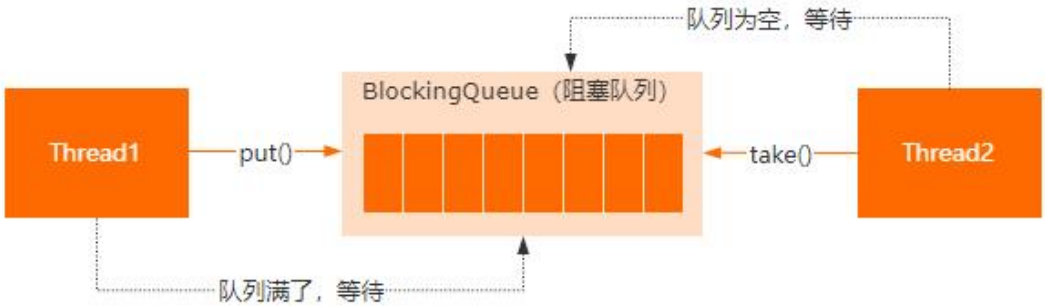
今天来分享一道“饿了么”的高级工程师的面试题。

“基于数组的阻塞队列 ArrayBlockingQueue”的实现原理。

关于这个问题，我们来看看高手的回答。

> 以下是高手的回答

1. （如图）阻塞队列（BlockingQueue）是在队列的基础上增加了两个附加操作，
   - 在队列为空的时候，获取元素的线程会等待队列变为非空。
   - 当队列满时，存储元素的线程会等待队列可用。



2. 由于阻塞队列的特性，可以非常容易实现生产者消费者模型，也就是生产者只需要关心数据的生产，消费者只需要关注数据的消费，所以如果队列满了，生产者就等待，同样，队列空了，消费者也需要等待。
3. 要实现这样的一个阻塞队列，需要用到两个关键的技术，队列元素的存储、以及线程阻塞和唤醒。
4. 而 ArrayBlockingQueue 是基于数组结构的阻塞队列，也就是队列元素是存储在一个数组结构里面，并且由于数组有长度限制，为了达到循环生产和循环消费的目的，ArrayBlockingQueue 用到了循环数组。
5. 而线程的阻塞和唤醒，用到了 J.U.C 包里面的 ReentrantLock 和 Condition。Condition 相当于 wait/notify 在 JUC 包里面的实现。

以上就是我对这个问题的理解。

53.111 什么是聚集索引和非聚集索引

一个去阿里面试并且第一面就挂了的粉丝私信我，被数据库里面几个问题难倒了，他说面试官问了事务隔离级别、MVCC、聚集索引/非聚集索引、B 树、B+树这些，没回答好。

大厂面试基本上是这样，由点到面去展开，如果你对这个技术理解不够全面，很容易就会被看出来。

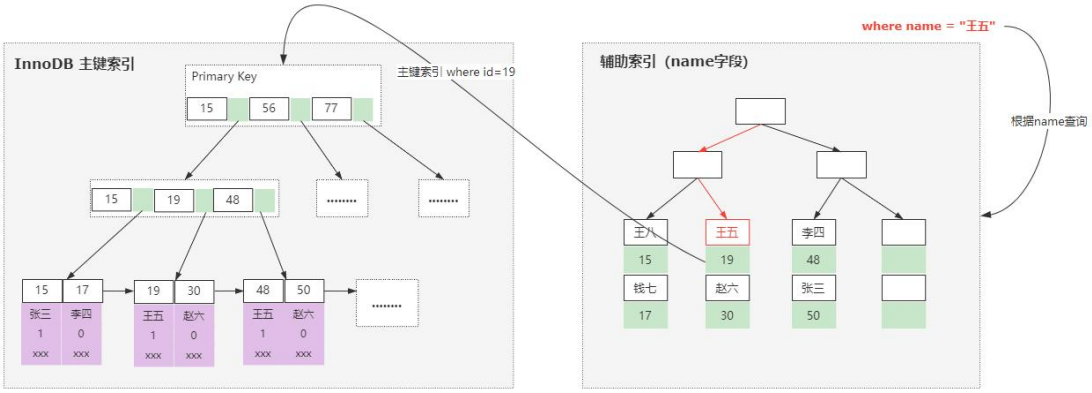
ok，关于“什么是聚集索引和非聚集索引”这个问题，看看普通人和高手的回答。

以下是高手的回答

1. 简单来说，聚集索引就是基于主键创建的索引，除了主键索引以外的其他索引，称为非聚集索引，也叫做二级索引。
2. 由于在 InnoDB 引擎里面，一张表的数据对应的物理文件本身就是按照 B+树来组织的一种索引结构，而聚集索引就是按照每张表的主键来构建一颗 B+树，然后叶子节点里面存储了这个表的每一行数据记录。
3. 所以基于 InnoDB 这样的特性，聚集索引并不仅仅是一种索引类型，还代表着一种数据的存储方式。
4. 同时也意味着每个表里面必须要有一个主键，如果没有主键，InnoDB 会默认选择或者添加一个隐藏列作为主键索引来存储这个表的数据行。一般情况是建议使用自

增 id 作为主键，这样的话 id 本身具有连续性使得对应的数据也会按照顺序存储在磁盘上，写入性能和检索性能都很高。否则，如果使用 uuid 这种随机 id，那么在频繁插入数据的时候，就会导致随机磁盘 IO，从而导致性能较低。

5. 需要注意的是，InnoDB 里面只能存在一个聚集索引，原因很简单，如果存在多个聚集索引，那么意味着这个表里面的数据存在多个副本，造成磁盘空间的浪费，以及数据维护的困难。
6. （如图）由于在 InnoDB 里面，主键索引表示的是一种数据存储结构，所以如果是基于非聚集索引来查询一条完整的记录，最终还是需要访问主键索引来检索。



53.112 什么是双亲委派?

Hi，大家好。

今天我们来分享一道关于 Java 类加载方面的面试题。在国内的一二线互联网公司面试的时候，面试官通常是使用这方面的问题来暖场，但往往造成的是冷场~

比如，什么是双亲委派？什么是类加载？ new String()生成了几个对象等等。

双亲委派的英文是 parent delegation model，我认为从真正的实现逻辑来看，正确的翻译应该是父委托模型。

不管它叫什么，我们先来看看遇到这个问题应该怎么回答。

以下是高手的回答

关于这个问题，需要从几个方面来回答。

首先，我简单说一下类的加载机制（如图），就是我们自己写的 java 源文件到最终运行，必须要经过编译和类加载两个阶段。

编译的过程就是把.java 文件编译成.class 文件。

类加载的过程，就是把 class 文件装载到 JVM 内存中，装载完成以后就会得到一个 Class 对象，我们就可以使用 new 关键字来实例化这个对象。

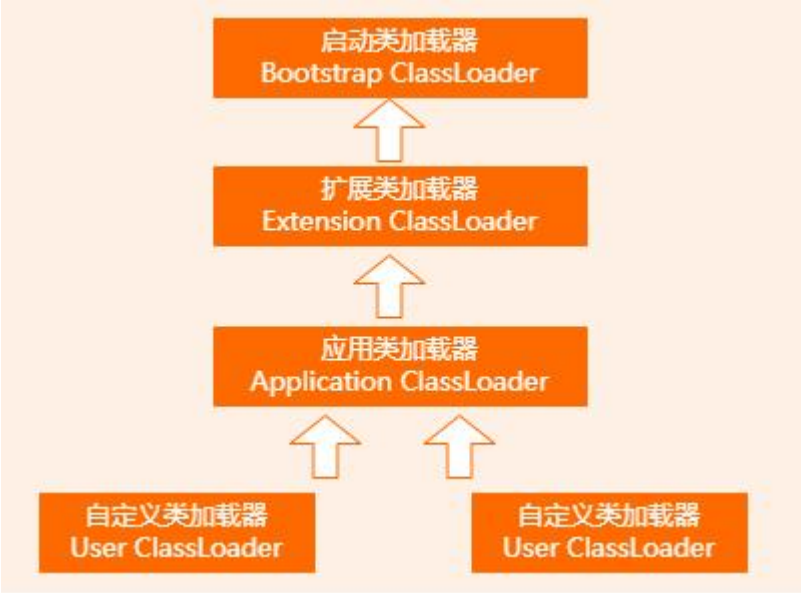
（如图）而类的加载过程，需要涉及到类加载器。

JVM 在运行的时候，会产生 3 个类加载器，这三个类加载器组成了一个层级关系

每个类加载器分别去加载不同作用范围的 jar 包，比如

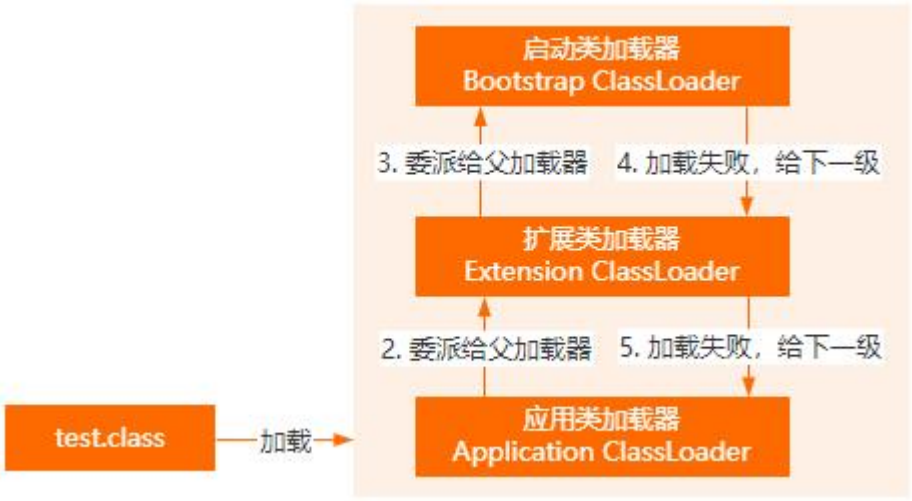
- Bootstrap ClassLoader，主要是负责 Java 核心类库的加载，也就是 %{JDK_HOME}\lib 下的 rt.jar、resources.jar 等
- Extension ClassLoader，主要负责%{JDK_HOME}\lib\ext 目录下的 jar 包和 class 文件
- Application ClassLoader，主要负责当前应用里面的 classpath 下的所有 jar 包和类文件

除了系统自己提供的类加载器以外，还可以通过 ClassLoader 类实现自定义加载器，去满足一些特殊场景的需求。



（如图）所谓的父委托模型，就是按照类加载器的层级关系，逐层进行委派。

比如当需要加载一个 class 文件的时候，首先会把这个 class 的查询和加载委派给父加载器去执行，如果父加载器都无法加载，再尝试自己来加载这个 class。



这样设计的好处，我认为有几个。

1. 安全性，因为这种层级关系实际上代表的是一种优先级，也就是所有的类的加载，优先给 Bootstrap ClassLoader。那对于核心类库中的类，就没办法去破坏，比如自己写一个 java.lang.String，最终还是会交给启动类加载器。再加上每个类加载器的作用范围，那么自己写的 java.lang.String 就没办法去覆盖类库中类。
2. 我认为这种层级关系的设计，可以避免重复加载导致程序混乱的问题，因为如果父加载器已经加载过了，那么子类就没必要去加载了。

以上就是我对这个问题的理解。

53. 113 怎么理解线程安全?

Hi，大家好，我是 Mic

一个工作了 4 年的小伙伴，遇到了一个非常抽象的面试题，说说你对线程安全性的理解。

这类问题，对于临时刷面试题来面试的小伙伴，往往是致命的。

一个是不知道从何说起，也就是语言组织比较困难。

其次就是，如果对于线程安全性没有一定程度的理解，一般很难说出你的理解。

ok，我们来看看这个问题的回答。

以下是高手的回答

简单来说，在多个线程访问某个方法或者对象的时候，不管通过任何的方式调用以及线程如何去交替执行。

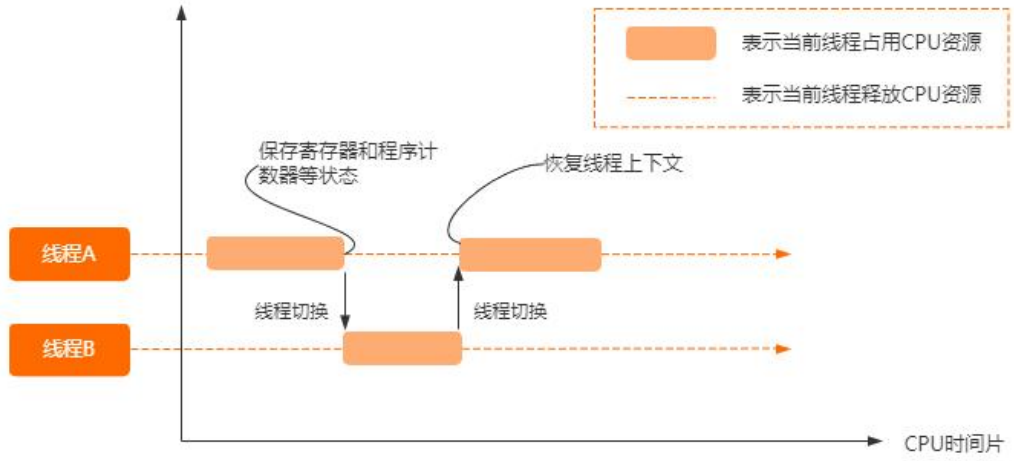
在程序中不做任何同步干预操作的情况下，这个方法或者对象的执行/修改都能按照预期的结果来反馈，那么这个类就是线程安全的。

实际上，线程安全问题的具体表现体现在三个方面，原子性、有序性、可见性。

原子性呢，是指当一个线程执行一系列程序指令操作的时候，它应该是不可中断的，因为一旦出现中断，站在多线程的视角来看，这一系列的程序指令会出现前后执行结果不一致的问题。

这个和数据库里面的原子性是一样的，简单来说就是一段程序只能由一个线程完整的执行完成，而不能存在多个线程干扰。

（如图）CPU 的上下文切换，是导致原子性问题的核心，而 JVM 里面提供了 Synchronized 关键字来解决原子性问题。



可见性，就是说在多线程环境下，由于读和写是发生在不同的线程里面，有可能出现某个线程对共享变量的修改，对其他线程不是实时可见的。

导致可见性问题的原因有很多，比如 CPU 的高速缓存、CPU 的指令重排序、编译器的指令重排序。

有序性，指的是程序编写的指令顺序和最终 CPU 运行的指令顺序可能出现不一致的现象，这种现象也可以称为指令重排序，所以有序性也会导致可见性问题。

可见性和有序性可以通过 JVM 里面提供了一个 Volatile 关键字来解决。

在我看来，导致有序性、原子性、可见性问题的本质，是计算机工程师为了最大化提升 CPU 利用率导致的。比如为了提升 CPU 利用率，设计了三级缓存、设计了 StoreBuffer、

设计了缓存行这种预读机制、在操作系统里面，设计了线程模型、在编译器里面，设计了编译器的深度优化机制。

一上就是我对这个问题的理解。

53.114 请简述一下伪共享的概念以及如何避免

一个工作 5 年的小伙伴，2 个星期时间，去应聘了 10 多家公司，结果每次在技术面试环节，被技术原理难倒了。

他现在很尴尬， 简历投递出去就像石沉大海，基本没什么面试邀约。

技术能力的提升也不是一两天的事情，所以他现在特别焦虑。

于是，我从他遇到过的面试题里面抽出来一道，给大家分享一下。

“请简述一下伪共享的概念以及避免的方法”

对于这个问题，看看高手该怎么回答。

以下是高手的回答

对于这个问题，要从几个方面来回答。

首先，计算机工程师为了提高 CPU 的利用率，平衡 CPU 和内存之间的速度差异，在 CPU 里面设计了三级缓存。

CPU 在向内存发起 IO 操作的时候，一次性会读取 64 个字节的数据作为一个缓存行，缓存到 CPU 的高速缓存里面。

在 Java 中一个 long 类型是 8 个字节，意味着一个缓存行可以存储 8 个 long 类型的变量。

这个设计是基于空间局部性原理来实现的，也就是说，如果一个存储器的位置被引用，那么将来它附近的位置也会被引用。

所以缓存行的设计对于 CPU 来说，可以有效的减少和内存的交互次数，从而避免了 CPU 的 IO 等待，以提升 CPU 的利用率。

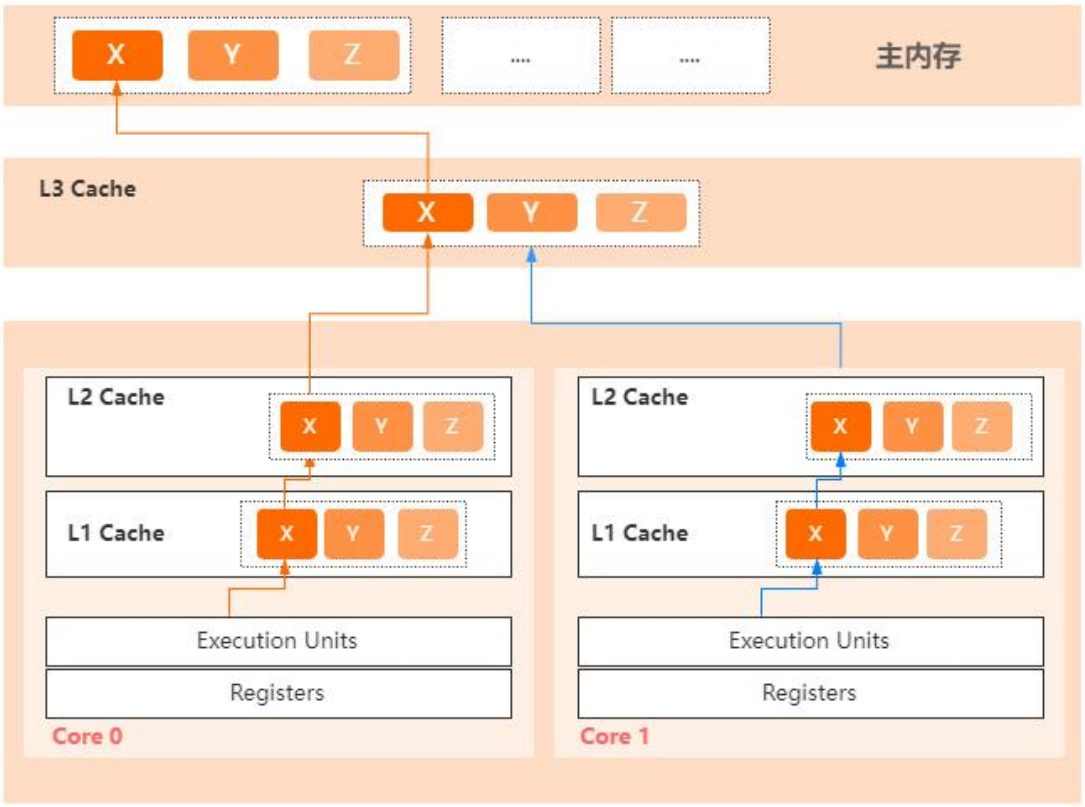
正是因为这种缓存行的设计，导致如果多个线程修改同一个缓存行里面的多个独立变量的时候，基于缓存一致性协议，就会无意中影响了彼此的性能，这就是伪共享的问题。

（如图）像这样一种情况，CPU0 上运行的线程想要更新变量 X、CPU1 上的线程想要更新变量 Y，而 X/Y/Z 都在同一个缓存行里面。

每个线程都需要去竞争缓存行的所有权对变量做更新，基于缓存一致性协议。

一旦运行在某个 CPU 上的线程获得了所有权并执行了修改，就会导致其他 CPU 中的缓存行失效。

这就是伪共享问题的原理。



因为伪共享会问题导致缓存锁的竞争，所以在并发场景中的程序执行效率一定会收到较大的影响。

这个问题的解决办法有两个：

1. 使用对齐填充，因为一个缓存行大小是 64 个字节，如果读取的目标数据小于 64 个字节，可以增加一些无意义的成员变量来填充。
2. 在 Java8 里面，提供了@Contented 注解，它也是通过缓存行填充来解决伪共享问题的，被@Contented 注解声明的类或者字段，会被加载到独立的缓存行上。

已上就是我对这个问题的理解！

53.115 为什么要使用 Spring 框架?

一个工作了 4 年的小伙伴，他说他从线下培训就开始接触 Spring，到现在已经快 5 年时间了。

从来没有想过，为什么要使用 Spring 框架。

结果在面试的时候，竟然遇到一个这样的问题。

大脑一时间短路了，来求助我，这类问题应该怎么去回答。

下面我们来看看普通人和高手的回答

以下是高手的回答

Spring 是一个轻量级应用框架，它提供了 IoC 和 AOP 这两个核心的功能。

它的核心目的是为了简化企业级应用程序的开发，使得开发者只需要关心业务需求，不需要关心 Bean 的管理，

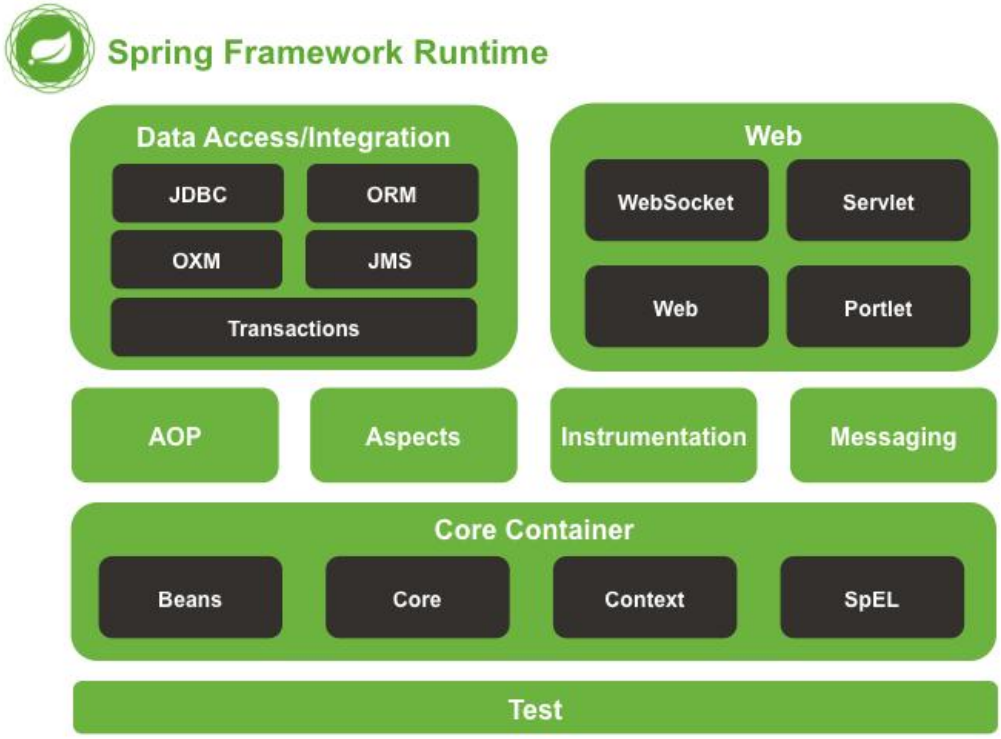
以及通过切面增强功能减少代码的侵入性。

从 Spring 本身的特性来看，我认为有几个关键点是我们选择 Spring 框架的原因。

- 轻量：Spring 是轻量的，基本的版本大约 2MB。
- IOC/DI：Spring 通过 IOC 容器实现了 Bean 的生命周期的管理，以及通过 DI 实现依赖注入，从而实现了对象依赖的松耦合管理。
- 面向切面的编程(AOP)：Spring 支持面向切面的编程，从而把应用业务逻辑和系统服务分开。
- MVC 框架：Spring MVC 提供了功能更加强大且更加灵活的 Web 框架支持
- 事务管理：Spring 通过 AOP 实现了事务的统一管理，对应用开发中的事务处理提供了非常灵活的支持

（如图）最后，Spring 从第一个版本发布到现在，它的生态已经非常庞大了。在业务开发领域，Spring 生态几乎提供了

非常完善的支持，更重要的是社区的活跃度和技术的成熟度都非常高，以上就是我对这个问题的理解。



53.116 Spring 中事务的传播行为有哪些?

一个工作了 2 年的粉丝，私信了一个比较简单的问题。

说： “Spring 中事务的传播行为有哪些？”

他说他能记得一些，但是在项目中基本上不需要配置，所以一下就忘记了。

结果导致面试被拒绝，有点遗憾！

ok，关于这个问题，看看普通人和高手的回答。

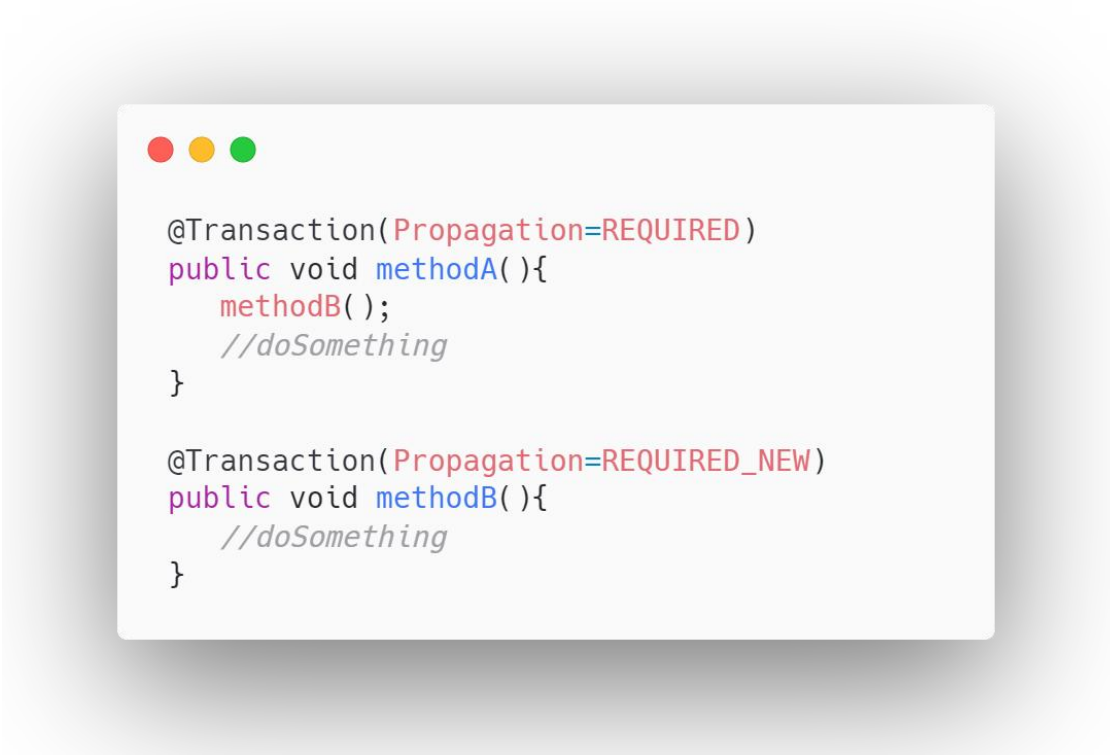
以下是高手的回答

对于这个问题，需要从几个方面去回答。

首选， 所谓的事务传播行为，就是多个声明了事务的方法相互调用的时候，这个事务应该如何传播。

（如图）比如说，methodA()调用 methodB()，两个方法都显示的开启了事务。

那么 methodB()是开启一个新事务，还是继续在 methodA()这个事务中执行？就取决于事务的传播行为。



```
@Transaction(Propagation=REQUIRED)
public void methodA(){
    methodB();
    //doSomething
}

@Transaction(Propagation=REQUIRED_NEW)
public void methodB(){
    //doSomething
}
```

在 Spring 中，定义了 7 种事务传播行为。

1. REQUIRED：默认的 Spring 事物传播级别，如果当前存在事务，则加入这个事务，如果不存在事务，就新建一个事务。
2. REQUIRE_NEW：不管是否存在事务，都会新开一个事务，新老事务相互独立。外部事务抛出异常回滚不会影响内部事务的正常提交。
3. NESTED：如果当前存在事务，则嵌套在当前事务中执行。如果当前没有事务，则新建一个事务，类似于 REQUIRE_NEW。
4. SUPPORTS：表示支持当前事务，如果当前不存在事务，以非事务的方式执行。
5. NOT_SUPPORTED：表示以非事务的方式来运行，如果当前存在事务，则把当前事务挂起。
6. MANDATORY：强制事务执行，若当前不存在事务，则抛出异常.
7. NEVER：以非事务的方式执行，如果当前存在事务，则抛出异常。

Spring 事务传播级别一般不需要定义，默认就是 PROPAGATION_REQUIRED，除非在嵌套事务的情况下需要重点了解。

以上就是我对这个问题的理解！

53.117 说一说 Mybatis 里面的缓存机制

一个工作了 5 年的程序员，在私信里面不断向我诉苦。

他说，他用了 Mybatis 这么久，怎么滴也算是精通 Mybatis 了吧。

结果竟然在 Mybatis 这个面试题上翻车了！ 真的好烦！

好吧，我们今天来看看“Mybatis 里面的缓存机制”，普通人和高手的回答。

以下是高手的回答

这个问题，有点复杂，我打算从几个方面来说明。

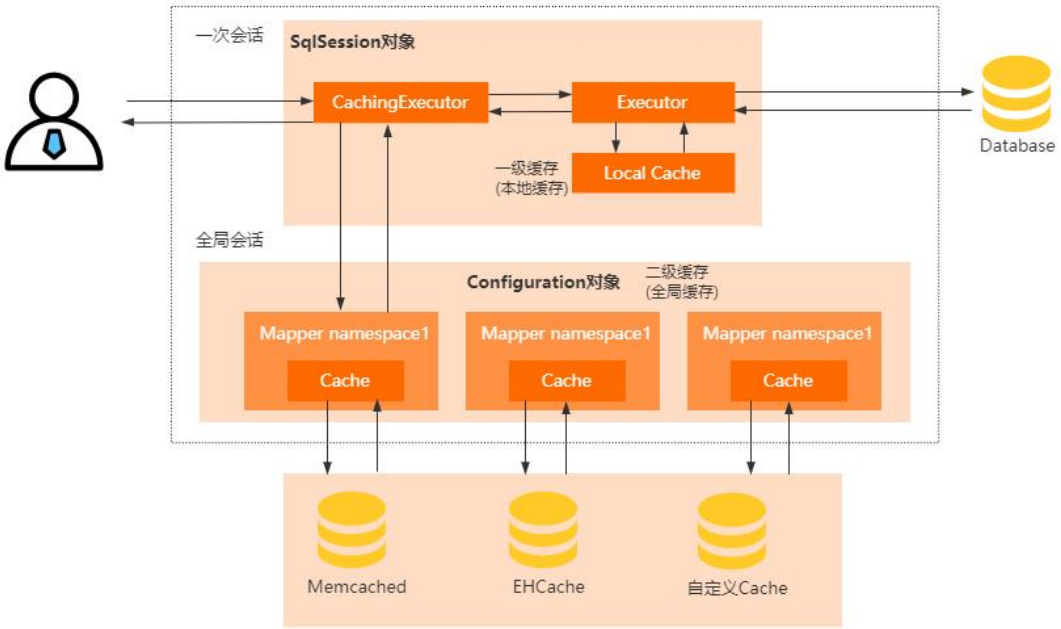
（如图）首先，Mybatis 里面设计了二级缓存来提升数据的检索效率，避免每次数据的访问都需要去查询数据库。

一级缓存，是 SqlSession 级别的缓存，也叫本地缓存，因为每个用户在执行查询的时候都需要使用 SqlSession 来执行，

为了避免每次都去查数据库，Mybatis 把查询出来的数据保存到 SqlSession 的本地缓存中，后续的 SQL 如果命中缓存，就可以直接从本地缓存读取了。

如果想要实现跨 SqlSession 级别的缓存？那么一级缓存就无法实现了，因此在 Mybatis 里面引入了二级缓存，就是当多个用户

在查询数据的时候，只有有任何一个 SqlSession 拿到了数据就会放入到二级缓存里面，其他的 SqlSession 就可以从二级缓存加载数据。

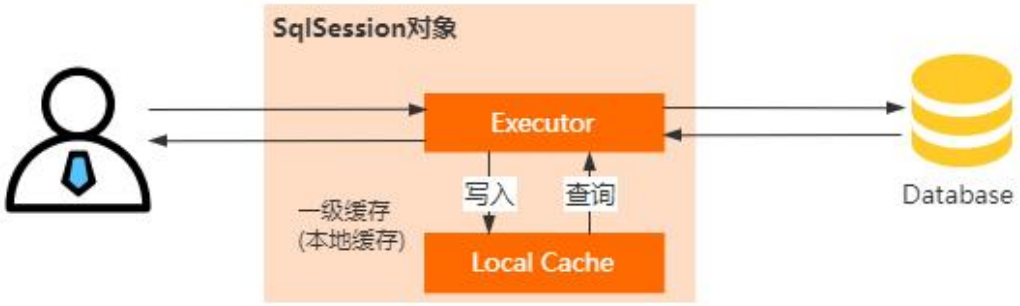


每个

(如图）一级缓存的具体实现原理是：

在 SqlSession 里面持有一个 Executor，每个 Executor 中有一个 LocalCache 对象。当用户发起查询的时候，Mybatis 会根据执行语句在 Local Cache 里面查询，如果没命中，再去查询数据库并写入到 LocalCache，否则直接返回。

所以，以及缓存的生命周期是 SqlSessiion，而且在多个 Sqlsession 或者分布式环境下，可能会导致数据库写操作出现脏数据。

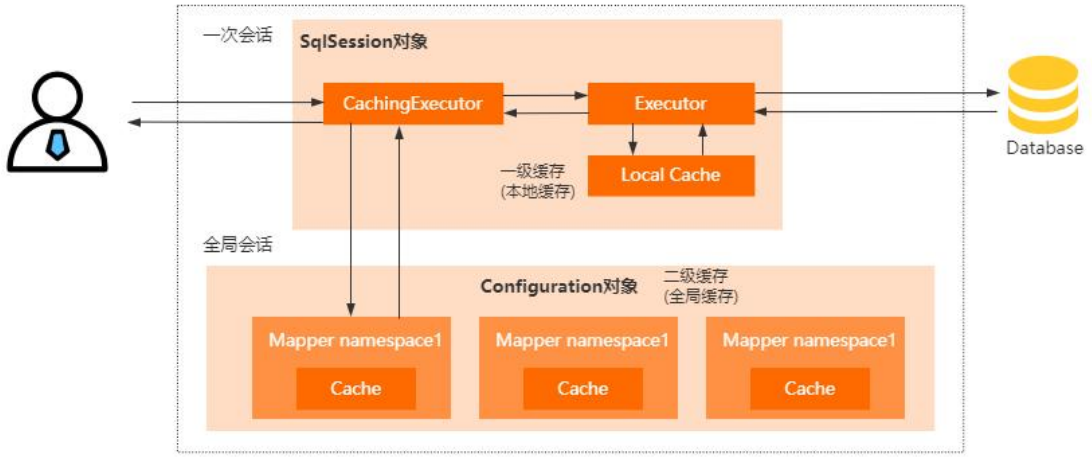


（如图）二级缓存的具体实现原理是：

使用 CachingExecutor 装饰了 Executor，所以在进入一级缓存的查询流程之前，会先通过 CachingExecutor 进行二级缓存的查询。

开启二级缓存以后，会被多个 SqlSession 共享，所以它是一个全局缓存。因此它的查询流程是先查二级缓存，再查一级缓存，最后再查数据库。

另外，MyBatis 的二级缓存相对于一级缓存来说，实现了 SqlSession 之间缓存数据的共享，同时缓存粒度也能够到 namespace 级别，并且还可以通过 Cache 接口实现类不同的组合，对 Cache 的可控性也更强。



以上就是我对这个问题的理解。

53.118 Dubbo 是如何动态感知服务下线的?

什么情况？ 一个工作了 5 年的 Java 程序员，竟然无法回答出这个问题?

“Dubbo 是如何动态感知服务下线的”？

好吧，对于这个问题，看看普通人和高手的回答!

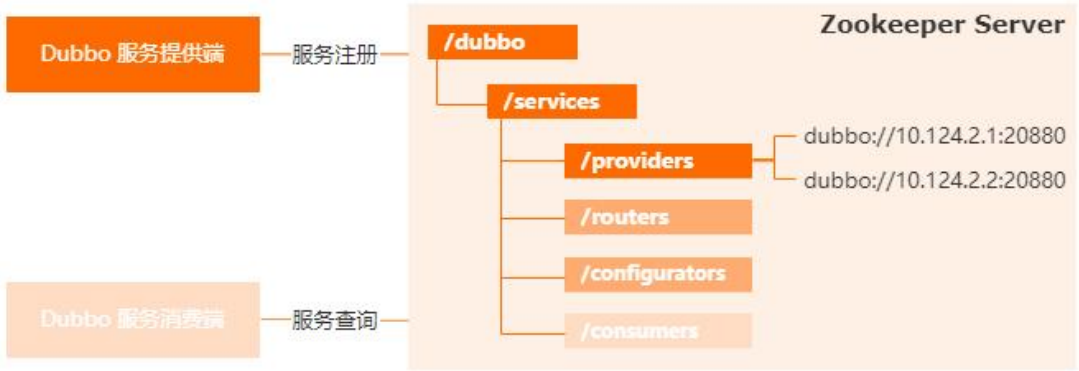
> 以下是高手的回答

好的，面试官，关于这个问题，我从几个方面来回答。

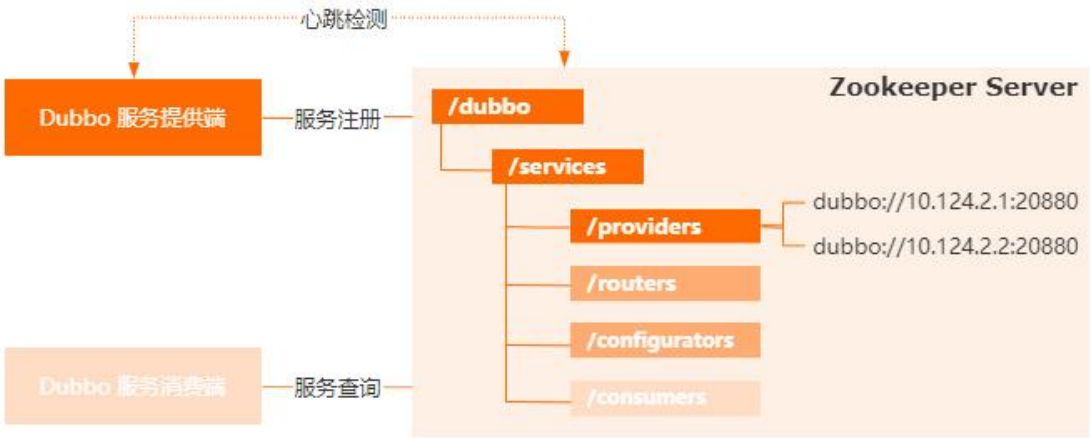
首先，Dubbo 默认采用 Zookeeper 实现服务的注册与服务发现，简单来说啊，就是多个 Dubbo 服务之间的通信地址，是使用 Zookeeper 来维护的。

（如图）在 Zookeeper 上，会采用树形结构的方式来维护 Dubbo 服务提供端的协议地址，

Dubbo 服务消费端会从 Zookeeper Server 上去查找目标服务的地址列表，从而完成服务的注册和消费的功能。



Zookeeper 会通过心跳检测机制，来判断 Dubbo 服务提供端的运行状态，来决定是否应该把这个服务从地址列表剔除。



当 Dubbo 服务提供方出现故障导致 Zookeeper 剔除了这个服务的地址，

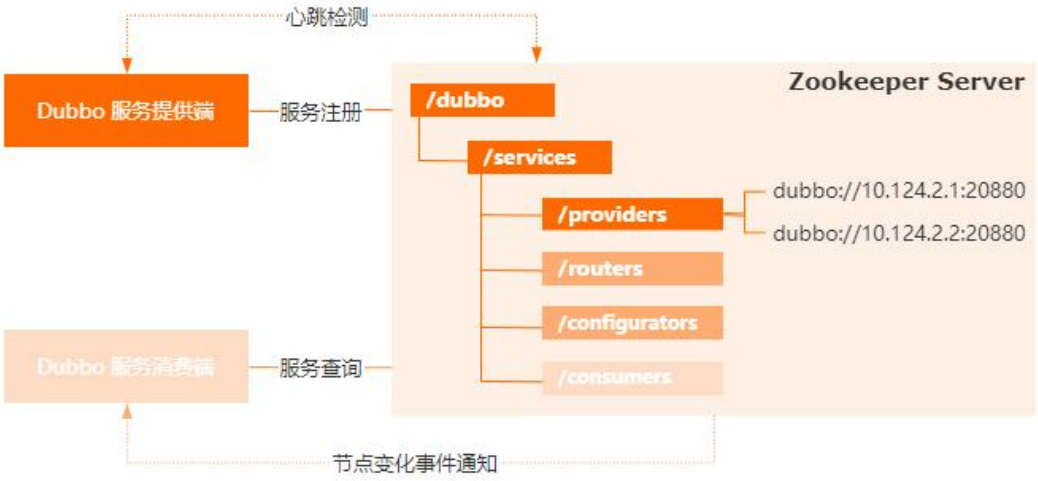
那么 Dubbo 服务消费端需要感知到地址的变化，从而避免后续的请求发送到故障节点，导致请求失败。

也就是说 Dubbo 要提供服务下线的动态感知能力。

（如图）这个能力是通过 Zookeeper 里面提供的 Watch 机制来实现的，

简单来说呢，Dubbo 服务消费端会使用 Zookeeper 里面的 Watch 来针对 Zookeeper Server 端的`/providers` 节点注册监听，

一旦这个节点下的子节点发生变化，Zookeeper Server 就会发送一个事件通知 Dubbo Client 端.



Dubbo Client 端收到事件以后，就会把本地缓存的这个服务地址删除，这样后续就不会把请求发送到失败的节点上，完成服务下线感知。

以上就是我对这个问题的理解！

53.119 Spring 中 Bean 的作用域有哪些?

一个工作 3 年的小伙子，去面试被问到 Spring 里面的问题。

这个问题比较简单，但是他却没有回答上来。

虽然他可以通过搜索引擎找到答案，但是如果没有理解，下次面试还是不会！

这个面试题是： “Spring 中的 Bean，作用域有哪些？”

对于这个问题，看看普通人和高手的回答。

> 以下是高手的回答

好的，这个问题可以从几个方面来回答。

首先呢，Spring 框架里面的 IOC 容器，可以非常方便的去帮助我们管理应用里面的 Bean 对象实例。

我们只需要按照 Spring 里面提供的 xml 或者注解等方式去告诉 IOC 容器，哪些 Bean 需要被 IOC 容器管理就行了。

其次呢，既然是 Bean 对象实例的管理，那意味着这些实例，是存在生命周期，也就是所谓的作用域。

理论上来说，常规的生命周期只有两种：

- singleton， 也就是单例，意味着在整个 Spring 容器中只会存在一个 Bean 实例。
- prototype，翻译成原型，意味着每次从 IOC 容器去获取指定 Bean 的时候，都会返回一个新的实例对象。

但是在基于 Spring 框架下的 Web 应用里面，增加了一个会话纬度来控制 Bean 的生命周期，主要有三个选择

- request， 针对每一次 http 请求，都会创建一个新的 Bean
- session，以 sesssion 会话为纬度，同一个 session 共享同一个 Bean 实例，不同的 session 产生不同的 Bean 实例
- globalSession，针对全局 session 纬度，共享同一个 Bean 实例

以上就是我对这个问题的理解。

53.120 Zookeeper 中的 Watch 机制的原理?

一个工作了 7 年的粉丝，遇到了一个 Zookeeper 的问题。

因为接触过 Zookeeper 这个技术，不知道该怎么回答。

我说一个工作了 7 年的程序员，没有接触过主流技术，这不正常。

于是我问了他工资以后，我理解了！

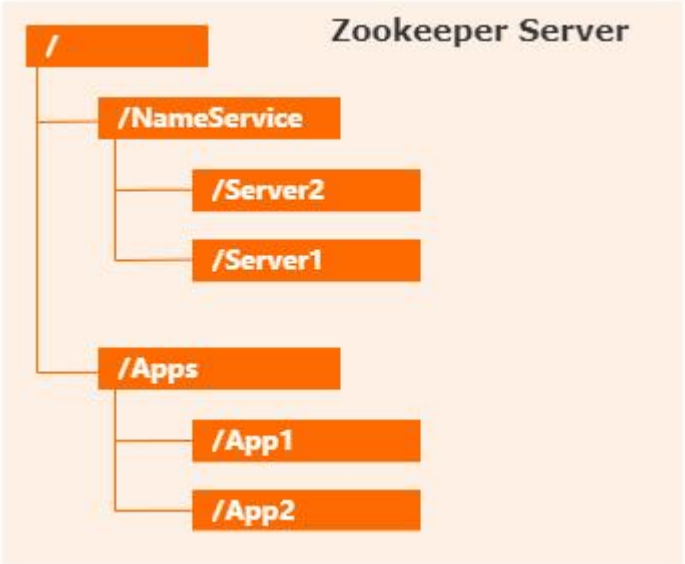
好吧，关于“Zookeeper 中 Watch 机制的实现原理”，看看普通人和高手的回答。

以下是高手的回答

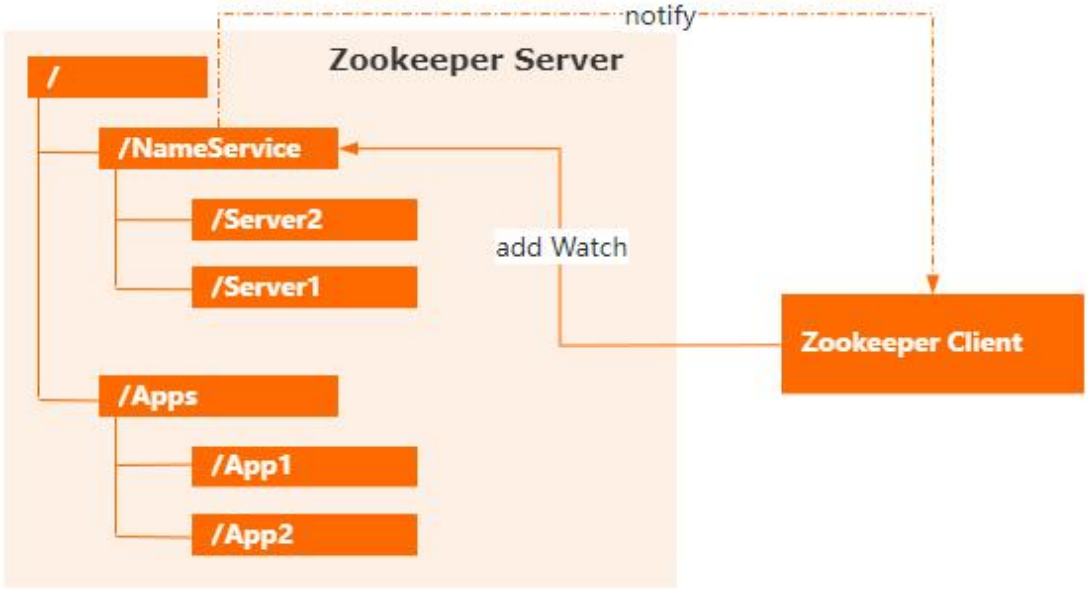
好的，这个问题我打算从两个方面来回答。

Zookeeper 是一个分布式协调组件，为分布式架构下的多个应用组件提供了顺序访问控制能力。

（如图）它的数据存储采用了类似于文件系统的树形结构，以节点的方式来管理存储在 Zookeeper 上的数据。



（如图）Zookeeper 提供了一个 Watch 机制，可以让客户端感知到 Zookeeper Server 上存储的数据变化，这样一种机制可以让 Zookeeper 实现很多的场景，比如配置中心、注册中心等。



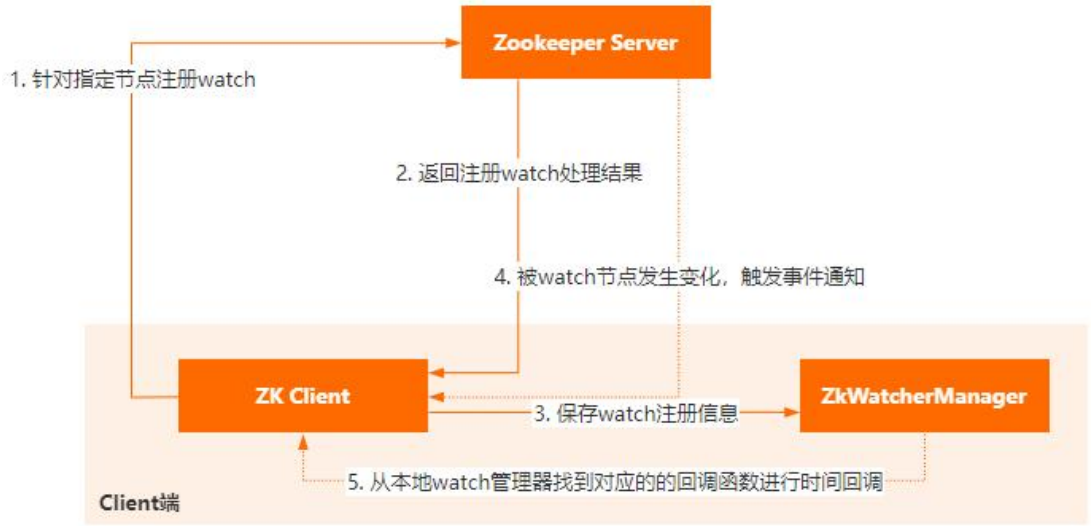
Watch 机制采用了 Push 的方式来实现，也就是说客户端和 Zookeeper Server 会建立一个长连接，一旦监听的指定节点发生了变化，就会通过这个长连接把变化的事件推送给客户端。

（如图） Watch 的具体流程分为几个部分：

- 首先，是客户端通过指定命令比如 exists、get，对特定路径增加 watch
- 然后服务端收到请求以后，用 HashMap 保存这个客户端会话以及对应关注的节点路径，同时客户端也会使用 HashMap

存储指定节点和事件回调函数的对应关系。

- 当服务端指定被 watch 的节点发生变化后，就会找到这个节点对应的会话，把变化的事件和节点信息发给这个客户端。
- 客户端收到请求以后，从 ZkWatcherManager 里面对应的回调方法进行调用，完成事件变更的通知。



以上就是我对这个问题的理解！

53.121 Spring 中有哪些方式可以把 Bean 注入到 IOC 容器?

今天收到一个工作 4 年的粉丝的面试题。

问题是： “Spring 中有哪些方式可以把 Bean 注入到 IOC 容器”。

他说这道题是所有面试题里面回答最好的，但是看面试官的表情，好像不太对。

我问他怎么回答的，他说： “接口注入”、“Setter 注入”、“构造器注入”。

为什么不对？来看看普通人和高手的回答。

以下是高手的回答

好的，把 Bean 注入到 IOC 容器里面的方式有 7 种方式

7. 使用 xml 的方式来声明 Bean 的定义，Spring 容器在启动的时候会加载并解析这个 xml，把 bean 装载到 IOC 容器中。
8. 使用@CompontScan 注解来扫描声明了@Controller、@Service、@Repository、@Component 注解的类。
9. 使用@Configuration 注解声明配置类，并使用@Bean 注解实现 Bean 的定义，这种方式其实是 xml 配置方式的一种演变，是 Spring 迈入到无配置化时代的里程碑。
10. 使用@Import 注解，导入配置类或者普通的 Bean
11. 使用 FactoryBean 工厂 bean，动态构建一个 Bean 实例，Spring Cloud OpenFeign 里面的动态代理实例就是使用 FactoryBean 来实现的。
12. 实现 ImportBeanDefinitionRegistrar 接口，可以动态注入 Bean 实例。这个在 Spring Boot 里面的启动注解有用到。
13. 实现 ImportSelector 接口，动态批量注入配置类或者 Bean 对象，这个在 Spring Boot 里面的自动装配机制里面有用到。

以上就是我对这个问题的理解。

53.122 Redis 存在线程安全问题吗？为什么？

一个工作了 5 年的粉丝私信我。

他说自己准备了半年时间，想如蚂蚁金服，结果第一面就挂了，非常难过。

问题是： “Redis 存在线程安全问题吗？”

关于这个问题，看看普通人和高手的回答。

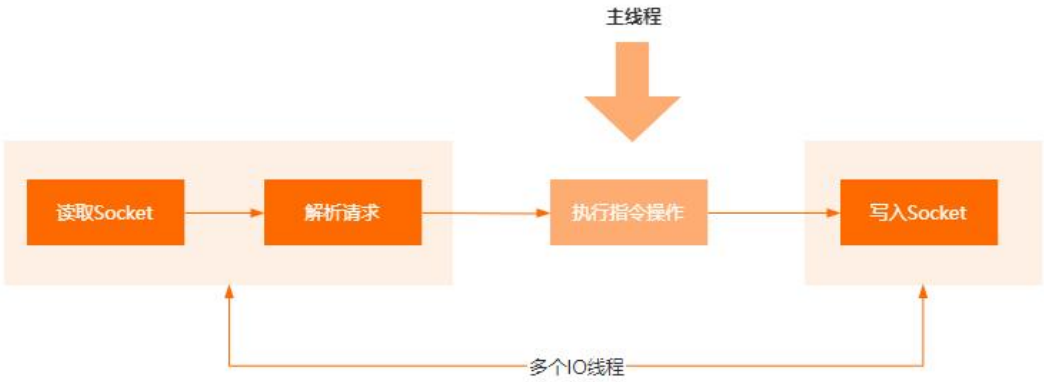
> 以下是高手的回答

好的，关于这个问题，我从两个方面来回答。

第一个，从 Redis 服务端层面。

Redis Server 本身是一个线程安全的 K-V 数据库，也就是说在 Redis Server 上执行的指令，不需要任何同步机制，不会存在线程安全问题。

（如图）虽然 Redis 6.0 里面，增加了多线程的模型，但是增加的多线程只是用来处理网络 IO 事件，对于指令的执行过程，仍然是由主线程来处理，所以不会存在多个线程通知执行操作指令的情况。



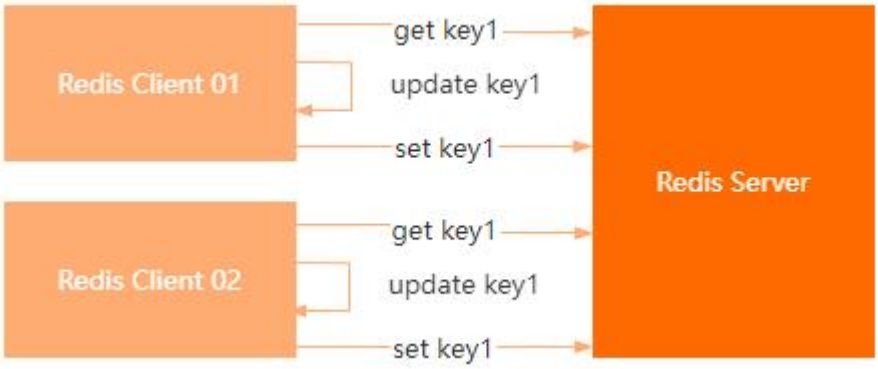
为什么 Redis 没有采用多线程来执行指令，我认为有几个方面的原因。

- Redis Server 本身可能出现的性能瓶颈点无非就是网络 IO、CPU、内存。但是 CPU 不是 Redis 的瓶颈点，所以没必要使用多线程来执行指令。
- 如果采用多线程，意味着对于 redis 的所有指令操作，都必须要考虑到线程安全问题，也就是说需要加锁来解决，这种方式带来的性能影响反而更大。

第二个，从 Redis 客户端层面。

（如图）虽然 Redis Server 中的指令执行是原子的，但是如果有多个 Redis 客户端同时执行多个指令的时候，就无法保证原子性。

假设两个 redis client 同时获取 Redis Server 上的 key1， 同时进行修改和写入，因为多线程环境下的原子性无法被保障，以及多进程情况下的共享资源访问的竞争问题，使得数据的安全性无法得到保障。



当然，对于客户端层面的线程安全性问题，解决方法有很多，比如尽可能的使用 Redis 里面的原子指令，或者对多个客户端的资源访问加锁，或者通过 Lua 脚本来实现多个指令的操作等等。

以上就是我对这个问题的理解。

53.123 Spring 中 BeanFactory 和 FactoryBean 的区别

一个工作了六年多的粉丝，胸有成竹的去京东面试。

然后被 Spring 里面的一个问题卡住，唉，我和他说，6 年啦，Spring 都没搞明白？

那怎么去让面试官给你通过呢？

这个问题是： Spring 中 BeanFactory 和 FactoryBean 的区别。

好吧，对于这个问题看看普通人和高手的回答。

> 以下是高手的回答

关于这个问题，我从几个方面来回答。

首先，Spring 里面的核心功能是 IOC 容器，所谓 IOC 容器呢，本质上就是一个 Bean 的容器或者是一个 Bean 的工厂。

它能够根据 xml 里面声明的 Bean 配置进行 bean 的加载和初始化，然后 BeanFactory 来生产我们需要的各种各样的 Bean。

所以我对 BeanFactory 的理解了有两个。

- BeanFactory 是所有 Spring Bean 容器的顶级接口，它为 Spring 的容器定义了一套规范，并提供像 getBean 这样的方法从容器中获取指定的 Bean 实例。
- BeanFactory 在产生 Bean 的同时，还提供了解决 Bean 之间的依赖注入的能力，也就是所谓的 DI。

FactoryBean 是一个工厂 Bean，它是一个接口，主要的功能是动态生成某一个类型的 Bean 的实例，也就是说，我们可以自定义一个 Bean 并且加载到 IOC 容器里面。

它里面有一个重要的方法叫 `getObject()`，这个方法里面就是用来实现动态构建 Bean 的过程。

Spring Cloud 里面的 OpenFeign 组件，客户端的代理类，就是使用了 FactoryBean 来实现的。

以上就是我对这个问题的理解。

53.124 什么是可重入，什么是可重入锁？它用来解决什么问题？

一个工作了 3 年的粉丝，去一个互联网公司面试，结果被面试官怼了。

面试官说：” 这么简单的问题你都不知道？ 没法聊了，回去等通知吧 “。

这个问题是： ” 什么是可重入锁，以及它的作用是什么？ “

对于这个问题，来看看普通人和高手的回答吧

以下是高手的回答

好的。

可重入是多线程并发编程里面一个比较重要的概念，

简单来说，就是在运行的某个函数或者代码，因为抢占资源或者中断等原因导致函数或者代码的运行中断，

等待中断程序执行结束后，重新进入到这个函数或者代码中运行，并且运行结果不会受到影响，那么这个函数或者代码就是可重入的。

(如图) 而可重入锁，简单来说就是一个线程如果抢占到了互斥锁资源，在锁释放之前再去竞争同一把锁的时候，不需要等待，只需要记录重入次数。

在多线程并发编程里面，绝大部分锁都是可重入的，比如 Synchronized、ReentrantLock 等，但是也有不支持重入的锁，比如 JDK8 里面提供的读写锁 StampedLock。

```
public static synchronized void lock1(){
  //ThreadX 获取到了lock1中的Synchronized锁,
  // 再次调用另外一个加同步锁的lock2()方法
    lock2();
}

public static synchronized void lock2(){
    //doSomething
}
```

锁的可重入性，主要解决的问题是避免线程死锁的问题。

因为一个已经获得同步锁 X 的线程，在释放锁 X 之前再去竞争锁 X 的时候，相当于会出现自己要等待自己释放锁，这很显然是无法成立的。

以上就是我对这个问题的理解。

53.125 请你简单说一下 Mysql 的事务隔离级别

一个工作了 6 年的粉丝，去阿里面试，在第一面的时候被问到”Mysql 的事务隔离级别“。

他竟然没有回答上来，一直在私信向我诉苦。

我说，你只能怪年轻时候的你，那个时候不够努力导致现在的你技术水平不够。

好吧，关于这个问题，看看普通人和高手的回答。

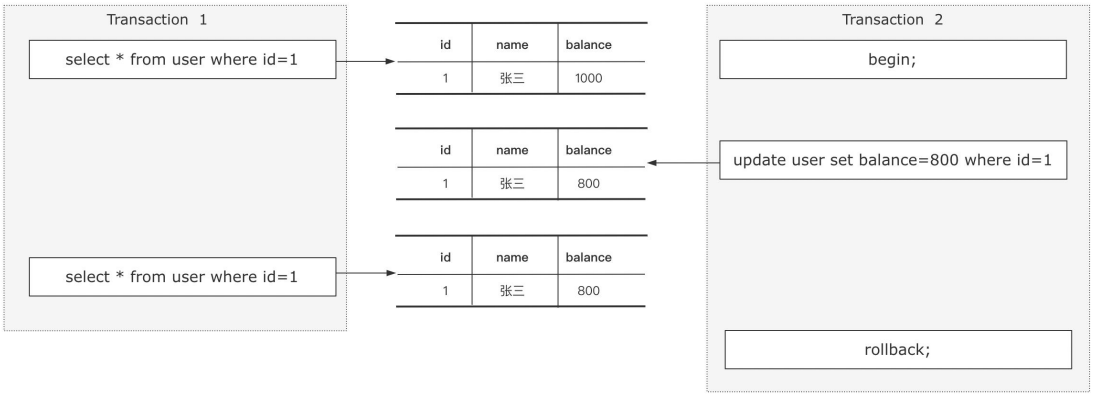
以下是高手的回答

好的，关于这个问题，我会从几个方面来回答。

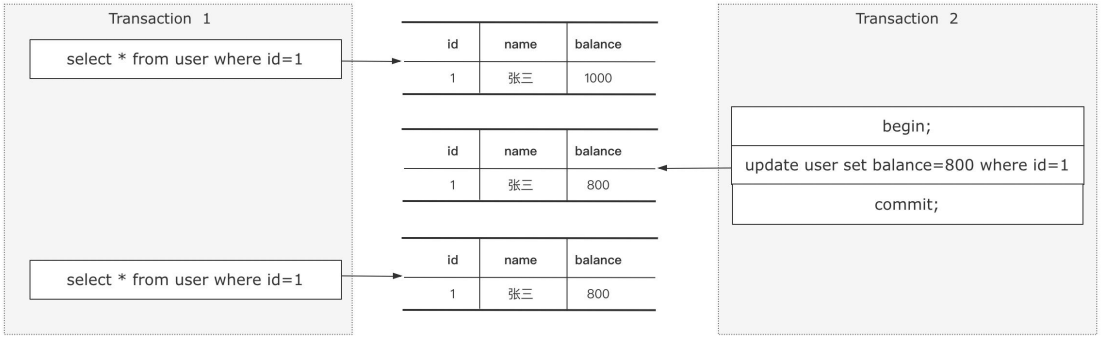
首先，事务隔离级别，是为了解决多个并行事务竞争导致的数据安全问题的一种规范。

具体来说，多个事务竞争可能会产生三种不同的现象。

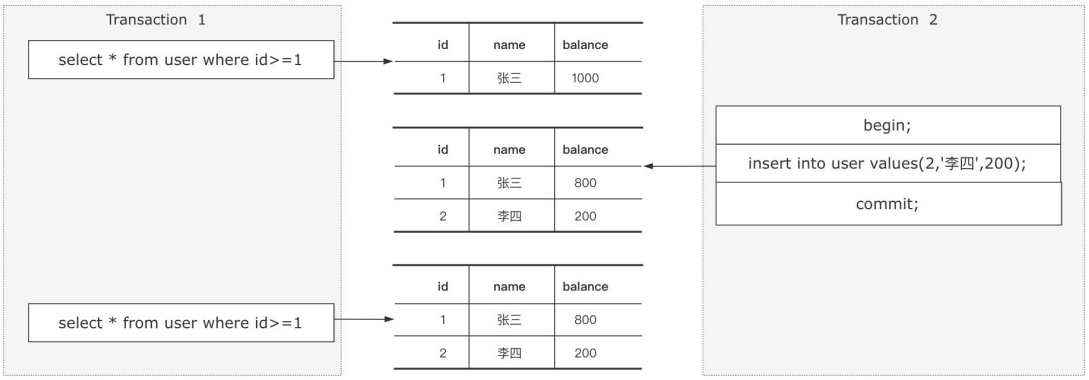
1. （如图）假设有两个事务 T1/T2 同时在执行，T1 事务有可能会读取到 T2 事务未提交的数据，但是未提交的事务 T2 可能会回滚，也就导致了 T1 事务读取到最终不一定存在的数据产生脏读的现象。



2. （如图）假设有两个事务 T1/T2 同时执行，事务 T1 在不同的时刻读取同一行数据的时候结果可能不一样，从而导致不可重复读的问题。



3. （如图），假设有两个事务 T1/T2 同时执行，事务 T1 执行范围查询或者范围修改的过程中，事务 T2 插入了一条属于事务 T1 范围内的数据并且提交了，这时候在事务 T1 查询发现多出来了一条数据，或者在 T1 事务发现这条数据没有被修改，看起来像是产生了幻觉，这种现象称为幻读。



而这三种现象在实际应用中，可能有些场景不能接受某些现象的存在，所以在 SQL 标准中定义了四种隔离级别，分别是：

4. 读未提交，在这种隔离级别下，可能会产生脏读、不可重复读、幻读。
5. 读已提交（RC)，在这种隔离级别下，可能会产生不可重复读和幻读。
6. 可重复读（RR），在这种隔离级别下，可能会产生幻读
7. 串行化，在这种隔离级别下，多个并行事务串行化执行，不会产生安全性问题。

这四种隔离级别里面，只有串行化解决了全部的问题，但也意味着这种隔离级别的性能是最低的。

在 Mysql 里面，InnoDB 引擎默认的隔离级别是 RR（可重复读），因为它需要保证事务 ACID 特性中的隔离性特征。

以上就是我对这个问题的理解。

53.126 请说一下 ReentrantLock 的实现原理?

一个工作了 3 年的粉丝私信我，在面试的时候遇到了这样一个问题。

”请说一下 ReentrantLock 的实现原理“，他当时根据自己的理解零零散散的说了一些。

但是似乎没有说到关键点上，让我出一期视频说一下回答思路。

好吧，关于这个问题，我们来看看普通人和高手的回答。

以下是高手的回答

好的，面试官，关于这个问题，我会从这几个方面来回答。（注意下面这几个点直接展示，不需要口播）

- 什么是 ReentrantLock
- ReentrantLock 的特性
- ReentrantLock 的实现原理

首先，ReentrantLock 是一种可重入的排它锁，主要用来解决多线程对共享资源竞争的问题。

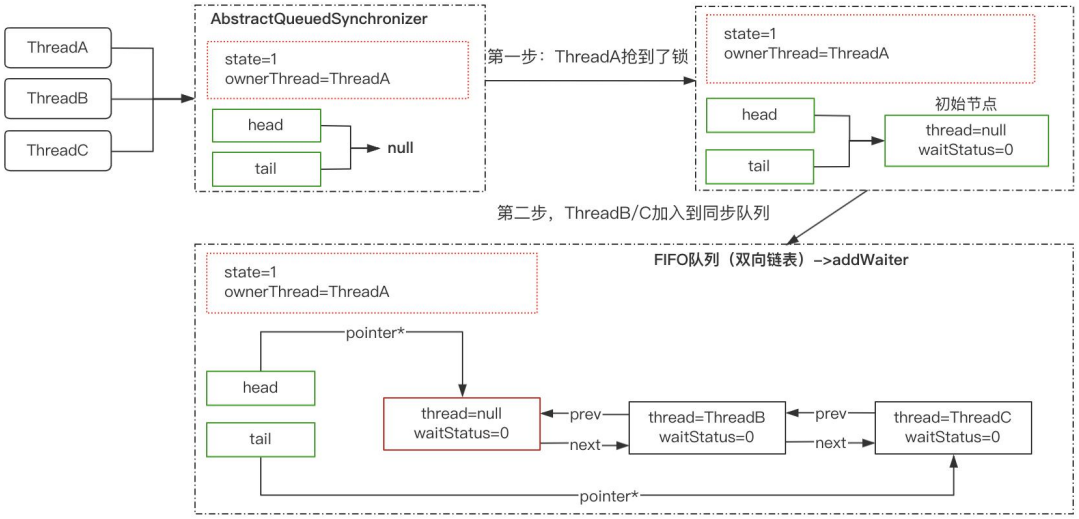
它的核心特性有几个：

1. 它支持可重入，也就是获得锁的线程在释放锁之前再次去竞争同一把锁的时候，不需要加锁就可以直接访问。
2. 它支持公平和非公平特性
3. 它提供了阻塞竞争锁和非阻塞竞争锁的两种方法，分别是 lock()和 tryLock()。

（如图）然后，ReentrantLock 的底层实现有几个非常关键的技术。

4. 锁的竞争，ReentrantLock 是通过互斥变量，使用 CAS 机制来实现的。
5. 没有竞争到锁的线程，使用了 AbstractQueuedSynchronizer 这样一个队列同步器来存储，底层是通过双向链表来实现的。当锁被释放之后，会从 AQS 队列里面的头部唤醒下一个等待锁的线程。
6. 公平和非公平的特性，主要是体现在竞争锁的时候，是否需要判断 AQS 队列存在等待中的线程。
7. 最后，关于锁的重入特性，在 AQS 里面有一个成员变量来保存当前获得锁的线程，当同一个线程下次再来竞争锁的时候，就不会去走锁竞争的逻辑，而是直接增加重入次数。

以上就是我对这个问题的理解。



53.127 Mybatis 中#{}和${}的区别是什么?

一个工作 2 年的粉丝，被问到一个 Mybatis 里面的基础问题。

他跑过来调戏我，说 Mic 老师，你要是能把这个问题回答到一定高度，请我和一个月奶茶。

这个问题是：“ Mybatis 里面#{}和${}的区别是什么“

下面看看普通人和高手对这个问题的回答。

以下是高手的回答

好的，关于这个问题我从几个方面来回答。

首先，Mybatis 提供到的#号占位符和$号占位符，都是实现动态 SQL 的一种方式，通过这两种方式把参数传递到 XML 之后，

在执行操作之前，Mybatis 会对这两种占位符进行动态解析。

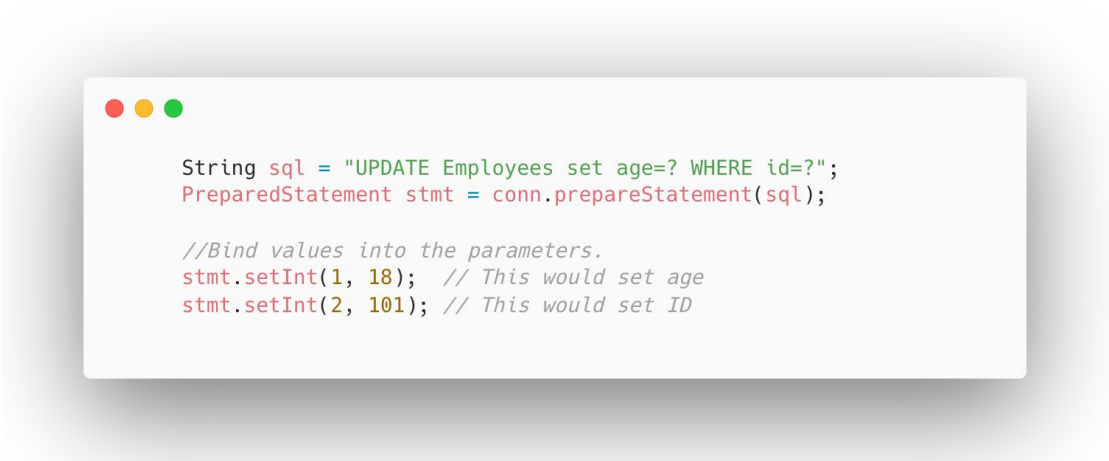
（如图）#号占位符，等同于 jdbc 里面的？号占位符。

它相当于向 PreparedStatement 中的预处理语句中设置参数，

而 PreparedStatement 中的 sql 语句是预编译的，SQL 语句中使用了占位符，规定了 sql 语句的结构。

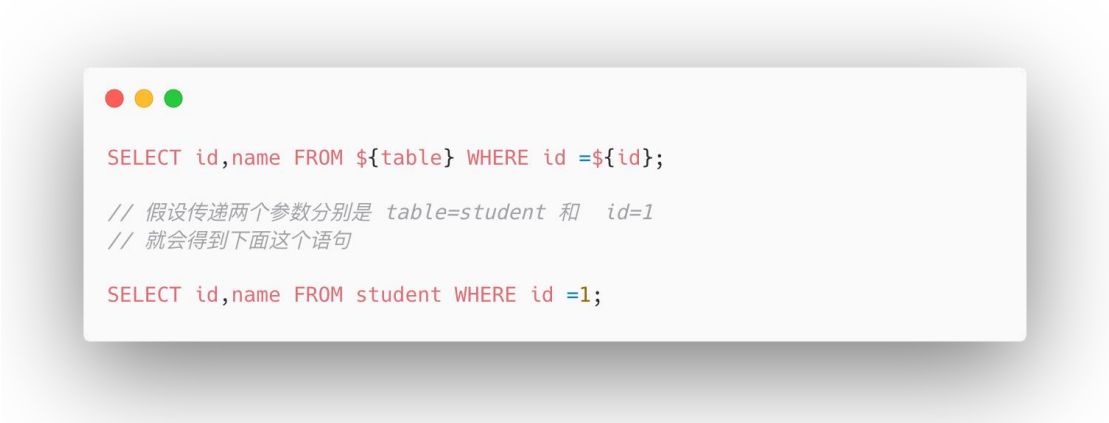
并且在设置参数的时候，如果有特殊字符，会自动进行转义。

所以#号占位符可以防止 SQL 注入。



```
String sql = "UPDATE Employees set age=? WHERE id=?";
PreparedStatement stmt = conn.prepareStatement(sql);

//Bind values into the parameters.
stmt.setInt(1, 18);  // This would set age
stmt.setInt(2, 101); // This would set ID
```

（如图）而使用$的方式传参，相当于直接把参数拼接到了原始的 SQL 里面，Mybatis 不会对它进行特殊处理。



```
SELECT id,name FROM ${table} WHERE id =${id};

// 假设传递两个参数分别是 table=student 和  id=1
// 就会得到下面这个语句

SELECT id,name FROM student WHERE id =1;
```

所以$和#最大的区别在于，前者是动态参数，后者是占位符， 动态参数无法防止 SQL 注入的问题，所以在实际应用中，应该尽可能的使用#号占位符。

另外，$符号的动态传参，可以适合应用在一些动态 SQL 场景中，比如动态传递表名、动态设置排序字段等。

以上就是我对这个问题的理解。

53.128 Mysql 为什么使用 B+Tree 作为索引结构

一个工作 8 年的粉丝私信了我一个问题。

他说这个问题是去阿里面试的时候被问到的，自己查了很多资料也没搞明白，希望我帮他解答。

问题是： “Mysql 为什么使用 B+Tree 作为索引结构”

关于这个问题，看看普通人和高手的回答。

以下是高手的回答

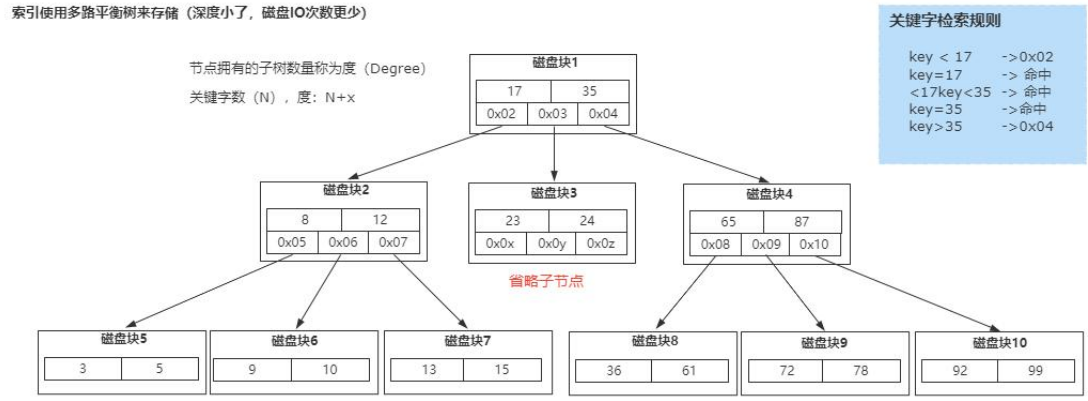
关于这个问题 ，我从几个方面来回答。

首先，常规的数据库存储引擎，一般都是采用 B 树或者 B+树来实现索引的存储。

（如图）因为 B 树是一种多路平衡树，用这种存储结构来存储大量数据，它的整个高度会相比二叉树来说，会矮很多。

而对于数据库来说，所有的数据必然都是存储在磁盘上的，而磁盘 IO 的效率实际上是很低的，特别是在随机磁盘 IO 的情况下效率更低。

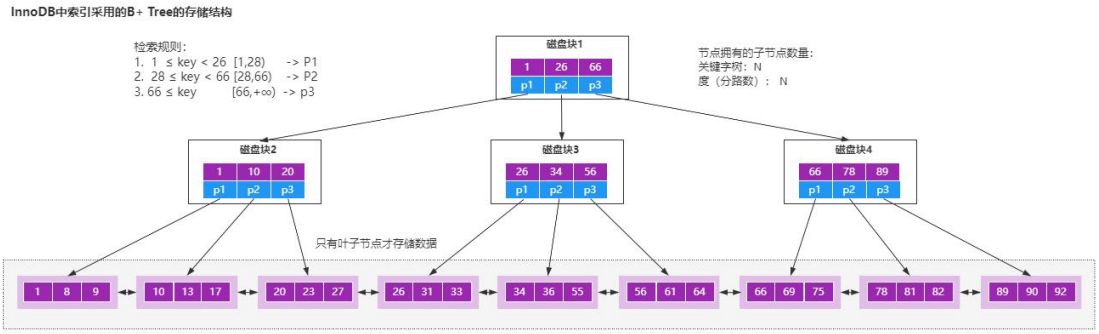
所以树的高度能够决定磁盘 IO 的次数，磁盘 IO 次数越少，对于性能的提升就越大，这也是为什么采用 B 树作为索引存储结构的原因。



（如图）但是在 Mysql 的 InnoDB 存储引擎里面，它用了一种增强的 B 树结构，也就是 B+树来作为索引和数据的存储结构。

相比较于 B 树结构，B+树做了几个方面的优化。

1. B+树的所有数据都存储在叶子节点，非叶子节点只存储索引。
2. 叶子节点中的数据使用双向链表的方式进行关联。



使用 B+树来实现索引的原因，我认为有几个方面。

3. B+树非叶子节点不存储数据，所以每一层能够存储的索引数量会增加，意味着 B+树在层高相同的情况下存储的数据量要比 B 树要多，使得磁盘 IO 次数更少。
4. 在 Mysql 里面，范围查询是一个比较常用的操作，而 B+树的所有存储在叶子节点的数据使用了双向链表来关联，所以在查询的时候只需查两个节点进行遍历就行，而 B 树需要获取所有节点，所以 B+树在范围查询上效率更高。
5. 在数据检索方面，由于所有的数据都存储在叶子节点，所以 B+树的 IO 次数会更加稳定一些。
6. 因为叶子节点存储所有数据，所以 B+树的全局扫描能力更强一些，因为它只需要扫描叶子节点。但是 B 树需要遍历整个树。

另外，基于 B+树这样一种结构，如果采用自增的整型数据作为主键，还能更好的避免增加数据的时候，带来叶子节点分裂导致的大量运算的问题。

总的来说，我认为技术方案的选型，更多的是去解决当前场景下的特定问题，并不一定是说 B+树就是最好的选择，就像 MongoDB 里面采用 B 树结构，本质上来说，其实是关系型数据库和非关系型数据库的差异。

以上就是我对这个问题的理解。

53.129 数据库连接池有什么用？它有哪些关键参数?

一个工作 5 年的粉丝找到我，他说参加美团面试，遇到一个基础题没回答上来。

这个问题是：“数据库连接池有什么用？以及它有哪些关键参数”？

我说，这个问题都不知道，那你项目里面的连接池配置怎么设置的？ 你们猜他怎么回答。懂得懂得啊。

好的，关于这个问题，我们来看看高手的回答。

> 以下是高手的回答

关于这个问题，我从这几个方面来回答。

首先，数据库连接池是一种池化技术，池化技术的核心思想是实现资源的复用，避免资源重复创建销毁的开销。

而在数据库的应用场景里面，应用程序每次向数据库发起 CRUD 操作的时候，都需要创建连接

在数据库访问量较大的情况下，频繁的创建连接会带来较大的性能开销。

（如图）而连接池的核心思想，就是应用程序在启动的时候提前初始化一部分连接保存到连接池里面，当应用需要使用连接的时候，直接从连接池获取一个已经建立好的链接。连接池的设计，避免了每次连接的建立和释放带来的开销。

连接池的参数有很多，不过关键参数就几个：

首先是，连接池初始化的时候会有几个关键参数：

1. 初始化连接数，表示启动的时候初始多少个连接保存到连接池里面。
2. 最大连接数，表示同时最多能支持多少连接，如果连接数不够，后续要获取连接的线程会阻塞。
3. 最大空闲连接数，表示没有请求的时候，连接池中要保留的最大空闲连接。
4. 最小空闲连接，当连接数小于这个值的时候，连接池需要再创建连接来补充到这个值。

然后，就是在使用连接的时候的关键参数：

5. 最大等待时间，就是连接池里面的连接用完了以后，新的请求要等待的时间，超过这个时间就会提示超时异常。
6. 无效连接清除， 清理连接池里面的无效连接，避免使用这个连接操作的时候出现错误。

不同的连接池框架，除了核心的参数以外，还有很多业务型的参数，比如是否要检测连接 sql 的有效性、连接初始化 SQL 等等，这些配置参数可以在使用的时候去查询 api 文档就可以知道。

以上就是我对这个问题的理解。

53.130 简述一下你对线程池的理解？

到底是什么面试题，

让一个工作了 4 年的精神小伙，只是去参加了一场技术面试，

就被搞得精神萎靡。郁郁寡欢！

这一切的背后到底是道德的沦丧，还是人性的扭曲。

让我们一起揭秘一下这道面试题。

关于，“简述你对线程池的理解”，看看普通人和高手的回答。

以下是高手的回答

关于这个问题，我会从几个方面来回答。

首先，线程池本质上是一种池化技术，而池化技术是一种资源复用的思想，比较常见的有连接池、内存池、对象池。

而线程池里面复用的是线程资源，它的核心设计目标，我认为有两个：

1. 减少线程的频繁创建和销毁带来的性能开销，因为线程创建会涉及到 CPU 上下文切换、内存分配等工作。
2. 线程池本身会有参数来控制线程创建的数量，这样就可以避免无休止的创建线程带来的资源利用率过高的问题，

起到了资源保护的作用。

其次，我简单说一下线程池里面的线程复用技术。因为线程本身并不是一个受控的技术，也就是说线程的生命周期时由任务运行的状态决定的，无法人为控制。

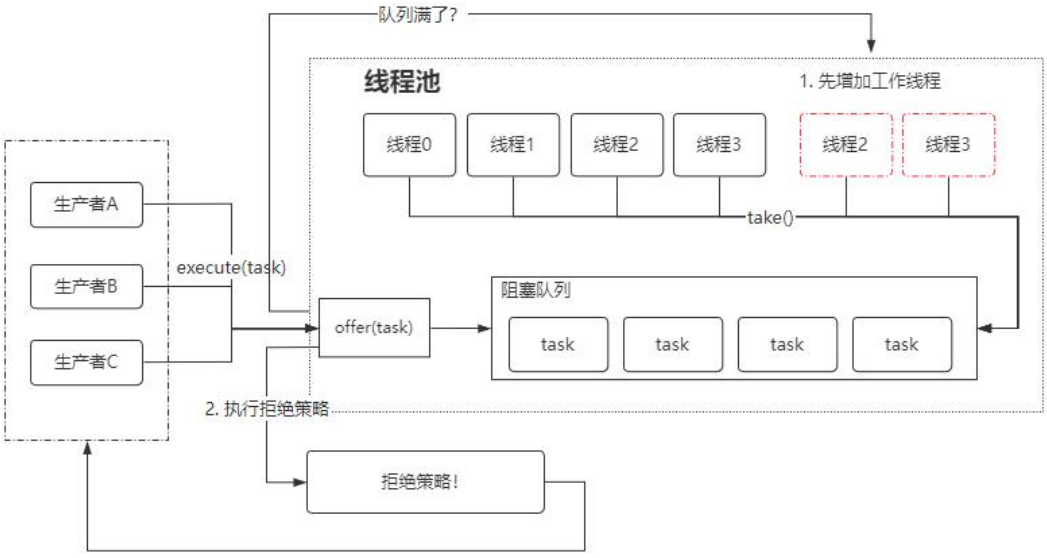
（图片）所以为了实现线程的复用，线程池里面用到了阻塞队列，简单来说就是线程池里面的工作线程处于一直运行状态，它会从阻塞队列中去获取待执行的任务，一旦队列空了，那这个工作线程就会被阻塞，直到下次有新的任务进来。

也就是说，工作线程是根据任务的情况实现阻塞和唤醒，从而达到线程复用的目的。

最后，线程池里面的资源限制，是通过几个关键参数来控制的，分别是核心线程数、最大线程数。

核心线程数表示默认长期存在的工作线程，而最大线程数是根据任务的情况动态创建的线程，主要是提高阻塞队列中任务的

处理效率。



以上就是我对这个问题的理解!

53.131 如何理解 Spring Boot 中的 Starter?

一个工作了 3 年的 Java 程序员，遇到一个 Spring Boot 的问题。

他对这个问题有一些了解，但是回答得不是很好，希望参考我的高手回答。

这个问题是：“如何理解 Spring Boot 中的 Starter”。

对于这个问题，看看普通人和高手的回答。

以下是高手的回答

Starter 是 Spring Boot 的四大核心功能特性之一，除此之外，Spring Boot 还有自动装配、Actuator 监控等特性。

Spring Boot 里面的这些特性，都是为了让开发者在开发基于 Spring 生态下的企业级应用时，只需要关心业务逻辑，

减少对配置和外部环境的依赖。

（如图）其中，Starter 是启动依赖，它的主要作用有几个。

1. Starter 组件以功能为纬度，来维护对应的 jar 包的版本依赖，

使得开发者可以不需要去关心这些版本冲突这种容易出错的细节。

2. Starter 组件会把对应功能的所有 jar 包依赖全部导入进来，避免了开发者自己去引入依赖带来的麻烦。

3. Starter 内部集成了自动装配的机制，也就说在程序中依赖对应的 starter 组件以后，

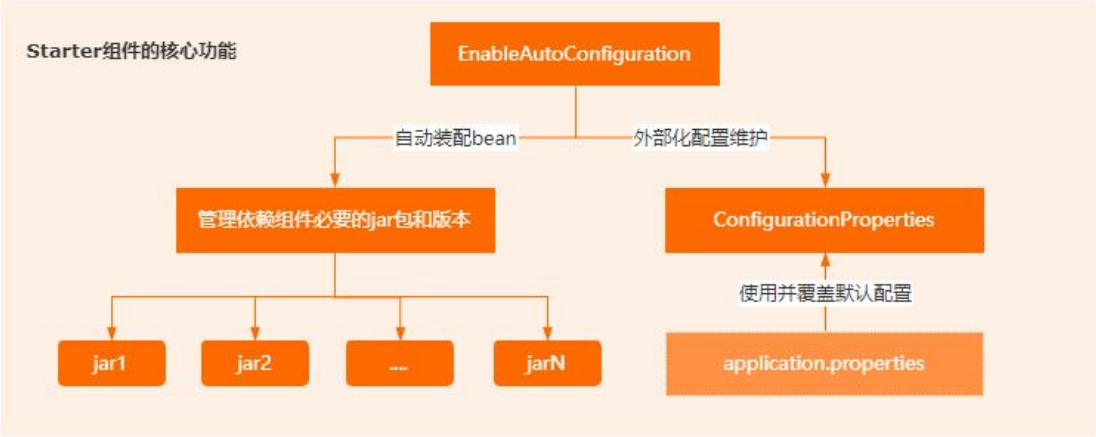
这个组件自动会集成到 Spring 生态下，并且对于相关 Bean 的管理，也是基于自动装配机制来完成。

4. 依赖 Starter 组件后，这个组件对应的功能所需要维护的外部化配置，会自动集成到 Spring Boot 里面，

我们只需要在 application.properties 文件里面进行维护就行了，比如 Redis 这个 starter，只需要在 application.properties

文件里面添加 redis 的连接信息就可以直接使用了。

在我看来，Starter 组件几乎完美的体现了 Spring Boot 里面约定优于配置的理念。



另外，Spring Boot 官方提供了很多的 Starter 组件，比如 Redis、JPA、MongoDB 等等。

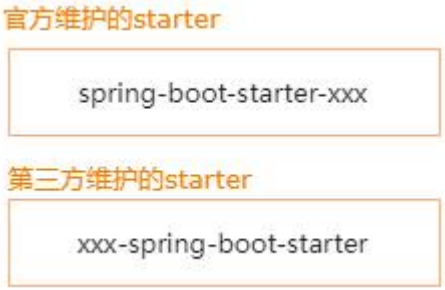
但是官方并不一定维护了所有中间件的 Starter，所以对于不存在的 Starter，第三方组件一般会自己去维护一个。

（如图）官方的 starter 和第三方的 starter 组件，最大的区别在于命名上。

官方维护的 starter 的以 spring-boot-starter 开头的前缀。

第三方维护的 starter 是以 spring-boot-starter 结尾的后缀

这也是一种约定优于配置的体现。



以上就是我对这个问题的理解。

53.132 IO 和 NIO 有什么区别?

IO 问题一直是面试的重灾区之一

但又是非常重要而且面试必问的知识点

一个工作了 7 年的粉丝私信我，他去面试了 4 家互联网公司，

有三个公司问他网络 IO 的问题，另外一个公司问了 Netty，结果都没回答上来。

好吧，对于“IO 和 NIO 的区别”，看看普通人和高手的回答。

以下是高手的回答

好的，关于这个问题，我会从下面几个方面来回答。

首先，I/O ，指的是 IO 流， 它可以实现数据从磁盘中的读取以及写入。

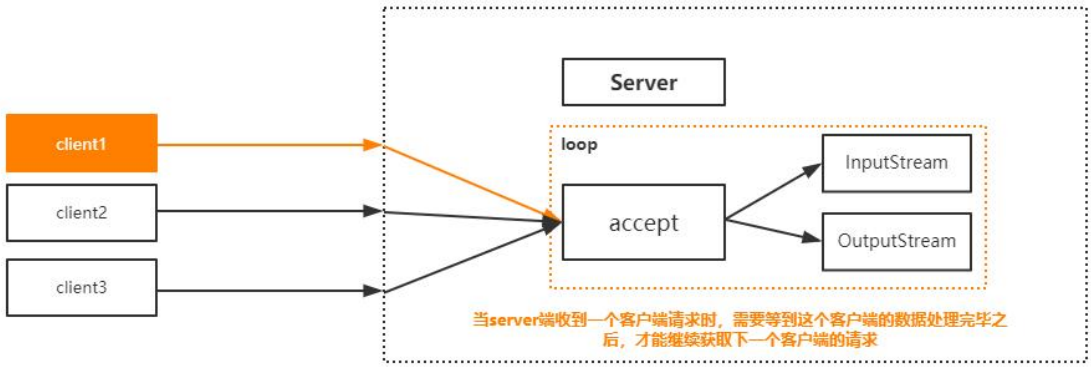
实际上，除了磁盘以外，内存、网络都可以作为 I/O 流的数据来源和目的地。

在 Java 里面，提供了字符流和字节流两种方式来实现数据流的操作。

其次，当程序是面向网络进行数据的 IO 操作的时候，Java 里面提供了 Socket 的方式来实现。

通过这种方式可以实现数据的网络传输。

（如图）基于 Socket 的 IO 通信，它是属于阻塞式 IO，也就是说，在连接以及 IO 事件未就绪的情况下，当前的连接会处于阻塞等待的状态。



如果一旦某个连接处于阻塞状态，那么后续的连接都得等待。所以服务端能够处理的连接数量非常有限。

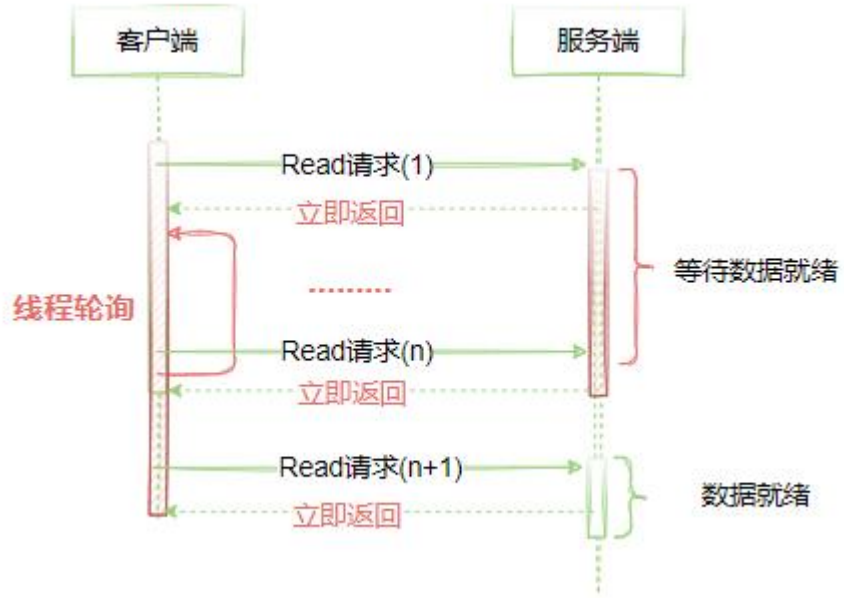
NIO，是 JDK1.4 里面新增的一种 NEW IO 机制，相比于传统的 IO，NIO 在效率上做了很大的优化，并且新增了几个核心组件。

Channel、Buffer、Selectors。

(如图）另外，还提供了非阻塞的特性，所以，对于网络 IO 来说，NIO 通常也称为 No-Block IO，非阻塞 IO。

也就是说，通过 NIO 进行网络数据传输的时候，如果连接未就绪或者 IO 事件未就绪的情况下，服务端不会阻塞当前连接，而是继续去轮询后续的连接来处理。

所以在 NIO 里面，服务端能够并行处理的链接数量更多。



因此，总的来说，IO 和 NIO 的区别，站在网络 IO 的视角来说，前者是阻塞 IO，后者是非阻塞 IO。

以上就是我对这个问题的理解。

53.133 什么是幂等？如何解决幂等性问题？

一个在传统行业工作了 7 年的粉丝私信我。

他最近去很多互联网公司面试，遇到的很多技术和概念都没听过。

其中就有一道题是：”什么是幂等、如何解决幂等性问题“？

他说，这个概念听都没听过，怎么可能回答出来。

好的，对于这个问题，看看普通人和高手的回答。

以下是高手的回答

好的。

所谓幂等，其实它是一个数学上的概念，在计算机编程领域中，幂等是指一个方法被多次重复执行的时候产生的影响和第一次执行的影响相同。

之所以要考虑到幂等性问题，是因为在网络通信中，存在两种行为可能会导致接口被重复执行。

1. 用户的重复提交或者用户的恶意攻击，导致这个请求会被多次重复执行。
2. 在分布式架构中，为了避免网络通信导致的数据丢失，在服务之间进行通信的时候都会设计超时重试的机制，而这种机制有可能导致服务端接口被重复调用。

所以在程序设计中，对于数据变更类操作的接口，需要保证接口的幂等性。

而幂等性的核心思想，其实就是保证这个接口的执行结果只影响一次，后续即便再次调用，也不能对数据产生影响，所以基于这样一个诉求，常见的解决方法有很多。

3. 使用数据库的唯一约束实现幂等，比如对于数据插入类的场景，比如创建订单，因为订单号肯定是唯一的，所以如果是多次调用就会触发数据库的唯一约束异常，从而避免一个请求创建多个订单的问题。
4. 使用 redis 里面提供的 setNX 指令，比如对于 MQ 消费的场景，为了避免 MQ 重复消费导致数据多次被修改的问题，可以在接受到 MQ 的消息时，把这个消息通过 setNx 写入到 redis 里面，一旦这个消息被消费过，就不会再次消费。
5. 使用状态机来实现幂等，所谓的状态机是指一条数据的完整运行状态的转换流程，比如订单状态，因为它的状态只会向前变更，所以多次修改同一条数据的时候，一旦状态发生变更，那么对这条数据修改造成的影响只会发生一次。

当然，除了这些方法以外，还可以基于 token 机制、去重表等方法来实现，但是不管是什么方法，无非就是两种，

- 要么就是接口只允许调用一次，比如唯一约束、基于 redis 的锁机制。
- 要么就是对数据的影响只会触发一次，比如幂等性、乐观锁

以上就是我对这个问题的理解。

53.134 如何中断一个正在运行的线程?

一个去京东面试的工作了 5 年的粉丝来找我说：

Mic 老师，你说并发编程很重要，果然我今天又挂在一道并发编程的面试题上了。

我问他问题是什么，他说：”如何中断一个正在运行中的线程？ “。

我说这个问题很多工作 2 年的人都知道~

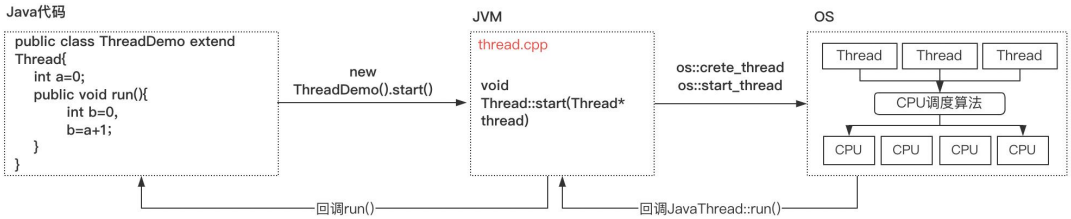
好吧，对于这个问题，来看看普通人和高手的回答。

以下是高手的回答

关于这个问题，我从几个方面来回答。

（如图）首先，线程是系统级别的概念，在 Java 里面实现的线程，最终的执行和调度都是由操作系统来决定的，JVM 只是对操作系统层面的线程做了一层包装而已。

所以我们在 Java 里面通过 start 方法启动一个线程的时候，只是告诉操作系统这个线程可以被执行，但是最终交给 CPU 来执行是操作系统的调度算法来决定的。



因此，理论上来说，要在 Java 层面去中断一个正在运行的线程，只能像类似于 Linux 里面的 kill 命令结束进程的方式一样，强制终止。

所以，Java Thread 里面提供了一个 stop 方法可以强行终止，但是这种方式是不安全的，因为有可能线程的任务还没有，导致出现运行结果不正确的问题。

要想安全的中断一个正在运行的线程，只能在线程内部埋下一个钩子，外部程序通过这个钩子来触发线程的中断命令。

（如图）因此，在 Java Thread 里面提供了一个 interrupt()方法，这个方法配合 isInterrupted()方法使用，就可以实现安全的中断机制。

这种实现方法并不是强制中断，而是告诉正在运行的线程，你可以停止了，不过是否要中断，取决于正在运行的线程，所以它能够保证线程运行结果的安全性。

以上就是我对这个问题的理解！

```java
class Task extends Thread{
    @Override
    public void run() {
        while (true) {
            if (this.isInterrupted()){  //中断信号判断
                System.out.println(Thread.currentThread().getName() + "线程停止了");
                break; //跳出循环，run方法执行结束，从而终止线程
            }
            System.out.println(i);
        }
    }
}

public class InterruptExample {
    public static void main(String[] args) throws InterruptedException {
        Task thread = new Task();
        thread.start();
        Thread.sleep(1000);

        //调用 interrupt 方法，给 thread 线程发送一个中断信号
        thread.interrupt();
    }
}
```

53.135 JVM 如何判断一个对象可以被回收

Hi， 我是 Mic。

今天分享一道一线互联网公司必问的面试题。

”JVM 如何判断一个对象可以被回收“

关于这个问题，来看看高手的回答。

以下是高手的回答

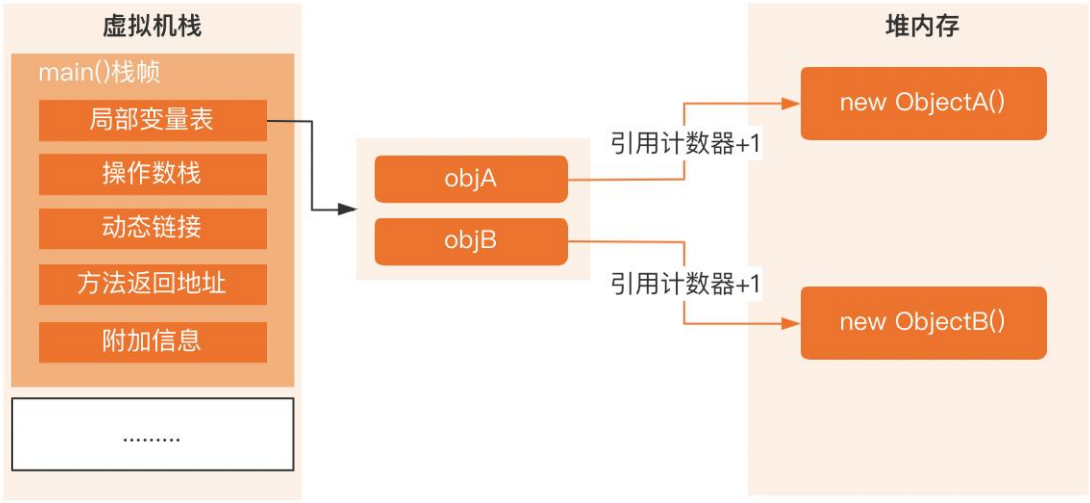
好的，面试官。

在 JVM 里面，要判断一个对象是否可以被回收，最重要的是判断这个对象是否还在被使用，只有没被使用的对象才能回收。

1. （如图）引用计数器，也就是为每一个对象添加一个引用计数器，用来统计指向当前对象的引用次数，

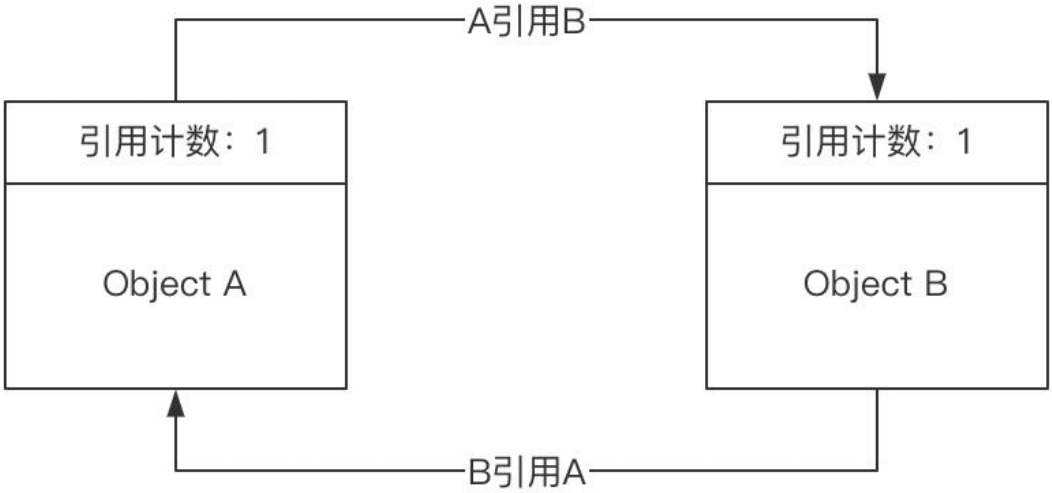
如果当前对象存在应用的更新，那么就对这个引用计数器进行增加，一旦这个引用计数器变成 0，就意味着它可以被回收了。

这种方法需要额外的空间来存储引用计数器，但是它的实现很简单，而且效率也比较高。



(如图）不过主流的 JVM 都没有采用这种方式，因为引用计数器在处理一些复杂的循环引用或者相互依赖的情况时，

可能会出现一些不再使用但是又无法回收的内存，造成内存泄露的问题。



2. 可达性分析，它的主要思想是，首先确定一系列肯定不能回收的对象作为 GC root，比如虚拟机栈里面的引用对象、本地方法栈引用的对象等，然后以 GC ROOT 作为起始节点，

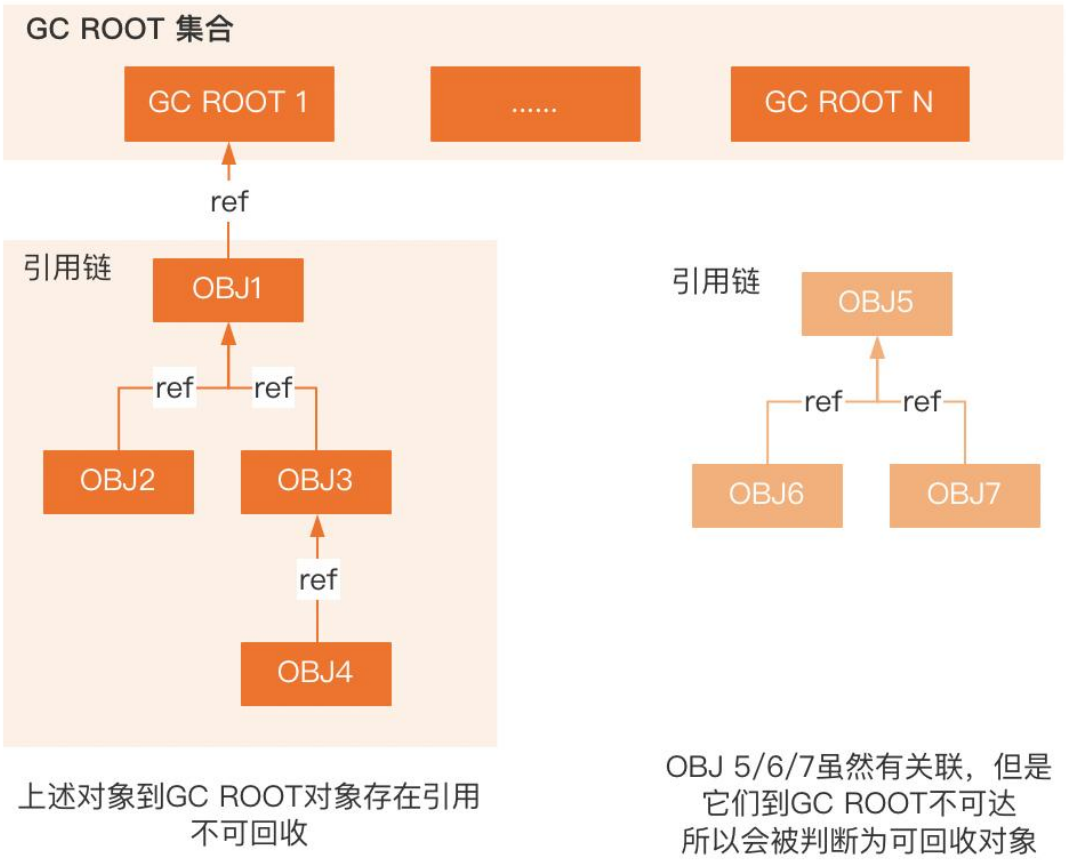
从这些节点开始向下搜索，去寻找它的直接和间接引用的对象，当遍历完之后如果发现有一些对象不可到达，

那么就认为这些对象已经没有用了，需要被回收。

在垃圾回收的时候，JVM 会首先找到所有的 GC root，这个过程会暂停所有用户线程，也就是 stop the world，然后再从 GC Roots 这些根节点向下搜索，可达的对象保留，不可达的就会回收掉。

可达性分析是目前主流 JVM 使用的算法。

以上就是我对这个问题的理解。



53.136 说说你对 Spring MVC 的理解

一个工作了 7 年的粉丝，他说在面试之前，Spring 这块的内容准备得很充分。

而且各种面试题也刷了，结果在面试的时候，面试官问：”说说你对 Spring MVC 的理解“。

这个问题一下给他整不会了，就是那种突然不知道怎么组织语言，最后因为回答比较混乱没通过面试。

ok，对于这个问题，我们来看看高手的回答。

> 以下是高手的回答

好的，关于这个问题，我会从几个方面来回答。

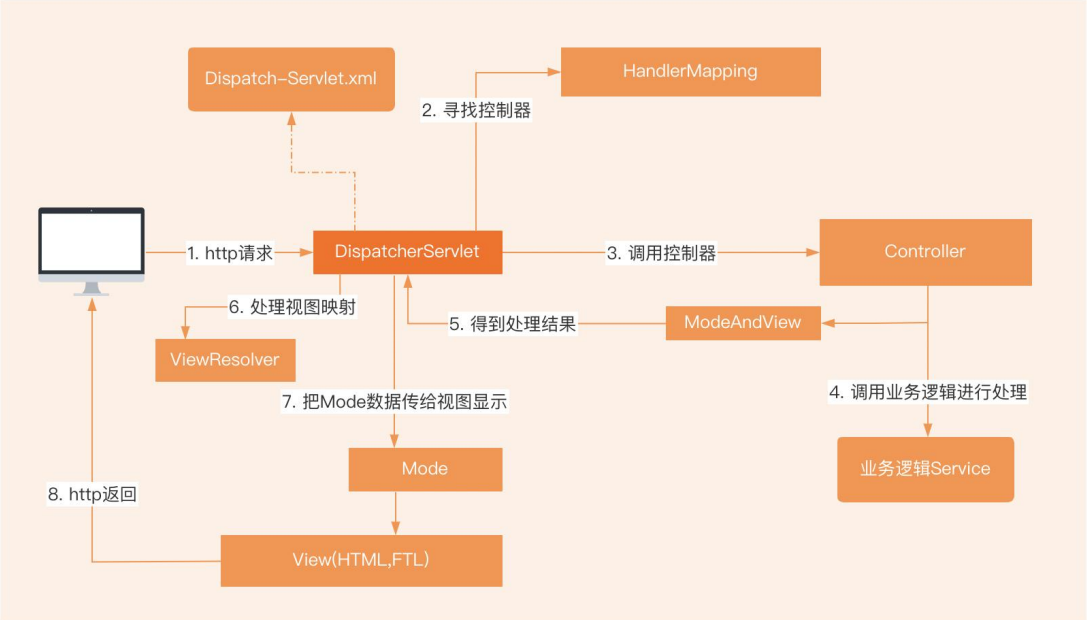
首先，Spring MVC 是是属于 Spring Framework 生态里面的一个模块，它是在 Servlet 基础上构建并且使用 MVC 模式设计的一个 Web 框架，
主要的目的是简化传统 Servlet+JSP 模式下的 Web 开发方式。

其次， Spring MVC 的整体架构设计对 Java Web 里面的 MVC 架构模式做了增强和扩展，主要有几个方面。

1. 把传统 MVC 框架里面的 Controller 控制器做了拆分，分成了前端控制器 DispatcherServlet 和后端控制器 Controller。
2. 把 Model 模型拆分成业务层 Service 和数据访问层 Repository。
3. 在视图层，可以支持不同的视图，比如 Freemark、velocity、JSP 等等。

所以，Spring MVC 天生就是为了 MVC 模式而设计的，因此在开发 MVC 应用的时候会更加方便和灵活。

（如图）Spring MVC 的具体工作流程是，浏览器的请求首先会经过 SpringMVC 里面的核心控制器 DispatcherServlet，它负责对请求进行分发到对应的 Controller。
Controller 里面处理完业务逻辑之后，返回 ModeAndView。
然后 DispatcherServlet 寻找一个或者多个 ViewResolver 视图解析器，找到 ModeAndView 指定的视图，并把数据显示到客户端。



以上就是我对 Spring MVC 的理解。

53.137 请说一下 Mysql 索引的优点和缺点?

Hi，我是 Mic

今天分享的这道面试题，让一个工作 4 年的小伙子去大众点评拿了 60W 年薪。

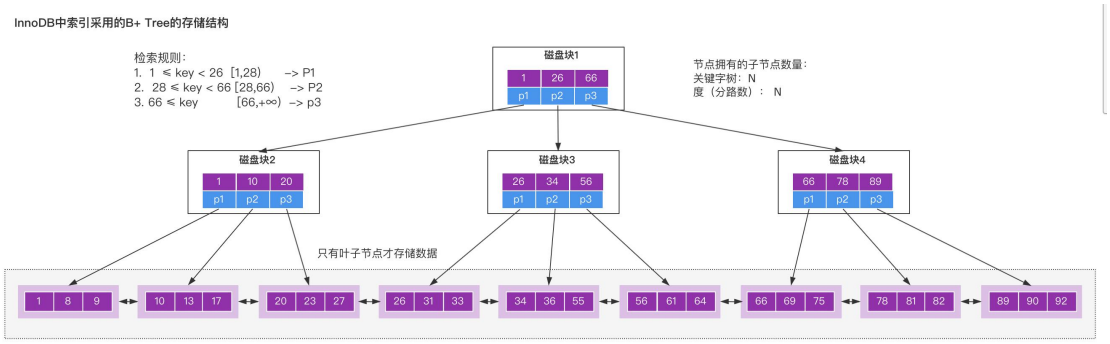
这道面试题是：“请你说一下 Mysql 索引的优点和缺点“

关于这道题，看看高手的回答

以下是高手的回答

（图片）索引，是一种能够帮助 Mysql 高效从磁盘上检索数据的一种数据结构。

在 Mysql 中的 InnoDB 引擎中，采用了 B+树的结构来实现索引和数据的存储



在我看来，Mysql 里面的索引的优点有很多

1. 通过 B+树的结构来存储数据，可以大大减少数据检索时的磁盘 IO 次数，从而提升数据查询的性能
2. B+树索引在进行范围查找的时候，只需要找到起始节点，然后基于叶子节点的链表结构往下读取即可，查询效率较高。
3. 通过唯一索引约束，可以保证数据表中每一行数据的唯一性

当然，索引的不合理使用，也会有带来很多的缺点。

4. 数据的增加、修改、删除，需要涉及到索引的维护，当数据量较大的情况下，索引的维护会带来较大的性能开销。
5. 一个表中允许存在一个聚簇索引和多个非聚簇索引，但是索引数不能创建太多，否则造成的索引维护成本过高。
6. 创建索引的时候，需要考虑到索引字段值的分散性，如果字段的重复数据过多，创建索引反而会带来性能降低。

在我看来，任何技术方案都会有两面性，大部分情况下，技术方案的选择更多的是看中它的优势 和当前问题的匹配度。

以上就是我对这个问题的理解。

53.138 new String("abc")到底创建了几个对象?

一个工作了 6 年的粉丝和我说，

最近面试感觉越来越难的，基本上都会问技术底层原理，甚至有些还会问到操作系统层面的知识。

我说，现在各个一线大厂有很多优秀的程序员毕业了，再加上市场大环境不好对程序员的需求量也在减少。

如果技术底子不好，确实找工作会很困难。

今天分享的问题是：” new String(“abc”)到底创建了几个对象?

关于这个问题，看看高手的回答。

> 以下是高手的回答

好的，面试官。

首先，这个代码里面有一个 `new` 关键字，这个关键字是在程序运行时，根据已经加载的系统类 String，在堆内存里面实例化的一个字符串对象。

然后，在这个 String 的构造方法里面，传递了一个“abc”字符串，因为 String 里面的字符串成员变量是 final 修饰的，所以它是一个字符串常量。

接下来，JVM 会拿字面量“abc” 去字符串常量池里面试图去获取它对应的 String 对象引用，如果拿不到，就会在堆内存里面创建一个”abc”的 String 对象

并且把引用保存到字符串常量池里面。

后续如果再有字面量“abc”的定义，因为字符串常量池里面已经存在了字面量“abc”的引用，所以只需要从常量池获取对应的引用就可以了，不需要再创建。

所以，对于这个问题，我认为的答案是

1. 如果 `abc` 这个字符串常量不存在，则创建两个对象，分别是 `abc` 这个字符串常量，以及 `new String` 这个实例对象。
2. 如果 `abc` 这字符串常量存在，则只会创建一个对象

53.139 常见的限流算法有哪些?

一个在传统行业工作了 5 年的程序员。

去一个互联网公司面试，遇到了一个限流的问题。

因为之前完全没接触过分布式这块的项目，所以根本就回答不上来，向我来求助。

其中有一个问题是：”请你说一下常见的限流算法“。

对于这个问题，看看高手的回答。

以下是高手的回答

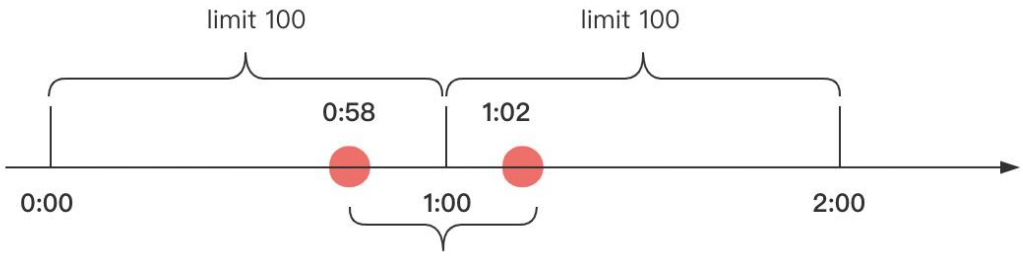
好的，关于这个问题，我会从几个方面来回答。

首先，限流算法是一种系统保护策略，主要是避免在流量高峰导致系统被压垮，造成系统不可用的问题。

常见的限流算法有 5 种。

1. （如图）计数器限流，一般用在单一维度的访问频率限制上，比如短信验证码每隔 60s 只能发送一次，或者接口调用次数等

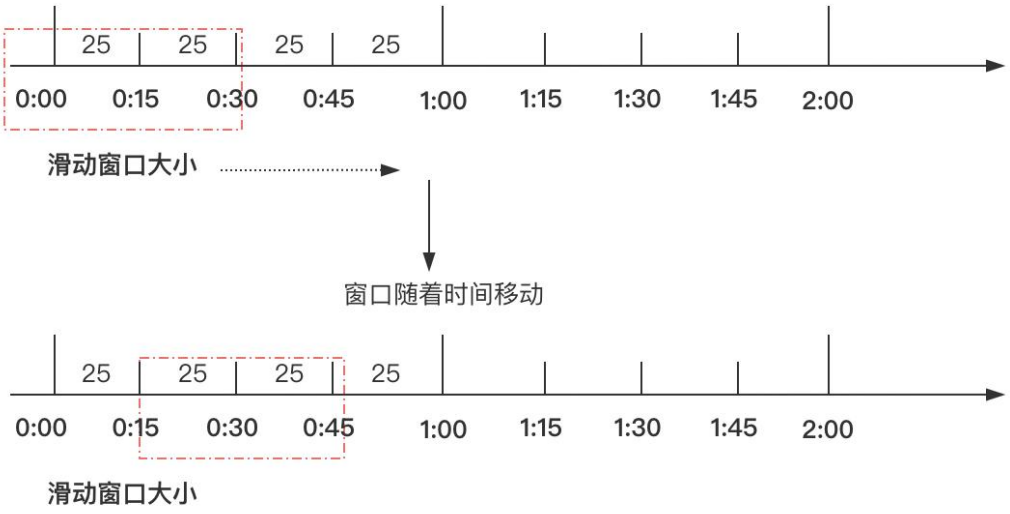
它的实现方法很简单，每调用一次就加 1，处理结束以后减一。



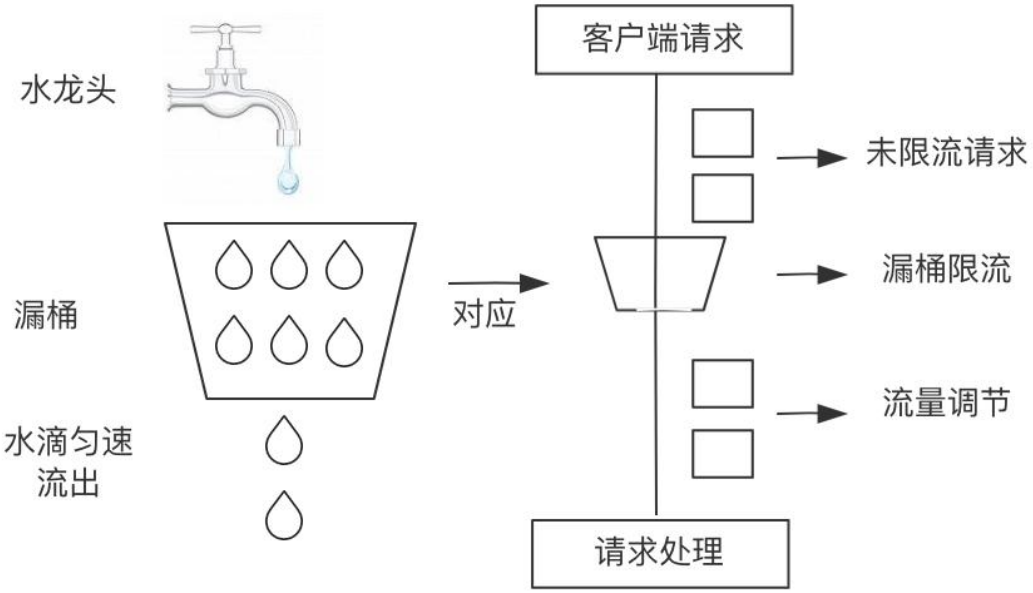
2. （如图）滑动窗口限流，本质上也是一种计数器，只是通过以时间为维度的可滑动窗口设计，来减少了临界值带来的并发超过阈值的问题。

每次进行数据统计的时候，只需要统计这个窗口内每个时间刻度的访问量就可以了。

Spring Cloud 里面的熔断框架 Hystrix ，以及 Spring Cloud Alibaba 里面的 Sentinel 都采用了滑动窗口来做数据统计。



3. (如图)漏桶算法，它是一种恒定速率的限流算法，不管请求量是多少，服务端的处理效率是恒定的。基于 MQ 来实现的生产者消费者模型，其实算是一种漏桶限流算法。



4. （如图）令牌桶算法，相对漏桶算法来说，它可以处理突发流量的问题。

它的核心思想是，令牌桶以恒定速率去生成令牌保存到令牌桶里面，桶的大小是固定的，令牌桶满了以后就不再生成令牌。

每个客户端请求进来的时候，必须要从令牌桶获得一个令牌才能访问，否则排队等待。在流量低峰的时候，令牌桶会出现堆积，因此当出现瞬时高峰的时候，有足够多的令牌可以获取，因此令牌桶能够允许瞬时流量的处理。

网关层面的限流、或者接口调用的限流，都可以使用令牌桶算法，像 Google 的 Guava，和 Redisson 的限流，都用到了令牌桶算法

在我看来，限流的本质是实现系统保护，最终选择什么样的算法，一方面取决于统计的精准度，另一方面考虑限流维度和场景的需求。

以上就是我对这个问题的理解

53.140 TCP 协议为什么要设计三次握手?

一个工作 5 年的粉丝，最近去面试了很多公司，每次都被各种技术原理题问得语无伦次。

由于找了快 1 个月时间的工作，有点焦虑，来向我求助。

我能做的只是保证每天更新一个面试题，然后问他印象最深刻的一个面试题是什么，他说。

“TCP 协议为什么要设计三次握手”。

这个问题的高手回答，我整理成了文档，大家可以在我主页加 V 领取。

好的，关于这个问题，我们来看看高手的回答。

以下是高手的回答

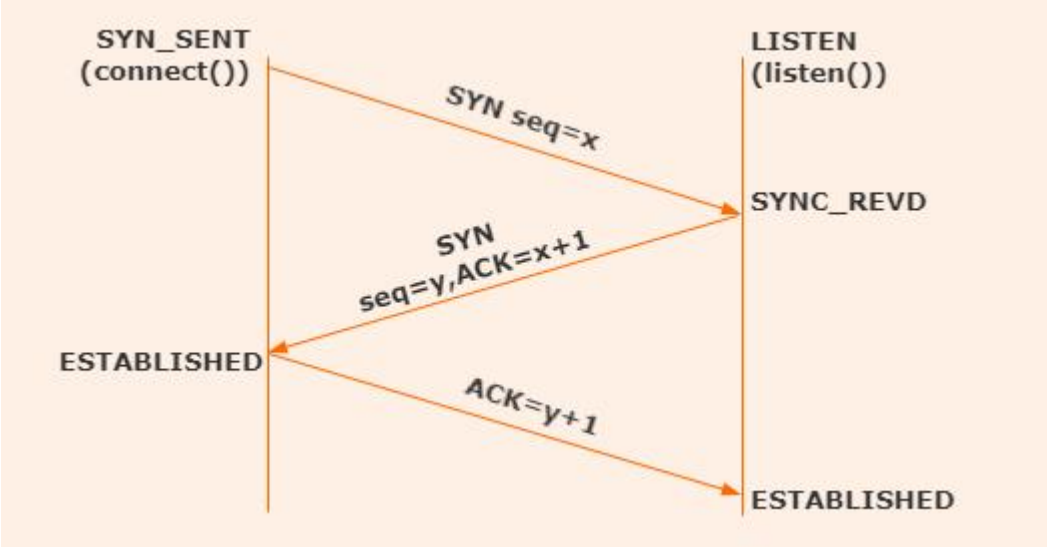
关于这个问题，我会从下面 3 个方面来回答。

1. TCP 协议，是一种可靠的，基于字节流的，面向连接的传输层协议。
    - 可靠性体现在 TCP 协议通信双方的数据传输是稳定的，即便是在网络不好的情况下，TCP 都能够保证数据传输到目标端，而这个可靠性是基于数据包确认机制来实现的。
    - TCP 通信双方的数据传输是通过字节流来实现传输的
    - 面向连接，是说数据传输之前，必须要建立一个连接，然后基于这个连接进行数据传输
2. （如图）因为 TCP 是面向连接的协议，所以在进行数据通信之前，需要建立一个可靠的连接，TCP 采用了三次握手的方式来实现连接的建立。

所谓的三次握手，就是通信双方一共需要发送三次请求，才能确保这个连接的建立。

- 客户端向服务端发送连接请求并携带同步序列号 SYN。

- 服务端收到请求后，发送 SYN 和 ACK， 这里的 SYN 表示服务端的同步序列号，ACK 表示对前面收到请求的一个确认，表示告诉客户端，我收到了你的请求。
- 客户端收到服务端的请求后，再次发送 ACK，这个 ACK 是针对服务端连接的一个确认，表示告诉服务端，我收到了你的请求。



3. 之所以 TCP 要设计三次握手，我认为有三个方面的原因：
    - TCP 是可靠性通信协议，所以 TCP 协议的通信双方都必须要维护一个序列号，去标记已经发送出去的数据包，哪些是已经被对方签收的。而三次握手就是通信双方相互告知序列号的起始值，为了确保这个序列号被收到，所以双方都需要有一个确认的操作。
    - TCP 协议需要在一个不可靠的网络环境下实现可靠的数据传输，意味着通信双方必须要通过某种手段来实现一个可靠的数据传输通道，而三次通信是建立这样一个通道的最小值。当然还可以四次、五次，只是没必要浪费这个资源。
    - 防止历史的重复连接初始化造成的混乱问题，比如说在网络比较差的情况下，客户端连续多次发送建立连接的请求，假设只有两次握手，那么服务端只能选择接受或者拒绝这个连接请求，但是服务端不知道这次请求是不是之前因为网络堵塞而过期的请求，也就是说服务端不知道当前客户端的连接是有效还是无效。

以上就是我对这个问题的理解。

53.141 请简单说一下你对受检异常和非受检异常的理解

Hi，我是 Mic

今天给大家分享一道阿里一面的面试题。

这道题目比较基础，但是确难倒了很多人。

关于”受检异常和非受检异常的理解“

我们来看看高手的回答。

> 以下是高手的回答

好的。

(如图)所谓的受检异常，表示在编译的时候强制检查的异常，这种异常需要显示的通过 try/catch 来捕捉，或者通过 throws 抛出去，否则从程序无法通过编译。

而非受检异常，表示在编译器可以不需要强制检查的异常，这种异常不需要显示去捕捉。

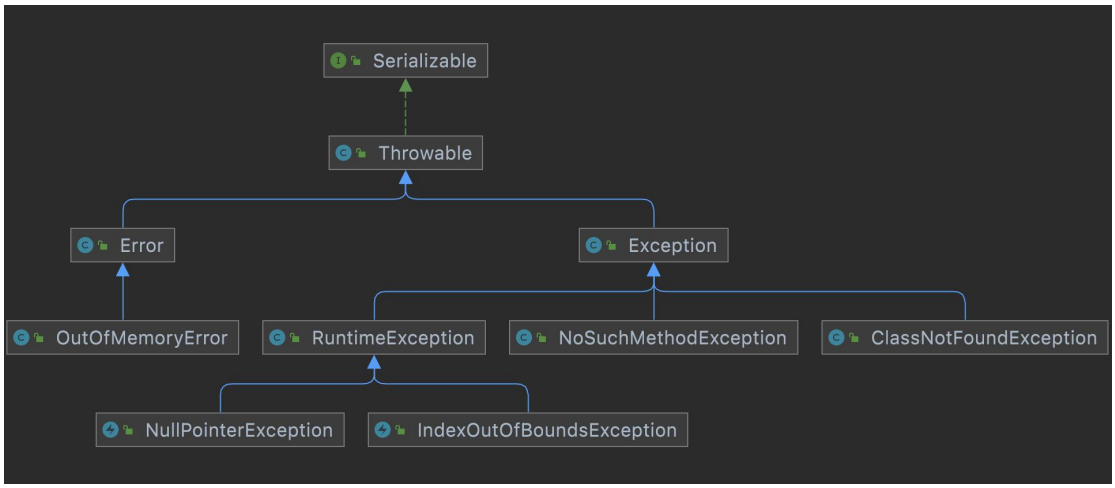
（如图）在 Java 里面，所有的异常都是继承自 java.lang.Throwable 类，Throwable 有两个直接子类，Error 和 Exception。

Error 用来表示程序底层或者硬件有关的错误，这种错误和程序本身无关，比如常见的 OOM 异常。这种异常和程序本身无关，所以不需要检查，属于非受检异常。

Exception 表示程序中的异常，可能是由于程序不严谨导致的，比如 NullPointerException。

Exception 下面派生了 RuntimeException 和其他异常，其中 RuntimeException 运行时异常，也是属于非受检异常。

所以，除了 Error 和 RuntimeException 及派生类以外，其他异常都是属于受检异常，比如 IOException、SQLException。



之所以在 Java 中要设计一些强制检查的异常，我认为主要原因是考虑到程序的正确性、稳定性和可靠性。

比如数据库异常、文件读取异常，这些异常是程序无法提前预料到的，但是一旦出现问题，就会造成资源被占用导致程序出现问题。

所以这些异常我们需要主动捕获，一旦出现问题，我们可以做出相应的处理，比如关闭数据库连接、文件流的释放等。

以上就是我对这个问题的理解！

53.142 为什么引入偏向锁、轻量级锁，介绍下升级流程

Hi，我是 Mic

一个工作了 7 年的粉丝来找我，他说最近被各种锁搞晕了。

比如，共享锁、排它锁、偏向锁、轻量级锁、自旋锁、重量级锁、

间隙锁、临键锁、意向锁、读写锁、乐观锁、悲观锁、表锁、行锁。

然后前两天去面试，被问到偏向锁、轻量级锁，结果没回答上来。

ok，关于 Synchronized 锁升级的原理，看看普通人和高手的回答。

以下是高手的回答

好的，面试官。

1. Synchronized 在 jdk1.6 版本之前，是通过重量级锁的方式来实现线程之间锁的竞争。

之所以称它为重量级锁，是因为它的底层底层依赖操作系统的 Mutex Lock 来实现互斥功能。

（如图）Mutex 是系统方法，由于权限隔离的关系，应用程序调用系统方法时需要切换到内核态来执行。

这里涉及到用户态向内核态的切换，这个切换会带来性能的损耗。

2. 在 jdk1.6 版本中，synchronized 增加了锁升级的机制，来平衡数据安全性和性能。简单来说，就是线程去访问 synchronized 同步代码块的时候，synchronized 根据

线程竞争情况，会先尝试在不加重量级锁的情况下去保证线程安全性。所以引入了偏向锁和轻量级锁的机制。

偏向锁，就是直接把当前锁偏向于某个线程，简单来说就是通过 CAS 修改偏向锁标记，这种锁适合同一个线程多次去申请同一个锁资源并且没有其他线程竞争的场景。

轻量级锁也可以称为自旋锁，基于自适应自旋的机制，通过多次自旋重试去竞争锁。自旋锁优点在于它避免避免了用户态到内核态的切换带来的性能开销。

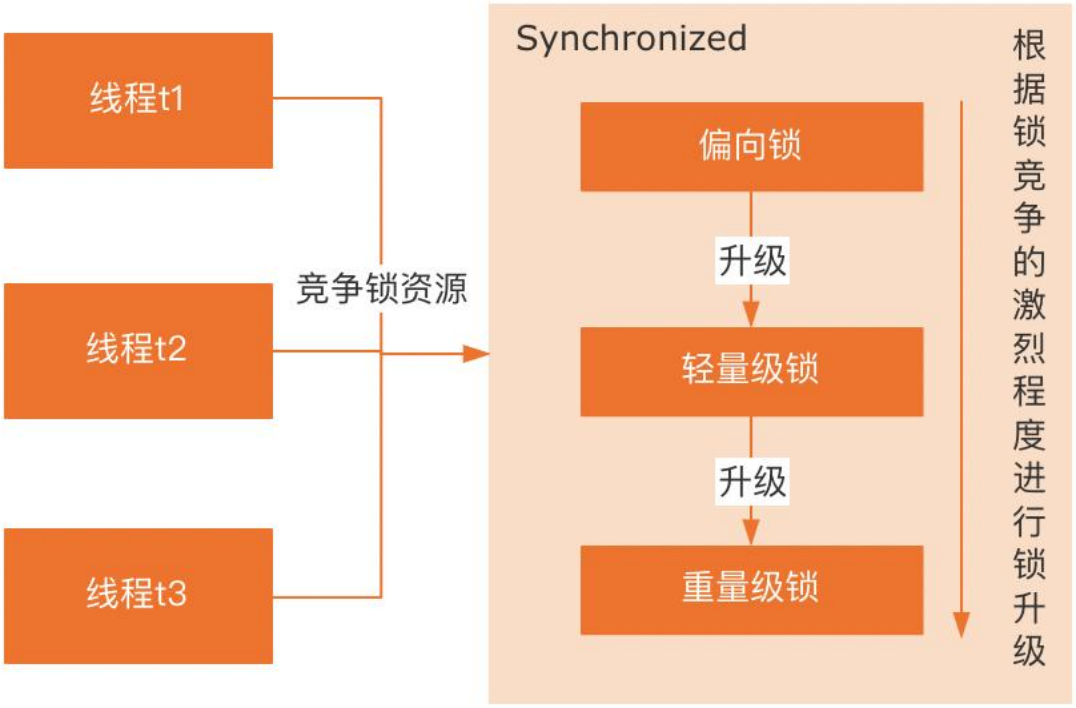
3. （如图）Synchronized 引入了锁升级的机制之后，如果有线程去竞争锁：

首先，synchronized 会尝试使用偏向锁的方式去竞争锁资源，如果能够竞争到偏向锁，表示加锁成功直接返回。如果竞争锁失败，说明当前锁已经偏向了其他线程。

需要将锁升级到轻量级锁，在轻量级锁状态下，竞争锁的线程根据自适应自旋次数去尝试抢占锁资源，如果在轻量级锁状态下还是没有竞争到锁，

就只能升级到重量级锁，在重量级锁状态下，没有竞争到锁的线程就会被阻塞，线程状态是 Blocked。

处于锁等待状态的线程需要等待获得锁的线程来触发唤醒。



总的来说， Synchronized 的锁升级的设计思想，在我看来本质上是一种性能和安全性的平衡，也就是如何在不加锁的情况下能够保证线程安全性。

这种思想在编程领域比较常见，比如 Mysql 里面的 MVCC 使用版本链的方式来解决多个并行事务的竞争问题。

以上就是我对这个问题的理解。

53.143 介绍下 Spring IoC 的工作流程

Hi，我是 Mic

一个工作了 4 年的粉丝，在面试的时候遇到一个这样的问题。

“介绍一下 Spring IOC 的工作流程”

他说回答得不是很好，希望我能帮他梳理一下。

这个问题高手部分的回答已经整理成了文档，可以在主页加 V 领取。

关于这个问题，我们来看看高手的回答。

> 以下是高手的回答

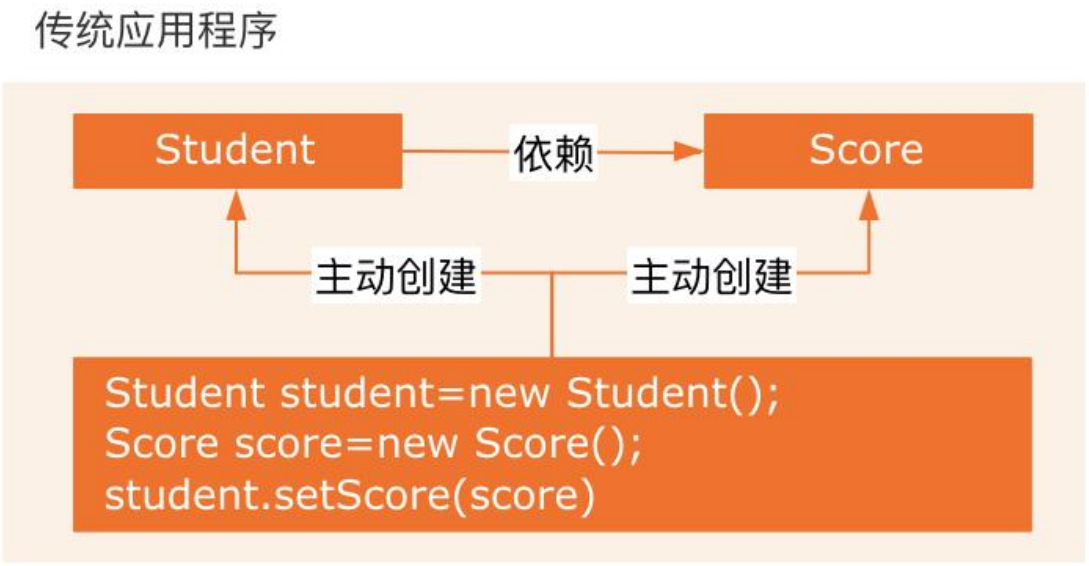
好的，这个问题我会从几个方面来回答。

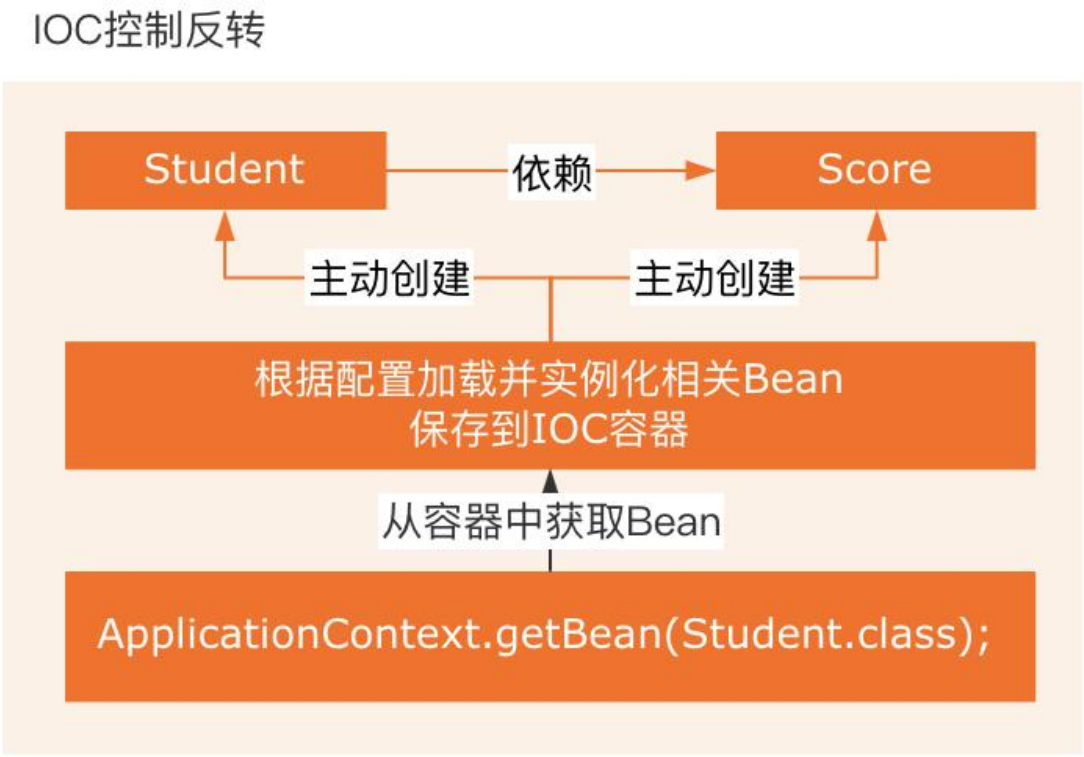
- IOC 是什么
- Bean 的声明方式
- IOC 的工作流程

IOC 的全称是 Inversion Of Control，也就是控制反转，它的核心思想是把对象的管理权限交给容器。（展示图 1）

（动态出现图 2）应用程序如果需要使用到某个对象实例，直接从 IOC 容器中去获取就行，这样设计的好处是降低了程序里面对象与对象之间的耦合性。

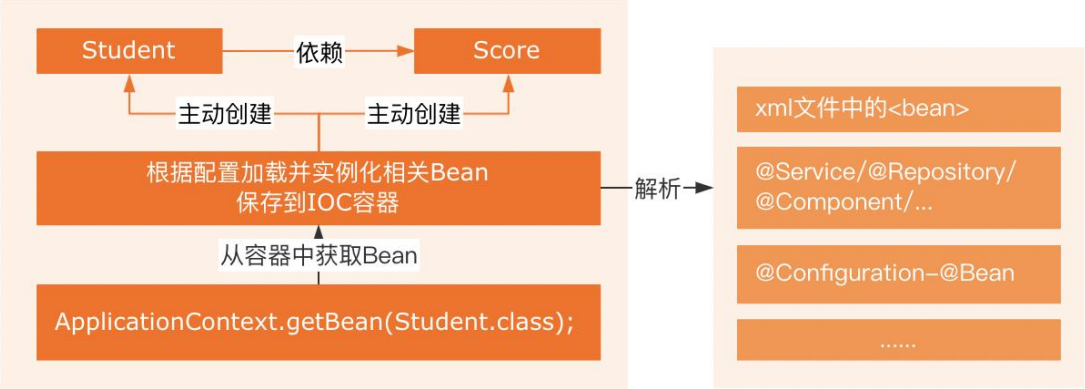
使得程序的整个体系结构变得更加灵活。





Spring 里面很多方式去定义 Bean，(如图)比如 XML 里面的<bean>标签、@Service、@Component、@Repository、@Configuration 配置类中的@Bean 注解等等。

Spring 在启动的时候，会去解析这些 Bean 然后保存到 IOC 容器里面。

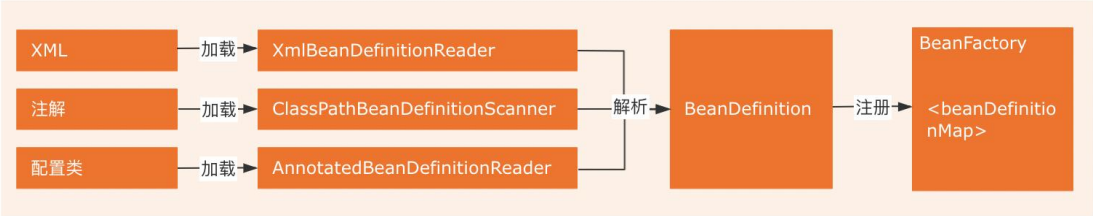


Spring IOC 的工作流程大致可以分为两个阶段。

第一个阶段，就是 IOC 容器的初始化

这个阶段主要是根据程序中定义的 XML 或者注解等 Bean 的声明方式

(如图）通过解析和加载后生成 BeanDefinition，然后把 BeanDefinition 注册到 IOC 容器。



通过注解或者 xml 声明的 bean 都会解析得到一个 BeanDefinition 实体，实体中包含这个 bean 中定义的基本属性。

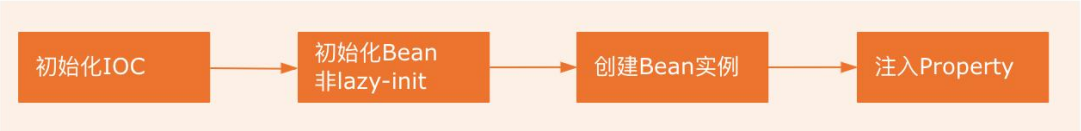
最后把这个 BeanDefinition 保存到一个 Map 集合里面，从而完成了 IOC 的初始化。

IoC 容器的作用就是对这些注册的 Bean 的定义信息进行处理和维护，它 IoC 容器控制反转的核心。

第二个阶段，完成 Bean 初始化及依赖注入

然后进入到第二个阶段，这个阶段会做两个事情（如图）

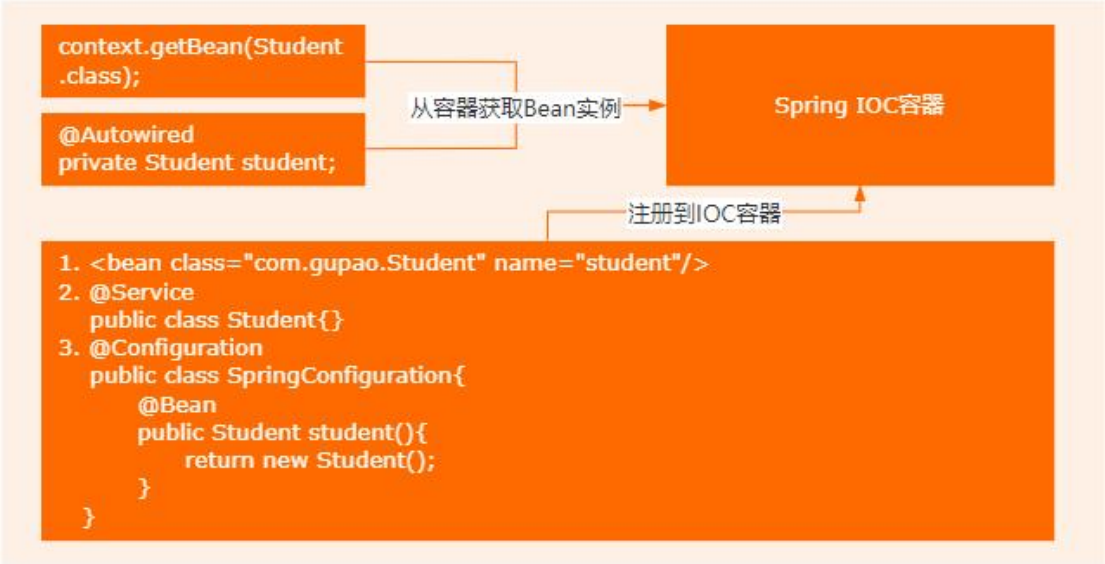
1. 通过反射针对没有设置 lazy-init 属性的单例 bean 进行初始化。
2. 完成 Bean 的依赖注入。



第三个阶段，Bean 的使用

（如图）通常我们会通过@Autowired 或者 BeanFactory.getBean()从 IOC 容器中获取指定的 bean 实例。

另外，针对设置 layy-init 属性以及非单例 bean 的实例化，是在每次获取 bean 对象的时候，调用 bean 的初始化方法来完成实例化的，并且 Spring IOC 容器不会去管理这些 Bean。



以上就是我对这个问题的理解。

53.144 @Resource 和 @Autowired 的区别

Hi，大家好，我是 Mic。

一个工作 2 年的粉丝，问我一个 Spring 里面的问题。

希望我能从不同的视角去分析，然后碾压面试官。

这个问题是： “@Resource 和@Autowired”的区别。

高手部分的回答我已经整理成了文档，需要的小伙伴可以在主页加 V 领取。

以下是高手的回答

好的，面试官。

@Resource 和@Autowired 这两个注解的作用都是在 Spring 生态里面去实现 Bean 的依赖注入。

下面我分别说一下@Autowired 和@Resource 这两个注解。

闪现［@Autowired 的作用详解 ］几个字。

（如图）首先，@Autowired 是 Spring 里面提供的一个注解，默认是根据类型来实现 Bean 的依赖注入。

@Autowired 注解里面有一个 `required` 属性默认值是 true，表示强制要求 bean 实例的注入，

在应用启动的时候，如果 IOC 容器里面不存在对应类型的 Bean，就会报错。

当然，如果不希望自动注入，可以把这个属性设置成 false。

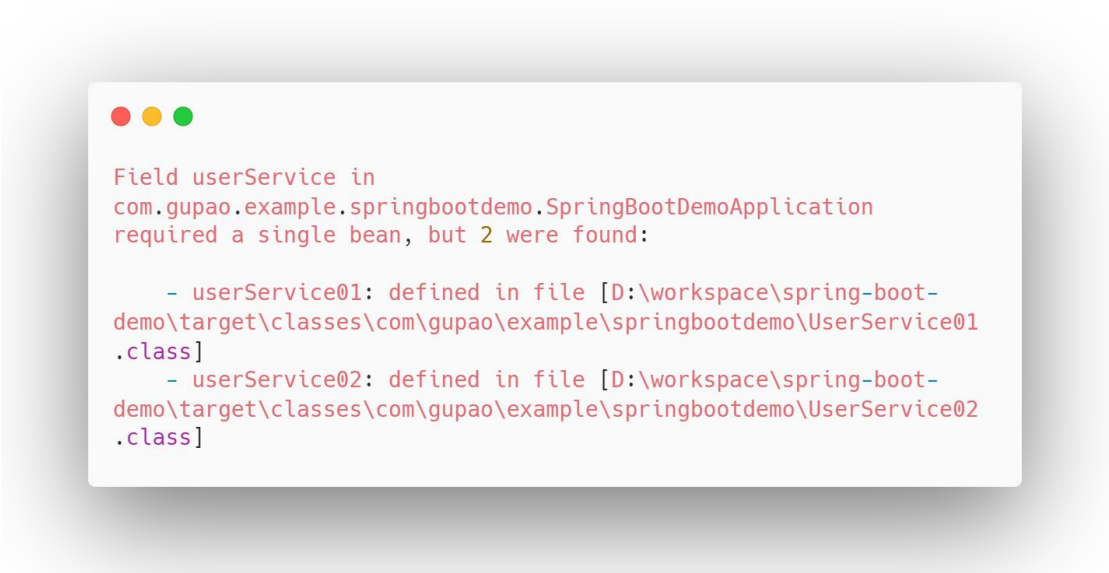
```java
@Service
public class HelloService {
    public void sayHi() {
      //TODO
    }
}

@Service
public class UserService {

    @Autowired(required = true)
    private HelloService helloService;

    public void user() {
      helloService.sayHi();
    }
}
```

（如图）其次呢， 如果在 Spring IOC 容器里面存在多个相同类型的 Bean 实例。由于 @Autowired 注解是根据类型来注入 Bean 实例的

```java
@Configuration
public class SpringConfiguration{

  @Bean("hello1")
  public HelloService hello1(){
    return new HelloService();
  }
  @Bean("hello2")
  public HelloService hello2(){
    return new HelloService();
  }

}

@Service
public class UserService{
  @Autowired
  private HelloService helloService;
}
```

（如图）所以 Spring 启动的时候，会提示一个错误，大概意思原本只能注入一个单实例 Bean，

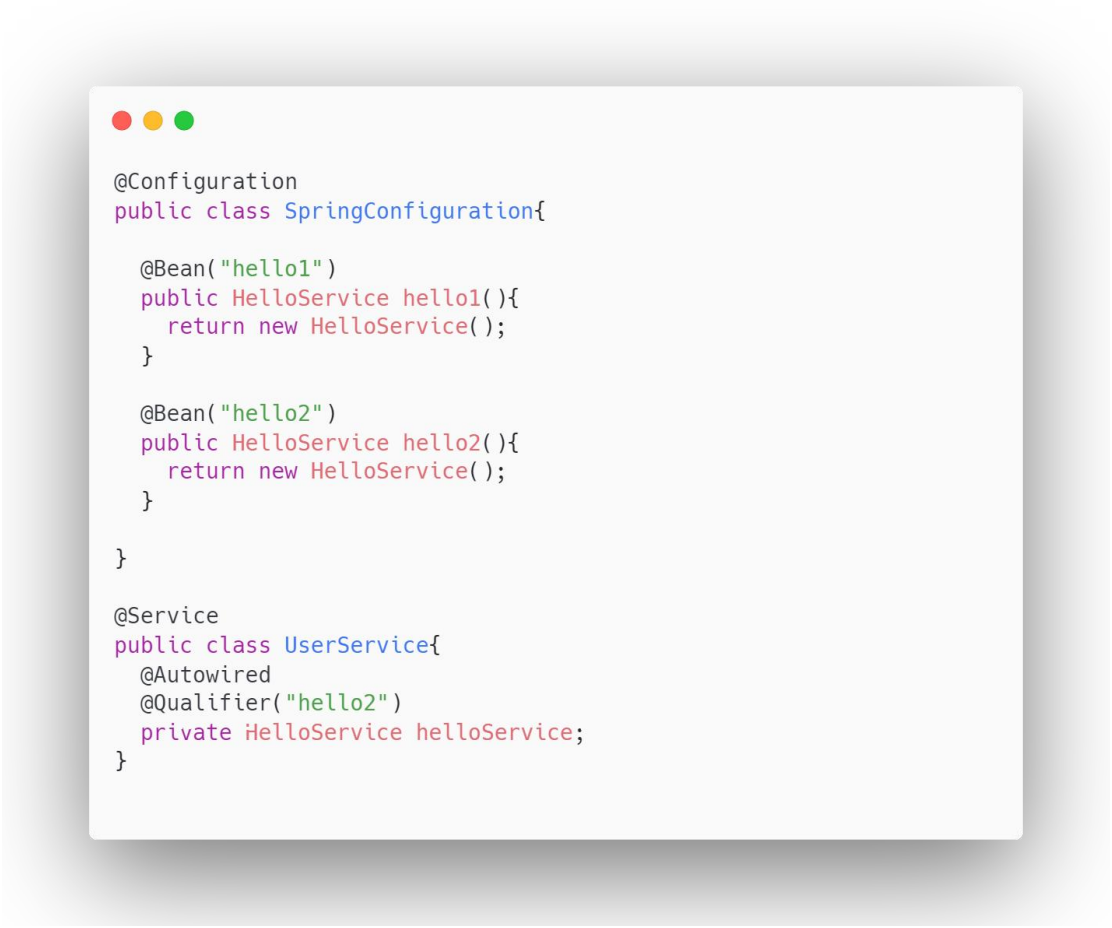
但是在 IOC 容器里面却发现有多个，导致注入失败。



当然，针对这个问题，我们可以使用 @Primary 或者@Qualifier 这两个注解来解决。

@Primary 表示主要的 bean，当存在多个相同类型的 Bean 的时候，优先使用声明了 @Primary 的 Bean。

（如图）@Qualifier 的作用类似于条件筛选，它可以根据 Bean 的名字找到需要装配的目标 Bean。



```
@Configuration
public class SpringConfiguration{

  @Bean("hello1")
  public HelloService hello1(){
    return new HelloService();
  }

  @Bean("hello2")
  public HelloService hello2(){
    return new HelloService();
  }

}

@Service
public class UserService{
  @Autowired
  @Qualifier("hello2")
  private HelloService helloService;
}
```

闪现［@Resource 的作用详解 ］几个字。

接下来，我再解释一下@Resource 注解。

@Resource 是 JDK 提供的注解，只是 Spring 在实现上提供了这个注解的功能支持。

它的使用方式和@Autowired 完全相同，（如图）最大的差异于@Resource 可以支持 ByName 和 ByType 两种注入方式。

如果使用 name，Spring 就根据 bean 的名字进行依赖注入，如果使用 type，Spring 就根据类型实现依赖注入。

如果两个属性都没配置，就先根据定义的属性名字去匹配，如果没匹配成功，再根据类型匹配。两个都没匹配到，就报错。

```java
@Service
public class UserService{
    @Resource(name = "userService01")
    private UserService01 userService;

    @Resource(type = UserService02.class)
    private UserService02 userService02;
}
```

最后，我再总结一下。

- @Autowired 是根据 type 来匹配，@Resource 可以根据 name 和 type 来匹配，默认是 name 匹配。
- @Autowired 是 Spring 定义的注解，@Resource 是 JSR 250 规范里面定义的注解，而 Spring 对 JSR 250 规范提供了支持。
- @Autowired 如果需要支持 name 匹配，就需要配合@Primary 或者@Qualifier 来实现。

以上就是我对这个问题的理解。

53.145 Spring 中，有两个 id 相同的 bean，会报错吗，如果会报错，在哪个阶段报错

Hi，大家好，我是 Mic

一个工作 3 年的粉丝，早上 6 点给我微信发语音，把我直接吓醒。

我以为什么天大的事情，结果一问才知道。

面试官问了他一个问题没答上来，问题是“Spring 里面，如果两个 id 相同的 bean 会报错吗？如果会，在哪个阶段报错？”

> 以下是高手的回答

好的，关于这个问题，我从几个点来回答。

首先，在同一个 XML 配置文件里面，不能存在 id 相同的两个 bean，否则 spring 容器启动的时候会报错（如图）。

因为 id 这个属性表示一个 Bean 的唯一标志符号，所以 Spring 在启动的时候会去验证 id 的唯一性，一旦发现重复就会报错，

这个错误发生 Spring 对 XML 文件进行解析转化为 BeanDefinition 的阶段。

但是在两个不同的 Spring 配置文件里面，可以存在 id 相同的两个 bean。 IOC 容器在加载 Bean 的时候，默认会多个相同 id

的 bean 进行覆盖。

在 Spring3.x 版本以后，这个问题发生了变化

我们知道 Spring3.x 里面提供@Configuration 注解去声明一个配置类，然后使用 @Bean 注解实现 Bean 的声明，这种方式完全取代了 XML。

在这种情况下，（如图）如果我们在同一个配置类里面声明多个相同名字的 bean，在 Spring IOC 容器中只会注册第一个声明的 Bean 的实例。

后续重复名字的 Bean 就不会再注册了。

像这样一段代码，在 Spring IOC 容器里面，只会保存 UserService01 这个实例，后续相同名字的实例不会再加载。

```java
@Configuration
public class SpringConfiguration {

    @Bean(name = "userService")
    public UserService01 userService01(){
        return new UserService01();
    }

    @Bean(name = "userService")
    public UserService02 userService02(){
        return new UserService02();
    }
}
```

（如图）如果使用@Autowired 注解根据类型实现依赖注入，因为 IOC 容器只有 UserService01 的实例，所以启动的时候会提示找不到 UserService02 这个实例。

```java
    @Autowired
    private UserService01 userService;
    @Autowired
    private UserService02 userService02;
```

如果使用@Resource 注解根据名词实现依赖注入，在 IOC 容器里面得到的实例对象是 UserService01，

于是 Spring 把 UserService01 这个实例赋值给 UserService02，就会提示类型不匹配错误。

```
@Resource(name="userService")
private UserService01 userService;
@Resource(name="userService")
private UserService02 userService02;
```

这个错误，是在 Spring IOC 容器里面的 Bean 初始化之后的依赖注入阶段发生的。

以上就是我对这个问题的理解。

53.146 Kafka 怎么避免重复消费

Hi，大家好，我是 Mic

一个工作 5 年的粉丝找到我。

他说： “Mic 老师，你要是能回答出这个问题，我就佩服你”

我当场就懵了，现在打赌都这么随意了吗?

我问他问题是什么，他说“Kafka 如何避免重复消费的问题！”

这个问题的高手部分的回答我已经整理成了文档，需要的小伙伴可以在主页加 V 领取。

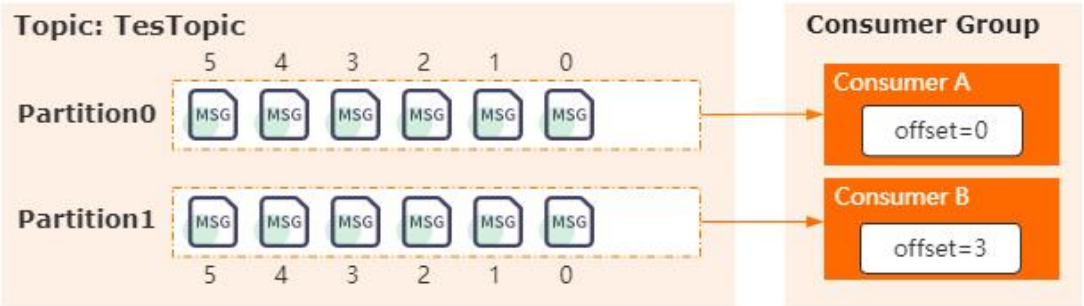
以下是高手的回答

好的，关于这问题，我从几个方面来回答。

首先，（如图）Kafka Broker 上存储的消息，都有一个 Offset 标记。

然后 kafka 的消费者是通过 offSet 标记来维护当前已经消费的数据，

每消费一批数据，Kafka Broker 就会更新 OffSet 的值，避免重复消费。



默认情况下，消息消费完以后，会自动提交 Offset 的值，避免重复消费。

Kafka 消费端的自动提交逻辑有一个默认的 5 秒间隔，也就是说在 5 秒之后的下一次向 Broker 拉取消息的时候提交。

所以在 Consumer 消费的过程中，应用程序被强制 kill 掉或者宕机，可能会导致 Offset 没提交，从而产生重复提交的问题。

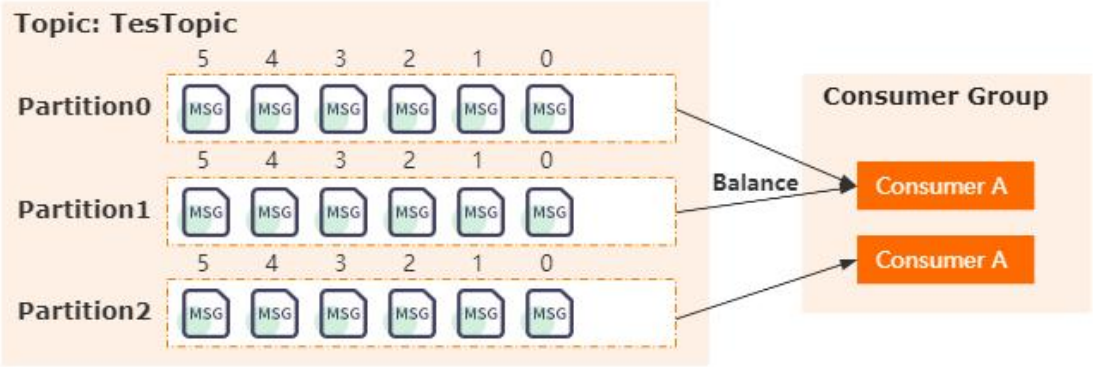
除此之外，还有另外一种情况也会出现重复消费。

（如图）在 Kafka 里面有一个 Partition Balance 机制，就是把多个 Partition 均衡的分配给多个消费者。

Consumer 端会从分配的 Partition 里面去消费消息，如果 Consumer 在默认的 5 分钟内没办法处理完这一批消息。

就会触发 Kafka 的 Rebalance 机制，从而导致 Offset 自动提交失败。

而在重新 Rebalance 之后，Consumer 还是会从之前没提交的 Offset 位置开始消费，也会导致消息重复消费的问题。



基于这样的背景下，我认为解决重复消费消息问题的方法有几个。

4. 提高消费端的处理性能避免触发 Balance，比如可以用异步的方式来处理消息，缩短单个消息消费的市场。或者还可以调整消息处理的超时时间。还可以减少一次性从 Broker 上拉取数据的条数。
5. 可以针对消息生成 md5 然后保存到 mysql 或者 redis 里面，在处理消息之前先去 mysql 或者 redis 里面判断是否已经消费过。这个方案其实就是利用幂等性的思想。

以上就是我对这个问题的理解。

53.147 什么是 ISR，为什么需要引入 ISR

Hi，大家好，我是 Mic。

一个工作 5 年的粉丝，在简历上写精通 Kafka。

结果在面试的时候直接打脸。

面试官问他： “什么是 ISR，为什么需要设计 ISR”

然后他一脸懵逼的看着面试官.

关于这个问题的回答，我整理成了文档，有需要的小伙伴可以在主页加 v 领取。

以下是高手的回答

好的，关于这个问题，我需要从几个方面来回答。

首先，发送到 Kafka Broker 上的消息，最终是以 Partition 的物理形态来存储到磁盘上的。

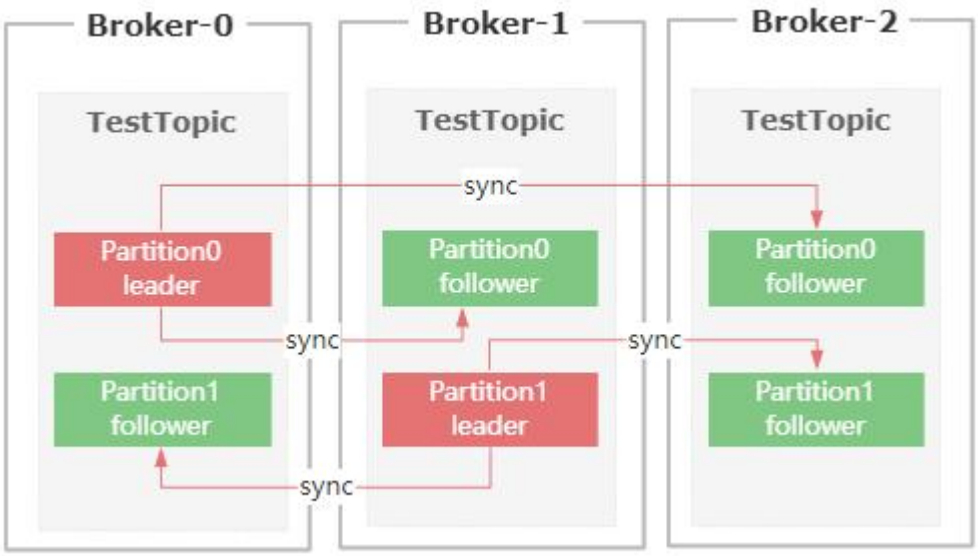
（如图）而 Kafka 为了保证 Parititon 的可靠性，提供了 Paritition 的副本机制，然后在这些 Partition 副本集里面。

存在 Leader Partition 和 Flollower Partition。

生产者发送过来的消息，会先存到 Leader Partition 里面，然后再把消息复制到 Follower Partition，

这样设计的好处就是一旦 Leader Partition 所在的节点挂了，可以重新从剩余的 Partition 副本里面选举出新的 Leader。

然后消费者可以继续从新的 Leader Partition 里面获取未消费的数据。



在 Partition 多副本设计的方案里面，有两个很关键的需求。

- 副本数据的同步

- 新 Leader 的选举

这两个需求都需要涉及到网络通信，Kafka 为了避免网络通信延迟带来的性能问题，以及尽可能的保证新选举出来的 Leader Partition 里面的数据是最新的，所以设计了 ISR 这样一个方案。

ISR 全称是 in-sync replica，它是一个集合列表，里面保存的是和 Leader Parition 节点数据最接近的 Follower Partition

如果某个 Follower Partition 里面的数据落后 Leader 太多，就会被剔除 ISR 列表。

简单来说，ISR 列表里面的节点，同步的数据一定是最新的，所以后续的 Leader 选举，只需要从 ISR 列表里面筛选就行了。

所以，我认为引入 ISR 这个方案的原因有两个

1. 尽可能的保证数据同步的效率，因为同步效率不高的节点都会被踢出 ISR 列表。
2. 避免数据的丢失，因为 ISR 里面的节点数据是和 Leader 副本最接近的。

以上就是我对这个问题的理解。

53.148 RDB 和 AOF 的实现原理、优缺点

Hi，大家好，我是 Mic。

一个工作了 5 年的粉丝私信我，最近面试碰到很多 Redis 相关的问题。

其中一个面试官问他 Redis 里面的持久化机制，没有回答得很好。

希望我帮他系统回答一下。

关于 Redis 里面的 RDB 和 AOF 两种持久化机制的原理和优缺点这个问题。

我把高手的回答整理成了很详细的文档，有需要的小伙伴可以在主页加 V 领取。

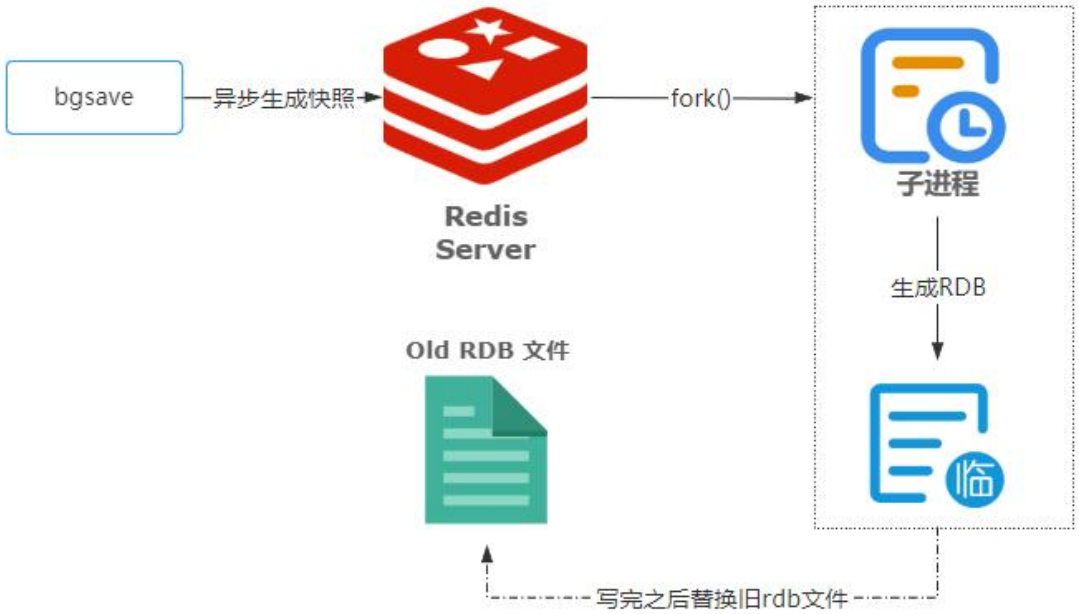
以下是高手的回答

好的，关于这个问题，我从几个点来回答。

首先，Redis 本身是一个基于 Key-Value 结构的内存数据库，为了避免 Redis 故障导致数据丢失的问题，所以提供了 RDB 和 AOF 两种持久化机制。

（如图）RDB 是通过快照的方式来实现持久化的，也就是说会根据快照的触发条件，把内存里面的数据快照写入到磁盘，

以二进制的压缩文件进行存储。



RDB 快照的触发方式有很多，比如

- 执行 bgsave 命令触发异步快照，执行 save 命令触发同步快照，同步快照会阻塞客户端的执行指令。
- 根据 redis.conf 文件里面的配置，自动触发 bgsave
- 主从复制的时候触发

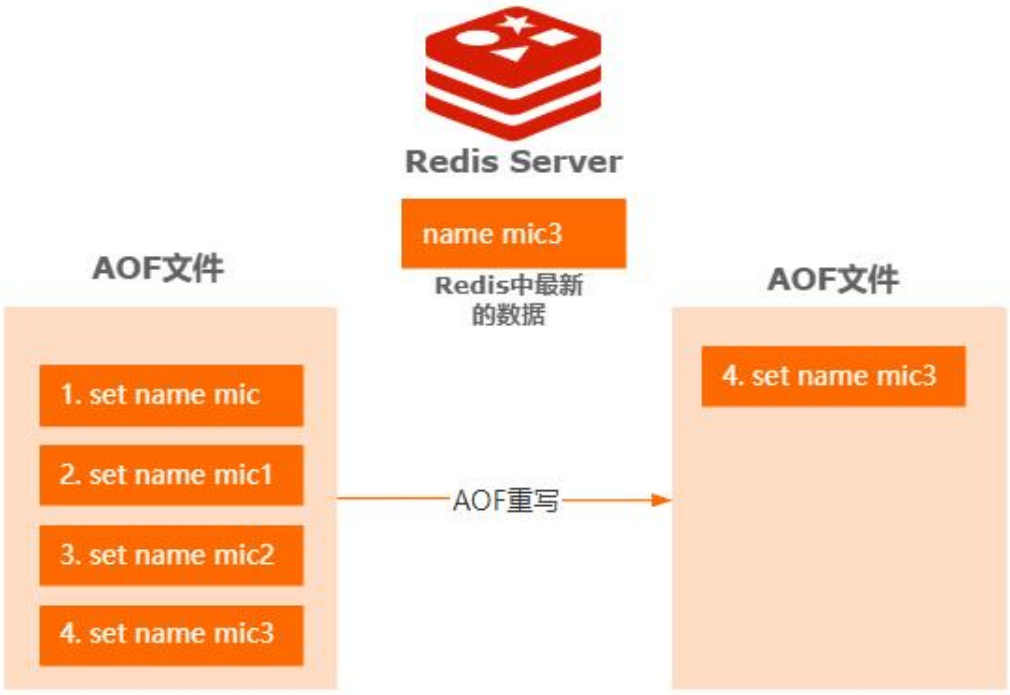
AOF 持久化，它是一种近乎实时的方式，把 Redis Server 执行的事务命令进行追加存储。简单来说，

（如图）就是客户端执行一个数据变更的操作，Redis Server 就会把这个命令追加到 aof 缓冲区的末尾，

然后再把缓冲区的数据写入到磁盘的 AOF 文件里面，至于最终什么时候真正持久化到磁盘，是根据刷盘的策略来决定的。

另外，因为 AOF 这种指令追加的方式，会造成 AOF 文件过大，带来明显的 IO 性能问题，所以 Redis 针对这种情况提供了

AOF 重写机制（如图），也就是说当 AOF 文件的大小达到某个阈值的时候，就会把这个文件里面相同的指令进行压缩。



因此，基于对 RDB 和 AOF 的工作原理的理解，我认为 RDB 和 AOF 的优缺点有两个。

- RDB 是每隔一段时间触发持久化，因此数据安全性低，AOF 可以做到实时持久化，数据安全性较高
- RDB 文件默认采用压缩的方式持久化，AOF 存储的是执行指令，所以 RDB 在数据恢复的时候性能比 AOF 要好

在我看来，所谓优缺点，本质上其实是哪种方案更适合当前的应用场景而已。

以上就是我对这个问题的理解！

53.149 简单说一下你对序列化和反序列化的理解

Hi，大家好，我是 Mic

一个工作 4 年的粉丝，投了很多简历

好不容易接到一个互联网公司的面试邀约。

在面试第一轮就被干掉了，原因是对主流互联网技术理解太浅了。

其中就有一个这样的问题： “简单说一下你对序列化和反序列化的理解”

这个问题的回答我整理成了文档，可以在我主页加 V 领取。

> 以下是高手的回答

好的，关于这个问题，我需要从几个方面来回答。

首先，我认为，之所以需要序列化，核心目的是为了解决网络通信之间的对象传输问题。（如图）也就是说，如何把当前 JVM 进程里面的一个对象，跨网络传输到另外一个 JVM 进程里面。

而序列化，就是把内存里面的对象转化为字节流，以便用来实现存储或者传输。

反序列化，就是根据从文件或者网络上获取到的对象的字节流，根据字节流里面保存的对象描述信息和状态。

重新构建一个新的对象。

其次呢，序列化的前提是保证通信双方对于对象的可识别性，所以很多时候，我们会把对象先转化为通用的解析格式，

比如 json、xml 等。然后再把他们转化为数据流进行网络传输，从而实现跨平台和跨语言的可识别性。

最后，我再补充一下序列化选择。

市面上开源的序列化技术非常多，比如 Json、Xml、Protobuf、Kyro、hessian 等等。

那在实际应用里面，哪种序列化最合适，我认为有几个关键因素。

- 序列化之后的数据大小，因为数据大小会影响传输性能
- 序列化的性能，序列化耗时较长会影响业务的性能
- 是否支持跨平台和跨语言
- 技术的成熟度，越成熟的方案使用的公司越多，也就越稳定。

以上就是我对这个问题的理解！

53.150 什么是守护线程，它有什么特点

Hi，大家好，我是 Mic

一个工作了 3 年的粉丝，在面试的时候遇到一个线程相关问题。

想让我帮他解答一下。

问题是，“什么是守护线程，它有什么特点”

这个问题的高手回答已经整理成了文档，需要的小伙伴可以在主页加 V 领取。

以下是高手的回答

好的。

守护线程，它是一种专门为用户线程提供服务的线程，它的生命周期依赖于用户线程。

只有 JVM 中仍然还存在用户线程正在运行的情况下，守护线程才会有存在的意义。

否则，一旦 JVM 进程结束，那守护线程也会随之结束。

也就是说，守护线程不会阻止 JVM 的退出。但是用户线程会！

守护线程和用户线程的创建方式是完全相同的，我们只需要调用用户线程里面的 setDaemon 方法并且设置成 true，

就表示这个线程是守护线程。

因为守护线程拥有自己结束自己生命的特性，所以它适合用在一些后台的通用服务场景里面。

比如 JVM 里面的垃圾回收线程，就是典型的使用场景。

这个场景的特殊之处在于，当 JVM 进程技术的时候，内存回收线程存在的意义也就不存在了。

所以不能因为正在进行垃圾回收导致 JVM 进程无法技术的问题。

但是守护线程不能用在线程池或者一些 IO 任务的场景里面，因为一旦 JVM 退出之后，守护线程也会直接退出。

就会可能导致任务没有执行完或者资源没有正确释放的问题。

以上就是我对这个问题的理解。

53.151 请谈谈 AQS 是怎么回事儿?

Hi，大家好，我是 Mic。

今年的市场环境是真的很难。很多工作一年的人，面试的难度相当于一个 4 年经验的人。

越是这样，我们越应该强大自己，才能在逆境中获得更多的机会。

今天一个一年经验的粉丝，被问到“AQS 的实现原理”，来找我求助。

这个问题的回答我整理成了文档，大家可以在主页加 V 领取。

下面看看普通人和高手对于这个问题的回答。

以下是高手的回答

好的，关于这个问题我需要从几个方面来回答。

AQS 它是 J.U.C 这个包里面非常核心的一个抽象类，它为多线程访问共享资源提供了一个队列同步器。

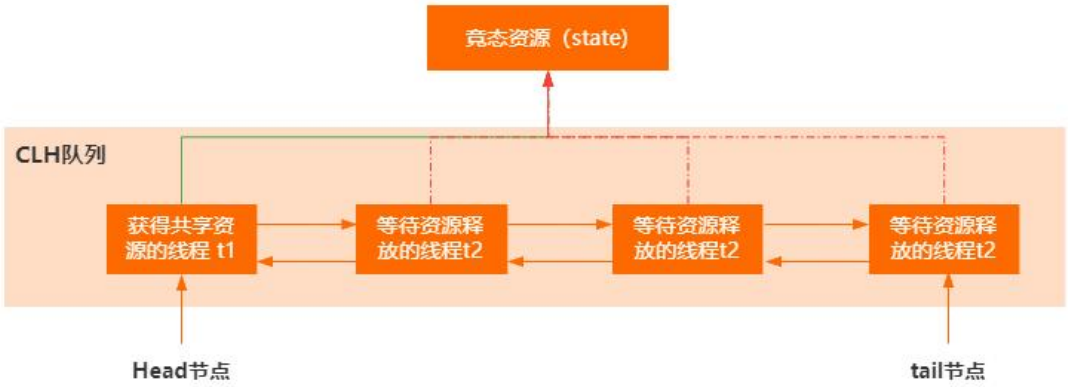
在 J.U.C 这个包里面，很多组件都依赖 AQS 实现线程的同步和唤醒，比如 Lock、Semaphore、CountDownLatch 等等。

（如图），AQS 内部由两个核心部分组成：

- 一个 volatile 修饰的 state 变量，作为一个竞态条件
- 用双向链表结构维护的 FIFO 线程等待队列

它的具体工作原理是，多个线程通过对这个 state 共享变量进行修改来实现竞态条件，竞争失败的线程加入到 FIFO 队列并且阻塞，

抢占到竞态资源的线程释放之后，后续的线程按照 FIFO 顺序实现有序唤醒。



AQS 里面提供了两种资源共享方式，

一种是独占资源，同一个时刻只能有一个线程获得竞态资源。比如 ReentrantLock 就是使用这种方式实现排他锁

另一种是共享资源，同一个时刻，多个线程可以同时获得竞态资源。CountDownLatch 或者 Semaphore 就是使用共享资源的方式，实现同时唤醒多个线程。

以上就是我对这个问题的理解。

53.152 说一说你对 Spring Cloud 的理解

Hi，大家好，我是 Mic

一个工作了 7 年的 Java 粉丝，竟然连这个问题都回答不出来。

难怪被裁员之后，每次面试都被技术问题卡脖子，导致一直找不到工作。

今天他遇到这样一个问题，希望我能给出一个回答建议。

“请你说一下你对 Spring Cloud 的理解”。

这个问题的回答，我整理成了文档，大家可以在我的主页加 V 领取

以下是高手的回答

好的。

Spring Cloud 是 Spring 官方推出来的一套微服务解决方案。

准确来说，我认为 Spring Cloud 其实是对微服务架构里面出现各种技术场景，定义了一套标准规范。

然后在这套标准里面，Spring 集成了 Netflix 公司的 OSS 开源套件，比如

Zuul 实现应用网关、

Eureka 实现服务注册与发现、

Ribbon 实现负载均衡、

Hystrix 实现服务熔断

我们可以使用 Spring Cloud Netflix 这套组件，快速落地微服务架构以及解决微服务治理等一系列问题。

但是随着 Netflix OSS 相关技术组件的闭源和停止维护，所以 Spring 官方也自研了一些组件，

比如 Gateway 实现网关、 LoadBalancer 实现负载均衡。

另外，Alibaba 里面的开源组件也实现了 Spring Cloud 的标准，成为了 Spring Cloud 里面的另外一套微服务解决方案。

包括 Dubbo 做 rpc 通信、Nacos 实现服务注册与发现以及动态配置中心、Sentinel 实现服务限流和服务降级等等。

以上就是我对 Spring Cloud 的理解，另外，我再补充一下，我认为 Spring Cloud 生态的出现有两个很重要的意义。

1. 在 Spring Cloud 出现之前，为了解决微服务架构里面的各种技术问题，需要去集成各种开源框架，因为标准和兼容性问题，所以在实践的时候很麻烦，而 Spring Cloud 统一了这样一个标准。
2. 降低了微服务架构的开发难度，只需要在 Spring Boot 的项目基础上通过 starter 启动依赖集成相关组件就能轻松解决各种问题。

以上就是我对这个问题的理解。

53.153 什么是 SPI，它有什么用?

Hi，大家好，我是 Mic

一个工作了 4 年的粉丝，私信了一个问题。

我看到问题的时候有点惊讶，很多主流框架都用到了这个机制

你竟然不知道?

看来你是凭实力那低薪!

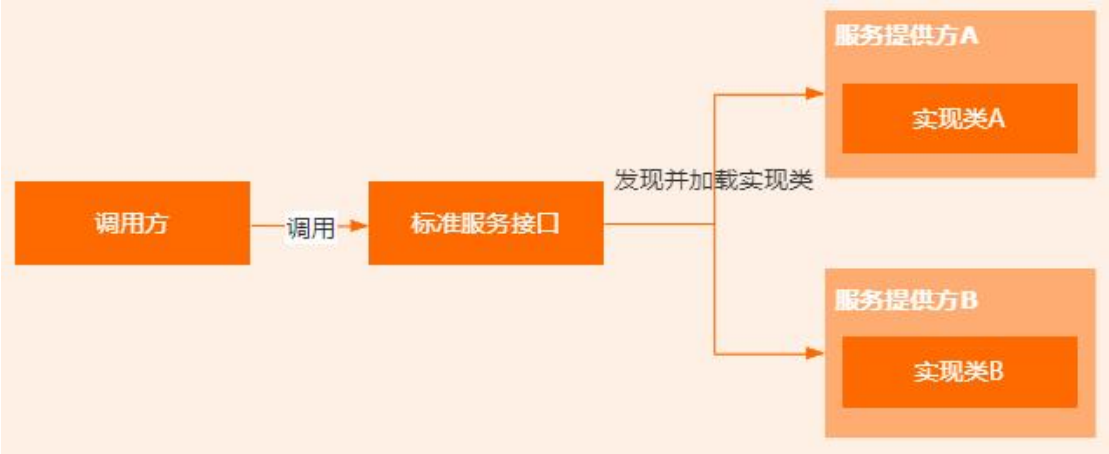
这个问题是： “什么是 SPI，它有什么用”

> 以下是高手的回答

好的

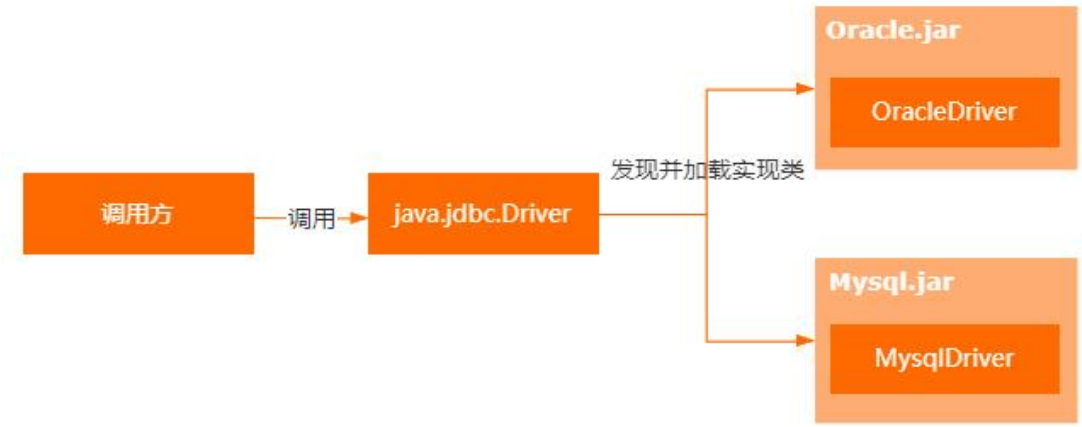
SPI 全称是 Service Provider Interface ，它是 JDK 内置的一种动态扩展点的实现。（如图），简单来说，就是我们可以定义一个标准的接口，然后第三方的库里面可以实现这个接口。

那么，程序在运行的时候，会根据配置信息动态加载第三方实现的类，从而完成功能的动态扩展机制。



在 Java 里面，SPI 机制有一个非常典型的实现案例，就是数据库驱动 java.jdbc.Driver（如图），JDK 里面定义了数据库驱动类 Driver，它是一个接口，JDK 并没有提供实现。具体的实现是由第三方数据库厂商来完成的。

在程序运行的时候，会根据我们声明的驱动类型，来动态加载对应的扩展实现，从而完成数据库的连接。



除此之外，在很多开源框架里面都借鉴了 Java SPI 的思想，提供了自己的 SPI 框架，比如

Dubbo 定义了 ExtensionLoader，实现功能的扩展。

Spring 提供了 SpringFactoriesLoader，实现外部功能的集成。

以上就是我对这个问题的理解！

53.154 请描述一下 Redis 中 AOF 重写的过程

Hi，大家好，我是 Mic

一个工作了 5 年的粉丝私信我。他说他在简历里面写精通 Redis，

结果面试官一直围绕 Redis 来问，后来越问越深，面试官后面回了一句话：“也没有想象中那么深嘛”

今天给大家分享一道年薪 50W 的面试题，“请描述一下 Redis 里面 AOF 的重写过程”

这个问题的回答我整理成了文档，有需要的小伙伴可以在主页加 V 领取

下面看看普通人和高手对这个问题的回答

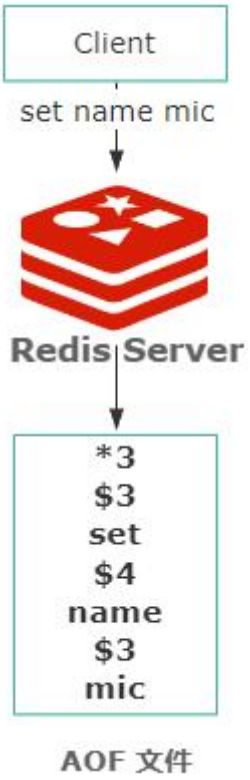
> 以下是高手的回答

好的。

AOF 是 Redis 里面的一种数据持久化方式，它采用了指令追加的方式（如图）。

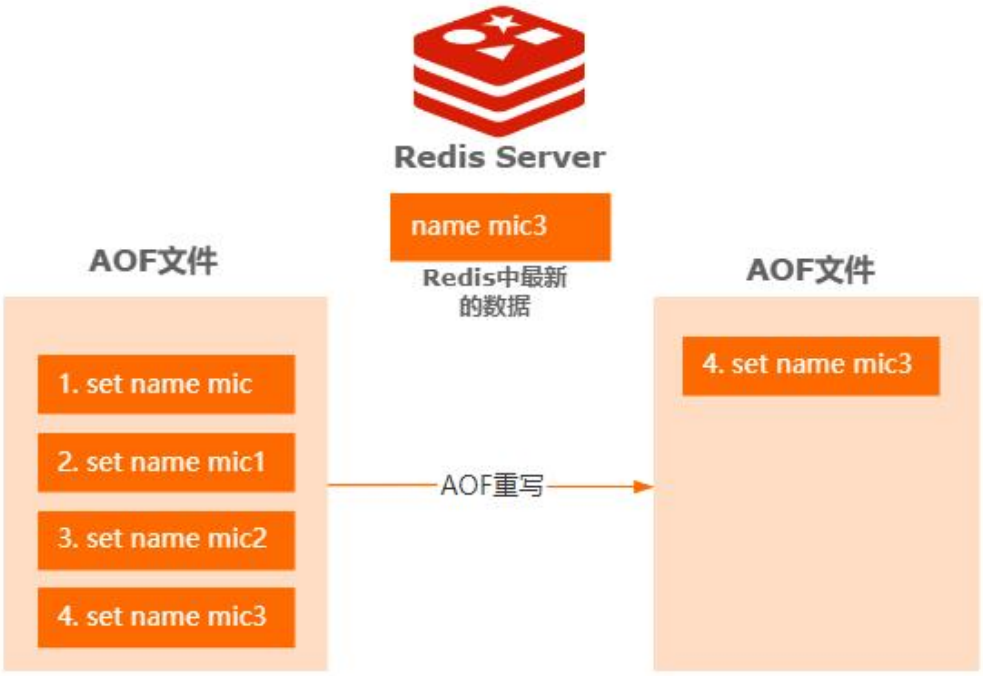
近乎实时的去实现数据指令的持久化，因为 AOF，会把每个数据更改的操作指令，追加存储到 aof 文件里面。

所以很容易导致 AOF 文件出现过大，造成 IO 性能问题。



Redis 为了解决这个问题，设计了 AOF 重写机制，也就是说把 AOF 文件里面相同的指令进行压缩，只保留最新的数据指令。

（如图），简单来说，如果 aof 文件里面存储了某个 key 的多次变更记录，但是实际上，最终在做数据恢复的时候，只需要执行最新的指令操作就行了，历史的数据就没必要存在这个文件里面占空间。



AOF 文件重写的具体过程分为几步：

首先，根据当前 Redis 内存里面的数据，重新构建一个新的 AOF 文件

然后，读取当前 Redis 里面的数据，写入到新的 AOF 文件里面

最后，重写完成以后，用新的 AOF 文件覆盖现有的 AOF 文件

另外，因为 AOF 在重写的过程中需要读取当前内存里面所有的键值数据，再生成对应的一条指令进行保存。

而这个过程是比较耗时的，对业务会产生影响。

所以 Redis 把重写的过程放在一个后台子进程里面来完成，

这样一来，子进程在做重写的时候，主进程依然可以继续处理客户端请求。

最后，为了避免子进程在重写过程中，主进程的数据发生变化，

导致 AOF 文件和 Redis 内存中的数据不一致的问题，Redis 还做了一层优化。

就是子进程在重写的过程中，主进程的数据变更需要追加到 AOF 重写缓冲区里面。

等到 AOF 文件重写完成以后，再把 AOF 重写缓冲区里面的内容追加到新的 AOF 文件里面。

以上就是我对这个问题的理解。

53.155 HashMap 是如何解决 hash 冲突的?

Hi，大家好，我是 Mic。

今天我们来分享一道工作一年左右的面试题。

最近很多粉丝在面试的时候都遇到了这个问题

问题是：“HashMap 是如何解决 Hash 冲突的”？

很多人觉得这个问题很简单，但是我认为高手的回答会更好一点。

文字版整理好了，大家可以在主页去加 v 领取。

下面看看普通人和高手对这个问题的回答。

> 以下是高手的回答

好的，这个问题我需要从几个方面来回答。

首先，HashMap 底层采用了数组的结构来存储数据元素，数组的默认长度是 16，

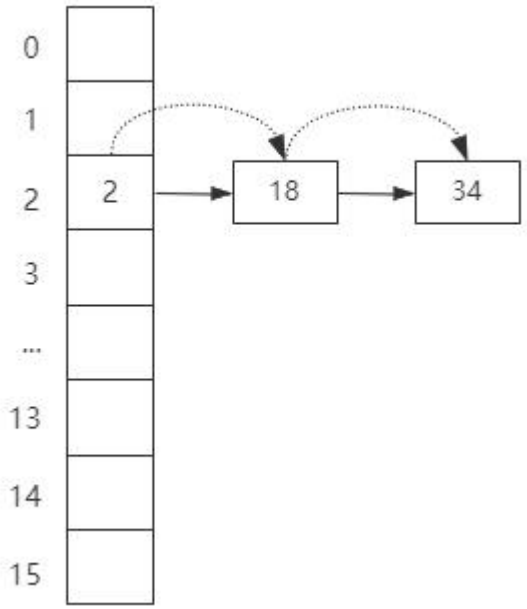
当我们通过 put 方法添加数据的时候，HashMap 根据 Key 的 hash 值进行取模运算。

最终保存到数组的指定位置。

但是这种设计会存在 hash 冲突问题，也就是两个不同 hash 值的 key，最终取模后会落到同一个数组下标。

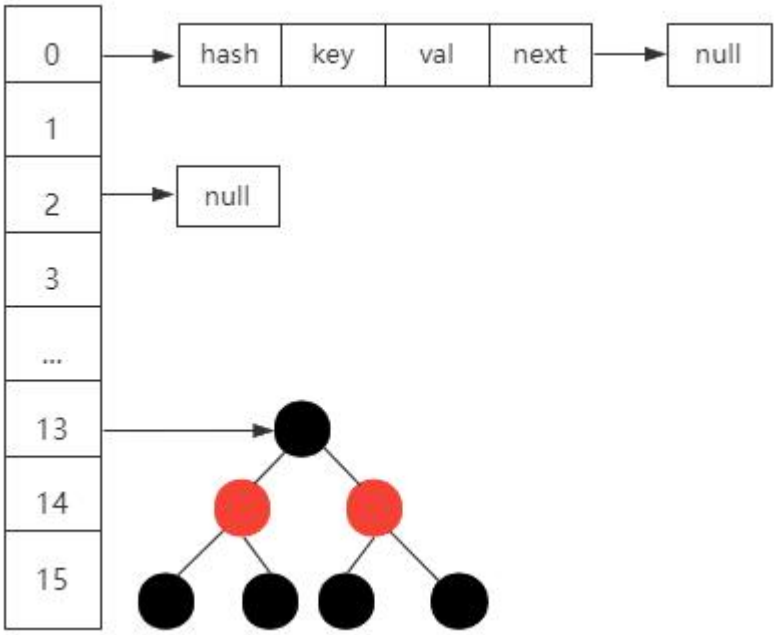
（如图）所以 HashMap 引入了链式寻址法来解决 hash 冲突问题， 对于存在冲突的 key，HashMap 把这些 key 组成一个单向链表。

然后采用尾插法把这个 key 保存到链表的尾部。



另外，为了避免链表过长的问题，当链表长度大于 8 并且数组长度大于等于 64 的时候，HashMap 会把链表转化为红黑树（如图）。

从而减少链表数据查询的时间复杂度问题，提升查询性能。



最后，我再补充一下，解决 hash 冲突问题的方法有很多，比如

- 再 hash 法，就是如果某个 hash 函数产生了冲突，再用另外一个 hash 进行计算，比如布隆过滤器就采用了这种方法。
- 开放寻址法，就是直接从冲突的数组位置往下寻找一个空的数组下标进行数据存储，这个在 ThreadLocal 里面有使用到。
- 建立公共溢出区，也就是把存在冲突的 key 统一放在一个公共溢出区里面。

以上就是我对这个问题的理解。

53.145 ReentrantLock 是如何实现锁公平和非公平性的 ？

一个工作了 3 年的程序员竟然连这个问题都回答不上?

Hi，大家好，我是 Mic。

今天分享一道 3 年经验，频率较高的面试题。

“ReentrantLock 是如何实现锁的公平和非公平性的”？

这个问题的回答我整理成了文档，有需要的小伙伴可以在主页加 V 领取。

以下是高手的回答

好的。

我先解释一下个公平和非公平的概念。

公平，指的是竞争锁资源的线程，严格按照请求顺序来分配锁。

非公平，表示竞争锁资源的线程，允许插队来抢占锁资源。

ReentrantLock 默认采用了非公平锁的策略来实现锁的竞争逻辑。

（如图）其次，ReentrantLock 内部使用了 AQS 来实现锁资源的竞争，

没有竞争到锁资源的线程，会加入到 AQS 的同步队列里面，这个队列是一个 FIFO 的双向链表。

在这样的一个背景下，公平锁的实现方式就是，线程在竞争锁资源的时候判断 AQS 同步队列里面有没有等待的线程。

如果有，就加入到队列的尾部等待。

而非公平锁的实现方式，就是不管队列里面有没有线程等待，它都会先去尝试抢占锁资源，如果抢不到，再加入到

AQS 同步队列等待。

ReentrantLock 和 Synchronized 默认都是非公平锁的策略，之所以要这么设计，我认为还是考虑到了性能这个方面的原因。

因为一个竞争锁的线程如果按照公平的策略去阻塞等待，同时 AQS 再把等待队列里面的线程唤醒，这里会涉及到内核态

的切换，对性能的影响比较大。

如果是非公平策略，当前线程正好在上一个线程释放锁的临界点抢占到了锁，就意味着这个线程不需要切换到内核态，

虽然对原本应该要被唤醒的线程不公平，但是提升了锁竞争的性能。

以上就是我对这个问题的理解。

53.147 Zookeeper 如何实现 Leader 选举

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

一个工作了 7 年的粉丝，最近去面试遇到 Zookeeper 里面的一个问题。

因为平时很少研究，所以面试的时候只能一个劲的说：不知道。

他觉得很尴尬，于是来问我，：“Zookeeper 是如何实现 Leader 选举的”。

我说，这个问题已经用文档整理好可，你直接在我主页加 V 领取就行。

以下是高手的回答

好的。

（如图）首先，Zookeeper 集群节点由三种角色组成，分别是

- Leader，负责所有事务请求的处理，以及过半提交的投票发起和决策。
- Follower，负责接收客户端的非事务请求，而事务请求会转发给 Leader 节点来处理，

另外，Follower 节点还会参与 Leader 选举的投票。

- Observer，负责接收客户端的非事务请求，事务请求会转发给 Leader 节点来处理，

另外 Observer 节点不参与任何投票，

只是为了扩展 Zookeeper 集群来分担读操作的压力。

其次，Zookeeper 集群是一种典型的中心化架构，也就是会有一个 Leader 作为决策节点，

专门负责事务请求的处理和数据的同步。

这种架构的好处是可以减少集群架构里面数据同步的复杂度，集群管理会更加简单和稳定。

但是，会带来 Leader 选举的一个问题，也就是说，（如图）如果 Leader 节点宕机了，为了保证集群继续提供可靠的服务，

Zookeeper 需要从剩下的 Follower 节点里面去选举一个新的节点作为 Leader，也就是所谓的 Leader 选举!

[备注： 下面这个图，做成动画形式，设计的时候找 Mic 讨论一下]

（如图）具体的实现是，

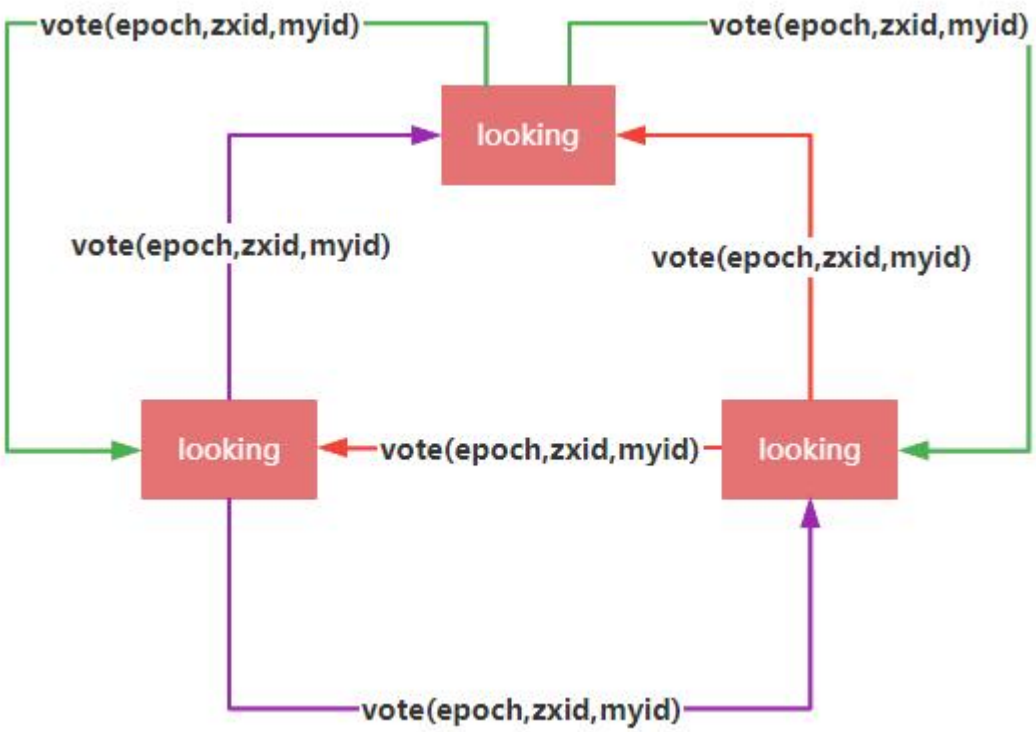
每一个节点都会向集群里面的其他节点发送一个票据 Vote，这个票据包括三个属性。

- epoch， 逻辑时钟，用来表示当前票据是否过期。
- zxid，事务 id，表示当前节点最新存储的数据的事务编号
- myid，服务器 id，在 myid 文件里面填写的数字。

每个节点都会选自己当 Leader，所以第一次投票的时候携带的是当前节点的信息。

接下来每个节点用收到的票据和自己节点的票据做比较，根据 epoch、zxid、myid 的顺序逐一比较，

以值最大的一方获胜。比较结束以后这个节点下次再投票的时候，发送的投票请求就是获胜的 Vote 信息。



然后通过多轮投票以后，每个节点都会去统计当前达成一致的票据，以少数服从多数的方式，最终

获得票据最多的节点成为 Leader。

以上就是我对这个问题的理解。

最后我再补充一下，选择 epoch/zxid/myid 作为投票评判依据的原因，我是这么理解的。

epoch ，因为网络通信延迟的可能性，有可能在新一轮的投票里面收到上一轮投票的票据，这种数据应该丢弃，否则会影响投票的结果和效率。

zxid， zxid 越大，说明这个节点的数据越接近 leader，所以用 zxid 做判断条件是为了避免数据丢失的问题

myid， 服务器 id，这个是避免投票时间过长，直接用 myid 最大值作为快速终结投票的属性。

嗯！

53.148 说一下你对 CompletableFuture 的理解

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

一个工作了 4 年的粉丝，很兴奋的和我说，他拿到了字节跳动的 offer

评级是 2-1 ，相当于阿里的 p6。

他说多亏了面试之前每天刷我的面试短视频。

这种正向反馈，让我觉得做这个事情很有价值。

好的，今天给大家分享一道并发编程的面试题。

“请你说一下你对 CompletableFuture 的理解” 。

关于这个问题的高手部分的回答，我整理成了文档，可以在我的主页加 V 领取。

以下是高手的回答

好的，面试官。

CompletableFuture 是 JDK1.8 里面引入的一个基于事件驱动的异步回调类。

简单来说，就是当使用异步线程去执行一个任务的时候，我们希望在任务结束以后触发一个后续的动作。

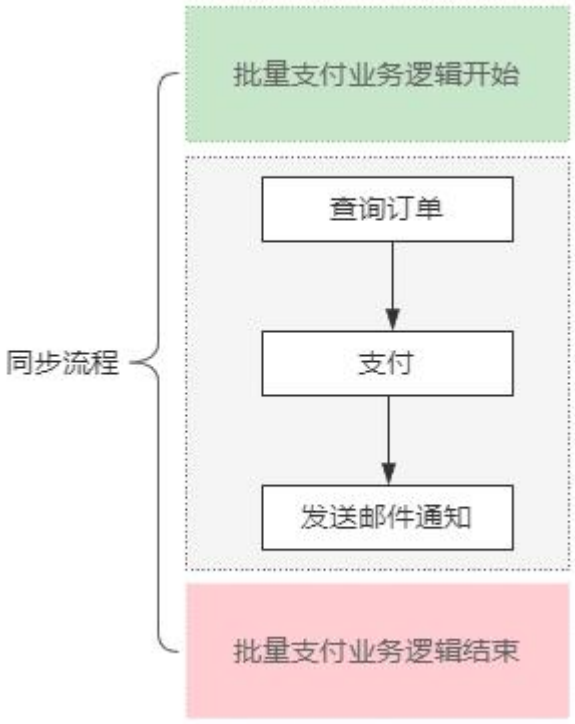
而 CompletableFuture 就可以实现这个功能。

（如图），举个简单的例子，比如在一个批量支付的业务逻辑里面，

涉及到查询订单、支付、发送邮件通知这三个逻辑。

这三个逻辑是按照顺序同步去实现的，也就是先查询到订单以后，再针对这个订单发起支付，支付成功以后再发送邮件通知。

而这种设计方式导致这个方法的执行性能比较慢。



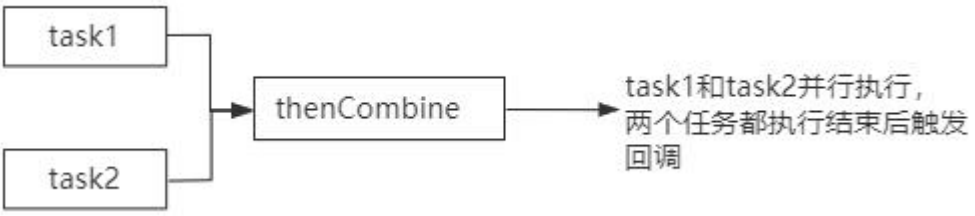
所以，这里可以直接使用 CompletableFuture，（如图），也就是说把查询订单的逻辑放在一个异步线程池里面去处理。

然后基于 CompletableFuture 的事件回调机制的特性，可以配置查询订单结束后自动触发支付，支付结束后自动触发邮件通知。

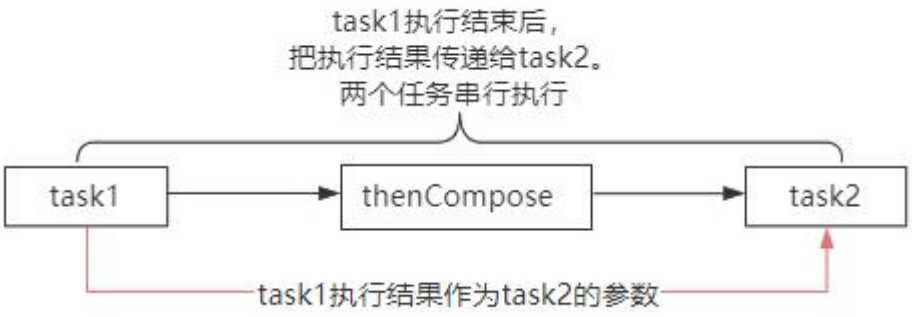
从而极大的提升这个这个业务场景的处理性能！

CompletableFuture 提供了 5 种不同的方式，把多个异步任务组成一个具有先后关系的处理链，然后基于事件驱动任务链的执行。

- 第一种，thenCombine（如图），把两个任务组合在一起，当两个任务都执行结束以后触发事件回调。



- 第二种，thenCompose（如图），把两个任务组合在一起，这两个任务串行执行，也就是第一个任务执行完以后自动触发执行第二个任务。



- 第三种，thenAccept(如图)，第一个任务执行结束后触发第二个任务，

并且第一个任务的执行结果作为第二个任务的参数，这个方法是纯粹接受上一个任务的结果，不返回新的计算值。

- 第四种，thenApply（如图），和 thenAccept 一样，但是它有返回值。

- 第五种，thenRun（如图），就是第一个任务执行完成后触发执行一个实现了 Runnable 接口的任务。

最后，我认为，CompletableFuture 弥补了原本 Future 的不足，使得程序可以在非阻塞的状态下完成异步的回调机制。

以上就是我对这个问题的理解。

53. 149 Spring 里面的事务和分布式事务的使用如何区分，以及这两个事务之间有什么关联?

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

昨天一个粉丝私信我一个问题。

他去面试的时候遇到一个这样的问题。

“Spring 里面的事务和分布式事务的使用如何区分，以及这两个事务之间有什么关联”

我没有想到，还有人回答不了这个问题。

所以，今天我们就来说一下这个面试题。

这个问题高手的回答已经整理成了文档，大家可以在主页加 V 领取。

以下是高手的回答

好的，面试官。

首先， 在 Spring 里面并没有提供事务，它只是提供了对数据库事务管理的封装。

通过声明式的事务配置，使得开发人员可以从一些复杂的事务处理中得到解脱，

我们不再需要关心连接的获取、连接的关闭、事务提交、事务回滚这些操作。

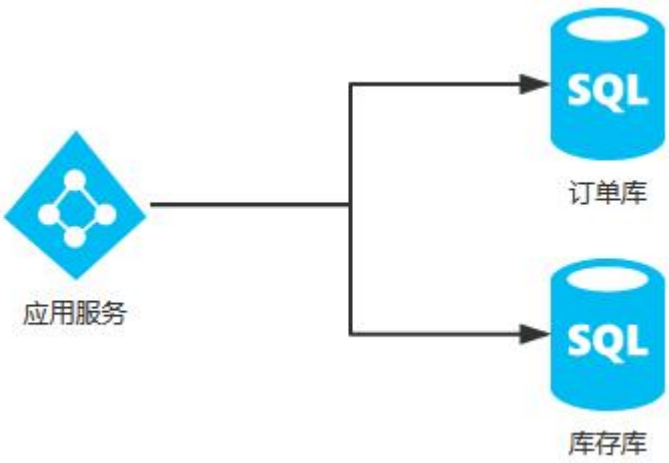
更加聚焦在业务开发层面。

所以，Spring 里面的事务，本质上就是数据库层面的事务，

这种事务的管理，主要是针对单个数据库里面多个数据表操作的，去满足事务的 ACID 特性。

（如图）分布式事务，是解决多个数据库的事务操作的数据一致性问题，

传统的关系型数据库不支持跨库事务的操作，所以需要引入分布式事务的解决方案。



而 Spring 并没有提供分布式事务场景的支持，所以 Spring 事务和分布式事务在使用上并没有直接的关联性。

但是我们可以使用一些主流的事务解决框架，比如 Seata，集成到 Spring 生态里面，去解决分布式事务的问题。

以上就是我对这个问题的理解！

53.150 Cookie 和 Session 的区别

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

今天分享一道比较基础的面试题。

但是我可以保证很多人不一定回答得很好。

具体问题是：Cookie 和 Session 的区别。

这个问题的高手回答已经整理成了文档，可以在我的主页加 V 领取

下面看看普通人和高手对这个问题的回答。

> 以下是高手的回答

好的，面试官。

我先解释一下 Cookie，它是客户端浏览器用来保存服务端数据的一种机制。

当通过浏览器进行网页访问的时候，服务器可以把某一些状态数据以 key-value 的方式写入到 Cookie 里面存储到客户端浏览器。

然后客户端下一次再访问服务器的时候，就可以携带这些状态数据发送到服务器端，服务端

可以根据 Cookie 里面携带的内容来识别使用者。

Session 表示一个会话，它是属于服务器端的容器对象，默认情况下，针对每一个浏览器的请求。

Servlet 容器都会分配一个 Session。

Session 本质上是一个 ConcurrentHashMap，可以存储当前会话产生的一些状态数据。

我们都知道，Http 协议本身是一个无状态协议，也就是服务器并不知道客户端发送过来的多次请求是属于同一个用户。

所以 Session 是用来弥补 Http 无状态的不足，简单来说，服务器端可以利用 session 来存储客户端在同一个会话里面的多次
请求记录。

基于服务端的 session 存储机制，再结合客户端的 Cookie 机制，就可以实现有状态的 Http 协议。

（如图）具体的工作原理是：

客户端第一次访问服务端的时候，服务端会针对这次请求创建一个会话，并生成一个唯一的 sessionId 来标注这个会话。

然后服务端把这个 sessionid 写入到客户端浏览器的 cookie 里面，用来实现客户端状态的保存。

在后续的请求里面，每次都会携带 sessionid，服务器端就可以根据这个 sessionid 来识别当前的会话状态。

所以，总的来看，Cookie 是客户端的存储机制，Session 是服务端的存储机制。

这两者结合使用，来实现会话状态的存储，以上就是我对这个问题的理解！

53.151 线程状态，BLOCKED 和 WAITING 有什么区别

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

一个在北京工作了 3 年的粉丝，在一个公司待了 3 年没有跳槽。

而且他在现在公司里面担任一个核心开发，自认为能力还不错，想出去找一份高薪工作。

结果去面试的时候被一道简单的问题难住了，面试官问他：“线程状态 BLOCKED 和 WAITING 有什么区别”！

因为平时主要是做业务开发，所以线程这方面的研究很少，最后很遗憾没有通过面试。

这个问题的高手回答已经整理成了文档，可以在我主页加 V 领取。

下面看看普通人和高手对这个问题的回答。

以下是高手的回答

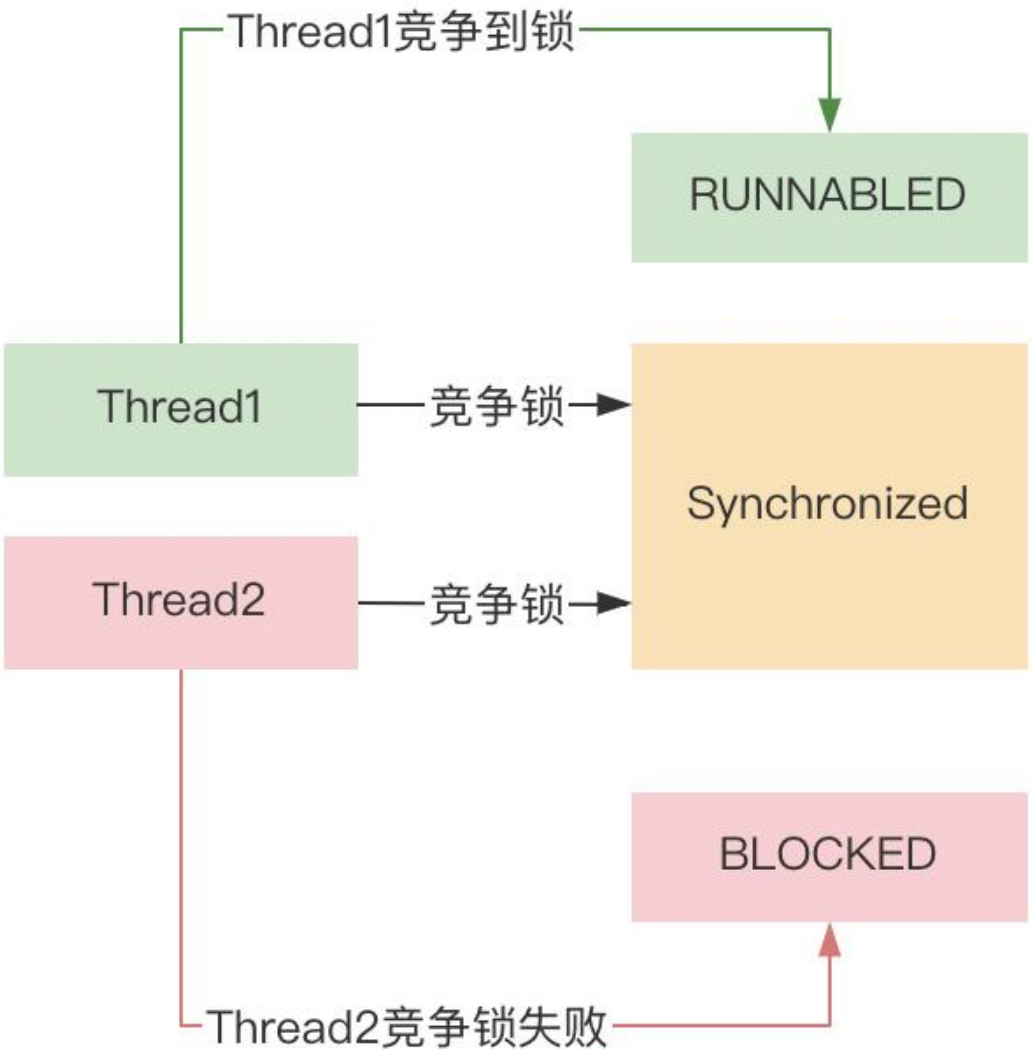
好的，面试官。

BLOCKED 和 WAITING 都是属于线程的阻塞等待状态。

BLOCKED 状态是指线程在等待监视器锁的时候的阻塞状态。

（如图）也就是在多个线程去竞争 Synchronized 同步锁的时候，没有竞争到锁资源的线程，会被阻塞等待，这个时候线程状态就是 BLOCKED。

在线程的整个生命周期里面，只有 Synchronized 同步锁等待才会存在这个状态。



WAITING 状态，表示线程的等待状态，在这种状态下，线程需要等待某个线程的特定操作才会被唤醒。我们可以使用 Object.wait()、Object.join()、LockSupport.park()这些方法

使得线程进入到 WAITING 状态，在这个状态下，必须要等待特定的方法来唤醒，

比如 Object.notify 方法可以唤醒 Object.wait()方法阻塞的线程

LockSupport.unpark()可以唤醒 LockSupport.park()方法阻塞的线程。

所以，在我看来，BLOCKED 和 WAITING 两个状态最大的区别有两个：

- BLOCKED 是锁竞争失败后被被动触发的状态，WAITING 是人为的主动触发的状态
- BLCKED 的唤醒时自动触发的，而 WAITING 状态是必须要通过特定的方法来主动唤醒

以上就是我对这个问题的理解。

53.152 Kafka 如何保证消息消费的顺序性?

最近很多同学去面试，都被问到“MQ 如何保证消费顺序性”这样一个问题

很多人都没回到上来，特别是面试官在面试的时候，还会再补充一句：“还有没有其他方案”。

然后求职者一脸懵逼，实在不知道该怎么回答。

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

今天我们来分析一道 这样的面试题，“MQ 是如何保证消费顺序性”

这个问题的回答已经整理成了文档，可以在主页加 V 领取。

以下是高手的回答

好的，这个问题我从两个方面来回答。

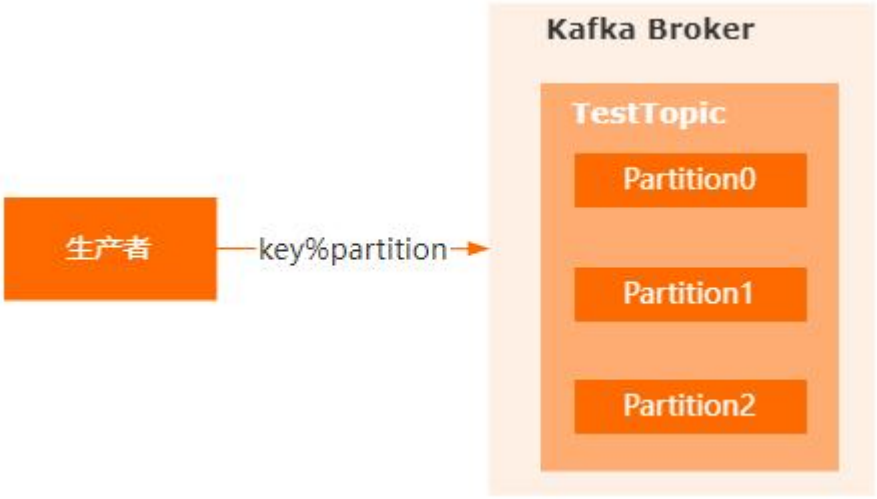
[这两个直接显示在屏幕，不需要念]

- kafka 为什么会存在无序消费
- kafka 如何保证有序消费

首先，在 kafka 的架构里面，用到了 Partition 分区机制来实现消息的物理存储 (如图)，在同一个 topic 下面，可以维护多个 partition 来实现消息的分片。

生产者在发送消息的时候，会根据消息的 key 进行取模（如图），来决定把当前消息存储到哪个 partition 里面。

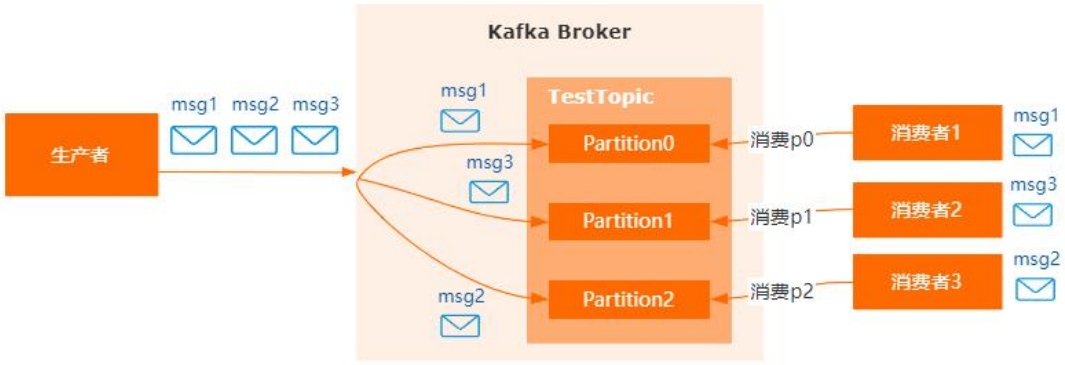
并且消息是按照先后顺序有序存储到 partition 里面的。



在这种情况下，（如图），假设有一个 topic 存在三个 partition，而消息正好被路由到三个独立的 partition 里面。

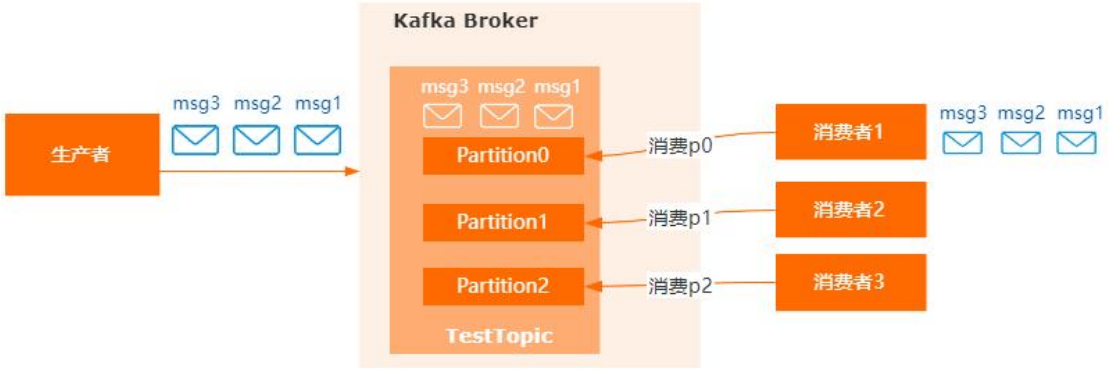
然后消费端有三个消费者通过 balance 机制分别指派了对应消费分区。因为消费者是完全独立的网络节点，

所有可能会出现，消息的消费顺序不是按照发送顺序来实现的，从而导致乱序的问题。



针对这个问题，一般的解决办法就是自定义消息分区路由的算法，然后把指定的 key 都发送到同一个 Partition 里面。（如图）

接着指定一个消费者专门来消费某个分区的数据，这样就能保证消息的顺序消费了。



另外，有些设计方案里面，在消费端会采用异步线程的方式来消费数据来提高消息的处理效率，那这种情况下，

因为每个线程的消息处理效率是不同的，所以即便是采用单个分区的存储和消费也可能会出现无序问题，

针对这个问题的解决办法就是在消费者这边使用一个阻塞队列，把获取到的消息先保存到阻塞队列里面，然后异步线程从阻塞队列里面去获取消息来消费。

以上就是我对这个问题的理解。

53.153 Mysql 事务的实现原理

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

今天分享的面试题很有意思，去大厂面试的时候，百分之 90 的可能性会问到。

但是真正能够完整回答出来的同学缺很少。

最近一个工作了 11 年的粉丝去面试，就被面试到这个问题。

问题是： Mysql 事务的底层实现原理。

这个问题的高手回答已经整理成了文档，大家可以在主页加 V 领取。

下面看看普通人和高手对这个问题的回答

以下是高手的回答

好的，面试官

Mysql 里面的事务，满足 ACID 特性，所以在我看来，Mysql 的事务实现原理，就是 InnoDB 是如何保证 ACID 特性的。

首先，A 表示 Atomic 原子性，也就是需要保证多个 DML 操作是原子的，要么都成功，要么都失败。

那么，失败就意味着要对原本执行成功的数据进行回滚，所以 InnoDB 设计了一个 UNDO_LOG 表，在事务执行的过程中，

把修改之前的数据快照保存到UNDO_LOG里面，一旦出现错误，就直接从UNDO_LOG里面读取数据执行反向操作就行了。

其次，C 表示一致性，表示数据的完整性约束没有被破坏，这个更多是依赖于业务层面的保证，数据库本身也提供了一些，比如主键的唯一余数，字段长度和类型的保证等等。

接着，I 表示事物的隔离性，也就是多个并行事务对同一个数据进行操作的时候，如何避免多个事务的干扰导致数据混乱的问题。

而 InnoDB 实现了 SQL92 的标准，提供了四种隔离级别的实现。分别是：

RU（未提交读）

RC（已提交读）

RR（可重复读）

Serializable（串行化）

InnoDB 默认的隔离级别是 RR（可重复读），然后使用了 MVCC 机制解决了脏读和不可重复读的问题，然后使用了行锁/表锁的方式解决了幻读的问题。

最后一个是 D，表示持久性，也就是只要事务提交成功，那对于这个数据的结果的影响一定是永久性的。

不能因为宕机或者其他原因导致数据变更失效。

理论上来说，事务提交之后直接把数据持久化到磁盘就行了，但是因为随机磁盘 IO 的效率确实很低，所以 InnoDB 设计了

Buffer Pool 缓冲区来优化，也就是数据发生变更的时候先更新内存缓冲区，然后在合适的时机再持久化到磁盘。

那在持久化这个过程中，如果数据库宕机，就会导致数据丢失，也就无法满足持久性了。

所以 InnoDB 引入了 Redo_LOG 文件，这个文件存储了数据被修改之后的值，当我们通过事务对数据进行变更操作的时候，除了修改内存缓冲区里面的数据以外，还会把本次修改的值追加到 REDO_LOG 里面。

当提交事务的时候，直接把 REDO_LOG 日志刷到磁盘上持久化，一旦数据库出现宕机，在 Mysql 重启在以后可以直接用 REDO_LOG 里面保存的重写日志读取出来，再执行一遍从而保证持久性。

因此，在我看来，事务的实现原理的核心本质就是如何满足 ACID 的，在 InnDB 里面用到了 MVCC、行锁表锁、UNDO_LOG、REDO_LOG 等机制来保证。

以上就是我对这个问题的理解！

53.154 String、StringBuffer、StringBuilder 区别

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

今天分享一个非常基础但是有点深度的面试题。

记得 10 多年前我去找工作的时候，经常会碰到这个问题。

现在这一类的问题，考察的是应届生会比较多一点吧。

问题是：　“String、StringBuffer、StringBuilder 的区别”

这个问题的回答已经整理成了文档，可以在我的主页加 V 领取。

下面来看看普通人和高手对这个问题的回答

> 以下是高手的回答

嗯，好的，面试官。

关于 String、StringBuffer、StringBuilder 的区别，我想从四个角度来说明。

第一个，可变性，

String 内部的 value 值是 final 修饰的，所以它是不可变类。所以每次修改 String 的值，都会产生一个新的对象。

StringBuffer 和 StringBuilder 是可变类，字符串的变更不会产生新的对象。

第二个，线程安全性，

String 是不可变类，所以它是线程安全的。

StringBuffer 是线程安全的，因为它每个操作方法都加了 synchronized 同步关键字。

StringBuilder 不是线程安全的。

所以在多线程环境下对字符串进行操作，应该使用 StringBuffer，否则使用 StringBuilder

第三个，性能方面。

String 的性能是最的低的，因为不可变意味着在做字符串拼接和修改的时候，需要重新创建新的对象以及分配内存。

其次是 StringBuffer 要比 String 性能高，因为它的可变性使得字符串可以直接被修改

最后是 StringBuilder，它比 StringBuffer 的性能高，因为 StringBuffer 加了同步锁。

第四个，存储方面。

String 存储在字符串常量池里面

StringBuffer 和 StringBuilder 存储在堆内存空间。

以上就是我对这个问题的理解！

最后再补充一下， StringBuilder 和 StringBuffer 都是派生自 AbstractStringBuilder 这个抽象类。

53.155 说说你对一致性 Hash 算法的理解

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

最近一个工作 4 年的粉丝，去美团面试，被面试官问到一个一致性 hash 算法的问题他没回答上来，然后跑过来找我希望我能帮他分析一下这个问题。

具体问题是： “说说你对一致性 Hash 算法的理解”

> 以下是高手的回答

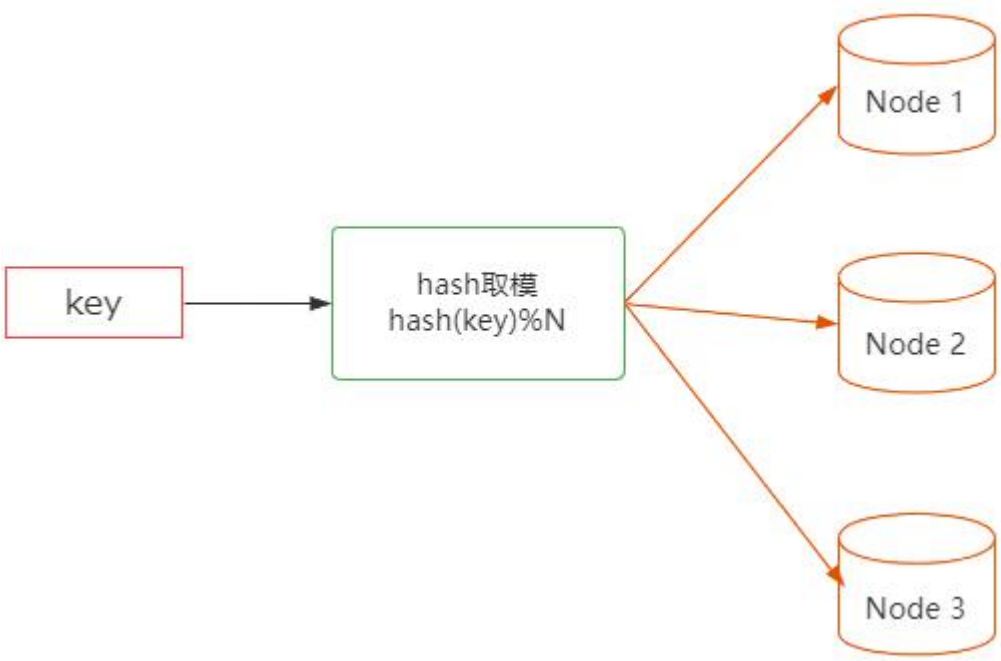
好的，面试官。

一致性 hash，是一种比较特殊的 hash 算法，它的核心思想是解决在分布式环境下，hash 表中可能存在的动态扩容和缩容的问题。

> 为什么会出现一致性 hash（直接在面试官的屏幕上提示就行）

一般情况下，我们会使用 hash 表的方式以 key-value 的方式来存储数据，但是当数据量比较大的时候，我们就会把数据存储

到多个节点上，（如图）然后通过 hash 取模的方法来决定当前 key 存储到哪个节点上。



这种方式有一个非常明显的问题，就是当存储节点增加或者减少的时候，原本的映射关系就会发生变化。

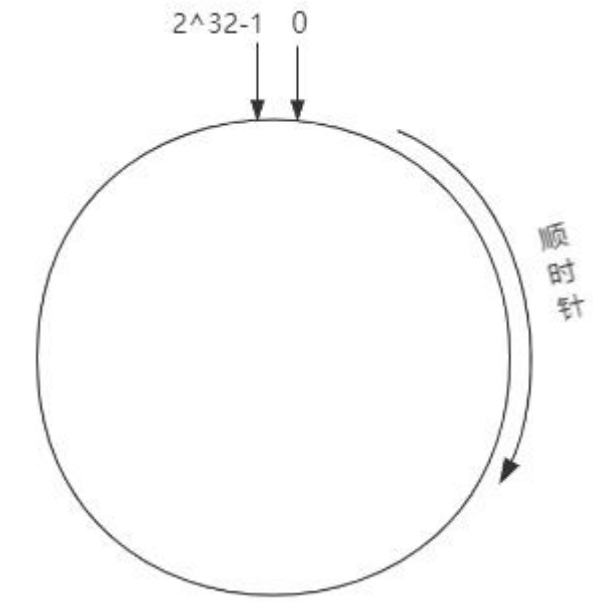
也就是需要对所有数据按照新的节点数量重新映射一遍，这个涉及到大量的数据迁移和重新映射，迁移代价很大。

> 一致性 hash 的工作原理（直接在面试官的屏幕上提示就行）

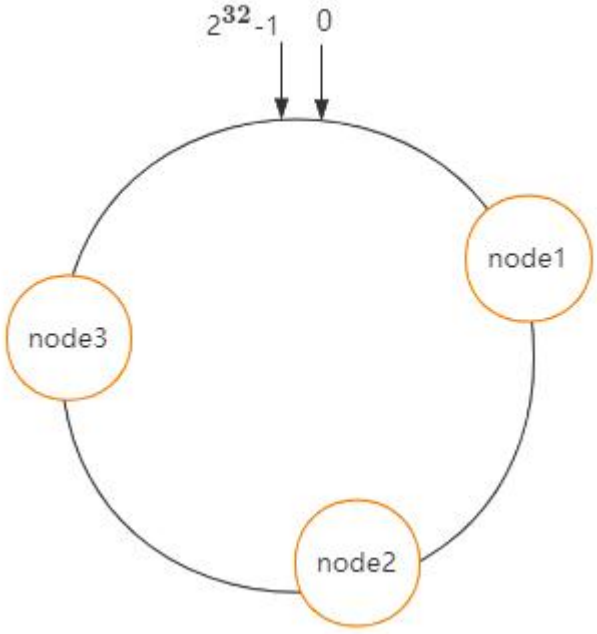
而一致性 hash 就是用来优化这种动态变化场景的算法，它的具体工作原理也很简单。

首先，一致性 Hash 是通过一个 Hash 环的数据结构来实现的，（如图），这个环的起点是 0，终点是 2^32-1。

也就是这个环的数据分布范围是[0,2^32-1]。

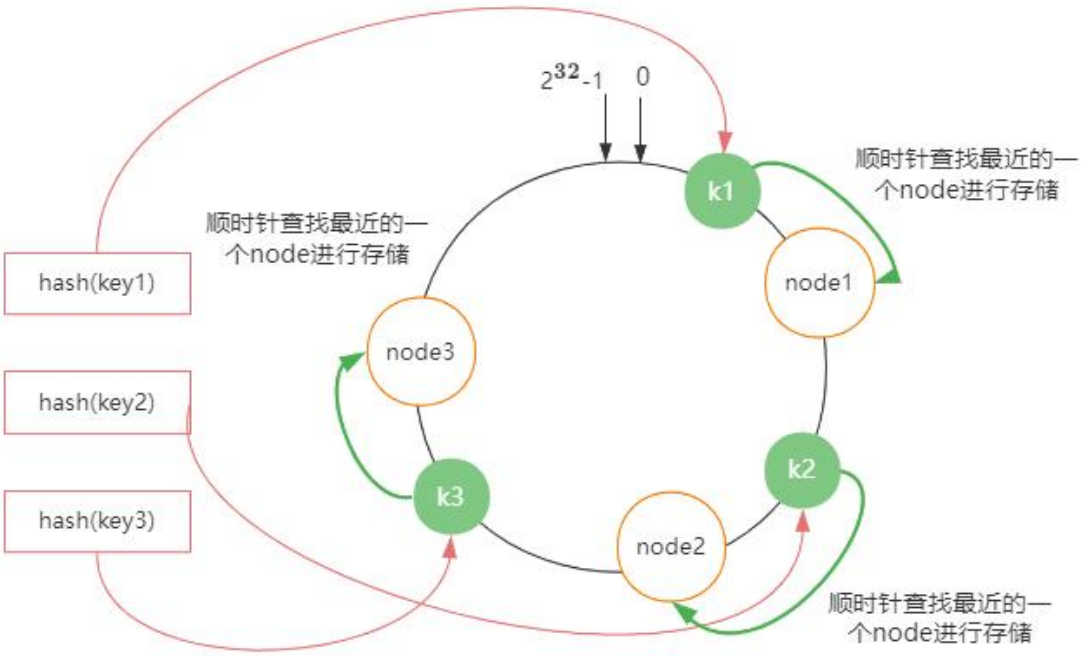


（如图）然后我们把存储节点的 ip 地址作为 key 进行 hash 之后，会在 Hash 环上确定一个位置。



接下来，（如图）就是把需要存储的目标 key 使用 hash 算法计算后得到一个 hash 值，同样也会落到 hash 环的某个位置上。

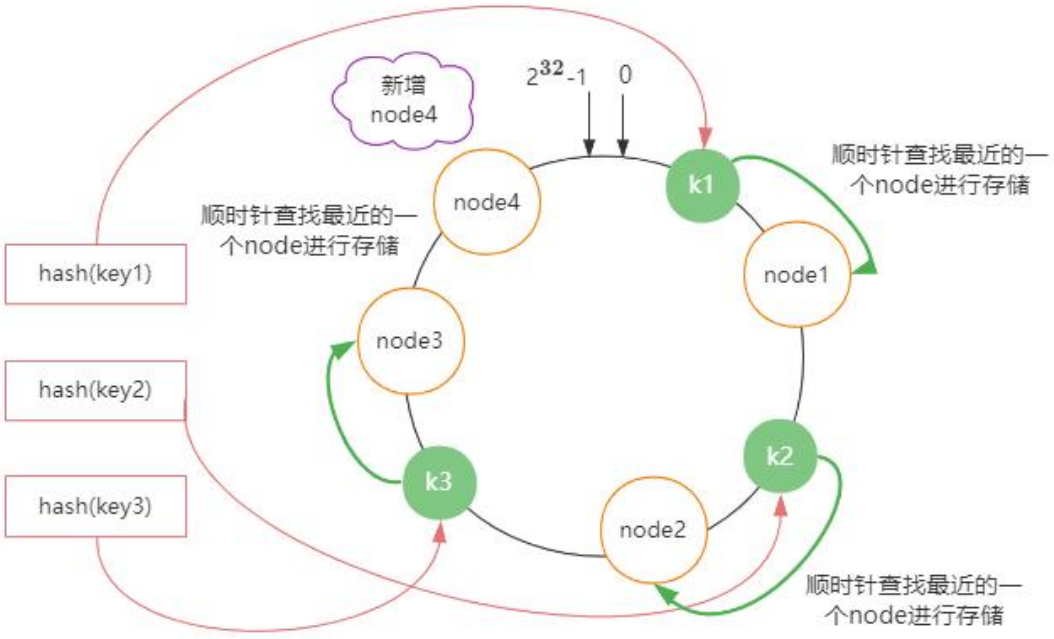
然后这个目标 key 会按照顺时针的方向找到离自己最近的一个节点进行数据存储。



> 为什么一致性 Hash 比普通 hash 算法好（字幕提示）

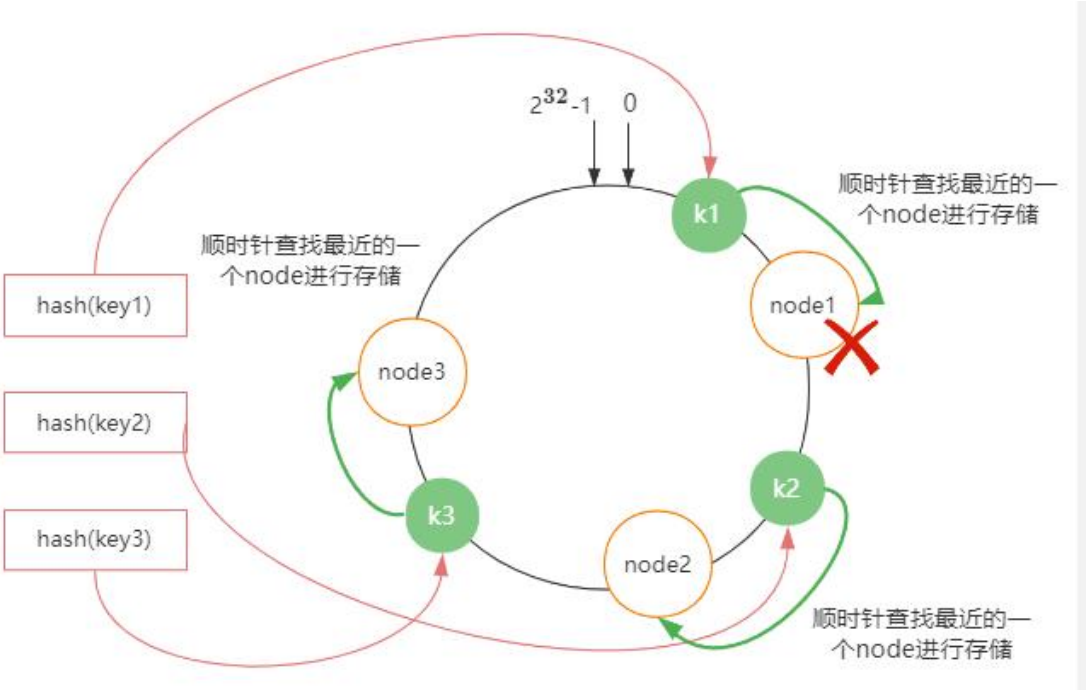
假设现在需要新增一个节点(如图)node4,那数据的映射关系的影响范围只限于node3和 node1,

只有少部分的数据需要重新映射迁移就行了。



如果是已经存在的节点 node1 因为故障下线了（如图），只那只需要把原本分配在 node1 上的数据重新分配到 node2 上就行了。

同样对数据影响的范围非常小。



所以，在我看来，一致性 hash 算法的好处是扩展性很强，在增加或者减少服务器的时候，数据迁移范围比较小。

另外，在一致性 Hash 算范里面，为了避免 hash 倾斜导致数据分配不均匀的情况，我们可以使用虚拟节点的方式来解决。

以上就是我对这个问题的理解。

53.156 Thread 和 Runnable 的区别

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

昨天一个工作 1 年的粉丝出去面试。

遇到这样一个问题：thread 和 runnable 的区别

其实这个问题很简单，但是如果能够按照高手的回答思路去说明，效果会更好

以下是高手的回答

好的，我认为 Thread 和 Runnable 接口的区别有 4 个。

1. Thread 是一个类，Runnable 是接口，因为在 Java 语言里面的继承特性，接口可以支持多继承，而类只能单一继承。

所以如果在已经存在继承关系的类里面要实现线程的话，只能实现 Runnable 接口。

2. Runnable 表示一个线程的顶级接口，Thread 类其实也是实现了 Runnable 接口。

3. 站在面向对象的思想来说，Runnable 相当于一个任务，而 Thread 才是真正处理的线程，所以我们只需要用 Runnable 去定义一个具体的任务，然后交给 Thread 去处理就可以了，这样达到了松耦合的设计目的。

4. Runnable 接口定义了线程执行任务的标准方法 `run`，所以它

所以，基于这四个点的原因，所以在实际应用中，建议实现 Runnable 接口实现线程的任务定义，然后使用 Thread 的 start 方法

去启动启动线程并执行 Runnable 这个任务。

以上就是我对这个问题的理解。

53.157 说说你对一致性 Hash 算法的理解

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

最近一个工作 4 年的粉丝，去美团面试，被面试官问到一个一致性 hash 算法的问题他没回答上来，然后跑过来找我希望我能帮他分析一下这个问题。

具体问题是： “说说你对一致性 Hash 算法的理解”

以下是高手的回答

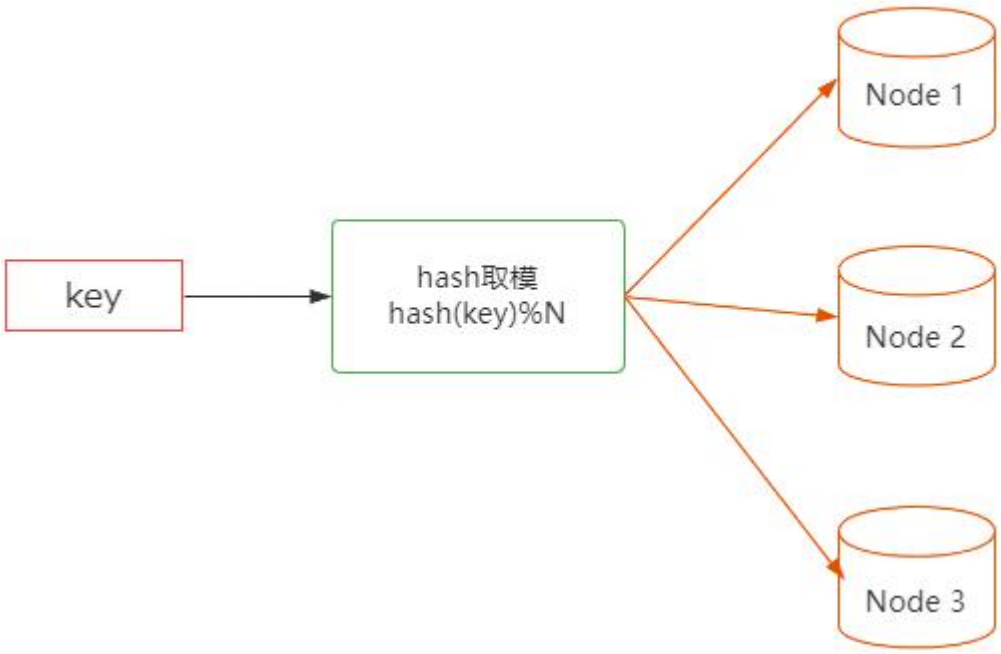
好的，面试官。

一致性 hash，是一种比较特殊的 hash 算法，它的核心思想是解决在分布式环境下，hash 表中可能存在的动态扩容和缩容的问题。

> 为什么会出现一致性 hash（直接在面试官的屏幕上提示就行）

一般情况下，我们会使用 hash 表的方式以 key-value 的方式来存储数据，但是当数据量比较大的时候，我们就会把数据存储

到多个节点上，（如图）然后通过 hash 取模的方法来决定当前 key 存储到哪个节点上。



这种方式有一个非常明显的问题，就是当存储节点增加或者减少的时候，原本的映射关系就会发生变化。

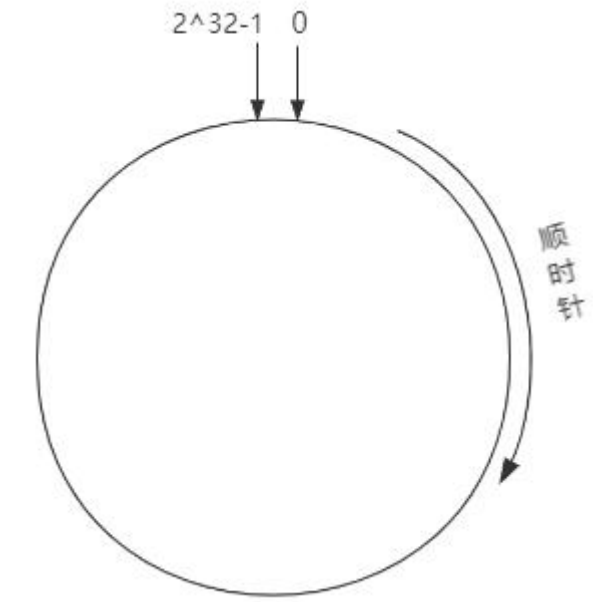
也就是需要对所有数据按照新的节点数量重新映射一遍，这个涉及到大量的数据迁移和重新映射，迁移代价很大。

> 一致性 hash 的工作原理（直接在面试官的屏幕上提示就行）

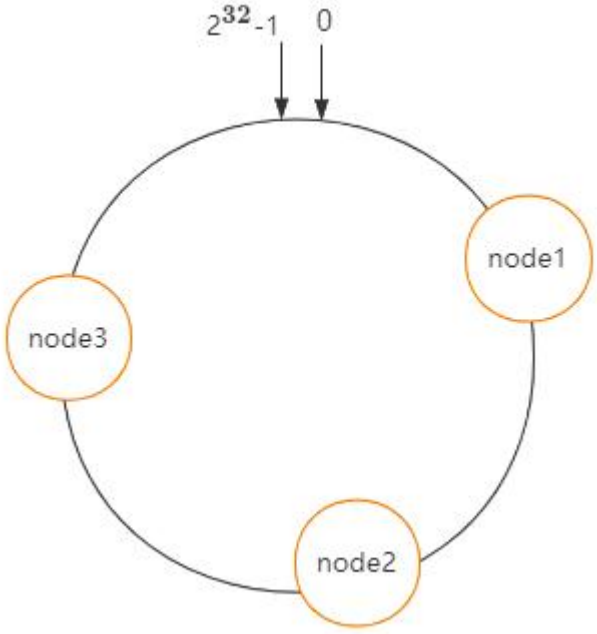
而一致性 hash 就是用来优化这种动态变化场景的算法，它的具体工作原理也很简单。

首先，一致性 Hash 是通过一个 Hash 环的数据结构来实现的，（如图），这个环的起点是 0，终点是 2^32-1。

也就是这个环的数据分布范围是[0,2^32-1]。

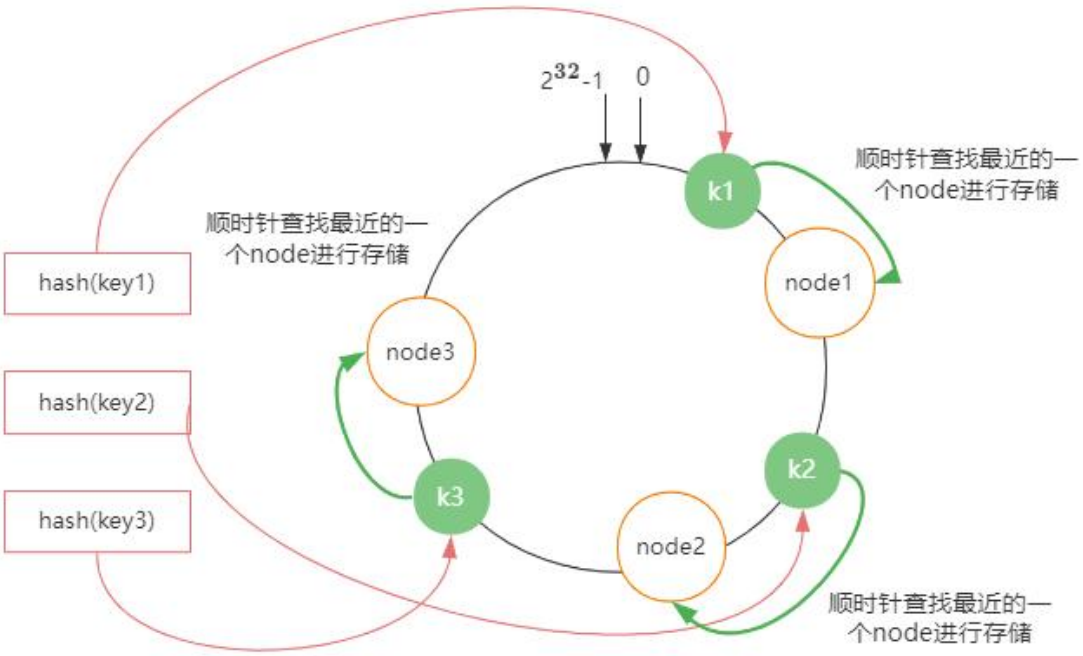


（如图）然后我们把存储节点的 ip 地址作为 key 进行 hash 之后，会在 Hash 环上确定一个位置。



接下来，（如图）就是把需要存储的目标 key 使用 hash 算法计算后得到一个 hash 值，同样也会落到 hash 环的某个位置上。

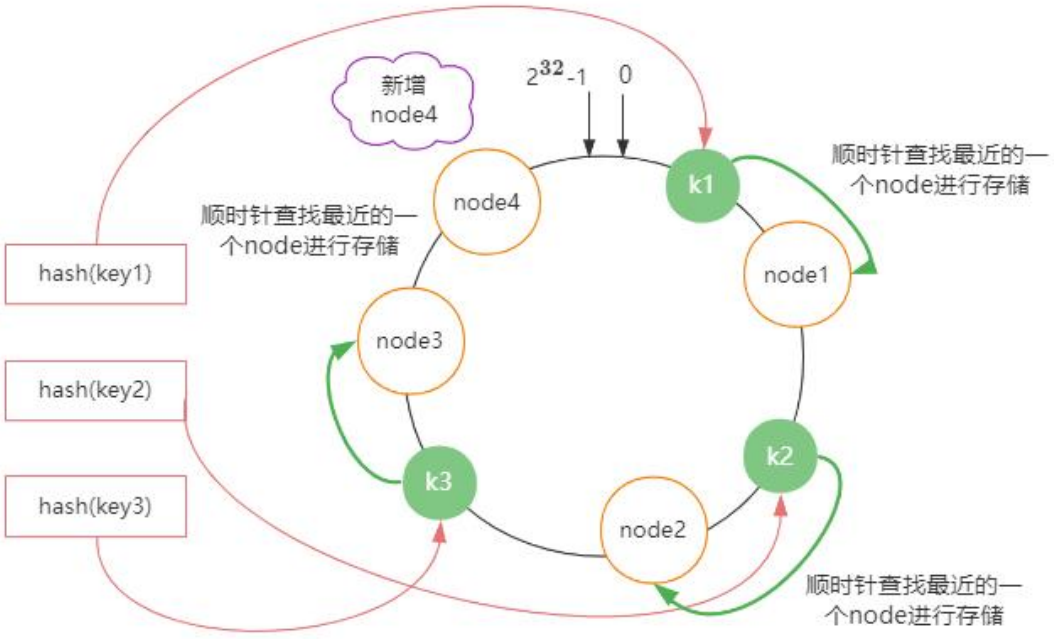
然后这个目标 key 会按照顺时针的方向找到离自己最近的一个节点进行数据存储。



> 为什么一致性 Hash 比普通 hash 算法好（字幕提示）

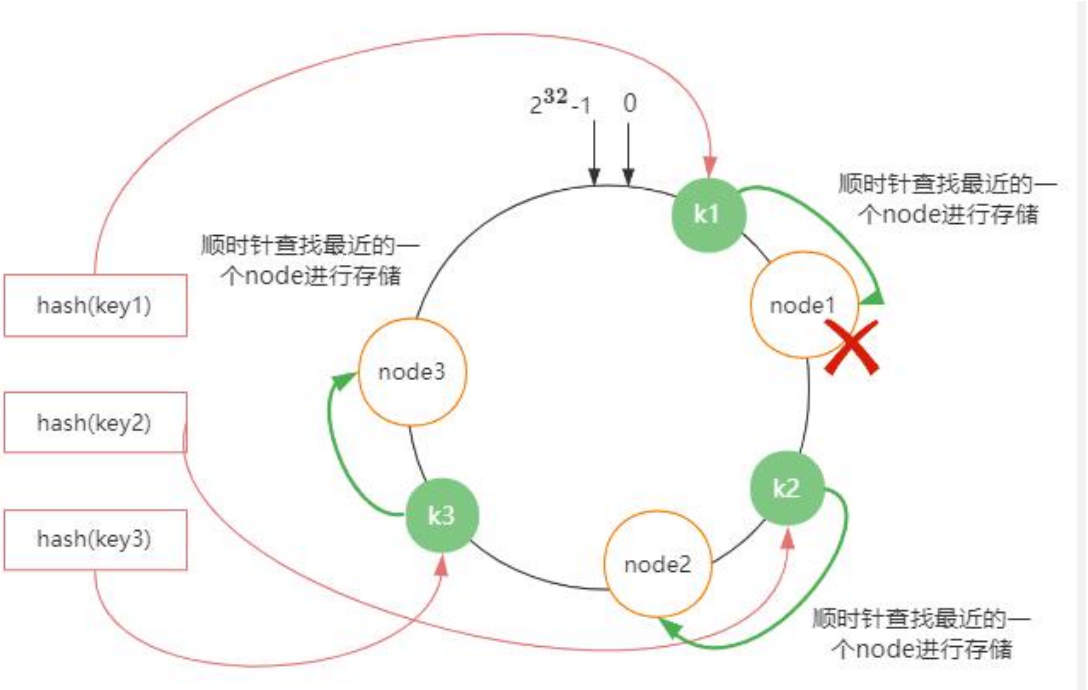
假设现在需要新增一个节点(如图)node4,那数据的映射关系的影响范围只限于node3和 node1,

只有少部分的数据需要重新映射迁移就行了。



如果是已经存在的节点 node1 因为故障下线了（如图），只那只需要把原本分配在 node1 上的数据重新分配到 node2 上就行了。

同样对数据影响的范围非常小。



所以，在我看来，一致性 hash 算法的好处是扩展性很强，在增加或者减少服务器的时候，数据迁移范围比较小。

另外，在一致性 Hash 算范里面，为了避免 hash 倾斜导致数据分配不均匀的情况，我们可以使用虚拟节点的方式来解决。

以上就是我对这个问题的理解。

53.158 Thread 和 Runnable 的区别

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

昨天一个工作 1 年的粉丝出去面试。

遇到这样一个问题：thread 和 runnable 的区别

其实这个问题很简单，但是如果能够按照高手的回答思路去说明，效果会更好

以下是高手的回答

好的，我认为 Thread 和 Runnable 接口的区别有四个。

1. Thread 是一个类，Runnable 是接口，因为在 Java 语言里面的继承特性，接口可以支持多继承，而类只能单一继承。

所以如果在已经存在继承关系的类里面要实现线程的话，只能实现 Runnable 接口。

2. Runnable 表示一个线程的顶级接口，Thread 类其实是实现了 Runnable 这个接口，我们在使用的时候都需要实现 run 方法。
3. 站在面向对象的思想来说，Runnable 相当于一个任务，而 Thread 才是真正处理的线程，

所以我们只需要用 Runnable 去定义一个具体的任务，然后交给 Thread 去处理就可以了，这样达到了松耦合的设计目的。

4. 接口表示一种规范或者标准，而实现类表示对这个规范或者标准的实现，所以站在线程的角度来说，Thread 才是真正意义上的线程实现。

Runnable 表示线程要执行的任务，因此在线程池里面，提交一个任务传递的类型是 Runnable。

总的来说，在我看来，Thread 只是实现了 Runnable 接口并做了扩展，所以这两者我认为没什么可比性。
以上就是我对这个问题的理解。

53.159 Integer 使用不当导致生产的事故

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。
昨天一个工作 4 年的粉丝私信我，他说最近背了一个生产事故，想让我来给大家分享一下避免采坑。
他是做理财这块业务的，他每天会收到一个基金公司的收益文件，
然后他需要把这个文件解析并且保存每个用户的收益数据到数据库。
在解析文件的时候，他需要对数据的条数做校验，于是用到了 Integer 这个对象并且使用`==`来判断。
测试环境都没问题，但是到了生产环境上出现用户收益没有到账的问题，造成了大规模的投诉。
最后定位才发现是收益文件验证失败导致没有被解析入库。
所以这里就出现一个问题：“为什么两个 Integer 的对象不能用==号来判断？为什么测试环境没有把这问题测试出来”。

以下是高手的回答

好的，面试官。

Integer 是一个封装类型。它是对应一个 int 类型的包装。

在 Java 里面之所以要提供 Integer 这种基本类型的封装类，是因为 Java 是一个面向对象的语言，

而基本类型不具备对象的特征，所以在基本类型上做了一层对象的包装并且提供了相关的属性和访问方法来完善基本类型的操作。

在 Integer 这个封装类里面，除了基本的 int 类型的操作之外，还引入了享元模式的设计，

对-128 到 127 之间的数据做了一层缓存(如图），也就是说，如果 Integer 类型的目标值在-128 到 127 之间，

就直接从缓存里面获取 Integer 这个对象实例并返回，否则创建一个新的 Integer 对象。

```
public final class Integer extends Number implements Comparable<Integer> {
    ...
     public static Integer valueOf(int i) {
          assert IntegerCache.high >= 127;
          if (i >= IntegerCache.low && i <= IntegerCache.high)
               return IntegerCache.cache[i + (-IntegerCache.low)];
          return new Integer(i);
    }
    ...
}
```

这么设计的好处是减少频繁创建 Integer 对象带来的内存消耗从而提升性能。

因此在这样一个前提下，如果定义两个 Integer 对象，并且这两个 Integer 的取值范围正好在-128 到 127 之间。

如果直接用==号来判断，返回的结果必然是 true，因为这两个 Integer 指向的内存地址是同一个。

否则，返回的结果是 false。

之所以在测试环境上没有把这个问题暴露出来，是因为测试环境上验证的数据量有限，使得取值的范围正好在 Integer

的缓存区间，从而通过了测试。

但是在实际的应用里面，数据量远远超过 IntegerCache 的取值范围，所以就导致了校验失败的问题。

以上就是我对这个问题的理解。

53.160 JVM 分代年龄为什么是 15 次？可以 25 次吗?

一个工作了 7 年的粉丝去京东面试，遇到一个很有意思的问题。

这是一个关于 JVM 底层相关的问题，如果平时没有去花时间是肯定回答不出来的。

Hello，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

今天给大家分享的这道面试题是：“JVM 分代年龄为什么是 15 次，可以是 25 次吗？”

以下是高手的回答

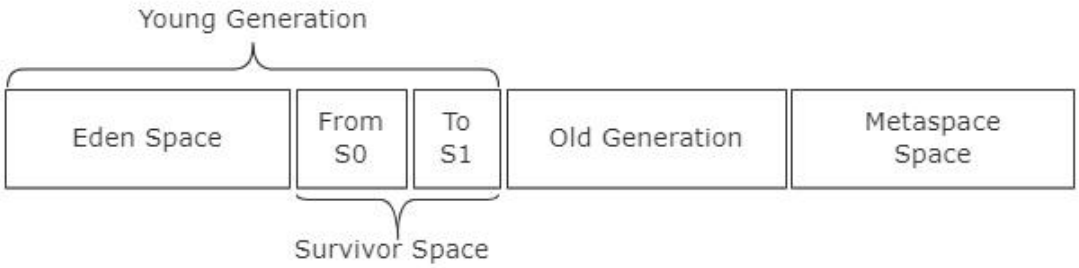
好的，这个问题我会从几个方面来回答。

首先，在 JVM 的 heap 内存里面，分为 Eden Space、Survivor Space、Old Generation（如图）。

当我们在 Java 里面使用 new 关键字创建一个对象的时候，JVM 会在 Eden Space 分配一块内存空间来存储这个对象。

当 Eden Space 的内存空间不足的时候，会触发 Young GC 进行对象回收。

那些因为存在引用关系而无法回收的对象，JVM 会把它们转移到 Survivor Space。



（如图）Survivor Space 内部又分为 From 区和 To 区，刚从 Eden 区转移过来的对象会分配到 From 区，

每经历一次 Young GC，这些没有办法被回收的对象就会在 From 区和 To 区来回移动，每移动一次，这个对象的

GC 年龄就加 1。默认情况下 GC 年龄达到 15 的时候，JVM 就会把这个对象移动到 Old Generation。

其次呢，一个对象的 GC 年龄，是存储在对象头里面的（如图），一个 Java 对象在 JVM 内存中的布局由三个部分组成，

分别是对象头、实例数据、对齐填充。而对象头里面有 4 个 bit 位来存储 GC 年龄。

锁状态	32bit				
	25bit		4bit	1bit	2bit
	23bit	2bit		偏向模式	标志位
未锁定	对象哈希码		分代年龄	0	01
轻量级锁定	指向调用栈中锁记录的指针				00
重量级锁定（锁膨胀）	指向重量级锁的指针				10
GC 标记	空				11
可偏向	线程 ID	Epoch	分代年龄	1	01

而 4 个 bit 位能够存储的最大数值是 15，所以从这个角度来说，JVM 分代年龄之所以设置成 15 次是因为它最大能够存储的数值就是 15。

虽然 JVM 提供了参数来设置分代年龄的大小，但是这个大小不能超过 15。

而从设计角度来看，当一个对象触发了最大值 15 次 gc，还没有办法被回收，就只能移动到 old generation 了。

另外，设计者还引入了动态对象年龄判断的方式来决定把对象转移到 old generation，也就是说

不管这个对象的 gc 年龄是否达到了 15 次，只要满足动态年龄判断的依据，也同样会转移到 old generation。

以上就是我对这个问题的理解。

53.161 可以讲一下 ArrayList 的自动扩容机制吗?

一个工作了 3 年的粉丝，最近在面试的时候遇到一个集合方面的问题。

这个问题其实很简单，可能大家没有去关注集合方面的底层实现原理。

导致在面试的时候容易栽跟头。

Hi，大家好，我是 Mic，一个工作了 14 年的程序员和创业者。

今天分享的这个面试题是： “ArrayList 的自动扩容机制的实现原理”

> 以下是高手的回答

好的，面试官。

ArrayList 是一个数组结构的存储容器，默认情况下，数组的长度是 10.

当然我们也可以在构建 ArrayList 对象的时候自己指定初始长度。

随着在程序里面不断的往 ArrayList 中添加数据，当添加的数据达到 10 个的时候，ArrayList 就没有多余容量可以存储后续的数据。

这个时候 ArrayList 会自动触发扩容。

扩容的具体流程很简单，

1. 首先，创建一个新的数组，这个新数组的长度是原来数组长度的 1.5 倍。
2. 然后使用 Arrays.copyOf 方法把老数组里面的数据拷贝到新的数组里面。

扩容完成后再把当前要添加的元素加入到新的数组里面，从而完成动态扩容的过程。

以上就是我对这个我对这个问题的理解！

53.162 Eureka server 数据同步原理能说下吗

Hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

昨天，一个工作了 4 年的 Java 程序员，去一个互联网公司面试，在面试的时候面试官一个劲的

问他 Spring Cloud Netflix 组件的底层原理，开始还能回答出来，越到后面越懵逼，最后面试没过。

他有点不理解面试官怎么会问这么深，当他把简历发给我的时候，我看到简历上写了精通 Spring Cloud。

瞬间明白了原因。

Spring Cloud Netflix 里面的组件可以考察的方向太多了，比如今天分享的这道面试题：

“Eureka server 数据同步原理”，相信 80%的同学都回答不出来，不过没关系，我已经把这个问题的答案

以下是高手的回答

好的，面试官

Eureka 是一个服务注册中心，在 Eureka 的设计里面，为了保证 Eureka 的高可用性，提供了集群的部署方式。

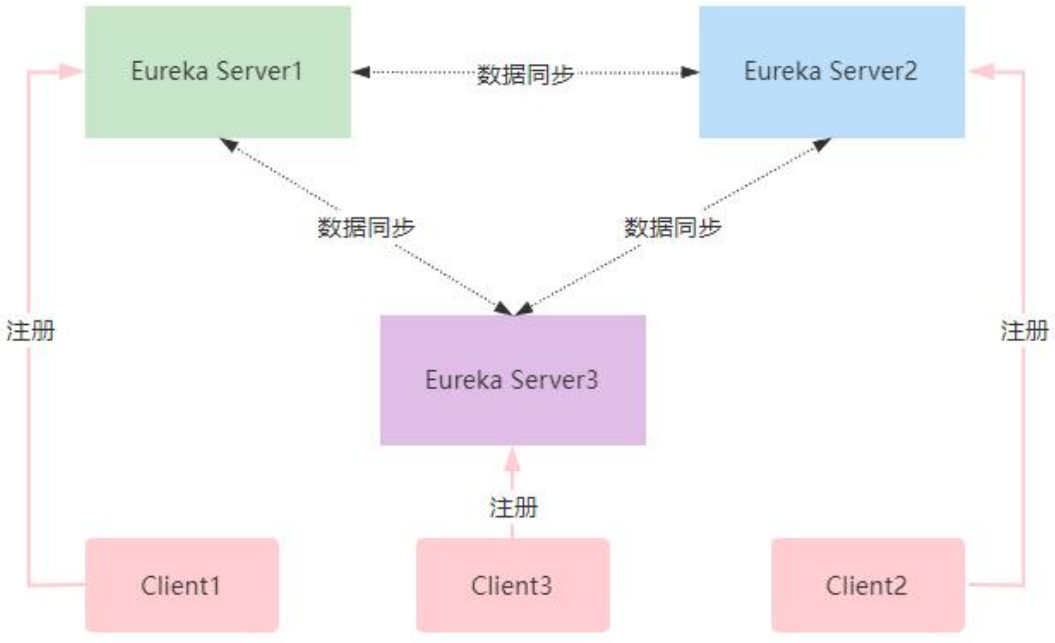
Eureka 的集群部署采用的是两两相互注册的方式来实现，也就是说每个 Eureka Server 节点都需要发现

集群中的其他节点并建立连接，然后通过心跳的方式来维持这个连接的状态。

（如图），Eureka Server 集群节点之间的数据同步方式非常简单粗暴，使用的是对等复制的方式来实现数据同步。

也就是说，在 Eureka Server 集群中，不存在所谓主从节点，任何一个节点都可以接收或者写入数据。

一旦集群中的任意一个节点接收到了数据的变更，就直接同步到其他节点上。



这种无中心化节点的数据同步，需要考虑到一个数据同步死循环的问题，也就是需要区分 Eureka Server 收到的数据

是属于客户端传递来的数据还是集群中其他节点发过来的同步数据。

Eureka 使用了一个时间戳的标记来实现类似于数据的版本号来解决这个问题。

另外，从 Eureka 的数据同步方案来看，Eureka 集群采用的是 AP 模型，也就是只提供高可用保障，而不提供数据强一致性保障。

之所以采用 AP，我认为注册中心它只是维护服务之间的通信地址，数据是否一致对于服务之间的通信影响并不大。

而注册中心对 Eureka 的高可用性要求会比较高，不能出现因为 Eureka 的故障导致服务之间无法通信的问题。

以上就是我对这个问题的理解。

53.163 请说一下你对分布式和微服务的理解

很难想象，在分布式架构以及微服务架构普及了近 10 年时间的现在，还有人不清楚微服务架构和分布式架构。

这不，一个工作了 3 年的粉丝，就遇到了这个方面的问题，希望我能出一个视频。

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

今天给大家分享的面试题是： “请你说一下你对分布式和微服务的理解” 。

以下是高手的回答

好的，面试官。

首先我先解释一下分布式系统。

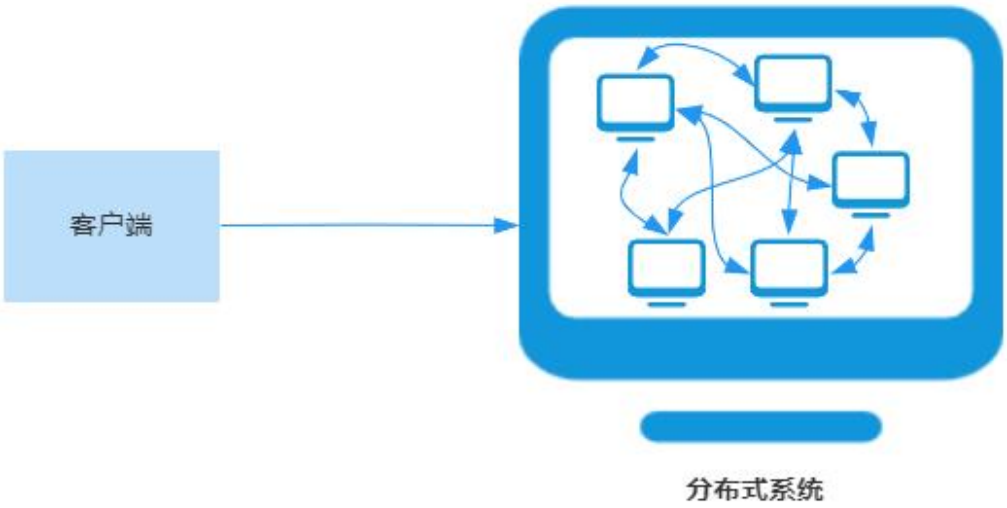
简单来说，分布式是一组通过网络进行通信，并且为了完成共同的计算任务的计算机节点组成的系统。

分布式系统的设计理念，其实是来自于小型机或者大型机的计算能力的瓶颈和成本的增加。

在集中式系统里面，要想提升程序的运行性能，只能不断的升级 CPU 以及增加内存，但是硬件的提升本身也是有瓶颈的，所以当企业对于计算要求越来越高的时候，集中式架构已经无法满足需求了。

在这样的背景下， 就产生了分布式计算，（如图），也就是把一个计算任务分配给多个计算机节点去运行。

但是对于用户或者客户端来说，感知不到背后的逻辑，就像访问单个计算机一样，他看到的仍然是一个整体。



在分布式系统中，软件架构也需要作出相应的调整，需要把原本的单体应用进行拆分，部署到多个计算机节点上，然后各个服务之间使用远程通信协议实现计算结果的数据交互。

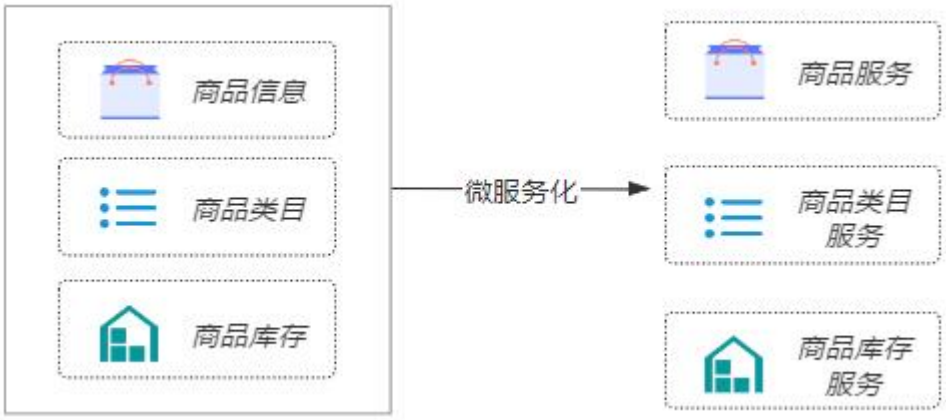
针对这种分布式部署的应用架构，我们称为 SOA（面向服务）的架构。

其次，我再解释一下微服务架构。

其实微服务架构本身就是一种分布式架构，它强调的是对部署在各个计算机上的应用服务的粒度。

（如图）它的核心思想是，针对拆分的服务节点做更进一步的解耦。

也就是说，针对 SOA 服务化架构下的单个业务服务，以更加细粒度的方式进一步拆分。每个拆分出来的微服务由独立的小团队负责，最好在 3 人左右。



拆分的好处是使得程序的扩展性更强，开发迭代效率更高。

对于一些大型的互联网项目来说，微服务能够在不影响用户使用的情况下非常方便的实现产品功能的创新和上线。

以上就是我对这个问题的理解。

53.164 请你说一下你对滑动窗口算法的理解

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

一个工作 5 年的程序员，去滴滴面试，被问到了 Sentinel 组件里面关于滑动窗口算法的问题。

他很遗憾，平时写 CRUD 比较多，技术底层的回答只能靠运气。

于是面试没有通过，错过了一个很好的机会。

关于：“请你说一下你对滑动窗口算法的理解” 这个问题。

> 以下是高手的回答

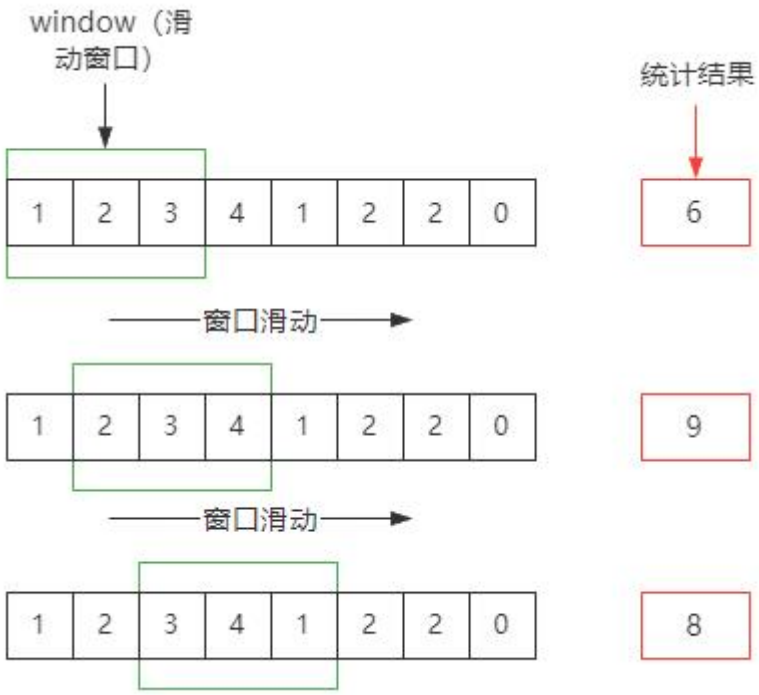
好的，面试官，

滑动窗口是一种比较常用的数据统计算法。

（如图）简单来说，就是在一个大的数组上，定义一个固定长度的滑动窗口，然后这个窗口在数组上进行滑动。

在窗口滑动的过程中，左边会出一个元素，右边会进一个元素， 最后，我们根据当前窗口内记录的数据进行计算。

从而达到数据统计的目的。



滑动窗口一般可以用来解决数组的统计问题，比如。

- 解决数组/字符串的子元素问题。
- 把嵌套的 for 循环问题，转换为单循环问题，降低时间复杂度。

在 Hystrix 中，用到了滑动窗口来实现熔断触发的数据统计。

另外在 Sentinel 限流框架中，也使用了滑动窗口来实现限流的数据统计。

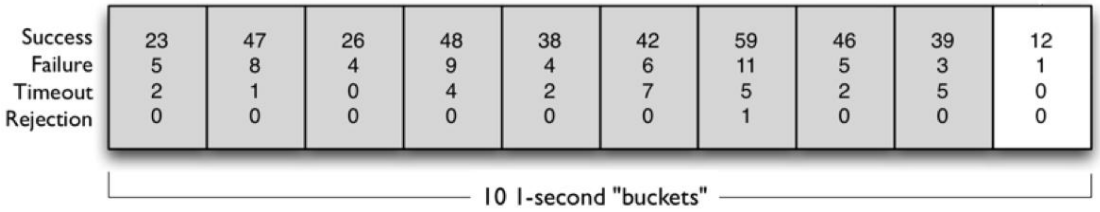
不过在这两个组件中使用的滑动窗口都做了一些调整，他们是通过时间线来驱动窗口往前滑动的。

简单来说，以 Hystrix 为例，（如图）它定义一个 10 个长度的数组，每个数组表示一个 1 秒的时间区间跨度，

然后在每个区间中记录当前时间端内发生的所有请求的成功、失败的次数。

Hystrix 只需要统计当前 10 秒内对应的 10 个窗口总的成功、失败次数。

最后根据配置的阈值决定是否要触发熔断功能。



以上就是我对这个问题的理解。

53.165 什么是深拷贝和浅拷贝?

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

一个工作了 3 年的小伙子，委屈巴巴的跑过来私信我，说最近面试也太难了。

只是找个 CRUD 的工作，竟然还问我深拷贝和浅拷贝，这个问题又不影响我写 CRUD，唉。

然后深情的望向天空，眼里充满了迷茫了焦虑。

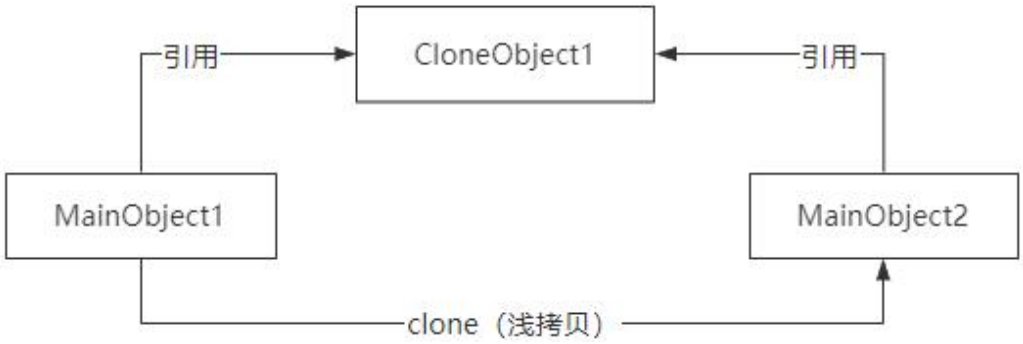
以下是高手的回答

好的，面试官。

深拷贝和浅拷贝是用来描述对象或者对象数组这种引用数据类型的复制场景的。

浅拷贝，（如图）就是只复制某个对象的指针，而不复制对象本身。

这种复制方式意味着两个引用指针指向被复制对象的同一块内存地址。



深拷贝，（如图）会完全创建一个一模一样的新对象，新对象和老对象不共享内存，也就意味着对新对象的修改不会影响老对象的值。

在 Java 里面，无论是深拷贝还是浅拷贝，都需要通过实现 Cloneable 接口，并实现 clone()方法。

然后我们可以在 clone()方法里面实现浅拷贝或者深拷贝的逻辑。

实现深拷贝的方法有很多，比如

1. 通过序列化的方式实现，也就是把一个对象先序列化一遍，然后再反序列化回来，就会得到一个完整的新对象。
2. 在 clone()方法里面重写克隆逻辑，也就是对克隆对象内部的引用变量再进行一次克隆。

以上就是我对这个问题的理解。

53.166 谈谈你对 Spring IOC 和 DI 的理解?

一个工作 4 年的 Java 程序员，Spring 都用了 4 年了，竟然连 Spring 里面这么基础的问题都回答不好。

还以为像 3~4 年前一样，随便准备一下就可以拿到一个薪资不错的 Offer。

同学们，这种好日子已经过去了，对于 Java 程序员来说，未来找工作一定会越来越难。

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

今天分享的面试提示：谈谈你对 Spring IOC 和 DI 的理解。

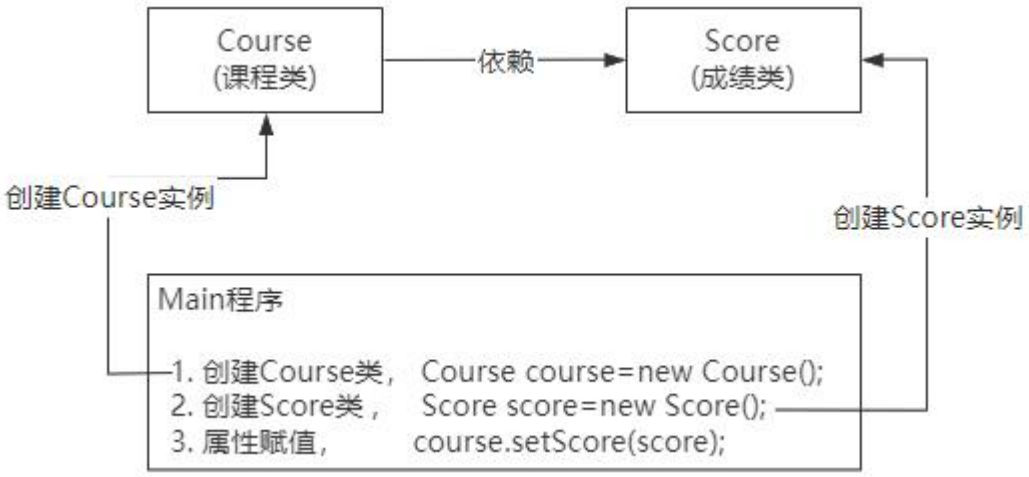
以下是高手的回答

好的，面试官。

首先，Spring IOC，全称控制反转（Inversion of Control）。

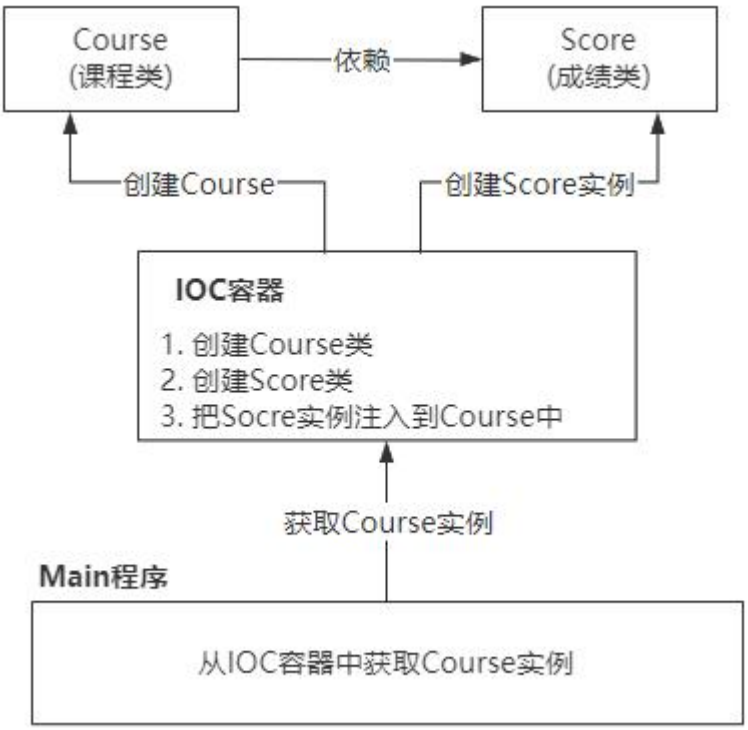
（如图）在传统的 Java 程序开发中，我们只能通过 new 关键字来创建对象，

这种导致程序中对象的依赖关系比较复杂，耦合度较高。



（如图）而 IOC 的主要作用是实现了对象的管理，也就是我们把设计好的对象交给了 IOC 容器控制，

然后在需要用到目标对象的时候，直接从容器中去获取。



有了 IOC 容器来管理 Bean 以后，相当于把对象的创建和查找依赖对象的控制权交给了容器，这种设计理念

使得对象与对象之间是一种松耦合状态，极大提升了程序的灵活性以及功能的复用性。

然后，DI 表示依赖注入，也就是对于 IOC 容器中管理的 Bean，如果 Bean 之间存在依赖关系，那么 IOC 容器需要

自动实现依赖对象的实例注入，通常有三种方法来描述 Bean 之间的依赖关系。

1. 接口注入
2. setter 注入
3. 构造器注入

另外，为了更加灵活的实现 Bean 实例的依赖注入，Spring 还提供了@Resource 和 @Autowired 这两个注解。

分别是根据 bean 的 id 和 bean 的类型来实现依赖注入。

以上就是我对这个问题的理解！

53.167 wait 和 sleep 是否会触发锁的释放以及 CPU 资源的释放?

通过最近这几个月的私信发现一个问题，很多工作了 5~6 年的程序员，去面试的时候但凡问到技术原理。

基本上都是回答不出来的，有些同学侥幸靠背面试题通过面试，但是这种无法掌控自己选择权的感觉，

你不觉得很难受吗?

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

一个工作 5 年的粉丝，去美团面试，遇到了这样一个问题。

“wait 和 sleep 是否会触发锁的释放以及 CPU 资源的释放？”

其实这个问题还比较简单，它的结论回答出来了，但是后面面试官又问了一个为什么，他就懵了。

以下是高手的回答

好的，面试官。

Object.wait()方法，会释放锁资源以及 CPU 资源。

Thread.sleep()方法，不会释放锁资源，但是会释放 CPU 资源。

首先，wait()方法是让一个线程进入到阻塞状态，而这个方法必须要写在一个 Synchronized 同步代码块里面。

因为 wait/notify 是基于共享内存来实现线程通信的工具，这个通信涉及到条件的竞争，所以在调用这两个方法之前必须要竞争锁资源。

当线程调用 wait 方法的时候，表示当前线程的工作处理完了，意味着让其他竞争同一个共享资源的线程有机会去执行。

但前提是其他线程需要竞争到锁资源，所以 wait 方法必须要释放锁，否则就会导致死锁的问题。

然后，Thread.sleep()方法，只是让一个线程单纯进入睡眠状态，这个方法并没有强制要求加 synchronized 同步锁。

而且从它的功能和语义来说，也没有这个必要。

当然，如果是在一个 Synchronized 同步代码块里面调用这个 Thread.sleep，也并不会触发锁的释放。

最后，凡是让线程进入阻塞状态的方法，操作系统都会重新调度实现 CPU 时间片切换，这样设计的目的是提升 CPU 的利用率。

以上就是我对这个问题的理解。

53.168 AQS 为什么要使用双向链表?

一个工作 4 年的程序员，简历上写精通并发编程，并且阅读过 AQS（AbstractQueuedSynchronizer）的源码，然后面试官只问了他一个问题，然后就垮了！

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

AQS 大家都不陌生，是 J.U.C 包里面一个非常重要的线程同步器。

面试官提了这样一个问题：“AQS 为什么要采用双向链表结构”？

以下是高手的回答

好的，面试官。

首先，双向链表的特点是它有两个指针，一个指针指向前置节点，一个指针指向后继节点。

所以，双向链表可以支持 常量 O(1) 时间复杂度的情况下找到前驱结点，基于这样的特点。

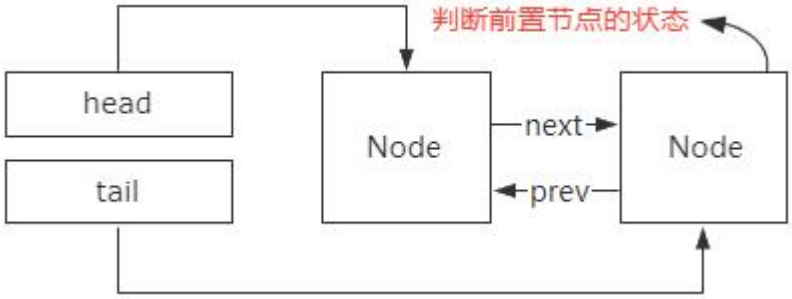
双向链表在插入和删除操作的时候，要比单向链表简单、高效。

因此，从双向链表的特性来看，我认为 AQS 使用双向链表有三个方面的考虑。

- 第一个方面，没有竞争到锁的线程加入到阻塞队列，并且阻塞等待的前提是，当前线程所在节点的前置节点是正常状态，

这样设计是为了避免链表中存在异常线程导致无法唤醒后续线程的问题。

（如图）所以线程阻塞之前需要判断前置节点的状态，如果没有指针指向前置节点，就需要从 head 节点开始遍历，性能非常低。



- 第二个方面，在 Lock 接口里面有一个，lockInterruptibly()方法，这个方法表示处于锁阻塞的线程允许被中断。

也就是说，没有竞争到锁的线程加入到同步队列等待以后，是允许外部线程通过 interrupt()方法触发唤醒并中断的。

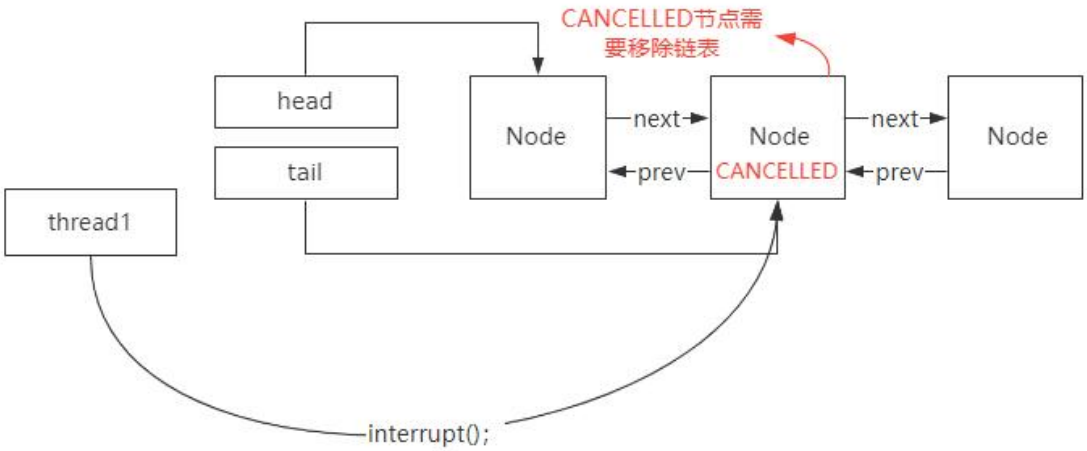
这个时候，被中断的线程的状态会修改成 CANCELLED。

（如图）被标记为 CANCELLED 状态的线程，是不需要去竞争锁的，但是它仍然存在于双向链表里面。

意味着在后续的锁竞争中，需要把这个节点从链表里面移除，否则会导致锁阻塞的线程无法被正常唤醒。

在这种情况下，如果是单向链表，就需要从 Head 节点开始往下逐个遍历，找到并移除异常状态的节点。

同样效率也比较低，还会导致锁唤醒的操作和遍历操作之间的竞争。



- 第三个方面，为了避免线程阻塞和唤醒的开销，所以刚加入到链表的线程，首先会通过自旋的方式尝试去竞争锁。

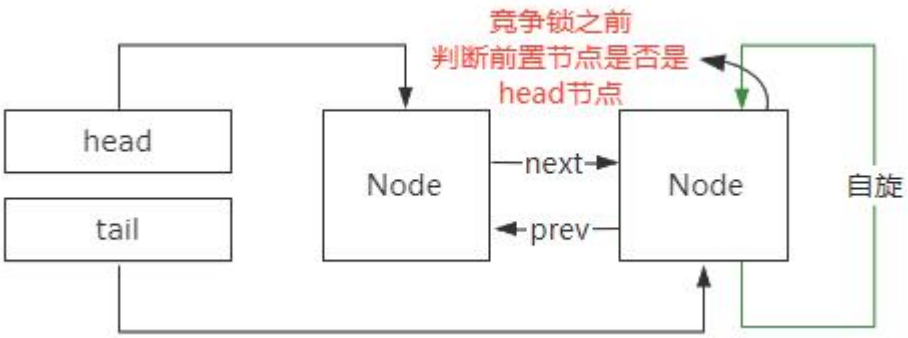
但是实际上按照公平锁的设计，只有头节点的下一个节点才有必要去竞争锁，后续的节点竞争锁的意义不大。

否则，就会造成羊群效应，也就是大量的线程在阻塞之前尝试去竞争锁带来比较大的性能开销。

所以，（如图）为了避免这个问题，加入到链表中的节点在尝试竞争锁之前，需要判断前置节点是不是头节点，

如果不是头节点，就没必要再去触发锁竞争的动作。

所以这里会涉及到前置节点的查找，如果是单向链表，那么这个功能的实现会非常复杂。



以上就是我对这个问题的理解！

53.169 ConcurrentHashMap 的 size()方法是线程安全的吗？为什么

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

一个工作了 5 年的粉丝，最近去蚂蚁面试，在第一面的时候，本问到了几个 Java 基础的问题。

其中有一个问题比较有意思

面试官问：“ConcurrentHashMap 的 size()方法是线程安全的吗？为什么？”

关于这个问题的高手回答，我整理到了一个 10W 字的面试文档里面，大家可以在我的主页加 V 去领取。

> 以下是高手的回答

好的，面试官。

ConcurrentHashMap 的 size()方法是非线程安全的。

也就是说，当有线程调用 put 方法在添加元素的时候，其他线程在调用 size()方法获取的元素个数

和实际存储元素个数是不一致的。

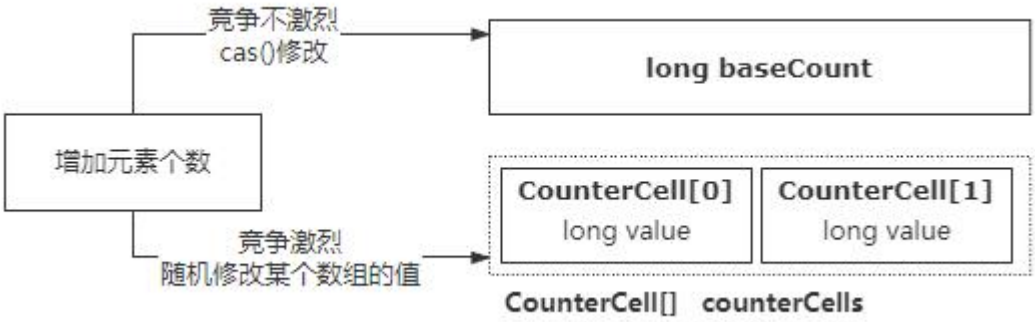
原因是 size()方法是一个非同步方法，put()方法和 size()方法并没有实现同步锁。

put()方法的实现逻辑是：在 hash 表上添加或者修改某个元素，然后再对总的元素个数进行累加。

其中，线程的安全性仅仅局限在 hash 表数组粒度的锁同步，避免同一个节点出现数据竞争带来线程安全问题。

（如图）数组元素个数的累加方式用到了两个方案：

- 当线程竞争不激烈的时候，直接用 cas 的方式对一个 long 类型的变量做原子递增。
- 当线程竞争比较激烈的时候，使用一个 CounterCell 数组，用分而治之的思想减少多线程竞争，从而实现元素个数的原子累加。



size()方法的逻辑就是遍历 CounterCell 数组中的每个 value 值进行累加，再加上 baseCount，汇总得到一个结果。

所以很明显，size()方法得到的数据和真实数据必然是不一致的。

因此从 size()方法本身来看，它的整个计算过程是线程安全的，因为这里用到了 CAS 的方式解决了并发更新问题。

但是站在 ConcurrentHashMap 全局角度来看，put()方法和 size()方法之间的数据是不一致的，因此也就不是线程安全的。

之所以不像 HashTable 那样，直接在方法级别加同步锁。在我看来有两个考虑点。

1. 直接在 size()方法加锁，就会造成数据写入的并发冲突，对性能造成影响，

当然有些朋友会说可以加读写锁，但是同样会造成 put 方法锁的范围扩大，性能影响极大!

2. ConcurrentHashMap 并发集合中，对于 size()数量的一致性需求并不大，并发集合更多的是去保证数据存储的安全性。

以上就是我对这个问题的理解。

53.170 Redis 多线程模型怎么理解，那它会有线程安全问题吗?

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

昨天，一个工作了 7 年的粉丝私信我这样一个问题。

他说 Redis6.0 已经支持多线程了，那是不是会存在线程安全问题，如果有线程安全问题，它是怎么解决的。

这个问题说简单也简单，说难也挺难的，毕竟不仅仅只是涉及到多线程的问题，还设计到 NIO 里面的 Reactor 模型问题。

关于：“Redis 多线程模型怎么理解，那它会有线程安全问题吗？”这个问题。

以下是高手的回答

好的，面试官，这个问题我需要从几个方面回答。

首先，Redis 在 6.0 支持的多线程，并不是说指令操作的多线程，而是针对网络 IO 的多线程支持。

也就是 Redis 的命令操作，仍然是线程安全的。

其次， Redis 本身的性能瓶颈，取决于三个纬度，网络、CPU、内存。

而真正影响内存的关键问题是像内存和网络。

而 Redis6.0 的多线程，本质上解决网络 IO 的处理效率问题。

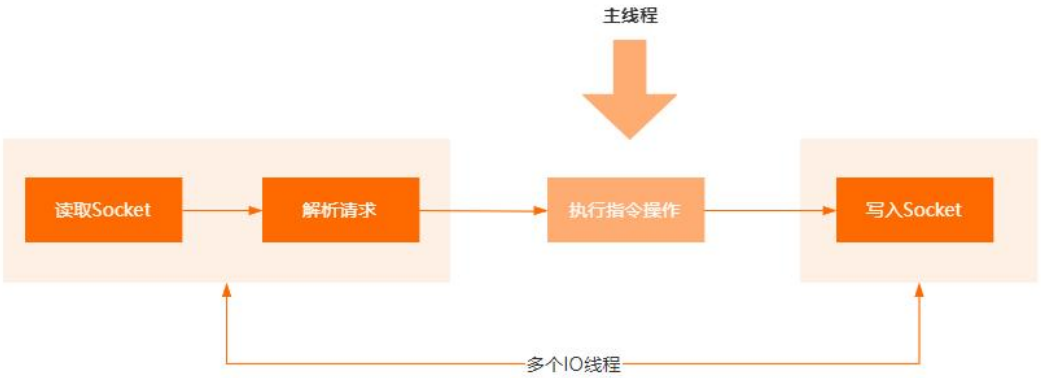
在 Redis6.0 之前。Redis Server 端处理接受到客户端请求的时候，Socket 连接建立到指令的读取、解析、执行、

写回都是由一个线程来处理，

这种方式，在客户端请求比较多的情况下，单个线程的网络处理效率太慢，导致客户端的请求处理效率较低。

于是在 Redis6.0 里面，（如图）针对网络 IO 的处理方式改成了多线程，通过多线程并行的方式提升了网络 IO 的处理效率。

但是对于客户端指令的执行过程，还是使用单线程方式来执行。



最后，Redis6.0 里面多线程默认是关闭的，需要在 redis.conf 文件里面修改 io-threads-do-reads 配置才能开启。

另外，之所以指令执行不使用多线程，我认为有两个方面的原因。

- 内存的 IO 操作，本身不存在性能瓶颈，Redis 在数据结构上已经做了非常多的优化。
- 如果指令的执行使用多线程，那 Redis 为了解决线程安全问题，需要对数据操作增加锁的同步，

不仅仅增加了复杂度，还会影响性能，代价太大不合算。

以上就是我对这个问题的理解！

53.171 Nacos 配置更新的工作流程

Nacos 作为阿里的开源中间件，在加入到 Spring Cloud 生态以后。

不管是作为配置中心还是注册中心，它的简单易用的特性，被广泛适用在各个互联网公司里面。

然后大家会发现 Nacos 相关的面试也越来越多了。

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

昨天，一个工作了 8 年的粉丝，被面试官问到了这样一个问题：

“请你详细说一下 Nacos 客户端是如何实现配置的动态更新的”。

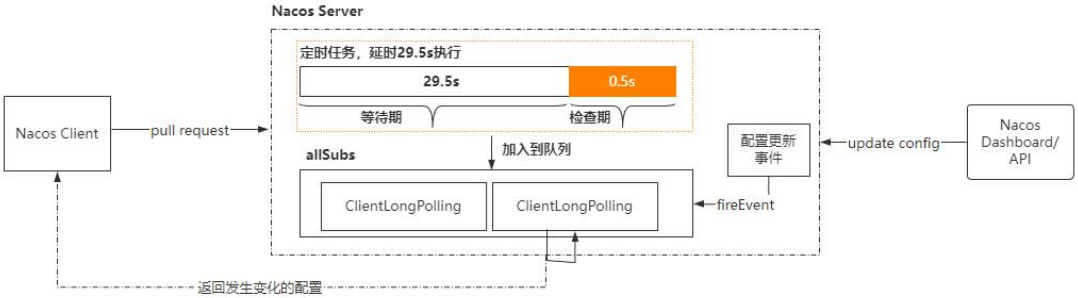
以下是高手的回答

好的，面试官，这个问题我需要从几个方面来回答。

首先，Nacos 是采用长轮训的方式向 Nacos Server 端发起配置更新查询的功能。

所谓长轮训，（如图）就是客户端发起一次轮训请求到服务端，当服务端配置没有任何变更的时候，这个连接一直打开。

直到服务端有配置或者连接超时后返回。



Nacos Client 端需要获取服务端变更的配置，前提是要有一个比较，

也就是拿客户端本地的配置信息和服务端的配置信息进行比较。

一旦发现和服务端的配置有差异，就表示服务端配置有更新，于是把更新的配置拉到本地。

在这个过程中，有可能因为客户端配置比较多，导致比较的时间较长，使得配置同步较慢的问题。

于是 Nacos 针对这个场景，做了两个方面的优化。

1. 减少网络通信的数据量，客户端把需要进行比较的配置进行分片，每一个分片大小是 3000，

也就是说，每次最多拿 3000 个配置去 Nacos Server 端进行比较。

2. 分阶段进行比较和更新，

第一阶段，客户端把这 3000 个配置的 key 以及对应的 value 值的 md5 拼接成一个字符串，然后发送到 Nacos Server 端

进行判断，服务端会逐个比较这些配置中 md5 不同的 key，把存在更新的 key 返回给客户端。

第二阶段，客户端拿到这些变更的 key，循环逐个去调用服务单获取这些 key 的 value 值。

这两个优化，核心目的是减少网络通信数据包的大小，把一次大的数据包通信拆分成了多次小的数据包通信。

虽然会增加网络通信次数，但是对整体的性能有较大的提升。

最后，再采用长连接这种方式，既减少了 pull 轮询次数，又利用了长连接的优势，很好的实现了配置的动态更新同步功能。

以上就是我对这个问题的理解。

53.172 什么是时间轮，请你说一下你对时间轮的理解

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

一个工作了 7 年的程序员，去字节面试，被问到时间轮的问题。

他面试回来和我说，这个问题超出了他的知识面，自己也在网上找了一些文章去学习，但是理解不是很深刻。

想让我出一个关于时间轮问题的面试视频。

谁叫我这么善良呢？立刻就给这个粉丝安排了。

关于“什么是时间轮，请你说一下你对时间轮的理解”这个问题，

> 以下是高手的回答

好的，面试官。

时间轮，简单理解就是一种用来存储一系列定时任务的环状数组，它的整个工作原理和我们的钟表的表盘类似。

它由两个部分组成， 一个是环状数组，另一个是遍历环状数组的指针。

（如图）首先，定义一个固定长度的环状数组，然后数组的每一个元素代表一个时间刻度，

假设是 1s，那么如果是长度为 8 的数组，就代表 8 秒钟。

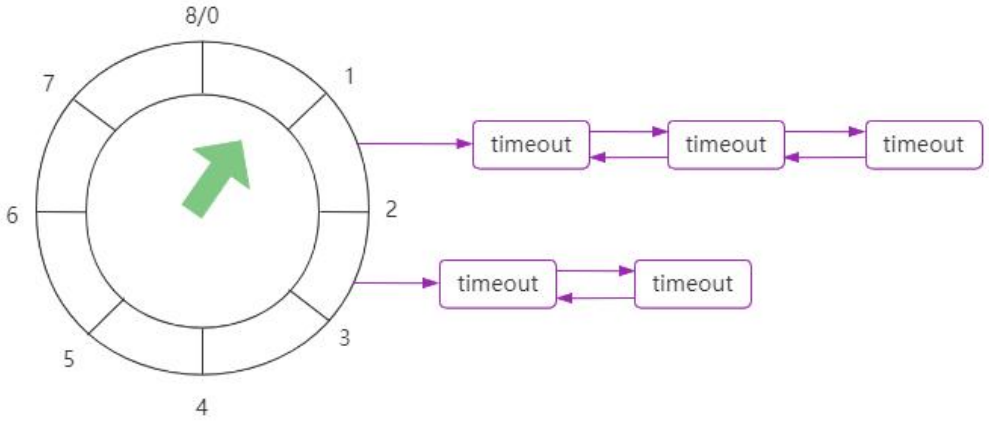
然后，有一个指针，这个指针按照顺时针无线循环这个数组，每隔最小时间单位前进一个数组索引。

这个指针转一圈代表 8 秒钟，转两圈表示 16 秒，假设从 0 点 0 分 0 秒开始，转一圈以后就到了 0 点 0 分 9 秒钟。

环状数组里面的每个元素都是用来存储定时任务的容器，

（如图）当我们向时间轮里面添加一个定时任务的时候，我们会根据定时任务的执行时间计算它所存储的数组下标。

有可能在某个时间刻度上存在多个定时任务，那这个时候会采用双向链表的方式来存储。



当指针指向某个数组的时候，就会把这个数组中存储的任务取出来，然后遍历这个链表逐个运行里面的任务。

如果某个定时任务的执行时间大于环形数组所表示的长度，一般可以使用一个圈数来表示该任务的延迟执行时间。

也就是说，如果是第 16 秒要执行的任务，那意味着这个任务应该是在第二圈的数组下标 0 位置执行。

使用时间轮的方式来管理多个定时任务的好处有很多，我认为有两个核心原因：

- 减少定时任务添加和删除的时间复杂度，提升性能。
- 可保证每次执行定时器任务都是 O（1）复杂度，在定时器任务密集的情况下，性能优势非常明显。

当然，它也有缺点，对于执行时间非常严格的任务，时间轮不是很适合，因为时间轮算法的精度取决于最小时间单元的粒度。假设以 1s 为一个时间刻度，那小于 1s 的任务就无法被时间轮调度。

时间轮算法在很多地方都有用到，比如 Dubbo、Netty、Kafka 等。

以上就是我对这个问题的理解。

53.173 RabbitMQ 的消息如何实现路由?

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。今天分享的这个面试题，没什么特别复杂和特殊的地方。

就是一个 RabbitMQ 里面的普通面试题

这个问题一般是去互联网公司里面 去考察 3~5 年的求职者。

问题是： “RabbitMQ 的消息如何实现路由”？

以下是高手的回答

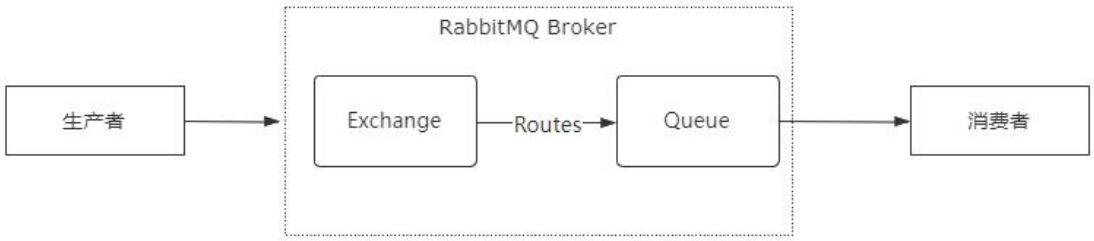
好的，面试官。

RabbitMQ 是一个基于 AMQP 协议实现的分布式消息中间件。

（如图）AMQP 的具体工作机制是，生产者把消息发送到 RabbitMQ Broker 上的 Exchange 交换机上。

Exchange 交换机把收到的消息根据路由规则发给绑定的队列（Queue）。

最后再把消息投递给订阅了这个队列的消费者，从而完成消息的异步通讯。



其中，Exchange 是一个消息交换机，它里面定义了消息路由的规则，也就是这个消息路由到那个队列。

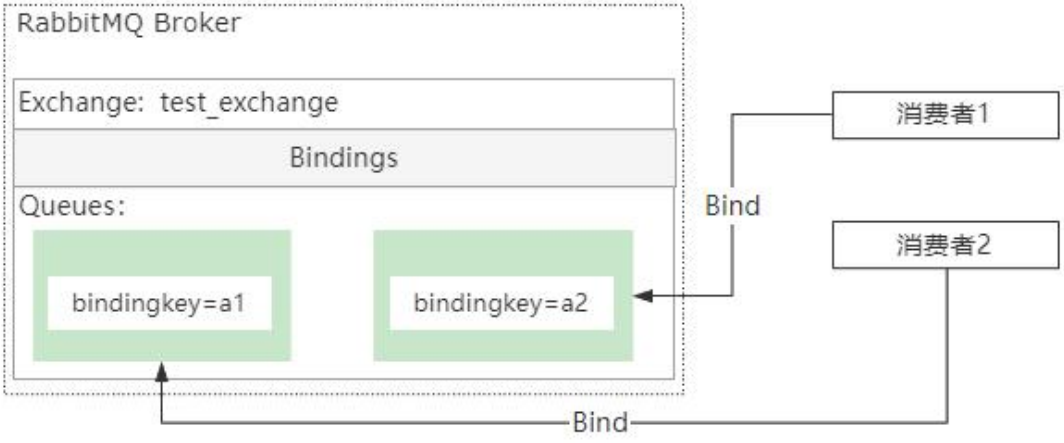
然后 Queue 表示消息的载体，每个消息可以根据路由规则路由到一个或者多个队列里面。

而关于消息的路由机制，核心的组件是 Exchange。

它负责接收生产者的消息然后把消息路由到消息队列，而消息的路由规则由 ExchangeType 和 Binding 决定。

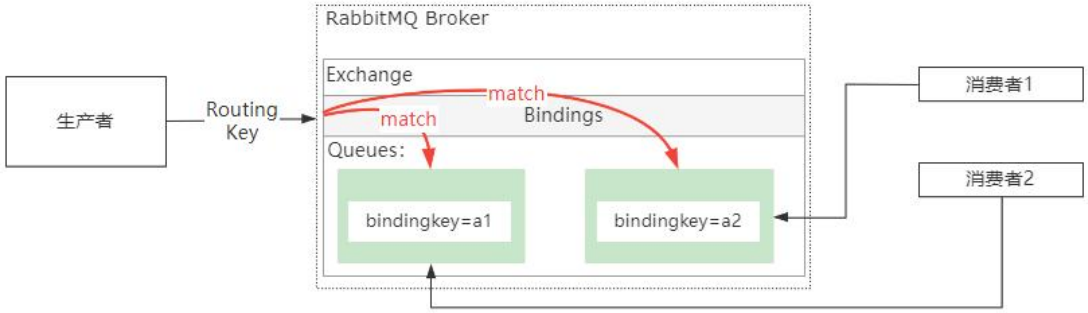
（如图）Binding 表示建立 Queue 和 Exchange 之间的绑定关系，每一个绑定关系会存在一个 BindingKey。

通过这种方式相当于在 Exchange 中建立了一个路由关系表。



（如图）生产者发送消息的时候，需要声明一个 routingKey（路由键），Exchange 拿到 routingKey 之后，

根据 RoutingKey 和路由表里面的 BindingKey 进行匹配，而匹配的规则是通过 ExchangeType 来决定的。



在 RabbitMQ 中，有三种类型的 Exchange：direct ，fanout 和 topic。

- direct： 完整匹配方式，也就是 Routing key 和 Binding Key 完全一致，相当于点对点的发送。
- fanout： 广播机制，这种方式不会基于 Routing key 来匹配，而是把消息广播给绑定到当前 Exchange 上的所有队列上。
- topic： 正则表达式匹配，根据 Routing Key 使用正则表达式进行匹配，符合匹配规则的 Queue 都会收到这个消息

以上就是我对这个问题的理解！

53.174 如何保证 RabbitMQ 的消息可靠传输

最近一段时间，很多粉丝反馈，投递了很多简历出去，没什么面试机会。

而且即便有面试机会，面试也基本同步过。

确实，今年的面试难度特别高，所以大家可以趁这段时间好好积累一下，为金九银十做好准备。

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

今天分享的问题是：如何保证 RabbitMQ 的消息可靠传输？

> 以下是高手的回答

好的，面试官，这个问题我从几个方面来说明。

首先，在 RabbitMQ 的整个消息传递过程中，有三种情况会存在丢失。

- 生产者把消息发送到 RabbitMQ Server 的过程中丢失
- RabbitMQ Server 收到消息后在持久化之前宕机导致数据丢失
- 消费端收到消息还没来得及处理宕机，导致 RabbitMQ Server 认为这个消息已签收

所以，我认为只需要从这三个纬度去保证消息的可靠性传输就行了。

从生产者发送消息的角度来说，RabbitMQ 提供了一个 Confirm（消息确认）机制，生产者发送消息到 Server 端以后，如果消息处理成功，Server 端会返回一个 ack 消息。客户端可以根据消息的处理结果来决定是否要做消息的重新发送，从而确保消息一定到达 RabbitMQ Server 上。

从 RabbitMQ Server 端来说，可以开启消息的持久化机制，也就是收到消息之后持久化到磁盘里面。

设置消息的持久化有两个步骤。

- 创建 Queue 的时候设置为持久化
- 发送消息的时候，把消息投递模式设置为持久化投递

不过虽然设置了持久化消息，但是有可能会出现，消息刷新到磁盘之前，RabbitMQ Server 宕机导致消息丢失的问题。

所以为了确保万无一失，需要结合 Confirm 消息确认机制一起使用。

从消费端的角度来说，我们可以把消息的自动确认机制修改成手动确认，也就是说消费端只有手动调用

消息确认方法才表示消息已经被签收。

这种方式可能会造成重复消费问题，所以这里需要考虑到幂等性的设计。

以上就是我对这个问题的理解！

53.175 请说一下 Netty 中 Reactor 模式的理解

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

今天一个工作了 6 年的粉丝，去美团面试的时候遇到一个比较有意思的问题。

应该大部分同学对这个领域都比较陌生，因为网络编程在实际开发中接触还是比较少的。

这个问题是：“请说一下 Netty 中 Reactor 模式的理解”？

以下是高手的回答

好的，面试官。

Reactor 其实是在 NIO 多路复用的基础上提出的一个高性能 IO 设计模式。

它的核心思想是把响应 IO 事件和业务处理进行分离，通过一个或者多个线程来处理 IO 事件。

然后把就绪的事件分发给业务线程进行异步处理。

Reactor 模型有三个重要的组件：

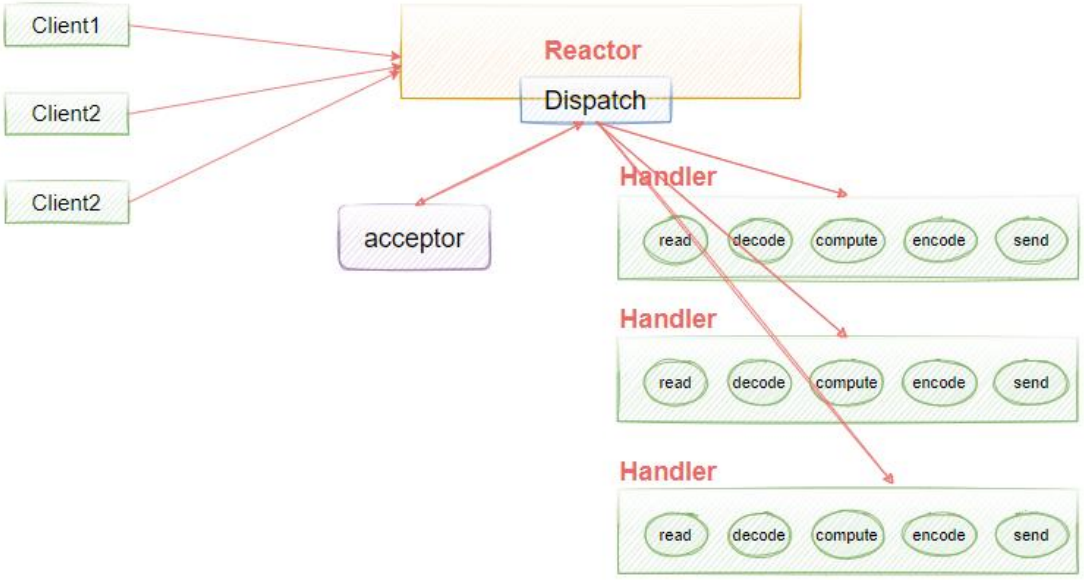
- **Reactor** ：把 I/O 事件分发给对应的 Handler
- **Acceptor** ：处理客户端连接请求
- **Handlers** ：执行非阻塞读/写，也就是针对收到的消息进行业务处理。

在 Reactor 的这种设计中，有三种模型分别是

- 单线程 Reactor 模型。
- 多线程 Reactor 模型。

- 主从多 Reactor 多线程模型。

（如图），单线程 Reactor 模型，就是由同一个线程来负责处理 IO 事件以及业务逻辑。这种方式的缺点在于 handler 的执行过程是串行，如果有任何一个 handler 处理线程阻塞，就会影响整个服务的吞吐量。



所以，就有了多线程 Reactor 模型（如图）。

也就是把处理 IO 就绪事件的线程和处理 Handler 业务逻辑的线程进行分离，

每个 Handler 由一个独立线程来处理，

在这种设计下，即便是存在 Handler 线程阻塞问题，也不会对 IO 线程造成影响。

在多线程 Reactor 模型下，所有的 IO 操作都是由一个 Reactor 来完成的，而且 Reactor 运行在单个线程里面。

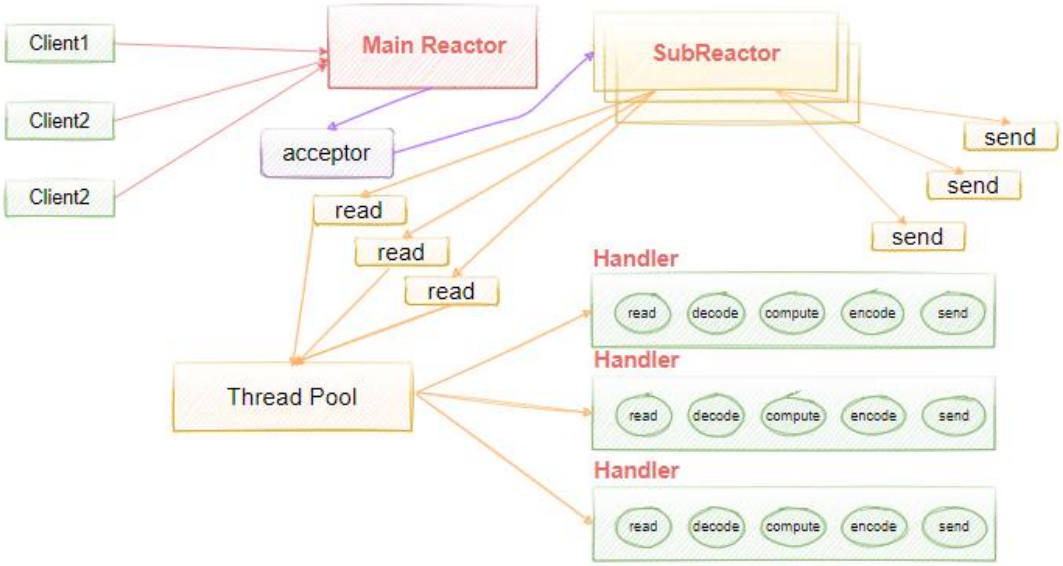
对于并发较高的场景下，Reactor 就成为了性能瓶颈，所以在这个基础上做了更进一步优化。

提出了多 Reactor 多线程模型（如图），这种模式也叫 Master-Workers 模式。

它把原本单个 Reactor 拆分成了 Main Reactor 和多个 SubReactor，

Main Reactor 负责接收客户端的链接，然后把接收到的连接请求随机分配到多个 subReactor 里面。

SubReactor 负责 IO 事件的处理。



这种方式另外一个好处就是可以对 subReactor 做灵活扩展，从而

适应不同的并发量，解决了单个 Reactor 模式的性能瓶颈问题。

以上就是我对 Reactor 模型的理解。

53.176 HashMap 中的 hash 方法为什么要右移 16 位异或?

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

今天一个工作 5 年的程序员，去面试的时候遇到一个非常奇怪的问题。

“HashMap 中的 hash 方法为什么要右移 16 位异或”。

他听到这个问题的时候完全懵了，在他的认知里面完全就没有这方面的概念。

> 以下是高手的回答

好的，面试官。

（如图）

```java
static final int hash(Object key) {
        int h;
        return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

之所以要对 hashCode 无符号右移 16 位并且异或，核心目的是为了让 hash 值的散列度更高，

尽可能减少 hash 表的 hash 冲突，从而提升数据查找的性能。

（如图）在 HashMap 的 put 方法里面，是通过 Key 的 hash 值与数组的长度取模计算得到数组的位置。

而在绝大部分的情况下，n 的值一般都会小于 2^16 次方，也就是 65536。

所以也就意味着 i 的值 ， 始终是使用 hash 值的低 16 位与(n-1)进行取模运算，这个是由与运算符&的特性决定的。

这样就会造成 key 的散列度不高，导致大量的 key 集中存储在固定的几个数组位置，很显然会影响到数据查找性能。

```java
final V putVal(int hash, K key, V value, boolean onlyIfAbsent,boolean evict) {
  Node<K,V>[] tab; Node<K,V> p; int n, i;
  //......
  if ((p = tab[i = (n - 1) & hash]) == null)
      tab[i] = newNode(hash, key, value, null);
  //......
}
```

因此，为了提升 key 的 hash 值的散列度，在 hash 方法里面，做了位移运算。

（如图）

首先使用 key 的 hashCode 无符号右移 16 位，意味着把 hashCode 的高位移动到了低位。

然后再用 hashCode 与右移之后的值进行异或运算，就相当于把高位和低位的特征进行和组合。

从而降低了 hash 冲突的概率。

```
static final int hash(Object key) {
        int h;
        return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

以上就是我对这个问题的理解！

53.177 DCL 单例模式设计为什么需要 volatile 修饰实例对象

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

今天给大家分享一道字节跳动的面试题。

“DCL 单例模式设计为什么需要 volatile 修饰实例对象”

这也是一个比较常见的问题，涉及到多线程领域关于指令重排序的知识。

以下是高手的回答

好的，面试官。

（如图）， 我所理解的 DCL 问题，是在基于双重检查锁设计下的单例模式中，存在不完整对象的问题。

而这个不完整对象的本质，是因为指令重排序导致的。

```
public class DCLExample {
    private static DCLExample instance;

    public static DCLExample getInstance(){
        if (instance==null){
            synchronized(DCLExample.class){
                if(instance==null){
                  instance = new DCLExample();
                }
            }
        }
        return instance;
    }
}
```

当我们使用 `instance=new DCLExample()`构建一个实例对象的时候，因为 new 这个操作并不是原子的。

所以这段代码最终会被编译成 3 条指令（如图）。

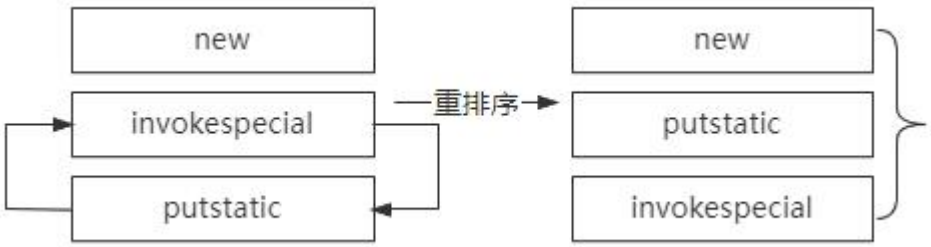
- 为对象分配内存空间
- 初始化对象
- 把实例对象赋值给 instance 引用

```
new  #3 <T>  //在java堆上为DCLExample对象分配内存空间，并将地址压入操作数栈顶
invokespecial #4  "<init>":()V  //调用实例初始化方法
putstatic    //把实例对象赋值给instance引用
```

由于这是三个指令并不是原子的（如图）。

按照重排序规则，在不影响单线程执行结果的情况下，两个不存在依赖关系的指令允许重排序，也就是不一定会

按照代码编写顺序来执行。



这样一来，(如图) 就会导致其他线程可能拿到一个不完整的对象，也就是这个 instance 已经分配了引用实例，

但是这个实例的初始化指令还没执行。

时间	线程A	线程B
t1	A1：为对象分配内存空间	
t2	A3：把实例对象赋值给instance引用	
t3		B1：判断instance是否为空
t4		B2：由于instance!=null，所以直接获取instance引用对象
t5	A2：初始化实例对象	
t6	A4：访问instance引用的对象	

解决办法就是可以在 instance 这个变量上增加一个 volatile 关键字修饰，

volatile 底层使用了内存屏障机制来避免指令重排序。

以上就是我对这个问题的理解。

53.178 说一下你对行锁、临键锁、间隙锁的理解

一个工作了 6 年的程序员，最近去阿里面试 p6 这个岗位。

面试之前信心满满的和我说，这次一定要拿下 35k 月薪的 offer。

然后，在第一面的时候，被 Mysql 里面的一个问题难住了。

hi，大家好，我是 Mic，一个没有才华只能靠颜值混饭吃的 Java 程序员。

今天给大家分享阿里的一道面试题，“说一下你对行锁、临键锁、间隙锁的理解”

以下是高手的回答

好的，面试官。

行锁、临键锁、间隙锁，都是 Mysql 里面 InnoDB 引擎下解决事务隔离性的一系列排他锁。

下面请允许我分别介绍一下这三种锁。

行锁，也称为记录锁。（如图）

当我们针对主键或者唯一索引加锁的时候，Mysql 默认会对查询的这一行数据加行锁，避免其他事务对这一行数据进行修改。

```
--其中id为主键索引
SELECT * FROM `test` WHERE `id`=1 FOR UPDATE;
```

间隙锁，顾名思义，就是锁定一个索引区间。

在普通索引或者唯一索引列上，由于索引是基于 B+树的结构存储，所以默认会存在一个索引区间。（如图），

而间隙锁，就是某个事物对索引列加锁的时候，默认锁定对应索引的左右开区间范围。

```sql
CREATE TABLE `test` (
  `id` int(1) NOT NULL AUTO_INCREMENT,
  `name` varchar(8) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;

INSERT INTO `test` VALUES ('1', '小A');
INSERT INTO `test` VALUES ('5', '小B');
INSERT INTO `test` VALUES ('7', '小C');
INSERT INTO `test` VALUES ('11', '小D');
```

默认存在的间隙锁区间

$$(-\infty, 1)\ ;\ (1, 5)\ ;\ (5, 7)\ ;\ (7, 11)\ ;\ (11, +\infty)$$

在基于索引列的范围查询，无论是否是唯一索引，都会自动触发间隙锁。

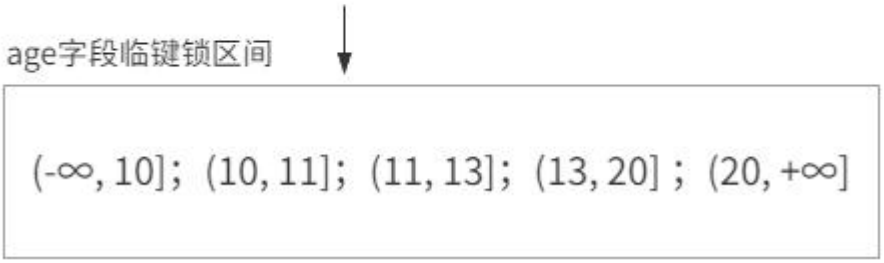
比如基于 between 的范围查询，就会产生一个左右开区间的间隙锁。

```sql
--锁定：(5, 7)
SELECT * FROM `test` WHERE `id` BETWEEN 5 AND 7 FOR UPDATE;
```

最后一个是临键锁，（如图）它相当于行锁+间隙锁的组合，也就是它的锁定范围既包含了索引记录，也包含了索引区间

它会锁定一个左开右闭区间的数据范围。

```sql
CREATE TABLE `test`(
 `id` int(1) NOT NULL AUTO_INCREMENT,
 `name` varchar(8) DEFAULT NULL,
  `age` int(4) NOT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_age` (age)
) ENGINE=INNODB DEFAULT CHARSET=utf8;

INSERT INTO `test` VALUES(1,'小A',10);
INSERT INTO `test` VALUES(2,'小B',11);
INSERT INTO `test` VALUES(3,'小C',13);
INSERT INTO `test` VALUES(4,'小D',20);
```



（如图）假设我们使用非唯一索引列进行查询的时候，默认会加一个临键锁，锁定一个左开右闭区间的范围。

```sql
-- 临键锁，锁定区间 (10,11]
select * from test where age=11 for update;
```

所以总的来说，行锁、临键锁、间隙锁只是表示锁定数据的范围，最终目的是为了解决幻读的问题。

而临键锁相当于行锁+间隙锁，因此当我们使用非唯一索引进行精准匹配的时候，会默认加临键锁，

因为它需要锁定匹配的这一行数据，还需要锁定这一行数据对应的左开右闭区间。

因此在实际应用中，尽可能使用唯一索引或者主键索引进行查询，避免大面积的锁定造成性能影响。

以上就是我对这个问题的理解。

53.179 生产环境服务器变慢，如何诊断处理？

“生产环境服务器变慢？如何诊断处理”

这是最近一些工作 5 年以上的粉丝反馈给我的问题，他们去一线大厂面试，都被问到了这一类的问题。

Hi，大家好， 我是 Mic，一个工作了 14 年的 Java 程序员。

今天给大家分享一下，面试过程中遇到这个问题，我们应该怎么回答。

以下是高手的回答

好的，面试官。

生产环境服务器处理效率变慢，我认为主要会涉及到三个纬度：

- CPU 的利用率
- 磁盘 IO 效率
- 内存

CPU 利用率过高或者 CPU 利用率过低，都会影响程序的处理效率。

利用率过高，说明当前服务器要处理的指令比较多，当 CPU 忙不过来的时候，指令的运算效率自然就会下降。

反馈在用户上的感受就是程序响应变慢了。

针对这个问题，我们可以使用 top 命令查询当前系统中占用 CPU 过高的进程，以及定位到这个进程中比较活跃的线程。

再通过 jstack 命令打印当前虚拟机的线程快照，然后根据快照日志排查问题代码。

如果 CPU 利用率过低，说明程序资源使用不够，可以增加线程数量提升程序性能。

程序运算过程中，会直接或者间接涉及到一些磁盘 IO 相关的操作，比如程序直接读写磁盘，

或者程序依赖的第三方组件涉及到磁盘的持久化存储，所以磁盘的 IO 效率也会对程序运行效率产生影响。

针对这个情况，可以使用 iostat 命令查看，如果磁盘负载较高，可以针对性的进行优化，比如

- 借助缓存系统，减少磁盘 IO 次数
- 用顺序写替代随机写入，减少寻址开销
- 使用 mmap 替代 read/write，减少内存拷贝次数

另外，系统 IO 的瓶颈可以通过 CPU 和负载的非线性关系体现出来。当负载增大时，系统吞吐量不能有效增大，

CPU 不能线性增长，其中一种可能是 IO 出现阻塞。

最后，就是内存的瓶颈，内存作为一块临时存储数据的组件，所有 CPU 运算的指令都需要从内存中去读写。

内存的合理使用，可以减少应用和磁盘的直接 IO 频率，以及减少网络 IO 的频率，极大提升 IO 性能。

其次，作为 Java 应用程序的运行平台 JVM，对于内存的合理分配，能够避免频繁的 YGC 和 FULL GC。

内存使用率比较高的时候， 可以 dump 出 JVM 堆内存，然后借助 MAT 工具进行分析，

查出大对象或者占用最多的对象，以及排查是否存在内存泄漏的问题。

如果 dump 出的堆内存文件正常，此时可以考虑堆外内存被大量使用导致出现问题，需要借助操作系统指令 pmap 查出进程的内存分配情况。

如果 CPU 和 内存使用率都很正常，那就需要进一步开启 GC 日志，分析用户线程暂停的时间、

各部分内存区域 GC 次数和时间等指标，可以借助 jstat 或可视化工具 GCeasy 等，如果问题出在 GC 上面的话，考虑是否是内存不够、根据垃圾对象的特点进行参数调优、使用更适合的垃圾收集器；

分析 jstack 出来的各个线程状态。如果问题实在比较隐蔽，考虑是否可以开启 jmx，使用 visualmv 等可视化工具远程监控与分析。

53.180 HashMap 啥时候扩容，为什么扩容?

“HashMap 啥时候扩容，为什么扩容？”

这是一个针对 1 到 3 年左右 Java 开发人员的面试题，

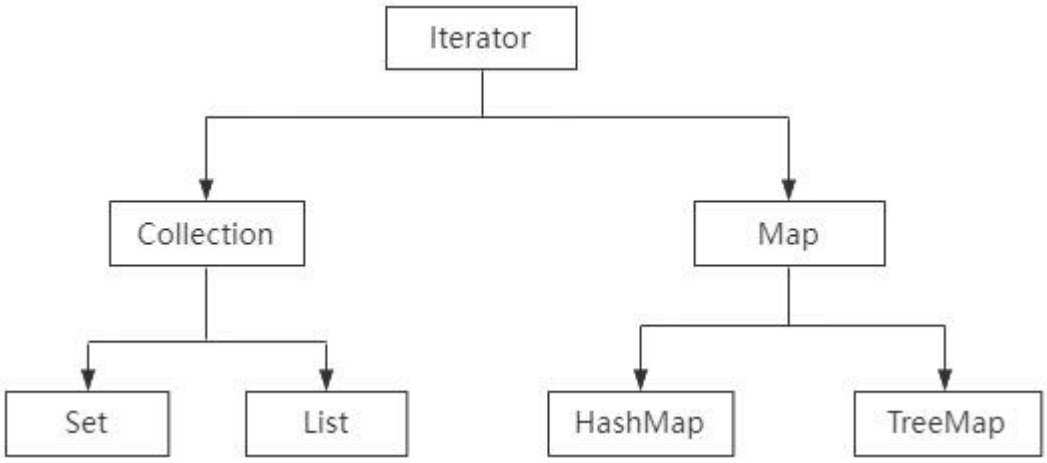
问题本身不是很难，但是对于这个阶段粉丝来说，由于不怎么关注

所以会难住一部分同学。

以下是高手的回答

在任何语言中，我们希望在内存中临时存放一些数据，可以用一些官方封装好的集合（如图），

比如 List、HashMap、Set 等等。作为数据存储的容器。



> 容器的大小

当我们创建一个集合对象的时候，实际上就是在内存中一次性申请一块内存空间。

而这个内存空间大小是在创建集合对象的时候指定的。

比如 List 的默认大小是 10、HashMap 的默认大小是 16。

> 长度不够怎么办

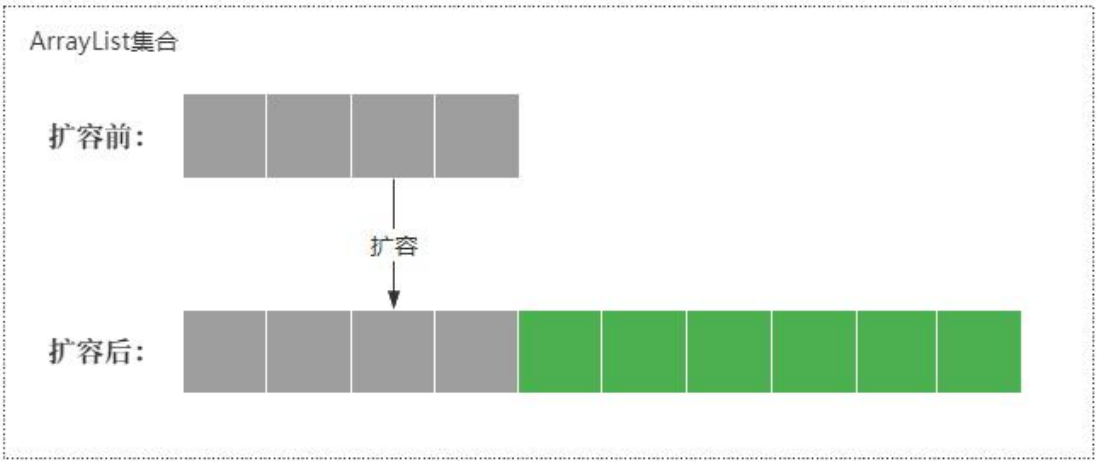
在实际开发中，我们需要存储的数据量往往大于存储容器的大小。

针对这种情况，通常的做法就是扩容。

当集合的存储容量达到某个阈值的时候，集合就会进行动态扩容，从而更好的满足更多数据的存储。

（如图）List 和 HashMap，本质上都是一个数组结构，所以基本上只需要新建一个更长的数组

然后把原来数组中的数据拷贝到新数组就行了。



以 HashMap 为例，它是什么时候触发扩容以及扩容的原理是什么呢?

> HashMap 是如何扩容的?

当 HashMap 中元素个数超过临界值时会自动触发扩容，这个临界值有一个计算公式。

threashold=loadFactor*capacity（屏幕显示）。

loadFactor 的默认值是 0.75，capacity 的默认值是 16，也就是元素个数达到 12 的时候触发扩容。

扩容后的大小是原来的 2 倍。

由于动态扩容机制的存在，所以我们在实际应用中，需要注意在集合初始化的时候明确指定集合的大小。

避免频繁扩容带来性能上的影响。

假设我们要向 HashMap 中存储 1024 个元素，如果按照默认值 16，随着元素的不断增加，会造成 7 次扩容。

而这 7 次扩容需要重新创建 Hash 表，并且进行数据迁移，对性能影响非常大。

最后，可能有些面试官会继续问，为什么扩容因子是 0.75?

> 为什么扩容因子是 0.75?

扩容因子表示 Hash 表中元素的填充程度，扩容因子的值越大，那么触发扩容的元素个数更多，

虽然空间利用率比较高，但是 hash 冲突的概率会增加。

扩容因子的值越小，触发扩容的元素个数就越少，也意味着 hash 冲突的概率减少，

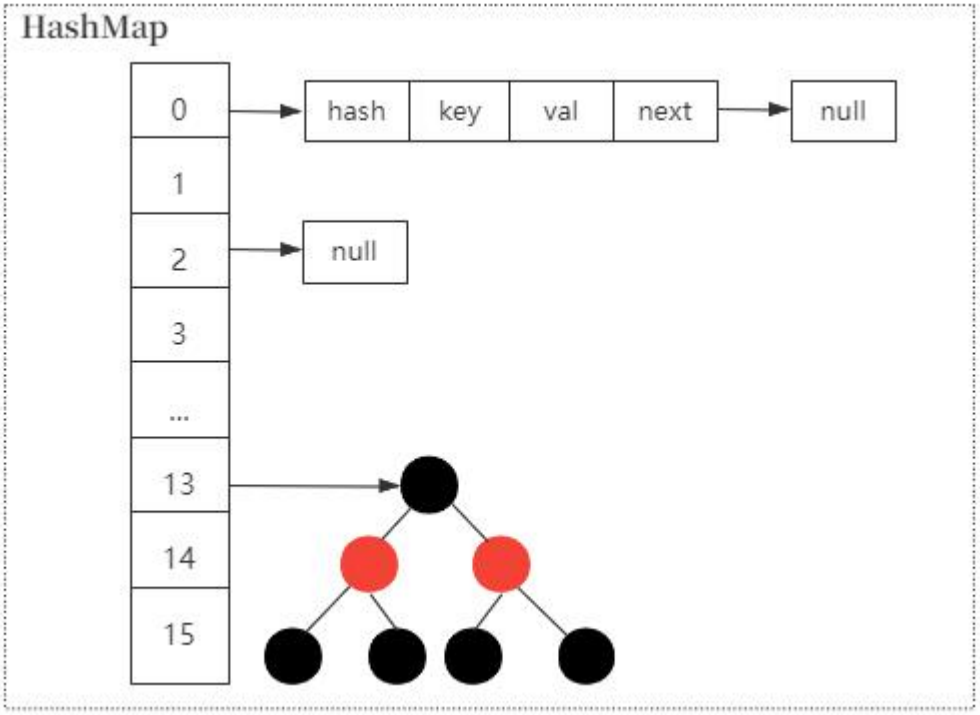
但是对内存空间的浪费就比较多，而且还会增加扩容的频率。

因此，扩容因子的值的设置，本质上就是在 冲突的概率 以及 空间利用率之间的平衡。

0.75 这个值的来源，和统计学里面的泊松分布有关。

（如图）我们知道，HashMap 里面采用链式寻址法来解决 hash 冲突问题，为了避免链表过长带来时间复杂度的增加

所以链表长度大于等于 7 的时候，就会转化为红黑树，提升检索效率。



当扩容因子在 0.75 的时候，链表长度达到 8 的可能性几乎为 0，也就是比较好的达到了空间成本和时间成本的平衡。

以上就是关于这个问题的完整理解。

在面试的时候，我们可以这么回答。

> 面试题的标准回答

当 HashMap 元素个数达到扩容阈值，默认是 12 的时候，会触发扩容。

默认扩容的大小是原来数组长度的 2 倍，HashMap 的最大容量是 Integer.MAX_VALUE，也就是 2 的 31 次方-1。

53.181 讲下线程池的线程回收

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

最近很多小伙伴私信我，让我说一些线程池相关的问题。

线程池这个方向考察的点还挺多的，如果只是靠刷面试题

面试官很容易就能识别出来，我随便举几个。

- 线程池是如何实现线程的回收的
- 核心线程是否能够回收
- 当调用线程池的 shutdown 方法，会发生什么?

面试一定是连环问，从而确定求职者对这个领域的理解程度。

以下是高手的回答

好的，面试官，这个问题我需要从 3 个方面来回答。

首先，线程池里面分为核心线程和非核心线程。

核心线程是常驻在线程池里面的工作线程，它有两种方式初始化。

- 向线程池里面添加任务的时候，被动初始化
- 主动调用 `prestartAllCoreThreads` 方法

当线程池里面的队列满了的情况下，为了增加线程池的任务处理能力。

线程池会增加非核心线程。

核心线程和非核心线程的数量，是在构造线程池的时候设置的，也可以动态进行更改。

由于非核心线程是为了解决任务过多的时候临时增加的，所以当任务处理完成后，

工作线程处于空闲状态的时候，就需要回收。

因为所有工作线程都是从阻塞队列中去获取要执行的任务，所以只要在一定时间内，

阻塞队列没有任何可以处理的任务，那这个线程就可以结束了。

这个功能是通过阻塞队列里面的 poll 方法来完成的。这个方法提供了超时时间和超时时间单位这两个参数

当超过指定时间没有获取到任务的时候，poll 方法返回 null，从而终止当前线程完成线程回收。

默认情况下，线程池只会回收非核心线程，如果希望核心线程也要回收，可以

设置 allowCoreThreadTimeOut 这个属性为 true，一般情况下我们不会去回收核心线程。

因为线程池本身就是实现线程的复用，而且这些核心线程在没有任务要处理的时候是处于阻塞状态

并没有占用 CPU 资源。

以上就是我对这个问题的理解。

53.182 索引什么时候失效?

“索引什么时候失效？”

面试过程中，突如其来的一个问题，是不是有点懵?

没关系，关注我，面试不迷路。

我是 Mic，一个工作了 14 年的 Java 程序员。

索引失效涉及到的知识点非常多，所以我把这个问题的回答整理到了

以下是高手的回答

好的，面试官。

InnoDB 引擎里面有两种索引类型，一种是主键索引、一种是普通索引。

InnoDB 用了 B+树的结构来存储索引数据。

当使用索引列进行数据查询的时候，最终会到主键索引树中查询对应的数据行进行返回。

理论上来说，使用索引列查询，就能很好的提升查询效率，但是不规范的使用

会导致索引失效，从而无法发挥索引本身的价值。

导致索引失效的情况有很多：

1. 在索引列上做运算，比如使用函数，Mysql 在生成执行计划的时候，它是根据统计信息来判断是否要使用索引的。

而在索引列上加函数运算，导致 Mysql 无法识别索引列，也就不会再走索引了。

不过从 Mysql8 开始，增加了函数索引可以解决这个问题。

2. 在一个由多列构成的组合索引中，需要按照最左匹配法则，也就是从索引的最左列开始顺序检索，否则不会走索引。

在组合索引中，索引的存储结构是按照索引列的顺序来存储的，因此在 sql 中也需要按照这个顺序才能进行逐一匹配。

否则 InnoDB 无法识别索引导致索引失效。

3. 当索引列存在隐式转化的时候， 比如索引列是字符串类型，但是在 sql 查询中没有使用引号。

那么 Mysql 会自动进行类型转化，从而导致索引失效

4. 在索引列使用不等于号、`not` 查询的时候，由于索引数据的检索效率非常低，因此 Mysql 引擎会判断不走索引。

5. 使用 like 通配符匹配后缀`%xxx`的时候，由于这种方式不符合索引的最左匹配原则，所以也不会走索引。

但是反过来，如果通配符匹配的是前缀 `xxx%`，符合最左匹配，也会走索引。

6. 使用 `or` 连接查询的时候，`or` 语句前后没有同时使用索引，那么索引会失效。只有 `or` 左右查询字段都是索引列的时候，才会生效。

除了这些场景以外，对于多表连接查询的场景中，连接顺序也会影响索引的使用。

不过最终是否走索引，我们可以使用 explain 命令来查看 sql 的执行计划，然后针对性的进行调优即可。

以上就是我对这个问题的理解。

53.183 怎么防止缓存击穿的问题?

“怎么防止缓存击穿？”

这是很多一二线大厂面试的时候考察频率较高的问题。

在并发量较高的系统中，缓存可以提升数据查询的性能，还能缓解后端存储系统的并发压力。

可谓是屡试不爽的利器。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

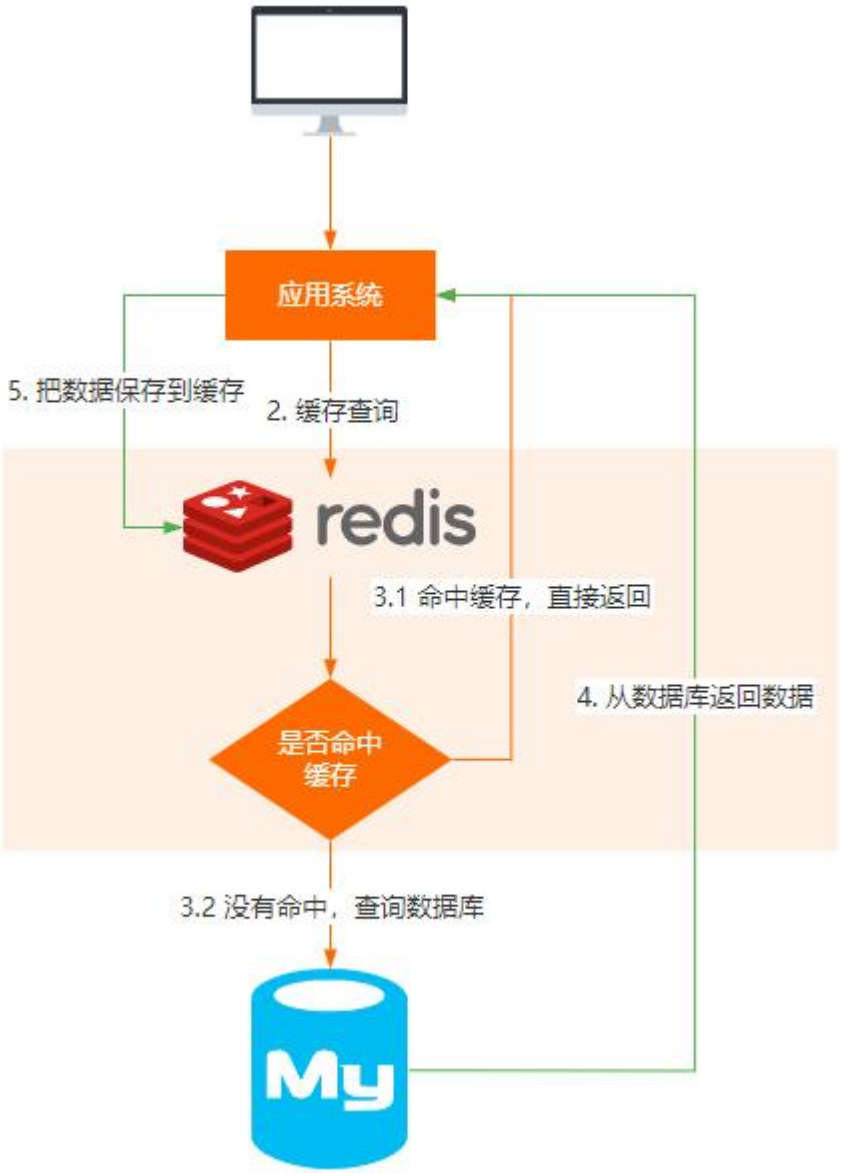
> 以下是高手的回答

好的，面试官。

在实际应用中，我们会在程序和数据库之间增加一个缓存层（如图）。

一方面是为了提升数据检索效率，提升程序性能

另一方面是为了缓解数据库的并发压力。



缓存击穿，表示请求因为某些原因全部打到了数据库，缓存并没有起到流量缓冲的作用。

我认为有 2 种情况会导致缓存击穿。

- 在 Redis 里面保存的热点 key，在缓存过期的瞬间，有大量请求进来，导致请求全部打在数据库上。

- 客户端恶意发起大量不存在的 key 的请求，由于访问的 key 对应的数据本身就不存在，

所以每次必然都会穿透到数据库，导致缓存成为了摆设。

总之，当 Redis 承担了流量缓冲功能的时候，就需要考虑到 Redis 失效导致并发压力过大对后端存储设备造成冲击的问题。

因此，我认为可以通过几种方法来解决。

1. 对于热点数据，我们可以不设置过期时间，或者在访问数据的时候对数据过期时间进行续期。

2. 对于访问量较高的缓存数据，我们可以设计多级缓存，尽量减少后端存储设备的压力。

3. 使用分布式锁，当发现缓存失效的时候，不是先从数据库加载，而是先获取分布式锁，

   获得分布式锁的线程从数据库查询数据后写回到缓存里面。

   后续没有获得锁的线程就只需要等待和重试即可。

   这个方案牺牲了一定的性能，但是确保护了数据库避免被压垮。

4. 对于恶意攻击类的场景，可以使用布隆过滤器，应用启动的时候把存在的数据缓存到布隆过滤器里面。

每一次请求进来的时候先访问布隆过滤器，

如果不存在，则说明这个数据一定没有在数据库里面，就没必要再去访问数据库了。

另外，我们在整个缓存架构设计中，除了尽可能避免缓存穿透的问题，还需要从全局视角做整体考虑

比如业务隔离、多级缓存、部署隔离、安全性考虑等。

以上就是我对这个问题的理解。

53.184 强引用、软引用、弱引用、虚引用有什么区别?

“强引用、软引用、弱引用、虚引用有什么区别？”

这个问题难倒了很多资深 Java 工程师，不是因为这个问题本身有多难。

而是确实它是一个比较小众的知识点。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

今天给大家分享一下这道面试题的标准回答。

以下是高手的回答

好的，面试官。

不同的引用类型，主要体现的是对象不同的可达性状态和对垃圾收集的影响。

强引用，就是普通对象的引用，只要还有强引用指向一个对象，就能表示对象还“活着”，

垃圾收集器无法回收这一类对象。

只有在没有其他引用关系，或者超过了引用的作用域，再或者显示的把引用赋值为 null 的时候，

垃圾回收器才能进行内存回收。

软引用，是一种相对强引用弱化一些的引用，可以让对象豁免一些垃圾收集，

只有当 JVM 认为内存不足时，才会去试图回收软引用指向的对象。

软引用通常用来实现内存敏感的缓存，如果还有空闲内存，就可以暂时保留缓存，

当内存不足时清理掉，这样就保证了使用缓存的同时，不会耗尽内存。

弱引用，相对强引用而言，它允许在存在引用关联的情况下被垃圾回收的对象

在垃圾回收器线程扫描它所管辖的内存区域的过程中，

一旦发现了只具有**弱引用**的对象，不管当前**内存空间足够与否**，垃圾回收期都会**回收**该内存

虚引用，它不会决定对象的生命周期，它提供了一种确保对象被 finalize 以后，去做某些事情的机制。

当垃圾回收器准备回收一个对象时，如果发现它还有虚引用，

就会在回收对象的内存之前，把这个虚引用加入到与之关联的引用队列中。

程序可以通过判断引用**队列**中是否已经加入了**虚引用**，

来了解被引用的对象是否将要进行**垃圾回收**，然后我们就可以在引用的对象的内存回收之前采取必要的行动。

以上就是我对这个问题的理解！

53.185 什么是 IO 的多路复用机制?

“什么是 IO 的多路复用机制？”

这是一道年薪 50W 的面试题，很遗憾，99%的人都回答不出来。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

今天，给大家分享一道网络 IO 的面试题。

以下是高手的回答

好的，面试官。

IO 多路复用机制，核心思想是让单个线程去监视多个连接，一旦某个连接就绪，也就是触发了读/写事件。

就通知应用程序，去获取这个就绪的连接进行读写操作。

也就是在应用程序里面可以使用单个线程同时处理多个客户端连接，

在对系统资源消耗较少的情况下提升服务端的链接处理数量。

在 IO 多路复用机制的实现原理中，

客户端请求到服务端后，此时客户端在传输数据过程中（如图），

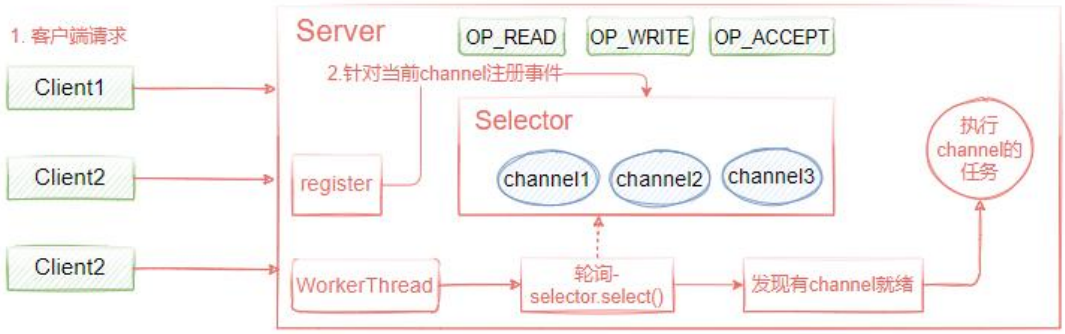
为了避免 Server 端在 read 客户端数据过程中阻塞，服务端会把该请求注册到 Selector 复路器上，

服务端此时不需要等待，只需要启动一个线程，

通过 selector.select()阻塞轮询复路器上就绪的 channel 即可，

也就是说，如果某个客户端连接数据传输完成，

那么 select()方法会返回就绪的 channel，然后执行相关的处理就可以了。



常见的 IO 多路复用机制的实现方式有： select 、poll、epoll。

这些都是 Linux 系统提供的 IO 复用机制的实现，其中 select 和 poll 是基于轮询的方式去获取就绪连接。

而 epoll 是基于事件驱动的方式获取就绪连接。从性能的角度来看，基于事件驱动的方式要优于轮询的方式。

以上就是我对这个问题的理解。

53.186 Java 有几种文件拷贝方式，哪一种效率最高?

“Java 有几种文件拷贝方式，哪一种效率最高？”

屏幕前的你知道吗?

这个问题是京东一面的时候，针对 4 年经验的同学的一个面试题。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

以下是高手的回答

好的，面试官。

第一种，使用 java.io 包下的库，使用 FileInputStream 读取，再使用 FileOutputStream 写出。

第二种，利用 java.nio 包下的库，使用 transferTo 或 transfFrom 方法实现。

第三种，Java 标准类库本身已经提供了 Files.copy 的实现。

对于 Copy 的效率，这个其实与操作系统和配置等情况相关，

在传统的文件 IO 操作里面，我们都是调用操作系统提供的底层标准 IO 系统调用函数 read()、write() ，

由于内核指令的调用会使得当前用户线程切换到内核态，然后内核线程负责把相应的文件数据读取到内核的 IO 缓冲区，

再把数据从内核IO缓冲区拷贝到进程的私有地址空间中去，这样便完成了一次IO操作。

而 NIO 里面提供的 NIO transferTo 和 transfFrom 方法，也就是常说的零拷贝实现。

它能够利用现代操作系统底层机制，避免不必要拷贝和上下文切换，因此在性能上表现比较好。

以上就是我对这个问题的理解。

53.187 聊聊你知道的设计模式

“聊聊你知道的设计模式！”

这个问题很简单，但是要让面试官认可你的回答，那还是得花点心思。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

今天给大家分享一下，当遇到这种比较泛类型的问题的时候，如何回答才能让面试官满意。

以下是高手的回答

大致按照模式的应用目标分类，设计模式可以分为创建型模式、结构型模式和行为型模式。

创建型模式，是对对象创建过程的各种问题和解决方案的总结，包括各种工厂模式、单例模式、构建器模式、原型模式。

结构型模式，是针对软件设计结构的总结，关注于类、对象继承、组合方式的实践经验。

常见的结构型模式，包括桥接模式、适配器模式、装饰者模式、代理模式、组合模式、外观模式、享元模式等。

行为型模式，是从类或对象之间交互、职责划分等角度总结的模式。

比较常见的行为型模式有策略模式、解释器模式、命令模式、观察者模式、迭代器模式、模板方法模式、访问者模式。

以上就是我对这个问题的理解。

53.188 如果一个线程两次调用 start()，会出现什么问题？

“如果一个线程两次调用 start()，会出现什么问题？”

如果这个问题出自阿里 p6 岗位第一面的提问，你能回答出来吗？

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

以下是高手的回答

好的，面试官。

在 Java 里面，一个线程只能调用一次 start()方法，第二次调用会抛出 IllegalThreadStateException。

一个线程本身是具备一个生命周期的。

在 Java 里面，线程的生命周期包括 6 种状态。

- NEW，线程被创建还没有调用 start 启动
- RUNNABLE，在这个状态下的线程有可能是正在运行，也可能是在就绪队列里面等待操作系统进行调度分配 CPU 资源。
- BLOCKED，线程处于锁等待状态
- WAITING，表示线程处于条件等待状态，当触发条件后唤醒，比如 wait/notify。
- TIMED_WAIT，和 WAITING 状态相同，只是它多了一个超时条件触发
- TERMINATED，表示线程执行结束

当我们第一次调用 start()方法的时候，线程的状态可能处于终止或者非 NEW 状态下的其他状态。

再调用一次 start()，相当于让这个正在运行的线程重新运行，不管从线程的安全性角度，还是从线程本身的执行逻辑，都是不合理的。

因此为了避免这个问题，在线程运行的时候会先判断当前线程的运行状态。

以上就是我对这个问题的理解。

53.189 Java 官方提供了哪几种线程池，分别有什么特点？

“Java 官方提供了哪几种线程池，分别有什么特点？”

这是一道针对工作 3 年左右的面试题，屏幕前的小伙伴，你能回答上来吗？

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

以下是高手的回答

好的。

JDK 中幕刃提供了 5 中不同线程池的创建方式，

下面我分别说一下每一种线程池以及它的特点。

- newCachedThreadPool， 是一种可以缓存的线程池，它可以用来处理大量短期的突发流量。

它的特点有三个，最大线程数是 Integer.MaxValue，线程存活时间是 60 秒，
阻塞队列用的是 SynchronousQueue，这是一种不存才任何元素的阻塞队列，
也就是每提交一个任务给到线程池，都会分配一个工作线程来处理，由于最大线程数没有限制。
所以它可以处理大量的任务，另外每个工作线程又可以存活 60s，使得这些工作线程可以缓存起来
应对更多任务的处理。

- newFixedThreadPool，是一种固定线程数量的线程池。

它的特点是核心线程和最大线程数量都是一个固定的值
如果任务比较多工作线程处理不过来，就会加入到阻塞队列里面等待。

- newSingleThreadExecutor，只有一个工作线程的线程池。

并且线程数量无法动态更改，因此可以保证所有的任务都按照 FIFO 的方式顺序执行。

- newScheduledThreadPool，具有延迟执行功能的线程池

可以用它来实现定时调度

- newWorkStealingPool，Java8 里面新加入的一个线程池

它内部会构建一个 ForkJoinPool，利用工作窃取的算法并行处理请求。

这些线程都是通过工具类 Executors 来构建的，
线程池的最终实现类是 ThreadPoolExecutor。
以上就是我对这个问题的理解。

53.190 请你说一下你对 Happens-Before 的理解。

“请你说一下你对 Happens-Before 的理解”

屏幕前的你，听到这个问题的时候，知道怎么回答吗?

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

并发编程是面试过程中重点考察的方向，能够考察的方向有很多

以下是高手的回答

好的，这个问题我需要从几个方面来回答。

首先，Happens-Before 是一种可见性模型，也就是说，在多线程环境下。

原本因为指令重排序的存在会导致数据的可见性问题，也就是 A 线程修改某个共享变量对 B 线程不可见。

因此，JMM 通过 Happens-Before 关系向开发人员提供跨越线程的内存可见性保证。

如果一个操作的执行结果对另外一个操作可见，那么这两个操作之间必然存在 Happens-Before 管理。

其次，Happens-Before 关系只是描述结果的可见性，并不表示指令执行的先后顺序，也就是说

只要不对结果产生影响，仍然允许指令的重排序。

最后，在 JMM 中存在很多的 Happens-Before 规则。

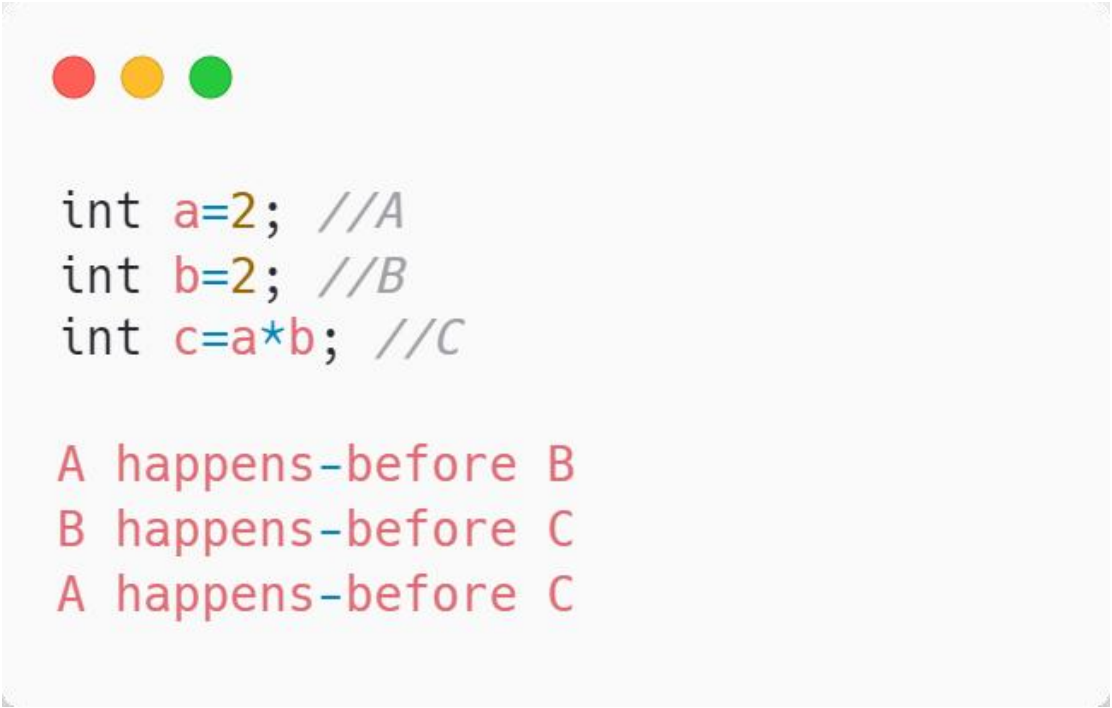
- 程序顺序规则，一个线程中的每个操作，

happens-before 这个线程中的任意后续操作，可以简单认为是 as-if-serial

也就是不管怎么重排序，单线程的程序的执行结果不能改变

- 传递性规则（如图），也就是 A Happens-Before B，B Happens-Before C。

就可以推导出 A Happens-Before C。



- volatile 变量规则，对一个 volatile 修饰的变量的写一定 happens-before 于任意后续对这个 volatile 变量的读操作
- 监视器锁规则（如图），一个线程对于一个锁的释放锁操作，

一定 happens-before 与后续线程对这个锁的加锁操作

在这个场景中，如果线程 A 获得了锁并且把 x 修改成了 12，那么后续的线程获得锁之后

得到的 x 的值一定是 12.

```
int x=10;
synchronized (this) { // 此处自动加
锁
 // x 是共享变量, 初始值 =10
 if (this.x < 12) {
    this.x = 12;
  }
} // 此处自动解锁
```

- 线程启动规则（如图），如果线程 A 执行操作 ThreadB.start(),那么线程 A 的 ThreadB.start()之前的操作 happens-before 线程 B 中的任意操作。

在这样一个场景中，t1 线程启动之前对于 x=10 的赋值操作，t1 线程启动以后读取 x 的值一定是 10.

```
public StartDemo{
    int x=0;
    public void test(){
      Thread t1 = new Thread(()->{
        // 主线程调用 t1.start() 之前
        // 所有对共享变量的修改，此处皆可见
        // 此例中，x==10
      });
      // 此处对共享变量 x修改
      x = 10;
      // 主线程启动子线程
      t1.start();
    }
}
```

- join 规则（如图），如果线程 A 执行操作 ThreadB.join()并成功返回，

那么线程 B 中的任意操作 happens-before 于线程 A 从 ThreadB.join()操作成功的返回。

```
Thread t1 = new Thread(()->{
  // 此处对共享变量 x 修改
  x= 100;
});
// 例如此处对共享变量修改，
// 则这个修改结果对线程 t1 可见
// 主线程启动子线程
t1.start();
t1.join()
// 子线程所有对共享变量的修改
// 在主线程调用 t1.join() 之后皆可见
// 此例中，x==100
```

以上就是我对 Happens-Before 的理解。

53.191 谈谈常用的分布式 ID 设计方案

“谈谈常用的分布式 ID 设计方案”！

一个工作了 7 年的同学，被问到了这样一个问题。

问题并不难，但是在实际面试的时候，如果只是回答 1，2，3

很难通过面试，因为作为一个高级程序员，还需要有自己的理解和思考。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

以下是高手的回答

好的，这个问题我需要从几个方面来回答。

首先，分布式全局 ID 的的解决方案有很多，比如：

- 使用 Mysql 的全局表
- 使用 Zookeeper 的有序节点
- 使用 MongoDB 的 objectid
- redis 的自增 id

- UUID 等等
- ……

这些方案只是解决基础的 id 唯一性问题，在实际生产环境中，需要构建一个全局唯一 id

还需要考虑更多的因素：

- 有序性， 有序的 ID 能够更好的确认数据的位置，

以及 B+数的存储结构中，范围查询的效率更高，并且可以提升 B+树数据维护的效率。

- 安全性，避免恶意爬去数据造成数据泄露
- 可用性，ID 生成系统的可用性要求非常高，一旦出现故障就会造成业务不可用的问题
- 性能，全局 id 生成系统需要满足整个公司的业务需求，涉及到亿级别的调用，对性能要求较高

因此，在如果我们选择数据库的全局表，你没获取一次 id 就需要更新数据库，性能上限比较明显，

而且基于数据库构建高扩展和高性能的解决方案难度很大。

所以，目前市面上主流的解决方案是基于 Twitter 早期开源的 Snowflake 雪花算法（图片）。

它是由 64 位长度组成的全局 id 生成算法，通过对 64 位进行区间划分来表述不同含义实现唯一性。

它的好处是：

- 算法实现简单
- 不存在太多外部依赖
- 可以生成有意义的有序编号
- 基于位运算，性能也很好，Twitter 测试的峰值是 10 万个每秒。

另外，美团公司开源了一个全局唯一 id 生成系统 leaf，它里面也用到了雪花算法去构建全局唯一 id

并且在高性能和高可用方面，做了很多的优化，为美团内部业务提供了每天上亿次的调用。

以上就是我对这个问题的理解。

53.192 线程池是如何实现线程复用的?

“线程池是如何实现线程复用的？”

一个工作 2 年的 Java 程序员，在面试互联网公司的时候被这个问题难住了。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

多线程是非常重要的一个技术领域，在实际开发中使用比较多。

而线程池是属于线程的复用技术，因此对于这个问题，我把

以下是高手的回答

好的，面试官。

线程池里面采用了生产者消费者的模式，来实现线程复用。

生产者消费者模型，其实就是通过一个中间容器来解耦生产者和消费者的任务处理过程。

生产者不断生产任务保存到容器，消费者不断从容器中消费任务。

在线程池里面，因为需要保证工作线程的重复使用，

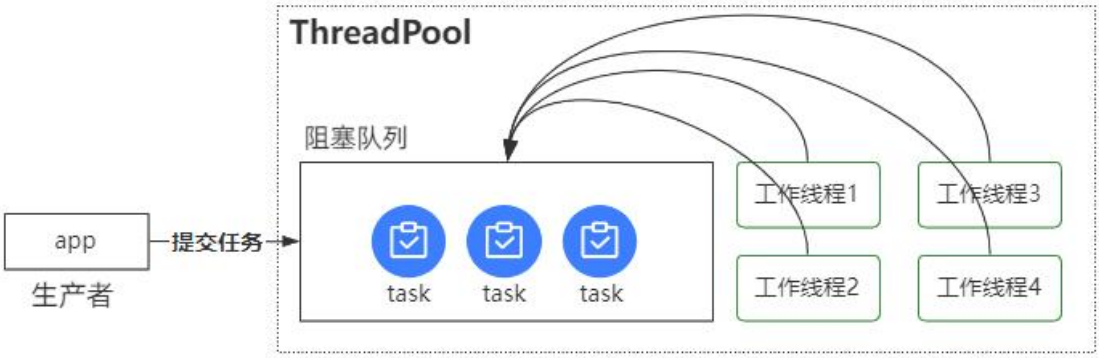
并且这些线程应该是有任务的时候执行，没任务的时候等待并释放 CPU 资源。

因此（如图），它使用了阻塞队列来实现这样一个需求。

提交任务到线程池里面的线程称为生产者线程，它不断往线程池里面传递任务。

这些任务会保存到线程池的阻塞队列里面。

然后线程池里面的工作线程不断从阻塞队列获取任务去执行。



基于阻塞队列的特性，使得阻塞队列中如果没有任务的时候，这些工作线程就会阻塞等待。

直到又有新的任务进来，这些工作线程再次被唤醒。

从而达到线程复用的目的，以上就是我对这个问题的理解！

53.193 可以说下阻塞队列被异步消费怎么保持顺序吗？

你知道“阻塞队列被异步消费是如何保证消费顺序的吗？”

Hi，大家好，我是 Mic，一个工作 14 年的 Java 程序员。

以下是高手的回答

好的，这个问题我需要从三个方面来回答。

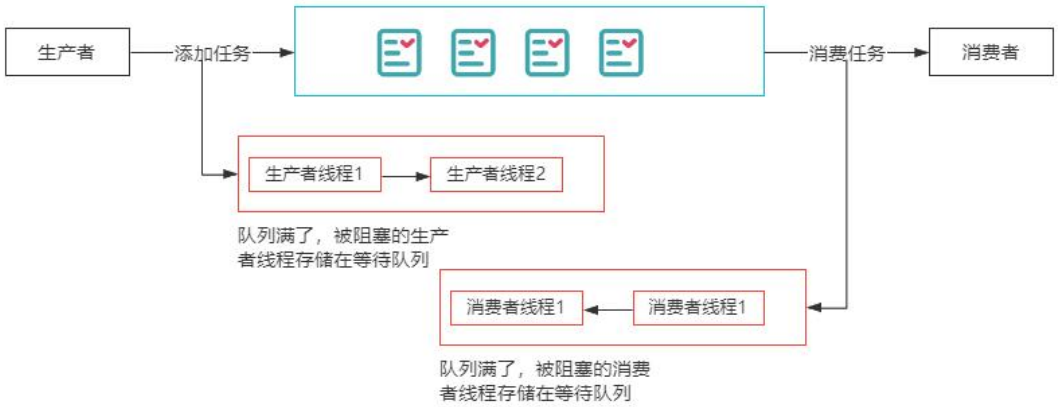
首先，阻塞队列本身是符合 FIFO 特性的队列，也就是存储进去的元素符合先进先出的规则。

其次，在阻塞队列里面，使用了 condition 条件等待来维护了两个等待队列（如图），

一个是队列为空的时候存储被阻塞的消费者

另一个是队列满了的时候存储被阻塞的生产者

并且存储在等待队列里面的线程，都符合 FIFO 的特性。



最后，对于阻塞队列的消费过程，有两种情况。

- 第一种，就是阻塞队列里面已经包含了很多任务，这个时候启动多个消费者去消费的时候，

它的有序性保证是通过加锁来实现的，也就是每个消费者线程去阻塞队列获取任务的时候

必须要先获得排他锁。

- 第二种，如果有多个消费者线程因为阻塞队列中没有任务而阻塞，这个时候这些线程是按照 FIFO 的顺序存储到

condition 条件等待队列中的。 当阻塞队列中开始有任务要处理的时候，这些被阻塞的消费者线程

会严格按照 FIFO 的顺序来唤醒，从而保证了消费的顺序型。

以上就是我对这个问题的理解。

53.194 当任务数超过线程池的核心线程数时，如何让它不进入队列，而是直接启用最大线程数

你们知道，“当任务数超过线程池的核心线程数时，如何让它不进入队列，而是直接启用最大线程数”吗?

HI，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

刚刚这个问题是一个工作 5 年的粉丝最近去某互联网公司面试遇到的。

好的，面试官。

当我们提交一个任务到线程池的时候，它的工作原理分为四步。

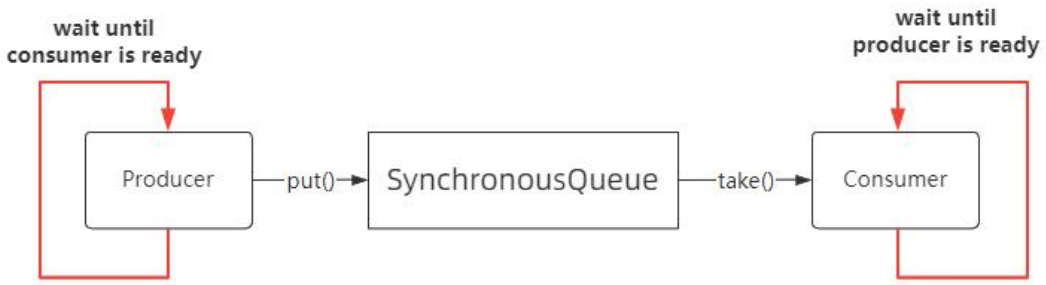
- 第一步，预热核心线程
- 第二步，把任务添加到阻塞队列
- 第三步，如果添加到阻塞队列失败，则创建非核心线程增加处理效率
- 第四步，如果非核心线程数达到了阈值，就触发拒绝策略

所以，如果希望这个任务不进入队列，那么只需要去影响第二步的执行逻辑就行了。

Java 中线程池提供的构造方法里面，有一个参数可以修改阻塞队列的类型。

其中，就有一个阻塞队列叫 SynchronousQueue（如图）， 这个队列不能存储任何元素。

它的特性是，每生产一个任务，就必须要指派一个消费者来处理，否则就会阻塞生产者。



基于这个特性，只要把线程池的阻塞队列替换成 SynchronousQueue。

就能够避免任务进入到阻塞队列，而是直接启动最大线程数去处理这个任务。

以上就是我对这个问题的理解。

53.195 请描述 Redis 的缓存淘汰策略

“请你描述一下 Redis 的缓存淘汰策略”

手机屏幕前的你，如果你正好遇到这个问题，想好怎么回答了吗？

Hi，大家好，我是 Mic

关于这个问题，我把高手的回答整理到了 15W 字的面试文档里面

大家可以在我的主页加 V 领取。

下面看看普通人和高手的回答。

> 以下是高手的回答

好的，面试官。这个问题我需要从三个方面来回答。

第一个方面：

当 Redis 使用的内存达到 maxmemory 参数配置的阈值的时候，Redis 就会根据配置的内存淘汰策略。

把访问频率不高的 key 从内存中移除。

maxmemory 默认情况是当前服务器的最大内存。

第二个方面：

Redis 默认提供了 8 种缓存淘汰策略，这 8 种缓存淘汰策略总的来说，我认为可以归类成五种

- 第一种， 采用 LRU 策略，就是把不经常使用的 key 淘汰掉
- 第二种，采用 LFU 策略，它在 LRU 算法上做了优化，增加了数据访问次数，从而确保淘汰的是非热点 key。
- 第三种，随机策略，也就是是随机删除一些 key
- 第四种，ttl 策略，从设置了过期时间的 key 里面，挑选出过期时间最近的 key 进行优先淘汰
- 第五种，当内存不够的时候，直接报错，这是默认的策略。

这些策略可以在 redis.conf 文件中手动配置和修改，我们可以根据缓存的类型和缓存使用的场景来选择

合适的淘汰策略。

最后一个方面，

我们在使用缓存的时候，建议是增加这些缓存的过期时间。

因为我们知道这些缓存大概的生命周期，从而更好的利用内存。

以上就是我对这个问题的理解。

53.196 SimpleDateFormat 是线程安全的吗？为什么？

“SimpleDateFormat 是线程安全的吗？为什么？”

这是一个针对 1~3 年的 Java 开发经验的面试题

屏幕前的你能回答出来吗？

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

以下是高手的回答

好的，面试官。

SimpleDateFormat 不是线程安全的，

SimpleDateFormat 类内部有一个 Calendar 对象引用，

它用来储存和这个 SimpleDateFormat 相关的日期信息。

当我们把 SimpleDateFormat 作为多个线程的共享资源来使用的时候。

意味着多个线程会共享 SimpleDateFormat 里面的 Calendar 引用，

多个线程对于同一个 Calendar 的操作，会出现数据脏读现象导致一些不可预料的错误。

在实际应用中，我认为有 4 种方法可以解决这个问题。

- 第一种，把 SimpleDateFormat 定义成局部变量，每个线程调用的时候都创建一个新的实例。
- 第二种，使用 ThreadLocal 工具，把 SimpleDateFormat 变成线程私有的
- 第三种，加同步锁，在同一时刻只允许一个线程操作 SimpleDateFormat
- 第四种，在 Java8 里面引入了一些线程安全的日期 API，比如 LocalDateTimer、DateTimeFormatter 等。

以上就是我对这个问题的理解。

53.197 Http 协议和 RPC 协议有什么区别?

“Http 协议和 RPC 协议有什么区别？”

最近很多人问我这个问题，他们都不知道怎么回答。

今天我们就来了解一下这个问题的高手回答。

另外，我把文字版本的内容整理到了一个 15W 字的面试文档里了。

以下是高手的回答

好的，面试官，这个问题我想从三个层面来回答。

从功能特性来说。

- http 是一个属于应用层的超文本传输协议，是万维网数据通信的基础，主要服务在网页端和服务端的数据传输上。
- RPC 是一个远程过程调用协议，它的定位是实现不同计算机应用之间的数据通信，

屏蔽通信底层的复杂性，让开发者就像调用本地服务一样完成远程服务的调用。

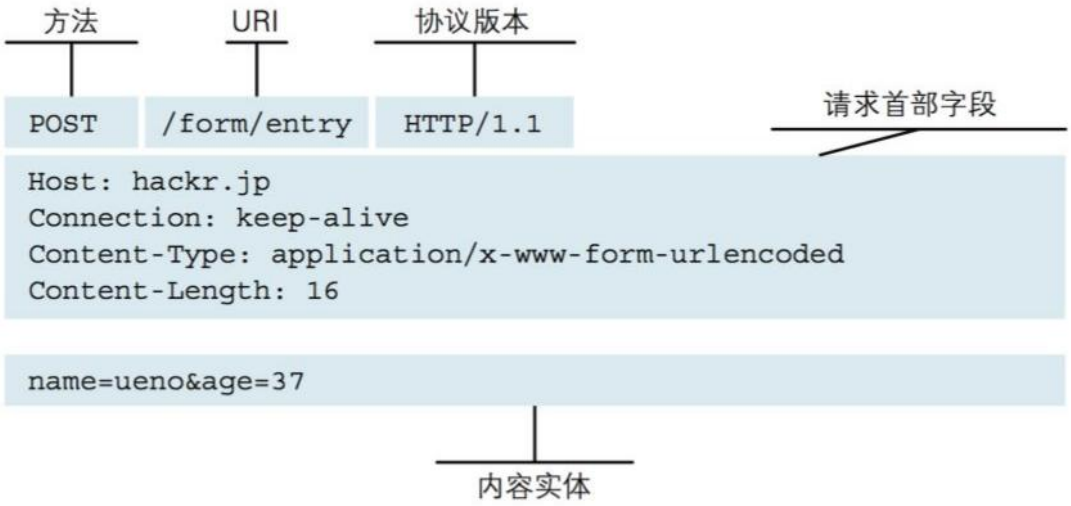
因此，这两个协议在定位层面就完全不同。

其次，从实现原理来说。

- http 协议是一个已经实现并且成熟的应用层协议（如图），它定义了通信的报文格式 Request Body 和 Request Header，

以及 Response Body 和 Response Header。

也就是说，符合这样一个协议特征的通信协议，才是 http 协议。



- RPC 只是一种协议的规范，它并没有具体实现，只有按照 RPC 通信协议规范实现的通信框架，

也就是 RPC 框架，才是协议的具体实现，比如 Dubbo、gRPC 等。

因此，我们可以在实现 RPC 框架的时候，

自定义报文通信的协议规范、自定义序列化方式、自定义网络通信协议的类型等等

因此，从这个层面来说，http 是成熟的应用协议，而 RPC 只是定义了不同服务之间的通信规范。

最后，应用层面来说。

- http 协议和实现了 RPC 协议的框架都能够实现跨网络节点的服务之间通信。

并且他们底层都是使用 TCP 协议作为通信基础。

但是，由于 RPC 只是一种标准协议，只要符合 RPC 协议的框架都属于 RPC 框架。

因此，RPC 的网络通信层也可以使用 HTTP 协议来实现，比如 gRPC、OpenFeign 底层都采用了 http 协议。

以上就是我对这个问题的理解。

53.198 一个空 Object 对象的占多大空间?

“一个空 Object 对象的占多大空间?”

一个工作了 5 年的 Java 程序员直接被搞蒙了。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

以下是高手的回答

好的，面试官。

在开启了压缩指针的情况下，Object 默认会占用 12 个字节，

但是为了避免伪共享问题，JVM 会按照 8 个字节的倍数进行填充，所以会填充 4 个字节变成 16 个字节长度。

在关闭压缩指针的情况下，Object 默认会占用 16 个字节，16 个字节正好是 8 的整数倍，因此不需要填充。

在 HotSpot 虚拟机里面，（如图）一个对象在堆内存里面的内存布局是使用 OOP 结构来表示的，

它主要分为三个部分。

- （如图）对象头，包括 Markword、类元指针、数组长度

其中 Markword 用来存储对象运行时的相关数据，比如 hashCode、gc 分代年龄等。

在 64 位操作系统中占 8 个字节，32 位操作系统中占 4 个字节

类元指针指向当前实例对象所属哪个类，开启指针压缩的情况下占 4 个字节，未开启则占 8 个字节

数组长度只有对象数组才会存在，占 4 个字节

- 实例数据，存储对象中的字段信息
- 对齐填充，Java 对象的大小需要按照 8 个字节或者 8 个字节的倍数对齐，避免伪共享问题。

因此，一个空的对象，在开启压缩指针的情况下，占 16 个字节

其中 Markword 占 8 个字节、类元指针占 4 个字节， 对齐填充占 4 个字节。

以上就是我对这个问题的理解。

53.199 什么是 Java 虚拟机，为什么要使用?

“什么是 Java 虚拟机，为什么要使用”。

最近一个 1 年 Java 开发经验的同学去面试阿里，遇到这个问题向我求助。

Hi，大家好，我是 Mic，一个工作 14 年的 Java 程序员。

那么，这个问题，面试官希望考察什么呢?

53.199.1 问题解析

Java 虚拟机，是 Java 应用程序运行的平台。

很多初学者，第一步基本上都是学习怎么写代码，并没有关注 Java 代码所运行的平台。

因此，虽然写了几年代码，但是对 Java 本身的理解不够深刻，程序一旦出现问题，很难排查和解决。

面试官考察这个问题的出发点，我认为有三个

- 了解求职者对于 Java 语言的理解深度，这个方面有助于提升代码编写的质量
- 了解求职者对于 JVM 基础的掌握程度，良好的基础有助于快速解决 GC 问题、内存问题等
- 考察求职者的潜质，一个对技术有热情的人，有助于更好的陪伴公司成长

所以，对于这个问题来说，我们只需要从 JVM 关键特性 Write Once、Run Anywhere 这个角度去切入解释就行了。

下面我们来看看高手应该怎么回答。

53.199.2 高手的回答

Java 虚拟机是 Java 语言的运行环境。

之所以需要 Java 虚拟机，主要是为 Java 语言提供 Write Once，Run Anywhere 能力。

实际上，一次编写，到处运行这个能力本身是不可能实现的。因为不同的操作系统和硬件。

最终执行的指令会有较大的差异。

而 Java 虚拟机就是解决这个问题的，它能根据不同的操作系统和硬件差异，生成符合这个平台机器指令。

简单理解，它就相当于一个翻译工具，在 window 下，翻译成 window 可执行的指令，在 linux 下，

翻译成 linux 下可执行的指令。

除了这个因素以为，我认为自动回收垃圾这个功能也是原因之一，它让开发者省去了垃圾回收这个工作。

减少了程序开发的复杂性。

53.200 finally 块一定会执行吗?

“finally 块一定会执行吗？”

这是最近一个工作 3 年的小伙伴去面试的时候遇到的问题。

屏幕前的你遇到这个问题会怎么回答呢?

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

对于这个问题，面试官想考察什么呢?

53.200.1 问题解析

这个问题，很明显是考察 Java 基础。

finally 语句块在实际开发中使用得非常多，它是和 try 语句块组合使用

通常情况下，不管有没有触发异常，finally 语句块中的代码是必然会执行的

所以我们会把资源的释放、或者业务日志的打印放在 finally 语句块里面。

所以，当大家把这个理念当成是固定的公式以后，就很少会去思考 finally 语句块什么情况下不执行。

这也是难倒很多求职者的原因，所以我认为这个问题主要考察两个方面：

- 对 finally 关键字的理解程度，其实就是考察 Java 基础，良好的 Java 基础能够写出更加稳定和健壮性的代码
- 是否具备对技术的探索精神，这样的人在技术的成长速度上会比一般人更快

下面来看看高手怎么回答这个问题吧!

53.200.2 高手的回答

finally 语句块在两种情况下不会执行：

- 程序没有进入到 try 语句块因为异常导致程序终止，这个问题主要是开发人员在编写代码的时候，异常捕获的范围不够。
- 在 try 或者 cache 语句块中，执行了 System.exit(0)语句，导致 JVM 直接退出

53.201 并行和并发有什么区别?

“并行和并发有什么区别？”

关于这个问题，很多工作 5 年以上的同学都回答不出来。

或者说，自己有一定的理解，但是不知道怎么表达

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

关于这个问题，面试官想考察什么呢?

53.201.1 问题解析

并行和并发最早其实描述的是 Java 并发编程里面的概念。

他们强调的是 CPU 处理任务的能力。

简单来说，

并发，就是同一个时刻，CPU 能够处理的任务数量，并且对于应用程序来说，不会出现卡顿现象。

并行，就是同一个时刻，允许多个任务同时执行，在多核 CPU 架构中，同时执行的任务数量是由核心数决定的，

比如在 4 核 4 线程的 CPU 中，只能同时执行 4 个线程。

这两个概念看起来类似，但其实描述的纬度是不同的，并发描述的是程序处理能力的视角

并行描述的是 CPU 处理任务方式的视角，一个是宏观层面，一个是微观层面。

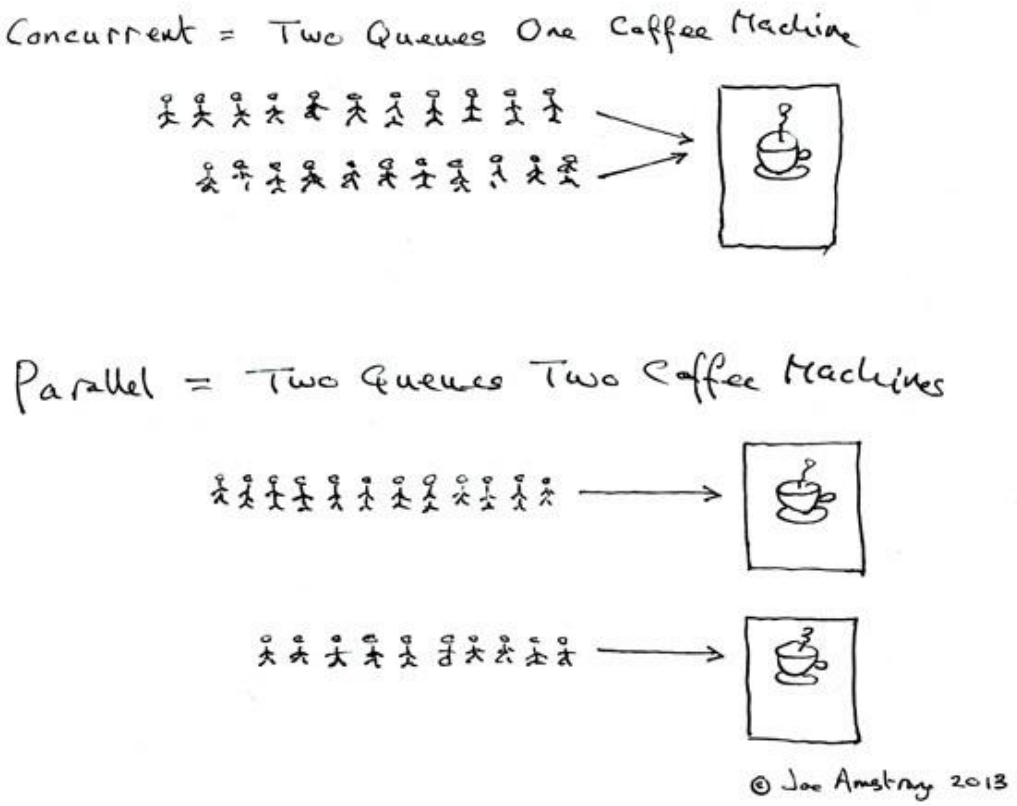
他们两个又是相辅相成的，CPU 并行执行任务的能力，又能提升程序的并发处理性能。

所以多核 CPU 的性能要比单核 CPU 好。

当然，如果是单核 CPU，也可以通过时间片切换的方式提升并发能力。

（如图）Erlang 之父 Joe Armstrong 用了一张图片解释了并行和并发的区别。

并发就是两个队列交替使用一台咖啡机，并行是两个队列同时使用两台咖啡机。



所以，在我看来，这个面试题可以很好的考察求职者 Java 并发编程的理解程度。

网上有很多的文章都在尝试解释这个概念，但是这些解释反而让这个问题越来越复杂。

我认为只有对线程的底层原理有深度理解，才能很好的回答这个问题。

53.201.2 高手的回答

并行和并发是 Java 并发编程里面的概念。

并行，是指在多核 CPU 架构下，同一时刻同时可以执行多个线程的能力。

在单核 CPU 架构中，同一时刻只能运行一个线程。

在 4 核 4 线程的 CPU 架构中，同一时刻可以运行 4 个线程，那这 4 个线程就是并行执行的。

并发，是指在同一时刻 CPU 能够处理的任务数量，也可以理解成 CPU 的并发能力。

在单核 CPU 架构中，操作系统通过 CPU 时间片机制提升 CPU 的并发能力

在多核 CPU 架构中,基于任务的并行执行能力以及 CPU 时间片切换的能力来提升 CPU 的并发能力。

所以，总的来说，并发是一个宏观概念，它指的是 CPU 能够承载的压力大小，

并行是一个微观概念，它描述 CPU 同时执行多个任务的能力。

53.202 JVM 为什么使用元空间替换了永久代?

“JVM 为什么使用元空间替换了永久代？”

这是一个工作 6 年的同学去字节第一面遇到的问题，很遗憾，他没有回答出来

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

关于这个问题，我们怎么回答？面试官到底关注什么呢?

53.202.1 面试解析

我们都知道 Java8 以及以后的版本中，JVM 运行时数据区的结构都在慢慢调整和优化。

但实际上这些变化，对于业务开发的小伙伴来说，没有任何影响。

因此我可以说，99%的人都回答不出这个问题。

但是互联网大厂的面试就是筛选那 1%的优秀人才，因此通过这道题，

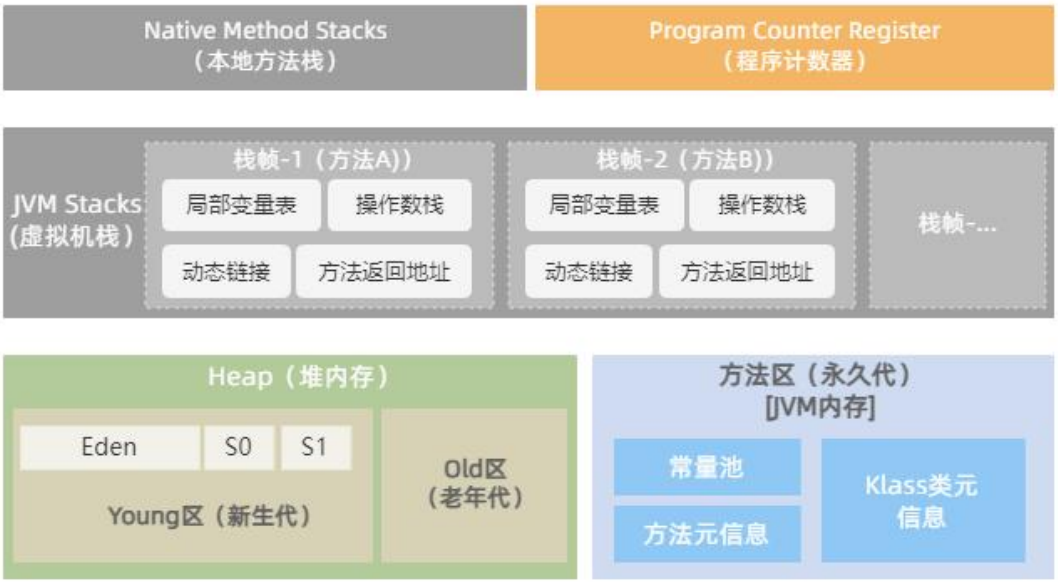
- 既可以考察求职者对 JVM 原理的理解程度
- 又能够考察求职者基本功的扎实程度
- 还能实现高级人才的筛选

在 Java7 里面，JVM 运行时数据区是这样的（如图）。

在 Hotspot 虚拟机中，方法区的实现是在永久代里面，它里面主要存储运行时常量池、Klass 类元信息等。

永久代属于 JVM 运行时内存中的一块存储空间，我们可以通过-XX:PermSize 来设置永久代的大小。

当内存不够的时候，会触发垃圾回收。



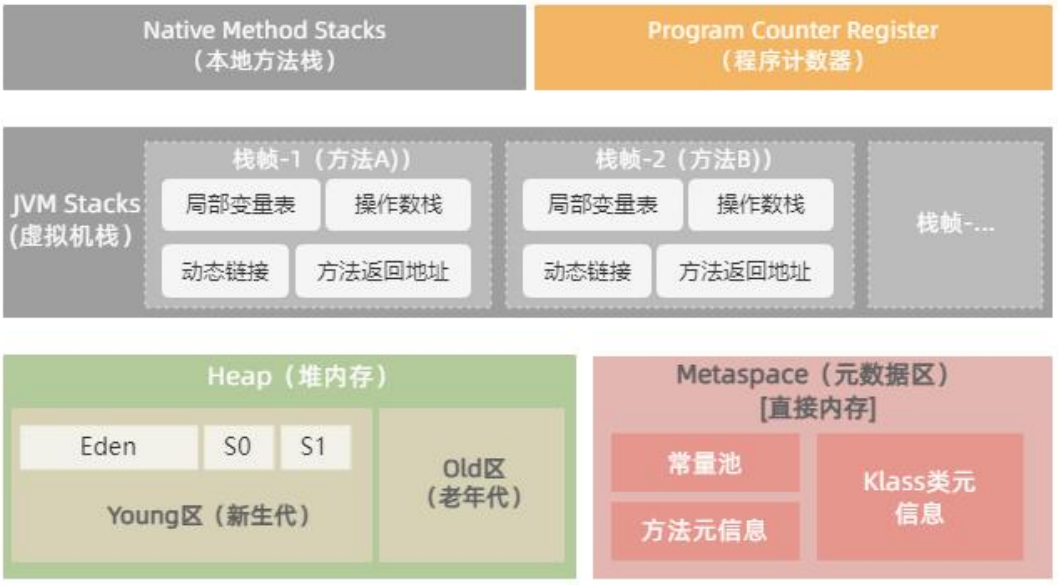
JDK1.7 运行时数据区

在 JDK1.8 里面，JVM 运行时数据区是这样的（如图）

在 Hotspot 虚拟机中，取消了永久代，由元空间来实现方法区的数据存储。

元空间不属于 JVM 内存，而是直接使用本地内存，因此不需要考虑 GC 问题。

默认情况下元空间是可以无限制的使用本地内存的，但是我们也可以使用 JVM 参数来限制内存使用大小。



JDK1.8 运行时数据区

为什么要使用元空间来替换永久代，背后必然有它的道理，但是如果求职者能够回答出来。

必然对于 JVM 底层原理是有一定了解的。

我们来看看高手的回答。

53.202.2 高手回答

我认为有三个方面的原因：

- 在 1.7 版本里面，永久代内存是有上限的，虽然我们可以通过参数来设置，但是 JVM 加载的 class 总数、大小是很难确定的。

所以很容易出现 OOM 问题。

但是元空间是存储在本地内存里面，内存上限比较大，可以很好的避免这个问题。

- 永久代的对象是通过 FullGC 进行垃圾收集，也就是和老年代同时实现垃圾收集。

替换成元空间以后，简化了 Full GC。可以在不进行暂停的情况下并发地释放类数据，同时也提升了 GC 的性能

- Oracle 要合并 Hotspot 和 JRockit 的代码，而 JRockit 没有永久代。

以上就是我对这个问题的理解。

53.203 如何解决 TCC 中的悬挂问题

“如何解决 TCC 中的悬挂问题”！

一个工作了 4 年的 Java 程序员，去京东面试，被问到这个问题。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这个问题面试官想考察什么方面的知识？我们又该怎么回答呢？

53.203.1 问题解析

TCC 是分布式事务问题里面的解决方案，一般在应聘互联网公司的时候问的比较多。

实际上，在 TCC 这个事务解决方案里面，除了悬挂问题以外，还有空回滚、幂等性需要考虑。

但是我们在应用的时候都是采用一些成熟的框架，比如 Seata，这些框架本身就帮我们解决了。

导致大部分人不知道这个问题的意思。

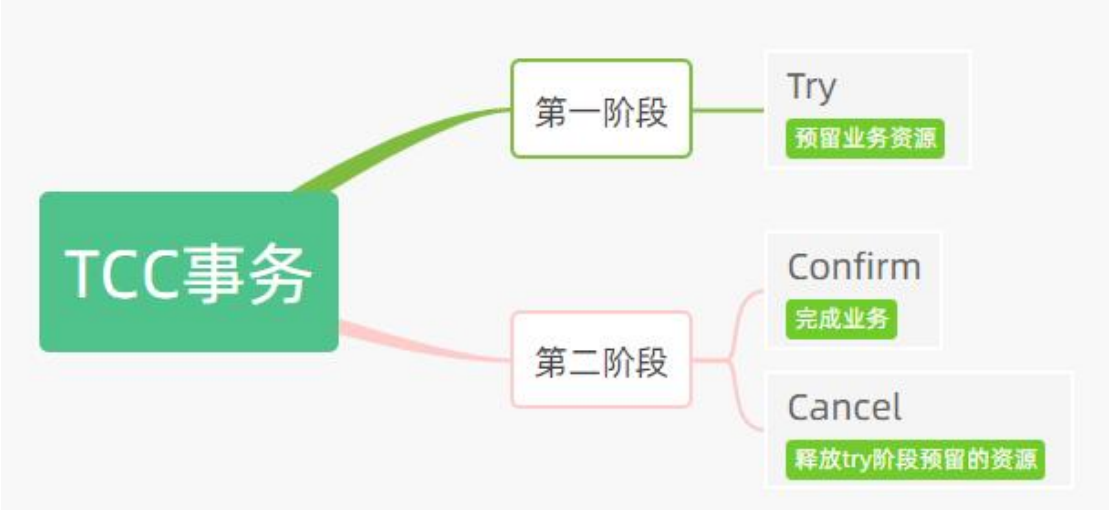
所谓 TCC，其实就是（Try-Confirm-Cancel），也就是把一个事务拆分成两个阶段，类似于传统的 XA 事务模型。

（如图）

Try 这个阶段，是实现业务的检查，预留必要的业务资源。

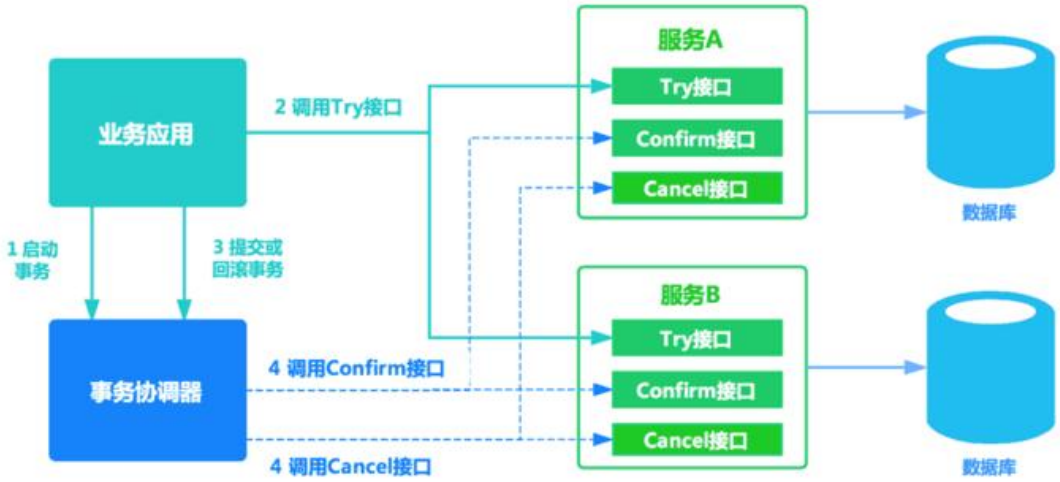
Confirm，真正执行业务逻辑，只需要使用 try 阶段预留的业务资源进行处理就行。

Cancel，如果事务执行失败，就通过 cancel 方法释放 try 阶段预留的资源。



（如图）在 TCC 事务模式下，我们通过一个事务协调器来管理多个事务，每个事务先执行 try 方法。

当所有事务参与者的 try 方法执行成功，就执行 confirm 方法完成真正逻辑的执行，一旦任意一个事务参与者出现异常，就通过 cancel 接口触发事务回滚，释放 Try 阶段占用的资源。



很显然，这是一个最终一致性的实现方案，因此当 Try 执行成功，就必须确保 Confirm 执行成功。

当 Try 执行失败，就必须确保 Cancel 实现资源释放。

而面试题中提到悬挂问题，指的是 TCC 执行 Try 接口出现网络超时时候，使得 TCC 触发 Cancel 接口回滚，

但可能在回滚之后，这个超时的 Try 接口才被真正执行，也就导致 Cancel 接口比 Try 接口先执行。

从而造成 Try 接口预留的资源一直无法释放，这种情况就是悬挂。

以上就是 TCC 悬挂问题的背景，它确实是每个成熟的高级开发必须要了解的细节。

因为有可能会造成比较严重的生产事故。

了解了背景之后，我们应该如何解决呢？下面来看看高手的回答。

53.203.2 高手的回答

对于悬挂问题，我认为只需要保证 Cancel 接口执行完以后，Try 接口不允许在执行就可以了。

所以，我们可以在 Try 接口里面，先判断 Cancel 接口有没有执行过，如果已经执行过，就不再执行。

是否执行过的这个判断，可以在事务控制表里面插入一条事务控制记录来标记这个事务的回滚状态。

然后在 Try 接口中只需要读取这个状态来判断就行了。

53.204 什么是令牌桶限流算法

当面试官问你，“什么是令牌桶限流算法”！

屏幕前的你，知道要怎么回答，才能获得面试官的青睐吗？

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

关于这个问题，面试官想考察哪些纬度？我们又该怎么回答呢？

53.204.1 问题解析

限流策略，是在高并发流量下保护系统稳定性的一种策略。

所以这个问题，主要是互联网公司会去考察。

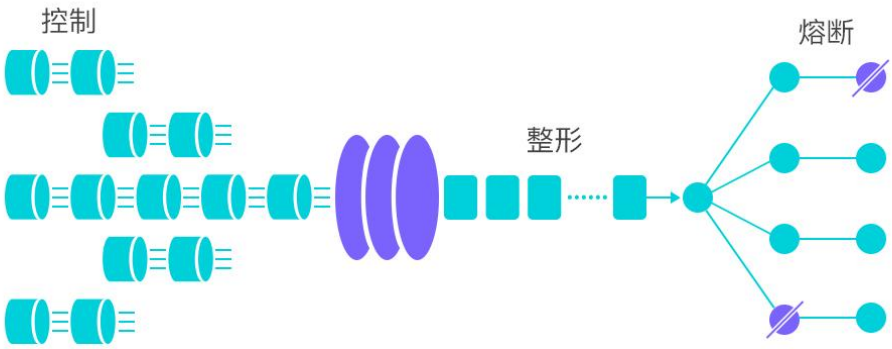
当然，在实际业务开发中，限流无处不在，比如

- 线程池、连接池这些通过限制总的并发数量避免资源过度使用。
- Nginx 反向代理服务器上通过 limit_conn 模块限制瞬时并发连接数
- 在方法层面通过 Sentinel、RateLimiter 等工具限制接口的并发请求数量等等

他们的核心目标，都是限制并发请求数量，避免系统被压垮导致不可用的问题。

在限流的整个体系里面（如图），我认为有三个比较重要的纬度

- 资源，也就是针对什么资源进行限流，比如接口，或者连接等
- 阈值，流量峰值达到多少后限制后续流量的访问
- 触发限流后的行为，比如熔断、降级等

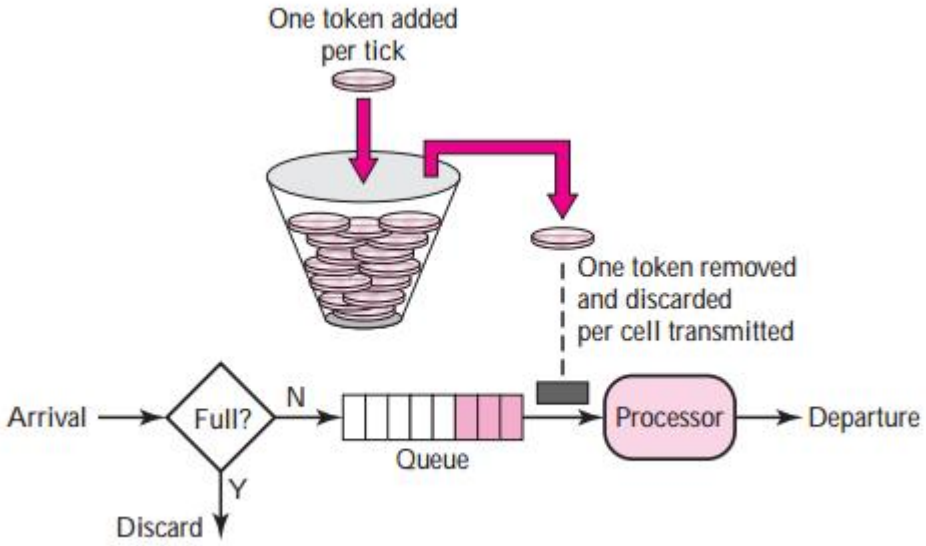


限流算法是整个限流实现的核心，不同限流算法，能够对流量的精准控制粒度，以及是否能支持突发流量等情况进行控制

常见的限流算法，滑动窗口、令牌桶、漏桶等。

其中，（如图）令牌桶是一种能够处理突发流量的限流算法，系统以恒定速率向令牌桶里面添加令牌，

然后每个请求都需要从令牌桶去获取令牌才能访问，如果获取不到，就会触发限流。



所以，我认为这道题考察两个方面

- 对限流的整体认知
- 了解限流算法对于限流本身的重要性

下面看看高手应该怎么回答。

53.204.2 高手回答

令牌桶是一种控制请求访问速率的算法。

它具体工作原理是：系统以一定速率生成令牌并放到令牌桶里面。

然后所有的客户端请求进入到系统后，先从令牌桶里面获取令牌，成功获取到令牌表示可以正常访问。

如果取不到令牌，说明请求流量大于令牌生成速率，也就是并发数超过系统承载的阈值，就会触发限流的动作。

在流量较低的情况下，令牌桶可以缓存一定数量的令牌，所以令牌桶可以处理瞬时突发流量。

53.205 Mysql 如何解决幻读问题

“Mysql 如何解决幻读问题”

一个工作了 4 年小伙伴，去一个美团面试，遇到了这样一个问题。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

关于这个问题，面试官想考察什么？我们应该如何回答呢？

53.205.1 问题解析

这个问题至少考察的是 3 年以上开发经验的同学。

Mysql 底层去解决并发事务问题，至少是要有一定的技术积累才能真正理解。

而如果作为一个刚工作没多久的程序员，必须要知道数据库的事务隔离级别的问题。

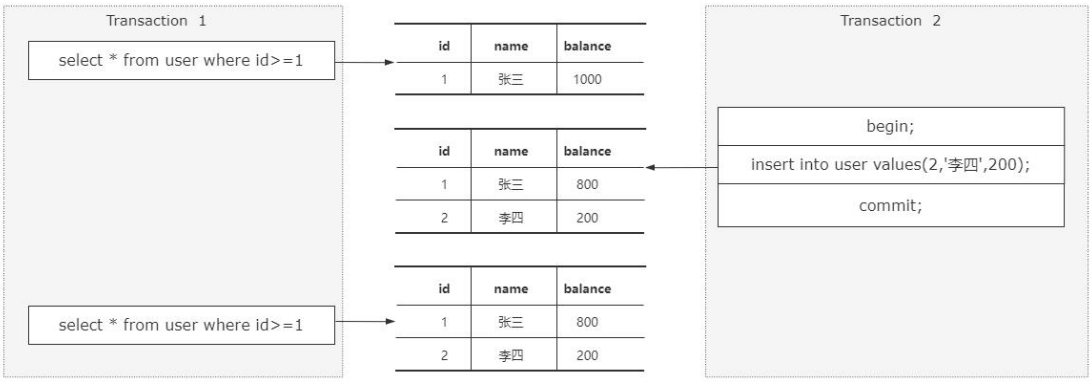
因为不同的隔离级别对于数据的安全性影响是不同的。

也就是存在脏读、幻读、不可重复读等问题。

所谓幻读，就是一个事务前后两次读取到的数据条数不一致。

（如图）在第一个事务里面执行一个范围查询，这个时候满足查询的数据只有一条。

接着第二个事务里面插入一条数据并且提交了，然后在第一个事务里面再次查询的时候发现有两条数据满足条件。



在 RR 事务隔离级别下，引入了 MVCC 和 LBCC 这两种方式来解决幻读问题。

MVCC 类似于一种乐观锁的设计，简单来说就是针对每个事务生成一个事务版本，然后针对这个版本定义了访问规则

- 一个事务只能看到第一次查询之前已经提交的事务以及当前事务的修改。
- 一个事务不能看到当前事务第一次查询之后创建的事务，以及未提交的事务修改。

但是，如果在一个事务里面存在当前读的情况下，MVCC 还是会存在幻读问题，

因为当前读不是读快照，而是直接读内存。

所以针对这种情况，可以使用 LBCC 也就是基于锁的机制来解决，也就是常说的行锁、表锁、间隙锁等

基于对上述知识的理解，如果没有对 Mysql 不同事务隔离级别的底层实现原理有一个清晰认识的同学

在回答这个问题的时候，要么就是很生硬，要么就是无法扩展，就会显得有点像是在背答案。

下面看看高手是怎么回答这个问题的吧。

53.205.2 高手回答

在 RR(也就是可重复读)的事务隔离级别下，InnoDB 采用了 MVCC 机制来解决幻读问题。

MVCC 就是一种乐观锁的机制，它通过对不同事务生成不同的快照版本，通过 UNDO 版本链进行管理

并且在 MVCC 里面，规定了高版本能够看到低版本的事务变更，低版本看不到高版本的事务变更

从而实现了不同事务之间的数据隔离，解决了幻读的问题。

但是在当前读的情况下，是直接读取内存的数据，跳过了快照度，所以还是会出现幻读问题。

我认为可以通过两个方式来解决。

1. 第一种是尽量避免当前读的情况
2. 第二种是引入 LBCC 的方式

以上就是我对这个问题的理解。

53.206 Java 常见的垃圾收集器有哪些?

实际上，垃圾收集器（GC，Garbage Collector）是和具体 JVM 实现紧密相关的，不同厂商（IBM、Oracle），不同版本的 JVM，提供的选择也不同。接下来，我来谈谈最主流的 Oracle JDK。

Serial GC，它是最古老的垃圾收集器，“Serial”体现在其收集工作是单线程的，并且在进行垃圾收集过程中，会进入臭名昭著的“Stop-The-World”状态。当然，其单线程设计也意味着精简的 GC 实现，无需维护复杂的数据结构，初始化也简单，所以一直是 Client 模式下 JVM 的默认选项。从年代的角度，通常将其老年代实现单独称作

Serial Old，它采用了标记 - 整理（Mark Compact）算法，区别于新生代的复制算法。Serial GC 的对应 JVM 参数是

-XX:+UseSerialGC

ParNew GC，很明显是个新生代 GC 实现，它实际是 Serial GC 的多线程版本，最常见的应用场景是配合老年代的 CMS GC 工作，下面是对应参数

-XX:+UseConcMarkSweepGC -XX:+UseParNewGC

CMS（Concurrent Mark Sweep） GC，基于标记 - 清除（Mark-Sweep）算法，设计目标是尽量减少停顿时间，这一点对于 Web 等反应时间敏感的应用非常重要，一直到今天，仍然有很多系统使用 CMS GC。但是，CMS 采用的标记 - 清除算法，存在着内存碎片化问题，所以难以避免在长时间运行等情况下发生 full GC，导致恶劣的停顿。另外，既然强调了并发（Concurrent），CMS 会占用更多 CPU 资源，并和用户线程争抢。Parrallel GC，在早期 JDK 8 等版本中，它是 server 模式 JVM 的默认 GC 选择，也被称作是吞吐量优先的 GC。它的算法和 Serial GC 比较相似，尽管实现要复杂的多，其特点是新生代和老年代 GC 都是并行进行的，在常见的服务器环境中更加高效。开启选项是：

-XX:+UseParallelGC

另外，Parallel GC 引入了开发者友好的配置项，我们可以直接设置暂停时间或吞吐量等目标，JVM 会自动进行适应性调整，例如下面参数：

-XX:MaxGCPauseMillis=value

-XX:GCTimeRatio=N // GC 时间和用户时间比例= 1 / (N+1)

G1 GC 这是一种兼顾吞吐量和停顿时间的 GC 实现，是 Oracle JDK 9 以后的默认 GC 选项。G1 可以直观的设定停顿时间的目标，相比于 CMS GC，G1 未必能做到 CMS 在最好情况下的延时停顿，但是最差情况要好很多。G1 GC 仍然存在着年代的概念，但是其内存结构并不是简单的条带式划分，而是类似棋盘的一个个 region。Region 之间是复制算法，但整体上实际可看作是标记 - 整理（Mark-Compact）算法，可以有效地避免内存碎片，尤其是当 Java 堆非常大的时候，G1 的优势更加明显。G1 吞吐量和停顿表现都非常不错，并且仍然在不断地完善，与此同时 CMS 已经在 JDK9 中被标记为废弃（deprecated），所以 G1 GC 值得你深入掌握。

53.207 RabbitMQ 如何实现高可用

“RabbitMQ 如何实现高可用”？

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

下面我们来分析一下面试官对于这个问题的考察意图

53.207.1 考察目标

这个问题就是简单的考察 RabbitMQ 相关知识点的了解。

难度并不大，主要考察 3 年以上开发经验的同学。

但是这个问题只是一个切入点，我认为接下来会根据求职者的背景

针对这个问题做更进一步去深度考察。

53.207.2 问题解析

在分布式架构下，高可用是最基础的设计。

也就是说，一旦依赖的某个服务出现故障，不能影响业务的正常执行。

RabbitMQ 提供了两种集群模式：

- 普通集群模式
- 镜像集群模式

先来看普通集群（如图）

这种集群模式下，各个节点只同步元数据，不同步队列中的消息。

其中元数据包含队列的名称、交换机名称及属性、交换机与队列的绑定关系等。

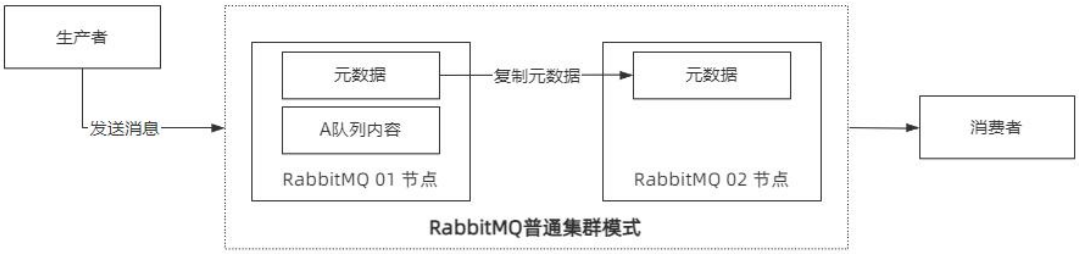
当我们发送消息和消费消息的时候，不管请求发送到 RabbitMQ 集群的哪个节点。

最终都会通过元数据定位到队列所在的节点去存储以及拉取数据。

很显然，这种集群方式并不能保证 Queue 的高可用，因为一旦 Queue 所在的节点挂了，那么这个 Queue

的消息就没办法访问了。

它的好处是通过多个节点分担了流量的压力，提升了消息的吞吐能力。

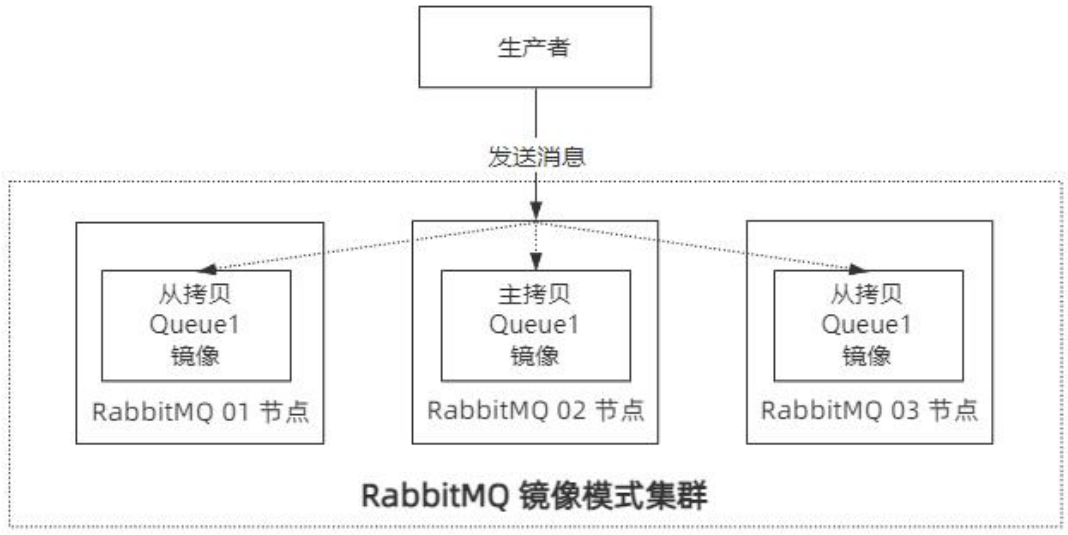


第二种是镜像集群（如图）

它和普通集群的区别在于，镜像集群中 Queue 的数据会在 RabbitMQ 集群的每个节点存储一份。

一旦任意一个节点发生故障，其他节点仍然可以继续提供服务。

所以这种集群模式实现了真正意义上的高可用。



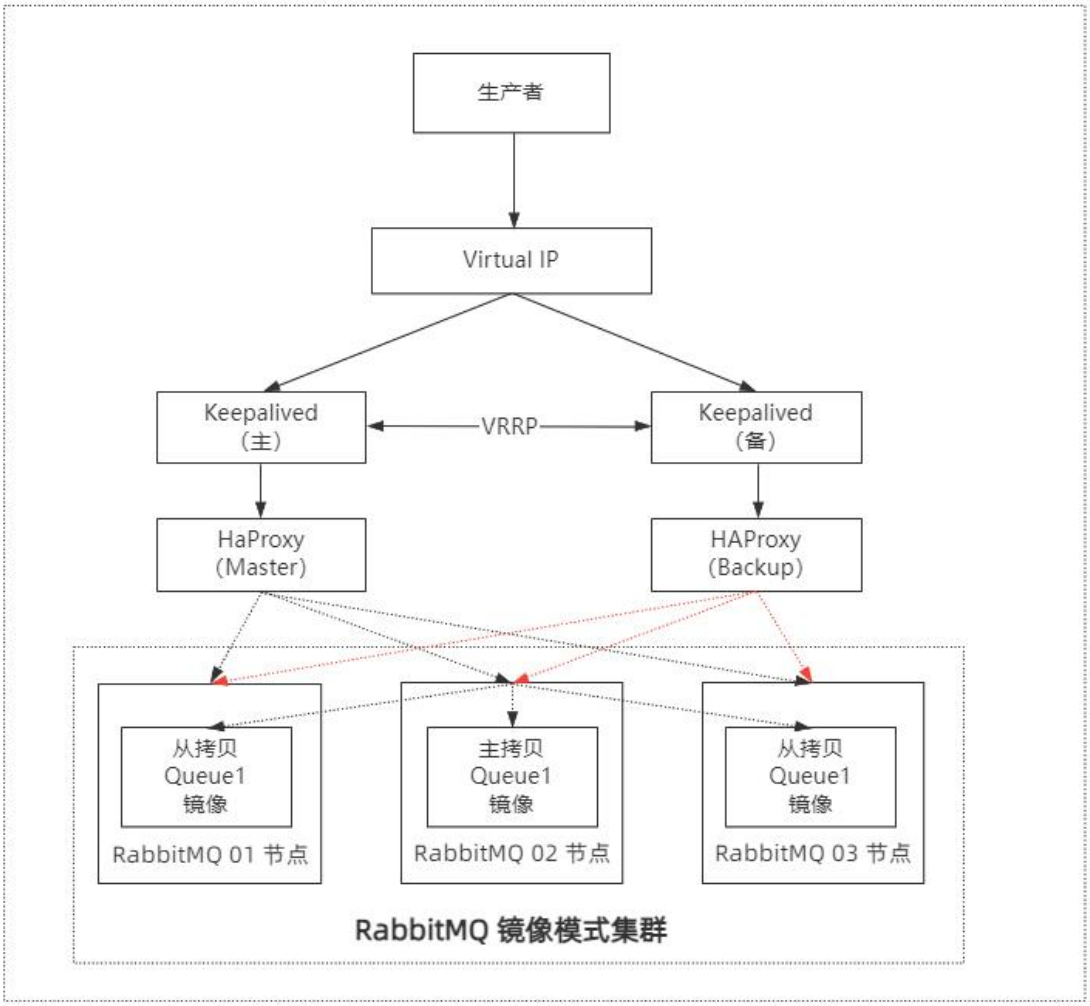
最后，在镜像集群的模式下，我们可以通过 Keepalived+HAProxy 来实现 RabbitMQ 集群的负载均衡（如图）。

其中：

- HAProxy 是一个能支持四层和七层的负载均衡器，可以实现对 RabbitMQ 集群的负载均衡
- 同时为了避免 HAProxy 的单点故障，可以再增加 Keepalived 实现 HAProxy 的主备，如果 HAProxy 主节点出现故障

那么备份节点就会接管主节点提供服务。

Keepalived 提供了一个虚拟 IP，业务只需要连接到虚拟 IP 即可。



好了，这就是 RabbitMQ 的常见高可用实现方案。

那么在面试的时候，怎么回答比较好呢?

53.207.3 高手回答

好的。

RabbitMQ 高可用实现方式有两种，

- 第一种是普通集群模式，在这种模式下，一个 Queue 的消息只会存在集群的一个节点上，

集群里面的其他节点会同步 Queue 所在节点的元数据，消息在生产和消费的时候，不管请求发送到

集群的哪个节点，最终都会路由到 Queue 所在节点上去存储和拉取消息。

这种方式并不能保证 Queue 的高可用性，但是它可以提升 RabbitMQ 的消息吞吐能力

- 第二种是镜像集群，也就是集群里面的每个节点都会存储 Queue 的数据副本。

意味着每次生产消息的时候，都需要把消息内容同步给集群中的其他节点。

这种方式能够保证 Queue 的高可用性，但是集群副本之间的同步会带来性能的损耗。

另外，由于每个节点都保存了副本，所以我们还可以通过 HAProxy 实现负载均衡。

以上就是我的理解。

53.208 Spring 如何解决循依赖问题

“Spring 如何解决循依赖问题”！

这是一道非常经典并且考察频率很高的面试题。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

下面我们来分析一下面试官对于这个问题的考察意图

53.208.1 考察目的

这个问题考察的求职者年限还挺广的，工作 1 年到工作 7 年之间都会遇到。

我认为考察目的有三个：

- Spring 是 Java 开发的基础应用框架，所以考察的是基本功
- 考察技术深度，判断你的能力高低以及
- 作为人才筛选的区分度

因此对于这个问题，不仅仅要回答原因，还需要有自己的思考和总结。

另外还需要注意，这个问题问的是 Spring 如何解决循环依赖，因此对于 Spring 无法解决的循环依赖

我们没必要在这里展开。

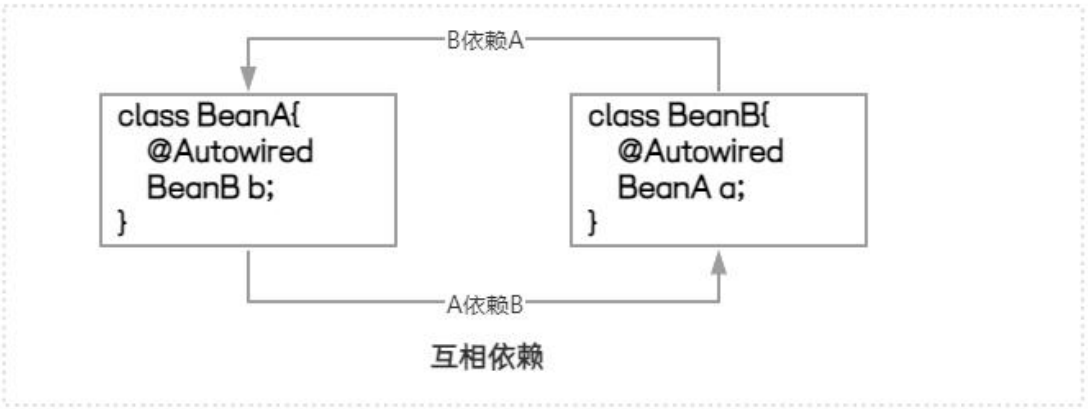
下面我们来分析一下这个问题的产生背景和解决方案。

53.208.2 问题解析

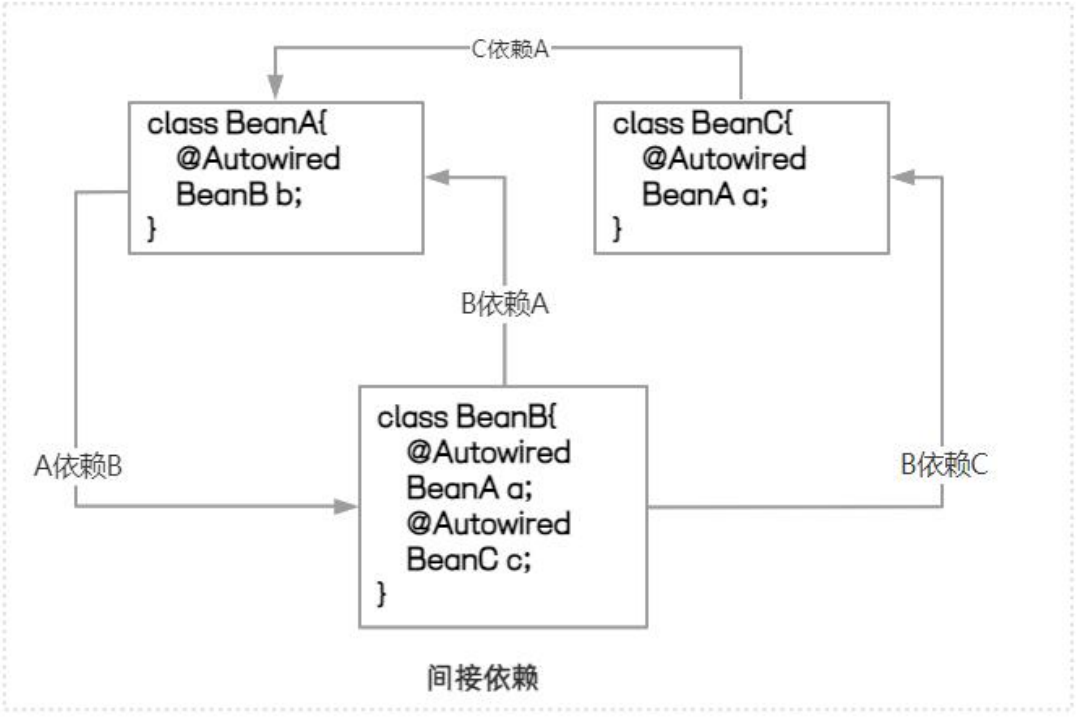
循环依赖是指一个或多个 Bean 实例之间存在直接或间接的依赖关系，构成循环调用。

通常表现为三种形态。

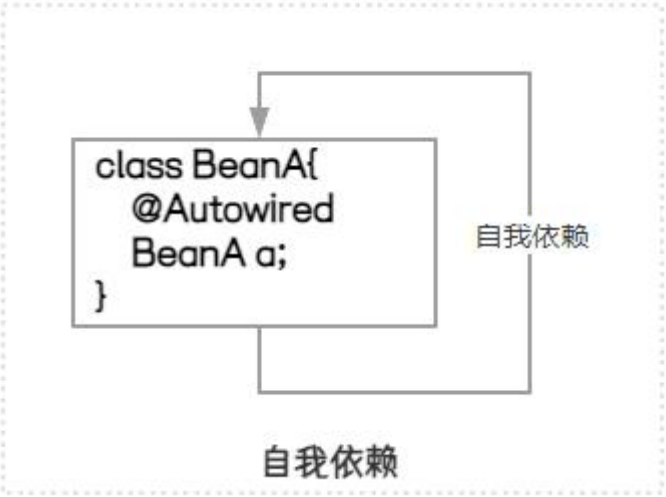
- （如图）互相依赖，也就是 A 依赖 B，B 依赖 A



- （如图）间接依赖，两个以上的 Bean 存在间接依赖关系造成循环调用。



- （如图）自我依赖，自己依赖自己造成了循环依赖



Spring 本身也考虑到了这方面的问题，所以它设计了三级缓存来解决部分循环依赖的问题。

所谓三级缓存，其实就是用来存放不同类型的 Bean。

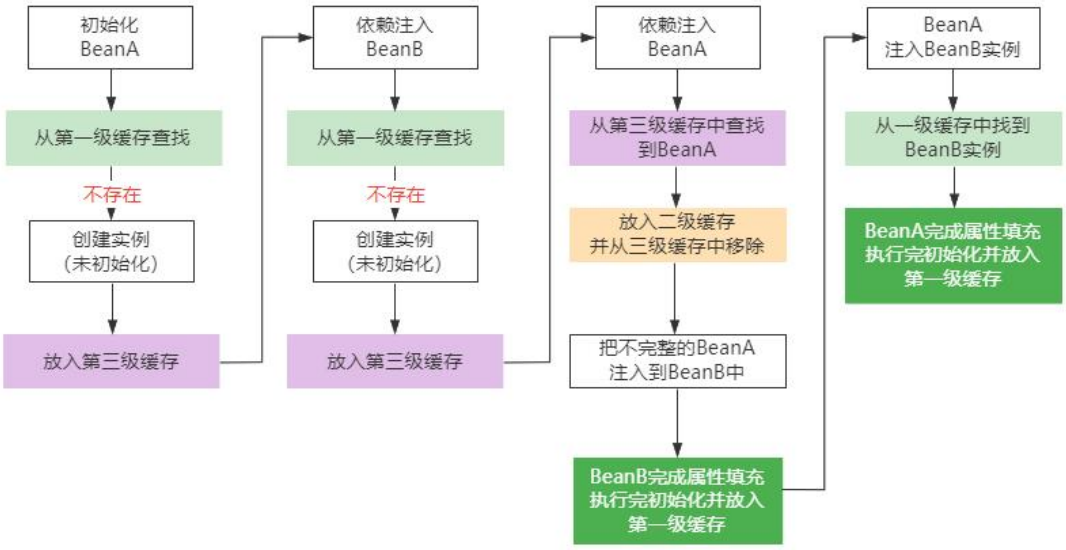
- 第一级缓存存放完全初始化好的 Bean，这个 Bean 可以直接使用了
- 第二级缓存存放原始的 Bean 对象，也就是说 Bean 里面的属性还没有进行赋值
- 第三级缓存存放 Bean 工厂对象，用来生成原始 Bean 对象并放入到二级缓存中

假设 BeanA 和 BeanB 存在循环依赖，那么在三级缓存的设计下，我画了这样一个图来描述工作原理。

- 初始化 BeanA，先把 BeanA 实例化，然后把 BeanA 包装成 ObjectFactory 对象保存到三级缓存中。
- 接着 BeanA 开始对属性 BeanB 进行依赖注入，于是开始初始化 BeanB，同样做两件事，

创建 BeanB 实例，以及加入到三级缓存。

- 然后，BeanB 也开始进行依赖注入，在三级缓存中找到了 BeanA，于是完成 BeanA 的依赖注入
- BeanB 初始化成功以后保存到一级缓存，于是 BeanA 可以成功拿到 BeanB 的实例，从而完成正常的依赖注入。



整个流程看起来很复杂，但是它的核心思想就是把 Bean 的实例化和 Bean 中属性的依赖注入这两个过程分离出来。

不过要注意的是，Spring 本身只能解决单实例存在的循环引用问题，但是存在以下四种情况需要人为干预：

- 多实例的 Setter 注入导致的循环依赖，需要把 Bean 改成单例。
- 构造器注入导致的循环依赖，可以通过@Lazy 注解
- DependsOn 导致的循环依赖，找到注解循环依赖的地方，迫使它不循环依赖。
- 单例的代理对象 Setter 注入导致的循环依赖，
  - 可以使用@Lazy 注解，
  - 或者使用@DependsOn 注解指定加载先后关系。

在实际开发中，出现循环依赖的根本原因还是在代码设计的时候，因为模块的耦合度较高，

依赖关系复杂导致的，我们应该尽可能的从系统设计角度去考虑模块之间的依赖关系，避免循环依赖的问题。

下面我们来看看高手的回答。

53.208.3 高手解答

好的，

Spring 设计了三级缓存来解决循环依赖问题。

- 第一级缓存里面存储完整的 Bean 实例，这些实例是可以直接被使用的。
- 第二级缓存里面存储的是实例化以后，但是还没有设置属性值的 Bean 实例，也就是 Bean 里面的依赖注入还没有做。
- 第三级缓存用来存放 Bean 工厂，它主要用来生成原始 Bean 对象并且放到第二级缓存里面。

三级缓存的核心思想，就是把 Bean 的实例化，和 Bean 里面的依赖注入进行分离。

采用一级缓存存储完整的 Bean 实例，采用二级缓存来存储不完整的 Bean 实例，

通过不完整的 Bean 实例作为突破口，解决循环依赖的问题。

至于第三级缓存，主要是解决代理对象的循环依赖问题。

以上就是我的理解。

53.209 如何解决死锁问题？

“如何解决死锁问题？”

如果你遇到这个问题，不知道该怎么回答，一定要看完这个视频。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这个问题，面试官的考察目的是什么呢？

53.209.1 考察目标

这个问题还是有一点难度，首先他考察的是并发编程相关领域的知识。

其次，对于死锁这个问题，平时我们遇到得比较少，即便是看过相关的问题，也不一定能够记住

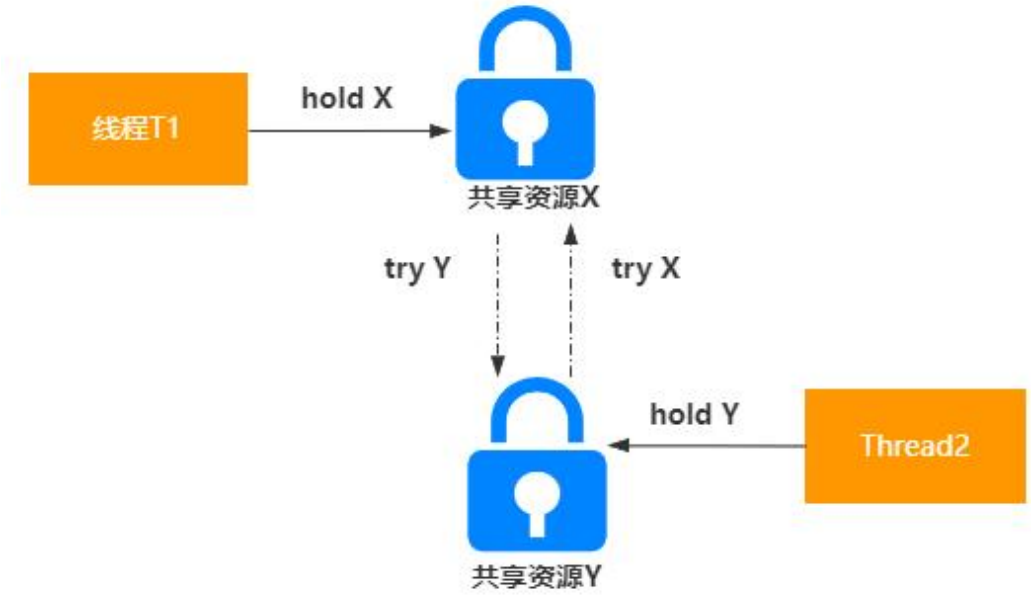
所以，在一定程度上，能够回答清楚这个问题，说明这个求职者的基本功还不错。

53.209.2 问题解析

（如图）

死锁，就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致互相等待的现象。

在没有外部干预的情况下，线程会一直处于阻塞状态，无法往下执行。



不过，要想真正产生死锁，必须同时满足四个条件。

- 互斥条件，共享资源 x 和 y 只能被一个线程占用

- 请求和保持条件， 线程 t1 已经获取共享资源 x，在等待共享资源 y 的时候，不释放共享资源 x
- 不可抢占条件，其他线程不能强行抢占线程 t1 占有的资源
- 循环等待条件，线程 t1 等到线程 t2 占有的资源，线程 t2 等待线程 t1 占有的资源，形成循环等待

线程在产生死锁以后，只能通过外部干预来解决，比如重启、或者 kill 线程等。

所以我们在写代码的时候，就应该去刻意规避死锁的问题。

也就是避免同时满足这四个条件。

在这四个条件里面，互斥条件是锁本身的特性，无法被破坏，其他三个条件都可以被破坏。

对于请求和保持条件，我们可以在第一次执行的时候一次性申请所有的共享资源

对于不可抢占条件，占用部分资源的线程在进一步申请其他资源的时候，如果申请不到，就主动释放它占有的资源。

对于循环等待条件，可以按照顺序来申请资源，相当于给资源编号，按照编号顺序申请就可以避免循环等待。

当然，死锁问题不仅仅局限在多线程领域，单反涉及到互斥锁的地方都有可能出现，

比如 Mysql 数据库的行锁、表锁，以及分布式锁等。

在底层原理上都是相同的。

下面来看看高手的回答。

53.209.3 高手回答

好的，

程序出现死锁，是因为在多线程环境里面两个或两个以上的线程同时满足

互斥条件、请求保持条件、不可抢占条件、循环等待条件。

出现死锁以后，可以通过 jstack 命令去导出线程的 dump 日志，

然后从 dump 日志里面定位到具体死锁的程序代码。

通过修改程序代码去破坏这四个条件里面的任意一个，就可以解决死锁问题。

当然，因为互斥条件因为是锁本身的特性，所以不能被破坏。

以上就是我的理解。

53.210 为什么 ConcurrentHashMap 中 key 不允许为 null

“为什么 ConcurrentHashMap 中 key 不允许为 null”！

听到这个问题，大家有没有感受到面试过程中的压迫感

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这个问题，面试官的考察目的是什么呢?

53.210.1 考察目标

这是一个基础问题，主要考察 1 到 3 年经验的开发人员

ConcurrentHashMap 在实际应用中使用频率较高

考察这个问题的目的，是了解求职者的基本功。

所以为了表现更好，可以从 ConcurrentHashMap 的设计角度去回答。

53.210.2 问题解析

打开 ConcurrentHashMap 的源码

在 put 方法里面，可以看到这样一段代码（如图）

如果 key 或者 value 为空，则抛出空指针异常。

```
final V putVal(K key, V value, boolean onlyIfAbsent) {
        if (key == null || value == null) throw new
NullPointerException();
```

但是为什么 ConcurrentHashMap 不允许 key 或者 value 为空呢?

简单来说，就是为了避免在多线程环境下出现歧义问题。

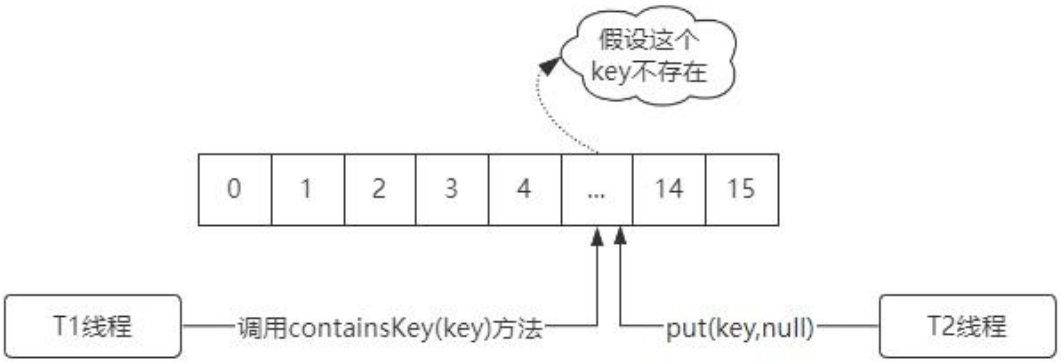
所谓歧义问题，就是如果 key 或者 value 为 null，当我们通过 get(key)获取对应的 value 的时候，如果返回的结果是 null

我们没办法判断，它是 put(k,v)的时候，value 本身为 null 值，还是这个 key 本身就不存在。

比如在这样一种情况下（如图），线程 t1 调用 containsKey 方法判断 key 是否存在，假设当前这个 key 不存在，本来应该返回 false。

但是在 T1 线程返回之前，正好有一个 T2 线程插入了这个 key，但是 value 为 null。

这就导致原本 T1 线程返回的结果有可能是 true，有可能是 false，取决于 T1 和 T2 线程的执行顺序。



这种现象我们可以认为是线程安全性问题，而 ConcurrentHashMap 又是一个线程安全的集合，

所以自然就不允许 key 或者 value 为 null。

而 HashMap 中是允许存 null 的，因为它不需要考虑到线程安全性问题。

所以这个问题的核心本质还是 ConcurrentHashMap 这个并发安全性集合的特性。

当然。Doug Lea 还认为，不管是否是并发安全的集合，它都不应该允许存储 null。

53.210.3 高手回答

好的。

ConcurrentHashMap 这么设计的原因是为了避免在多线程并发场景下的歧义问题。

也就是说，当一个线程从 ConcurrentHashMap 获取某个 key，如果返回的结果是 null 的时候。

这个线程无法确认，这个 null 表示的是确实不存在这个 key，还是说存在 key，但是 value 为空。

这种不确定性会造成线程安全性问题，而 ConcurrentHashMap 本身又是一个线程安全的集合。

所以才这么设计！

以上就是我的理解。

53.211 ThreadLocal 会出现内存泄漏吗?

“ThreadLocal 会出现内存泄漏吗？”

这个问题在网上存在很多争论，有些博主说了半天也没说明白

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这个问题，面试官考察什么呢?

53.211.1 考察目的

这是并发编程里面的知识，所以考察的还是技术基础。

Java 基础是每个公司必然都会考察的，不管你是工作 1 年还是工作 10 年。

因为所有的应用框架和中间件，都是在 Java 基础上构建出来的。

基本功扎实的人，不仅仅写的代码更加可靠，而且学习新技术也更加容易

53.211.2 问题解析

ThreadLocal 是一个用来解决线程安全性问题的工具。

它相当于让每个线程都开辟一块内存空间，用来存储共享变量的副本。

然后每个线程只需要访问和操作自己的共享变量副本即可，从而避免多线程竞争同一个共享资源。

它的工作原理很简单（如图）

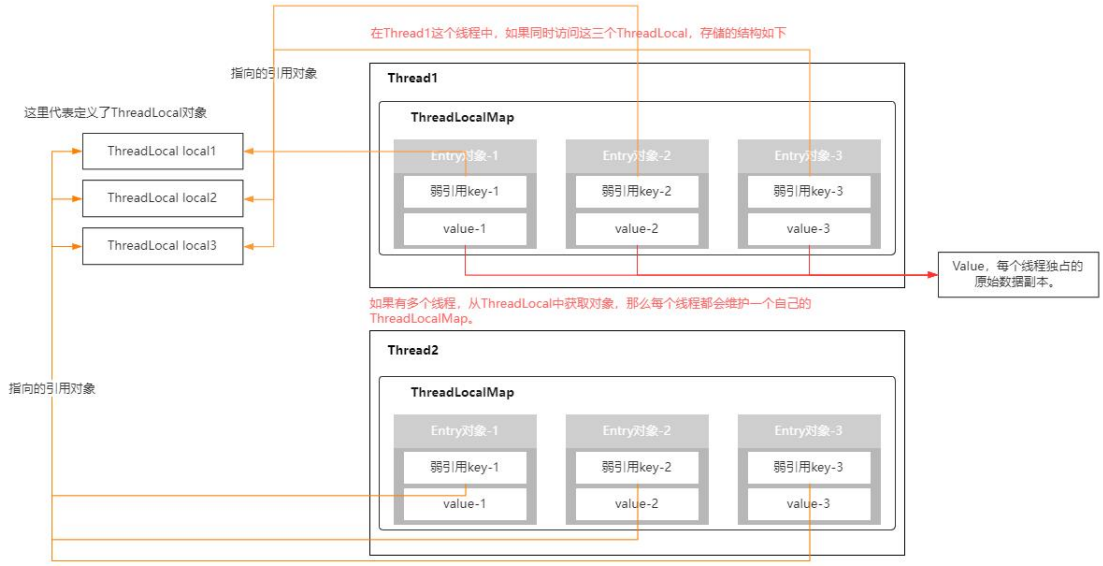
每个线程里面有一个成员变量 ThreadLocalMap。

当线程访问用 ThreadLocal 修饰的共享数据的时候

这个线程就会在自己成员变量 ThreadLocalMap 里面保存一份数据副本。

key 指向 ThreadLocal 这个引用，并且是弱引用关系，而 value 保存的是共享数据的副本。

因为每个线程都持有一个副本，所以就解决了线程安全性问题。



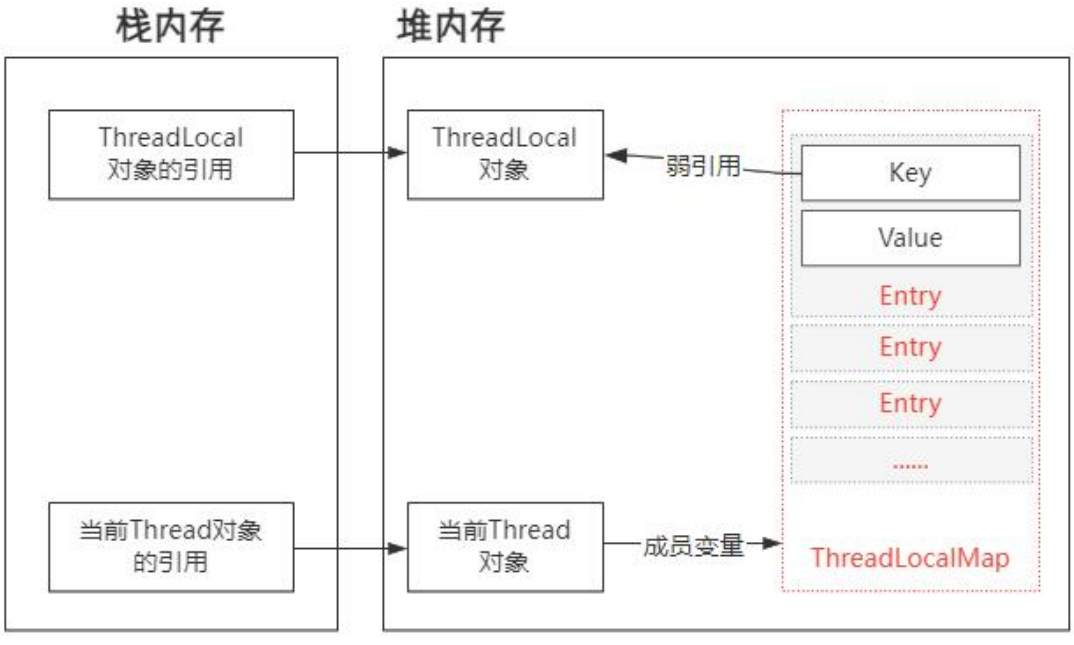
这个问题考察的是内存泄漏，所以必然和对象引用有关系。

ThreadLocal 中的引用关系如图所示（如图），

Thread 中的成员变量 ThreadLocalMap，它里面的可以 key 指向 ThreadLocal 这个成员变量，并且它是一个弱引用

所谓弱引用，就是说成员变量 ThreadLocal 允许在这种引用关系存在的情况下，被 GC 回收。

一旦被回收，key 的引用就变成了 null，就会导致这个内存永远无法被访问，造成内存泄漏。



那到底 ThreadLocal 会不会存在内存泄漏呢?

从 ThreadLocal 本身的设计上来看，是一定存在的。

可能有些小伙伴忍不住想怼我了，Mic 老师，如果这个线程被回收了，那线程里面的成员变量都会被回收。

就不会存在内存泄漏问题啊?

这样理解没问题，但是在实际应用中，我们一般都是使用线程池，而线程池本身是重复利用的

所以还是会存在内存泄漏的问题。

除此之外啊，ThreadLocal 为了避免内存泄漏问题，当我们在进行数据的读写时，ThreadLocal 默认会去尝试做一些清理动作，找到并清理 Entry 里面 key 为 null 的数据。

但是，它仍然不能完全避免，有同学就问了，那怎么办啊！！！

有两个方法可以避免:

- 每次使用完 ThreadLocal 以后，主动调用 remove()方法移除数据
- 把 ThreadLocal 声明称全局变量，使得它无法被回收

ThreadLocal 本身的设计并不复杂，要想深入了解，建议大家去看看源码!

53.211.3 高手回答

好的，

我认为，不恰当的使用 ThreadLocal，会造成内存泄漏问题。

主要原因是，线程的私有变量 ThreadLocalMap 里面的 key 是一个弱引用。

弱引用的特性，就是不管是否存在直接引用关系，

当成员 ThreadLocal 没用其他的强引用关系的时候，这个对象会被 GC 回收掉。

从而导致 key 可能变成 null，造成这块内存永远无法访问，出现内存泄漏的问题。

规避内存泄漏的方法有两个:

- 通过扩大成员变量 ThreadLoca 的作用域，避免被 GC 回收
- 每次使用完 ThreadLocal 以后，调用 remove 方法移除对应的数据

第一种方法虽然不会造成key为null的现象,但是如果后续线程不再继续访问这个key。

也会导致这个内存一直占用不释放，最后造成内存溢出的问题。

所以我认为最好是在使用完以后调用 remove 方法移除

以上就是我的理解。

53.212 在 Java 中实现单例模式有哪些方法

“在 Java 中实现单例模式有哪些方法”！

屏幕前的你们，是不是感觉这个问题很简单。

但是实际上，有一个同学曾经去快手面试的时候，被更进一步问到。

写一个性能最好的单例模式，结果很显然就没有回答出来

HI，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这个问题，面试官想考察什么呢?

53.212.1 考察目的

这是属于设计模式里面的一个问题。

我们知道 Java 有 23 三种设计模式，但是真正在实际开发中，能够熟练使用设计模式的人很少。

很多人说没有场景，但其实只是因为他们只理解了设计模式的概念。

这个问题考察求职者对于设计模式的理解和应用。

本质上还是考察基本功，当然，针对不同工作年限，考察的深度不同。

对于刚工作的同学，只需要了解什么是单例以及如何写出一个单例就行

对于工作年限较长的同学，还需要考察单例模式的性能、以及避免破坏单例的情况等

53.212.2 问题解析

单例模式，就是一个类在任何情况下绝对只有一个实例，并且提供一个全局访问点来获取该实例。

要实现单例，至少需要满足两个点：

- 私有化构造方法，防止被外部实例化造成多实例问题
- 提供一个静态方位作为全局访问点来获取唯一的实例对象

在 Java 里面，至少有 6 种方法来实现单例。

（如图）第一种，是最简单的实现，通过延迟加载的方式进行实例化，并且增加了同步锁机制避免多线程环境下

的线程安全问题。

```
public class Singleton {
 private static Singleton instance;
 private Singleton (){}
 public static synchronized Singleton getInstance() {
   if (instance == null) {
       instance = new Singleton();
   }
   return instance;
}
```

但是这种加锁会造成性能问题，而且同步锁只有在第一次实例化的时候才产生作用，后续不需要。

（如图）于是有了第二种改进方案，通过双重检查锁的方式，减少了锁的范围来提升性能

（如图）第三种，通过饿汉式实现单例。

这种方式在类加载的时候就触发了实例化，从而避免了多线程同步问题。

```
public class Singleton {
 private static Singleton instance = new Singleton();
 private Singleton (){}
 public static Singleton getInstance() {
    return instance;
 }
}
```

还有一种与这个方式类似的实现（如图）

通过在静态块里面实例化，而静态块是在类加载的时候触发执行的，所以也只会执行一次。

```
public class Singleton {
  private Singleton instance = null;
  static {
    instance = new Singleton();
  }
  private Singleton (){}
  public static Singleton getInstance() {
    return this.instance;
  }
}
```

上面两种方式，都是在类加载的时候初始化，没有达到延迟加载的效果，当然本身影响不大，但是

其实还是可以更进一步优化，就是可以在使用的时候去触发初始化（如图）。

像这种写法，把 INSTANCE 写在一个静态内部类里面，

由于静态内部类只有调用静态内部类的方法，静态域，或者构造方法的时候才会加载静态内部类。

所以当 Singleton 被加载的时候不会初始化 INSTANCE，从而实现了延迟加载。

```
public class Singleton {
 private static class SingletonHolder {
    private static final Singleton INSTANCE = new Singleton();
 }
 private Singleton (){}
 public static final Singleton getInstance() {
    return SingletonHolder.INSTANCE;
 }
}
```

另外，我们还可以使用枚举类来实现（如图）。

这种写法既能避免多线程同步问题，又能防止反序列化重新创建新对象，也是一个比较好的方案。

```
public enum Singleton {
  INSTANCE;
  public void doSomeThing() {
    //TODO
  }
}
```

当然，除了这些方案以外，也许还有更多的写法，只需要满足单例模式的特性就行了。

53.212.3 高手解析

好的

我认为可以通过 3 种方式来实现单例，

第一种是通过双重检查锁的方式，它是一种线程安全并且是延迟实例化的方式，

但是因为加锁，所以会有性能上的影响。

第二种是通过静态内部类的方式实现，它也是一种延迟实例化，

由于它是静态内部类，所以只会使用的时候加载一次，不存在线程安全问题。

第三种是通过枚举类的方式实现，它既是线程安全的，又能防止反序列化导致破坏单例问题。

但是，多线程、克隆、反序列化、反射，都有可能会造成单例的破坏。

而我认为，通过枚举的方式实现单例，是能够解决所有可能被破坏的情况。

以上就是我的理解。

53.213 请简要说明 Mysql 中 MyISAM 和 InnoDB 引擎的区别

“请简要说明 Mysql 中 MyISAM 和 InnoDB 引擎的区别”。

屏幕前有多少同学在面试过程与遇到过类似问题， 可以在评论区留言：遇到过。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这个问题面试官想考察什么？我们又该怎么回答呢？

53.213.1 考察目的

对于 xxxx 技术的区别，在面试中是很常见的一个问题

一般情况下，面试官会通过这类问题来热场，打开接下来沟通的话题，

然后沿着你回答的内容层层递进去做更深入的了解。

当然，另外一个更加深层次的原因，就是考察求职者对于这两个技术的理解层次。

因为能够通过自己的理解总结出他们的区别，至少说明你是有比较深入的研究的。

这个问题考察难度算是比较大的，一般面向 3 年以上开发经验的同学。

53.213.2 问题解析

MyISAM 和 InnoDB 都是 Mysql 里面的两个存储引擎。

在 Mysql 里面，存储引擎是可以自己扩展的，它的本质其实是定义数据存储的方式

以及数据读取的实现逻辑。

而不同存储引擎本身的特性，使得我们可以针对性的选择合适的引擎来实现不同的业务场景。

从而获得更好的性能。

在 Mysql 5.5 之前，默认的存储引擎是 MyISAM，从 5.5 以后，InnoDB 就作为了默认的存储引擎。

在实际应用开发中，我们基本上都是采用 InnoDB 引擎。

我们先来看一下 MyISAM 引擎。

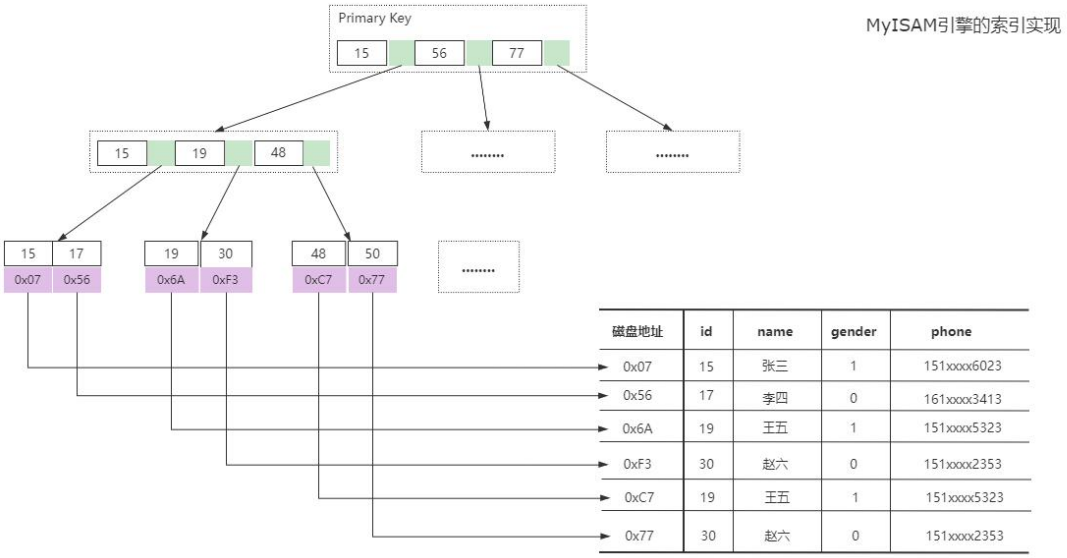
MyISAM 引擎的数据是通过二进制的方式存储在磁盘上，它在磁盘上体现为两个文件

- 一个是.MYD 文件，D 代表 Data，是 MyISAM 的数据文件，存放数据记录，
- 一个是.MYI 文件，I 代表 Index，是 MyISAM 的索引文件，存放索引

实现机制如图所示（如图）。

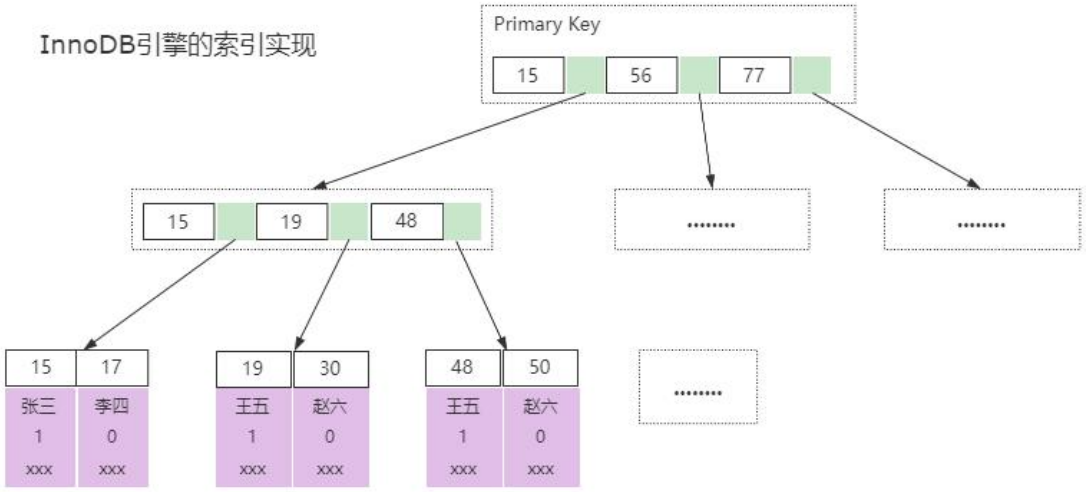
因为索引和数据是分离的，所以在进行查找的时候，先从索引文件中找到数据的磁盘位置，再到数据文件

中找到索引对应的数据内容。



在 InnoDB 存储引擎中，数据同样存储在磁盘上，它在磁盘上只有一个 ibd 文件，里面包含索引和数据。

（如图），它的整体结构如图所示，在 B+树的叶子节点里面存储了索引对应的数据，在通过索引进行检索的时候，命中叶子节点，就可以直接从叶子节点中取出行数据。



了解了这两个存储引擎以后，我们在面试的时候该怎么回答呢？

53.213.3 高手回答

好的。

基于我的理解，我认为 MyISAm 和 InnoDB 的区别有 4 个，

- 第一个，数据存储的方式不同，MyISAM 中的数据和索引是分开存储的，

而 InnoDB 是把索引和数据存储在同一个文件里面。

- 第二个，对于事务的支持不同，MyISAM 不支持事务，而 InnoDB 支持 ACID 特性的事务处理
- 第三个，对于锁的支持不同，MyISAM 只支持表锁，而 InnoDB 可以根据不同的情况，支持行锁，表锁，间隙锁，临键锁
- 第四个，MyISAM 不支持外键，InnoDB 支持外键

因此基于这些特性，我们在实际应用中，可以根据不同的场景来选择合适的存储引擎。

比如如果需要支持事务，那必须要选择 InnoDB。

如果大部分的表操作都是查询，可以选择 MyISAM。

以上就是我的理解。

53.214 Mybatis 是如何进行分页的 、

“Mybatis 是如何进行分页的”？

这是一个工作了 3 年的同学，在面试的时候遇到的问题。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

下面我们来分析一下面试官对于这个问题的考察意图。

53.214.1 考察目标

Mybatis 是 Java 应用开发的基础框架，而分页又是我们实时都在使用的功能。

因此，在我看来，一方面考察的是求职者对 Mybatis 框架的使用能力

另一方面，以此为切入点去深度挖掘 Mybatis 里面更多的问题，从而了解求职者对它的理解深度。

这道题考察难度不大，主要考察 1-3 年 Java 开发经验的同学。

53.214.2 问题解析

数据进行分页是最基础的功能，一般可以把分页分成两类：

- 逻辑分页，先查询出所有的数据缓存到内存，再根据业务相关需求，从内存数据中筛选出合适的数据进行分页。
- 物理分页 ，直接利用数据库支持的分页语法来实现，比如 Mysql 里面提供了分页关键词 Limit

Mybatis 提供了四种分页方式：

- 在 Mybatis Mapper 配置文件里面直接写分页 SQL，这种方式比较灵活，实现也简单。

- RowBounds 实现逻辑分页，也就是一次性加载所有符合查询条件的目标数据，根据分页参数值在内存中实现分页。

当然，在数据量比较大的情况下，JDBC 驱动本身会做一些优化，也就是不会把所有结果存储在 ResultSet 里面，

而是只加载一部分数据，再根据需求去数据库里面加载。

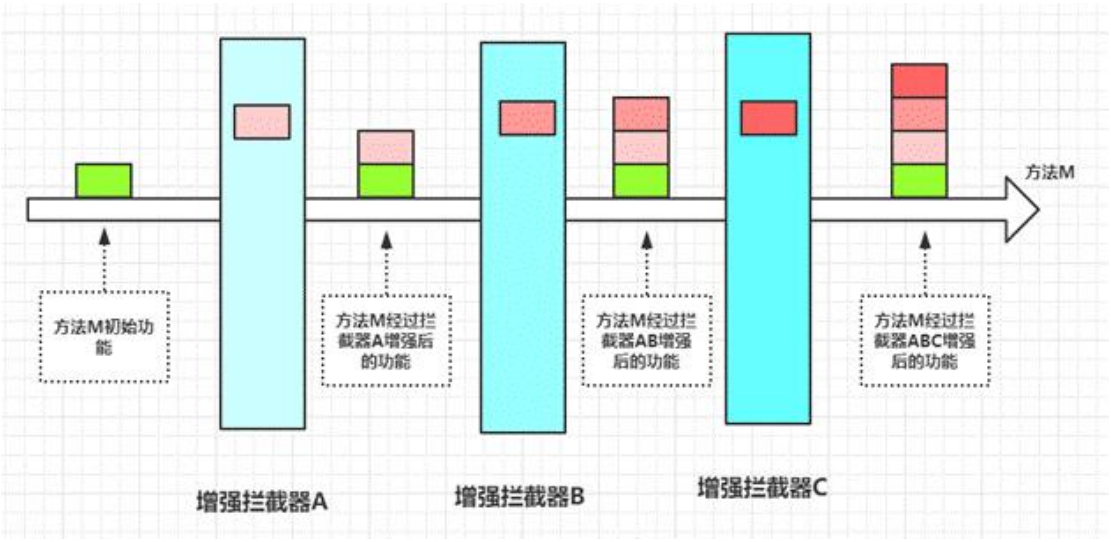
这种方式不适合数据量较大的场景，而且有可能会频繁访问数据库造成比较大的压力。

- Interceptor 拦截器实现，通过拦截需要分页的 select 语句，然后在这个 sql 语句里面动态拼接分页关键字，从而实现分页查询。

（如图）Interceptor 是 Mybatis 提供的一种针对不同生命周期的拦截器，比如：

  - 拦截执行器方法
  - 拦截参数的处理
  - 拦截结果集的处理
  - 拦截 SQL 语法构建的处理

我们可以拦截不同阶段的处理，来实现 Mybatis 相关功能的扩展。



这种方式的好处，就是可以提供统一的处理机制，不需要我们再单独去维护分页相关的功能。

- 插件（PageHelper）及（MyBaits-Plus、tkmybatis）框架实现

这些插件本质上也是使用 Mybatis 的拦截器来实现的。

只是他们帮我们实现了扩展和封装，节省了分页扩展封装的工作量，在实际开发中，只需要拿来即用即可。

总结一下，对于任何 ORM 框架，分页的实现逻辑无外乎两种，不管怎么包装，最终给到开发者的，只是使用上

的差异而已。

那么，我们来看看高手该如何回答。

53.214.3 高手回答

好的。

我认为有三种方式来实现分页：

- 第一种，直接在 Select 语句上增加数据库提供的分页关键字，然后在应用程序里面传递当前页，以及每页展示条数即可。
- 第二种，使用 Mybatis 提供的 RowBounds 对象，实现内存级别分页。
- 第三种，基于 Mybatis 里面的 Interceptor 拦截器，在 select 语句执行之前动态拼接分页关键字。

以上就是我的理解。

53.215 Java SPI 是什么？有什么用？

“Java SPI 是什么？有什么用？”

这是阿里 p6 面试过程中，第二面的时候遇到的一个真实的问题。

如果你不理解 SPI，建议你看完整个视频。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

下面来看看这个问题考察的目的

53.215.1 考察目标

这道题考察难度偏中等，对于没怎么去研究过源码的同学来说，SPI 是非常陌生的概念

考察人群主要还是 3 到 5 年比较多。

3~5 年属于中高端 Java 开发人群，因此考察目的也很明显：

- 了解求职者对于技术领域的理解程度，
- 实现高级开发的人才选拔

Java 这个行业没有人才评级标准，所以在面试的时候，面试官也比较难去界定你的职级。

所以在互联网企业，技术面的考察会比较深。

所以，要想回答好这个问题，还是要有一些自己的见解。

53.215.2 问题解析

Java SPI，全称是 Service Provider Interface。

它是一种基于接口的动态扩展机制，相当于 Java 里面提供了一套接口。

然后第三方可以实现这个接口来完成功能的扩展和实现。

（如图），举个简单的例子。

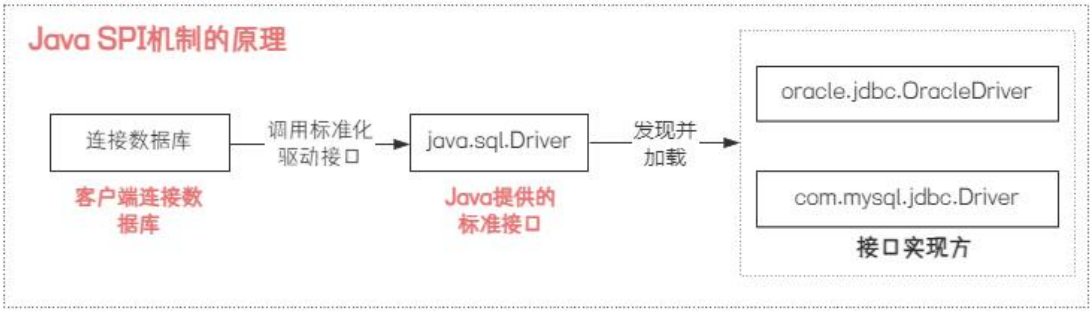
在 Java 的 SDK 里面，提供了一个数据库驱动的接口 java.sql.Driver。

它的作用是提供数据库的访问能力。

不过，在 Java 里面并没有提供实现，因为不同的数据库厂商，会有不同的语法和实现。

所以只能由第三方数据库厂商来实现，比如 Oracle 是 oracle.jdbc.OracleDriver，mysql 是 com.mysql.jdbc.Driver.

然后在应用开发的时候，根据集成的驱动实现连接到对应数据库。



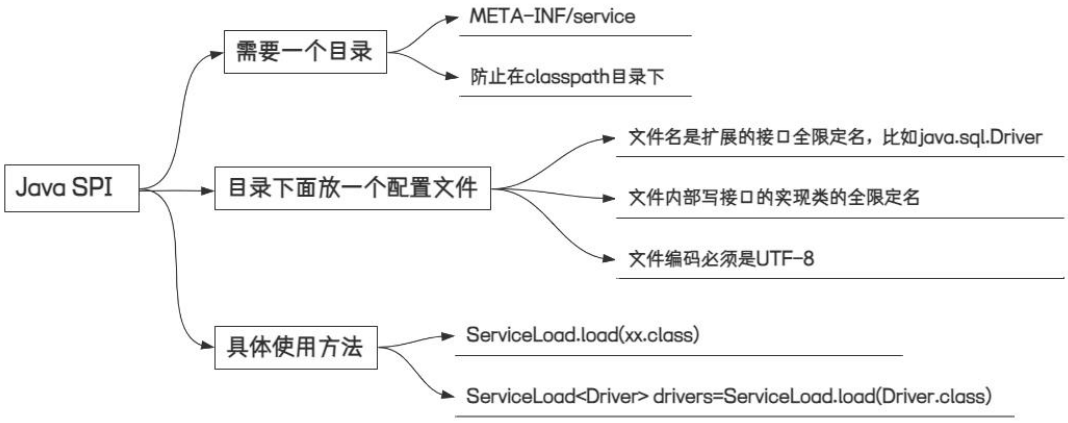
Java 中 SPI 机制主要思想是将装配的控制权移到程序之外实现标准和实现的解耦，以及提供动态可插拔的能力，

在模块化的设立中，这种思想非常重要。

实现 Java SPI，需要满足几个基本的格式（如图）：

- 需要先定义一个接口，作为扩展的标准
- 在 classpath 目录下创建 META-INF/service 文件目录
- 在这个目录下，以接口的全限定名命名的配置文件， 文件内容是这个接口的实现类
- 在应用程序里面，使用 ServiceLoad，就可以根据接口名称找到 classpath 所有的扩展时间

然后根据上下文场景选择实现类完成功能的调用。



Java SPI 有一定的不足之处，比如，不能根据需求去加载扩展实现，每次都会加载扩展接口的所有实现类并进行实例化，

实例化会造成性能开销，并且加载一些不需要用到的实现类，会导致内存资源的浪费，好了，下面看看高手的回答。

53.215.3 高手回答

Java SPI 是 Java 里面提供的一种接口扩展机制。

它的作用我认为有两个：

- 把标准定义和接口实现分离，在模块化开发中很好的实现了解耦
- 实现功能的扩展，更好的满足定制化的需求

除了 Java 的 SPI 以外，基于 SPI 思想的扩展实现还有很多，比如 Spring 里面的 SpringFactoriesLoader。

Dubbo 里面的 ExtensionLoader，并且 Dubbo 还在 SPI 基础上做了更进一步优化，提供了激活扩展点、自适应扩展点。

以上就是我的理解！

53.216 存储 MD5 的值应该用 VARCHAR 还是 CHAR

昨天一个工作 4 年的粉丝，遇到了一个很有意思的面试题。

“存储 MD5 的值应该用 VARCHAR 还是 CHAR”！

觉得是用 VARCHAR 类型的小伙伴，请仔细看完这一个视频。

Hi，大家好，我是 Mic，一个工作 14 年的 Java 程序员

53.216.1 考察目的

这个问题考察 1~4 年左右开发经验的同学。

大家肯定会觉得这个问题很简单啊，直接回答 VARCHAR 就行了。

但如果只是这么简单，你就太单纯了。

这个问题考察的目的至少有两个：

- 考察数据库里面的基本数据类型的理解
- 基于这个问题作为切入口，了解求职者对数据库的掌握程度

这个问题后面，一定伴随这“为什么”，

面试官一定是不不不去挖坑，让你往里面跳，直到你回答不上来位置。

所以我们在回答这个问题的时候，就需要更加全面的去回答。

53.216.2 问题分析

MD5 是由数字和字母组成的一个 16 位或者 32 位长度的字符串，一般在应用开发中都是使用 32 位。

看起来，我们用 varchar(32)或者 char(32)都可以存储，那用哪种更好呢?

要回答这个问题，必须要了解这两个类型的功能特性和区别。

- 第一个，char 是一个固定长度的字符串，Varchar 是一个可变长度的字符串

假设声明一个 char(10)的长度，如果存储字符串“abc”，虽然实际字符长度只有 3，但是 char 还是会占 10 个字节长度。

同样，如果用 varchar 存储，那它只会使用 3 个字符的实际长度来存储。

- 第二个，存储的效率不同，char 类型每次修改以后存储空间的长度不变，所以效率更高

varchar 每次修改数据都需要更新存储空间长度，效率较低

- 第三个，存储空间不同，char 不管实际数据大小，存储空间是固定的，而 varchar 存储空间等于实际数据长度，

所以 varchar 实际存储空间的使用要比 char 更小

基于他们特性的分析，可以得出一个基本的结论：

- char 适合存储比较短的且是固定长度的字符串
- varchar 适合存储可变长度的字符串

53.216.3 高手回答

好的，

我认为应该使用 Char 类型，原因是：

char 类型是固定长度的字符串，varchar 是可变长度字符串。

而 MD5 是一个固定长度的字符，不管数据怎么修改，长度不变，这个点很符合 char 类型。

另外，由于是固定长度，所以在数据变更的时候，不需要去调整存储空间大小，在效率上会比 varchar 好。

以上就是我的理解。

53.217 如何破坏双亲委派模型

“如何破坏双亲委派模型”！

这是一个工作 5 年的同学去美团面试遇到的面试题。

Hi，大家好，我是 Mic，咕泡科技联合创始人

下面来分析一下这个问题的考察目的

53.217.1 考察目的

这是一个偏 Java 基础的问题，考察 3 年以上的 Java 程序员。

这个问题考察目的有两个：

1. 了解求职者对于 Java 基础的掌握深度，类加载相关知识点挺多的，涉及到类加载器、类的生命周期、JVM 的工作原理，掌握这些基础可以快速解决程序中的一些问题，比如加载的类版本错误导致 NoSuchMethodException
2. 平时的工作中几乎不会用到类加载器，也不需要涉及到这方面的专业知识，所以这个问题很好的实现了人才能力的筛选

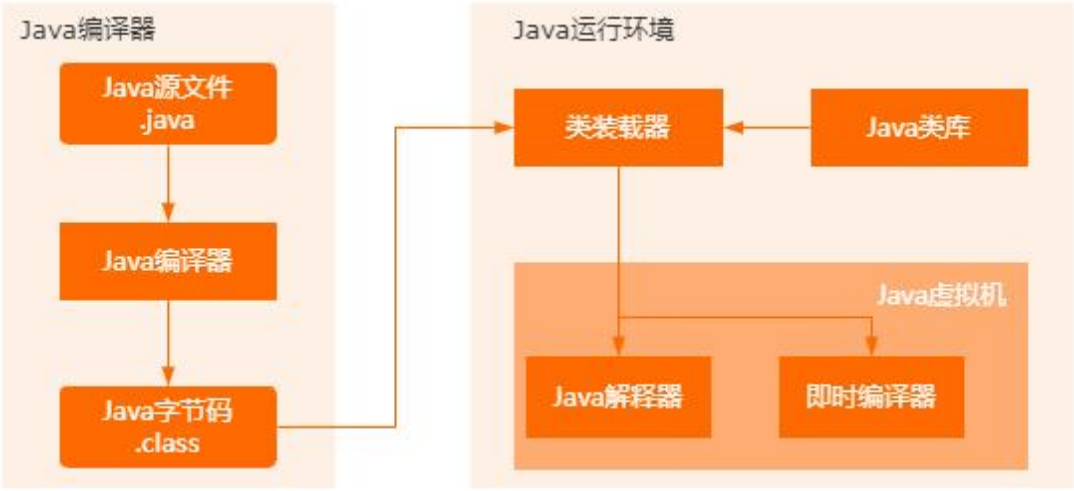
下面来分析一下这个问题的具体背景。

53.217.2 问题分析

我们自己写的 java 源文件到最终运行，必须要经过编译和类加载两个阶段（如图）。

编译的过程就是把.java 文件编译成.class 文件。

类加载的过程，就是把 class 文件装载到 JVM 内存中，装载完成以后就会得到一个 Class 对象，我们就可以使用 new 关键字来实例化这个对象。



（如图）而类的加载过程，需要涉及到类加载器。

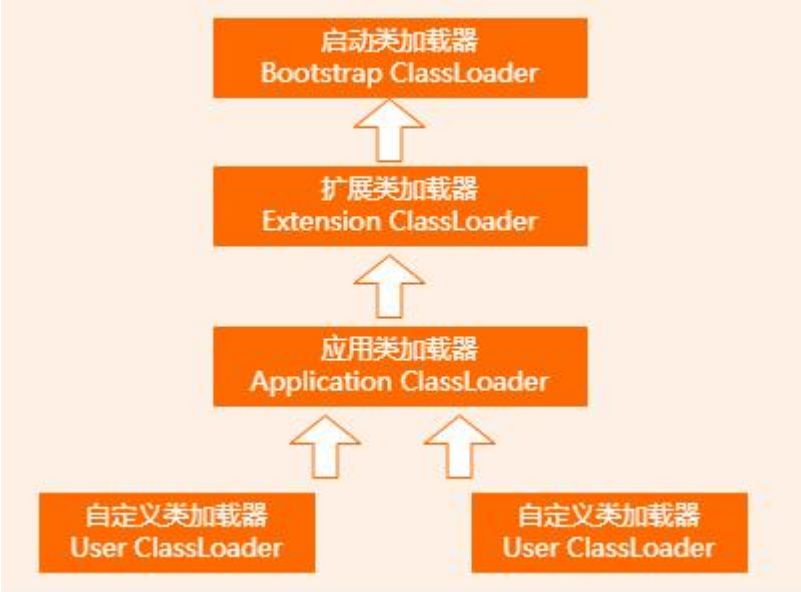
JVM 在运行的时候，会产生 3 个类加载器，这三个类加载器组成了一个层级关系

每个类加载器分别去加载不同作用范围的 jar 包，比如

- Bootstrap ClassLoader，主要是负责 Java 核心类库的加载，也就是 %{JDK_HOME}\lib 下的 rt.jar、resources.jar 等
- Extension ClassLoader，主要负责%{JDK_HOME}\lib\ext 目录下的 jar 包和 class 文件

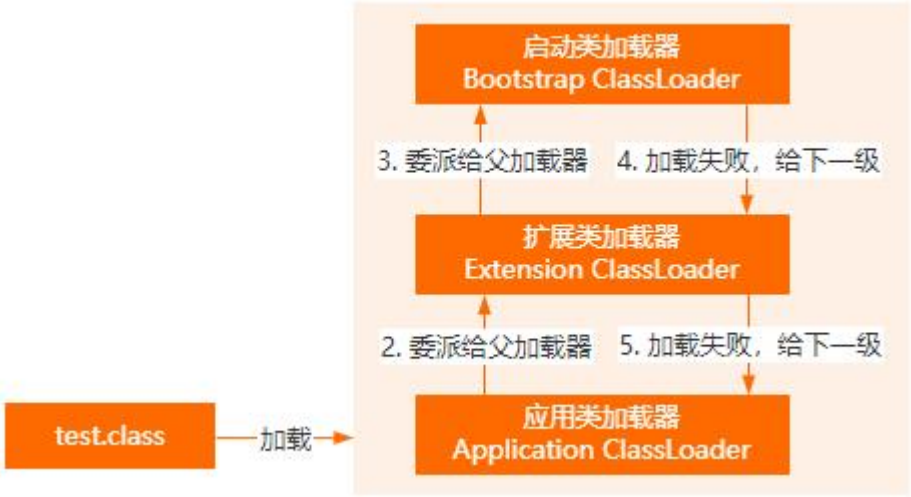
- Application ClassLoader，主要负责当前应用里面的 classpath 下的所有 jar 包和类文件

除了系统自己提供的类加载器以外，还可以通过 ClassLoader 类实现自定义加载器，去满足一些特殊场景的需求。



（如图）而双亲模型，就是按照类加载器的层级关系，逐层进行委派。

比如当需要加载一个 class 文件的时候，首先会把这个 class 的查询和加载委派给父加载器去执行，如果父加载器都无法加载，再尝试自己来加载这个 class。



不过，双亲委派并不是一个强制性的约束模型，我们可以通过一些方式去打破双亲委派模型。

这个打破的意思，就是类加载器可以加载不属于当前作用范围的类，实际上，JVM 本身就存在双亲委派被破坏的情况。

第一种情况，双亲委派是在 JDK1.2 版本发布的，而类加载器和抽象类 ClassLoader 在 JDK1.0 就已经存在了，用户可以通过重写 ClassLoader 里面的 loadClass()方法实现自定义类加载，JDK1.2 为了向前兼容，所以在设计的时候需要兼容 loadClass()重写的实现，导致双亲委派被破坏的情况。

同时，为了避免后续再出现这样的问题，不在提倡重写 loadClass()方法，而是使用 JDK1.2 中 ClassLoader 中提供了 findClass 方法来实现符合双亲委派规则的类加载逻辑。

第二种情况，在这个类加载模型中，有可能存在顶层类加载器加载的类，需要调用用户类加载器实现的代码的情况。

比如 java.jdbc.Driver 接口，它只是一个数据库驱动接口，这个接口是由启动类加载器加载的。

但是 java.jdbc.Driver 接口的实现是由各大数据库厂商来完成的，既然是自己实现的代码，就应该由应用类加载器来加载。

于是就出现了启动类加载器加载的类要调用应用类加载器加载的实现。

为了解决这个问题，在 JVM 中引入了线程上下文类加载器，它可以把原本需要启动类加载器加载的类，由应用类加载器进行加载。

除此之外，像 Tomcat 容器，也存在破坏双亲委派的情况，来实现不同应用之间的资源隔离。

了解了这些背景之后，我们来看看高手该怎么回答。

53.217.3 高手回答

好的，面试官。

我知道有两种方式来破坏双亲委派模型

1. 第一种，集成 ClassLoader 抽象类，重写 loadClass 方法，在这个方法可以自定义要加载的类使用的类加载器。

2. 第二种，使用线程上下文加载器，可以通过 java.lang.Thread 类的 setContextClassLoader()方法来设置当前类使用的类加载器类型。

53.218 Redis 哨兵机制和集群有什么区别?

“Redis 哨兵机制和集群有什么区别？”

这是一个工作了 2 年的同学面试的时候遇到的问题，如果你不知道怎么回答这个问题，记得认真看完这个视频。

Hi，大家好，我是 Mic，咕泡科技联合创始人

下面来分析这个问题的考察目的

53.218.1 考察目的

这个问题考察难度并不大，工作 3 年以上都会遇到，只是不同的工作年限对于面试的深度会有差异。

考察这个问题的目的无非就是看看求职者是否了解 Redis 的集群，以及是否有自己去搭建过 Redis 集群。

对于工作 2 年的同学来说，回答不需要太深入，但是 5 年以上的同学，会以这个问题作为切入口去深度考察求职者

对 Redis 集群和哨兵机制的底层原理。

53.218.2 问题分析

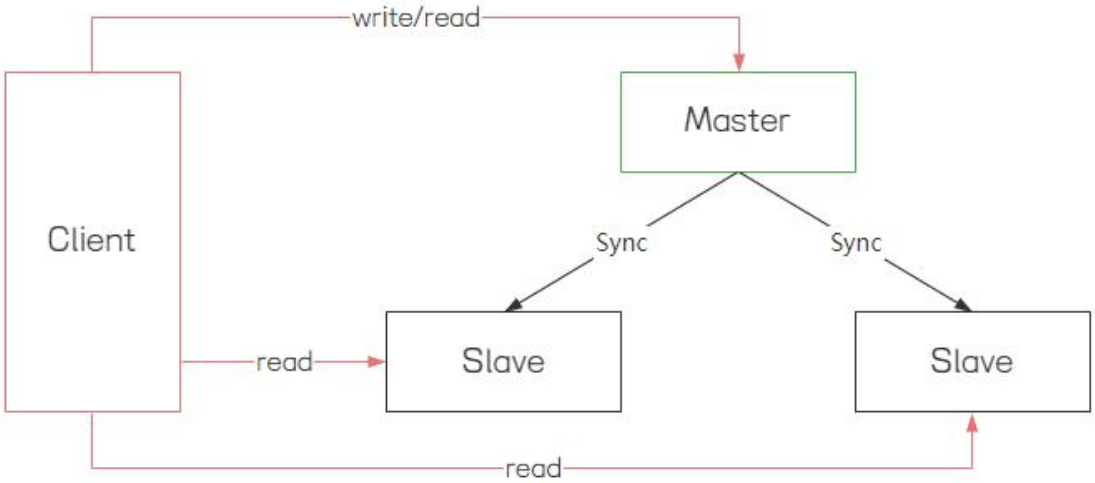
Redis 集群有几种实现方式，一个是主从集群、一个是 Redis Cluster。

（如图）主从集群，就是在 Redis 集中包括一个 Master 节点和多个 Slave 节点。

Master 负责数据的读写，Slave 节点负责数据的读取。

Master 上收到的数据变更，会同步到 Slave 节点上实现数据的同步。

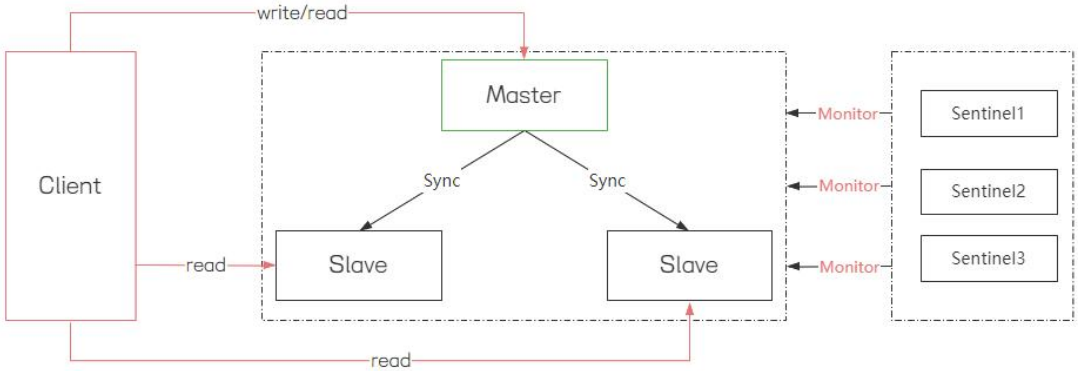
通过这种架构实现可以 Redis 的读写分离，提升数据的查询性能。



Redis 主从集群不提供容错和恢复功能，一旦 Master 节点挂了，不会自动选出新的 Master，导致后续客户端所有写请求直接失败。

所以Redis提供了哨兵机制，专门用来监听Redis主从集群提供故障的自动处理能力(如图）。

哨兵会监控 Redis 主从节点的状态，当 Master 节点出现故障，会自动从剩余的 Slave 节点中选一个新的 Master。



哨兵模式下虽然解决了 Master 选举的问题，但是在线扩容的问题还是没有解决。

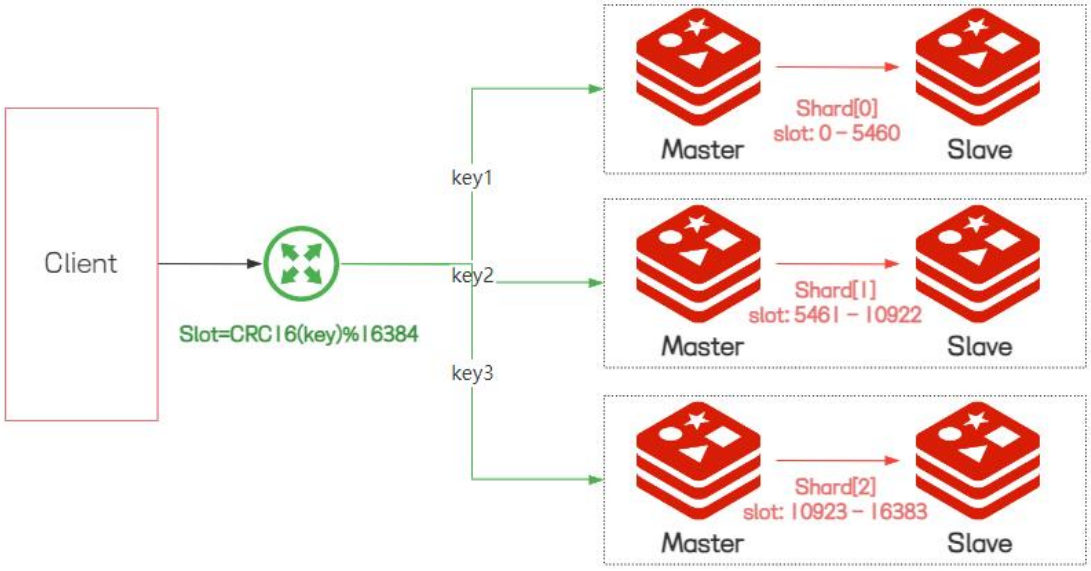
于是就有了第三种集群方式，Redis Cluster（如图），它实现了 Redis 的分布式存储，也就是每个节点存储不同的数据实现数据的分片。

在Redis Cluster中，引入了Slot槽来实现数据分片，Slot的整体取值范围是0~16383，每个节点会分配一个 Slot 区间

当我们存取 Key 的时候，Redis 根据 key 计算得到一个 Slot 的值，然后找到对应的节点进行数据的读写。

在高可用方面，Redis Cluster 引入了主从复制模式， 一个 Master 节点对应一个或多个 Slave 节点，当 Master 出现故障，

会从 Slave 节点中选举一个新的 Master 继续提供服务。



Redis Cluster 虽然解决了在线扩容以及故障转移的能力，但也同样有缺点，比如

- 客户端的实现会更加复杂
- Slave 节点只是一个冷备节点，不提供分担读操作的压力
- 对于 Redis 里面的批量操作指令会有限制

因此主从模式和 Cluster 模式各有优缺点，在使用的时候需要根据场景需求来选择。

53.218.3 高手回答

好的，面试官

因为 Redis 集群有两种，一种是主从复制，一种是 Redis Cluster，我不清楚您问的是哪一种。

按照我的理解，我认为您可能说的是 Redis 哨兵集群和 Redis Cluster 的区别。

对于这个问题，我认为可以从 3 个方面来回答

1. Redis 哨兵集群是基于主从复制来实现的，所以它可以实现读写分离，分担 Redis 读操作的压力

而 Redis Cluster 集群的 Slave 节点只是实现冷备机制，它只有在 Master 宕机之后才会工作。

2. Redis 哨兵集群无法在线扩容，所以它的并发压力受限于单个服务器的资源配置。Redis Cluster 提供了基于 Slot 槽的数据分片机制，可以实现在线扩容提升写数据的性能

3. 从集群架构上来说，Redis 哨兵集群是一主多从， 而 Redis Cluster 是多主多从

53.219 @Conditional 注解有什么用?

“@Conditional 注解有什么用？”

但凡有 Java 开发经验的同学，这个问题应该都要能回答出来。

如果回答不上来，建议认真看完这条视频。

Hi，大家好，我是 Mic，咕泡科技的联合创始人。

53.219.1 考察目的

@Condition 是 Spring4.x 版本引入的一个注解，由于这个问题本身比较简单，所以考察范围一般是工作 1 到 3 年左右。

考察目的就是了解求职者对 Spring Framework 里面注解的了解情况。

作为求职者，回答的时候不过过度解读面试官的意图，简单明了的回复即可。

53.219.2 问题分析

@Conditional 是 Spring4 版本里面提供的注解，它的作用是给需要装载的 Bean 增加一个条件判断，

只有满足条件的 Bean 才会装载到 IOC 容器。

@Conditional 注解的定义如图所示，从这个注解中可以了解到几个关键信息

- @Conditional 注解可以修饰在类或者方法上
- @Conditional 注解可以接收一个或多个实现了 Condition 接口的类。

```
//此注解可以标注在类和方法上
@Target({ElementType.TYPE, ElementType.METHOD})
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Conditional {
    Class<? extends Condition>[] value();
}
```

Condition 接口的定义如图所示，它提供了一个返回值为 boolean 的 matches 方法，

基于@Conditional 本身的作用

不难猜出它应该是用来实现 Bean 是否能被装载的判断逻辑的。

```
@FunctionalInterface
public interface Condition {
 boolean matches(ConditionContext context,
AnnotatedTypeMetadata metadata);

}
```

@Conditional 注解既然是用来判断 Bean 是否能被装载的条件，那么意味着我们可以

在 Bean 的描述逻辑上增加这样一个注解

然后通过重写 Condition 接口的 matches 方法，自定义 Bean 装载的条件。

比如下图这种使用方法，当 Spring 解析这个配置类的时候，HelloService 这个 bean

是否能被装载到 IOC 容器，

取决于 CustomizeCondition 里面的 matches 方法的返回值，返回 true 才可以被装载。

```
@Configuration
public class ConditionalConfiguration {

    @Conditional(CustomizeCondition.class)
    @Bean
    public HelloService helloService(){
        return new HelloService();
    }
}
```

这就给我们 Bean 的装载过程增加了很多的灵活性。

53.219.3 高手回答

@Conditional 注解的作用是为 Bean 的装载提供了一个条件判断。

只有满足条件的情况下，Spring 才会把当前 Bean 装载到 IOC 容器中。

这个条件的实现逻辑，我们可以实现 Condition 接口并重写 matches 方法自己去实现。

所以@Conditional 注解增加了 Bean 装载的灵活性。

在 Spring Boot 里面，对@Conditional 注解做了更进一步的扩展，比如增加了

@ConditionalOnClass、@ConditionalOnBean

等注解，使得我们在使用的过程中不再需要去写条件的逻辑。

53.220 请你说一下你对服务降级的理解

“请你说一下你对服务降级的理解”！

这是一个工作 5 年的同学去京东面试的时候遇到的问题。

如果屏幕前的你不知道怎么回答这个问题，可以认真看完这条视频。

Hi，大家好，我是 Mic，咕泡科技的联合创始人

下面我们来分析一下这个问题的考察意图

53.220.1 考察目的

服务降级这个问题主要考察工作 5 年以上的同学。

主要考察求职者是否了解服务降级，以及在实际工作中是否有参与过相关的设计。

服务降级本身就是一种兜底的设计方案，主要是出现在分布式架构的设计场景中。

问题难度不大，但是回答的时候逻辑性很重要。

53.220.2 问题分析

服务降级是一种提升系统稳定性和可用性的策略。

简单来说，就是当服务器压力增加的情况下，根据实际业务的需求和流量的情况，不对外提供部分服务的功能。

从而释放服务器的资源去保证核心业务的正常运行。

服务降级有两种方式，一种是主动降级，一种是基于特定情况的被动降级。

- 主动降级：这种方式在大促的时候使用比较多，比如在电商平台中，核心服务是下单、支付。

所以一般会把非核心服务比如评论服务关闭掉，这样就使得评论服务不会占用计算资源，从而保证核心服务的稳定运行

- 被动降级：它有两种主要的触发场景
  - 熔断触发降级， 在一个请求链路中，为了避免某个服务节点出现故障导致请求堆积，造成资源消耗是的服务崩溃的问题，一般会采取熔断策略。

当触发了熔断机制以后，如果后续再向故障节点发起请求的时候，这个请求不会发送到故障节点上，而是直接置为失败，这样就避免了请求堆积的问题。

而直接置为失败之后需要给到用户一个反馈，而这个反馈就是降级策略，就相当于给用户一个处理结果。

比如返回一个“系统繁忙”之类的信息。

  - 限流触发降级，因为系统资源是有限的，为了避免高并发流量把系统压垮导致不可用问题，所以我们会采取限流的策略去保护系统。通过限流去限制一部分用户的访问，然后保证整个系统的稳定运行

同样，触发了限流之后，需要给到用户一个反馈，这个反馈同样也称为降级策略。

比如可以反馈“当前访问人数较多，请稍候再试”，或者让这些用户排队，并显示当前排队的情况等。

因此，降级带来的结果是使得用户的体验下降，但是却保证了系统的稳定性和可用性。

53.220.3 高手回答

好的，面试官。

我认为，服务降级其实就是降低服务的能力等级。

在高并发流量下，因为系统资源有限，导致系统无法为高并发流量提供稳定可靠的支撑。

所以我们可以把一些非核心服务下掉，或者提供一些默认的处理结果，把这些计算资源腾出来给到核心服务去使用。

从而保证核心服务的稳定运行。

53.221 服务注册中心应该是 AP 还是 CP

在互联网去面试的时候，一定会考察求职者分布式架构领域相关的知识。

而注册中心是微服务架构里面最重要的核心组件，所以面试频率会比较高。

其中“服务注册中心应该是 AP 还是 CP 这个问题，就是最近一个工作了 5 年的粉丝遇到的一个问题。

Hi，大家好，我是 Mic，咕泡科技联合创始人

下面我们来分析一下面试官对于这个问题的考察意图。

53.221.1 考察目的

这个问题考察的难度中等偏上，主要考察工作 5 年以上的同学。

考察目的有两个：

4. 应聘的公司肯定是需要用到微服务架构，所以要了解求职者是否用过注册中心
5. 了解求职者是否深入理解注册中心，能够回答这个问题，至少说明对于注册中心的工作原理和价值是有清晰认知的

所以，对于这个问题，有两个回答的建议

6. 要有自己的总结和理解
7. 表达逻辑要足够清晰，不能想到哪说到哪。

53.221.2 问题分析

首先，要先了解 CAP 模型。

CAP 模型是说，在一个分布式系统里面，不可能同时满足三个点

- 一致性（Consistency），访问分布式系统中的每一个节点都能获得最新的数据。
- 可用性（Availability），每次请求都能获得一个有效的访问，但不保证数据是最新的。
- 分区容错性（Partition tolerance），分区相当于对通信耗时的要求，系统如果不能在时限范围内达成数据一致，

就意味着发生了分区的情况。

在 CAP 模型中只能满足 CP 或者 AP，之所以不能满足 CA，因为网络通信的不确定性可能会导致分区容错，

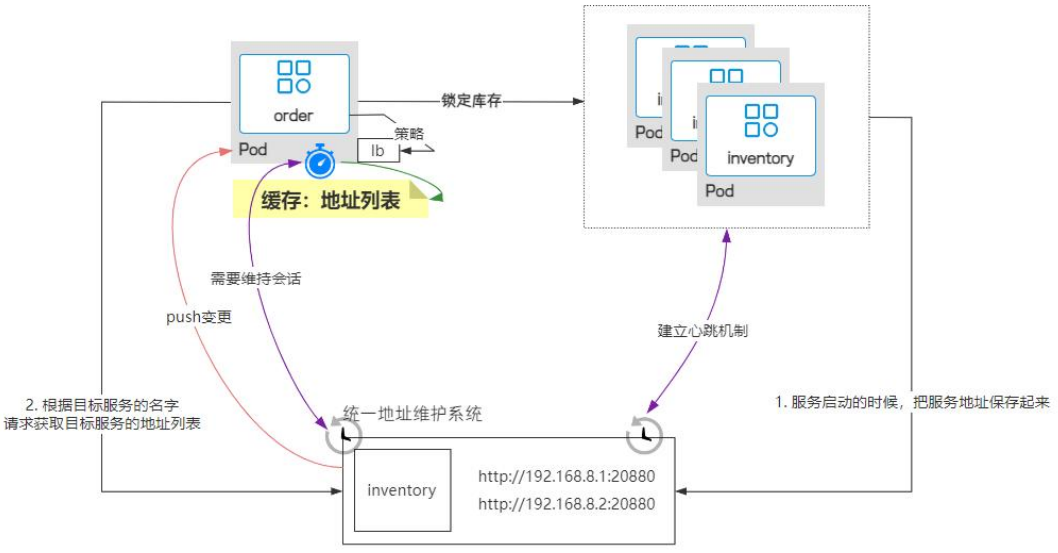
也就是分区容错性必然是存在的，因此我们只能在一致性和可用性之间做选择。

再回到注册中心（如图），服务注册中心的本质是为了提供服务地址的统一管理，以及提供一个服务动态感知的能力。

所以，注册中心应该要保证高可用性，也就是无论什么情况下，应用都能正常从注册中心获取到目标服务的通信地址。

当注册中心不可用的时候，不能影响服务之间的正常通信。

因此，从这个角度来说，注册中心应该是 AP 模型。



另外，对于服务动态感知这个场景来说，从服务地址失效到最终客户端感知到变化，必然会存在延迟。

也就是意味着客户端无法实时感知到出现故障的服务端节点。

既然一定会出现数据不一致的问题，就更加没必要去搭建一个 CP 模型的注册中心集群了。

否则反而会降低请求的性能。

53.221.3 高手回答

我认为注册中心应该是 AP 模型，原因有两个

1. 注册中心只是一个地址维护的平台，它如果出现故障，也不能影响服务之间的正常通信。
2. 注册中心的地址感知，本身就存在延迟，所以设计一个 CP 模型的架构意义并不大。

53.222 说说你对 CAP 的理解

分布式架构之所以复杂，就是因为增加了网络通信，而网络通信本身具有不确定性。

但是作为业务支撑的整个技术架构，所有业务的处理必须要具备确定性。

因此在这样一个矛盾下，导致架构变得更加复杂。

面试题”说说你对 CAP 的理解“这个问题，就是在这个背景下产生的

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员。

下面我们来分析一下面试官对于这个问题的考察意图。

53.222.1 考察目的

这个问题考察难度算中等，主要针对 5 年以上开发经验的同学。

考察目的有两个，

1. 了解求职者对于分布式架构的理解
2. 在分布式架构中，很多技术方案在落地的时候需要有取舍，要么是 CP、要么是 AP，理解 CAP 定理，能够更好的做出合适的选择

53.222.2 高手回答

CAP 模型，在一个分布式系统里面，不可能同时满足三个点

- 一致性（Consistency），访问分布式系统中的每一个节点都能获得最新的数据。

- 可用性（Availability），每次请求都能获得一个有效的访问，但不保证数据是最新的。
- 分区容错性（Partition tolerance），分区相当于对通信耗时的要求，系统如果不能在时限范围内达成数据一致，

就意味着发生了分区的情况。

在 CAP 模型中只能满足 CP 或者 AP，之所以不能满足 CA，因为网络通信的不确定性可能会导致分区容错，

也就是分区容错性必然是存在的，因此我们只能在一致性和可用性之间做选择。

以上就是我的理解。

53.223 请说一下分布式事务的理解和解决方案?

”请说一下分布式事务的理解和解决方案“

这是一个工作了 4 年的同学，在面试的时候遇到的问题。

Hi，大家好，我是 Mic，咕泡科技联合创始人

下面我们来分析一下面试官对于这个问题的考察意图。

53.223.1 考察目的

这个问题考察难度不算大，考察 4 年以上 Java 开发经验的同学

分布式事务主要涉及到跨库事务处理问题，除了考察求职者对这方面知识的了解以外，还想了解一下求职者的实际处理经验。

53.223.2 问题分析

通常情况下，传统的关系型数据库只能保证单个数据库中多个数据表的事务特性。

一旦多个 SQL 操作涉及到多个数据库，这类的事务无法解决跨库事务问题。

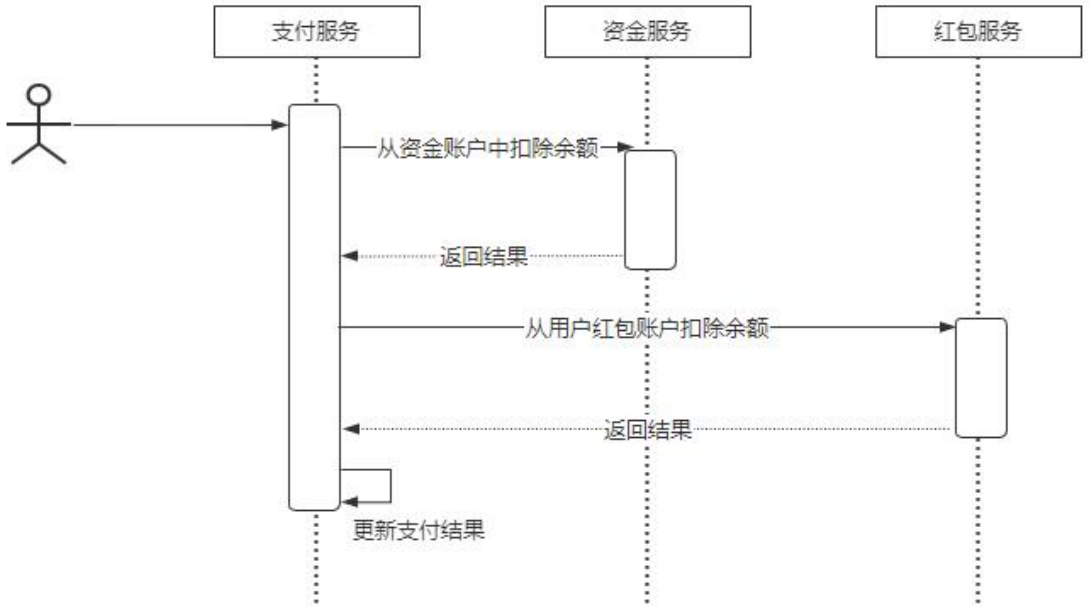
在传统架构下，这种问题出现的情况非常少，但是在分布式微服务架构中，分布式事务的问题变得更加突出。

以电商项目为例，假设我们要实现电商系统中的支付功能，它的实现流程如下（如图）。

在微服务架构中，应用被拆分成以业务模块为单元的服务，并且每个服务有自己的数据库系统。

当用户发起支付时，会涉及到以下几个事务操作：

1. 创建支付订单
2. 从资金服务中扣除余额
3. 从红包服务中扣除余额
4. 更新支付结果



这是四个典型的事务操作，而且这些操作分别属于不同的数据库，最终期望的结果是希望这三个服务所对应的数据是一致的，很显然传统的事务无法解决这个问题！

因此就产生了分布式事务的问题，所谓分布式事务，就是事务具有分布式特性，简单理解就是如何实现多个跨数据库的小事务组成的大事务的 ACID 特性。

下面来看看高手的回答。

53.223.3 高手回答

好的

分布式事务是指存在多个跨库事务的事务一致性问题，

或者是指在分布式架构下由多个应用节点组成的多个事务之间的事务一致性问题。

目前主流的分布式事务解决方案有两种

- 一种是基于 XA 协议实现的强一致性事务方案，比如 Atomikos、Seata 中的 XA 事务模型。

基于 CAP 理论可以知道，如果要保证分布式事务的强一致性，就必然会带来性能的影响从而影响到可用性。
所以强一致性事务性能会比较低。

- 另一种是基于 BASE 理论下的弱一致性事务解决方案，比如 TCC 事务模型、基于可靠性消息的最终一致性方案、Seata 的 Saga 事务模型等。

最终一致性事务损失了数据的强一致性，通过异步补偿的方式达到数据的最终一致。
因此在性能上比较好，适用于并发量比较高的场景。

53.224 Redis 主从复制的原理

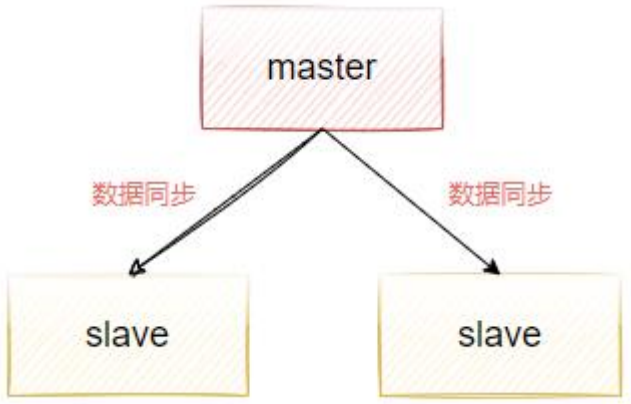
一个工作了 5 年的粉丝，在面试的时候遇到“Redis 主从复制的原理”这个问题。
他回答这个问题的时候断断续续的，没有一个比较清晰的逻辑性，导致面试没发挥好。
今天这个视频，给大家分享一下遇到这类问题，正确的回答方法。
Hi，大家好，我是 Mic，咕泡科技联合创始人
下面来分析一下面试官的考察意图。

53.224.1 考察目的

这个问题还是有一定的深度，平时在工作中很少会涉及到这方面的内容。
从这个问题的考察意图来看，很明显是考察求职者对于 Redis 底层原理的理解程度。
另外，目标公司可能需要涉及到 Redis 中间件的日常运维。
同时，面试官也是通过技术原理来达到筛选高级或者资深 Java 开发的目的。

53.224.2 问题分析

Redis 主从复制，是指在 Redis 集群里面（如图），Master 节点和 Slave 节点数据同步的一种机制。
简单来说就是把一台 Redis 服务器的数据，复制到其他 Redis 服务器中。
其中负责复制数据的来源称为 master，被动接收数据并同步的节点称为 slave



在 Redis 里面，提供了全量复制和增量复制两种模式。

全量复制一般发生在 Slave 节点初始化阶段，这个时候需要把 master 上所有数据都复制一份。

具体的工作原理是：

1. Slave 向 Master 发送 SYNC 命令，Master 收到命令以后生成数据快照
2. 把快照数据发送给 Slave 节点，Salve 节点收到数据后丢弃旧的数据，并重新载入新的数据

需要注意，在主从复制过程中，Redis 并没有采用实现强数据一致性，因此会存在一定时间的数据不一致问题。

增量复制，就是指 Master 收到数据变更之后，把变更的数据同步给所有 Slave 节点。

增量复制的原理是，Master 和 Slave 都会维护一个复制偏移量（offset），用来表示 Master 向 Slave 传递的字节数。

每次传输数据，Master 和 Slave 维护的 Offset 都会增加对应的字节数量。

Redis 只需要根据 Offset 就可以实现增量数据同步了。

53.224.3 高手回答

好的，Redis 主从复制包括全量复制和增量复制。

全量复制是发生在初始化阶段，从节点会主动向主节点发起一个同步请求，主节点收到请求后会

会生成一份当前数据的快照发送给从节点，从节点收到数据进行加载后完成全量复制。

增量复制是发生在每次 Master 数据发生变化的过程中，会把变化的数据同步给所有的从节点。

增量复制是通过维护 Offset 这个复制偏移量来实现的。

以上就是我的理解。

53.225 简述雪花算法的实现原理

一个工作了 4 年的粉丝，去京东面试，遇到了这样一个问题

”请你简述雪花算法的实现原理”

屏幕前的小伙伴，如果你遇到这个问题，知道怎么回答吗?

Hi，大家好，我是 Mic，咕泡科技联合创始人

下面来分析一下面试官的考察意图。

53.225.1 考察目的

雪花算法是一个全局唯一算法，它主要出现在像分库分表场景中作为业务主键、

或者作为一些像订单号这类的 id 生成器。

这个问题就是考察求职者对于雪花算法的了解。

而且面试官问的是实现原理，那么意味着要至少要说明雪花算法的整体设计、以及实现方式。

53.225.2 问题分析

雪花算法一般用来实现全局唯一的业务主键，解决分库分表之后主键 id 的唯一性问题。

所以单纯就全局唯一性质来说，有很多的实现方式，比如

- UUID
- Redis 的原子递增
- 数据库全局表的自增 id
- 等等

但是在实际应用中，还需要满足有序递增、高性能、带时间戳等。

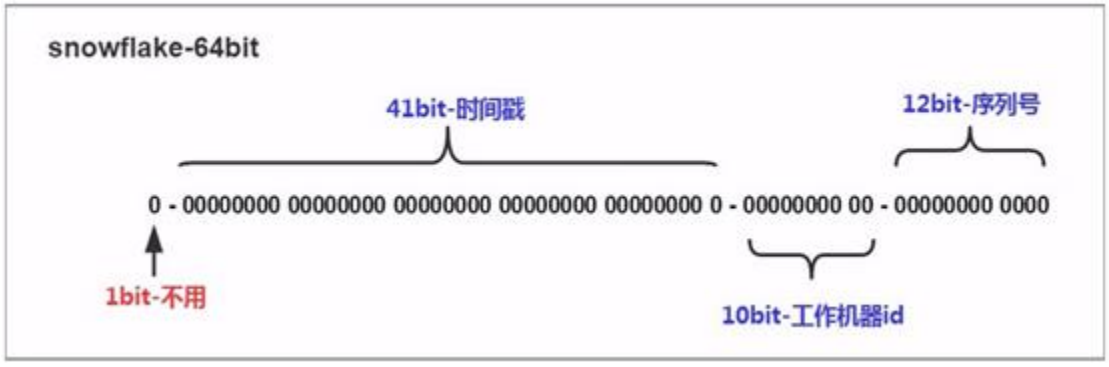
而雪花算法就是一个比较符合这类特征的全局唯一算法，在美团的 Leaf 组件中也有用到。

它是由一个 64 位的 long 类型数字组成，分为四个部分。

- 第一部分，用 1 个 bit 表示符号位，一般情况下是 0
- 第二部分，用 41 个 bit 来表示时间戳，使用系统时间的毫秒数
- 第三部分，用 10 个 bit 来记录工作机器 id，这样就可以保证在多个服务器上生成的 id 的唯一性。

如果存在跨机房部署，我们还可以把它分成两个 5bit，前面 5 个 bit 可以表示机房 id，后面 5 个 bit 可以表示机器 id。

- 第四个部分，用 12 个 bit 表示序列号，表示一个递增序列，用来记录同毫秒内产生的不同 id



雪花算法，就是根据这四个部分的规则，生成对应的 bit 位数据，然后组装在一起形成一个全局唯一 id。

53.225.3 高手回答

好的。

雪花算法是一种生成分布式全局唯一 ID 的算法，它会得到一个 64 位长度的 long 类型数据。

其中这 64 位的数据，由 4 个部分组成

- 第一个 bit 位是符号位，因为 id 不会是负数，所以它一般是 0
- 接着用 41 个 bit 位来表示毫秒单位的时间戳
- 再用 10 个 bit 位来表示工作机器 id
- 最后 12 个 bit 位表示递增的序列号

把这 64 个 bit 位拼接成一个 long 类型的数字，就是雪花算法的实现。

53.226 Integer 和 int 的区别？Java 为什么要设计封装类?

“Integer 和 int 的区别？Java 为什么要设计封装类？”

虽然经常使用，但是很少去关注为什么，导致面试遇到这个问题就懵了。

Hi，大家好，我是咕泡科技的联合创始人 Mic

关于这个问题，我们来分析一下面试官的考察意图

53.226.1 考察目的

这是一个典型的 Java 基础问题，本质上来说，这个问题是考察求职者对于面向对象的理解程度。

也是在考察求职者的基本功，越是简单常见的东西，就越能体现求职者的基础扎实程度。

这类问题一般是考察 1~3 年开发经验的同学。

在回答这个问题的时候，尽量从封装类型的特性和功能全方位的去回答。

53.226.2 问题分析

Integer 是基本数据类型 int 的封装类

在 Java 里面，有八种基本数据类型，他们都有一一对应的封装类型。

基本类型和封装类型的区别有很多，比如

- int 类型，我们可以直接定义一个变量名赋值即可，但是 Integer 需要使用 new 关键字创建对象

- 基本类型和 Integer 类型混合使用时，Java 会自动通过拆箱和装箱实现类型转换
- Integer 作为一个对象类型，封装了一些方法和属性，我们可以利用这些方法来操作数据。
- 作为成员变量，Integer 的默认值是 null，而 int 的默认值是 0

要是真正列数出来，还可以挖掘更多的差异点。

在 Java 里面，之所以要对基础类型设计一个对应的封装类型。

是因为 Java 本身是一门面向对象的语言，对象是 Java 语言的基础单元，我们时时刻刻都在创建对象，也随时都在使用对象，

很多时候在传递数据时也需要对象类型，比如像 ArrayList、HashMap 这些集合，只能存储对象类型，

因此从这个点来说，封装类型存在的意义就很大。

其次，封装类型还有很多好处，比如

- 安全性较好，可以避免外部操作随意修改成员变量的值，保证了成员变量和数据传递的安全性
- 隐藏了实现细节，对使用者更加友好，只需要调用对象提供的方法就可以完成对应的操作

下面来看看高手的回答

53.226.3 高手回答

好的

Integer 和 int 的区别有很多，我简单说 3 个方面

- Integer 的初始值是 null，int 的初始值是 0
- Integer 存储在堆内存，int 类型是直接存储在栈空间
- Integer 是对象类型，它封装了很多的方法和属性，我们在使用的时候更加灵活。

至于为什么要设计封装类型，最主要的原因是 Java 本身是面向对象的语言，一切操作都是以对象作为基础。

比如像集合里面存储的元素，也只支持存储 Object 类型，普通类型无法通过集合来存储。

以上就是我的理解

53.227 Integer a1=100 Integer a2=100，a1==a2?的运行结果?

一个工作了 3 年的同学去面试遇到这样一个问题。

“Integer a1=100、Integer a2=100，请问 a1==a2 的运行结果以及为什么？”

如果屏幕前的你们也同样不知道，记得看完整个视频。

下面来看看面试官的考察目的

53.227.1 考察目的

这个问题主要考察 Java 基础知识，涉及到的知识点还挺多的。

比如，==号表示的内存地址匹配、装箱拆箱、Integer 内部内部的设计原理。

大部分同学能够熟练使用 Integer，但是不一定去了解过原理，但是作为一个 3 年以上的开发。

对于 Java 基础必须要知其然还得知其所以然。

53.227.2 问题分析

按照大家对于 Java 基础的认知，两个独立的对象用==进行比较，是比较两个对象的内存地址。

那得到的结果必然是 false。但是在这个场景中，得到的结果是 true。

为什么呢?

首先， Integer a1=100， 把一个 int 数字赋值给一个封装类型，Java 会默认进行装箱操作，也就是调用 Integer.valueOf()

方法，把数字 100 包装成封装类型 Integer。

其次，在 Integer 内部设计中，用到了享元模式的设计，享元模式的核心思想是通过复用对象，减少对象的创建数量，

从而减少内存占用和提升性能。

Integer 内部维护了一个 IntegerCache，它缓存了-128 到 127 这个区间的数值对应的 Integer 类型。

一旦程序调用 valueOf 方法，如果数字是在-128 到 127 之间就直接在 cache 缓存数组中去取 Integer 对象。

否则，就会创建一个新的对象。

所以，对于这个面试题来说，两个 Integer 对象，因为值都是 100，并且默认通过装箱机制调用了 valueOf 方法。

从 IntegerCache 中拿到了两个完全相同的 Integer 实例。

因此用等号比较得到的结果必然是 true。

53.227.3 高手回答

好的，

a1==a2 的执行结果是 true

原因是 Integer 内部用到了享元模式的设计，针对-128 到 127 之间的数字做了缓存。

使用 Integer a1=100 这个方式赋值时，Java 默认会通过 valueOf 对 100 这个数字进行装箱操作，

从而触发了缓存机制，使得 a1 和 a2 指向了同一个 Integer 地址空间。

以上就是我的理解。

53.228 HashMap 与 HashTable 区别

“HashMap 和 HashTable 有什么区别？”

这是一道非常非常基础的面试题，但很多人却回答不好，没有逻辑性

今天这个视频就给大家分享一下，高手和普通人遇到这个问题的区别。

下面先来分享一下这个问题的考察目的

53.228.1 考察目的

这个问题一般考察 1~3 年开发经验。

考察目的仍然是看求职者对 Java 基础的掌握程度。

集合是 Java 基础中非常重要的组件，除了根据不同数据结构选择合适的集合类。

还需要从安全性、性能、功能特性角度去了解集合的差异性以及底层工作原理。

如果达不到这个层次，在使用的时候遇到问题影响就比较大。

因此这也是人才筛选比较基础的一部分。

53.228.2 问题分析

Hashtable 和 HashMap 都是一个基于 hash 表实现的 K-V 结构的集合。

Hashtable 是 JDK1.0 引入的一个线程安全的集合类，因为所有数据访问的方法都加了一个 Synchronized 同步锁。

Hashtable 内部采用数组加链表来实现，链表用来解决 hash 冲突的问题。

HashMap 是 JDK1.2 引入的一个线程不安全的集合类，

HashMap 内部也是采用了数组加链表实现，在 JDK1.8 版本里面做了优化，引入了红黑树。

当链表长度大于等于 8 并且数组长度大于 64 的时候，就会把链表转化为红黑树，提升数据查找性能。

53.228.3 高手回答

好的，

- 从功能特性的角度来说
  - HashTable 是线程安全的，而 HashMap 不是。
  - HashMap 的性能要比 HashTable 更好，因为，HashTable 采用了全局同步锁来保证安全性，对性能影响较大
- 从内部实现的角度来说
  - HashTable 使用数组加链表、HashMap 采用了数组+链表+红黑树
  - HashMap 初始容量是 16、HashTable 初始容量是 11
  - HashMap 可以使用 null 作为 key，HashMap 会把 null 转化为 0 进行存储，而 Hashtable 不允许。

最后，他们两个的 key 的散列算法不同，HashTable 直接是使用 key 的 hashcode 对数组长度做取模。

而 HashMap 对 key 的 hashcode 做了二次散列，从而避免 key 的分布不均匀问题影响到查询性能。

以上就是我的理解。

53.229 介绍下策略模式和观察者模式?

一个工作了 5 年的粉丝，去字节面试，在第一面遇到这样一个问题

“请你介绍一些策略模式和观察者模式？”

这个粉丝思路突然就短路了，不知道该怎么回答?

他要是早点看到这个视频，估计就已经拿到 Offer 了。

53.229.1 问题分析

在 Java 里面，有 23 种设计模式

而在实际开发中，用到的设计模式屈指可数，主要有两方面的原因

- 目前的开发模式，基本上按照 MVC 这一套在搞，大部分业务逻辑的实现都不复杂
- 对设计模式的理解不够，只能生搬硬套，不仅仅没带来好处，还让程序处理变得更麻烦

但是，设计模式确实是无数前辈在软件开发过程中总结的一些经验，他们能够使得程序更加灵活可扩展

有人把它总结成了公式化的 23 种设计模式，导致大家以为按照这个公式去搬运就可以，但实际上我认为。

设计模式应该是一种软件设计的思想或者方法论，它不应该固化成某种特定的公式，它的运用应该更加灵活。

这 23 种设计模式可以分成三种类型分别是创建型、结构型、行为型。

策略模式和观察者模式属于行为型模式。

行为型模式主要用来描述多个类和对象之间的相互协同完成单个对象无法单独完成的任务，除了这两种以外，

还包括模版方法、状态模式、责任链模式、解释器模式等。

53.229.2 高手回答

策略模式和观察者模式属于行为型模式。

策略模式主要是用在根据上下文动态控制类的行为的场景，

- 一方面可以解决多个 if...else 判断带来的代码复杂性和维护性问题

- 另一方面，把类的不同行为进行封装，使得程序可以进行动态的扩展和替换，增加了程序的灵活性。

像支付路由这种场景，就可以使用策略模式实现。

观察者模式主要用在一对多的对象依赖关系的中，实现某一个对象状态变更之后的感知的场景

- 一方面可以降低对象依赖关系的耦合度，弱化依赖关系。
- 另一方面，通过这种状态通知机制，可以保证这些依赖对象之间的状态协同。

在 Spring 源码里面有大量运用这种观察者模式实现事件的传播和感知。

以上就是我的理解。

53.230 为什么重写 equals() 就一定要重写 hashCode() 方法?

“为什么重写 equals() 就一定要重写 hashCode() 方法?”

一个工作了 4 年的粉丝，好不容易拿到一个面试机会，结果就被这个问题暴击了

没办法，只能来向我求助了。

回答这个问题之前，我们先来分析一下这个问题的背景。

53.230.1 问题分析

关于这个问题，首先需要深入了解一下 equals 这个方法。

这个 `equals` 方法是 String 这个类里面的实现。

从代码中可以看到，当调用 equals 比较两个对象的时候，会做两个操作

1. 用`==`号比较两个对象的内存地址，如果地址相同则返回 true
2. 否则，继续比较字符串的值，如果两个字符串的值完全相等，同样返回 true

```java
public boolean equals(Object anObject) {
      if (this == anObject) { //1:  == 比较内存
          return true;
      }
      if (anObject instanceof String) { //2: 比较字符串的值
          String anotherString = (String)anObject;
          int n = value.length;
          if (n == anotherString.value.length) {
              char v1[] = value;
              char v2[] = anotherString.value;
              int i = 0;
              while (n-- != 0) {
                  if (v1[i] != v2[i])
                      return false;
                  i++;
              }
              return true;
          }
      }
      return false;
  }
```

那 equals 和 hashCode()有什么关系呢?

- 首先，Java 里面任何一个对象都有一个 native 的 hashCode()方法
- 其次，这个方法在散列集合中会用到，比如 HashTable、HashMap 这些，当添加元素的时候，需要判断元素是否存在，

而如果用 equals 效率太低，所以一般是直接用对象的 hashCode 的值进行取模运算。

  - 如果 table 中没有该 hashcode 值，它就可以直接存进去，不用再进行任何比较了；
  - 如果存在该 hashcode 值， 就调用它的 equals 方法与新元素进行比较，相同的话就不存了，不相同就散列其它的地址，所以这里存在一个冲突解决的问题，这样一来实际调用 equals 方法的次数就大大降低了.

hashCode 的值默认是 JVM 使用随机数来生成的，两个不同的对象，可能生成的 HashCode 会相同。

这种情况在 Hash 表里面就是所谓的哈希冲突，通常会使用链表或者线性探测等方式来解决冲突问题。

但是如果两个完全相同的对象，也就是内存地址指向同一个，那么他们的 hashCode 一定是相同的。

了解了 equals 和 hashCode 的关系以后，再来分析这个面试题。
在理论情况下，如果 `x.equals(y)==true`，如果没有重写 equals 方法，那么这两个对象的内存地址是同一个，意味着 hashCode 必然相等。
但是如果我们只重写了 equals 方法，就有可能导致 hashCode 不相同。
一旦出现这种情况，就导致这个类无法和所有集合类一起工作。
所以，在实际开发中，约定俗成了一条规则，重写 `equals` 方法的同时也需要重写 `hashCode` 方法。

53.230.2 高手回答

好的。
如果只重写 equals 方法，不重写 hashCode 方法。
就有可能导致 `a.equals(b)`这个表达式成立，但是 hashCode 却不同。
那么这个只重写了 `equals` 方法的对象，在使用散列集合进行存储的时候就会出现问题。
因为散列结合是使用 hashCode 来计算 key 的存储位置，如果存储两个完全相同的对象，但是有不同的 hashcode
就会导致这两个对象存储在 hash 表的不同位置，当我们想根据这个对象去获取数据的时候，就会出现一个悖论
一个完全相同的对象会在存储在 hash 表的两个位置，造成大家约定俗成的规则，出现一些不可预料的错误。

53.231 Java 反射的优缺点?

“请你 Java 反射的优缺点？”
这是一个工作了 5 年的同学，去阿里面试第一面遇到的问题。
今天这个视频给大家分享一下我们遇到这个问题该怎么回答，才能获得面试官的认可。

53.231.1 问题分析

反射是 Java 语言里面比较重要的一个特征。

它能够在程序运行的过程中去构造任意一个类对象、并且可以获取任意一个类的成员变量、成员方法、属性，以及调用任意一个对象的方法。

通过反射的能力，可以让 Java 语言支持动态获取程序信息以及动态调用方法的能力。

在 Java 里面，专门有一个 java.lang.reflect 用来实现反射相关的类库，包括 Construct、Field、Method 等类，

分别用来获取类的构造方法、成员变量、方法信息。

反射的使用场景还挺多的，比如在动态代理的场景中，使用动态生成的代理类来提升代码的复用性。

在 Spring 框架中，有大量用到反射，比如用反射来实例化 Bean 对象。

53.231.2 高手回答

好的，

Java 反射的优点：

- 增加程序的灵活性，可以在运行的过程中动态对类进行修改和操作
- 提高代码的复用率，比如动态代理，就是用到了反射来实现
- 可以在运行时轻松获取任意一个类的方法、属性，并且还能通过反射进行动态调用

Java 反射的缺点：

- 反射会涉及到动态类型的解析，所以 JVM 无法对这些代码进行优化，导致性能要比非反射调用更低。
- 使用反射以后，代码的可读性会下降
- 反射可以绕过一些限制访问的属性或者方法，可能会导致破坏了代码本身的抽象性

53.232 Redis 有哪些过期策略

昨天，一个工作了 6 年的粉丝去面试，他顺利的通过了三轮技术面试并拿到了 Offer。

然后向我表达了感激，因为这段时间他一直在刷我的面试视频和面试文档。

他说面试中遇到的问题，都在这个文档里面写了。

如果大家要去面试，可以去评论区置顶中领取这个文档。

今天给大家分享的面试题是：Redis 有哪些过期策略。

在 Redis 里面，有三种过期策略分别是：定时过期、惰性过期、定期过期

定时过期，是指每个设置过期时间的 key 都需要创建一个定时器，到过期时间就会立即对 key 进行清除。

该策略可以立即清除过期的数据，对内存很友好；但是会占用大量的 CPU 资源去处理过期的数据，从而影响缓存的响应时间和吞吐量

惰性过期：是指被动访问某个 key 的时候，才会判断 key 是否已过期，过期则清除。

该策略可以最大化地节省 CPU 资源，却对内存非常不友好。极端情况可能出现大量的过期 key 没有再次被访问，从而不会被清除，占用大量内存。

但是这两种方案都有点极端，Redis 里面还有另外一种折中的方案，就是定期过期。

这种方案会周期性轮询 redis 库中的时效性数据，采用随机抽取的策略，利用过期数据占比的方式控制删除频率。

这种方式可以通过调整定时扫描的时间间隔和每次扫描耗时，实现在不同情况下使得 CPU 和内存资源达到最优的平衡效果。

而 Redis 中同时使用了**惰性过期和定期过期**两种过期策略。

好的，今天的分享就到这里，感谢大家的关注和点赞，再见。

53.233 什么是热 Key 问题，如何解决热 key 问题

昨天，一个工作 3 年的粉丝去面试，被问到：什么是热点 key 问题以及如何解决。

如果不知道这个问题该怎么回答，可以在我整理的 30W 字的大厂面筋指南中找到，需要的小伙伴可以在评论区的置顶中获取。

下面我给大家分享一下我的回答。

关于热点 key，简单理解，就是访问频率发很高的 key，由于访问量较大，热点 key 有可能会导致服务器资源不足出现宕机的问题。

热点 key 一般有几种产生的情况：

- 在一些高并发的场景中，比如秒杀、热搜等。
- 由于资源分配不平衡导致访问热点问题

一般情况下，我们可以通过监控工具、或者客户端程序上报的方式来识别热点 key。

由于热点 Key 可能存在的影响， 所以我们一般需要去尽可能的规避，从而避免出现性能上和稳定性问题，通常的解决方案有以下几种：

- 把热点 key 进行拆分，实现并发流量的分流
- 使用多级缓存的设计，通过增加本地缓存的方式减少目标节点的访问
- 通过对访问频率较高的节点进行扩容，通过负载均衡的方式分散流量

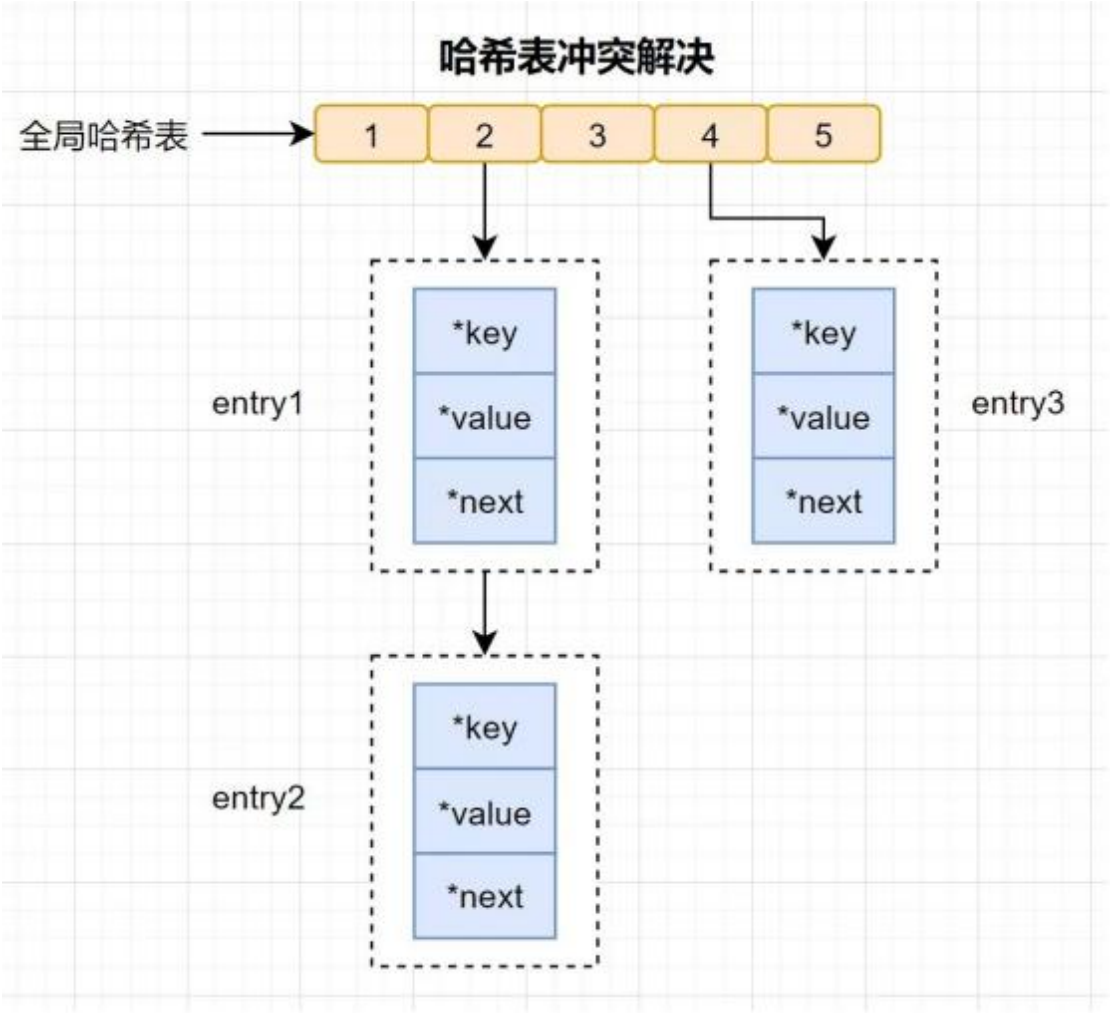
好的，今天的分享就到这里，感谢大家的关注和点赞，再见。

53.234 Redis 遇到 Hash 冲突怎么办?

昨天，一个工作 5 年的粉丝去面试，被问到 Redis 遇到 Hash 冲突的问题时，是怎么去解决的。

这其实是一个比较通用的问题，在我整理的大厂面筋指南中有这个问题的完整回答，有需要的小伙伴可以在评论区的置顶中去领取。

首先，Redis 里面的的 Hash 结构的底层数据结构（如图），是使用了一张全局 hash 表来保存的所有键值对，这张哈希表，有多个哈希桶组成，哈希桶中的 entry 元素保存了 *key* 和 value 指针，其中 *key* 指向了实际的键，value 指向了实际的值。



所谓 hash 冲突，是指不同的 key，计算出来的结果落到了同一个 hash 桶中。

Redis 为了解决哈希冲突，采用了链式寻址法，也就是采用链表的方式来保存同一个 hash 桶中的多个元素。

这个部分的实现和 Java 中的 HashMap 是一样的。

有些粉丝可能会问，如果出现大量的 key 的冲突导致链表过长的情况下，会导致数据的检索效率变慢，Redis 是怎么解决这个问题的呢?

为了保持高效，Redis 会对**哈希表做 rehash** 操作，也就通过增加哈希桶来减少冲突。

为了 rehash 更高效，Redis 还默认使用了两个全局哈希表，一个用于当前使用，称为主哈希表，**一个用于扩容，称为备用哈希表**

好的，今天的分享就到这里了，感谢大家的点赞和评论，byebye。

53.235 如何处理消息队列的消息积压问题

昨天，一个工作 5 年的粉丝去小米金融面试，被问到：如何处理消息队列中的消息积压问题。

由于之前在项目中没遇到过类似的问题，所以没有回答出来。

这个问题在我整理的大厂面筋指南里是有的，有需要的小伙伴可以在评论区置顶中领取。

通常来说，消息积压的原因是生产者的消息生产速度大于消费者的消费速度，遇到这个问题的时候，需要排查具体的原因再提出解决方案。

如果当前不是因为系统 bug 导致的，那我们可以优化消费端的逻辑，比如通过异步的方式来处理消息、或者通过批量处理的方式来消费。

如果通过这两种优化方式还没有缓解，可以考虑对消费端进行水平扩容，从而扩大消费端的消费能力。

如果是因为系统 bug 导致大量消息堆积，那么首先需要解决系统 bug，然后临时做紧急扩容来完成大量消息的消费。

1. 首先解决消费端的 bug，来保证消费端的正常消息处理工作。
2. 接着把现在所有的消费端停止，然后新建一个 Topic，然后把 Partition 分区数量调整成原来的 10 倍。

3. 接着写一个用来实现数据分发的 Consumer 程序，这个程序专门去消费现在积压的数据，消费后不做处理，而是直接再把这些数据写入临时建立的 Topic 的 10 个 Partition 中。
4. 然后临时增加 10 倍的消费者节点来部署 Consumer，专门来消费临时的 Partition 分区数据。

通过上面这种方法，可以快速把现在堆积的消息处理完。等积压的消息处理结束后，再把恢复成原来的部署架构，把临时的 Topic 和临时申请的机器释放掉。

好的，今天的分享就到这里，感谢大家的关注和点赞，再见。

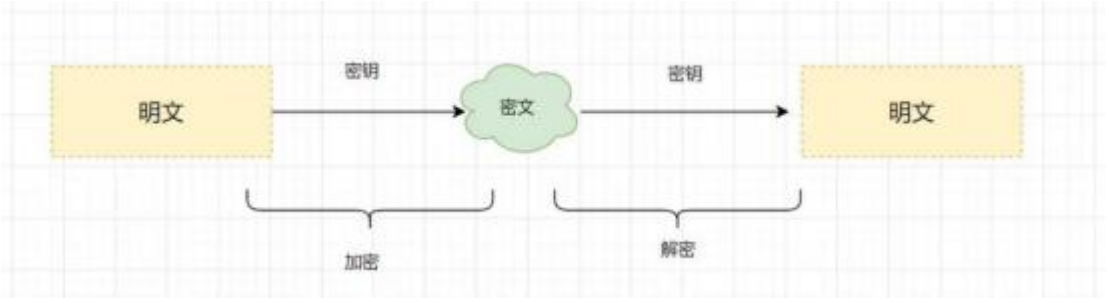
53.236 对称加密与非对称加密有什么区别

昨天，一个工作了 5 年的粉丝，去蚂蚁一面，被问到这个问题。

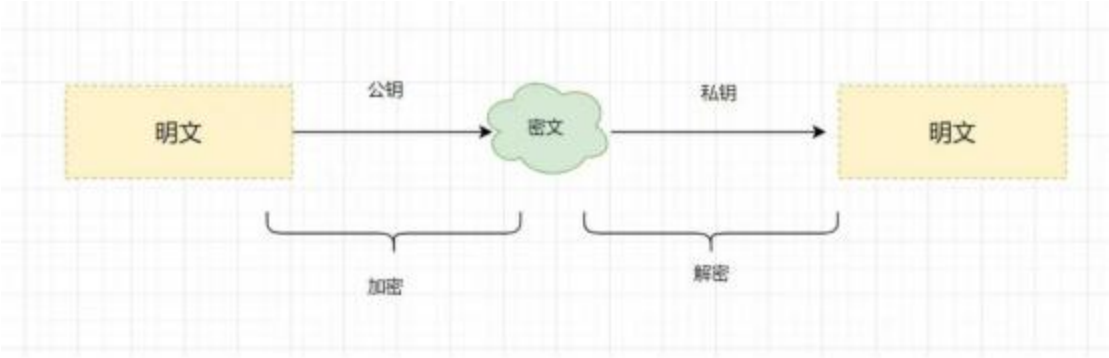
原因是它的项目经理中有包含这块的工作。

今天就给大家分享一下这个问题的回答，如果大家获取文字版本，可以点击评论区的置顶中去领取我整理的大厂面试指南。

所谓对称加密：指加密和解密使用同一密钥，优点是运算速度较快，常见的对称加密算法有：DES、AES 等



非对称加密：指的是加密和解密使用不同的密钥，也称为公钥和私钥。公钥与私钥是成对存在的，如果用公钥对数据进行加密，只有对应的私钥才能解密。反之，如果采用私钥加密，只能使用对应的公钥才能解密，RSA 是比较常见的非对称加密算法。



好的，今天的分享就到这里，感谢大家的关注和点赞。

53.237 JVM 中一次完整的 GC 流程是怎样的，对象如何晋升到老年代

关于：“JVM 中一次完整的 GC 流程是怎样的，对象如何晋升到老年代”这个问题，在面试过程中遇到的频率非常高。

今天正好有空，给大家分享一下这个问题的回答思路。

如果需要文字版本和完整的大厂面试手册，可以在评论区的置顶中领取。

关于这个问题，我们可以先描述 Java 堆内存的划分，再解释 Minor GC、Major GC、Full GC，最后再描述他们之间的转化流程。

以下是我的回答思路：

Java 堆内存是由新生代和老年代组成，其中新生代又包括 Eden 区、Survivor 区[S0、S1]。

一个新的对象，首先会分配在 Eden 区（如果是大对象，会直接进入老年代），当 Eden 区空间满了以后，JVM 会触发一次 Minor GC，用来回收 Eden 区的来及。

Eden 区存活下来的对象，会转移到 Survivor 区。

如果对象是在 Eden 中出生，并经过第一次 Minor GC 后仍然存活，那这个时候，该对象的 GC 年龄会设置成 1

每熬过一次 Minor GC，该对象的 GC 年龄都会进行累加，如果超过默认的 GC 次数 15 次，这个对象就会转移到老年代。

当老年代满了无法容纳更多对象的时候，就会触发 Full GC，Full GC 会清理整个内存堆 包括年轻代、老年代。

需要注意的是，在进行 Full GC 之前，通常会先进行一次 Young GC，以尽可能地清理掉 Eden 区和 Survivor 区中的垃圾对象，

以减少 Full GC 的压力和耗时。在 Young GC 之后，存活的对象将被复制到 Survivor 区或 Old 区，而 Eden 区会被完全清空，等待下一次对象的分配。

Major GC 其实是 Full GC 的另外一种叫法。

好的，今天的分享就到这里，感谢大家的关注和点赞。

53.238 谈谈什么是零拷贝？

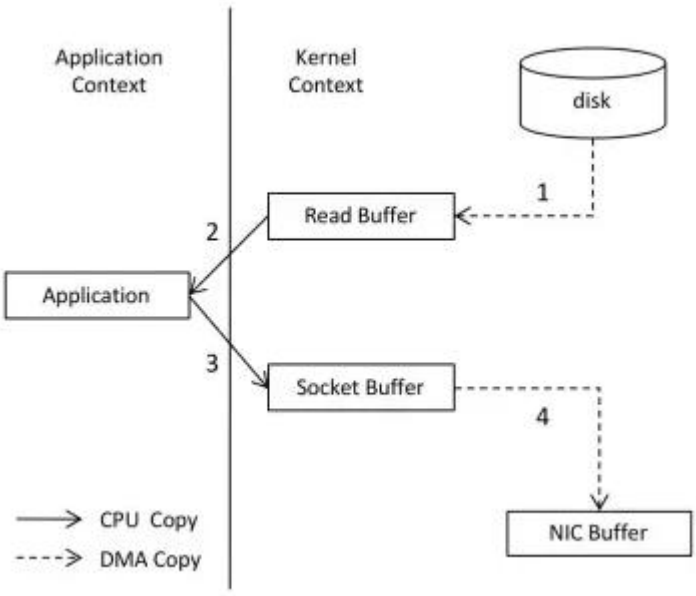
昨天，一个工作 5 年的粉丝去滴滴面试，在第二面的时候遇到了这个问题：” 请你简单说一下零拷贝原理 “，他没回答出来没通过面试。

现在找工作确实很难，所以大家还是要做好充分准备，该刷的面试题还是要刷。

之前我就分享过一本《大厂面筋指南》，大家记得去看一下，没领到的可以在评论区置顶中领取。

在实际应用中，如果我们需要把磁盘中的某个文件内容发送到远程服务器上，那么它必须要经过几个拷贝的过程，。从磁盘中读取目标文件内容拷贝到内核缓冲区，CPU 控制器再把内核缓冲区的数据赋值到用户空间的缓冲区中，

接着在应用程序中，调用 write() 方法，把用户空间缓冲区中的数据拷贝到内核下的 Socket Buffer 中。最后，把在内核模式下的 SocketBuffer 中的数据赋值到网卡缓冲区（NIC Buffer)网卡缓冲区再把数据传输到目标服务器上。



在这个过程中我们可以发现，数据从磁盘到最终发送出去，要经历 4 次拷贝，而在这四次拷贝过程中，有两次拷贝是浪费的，分别是：

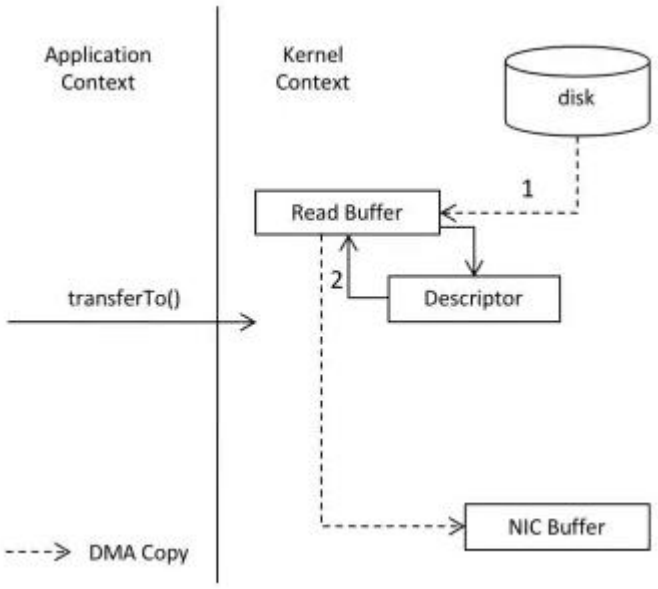
- 从内核空间赋值到用户空间
- 从用户空间再次复制到内核空间

除此之外，由于用户空间和内核空间的切换会带来 CPU 的上线文切换，对于 CPU 性能也会造成性能影响。而零拷贝，就是把这两次多于的拷贝省略掉，应用程序可以直接把磁盘中的数据从内核中直接传输给 Socket，而不需要再经过应用程序所在的用户空间（如图）。

零拷贝通过 DMA（Direct Memory Access）技术把文件内容复制到内核空间中的 Read Buffer，

接着把包含数据位置和长度信息的文件描述符加载到 Socket Buffer 中，DMA 引擎直接可以把数据从内核空间中传递给网卡设备。

在这个流程中，数据只经历了两次拷贝就发送到了网卡中，并且减少了 2 次 cpu 的上下文切换，对于效率有非常大的提高



所以，所谓零拷贝，并不是完全没有数据复制，只是相对于用户空间来说，不再需要进行数据拷贝。对于前面说的整个流程来说，零拷贝只是减少了不必要的拷贝次数而已。

在程序中如何实现零拷贝呢?

- 在 Linux 中，零拷贝技术依赖于底层的 sendfile()方法实现
- 在 Java 中，FileChannal.transferTo()方法的底层实现就是 sendfile()方法。

- 除此之外，还有一个 mmap 的文件映射机制

好的，今天的分享就到这里，感谢大家的关注和点赞，bye bye。

53.239 Mysql 索引在什么情况下会失效

昨天，一个工作 1 年的粉丝，好不容拿到一个面试机会，结果在这个简单的问题上挂了，有点可惜。

Mysql 索引失效这个问题，面试频率高，但同时也很简单。

我把完整的回答思路整理到了一个 30 万字的大厂面试指南里面，大家可以在我的评论区置顶中领取。

下面说一下我的回答：

1. 没有使用索引列作为 WHERE 子句的查询条件。
2. 对索引列进行函数操作。比如字符串操作、日期操作等，MySQL 将无法使用索引，此时索引失效。因为 MySQL 无法在运行时使用函数计算来匹配索引。
3. 对索引列进行类型转换。如果索引列是数字类型，但是你传入的值是字符串，那这个时候 Mysql 会默认对类型进行转化，同样会导致 MySQL 将无法使用索引，MySQL 在使用索引时必须将查询条件与索引列的数据类型匹配。
4. LIKE 查询的查询字符串以通配符开头。当使用 LIKE 查询并且查询字符串以通配符（例如%或_）开头时，MySQL 无法使用索引，因为通配符在开头时无法进行前缀匹配。
5. OR 条件查询。当查询中包含 OR 条件时，如果 OR 条件中的每个条件都不涉及索引列，MySQL 无法使用索引，此时索引失效。
6. 查询条件涉及到大量数据。当查询条件涉及到大量数据时，例如返回表中大部分数据的查询，MySQL 可能会认为使用索引并不高效，因此会放弃使用索引。

对于这个问题，尽可能多说一点，但是说不完整没关系，把核心的一些点说出来就够了

好的，今天的分享就到这里，感谢大家的关注和点赞，bye bye。

53.240 MySql 存储引擎 InnoDB 与 MyISAM 有什么区别

昨天，一个工作 3 年的粉丝去面试，被问到 InnoDB 和 MyISAM 引擎的区别时，回答的不是很理想。

然后过来找到我希望我帮他梳理一下回答思路。

下面我来分享一下我的回答，如果大家不方便看视频需要我整理的大厂面试指南的，可以在评论置顶中领取。

- 事务支持不同，InnoDB 支持事务处理，而 MyISAM 不支持。
- 并发处理不同：InnoDB 支持行级锁，而 MyISAM 支持表级锁
- 外键支持不同：InnoDB 支持外键约束，而 MyISAM 不支持
- 性能上存在差异：MyISAM 的读取速度比 InnoDB 快，但是在高并发环境下，InnoDB 的性能更好。这是因为 InnoDB 支持行级锁和事务处理，而 MyISAM 不支持。

所以，如果是读多写少的情况下，使用 MyISAM 引擎会更合适。

- 数据安全不同：InnoDB 支持崩溃恢复和数据恢复，而 MyISAM 不支持。如果 MySQL 崩溃了或者发生意外故障，InnoDB 可以通过恢复日志来恢复数据。

基本上回答出来这 5 个点，面试官一定会露出姨母般的微笑。

好的，今天的分享就到这里，感谢大家的点赞和关注。bye 。

53.241 select 和 epoll 的区别

这个问题，还真不是一般的程序员能回答出来的，大家如果不清楚，请诚实的说不知道。

昨天一个工作 8 年的粉丝被问到这个问题，他是一个连 NIO 都不知道的人，根本就不知道怎么回答。

如果工作中不涉及，但是又没有时间去准备，可以去评论区置顶中领取我整理的 30W 字的大厂面试指南。

下面我来总结一下这两者的区别：

`select` 和 `epoll` 都是 I/O 多路复用的机制，它们可以让一个进程监听多个文件描述符的 IO 事件或者连接事件，

只要其中任意一个或多个文件描述符就绪，就会触发阻塞唤醒，使得应用程序可以直接进行数据的读取或者写入。

它们的区别主要有几个方面：

1. `select` 是基于轮询的机制，它需要遍历整个监听集合，直到找到就绪的文件描述符。

而 `epoll` 是基于事件通知的机制，它只需要遍历当前就绪的文件描述符集合，大大减少了遍历的次数和开销。

2. `select` 的监听集合大小有限，一般受到操作系统的限制，而 `epoll` 没有这个限制，可以监听大量的文件描述符。
3. 在处理大量文件描述符时，`select` 的性能随着监听集合的增大而逐渐下降，而 `epoll` 的性能则能够保持稳定。
4. 在多线程环境下，`select` 需要将监听集合传递给每个线程，而 `epoll` 可以在一个线程中处理多个文件描述符，避免了线程间的切换和数据复制等开销。

从对比中不难发现，`epoll` 相比于 `select` 在性能和扩展性方面都有很大的优势，所以通常在企业级应用中使用较多的还是 epoll。

但是在一些特定的场景下，`select` 也有它的优势，例如在处理少量文件描述符或需要跨平台支持的情况下，`select` 是一个更好的选择。

好的，今天的分享就到这里，感谢大家的关注和点赞，bye.

53.242 如果发生内存泄漏怎么排查

“如果发生内存泄漏怎么排查”

昨天，一个工作 3 年的粉丝，在投递了 200 多家公司以后，终于约到了面试机会。

由于没有实操过，所以被问到：“如果发生内存泄漏怎么排查这个问题的时候”，就不知道该怎么回答了。

这个问题在我之前整理的大厂面试指南里面，有标准的回答，大家可以去评论区置顶中领取。

53.242.1 问题解析

内存泄露的面试频率还挺高的，一方面它是一个比较重要的基础知识，另一方面，我们写的所有程序都是运行在 JVM 上，

在出现故障的时候，需要开发人员具备解决该问题的能力。

这个问题分为两个部分：

1. 什么是内存泄露以及会带来什么影响
2. 内存泄露的排查和解决方法

内存泄漏指的是在程序运行过程中，因为某些原因导致不需要使用的对象，
仍然占用 JVM 的内存空间并且这块内存还无法被回收。
最终导致程序占用的内存越来越大从而出现 OOM 错误或者影响程序性能。
一般情况下，除了 OOM 这种错误以外，内存泄露会有一些比较明显的现象，比如频繁的 Full GC； 内存占用量过大一直无法释放等。
下面来看一下这个问题的回答。

53.242.2 问题答案

内存泄露的排查，我一般会根据现象去定位问题。
所以第一步，会先去定位是否是内存泄露，比如老年代逐步增长、fullGC 卡顿、年轻代的内存一直在高位无法释放、频繁 full gc 等。
这些现象基本上都是内存出现异常。
要了解 gc 的情况，可以使用 jstat 命令，查看虚拟机中各个内存区域的使用情况和 gc 情况。
然后使用 dump 工具，把当前内存 dump 下来，然后使用 MAT 工具来分析。
如果 dump 的文件比较大，可以使用轻量级的在线分析工具 jmap。
MAT 工具会自动分析 dump 文件的内容，给出一个分析结果并定位到有问题的类，然后去对这部分代码进行优化即可
一般可能是循环引用、内存对象泄露没有被销毁、动态分配内存以后未释放、长期持有对象引用、资源未关闭等。
以上就是我的理解！

53.243 MVCC 过程中会加锁吗?

“MVCC 过程中会加锁吗？”
昨天，一个工作了 4 年的粉丝在面试的时候，被问到这个问题，他果断而自信的回答了说不会。
然后面试官就拒绝了他。

后来在我给他的《大厂面试指南》里面看到答案以后，恍然大悟。

经过 2 年时间的整理，大厂面试指南已经涵盖了市面上 99%以上的问题，大家可以去评论区置顶中领取。

53.243.1 问题解析

MVCC 机制，全称（Multi-Version Concurrency Control）多版本并发控制，是确保在高并发下，

多个事务读取数据时不加锁也可以多次读取相同的值。

MVCC 在读已提交（READ COMMITTED）、可重复读（REPEATABLE READ 简称 RR）模式下才生效。

MVCC 在可重复读的事物隔离级别下，可以解决脏读、脏写、不可重复读等问题。

我们知道，MVCC 是基于乐观锁的实现，所以很自然的想到 MVCC 是不是不会加锁。

这个问题也要看情况来回答，下面看看回答建议。

53.243.2 问题答案

在 MVCC 中，通常不需要加锁来控制并发访问。

相反，每个事务都可以读取已提交的快照，而不需要获得共享锁或排它锁。

在写操作的时候，MVCC 会使用一种叫为“写时复制”（Copy-On-Write）的技术，

也就是在修改数据之前先将数据复制一份，从而创建一个新的快照。

当一个事务需要修改数据时，MVCC 会首先检查修改数据的快照版本号

是否与该事务的快照版本一致，如果一致则表示可以修改这条数据，

否则该事务需要等待其他事务完成对该数据的修改。

另外，这个事物在新快照之上修改的结果，不会影响原始数据，

其他事务可以继续读取原始数据的快照，从而解决了脏读、不可重复度问题。

所以，正是有了 MVCC 机制，让多个事务对同一条数据进行读写时，不需要加锁也不会出现读写冲突。

53.244 在 2G 大小的文件中，找出高频 top100 的单词

“在 1G 大小的文件中，找出高频 top100 的单词？”

昨天，一个工作了 5 年的粉丝在面试的时候，被这个问题之间问傻眼了

脑子完全转不过来。

这种问题在面试中一般问得很少，但是如果运气好遇到这个问题。

可以在我整理的《大厂面试指南》里面找到对应的回答。

经过 2 年时间的整理，大厂面试指南已经涵盖了市面上 99%以上的问题，大家可以去评论区置顶中领取。

53.244.1 问题解析

这是一个典型的 top k 问题，在面试的时候，会产生很多变体。

但是不管怎么变，top k 问题的本质是一样的。

另外，对于这类问题，我们可以发散自己的思维去回答，因为这类问题本身没有啥标准答案。

面试官更多的是去考察候选人的技术思维和技术积累，因此大胆一点回答没有关系。

这个问题的关键因素有两个：

1. 2G 大小的文件，意味着文件很大并且也无法一次性 load 到内存里面
2. 需要从这么大的文件中做筛选，如果用普通的思维方法，查找速度很慢

因此可以从这两个方面着手去思考回答思路。

53.244.2 问题答案

关于这个问题，我说一下我的回答思路

1. 把 2G 的文件进行分割成大小为 512KB 小文件，总共得到 2048 个小文件，避免一次性读入整个文件造成内存不足。
2. 定义一个长度为 2048 的 hash 表数组，用来统计每个小文件中单词出现的频率。
3. 使用多线程并行遍历 2048 个小文件，针对每个单词进行 hash 取模运算分别存储到长度为 2048 的 hash 表数组中

```
inthash=Math.abs(word.hashCode() %hashTableSize);
hashTables[hash].merge(word, 1, Integer::sum);
```

4. 接着再遍历这 2048 个 hash 表，把频率前 100 的单词存入小顶堆中
5. 最后，小顶堆中最终得到的 100 个单词，就是 top 100 了。

这种解决方案的核心思想是将大文件分割为多个小文件，然后采用分治和堆的算法，来解决这个问题。

53.245 Java 之父直播面试被吊打?

听说了吗？ 昨天 IT 界发生了一件大事，这个事情导致 Java 之父余胜军直播间人数狂飙到 2000 人。

评论区言论一边倒，引发了对 Java 之父这个头衔的质疑和嘲讽！

事情是这样的，

昨天晚上， Java 之父余胜军和陆总监直播连麦进行互动，本来聊天氛围和直播间氛围都还不错。

结果陆总监突然向 Java 之父提了一个面试题：

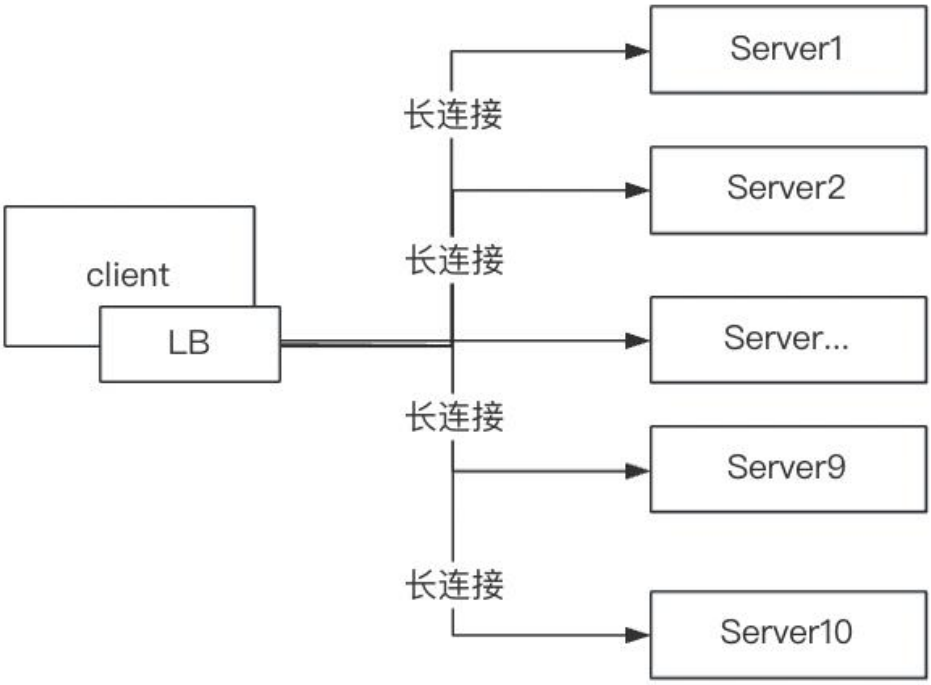
「播放视频」

结果，Java 之父竟然没回答上来，最精彩的是，评论区一堆看热闹不嫌事大的人开始带节奏。

事情愈演愈烈，直播间人数从 1000 直接狂飙到 2000 人。

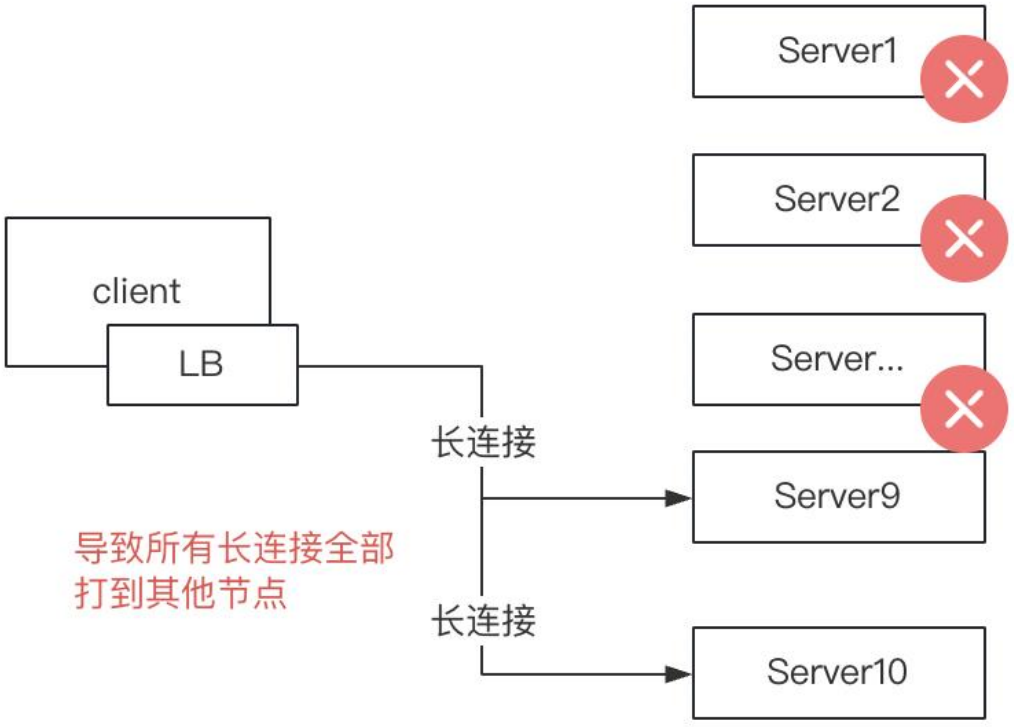
这个问题其实不难，我梳理了一下这个问题的背景，给大家分享一下怎么解决（如图）

客户端和多个目标服务节点保持长连接，然后通过负载均衡机制对连接进行分发，形成这样的一个结构。



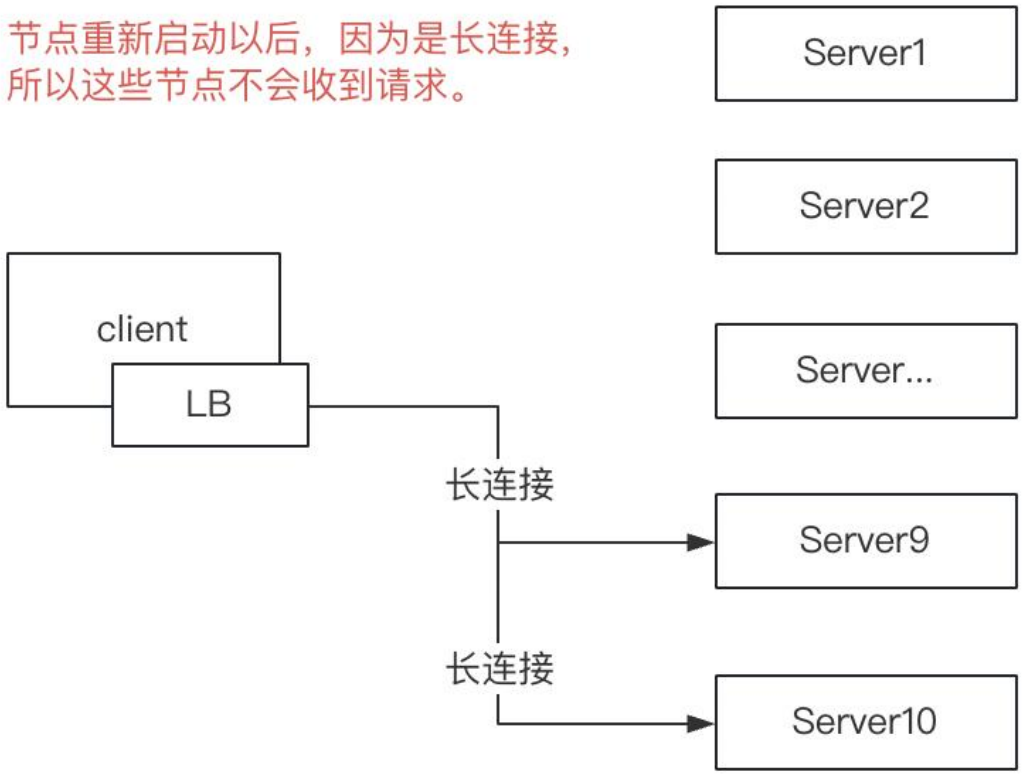
假设现在需要进行停机发布，按照发布流程，会先切掉一部分机器的流量进行重启，这就会导致原本的长连接断开（如图）。

客户端出发重连操作，使得这些长连接全部连接到其他没有重启的节点上。



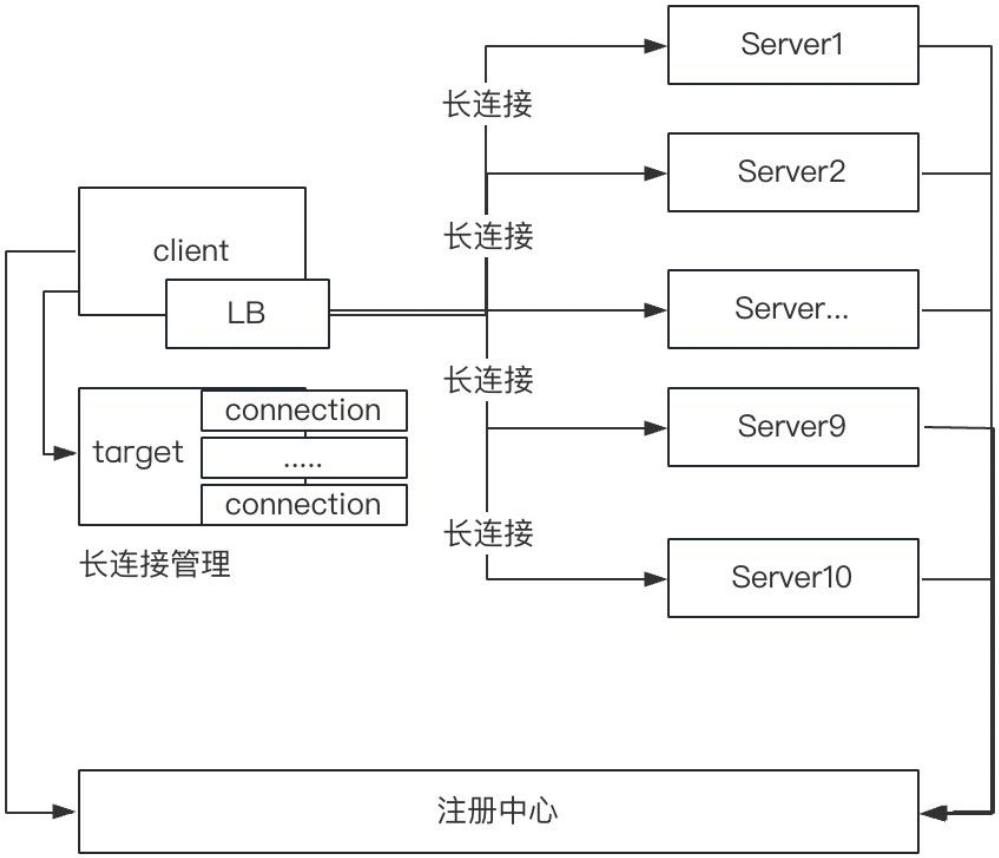
问题来了，因为是客户端建立的是长连接，所以节点重新启动以后（如图），客户端不会触发重连，使得新的节点没有任何连接。

而老的节点连接数量暴增，出现"早的早死涝的涝死"的现象，问，怎么解决这个问题。



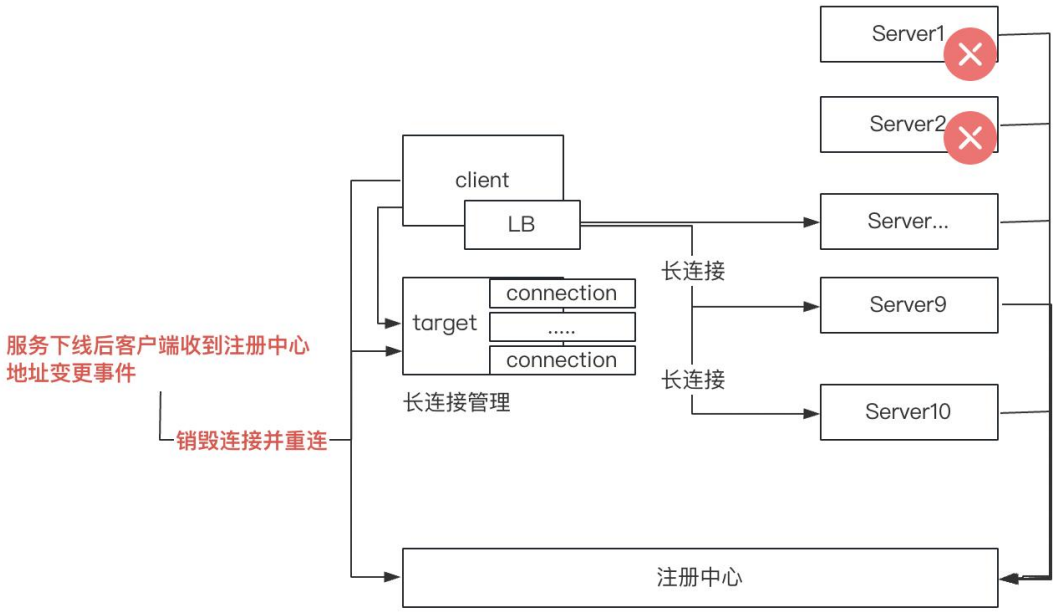
这个问题第一反应可能觉得没有思路，但是我们仔细来分析一下这个问题的背景，这里存在几个已知的情况（如图）。

1. 客户端和多个目标服务节点建立连接，并且实现动态负载均衡，意味着需要用到注册中心实现服务上下线感知
2. 客户端层面存在一个长连接管理机制，用来维护每个连接的状态。

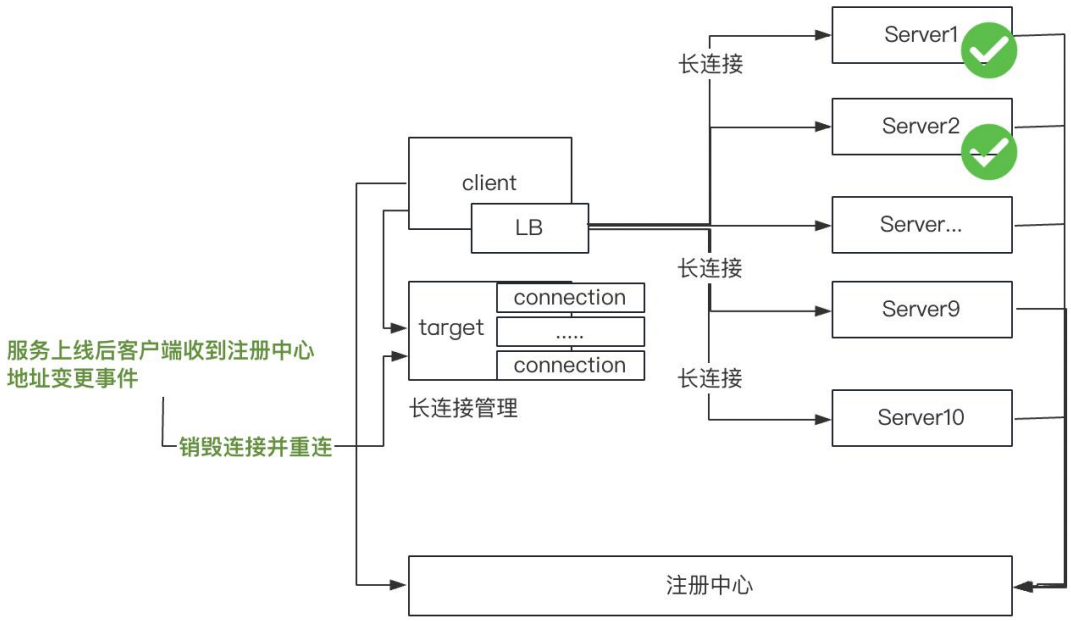


有了这样一个背景的分析，那这个问题就很好解决了，如果服务下线以后（如图），通过注册中心的感知机制，客户端会收到通知。

于是，客户端把所有失败的连接删除，并重新向健康状态的节点发起连接即可。



同样的道理，当发布的服务节点重新启动以后，注册中心也同样会收到通知，于是，把本地长连接池里面的长连接销毁，并重新建立新的连接出发新的 reblance 平衡即可。



直播间陆总监说了一个 kafka 的 rebalance 的解决方案，我们确实可以借鉴这个思路去解决这个问题。

不过 rebalance 机制，其实也是由一个第三方的 Coordinator 协调机制，和我上面说的思想是类似的。

本质上来说，都需要一个第三方节点来感知服务上下线状态，以及重新出发新的连接平衡机制。

最后我还是要说一下，任何一个没有相关经验的人，遇到类似问题要立刻给出解决方案，并且还是上千人的直播间。

确实是有一定的难度， 在企业级开发中，我们在设计一个技术解决方案的时候，同样也是会经过多次思考和修改，并且

还需要做技术方案的评审，最后才会落地实施。

所以大家还是要鼓励这些通过付出自己的时间和精力给大家带来技术提升的知识博主。

我是 Mic，今天的分享就到这里，欢迎关注和点赞。

53.246 Redis 中的哨兵选举算法是如何实现的?

“Redis 中的哨兵选举算法是如何实现的?”

昨天，一个工作 7 年的粉丝，去某外包公司面试，被问到这个问题不知道该怎么回答。

今天正好有空，给大家分享一下这个问题的回答思路。

对了，这个问题在我之前整理的 50 万字的大厂面试指南里面，有标准的回答，大家可以去评论区置顶中领取。

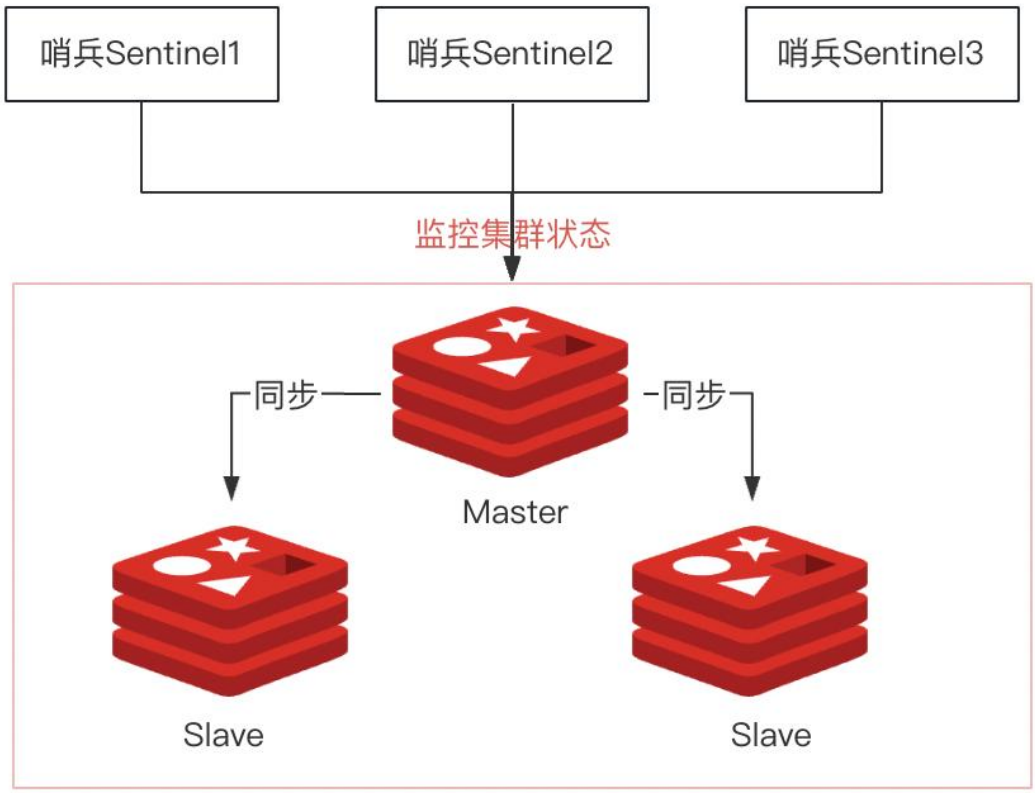
53.246.1 问题解析

Redis 里面的 Master-Slave 集群，是不具备故障恢复能力的。

也就是 Master 节点挂了以后，需要从集群中的其他 Slave 节点选举新的 Master 继续提供服务。

（如图）因此在 Redis 里面引入了 Sentinel 哨兵机制，通过哨兵来监控集群的状态，实现 Master 选举。

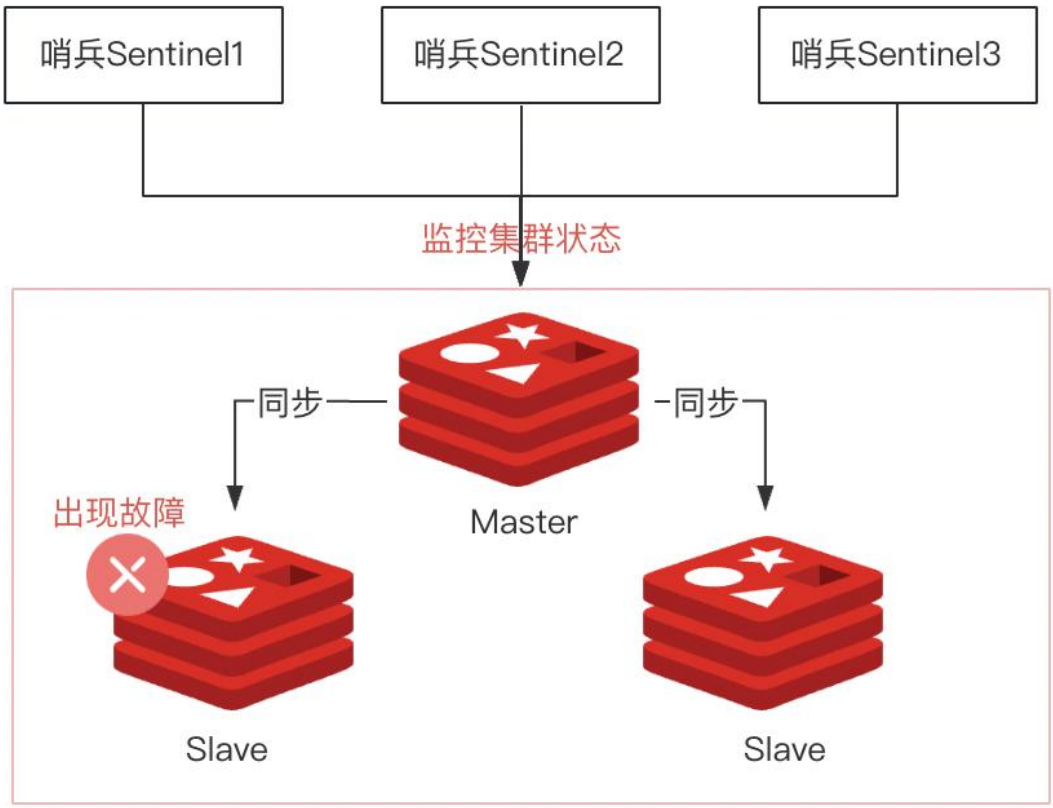
哨兵是一个单独的进程，所以为了保证哨兵的可靠性，我们也会对哨兵做部署集群。



假设哨兵节点有 3 个（如图），那这个时候这三个节点分别去监听 Redis 的三个主从节点，这里就存在一个问题：

一旦 Redis 主从集群的某个节点出现故障，而故障节点被其中一个 Sentinel 哨兵节点检测到，但是另外两个节点还没检测到

，那三个哨兵节点如何在意见上达成意见上的一致呢?



同时，哨兵节点怎么判断哪一个 Slave 节点应该成为 Master 呢?

这就是这个问题的底层逻辑，也是候选人在面对这个问题的时候需要思考和回答的方向，下面来看一下高手的回答。

53.246.2 问题答案

当 Redis 集群中的 Master 节点出现故障，哨兵节点检测到以后，会从 Redis 集群中的其他 Slave 节点选举出一个作为新的 Master。

具体的判断依据有两个部分：

第一部分： 筛选

第二部分： 综合评估

在筛选阶段，会过滤掉不健康的节点，比如（下线或者断线），或者没有回复 Sentinel 哨兵心跳响应的 Slave 节点。

同时，还会评估实例过往的网络连接情况，如果在一定时间内，Slave 和 Master 经常性断链，而且超出了一定的阈值，也不会考虑。

经过筛选后，留下的都是健康的节点了。

接下来就对健康节点进行综合评估，具体有三个维度，按照顺序来判断。

1. 根据 Slave 优先级来判断，通过 slave-priority 配置项（redis.conf），可以给不同的从库设置不同优先级，优先级高的优先成为 master。
2. 选择数据偏移量差距最小的，即 slave_repl_offset 与 master_repl_offset 进度差距，其实就是比较 slave 与 原 master 复制进度差距，避免丢失过多数据的问题。
3. slave runID，在优先级和复制进度都相同的情况下，选用 runID 最好的，runID 越小说明创建时间越早，优先选为 master。

经过以上步骤，就可以选举出新的 Master 节点了。

另外，如果哨兵存在集群的情况下，如果其中一个哨兵节点认为 Redis 集群主线故障，另外两个哨兵还没感知到的情况下。

在进行 Master 选举之前，Sentinel 哨兵集群需要通过共识算法来达成一致，这里用到了 Raft 协议。

以上就是我的理解。

53.247 SkipList 的索引过程，能否越两级搜索

“Redis 中的哨兵选举算法是如何实现的？”

昨天，一个工作 7 年的粉丝，去某外包公司面试，被问到这个问题不知道该怎么回答。

今天正好有空，给大家分享一下这个问题的回答思路。

对了，这个问题在我之前整理的 50 万字的大厂面试指南里面，有标准的回答，大家可以去评论区置顶中领取。

53.247.1 问题解析

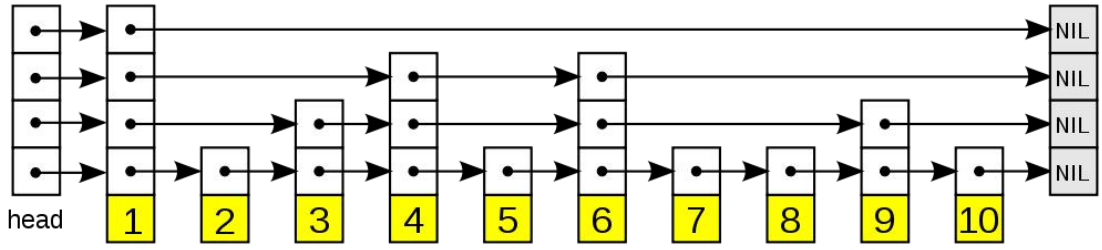
SkipList，很多人可能没听过，它还有另外一个名字叫跳跃表（如图）。

跳跃表主要由以下部分构成：

- 表头（head）：负责维护跳跃表的节点指针。
- 跳跃表节点：保存着元素值，以及多个层。
- 层：保存着指向其他元素的指针。

高层的指针越过的元素数量大于等于低层的指针，为了提高查找的效率，程序总是从高层先开始访问，然后随着元素值范围的缩小，慢慢降低层次。

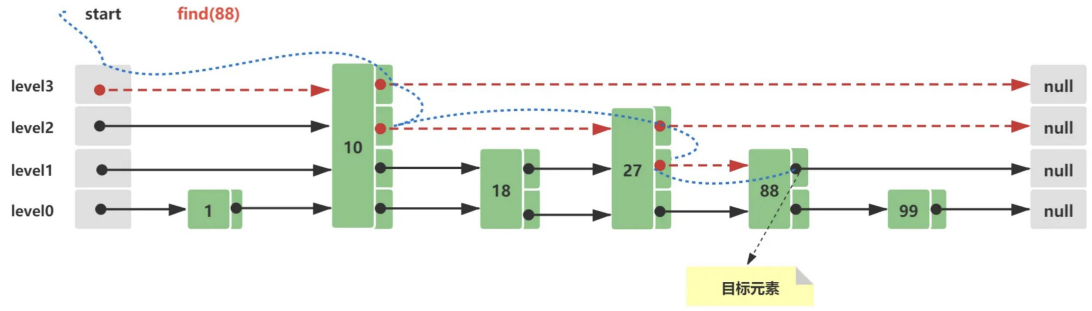
- 表尾：全部由 NULL 组成，表示跳跃表的末尾。



我这样说，大家理解起来觉得有点复杂，简单来说，就是在一个链表的基础上，增加了多层索引，这个层数是随机的。

每一层索引都是原始链表的一个子集，并且索引元素的间隔逐层增大，这样就可以在较高的层级上进行快速搜索，从而减少搜索的时间复杂度。

比如，在（如图）这样一个跳跃表中，想要查找 88 这个数字的位置，它可以通过逐层跳跃快速找到，相比于链表来说，查找性能要快很多。



理解了这样一个背景，再来看一下这个问题的回答。

53.247.2 问题答案

在一般情况下，跳表的查找过程，是从最高层开始逐层向下查找的，每次查找都是在当前层中找到小于目标元素的最大元素，

然后跳转到下一层，继续进行查找。

如果最后找到了目标元素，就返回这个元素所在的节点，否则返回空。

所以，对于跳跃表中的索引过程，并没有直接跳跃两层检索的情况，也是逐层进行判断最终获得查找的结果。

以上就是我的理解。

53.248 Kafka 消息队列怎么保证 exactlyOnce，怎么实现顺序消费

“Kafka 消息队列怎么保证 exactlyOnce，怎么实现顺序消费”

这个问题，难倒了很多工作 5 年以上的程序员。

而且这个问题的面试频率也非常高，建议大家收藏反复观看。

另外，我把这个问题的回答整理到了 50 万字的大厂面试指南里面，大家可以去评论区置顶中领取。

53.248.1 问题解析

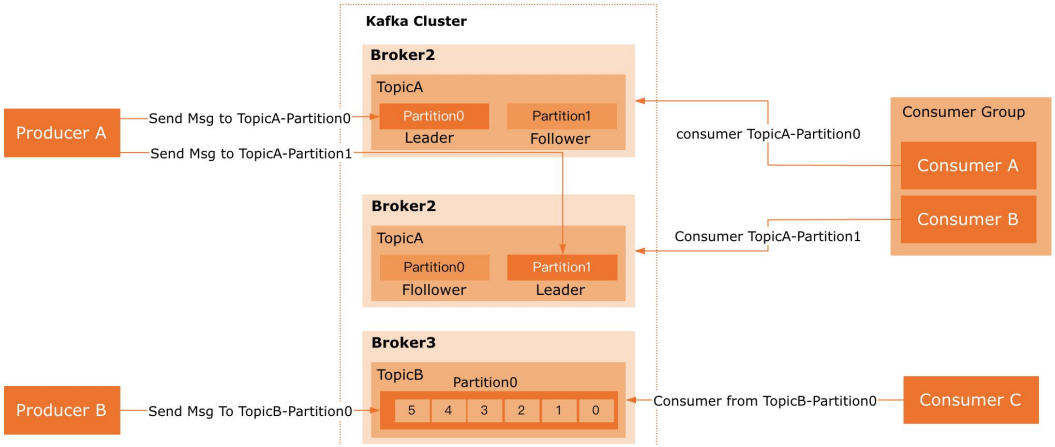
在回答这个问题之前，先来了解一下 Kafka 的运行机制（如图）

当我们向某个 Topic 发送消息的时候，在 Kafka 的 Broker 上，会通过 Partition 分区的机制来实现消息的物理存储。

一个 Topic 可以有多个 Partition，相当于把一个 Topic 里面的 N 个消息数据进行分片存储。

消费端去消费消息的时候，会从指定的 Partition 中去获取。

在同一个消费组中，一个消费者可以消费多个 Partition 中的数据。但是消费者的数量只能小于或者等于 Partition 分区数量。



理解了 Kafka 的工作机制以后，再来理解一下 exactlyOnce 的意思，在 MQ 的消息投递的语义有三种：

- At Most Once: 消息投递至多一次，可能会丢但不会出现重复。
- At Least Once: 消息投递至少一次，可能会出现重复但不会丢。
- Exactly Once: 消息投递正好一次，不会出现重复也不会丢。

所以，要回答好这个问题，必须要从上面这两个角度去切入，下面看看这个问题的回答。

53.248.2 问题答案

首先我先回答 Kafka 如何保证 ExactlyOnce。

准确来说，目前市面上的 MQ 产品，基本上都没有提供 Exactly Once 语义的实现。

我们只能通过一些其他手段来达到 Exactly Once 的效果。

也就是确保生产者只发送一次，消费端只接受一次

1. 生产者可以采用事物消息的方式，事务可以支持多分区的数据完整性，原子性。

并且支持跨会话的 exactly once 处理语义，即使 producer 宕机重启，依旧能保证数据只处理一次

开启事务首先需要开启幂等性，即设置 `enable.idempotence` 为 true。然后对 producer 消息发送做事务控制。

如果出现导致生产者重试的错误，同样的消息，仍由同样的生产者发送多次，这个消息只被写到 Kafka broker 的日志中一次

2. 虽然生产者能保证在 Kafka broker 上只记录唯一一条消息，但是由于网络延迟的存在，有可能会导致 Broker 在投递消息给消费者的时候，触发重试导致投递多次。

所以消费端，可以采用幂等性的机制来避免重试带来的重复消费问题。

其次，关于实现顺序消费问题。

在 Kafka 里面，每个 Partition 分区的消息本身就是按照顺序存储的。

所以只需要针对 Topic 设置一个 Partition，这样就保证了所有消息都写入到这一个 Partition 中。

而消费者这边只需要消费这个分区，就可以实现消息的顺序消费处理。

以上就是我的理解！

53.249 说一下 Kafka 中 Partition 分区副本的 Leader 选举算法

“说一下 Kafka 中 Partition 分区副本的 Leader 选举算法”

我敢说，这个问题百分之 99 的程序员答不上来，不服，来评论区打我的脸。

这个问题确实有一定的难度，不知道也可以理解，我把答案整理在 50 万字的大厂面试指南里面，大家可以在评论区领取。

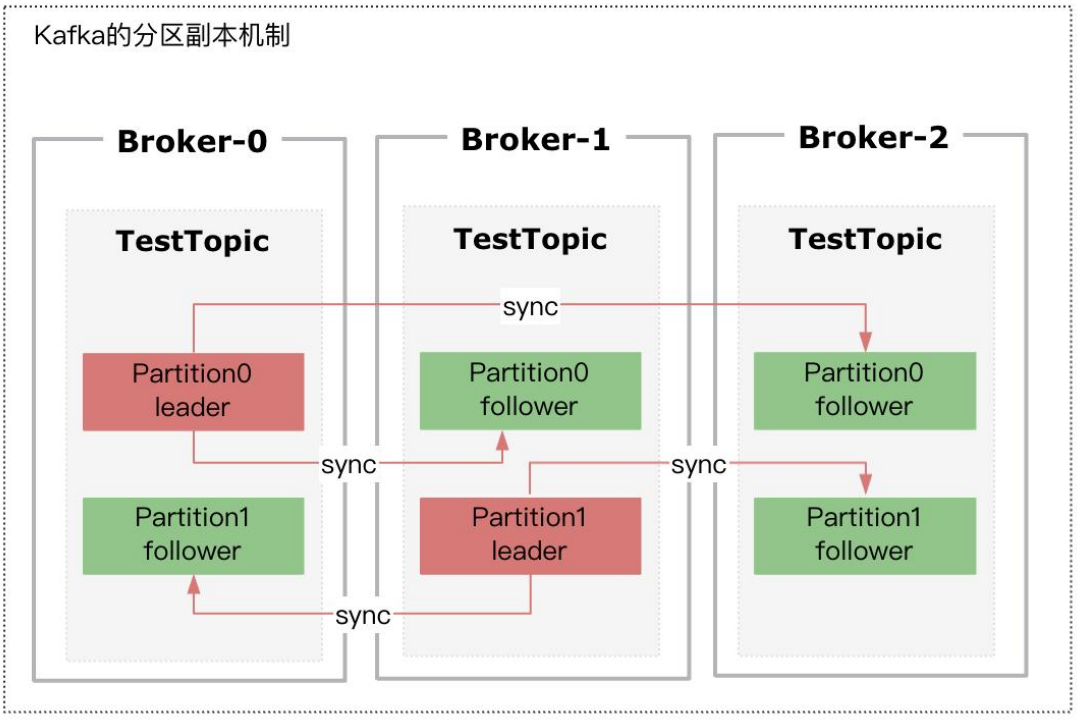
53.249.1 问题分析

在 Kafka 的架构中，一个 Topic 逻辑主题，可以分成多个 Partition 分区实现消息内容的物理存储。（如图）

同时，为了保证 Partition 分区的可靠性，Kafka 设计了分区副本的概念，也就是一个 Partition 可以设置多个副本。

在多个副本中，由于设计到数据的同步，所以 Kafka 针对 Partition 分区副本集，设置了 Leader 副本和 Follower 副本

Leader 副本负责处理所有的读写请求，Follower 副本，只负责从 Leader 副本同步数据。



而这个问题，就是去了解候选人对于 Partition 分区副本集中的 Leader 选举机制的了解，所以如果希望获得面试官的青睐

至少要回到到具体的点上。

因为在 Kafka 中，除了 Partition 分区副本的 Leader 选举以外，还有 Kafka 集群本身的 Leader 选举，这两个不是同一个东西，千万别搞混了。

53.249.2 问题答案

Kafka 首先会选择一个具有最新数据的副本作为新的 Leader，也就是 ISR 集合中的副本。

其中，ISR（In-Sync Replica）是指与 Leader 同步的副本集合，它们的数据同步状态与 Leader 最接近，

并且它们与 Leader 副本的网络通信延迟最小。

如果 ISR 集合中没有可用的副本，Kafka 会从所有副本中选择一个具有最新数据的副本作为新的 Leader。

在这种情况下选举出来的 Leader，由于和原来老的 Leader 节点的数据存在较大的延迟，会造成数据丢失的情况

所以 Kafka 设计者把这个功能开关的选择交给了开发者，如果愿意接受这种情况，可以通过 unclean.leader.election.enable 参数

来设置。

开启之后虽然会造成数据丢失，但是至少可以保证依然能对外提供服务，保证了可用性。

53.250 Kafka 中一个 Topic 有三个 Partition，同一个消费组中两个消费者如何消费的?

"Kafka 中一个 Topic 有三个 Partition，同一个消费组中两个消费者如何消费的？"

我敢保证，刷到这个视屏的人，100 个人里面只有 1 个人能回答出来。

请在不要查资料的情况下留言留下你的回答，我不相信能打脸。

另外，这个问题的答案我整理到了 50 万字的面试文档里面，有需要的小伙伴可以在评论区置顶中去领取。

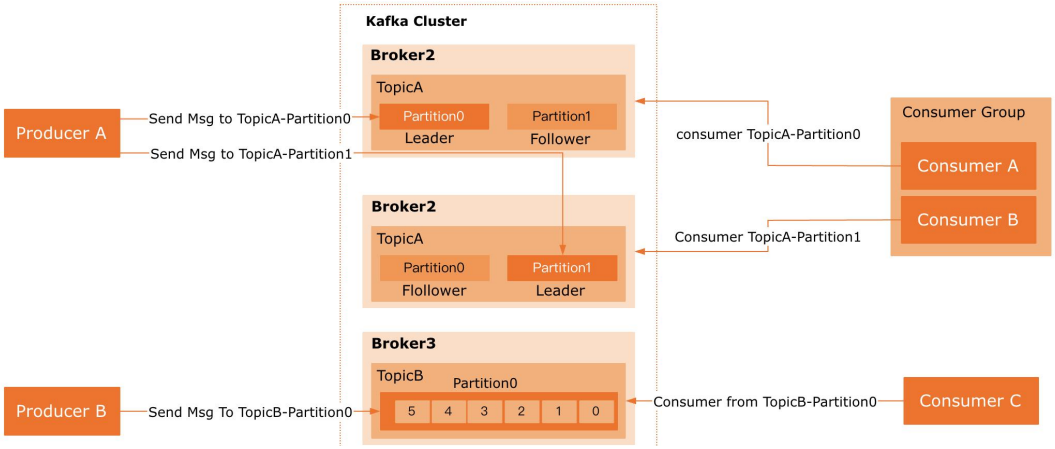
53.250.1 问题分析

在回答这个问题之前，先来了解一下 Kafka 的运行机制（如图）

当我们向某个 Topic 发送消息的时候，在 Kafka 的 Broker 上，会通过 Partition 分区的机制来实现消息的物理存储。

一个 Topic 可以有多个 Partition，相当于把一个 Topic 里面的 N 个消息数据进行分片存储。

消费端去消费消息的时候，会从指定的 Partition 中去获取。



在同一个消费组中，一个消费者可以消费多个 Partition 中的数据。但是消费者的数量只能小于或者等于 Partition 分区数量。

而这里提出来的问题是，一个消费组中两个消费者去消费三个 Partition，很自然的想到其中一个消费者需要消费两个 Partition

但是哪个消费者消费两个 partition 呢？这就是这个问题的关键，下面看下这个问题的回答。

53.250.2 问题答案

这个问题涉及到Kafka里面的Consumer Group Coordinator ，也就是消费组协调器。

它会根据消费者订阅的 Topic 中的 partition 数量、

和消费组中的消费者实例数量来决定每个消费者消费哪些 Partition。

这个算法会在消费组中选择一个消费者实例作为 Leader，

Leader 负责分配 Partition 给消费者实例，并协调消费者实例之间的 Partition 分配和 Reblance

当一个消费者实例加入或离开消费组的时候，协调器会触发 Partition 的重新分配，确保所有 Partition 都能被消费者实例均匀地消费。

Kafka 还提供了三种 Partition 分配策略，

1. "Round-robin(轮训)"，它会将 Partition 均匀地分配给消费者实例
2. "Range（范围）"，它会按照 Partition 的范围进行分配。
3. ” Sticky（粘滞分配），它会尽可能地将同一 Partition 分配给同一个消费者实例

以上就是我的理解。

53.251 JVM 中的三色标记法是什么?

“JVM 中的三色标记法是什么？”

这是一个年薪 40 万的真实面试题！ 大家可以先点赞收藏反复观看。

我把这个问题的完整回答，也同步整理到了 50W 字的面试文档里面，有需要的小伙伴可以在评论区置顶中领取。

53.251.1 问题分析

三色标记法是 Java 虚拟机（JVM）中垃圾回收算法的一种，主要用来标记内存中存活和需要回收的对象。

它的好处是，可以让 JVM 不发生或仅短时间发生 STW(Stop The World)，从而达到清除 JVM 内存垃圾的目的

JVM 中的「**CMS、G1 垃圾回收器**」都用到了三色标记法。

下面来看一下这个问题的解答。

53.251.2 问题答案

在三色标记法中，Java 虚拟机将内存中的对象分为三个颜色：

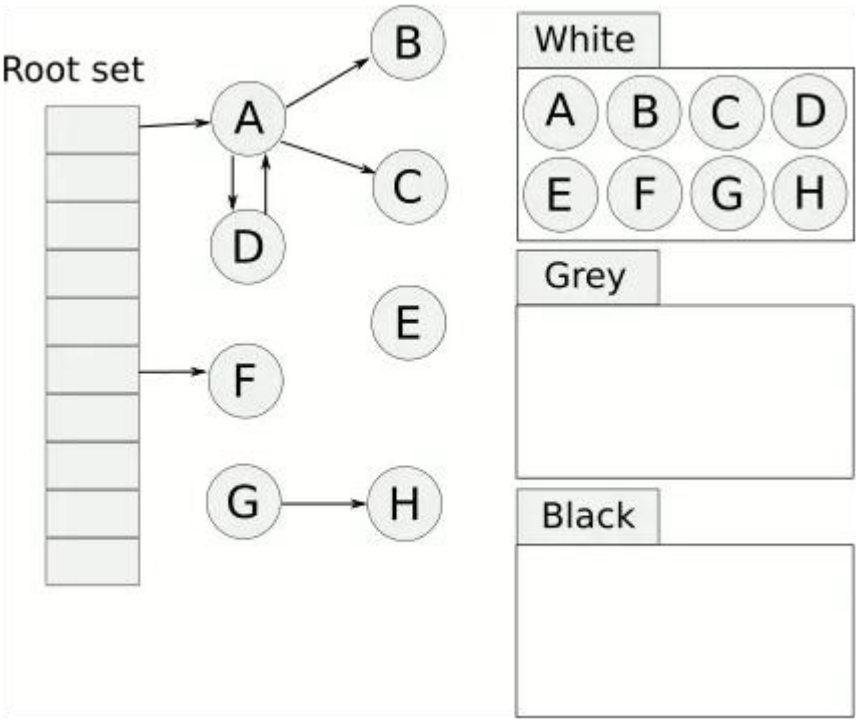
4. 白色：表示还没有被垃圾回收器扫描的对象；
5. 黑色：表示已经被垃圾回收器扫描过，且对象及其引用的其他对象都是存活的；
6. 灰色：表示已经被垃圾回收器扫描过，但对象引用的其他对象尚未被扫描。

在 GC 开始时（如图），先将所有对象都标记为白色，然后从根对象开始遍历内存中的对象，接着把直接引用的对象标记为灰色。

再判断灰色集合中的对象是否存在子引用，不存在咋放入黑色集合，如果存在，就把子引用对象放入到灰色集合。

按照这样一个步骤不断推导，直到灰色集合中所有的对象变黑后，本轮标记完成。

最后，还处于白色标记的对象就是不可达对象，可以直接被回收。



53.252 数据量达到多少的时候要开始分库分表

昨天，一个工作 7 年的粉丝去平安面试，在第一面的时候被问到这个问题，没回答得很好

最后比较遗憾错过了这个 offer

数据量达到多少的时候要开始分库分表这个问题，接下来我说一下我的观点。

如果需要文字版本的完整回答，可以直接在评论区置顶中领取 50W 字的大厂面筋指南。

53.252.1 问题分析

这个其实是一个开放性问题，因为分库分表有很多目的，比如查询性能下降、或者是业务解耦。

另外，如果单纯基于数据量来推导分库分表的行为是不严谨的。

里面涉及到的变量比较多，

比如数据行的大小、索引大小会决定 B+树的层高，而层高又间接决定不同数据量级的查询性能。

对该表的查询维度、查询条件的复杂度，即便在几十万数据的表中也可能会出现查询性能低的问题。

硬件资源的配置性能，比如带宽、内存、磁盘也同样会影响查询效率。

甚至是表结构设计不合理也有影响，比如如果建立的是宽表，也就是一个表包含太多的列，也会影响查询效率

所以，面对这个问题，建议是可以发散思维去分析，通过知识的宽度来获得认可。

53.252.2 问题答案

这个问题需要结合具体业务场景和系统架构来考虑，基于我的理解，通常有以下几个考虑因素：

1. 单表数据量：如果单个表的数据量已经非常大，例如超过了百万级别，就需要开始考虑分表。
2. 数据库性能：当单个数据库的性能无法满足业务需求时，就需要考虑分库。
3. 数据访问频率：如果某些表的数据访问频率非常高，单个数据库节点无法满足高并发请求，

就需要考虑将这些表分到不同的库或表中，以提高性能。

4. 业务拆分：当系统的业务逻辑越来越复杂，不同的业务之间的数据耦合度越来越低，就需要考虑对系统进行拆分，以方便管理和扩展。

以上就是我的理解！

53.253 表数据量大的时候，影响查询效率的主要原因有哪些

”表数据量大的时候，影响查询效率的主要原因有哪些“

最近有在面试过程中遇到这个问题的，可以刷个 1 告知我一下。

这个问题也是一个一线大厂的真题，当时那个工作 6 年的粉丝没回答出来

今天给大家分享一下这个问题的解答思路。

另外，如果你们需要文字版本的回答，可以在评论区置顶中去领取。

53.253.1 问题分析

这个是一个比较开放性的问题，没有标准答案。

大家可以把自己所知道的都说出来就好了。

开放性问题在面试中比较常见，主要目的是去挖掘候选人的技术能力，让候选人尽可能多的说自己的技术理解和边界。

从而为后续的面试问题提供依据。

下面来看看这个问题的回答。

53.253.2 问题答案

表数据量大的时候，主要有几个方面的因素影响查询效率

1. 磁盘 IO：数据量大意味着需要从磁盘中读取更多的数据，而磁盘 IO 速度是相对较慢的，因此会影响查询效率。
2. 索引失效：索引是提高查询效率的重要手段，但是如果索引失效，就会导致查询效率下降。

索引失效的原因可能是查询条件中使用了不支持索引的操作符，或者是数据分布不均匀导致索引失效。

3. 数据分页：当需要查询大量数据的时候，数据库需要进行数据分页，而数据分页的过程需要占用大量的 CPU 资源，因此也会影响查询效率。
4. 锁竞争：当多个事物同时对同一个表进行读写操作时，就会产生锁竞争，而锁竞争会导致查询效率下降。
5. 内存使用：当表数据量大的时候，需要占用更多的内存空间来缓存数据，而如果内存不足，就会导致数据库频繁地进行磁盘 IO，从而影响查询效率。

以上就是我的理解。

53.254 应用程序中存在包冲突的情况下，怎么发现和解决

你们能想象，一个工作 4 年的程序员，竟然连这个问题都搞不清楚。

应用程序中存在包冲突的情况下，解决的问题应该很容易吧，甚至大家在工作中应该也遇到过啊。

好吧，今天抽空给大家分享一下这个问题的回答思路。

另外，我把完整的回答整理到了 50 万字的面试文档里面，有需要的小伙伴可以在评论区置顶中领取。

53.254.1 问题分析

这个就是一个简单的经验问题，一般在工作中都有可能遇到过。

即便没遇到过，也能基于自己已有的经验去回答出来。

无非就是把存在冲突版本的包排除就行了。下面来看看完整的回答。

53.254.2 问题答案

应用程序中存在包冲突的情况下，一般不需要主动去发现，程序在运行的时候会提示异常，比如常见的 NoSuchMethodException。

异常的原因是不同版本的依赖包中的类或者方法存在差异。

所以我们可以根据错误日志定位到冲突的类或者方法，然后找到所属的包。

接着使用 Intellij IDEA 这个工具自带的依赖关系分析或者 `mvn dependency:tree`，来分析当前应用中的包依赖关系，

定位到哪些依赖包中引用了不同的版本库。

最后，可以在 Pom 文件对应的依赖包中，通过`<exclusive>`标签排除掉低版本的依赖。

53.255 Zookeeper 如何实现分布式锁

”Zookeeper 如何实现分布式锁“

这个问题特别简单，我可以分解出至少 3 种回答，听完我的分解，分分钟吊打面试官。

我把这个问题的完整分析和回答整理到了 50 万字的面试文档里面，有需要的小伙伴可以在评论区置顶中领取

53.255.1 问题分析

关于分布式锁的问题，不管是用什么技术，锁的本质是不变的。

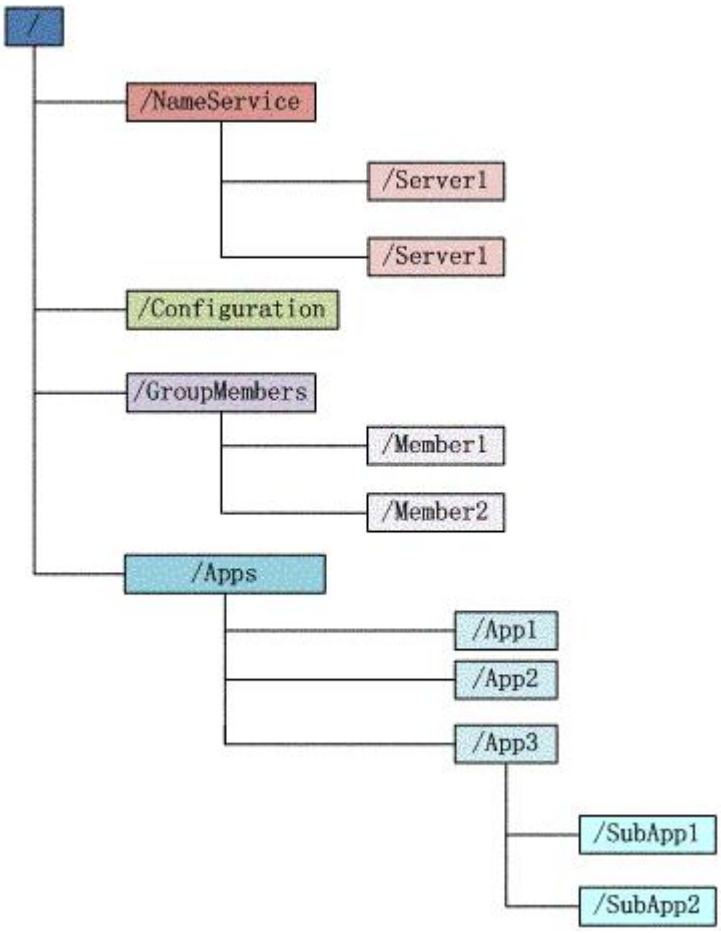
如果是排他锁，意味着在同一个时刻只有一个线程或进程拿到锁资源，获得访问权限。

所以基于这个点去推演 Zookeeper 怎么去达到这个目的就行了(如图)

Zookeeper 的存储结构类似于一个具有层次关系的文件系统，每一个节点代表一个 k-v 结构的数据。

这些节点有几个很重要的特性。

- 同一级节点不能存在重复的数据
- 提供了有序节点的特性，可以根据创建的先后分配一个有序的编号



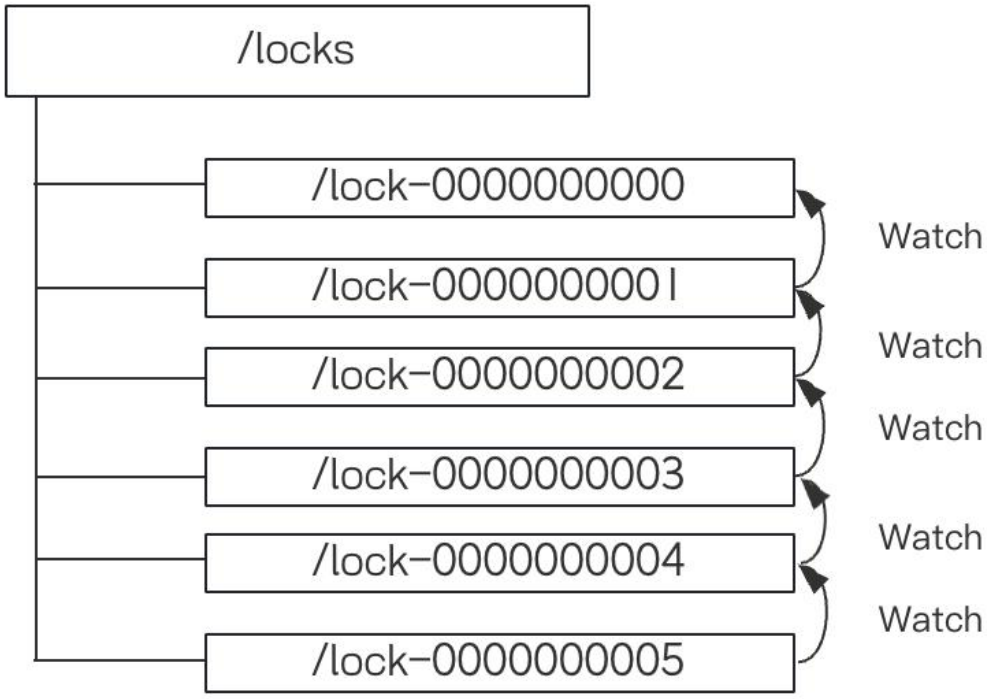
这几个特性是实现分布式锁的关键，下面来看看这个问题的回答。

53.255.2 问题答案

我们可以用到 Zookeeper 里面的有序节点来实现分布式锁，具体的实现过程是(如图)：

1. 在 ZooKeeper 上创建一个持久化的节点`/exclusive_lock`，作为锁的根节点。
2. 每个客户端去竞争锁的时候，直接在根节点下创建一个临时有序节点，节点的名称为“lock-”前缀加自动生成的序号
3. 接着再判断当前竞争锁的线程创建的节点，在所有子节点中是否是最小的节点，如果是就表示获得了锁

4. 如果不是，表示没有竞争到锁，那当前客户端需要监听自己在 ZooKeeper 上创建的节点的前一个节点的删除事件，当节点删除时，再次尝试获取锁。



除了这个方案以外，还可以用 Zookeeper 里面同一级节点的唯一性来实现分布式锁，以上就是我的理解。

53.256 说一些 CMS 垃圾回器的工作原理

昨天，一个工作 8 年的粉丝去面试，被问到这个问题

”说一些 CMS 垃圾回收算法的工作原理“这个问题，没回答出来。

说实话，在一个行业干了 8 年，这种基础原理都搞不明白，被淘汰或者找不到工作都很正常。

这个问题的回答，我整理到了 50 万字的面试文档里面，有需要的小伙伴可以在评论区置顶中领取。

53.256.1 问题分析

其实不管是什么类型的垃圾回收器，本质上就两个能力，识别垃圾和清理垃圾。

所以理解了这个本质，再去学习不同的垃圾回收器，就会变得更加简单。

识别垃圾的算法无非就是引用计数法、可达性分析算法。

而清理垃圾的算法包括标记清除、复制、标记整理、分代收集等。

下面来看下这个问题的正确回答。

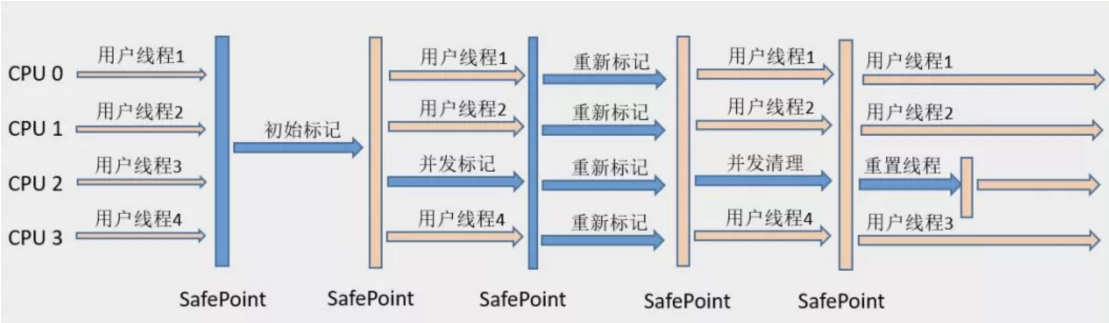
53.256.2 问题解答

CMS (Concurrent Mark and Sweep) 是一种低停顿垃圾回收器，

它主要通过初始标记阶段和并发标记阶段两个并发的阶段来实现垃圾回收：

它的整体流程可以分成四个步骤（如图）：

- 初始标记（CMS initial mark）：这个阶段需要 Stop The Word，来标记哪些对象是需要回收的，这个过程只需要标记 GC Roots 能够直接关联的对象，所以速度很快，对性能影响比较小。
- 并发标记（CMS concurrent mark）：扫描整个堆中的对象，标记所有不需要回收的对象。这个阶段不需要 Stop The Word，在应用程序运行过程中进行标记。
- 重新标记（CMS remark）：为了修正并发标记期间，应用程序同步运行导致标记产生变动的那一部分对象。这个阶段需要 Stop The Word。
- 并发清除（CMS concurrent sweep），CMS 会并发执行清除操作，同时应用程序继续运行，最大力度减少对性能的影响。



以上就是我的理解。

53.257 MySQL update 是锁行还是锁表?

“MySQL update 是锁行还是锁表？”

今天给大家分析一下这个问题的底层逻辑以及面试的时候的回答技巧

文字版本的回答，我整理到了 50W 字的面试文档中了，有需要的小伙伴可以在评论区置顶中领取。

53.257.1 问题分析

要回答好这个问题，需要先了解 Mysql 中为什么要引入锁。

在多个事物并行对同一个数据进行修改的时候，会产生事物的竞争造成脏读、幻读、不可重复读等问题。

所谓 Mysql 为了避免这类问题的出现，引入了事物隔离级别，其实本质上来说，最终解决的方式无非就是 LBCC 和 MVCC 两种。

而锁是解决事物竞争问题的底层实现方式。

通常来说，加锁会影响性能， 所以一般情况下都会考虑到性能和安全性的平衡

而 Mysql，也根据不同的作用范围，提供了不同的锁的实现方式。

而这个问题，就是考察候选人对锁范围的理解，下面来看下这个问题的回答思路。

53.257.2 问题解答

MySQL 的 Update 操作既可以锁行，也可以锁表，

具体使用哪种锁类型，取决于执行的 Update 语句的条件、事务隔离级别等因素。

- 如果 update 语句中的 where 条件包含了索引列，并且只更新一条数据，那这个时候就加行锁。

如果 where 条件中不包含索引列，这个时候会加表锁

- 另外，根据查询范围不同，Mysql 也会选择不同粒度的锁来避免幻读问题。

比如针对主键索引的 for update 操作：

```
SELECT * FROM t WHERE id = 10 FOR UPDATE;
```

Mysql 会增加 Next-Key Lock 来锁定 id=10 索引所在的区间

- 另外，针对于索引区间的查询或者修改

```
SELECT * FROM user WHERE id BETWEEN 1 AND 100 FOR UPDATE;
```

Mysql 会自动对索引间隙加锁，来解决幻读问题。

以上就是我的理解！

53.258 SortedSet 和 List 异同点?

“SortedSet 和 List 异同点？”

今天给大家分析一下这个问题的底层逻辑以及面试的时候的回答技巧

文字版本的回答，我整理到了 50W 字的面试文档中了，有需要的小伙伴可以在评论区置顶中领取。

53.258.1 问题分析

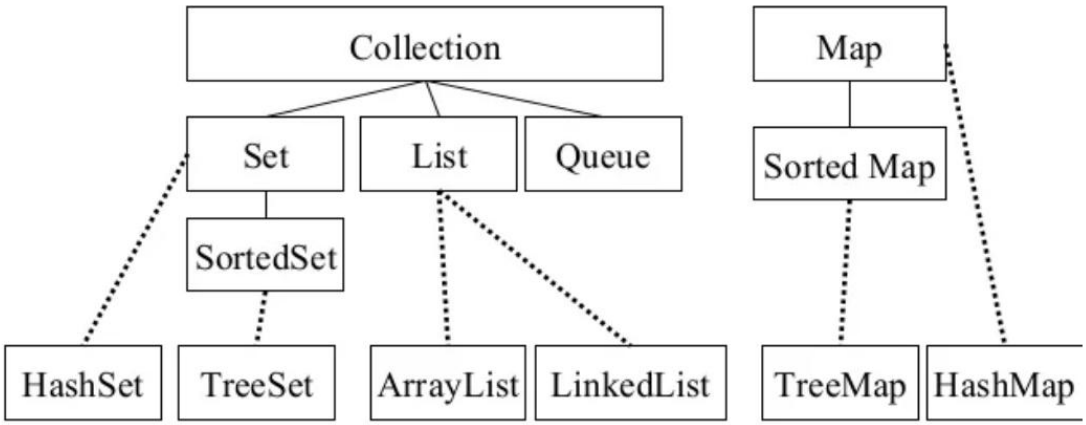
在面试过程中遇到的 xx 技术和 xx 技术的异同点问题。

大家必须要深刻了解这两种技术的特征和优缺点，如果单纯的去背诵，很难记住。

这个问题中，List 大家都已经非常熟悉了。我简单分享一下 SortedSet(如图）。

在 Java 的整个集合体系中，集合可以分成两个体系，一个是 Collection 存储单个对象的集合，另一个是 k-v 结构的 Map 集合

SortedSet 是 Collection 体系下 Set 接口下的派生类，而 Set 集合的特征是不包含重复的元素的集合。



了解了这个特点，才能更加准确的回答这个问题。

53.258.2 问题解答

首先我说一下相同点：

- 都可以用来存储一组有序的元素。
- 都支持随机访问和按照索引位置插入元素。
- 都是派生自 Collection 接口

不同点：

- SortedSet 是一个有序的集合，不允许元素的重复，而 List 是一个有序的列表，允许元素的重复。
- SortedSet 可以按照元素的自然顺序或者自定义比较器进行排序，而 List 只能按照元素的添加顺序排序。
- 在 SortedSet 中，元素的添加和删除操作的时间复杂度为 O(logn)，而在 List 中，元素的添加和删除操作的时间复杂度为 O(n)，因为需要移动其他元素的位置。
- SortedSet 可以方便地进行范围查询操作，例如获取某个区间内的元素，而 List 只能通过遍历实现范围查询。

以上就是我的理解！

53.259 SpringBoot 如何解决跨域问题?

“SpringBoot 如何解决跨域问题？”

今天给大家分析一下这个问题的底层逻辑以及面试的时候的回答技巧

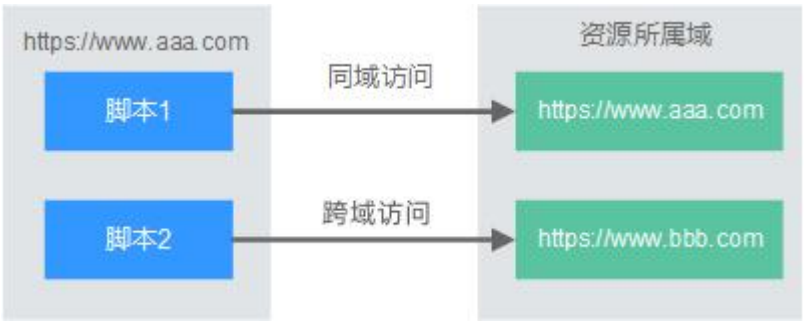
文字版本的回答，我整理到了 50W 字的面试文档中了，有需要的小伙伴可以在评论区置顶中领取。

53.259.1 问题分析

跨域指的是浏览器在执行网页中的 JavaScript 代码时，由于浏览器同源策略的限制（如图）。

只能访问同源（协议、域名、端口号均相同）的资源，而不能访问其他源（协议、域名、端口号任意一个不同）的资源。

（如图）比如像这样一种情况就体现了跨域的问题。



而解决跨域问题的方法，就是在不破坏同源策略的情况下，能够安全地实现数据共享和交互。

53.259.2 问题解答

常见的解决跨域问题的方法有两种，一种是 jsonp，另一种是 CORS。

其中，CORS 是一种在服务器后端解决跨域的方案，它的工作原理很简单。

如果一个网站需要访问另一个网站的资源，浏览器会先发送一个 OPTIONS 请求，根据服务器返回的 Access-Control-Allow-Origin 头信息，决定是否允许跨域访问。

所以，我们只需要在服务器端配置 `Access-Control-Allow-Origin` 属性，并配置允许哪些域名支持跨域请求即可。

在 Spring Boot 中，提供了两种配置 `Access-Control-Allow-Origin` 属性的方式来解决跨域问题

- 通过`@CrossOrigin(origins = "http://localhost:8080")`注解，指定允许哪些 origins 允许跨域
- 使用 WebMvcConfigurer 接口，重写 `addCorsMappings` 方法来配置允许跨域的请求源

```
@Configuration
publicclassCorsConfigimplementsWebMvcConfigurer {

    @Override
    publicvoidaddCorsMappings(CorsRegistryregistry) {
        registry.addMapping("/**")
                .allowedOrigins("http://localhost:8080")
                .allowedMethods("*");
```

```
    }
}
```

以上就是我的理解。

53.260 敏感数据怎么加解密和传输?

“敏感数据怎么加解密和传输?”

这是一个工作 4 年的粉丝去蚂蚁金服第一面遇到的面试真题

今天给大家分析一下这个问题的底层逻辑以及面试的时候的回答技巧

文字版本的回答，我整理到了 50W 字的面试文档中了，有需要的小伙伴可以在评论区置顶中领取。

53.260.1 问题分析

这是一个典型的信息安全性问题，面试官主要想通过这个问题测试你对安全方面的了解程度，

以及你在实际项目中如何保护敏感数据的能力。

候选人可以从加密算法的选择、加密方案设计、数据传输的安全性等方面来分析，下面来看看这个问题的回答

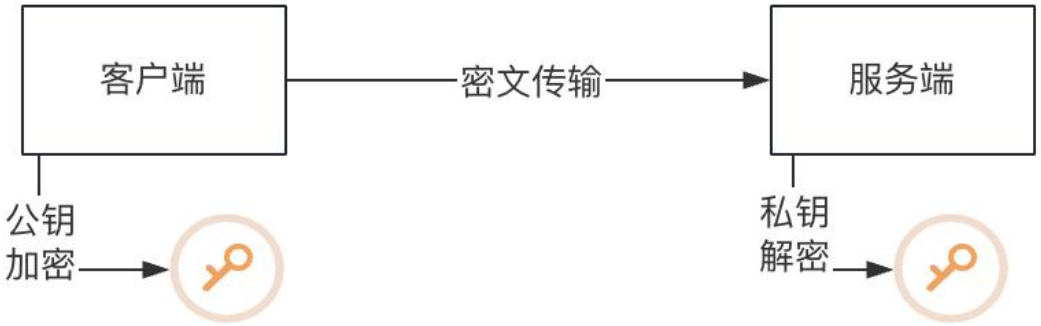
53.260.2 问题解答

常见的加密算法有两种，一种是对称加密，就是通信双方共享同一个密钥

另一种是非对称加密，也就是通过公钥和私钥两个密钥分别进行加密和解密。

对于客户端和服务端之间的数据传输（如图），可以采用非对称加密的方式实现。

首先客户端用提前分配好的公钥对数据加密，然后再把密文传输到服务端，服务端通过私钥来解密。



常见的非对称加密算法用 RSA 和 DSA 两种。

除了加密以外，还需要通过安全的通信协议进行传输，这里可以采用 https 协议。

最后，还需要确保公钥和私钥的安全性，防止被第三方拿到了密钥后破解内容。

以上就是我的理解。

53.261 字节跳动一面真题，如何提升接口的性能?

“如何让性能接口提升？”

昨天，一个工作 7 年的粉丝去高德面试，被问到这个问题时回答有点混乱。

今天正好有空，给大家分享一下这个问题的回答思路

文字版本的回答，我整理到了 50W 字的面试文档中了，有需要的小伙伴可以在评论区置顶中领取。

53.261.1 问题分析

这是一道比较开放性的面试题，面试官希望通过这类面试题打开面试过程中的沟通话题。

并且尽可能的让候选人多说，更好的去了解候选人的技术能力。

所以针对这个问题，可以打开回答的思路，从宏观到微观，把自己能知道的点都说出来就好了。

下面来看看这个问题的回答。

53.261.2 问题解答

影响接口性能的因素有很多，我分别从自己理解的几个维度来回答

1. 从接口本身的实现维度来说，可以从几个方面来优化
    a. 如果在接口中有操作数据库层面的代码，可以优化数据库 IO 的效率，比如 SQL 优化、数据库层面的优化等
    b. 如果存在部分频繁访问数据库的热数据，可以采用缓存机制
    c. 如果涉及到远程调用或者耗时的方法调用，可以采用异步方式避免同步阻塞，从而提升程序运行效率
    d. 代码本身的优化，可以利用合适的算法减少时间复杂度、避免一些很明显的重复计算等问题
2. 从宏观链路维度来说，可以关注几个方面
    a. 网络带宽，带宽的大小会影响数据的传输效率
    b. 服务器硬件资源如 CPU、内存等，会影响到接口中代码的执行效率
    c. 但个部署节点的计算能力瓶颈，也会影响接口性能，可以采用分布式部署的方式来优化

总的来说，接口的性能优化涉及到的因素比较多，如果真的出现性能问题，可以根据系统日志以及压测的情况去分析瓶颈点

再针对性的优化，以上就是我的理解。

53.262 什么是拆包和粘包？怎么解决?

“什么是粘包？怎么解决？”

这是一个工作 3 年提交给我的面试真题

今天我给大家分享一下这个问题的底层原理以及回答思路。

文字版本的回答，我整理到了 50W 字的面试文档中了，有需要的小伙伴可以在评论区置顶中领取。

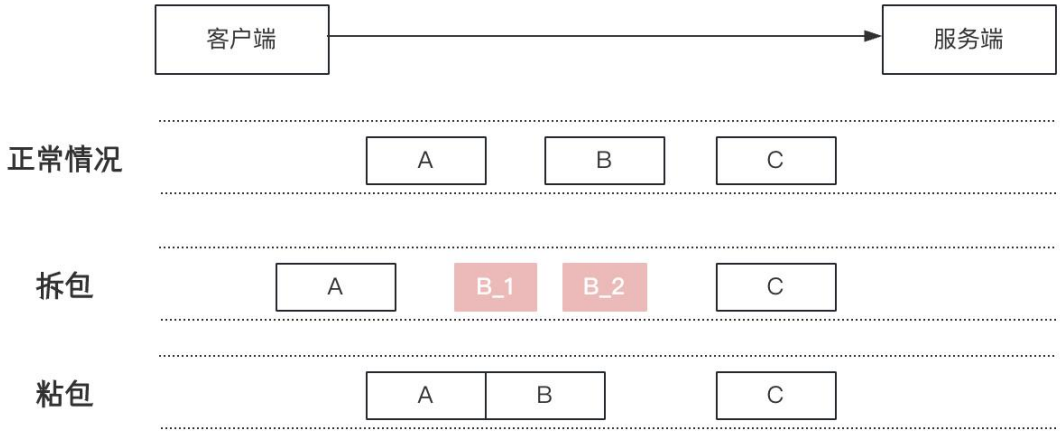
53.262.1 问题分析

拆包和粘包是在网络编程中比较常见的现象。

因为 TCP 协议底层是面向流的传输，所以数据在传输的过程中会被分割成一个个的数据包（如图）

接收端在接收数据时需要重新组装数据包，但是 TCP 协议不保证数据包与应用层的数据交互一一对应，

所以就可能会造成数据不完整的问题。



所以，候选人在回答这个问题之前，必须要了解这个问题的发生背景，否则回答不到点上就错过了这次机会。

53.262.2 问题解答

拆包指的是把一个完整的数据包拆分成多个小包进行发送，

而接收端可能无法一次性接收到所有小包，导致接收到的数据不完整。

粘包指的是把多个数据包粘合在一起一次性发送，

而接收端可能无法正确区分每个数据包，导致接收到的数据出现错位或混乱。

拆包和粘包现象是 TCP 协议的数据传输机制导致的，所以要解决这个问题，

就是需要让服务端知道如何判断一个数据包的完整性，因此可以采用以下几种方法：

- 在数据包中添加特殊字符或特殊标记，表示一个数据包的开始和结尾
- 通过自定义消息协议，并在协议头中保存数据包的长度信息，接收方可以根据这个长度来解析数据包来保证消息的完整性
- 基于定长消息，也就是发送端的消息长度是固定的，服务端按照固定长度来解析

以上就是我的理解！

53.263 为什么 Redis 集群的最大槽数是 16384 个?

“为什么 Redis 集群的最大槽数是 16384 个？”

昨天，一个工作 5 年的粉丝被这个问题搞懵了，他从来没往这方面想过。

今天我就给大家分享一下这个问题的回答思路

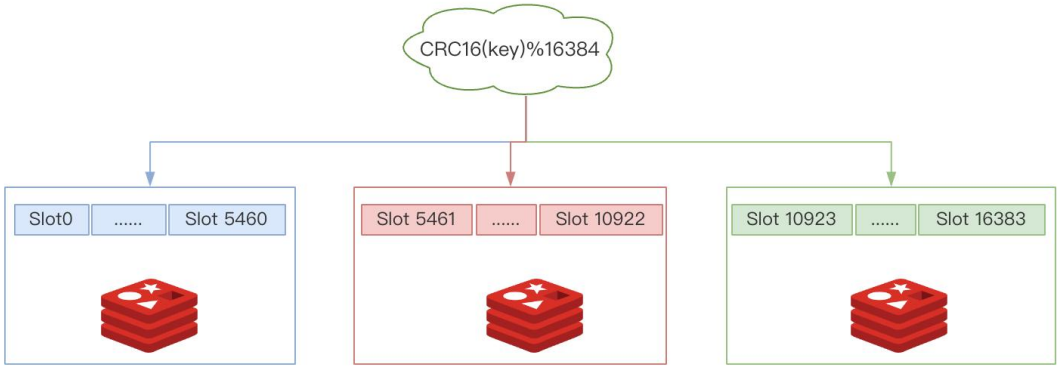
文字版本的回答，我整理到了 50W 字的面试文档中了，有需要的小伙伴可以在评论区置顶中领取。

53.263.1 问题分析

（如图）

Redis-Cluster 集群模式使用了哈希槽（hash slot）来实现数据的分片，hash slot 的默认长度是 16384.

应用程序去存储一个 key 的时候，会通过 CRC16 计算后取模路由到对应 hash slot 所在的节点。



这道面试题考察的是最大 hash 槽数为什么是 16384？ 有点类似于为什么 Hash 表的链表长度是 8 这个问题的味道。

意义并不大，下面来看看这个问题的回答。

53.263.2 问题解答

一般情况下，对于一个中间件中某些特定的阈值的设计，都是通过相关计算和测试得到的一个相对比较合适的值。

对于 Redis 中的 Hash Slot 为什么是 16384 这个问题，我认为有几个考虑因素：

- 对于网络通信开销的平衡：Redis 集群中每个节点会发送心跳消息，而心跳包中会携带节点的完整配置，

它能够以幂等的方式来更新配置。

如果采用 16384 个插槽，而每个插槽信息占用的位数为 1，

因此每个节点需要维护的配置信息占用空间大小就是 16384/节点数/8KB，假设是 3 个节点的集群，

则每个节点需要维护的配置信息占用空间大小为 2KB

其次，CRC16 算法产生的 hash 值有 16 位，如果按照 2^16 次方计算得到 65536 个槽

那就会导致每个节点维护的配置信息占 8kb。

8kb 数量的心跳数据看起来不大，但是这个心跳包每秒都需要把当前节点的信息同步给集群中的其他节点

相比于 16384 个 hash slot，整体增加了 4 倍，这样会对网络带块带来极大的浪费。

并且这个浪费并没有带来更好的效果。

- 集群规模的限制：Redis Cluster 不太可能扩展到超过 1000 个主节点，太多可能会导致网络拥堵等问题，

因此，在这个限制条件下，采用 16384 个插槽的范围比较合适

- 16384 个插槽可以确保每个 master 节点都有足够的插槽，同时也可以保证插槽数目不会过多或过少，从而保证了 Redis Cluster 的稳定性和高性能。

总之，16384 个槽的数量是经过考虑和实践得出的，既可以满足 Redis 集群的性能要求，又可以保证管理复杂度和通信开销的可控性。

53.264 为什么阿里 Java 手册禁止使用存储过程?

“为什么阿里 Java 手册禁止使用存储过程？”

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

昨天，一个工作 4 年的程序员去某互联网公司面试，被问到了这个问题。

如果大家不知道这个问题该怎么回答，可以在评论区置顶中领取我整理的 50W 字的大厂面试指南。

53.264.1 问题分析

一般来说，每个公司都会有自己的开发规范，开发规范存在的目的是减少一些低级错误和提升代码后期的维护性。

就这个问题来说，存储过程并不是绝对不能使用，只是在不同规模的公司，使用存储过程一定是会带来一定的影响。

因此，候选人可以从存储过程本身的缺点切入去解释就行了。

53.264.2 问题解答

禁止使用存储过程，主要有以下考虑点：

1. 存储过程的所有逻辑都在数据库层面，这会导致代码的可读性下降。
2. 存储过程可能包含复杂的业务逻辑，会导致数据库的负载增加，进而影响系统的性能。
3. 互联网公司的数据库会有专门的人来维护，开发人员是无法直接访问生产库的，当我们把业务逻辑写到数据库存储过程中，那后续在对业务进行升级的时候，就需要同步升级存储过程，不仅会造成工作职能的冲突，还会影响到业务逻辑的维护性问题。
4. 存储过程可能会难以调试和测试

以上就是我的理解，如果你喜欢我的作品，记得点赞收藏！

53.265 为什么 SQL 语句不要过多的 join?

“为什么 SQL 语句不要过多的 join？”

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这又是一个比较奇怪的问题，如果不清楚这个问题怎么回答。

建议去评论区领取一下我整理的 50 万字的大厂面试指南。

53.265.1 问题分析

面试官主要想了解面试者对于 SQL 优化的理解以及在实际工作中如何处理 SQL 语句的性能问题。

要回答好这个问题，只需要了解 join 操作会带来哪些影响，而这些影响对程序产生什么样的影响就行了。

53.265.2 问题解答

我认为主要有两个方面的原因：

1. 性能问题：每个 join 操作都需要对两个或多个表进行连接操作，这个操作需要消耗大量的计算资源和时间，如果 join 操作过多，会导致 SQL 的执行效率降低，从而影响整个系统的性能。
2. 可读性和维护性问题：join 操作会使 SQL 语句变得复杂，难以理解和维护，特别是当 join 操作涉及到多个表的时候，SQL 语句的复杂度会呈现指数级增长，给代码的可读性和可维护性带来挑战。

以上就是我的理解，如果你喜欢我的作品，记得点赞和关注！

53.266 为什么有些公司禁止使用@Transactional 声明式事务?

“为什么有些公司禁止使用@Transactional 声明式事务？”

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

屏幕前的粉丝朋友，如果不知道怎么回答这个问题，可以在评论区的置顶中领取这个问题的文字回答。

以及我们花了一年时间整理的 50 万字的大厂面试指南。

53.266.1 问题分析

@Transaction 注解在很多互联网公司也在使用，并不是每个公司都禁止。

所以是否要使用也和@Transaction 注解的特性和特定的环境有关系

因此大家可以沿着@Transaction 声明式事务可能存在的问题去说就好了。

下面来看一下这个问题的解答。

53.266.2 问题解答

我认为有几个方面的考虑

1. 在方法上增加@Transaction 声明式事务，如果一个方法中的存在较多耗时操作，就容易引发长事物问题，而长事物会带来锁的竞争影响性能，同时也会导致数据库连接池被耗尽，影响程序的正常执行。

2. 如果方法存在嵌套调用，而被嵌套调用的方法也声明了@Transaction 事物，就会出现事物的嵌套调用行为，容易引起事物的混乱造成程序运行结果出现异常
3. @Transaction 声明式事务是将事物控制逻辑放在注解中,如果项目的复杂度增加，事务的控制可能会变得更加复杂，导致代码可读性和维护性下降。

所以，为了避免这类问题，有些公司会推荐使用编程式事务，这样可以更加灵活地控制事务的范围，减少事务的锁定时间，提高系统的性能。以上就是我的理解，如果你喜欢这个作品，记得点赞和关注，我是 Mic，我们下期再见。

53.267 为什么阿里巴巴强制要求使用包装类型定义属性?

“为什么阿里巴巴强制要求使用包装类型定义属性？”

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

又是一个开发规范性的问题，问题本质是为什么企业强制要求使用包装类型定义属性

关于这个问题的完整回答，我整理到了一个 50 万字的面试文档中，有需要的小伙伴可以在评论区置顶中领取。

53.267.1 问题分析

这是一个开发规范的问题，开发规范更多的是为了后续程序的可维护性以及避免出现一些低级的错误影响程序的稳定。

所以对于一些规模比较大的公司，对规范的定义就会更加严格和细致。

因此回答这个问题的时候，可以往包装类型的优势去切入，从而保证自己的回答内容在面试官的预期内。

53.267.2 问题解答

我认为主要有以下几个方面的原因：

1. 默认值问题：使用基本数据类型定义属性时，如果没有给属性赋初始值，会使用默认值（如 int 的默认值为 0），

而使用包装类型定义属性，如果没有给属性赋初始值，属性的值为 null，这样可以更加清晰地表达属性的状态。

2. 拆箱问题：在一些特定场景下，如果使用基本数据类型定义属性，需要进行多次装箱和拆箱操作，这个操作会带来额外的性能开销和代码复杂度。而使用包装类型定义属性，可以避免这个问题，提高代码的效率和可读性。
3. Java 中的泛型中只能使用对象类型，如果要在泛型编程中使用基本类型，就必须使用对应的包装类型。
4. 提供了基本类型所不具备的方法和属性，比如 equals()、hashCode()、toString()，这些方法在某些特定场景中会比较有用

以上就是我的理解，如果你喜欢这个作品，记得点赞和关注，我是 Mic，我们下期再见。

53.268 过滤器和拦截器有什么区别?

”过滤器和拦截器有什么区别？“

我想大家可能是知道它们的区别的，但是真正要动口区说的时候就有点晕了。

这个问题的完整回答我整理到了 50 万字的面试文档里面，有需要的小伙伴可以在评论区置顶中领取

53.268.1 问题分析

这个问题一般考察 1 到 3 年左右的程序员，主要是想了解候选人对于 Web 应用程序中常用的技术的掌握程度，

以及对于请求处理流程和各个技术的应用场景的理解程度。

同时，这个问题还可以了解候选人对 Servlet 容器、Spring 框架等相关技术的掌握程度。因此大家在回答这个问题的时候。

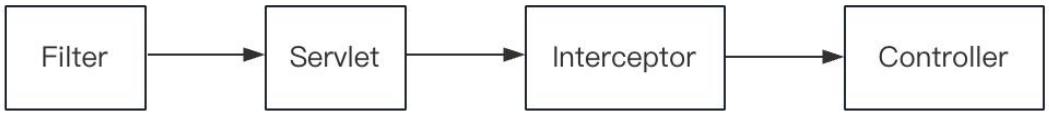
要结合自己的实际经验和理解，从技术实现、应用场景、性能等多个方面进行回答，

展示自己对于 Web 应用程序开发的深入理解和实际应用能力。

下面来看一下这个问题的回答。

53.268.2 问题解答

1. 运行顺序不同（如图）：过滤器是在 Servlet 容器接收到请求之后，但在 Servlet 被调用之前运行的；而拦截器则是在 Servlet 被调用之后，但在响应被发送到客户端之前运行的。



2. 配置方式不同：过滤器是在 web.xml 中进行配置；而拦截器的配置则是在 Spring 的配置文件中进行配置，或者使用注解进行配置。
3. Filter 依赖于 Servlet 容器，而 Interceptor 不依赖于 Servlet 容器
4. Filter 在过滤是只能对 request 和 response 进行操作，而 interceptor 可以对 request、response、handler、modelAndView、exception 进行操作。

以上就是我的理解，如果你喜欢这个作品，记得点赞和关注，我是 Mic，我们下期再见。

53.269 什么是微服务，说一下你对微服务的理解?

”什么是微服务，说一下你对微服务的理解？“

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

今天给大家分享一下，面试过程中遇到这个问题，我们应该怎么回答

这个问题的文字版本我整理到了一个 50 万字的面试文档中，有需要的小伙伴可以在评论区置顶中领取

53.269.1 问题分析

一般面试问这个问题，有两个目的

1. 是否真正理解微服务架构的特点和概念
2. 是否具备系统设计和架构能力，能否根据具体业务场景和需求设计合适的微服务架构

我认为这种问题千万不要去通过死记硬背来回答，这样容易导致回答思路比较混乱。

建议可以用理解的方式去回答，把一些关键点回答正确就差不多了。

53.269.2 问题解答

微服务是一种架构风格，我们可以把应用程序划分为一组小型的、松散耦合的服务，每个服务都运行在自己的进程中，并通过轻量级的通信机制进行通信

每个服务都是独立部署、独立扩展、独立更新的，从而提高了应用程序的可伸缩性、可维护性和可测试性

由于微服务是属于分布式架构下的一种针对应用架构的一种设计风格，所以我们会面临到分布式架构下的一些服务治理等复杂问题

因此 Spring 提供了一套 Spring Cloud 解决方案，帮助我们快速实现微服务的技术方案的落地。

以上就是我的理解，如果你喜欢这个作品，记得点赞和关注，我是 Mic，我们下期再见。

53.270 什么是 CompletableFuture?

”什么是 CompletableFuture? “

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

今天给大家分享一下，面试过程中遇到这个问题，我们应该怎么回答

这个问题的文字版本我整理到了一个 50 万字的面试文档中，有需要的小伙伴可以在评论区置顶中领取

53.270.1 问题分析

CompletableFuture 是 Java 8 引入的一个类，可以解决异步执行任务和处理异步任务的结果。

CompletableFuture 确实是一个很实用的组件

面试官想通过这个问题去考察候选人是否熟悉异步编程的概念，是否能够编写异步代码。

另外，CompletableFuture 可以优化系统的性能和响应速度，

所以，面试官想要考察候选人是否能够设计和优化高性能、高并发的系统，

并且能够使用 CompletableFuture 等工具来实现优化。

下面来看看这个问题的回答！

53.270.2 问题解答

CompletableFuture 是 Java 8 中引入的一个组件，它提供了一种简洁而强大的方式来处理异步任务和处理异步任务的结果。

在 CompletableFuture 出现之前，我们只能使用 Callable/Future 的机制来获取异步线程的执行结果，但 Future 的是通过阻塞等待的
方式来实现的，对性能不是很友好。
而使用 CompletableFuture 可以让我们将一个耗时的任务提交给线程池进行异步执行，然后可以继续执行其他的任务，
等到异步任务执行结束后会出发一个回调方法，我们可以在回调方法中处理异步任务的执行结果。
相当于优化了 Future 阻塞等待的问题。
CompletableFuture 提供了一些便捷的方法，例如 thenApply、thenAccept、thenRun 等，
可以让我们以链式的方式处理异步任务的结果，从而更加灵活地编写异步代码。
以上就是我的理解，如果你喜欢这个作品，记得点赞和关注，我是 Mic，我们下期再见。

53.271 对接第三方接口要考虑什么?

”对接第三方接口要考虑什么？“
Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员
今天给大家分享一下，面试过程中遇到这个问题，我们应该怎么回答
这个问题的文字版本我整理到了一个 50 万字的面试文档中，有需要的小伙伴可以在评论区置顶中领取

53.271.1 问题分析

这个问题考察候选人实际开发过程中是否有参与过第三方接口的对接。
以及在对接的过程中，是否有深度思考过对接过程中的一些问题。
很多候选人基本上都是按照第三方的接口文档去写，甚至成熟一点的三方接口都提供了封装好的工具包。
使得大家很少去关注。下面来看一下面试高回答。

53.271.2 问题解答

基于我的经验，主要考虑以下几个方面的问题：

1. 安全性问题，由于和第三方接口对接涉及到数据的跨网络传输，为了防止数据被拦截和篡改，需要采用安全的通信机制，比如 https 协议，以及数据签名等。
2. 接口稳定性和可靠性，这两个方面会直接影响用户体验和业务的正常流转，所以需要做相对充分的评估和测试
3. 接口是否存在访问限制或者费用，如果存在并发量的限制，需要评估是否满足当前业务需求； 如果存在费用，需要评估是否符合预算

以上就是我的理解，如果你喜欢这个作品，记得点赞和关注，我是 Mic，我们下期再见。

53.272 多线程异步和 MQ 有什么区别?

”多线程异步和 MQ 有什么区别？？ “

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

今天给大家分享一下，面试过程中遇到这个问题，我们应该怎么回答

这个问题的文字版本我整理到了一个 50 万字的面试文档中，有需要的小伙伴可以在评论区置顶中领取

53.272.1 问题分析

多线程异步和 MQ 异步其实在特性上有异曲同工之妙

多线程是进程内的概念、MQ 是分布式消息队列，两者不在同一个维度上；

至于区别，我想任何人都可以随便说出一二。

这个问题主要考察候选人对于并发编程和分布式消息队列的理解，

以及候选人如何考虑使用多线程异步和 MQ 来解决不同的问题。

下面看看完整的回答。

53.272.2 问题解答

多线程和 MQ 虽然在特性上都支持程序的异步操作，但是在实现本质上区别比较大，

我简单说一下较大区别的几个点

1. 处理任务的维度不同，多线程是同一个进程中的多个线程并行处理任务，MQ 是通过把消息发送到不同应用节点的不同进程来

处理任务

2. 数据可靠性不同，多线程异步处理任务时，数据是基于共享内存来交互的，一旦程序崩溃，内存中的数据会丢失；

使用 MQ 时，可以通过消息队列的持久化机制来保证消息的可靠性

3. 分布式能力方面，MQ 具备分布式能力，可以把消息分发到不同的节点存储和消费、多线程只能在一个进程中处理任务。

以上就是我的理解，如果你喜欢这个作品，记得点赞和关注，我是 Mic，我们下期再见。

53.273 什么情况下不建索引?

”多线程异步和 MQ 有什么区别？？“

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

今天给大家分享一下，面试过程中遇到这个问题，我们应该怎么回答

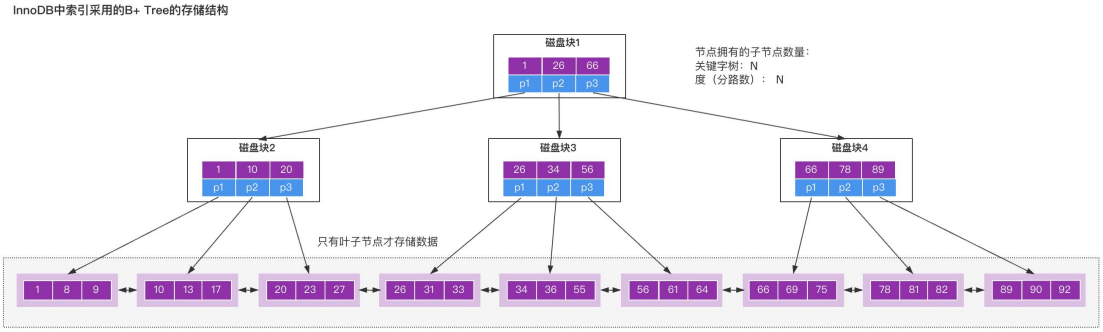
这个问题的文字版本我整理到了一个 50 万字的面试文档中，有需要的小伙伴可以在评论区置顶中领取

53.273.1 问题分析

一般情况下说的索引，都是 Mysql 里面 InnoDB 引擎的 B+树索引。

所以大家首先的知道索引的原理，大家都知道 B+树是一颗多路平衡二叉树（如图）

它的特点是，非叶子节点只存储索引，叶子节点存储数据，从而减少 B+树的层高降低磁盘 IO 次数啊从而提升数据检索效率。



通常情况下，加索引是能直接提升数据的检索效率，但面试官反其道而行问什么时候不建索引，所以这个问题考察候选人对于索引的理解深度。

如果期望这次面试有好的反馈，候选人需要能够分析出不建立索引的原因和影响，并根据具体情况进行详细说明。

53.273.2 问题解答

我认为有几种情况不适合建立索引：

1. 数据量太小的情况下，即使没有索引，查询的速度也比较快，这个时候建立索引反而会增加维护成本和查询时间
2. 数据离散度不高的列，比如性别、年龄这种，创建索引反而会降低检索效率，从底层原理来说，相当于增加了 B+树的扫描范围
3. 存在函数操作的情况，如果查询条件包含函数操作，那这个时候可能不会走索引，所以建了索引意义不大
4. 频繁变更的表，比如经常需要更新、删除或插入记录，那么对这个表建立索引的开销就会很大，甚至可能影响到整个数据库的性能。

以上就是我的理解，如果你喜欢这个作品，记得点赞和关注，我是 Mic，我们下期再见。

53.274 S*pring 中的 Bean 是线程安全的吗?*

金三银四的招聘季到了，Spring 作为最热门的框架，在很多大厂面试中都会问到相关的问题。

前几天，就有好几个同学就问我，在面试中被问到这样一个问题。Spring 中的 Bean 是不是线程安全的。大家总觉得在面试过程差了一点意思。但是又说不上来是什么原因。这是因为，大家可能对 Spring 的本质还欠缺一些深度的思考。

今天，咱们不兜圈子不绕弯，上来直接说答案，大家关注点个赞，本视频跟大家彻底讲明白。

其实，Spring 中的 Bean 是否线程安全，其实跟 Spring 容器本身无关。Spring 框架中没有提供线程安全的策略，因此，Spring 容器中在的 Bean 本身也不具备线程安全的特性。咱们要透彻理解这个结论，我们首先要知道 Spring 中的 Bean 是从哪里来的。

53.274.1 Spring 中 Bean 从哪里来的?

在 Spring 容器中，除了很多 Spring 内置的 Bean 以外，其他的 Bean 都是我们自己通过 Spring 配置来声明的，然后，由 Spring 容器统一加载。我们在 Spring 声明配置中通常会配置以下内容，如：class（全类名）、id（也就是 Bean 的唯一标识）、 scope

（作用域）以及 lazy-init（是否延时加载）等。之后，Spring 容器根据配置内容使用对应的策略来创建 Bean 的实例。因此，Spring 容器中的 Bean 其实都是根据我们自己写的类来创建的实例。因此，Spring 中的 Bean 是否线程安全，跟 Spring 容器无关，只是交由 Spring 容器托管而已。

那么，在 Spring 容器中，什么样的 Bean 会存在线程安全问题呢？回答，这个问题之前我们得先回顾一下 Spring Bean 的作用域。在 Spring 定义的作用域中，其中有 prototype（多例 Bean）和 singleton（单例 Bean）。那么，定义为 prototype 的 Bean，是在每次 getBean 的时候都会创建一个新的对象。定义为 singleton 的 Bean，在 Spring 容器中只会存在一个全局共享的实例。基于对以上 Spring Bean 作用域的理解，下面，我们来分析一下在 Spring 容器中，什么样的 Bean 会存在线程安全问题。

53.274.2 Spring 中什么样的 Bean 存在线程安全问题?

我们已经知道，多例 Bean 每次都会新创建新实例，也就是说线程之间不存在 Bean 共享的问题。因此，多例 Bean 是不存在线程安全问题的。

而单例 Bean 是所有线程共享一个实例，因此，就可能会存在线程安全问题。但是单例 Bean 又分为无状态 Bean 和有状态 Bean。在多线程操作中只会对 Bean 的成员变量进行查询操作，不会修改成员变量的值，这样的 Bean 称之为无状态 Bean。所以，可想而知，无状态的单例 Bean 是不存在线程安全问题的。但是，在多线程操作中如果需要对 Bean 中的成员变量进行数据更新操作，这样的 Bean 称之为有状态 Bean，所以，有状态的单例 Bean 就可能存在线程安全问题。

所以，最终我们得出结论，在 Spring 中，只有有状态的单例 Bean 才会存在线程安全问题。我们在使用 Spring 的过程中，经常会使用到有状态的单例 Bean，如果真正遇到了线程安全问题，我们又该如何处理呢?

53.274.3 如何处理 Spring Bean 的线程安全问题?

处理有状态单例 Bean 的线程安全问题有以下三种方法：

1、将 Bean 的作用域由 “singleton” 单例 改为 “prototype” 多例。

2、在 Bean 对象中避免定义可变的成员变量，当然，这样做不太现实，就当我没说。

3、在类中定义 ThreadLocal 的成员变量，并将需要的可变成员变量保存在 ThreadLocal 中，ThreadLocal 本身就具备线程隔离的特性，这就相当于为每个线程提供了一个独立的变量副本，每个线程只需要操作自己的线程副本变量，从而解决线程安全问题。

都已经看到这里了， 相信大家应该已经知道了 Spring 中的 Bean 是否线程安全以及如何处理 Bean 的线程安全问题。

下次再遇到这个面试题，你会回答了吗?

53.275 谈谈你对 Spring Bean 的理解

前几天，有位同学向我反馈，说在面试中问到这样这样一个面试题：谈谈你对 Spring Bean 的理解。今天咱们就针对这样一个面试题，给大家做一个详细的介绍。我一共分三段来介绍，首先，介绍什么是 Spring Bean？然后，定义 Spring Bean 有哪些方式？，最后，给大家介绍 Spring 容器是如何加载 Bean 的?

咱们先来看什么是 Spring Bean?

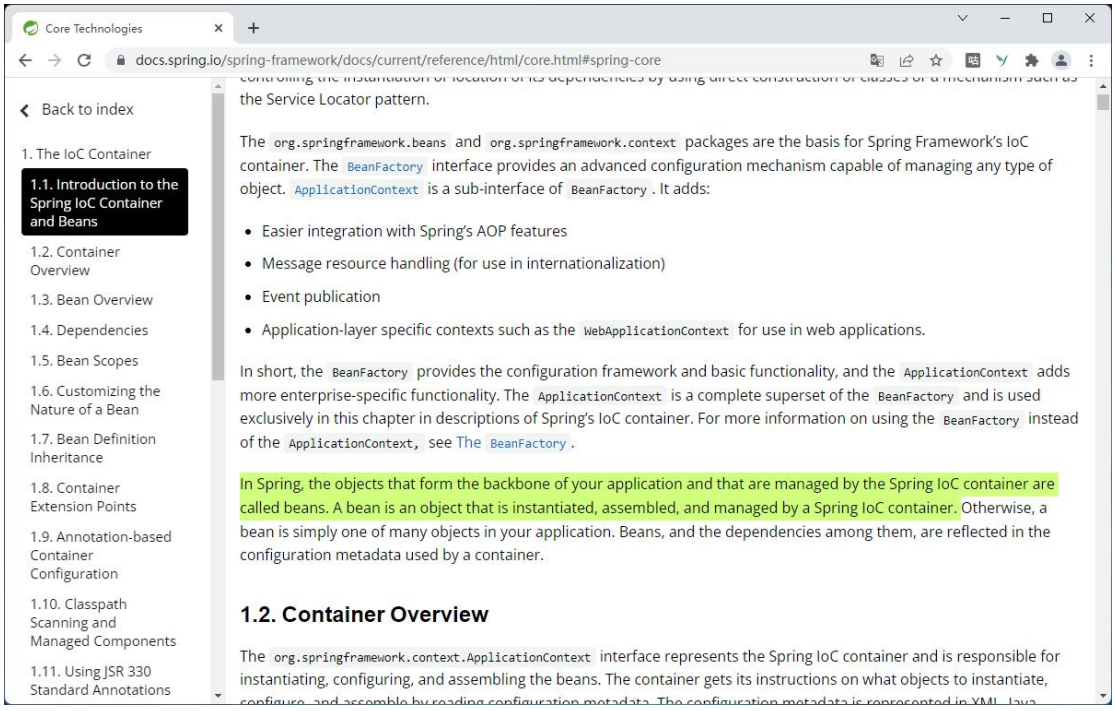
53.275.1 什么是 Spring Bean?

Spring Bean 是 Spring 中最基本的组成单元，Spring 官方文档对 Bean 的解释是这样的：

In Spring, the objects that form the backbone of your application and that are managed by the Spring IoC container are called beans. A bean is an object that is instantiated, assembled, and managed by a Spring IoC container.

翻译过来就是：

在 Spring 中，构成应用程序主干并由 Spring IoC 容器管理的对象称为 Bean。Bean 是一个由 Spring IoC 容器实例化、组装和管理的对象。



从官方定义中，我们可以提取出以下信息：

1、Bean 是对象，一个或者多个不限定。

2、Bean 托管在 Spring 中一个叫 IoC 的容器中。

3、我们的程序是由一个个 Bean 构成的。

Spring 是通过声明式配置的方式来定义 Bean 的，所有创建 Bean 需要的前置依赖或者参数都是通过配置文件先声明，Spring 启动以后会解析这些声明好的配置内容。那么，我们该如何去定义 Spring 中的 Bean 呢?

53.275.2 定义 Spring Bean 有哪些方式?

一般来说，Spring Bean 的定义配置有三种方式：

第一种：基于 XML 的方式配置

这种配置方式主要适用于以下两类场景：

1、Bean 实现类来自第三方类库、比如 DataSource 等

2、需要定义命名空间的配置，如：context、aop、mvc 等。

举个例子，来看一段代码

```
<beans>
    <import resource= “resource1.xml” />//导入其他配置文件 Bean 的定义
    <import resource= “resource2.xml” />
```

```
    <bean id="userService" class="com.gupaoedu.vip.UserService" init-method="init" destory-method="destory">
    </bean>
    <bean id="message" class="java.lang.String">
      <constructor-arg index="0" value="test"></constructor-arg>
    </bean>
</beans>
```

这段代码是标准的 Spring 配置内容，咱们从上往下来看，首先导入一个叫做 resource1.xml 的标准配置文件，然后，导入了第二个标准的配置文件 resource2.xml，接着，定义了一个叫做 userService 的 Bean，对应的类是 com.gupaoedu.vip.UserService，并且声明了在 userService 实例化之后要调用 init() 方法，在 userService 销毁之后调用 destory()方法。

第二种：基于注解扫描的方式配置

这种配置方式主要适用于：在开发中需要引用的类，如 Controller、Service、Dao 等。

两种配置方法：

```
<context:component-scan base-package="com.gupaoedu.vip">
    <context:include-filter type="regex" expression="com.gupaoedu.vip.mall.*"/>//包含的目标类
    <context:exclude-filter type="aspectj" expression="com.gupaoedu.vip.mall..*Controller+"/>        //排除的目标类
</context:component-scan>
```

在这段配置中，context:component-scan 相当于使用了 Spring 内置的扫描注解的组件 @ComponentScan ，声明了需要扫描的基础包路径 com.gupaoedu.vip，把所有 com.gupaoedu.vip.mall 下面的子包全部纳入扫描范围。并且排除了 com.gupaoedu.vip.mall 包下面所有以 Controller 结尾或者包含 Controller 结尾的类。

使用 Spring 容器管理组件的 beanName 规则，默认是类名首字母变小写，可以自己修改。Spring 主要提供了 4 种内置的注解用来声明 Bean。它们分别是@Controller，声明为控制层组件的 Bean，@Service，声明为业务逻辑层组件的 Bean，@Repository，声明为数据访问层组件的 Bean，当对组件的层次难以定位的时候使用@Component 注解来声明。

Spring 提供了四个注解，这些注解的作用与上面的 XML 定义 bean 效果一致，在于将组件交给 Spring 容器管理。组件的名称默认是类名（首字母变小写），可以自己修改：

第三种：基于 Java 类的配置

这种配置方式主要适用于以下两类场景：

1、需要通过代码控制对象创建逻辑的场景；

2、实现零配置，消除 XML 配置文件的场景。

同样还是举个例子，来看一段代码：

```
@Configuration
public class BeansConfiguration {

    @Bean
    public Student student(){

        Student student = new Student();
        student.setName("张三");
        student.setTeacher(teacher());
        return student;

    }

    @Bean
    public Teacher teacher(){

        Teacher teacher = new Teacher();
        teacher.setName("Tom");
        return teacher;

    }

}
```

通过分析这段代码，我可以了解到使用基于类的配置需要以下步骤：

1、首先，在 BeansConfiguration 类上配置@Configuration 注解，表示将 BeansConfiguration 类将定义 Bean 的元数据

2、在方法上使用@Bean 注解，方法名默认就是 Bean 的名称，方法返回值就是 Bean 的实例。

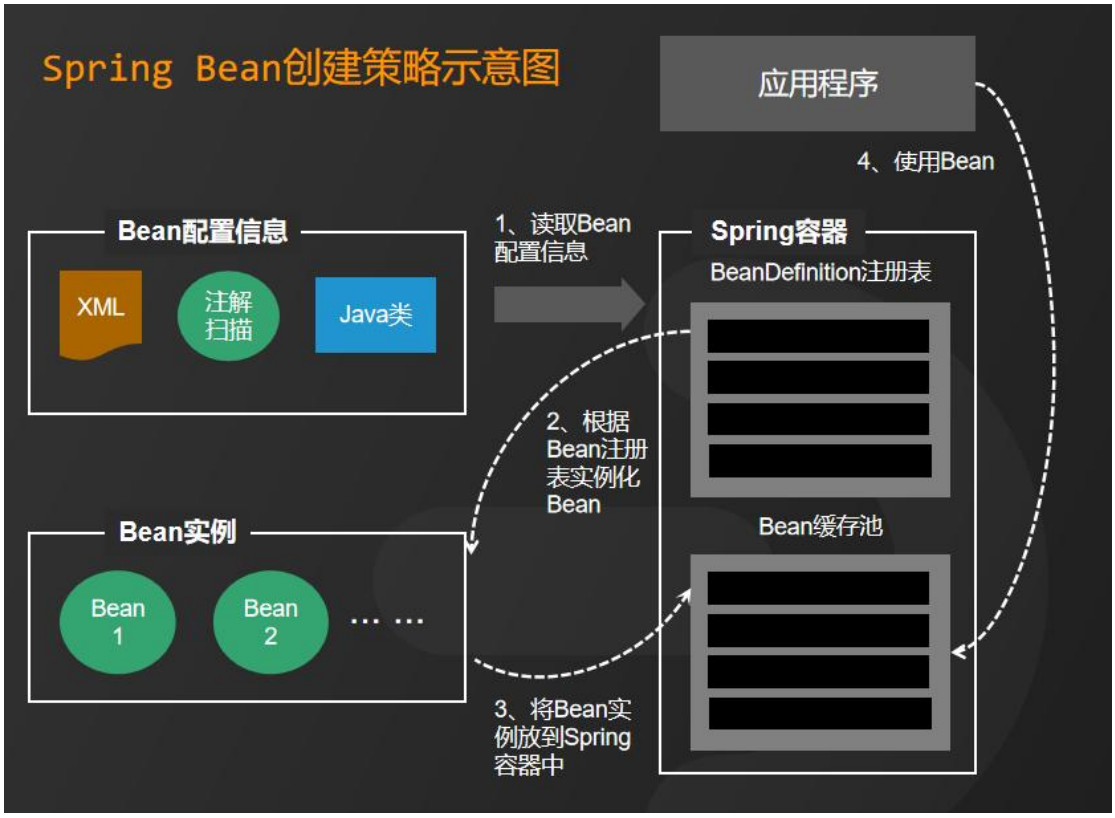
3、通过 AnnotationConfigApplicationContext 或子类来启动 Spring 容器，从而加载这些已经声明好的注解配置。

最后，总结一下，定义 Spring Bean 三种方式，分别是：基于 XML 的方式配置、基于注解扫描的方式配置、基于元数据类的配置。

那么 Bean 的配置定义好以后，Spring 又是如何加载这些 Bean 配置并创建 Bean 实例呢?

53.275.3 Spring 容器如何加载 Bean?

Spring 解析这些声明好的配置内容，将这些配置内容都转化为 BeanDefinition 对象，BeanDefinition 中几乎保存了配置文件中声明的所有内容，再将 BeanDefinition 存到一个叫做 beanDefinitionMap 中。以 beanName 作为 Key，以 BeanDefinition 对象作为 Value。之后 Spring 容器，根据 beanName 找到对应的 BeanDefinition，再去选择具体的创建策略。而 Spring 具体的创建策略如图所示，大家想要高清无码图的可以在评论区留言。



关于 Spring Bean 的面试题解析就到这里，相信大家也已经理解。如果想看 Spring BeanDefiniton 定义的小伙伴，可以关注我后续的更新内容。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，也可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

关注我，面试不再难！

53.276 Spring Bean 的定义包含哪些内容？（3.18 上午已拍摄）

大家好，我是被编程耽误的文艺 Tom。

前面我发了一个关于 Spring Bean 的视频。在这个视频中，我简单提到了 Spring Bean 的定义。其中，有几位同学就私信我，说老师能不能拍一期关于 Spring Bean 定义的详细介绍，今天我就来满足大家的要求。关于 Spring Bean 的定义我一共分为三部分来介绍，首先，介绍 Spring Bean 声明式配置内容；然后，介绍 BeanDefinition 与配置文件的关系；最后，介绍 Spring 如何解析配置文件？

咱们先来看 Spring Bean 声明式配置内容有哪些？

53.276.1 Spring Bean 声明式配置内容

关于 Spring Bean 的配置内容非常多，我主要列举九个关键的配置属性，比如：class、scope、lazy-init、depends-on、name、constructor-arg、properties、init-method、destroy-method 等。

这些属性都是要在 Spring 配置文件中声明的内容。在 Spring 容器启动后，这些配置内容都会映射到一个叫做 BeanDefinition 的对象中。然后，所有的 BeanDefinition 对象都会保存到一个叫做 beanDefinitionMap 的容器中，这个容器是 Map 类型，以 Bean 的唯一标识作为 key，以 BeanDefinition 对象实例作为值。这样 Spring 容器创建 Bean 时，就不需要再次读取和解析配置文件，只需要根据 Bean 的唯一标识，去 beanDefinitionMap 中取到对应的 BeanDefinition 对象即可。

那么，接下来我们看一下 BeanDefinition 是如何定义的。

53.276.2 BeanDefinition 与配置文件的关系

我们可以对照源码来看，BeanDefinition 的基础实现类 AbstractBeanDefinition 类，这个类下面的所有属性都能够和声明配置文件中的内容一一对应上，来看代码：

```
public AbstractBeanDefinition implements BeanDefinition {

    ...

    @Nullable
    private volatile Object beanClass;

    @Nullable
    private String scope = SCOPE_DEFAULT;

    private boolean lazyInit = false;

    @Nullable
    private String[] dependsOn;

    @Nullable
    private String factoryBeanName;

    @Nullable
    private ConstructorArgumentValues constructorArgumentValues;

    @Nullable
    private MutablePropertyValues propertyValues;

    @Nullable
    private String initMethodName;

    @Nullable
    private String destroyMethodName;

    ...

}
```

我们可以看到，BeanDefinition 中定义的属性和声明式的配置内容从命名上看比较类似。本期视频中，我重点介绍 5 个：

1、beanClass 对应的配置是 class，这个属性为必填项，用于指向一个具体存在的 Java 类，Spring 容器创建的 Bean 就是这个 Java 类的实例。

2、lazyInit 对应的配置是 lazy-init，用于指定 Bean 实例是否延时加载，我们能清楚地看到默认值是 false。也就是说容器启动时就会创建 Bean 对应的实例，如果设置为 true，则只有在首次获取 Bean 的实例时才创建。

3、dependsOn 对应的配置是 depends-on，用于定义 Bean 实例化的依赖关系。在 Spring 容器对 Bean 的实例初始化之前，有可能存在其他依赖，这需要需要保证其所以依赖的 Bean 需要提前实例化，depends-on 可以用来定义 Bean 的依赖顺序。在 BeanDefinition 中属性定义的数据类型是字符串数组，也就是说可以同时定义多个依赖对象。

4、factoryBeanName 对应的配置就是 name，这个属性用于定义 Bean 的唯一标识，且不能以大写字母开头。在 XML 配置中，使用 id 或 name 属性来指定。如果没有设值，Spring 默认使用类名首字母小写作为唯一标识。

5、constructorArgumentValues 对应的配置是 constructor-arg，它其实也是一个数组。如果 Java 类中定义了有参构造方法，则可以使用此属性给有参构造方法注入参数值。如果没有默认的无参构造方法，那么，这个属性必填。

其他的属性我相信小伙伴根据属性名称也能够自己一一对应上。我呢，也给大家整理成了一个表格，有需要完整表格的小伙伴可以在评论区留言，可以发给大家。

Spring Bean 声明式配置和 BeanDefinition 属性定义对照表

序号	配置	对应属性	说明
1	class	beanClass	这个属性为必填项，用于指向一个具体存在的Java类，Spring容器创建的Bean就是这个Java类的实例
2	scope	scope	用于定义作用域，默认值为singleton，单例，一共有singleton、prototype、request、session、globalSession　五个值可选，不同的作用域将由Spring选择特定的策略创建Bean的实例。
3	lazy-init	lazyInit	用于指定Bean实例是否延时加载。默认为false，也就是说容器启动时就会创建Bean对应的实例，如果设置为true，则只有在首次获取Bean的实例时才创建。
4	depends-on	dependsOn	用于定义Bean实例化的依赖关系。在Spring容器对Bean的实例初始化之前，有可能存在其他依赖，这需要需要保证其所以依赖的Bean需要提前实例化，depends-on可以用来定义Bean的依赖顺序。
5	name	factoryBeanName	这个属性用于定义Bean的唯一标识，且不能以大写字母开头。在XML配置中，使用id或name属性来指定。如果没有设值，Spring默认使用类名首字母小写作为唯一标识。
6	constructor-arg	constructorArgumentValues	如果Java类中定义了有参构造方法，则可以使用此属性给有参构造方法注入参数值。如果没有默认的无参构造方法，那么，这个属性必填。
7	properties	propertyValues	用于给Bean的指定属性赋默认值。
8	init-method	initMethodName	用于指定在Bean完成初始化之后需要调用的回调方法。当Bean的所有必需的属性被容器设置之后，意味着Bean初始化完成。
9	destroy-method	destroyMethodName	用于指定包含该Bean的实例被销毁时，需要调用的回调方法。

对照源码看完之后，大家应该非常清楚 Spring Bean 定义的关键内容包含哪些属性了。那么，Spring 又是如何解析这些配置文件变成 BeanDefinition 对象的呢?

53.276.3 Spring 如何解析配置文件?

Spring 容器启动之后，会调用 BeanDefinitionReader 工具类的 loadBeanDefinitions() 方法，启动对配置文件的加载和解析。 BeanDefinitionReader 的主要作用是读取 Spring 配置文件中的内容，将其转换为 BeanDefinition 对象。而 BeanDefinitionReader 又有非常多的实现类，每种类型的配置具体解析的过程又不一样，比如

XmlBeanDefinitionReader ， 用于读取 XML 文件并解析为 BeanDefinition 对象。

PropertiesBeanDefinitionReader，用于读取属性文件，将 Resource，Property 等解析为 BeanDefinition 对象。

GroovyBeanDefinitionReader，用于读取 Groovy 语言定义的 Bean，将它们解析为 BeanDefinition 对象。

以上就是关于 Spring Bean 定义的全部解析。我是被编程耽误的文艺 Tom，如果大家还有其他疑问，可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

53.277 为什么 Spring 中每个 Bean 都要定义作用域?（3.18 下午已拍摄）

大家好，我是被编程耽误的文艺 Tom。

前面的视频中都有提到过 Spring Bean 的作用域。本期视频呢，我针对 Spring Bean 作用域做一个详细的解答。关于 Spring Bean 的作用域，我一共分为两个部分来介绍。

首先，介绍 Spring Bean 作用域的定义，然后，介绍 Spring 为什么要定义作用域?

咱们先来看 Spring Bean 作用域的定义有哪些?

53.277.1 Spring Bean 作用域的定义

在 Spring 配置中，我们可以通过 scope 属性来定义 Spring Bean 的作用域，可以接受 5 个内建的值，分别代表 5 种作用域类型，下面给大家详细总结一下：

1、singleton，用来定义一个 Bean 为单例，也就是说在 Spring loC 容器中仅有唯一的一个实例对象，Spring 中的 Bean 默认都是单例的。它的作用域范围是 ApplicationContext 容器

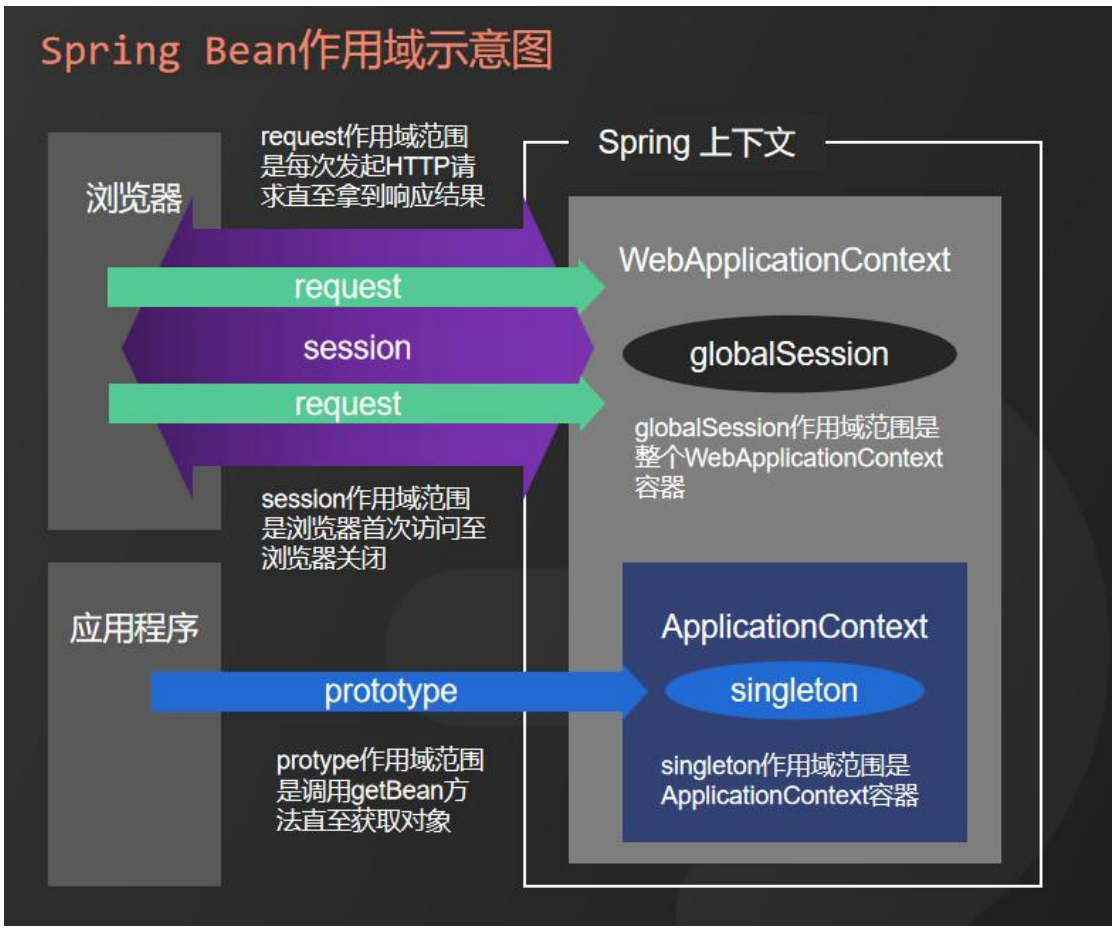
2、prototype，用来定义一个 Bean 为多例，也就是说在每次请求获取 Bean 的时都会重新创建实例，因此每次获取到的实例对象都是不同的。它的作用域范围是调用 getBean 方法直至获取对象。

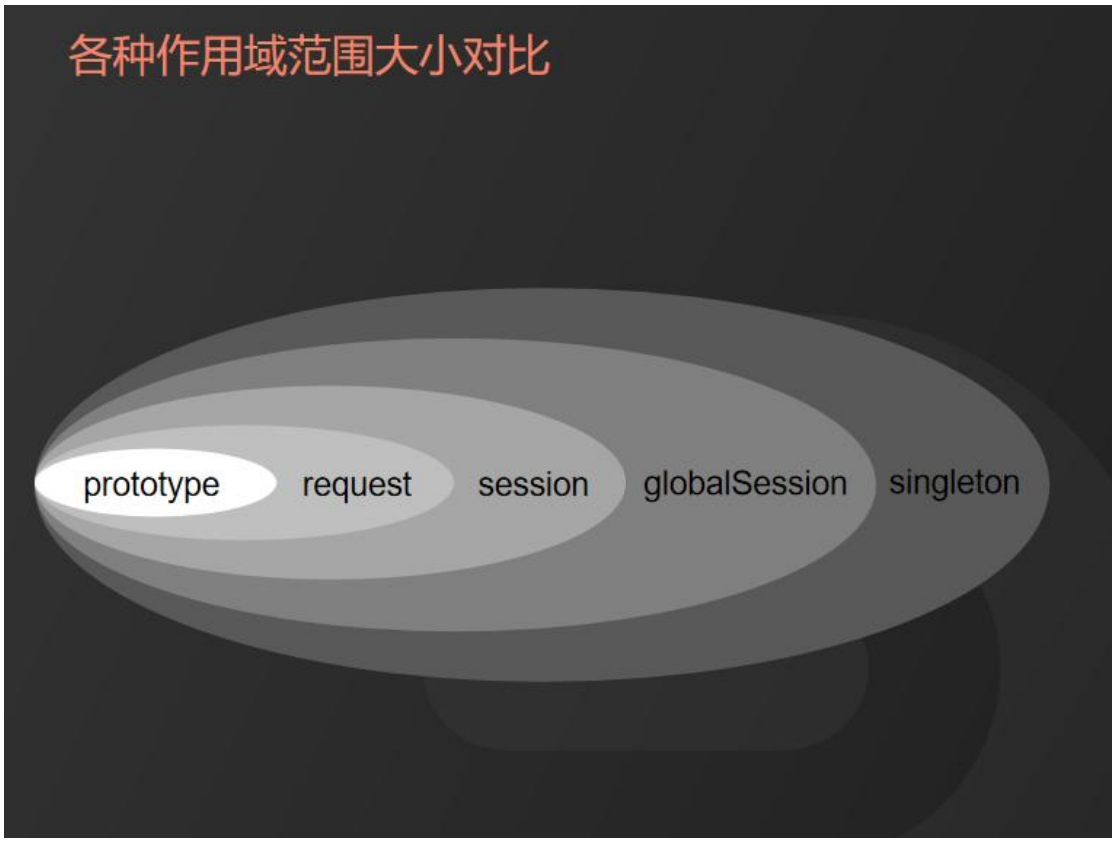
3、request，用来定义一个作用范围仅在 request 中的 Bean，也就是说在每次 HTTP 请求时会创建一个实例，该实例仅在当前 Request 中有效。它的作用域范围是每次发起 HTTP 请求直至拿到响应结果。

4、session，用来定义一个作用范围仅在 session 中的 Bean，也就是说在每次 HTTP 请求时会创建一个实例，该实例仅在当前 HTTP Session 中有效。它的作用域范围是浏览器首次访问至浏览器关闭。

5、globalSession，用来定义一个作用范围仅在中的 Bean。这种方式仅用于应用环境，也就是说该实例仅存在于 WebApplicationContext 环境中。它的作用域范围是整个 WebApplicationContext 容器。

第一个 singleton 和第二个 prototype 是比较常用的。其他三种仅适用于 Web 应用环境中，咱们也无须关心用什么样的框架，只需要符合 J2EE 规范即可生效。


Spring Bean作用域示意图
浏览器
request作用域范围
是每次发起HTTP请
求直至拿到响应结果
Spring 上下文
WebApplicationContext
request
session
globalSession
request
globalSession作用域范围是
整个WebApplicationContext
容器
session作用域范围
是浏览器首次访问至
浏览器关闭
应用程序
ApplicationContext
prototype
singleton
protype作用域范围
是调用getBean方
法直至获取对象
singleton作用域范围是
ApplicationContext容器


各种作用域范围大小对比
prototype
request
session
globalSession
singleton

这一张图呢，是表示各种作用域范围大小对比，其中 prototype 大于 request 大于 session 大于 globalSession 大于 singleton。大家可以私信我获取高清图，下载下来慢慢看，帮助大家更好地理解作用域范围。

53.277.2 Spring 为什么要定义作用域?

定义 Bean 的作用域，相当于用户可以通过配置的方式限制 Spring Bean 的使用范围，以起到保护 Bean 安全的作用。就好比孙悟空外出打妖怪前，给唐僧画了一个圈。唐僧只有待在圈里才能保证安全，出圈就可能会遇到危险。这样，唐僧访问不到圈外的资源，圈外的资源也无法触达到唐僧，以此形成一个安全的隔离区。

在日常开发中，我们可以根据业务需要，选择定义不同的作用域，以保护 Bean 的使用安全。

关于 Spring Bean 的作用域解析就到这里。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，也可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

关注我，面试不再难！

53.278 Spring Bean 的生命周期全过程

大家好，我是被编程耽误的文艺 Tom。今天能给大家介绍一下 Spring Bean 生命周期全过程，这道题呢也是大厂高频面试题。接下来我给大家做一个详细的分析和解答。

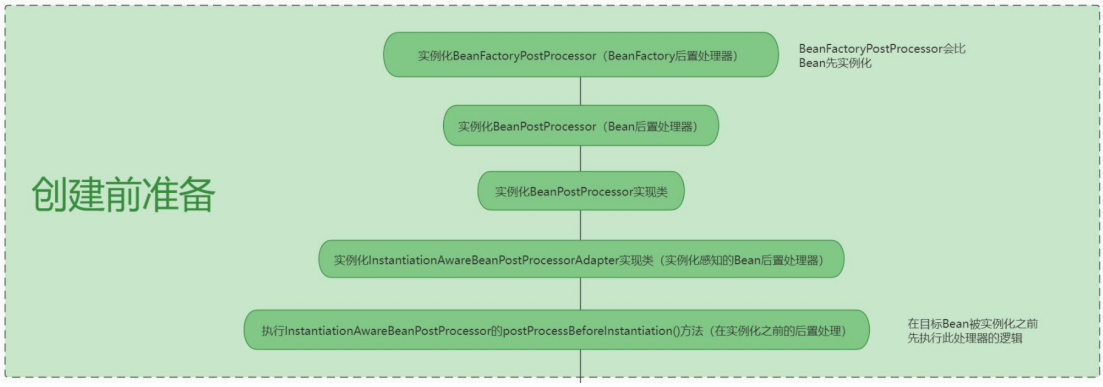
Spring 生命周期全过程大致分为五个阶段：创建前准备阶段、创建实例阶段、依赖注入阶段、

容器缓存阶段和销毁实例阶段。

这张图呢展示了 Spring Bean 生命周期完整流程，其中对每个阶段的具体操作做了详细介绍。下面我们来分析一下每个阶段的细节。

先来看第一阶段。

53.278.1 创建前准备阶段



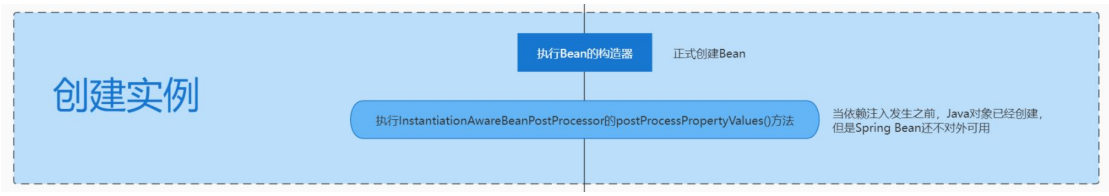
这个阶段主要是在开始 Bean 加载之前，从 Spring 上下文和相关配置中解析并查找 Bean 有关的配置内容，

比如 `init-method`-容器在初始化 bean 时调用的方法、`destory-method`，容器在销毁 Bean 时调用的方法。

以及，BeanFactoryPostProcessor 这类的 bean 加载过程中的前置和后置处理。

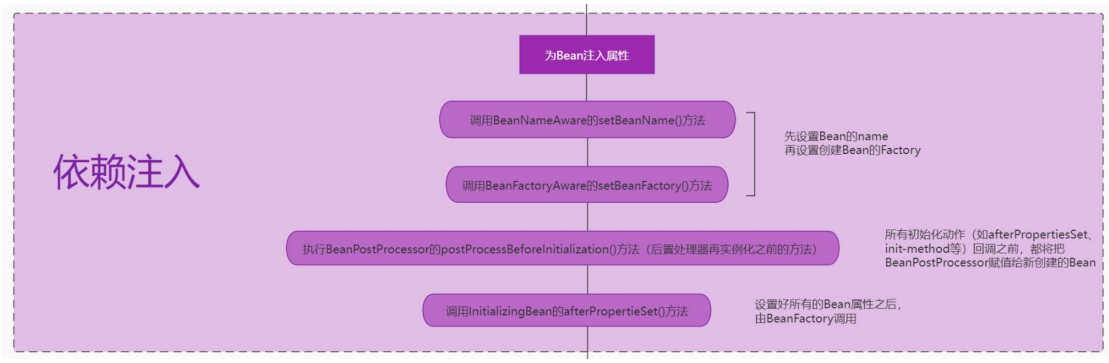
这些类或者配置其实是 Spring 提供给开发者，用来实现 Bean 加载过程中的扩展机制，在很多和 Spring 集成的中间件经常使用，比如 Dubbo。

53.278.2 创建实例阶段



这个阶段主要是通过反射来创建 Bean 的实例对象，并且扫描和解析 Bean 声明的一些属性。

53.278.3 依赖注入阶段



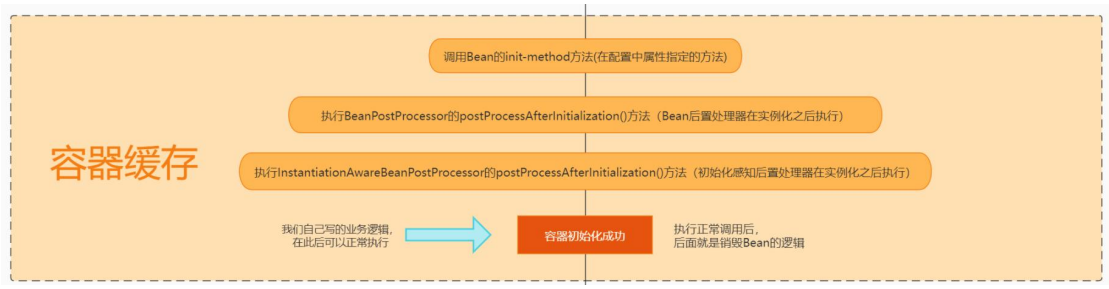
在这个阶段，会检测被实例化的 Bean 是否存在其他依赖，如果存在其他依赖，1 就需要对这些被依赖 Bean 进行注入。比如通过读取

@Autowired、@Setter 等依赖注入的配置。

在这个阶段还会触发一些扩展的调用，比如常见的扩展类：BeanPostProcessors（用来实现 Bean 初始化前后的回调）、

InitializingBean 类（这个类有一个 afterPropertiesSet()方法，给属性赋值）、还有 BeanFactoryAware 等等。

53.278.4 容器缓存阶段



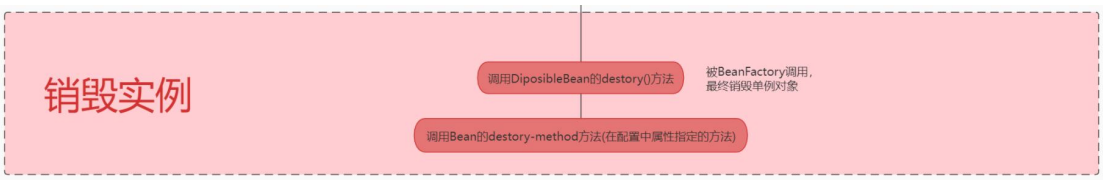
容器缓存阶段主要是把 Bean 保存到 IoC 容器中缓存起来，到了这个阶段，Bean 就可以被开发者使用了。

这个阶段涉及到的操作，常见的有，init-method 这个属性配置的方法，会在这个阶段调用。

在比如 BeanPostProcessors 方法中的后置处理器方法如：

postProcessAfterInitialization，也是在这个阶段触发的。

53.278.5 毁实例阶段

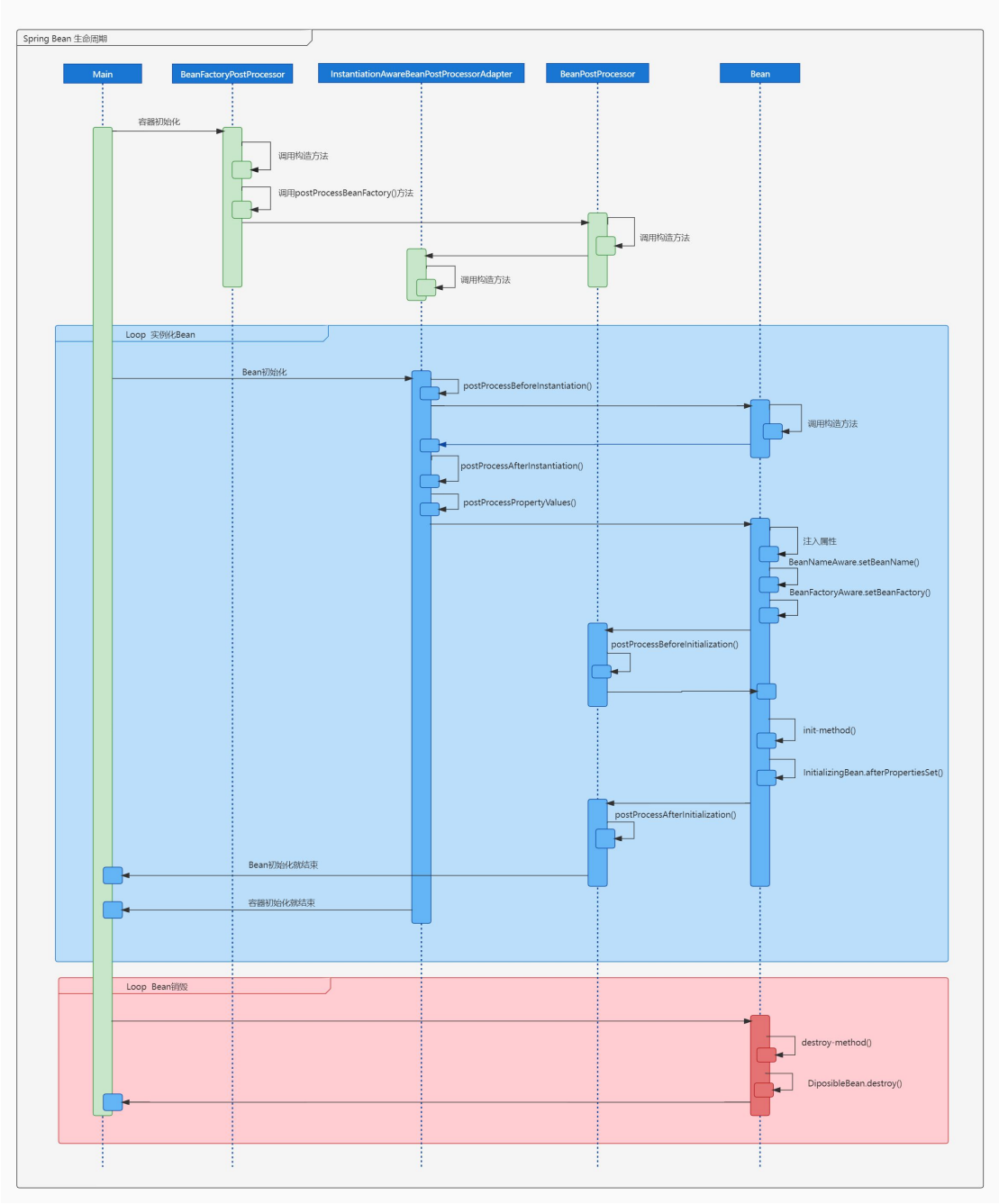


这个阶段，是完成 Spring 应用上下文关闭时，将销毁 Spring 上下文中所有的 Bean。

如果 Bean 实现了 DisposableBean 接口，或者配置了 destory-method 属 性，将会在这个阶段被调用。

看到这里，相信大家对 Spring Bean 的生命周期有了深刻的印象，需要视频中 Spring Bean 生

命周期的高清流程图，可以私信我。在附赠一张高清的时序图给大家！



我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.279 Spring 为何需要三级缓存解决循环依赖，而不是二级缓存?

今天给大家分享一道大厂面试真题，Spring 为何需要三级缓存解决循环依赖，而不是二级缓存？我一共分为五个部分来给大家介绍：

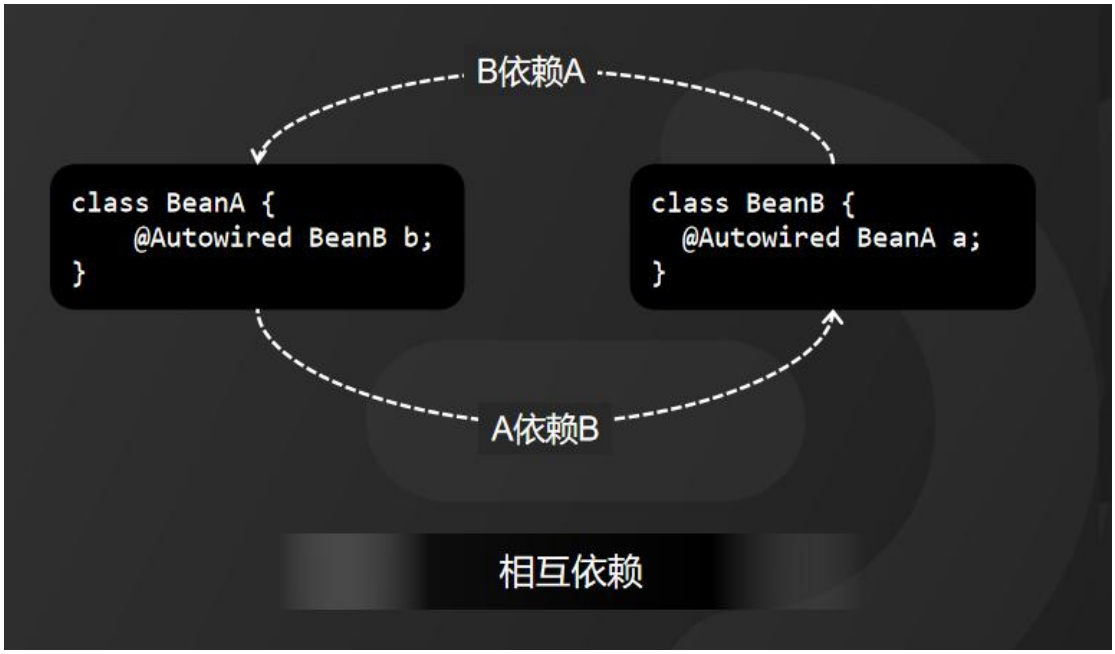
53.279.1 什么是循环依赖?

循环依赖就是指循环引用，是两个或多个 Bean 相互之间的持有对方的引用。在代码中，如果将两个或多个 Bean 互相之间持有对方的引用，因为 Spring 中加入了依赖注入机制，也就是自动给属性赋值。Spring 给属性赋值时，将会导致死循环。那么，哪些情况会出现循环依赖呢?

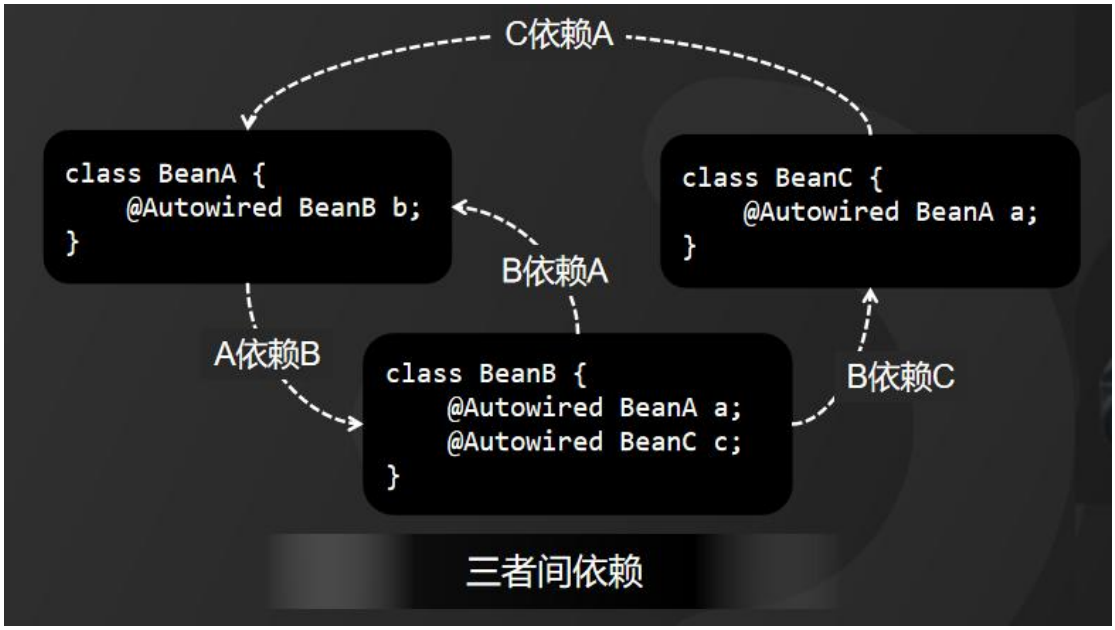
53.279.2 哪些情况会出现循环依赖?

循环依赖有三种形态:

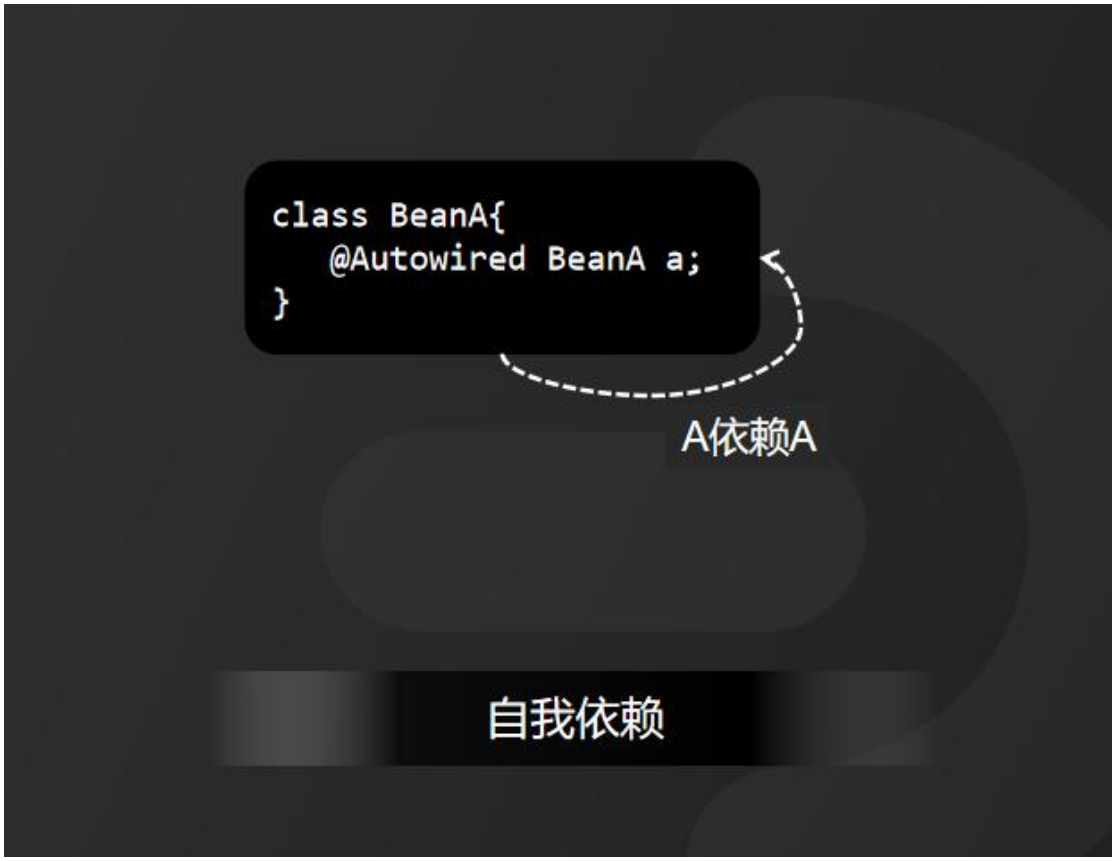
1、相互依赖，也就是 A 依赖 B，B 又依赖 A，它们之间形成了循环依赖。



2、三者间依赖，也就是 A 依赖 B，B 依赖 C，C 又依赖 A，形成了循环依赖。



3、自我依赖，也是 A 依赖 A 形成了循环依赖自己依赖自己。



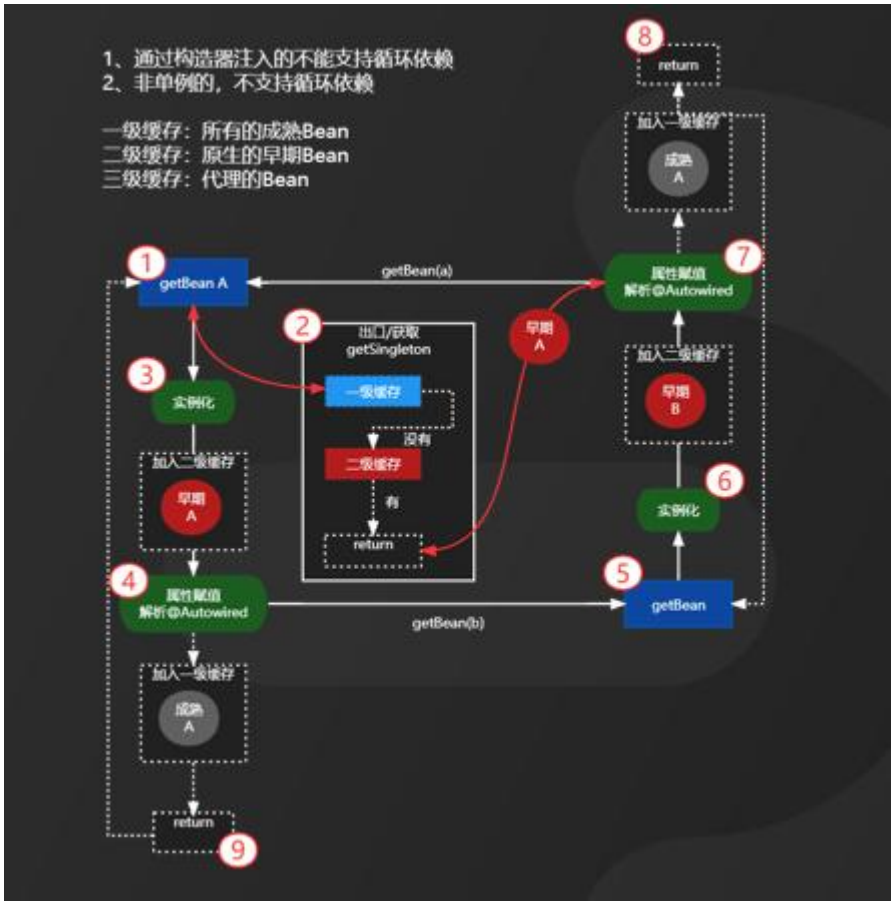
53.279.3 Spring 如何解决循环依赖问题?

Spring 中设计了三级缓存来解决循环依赖问题，当我们去调用 getBean()方法的时候，Spring 会先从一级缓存中去找到目标 Bean，如果发现一级缓存中没有 便会去二级缓存中去找，而如果一、二级缓存中都没有找到，意味着该目标 Bean 还没有实例化。于是，Spring 容器会实例化目标 Bean（PS：刚初始化的 Bean 称为早期 Bean），然后，将目标 Bean 放入到二级缓存中，同时，加上标记是否存在循环依赖。如果不存在循环依赖便会将目标 Bean 存入到二级缓存，否则，便会标记该 Bean 存在循环依赖，然后将等待下一次轮询赋值，也就是解析@Autowired 注解。等@Autowired 注解赋值完成后（PS：完成赋值的 Bean 称为成熟 Bean），会将目标 Bean 存入到一级缓存。

总结一下，Spring 一级缓存中存放所有的成熟 Bean，二级缓存中存放所有的早期 Bean，先取一级缓存，再去二级缓存。

53.279.4 为何需要三级缓存，而不是二级缓存?

那么，前面有提到三级缓存，三级缓存的作用是啥呢？来看这样一张图，三级缓存是用来存储代理 Bean，当调用 getBean()方法时，发现目标 Bean 需要通过代理工厂来创建，此时会将创建好的实例保存到三级缓存，最终也会将赋值好的 Bean 同步到一级缓存中。大家可以私信我或者在评论区留言获取高清大图。



5、Spring 中哪些情况下，不能解决循环依赖问题?

1.多例 Bean 通过 setter 注入的情况，不能解决循环依赖问题

2.构造器注入的 Bean 的情况，不能解决循环依赖问题

3.单例的代理 Bean 通过 Setter 注入的情况，不能解决循环依赖问题

4.设置了@DependsOn 的 Bean 的情况，不能解决循环依赖问题

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.280 请简述Spring MVC的执行流程(暂定3.25上午拍摄)

今天我给大家介绍一下 Spring MVC 的详细执行流程。我把 Spring MVC 的执行流程划分为三个阶段，配置阶段、初始化阶段和运行阶段。

我整理了一张完整的执行流程图，需要高清图的小伙伴可以私信我。下面详细介绍每个阶段的执行细节。

53.280.1 第一阶段：配置阶段

配置阶段，主要是完成对 xml 配置和注解配置。

具体步骤如下：

首先，从 web.xml 开始，配置 DispatcherServlet 的 url 匹配规则和 Spring 主配置文件的加载路径

然后，就是配置注解，比如@Controller、@Service、@Autowrited 以及@RequestMapping 等。

53.280.2 第二阶段：初始化阶段

初始化阶段，主要是加载并解析配置信息以及 IoC 容器、DI 操作和 HandlerMapping 的初始化。

具体步骤如下：

首先，Wer 容器启动以后，会由 Web 容器自动调用 DispatcherServlet 的 init()方法。

然后，在 init()方法中，会初始化 IoC 容器，IoC 容器其实就是个 Map。

紧接着，根据配置好的扫描包路径，扫描出相关的类，并利用反射进行实例化，存放到 IoC 容器中。

缓存之后，Spring 将再次迭代扫描 IoC 容器中的实例，给需要自动赋值的属性自动赋值。哪些属性需要自动赋值呢？比如加了@Autowrited 的属性。

最后，读取@RequestMapping 注解，获得请求 url，将 url 和 Method 建议一对一的映射关系并缓存起来。我们可以简单粗暴地理解为缓存在一个 Map 中，它的 Key 就是 url，它的值是 Method。

53.280.3 第三阶段：运行阶段

运行阶段，在 Spring 启动以后，等待用户请求，完成内部调度并返回响应结果。

具体步骤如下：

用户在浏览器输入 url 之后，Web 容器会接收到用户请求。Web 容器会自动调用 doGet()或者 doPost()方法。从 doGet()或者 doPost()方法中，我们可以获得两个对象，分别是 request 和 response。通过 request 可以获得用户请求带过来的信息，通过 response 可以往浏览器端输出响应结果。

然后，根据 request 中获得的请求 url，可以从 HandlerMapping 中找到对应 Method。

接着，还是利用反射调用方法，将获得方法调用的返回结果。

最后，将返回结果通过 response 输出到浏览器，用户就可以看到响应结果。

都已经看到这里了，大家是不是觉得 Spirng MVC 执行流程非常简单？

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，可以在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.281 刚折腾完 Log4J，又爆 Spring RCE 核弹级漏洞

继 Log4J 爆出安全漏洞之后，又在深夜，Spring 的 github 上又更新了一条可能造成 RCE（远程命令执行漏洞）的问题代码，随即在国内的安全圈炸开了锅。有安全专家建议升级到 JDK 9 以上，有些专家又建议回滚到 JDK 7 以下，一时间小伙伴们不知道该怎么办了。大家来看一段动画演示，怎么改都是“将军"。

大家不要慌，我给大家先临时支个招，后面再出教程。首先叫大家怎么排查哪些项目存在风险，然后，再介绍修复方案。

53.281.1 第一步：排查方法

排查的主要目的是确定你的项目是否使用了 Spring 框架。当然，你的项目有没有使用 Spring 框架开发者都知道。但是，如果公司项目比较多，为了规避风险，还要对一些老项目要进行排查。那老项目如何确定是否使用了 Spring 框架呢?

方法很简单，不管的你的项目是用 war 独立部署还是用 jar 包独立部署，只需要对应的 war 或者 jar 包，将后缀改为 zip 包，然后将 zip 解压。在解压后的目录中搜索是否存在 spring-beans-开头的 jar 包或者 CachedInrospectionResults.class 文件。如果存在就可以确定该项目使用了 Spring。

确定项目使用了 Spring 框架以后，如何来修复可能存在的风险呢?

53.281.2 第二步：修复方案

目前为止，Spring 官方还没有给出解决方案。我先教大家一个简单粗暴的方案，可以临时解决问题。

1、如果安装了 WAF 防护，也就是 Web 应用防火墙，只需要追加这样一个防护规则“class.*Class..class.***.Class”，防止远程下载。

2、如果没有安装 WAF，只需要在拦截器中增加对 class.*Class..class.***.Class 后缀请求的拦截就可以了。

按照这两步操作完之后，大家记得对业务运行情况进行测试，避免对已有功能造成影响。

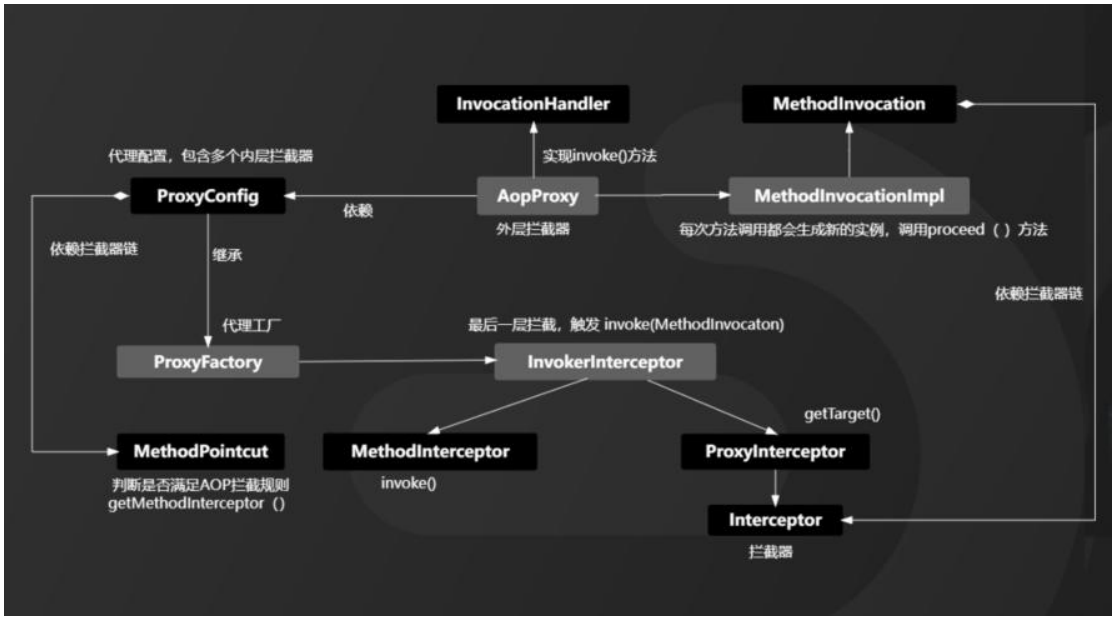
此次爆出的漏洞，攻击原理和之前 Log4J 爆出的漏洞原理差不多。如果大家对原理有兴趣的话，可以关注我发布的其他视频。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，技术不再难！

53.282 被面试官问烂的 Spring AOP 原理，你是怎么答的?

Spring AOP 在 Spring 体系中深不可测，Spring AOP 原理也是经常在互联网大厂面试时被问到，今天，我给大家抽丝剥茧，详细到你无法想象。我划分为四个阶段给大家介绍：创建代理对象阶段、拦截目标对象阶段、调用代理对象阶段、调用目标对象阶段。

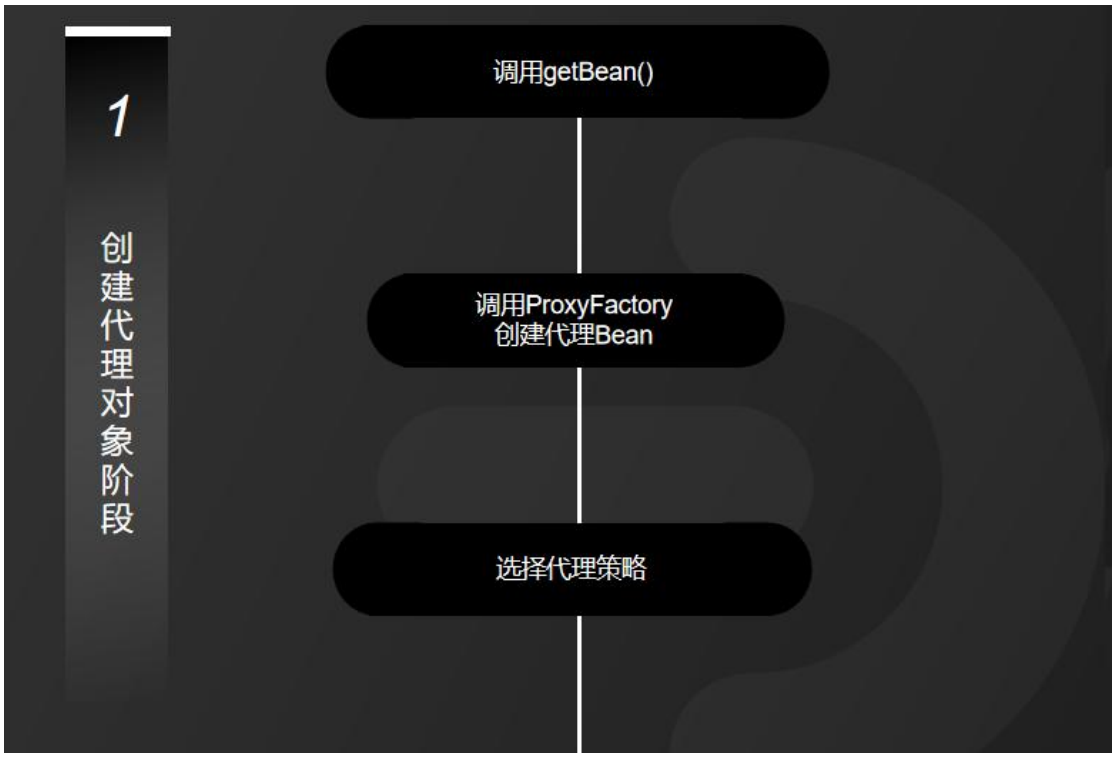
我整理了一张完整的 Spring AOP 设计原理 UML 图，需要高清图的小伙伴可以私信我。



下面详细介绍每个阶段的执行细节。

53.282.1 第一阶段：创建代理对象阶段

在 Spring 中，创建 Bean 实例都是从 getBean()方法开始的*（出现 getBean()**动画**）*，在实例创建之后，Spring 容器将根据 AOP 的配置去匹配目标类的类名，看目标类的类名是否满足切面规则。如果满足满足切面规则，就会调用 ProxyFactory 创建代理 Bean 并缓存到 IoC 容器中。*（出现**调用 ProxyFactory 创建代理 Bean 动画**）*根据目标对象的自动选择不同的代理策略。*（出现**选择代理策略动画**）*如果目标类实现了接口，Spring 会默认选择 JDK Proxy，如果目标类没有实现接口，Spring 会默认选择 Cglib Proxy，当然，我们也可以通过配置强制使用 Cglib Proxy。

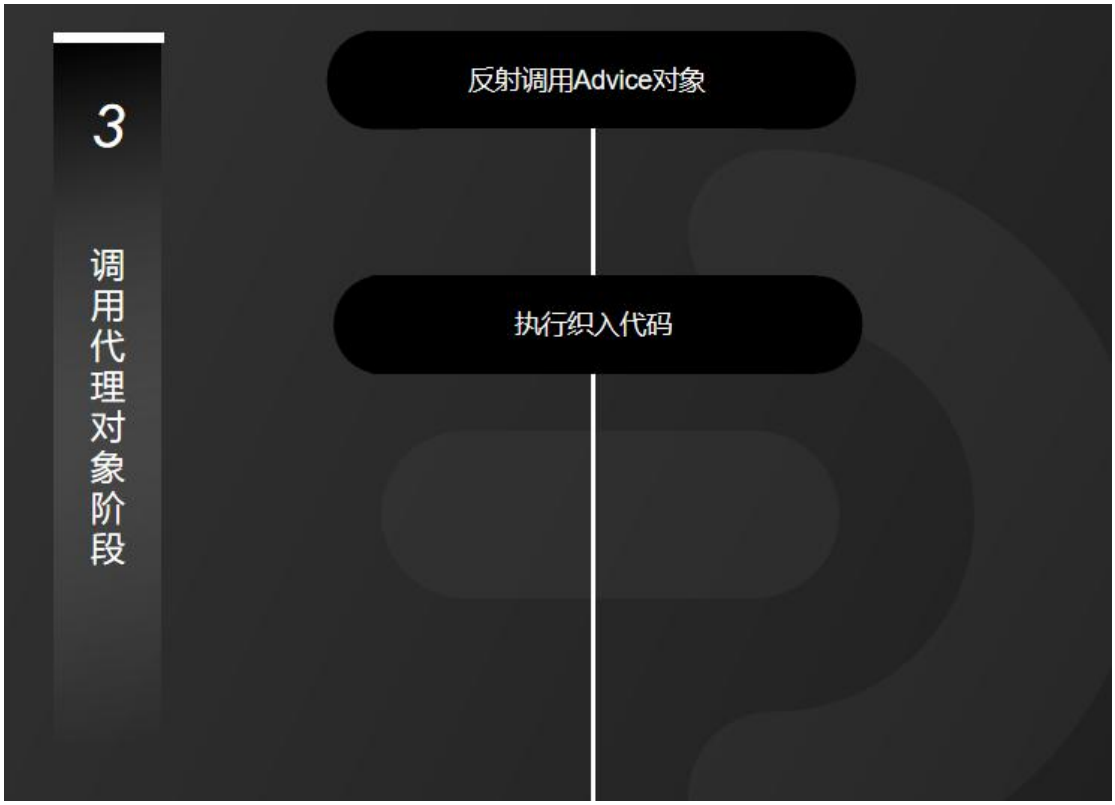


53.282.2 第二阶段：拦截目标对象阶段

当用户调用目标对象的某个方法时，*（出现**AopProxy 对象拦截**）*将会被一个叫做 AopProxy 的对象拦截,Spring 将所有的调用策略封装到了这个对象中，它默认实现了 InvocationHandler 接口，也就是调用代理对象的外层拦截器。*（出现**调用 InvocationHandler 的 invoke()方法动画**）*在这个接口的 invoke()方法中，*（出现**触发 MethodInvocation 的 proceed()方法动画**）*会触发 MethodInvocation 的 proceed()方法。在这个方法中会按顺序执行符合所有 AOP 拦截规则的拦截器链。

53.282.3 第三阶段：调用代理对象阶段

Spring AOP 拦截器链中的每个元素被命名为 MethodInterceptor，*（出现**反射调用 Advice 对象动画**）*其实就是切面配置中的 Advice 通知。这个回调通知可以简单地理解为是新生成的代理 Bean 中的方法。*（出现**执行织入代码动画**）*也就是我们常说的被织入的代码片段，这些被织入的代码片段会在这个阶段执行。



4、第四阶段：调用目标对象阶段

MethodInterceptor 接口也有一个 invoke()方法， *（出现执行 MethodInterceptor 的 invoke()方法动画）* 在 MethodInterceptor 的 invoke()方法中会触发对目标对象方法的调用，也就是反射调用目标对象的方法。

Spring AOP 原理就分析到这里，最后，总结一下不迷路：

（出现代理对象动画）

1、代理对象：就是由 Spring 代理策略生成的对象；

（出现目标对象动画）

2、目标对象：就是我们自己写的业务代码；

（出现织入代码动画）

3、织入代码：就是要在我们自己写的业务代码增加的代码片段；

（出现切面通知动画）

4、切面通知：就是封装织入代码片段的回调方法；

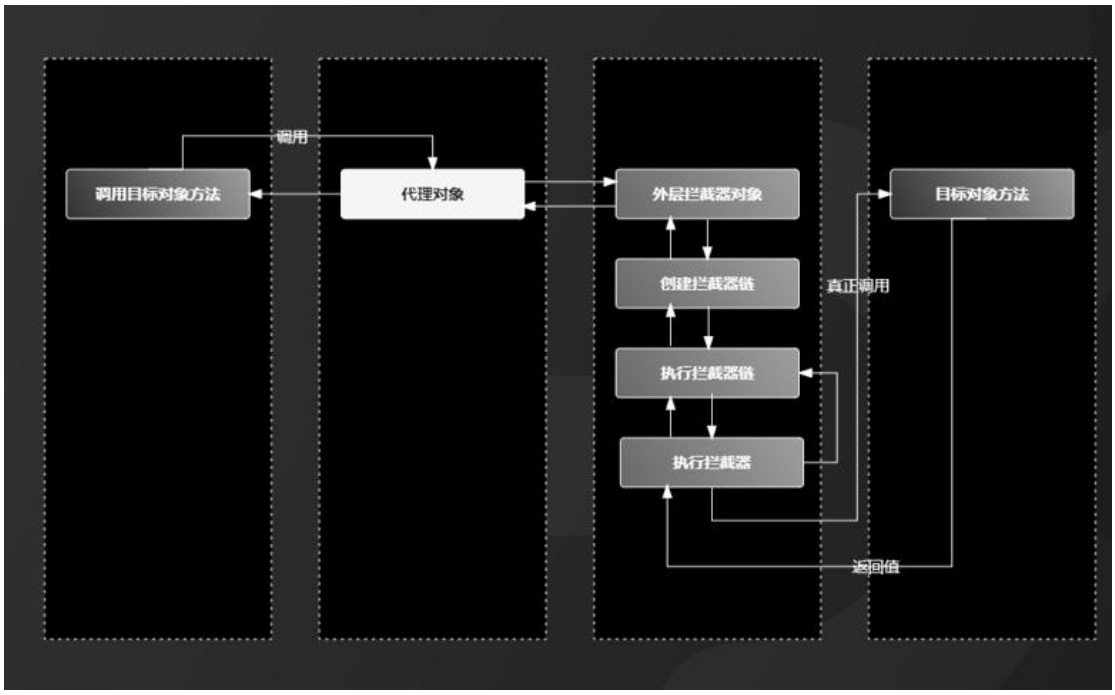
（出现 MethodInvocation 动画）

5、MethodInvocation：负责执行拦截器链，在 proceed()方法中执行；

（出现 MethodInterceptor 动画）

6、MethodInterceptor：负责执行织入的代码片段，在 invoke()方法中执行。

都看到这里了，你还觉得 Spirng AOP 原理难吗？我再送给大家一张精简版的 Spring AOP 执行流程图，需要的小伙伴立马关注点个赞。



我是被编程耽误的文艺 Tom，如果大家还有其他疑问，可以在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.283 3 分钟轻松理解单线程下的 HashMap 工作原理

HashMap 主要是用来处理键值对数据。随着 JDK 版本的更新，JDK1.8 对 HashMap 对底层也做了一些优化。今天我带大家一起来结合源码，深入浅出 HashMap 工作原理。
I

HashMap 是基于哈希表对 Map 接口的实现类，它的特点是访问数据的速度快，并不是按顺序来遍历。 HashMap 提供所有可选的映射操作，但不能保证映射顺序不变，并且允许使用空值和空键。HashMap 也并不是线程安全的，当存在多个线程同时写入时，可能会导致数据不一致的情况。

53.283.1 HashMap 中的关键属性

要透彻理解 HashMap 原理，首先需要对以下几个关键属性有一个基本的认识。

```java
public class HashMap<K,V> extends AbstractMap<K,V>
implements Map<K,V> {

    final float loadFactor;

    int threshold;

    transient int size;

    transient int modCount;

    static final int DEFAULT_INITIAL_CAPACITY = 1 << 4;
}
```

我们看到，HashMap 的源码片段：

第一个属性 loadFactor，它是负载因子，默认值是 0.75，表示扩容前 。

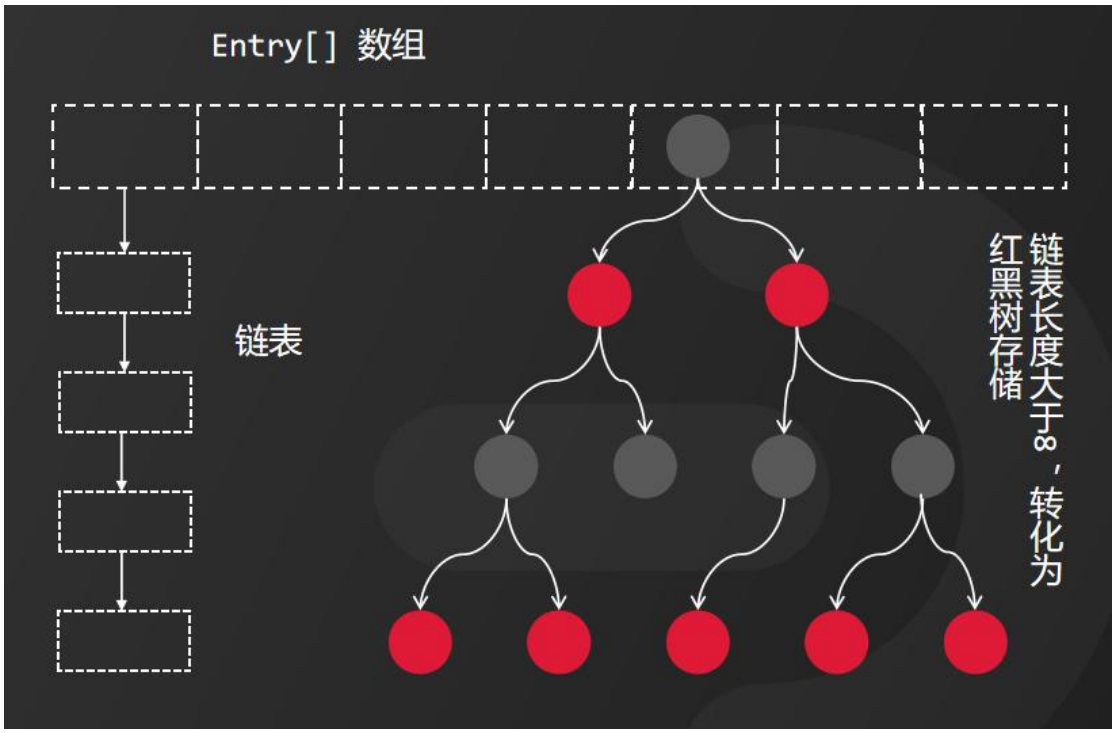
第二个属性 threshold 它是记录 HashMap 所能容纳的键值对的临界值，它的计算规则是负载因子 乘以 数组长度。

第三个属性 size，它用来记录 HashMap 实际存在的键值对的数量。

第四个属性 modCount，它用来记录 HashMap 内部结构发生变化的次数。

第五个是常量属性 DEFAULT_INITIAL_CAPACITY ，它规定 的默认容量是 16。

53.283.2 HashMap 的存储结构



HashMap 采用的是 的存储结构。HashMap 的数组部分称为 Hash 桶，数组元素保存在一个叫做 table 的属性中。当链表长度大于等于 8 时，链表数据将会以红黑树的形式进行存储，当长度降到 6 时，又会转成链表形式存储。

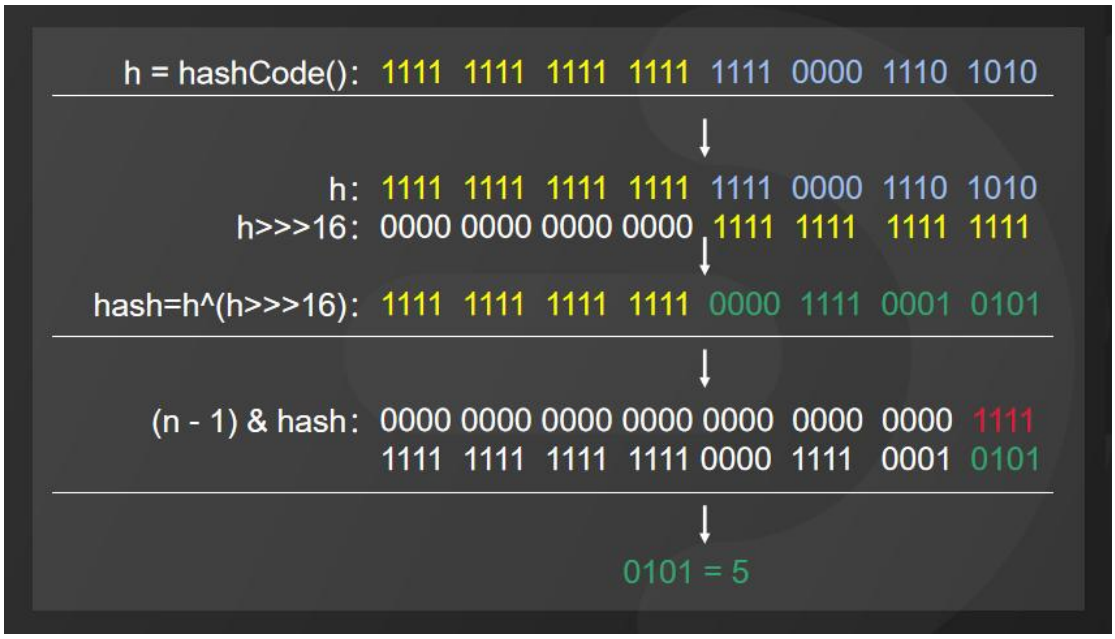
```
static class Node<K,V> implements Map.Entry<K,V> {
    ...

    final int hash;
    final K key;
    V value;
    Node<K,V> next;

    ...
}
```

每个 Node 节点，保存了用来定位数组索引位置的 hash 值、Key、Value 和链表指向的下一个 Node 节点。而 Node 类是 HashMap 的内部类，它实现了 Map.Entry 接口，它的本质其实可以简单的理解成就是一个键值对。来看一下源码。

3、HashMap 的工作原理

当我们向 HashMap 中插入数据时，首先要确定 Node 在数组中的位置。那如何确定 Node 的存储位置呢？以添加 Key 为字符串“e”的对象为例：

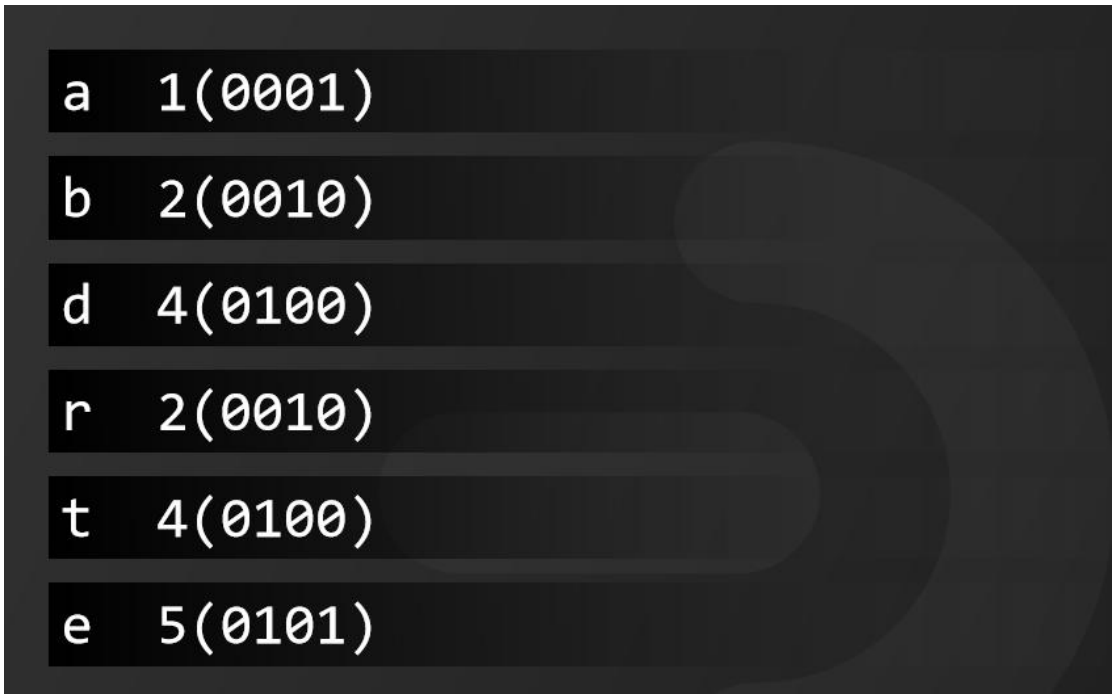
```
public static int hashCode(Object o) {
    return o != null ? o.hashCode() : 0;
}
```



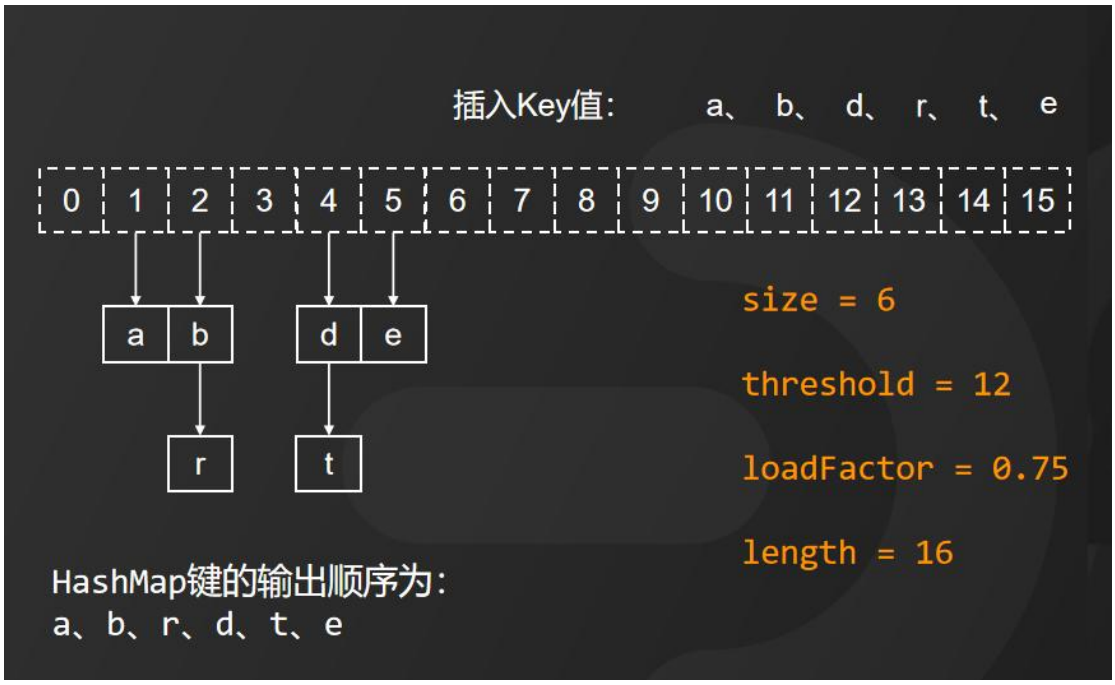
HashMap 首先调用 hashCode()方法，获取 Key 的 hashCode 值为 h。然后对 h 值进行高位运算；将 h 右移 16 位取得 h 的高 16 位，与 进行异或运算，最后得到 h 的值与 (table.length - 1)进行与运算获得该对象的保留位，最后计算出下标。当然，这是最官方的描述。有的小伙伴可能已经迷糊了。其实，这段运算过程，简单地理解成求模取余法。

就是用 hash 值和数组的长度减 1，取模，最后得到数组的下标，这样可以保证数组下标不越界。只不过，位运算是二进制运算，效率更高。

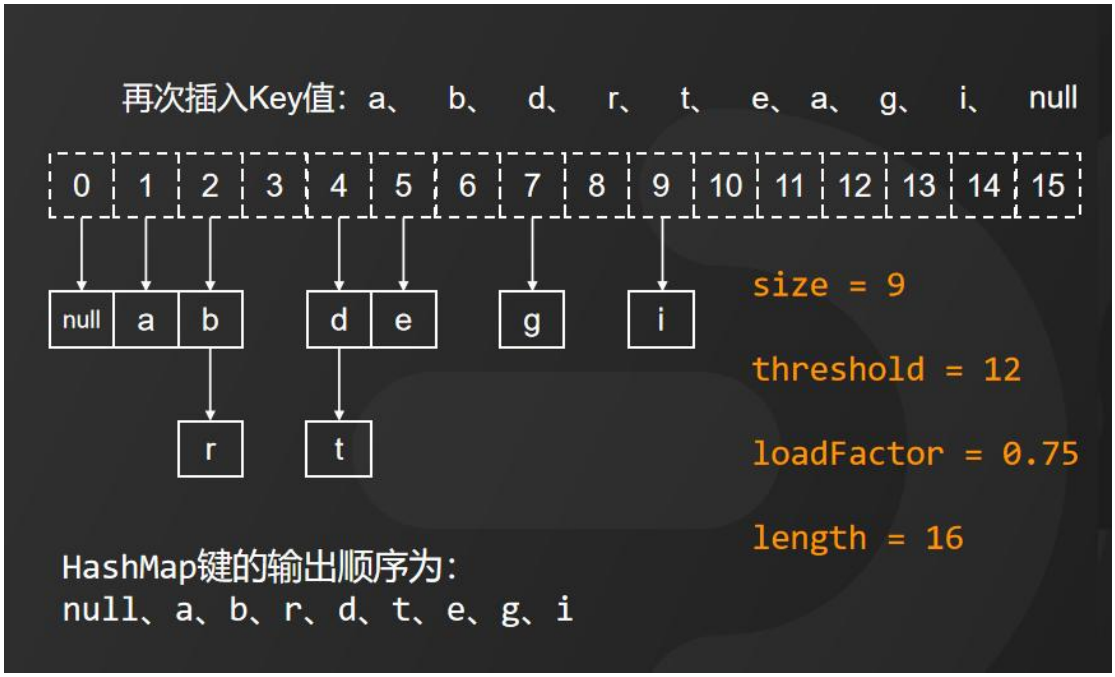
最后，来看一段动画演示，假设有“a”、“b”、“d”、“r”，“t”，“e”的 Key。通过计算得到的下标分别为 1 、2 、 4 、2 、4 、5



它们的插入顺序如动画所示。



如果我们再次插入 "a"， “g”， “i”，null 四个 Key，来看 HashMap 的内部变化。



当插入第二个以 a 为 Key 的对象时，会将新值赋值给 a 的值。当插入的对象大小超过临界值时，HashMap 将新建一个桶数组并重新赋值（当然，JDK1.7 和 1.8 重新赋值的方式略有不同）

这个时候，HashMap 键的输出顺序为 null、a、b、r、d、t、e、g、i

HashMap 的工作原理，你搞懂了吗?

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.284 什么是内存溢出，什么是内存泄漏?

什么是内存溢出，什么是内存泄漏？这是很多小伙伴经常问我的一个问题，今天花 3 分钟时间给大家介绍一下。先来介绍什么是内存溢出？

53.284.1 什么是内存溢出？

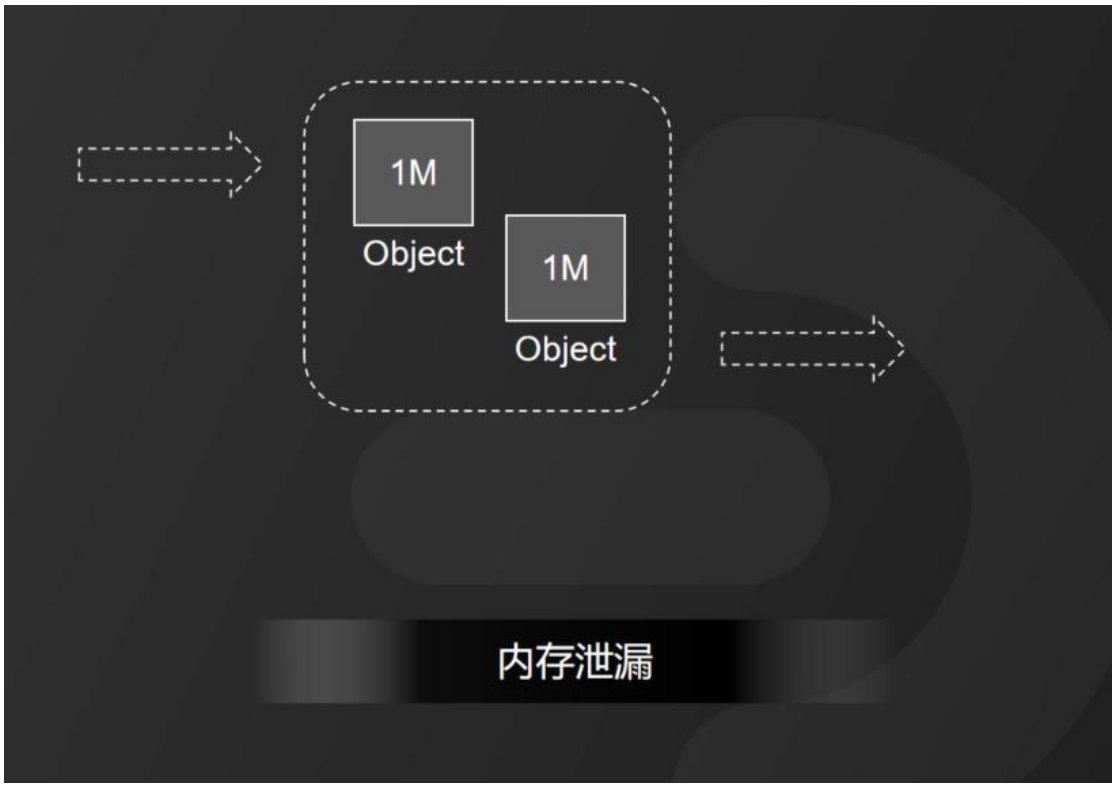
发生内存溢出。

我们来看到右侧的区域，假设我们 JVM 中可用的内存空间只剩下 3M，但是我们要创建一个 5M 的对象，那么，新创建的对象就放不进去了。这个时候，我们就叫做内存溢出。就好比是一个容量只有 300ml 的水杯，我们硬要往里面倒 500ml 的水，这时候，水就会溢出，倒不进去了，这就相当于是内存的溢出。

那么，内存泄漏又是怎么回事呢？

53.284.2 什么是内存泄漏？

还是来看这张图，



它代表业务代码执行时，所需要占用的内存空间。这段业务代码中创建了两个 1M 的对象，一起会占用 2M 内存。当对象使用完之后，这两个对象并没有释放，因此内存中会

留下 2M 的内存空间一直被占用。而我们的业务代码在程序中会被反复执行，每次执行都会留下 2M 不被释放，反复执行多次之后，随着时间的累积，就会有大量的对象用完不被释放，导致这些对象不能得到回收而发生内存溢出，这种情况就叫做内存泄漏。

也就说，在我们的业务代码执行过程中，有些对象它应该被回收，但是又有其他对象引用引用它，因此，GC 不能自动回收。所以，该回收的垃圾对象没有被回收，垃圾对象越堆越多，可用内存越来越少，若可用内存无法存放新的垃圾对象，最终导致内存泄漏。内存泄漏最终会导致内存溢出。

53.284.3 如何避免?

我们在 Code 过程中，特别是些一些流对象，

比如 OutputStream，Reader，BitMap，Document，很容易忘了 Close。最麻烦的是还要顺序回收，顺序错了还产生空指针，所以，大家在 Code 过程一定要注意，当然，现在有很多 IDE 会智能提示，也避免了很多低级错误。

以上就是我对内存溢出和内存泄漏的分析，听懂了的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

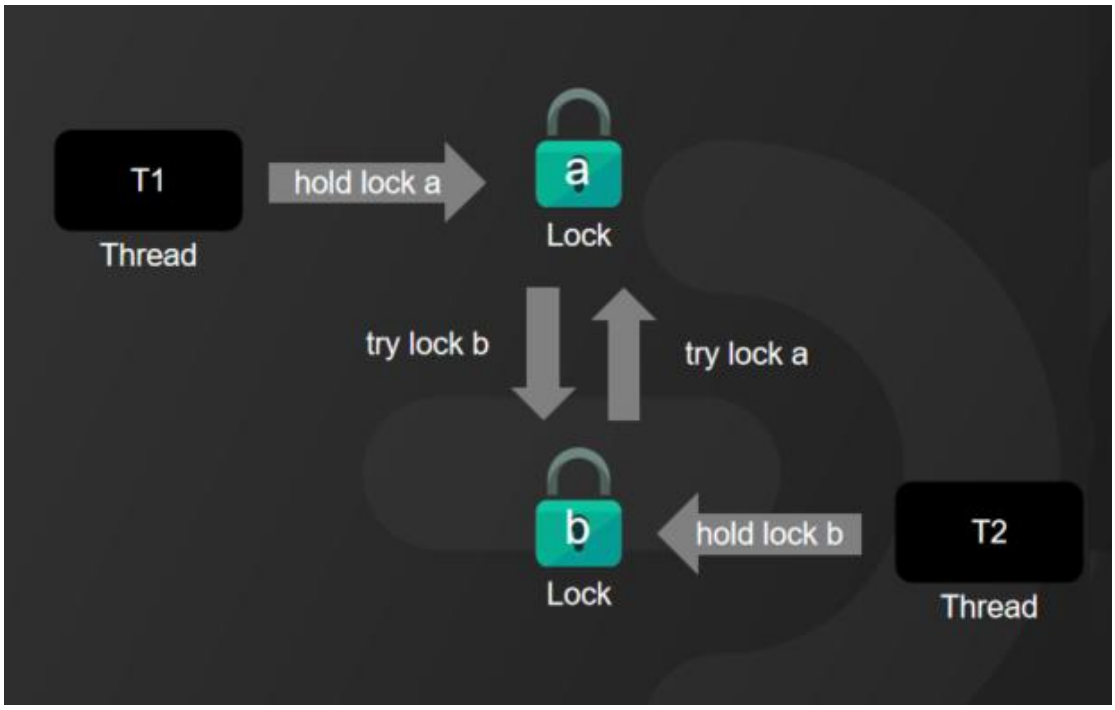
53.285 什么条件下会产出死锁，如何避免死锁?

一个去美团面试的小伙伴私我说，被面试官问到一个死锁的问题难道了，面试前还特意刷了题，面试的时候就是脑子一片空白不知道怎么回答。今天，我给大家彻底讲明白。

53.285.1 什么是死锁?

先来看这样一段视频，*(左侧区域播放视频 5 秒，人在右侧站立不动)*

看完你也许已经懂了，那到底什么是死锁呢?



死锁，简单来说就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致相互等待的现象。如果没有外部干预，线程会一直处于阻塞状态，无法往下执行。这样一直等待处于阻塞状态的线程，被称为死锁线程。

53.285.2 产生死锁的原因

产生死锁需要同时满足以下四个条件：

第一个：互斥条件，共享资源 a 和 b 只能被一个线程占用；

第二个：请求和保持条件，线程 T1 已经获取共享资源 a，在等待共享资源 b 的时候，不释放共享资源 a；

第三个：不可抢占条件，其他线程不能强行抢占线程 T1 占有的资源；

第四个：循环等待条件，线程 T1 等待线程 T2 占有的资源，线程 T2 等待线程 T1 占有的资源，这形成了循环等待。

53.285.3 如何避免死锁?

线程产生死锁之后，只能通过外部干预来解决问题，比如重启程序，或者 Kill 线程。所以，我们只能在写代码时规避死锁的产生。那么如何避免死锁产生呢？根据产生死锁的四个必要条件，我们只需要破坏其中任何一 个条件就可以解决。

第一个互斥条件是没有办法被破坏的，因为它是互斥锁的基本约束。其他三个条件都可以通过人工干预来破坏。比如请求保持条件，我们可以在首次执行一次性申请所有的资源，这样就不存在等待锁的问题了。

第二个，对于不可抢占条件来说，占用部分资源的线程在进一步申请其他资源的时候如果申请不到，我们可以主动释放它占有的资源。这样不可抢占这个条件就被破坏了。

第三个，对于循环等待条件来说，可以通过按序申请资源来预防死锁的产生。所谓按序申请，就是给资源编号，所有线程可以按照线性化的序号顺序去申请共享资源，先申请需要序号小的，再申请序号大的，这样循环等待自然就不存在了。

以上就是我对死锁的理解，听懂了的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.286 两个 Integer 对象比较大小，为什么 100 等于 100,1000 不等于 1000 ？

前几天，有位小伙伴向我反馈，在维护代码过程中，出现了一个莫名其妙的问题。明明上线之后程序跑得还好好的，可程序上线运行一段时间之后，所有，代码没有做任何修

改，发 cxcccccc 现运行结果和期望值恰好相反。因为涉及到金额造成了比较大的损失，最后，这位小伙伴还被公司辞退了，大家可以来评论一下，这位小伙伴背的这个锅值不值?

53.286.1 业务场景

大家来看，他的代码大致是这样写的：

```
public boolean xxx(Integer a,Integer b){
    ...
    if(a == b){
        ...
        return true;
    }
    ...
    return false;
}
```

a和b输入100，返回结果是：true

a和b输入1000，返回结果是：false

一般情况下，a 和 b 都输入 100 的时候，返回为 true，但当 a 和 b 都输入 1000 的时候，返回为 false。按照正常逻辑理解，100 等于等于 100，那 1000 为什么不等于等于 1000 呢？这位同学，百思不得其解。于是，这位同学，还特意写了一段测试代码

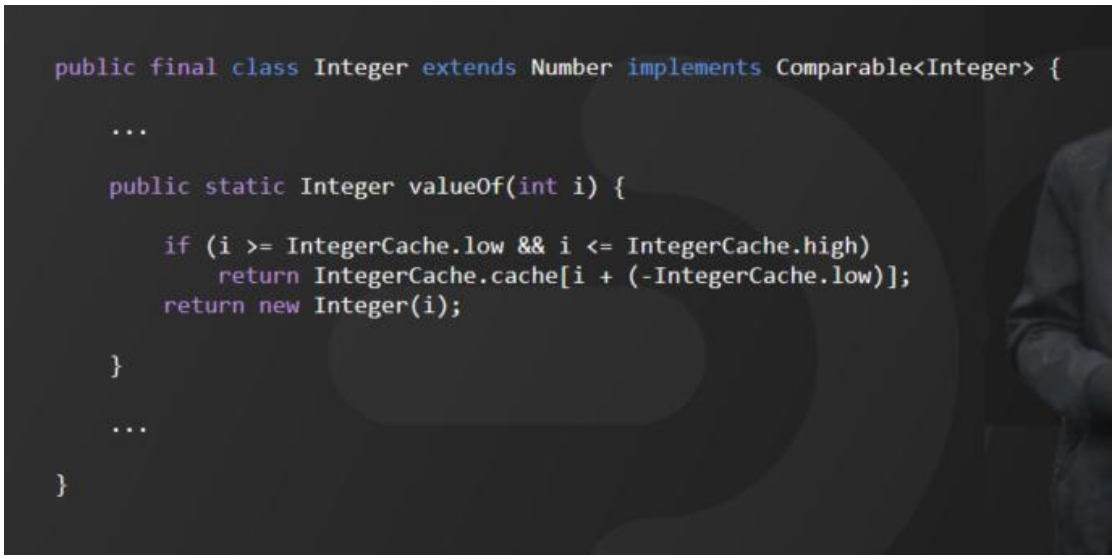
```
Integer a = 100,b = 100,c = 1000,d = 1000;

System.out.println((a == b) + “,” + (c == d));
```

运行结果是：true,false

这到底是什么原因呢？我们对照 Integer 的源码来进行分析：

53.286.2 源码分析

我摘取了 Integer 的源码片段，它有一个 valueOf()的方法：



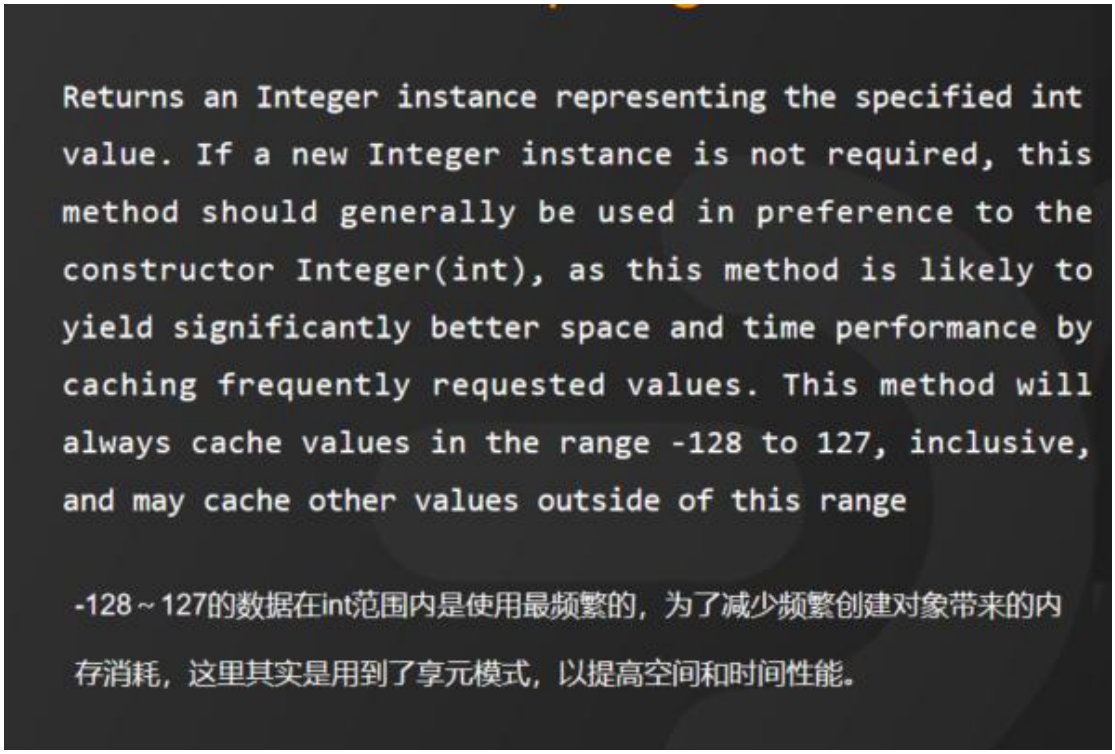
```
public final class Integer extends Number implements Comparable<Integer> {

    ...

    public static Integer valueOf(int i) {

        if (i >= IntegerCache.low && i <= IntegerCache.high)
            return IntegerCache.cache[i + (-IntegerCache.low)];
        return new Integer(i);

    }

    ...

}
```

我们可以看到，Integer 源码中的 valueOf()方法做了一个条件判断，其中 IntegerCache.low 的值为-128，

IntegerCache.high 的值为 127。

也就是说如果目标值在-128～127 之间，会直接从 cache 数组中取值，否则就会新建对象。

这里又有人会问了，那为什么默认是-128 - 127，怎么不是-200 - 200 或者是其他值呢？那 JDK 为何要这样做呢？



在 Java API 中是这样解释的：

> Returns an Integer instance representing the specified int value. If a new Integer instance is not required, this method should generally be used in preference to the constructor Integer(int), as this method is likely to yield

> significantly better space and time performance by caching frequently requested values. This method will always cache values in the range -128 to 127, inclusive, and may cache other values outside of this range

大致意思是：

-128～127 的数据在 int 范围内是使用最频繁的，为了减少频繁创建对象带来的内存消耗，这里其实是用到了享元模式，以提高空间和时间性能。

在 JDK 中，这样的应用不止 int，我给小伙伴们整理了一个表格，表格中的其他类型也都应用了享元模式，也就是说对数值做了缓存，只是缓存的范围不一样，具体如表中所示：

基本类型	大小	最小值	最大值	包装器类型	缓存范围	是否支持自定义
boolean	-	-	-	Bloolean	-	-
char	6bit	Unicode 0	Unic ode 2(16)-1	Character	0~127	否
byte	8bit	-128	127	Byte	-128~127	否
short	16bit	-2(15)	2(15)-1	Short	-128~127	否
int	32bit	-2(31)	2(31)-1	Integer	-128~127	支持
long	64bit	-2(63)	2(63)-1	Long	-128~127	否
float	32bit	IEEE754	IEEE754	Float	-	
double	64bit	IEEE754	IEEE754	Double	-	
void	-	-	-	Void	-	-

我需要这张表的小伙伴可以私信我，以上就是关于 Integer 对象比较大小的分析，听懂了的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

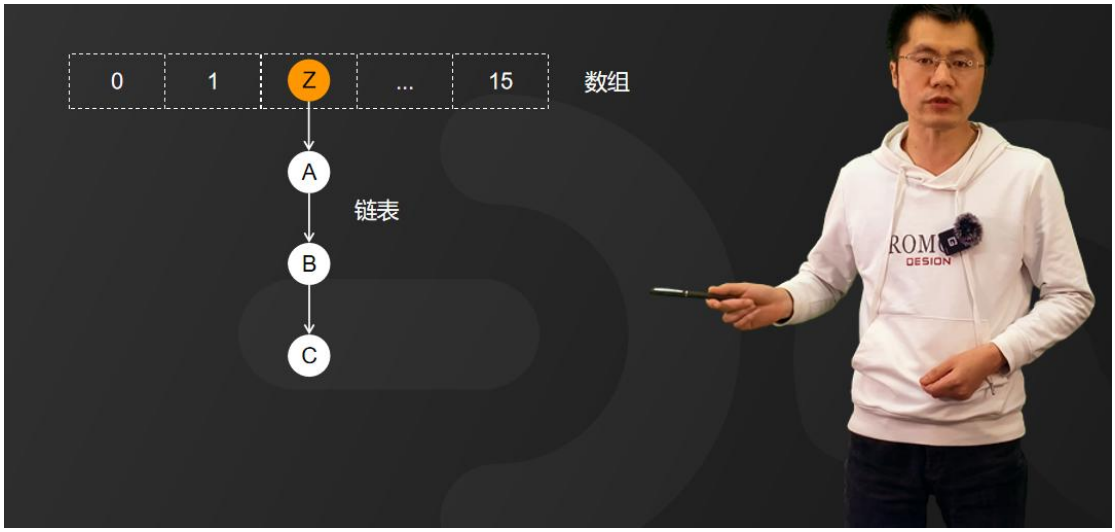
53.287 为什么 HashMap 会产生死循环?

HashMap 死循环是一个比较常见、也是比较经典的面试题，在大厂的面试中也经常被问到。HashMap 的死循环问题只在 JDK1.7 版本中会出现，主要是 HashMap 自身的工作机制，再加上并发操作，从而导致出现死循环。JDK1.8 以后，官方彻底解决了这个问题。

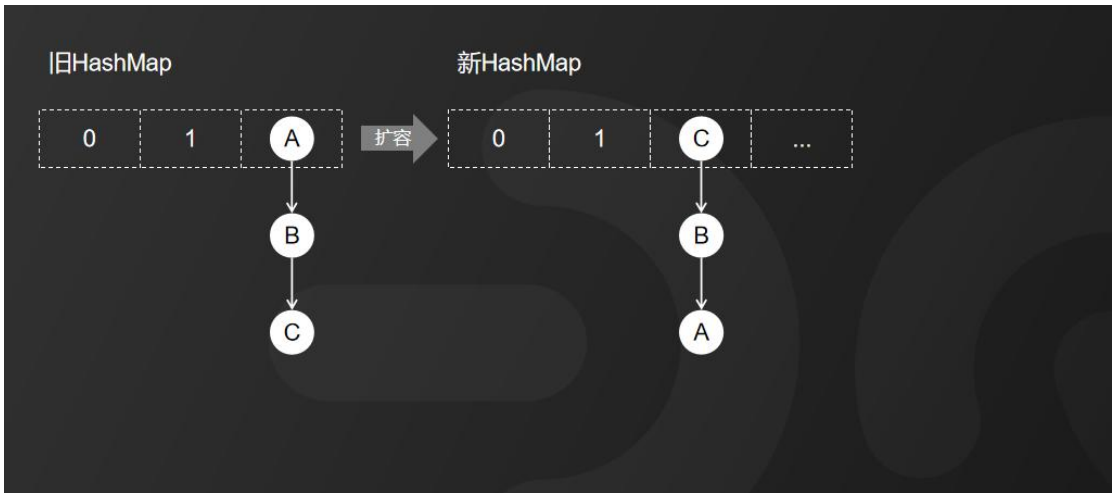
53.287.1 数据插入原理

在分析原因之前，我先带大家了解一下 JDK1.7 中 HashMap 插入数据的原理，来看动画演示：

由于 JDK 1.7 中 HashMap 的底层存储结构采用的是数组 加 链表的方式。



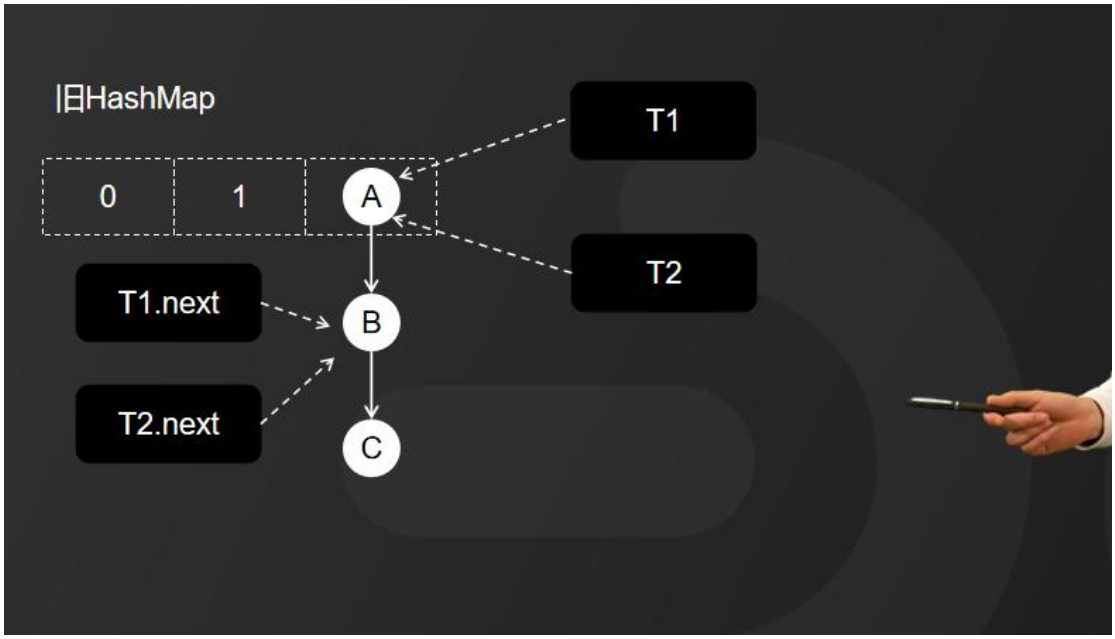
而 HashMap 在数据插入时又采用的是头插法，也就是说新插入的数据会从链表的头节点进行插入。



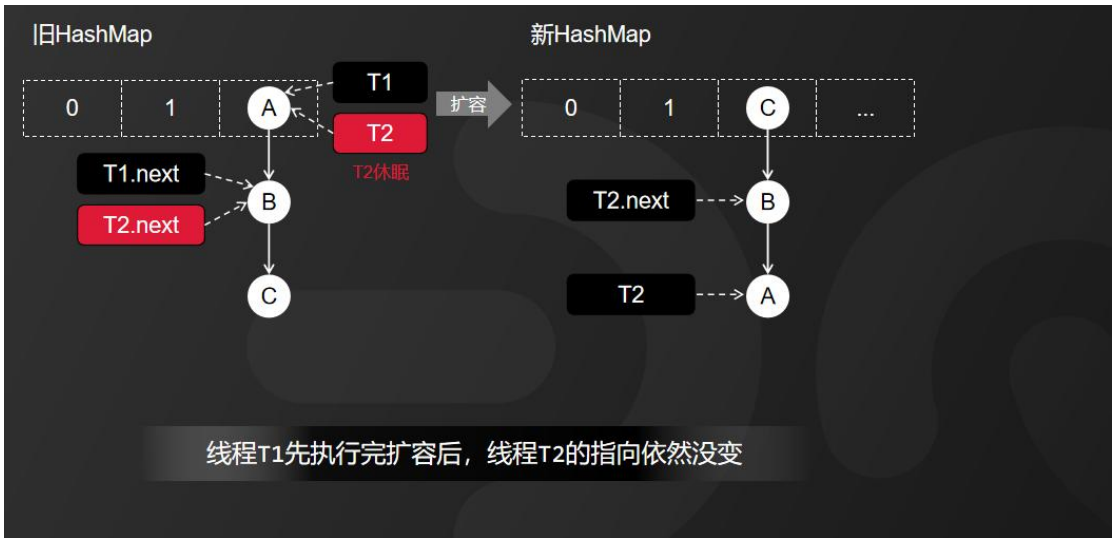
因此，HashMap 正常情况下的扩容就是是这样一个过程。我们来看，旧 HashMap 的节点会依次转移到新的 HashMap 中，旧 HashMap 转移链表元素的顺序是 A、B、C，而新 HashMap 使用的是头插法插入，所以，扩容完成后最终在新 HashMap 中链表元素的顺序是 C、B、A。

53.287.2 导致死循环的原因

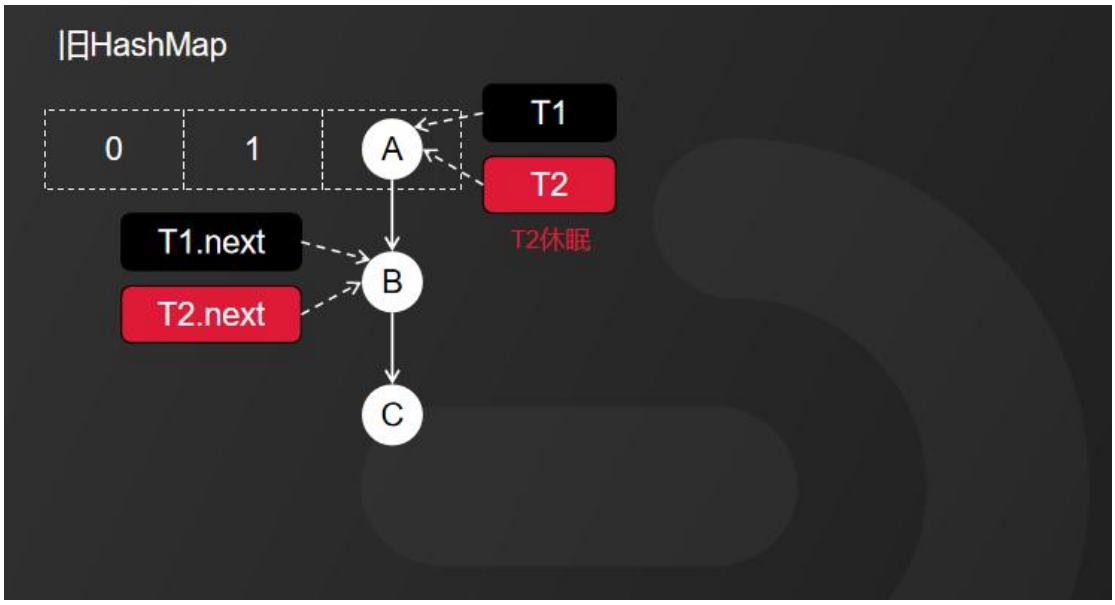
接下来，我通过动画演示的方式，带大家彻底理解造成 HashMap 死循环的原因。我们按以下三个步骤来还原并发场景下 HashMap 扩容导致的死循环问题：



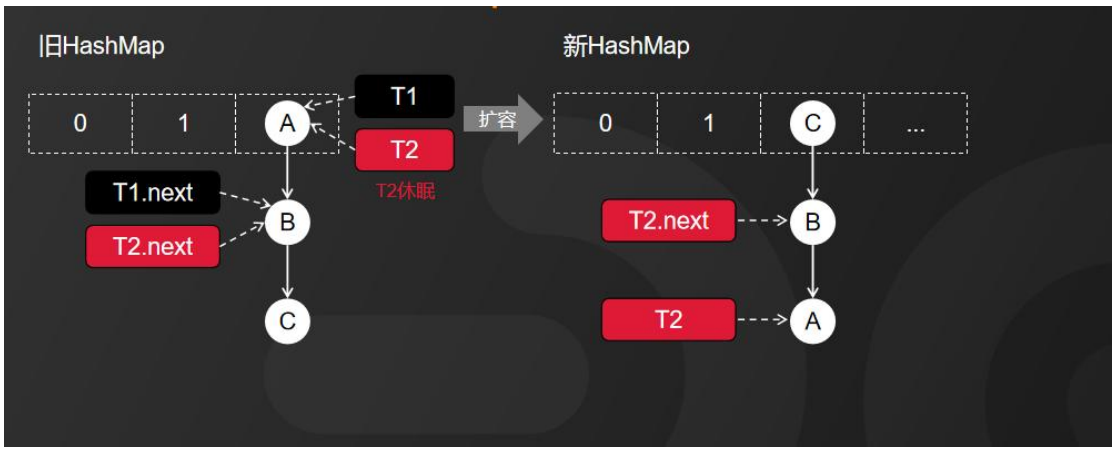
第一步：线程启动，有线程 T1 和线程 T2 都准备对 HashMap 进行扩容操作， 此时 T1 和 T2 指向的都是链表的头节点 A，而 T1 和 T2 的下一个节点分别是 T1.next 和 T2.next，它们都指向 B 节点。



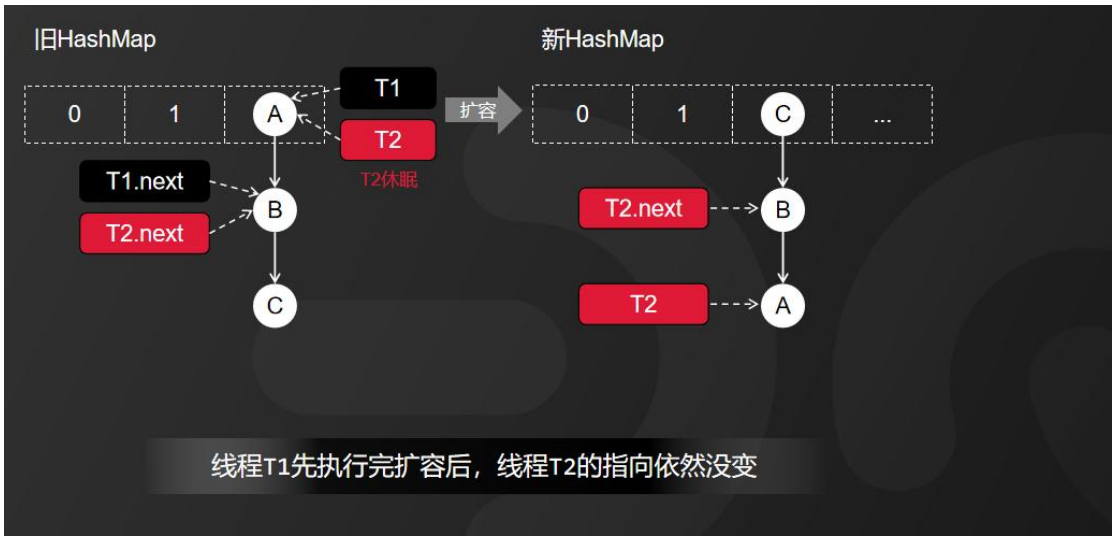
第二步：开始扩容，这时候，假设线程 T2 的时间片用完，进入了休眠状态，而线程 T1 开始执行扩容操作，一直到线程 T1 扩容完成后，线程 T2 才被唤醒。



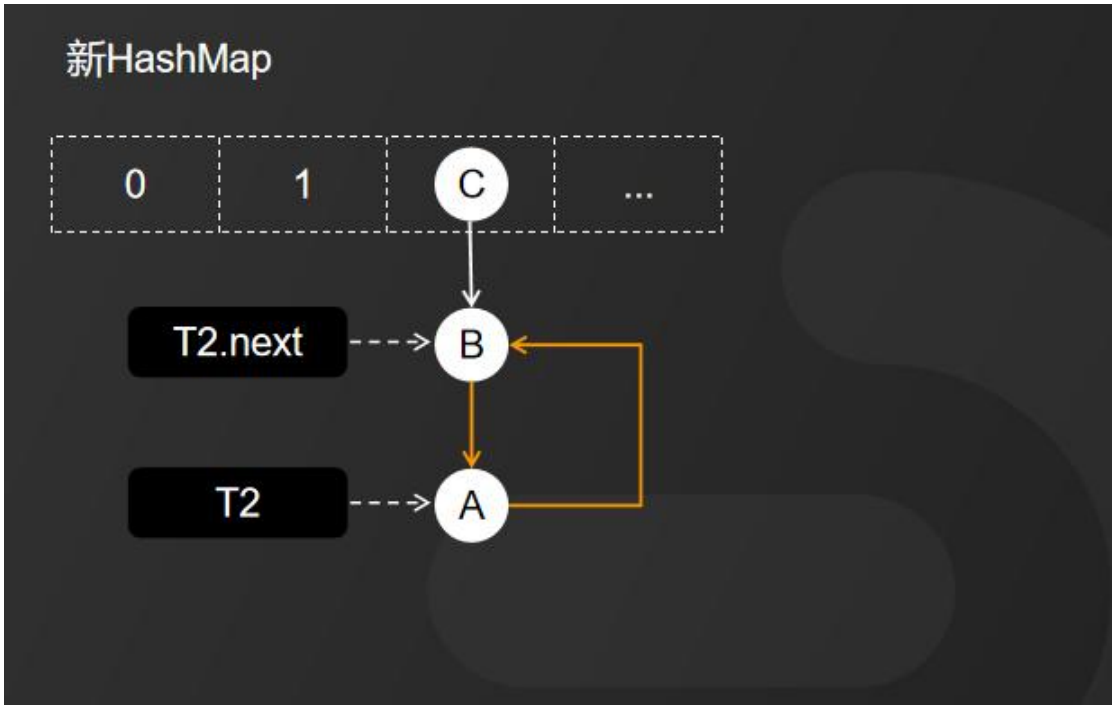
T1 完成扩容之后的场景就变成动画所示的这样。



因为 HashMap 扩容采用的是头插法，线程 T1 执行之后，链表中的节点顺序发生了改变。但线程 T2 对于发生的一切还是不可知的，所以它指向的节点引用依然没变。如图所示，T2 指向的是 A 节点，T2.next 指向的是 B 节点。



当线程 T1 执行完成之后，线程 T2 恢复执行时，死循环就发生了。



因为 T1 执行完扩容之后，B 节点的下一个节点是 A，而 T2 线程指向的首节点是 A，第二个节点是 B，这个顺序刚好和 T1 扩容之前的节点顺序是相反的。T1 执行完之后的顺序是 B 到 A，而 T2 的顺序是 A 到 B，这样 A 节点和 B 节点就形成了死循环。

53.287.3 解决方案

避免 HashMap 发生死循环的常用解决方案有三个：　1)、使用线程安全的 ConcurrentHashMap 替代 HashMap，个人推荐使用此方案。

2)、使用线程安全的容器 Hashtable 替代，但它性能较低，不建议使用。

3)、使用 synchronized 或 Lock 加锁之后，再进行操作，相当于多线程排队执行，也会影响性能，不建议使用。

53.287.4 总结

HashMap 死循环只发生在 JDK1.7 版本中，主要原因是 JDK1.7 中的 HashMap，在头插法 加 链表 加 多线程并发 加 扩容这几个情形累加到一起就会形成死循环。多线程环境下建议采用 ConcurrentHashMap 替代。在 JDK1.8 中，HashMap 改成了尾插法，解决了链表死循环的问题。

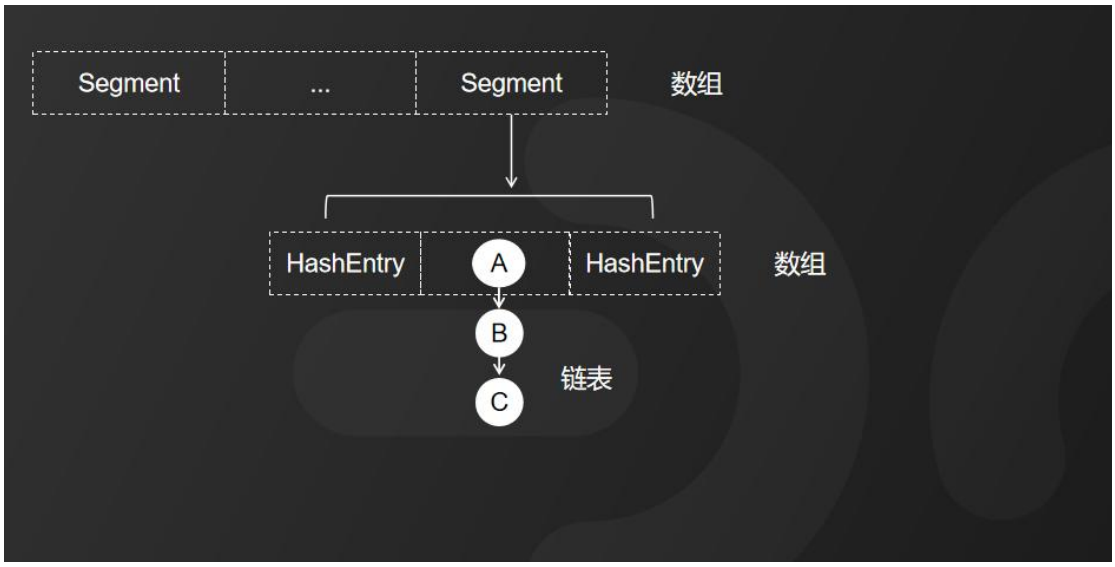
以上就是关于 HashMap 死循环原因的分析，听懂的小伙伴，关注点个赞，下次不迷路。我是被编程耽误的文艺 Tom，如果还有其他疑问或者需要高清无码图的小伙伴，请在评论区留言。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！3 人未读

53.288 ConcurrentHashMap 是如何保证线程安全的?

ConcurrentHashMap 相当于是 HashMap 的多线程版本，它的功能本质上和 HashMap 没什么区别。因为 HashMap 在并发操作的时候会出现各种问题，比如死循环问题、数据覆盖等问题。而这些问题，只要使用 ConcurrentHashMap 就可以完美地解决。那问题来到了，ConcurrentHashMap 它是如何保证线程安全的呢?

53.288.1 JDK1.7 实现原理

首先，我们来看 JDK 1.7 中 ConcurrentHashMap 的底层结构，它基本延续了 HashMap 的设计，采用的是数组 加 链表的形式。和 HashMap 不同的是，ConcurrentHashMap 中的数组设计　分为大数组 Segment 和小数组 HashEntry，来着这张图。

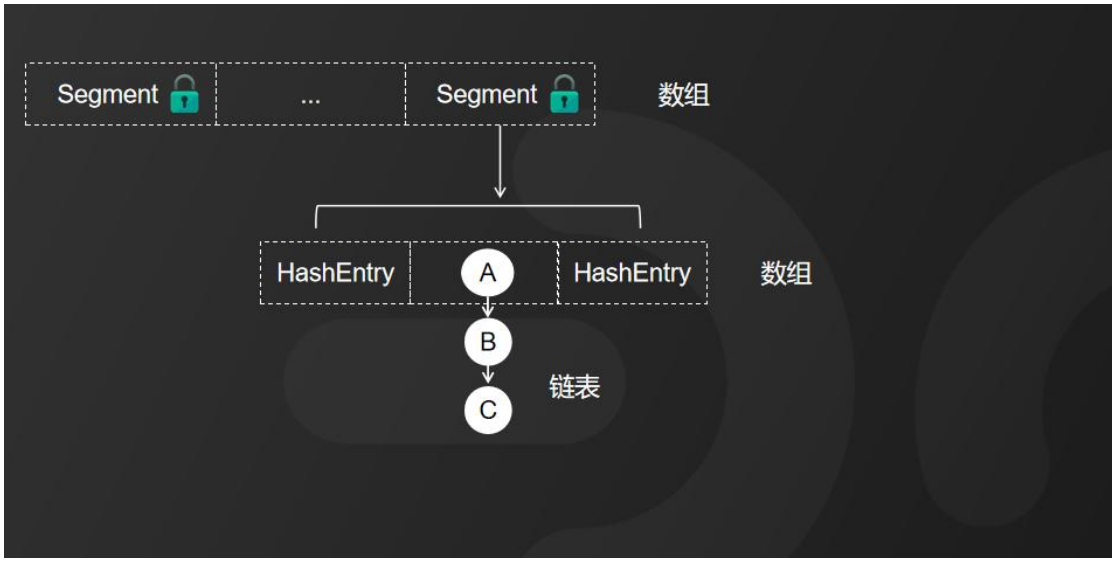


大树组 Segment 可以理解为一个数据库，而每个数据库（Segment）中又有很多张表（HashEntry），每个 HashEntry 中又有很多条数据，这些数据是用链表连接的。了解了 ConcurrentHashMap 的基本结构设计，我们再来看它的线程安全实现，就比较简单了。

接下来我们来对照 JDK1.7 中 ConcurrentHashMap 的 put()方法源码实现。

```
final V put(K key, int hash, V value, boolean onlyIfAbsent) {
    HashEntry<K,V> node = tryLock() ? null : scanAndLockForPut(key, hash, value);
    V oldValue;
    try {
        HashEntry<K,V>[] tab = table;
        int index = (tab.length - 1) & hash;
        HashEntry<K,V> first = entryAt(tab, index);
        for (HashEntry<K,V> e = first;;) {
            if (e != null) {
                K k;
                ...
            }
            else {
                ...
            }
        }
    } finally {
        unlock();
    }
    return oldValue;
}
```

因为 Segment 本身是基于 ReentrantLock 重入锁实现的加锁和释放锁的操作，这样就能保证多个线程同时访问 ConcurrentHashMap 时，同一时间只能有一个线程能够操作相应的节点，这样就保证了 ConcurrentHashMap 的线程安全。

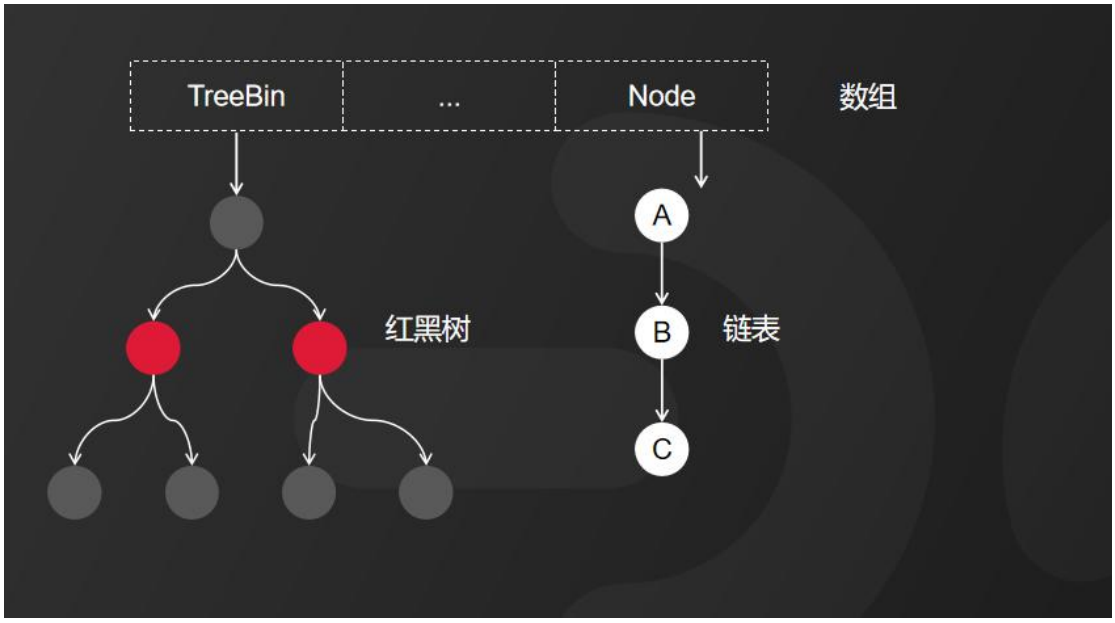


也就是说ConcurrentHashMap的线程安全是建立在Segment加锁的基础上的，所以，我们称它为分段锁或者片段锁，如图中所示。

那 JDK1.8 又是如何实现的呢？

53.288.2 JDK1.8 优化内容

在 JDK1.7 中，ConcurrentHashMap 虽然是线程安全的，但因为它的底层实现是数组加链表的形式，所以在数据比较多情况下，因为要遍历整个链表，会降低访问性能。所以，JDK1.8 以后采用了数组 加 链表 加 红黑树的方式优化了 ConcurrentHashMap 的实现，具体实现如图所示。



当链表长度大于 8，并且数组长度大于 64 时，链表就会升级为红黑树的结构。JDK 1.8 中的 ConcurrentHashMap 虽然保留了 Segment 的定义，但这，仅仅是为了保证序列化时的兼容性，不再有任何结构上的用处了。

那在 JDK 1.8 中 ConcurrentHashMap 的源码是如何实现的呢？它主要是使用了 CAS 加 volatile 或者 synchronized 的方式来保证线程安全。

```
final V putVal(K key, V value, boolean onlyIfAbsent) {
    if (key == null || value == null) throw new NullPointerException();
    int hash = spread(key.hashCode());
    int binCount = 0;
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
            if (casTabAt(tab, i, null, new Node<K,V>(hash, key, value, null)))
                break;
        }
        else if ((fh = f.hash) == MOVED)
            tab = helpTransfer(tab, f);
        else {
            V oldVal = null;
            synchronized (f) {
                ...
                }
            }
            if (binCount != 0) {
                if (binCount >= TREEIFY_THRESHOLD)
                    treeifyBin(tab, i);
                if (oldVal != null)
                    return oldVal;
                break;
            }
        }
    }
    addCount(1L, binCount);
    return null;
}
```

我们可以从源码片段中看到，添加元素时首先会判断容器是否为空，

如果为空则使用 volatile 加 CAS 来初始化，

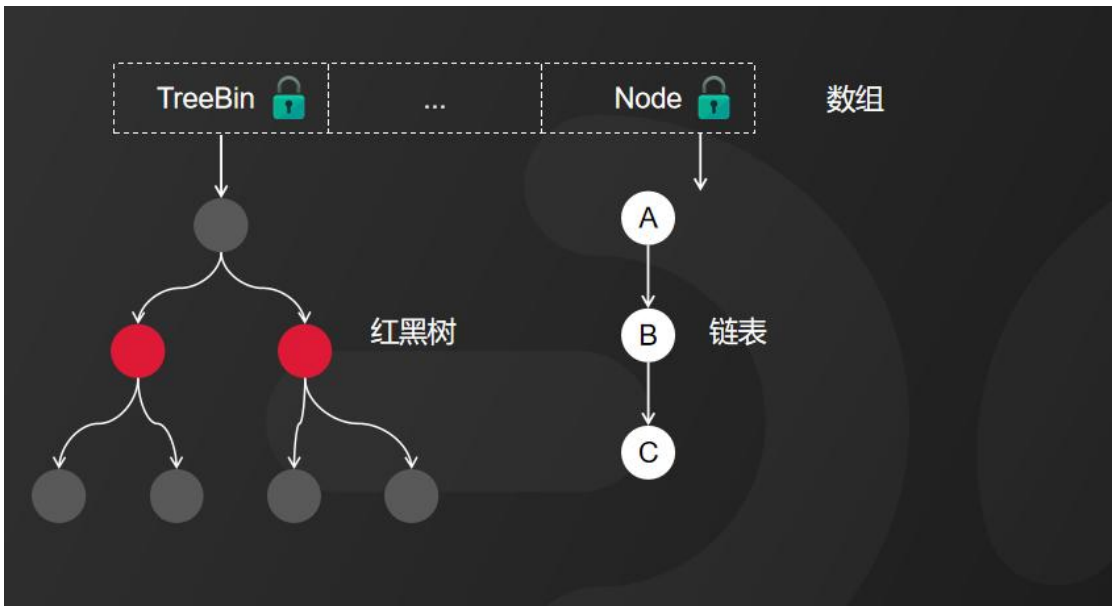
如果容器不为空，则根据存储的元素计算该位置是否为空。

如果根据存储的元素计算结果为空则利用 CAS 设置该节点；

如果根据存储的元素计算为空不为空，则使用 synchronized ，然后，遍历桶中的数据，并替换或新增节点到桶中，

最后再判断是否需要转为红黑树。这样就能保证并发访问时的线程安全了。

如果把上面的执行用一句话归纳的话，就相当于是 ConcurrentHashMap 通过对头结点加锁来保证线程安全的。



这样设计的好处是，使得锁的粒度相比 Segment 来说更小了，发生 hash 冲突 和 加锁的频率也降低了，在并发场景下的操作性能也提高了。而且，当数据量比较大的时候，查询性能也得到了很大的提升。

53.288.3 总结

最后，我们来总结一下：

1、ConcurrentHashMap 在 JDK 1.7 中使用的数组 加 链表的结构，其中数组分为两类，大树组 Segment 和 小数组 HashEntry，而加锁是通过给 Segment 添加 ReentrantLock 重入锁来保证线程安全的。

2、ConcurrentHashMap 在 JDK1.8 中使用的是数组 加 链表 加 红黑树的方式实现，它是通过 CAS 或者 synchronized 来保证线程安全的，并且缩小了锁的粒度，查询性能也更高。

ConcurrentHashMap 中有很多设计思想是值得我们去学习和借鉴的，比如说锁的粒度控制、分段锁的设计等等，都可以应用在实际的业务开发场景中。我们通过学习这些底层原理从中获取很多的设计思路，帮助我们更高效地去解决实际问题。

好了，关于 ConcurrentHashMap 的分享就讲到这里，听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果还有其他疑问或者需要 的小伙伴，请在评论区留言。

如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.289 为什么 ConcurrentHashMap 不允许插入 null 值?

在 Java 语言中，给 ConcurrentHashMap 和 Hashtable 这些线程安全的集合中的 Key 或者 Value 插入 null（空） 值的会报空指针异常，但是单线程操作的 HashMap 又允许 Key 或者 Value 插入 null（空） 值。这到底是为什么呢?

53.289.1 探寻源码

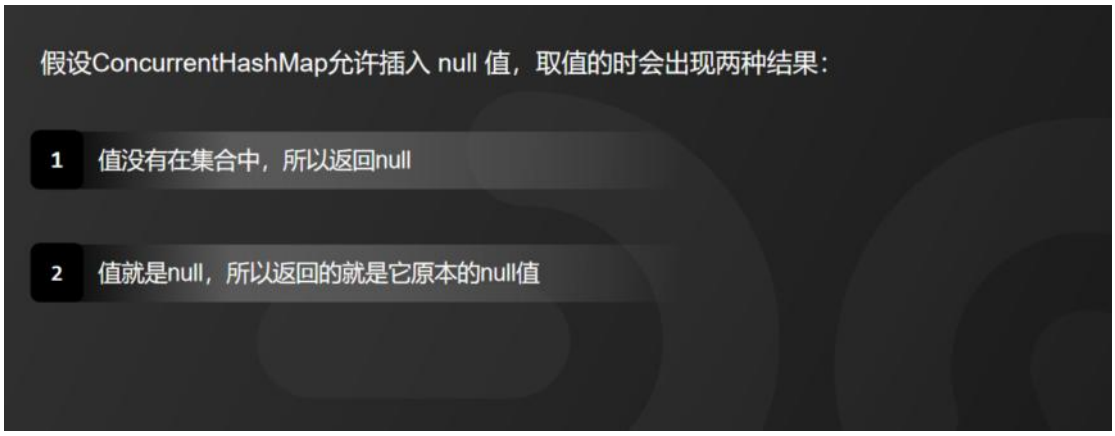
为了找到原因，我们先来看这样一段源码片段，打开 ConcurrentHashMap 的 putVal() 方法，源码中第一句就非常明确地做了判断，如果 Key 或者 Value 为 null（空） 值，就直接抛出空指针异常。

我们在源码中似乎已经找到了原因，你可以这样回答面试官，说 JDK 源码就是这么规定的。然而，这个原因是不能说服面试官的，虽然，源码是这样设计的，我们要思考的是，这样设计背后更深层次的原因。

那到底为什么 ConcurrentHashMap 不允许插入 null （空）值，HashMap 又允许插入呢?

53.289.2 歧义问题

因为给 ConcurrentHashMap 中插入 null （空）值会存在歧义。我们可以假设 ConcurrentHashMap 允许插入 null（空） 值，那么，我们取值的时候会出现两种结果：



1、值没有在集合中，所以返回的结果就是 null （空）；

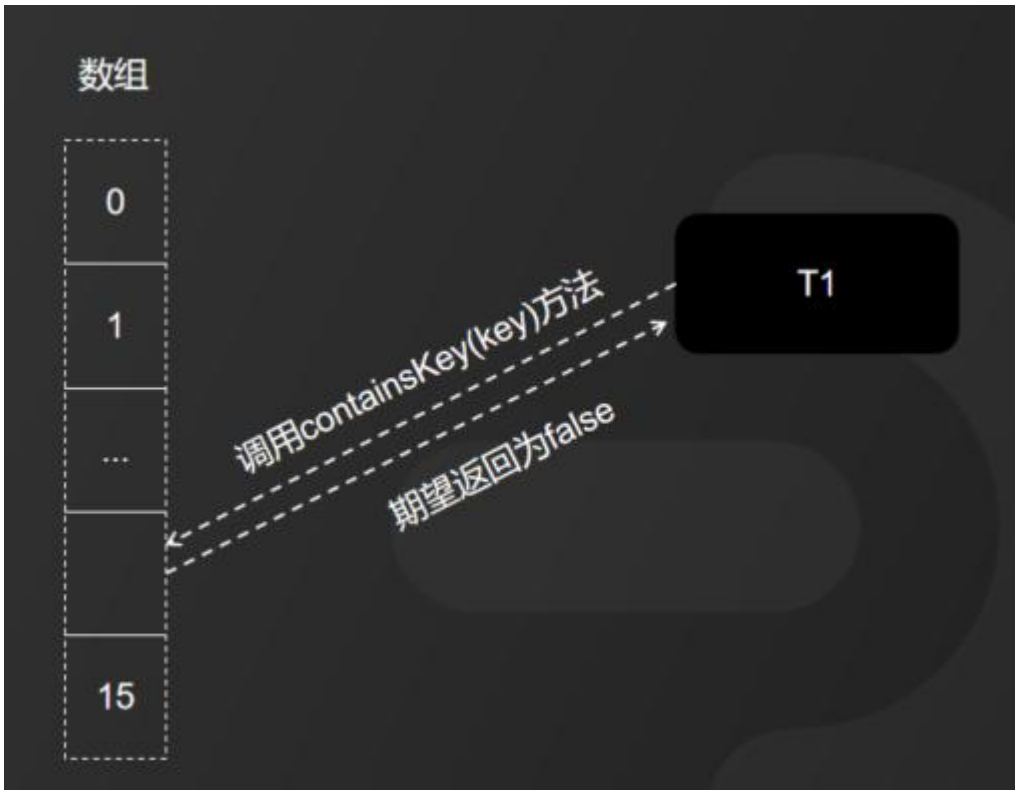
2、值就是 null（空），所以返回的结果就是它原本的 null（空） 值。

这就产生了歧义问题。

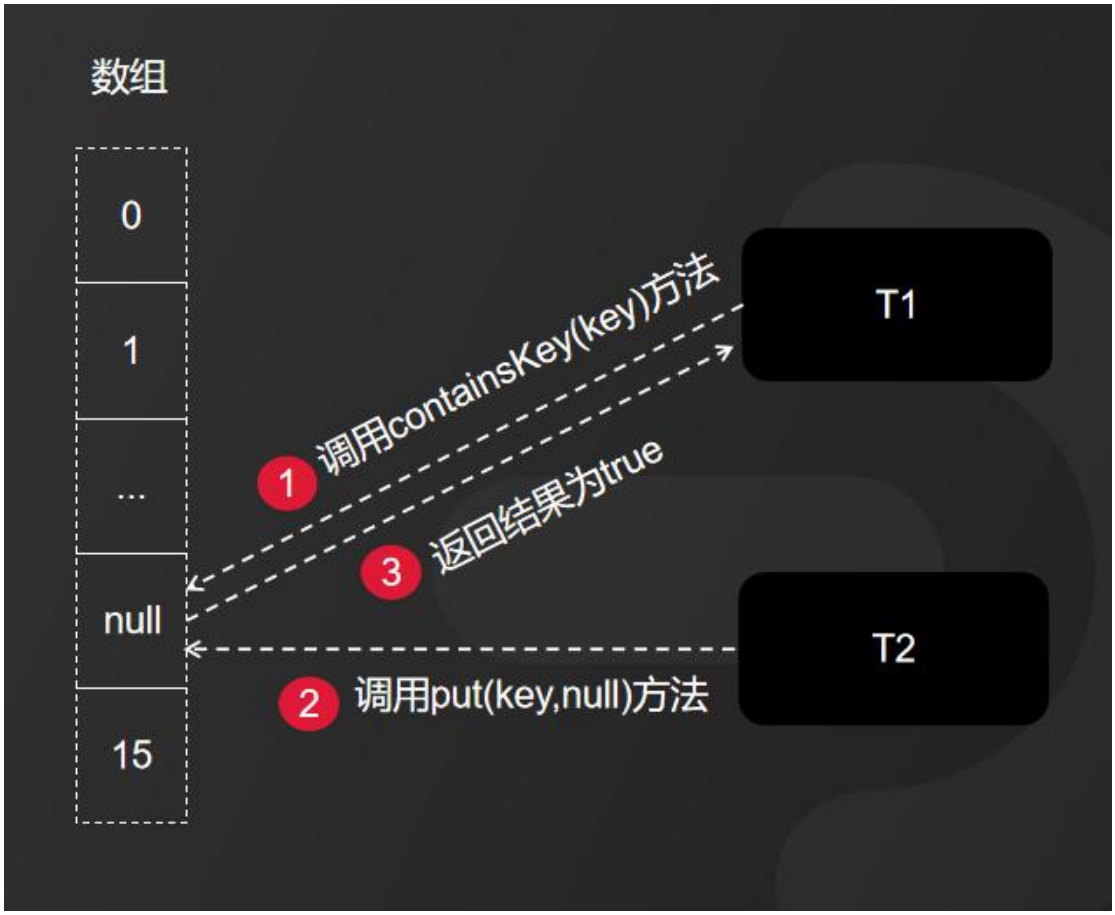
那 HashMap 允许插入 null（空） 值，难道它就不担心出现歧义吗？这是因为 HashMap 的设计是给单线程使用的，所以如果取到 null（空） 值，我们可以通过 HashMap 的 containsKey(key)方 法来区分这个 null（空） 值到底是插入值是 null（空），还是本就没有才返回的 null（空） 值。

而 ConcurrentHashMap 就不一样了，因为 ConcurrentHashMap 是在多线程场景下使用的，它的情况更加复杂。

举个例子，现在有线程 T1 调用了 ConcurrentHashMap 的 containsKey(key) 方法，我们期望返回的结果是 false，也就是说，T1 并没有往 ConcurrentHashMap 中 put null（空）值。



但是，恰恰出了个意外，在线程 T1 还没有得到返回结果之前，线程 T2 又调用了 ConcurrentHashMap 的 put() 方法，插入了一个 Key，并且存入的 Value 是 null（空） 值。那么，线程 T1 最终得到的返回结果就变成 true 了。



显然，这个结果和我们之前期望的 false 完全不一致。

也就是说，在多线程的复杂情况下，我们多线程的复杂情况下，到底是插入的 null（空）值，还是本就没有才返回的 null（空） 值。也就是说，产生的歧义不能被 证 伪，

53.289.3 作者回复

对于 ConcurrentHashMap 不允许插入 null 值的问题，有人问过 ConcurrentHashMap 的作者 Doug Lea，以下是他回复的邮件内容：

> The main reason that nulls aren't allowed in ConcurrentMaps (ConcurrentHashMaps, ConcurrentSkipListMaps) is that ambiguities that may be just barely tolerable in non-concurrent maps can't be accommodated. The main one is that if map.get(key) returns null, you can't detect whether the key explicitly maps to null vs the key isn't mapped.In a non-concurrent map, you can check this via map.contains(key),but in a

> concurrent one, the map might have changed between calls.
>
> Further digressing: I personally think that allowingnulls in Maps (also Sets) is an open invitation for programsto contain errors that remain undetected untilthey break at just the wrong time. (Whether to allow nulls evenin non-concurrent Maps/Sets is one of the few design issues surroundingCollections that Josh Bloch and I have long disagreed about.) It is very difficult to check for null keys and valuesin my entire application . Would it be easier to declare somewherestatic final Object NULL = new Object();and replace all use of nulls in uses of maps with NULL?
> -Doug

以上信件的主要意思是，Doug Lea 认为这样设计最主要的原因是：不容忍在并发场景下出现歧义！

53.289.4 总结

ConcurrentHashMap 在源码中加入不允许插入 null （空） 值的设计，主要目的是为了防止并发场景下的歧义问题。

以上就是我对关于 ConcurrentHashMap 为什么不允许插入 null （空） 值的解答，听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果还有其他疑问或者需要高清无码图的小伙伴，可以关注我的主页加 V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

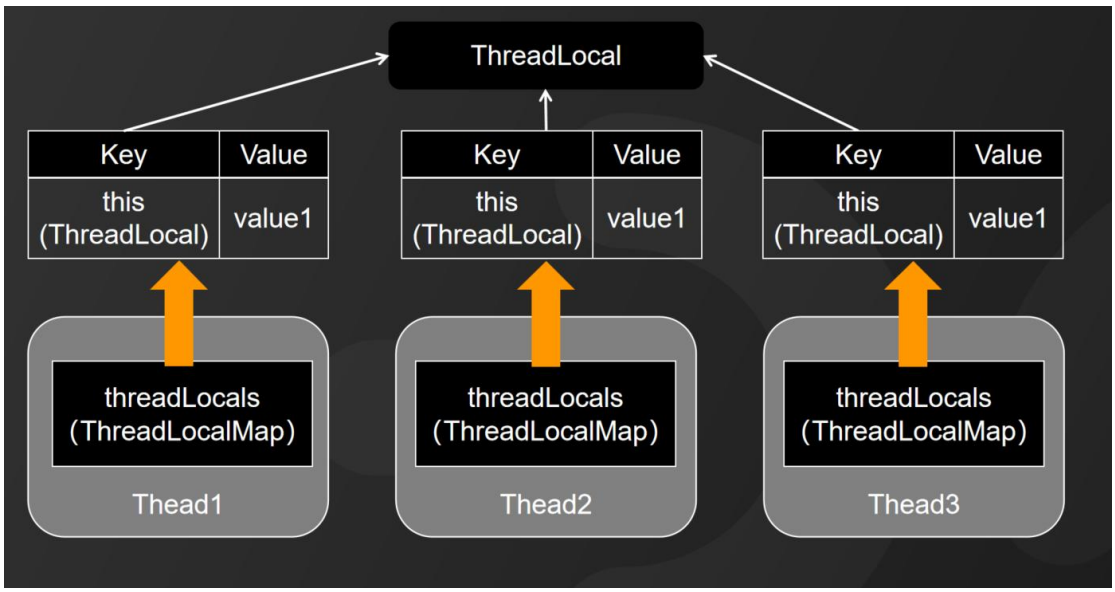
53.290 ThreadLocal 真的会造成内存泄漏吗?

ThreadLoca 在并发场景中，应用非常多。前几天有位小伙伴问我一个问题，说 ThreadLocal 是不是真的会造成内存泄漏？今天给大家做一个分享，个人见解，仅供参考。如果大家有其他见解可以在评论区讨论。

53.290.1 ThreadLocal 的基本原理

考虑到很多小伙伴可能还不太了解 ThreadLocal，我先简单介绍一下 ThreadLocal。在多线程并发访问同一个共享变量的情况下，如果不做同步控制的话，就可能会导致数据不一致的问题，所以，我们需要使用 synchronized 加锁来解决。

而 ThreadLocal 换了一个思路来处理多线程的情况，



ThreadLocal 本身并不存储数据，它使用了线程中的 threadLocals 属性，threadLocals 的类型就是在 ThreadLocal 中的定义的 ThreadLocalMap 对象，当调用 ThreadLocal 的 set(T value)方法时，ThreadLocal 将自身的引用也就是 this 作为 Key，然后，把用户传入的值作为 Value 存储到线程的 ThreadLocalMap 中，这就相当于每个线程的读写操作都是基于线程自身的一个私有副本，线程之间的数据是相互隔离的，互不影响。这样一来基于 ThreadLocal 的操作也就不存在线程安全问题了。它相当于采用了用空间来换时间的思路，从而提高程序的执行效率。

2、四种对象引用

在 ThreadLocalMap 内部，维护了一个 Entry 数组 table 的属性，用来存储键值对的映射关系，来看这样一段代码片段：

```
static class ThreadLocalMap {
    ...
    private Entry[] table;

    static class Entry implements WeakReference<ThreadLocal<?>> {
        Object value;
        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
    ...
}
```

```
static class ThreadLocalMap {

   ...

   private Entry[] table;


   static class Entry implements WeakReference<ThreadLocal<?>> {

   Object value;

        Entry(ThreadLocal<?> k, Object v) {

            super(k);

            value = v;

        }

   }

   ...

}
```

Entry 将 ThreadLocal 作为 Key，值作为 Value 保存，它继承自 WeakReference，注意构造函数里的第一行代码 super(k)，这意味着 ThreadLocal 对象是一个「弱引用」。有的小伙伴可能对「弱引用」不太熟悉，这里再介绍一下 Java 的四种引用关系。

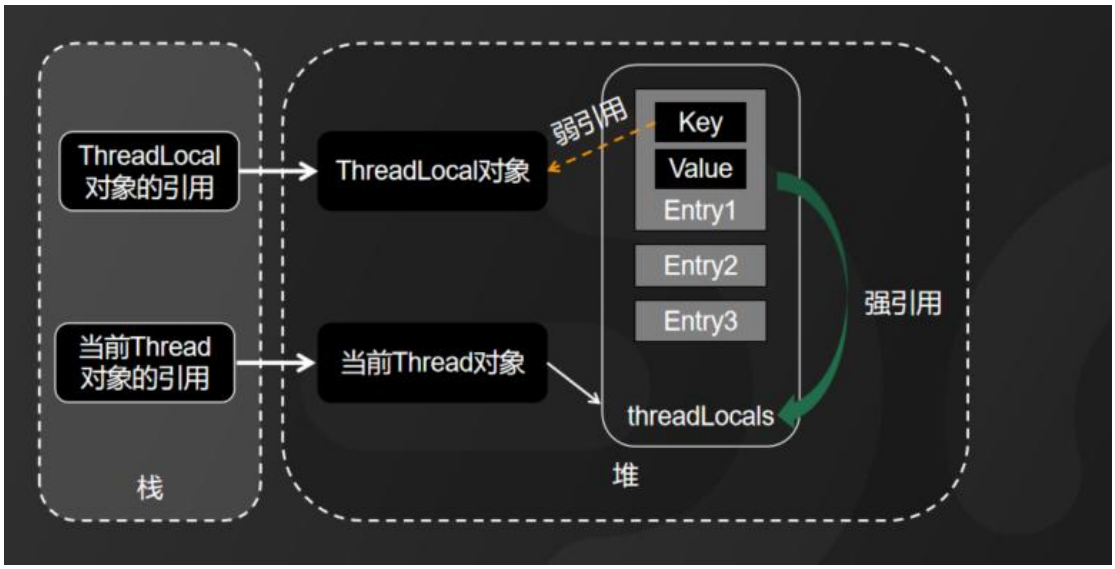
在 JDK1.2 之后，Java 对引用的概念做了一些扩充，将引用分为“强”、“软”、“弱”、“虚”四种，由强到弱依次为：

强引用：指代码中普遍存在的赋值行为，如：Object o = new Object()，只要强引用关系还在，对象就永远不会被回收。 **软引用**：还有用处，但不是必须存活的对象，JVM 会在内存溢出前对其进行回收，例如：缓存。 **弱引用**：非必须存活的对象，引用关系比软引用还弱，不管内存是否够用，下次 GC 一定回收。 **虚引用**：也称“幽灵引用”、“幻影引用”，最弱的引用关系，完全不影响对象的回收，等同于没有引用，虚引用的唯一的目的是对象被回收时会收到一个系统通知。

这个描述还是比较官方的，简单总结一下，大家应该都追过剧，强引用就好比是男主角，怎么都死不了。软引用就像女主角，虽有一段经历，还是没走到最后。弱引用就是男二号，注定用来牺牲的。虚引用就是路人甲了。

53.290.2 造成内存泄漏的原因

内存泄漏和 ThreadLocalMap 中定义的 Entry 类有非常大的关系。



这个动画完整地展示了 ThreadLocal 中对象引用的关系，需要这张高清图的小伙伴可以在评论区留言。

由于 ThreadLocal 对象是弱引用，如果外部没有强引用指向它，它就会被 GC 回收，导致 Entry 的 Key 为空（null），如果这时 Value 外部也没有强引用指向它，那么 Value 就永远也访问不到了，按理也应该被 GC 回收，但是由于 Entry 对象还在强引用 Value，导致 Value 无法被回收，这时「内存泄漏」就发生了，Value 成了一个永远也无法被访问，但是又无法被回收的对象。

Entry 对象属于 ThreadLocalMap，ThreadLocalMap 又属于 Thread，如果线程本身的生命周期很短，短时间内就会被销毁，那么「内存泄漏」立刻就会得到解决，只要线程被销毁，Value 也会随之被回收。

问题是，线程本身是非常珍贵的计算机资源，很少会去频繁的创建和销毁，一般都是通过线程池来使用，这就将线程的生命周期大大拉长，「内存泄漏」的影响也会越来越大。

造成内存泄漏的原因总结：

threadLocals对象中的Entry对象不再使用后，如果没有及时清除Entry对象，而程序自身也无法通过垃圾回收机制自动清除，就可能导致内存泄漏。

threadLocals 对象中的 Entry 对象不再使用后，如果没有及时清除 Entry 对象 ，而程序自身也无法通过垃圾回收机制自动清除，就可能导致内存泄漏。

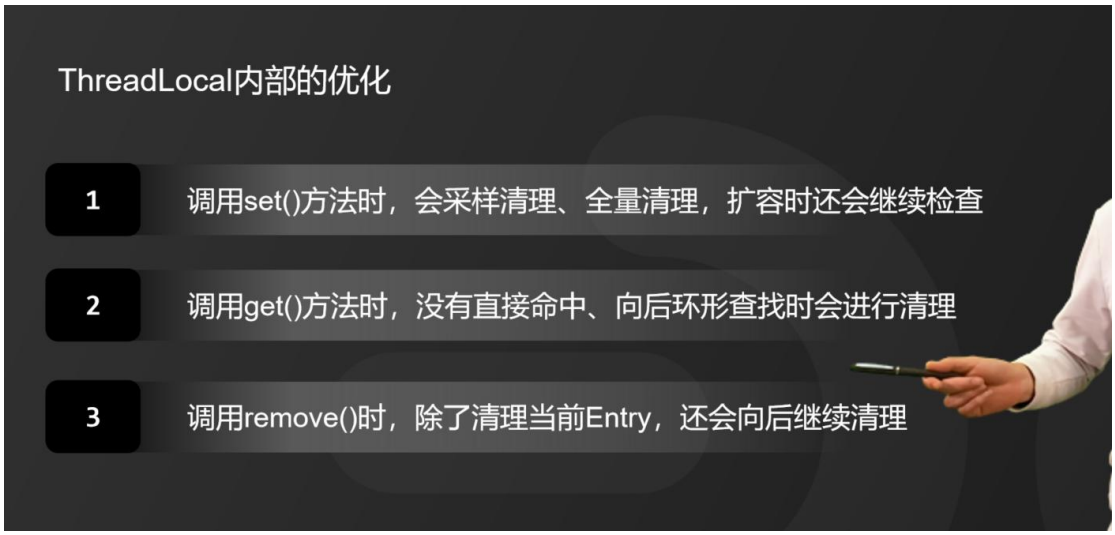
53.290.3 如何避免内存泄漏?

不要听到「内存泄漏」就不敢使用 ThreadLocal,只要规范化使用是不会有问题的。我给大家支几个招：

1、每次使用完 ThreadLocal 都记得调用 remove()方法清除数据。

2、将 ThreadLocal 变量尽可能地定义成 static final，避免频繁创建 ThreadLocal 实例。这样也就保证程序一直存在 ThreadLocal 的强引用，也能保证任何时候都能通过 ThreadLocal 的弱引用访问到 Entry 的 Value 值，进而清除掉。

当然，就是使用不规范，ThreadLocal 内部也做了一些优化，比如：



1、调用 set()方法时，ThreadLocal 会进行采样清理、全量清理，扩容时还会继续检查。

2、调用 get()方法时，如果没有直接命中或者向后环形查找时也会进行清理。

3、调用 remove()时，除了清理当前 Entry，还会向后继续清理。

所以，我真的希望小伙伴们，不要被制造焦虑的面试官们卷得太深。听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺 Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加 V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.291 Spring Boot 自动装配原理

昨天，有位在广州工作 4 年的小伙伴，在面试中被问到 SpringBoot 自动装配原理，当时，自我感觉比较好，他要的是 30K，但是都没有拿到 Offer。今天，我给大家分享一下我的理解。

53.291.1 Spring Boot 自动装配是什么?

SpringBoot 自动装配主要是基于注解编程 和 约定优于配置的思想来设计的。

自动装配就是由 Spring 自动把其他组件中的 Bean 装载到 IoC 容器中，不需要开发人员再去配置文件中添加大量的配置。我们只需要在 Spring Boot 的启动类上添加 @SpringBootApplication 注解，开启自动装配。

这种自动装配的思想，在 Spring 3.x 以后就开始支持，我们只要在类上添加 @Enable 注解就可以了，只是没有像 Spring Boot 这样全面地去设计。

因此，Spring 和 Spring Boot 的最大区别就是在于 Spring Boot 的自动装配。那自动装配的原理又是什么呢?

53.291.2 自动装配原理

@SpringBootApplication 这个注解是暴露给用户使用的入口，它的底层是由 @EnableAutoConfiguration 这个注解来实现的。

这样一个自动装配的实现，我把它归纳为以下三个核心步骤：

第一步：启动依赖组件的时候，组件中必须要包含 @Configuration 的配置类，在这个配置类里面声明为 @Bean 注解，就将方法的返回值或者属性值 注入到 IoC 容器中。

第二步：如果是使用第三方 jar 包，Spring Boot 采用 SPI 机制，只需要在/META-INF/目录下增加 spring.factories 配置文件。然后，Spring Boot 会根据约定规则，自动使用 SpringFactoriesLoader 来加载配置文件中的内容。

第三步：Spring 获取到第三方 jar 中的配置以后，会使用调用 ImportSelector 接口来完成动态加载。

这样的设计的好处在于，大幅减少了臃肿的配置文件，而且各模块之间的依赖实现了深度解耦。比如，我们使用 Spring 创建 Web 程序时需要导入非常多的 Maven 依赖，而 Spring Boot 只需要一个 Maven 依赖来创建 Web 程序，并且 Spring Boot 还把我们最常用的依赖都放到了一起，我们只需要 引入 spring-boot-starter-web 这一个依赖就可以完成一个简单的 Web 应用。

以前用 Spring 的时候需要 XML 文件配置开启一些功能，现在 Spring Boot 不用 XML 配置了，只需要写一个， 加了@Configuration 注解 或者实 现对应接口的配置类就可以了。

53.291.3 总结

最后，我总结一下，Spring Boot 自动装配是 Spring 的完善和扩展，就是为我们便捷开发，方便测试和部署，提高效率而诞生的框架技术。

小伙伴们，如果你被问到过 Spring Boot 自动装配原理的问题，你是怎么回答的呢？可以在评论区分享你的回答。

我是被编程耽误的文艺 Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加 V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

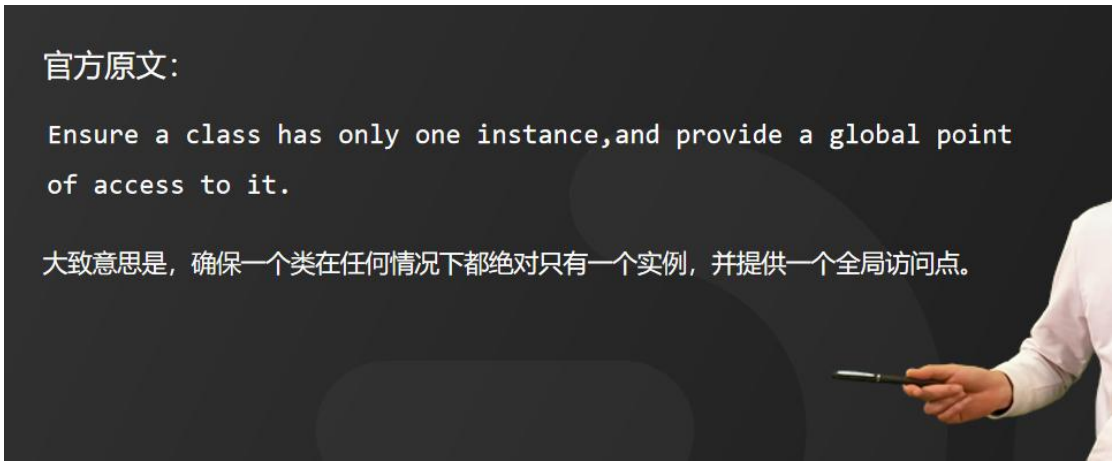
53.292 哪些情况下的单例对象可能会破坏?

昨天，有位小伙伴在评论区留言，希望我分享一些设计模式相关的面试题。设计模式本身是很抽象的，但是在很多面试中又经常被问到，很多小伙伴其实都能答得上，但是又不知道怎么样回答才能让面试官满意，往往越简单的知识越能够体现出核心竞争力。

今天，我给大家分享一个简单又不简单的单例模式，希望能够帮助到大家。先来看单例模式的定义。

53.292.1 单例模式的定义

关于单例模式的定义，官方原文是这样描述的：



Ensure a class has only one instance,and provide a global point of access to it.

大致意思是，确保一个类在任何情况下都绝对只有一个实例，并提供一个全局访问点。

单例模式的写法相信只要是程序员应该都会，也很非常简单，这里我就不一一列举了。

今天，我要重点要给大家分析的是，在 Java 中，哪些单例对象是最有可能被破坏的。

53.292.2 单例被破坏的五个场景

我把可能出现单例被破坏的情况，一共归纳为五种，分别为多线程破坏单例、指令重排破坏单例、克隆破坏单例、反序列化破坏单例、反射破坏单例。

下面我详细分析一下每种情况并给出解决方案：

第一种：多线程破坏单例

在多线程环境下，线程的时间片是由 CPU 自由分配的，具有随机性，而单例对象作为共享资源可能会同时被多个线程同时操作，从而导致同时创建多个对象。当然，这种情况只出现在懒汉式单例中。如果是饿汉式单例，在线程启动前就被初始化了，不存在线程再创建对象的情况。

如果懒汉式单例出现多线程破坏的情况，我给出以下两种解决方案：

1、改为 DCL 双重检查锁的写法。

2、使用静态内部类的写法，性能更高。

第二种：指令重排破坏单例

指令重排也可能导致懒汉式单例被破坏。来看这样一句代码：

instance = new Singleton();

看似简单的一段赋值语句：instance = new Singleton();

其实 JVM 内部已经被转换为多条执行指令：

```
memory = allocate();   分配对象的内存空间指令
ctorInstance(memory);   初始化对象
instance = memory;     将已分配存地址赋值给对象引用
```

1、分配对象的内存空间指令，调用 allocate()方法分配内存。

2、调用 ctorInstance()方法初始化对象

3、将已分配存地址赋值给对象引用

但是经过重排序后，执行顺序可能是这样的：

memory = allocate(); 分配对象的内存空间指令

instance = memory;   将已分配存地址赋值给对象引用

ctorInstance(memory); 初始化对象

1、分配对象的内存空间指令

2、设置 instance 指向刚分配的内存地址

3、初始化对象

我们可以看到指令重排之后，instance 指向分配好的内存放在了前面，而这段内存的初始化的指令被排在了后面，在线程 T1 初始化完成这段内存之前，线程 T2 虽然进不去同步代码块，但是在同步代码块之前的判断就会发现 instance 不为空，此时线程 T2 获得 instance 对象，如果直接使用就可能发生错误。

如果出现这种情况，我该如何解决呢？只需要在成员变量前加 volatile，保证所有线程的可见性就可以了。

private static volatile Singleton instance = null;

第三种：克隆破坏单例

在 Java 中，所有的类就继承自 Object，也就是说所有的类都实现了 clone()方法。如果是深 clone()，每次都会重新创建新的实例。那如果我们定义的是单例对象，岂不是也可调用 clone()方法来反复创建新的实例呢？确实，这种情况是有可能发生的。为了避免发生这样结果，我们可以在单例对象中重写 clone() 方法，将单例自身的引用作为返回值。这样，就能避免这种情况发生。

第四种：反序列化破坏单例

我们将 Java 对象序列化以后，对象通常会被持久化到磁盘或者数据库。如果我们要再次加载到内存，就需要将持久化的内容反序列化成 Java 对象。反序列化是基于字节码来操作的，我们要序列化以前的内容进行反序列化到内存，就需要重新分配内存，也就是说，要重新创建对象。那如果要反序列化的对象恰恰是单例对象，我们该怎么办呢？我告诉大家一种解决方案，在反序列的过程中，Java API 会调用 readResolve()方法，可以通过获取 readResolve()方法的返回值覆盖反序列化创建的对象。

（反序列化对象 指向 单例对象动画 出现 ）

因此，只需要重写 readResolve()方法，将返回值设置为已经存在的单例对象，就可以保证反序列化以后的对象是同一个了。之后再将反序列化后的对象中的值，克隆到单例对象中。

第五种：反射破坏单例

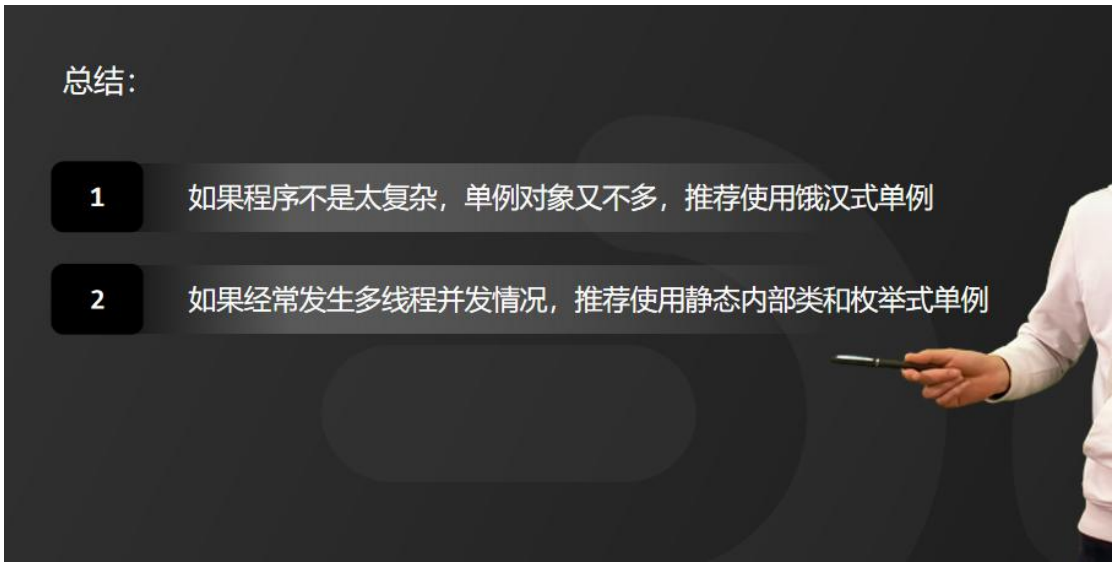
以上讲的所有单例情况都有可能被反射破坏。因为 Java 中的反射机制是可以拿到对象的私有的构造方法，也就是说，反射可以任意调用私有构造方法创建单例对象。当然，没有人会故意这样做，但是如果出现意外的情况，该如何处理呢？我推荐大家两种解决方案，

第一种方案是在所有的构造方法中第一行代码进行判断，检查单例对象是否已经被创建，如果已经被创建，则抛出异常。这样，构造方法将会被终止调用，也就无法创建新的实例。

第二种方案，将单例的实现方式改为枚举式单例，因为在 JDK 源码层面规定了，不允许反射访问枚举。

53.292.3 总结

最后总结一下：



1、在所有单例写法中，如果程序不是太复杂，单例对象又不多，推荐使用饿汉式单例。

2、但如果经常发生多线程并发情况下，推荐使用静态内部类和枚举式单例，我的《设计模式就该这样学》这本书中，也推荐这样的写法。

53.293 责任链模式的实现原理

责任链模式应用场景非常多、比如拦截器、过滤器等等。但是要彻底理解责任链的实现原理还是有一定难度的，因此，责任链模式的实现原理也就成为了一道互联网大厂的高频面试题。今天，我给小伙伴们来详细地掰一掰，保证让你彻底搞明白，不服来战。

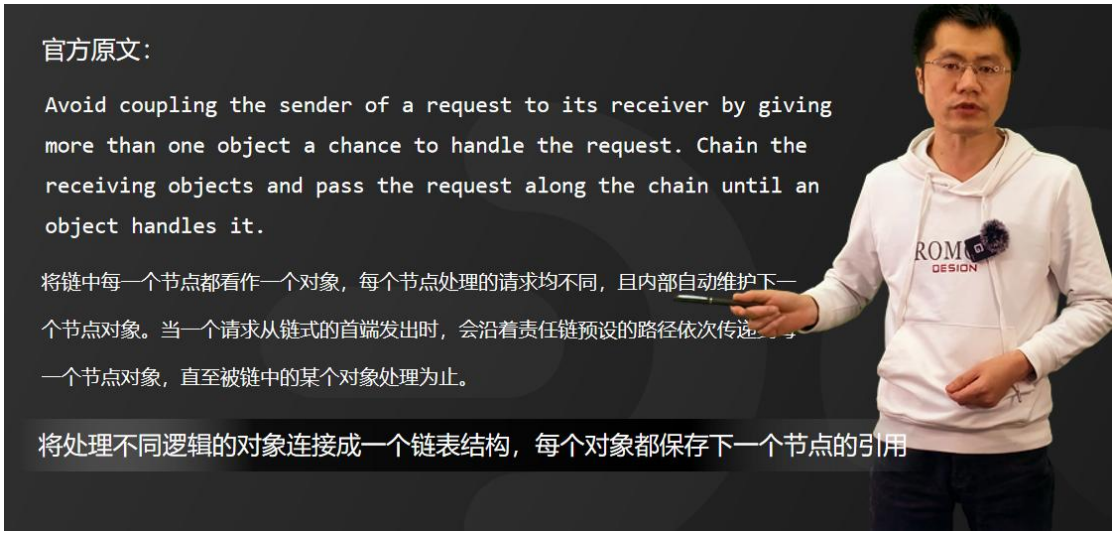
53.293.1 责任链模式的定义

关于责任链模式的定义，官方原文是这样描述的：

Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.

翻译过来就是：

将链中每一个节点都看作一个对象，每个节点处理的请求均不同，且内部自动维护下一个节点对象。当一个请求从 的首端发出时，会沿着责任链预设的路径依次传递到每一个节点对象，直至被链中的某个对象处理为止。



简答一句话总结就是：将处理不同逻辑的对象连接成一个链表结构，每个对象都保存下一个节点的引用。

我们在平时处理工作中的时候，往往需要各部门协同合作来完成某一个任务。因为，每个部门都有各自的职责，所以，很多时候事情完成一半，就要会转交到下一个部门，甚至还要盖章，直到所有部门都审批通过，事情才能完成。这就相当于将皮球层层往上踢。

还记得小时候追过的一个电视剧，80 后应该都看过，有位太极高手闯七层宝塔，每一层都住了一位武林高手，层层往上“过五关，斩六将”,打赢才能往上一层，打输了就踢回去，最后登顶.当时看得是热血沸腾，这也是责任链的一种场景。不过忘记电视剧叫什么名字了，小伙伴们可以评论区告诉我一下。

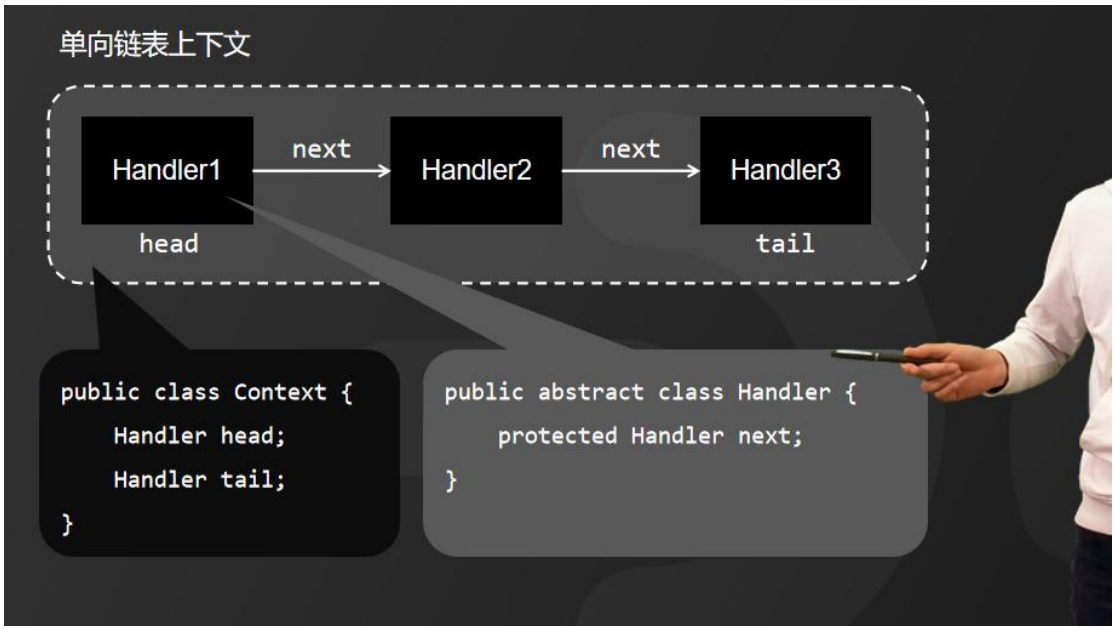
所以责任链模式又被戏称为“踢皮球”模式。那责任链模式的实现原理又是怎样的呢?

53.293.2 责任链模式的实现原理

责任链又分为单向责任链和双向责任链，单向责任链比较简单也容易理解，双向责任链相当于是一个执行闭环，较为复杂。

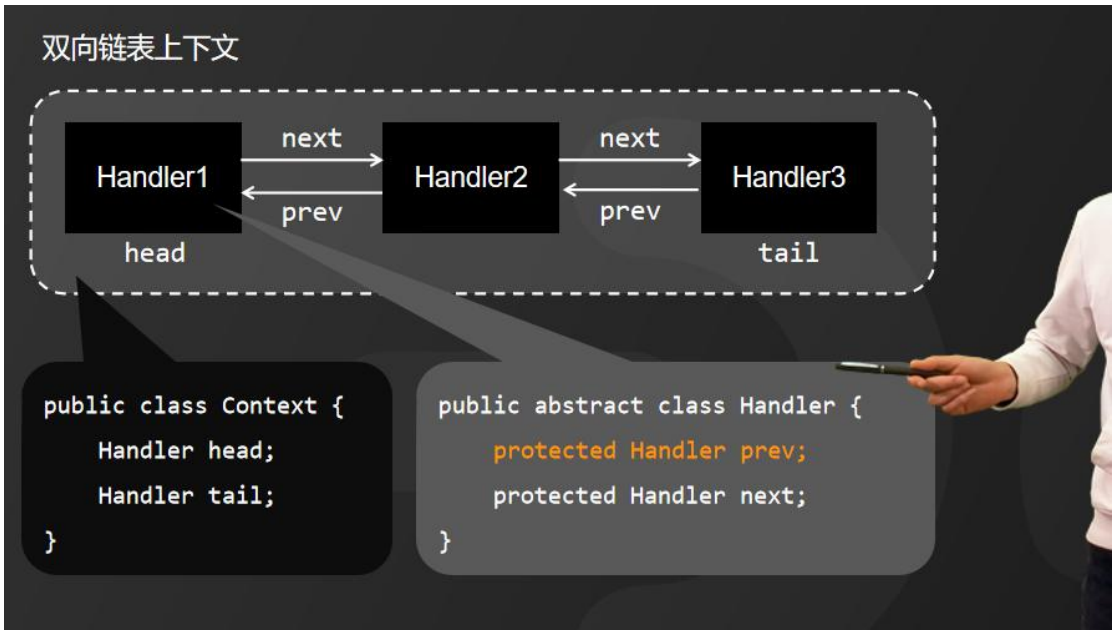
我们先来分析单向责任链。它的结构是这样设计的，首先，设计一个单向链表的上下文 Context，保存链表的头（head ）引用 和 尾（tail）引用，Context 的代码结构是这样的。

然后，在上下文中加入 Handler，也就是处理业务逻辑的节点类，每个 Hnandler 都保存了下一个执行节点的引用，形成一条完整的执行链路。Handler 的通用代码结构是这样的。



J2EE 中的 Filter 过滤器、Spring 中的 Interceptor 拦截器都是采用这样的单向链表的设计。

那双向链表又如何设计呢？它和单向链表的基本结构一致，我们来看，它只是在 Handler 中增加了对上一个节点的引用。这样责任链就行了一个执行闭环，就好比是环线地铁。来看它的通用代码结构是这样的。

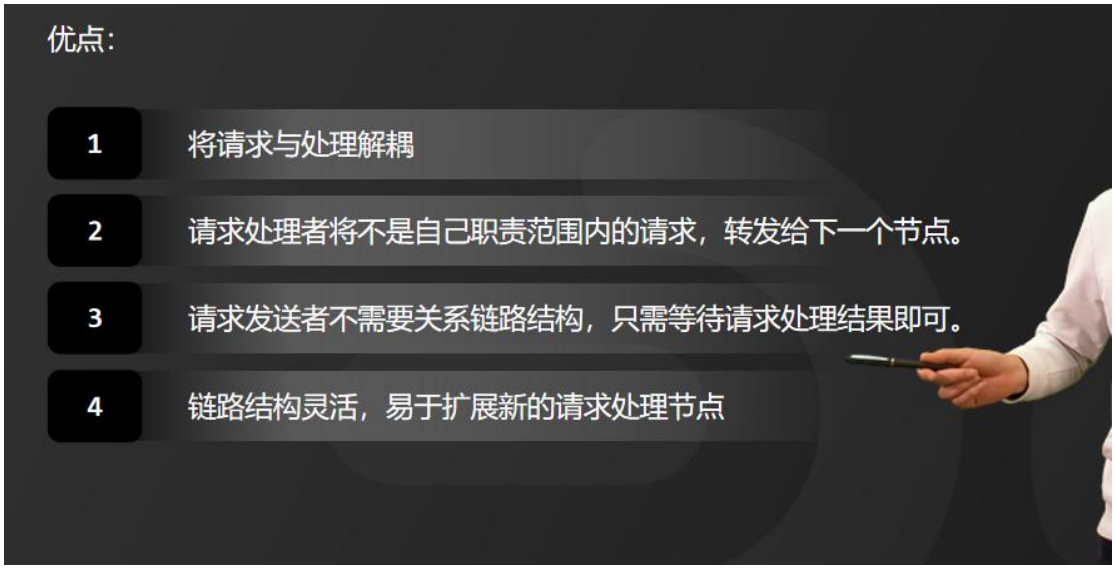


Netty 中的 Piepline 管道就是采用这样的双向链表的设计。

责任链模式一般还会结合建造者模式来使用，实现链式编程。

53.293.3 责任链模式的优缺点

那这样设计又有什么优点和缺点呢？先给大家总结一下它的优点：



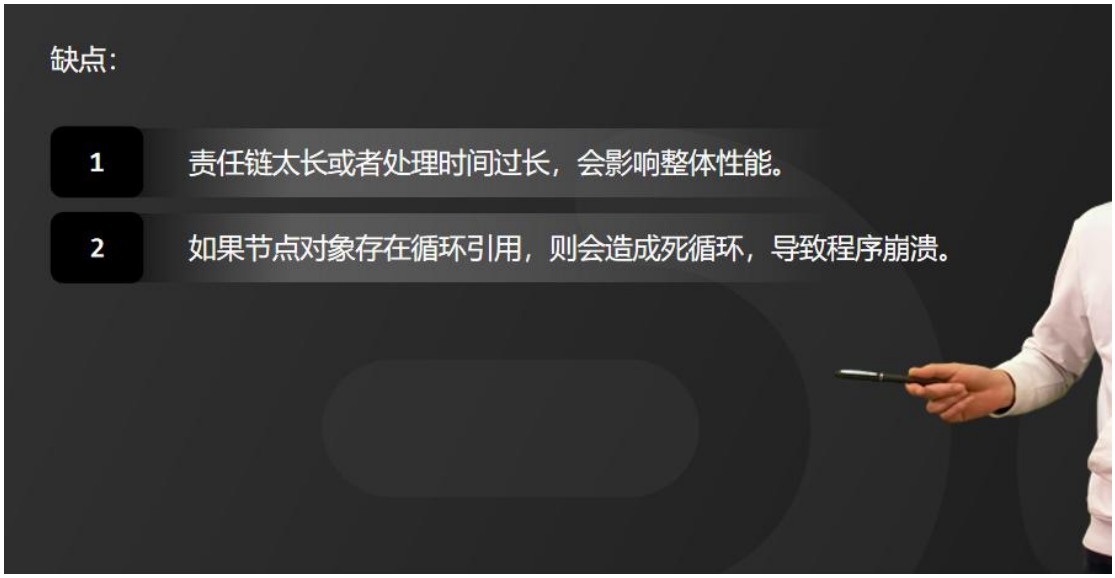
（1）实现了将请求与处理完全解耦。

（2）请求处理者只需关注自己职责范围内的请求进行处理即可，对于不是职责范围内的请求，直接转发给下一个节点对象。

（3）具备链式传递处理请求的功能，请求发送者不需要知晓链路结构，只需等待请求处理结果即可。

（4）链路结构灵活，可以通过改变链路结构动态地新增或删减责任。

那这样设计，会有哪些缺点呢？我也给大家总结一下：



（1）如果责任链路太长或者处理时间过长，会影响程序执行的整体性能。

（2）如果节点对象存在循环引用，有可能会造成死循环，从而导致程序崩溃。

小伙伴们，你还觉得责任链模式的实现原理难吗？听懂的小伙伴，请关注点个赞，下次不迷路。

53.294 为什么 Netty 线程池默认大小为 CPU 核数的 2 倍

有位工作 5 年的小伙伴问我说，为什么 Netty 线程池默认大小为 CPU 核数的 2 倍，今天，我花 2 分钟时间给大家专门分享一下我对这个问题的理解。

另外，我花了 1 个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

53.294.1 分析原因

我们都知道使用多线程的本质是为了提升程序的性能，总体来说有两个最核心的指标，一个延迟，一个吞吐量。延迟指的是发出请求到收到响应的时间，吞吐量指的是 。这两个指标之间有一定的关联，因为同等条件下延迟越短吞吐量越大，但由于它们是不同的维度，一个是时间，一个是空间，并不能相互转换。

因此，提升性能最主要的目的就是要降低延迟，提高吞吐量。

那我们如何来衡量这些性能指标呢?

53.294.2 如何衡量性能指标

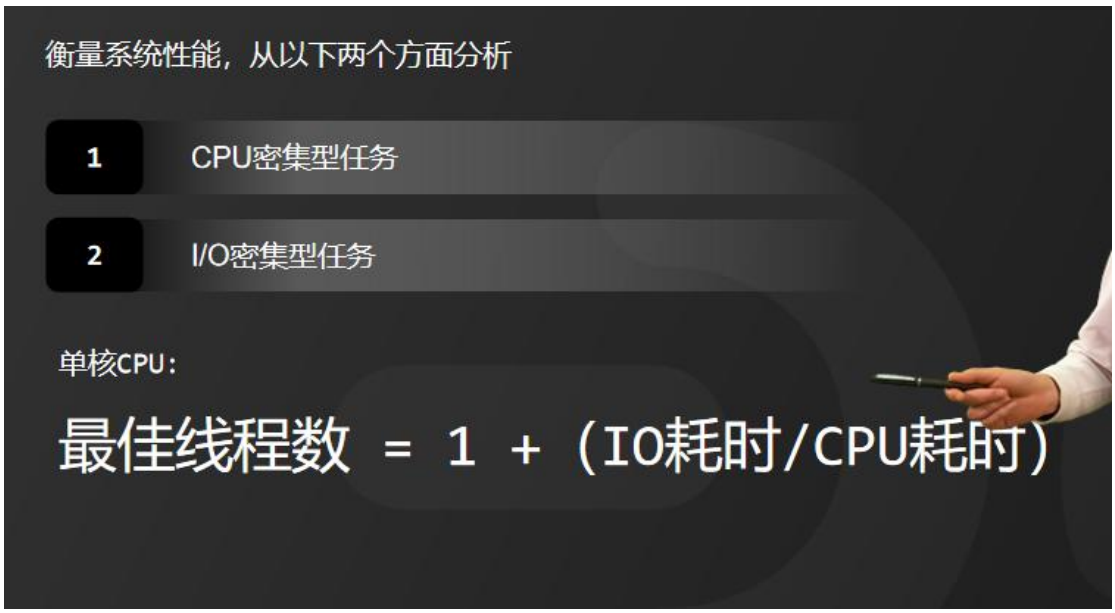
具体来说，要降低延时，就是要提高 CPU 的处理能力。而提高吞吐量，就是要提高 IO 读写效率。那么具体如何衡量系统性能，我从以下两个方面来分析：

我们可以将程序分为是 I/O 密集型任务和 CPU 密集型任务。

那么第 1 种情况，对于 CPU 密集型任务而言，理论上“线程的数量 = CPU 核数”就是合适的。但是，在实际应用中的线程数量一般会设置为“CPU 核数 + 1”。因为线程有可能因为内存页失效或其他原因导致阻塞，多设置一个线程可以保证 CPU 的利用率。

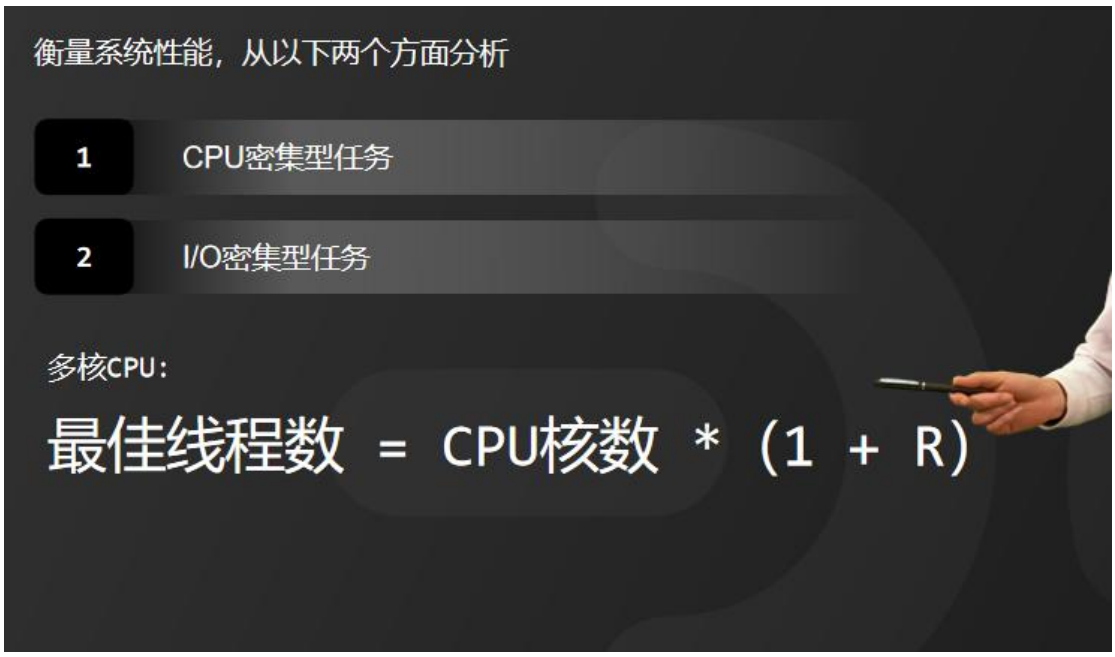
第 2 种情况，而对于 I/O 密集型任务而言，我们假设 CPU 计算和 I/O 操作的耗时比是 1:1，那么 2 个线程是最合适的。如果 CPU 计算和 I/O 操作的耗时比是 1:2，也就是说 3 个线程是合适的，这样 CPU 和 I/O 设备的利用率都可以达到 100%。根据这个推测，我们可以得到这样一个公式：

最佳线程数 = 1 + (IO 耗时/CPU 耗时)



不过上面这个公式是针对单核 CPU，如果是多核 CPU 只需要等比扩大就可以了，假设 IO 耗时和 CPU 耗时比为 R，那么计算公式如下：

最佳线程数 = CPU 核数 * (1 + R)



而 Netty 的默认线程池个数，就是假设了 I/O 耗时和 CPU 耗时的占比是 1:1，实际上 Netty 有一个参数叫 ioRatio，默认为 50，它表示在一个轮事件循环中，单个 I/O 线程执行 I/O 事件和执行异步任务的耗时占比为 1:1。相当于 R = 1，代入上面的公式，就可以得出 Netty 默认设置的线程池大小自然就是

默认线程池大小 = CPU 核数 * (1 + 1)

也就 2 倍 CPU 核数大小。而且 Netty 的应用场景主要是 I/O 密集型任务，所以，Netty 这样设计是有科学性的。

看到了这里，你是不是豁然开朗了呢?

53.294.3 总结与使用建议

通过前面的分析，我们已经知道了 Netty 线程池默认大小未 CPU 核数 2 倍的原因，我们在实际开发中，如何来得到一个比较准确的线程池大小呢?

我们可以提前压测，根据压测结果来进行微调。一般情况下，保证生产环境为压测环境的 75%即可。如果修改 Netty 的线程池大小，也一定要考虑 ioRatio 这个参数是否需要调整，因为 2 倍 CPU 核数的大小是假设的 I/O 耗时和 CPU 耗时为 1:1，调整线程大小之后，性能效果也不一定符合期望值。

在大部分场景下，没有必要太过于关注线程池大小怎么配置，I/O 密集型任务使用 Netty 默认配置就可以了。因为，提高吞吐量也不能只简单的只依赖线程池，还可以通过缓存、微服务拆分，优化业务逻辑、优化算法等方式来协作解决。

【结尾】

好了，以上就是我的分享和理解，对望能对大家有所帮助。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.295 谈谈分布式事务的 3 种解决方案

前几天，有一位 10 多年经验的架构师在面试互联网大厂时被问到这样一个问题，说请你谈谈分布式事务的解决方案。那今天，我给大家分享一下我对这个问题的理解。

另外，我花了 1 个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

53.295.1 什么是分布式事务

分布式事务是指事务的参与者和支持事务的服务器、资源服务器以及事务管理器，分别位于分布式系统的不同节点上，保证不同数据的一致性。

比如大型电商系统中的下单场景，会涉及到扣库存、优惠折扣计算、订单 ID 生成等这些服务，通常情况下库存、折扣、订单 ID 生成的服务都位于不同的服务器和数据库中，

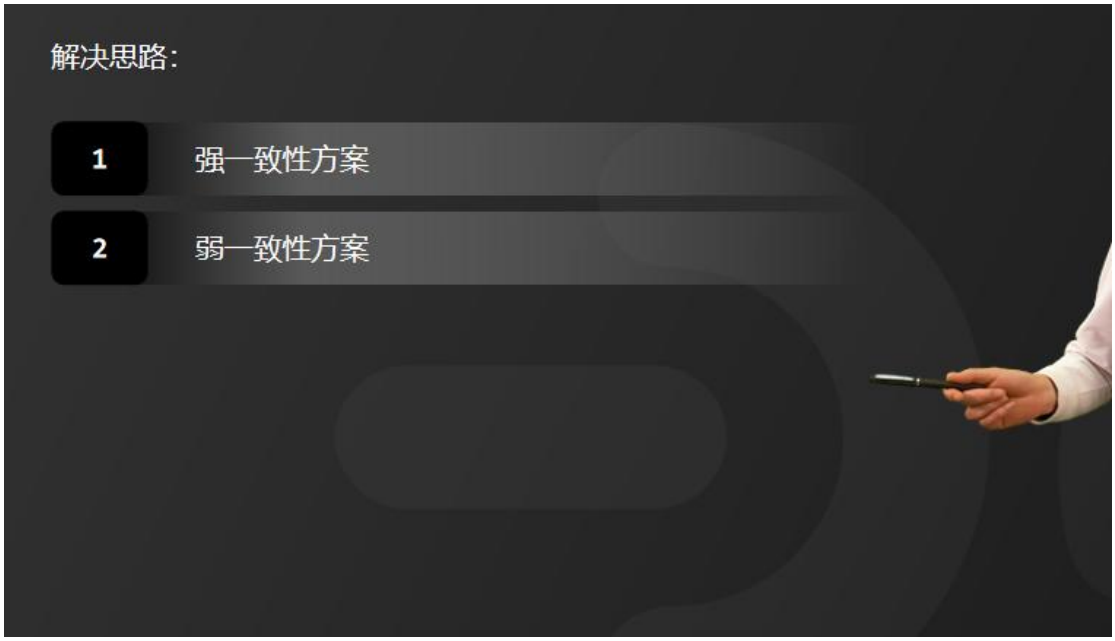
那么下单是否成功，不仅取决于本地节点的数据库操作，还需要依赖其他服务的执行结果结果，这个时候分布式事务就是保证这些操作要么全部成功，要么全部失败。

因此，本质上来说，分布式事务就是为了保证不同数据库数据的一致性。

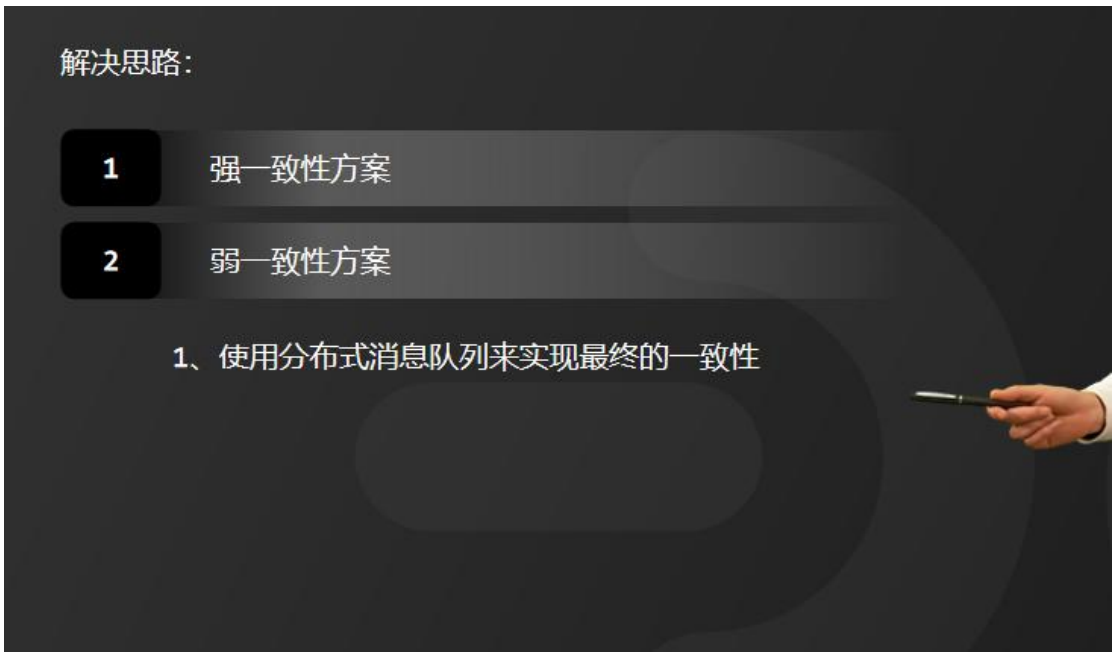
53.295.2 分布式事务解决方案

基于 CAP 定理和 Base 理论，我们可以知道，对于上述情况产生的分布式问题，我们要么采用强一致性方案，要么采用弱一致性方案。

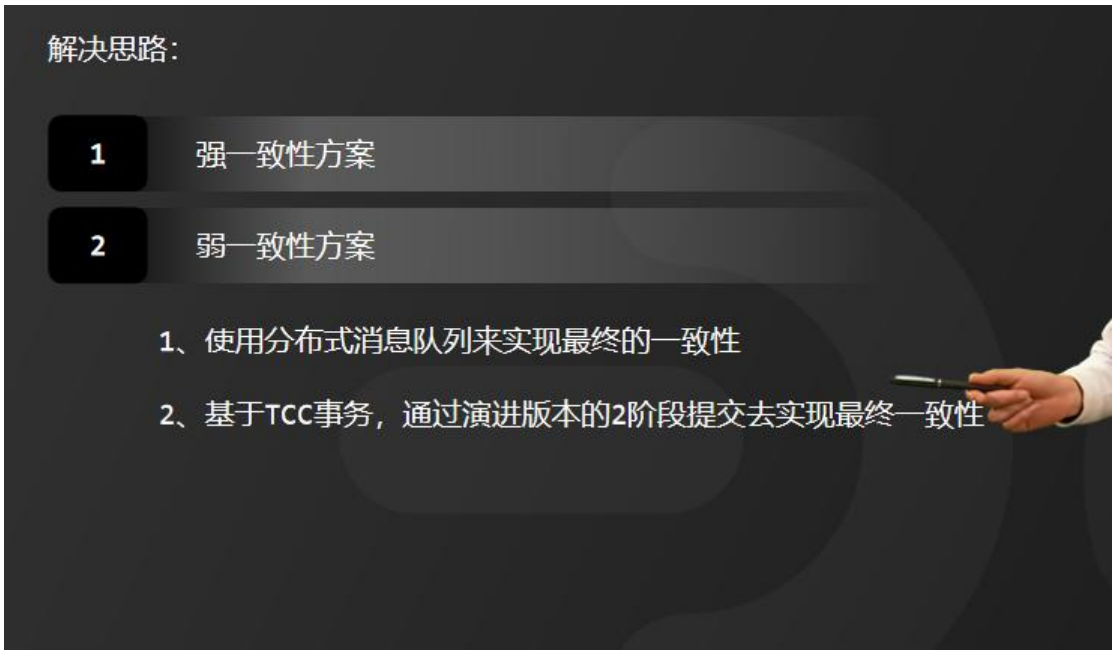
所谓强一致性方案，是指通过第三方事务管理器来协调多个节点的事务性，保证每一个节点的事务达到同时成功或者同时失败，为了实现这样一个需求。我们可以引入 X/Open DTP 模型提供的 XA 协议，基于 2 阶段提交或者 3 阶段提交的方式去实现，但是如果全局事务管理器中的多个节点，任意一个节点在进行事务提交确认的时候，由于网络通信延迟导致了阻塞，就会影响到所有节点的事务提交，而这个阻塞过程呢，也会影响到用户的请求线程，这对于用户体验以及整体的性能影响非常大。



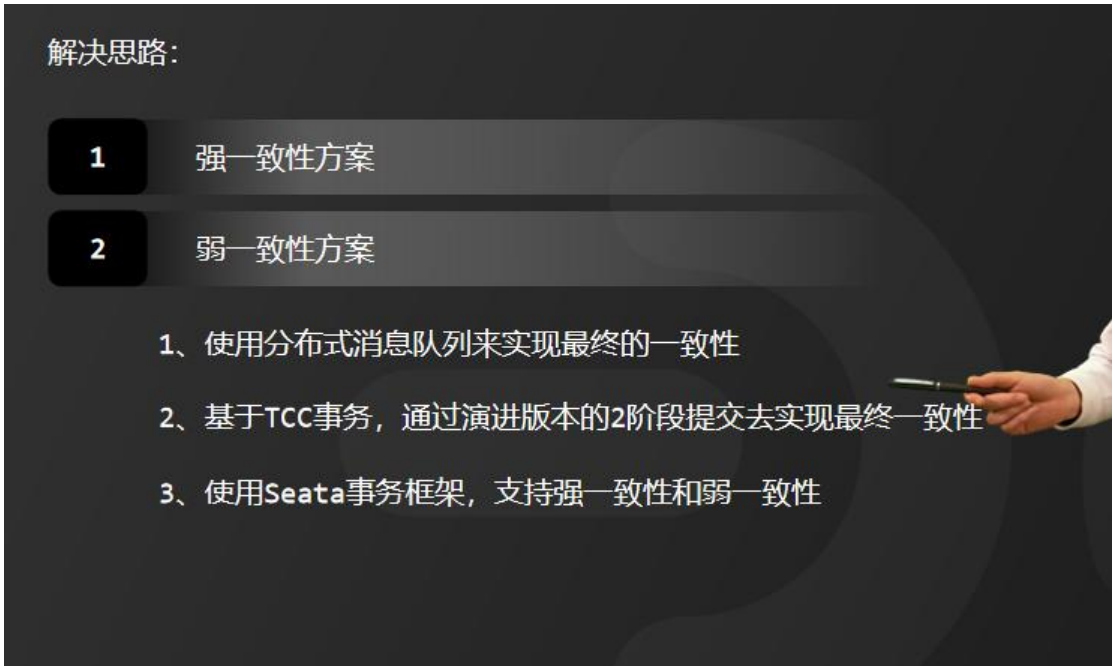
而弱一致性方案就是针对强一致性方案所衍生出来的性能和数据一致性平衡的一个方案。简单来说就是损失掉强一致性，数据在某一个时刻会存在不一致的状态，但是最终这些数据会达成一致，这样的好处是提升了系统的性能。在弱一致性方案中，常见的解决方案有 3 种：



第 1 个：使用分布式消息队列来实现最终的一致性。



第 2 个：基于 TCC 事务，通过演进版本的 2 阶段提交去实现最终一致性。



第 3 个：使用 Seata 事务框架，它提供了多种事务模型。比如说 AT、XA 、Saga、TCC 等，不同的模型提供的是强一致性或者弱一致性的支持。

【结尾】

以上就是我对分布式事务的理解。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

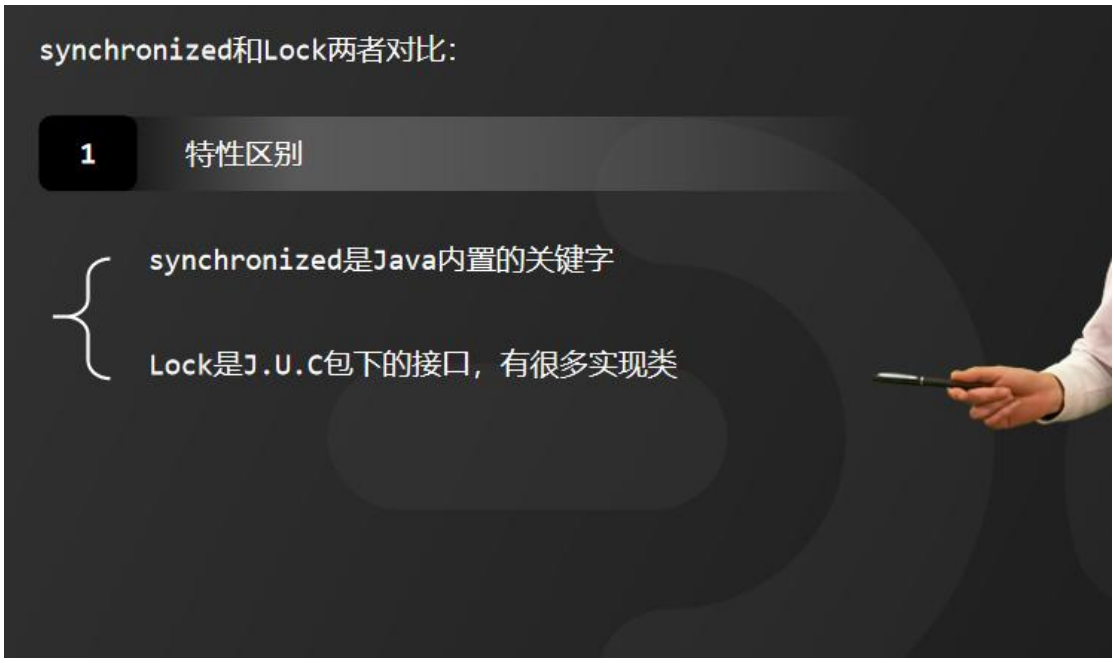
53.296 synchronized 和 Lock 的区别

最近有多位粉丝被问到 synchronized 和 Lock，据说还是阿里一面的面试题。在分布式开发中，锁是控制线程的重要方式。Java 提供了两种锁机制 synchronized 和 Lock。接下来，我给大家分享一下我对 synchronized 和 Lock 的理解。

以下是高手回答

synchronized 和 Lock 都是 Java 中用来解决线程安全问题的一个工具，那么关于 synchronized 和 Lock 的区别，我从以下 4 个方面来给大家来做一个详细的分析：

1）、特性区别

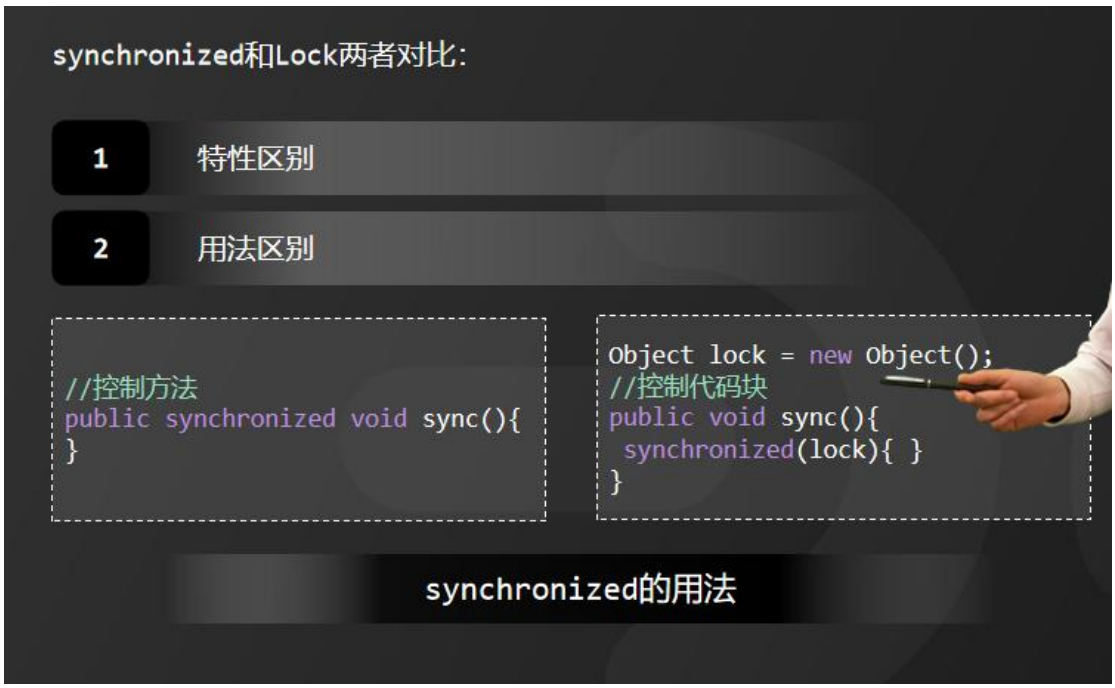


synchronized 是 Java 内置的一个线程同步关键字，

而 Lock 是 J.U.C 包下面的一个接口，它有很多实现类，比如 ReentrantLock 就是它的一个实现类。

2）、用法区别

synchronized 可以写在需要同步的对象、方法或者是特定代的码块中。主要有两种写法，比如这样：



一种是把 synchronized 修饰在方法上
//控制方法
public synchronized void sync(){
}
一种是把 synchronized 修饰在代码块上
Object lock = new Object();
//控制代码块
public void sync(){
synchronized(lock){

}
}
//用这种方式来控制锁的生命周期。而 Lock 控制锁的粒度是通过 lock() 和 unlock() 方法来实现的，以 ReentrantLock 为例，来看这样一段代码：
Lock lock = new ReentrantLock();
public void sync(){
lock.lock(); //添加锁
//TODO 线程安全的代码
lock.unlock(); //释放锁
}

这种方式，是可以保证 lock()方法和 unlock()方法之间的代码是线程安全的。而锁的作用域，取决于 Lock 实例的生命周期。

Lock 比 synchronized 在使用上相对来说要更加灵活一些。Lock 可以自主地去决定什么时候加锁，什么时候释放锁。只需要调用 lock()和 unlock()这两个方法就可以了。需要注意的是，为了避免死锁，一般我们 unlock()方法写在 finally 块中。

另外，Lock 还提供了非阻塞的竞争锁的方法叫 trylock()，这个方法可以通过返回 true 或者 fasle 来告诉当前线程是否已经有其他线程正在使用锁。

而 synchronized 是关键字，无法去扩展实现非阻塞竞争锁的方法。另外，synchronized 只有代码块执行结束或者代码出现异常的时候才会释放锁，因此，它对锁的释放是被动的。

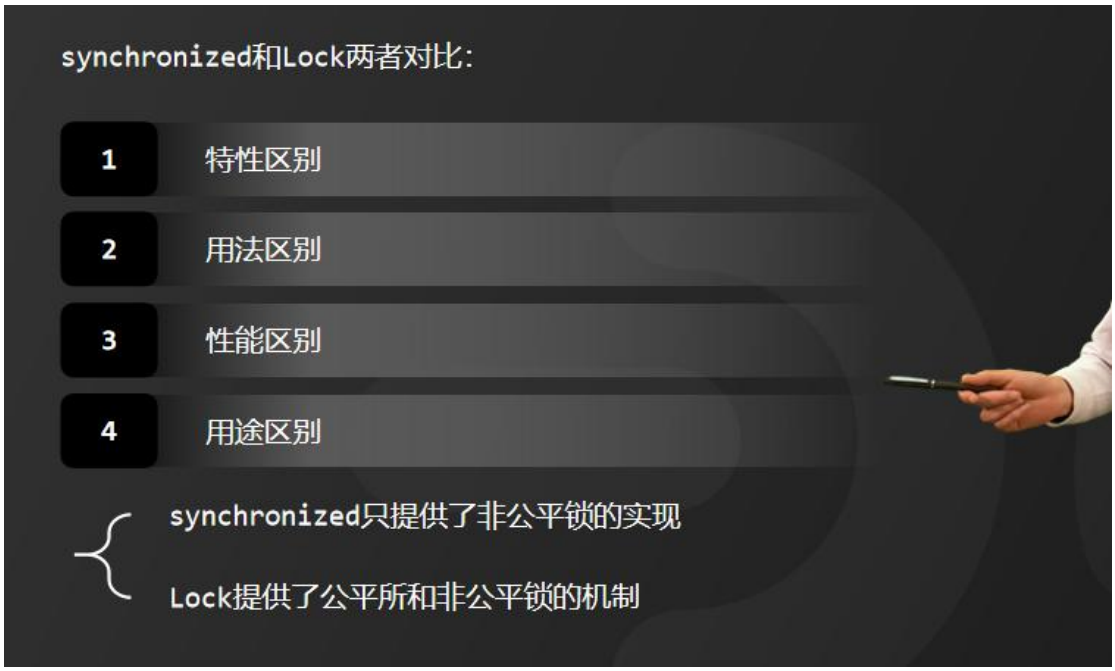
3）、性能区别

synchronized 和 Lock 在性能上差别不大。在实现上有一些区别，

synchronized 采用的是悲观锁机制，synchronized 是托管给 JVM 执行的。在 JDK1.6 以后采用了偏向锁、轻量级锁、重量级锁及锁升级的方式进行优化。

而 Lock 用的是乐观锁机制。控制锁的代码由用于自定义，也采用 CAS 自旋锁进行了优化。

4）、用途区别



二者在一般情况下没有什么区别，但是在非常复杂的同步应用中，建议使用 Lock。

因为 synchronized 只提供了非公平锁的实现，而 Lock 提供了公平所和非公平锁的机制。

公平锁是指线程竞争锁资源的时候，如果已经有其他线程正在排队或者等待锁释放，那么当前竞争锁的线程是无法去插队的。

而非公平锁就是不管是否线程再排队等待锁，它都会去尝试竞争一次锁。

以上，就是我对 synchronized 和 Lock 的理解，我还专门整理了一张表格帮助大家更好地理解，有需要这张表的小伙伴可以在我的主页简介中获取。

项	synchronized	Lock
特性	Java的关键字，在JVM层面	J.U.C包中的接口
获取	A获得锁，B等待。A阻塞，B一直等待	可尝试获得锁，线程可以不用一直等待
释放	执行完同步代码或者发生异常，被动释放	在finally中释放锁，避免死锁
状态	无法判断	可以判断
类型	可以重入，不可中断，非公平	可重入，可判断，可公平，可非公平
性能	少量同步	大量同步

【结尾】

我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.297 Dubbo 和 SpringCloud 的优缺点对比

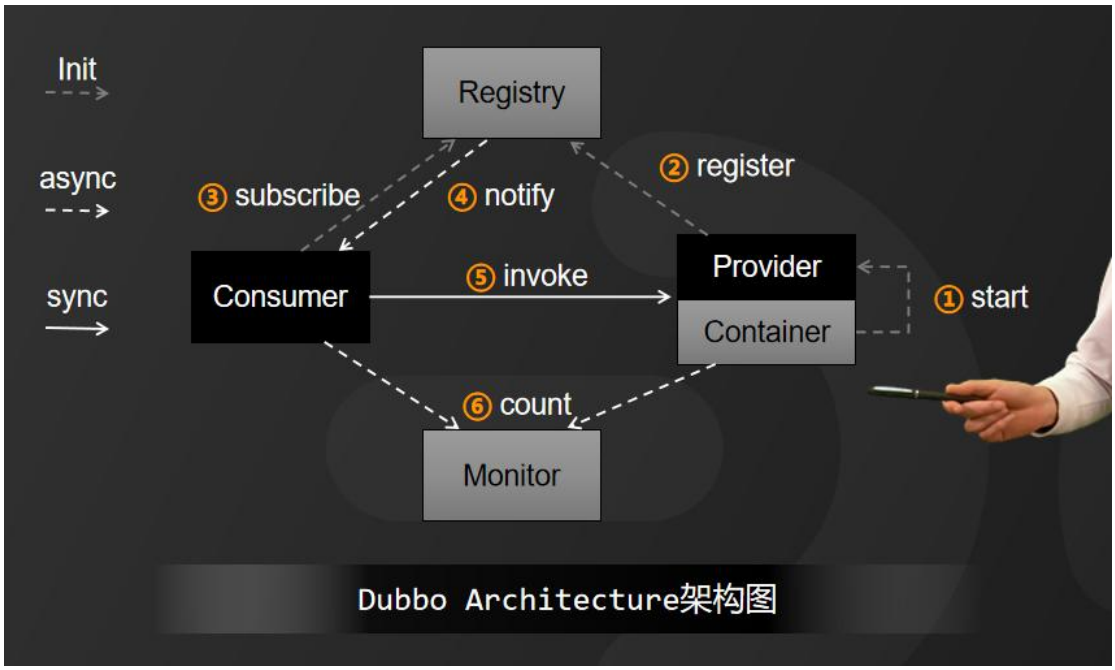
其实我个人不太愿意，拿 Dubbo 和 Spring Cloud 进行对比，因为它俩最初出现并不是为了解决同一类问题。但是，国内技术是在太卷，加上微服务的盛行，很多互联网大厂也经常会问到这个问题。那么今天，我还是给大家来详细聊一聊。

> 以下是问题分析和高手回答

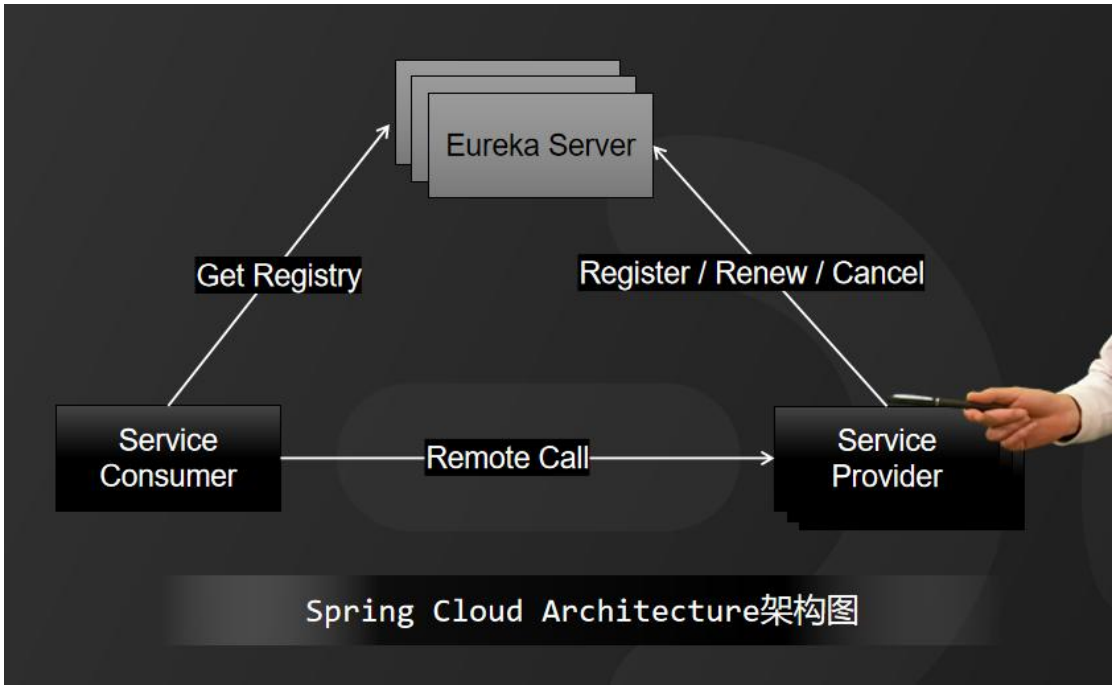
关于 Dubbo 和 Spring Cloud 的优缺点，我以奈菲（Netflix）版本为例，从以下 5 个方面来分析：

1）、从整体架构上来看

Dubbo 和 SpringCloud 的模式都比较接近，都需要服务提供方，注册中心，服务消费方。差异并不大。Dubbo 的架构图是这样的，



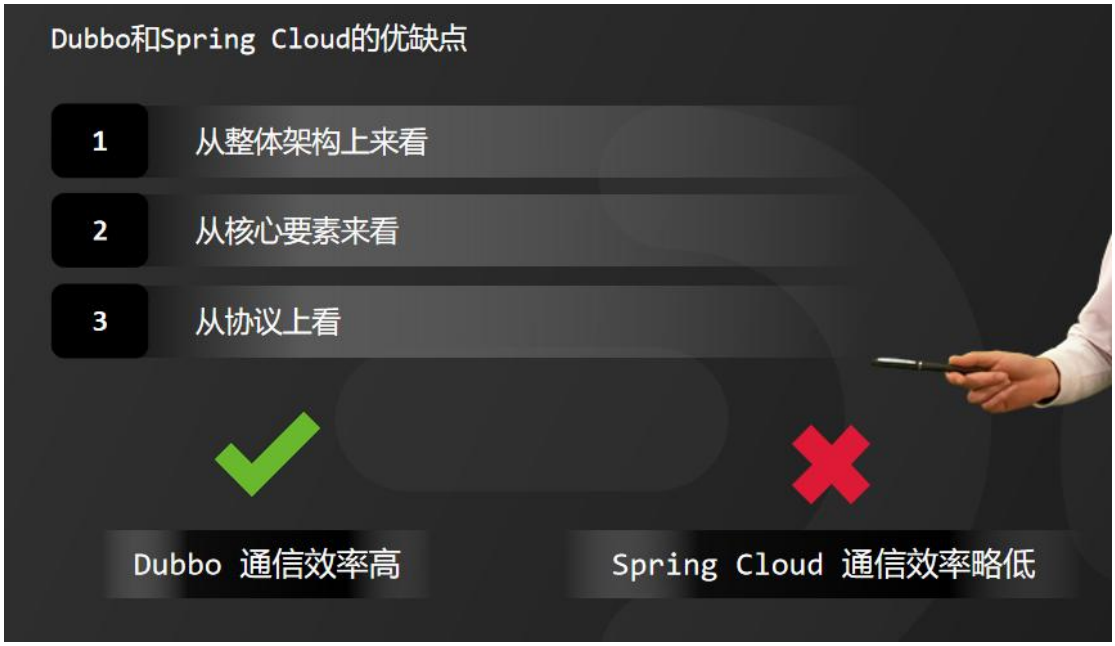
而 Spring Cloud 的架构图是这样的



2）、从核心要素来看

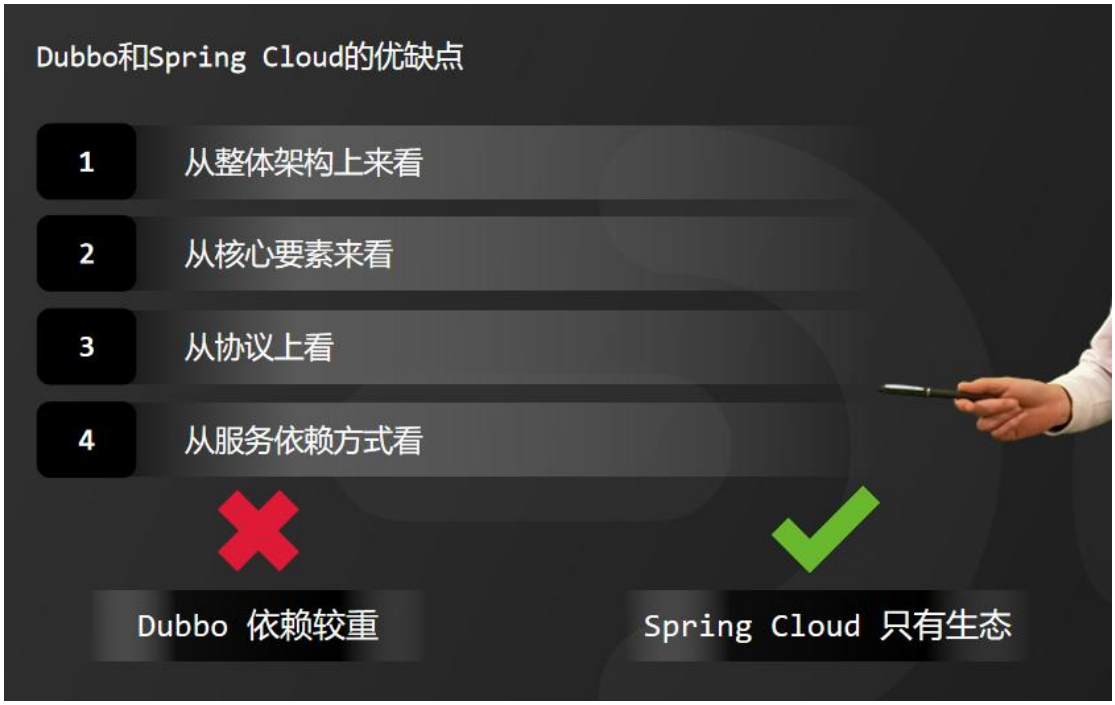
Spring Cloud 更胜一筹，在开发过程中只要整合 Spring Cloud 的子项目就可以顺利的完成各种组件的融合，而 Dubbo 需要通过实现各种 Filter 来做定制，开发成本以及技术难度略高。

3）、从协议上看



Dubbo默认采用的是单一长连接和NIO异步通讯，适合于小数据量大并发的服务调用，以及服务消费者机器数远大于服务提供者机器数的情况。Dubbo 还支持其他各种通信协议，而 Spring Cloud 使用 HTTP 协议的 REST API。因此，在通信速度上 Dubbo 略胜。

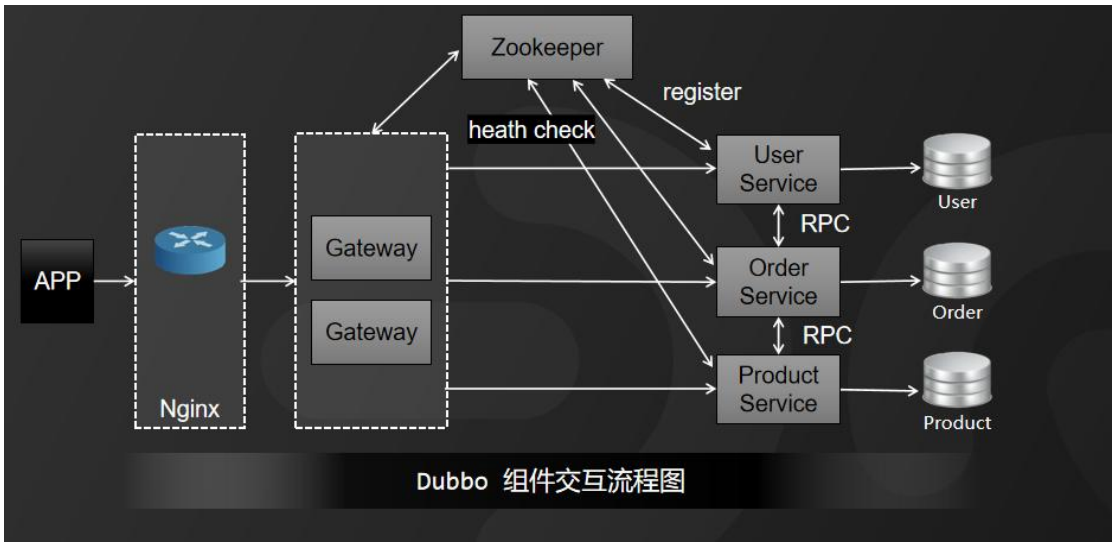
4）、从服务依赖方式看



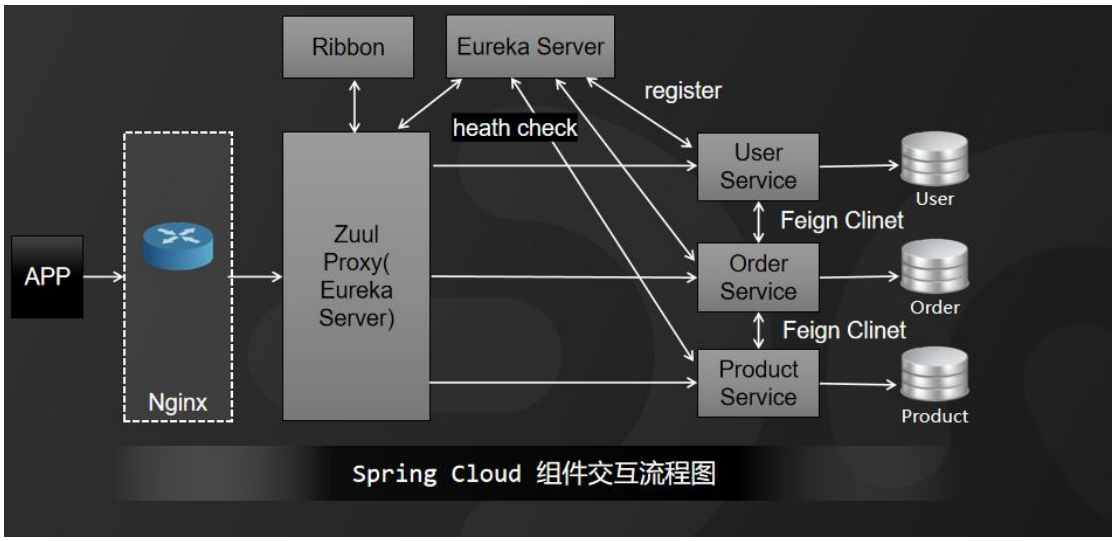
Dubbo 服务依赖比较重，需要有完善的版本管理机制，但是程序入侵少。而 Spring Cloud 是自有生态，省略了版本管理的问题，它使用 JSON 进行交互，为跨平台调用奠定了基础。

5）、从组件运行流程看

Dubbo 的每个组件都是需要部署在单独的服务器上， 用来接收前端请求、聚合服务，并批量调用后台原子服务。每个 Service 层和单独的 DB 交互。



而 Spring Cloud 所有请求都统一通过 API 网关（Zuul）来访问内部服务。网关接收到请求后，从注册中心（Eureka）获取可用服务。由 Ribbon 进行均衡负载后，分发到后端的具体实例。微服务之间通过 Feign 进行通信处理业务。



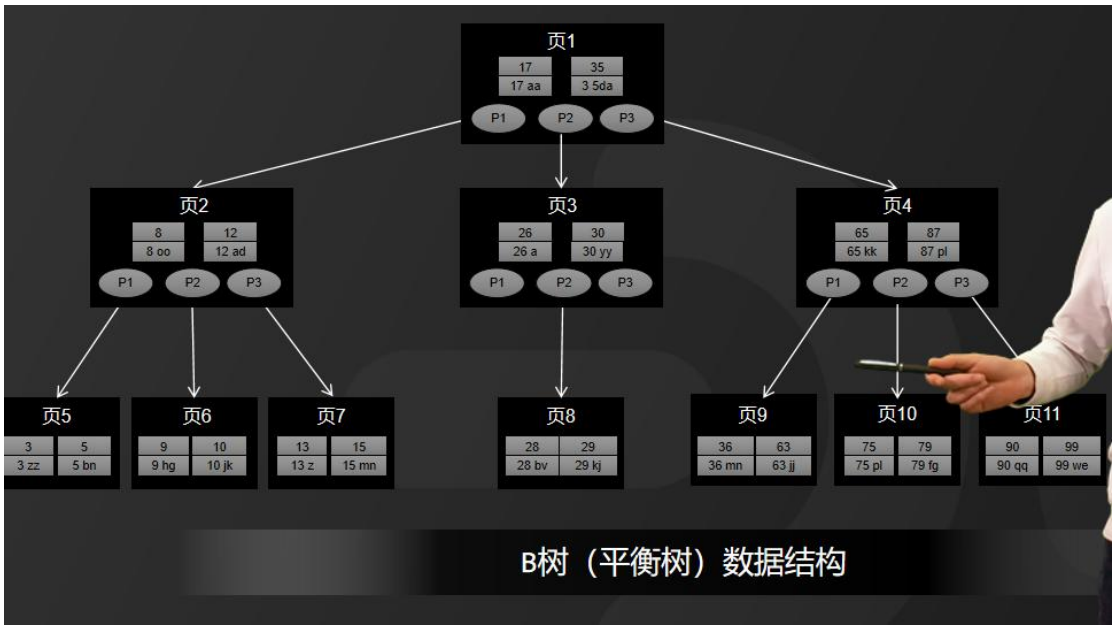
但是，两者的业务部署方式相同，都需要前置一个网关来隔绝外部直接调用原子服务的风险。Dubbo 需要自己开发一套 API 网关，而 Spring Cloud 则可以通过 Zuul 配置就可以完成网关定制。所以，从使用方式上 Spring Cloud 更加方便。

53.298 为什么 MySQL 索引结构采用 B+树?

一位 6 年经验的小伙伴去字节面试的时候被问到这样一个问题，为什么 MySQL 索引结构要采用 B+树？这位小伙伴从来就没有思考过这个问题。只因为现在都这么卷，后面还特意查了很多资料，他也希望听听我的见解。

53.298.1 B 树和 B+树

一般来说，数据库的存储引擎都是采用 B 树或者 B+树来实现索引的存储。首先来看 B 树，如图所示。



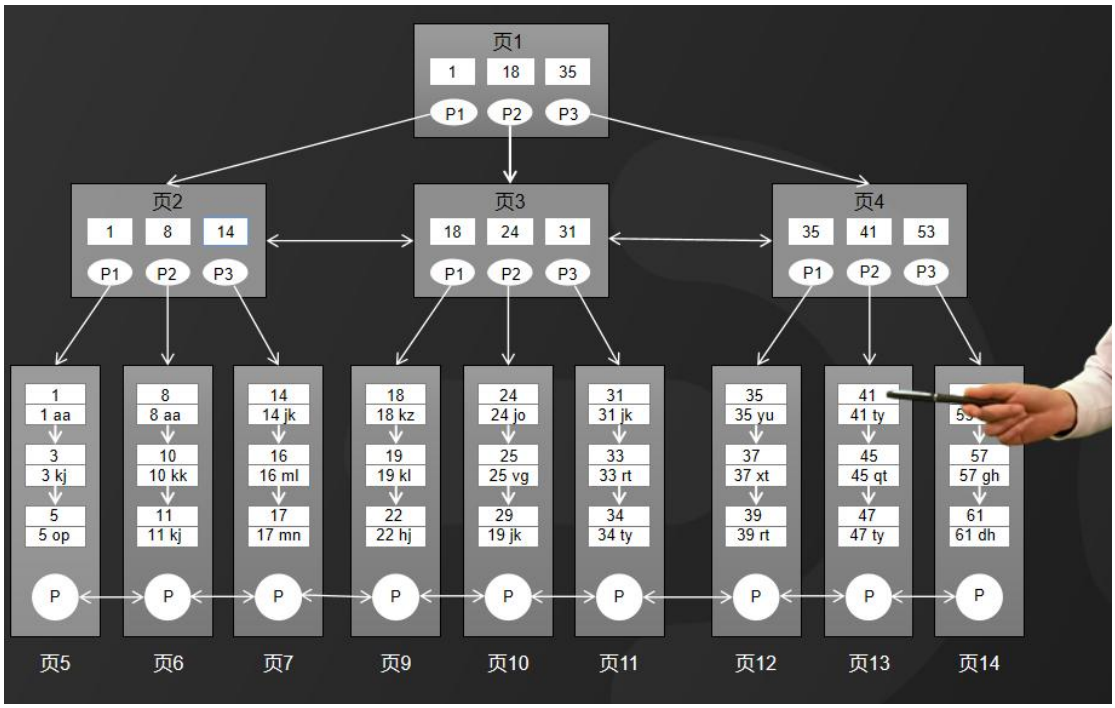
B 树是一种多路平衡树，用这种存储结构来存储大量数据，它的整个高度会相比二叉树来说，会矮很多。

而对于数据库而言，所有的数据都将会保存到磁盘上，而磁盘 I/O 的效率又比较低，特别是在随机磁盘 I/O 的情况下效率更低。

所以 高度决定了磁盘 I/O 的次数，磁盘 I/O 次数越少，对于性能的提升就越大，这也是为什么采用 B 树作为索引存储结构的原因，如图所示。

而 MySQL 的 InnoDB 存储引擎，它用了一种增强的 B 树结构，也就是 B+树来作为索引和数据的存储结构。

相比较于 B 树结构来说，B+树做了两个方面的优化，如图所示。



1、B+树的所有数据都存储在叶子节点，非叶子节点只存储索引。

2、叶子节点中的数据使用双向链表的方式进行关联。

53.298.2 原因分析

我认为，MySQL 索引结构采用 B+树，有以下 4 个原因：

1、从磁盘 I/O 效率方面来看：B+树的非叶子节点不存储数据，所以树的每一层就能够存储更多的索引数量，也就是说，B+树在层高相同的情况下，比 B 树的存储数据量更多，间接会减少磁盘 I/O 的次数。

2、从范围查询效率方面来看：在 MySQL 中，范围查询是一个比较常用的操作，而 B+树的所有存储在叶子节点的数据使用了双向链表来关联，所以 B+树在查询的时候只需查两个节点进行遍历就行，而 B 树需要获取所有节点，因此，B+树在范围查询上效率更高。

3、从全表扫描方面来看：因为，B+树的叶子节点存储所有数据，所以 B+树的全局扫描能力更强一些，因为它只需要扫描叶子节点。而 B 树需要遍历整个树。

4、从自增 ID 方面来看：基于 B+树的这样一种数据结构，如果采用自增的整型数据作为主键，还能更好的避免增加数据的时候，带来叶子节点分裂导致的大量运算的问题。

53.298.3 总结

总体来说，我认为技术方案的选型，更多的要根据具体的业务场景来决定，并不一定是说 B+树就是最好的选择，就像 MongoDB 里面采用 B 树结构，本质上来说，其实是关系型数据库和非关系型数据库的差异。

53.299 谈谈你对 MySQL 事务隔离级别的理解

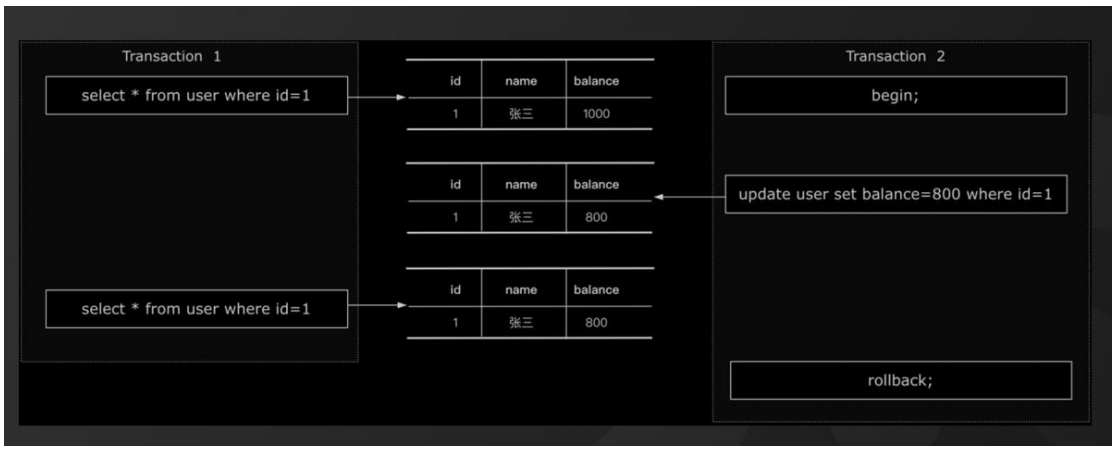
一位 5 年工作经验的粉丝，去阿里面试被问到一个关于数据库事务隔离级别的问题，当时，没有问答上来，希望给他一个参考答案。那么，今天我给大家谈谈我的理解。

另外，我花了 1 个多星期把往期的面试题解析配套文档准备好了，一共有 10W 字，想获取的小伙伴可以从我的个人煮叶简介中找到。

53.299.1 脏读、幻读、不可重复读

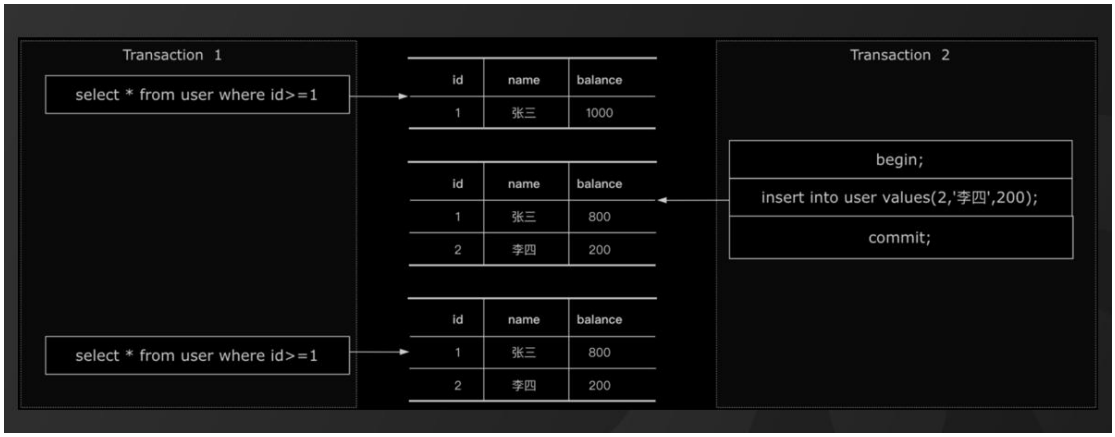
在 SQL 操作中，多个事务竞争可能会产生三种不同的现象，分别是脏读、幻读、不可重复读。

首先来看脏读，如图所示，



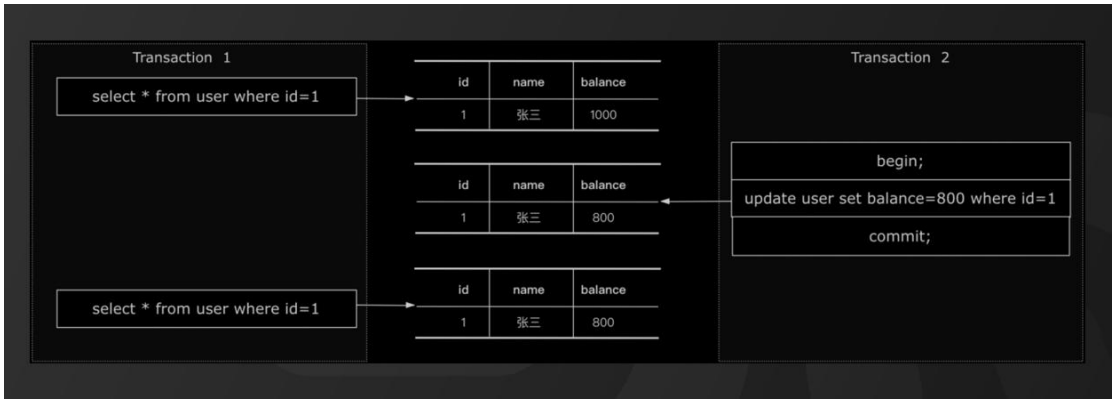
假设有两个事务 T1/T2 同时在执行，T1 事务有可能会读取到 T2 事务未提交的数据，但是未提交的事务 T2 可能会回滚，也就导致了 T1 事务读取到最终不一定存在的数据产生脏读的现象。

然后来看幻读，如图所示：



假设有两个事务 T1/T2 同时执行，事务 T1 执行范围查询或者范围修改的过程中，事务 T2 插入了一条属于事务 T1 范围内的数据并且提交了，这时候在事务 T1 查询发现多出来了一条数据，或者在 T1 事务发现这条数据没有被修改，看起来像是产生了幻觉，这种现象称为幻读。

最后来看，不可重复读，如图所示：



假设有两个事务 T1/T2 同时执行，事务 T1 在不同的时刻读取同一行数据的时候结果可能不一样，从而导致不可重复读的问题。

53.299.2 事务隔离级别

那么事务隔离级别，就是是为了解决多个并行事务竞争， 。而这脏读、幻读、不可重复读这三种现象在实际应用中，有些业务场景是不能接受这些现象存在的，所以在 SQL 标准中定义了四种隔离级别，分别是：

读未提交，在这种隔离级别下，可能会产生脏读、不可重复读、幻读。

读已提交（RC)，在这种隔离级别下，可能会产生不可重复读和幻读。

可重复读（RR），在这种隔离级别下，可能会产生幻读

串行化，在这种隔离级别下，多个并行事务串行化执行，不会产生安全性问题。

这四种隔离级别里面，只有串行化解决了全部的问题，但这种隔离级别的性能是最低的。

在 MySQL 里面，InnoDB 引擎默认的隔离级别是 RR（可重复读），因为它需要保证事务 ACID 特性中的隔离性特征。

【结尾】

以上就是我对 MySQL 事务隔离级别的理解。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.300 什么是双亲委派机制?

今天给大家分享一道国内的一二线互联网公司，高频次出现的面试题。比如，什么是类加载？什么是双亲委派？ 等等。首先，我们来看双亲委派，它全称是 Parent Delegation

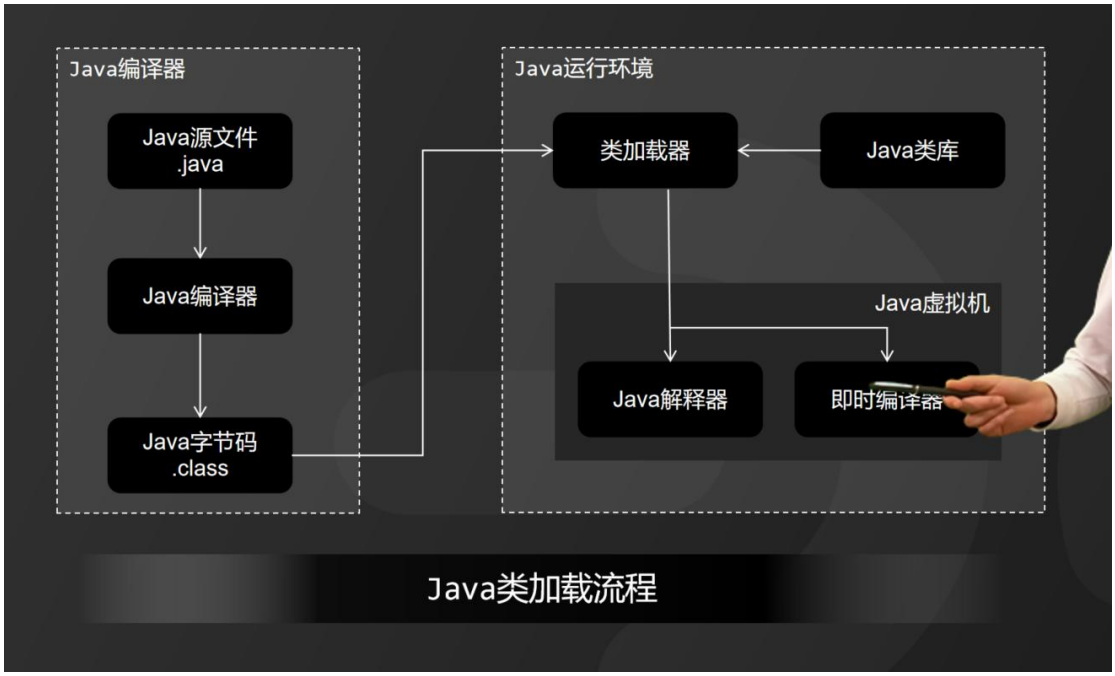
Model，直译过来可能叫做父级委托模型更容易理解。不管它叫什么，如果是你被问到这样的问题，你会不会冷场呢?

那么，今天我给大家分享一下我的理解。

我花了 1 个多星期把往期的面试题解析配套文档准备好了，一共有 10W 字，想获取的小伙伴可以从我的个人煮叶简介中找到。

53.300.1 类加载机制

要理解双亲委派，首先要理解 Java 的类加载机制。接下来，我简单介绍一下 Java 的类加载机制，如图所示：



Java 编译器将 Java 源文件编译成.class 文件，再由 JVM 加载.class 文件到内存中，JVM 装载完成后得到一个 Class 字节码对象。拿到字节码对象之后，我们就可以实例化了。

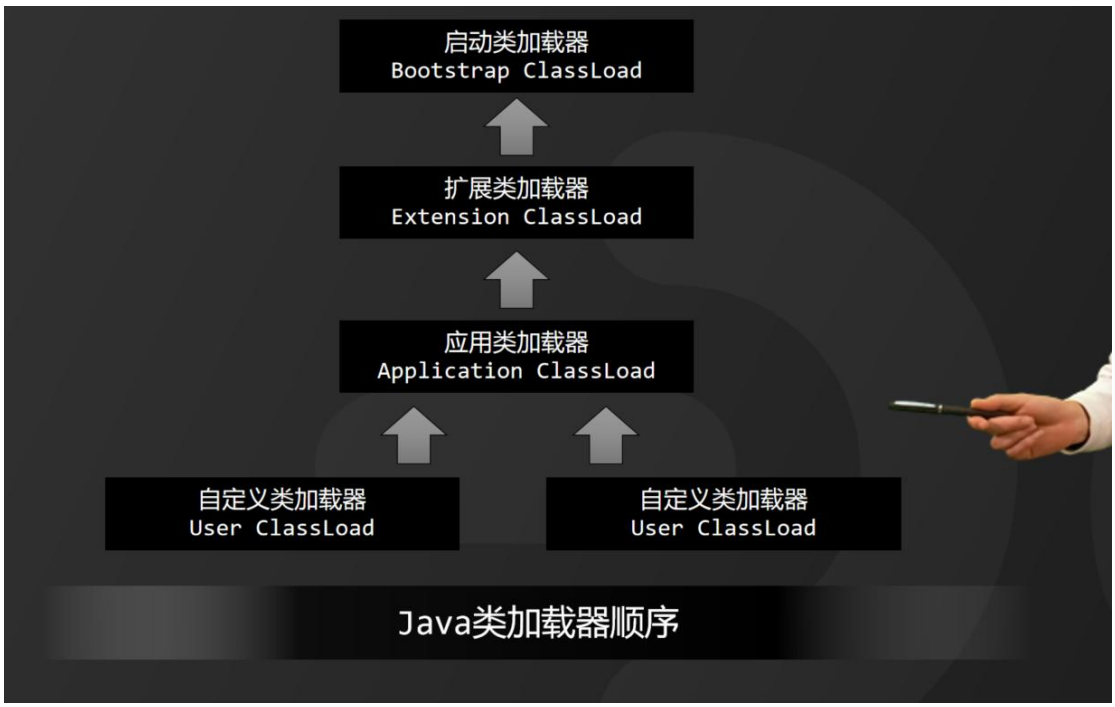
那么，类的加载过程需要使用到加载器。JVM 设计了 3 个类加载器，它们分别是：Bootstrap 类加载器、Extension 类加载器和 类加载器，这些类加载器分别加载不同作用范围的 jar 包和.class 文件。下面，我给大家详细介绍一下：

1、Bootstrap ClassLoader，主要是负责 Java 核心类库的加载，也就是 %{JDK_HOME}\lib 下的 rt.jar、resources.jar 等

2、Extension ClassLoader，主要负责%{JDK_HOME}\lib\ext 目录下的 jar 包和 class 文件

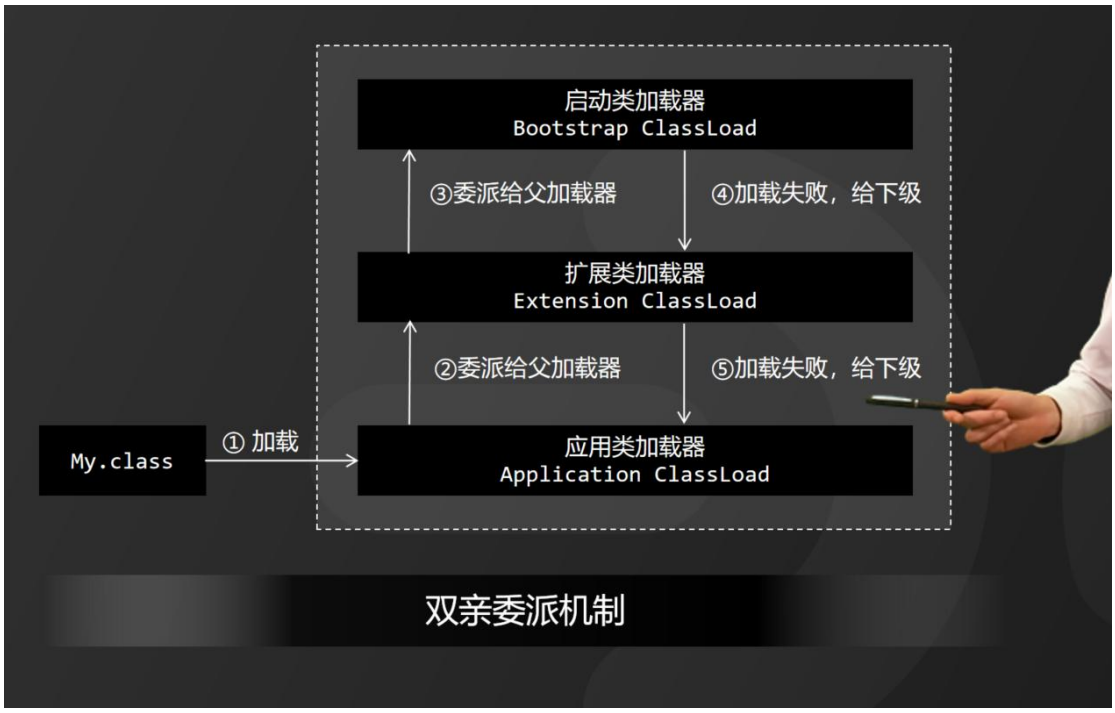
3、Application ClassLoader，主要负责当前应用里面的 classpath 下的所有 jar 包和类文件

除了系统自己提供的类加载器以外，还可以通过 ClassLoader 类实现自定义加载器，去满足一些特殊场景的需求。



53.300.2 双亲委派机制

所谓双亲委派机制，或者叫父级委托模型，就是指按照类加载器的层级关系，逐层进行委派。如图所示：



比如，我们需要加载一个 class 文件的时候，首先会把这个 class 的查询和加载委派给父加载器去执行，如果父加载器都无法加载，再尝试自己来加载这个 class。

53.300.3 总结

我认为，双亲委派机制的设计有 2 个好处：

1. 保证安全性，因为这种层级关系实际上代表的是一种优先级，也就是所有的类的加载，优先给 Bootstrap ClassLoader。那对于核心类库中的类，就没办法去破坏，

比如自己写一个 java.lang.String，最终还是会交给 Bootstrap 类加载器加载。再加上每个类加载器的都有不同的作用范围，这就意味着自己写的 java.lang.String 就没办法去覆盖核心类库中类。

2. 避免重复加载，我认为这种层级关系的设计，可以避免重复加载导致程序混乱的问题。如果父加载器已经加载过了，那么子加载器就没必要去加载了。

53.301 Netty 是什么，为什么要使用 Netty?

最近，也不知道什么原因，经常有粉丝问我关于 Netty 的问题。难道是大厂面试更卷了，开始关注更加底层的框架了？先不深究什么原因了，今天，我给大家分享一下什么是 Netty，它能解决什么问题?

另外，我花了 1 个多星期，准备了一份 10W 字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

53.301.1 Netty 是什么

一句话总结，Netty 就是一个基于 Java NIO 封装的高性能的网络通信框架。我从以下三个方面给大家归纳一下：

1、Netty 提供了比 NIO 更简单易用的 API，我们可以利用这些封装好的 API 快速开发自己的网络通信程序。

2、Netty 在 NIO 的基础上还做了很多优化，比如零拷贝机制、、内存池管理等等，因此，总体运行性能会比原生的 NIO 更高。

3、Netty 内置支持了多种通信协议，如 HTTP、WebSocket 等，并且针对数据通信的拆包、黏包问题，Netty 还内置了解决方案。

53.301.2 为什么要用 Netty?

Nety 相比于直接使 Java 原生 NIO 的 API 来说，选择 Netty 具备以下优势：

1、Netty 提供统一的 API，支持多种通信模型，如阻塞、非阻塞， 以及 epoll、poll 等模型。

2、Netty 可以使用很少的代码实现 Reactor 多线程模型以及主从线程模型。

3、可以使用自带的编解码器解决 TCP 拆包/粘包问题。

4、Netty 默认提供了多协议的通信支持。

5、Netty 处理高吞吐量、低延迟、低资源消耗，比 Java 原生 NIO 的 API 更有优势。

6、经典的开源项目底层也使用到了 Netty 通信框架， 比如 Zookeeper、Dubbo、RocketMQ 等等，经历了大型项目的使用和考验更加成熟稳定。

7、Netty 对安全性支持也不错，比如支持 SSL/TLS 等。

53.302 Netty 中提供了哪些线程模型?

最近，我更新了一些 Netty 相关的内容，于是有很多粉丝开始私信问我一些关于 Netty 的问题。今天，给大家分享一个大家问得比较多问题，Netty 中提供了哪些线程模型?

另外，我花了 1 个多星期，准备了一份 10W 字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

说到线程模型，又不得不说 Netty 中的 Reactor，Reactor 直译过来叫做反应堆，它是 Netty 支持异步多线程的核心组件。常见的 Reactor 线程模型有三种，分别是：Reactor 单线程模型、Reactor 多线程模型、主从 Reactor 多线程模型；

53.302.1 单线程单 Reactor 模型

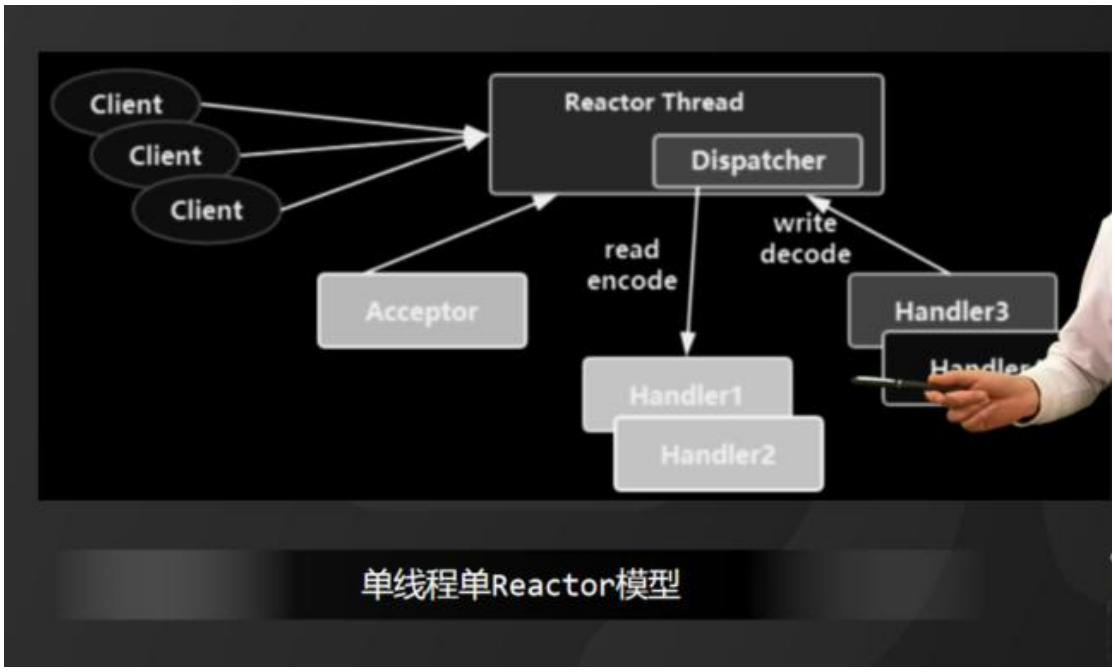
在 Reactor 模型有三个重要的组件：

1、Reactor ： 主要负责将 I/O 事件发派给对应的 Handler

2、Acceptor ： 用于处理客户端连接请求

3、Handlers ： 执行非阻塞的 I/O 读写任务

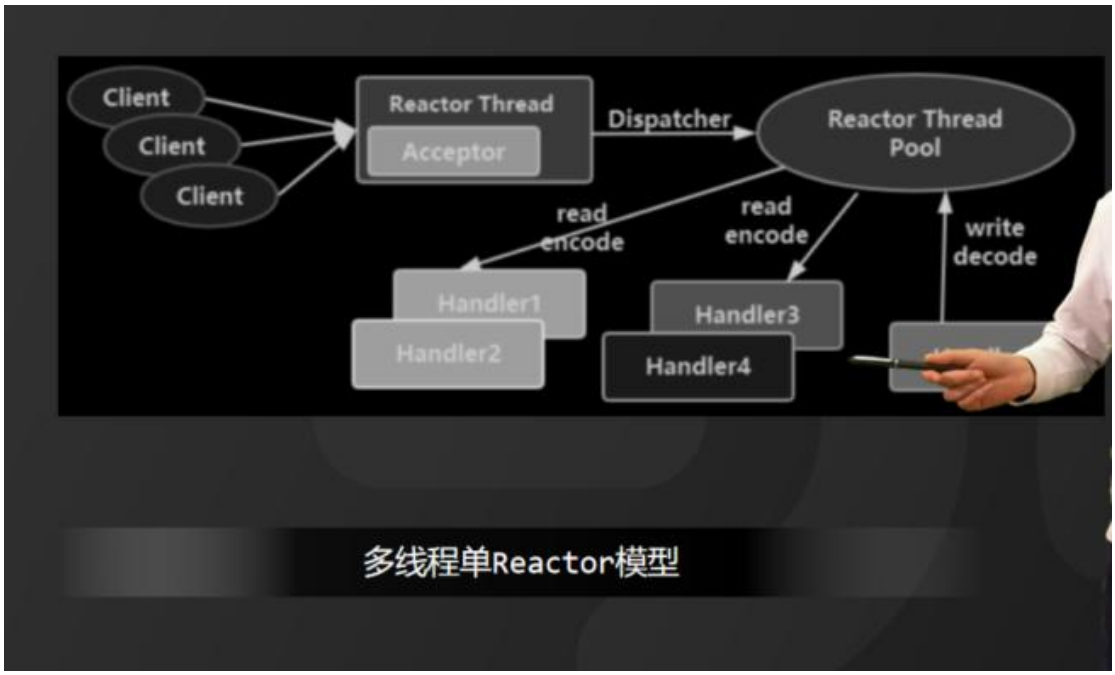
首先来看单线程单 Reactor 模型，如图所示：



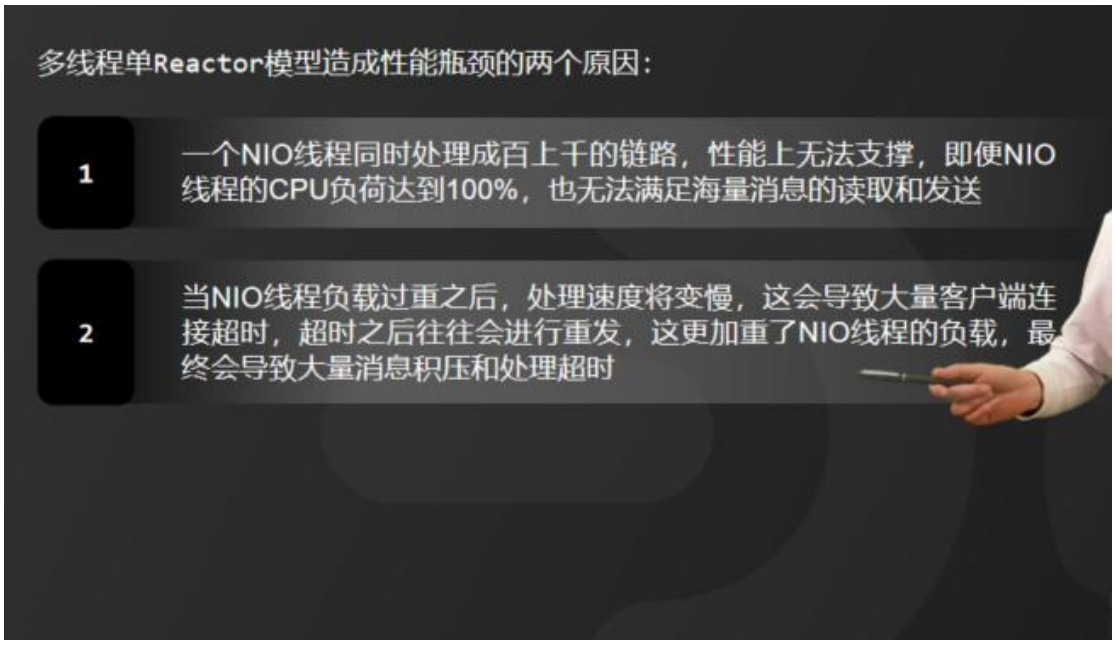
单线程 Reactor 这种实现方式存在缺点，因为，Handler 的执行是串行的，如果其中一个 Handler 处理线程阻塞，将导致其他的业务处理也会阻塞。而 Handler 和 Reactor 在同一个线程中的执行，这也将导致无法接收新的请求。

53.302.2 多线程单 Reactor 模型

为了解决单线程 Reactor 的问题，有人提出使用多线程的方式来处理业务逻辑，也就是在业务处理的地方加入线程池，实现异步处理，这样将 Reactor 和 Handler 就放在不同的线程中来执行，这就是多线程单 Reactor 模型。



但是，问题又来了，在多线程单 Reactor 模型中，所有的 I/O 操作是由一个 Reactor 来完成，而 运行在单个线程中，它需要处理包括 accept()/read() /write()/connect() 等操作，在并发量小的情况下影响不大。一旦并发量上来，出现高负载、高并发或大数据量的应用场景时，容易成为瓶颈，主要有以下 2 个原因：

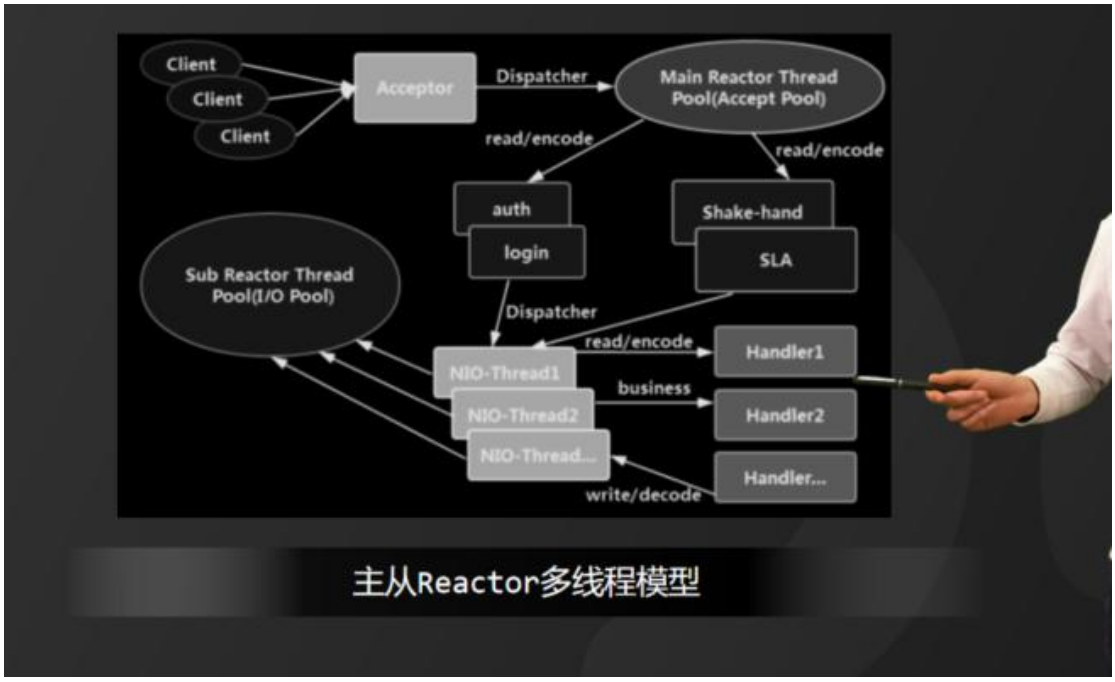


1、一个 NIO 线程同时处理成百上千的链路，性能上无法支撑，即便 NIO 线程的 CPU 达到 100%，也无法满足海量消息的读取和发送；

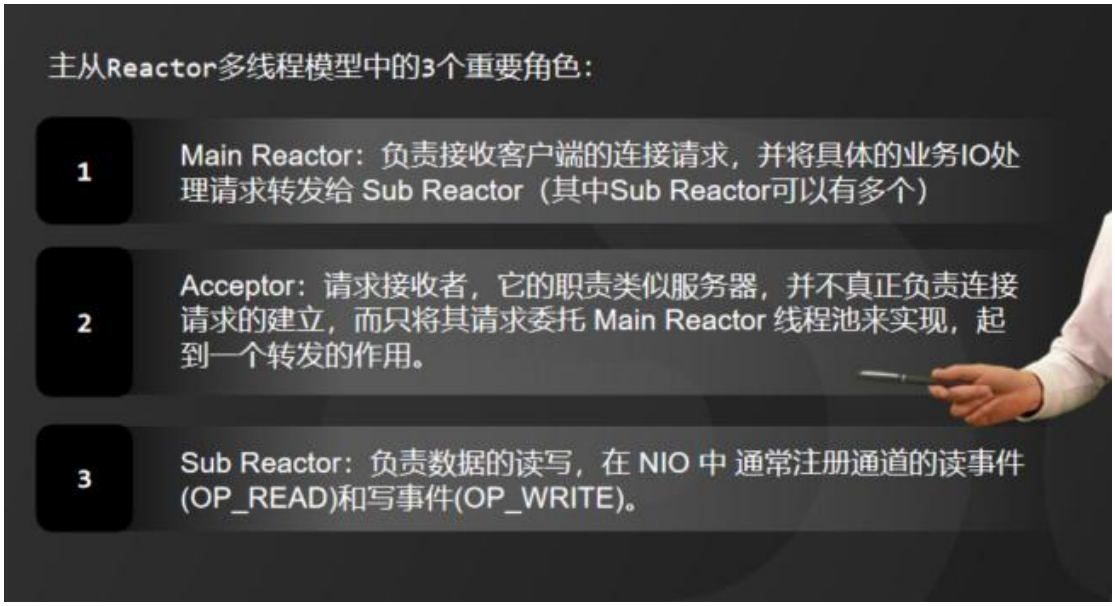
2、当 NIO 线程负载过重之后，处理速度将变慢，这会导致大量客户端连接超时，超时之后往往会进行重发，这更加重了 NIO 线程的负载，最终会导致大量消息积压和处理超时，成为系统的性能瓶颈；

所以，我们还可以更进一步优化，引入了主从 Reactor 多线程模式：

53.302.3 主从 Reactor 多线程模型



如图所示，在主从 Reactor 多线程模型中，由三个主要的角色：



Main Reactor：负责接收客户端的连接请求，并将具体的业务 IO 处理请求转发给 Sub Reactor（其中 Sub Reactor 可以有多个）。

Acceptor：请求接收者，它的职责类似服务器，并不真正负责连接请求的建立，而只将其请求委托 Main Reactor 线程池来实现，起到一个转发的作用。

Sub Reactor：负责数据的读写，在 NIO 中 通常注册通道的读事件(OP_READ)和写事件(OP_WRITE)。

53.303 Netty 中有哪些核心组件?

最近又有粉丝问我这样一个问题，说 Netty 中最核心的组件有哪些？它们都起什么作用？今天，给大家详细聊一聊

另外，我花了 1 个多星期，准备了一份 10W 字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

53.303.1 组件分层

我把 Netty 的核心组件分为三层，分别是网络通信层、事件调度层和服务编排层。

53.303.2 网络通信层

在网络通信层有三个核心组件：Bootstrap、ServerBootStrap、Channel

Bootstrap：负责客户端启动并用来链接远程 Netty Server；

ServerBootStrap：负责服务端监听，用来监听指定端口；

Channel：相当于完成网络通信的载体。

53.303.3 事件调度层

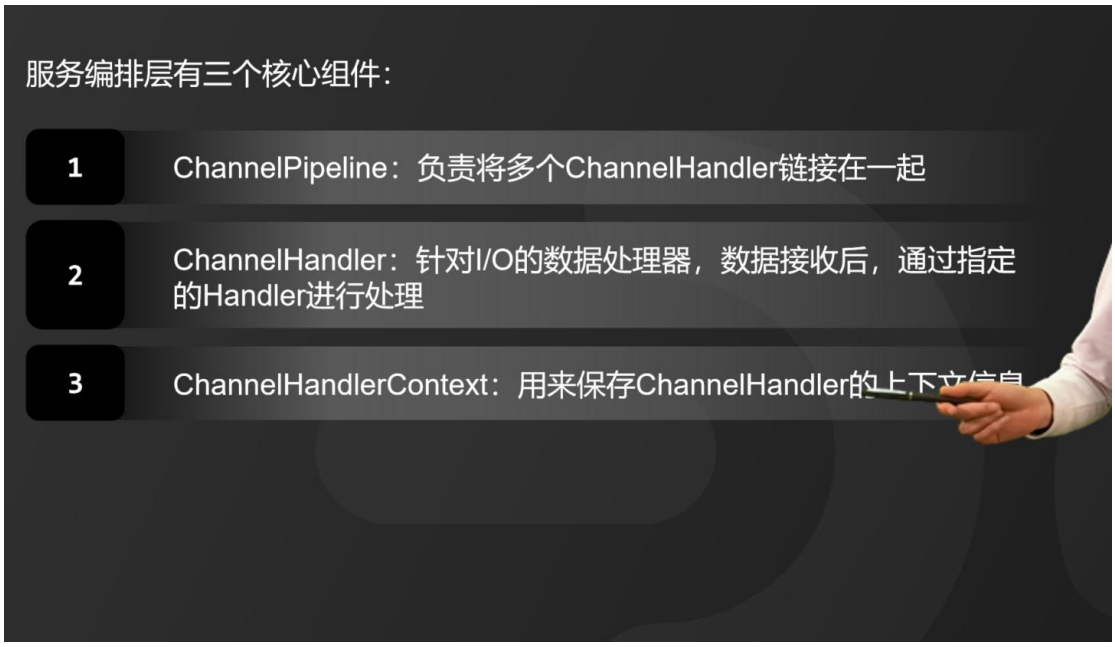
事件调度器有两个核心组件：EventLoopGroup 与 EventLoop

EventLoopGroup：本质上是一个线程池，主要负责接收 I/O 请求，并分配线程执行处理请求。

EventLoop：相当于线程池中的线程

53.303.4 服务编排层

在服务编排层有三个核心组件 ChannelPipeline、ChannelHandler、ChannelHandlerContext



ChannelPipeline：负责将多个 ChannelHandler 链接在一起

ChannelHandler：针对 I/O 的数据处理器，数据接收后，通过指定的 Handler 进行处理。

ChannelHandlerContext：用来保存 ChannelHandler 的上下文信息

【结尾】

以上就是我对 Netty 核心组件的理解。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.304 MyBatis 配置中的#{}和${}有什么区别?

前几天，一位应届生去面试，被问到一个 MyBatis 中比较基础的问题，说 MyBatis 中的#号和$符号有什么区别？今天，我给大家来详细介绍一下。

53.304.1 两者区别

Mybatis 提供到的#号和$号，都是实现动态 SQL 的一种方式，通过这两种方式把参数传递到 XML 之后，在执行操作之前，Mybatis 会对这两种占位符进行动态解析。

下面我给家详细介绍一下#号和$号的区别，

首先，来看#号，等同于 JDBC 里面的？号（占位符）。

它相当于向 PreparedStatement 预处理语句中设置参数，而 PreparedStatement 中的 SQL 语句是预编译的，如果在设置的参数包含特殊字符，会自动进行转义。所以#号占位符可以防止 SQL 注入。

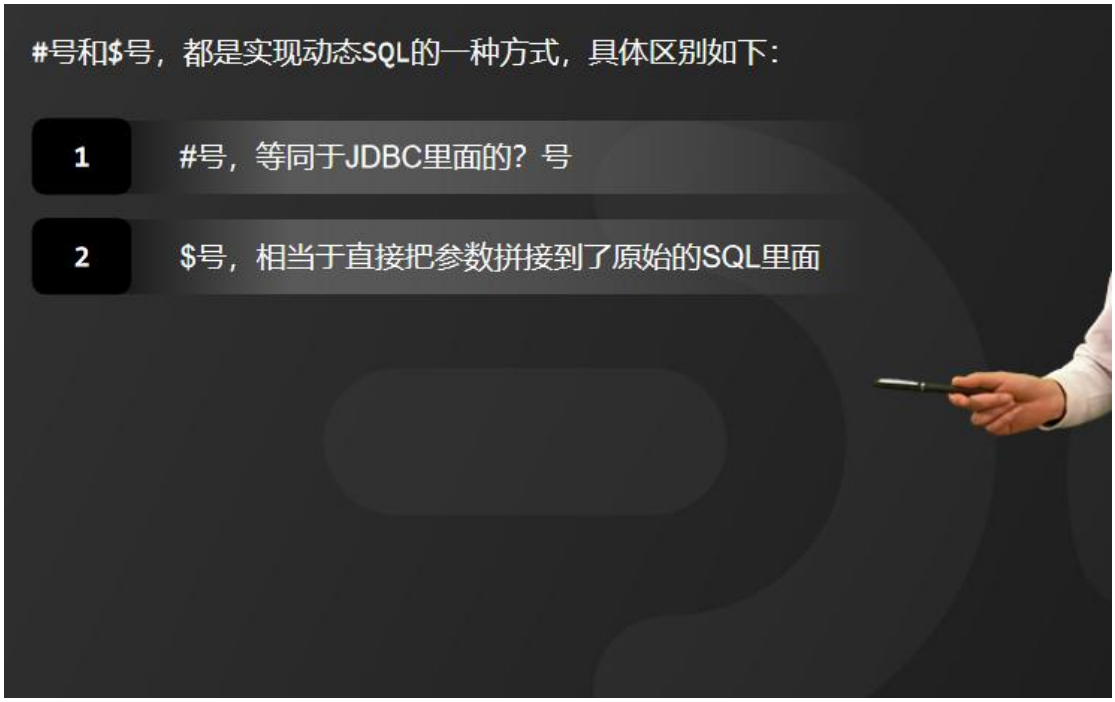
比如：这样一段代码：

解析前：

select * from user order by #{age} desc;

解析后：

select * from user order by ? desc;



而使用$号，相当于直接把参数拼接到了原始的 SQL 里面，MyBatis 不会对它进行特殊处理。

解析前：

select * from user order by ${age} desc;

解析后：

select * from user order by age desc;

所以$和#最大的区别在于，前者是动态参数，后者是占位符， 动态参数无法防止 SQL 注入的问题，所以在实际应用中，应该尽可能的使用#号占位符。

另外，$符号的动态传参，可以适合应用在一些动态 SQL 场景中，比如动态传递表名、动态设置排序字段等。

53.304.2 总结

一些小的细节如果不注意，就有可能造成巨大的经济损失。在技术如此成熟的互联网时代，还是会有一些网站经常出现 SQL 注入导致信息泄露的问题。

【结尾】

以上就是我对 MyBatis 配置#号和$号的理解。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。

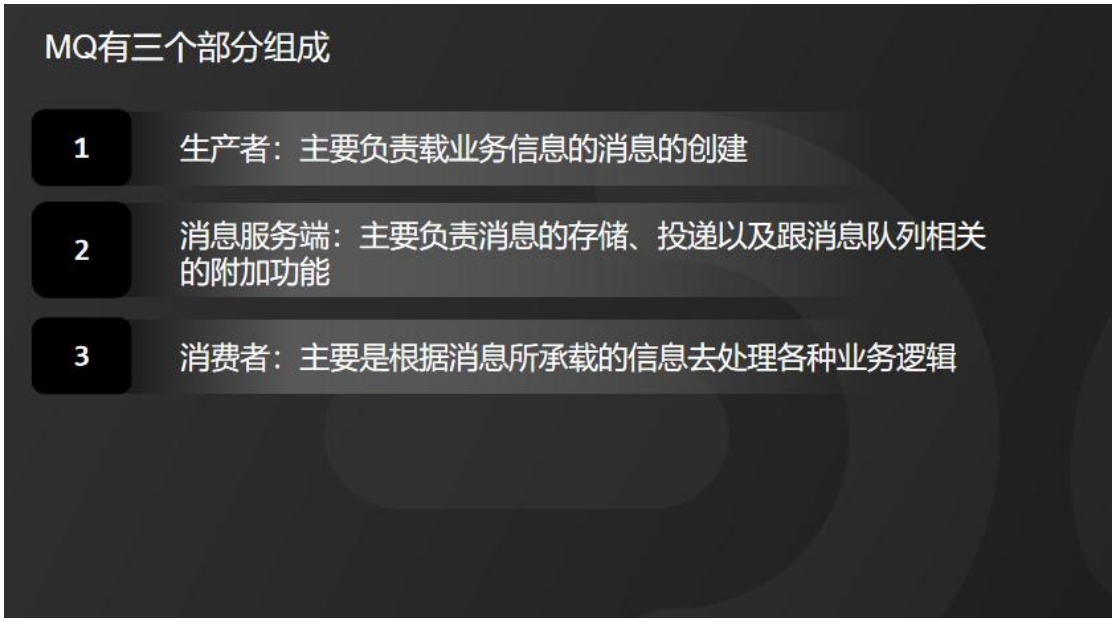
53.305 谈谈你对 MQ(消息中间件)的理解

昨天，有位 56 年工作经验的粉丝面试被问到，什么是消息中间件。平时只关注使用，并没有去总结过，竟然被这个问题住了。今天，我给大家来聊一聊。

53.305.1 什么是 MQ

MQ 全称是 Message Queue，直译过来叫做消息队列，主要是作为分布式应用之间实现异步通信的方式。

主要由三个部分组成，分别是生产者、消息服务端和消费者。



生产者（Producer），是生产消息的一端，相当于消息的发起方，主要负责载业务信息的消息的创建。

然后是消息服务端（Server），是处理消息的单元，本质就是用来创建和保存消息队列，它主要负责消息的存储、投递以及跟消息队列相关的附加功能。消息服务端是整个消息队列最核心的组成部分。

第三个是消费者（Consumer），是消费消息的一端，主要是根据消息所承载的信息去处理各种业务逻辑。

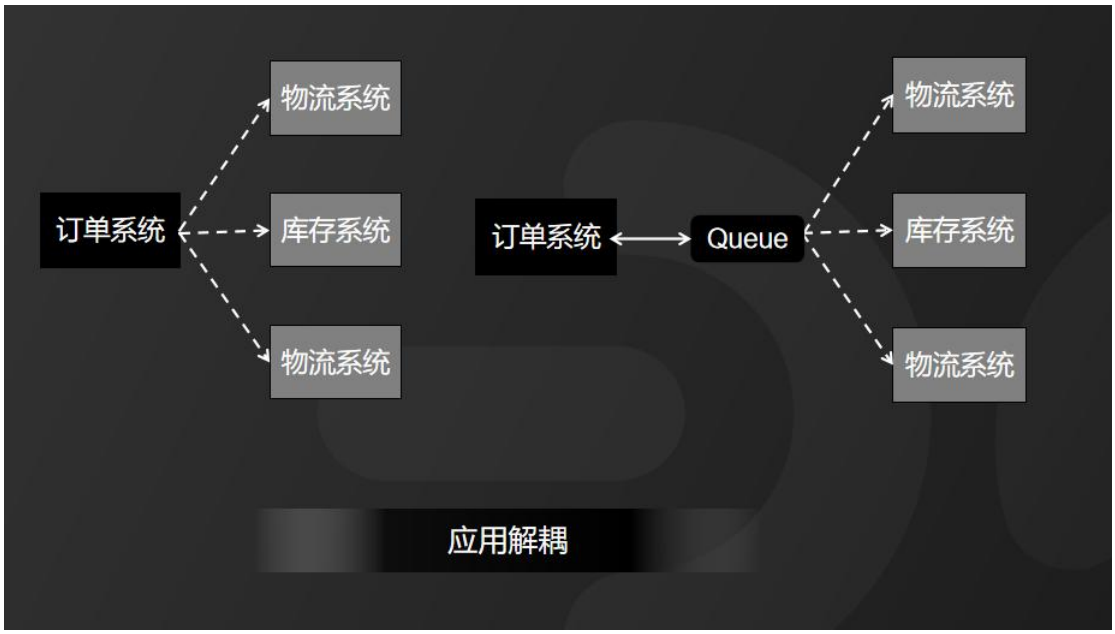
53.305.2 MQ 的应用场景

那么，我们为什么要用 MQ，它能解决什么问题呢？我从以下三个方面来分析：

第一个是流量消峰，主要是针对大流量入口过大，而业务需求又要短时间响应。但是服务器性能无法满足，导致大量请求积压，从而使得客户端大量等待超时的场景。为了保证高可用，可以把大量的并行任务发送给 MQ，MQ 再将请求分发给其他服务器，从而平稳地处理后续的业务。起到一个大流量缓冲的作用。

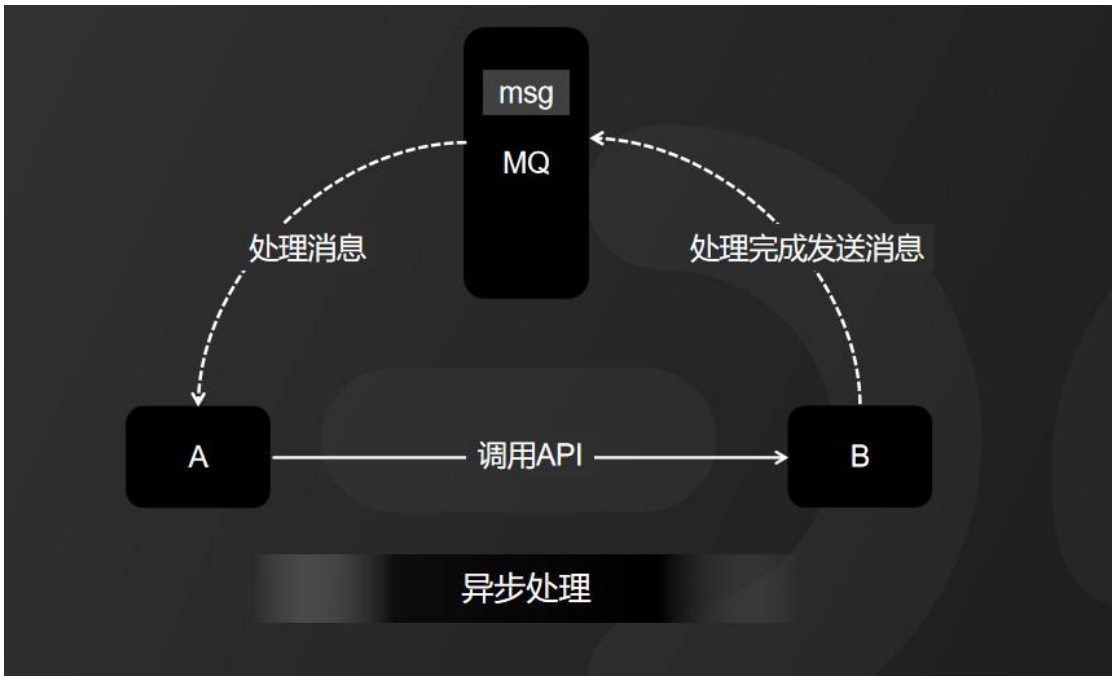
举个例子，假设订单系统最大 QPS 是 1 万，这个处理能力应付正常时段在 1 秒内就能返回结果。但是在流量高峰期，比如促销秒杀，如果 QPS 达到 2 万，订单系统就处理不过来了，只能在超过负载后不允许用户下单。如果使用消息队列做缓冲，我们可以取消这个限制，把超出负载的订单分散成一段时间来处理，这时有些用户可能在下单十几秒后才能收到下单成功的操作，但是比不能下单的体验要好。

第二个是应用解耦



可以理解为，把一些相关的、但耦合度不高的系统关联起来，以电商应用为例，比如订单系统、库存系统、物流系统、支付系统有关联，但是有没有那么紧密。每个系统将一些约定好的消息发送到 ，另外的系统直接去消费这些消息就可以了，它可以解决不同系统之间使用不同的框架或者不同编程语言的兼容性问题，从而提高整个系统的灵活性。

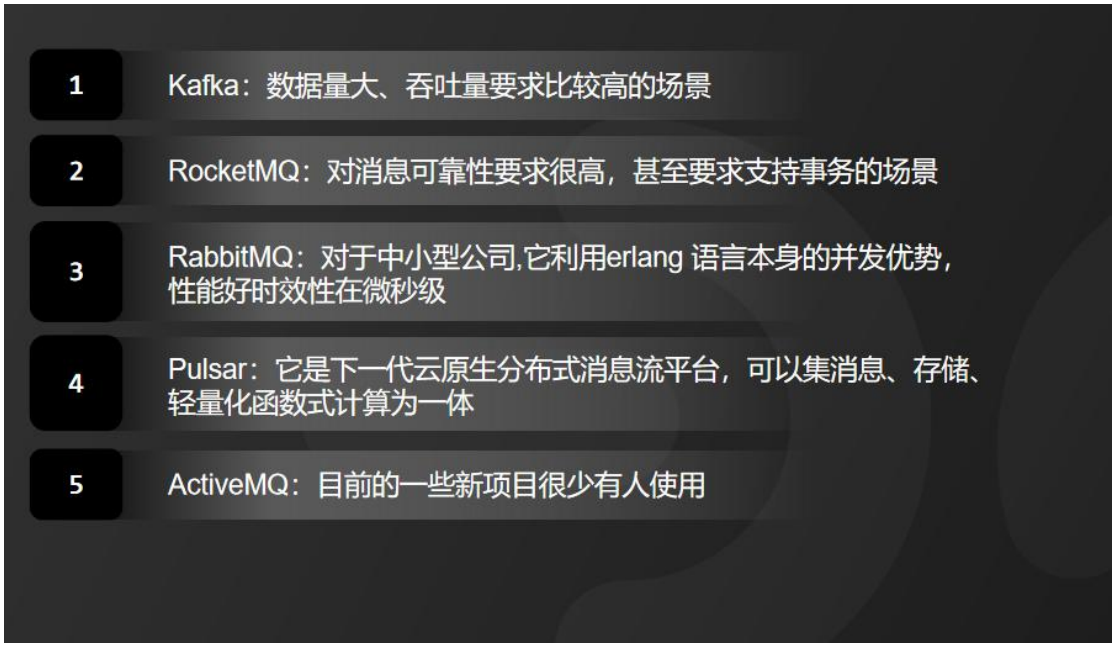
第三个是异步处理



主要应用于实时性要求不严格的场景，比如用户登录发送验证码，支付成功通知等等。服务提供方（生产者）只需要将协商好的消息发送到消息队列，之后的处理就由消费者，也不需要等待消费者的返回结果。

53.305.3 如何选择 MQ

目前，主流的消息中间件有 ActiveMQ、RabbitMQ、Kafka、RocketMQ、Pulsar。在技术选型的时候，可以根据具体的业务需求更合适的中间件。



数据量大、吞吐量要求比较高的场景一般采用 Kafka；

对消息可靠性要求很高，甚至要求支持事务的场景，比如金融互联网，可以选择 RocketMQ；

对于中小型公司来说， 可以选择 RabbitMQ，它利用 erlang 语言本身的并发优势，性能好 在微秒级。

而 Pulsar 近两年开始流行起来了，它是下一代云原生分布式消息流平台，可以集消息、存储、轻量化函数式计算为一体。

至于 ActiveMQ，目前的一些新项目很少有人用了。

53.306 谈谈你对 Redis 的理解

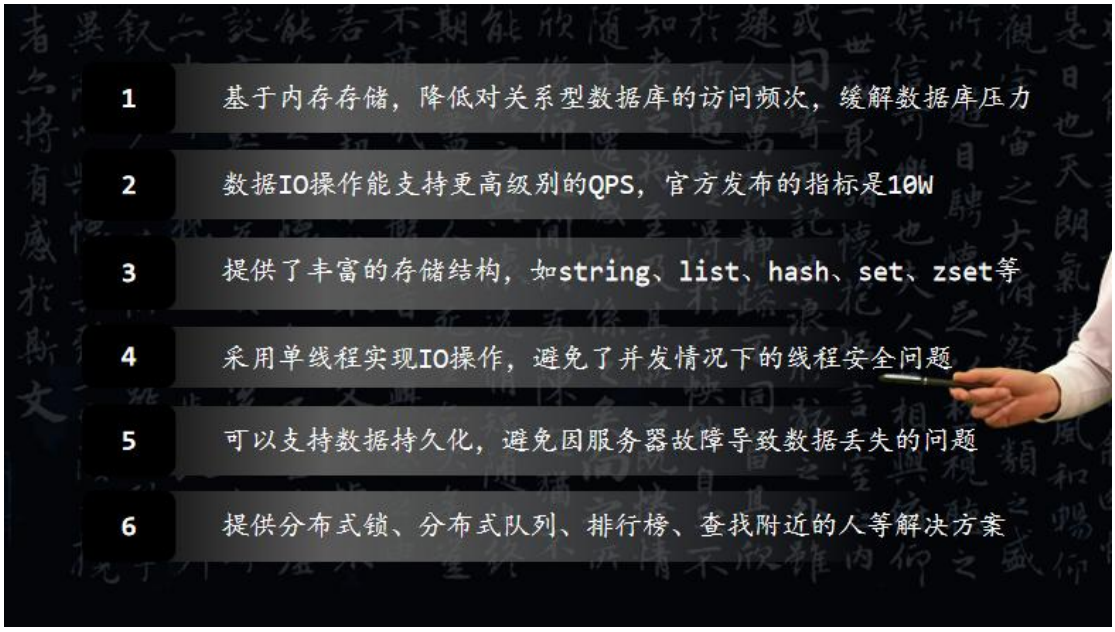
有位工作了 10 年的粉丝跟我说，去面试一家中小型公司，被要求解释什么是 Redis？后面，这位粉丝来问我，说这个面试官是不是内虚？至于是不是这个原因我不做评论，但是，养成总结的习惯还是有必要的。

53.306.1 什么是 Redis?

Redis 是一个高性能的基于 Key-Value 结构存储的 NoSQL 开源数据库。大部分公司采用 Redis 来实现分布式缓存，用来提高数据查询效率。

53.306.2 为什么选 Redis

在 Web 应用发展的初期，系统的访问和并发并不高，交互也比较少。但随着业务的扩大，访问量的提升，使得服务器负载和关系型数据库出现瓶颈，而导致瓶颈的源头，主要体现在磁盘 IO 上。随着互联网的进一步发展，对系统性能有了更高的要求，Redis 的出现，解决了很多问题。至于我们为什么要选择 Redis，我总结为以下六个原因：



1）、基于内存存储，可以降低对关系型数据库的访问频次，从而缓解数据库压力

2）、数据 IO 操作能支持更高级别的 QPS，官方发布的指标是 10W；

3）、提供了比较多的数据存储结构，比如 string、list、hash、set、zset 等等。

4）、采用单线程实现 IO 操作，避免了并发情况下的线程安全问题。

5）、可以支持数据持久化，避免因服务器故障导致数据丢失的问题

6）、Redis 还提供了更多高级功能，比如分布式锁、分布式队列、排行榜、查找附近的人等功能，为更复杂的需求提供了成熟的解决方案。

【结尾】

以上就是我对 Redis 的理解。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.307 放弃 FastDFS，拥抱 MinIO 的 7 大理由

目前可用于文件存储的网络服务选择也有不少，好比阿里云 OSS、七牛云、腾讯云等等，可是收费都有点小贵。为了帮公司节约成本，以前一直是使用 FastDFS 做为文件服务器，准确的说是图片服务器。直到我发现了 MinIO，我决定放弃 FastDFS。

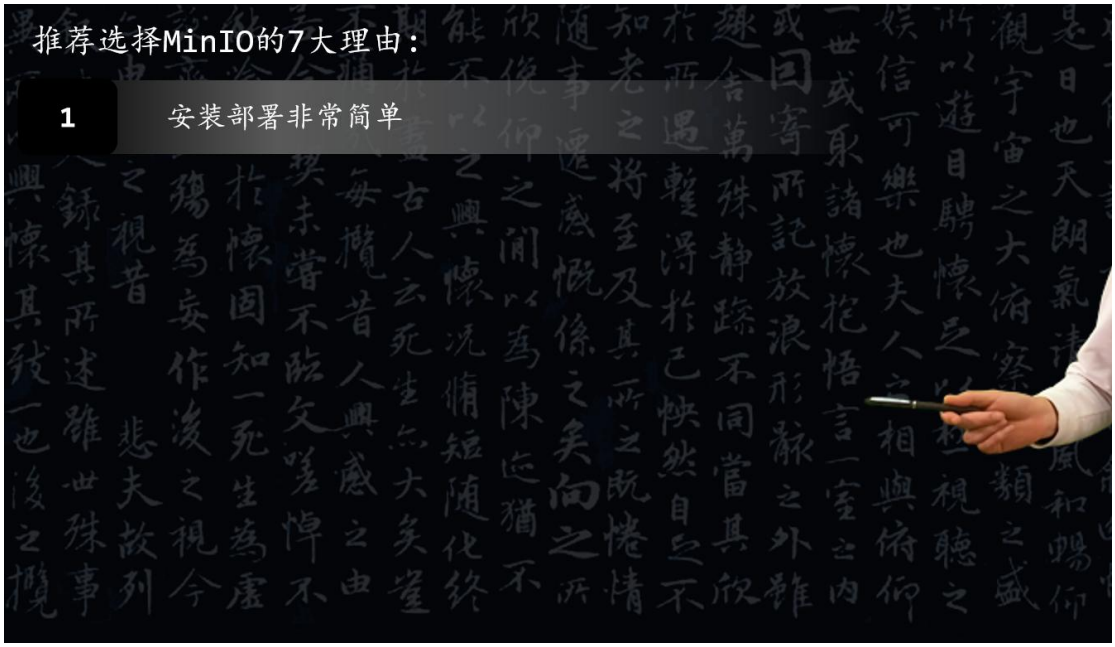
1、什么是 MinIO?

先给大家介绍一下什么是 MinIO。

MinIO 是专门为海量数据存储、人工智能、大数据分析而设计的对象存储系统。据官方介绍，单个对象最大可达 5TB。非常适合储海量图片、视频、日志文件、备份数据和容器/虚拟机镜像等。MinIO 主要采用 Golang 语言实现，整个系统都运行在操作系统的用户态空间，客户端与存储服务器之间采用 HTTP/HTTPs 通信协议。

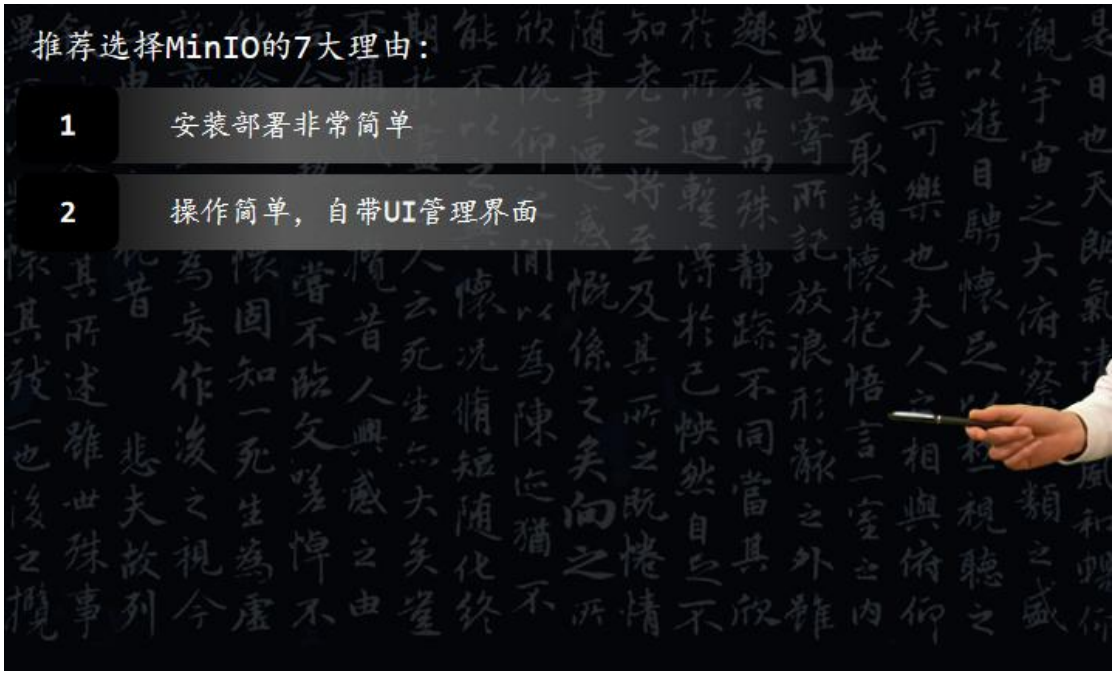
相比于其他方案来说，推荐选择 MinIO 的 7 大理由：

1）、安装部署非常简单

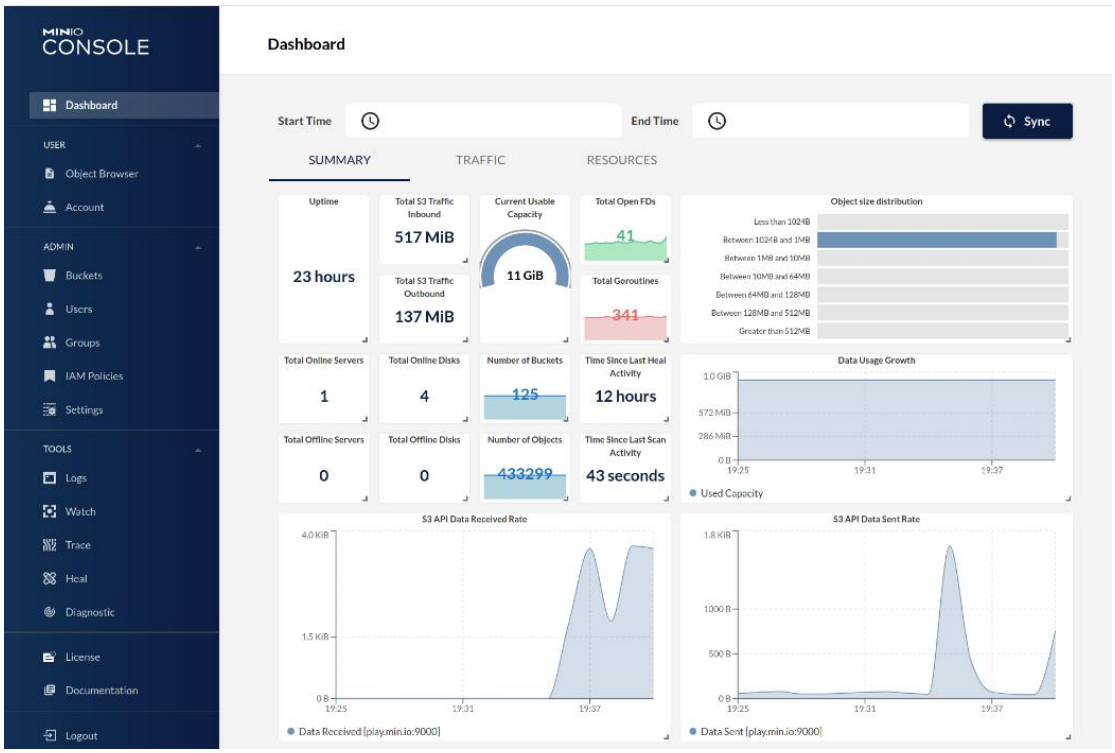


小伙伴们应该都知道，部署 FastDFS 的时，需要掌握 Linux 基础、分布式原理、C 语言编译、Nginx 安装、Ningx 防盗链插件这些前置知识。而 MinIO 只需要执行这几行命令便可轻松搞定，也大大降低了学习和维护成本。

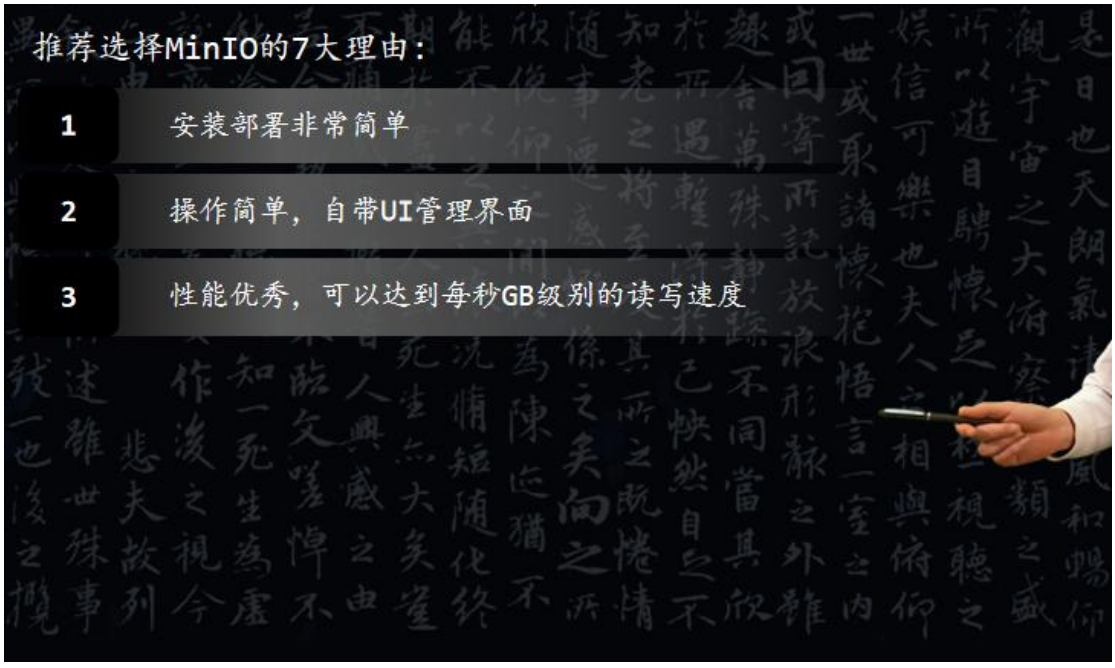
2）、操作简单，自带 UI 管理界面



MinIO 在服务端安装后就自带了 UI 界面，可谓开箱即用，而 FastDFS 默认是不带 UI 界面的。

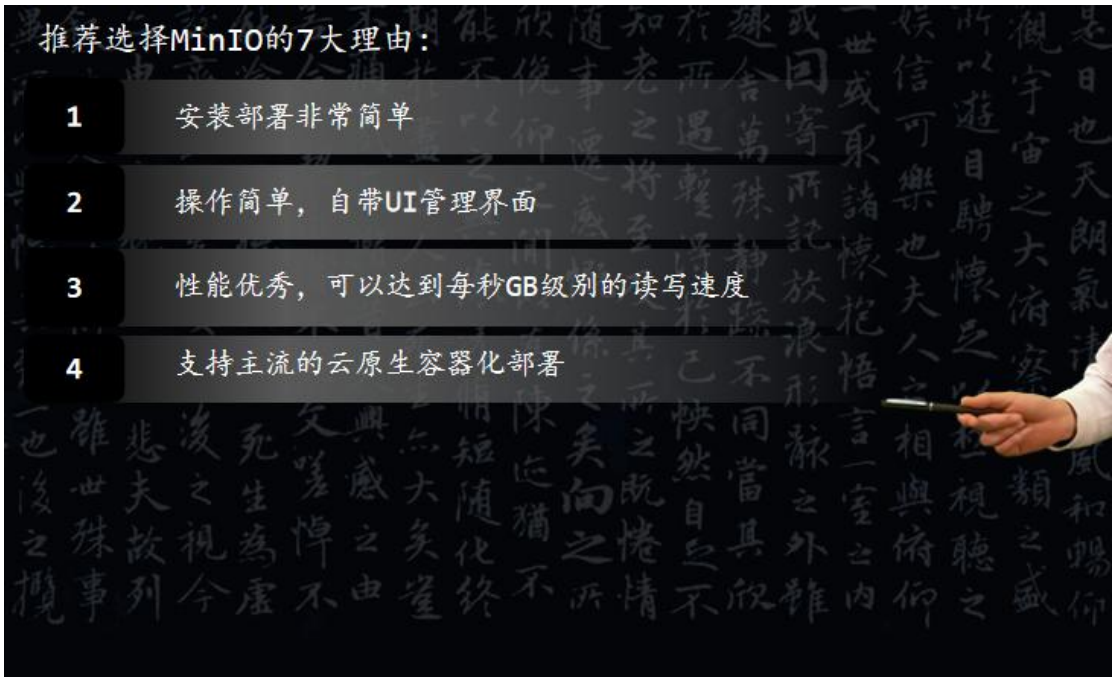


3）、性能优秀，可以达到每秒 GB 级别的读写速度



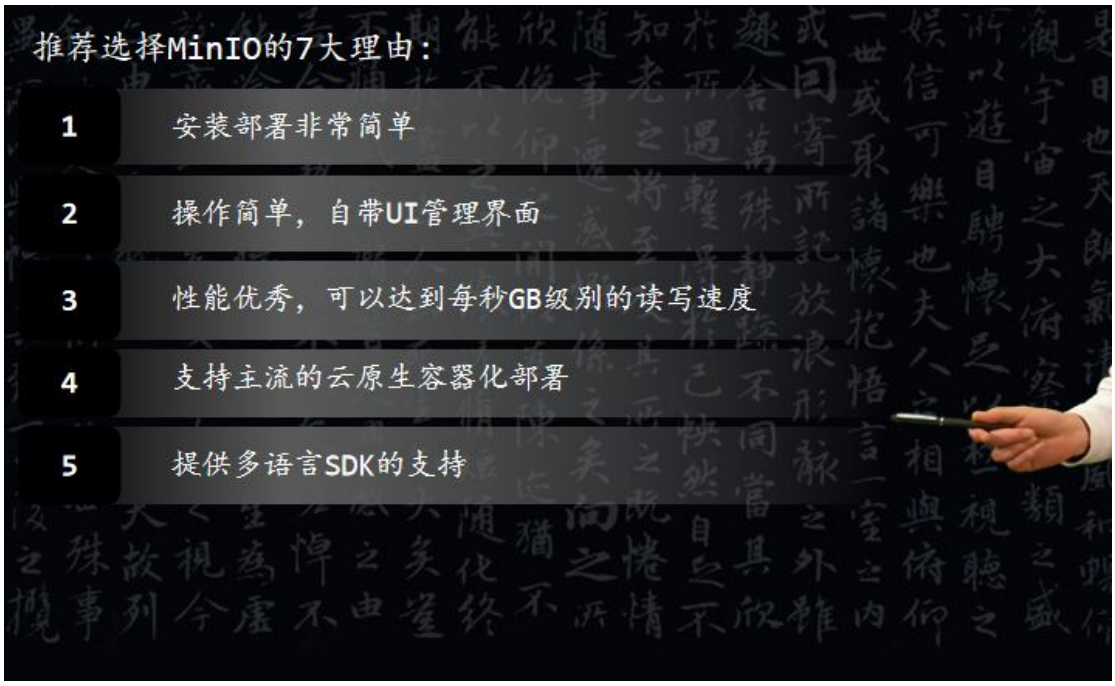
MinIO 官方号称是世界上速度最快的对象存储服务器。就算部署在普通的硬件上，读对象的速度能达到 183 GB/s，写对象的速度能达到 171 GB/s。这是 MinIO 和其他产品的性能对比图

4）、支持主流的云原生容器化部署



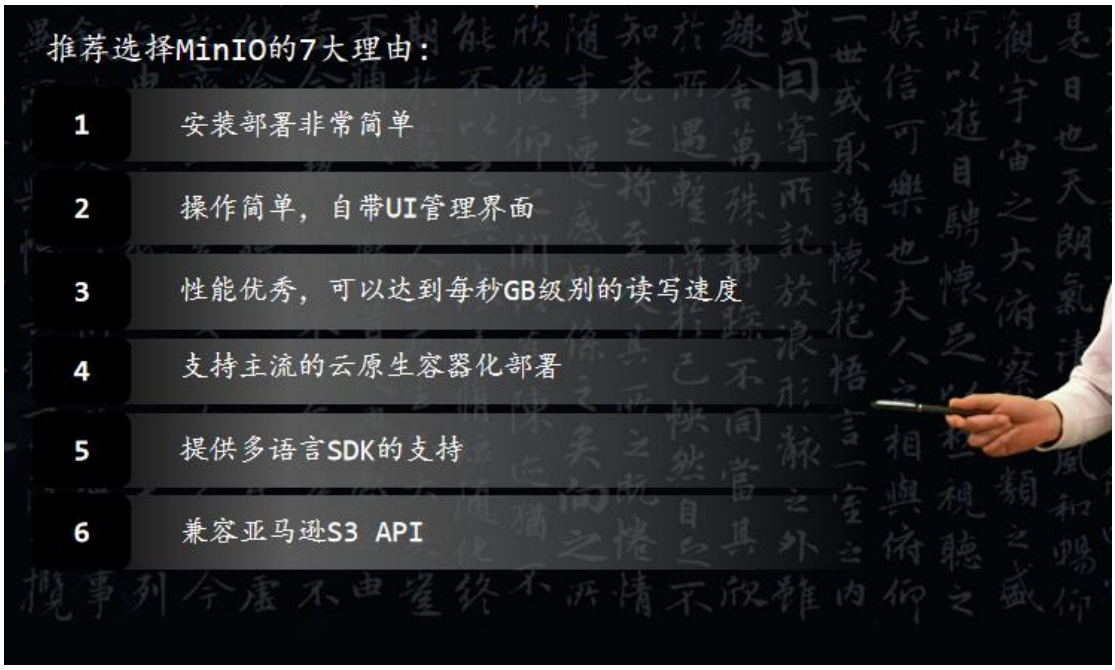
MinIO 提供了与 K8s、ETCD、Docker 等容器化技术深度集成方案，可以说 MinIO 就是为云环境而生的。

5）、提供多语言 SDK 的支持

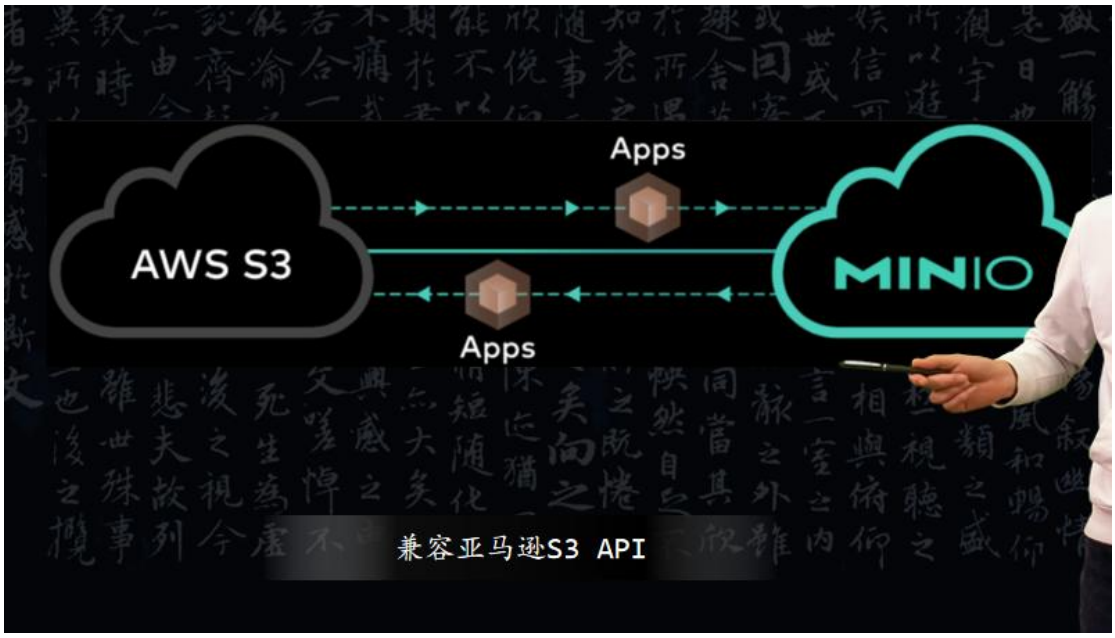


MinIO 几乎提供了全部主流开发语言的 SDK 以及文档，比如 Java、Python、Golang、JS、.NET 等等。

6）、兼容亚马逊 S3 API

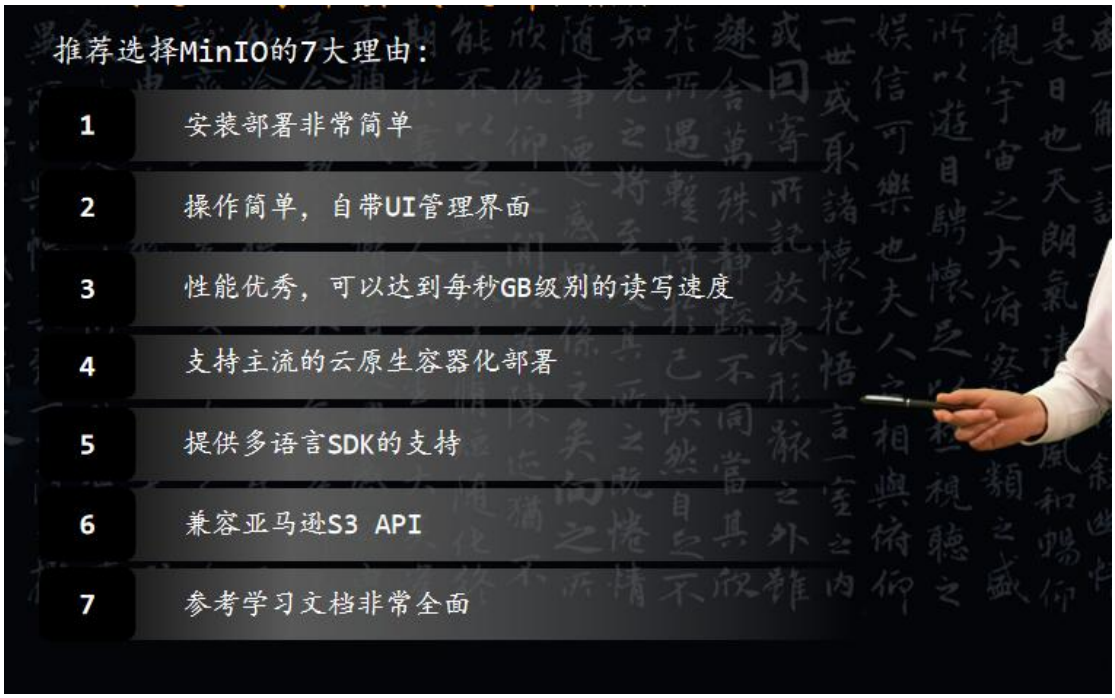


亚马逊云的 S3 API（接口协议） 是在全球范围内达到共识的对象存储的协议，是全世界认可的对象存储标准。而 MinIO 是第一个采用 S3 兼容协议的产品之一。



兼容 S3 API 有什么好处呢？相当于目前为了节约服务器成本，选择用 MinIO 自主开发对象存储系统，等企业壮大之后，不想再运维基础设施，可以直接将程序平移到云厂商，不需要重新开发。

7）、参考学习文档非常全面



官方提供了非常多的参考文档，而且 是全面开源的，在很多开源社区也非常活跃。

【结尾】

以上就是我对 MinIO 的理解。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.308 谈谈你对 Zookeeper 的理解

对一个没有接触过分布式的小伙伴来说，可能还不太清楚 Zookeeper 在分布式应用中的重要性，今天有位从传统行业转型到互联网的小小伙伴问我，希望跟他讲讲 Zookeeper 什么，能解决什么问 题？今天，我给大家详细介绍一下。

另外，我花了 1 个多星期，准备了一份 10W 字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

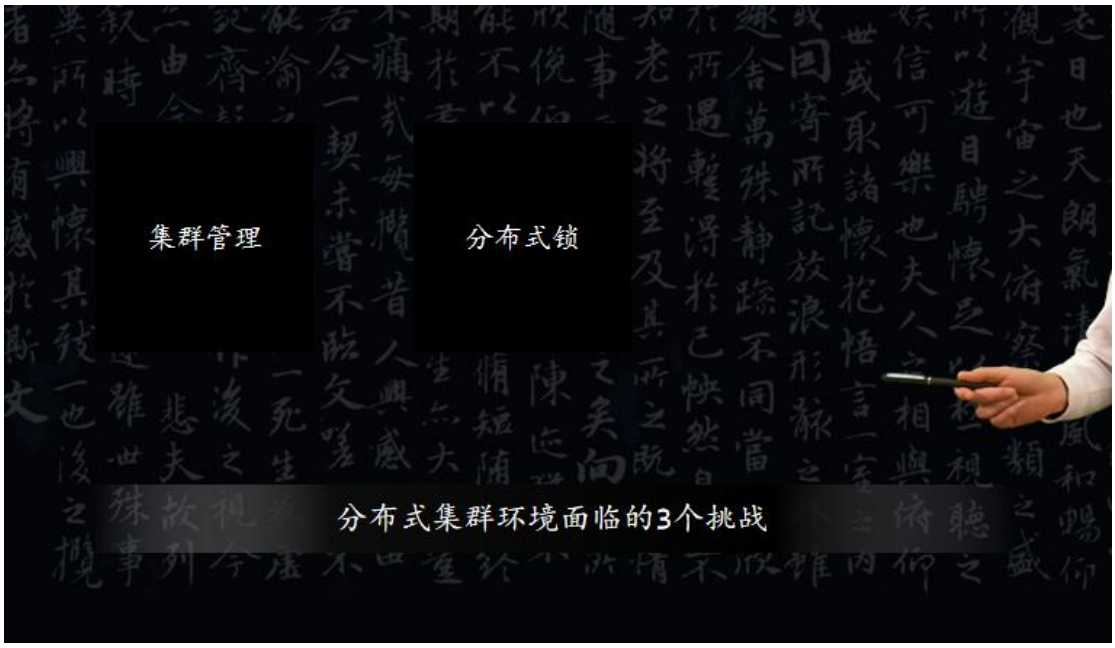
53.308.1 什么是 Zookeeper?

在讲什么 Zookeeper 之前，有必要先介绍一下分布式多个节点组成的集群环境中，所面临的 3 个常见的挑战。

第一种，集群管理

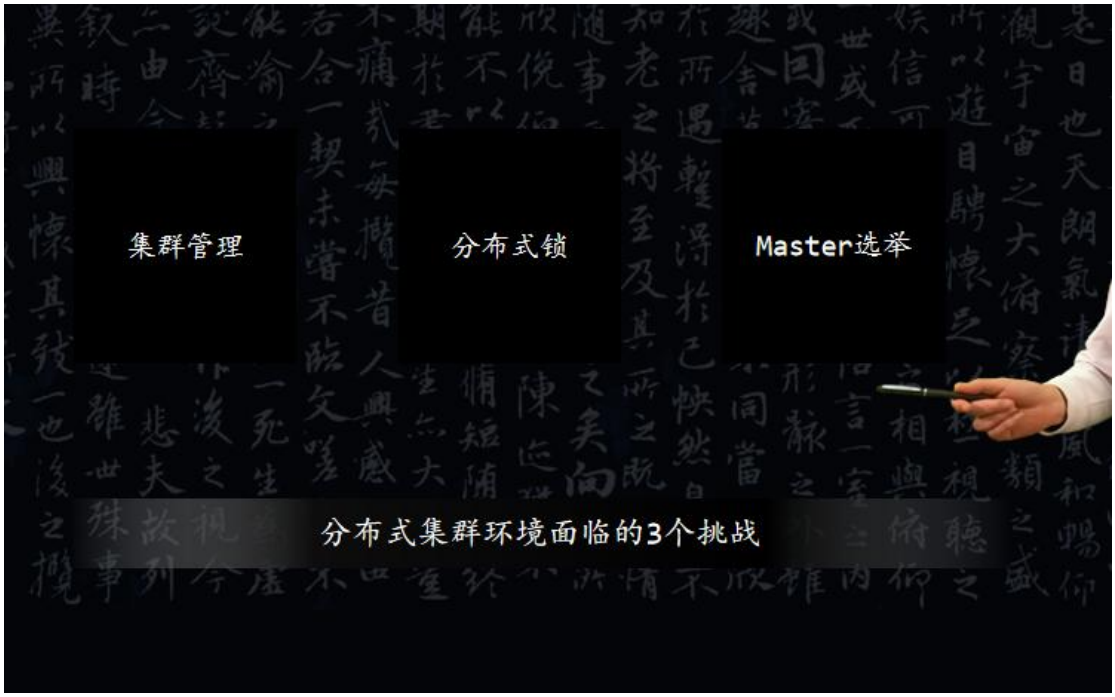
为了保证集群的高可用，每个节点都会创建一个数据副本。这种情况下，要保证客户端访问集群中的任意一个节点，都要是最新的数据。

第二种，分布式锁



对于分布式系统来说，如何保证共享资源的并发安全性也是一个比较大的挑战，为了达到这样一个目的，必须要使用跨进程的锁，也就是分布式锁来实现。

第三种，Master 选举



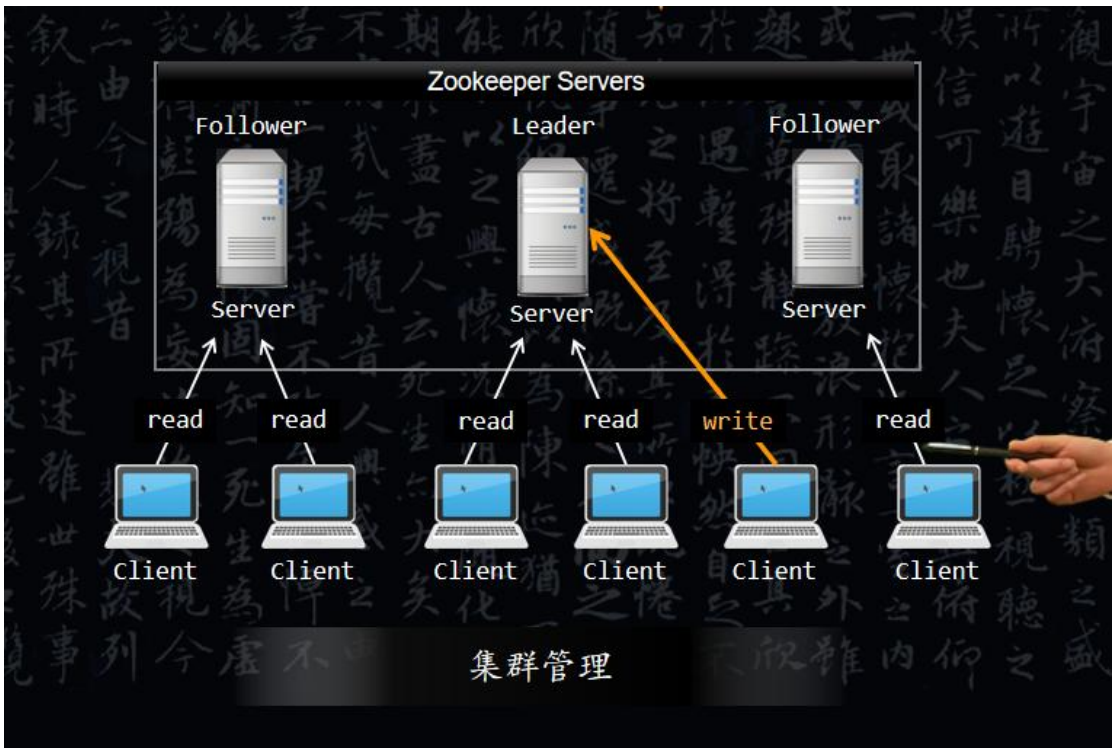
在多个节点集群中，为了降低集群数据同步的复杂度，一般会将节点设置为两个角色 Master 和 Slave，Master 一般负责数据的读写等事务性操作，而 Slave 只提供数据的读操作。但是，如何去确定某个节点是 Master 还是 Salve 也是一项比较复杂的工作。

Zookeeper 的出现，主要是为了满足分布式环境中，以上三种常见的场景需求，作为一个分布式的中间件而存在。它相当于是一个分布式开源的协调组件。简单理解，就相当于是一个裁判员的角色，专门负责协调和解决分布式系统中的各类问题。

53.308.2 分布式协调的基本原理

1、集群管理

Zookeeper 提供了 CP 模型，来保证集群中的每个节点的数据一致性。当然，ZK 本身的集群并不是强制一致性的，而是一个顺序一致性模型。如果我们需要去保证 CP 特性的话，我们需要调动“sync”同步方法进行同步。

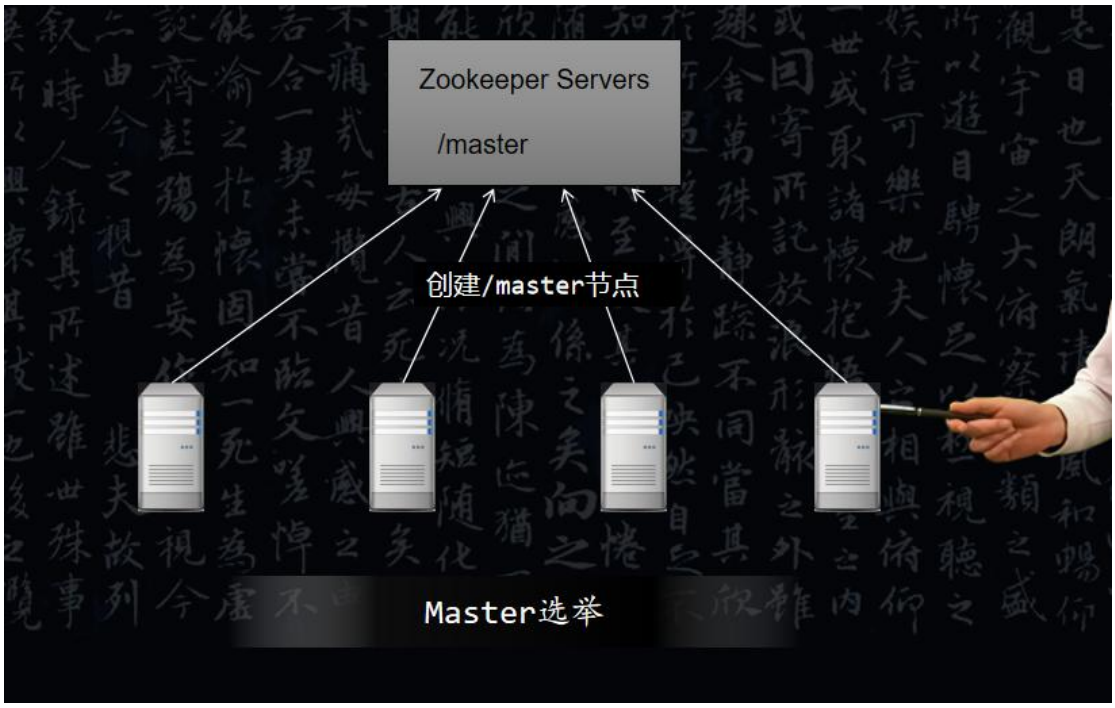


2、分布式锁

Zookeeper 提供了多种不同的节点类型，比如持久化节点、临时节点、有序节点和容器节点等等。对于分布式锁这个场景来说，Zookeeper 可以利用有序节点这样一个特性来实现。除此之外呢，还可以利用同一级节点的唯一特性来实现分布式锁。

3、Master 选举

Zookeeper 可以利用持久化节点来存储和管理其他集群节点的一些信息。从而去进行 Master 选举的一种机制，或者还可以利用集群中的一些有序节点的特性来实现 Master 选举。目前，主流的 Kafka、HBase、Hadoop 都是通过 Zookkeeper 来实现集群节点的主从选举。



53.308.3 总结

总的来说，Zookeeper 本就是一个经典的分布式数据一致性解决方案。它主要致力于分布式应用中的一些高性能、高可用，并且就严格访问顺序控制的一些能力模型，实现分布式的协调服务。它底层的数据一致性算法是基于 Paxos 算法演进而来的 ZAB 协议实现。

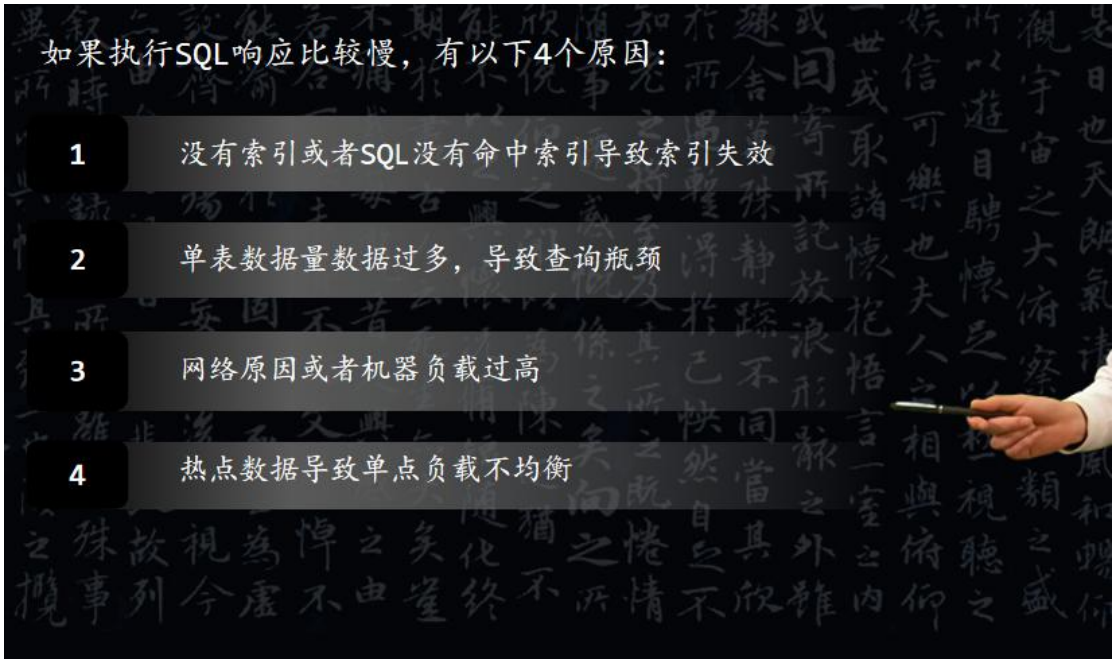
以上就是我对 Zookeeper 的理解。想更深入了解 Zookeeper Watch 机制、Leader 选举底层原理的小伙伴，可以在评论留言。

53.309 执行 SQL 响应比较慢，你有哪些排查思路?

如果面试问你，执行 SQL 响应慢，你有哪些排查思路和解决方案？这是一位去某里面试的小伙伴跟我分享的面试真题，那今天我给大家来分享一下我的思路。

53.309.1 排查思路

如果执行 SQL 响应比较慢，我觉得可能有以下 4 个原因：



第 1 个原因：没有索引或者 导致索引失效。

第 2 个原因：单表数据量数据过多，导致查询瓶颈

第 3 个原因：网络原因或者机器负载过高。

第 4 个原因：热点数据导致单点负载不均衡。

接下来，我针对以上几种情况，分别来聊一聊我的解决方案。

53.309.2 解决方案

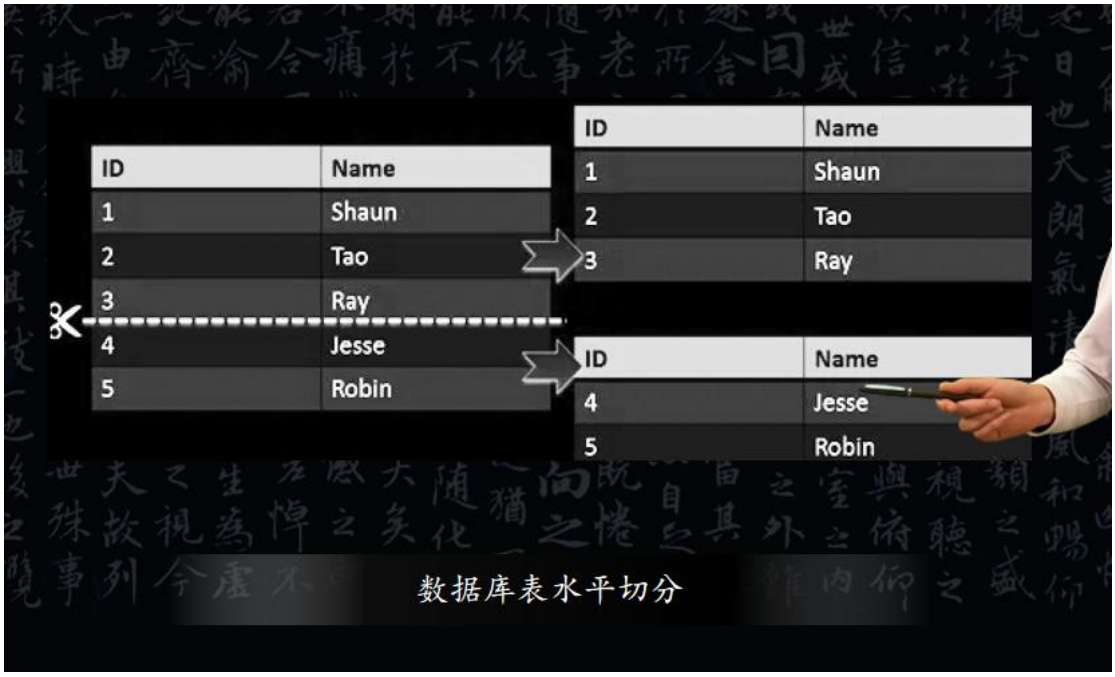
第 1 种情况：索引失效或者没有没有索引的情况

首先，可以打开 MySQL 的慢查询日志，收集一段时间的慢查询日志内容，然后找出耗时最长的 SQL 语句，对这些 SQL 语句进行分析。

比如可以利用执行计划 explain 去查看 SQL 是否有命中索引。如果发现慢查询的 SQL 没有命中索引，可以尝试去优化这些 SQL 语句，保证 SQL 走索引执行。如果 SQL 结构没有办法优化的话，可以考虑在表上再添加对应的索引。我们在优化 SQL 或者是添加索引的时候，都需要符合最左匹配原则。

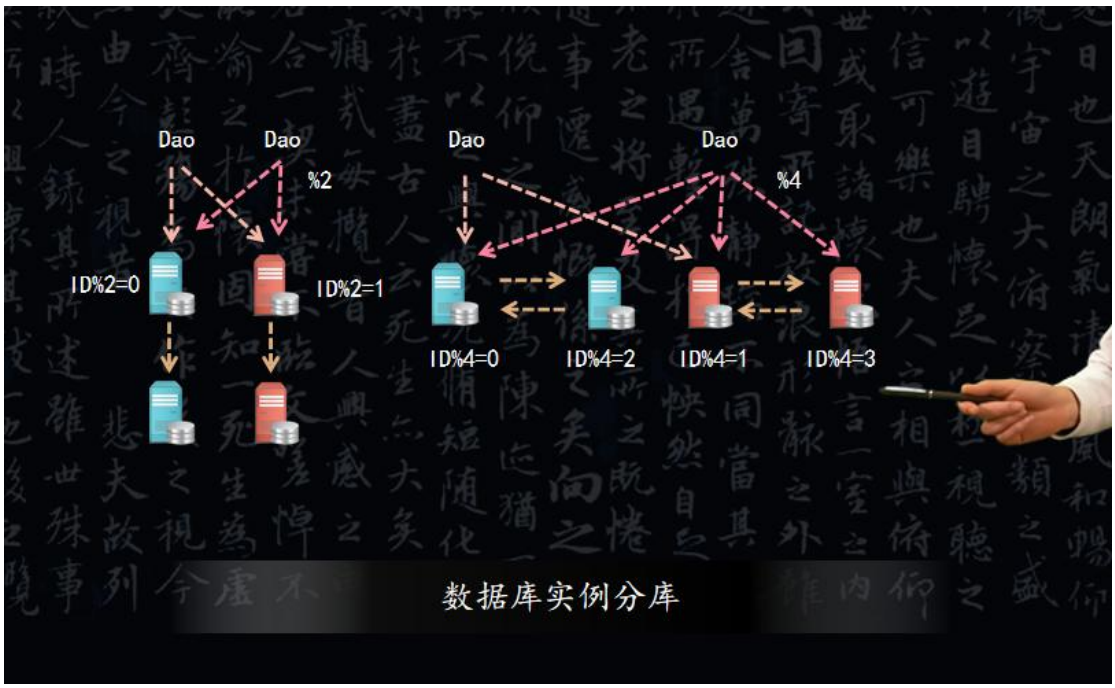
第 2 种情况：单表数据量数据过多，导致查询瓶颈的情况。即使 SQL 语句走了索引，表现性能也不会特别好。这个时候我们需要考虑对表进行切分。表切分规则一般分为两种，一种是水平切分，一种是垂直切分。

水平切分的意思是把一张数据行数达到千万级别的大表，按照业务主键切分为多张小表，这些小表可能达到 100 张甚至 1000 张。



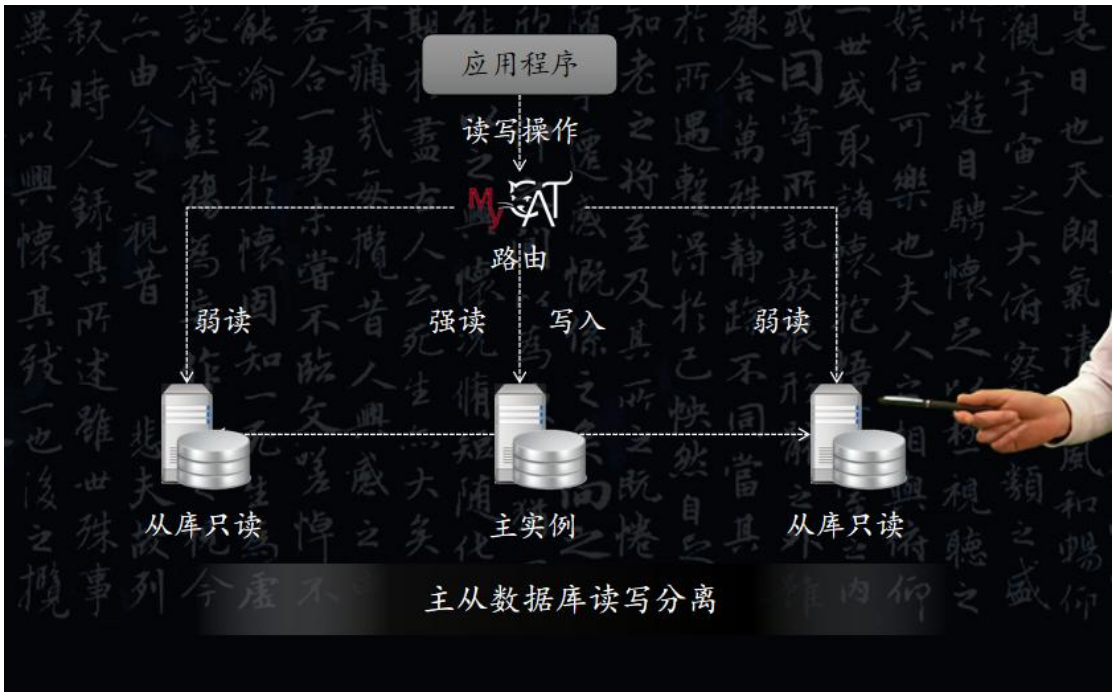
那垂直切分的意思是，将一张单表中的多个列，按照业务逻辑把关联性比较大的列放到同一张表中去。

除了这种分表之外，我们还可以分库，



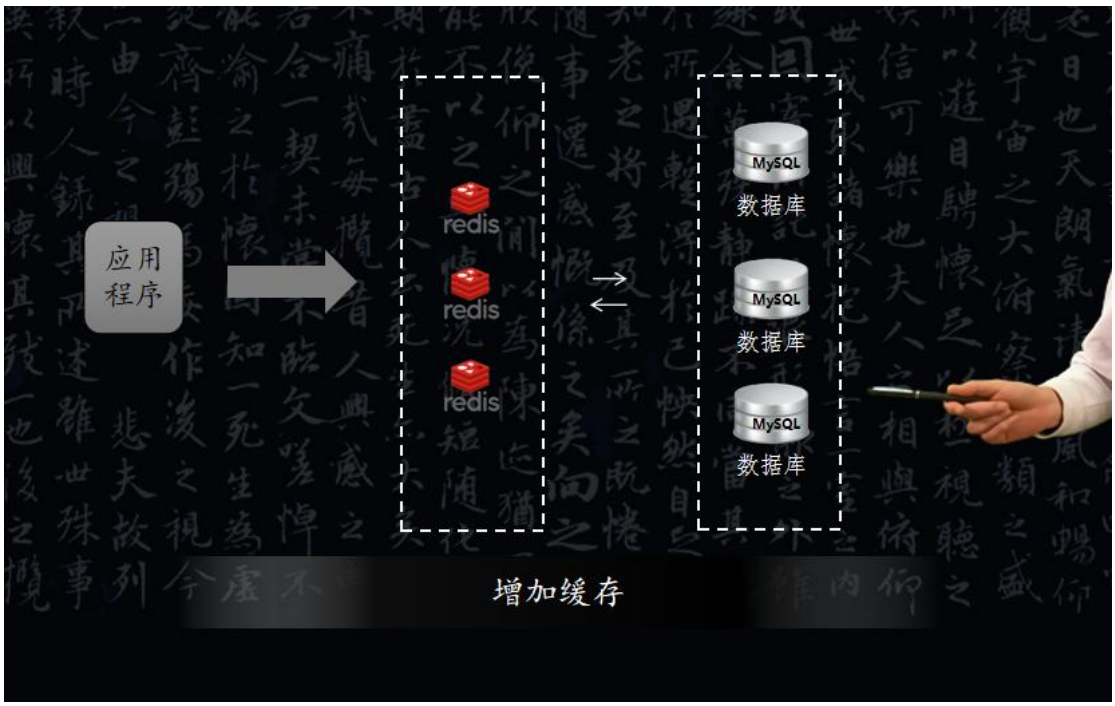
比如我们已经拆分完 1000 表，然后，把后缀为 0-100 的表放到同一个数据库实例中，然后，100-200 的表放到另一个数据库实例中，依此类推把 1000 表存放到 10 个数据库实例中。这样的话，我们就可以根据业务主键把请求路由到不同数据库实例，从而让每一个数据库实例承担的流量比较小，达到提高数据库性能的目的。

第 3 种情况：网络原因或者机器负载过高的情况，我们可以进行读写分离.



比如 MySQL 支持一主多从的分布式部署，我们可以将主库只用来处理写数据的操作，而多个从库只用来处理读操作。在流量比较大的场景中，可以增加从库来提高数据库的负载能力，从而提升数据库的总体性能。

第 4 种情况：热点数据导致单点负载不均衡的情况。



这种情况下，除了对数据库本身的调整以外，还可以增加缓存。将查询比较频繁的热点数据预存到缓存当中，比如 Redis、MongoDB、ES 等，以此来缓解数据的压力，从而提高数据库的响应速度。

【结尾】

以上就是我对执行 SQL 响应慢的排查思路和解决方案的理解，希望能够帮助到大家。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

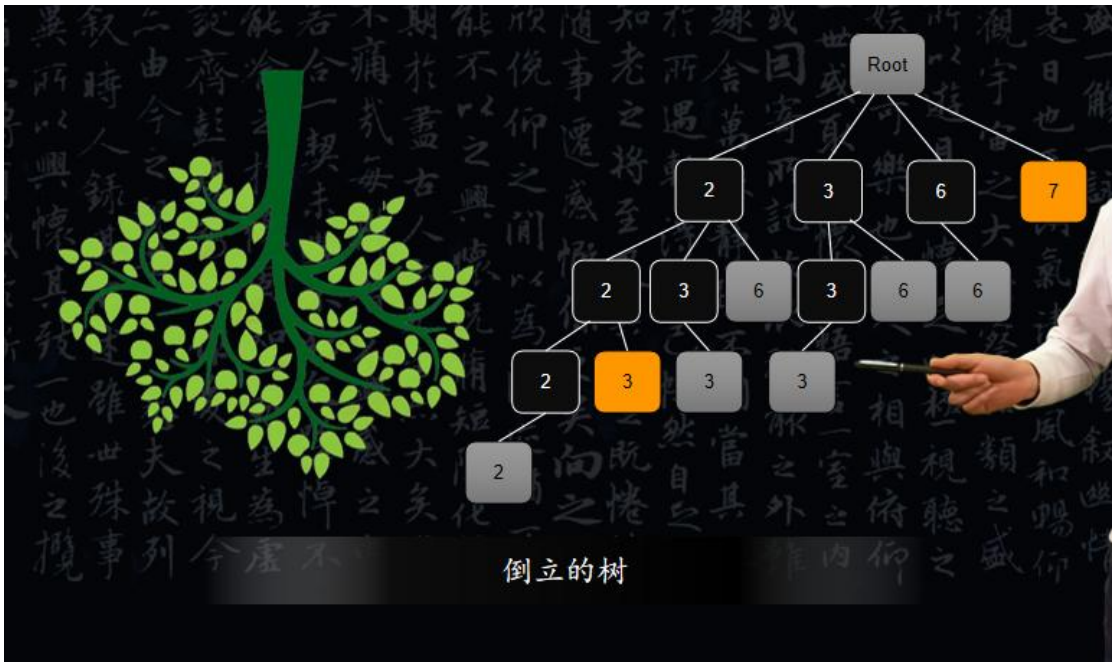
53.310 B 树和 B+树的区别什么?

今天，我给大家介绍一种面试中经常被问到数据结构树。大家可能也经常会听到二叉树、二叉查找树、AVL 平衡二叉树、B 树、 等等，那今天我给大家一次性讲清楚。

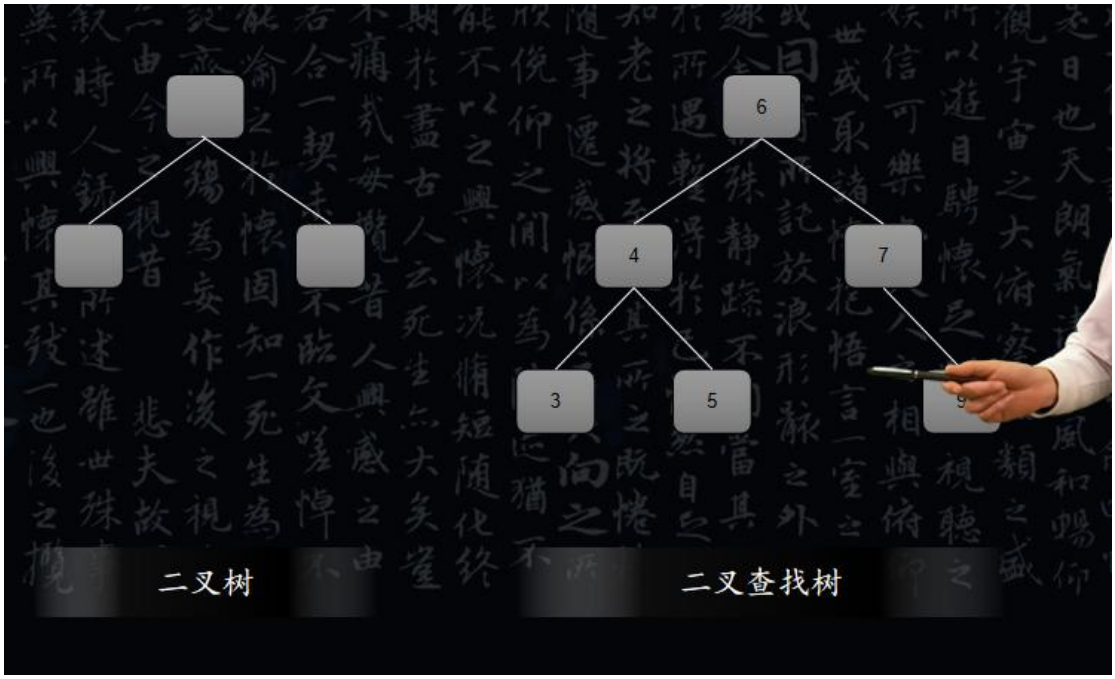
53.310.1 树形结构的演变历史

树是一种数据结构，它的结构形状如同一棵树木，但是是倒立的状态。

树的分叉就相当于树形数据结构中的节点，树上的节点可以从树根无限延伸。如图所示：

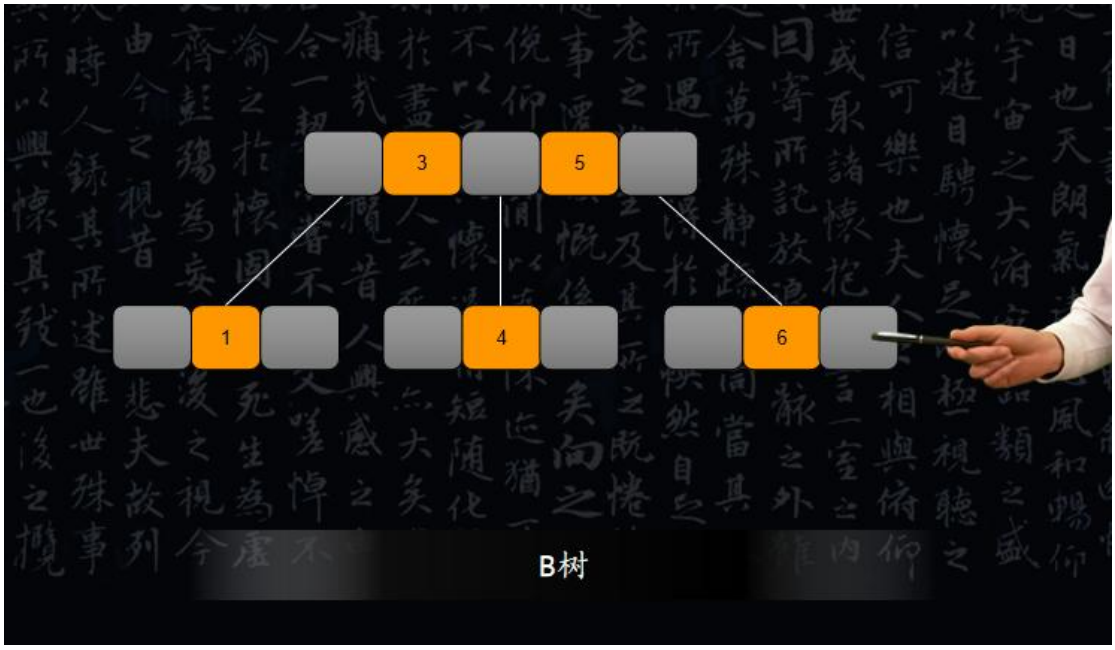


但是，这种无限延伸的树在实际开发中一般用于可无限扩展的树形功能菜单。这样的无限级、无限宽扩展的结构查询性能会比较低。因此，为了提高树形结构的查询性能，于是就出现了二叉树，来看这么一张图：



所谓二叉树是指每个节点最多支持两个分叉，相比于单向链表来说它多了一个分支。而二叉查找树，是在二叉树的基础上增加一个规则。它的规则是左子树的所有子节点都要小于它的根节点，而右侧子节点要大于它的根节点。

我们再来看这个图，二叉查找树有可能会出现斜树问题，从而导致时间复杂度会增加。因此，又引入了一种叫做平衡二叉树的机制。它具有二叉查找树的所有特点，同时还增加了一个规则。它的规则是左右两个子树的高度差的绝对值不能超过 1。 为了达到这样一个平衡，所以它会引入一个左旋和右旋的机制，来实现树的平衡。



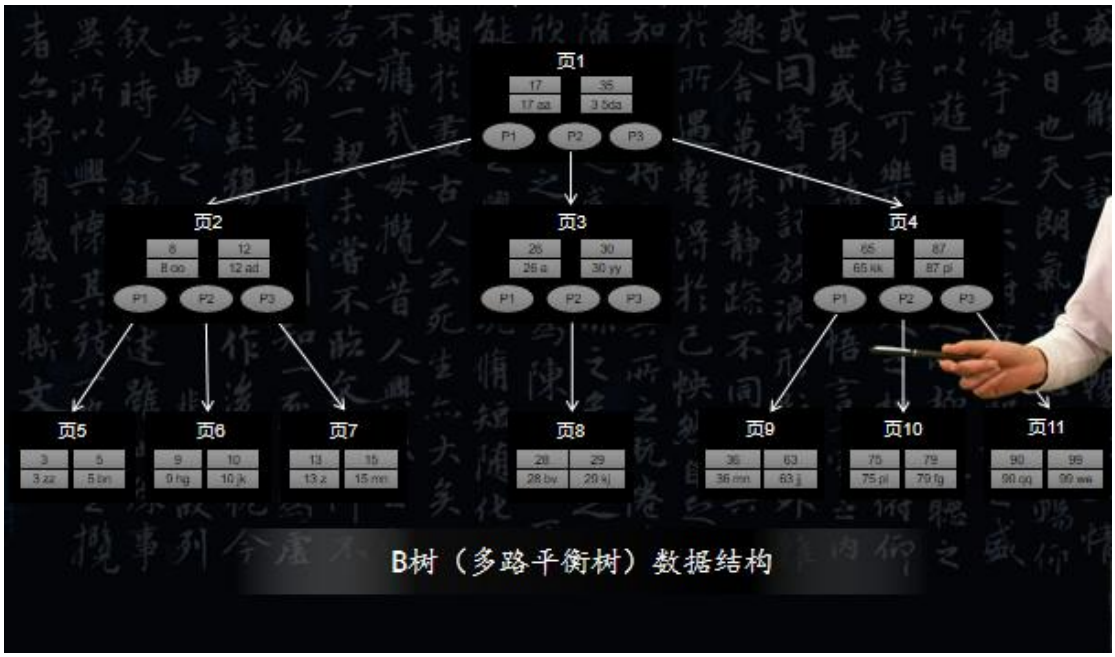
我们再来看这张图，这是 B 树。它是一种多路平衡查找树。它既满足平衡二叉树的规则，又可以有多个子路。子路的数量呢取决于它的关键字的数量。如图所示，根节点中有两个关键字 3 和 5，那么它能够拥有的子路数量等于关键字的数量加 1。因此，从这个特征来看，在存储同样数量的情况下，平衡二叉树的高度一定要大于 B 树的高度。

53.310.2 B 树和 B+树的区别

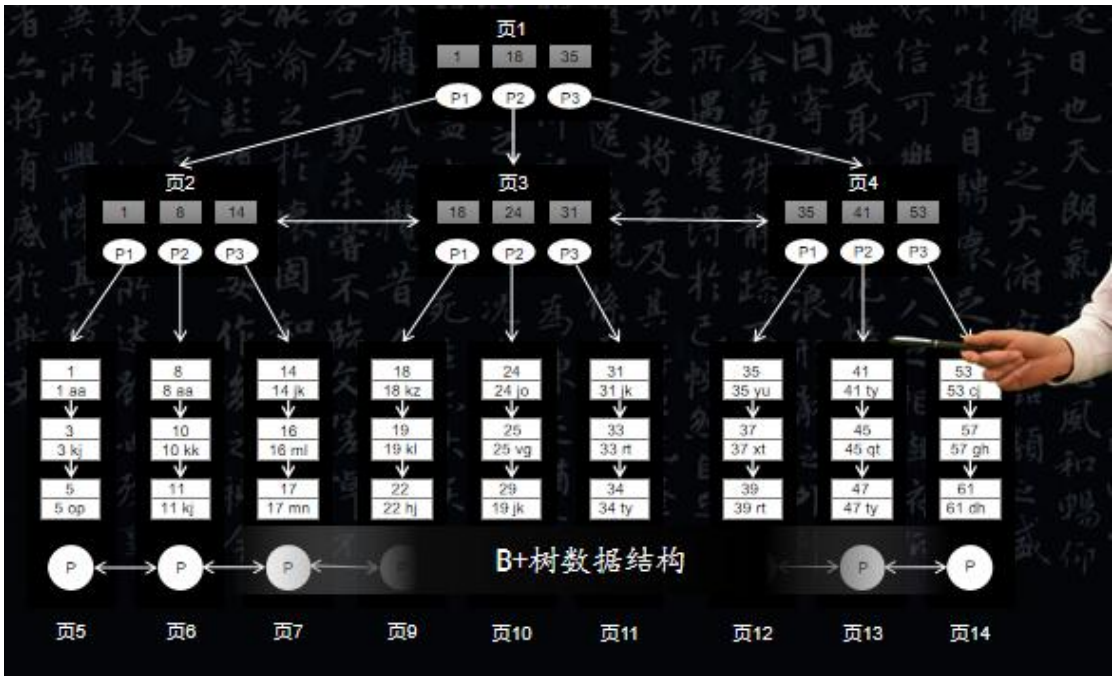
B+树呢，其实是在 B 树的基础上做了增强，和 B 树有两个最大的区别：

第 1 个：B 树的数据存储在每个节点上，而 B+树中的数据只存储在叶子节点上，并且通过链表的方式将所有叶子节点全部串联起来

第 2 个：B+树的子树数量等于它的关键字的数量，而 B 树是关键字数量加 1。我们来看这个图：



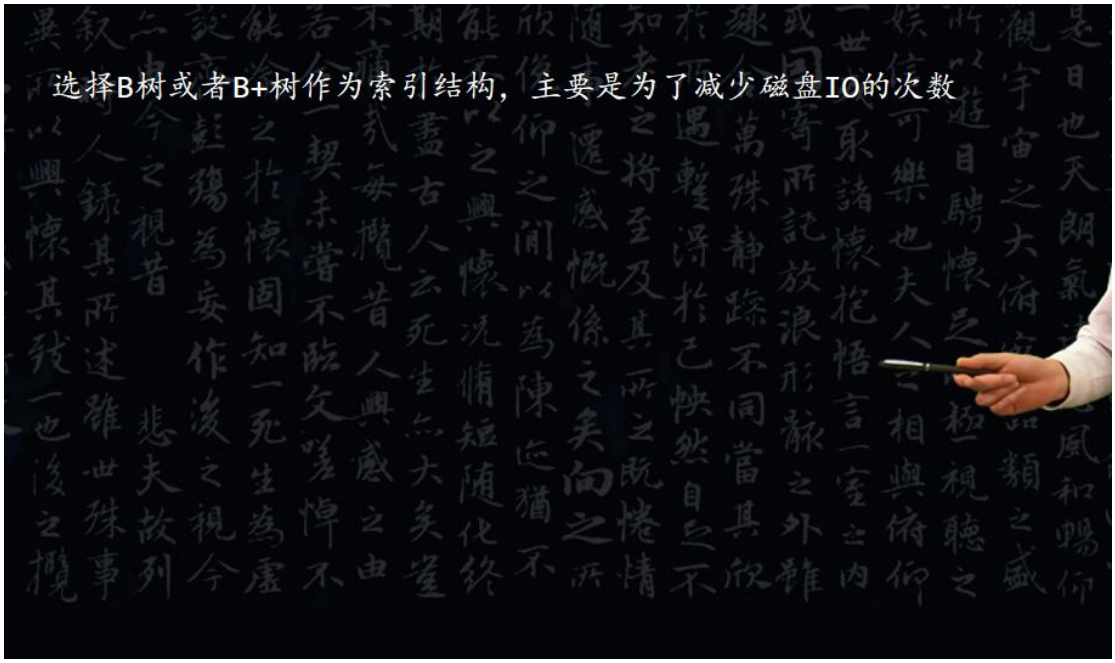
这个是 B 树的存储结构。从 B 树的结构上可以看到，每个节点都会存储数据。我们再来看这个图：



这是一个 B+树的结构。B+树的数据是存储在叶子节点上的，并且呢，叶子节点的数据是用双向链表来关联。

53.310.3 选择 B 树和 B+树的理由

那为什么要用 B 树或者 B+树来做索引结构呢?



因为 AVL 树的高度要比 B 树或者 B+树的高度更高，而高度就意味着磁盘 IO 的数量，所以，为了减少磁盘 IO 的次数，文件系统或者数据库才会使用 B 树，或者 B+树来做索引结构。

在比较经典的程序应用中，MongoDB 使用的是 B 树，MongoDB 中所有的节点都有 Data 域，只要找到指定索引就可以进行访问，无疑单次查询会更快。

而 MySQL 作为一个关系型数据库，数据的关联性是非常强的，区间访问是常见的一种情况，B+树由于数据全部存储在叶子节点，并且通过指针串在一起，这样就很容易的进行区间遍历甚至全部遍历。

【结尾】

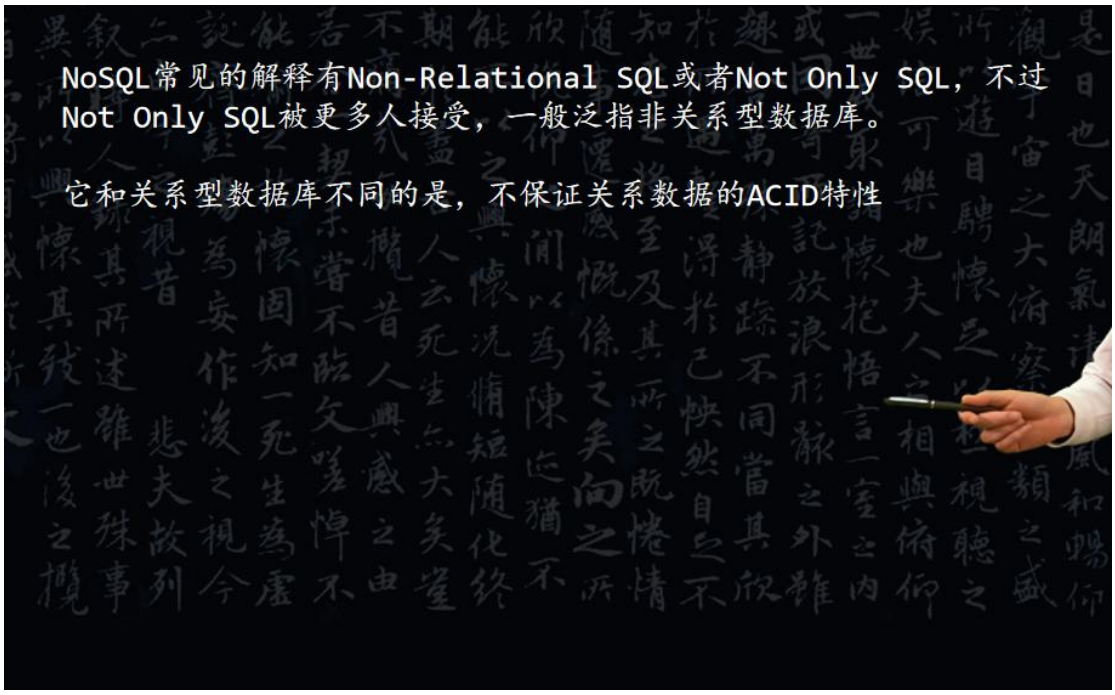
以上就是我对 B 树和 B+树的理解。程序的本质就是数据结构加算法。数据结构在实际开发中非常常见，比如数组、链表、双向链表、红黑树、跳跃表、B 树、B+树、队列等。所以，数据结构是编程最重要的基本功之一，很多大厂面试也经常会问到。同时，基本功也是决定大家在技术路上能够达到的高度的重要因素。

53.311 谈谈你对 NoSQL 的理解

有粉丝给我留言说，希望我更新一期关于 NoSQ 的视频，那今天，咱们就来聊一聊我对 NoSQL 的理解。如果你也有想要我分享的面试题，请在评论区告诉我，后续安排。

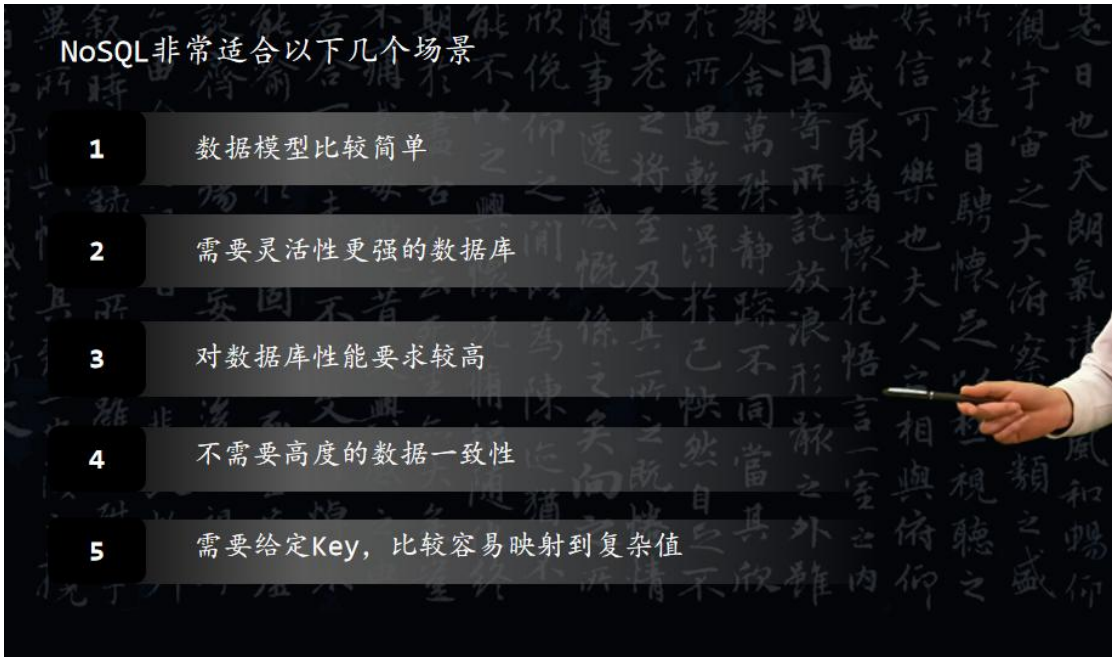
53.311.1 什么是 NoSQL?

NoSQL 呢常见的解释有 Non-Relational SQL 或者 Not Only SQL，不过 Not Only SQL 被更多人接受，一般泛指非关系型数据库。它和关系型数据库不同的是，不保证关系数据的 ACID 特性。



随着互联网的发展，NoSQL 数据库的产生就是为了解决超大规模和高并发系统中多重数据种类带来的挑战，特别是大数据应用的难题。目前具有代表性的 NoSQL 数据库有 Redis、HBase、ES、MongoDB 等等。

NoSQL 非常适合以下几个场景：



1、数据模型比较简单；

2、需要灵活性更强的数据库；

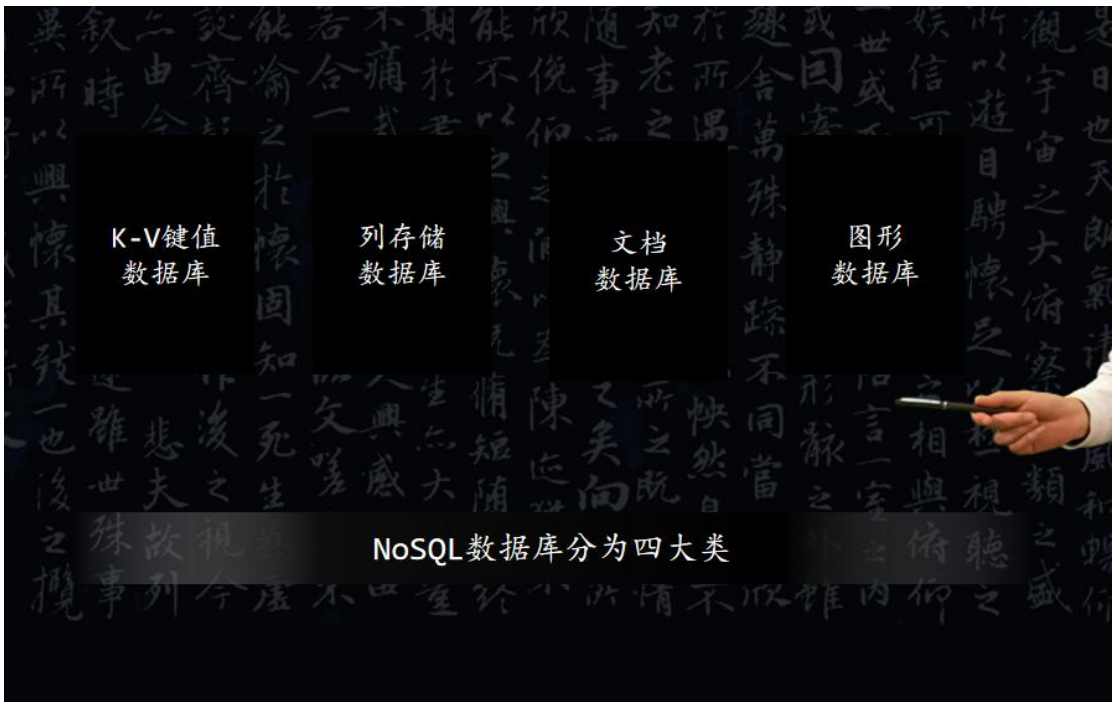
3、对数据库性能要求较高；

4、不需要高度的数据一致性；

5、对于给定 Key，比较容易映射复杂值的环境。

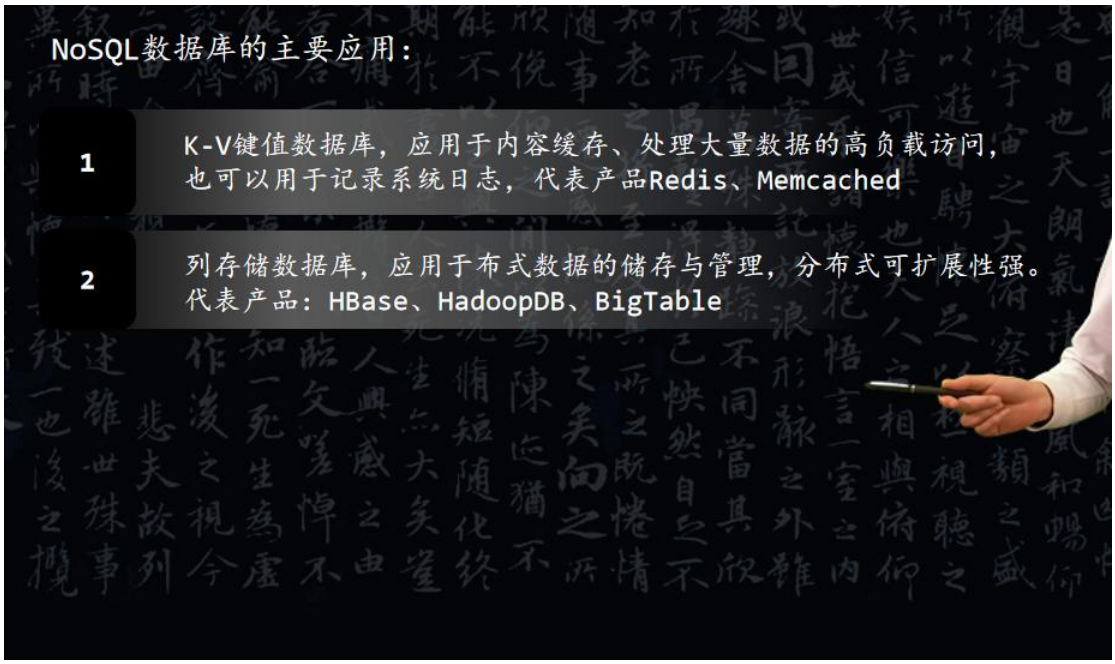
53.311.2 NoSQL 分类

通常来说，现在主流的 NoSQL 数据库可以分为四大类：K-V 键值数据库、列存储数据库、文档数据库和图形数据库。

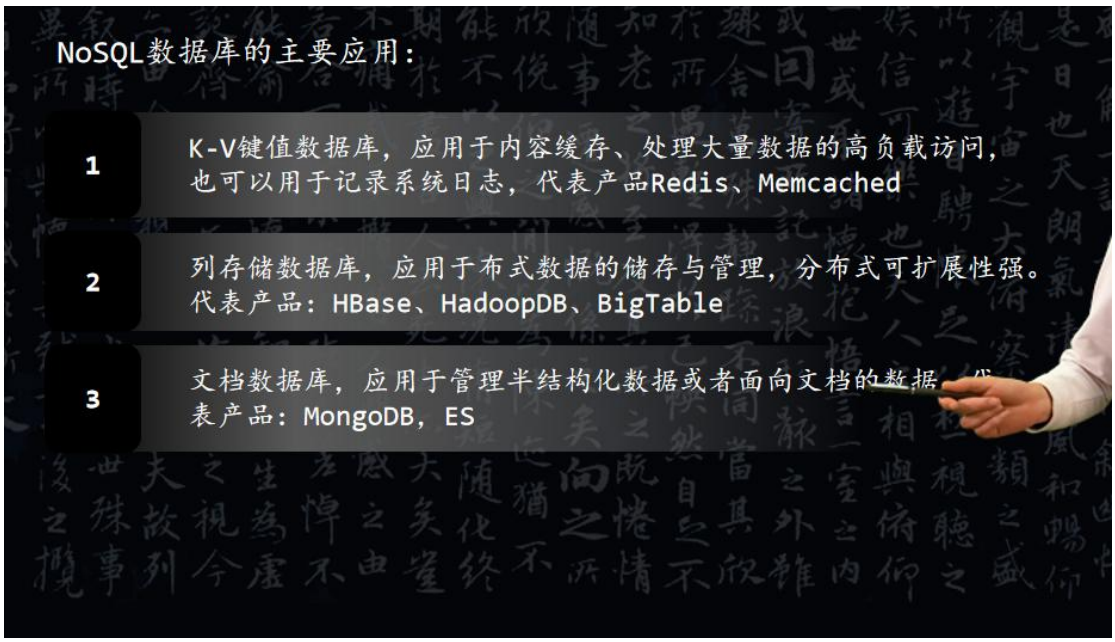


首先来看，K-V 键值数据库，主要应用于内容缓存、处理大量数据的高负载访问，也可以用于记录系统日志。比较有代表性的产品有 Redis、Memcached 等等。

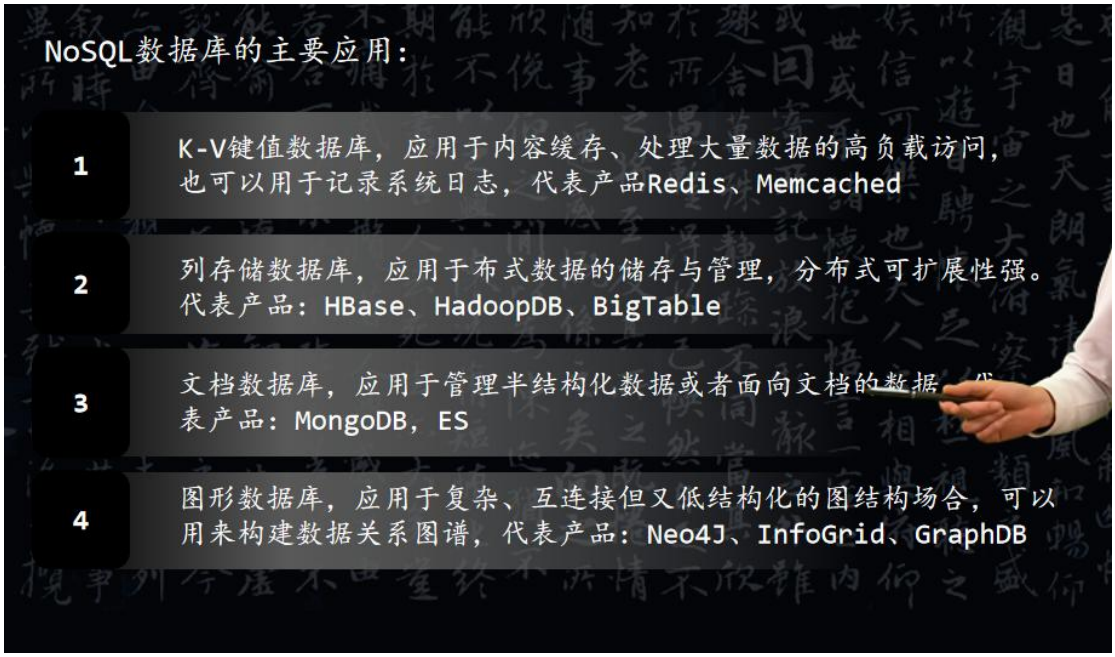
其次是，列存储数据库，主要应用于布式数据的储存与管理，分布式可扩展性强，比较有代表性的产品有 HBase、HadoopDB、BigTable 等等。



然后就是，文档数据库，主要应用于管理半结构化数据或者面向文档的数据，比较有代表性的产品有 MongoDB，ES 等等，当然，ES 不仅仅只是数据库，它还是一个分布式搜索引擎，是基于 Lucene 来开发的，现在已经发展成为一个自有生态 Elastic Stack。



最后就是，图形数据库，主要应用于复杂、互连接但又低结构化的图结构场合，可以用来构建数据关系图谱，Neo4J、InfoGrid、GraphDB 等等。

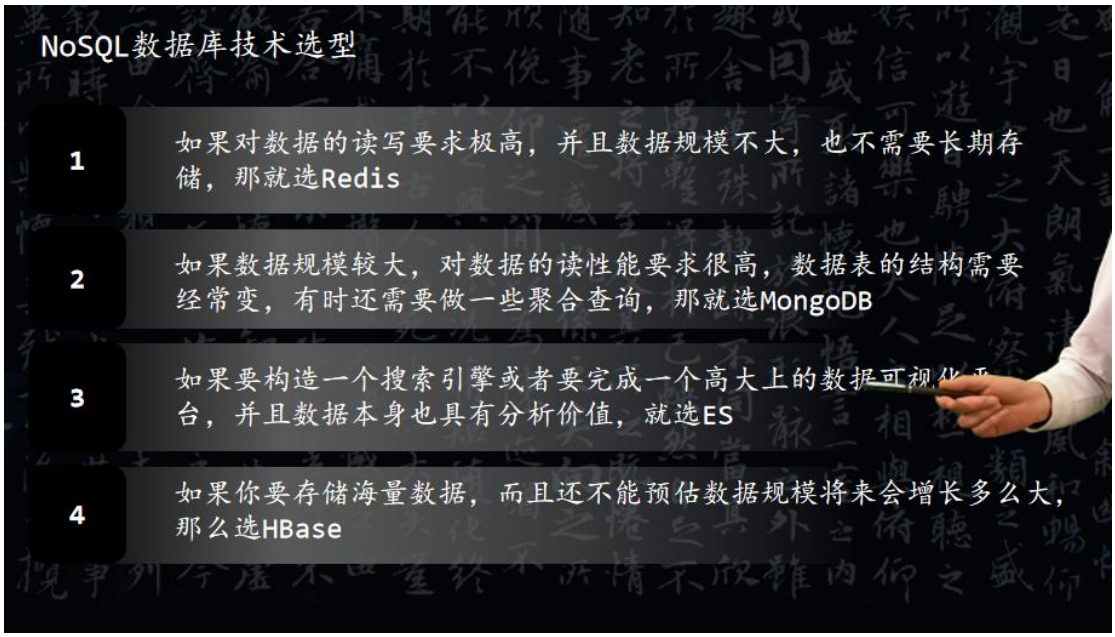


下面我把四种分类的 NoSQL 数据库整理成了一个表格，并总结了它们的优缺点

分类	代表产品	应用场景	数据模型	优点	缺点
K-V键值	Redis、Memcached	内容缓存、处理大量数据的高负载访问，记录系统日志	Key指向Value的键值对，通常用Hash Table来实现	查找速度快	数据无结构化，通常只被当作字符串或者二进制数据
列存储数据库	HBase、HadoopDB、BigTable	分布式数据的储存与管理，分布式可扩展性强	以列或列簇式存储，将同一列数据存在一起	查找速度快，可扩展性强，容易进行分布式扩展	功能相对局限
文档数据库	MongoDB，ES	管理半结构化数据或者面向文档的数据	Key-Value对应的键值对，Value为结构化数据	数据结构灵活，表结构可变，复杂性低	查询效率低，且缺乏统一的查询语言
图形数据库	Neo4J、InfoGrid、GraphDB	复杂、互连接但又低结构化的图结构场合，用来构建数据关系图谱	图结构	利用图结构相关算法，可用于构建复杂的关系图谱	复杂度高

53.311.3 主流 NoSQL 对比

对于 NoSQL 数据库对互联网开发的重要性就不多说了，在互联网分布式开发场景中，我认为掌握 Redis、HBase、Elastic、MongoDB 就能满足绝大部分需求。

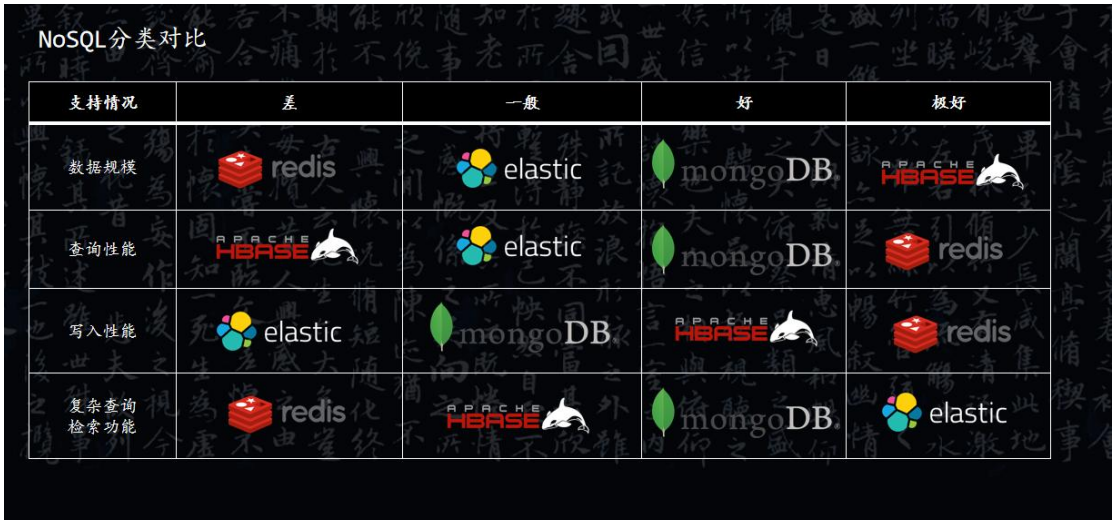


如果对数据的读写要求极高，并且数据规模不大，也不需要长期存储，那就选 Redis；如果数据规模较大，对数据的读性能要求很高，数据表的结构需要经常变，有时还需要做一些聚合查询，那就选 MongoDB；

如果要构造一个搜索引擎或者要完成一个高大上的数据可视化平台，并且数据本身也具有分析价值，就选 ES；

如果你要存储海量数据，而且还不能预估数据规模将来会增长多么大，那么选 HBase。

这里我也整理了一种表格，大家可以保存一下



最后，再给大家来个更加形象的对比：

Redis 就相当于开战斗机，MongoDB 就相当于坐高铁，HBase 就相当于航母，ES 那就是帝国铁骑，山地装甲机器人

【结尾】

好了，以上就是我对 NoSQL 的理解。我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.312 为什么 SQL 语句命中索引比不命中索引要快?

有位粉丝面试高开的时候被问到，为什么 SQL 语句命中索引比不命中索引要快？虽然自己也知道答案，但被问到的瞬间，就不知道如何组织语言了。今天，我给大家深度分析一下。

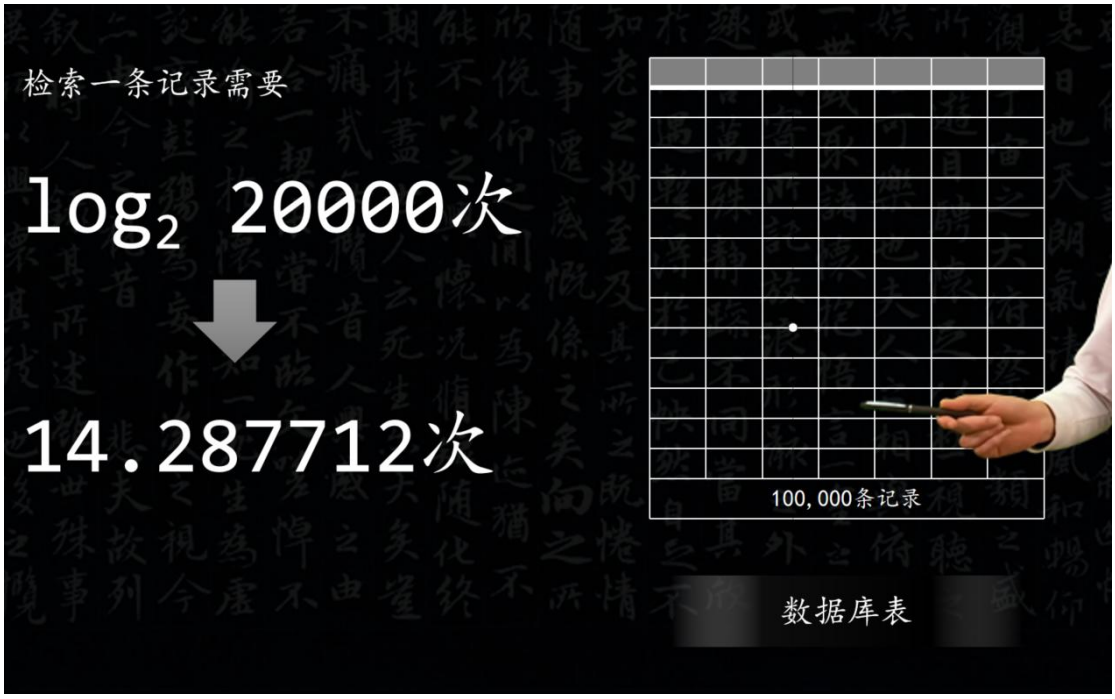
53.312.1 索引的作用

想象一下，现在有一本包含几十万字的字典，有几百页厚，同时里面的字是无序排列的。如果在不使用目录的情况下，我们如何从字典中找出需要的字来呢？毫无疑问，我们只能一页一页的翻，显然，这是一项反人类的的工作。

我们必然想的是先看目录，然后，找到相关的字或者偏旁，然后，找到对应的页码再去查找想要找的文字，这样，效率就大大提高了。而事实上，目录就是一种索引，我们说的数据库索引思想和目录的思想一脉相承。

数据库索引最主要的作用就是帮助我们快速检索到想要的数据，从而不至于每次查询都做全局扫描。

假设不使用任何算法的情况下，我们要查询 10 万条记录中的某一条，在最坏的情况下需要遍历 10 万次。

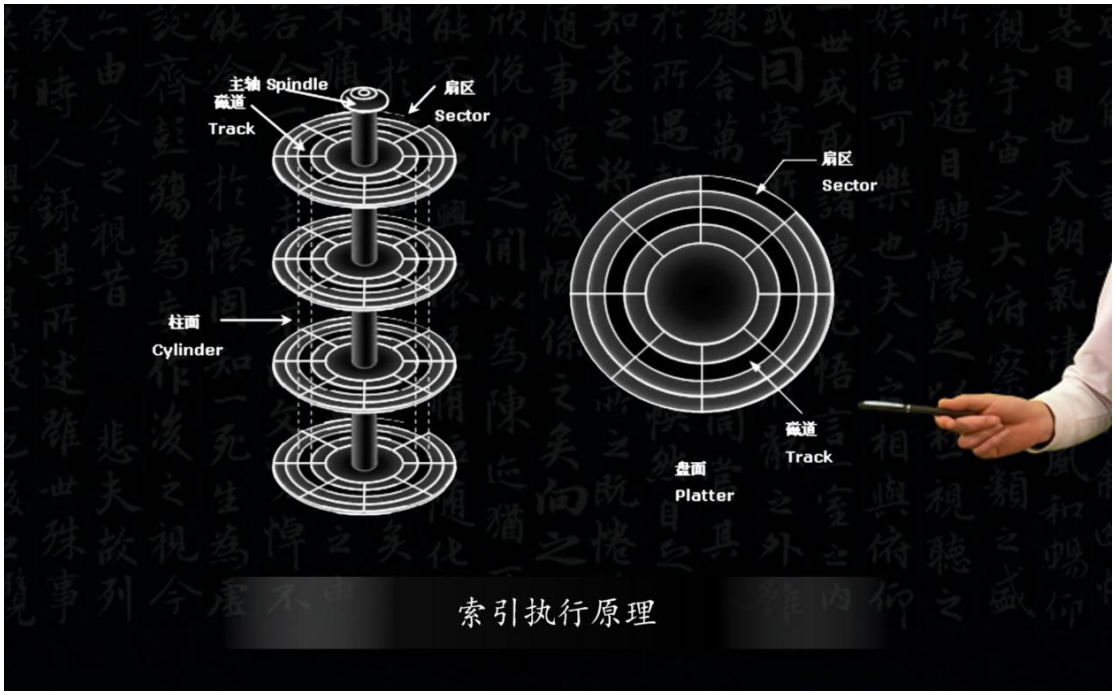


但如果使用二分查找算法，则只需要进行 log2 20000 次，也就是 14.287712 次即可。这意味着我们只需对排序后的值进行 14 次搜索，就可以使用二分查找到想要的唯一值，常见的索引数据结构有 B 树和 B+树。

下面我们，以 MySQL 的 InnoDB 引擎为例，分析一下索引的工作原理。

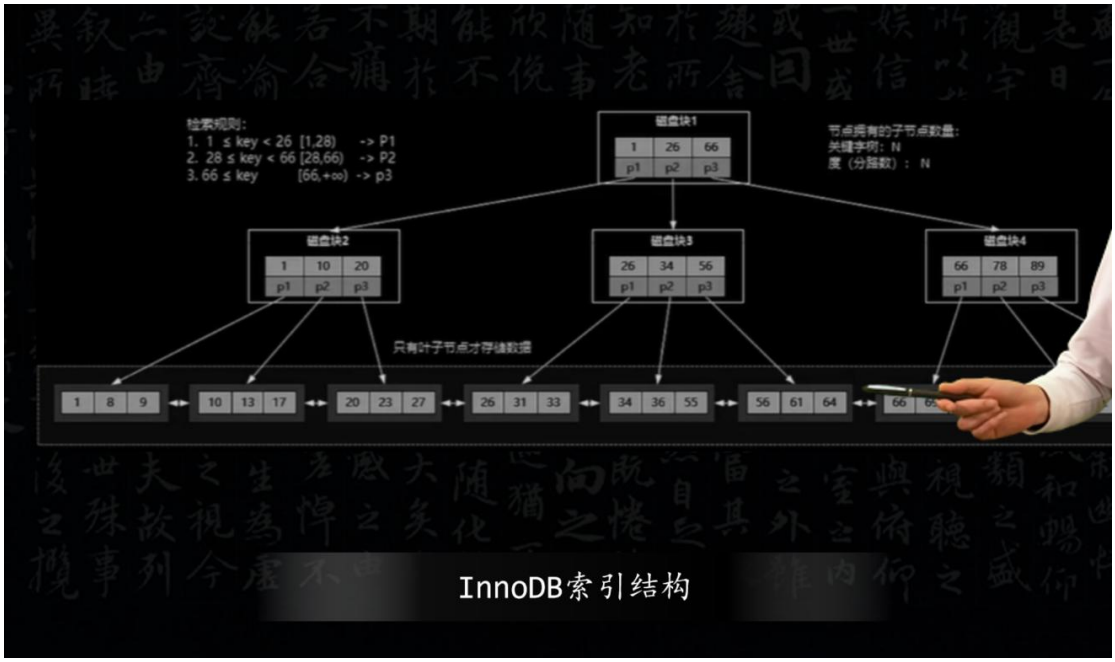
53.312.2 索引执行原理

我们知道 MySQL 的 InnoDB 引擎采用的是 B+树数据结构，当我们去执行 SELECT 语句查询数据的时候，InnoDB 需要从磁盘上去读取数据，而这个过程会涉及到磁盘 以及磁盘的随机 IO ，我们来看这么一个图：



系统会把数据的逻辑地址传给磁盘，磁盘控制线路按照寻址逻辑把逻辑地址翻译成物理地址。也就是确定要读取的数据在哪个磁道、哪个扇区。为了读取这个扇区的数据，需要把磁头放在这个扇区上面，为了实现这样一个点，磁盘会不断地去旋转。把目标扇区旋转到磁头下面，使得磁头能够去找到对应的磁道。这里还会涉及到寻道的时间以及旋转时间的一个损耗。很明显磁盘 IO 这个过程的性能开销是非常大的，尤其是查询的数据量比较多的情况下。

所以 InnotDB 里面，干脆对存储在磁盘上的数据建立一个索引，然后把索引数据以及索引列对应的磁盘地址以 B+树的方式进行存储。来看这么一个图：



当我们需要查找目标数据的时候，根据索引从 B+树中去查找目标数据就行了。由于 B+树的子树比较多，所以，只需要较少次数的磁盘 IO 就能够查找到目标数据。

至于 B+树的数据结构，在这里就不分析了。大家可以去我的个人主页看往期视频有讲到。

3、索引的弊端

虽然，使用索引能减少磁盘 IO 次数，提高查询效率，但是，索引也不能建立太多。如果一个表中所有字段的索引很大，也会导致性能 l 下降。想象一下，如果一个索引和一个表一样长，那么它将再次成为一个需要检查的开销。这就好比字典的目录非常详细，但是其长度已经和所有的文字一样长，这个时候目录本身的效率就大大下降了。

那索引有弊端吗？肯定是有的，索引可以提高查询读取性能，而它会将降低写入性能。当有索引时，如果更改一条记录，或者在数据库中插入一条新的记录，它将执行两个写入操作（一个操作是写入记录本身，另一个操作是将更新索引）。

因此，在定义索引时，必须牢记以下几点：

1、索引表中的每个字段将降低写入性能。

2、建议使用表中的唯一值为字段编制索引。

3、在关系数据库中充当外键的字段必须建立索引，因为它们有助于跨多个表进行复杂查询。

4、索引还使用磁盘空间，因此在选择要索引的字段时要小心。

【结尾】

我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.313 如何安全地中断一个正在运行的线程?

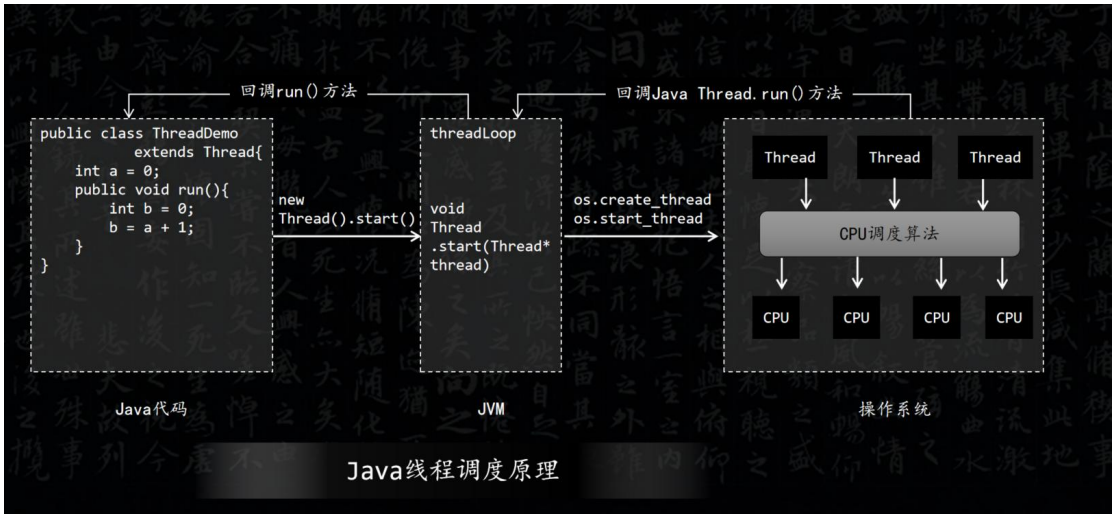
一个位 5 年的小伙伴去某东面试被一道并发编程的面试题给 Pass 了，说”如何中断一个正在运行中的线程？，这个问题很多工作 2 年的都知道，实在是有些遗憾。

今天，我给大家来分享一下我的回答。

53.313.1 什么是线程?

回答这个问题之前，先来回顾一下什么线程?

Thread，线程是操作系统进行运算调度的最小单位。所以，线程是系统级别的概念。



而在 Java 里面实现的线程，最终的执行和调度都是由操作系统来决定的，JVM 只是对操作系统层面的线程做了一层包装而已。

所以我们在 Java 里面调用 start() 方法启动一个线程的时候，只是告诉操作系统这个线程可以被执行，但是最终交给 CPU 来执行，是由操作系统的调度算法来决定的。

53.313.2 如何操作如何安全中断运行中的线程?

从理论上来说，要在 Java 层面去中断一个正在运行的线程，只能像类似于 Linux 里面的 kill 命令结束进程的方式一样，强制终止。

Java Thread 的 API 里面虽然提供了一个 stop() 方法，可以强行终止线程，但是这种方式是不安全的，因为有可能线程的任务还没有完成，突然中断会导致出现运行结果不正确的问题。

要想安全的中断一个正在运行的线程，只能在线程内部埋下一个钩子，外部程序 通过这个钩子来触发线程的中断命令。

因此，在 Java Thread 里面提供了一个 interrupt() 方法，这个方法要配合 isInterrupted()方法来使用，就可以实现安全地中断线程运行。

来看这段代码：

```
Runnable runnable = new Runnable(){
    public void run() {
        while (true) {
            if (Thread.currentThread().isInterrupted()){
                System.out.println("线程被中断了");
                return ;
            } else {
                System.out.println("线程没有被中断");
            }
        }
    }
};
Thread t = new Thread(runnable);
```

```
14  t.start();
15  Thread.sleep(500);
16  t.interrupt();
17  System.out.println("线程中断了，程序到这里了");
```

这种实现方法并不是强制中断，而是告诉正在运行的线程，你可以停止了。何时实际中断，取决于正在运行的线程，所以，它能够保证线程运行结果的安全性。

【结尾】

这个问题，很多工作了 5 年以上的小伙伴都不一定清楚。虽然 CRUD 能够熟练上手，但是，和其他人拉开差距的往往是工作之外的 8 个小时。

我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.314 谈谈你对 Spring Cloud 的理解

最近，有很多应届生也挺焦虑的，说大家都卷，不卷也不行。很多应届生在面试过程中八股文背得不错，但是一聊到项目经验就不知如何说起。

有位小伙伴来问我，说面试被问到对 Spring Cloud 的理解，自己也回答了，但感觉面试官不满意，说是没有结合项目经验来回答。

今天，我给大家分享一下我的理解，希望能够帮助到各位小伙伴。

53.314.1 Spring Cloud 的应用背景

Spring Cloud 是 Spring 官方推出来的一套微服务解决方案。

准确来说，Spring Cloud 其实是对微服务架构里面出现各种技术场景，定义了一套标准规范。

然后，在这套标准里面，Spring 集成了 Netflix 公司的 OSS 开源套件，比如

Zuul 实现应用网关、

Eureka 实现服务注册与发现、

Ribbon 实现负载均衡、

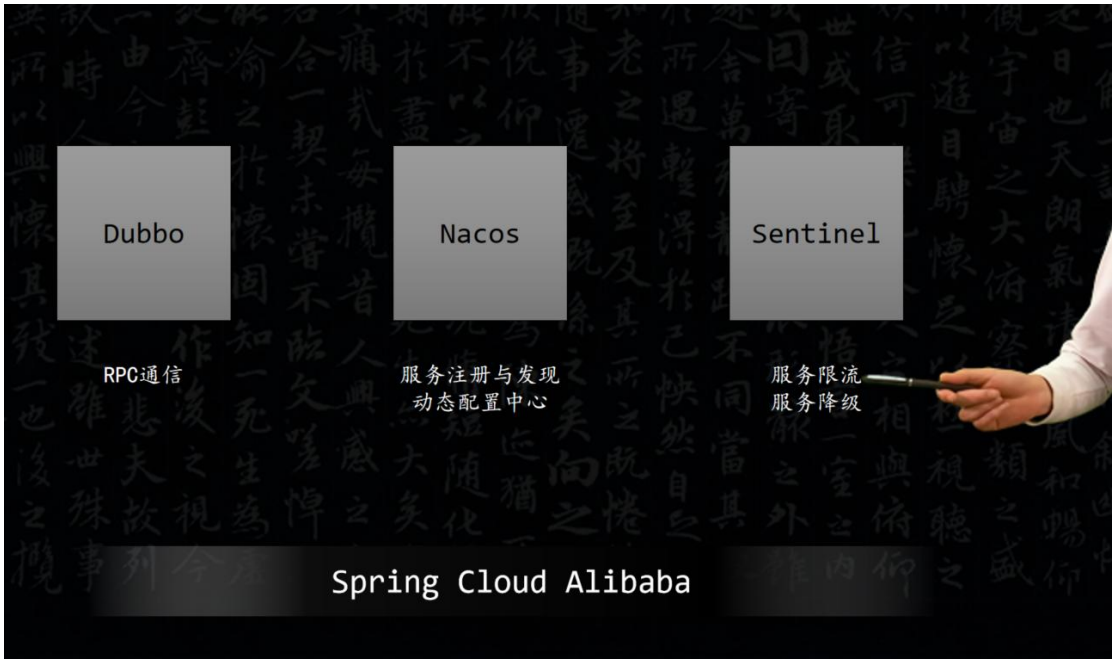
Hystrix 实现服务熔断

我们可以使用 Spring Cloud Netflix 这套组件，快速落地微服务架构以及解决微服务治理等一系列问题。

但是随着 Netflix OSS 相关技术组件的闭源和停止维护，所以 Spring 官方也自研了一些组件，

比如 Gateway 实现网关、 LoadBalancer 实现负载均衡。

另外，在国内的阿里也在为开源做出积极的贡献，它整合了 Spring Cloud 的标准，同时，加入了自身的技术，成为了 Spring Cloud 生态中另一套微服务解决方案 Spring Cloud Alibaba。



其中包括Dubbo做RPC通信、Nacos实现服务注册与发现以及动态配置中心、Sentinel实现服务限流和服务降级等等。

53.314.2 使用 Spring Cloud 的好处

以上就是我对 Spring Cloud 的理解。最后，我再补充一下，我认为 Spring Cloud 生态的出现有两个很重要的意义。

1、在 Spring Cloud 出现之前，为了解决微服务架构里面的各种技术问题，需要去集成各种开源框架，因为标准和兼容性问题，所以在实践的时候很麻烦。国内还有很些企业基于阿里的 Dubbo 进行二次开发自研出 Dubbo X，而 Spring Cloud 统一了这样一个标准。

2、使用 Spring Cloud 降低了微服务架构的开发难度，只需要在 Spring Boot 的项目基础上通过 Starter 启动依赖集成相关组件就能轻松解决各种问题。

【结尾】

在现在的互联网分布式应用开发中，Spring Cloud 的普及程度非常大了。很多应届生往往知道 Spring Cloud 怎么用，但是为什么用，以及在什么场景下用，却不一定能明白。

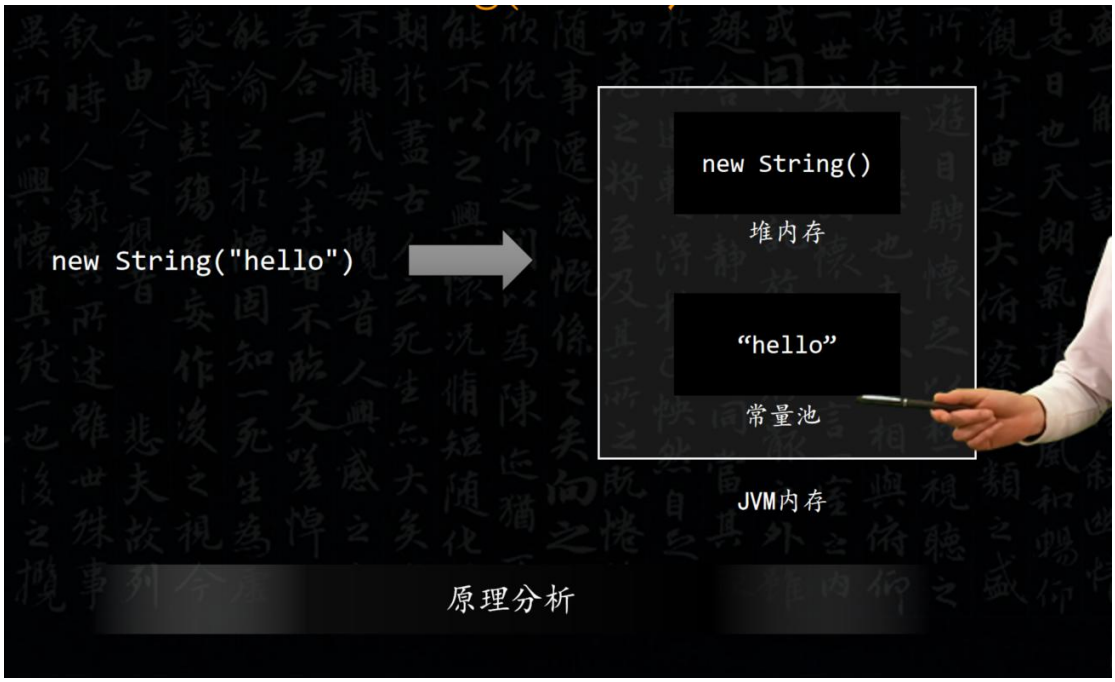
我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.315 new String("hello")之后，到底创建了几个对象?

最近，有很多优秀的程序员从大厂毕业， 再加上大环境的影响，很多正在找工作的小伙伴也感觉技术面试越来越难，基本上都会问技术底层原理，甚至有些还会问到操作系统层面的知识。 如果技术功底不扎实，确实很难找到合适的岗位。

今天，给大家分享这样一个问题，” new String(“hello”)之后，到底创建了几个对象?

53.315.1 原理分析



首先，这个代码里面有一个 new 关键字，这个关键字是在程序运行时，根据已经加载的系统类 String，在堆内存里面实例化的一个字符串对象。

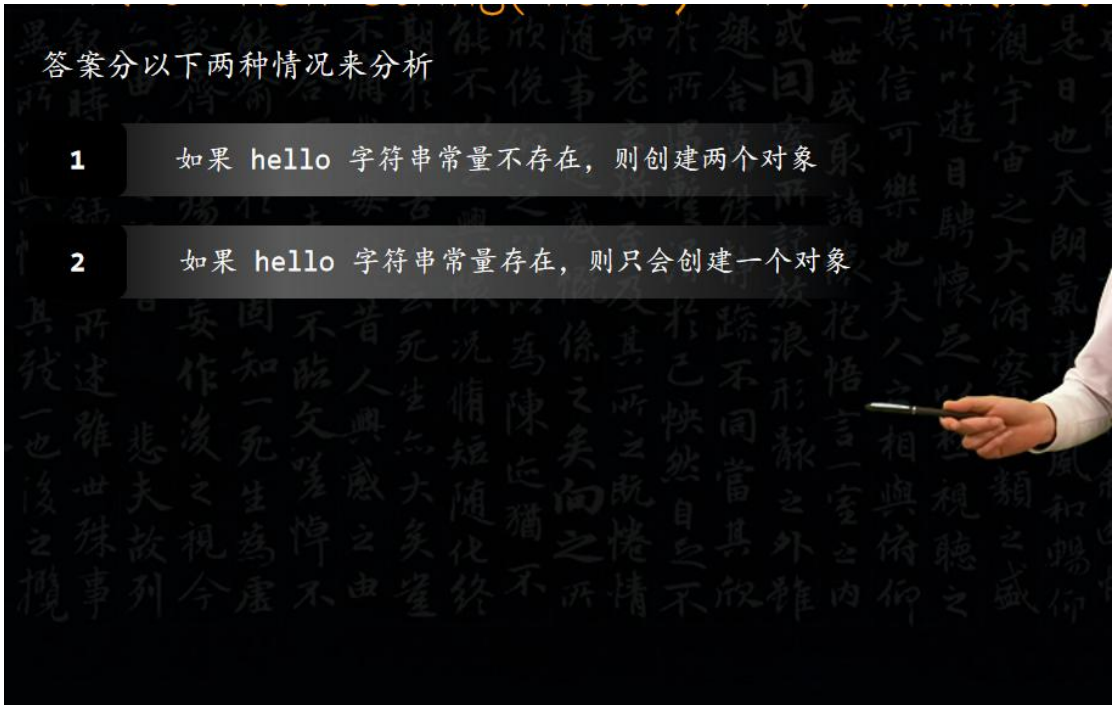
然后，在这个 String 的构造方法里面，传递了一个“hello”字符串，因为 String 里面的字符串成员变量是 final 修饰的，所以它是一个字符串常量。

接下来，JVM 会拿字面量“hello”去字符串常量池里面试图去获取它对应的 String 对象引用，如果拿不到，就会在堆内存里面创建一个”hello”的 String 对象，并且把引用保存到字符串常量池里面。

后续如果再有字面量“hello”的定义，因为字符串常量池里面已经存在了字面量“hello”的引用，所以只需要从常量池获取对应的引用就可以了，不需要再创建。

53.315.2 答案解析和知识扩展

所以，对于这个问题的答案分以下两种情况来分析：



如果 hello 这个字符串常量不存在，则创建两个对象，分别是 hello 这个字符串常量，以及 new String 这个实例对象。

如果 hello 这字符串常量存在，则只会创建一个对象。

小伙伴们再来看这样一段代码：

```
public static void main(String[] args) {
        String s1 = "hello";
```

```
        String s2 = "hello";
        String s3 = "he" + "llo";
        String s4 = "hel" + new String("lo");
        String s5 = new String("hello");
        String s6 = s5.intern();
        String s7 = "h";
        String s8 = "ello";
        String s9 = s7 + s8;
    }
```

它的输出结果如下：

```
        System.out.println(s1==s2);//true
        System.out.println(s1==s3);//true
        System.out.println(s1==s4);//false
        System.out.println(s1==s9);//false
        System.out.println(s4==s5);//false
        System.out.println(s1==s6);//true
```

接下来给大家简单分析一下：

由于 s2 指向的字面量"hello"在常量池中已经存在（s1 先于 s2），于是 JVM 就返回这个字面量绑定的引用，所以 s1==s2。

s3 中字面量的拼接其实就是"hello"， 在编译期间就已经对它进行了优化，所以 s1 和 s3 也是相等的。

s4 中的 new String("lo")生成了两个对象：lo 和 new String("lo")。lo 存在于字符串常量池中，new String("lo")存在于堆中，String s4 = "hel" + new String("lo")实质上是两个对象的相加，编译器不会进行优化，相加的结果存在于堆中，而 s1 存在于字符串常量池中，当然不相等。s1==s9 的原理也一样。

s4==s5 两个相加的结果都在堆中，不用说，肯定不相等。

s1s6 中，s5.intern()方法能使一个位于堆中的字符串，在运行期间动态地加入字符串常量池（字符串常量池的内容是在程序启动的时候就已经加载好了的）。如果字符串常量池中有该对象对应的字面量，则返回该字面量在字符串常量池中的引用；否则，复制一份该字面量到字符串常量池并返回它的引用。因此 s1s6 输出 true。

【结尾】

其实，要回答好这道题，需要对 JVM 的运行时内存划分以及 JVM 常量池有深刻的理解，这也是享元模式在 JDK 源码中的应用。现在的技术面试已经逐渐趋向于体系化的考查，不再是点状式的提问了。

我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.316 MyBatis 何时使用一级缓存，何时使用二级缓存?

一个 5 年工作经验的小伙伴，去面某东被问到 MyBatis 何时使用一级缓存，何时使用二级缓存？去之前还特地复习了 MyBatis 的相关知识，想着自己用 MyBatis 用得比较熟练了，竟然在这道题上翻车了。

今天，我给大家来分享一下 MyBatis 的缓存机制。

53.316.1 设计思想

Mybatis 里面设计了两级缓存来提升数据的检索效率，避免每次数据的访问都需要去查询数据库。

先来看一级缓存，它是 SqlSession 级别的缓存，也叫本地缓存，因为每个用户在执行查询的时候都需要使用 SqlSession 来执行，为了避免每次都去查数据库，MyBatis 把查询出来的数据保存到 SqlSession 的本地缓存中，后续的 SQL 如果命中缓存，就可以直接从本地缓存读取。

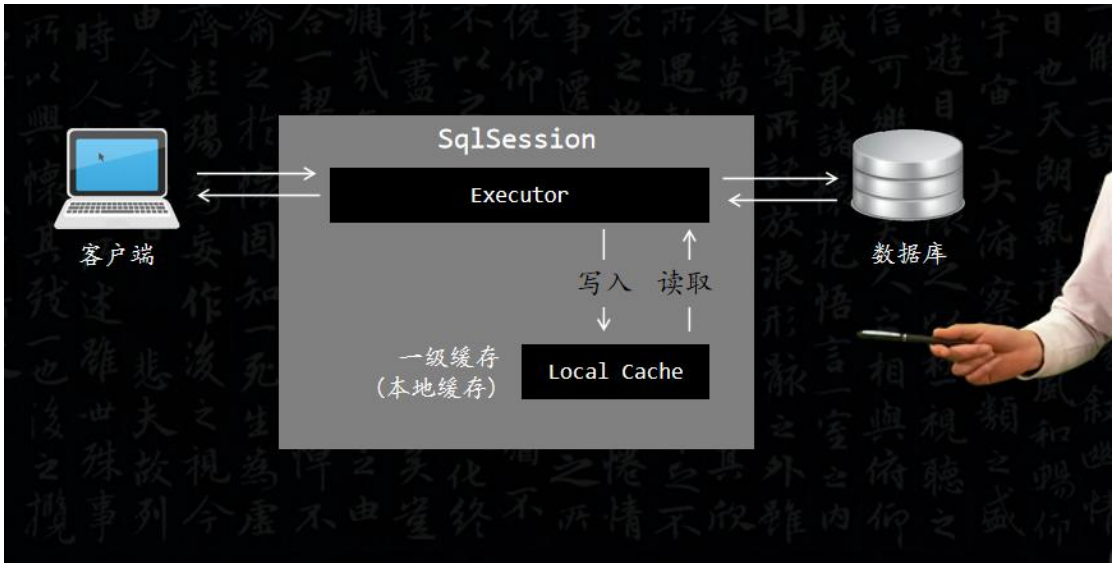
那如果想要实现跨 SqlSession 级别的缓存？一级缓存就无法实现了，因此，MyBatis 引入了二级缓存。当多个用户在查询数据的时候，只要有任何一个 SqlSession 拿到了数据就会放入到二级缓存里面，其他的 SqlSession 就可以从二级缓存加载数据。

53.316.2 原理分析

接下来，我给大家详细分析一下 MyBatis 缓存机制的实现原理。先来看一级缓存的实现原理：

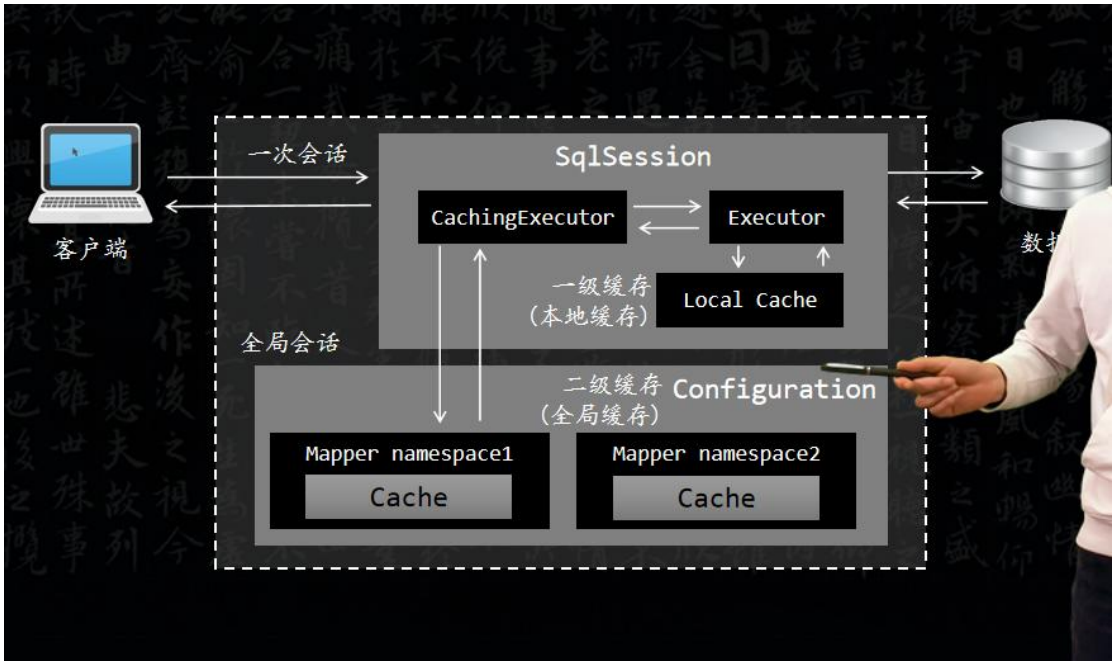
在 SqlSession 里面持有一个 Executor 的对象，每个 Executor 中有一个 LocalCache 对象。

当用户发起查询的时候，MyBatis 会根据执行语句在 Local Cache 里面查询，如果没命中，再去查询数据库并写入到 LocalCache，否则直接返回。



所以，一级缓存的生命周期只在 SqlSession 级别，而且在多个 SqlSession 或者分布式环境下，可能会导致数据库写操作出现脏数据。

那这个时候，就要使用二级缓存。下面来看二级缓存的具体实现原理：



二级缓存使用了一个叫做 CachingExecutor 的对象，对 Executor 进行了装饰，在进入一级缓存的查询流程之前，会先通过 CachingExecutor 进行二级缓存的查询。

开启二级缓存以后，会被多个 SqlSession 共享，所以它是一个全局缓存。因此它的查询流程是先查二级缓存，再查一级缓存，最后再查数据库。

另外，MyBatis 的二级缓存相对于一级缓存来说，实现了 SqlSession 之间缓存数据的共享，同时缓存粒度也能够控制到 Name Space 级别，并且还可以通过 Cache 接口实现类不同的组合，对 Cache 的可控性也更强。

【结尾】

以上就是我对这个问题的理解。在实际业务场景中，多级缓存的设计思想，非常值得我们学习和借鉴。学习底层技术的目的是为了提高技术思维能力和积累解决方案，才能够胜任更高的职位要求。

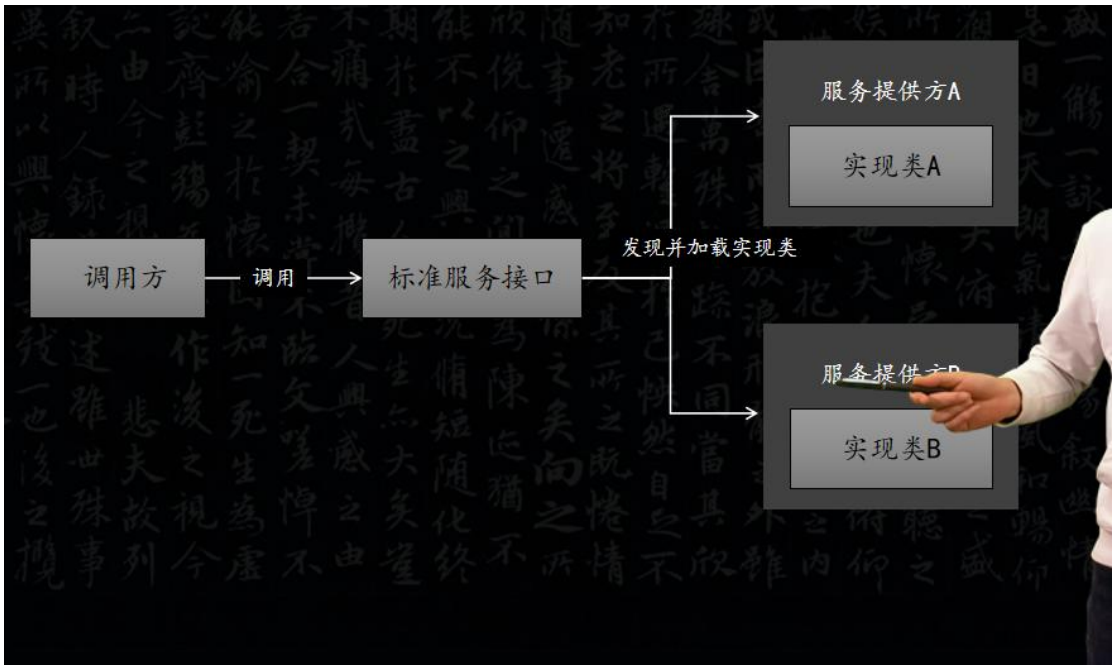
我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.317 什么是 Java SPI，它有什么作用?

很多主流框架都用到了 SPI 机制，比如 Dubbo、Spring 都有用到，可能很多小伙伴也听说过。今天给大家来聊一聊什么是 SPI，它用什么作用?

53.317.1 什么是 SPI

SPI 全称 Service Provider Interface，它是 Java 提供的一套用来被第三方实现或者扩展的 API。



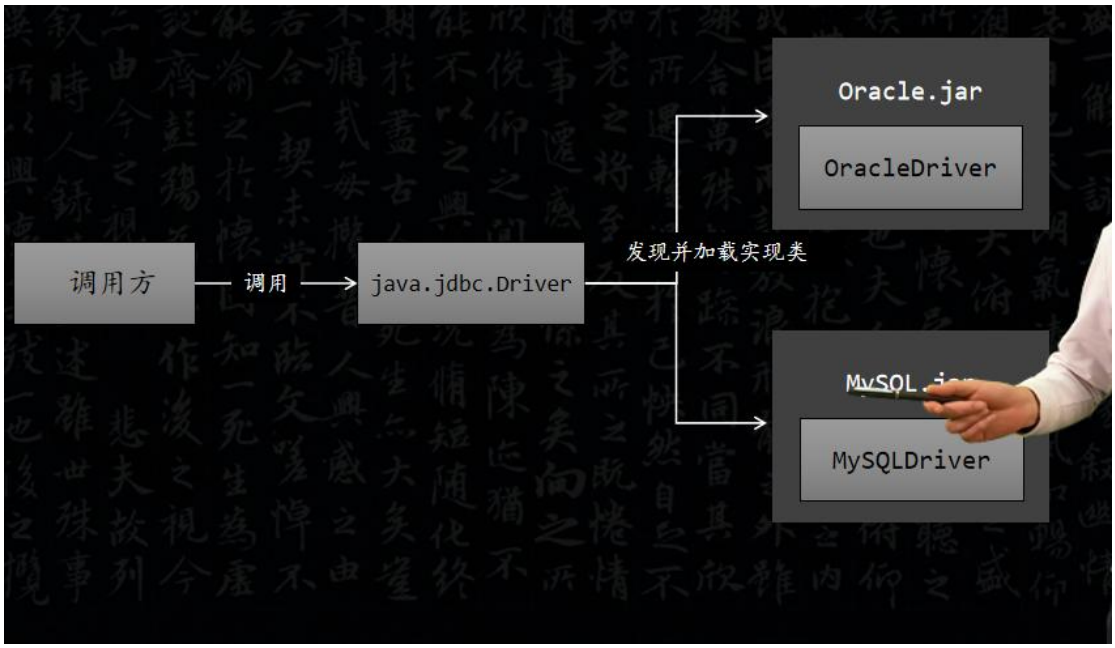
如图所示，简单来说，就是我们定义一个标准的接口，然后第三方的库里面可以实现这个接口。

那么，程序在运行的时候，会根据配置信息动态加载第三方实现的类，从而完成功能动态扩展的机制。

53.317.2 SPI 的应用场景

SPI 的主要作用是解耦。适用于调用者根据实际使用需要，启用、扩展、或者替换框架的实现策略。

在 Java 中，SPI 机制有一个非常典型的实现案例，就是数据库驱动 java.jdbc.Driver



如图所示，JDK 里面定义了数据库驱动类 Driver，它是一个接口，JDK 并没有提供实现，具体的实现是由第三方数据库厂商来完成的。

在程序运行的时候，会根据我们声明的驱动类型，来动态加载对应的扩展实现，从而完成数据库的连接。

除此之外，在很多开源框架里面都借鉴了 Java SPI 的思想，提供了自己的 SPI 框架，比如 Dubbo 定义了 ExtensionLoader，实现功能的扩展。

Spring 提供了 SpringFactoriesLoader，实现外部功能的集成，基于 Servlet 3.0 规范对 ServletContainerInitializer 的实现。

【结尾】

Java 中 SPI 的设计主要用到了桥接模式，在实际开发中，借鉴这种思想可以在不修改核心代码的情况下，完成提供功能的增强和扩展，而且不需要显式地依赖于继承或者接口。以上就是我对 Java SPI 的理解。

我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.318 假如我 Myql 的表很大，进行分页的时候，limit 1000000 加载很慢的话，你是怎么解决的呢?

【面试官】

假如我 Myql 的表很大，进行分页的时候，limit 1000000 加载很慢的话，你是怎么解决的呢?

求职者：好的，面试官，这个问题在生产环境也是比较常见的。

limit m，n ;其实去扫描 m+n 条数据，然后过滤掉前面的 m 条数据，当 m 越大，那么需要扫描的数据

也就越多，性能也会越来越慢。

针对这种情况，有以下几种方案可以进行一定的优化。

1.如果 id 是趋势递增的，那么每次查询都可以返回这次查询最大的 ID，然后下次查询，加上大于 上

次最大 id 的条件，这样会通过主键索引去扫描，并且扫描数量会少很多很多。因为只需要扫描 where

条件的数据

2.先 limit 出来主键 ID，然后用主表跟查询出来的 ID 进行 inner join 内连接，这样，也能一定上提速，

因为减少了回表，查询 ID 只需要走聚集索引就行。

3.当然，如果 mysql 级别优化不了了。我们也可以对分页数据进行缓存，比如 Redis 缓存，数据进行

变动的时候，做好缓存依赖即可。

4.因为越往后，一般用户行为触及不到，比如你去看淘宝，不会去翻后面几百页的数据，所以，业务

层面也可以做一些让步，比如不做后面几百页的数据。

53.319 谈谈你对分布式 ID 的理解

求职者：

我研究生毕业后，一直在杭州某互联网大厂担任 Java 开发。来贵司应聘高级开发，我的期

待薪资年包 50W

求职者：

没问题，您可以随便考察

求职者：

首先，分布式 ID，是因为随着业务的发展，数据或者 ID 的生成会在不同的地方，比如常见

的数据的分库分表，在分库分表的情况下，假如不同的表生成的 ID 不能保证唯一性，那么

就会导致一个 ID 不能保证一条数据的唯一性，就会带来问题。

所以分布式 ID 的主要目的：不论何时何地，生成的 ID 都是唯一的。

实现的方式有很多：

1.Mysql 的全局 ID 生成表

2.UUID（有些版本会重复）

3.Redis 的 incr 自增 ID

4.zk 的有序节点序号

5.mongoDB 的 objectId、

等等这些方案都能解决唯一性的问题。

但是除了唯一性以外，我们还需要考虑更多的其他因素：

1.比如是否趋势递增，否则在 Mysql 会导致主键索引树的分裂与合并。

2.安全性，通过 ID 是否会导致数据泄露

3.高可用以及高性能，满足业务的需求，不能生成一个 ID 很慢或者动不动不可用。

所以，为了性能与可靠性以及成本，市面上还有一种雪花算法是用得比较多，他是一种基于

位运算来实现的，并且实现简单，没有太多外部依赖。

很多大公司也有基于雪花写了自己的 ID 生成方案，比如美团的 Leaf

53.320 一条 mysql 查询语句很慢，怎么去排查优化

面试官：你们公司用的 mysql，假如，一条 mysql 查询语句很慢。你会怎么去排查优化?

面试者：好的。面试官。

1. 首先，我会去检查是否是语句查询问题，比如是否是连接数的问题，因为在高并发的场景，这个问

题也会经常出现的，可以通过 show status 来查看相关连接，里面包含现在的连接数等信息

2. 如果不是网络跟连接问题，那么就需要去优化 sql 语句。

优化 sql 语句，最主要的是要用 explain 进行分析，explain 里面有几个关键字段，

- 一个 type 连接类型，会有 all、index 、range 等等，一般最好能达到 index，如果是 all 那么就是全表扫描。
- 还有就是 key 里面用到的字段。
- 还有 rows,代表估计检查扫描的行，数量越大，性能越慢，
- extra 附加信息，order by 排序是否走到索引等等，可以在附加信息查看。

通过 explain 能找出一些比较明显的问题，比如是否用到索引，where 条件、orderby、group by 是

否用到了索引，如果没有用到，就要去看下索引是否满足最左匹配，或者用到了运算或者范围查询等
等使下一个索引列失效。
就需要去变动相关的索引列的顺序或者更改查询语句。
当然，还有一些小表驱动大表，这个就是存储引擎与 mysql server 层的一些交互优化，尽量减少
server 跟存储引擎的一些交互。
当不能快速找到问题的时候，就具体情况去具体分析，比如一些复杂的 sql 语句，那么可以一个一个
的去优化，把每个语句、或者每个查询条件单独拆开去分析，找到哪个语句或者哪个查询条件比较慢，
然后就能找到慢的问题了。

53.321 Mysql 里面的 RedoLog 和 BinLog，他们的作用在哪?

好的 面试官，RedoLog 是 innoDB 里面的事务日志，既然是事务日志，所以它的最终目的是保证我
的数据一致性，也是 InnoDB 满足 ACID 模型的一块很重要的支持。
binlog 不属于存储引擎级别，不管是什么存储引擎都会有的，是 Mysql 服务自己去实现的。
binlog 其实就是一些二进制日志记录文件，主要目的是数据灾备以及主从节点之间的数据同步。
面试官：那 innoDb 为什么不直接每次数据操作的时候，都保证同步到磁盘，而要加一个 RedoLog?

【灰灰老师回答】那为什么要设计一个 redoLog，是因为 InnoDB 存储引擎为了提高数据交互性能，
加了一个 bufferpool 内存缓存，如果我数据每次操作都同步给磁盘，就会很慢。

所以，InnoDB 采用的是异步的刷盘机制，但是异步刷盘又会导致数据丢失。因此，InnoDB 设计了
一个 RedoLog 的重做日志，每次操作实时同步到 RedoLog 的 file 文件，当然，默认也会实时刷新到磁
盘，但是解决了数据直接同步到磁盘的 2 个性能问题：
第一个数据交互的最小单位是 page，单次交互的量比较大，第二个是数据属于随机 IO，而 redolog
属于顺序 IO。
面试官：你刚才说 binlog 是用来同步主从数据的，那假如我 RedoLog 写成功了，但是 binlog 没有成
功，那是不是会出现主节点有数据，但是没有同步到 binlog？

【灰灰老师回答】好的，redoLog 如果成功了，binLog 没有，那么就会出现主有，从没有，如果先
写 binlog 再写 redolog，也会出现从有，但是主可能没有。
因而，这 2 个日志是尽可能的去保证都要写成功的。
所以，在 Mysql 里面，他们实现的机制是 2pc 二阶段提交，redoLog 先是准备状态，然后再去写
binLog，只有 binLog 写成功了，redolog 才会真正的刷新到磁盘。只有当 redoLog 跟 binLog 都有，
才能保证数据一致性。否则，数据回滚。

53.322 如果不使用 Spring，我们将如何开发？

曾经有一道面试题掀起了劲爆的浪潮，说如果不使用 Spring，我们将如何开发？好多家公司都模仿提问了这么一道面试题，而且好多人也都在各个社区给出了自己的答案。接下来看看网友们是怎么说的。
有说手写 IoC 的，有说用 Servlet 完成 Web 开发的，有说用纯 JDBC 完成数据库操作的等等，大家回答各有千秋。其实，想想我们上一代的程序员，没有 Spring 不也照样开发出非常多优秀的系统。

53.322.1 Spring 生态

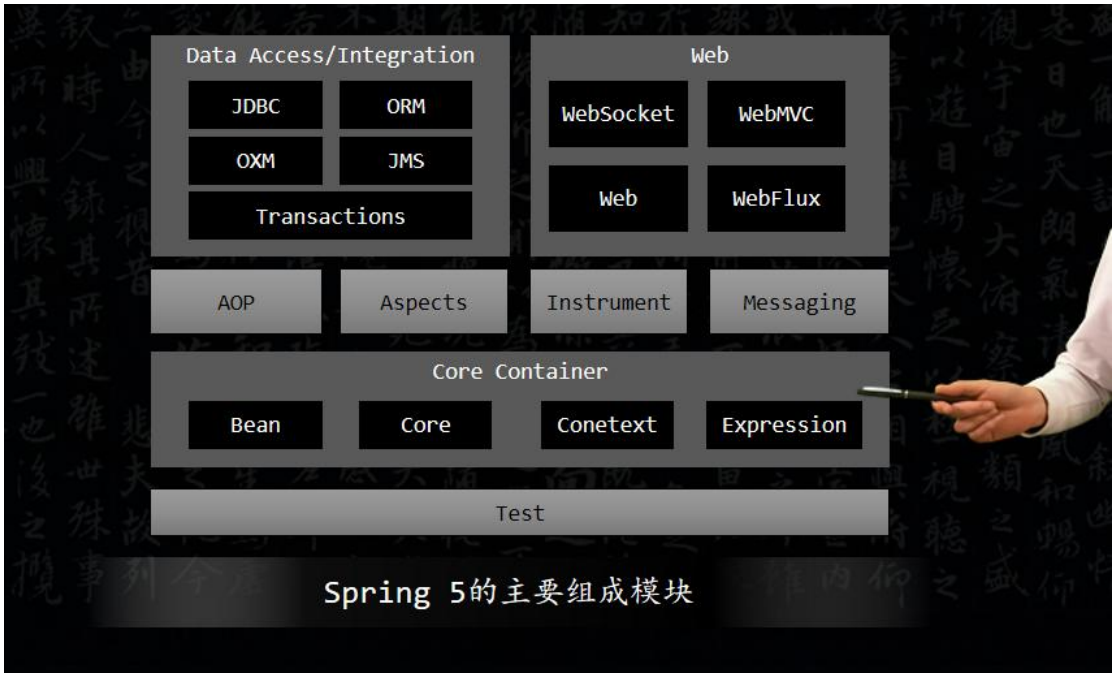
那我们使用 Spring 到底能给我们带来哪些好处呢?

根据我对 Spring 10 多年的研究经验，给大家做以下总结。

首先，Spring 是一个轻量级的应用框架。当然，众所周知，它提供了 IoC 和 AOP 这两个核心的功能。

它的核心目的是为了简化企业级应用程序的开发，使得开发者只需要关心业务需求，不需要关心 Bean 的管理，以及通过切面增强功能减少代码的侵入性。

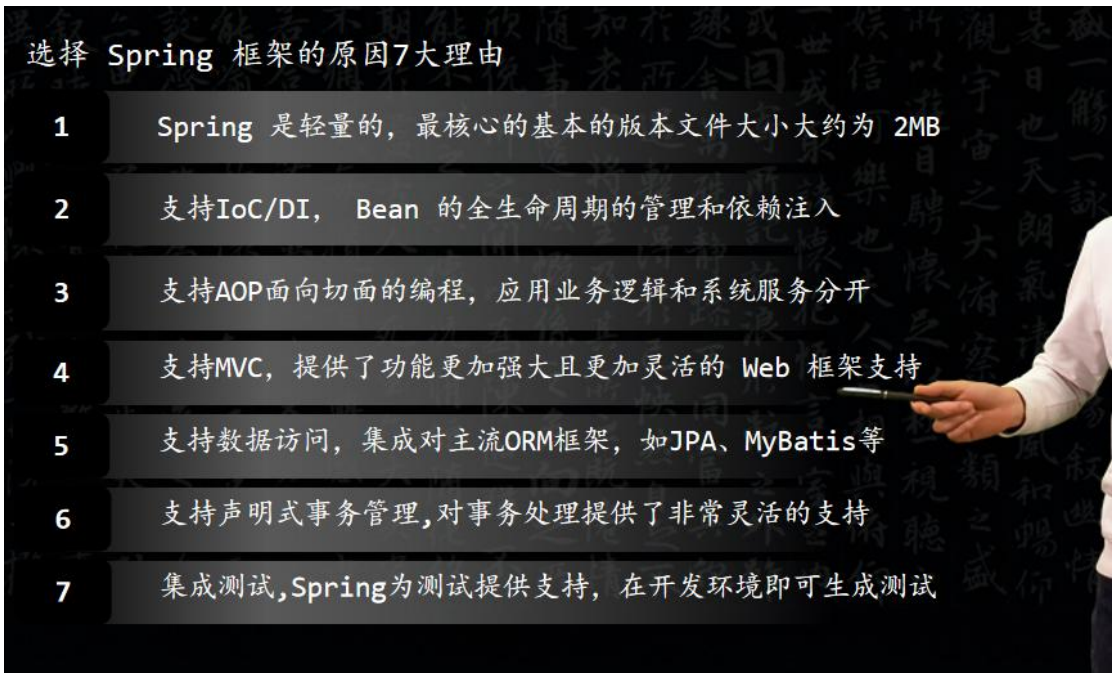
当然，Spring 发展到现在俨然不只是一个框架，而是一个生态，我们习惯性地称之为 Spring 全家桶。Spring 大约有 20 个模块，由 1300 多个不同的文件构成。这些模块可以分为核心容器、AOP 和设备支持、数据访问与集成、Web 组件、通信报文和集成测试、集成兼容等类。如图所示:



组成 Spring 框架的每个模块都可以单独存在，也可以将一个或多个模块联合实现。

53.322.2 使用 Spring 的好处

从 Spring 本身的特性来看，我认为有几个关键点是我们选择 Spring 框架的原因。



第 1 个，轻量：Spring 是轻量的，最核心的基本的版本文件大小大约为 2MB。

第 2 个，支持 IoC/DI：当然，IoC 不是 Spring 独有的。但最重要的是，可以使用 Spring 的 IoC 容器实现对 Bean 的全生命周期的管理，还可以通过 DI 实现依赖注入，从而实现了对象依赖的松耦合管理。

第 3 个，支持 AOP 面向切面的编程：Spring 支持面向切面的编程，从而把应用业务逻辑和系统服务分开。

第 4 个，支持 MVC：Spring MVC 提供了功能更加强大且更加灵活的 Web 框架支持

第 5 个，支持数据访问：Spring 集成对主流 ORM 框架的支持，比如 JPA、MyBatis 等等。

第 6 个，声明式事务管理：Spring 通过 AOP 实现了事务的统一管理，对应用开发中的事务处理提供了非常灵活的支持

第 7 个，集成测试：Spring 为测试提供支持，在开发环境即可生成测试

最后，Spring 生态非常庞大。使用 Spring 几乎可以满足所有业务场景的开发。更重要的是社区的活跃度和技术的成熟度都非常高。

【结尾】

以上就是我对 Spring 的理解。任何一个技术框架的出现，一定为了实现某些业务场景或者是某一类技术问题的解决方案。当大家有了的开发经验之后，对一些技术框架，要有自己的见解和思考。我自己在面试的时候也喜欢问这一类的问题。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

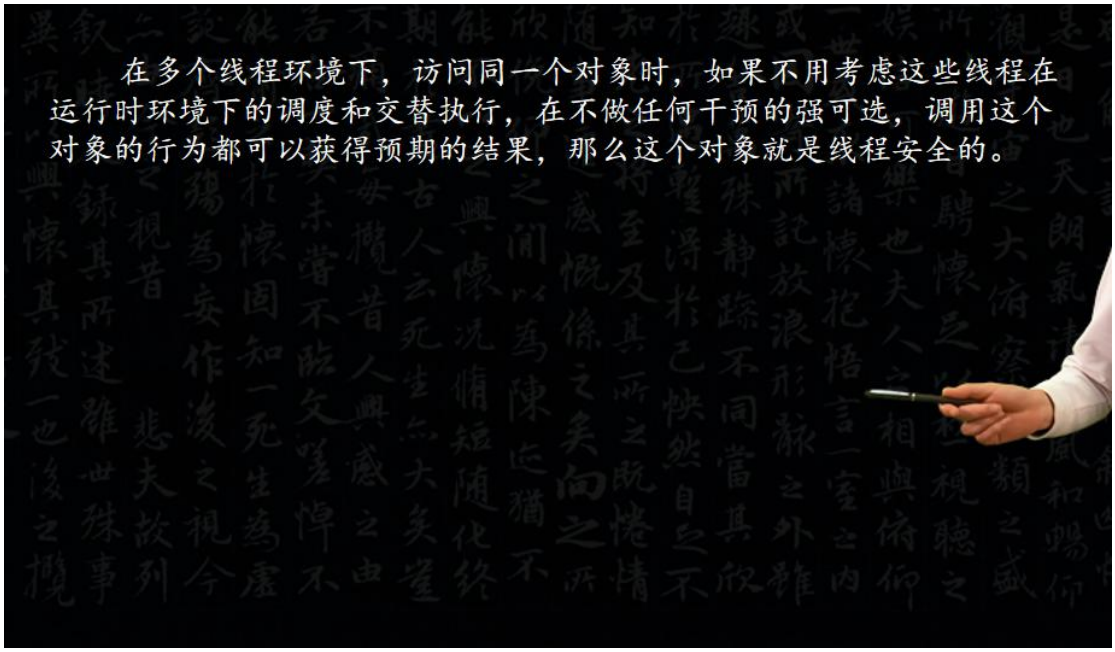
53.323 谈谈你对线程安全的理解？

一位 4 年工作经验的小伙伴，被问到一个非常抽象的问题，说，谈谈你对线程安全性的理解。如果平时只是刷刷面试题的话，遇到这种问题可能不知道如何说起了，往往需要自己组织语言。另外，如果平时积累不够的话，也很难说出一些自己独特的见解来。

今天，我分享一下我对线程安全的理解，希望能给小伙伴们作为参考。

53.323.1 官方定义

线程安全通常是相对于多线程或者并发的情况下而言的。如果是单线程操作的话，就无所谓线程安全了。

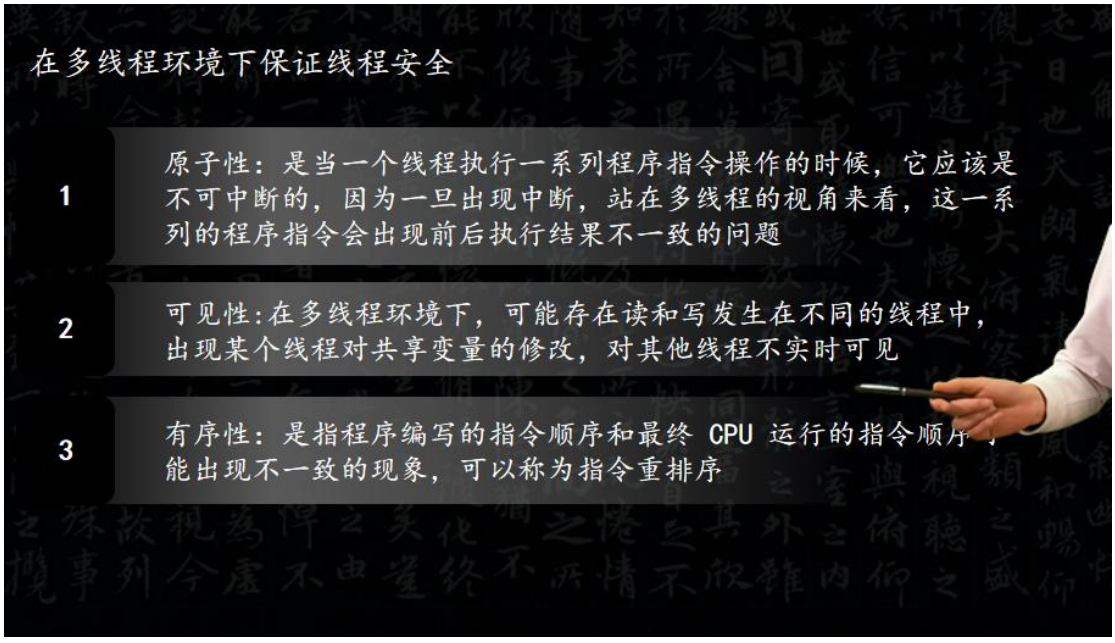


简单来说，就是在多个线程环境下，访问同一个对象时，如果不用考虑这些线程在运行时环境下的调度和交替执行，在不做任何干预的强可选，调用这个对象的行为都可以获得预期的结果，那么这个对象就是线程安全的。

看完这段，是不是还是很难理解？下面，我来分享一下看我是如何理解的。

53.323.2 高手的理解

我认为，在多线程环境下保证线程安全，无非就是保证对对象访问的原子性、有序性和多个线程之间的可见性。



原子性呢，说的是当一个线程执行一系列程序指令操作的时候，它应该是不可中断的，因为一旦出现中断，站在多线程的视角来看，这一系列的程序指令会出现前后执行结果不一致的问题。

这个和数据库里面的原子性是一样的，简单来说就是一段程序只能由一个线程完整的执行完成，而不能存在多个线程干扰。CPU 的上下文切换 ， 是导致出现多线程原子性问题的核心原因 ， 而 JDK 面也提供了 synchronized 关键字来解决原子性问题。

然后，就是可见性，相当于在多线程环境下，可能会存在读和写是发生在不同的线程里面，有可能出现某个线程对共享变量的修改，对其他线程不是实时可见的。导致可见性问题的原因有很多，比如 CPU 的高速缓存、CPU 的指令重排、编译器的指令重排等因素。

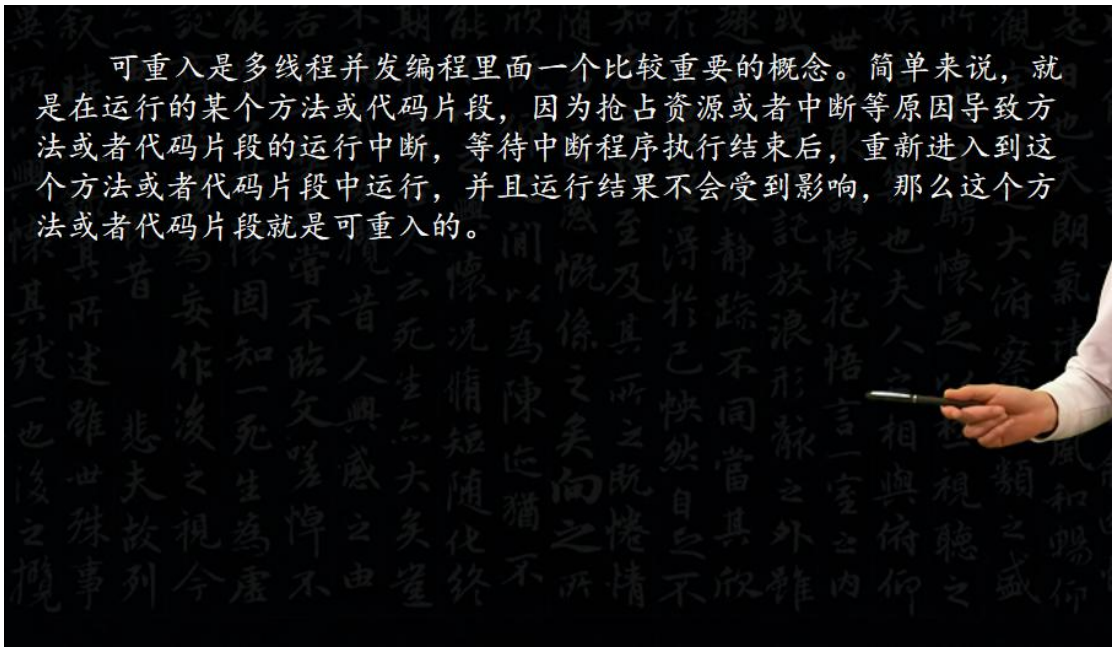
最后，就是有序性，指的是程序编写的指令顺序和最终 CPU 运行的指令顺序可能出现不一致的现象，这种现象也可以称为指令重排序，所以有序性也会导致可见性问题。可见性和有序性可以通过 JDK 提供的 volatile 关键字来解决。

我认为，导致有序性、原子性、可见性问题的本质，是计算机设计的时候，为了最大化提升 CPU 利用率导致的。比如 CPU 设计了三级缓存、设计了 StoreBuffer、设计了缓存行这种预读机制、在操作系统里面，设计了线程模型、在编译器里面，设计了编译器的深度优化机制等等。

53.324 什么是可重入锁?它能解决什么问题?

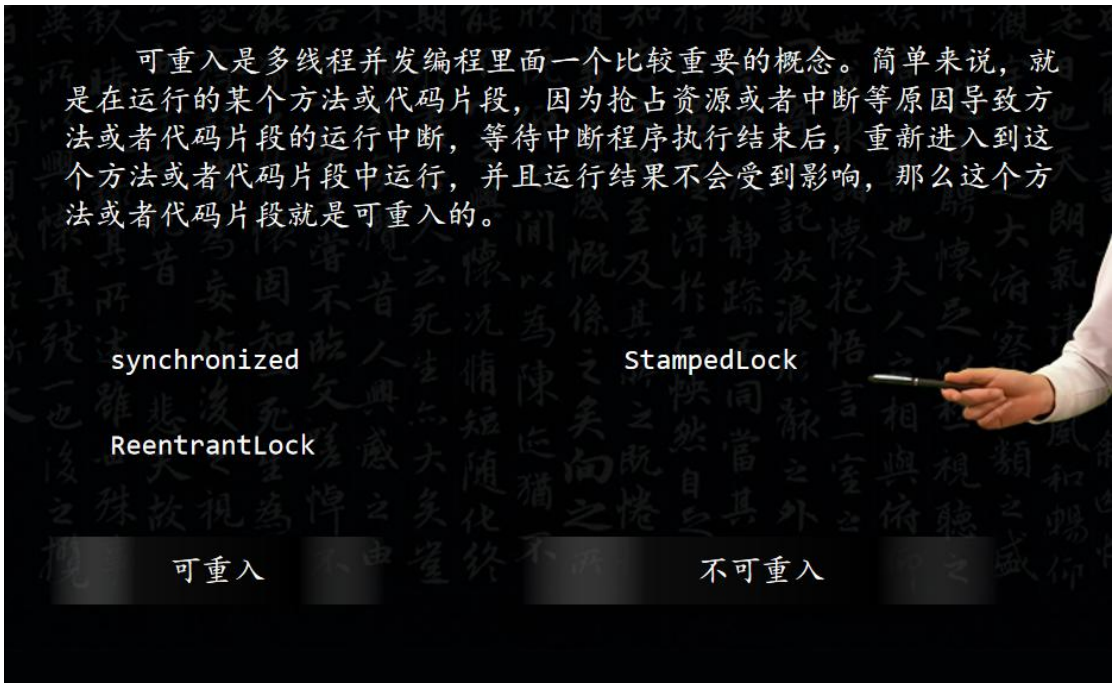
一位 3 年工作经验的小伙伴，去一家互联网公司面试，结果被面试官怼了。面试官说：”这么简单的问题你都不知道？没法聊了，回去等通知吧 “。然后，回来跟我是一阵诉苦。

53.324.1 可重入锁的官方定义



可重入是多线程并发编程里面一个比较重要的概念。简单来说，就是在运行的某个方法或代码片段，因为抢占资源或者中断等原因导致方法或者代码片段的运行中断，等待中断程序执行结束后，重新进入到这个方法或者代码片段中运行，并且运行结果不会受到影响，那么这个方法或者代码片段就是可重入的。

而可重入锁，简单来说就是一个线程如果抢占到了互斥锁资源，在锁释放之前再去竞争同一把锁的时候，不需要等待，只需要记录重入次数。

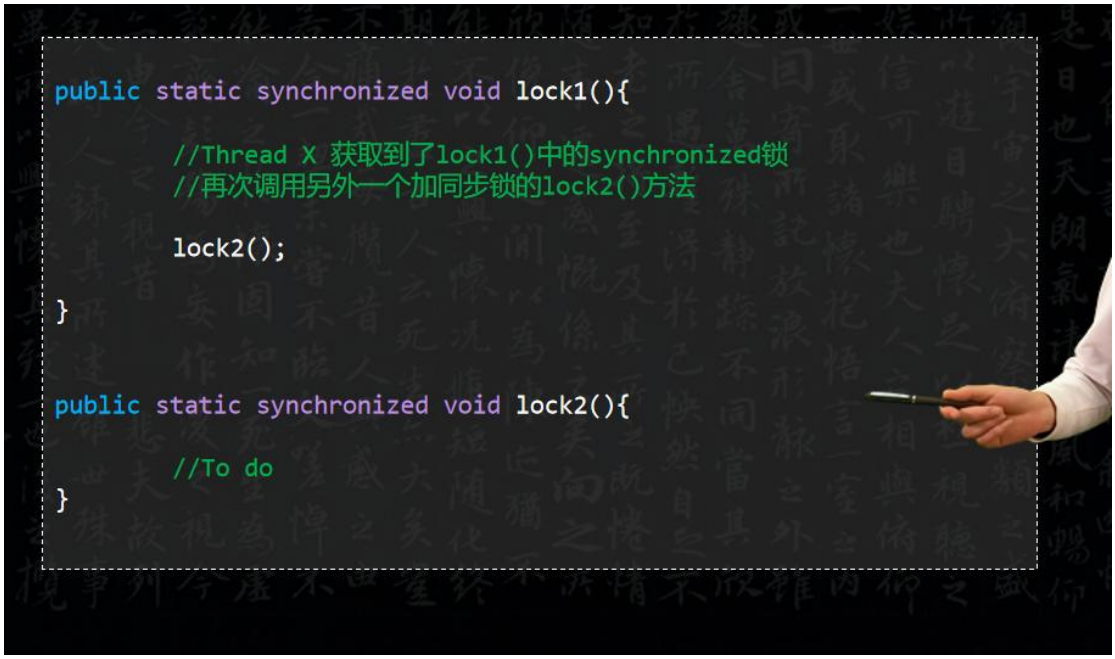


在多线程并发编程里面，绝大部分锁都是可重入的，比如 synchronized、ReentrantLock 等，但是也有不支持重入的锁，比如 JDK8 里面提供的读写锁 StampedLock。

53.324.2 可重入锁的应用场景

来看这么一段代码：

```
1  public static synchronized void lock1(){
2
3      //Thread X 获取到了 lock1()中的 synchronized 锁
4      //再次调用另外一个加同步锁的 lock2()方法
5
6      lock2();
7
8  }
9
10 public static synchronized void lock2(){
11
12     //To do
13 }
```



锁的可重入性，主要解决的问题是避免线程死锁的问题。因为一个已经获得同步锁 X 的线程，在释放锁 X 之前再去竞争锁 X 的时候，相当于会出现自己要等待自己释放锁，这很显然是无法成立的。

【结尾】

关于这个问题，其实是考察应聘者的基础知识。互联网大厂对基础的考察会特别深，各位想进互联网的小伙伴还是需要在工作之外去多花一点时间研究。对于 3 年工作经验的小伙伴，也应该要开始关注这一类问题了。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

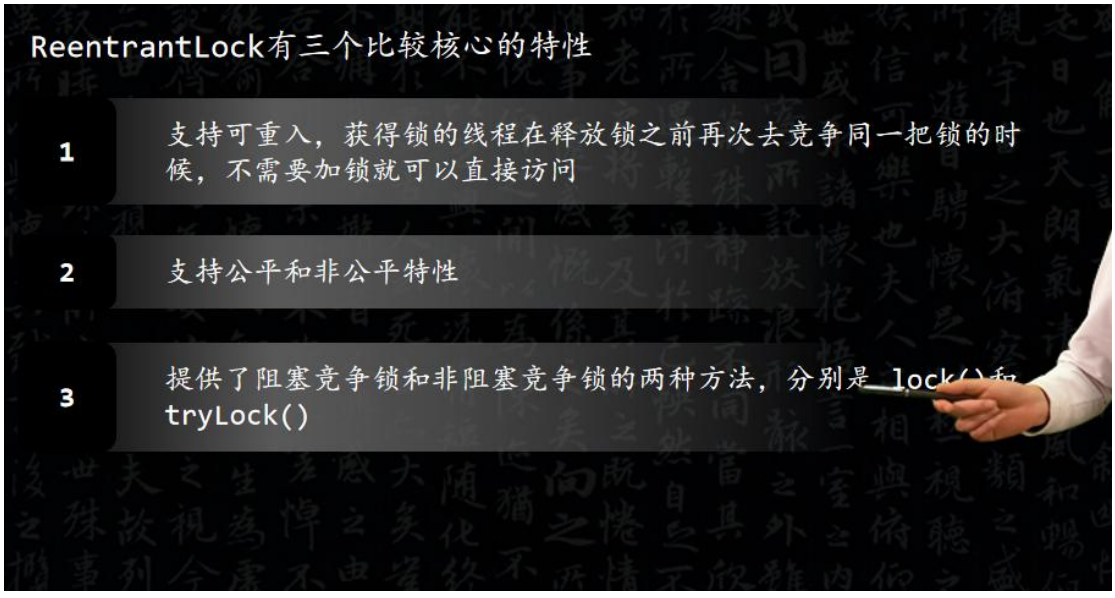
我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.325 谈谈你对 ReentrantLock 实现原理的理解?

一个 3 年工作经验的小伙伴，在面试的时候被这样一个问题。”谈谈你对 ReentrantLock 实现原理的理解“，他当时零零散散的说了一些。但好像没有说关键点。希望我分享一下我的理解。

53.325.1 什么是 ReentrantLock

ReentrantLock 是一种可重入的排它锁，主要用来解决多线程对共享资源竞争的问题。它有四个比较核心的特性：



第 1，它支持可重入，也就是获得锁的线程在释放锁之前再次去竞争同一把锁的时候，不需要加锁就可以直接访问。

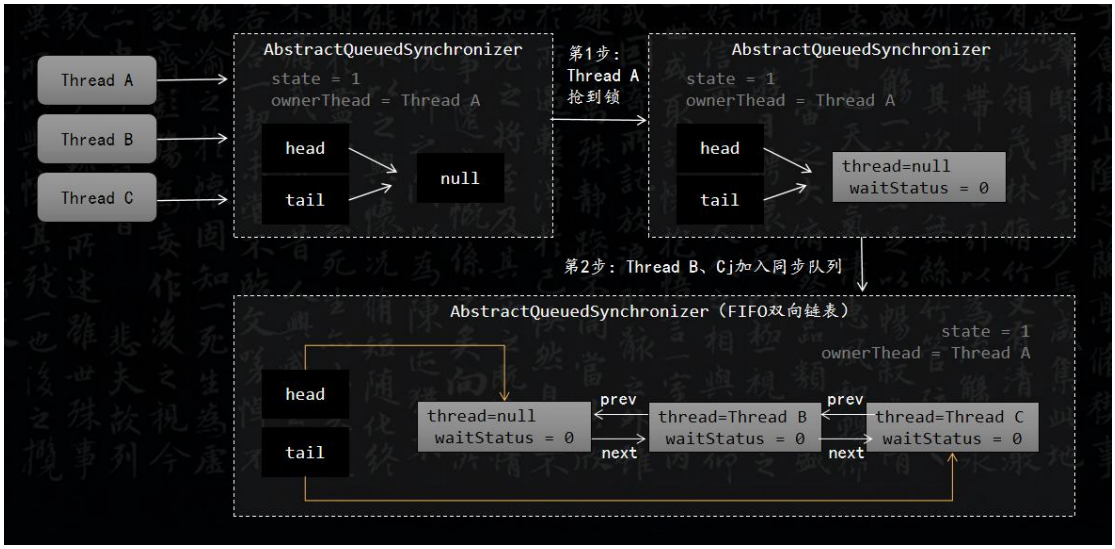
第 2，它支持公平和非公平特性

第 3，它提供了阻塞竞争锁和非阻塞竞争锁的两种方法，分别是 lock()和 tryLock()。

53.325.2 ReentrantLock 的实现原理

ReentrantLock 的底层实现有几个非常关键的技术。

第 1 个，锁的竞争，ReentrantLock 是通过互斥变量，使用 CAS 机制来实现的。



没有竞争到锁的线程，使用了 AbstractQueuedSynchronizer 这样一个队列同步器来存储，底层是通过双向链表来实现的。当锁被释放之后，会从 AQS 队列里面的头部唤醒下一个等待锁的线程。

第 2 个，公平和非公平的特性，主要是体现在竞争锁的时候，是否需要判断 AQS 队列存在等待中的线程。

第 3 个，锁的重入特性，在 AQS 里面有一个成员变量来保存当前获得锁的线程，当同一个线程下次再来竞争锁的时候，就不会去走锁竞争的逻辑，而是直接增加重入次数。

【结尾】

要回答好这样一道面试题，必须要理解 ReentrantLock 的设计思想，虽然很基础，但很重要。地基的深度决定了楼层的高度。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

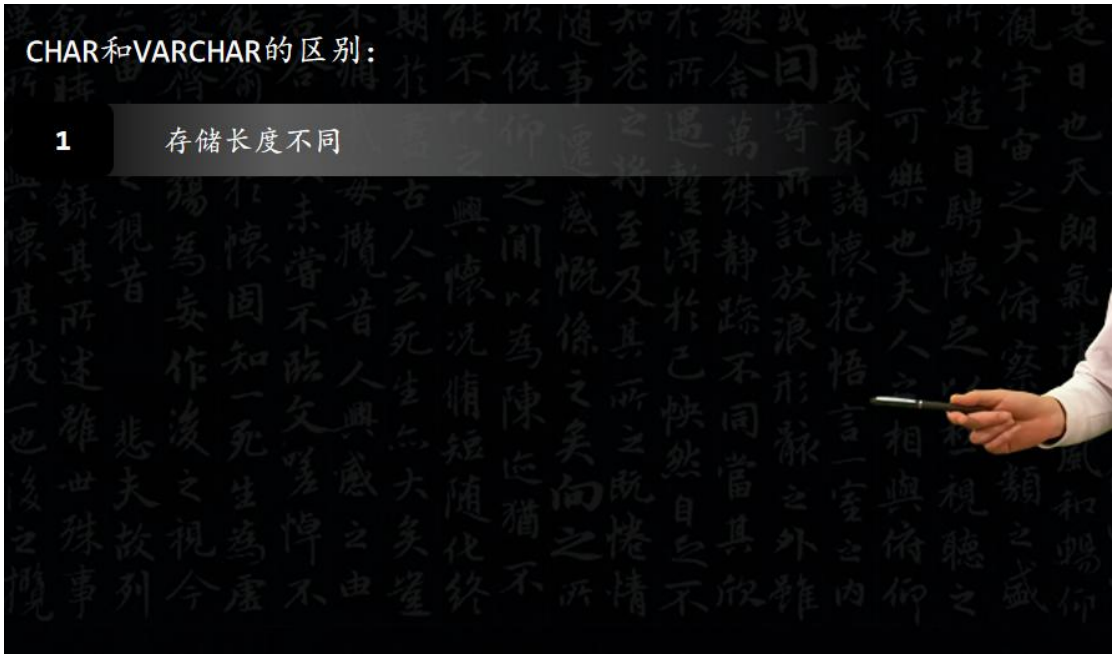
我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.326 存储 MD5 值应该用 VARCHAR 还是用 CHAR?

一个 5 年工作经验的小伙伴，在面试的时候被这样一个问题。说”存储 MD5 值应该用 VARCHAR 还是用 CHAR“，他一时间不只如何选择，感觉用 VARCHAR 也可以，用 CHAR 也行。希望我来帮忙分析一下。

53.326.1 两者区别

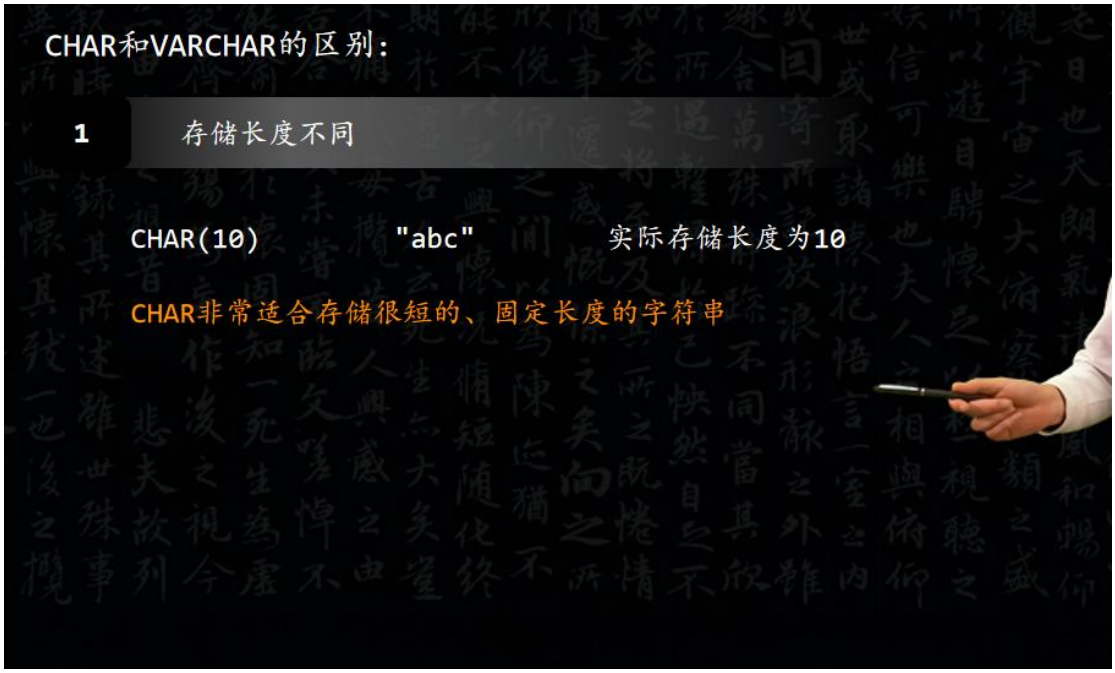
要回答好这个问题，得了解清楚 CHAR 和 VARCHAR 的区别：



第 1，存储长度不同

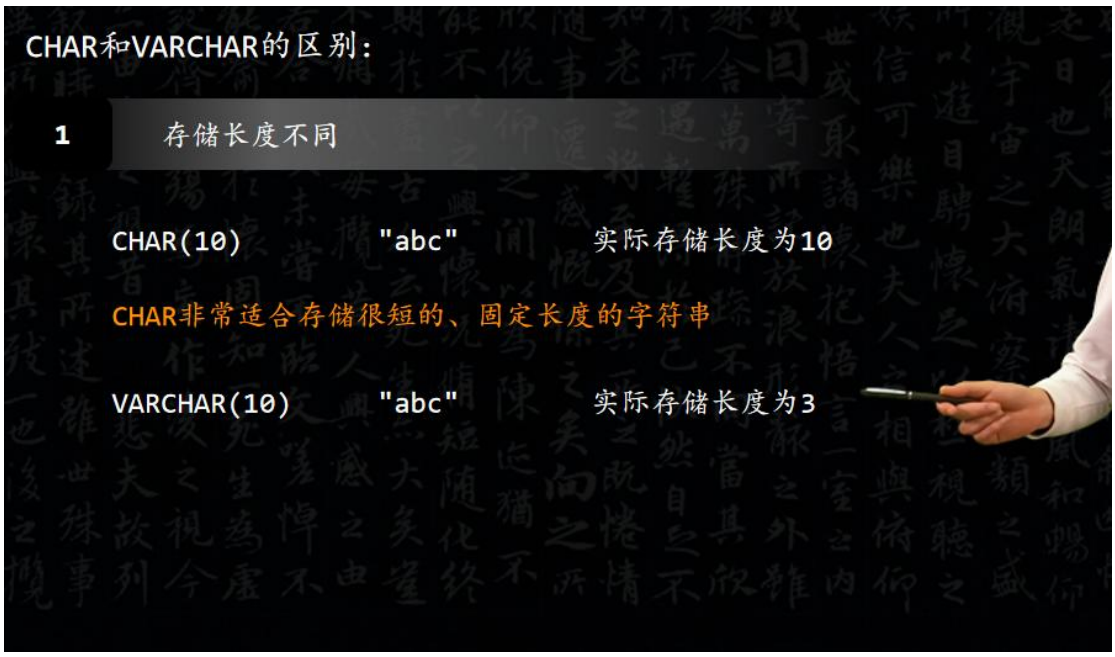
CHAR 类型的长度是固定的，VARCHAR 的长度是可变的。

假设，当我们当定义 CHAR(10)，输入的值是"abc"，但是它占用的空间一样是 10 个字节，会包含 7 个空字节。当输入的字符长度超过指定的数时，CHAR 会截取超出的字符。而且，当存储为 CHAR 的时候，MySQL 会自动删除输入字符串末尾的空格。

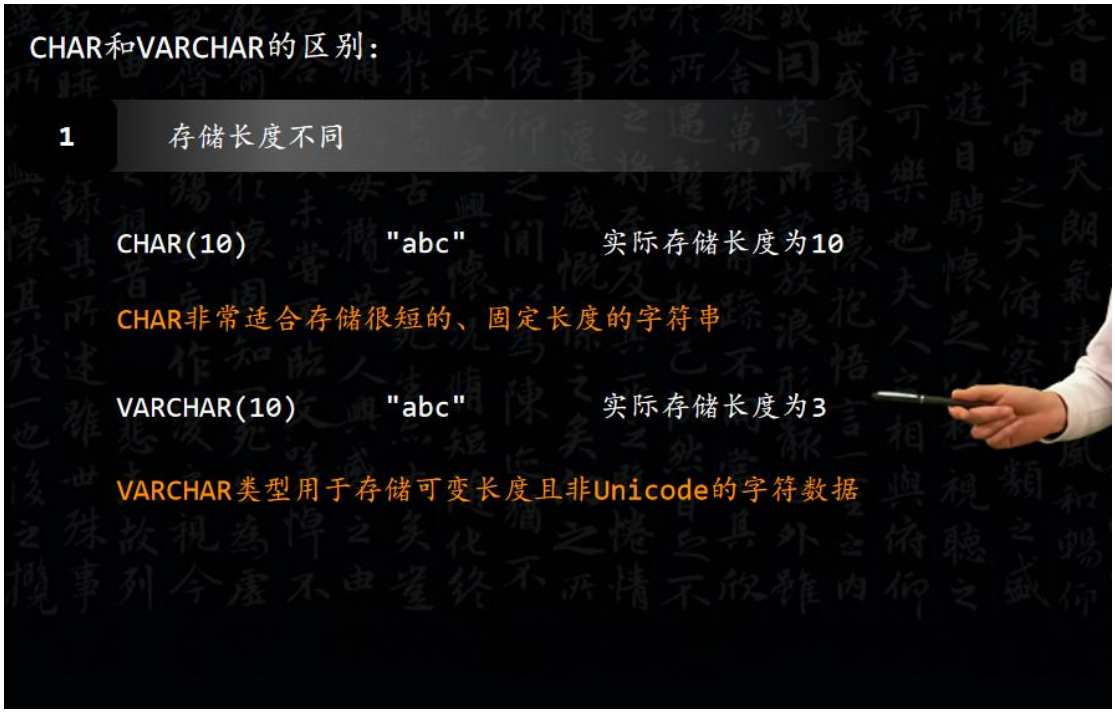


所以，CHAR 非常适合存储很短的、固定长度的字符串。

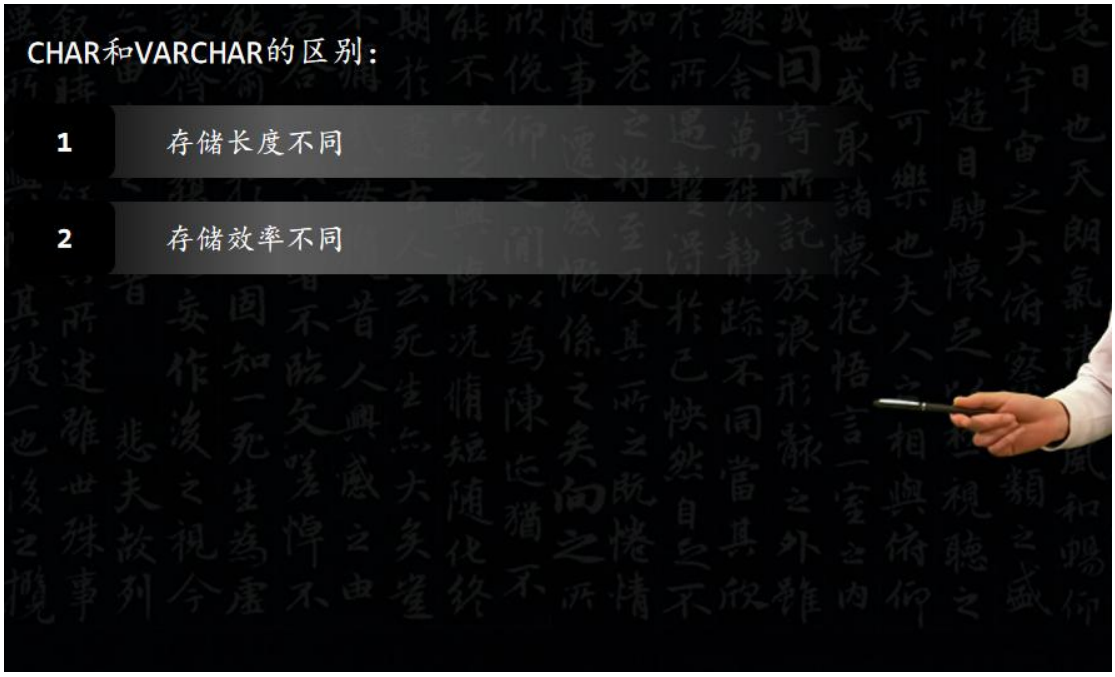
VARCHAR 类型用于存储可变长度且非 Unicode 的字符数据。但可变长度必须在 1 和 8000 之间的数值。



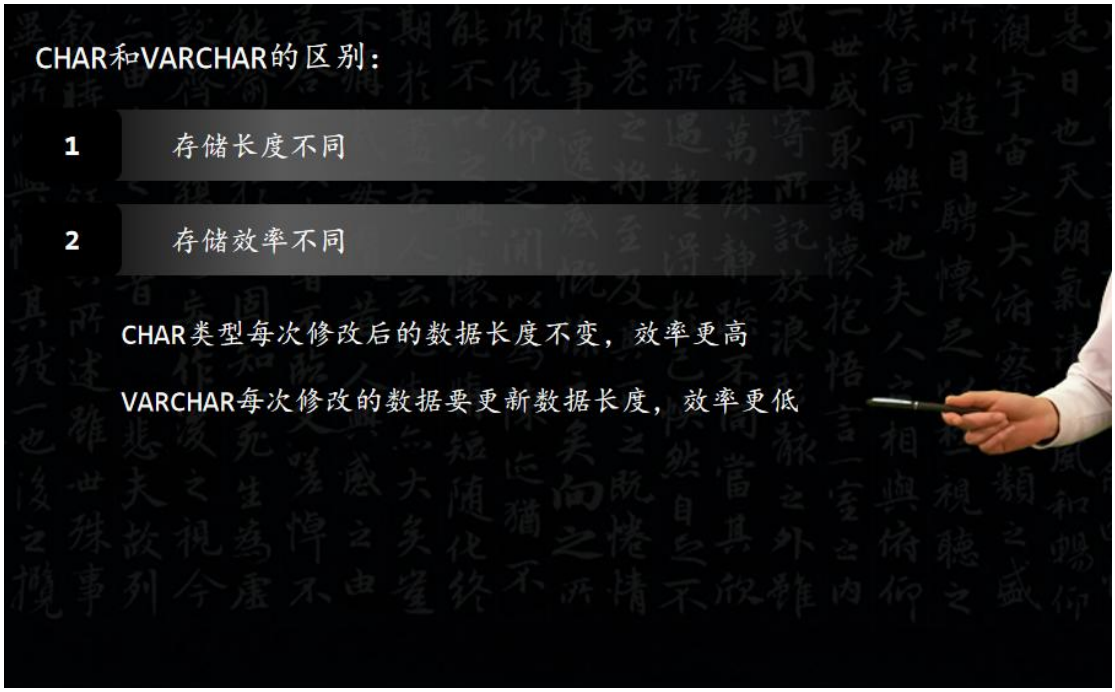
比如 VARCHAR(10)，然后输入 abc 三个字符，那么实际存储大小为 3 个字节。



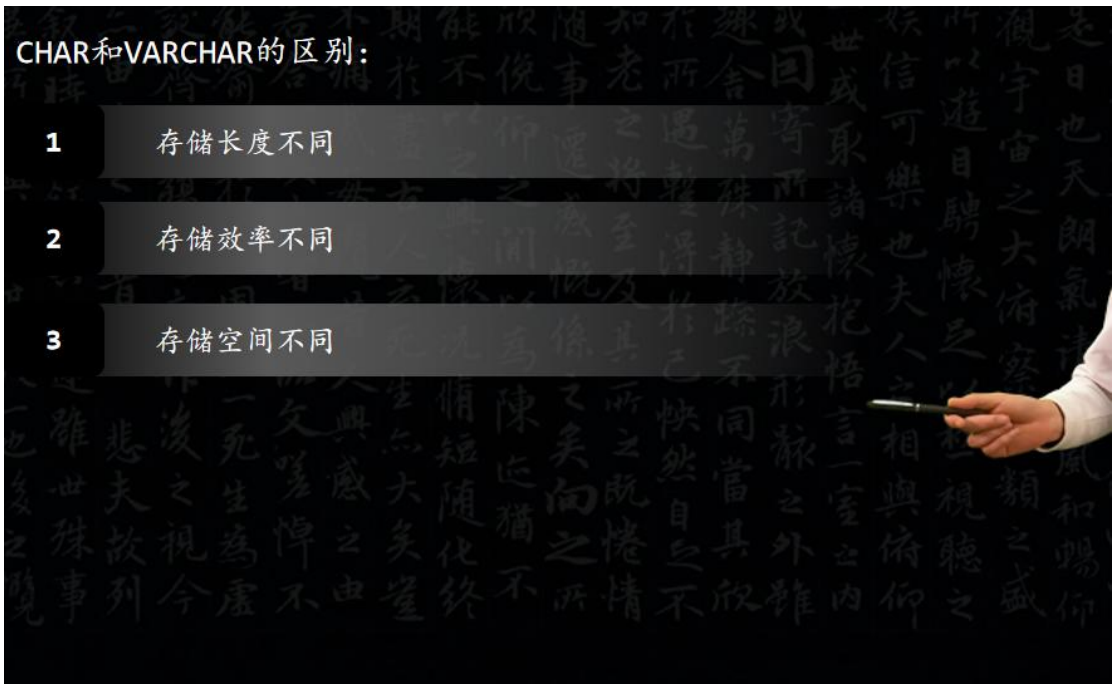
除此之外，VARCHAR 还会保留 1 个或 2 个额外的字节来记录字符串的实际长度。如果定义的最大长度小于等于 255 个字节，那么，就会预留 1 个字节；如果定义的最大长度大于 255 个字节，那么就会预留 2 个字节。



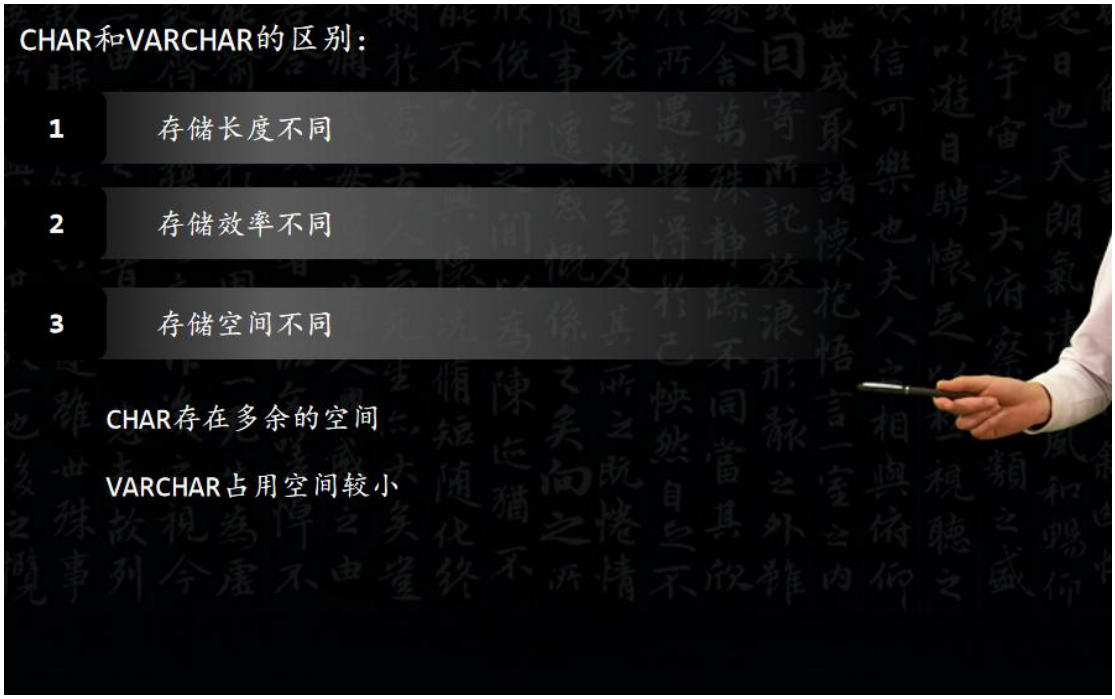
第 2，存储效率不同



CHAR 类型每次修改后的数据长度不变，效率更高。而 VARCHAR 每次修改的数据要更新数据长度，效率更低。



第 3，存储空间不同



CHAR 存储空间是初始的预计长度字符串再加上一个记录字符串长度的字节，可能会存在多余的空间。而 VARCHAR 存储空间的时候是实际字符串再加上一个记录字符串长度的字节，占用空间较小。

53.326.2 答案分析

根据以上的分析，CHAR 非常适合存储 MD5 值。因为 MD5 是一个定长的值，对于固定长度的非常短的列，CHAR 比 VARCHAR 效率也更高。

【结尾】

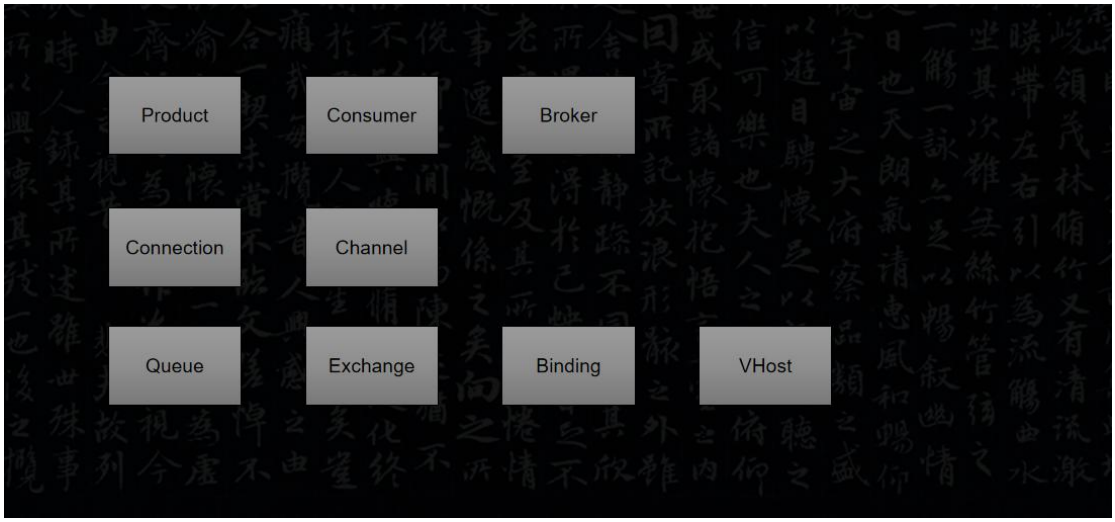
最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

53.327 谈谈你对 RabbitMQ 工作原理的理解

一个 5 年工作经验的小伙伴，在面试的时候被这样一个问题。谈谈你对 RabbitMQ 架构原理的理解。当时，这位小伙伴只解答说，我只会用，原理并没有关注过。那今天我给大家来分享一下我的理解。

53.327.1 核心组件

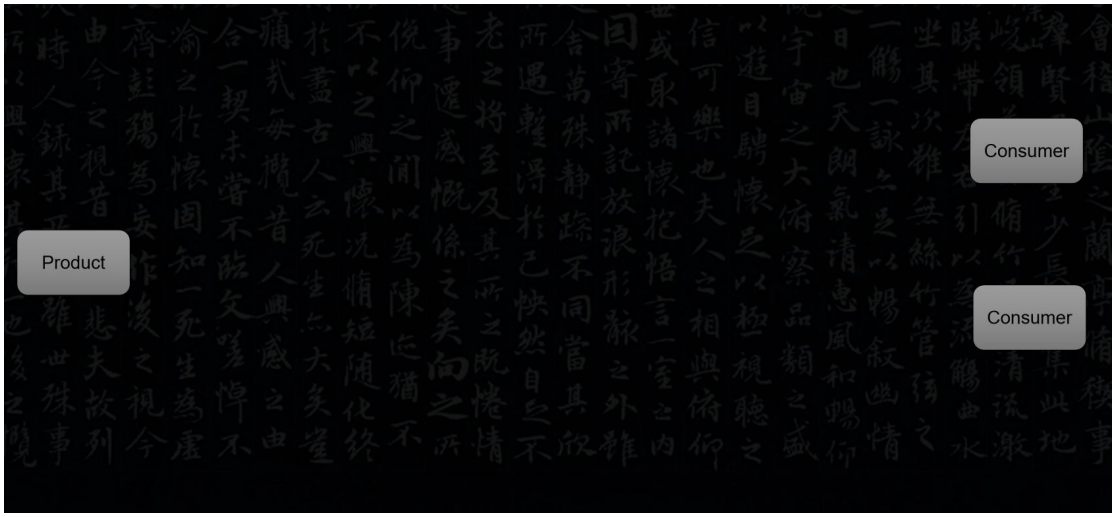
要回答这个问题呢，首先要知道 RabbitMQ 架构中几个核心组件，



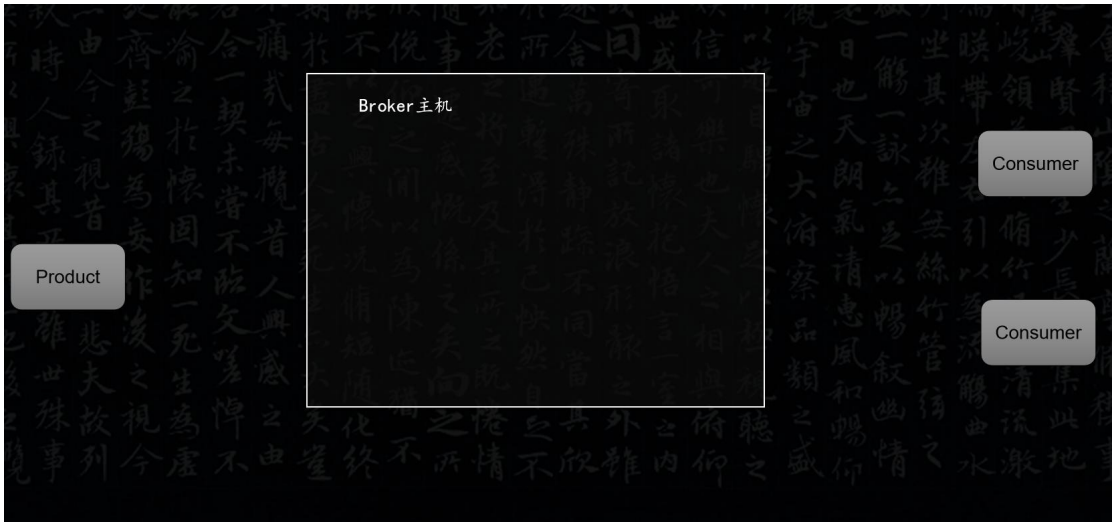
它们分别是 Product、Cunsumer、Broker、Connection、Channel、Queue、Exchange、Binding、VHost。

53.327.2 工作原理

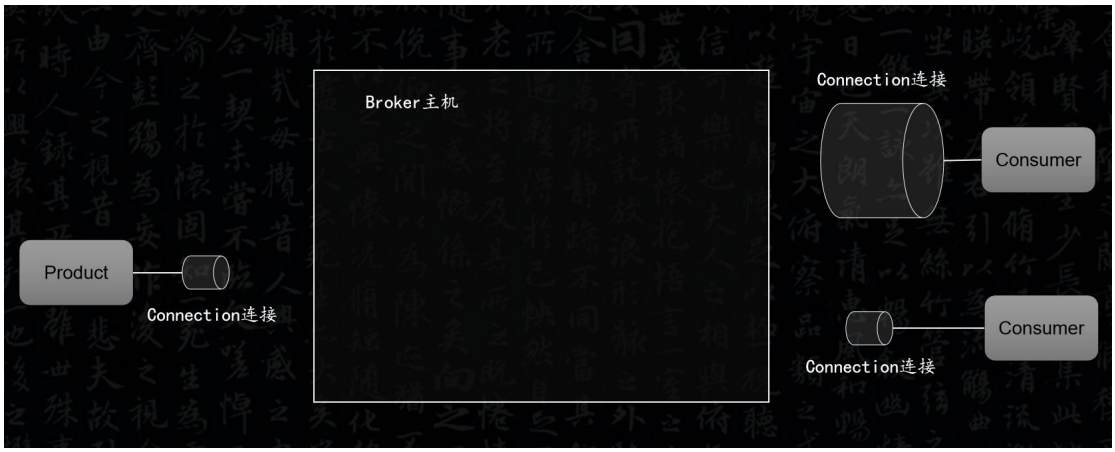
那么，这些组件之间是如何协调工作的呢？下面来看这么一个动画：



首先是，第 1 个和第 2 个，Product 和 Consumer 分别是消息的生产者和消息的消费者，这是所有的消息中间件都有两个角色，我就不做过多介绍了。



重点来看，第 3 个叫 Broker，中文翻译过来叫做代理或者中介，我可以把 Broker 理解为就是安装 RabbitMQ 的服务器，就像是一栋用来存储和转发消息的房子。

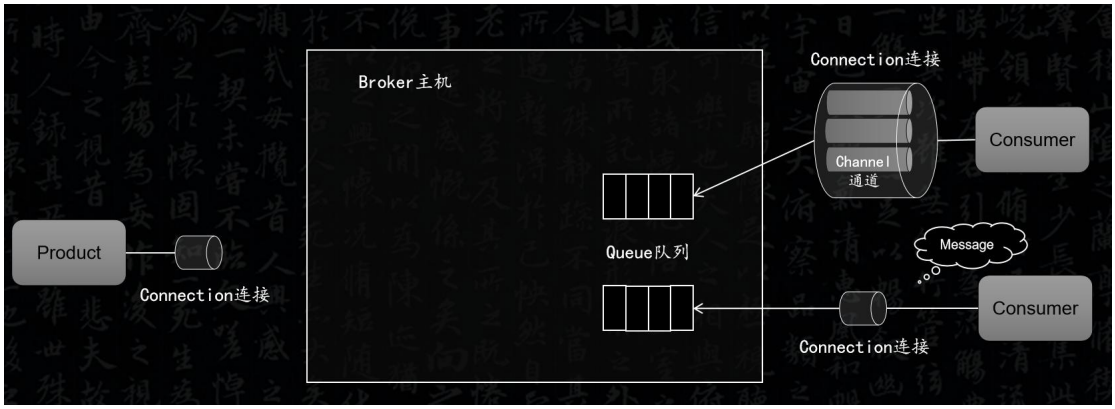


第 4 个就是，Connection 连接。无论是生产者发送消息，还是消费者接收消息，都必须要跟 Broker 之间建立一个连接，那 Connection 呢就相当于是一个 TCP 的长连接。

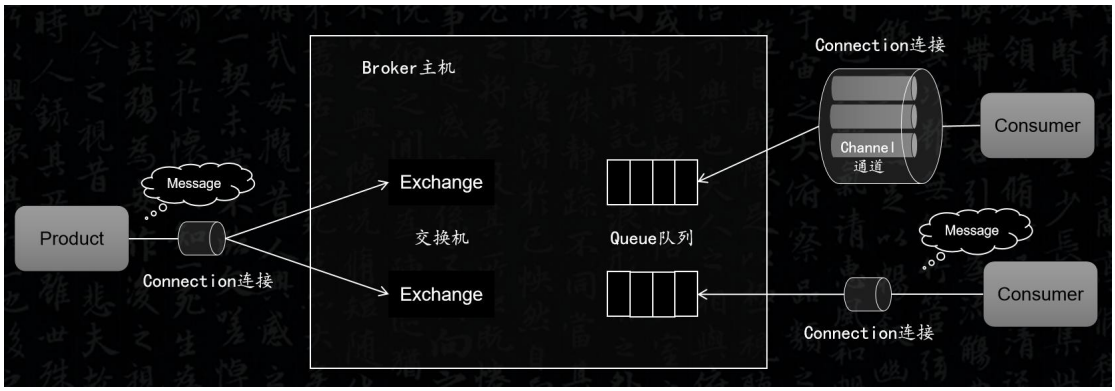
第 5 个 Channel 通道。如果所有的生产者发送消息和消费者接收消息，每次都创建和释放 TCP 长连接的话，对于 Broker 来说肯定会造成很大的性能损耗，也会浪费时间。所以在 AMQP 协议里面引入了 Channel 的概念，它相当于是一个虚拟的连接。这样我们就可以在已经连接好的 TCP 长连接里面去创建和释放 Channel，大大了减少了资源消耗。

不同的 Channel 是相互隔离的，每个 Channel 都有自己的编号。对于每个客户端线程来说，Channel 就没必要共享了，各自用自己的 Channel。

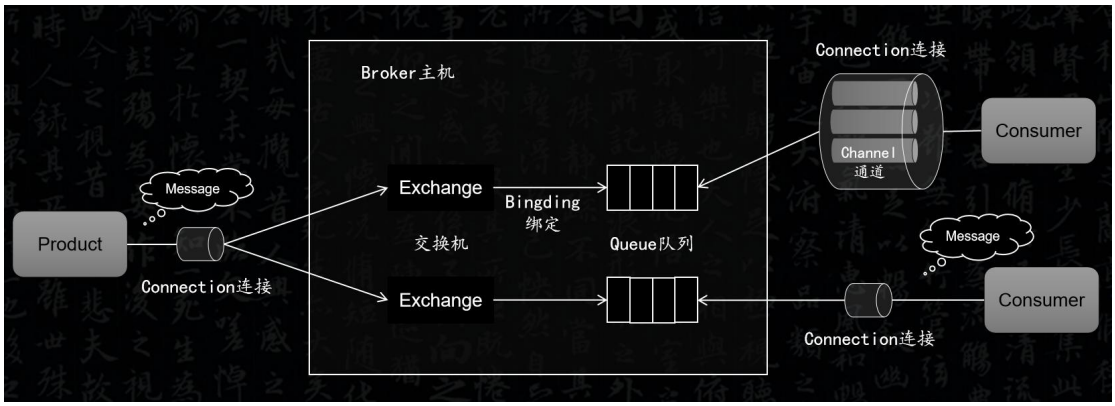
另外一个需要注意的是，Channel 是 RabbitMQ 原生 API 里面的最重要的编程接口，也就是说我们定义交换机、队列、绑定关系，发送消息，消费消息，调用的都是 Channel 接口上的方法。



第 6 个，Queue 队列。它专门用来存储消息，Queue 也是生产者和消费者的纽带，生产者发送的消息会存储到队列中，而消费者也是从队列中来消费消息。



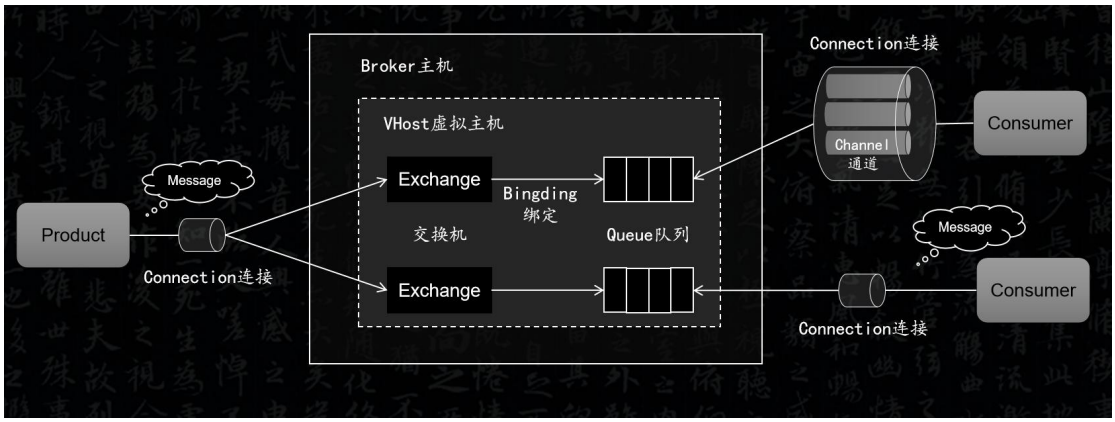
第 7 个，就是 Exchange 交换机。相当于消息的路由器。Exchange 不会存储消息，它只做一件事情，根据规则分发消息。



第 8 个，就是 Binding 绑定。Exchange 和存储消息的队列必须建立一个绑定关系，并且为每个队列指定一个特殊的标识。

Exchange 和队列是多对多的绑定关系，也就说，一个交换机的消息一个路由给多个队列，一个队列也可以接收来自多个交换机的消息。

绑定关系建立好之后，生产者发送消息到 Exchange，也会携带一个特殊的标识。当这个标识跟绑定的标识匹配的时候，消息就会发给一个或者多个符合规则的队列。



最后一个就是 Vhost，全称叫 Virtual Host 虚拟机。为了解决不同业务系统之间的消息隔离，节约硬件成本，我们可以利用 RabbitMQ 的 Vhost 来实现资源的隔离和权限的控制。它的功能和其他编程语言中的 NameSpace 比较类似。

【结尾】

以上就是我对 RabiitMQ 的架构原理的理解，，小伙伴们，你是不是也理解得更加透彻了呢?

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

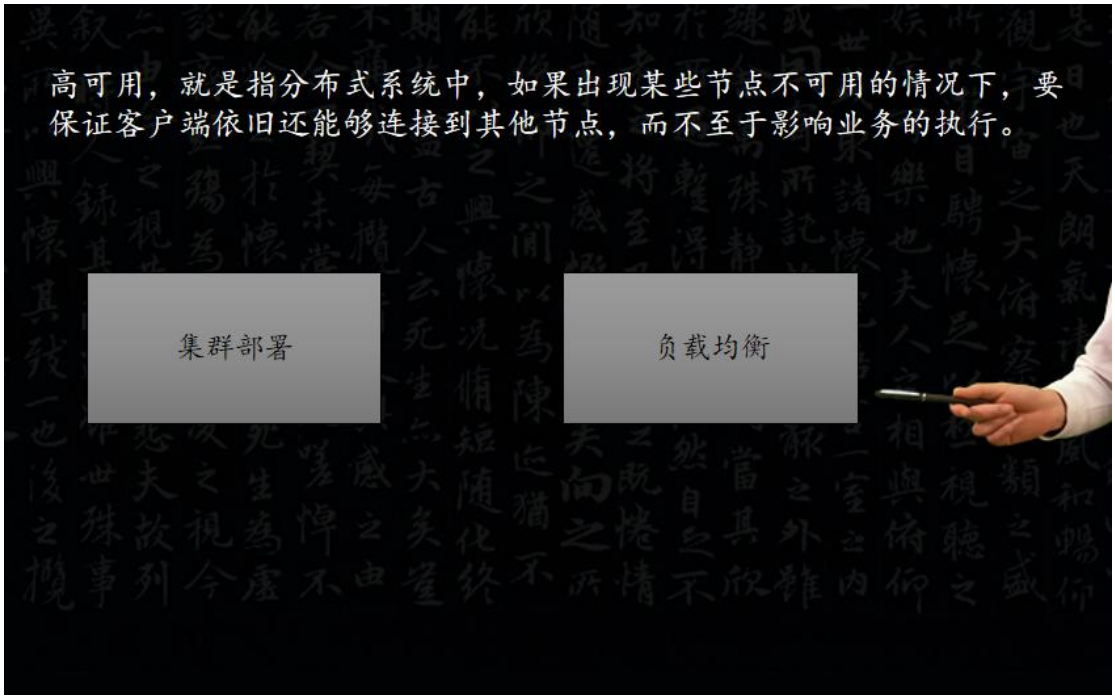
我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

53.328 如何保证 RabbitMQ 的高可用

一个 8 年工作经验的小伙伴，被问到这样一个问题，说如何保证 RabbitMQ 的高可用。关于这个问题呢，这位小伙伴倒是有个实操经验，就是不知道如何组织语言。所以，当时面试结果不太理想。今天，我给大家分享一下我的理解。

53.328.1 实现思路

所谓高可用呢，就是指分布式系统中，如果出现某些节点不可用的情况下，要保证客户端依旧还能够连接到其他节点，而不至于影响业务的执行。我主要从以下两个方面来实现高可用，一个是集群部署，一个是负载均衡。



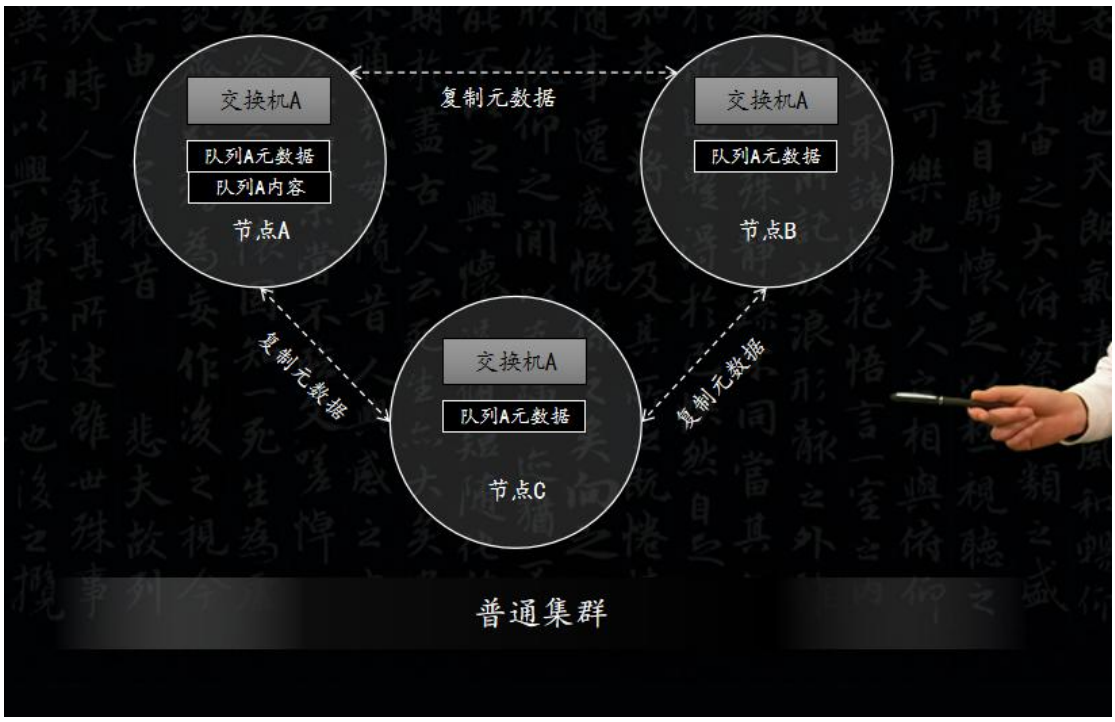
RabbitMQ 是用 Erlang 语言编写的，而 Erlang 天生具备分布式的特性，所以不需要通过引入 ZK 来实现服务协调。它是使用.erlang.cookie 来验证身份，它通过 25672 端口来实现集群。这个 cookie 就像暗号一样，只要喊出清楚明白 666，就知道是 Tom 的粉丝。

另外，RabbitMQ 的集群节点有两种，一种是磁盘节点（Disc Node），一种是内存节点（RAM Node）。在集群环境中至少需要一个磁盘节点用来持久化元数据，否则，当全部内存节点崩溃时，就无从同步元数据。

下面，分享一下 RabbitMQ 的集群模式，分别是普通集群模式和镜像队列模式。先来看普通集群模式。

53.328.2 普通集群

普通集群模式下，不同的节点之间只会相互同步元数据，而不会同步消息内容。



如图所示，队列 A 的消息只存储在 A 节点上。B 节点和 C 节点同步了队列 A 的元数据，但是没有同步队列中的内容。

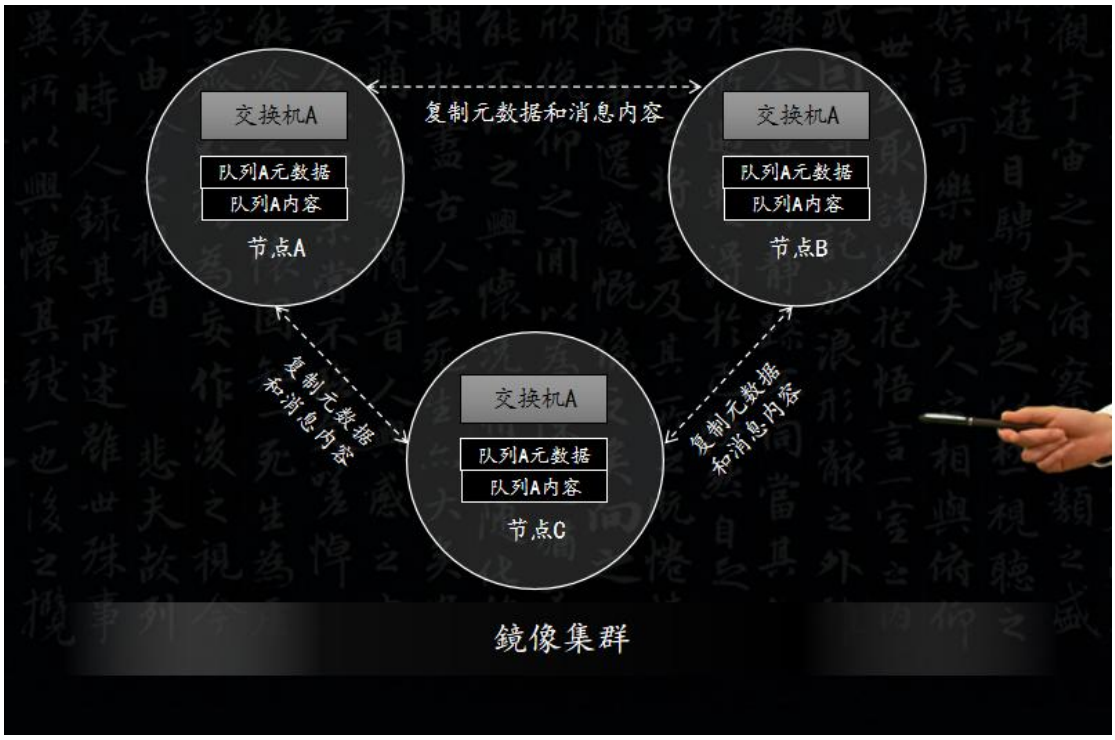
假如生产者连接的是 C 节点，要将消息通过交换机 A 路由到队列 A，最终消息还是会转发到 A 节点上存储，因为队列 A 存储在 A 节点上。

同理，如果消费者连接是 B 节点，同样也要从队列 A 上拉取消息，消息会从 A 节点转发到节点 B。其它节点只是起到一个路由的作用。

这样做的好处是，可以节约存储和同步数据的网络开销。但如果需要保证队列的高可用性，这时候，就需要用到镜像集群的模式。

53.328.3 镜像集群

下面来看镜像集群模式，如图所示，



其实也非常好理解，在镜像队列模式下，消息内容会在所有的镜像节点间同步，可用性更高。不过也有一定的副作用，系统性能会降低，节点过多的情况下同步数据的代价会比较大。

53.328.4 负载均衡

集群搭建成功后，要保证高可用，还需要一个负载均衡的组件（例如 HAProxy，LVS，Nignx），由负载的组件来做路由。对于，生产者或者消费者而言，只需要连接到负载组件的虚拟 IP 地址就可以了。

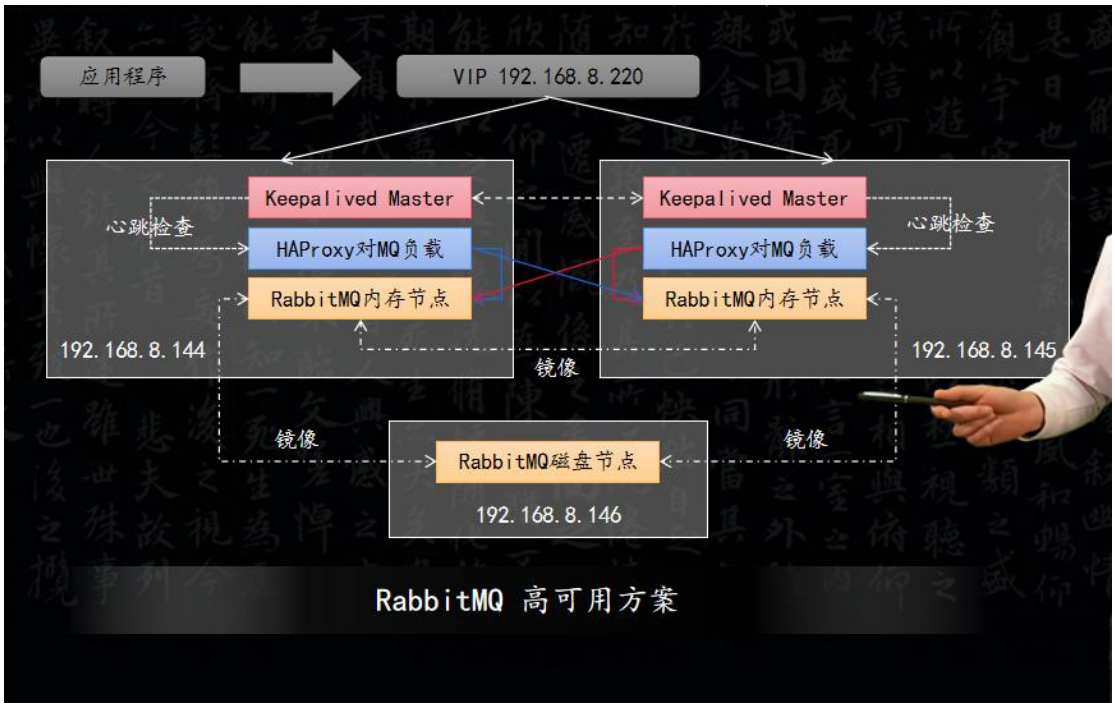
这个负载均衡组件需要满足以下几个条件：

1、它本身有负载功能，可以监控集群中节点的状态，如果某个节点出现异常或者发生故障，就把它剔除掉。

2、为了提高可用性，要能够支持部署多个服务，能够自主完成 Master 选举

3、Master 节点需要对外提供一个虚拟 IP，客户端只需要连接到这一个虚拟 IP 就可以自动完成真实 IP 的路由。

下面我以 HAProxy 加 Keepalived 组合，来实现 RabbitMQ 的高可用，具体架构如图所示：



当然，实现高可用的方案还有很多，比如我们还可以用 LVS，Nignx 等等来实现负载均衡。

【结尾】

关于 RabbitMQ 的高可用方案我就分享这么。我分享的这个方案，不仅仅只适用于 RabbitMQ，其实还可以应用于大部分的分布式高可用场景。小伙伴们如果你还有其他更加牛 B 的方案，可以在评论区告诉我相互学习一下。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺 Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.329 日常工作中你是怎么优化 SQL 的?

大家好，我是 MIC，一个工作了 13 年的 Java 程序员。

MySQL 作为一种免费开源的关系型数据库，深受着互联网公司的喜爱。

因此，它也成为了技术面试官最常问的问题之一。

今天，我们就通过高手的形式，带大家深入了解 MySQL 的性能优化技巧。

以下为高手的回答

MySQL 的性能优化我认为可以分为 4 大部分

6. 丨硬件和操作系统层面的优化
7. 丨架构设计层面的优化
8. 丨MySQL 程序配置优化
9. 丨SQL 优化

硬件及操作系统层面优化

从硬件层面来说，影响 Mysql 性能的因素有，CPU、可用内存大小、磁盘读写速度、网络带宽

从操作系层面来说，应用文件句柄数、操作系统网络的配置都会影响到 Mysql 性能。这部分的优化一般由 DBA 或者运维工程师去完成。

在硬件基础资源的优化中，我们重点应该关注服务本身承载的体量，然后提出合理的指标要求，避免出现资源浪费！

架构设计层面的优化

MySQL 是一个磁盘 IO 访问量非常频繁的关系型数据库

在高并发和高性能的场景中.MySQL 数据库必然会承受巨大的并发压力，而此时，我们的优化方式可以分为几个部分。

1. 搭建 Mysql 主从集群，单个 Mysql 服务容易单点故障，一旦服务器宕机，将会导致依赖 Mysql 数据库的应用全部无法响应。 主从集群或者主主集群可以保证服务的高可用性。
2. 读写分离设计，在读多写少的场景中，通过读写分离的方案，可以避免读写冲突导致的性能影响
3. 引入分库分表机制，通过分库可以降低单个服务器节点的 IO 压力，通过分表的方式可以降低单表数据量，从而提升 sql 查询的效率。
4. 针对热点数据，可以引入更为高效的分布式数据库，比如 Redis、MongoDB 等，他们可以很好的缓解 Mysql 的访问压力，同时还能提升数据检索性能。

MySQL 程序配置优化

MySQL 是一个经过互联网大厂验证过的生产级别的成熟数据库，对于 Mysql 数据库本身的优化，一般是通过 Mysql 中的配置文件 my.cnf 来完成的，比如。

Mysql5.7 版本默认的最大连接数是 151 个，这个值可以在 my.cnf 中修改。

binlog 日志，默认是不开启

缓存池 bufferpoll 的默认大小配置等。

由于这些配置一般都和用户安装的硬件环境以及使用场景有关系，因此这些配置官方只会提供一个默认值，具体情况还得由使用者来修改。

关于配置项的修改，需要关注两个方面。

10. 丨配置的作用域，分为会话级别和全局

11. 丨是否支持热加载

因此，针对这两个点，我们需要注意的是：

12. 丨全局参数的设定对于已经存在的会话无法生效

13. 丨会话参数的设定随着会话的销毁而失效

14. 丨全局类的统一配置建议配置在默认配置文件中，否则重启服务会导致配置失效

SQL 优化又能分为三步曲

丨第一、慢 SQL 的定位和排查

我们可以通过慢查询日志和慢查询日志分析工具得到有问题的 SQL 列表。

丨第二、执行计划分析

针对慢 SQL，我们可以使用关键字 explain 来查看当前 sql 的执行计划.可以重点关注 type key rows filterd 等字段 ，从而定位该 SQL 执行慢的根本原因。再有的放矢的进行优化

丨第三、使用 show profile 工具

Show Profile 是 MySQL 提供的可以用来分析当前会话中，SQL 语句资源消耗情况的工具，可用于 SQL 调优的测量。在当前会话中.默认情况下处于 show profile 是关闭状态，打开之后保存最近 15 次的运行结果

针对运行慢的 SQL，通过 profile 工具进行详细分析.可以得到 SQL 执行过程中所有的资源开销情况.

如 IO 开销,CPU 开销,内存开销等.

以上就是我对 MySQL 性能优化的理解。

53.330 说说分库与分表的设计

53.330.1 分库分表方案

1. 水平分库：以字段为依据，按照一定策略（hash、range 等），将一个库中的数据拆分到多个库中。
2. 水平分表：以字段为依据，按照一定策略（hash、range 等），将一个表中的数据拆分到多个表中。
3. 垂直分库：以表为依据，按照业务归属不同，将不同的表拆分到不同的库中。
4. 垂直分表：以字段为依据，按照字段的活跃性，将表中字段拆到不同的表（主表和扩展表）中。

53.330.2 常用的分库分表中间件：

5. sharding-jdbc（当当）
6. Mycat
7. TDDL（淘宝）
8. Oceanus(58 同城数据库中间件)
9. vitess（谷歌开发的数据库中间件）
10. Atlas(Qihoo 360)

53.330.3 分库分表可能遇到的问题

11. 事务问题：需要用分布式事务啦
12. 跨节点 Join 的问题：解决这一问题可以分两次查询实现
13. 跨节点的 count,order by,group by 以及聚合函数问题：分别在各个节点上得到结果后在应用程序端进行合并。
14. 数据迁移，容量规划，扩容等问题
15. ID 问题：数据库被切分后，不能再依赖数据库自身的主键生成机制啦，最简单可以考虑 UUID
16. 跨分片的排序分页问题（后台加大 pagesize 处理？）

53.330.4 三种应用场景

i. 只分库不分表

当数据库的读写访问量过高，还有可能会出现数据库连接不够用的情况。这个时候我们就需要考虑分库，通过增加数据库实例的方式来获得更多的数据库连接，从而提升系统的并发性能。

ii. 只分表不分库

当单表存储的数据量非常大的情况下，并且并发量也不高，数据库的连接也还够用。但是数据写入和查询的性能出现了瓶颈，这个时候就需要考虑分表了。将数据拆分到多张表中来减少单表存储的数据量，从而提升读写的效率。

iii. 既分库又分表

结合前面的两种情况，如果同时满足前面的两个条件，也就是数据连接也不够用，并且单表的数据量也很大，从而导致数据库读写速度变慢的情况，这个时候就要考虑既分库又分表。

53.331 InnoDB 与 MyISAM 的区别

“请简要说明 Mysql 中 MyISAM 和 InnoDB 引擎的区别”。

屏幕前有多少同学在面试过程与遇到过类似问题，可以在评论区留言：遇到过。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这个问题面试官想考察什么？我们又该怎么回答呢？

53.331.1 考察目的

对于 xxxx 技术的区别，在面试中是很常见的一个问题

一般情况下，面试官会通过这类问题来热场，打开接下来沟通的话题，然后沿着你回答的内容层层递进去做更深入的了解。

当然，另外一个更加深层次的原因，就是考察求职者对于这两个技术的理解层次。

因为能够通过自己的理解总结出他们的区别，至少说明你是有比较深入的研究的。

这个问题考察难度算是比较大的，一般面向 3 年以上开发经验的同学。

53.331.2 问题解析

MyISAM 和 InnoDB 都是 Mysql 里面的两个存储引擎。

在 Mysql 里面，存储引擎是可以自己扩展的，它的本质其实是定义数据存储的方式以及数据读取的实现逻辑。

而不同存储引擎本身的特性，使得我们可以针对性的选择合适的引擎来实现不同的业务场景。

从而获得更好的性能。

在 Mysql 5.5 之前，默认的存储引擎是 MyISAM，从 5.5 以后，InnoDB 就作为了默认的存储引擎。

在实际应用开发中，我们基本上都是采用 InnoDB 引擎。

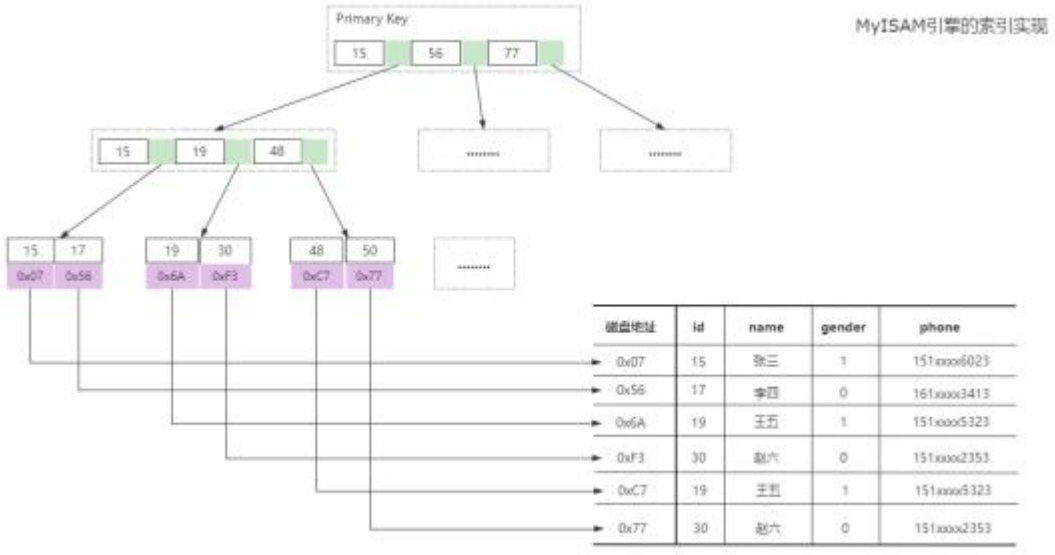
我们先来看一下 MyISAM 引擎。

MyISAM 引擎的数据是通过二进制的方式存储在磁盘上，它在磁盘上体现为两个文件

一个是.MYD 文件，D 代表 Data，是 MyISAM 的数据文件，存放数据记录，

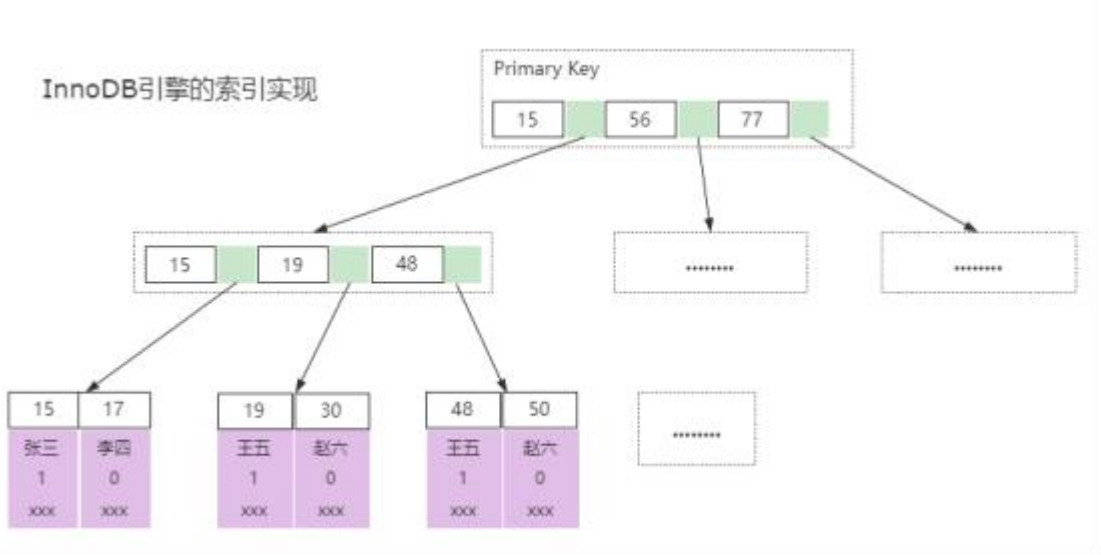
一个是.MYI 文件，I 代表 Index，是 MyISAM 的索引文件，存放索引实现机制如图所示。

因为索引和数据是分离的，所以在进行查找的时候，先从索引文件中找到数据的磁盘位置，再到数据文件中找到索引对应的数据内容。



在 InnoDB 存储引擎中，数据同样存储在磁盘上，它在磁盘上只有一个 ibd 文件，里面包含索引和数据。

它的整体结构如图所示，在 B+树的叶子节点里面存储了索引对应的数据，在通过索引进行检索的时候，命中叶子节点，就可以直接从叶子节点中取出行数据。



了解了这两个存储引擎以后，我们在面试的时候该怎么回答呢?

53.331.3 高手回答

基于我的理解，我认为 MyISAm 和 InnoDB 的区别有 4 个，第一个，数据存储的方式不同，MyISAM 中的数据和索引是分开存储的，而 InnoDB 是把索引和数据存储在同一个文件里面。

第二个，对于事务的支持不同，MyISAM 不支持事务，而 InnoDB 支持 ACID 特性的事务处理

第三个，对于锁的支持不同，MyISAM 只支持表锁，而 InnoDB 可以根据不同的情况，支持行锁，表锁，间隙锁，临键锁

第四个，MyISAM 不支持外键，InnoDB 支持外键因此基于这些特性，我们在实际应用中，可以根据不同的场景来选择合适的存储引擎。

比如如果需要支持事务，那必须要选择 InnoDB。

如果大部分的表操作都是查询，可以选择 MyISAM。

以上就是我的理解。

53.332 数据库索引的原理，为什么要用 B+树，为什么不用二叉树?

一个工作 8 年的粉丝私信了我一个问题。

他说这个问题是去阿里面试的时候被问到的，自己查了很多资料也没搞明白，希望我帮他解答。

问题是： “Mysql 为什么使用 B+Tree 作为索引结构”

以下为高手的回答

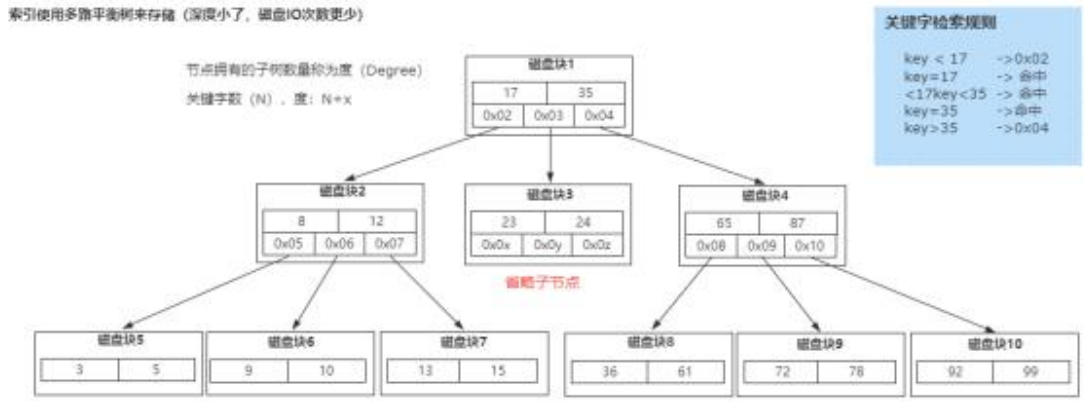
关于这个问题，我从几个方面来回答。

首先，常规的数据库存储引擎，一般都是采用 B 树或者 B+树来实现索引的存储。

因为 B 树是一种多路平衡树，用这种存储结构来存储大量数据，它的整个高度会相比二叉树来说，会矮很多。

而对于数据库来说，所有的数据必然都是存储在磁盘上的，而磁盘 IO 的效率实际上是很低的，特别是在随机磁盘 IO 的情况下效率更低。

所以树的高度能够决定磁盘 IO 的次数，磁盘 IO 次数越少，对于性能的提升就越大，这也是为什么采用 B 树作为索引存储结构的原因。

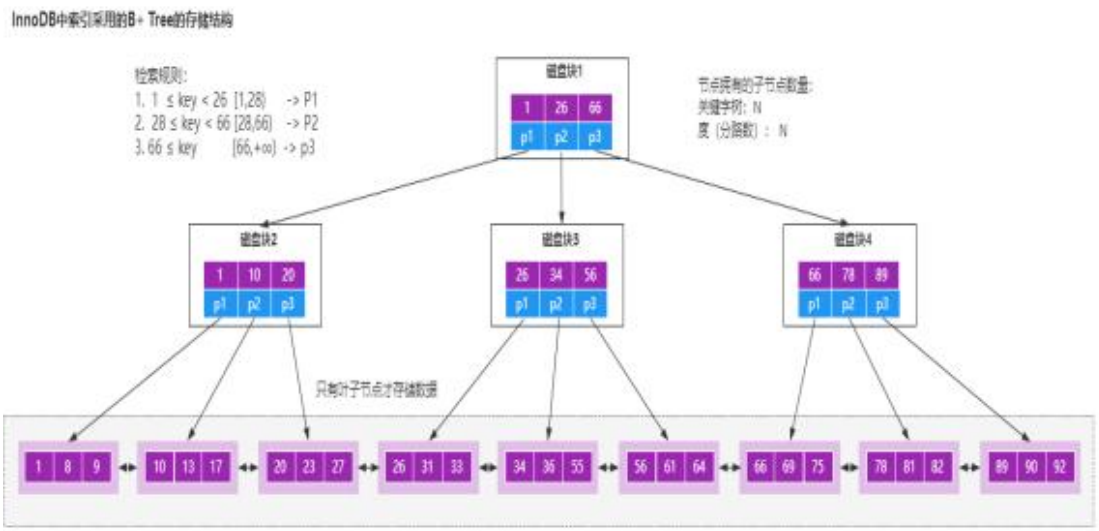


但是在 Mysql 的 InnoDB 存储引擎里面，它用了一种增强的 B 树结构，也就是 B+树来作为索引和数据的存储结构。

相比较于 B 树结构，B+树做了几个方面的优化。

B+树的所有数据都存储在叶子节点，非叶子节点只存储索引。

叶子节点中的数据使用双向链表的方式进行关联。



使用 B+树来实现索引的原因，我认为有几个方面。

B+树非叶子节点不存储数据，所以每一层能够存储的索引数量会增加，意味着 B+树在层高相同的情况下存储的数据量要比 B 树要多，使得磁盘 IO 次数更少。

在 Mysql 里面，范围查询是一个比较常用的操作，而 B+树的所有存储在叶子节点的数据使用了双向链表来关联，所以在查询的时候只需查两个节点进行遍历就行，而 B 树需要获取所有节点，所以 B+树在范围查询上效率更高。

在数据检索方面，由于所有的数据都存储在叶子节点，所以 B+树的 IO 次数会更加稳定一些。

因为叶子节点存储所有数据，所以 B+树的全局扫描能力更强一些，因为它只需要扫描叶子节点。但是 B 树需要遍历整个树。

另外，基于 B+树这样一种结构，如果采用自增的整型数据作为主键，还能更好的避免增加数据的时候，带来叶子节点分裂导致的大量运算的问题。

总的来说，我认为技术方案的选型，更多的是去解决当前场景下的特定问题，并不一定是说 B+树就是最好的选择，就像 MongoDB 里面采用 B 树结构，本质上来说，其实是关系型数据库和非关系型数据库的差异。

以上就是我对这个问题的理解。

53.333 聚集索引与非聚集索引的区别

一个去阿里面试并且第一面就挂了的粉丝私信我，被数据库里面几个问题难倒了，他说面试官问了事务隔离级别、MVCC、聚集索引/非聚集索引、B 树、B+树这些，没回答好。

大厂面试基本上是这样，由点到面去展开，如果你对这个技术理解不够全面，很容易就会被看出来。

ok，关于“什么是聚集索引和非聚集索引”这个问题，看看普通人和高手的回答。

以下为高手的回答

简单来说，聚集索引就是基于主键创建的索引，除了主键索引以外的其他索引，称为非聚集索引，也叫做二级索引。

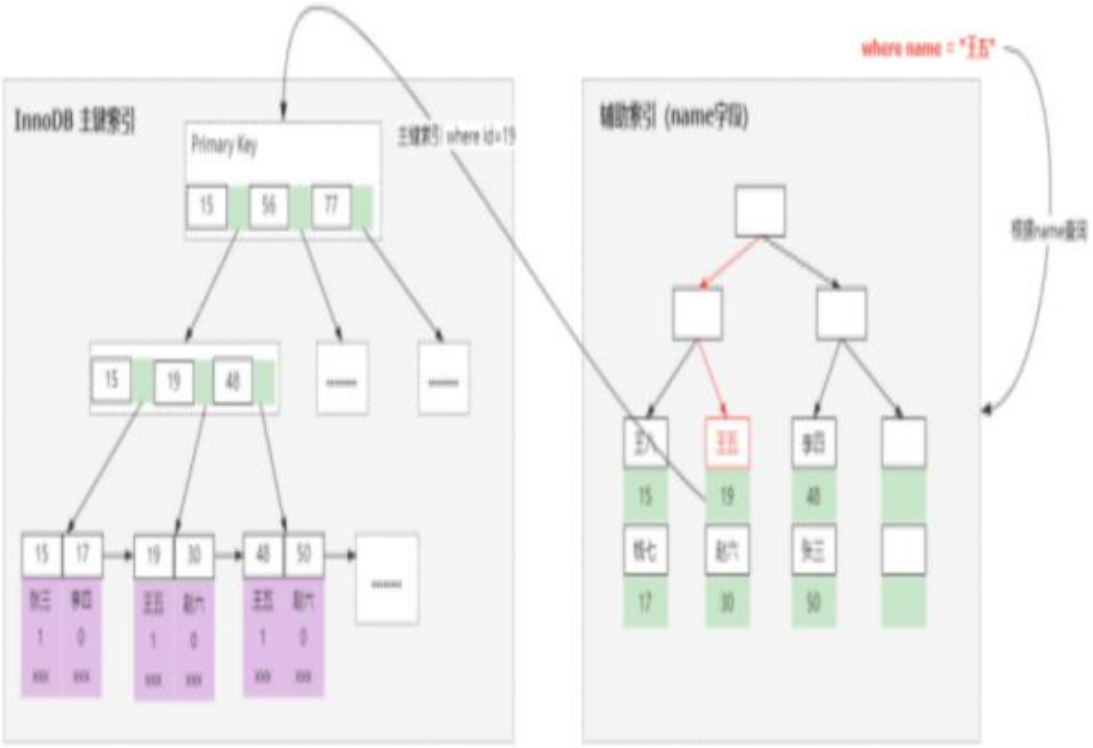
由于在 InnoDB 引擎里面，一张表的数据对应的物理文件本身就是按照 B+树来组织的一种索引结构，而聚集索引就是按照每张表的主键来构建一颗 B+树，然后叶子节点里面存储了这个表的每一行数据记录。

所以基于 InnoDB 这样的特性，聚集索引并不仅仅是一种索引类型，还代表着一种数据的存储方式。

同时也意味着每个表里面必须要有一个主键，如果没有主键，InnoDB 会默认选择或者添加一个隐藏列作为主键索引来存储这个表的数据行。一般情况是建议使用自增 id 作为主键，这样的话 id 本身具有连续性使得对应的数据也会按照顺序存储在磁盘上，写入性能和检索性能都很高。否则，如果使用 uuid 这种随机 id，那么在频繁插入数据的时候，就会导致随机磁盘 IO，从而导致性能较低。

需要注意的是，InnoDB 里面只能存在一个聚集索引，原因很简单，如果存在多个聚集索引，那么意味着这个表里面的数据存在多个副本，造成磁盘空间的浪费，以及数据维护的困难。

由于在 InnoDB 里面，主键索引表示的是一种数据存储结构，所以如果是基于非聚集索引来查询一条完整的记录，最终还是需要访问主键索引来检索。



54.334　MySQL 遇到过死锁问题吗，你是如何解决的?

“如何解决死锁问题？”

如果你遇到这个问题，不知道该怎么回答，一定要看完这个视频。

Hi，大家好，我是 Mic，一个工作了 14 年的 Java 程序员

这个问题，面试官的考察目的是什么呢?

53.334.1 考察目标

这个问题还是有一点难度，首先他考察的是并发编程相关领域的知识。

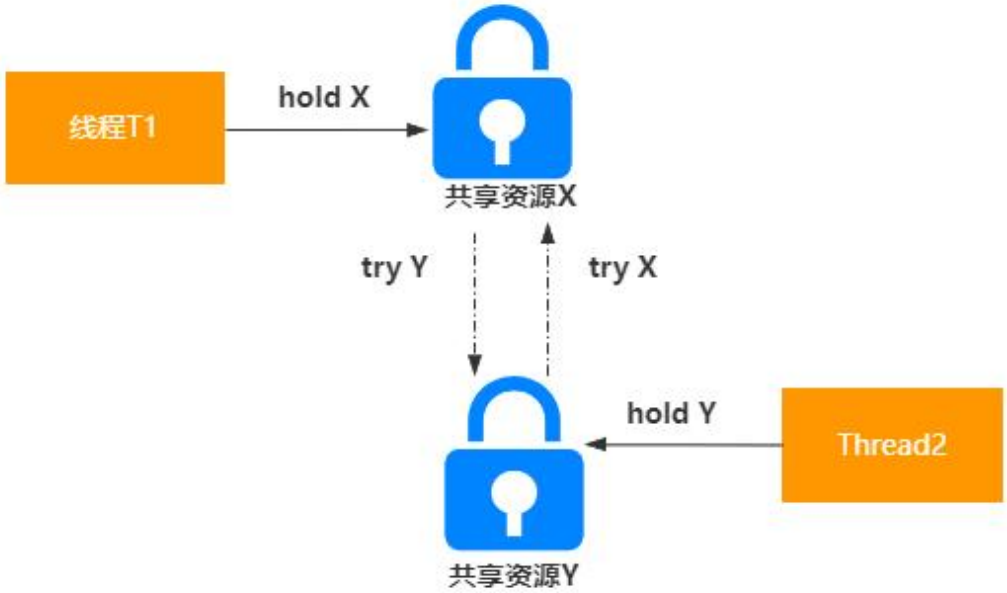
其次，对于死锁这个问题，平时我们遇到得比较少，即便是看过相关的问题，也不一定能够记住

所以，在一定程度上，能够回答清楚这个问题，说明这个求职者的基本功还不错。

53.334.2 问题解析

死锁，就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致互相等待的现象。

在没有外部干预的情况下，线程会一直处于阻塞状态，无法往下执行。



不过，要想真正产生死锁，必须同时满足四个条件。

互斥条件，共享资源 x 和 y 只能被一个线程占用

请求和保持条件，线程 t1 已经获取共享资源 x，在等待共享资源 y 的时候，不释放共享资源 x 不可抢占条件，其他线程不能强行抢占线程 t1 占有的资源

循环等待条件，线程 t1 等到线程 t2 占有的资源，线程 t2 等待线程 t1 占有的资源，形成循环等待

线程在产生死锁以后，只能通过外部干预来解决，比如重启、或者 kill 线程等。

所以我们在写代码的时候，就应该去刻意规避死锁的问题。

也就是避免同时满足这四个条件。

在这四个条件里面，互斥条件是锁本身的特性，无法被破坏，其他三个条件都可以被破坏。

对于请求和保持条件，我们可以在第一次执行的时候一次性申请所有的共享资源

对于不可抢占条件，占用部分资源的线程在进一步申请其他资源的时候，如果申请不到，就主动释放它占有的资源。

对于循环等待条件，可以按照顺序来申请资源，相当于给资源编号，按照编号顺序申请就可以避免循环等待。

当然，死锁问题不仅仅局限在多线程领域，单反涉及到互斥锁的地方都有可能出现，

比如 Mysql 数据库的行锁、表锁，以及分布式锁等。

在底层原理上都是相同的。

下面来看看高手的回答。

53.334.3 高手回答

程序出现死锁，是因为在多线程环境里面两个或两个以上的线程同时满足

互斥条件、请求保持条件、不可抢占条件、循环等待条件。

出现死锁以后，可以通过 jstack 命令去导出线程的 dump 日志，

然后从 dump 日志里面定位到具体死锁的程序代码。

通过修改程序代码去破坏这四个条件里面的任意一个，就可以解决死锁问题。

当然，因为互斥条件因为是锁本身的特性，所以不能被破坏。

以上就是我的理解。

54. [2024 真题合集]Java 程序员一线大厂面试真题

大家好，最近有面试了 oppo，给大家整理了面试真题的答案。希望对大家有帮助哈，一起学习，一起进步

54.1 聊聊你印象最深刻的项目，或者做了什么优化。

大家平时做的项目，如果很多知识点跟面试八股文相关的话，就可以相对条理清晰地写到简历去。

比如缓存数据库相关的，查询为空，你设置了一个 -1 到缓存，代表数据库没记录。下次判断 -1 ，就不查库了，以解决缓存穿透问题。

又比如你设置缓存过期时间比较分散，解决缓存击穿问题，都可以条理清晰写到简历去，这样面试官

很可能会问你相关的问题，这时候就对答如流啦。

还有平时你做的项目，有一些比较好的设计，都可以说一下哈，比如你是如何保证数据一致性的，怎么优化化接口性能的。

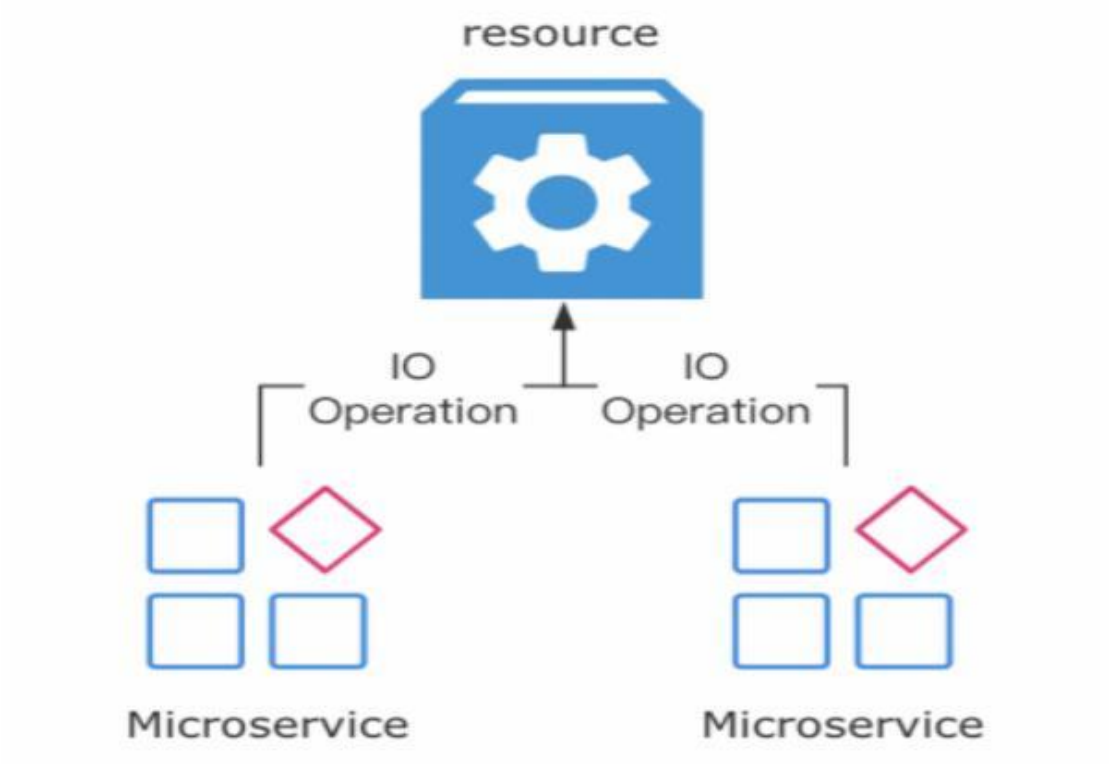
如果要讲优化接口这一块的话，你可以看下我这篇文章哈，结合来一起讲。其实就是缓存、分批、并发调用、异步等那几个关键知识点。

你可以挑个简单的来讲，比如复杂业务的 if 逻辑条件，可以调整顺序，让程序更高效，这样会让面试官眼前一亮哦。

54.2 你项目提到分布式锁，你们是怎么使用分布式锁的

分布式锁，是一种跨进程的跨机器节点的互斥锁，它可以用来保证多机器节点对于共享资源访问的排

他性



我觉得分布式锁和线程锁本质上是一样的，线程锁的生命周期是单进程多线程，分布式锁的声明周期

是多进程多机器节点。

在本质上，他们都需要满足锁的几个重要特性：

排他性，也就是说，同一时刻只能有一个节点去访问共享资源。

可重入性，允许一个已经获得锁的进程，在没有释放锁之前再次重新获得锁。

锁的获取、释放的方法

锁的失效机制，避免死锁的问题

所以，我认为，只要能够满足这些特性的技术组件都能够实现分布式锁。

关系型数据库，可以使用唯一约束来实现锁的排他性，

如果要针对某个方法加锁，就可以创建一个表包含方法名称字段，

并且把方法名设置成唯一的约束。

那抢占锁的逻辑就是：往表里面插入一条数据，如果已经有其他的线程获得了某个方法的锁，那这个

时候插入数据会失败，从而保证了互斥性。

这种方式虽然简单啊，但是要实现比较完整的分布式锁，还需要考虑重入性、锁失效机制、没抢占到

锁的线程要实现阻塞等，就会比较麻烦。

Redis，它里面提供了 SETNX 命令可以实现锁的排他性，当 key 不存在就返回 1，存在就返回 0。然后

还可以用 expire 命令设置锁的失效时间，从而避免死锁问题。

当然有可能存在锁过期了，但是业务逻辑还没执行完的情况。所以这种情况，可以写一个定时任务对

指定的 key 进行续期。

Redisson 这个开源组件，就提供了分布式锁的封装实现，并且也内置了一个 Watch Dog 机制来对

key 做续期。

我认为 Redis 里面这种分布式锁设计已经能够解决 99%的问题了，当然如果在 Redis 搭建了高可用集

群的情况下出现主从切换导致 key 失效，这个问题也有可能造成

多个线程抢占到同一个锁资源的情况，所以 Redis 官方也提供了一个 RedLock 的解决办法，但是实现

会相对复杂一些。

在我看来，分布式锁应该是一个 CP 模型，而 Redis 是一个 AP 模型，所以在集群架构下由于数据的一

致性问题导致极端情况下出现多个线程抢占到锁的情况很难避免。

那么基于 CP 模型又能实现分布式锁特性的组件，我认为可以选择 Zookeeper 或者 etcd，

在数据一致性方面，zookeeper 用到了 zab 协议来保证数据的一致性，etcd 用到了 raft 算法来保证数

据一致性。

在锁的互斥方面，zookeeper 可以基于有序节点再结合 Watch 机制实现互斥和唤醒，etcd 可以基于

Prefix 机制和 Watch 实现互斥和唤醒。

以上就是我对于分布式锁的理解！

54.3 常见分布式事务解决方案

分布式事务：就是指事务的参与者、支持事务的服务器、资源服务器以及事务管理器分别位于不同的分布式系统的不同节点之上。

简单来说，分布式事务指的就是分布式系统中的事务，它的存在就是为了保证不同数据库节点的数据一

致性

聊到分布式事务，大家记得这两个理论哈：CAP 理论 和 BASE 理论 分布式事务的几种解决方案：

- 2PC(二阶段提交)方案、3PC
- TCC（Try、Confirm、Cancel）
- 本地消息表
- 最大努力通知
- seata

2PC(二阶段提交)方案

2PC，即两阶段提交，它将分布式事务的提交拆分为 2 个阶段： prepare 和 commit/rollback ，即准备阶段和提交执行阶段。在 prepare 准备阶段需要等待所有参与子事务的反馈，因此可能造成数据库资源锁定时间过长，不适合并发

高以及子事务生命周长较长✂业务场景。并且协调者宕机，所有的参与者都收不到提交或回滚指令

3PC

两阶段提交分别是： CanCommit，PreCommit 和 doCommit ，这里不再详述。

3PC 利用超时机制解决了 2PC 的同步阻塞问题，避免资源被永久锁定，进一步加强了整个事务过程✂可靠性。但是 3PC 同样无法应对类似的宕机问题，

只不过出现多数据源中数据不一致问题的概率更小。

TCC

TCC 采用了补偿机制，其核心思想是：针对每个操作，都要注册一个与其对应的确认和补偿（撤销）操作。

它分为三个阶段： Try-Confirm-Cancel

- try 阶段：尝试去执行，完成所有业务的一致性检查，预留必须的业务资源。
- Confirm 阶段：该阶段对业务进行确认提交，不做任何检查，因为 try 阶段已经检查过了，默认 Confirm 阶段是不会出错的。
- Cancel 阶段：若业务执行失败，则进入该阶段，它会释放 try 阶段占用的所有业务资源，并回滚 Confirm 阶段执行的所有操作。

TCC 方案让应用可以自定义数据库操作的粒度，降低了锁冲突，可以提升性能。

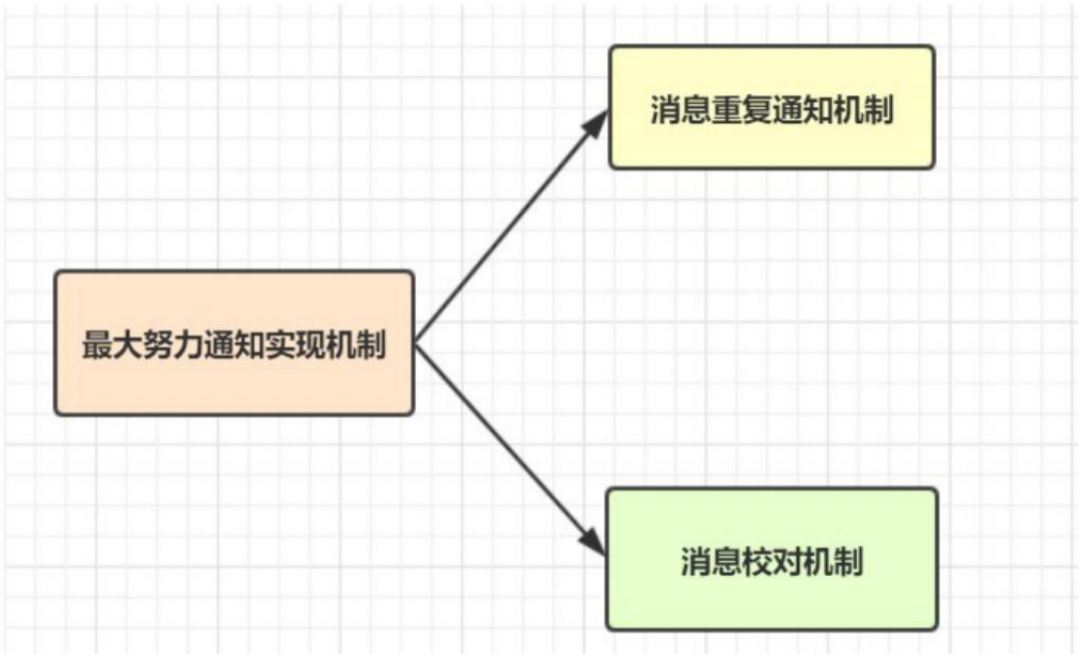
是应用侵入性强，try、confirm、cancel 三个阶段都需要业务逻辑实现

本地消息表

ebay 最初提出本地消息表这个方案，来解决分布式事务问题。业界目前使用这种方案是比较多的，它的核心思想就是将分布式事务拆分成本地事务进行处理。可以看一下基本的实现流程图

最大努力通知

最大努力通知方案的目标，就是发起通知方通过一定的机制，最大努力将业务处理结果通知到接收方



seata

Saga 模式

Seata 提供的长事务解决方案。核心思想是将长事务拆分为多个本地短事务，由 Saga 事务协调器协调，如果正常结束那就正常完成，

如果某个步骤失败，则根据相反顺序一次调用补偿操作。

Saga 的并发度高，但是一致性弱，对于转账，可能发生用户已扣款，最后转账又失败的情况。

54.4 你们的接口幂等 性如何保证 的?

如果你调用下游接口超时了，是不是考虑重试？如果重试，下有接口就需要支持幂等啦。

实现幂等一般有这 8 种方案：

- select+insert+主键 / 唯一索引冲突
- 直接 insert + 主键/唯一索引冲突
- 状态机幂等
- 抽取防重表
- token 令牌
- 悲观锁(如 select for update，很少用)
- 乐观锁

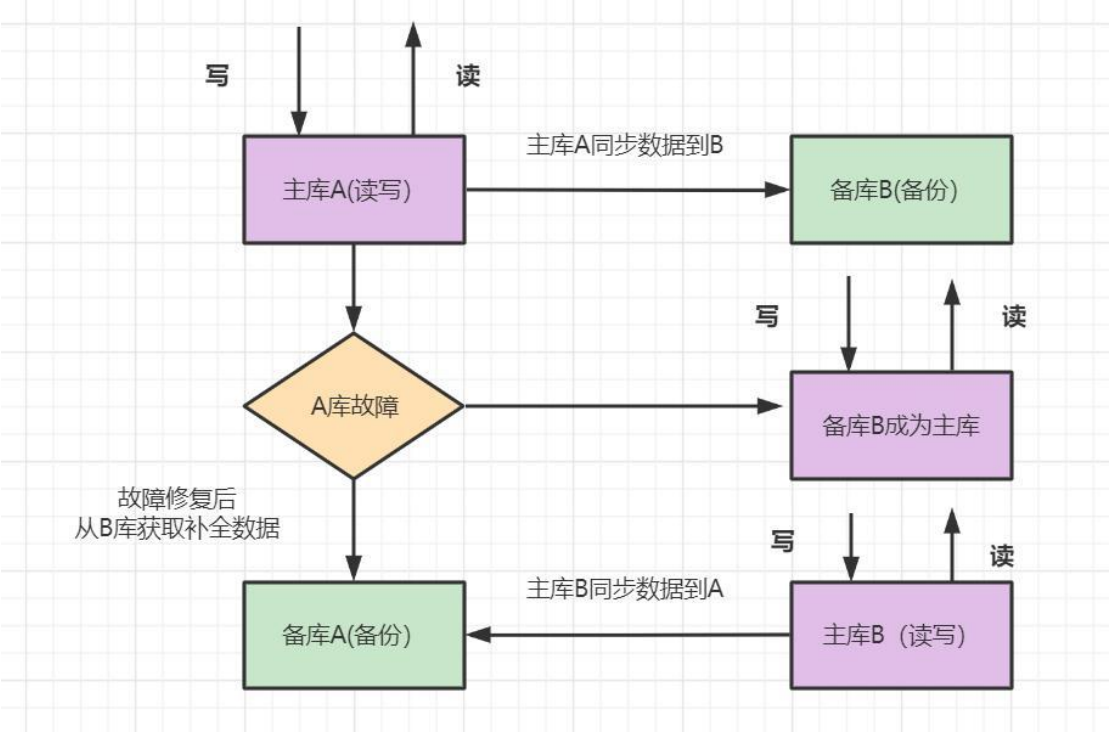
- 分布式锁

54.5 你们的 MySQL 架构 是怎样 的?

大家可以结合自己公司的 MySQL 架构聊聊。如果不是很清楚的话，可以结合数据库高可用方案

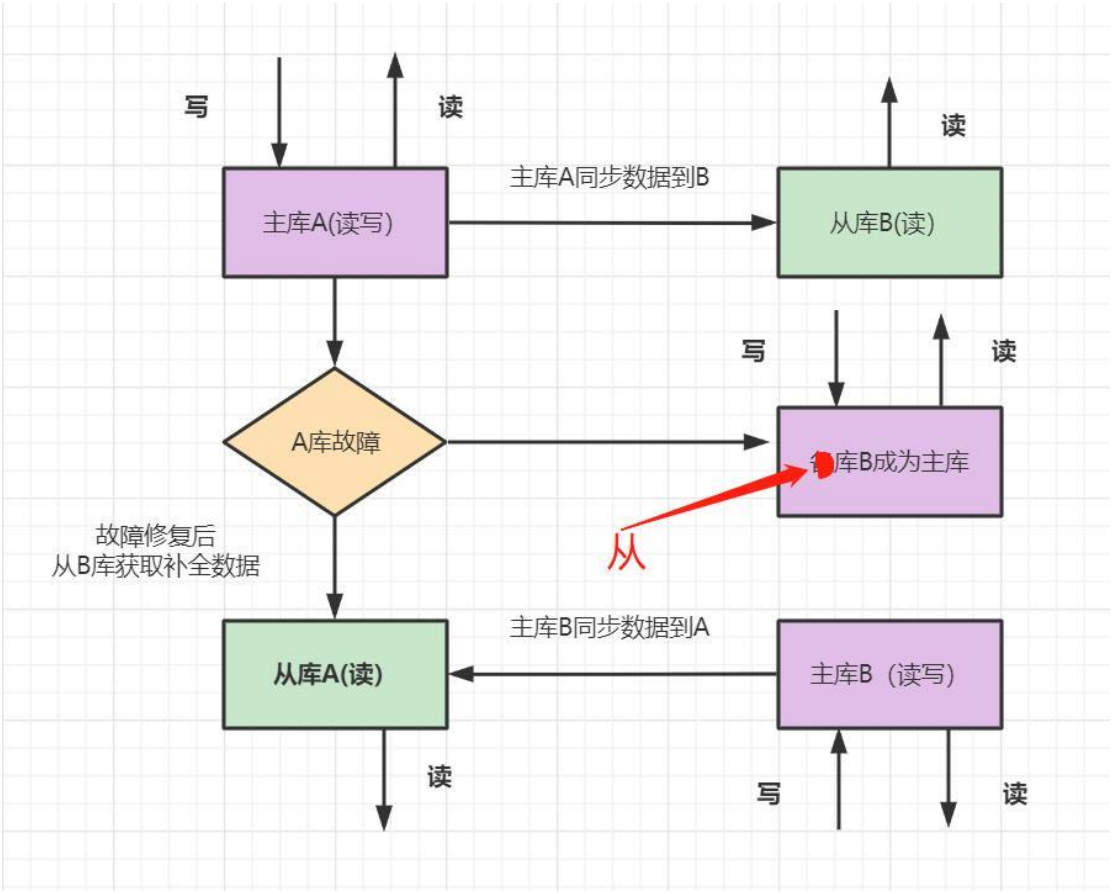
- 双机主备
- 一主一从
- 一主多从
- MariaDB 同步多主机
- 数据库中间件

双机主备



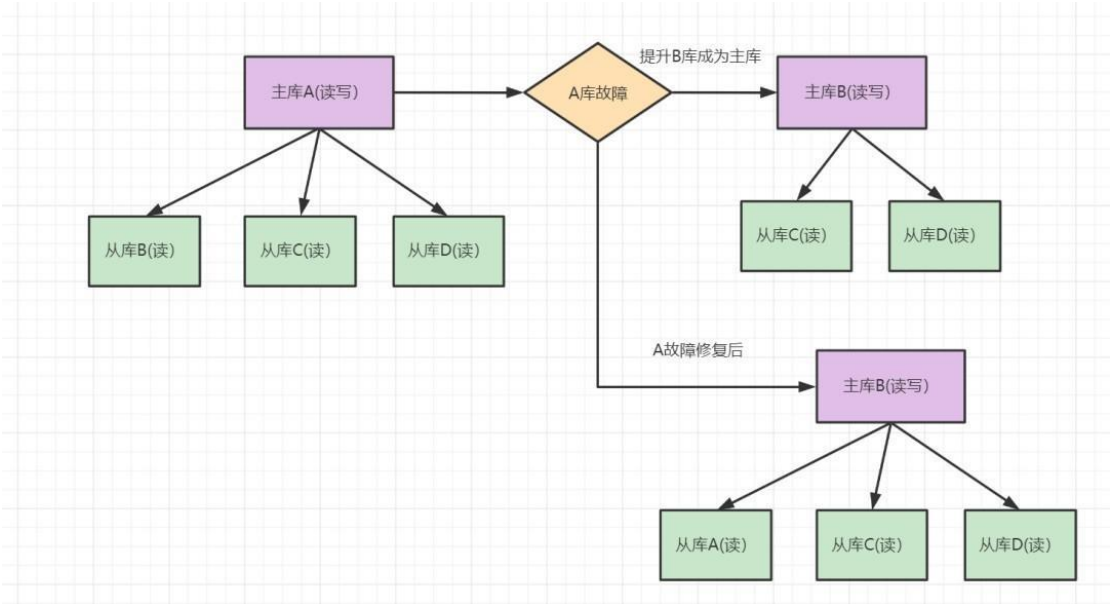
- 优点：一个机器故障了可以自动切换，操作比较简单。
- 缺点：只有一个库在工作，读写压力大，未能实现读写分离，并发也有一定限制

一主一从



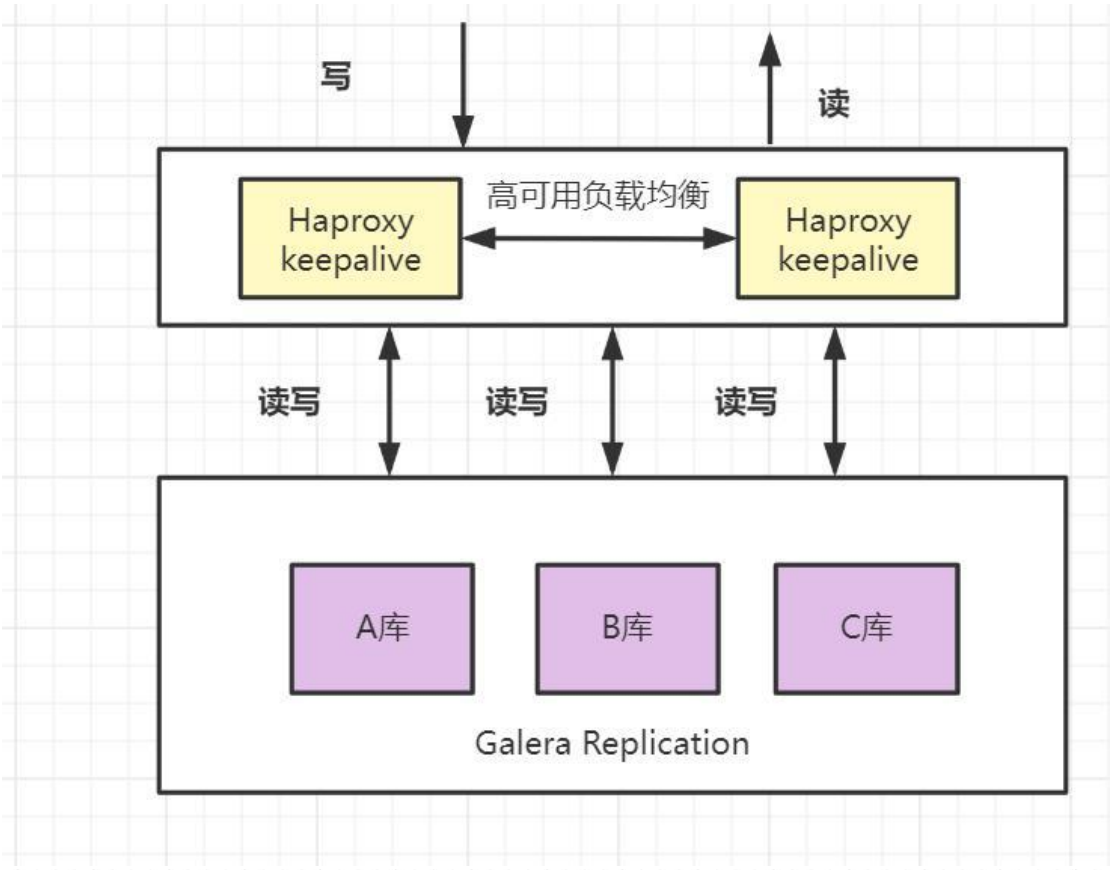
- 优点：从库支持读，分担了主库✂压力，提升了并发度。一个机器故障了可以自动切换，操作比较简单。
- 缺点：一台从库，并发支持还✂不够，并且一共两台机器，还是存在同时故障的机率，不够高可用

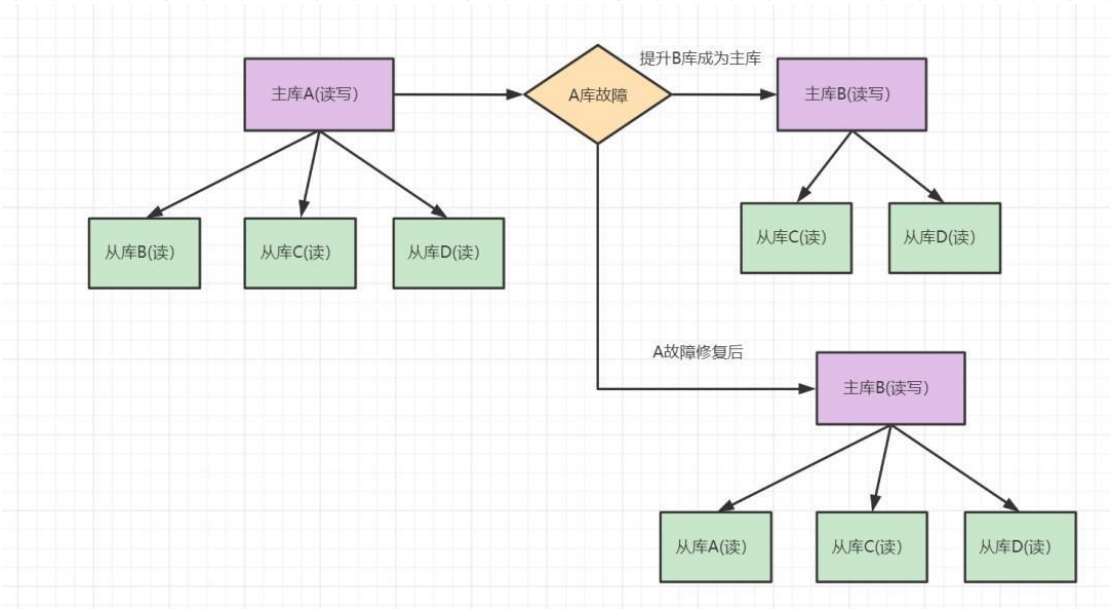
一主多从



- 优点：多个从库支持读，分担了主库的压力，明显提升了读的并发度。
- 缺点：只有一台主机写，因此写的并发度不高

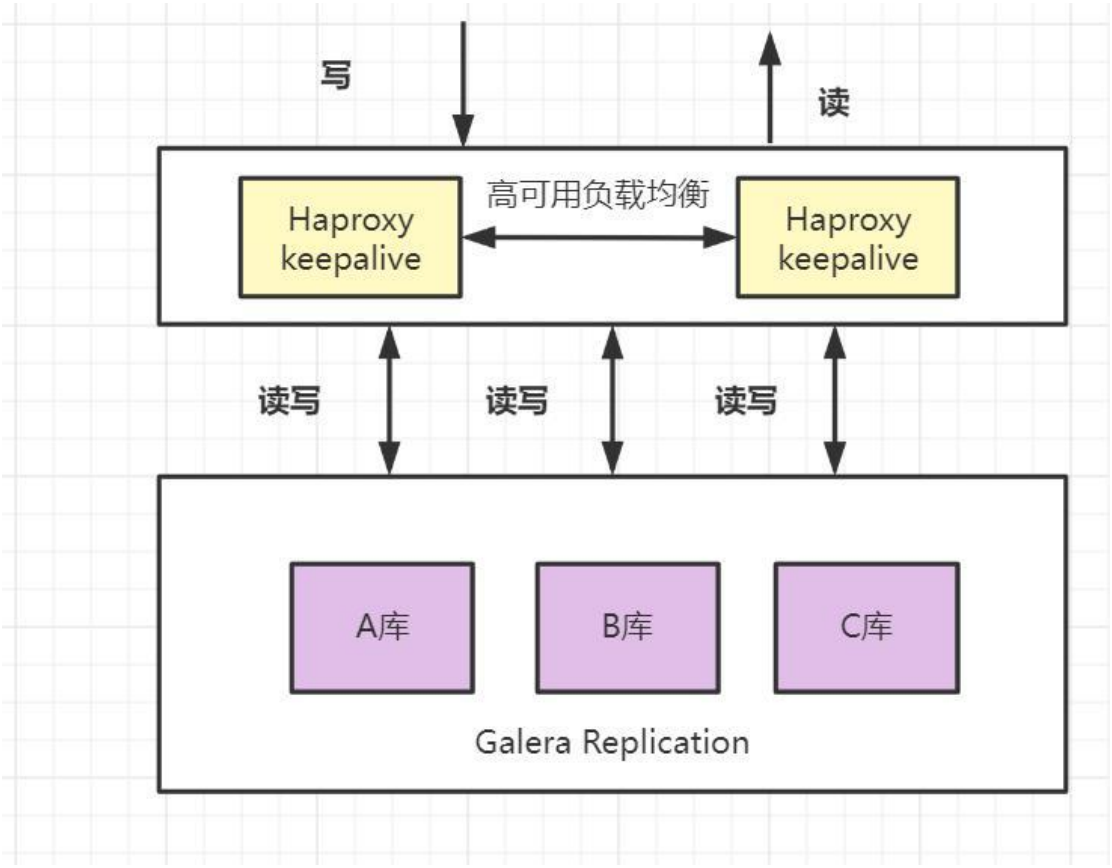
MariaDB 同步多主机集群





- 优点：多个从库支持读，分担了主库的压力，明显提升了读的并发度。
- 缺点：只有一台主机写，因此写的并发度不高

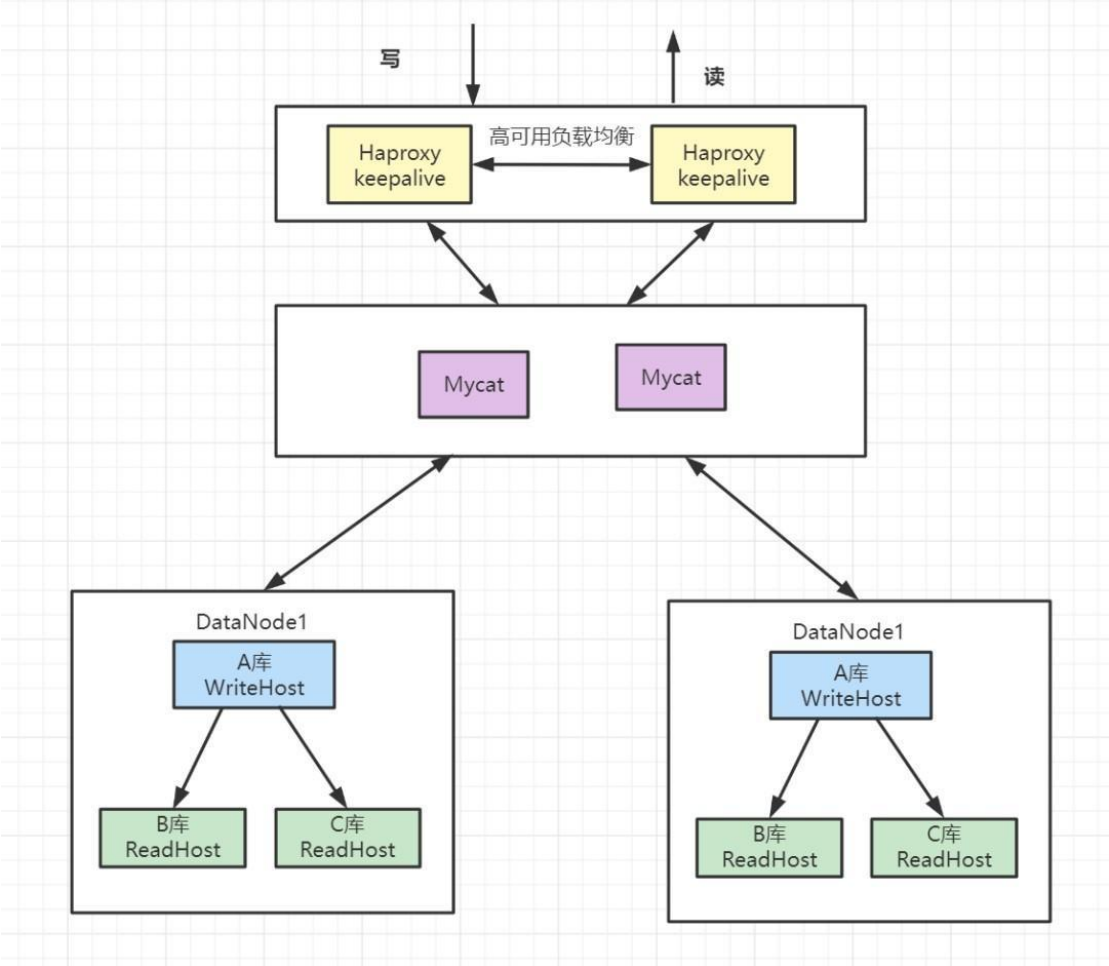
MariaDB 同步多主机集群



有代理层实现负载均衡，多个数据库可以同时进行读写操作；各个数据库之间可以通过Galera Replication 方法进行数据同步，每个库理论上数据的完全一致性。

- 优点：读写的并发度都明显提升，可以任意节点读写，可以自动剔除故障节点，具有较高的可靠性。
- 缺点：数据量不支持特别大。要避免大事务卡死，如果集群节点一个变慢，其他节点也会跟着变慢

数据库中间件



- mycat 分片存储，每个分片配置一主多从的集群。
- 优点：解决高并发高数据量的高可用方案
- 缺点：维护成本比较大

54.6 常见 的索引结构有？哈希表结构属于哪种场景？

哈希表、有序数组和搜索树。

- 哈希表这种结构适用于只有等值查询的场景
- 有序数组适合范围查询，用二分法快速得到，时间复杂度为 O(log(N))。查询还好，如果是插入，就得挪动后面所有的记录，成本比较高。因此它一般只适用静态存储引擎，比如保存 2018 年某个城市的所有人口信息。
- B+树适合范围查询，我们一般数据库的索引结构都是 B+树。

54.7 给你 ab,ac,abc 字段，你是如何加索引 的？

这主要考察联合索引的最左前缀原则知识点。

- 这个最左前缀可以是联合索引的最左 N 个字段。比如组合索引 （a,b,c） 可以相当于建立了 （a），（a,b）,(a,b,c) 三个索引，大大提高了索引复用能力。
- 最左前缀也可以是字符串索引的最左 M 个字符。因此给你 ab,ac,abc 字段，你可以直接加 abc 联合索引和 ac 联合索引即可

54.8 数据库隔离级别 是否了解？你们的数据库默认隔离级别？ 为什么选它？

四大数据库隔离级别，分别是 读未提交，读已提交，可重复读，串行化（Serializable）。

- 读未提交：事务即使未提交，却可以被别的事务读取到的，这级别的事务隔离有脏读、重复读、幻读的问题。
- 读已提交：当前事务只能读取到其他事务提交的数据，这种事务的隔离级别解决了脏读问题，但还是会存在不可重复读、幻读问题；
- 可重复读：限制了读取数据的时候，不可以进行修改，所以解决了不可重复读的问题，但是读取范围数据的时候，是可以插入数据，所以还会存在幻读问题
- 串行化：事务最高的隔离级别，在该级别下，所有事务都会进行串行化顺序执行。可以避免脏读、不可重复读与幻读所有并发问题。但是这种事务隔离级别下，事务执行很耗性能

MySQL 选择 Repeatable Read（可重复读） 作为默认隔离级别，我们的数据库隔离级别选择的是读已提交

54.9 为什么 MySQL 的默认隔离离别是 RR?

binlog 的格式也有三种：statement，row，mixed。默认设置为 statement 格式，binlog 记录的是 SQL 的原文。

又因为 MySQL 在主从复制的过程是通过 binlog 进行数据同步，如果设置为读已提交（RC）隔离级别，当出现事务乱序的时候，就会导致备库在 SQL 回放之后，结果和主库内容不一致。

比如一个表 t，表中有两条记录：

```
CREATE TABLE t (
a int(11) DEFAULT NULL,
b int(11) DEFAULT NULL,
PRIMARY KEY a (a),
KEY b(b)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
insert into t1 values(10,666),(20,233);
```

两个事务并发写操作，如下：

	事务1	事务2
1	设置session 为read commited	
2	set autocommit =0	
3	update t set a=30 where b=666	
4		设置session 为read commited
5		set autocommit =0
6		update t set b=666 where b=233
7		commit
8	commit	

在 读已提交（RC） 隔离级别下，两个事务执行完后，数据库的两条记录就变成了（30,666）、(20,666) 。

这两个事务执行完后，binlog 也就有两条记录，因为事务 binlog 用的是 statement 格式，事务 2 先提交，

因此 update t set b=666 where b=233 优先记录，而 update t set a=30 where b=666 记录在后面。

当 bin log 同步到从库后，执行 update t set b=666 where b=233 和 update t set a=30 where b=666 记录，数据库的记录就变成 （30,666）、(30,666) ，这时候主从数据不一致啦。

因此 MySQL 的默认隔离离别选择了 RR 而不是 RC 。 RR 隔离级别下，更新数据的时候不仅对更新的行加行级锁，还会加间隙锁 （gap lock） 。

事务 2 要执行时，因为事务 1 增加了间隙锁，就会导致事务 2 执行被卡住，只有等事务 1 提交或

者回滚后才能继续执行。

并且，MySQL 还禁止在使用 statement 格式的 binlog 的情况下，使用 READCOMMITTED 作为事务隔离级别。

54.10 我们的数据库隔离级别最后选的是读已提交（RC）

那为什么 MySQL 官方默认隔离级别的 RR，而有些大厂选择了 RC 作为默认的隔离级别呢?

- 提升并发，RC 在加锁的过程中，不需要添加 Gap Lock 和 Next-Key Lock ，只对要修改的记录添加行级锁就行了。因此 RC 支持的并发度比 RR 高得多，
- 减少死锁，正式因为 RR 隔离级别增加了 Gap Lock 和 Next-Key Lock 锁，因此它相对于 RC，更容易产生死锁

54.11 RR 隔离级别实现原理，它是如何解决不可重复读

什么 是不可重复读

先回忆下什么是不可重复读。假设现在有两个事务 A 和 B:

- 事务 A 先查询 Jay 的余额，查到结果是 100
- 这时候事务 B 对 Jay 的账户余额进行扣减，扣去 10 后，提交事务
- 事务 A 再去查询 Jay 的账户余额发现变成了 90

事务 A 被事务 B 干扰到了！在事务 A 范围内，两个相同的查询，读取同一条记录，却返回了不同 的数据，这就 是不可重复读。

undo log 版本链 + Read View 可见性规则

RR 隔离级别实现原理，就是 MVCC 多版本并发控制，而 MVCC 是通过 ReadView+ Undo Log 实现的，Undo Log 保存了历史快照，Read View 可见性规则帮助判断当前版本的数据是否可见。

Undo Log 版本链长这样:

Read view 的几个重要属性

- m_ids :当前系统中那些活跃(未提交)的读写事务 ID，它数据结构为一个 List。
- min_limit_id :表示在生成 Read View 时，当前系统中活跃的读写事务中最小的事务 id，即 m_ids 中的最小值。
- max_limit_id :表示生成 Read View 时，系统中应该分配给下一个事务的 id 值。
- creator_trx_id : 创建当前 Read View 的事务 ID

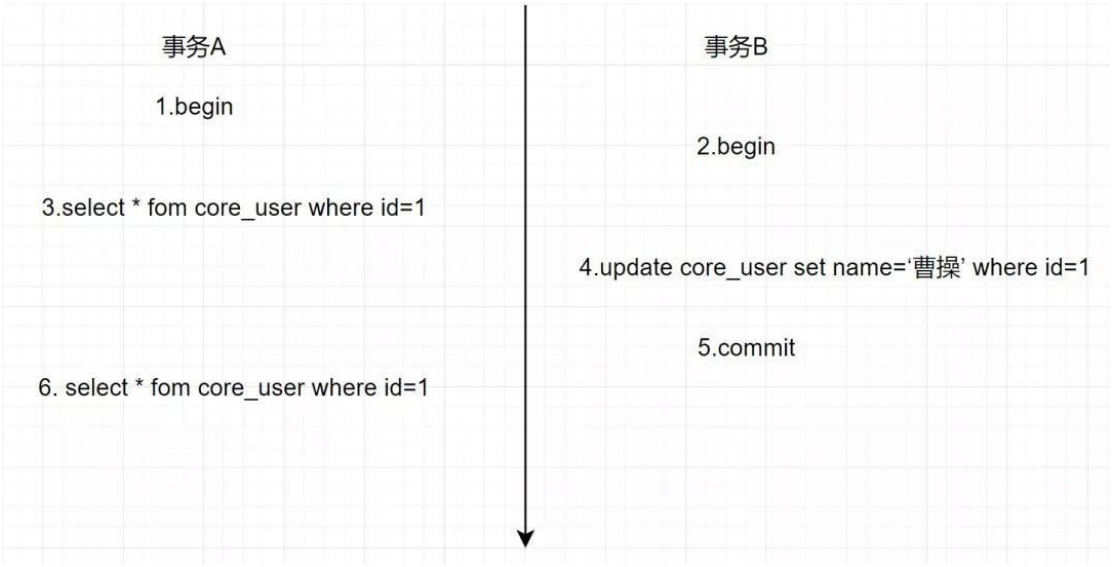
Read view 可见性规则如下：

1. 如果数据事务 ID trx_id < min_limit_id ，表明生成该版本的事务在生成 ReadView 前，已经提交(因为事务 ID 的递增的)，所以该版本可以被当前事务访问。
2. 如果 trx_id>= max_limit_id ，表明生成该版本的事务在生成 Read View 后才生成，所以该版本不可以被当前事务访问。
3. 如果 min_limit_id =<trx_id< max_limit_id ,需要分 3 种情况讨论
    - 如果 m_ids 包含 trx_id ,则代表 Read View 生成时刻，这个事务还未提交，但是如果数据的 trx_id 等于 creator_trx_id 的话，表明数据是自己生成的，因此是可见的。
    - 3.2 如果 m_ids 包含 trx_id ,并且 trx_id 不等于 creator_trx_id ,则Read View 生成时，事务未提交，并且不是自己生产的，所以当前事务也是看不见的;
    - 3.3 如果 m_ids 不包含 trx_id ，则说明你这个事务在 Read View 生成之前就已经提交了，修改的结果，当前事务是能看见的

RR 如何解决不可重复读

查询一条记录，基于 MVCC，是怎样的流程：

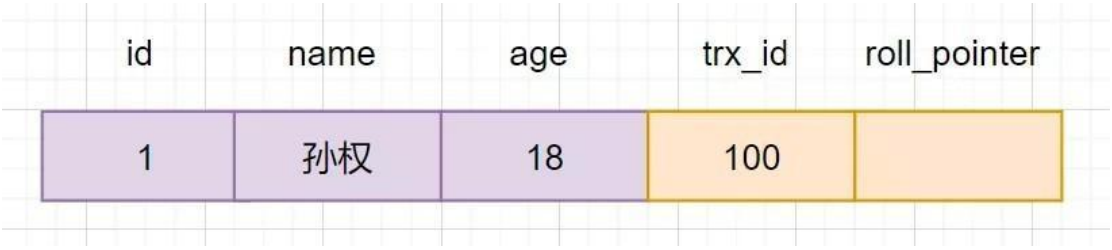
1. 获取事务自己的版本号，即事务 ID
2. 获取 Read View
3. 查询得到的数据，然后 Read View 中的事务版本号进行比较。
4. 如果不符合 Read View 的可见性规则， 即就需要 Undo log 中历史快照;
5. 最后返回符合规则的数据



假设存在事务 A 和 B，SQL 执行流程如下

在可重复读（RR）隔离级别下，一个事务里只会获取一次 Read View，都是副本共用的，从而保证每次查询的数据都是一样的。

假设当前有一张 core_user 表，插入一条初始化数据,如下：

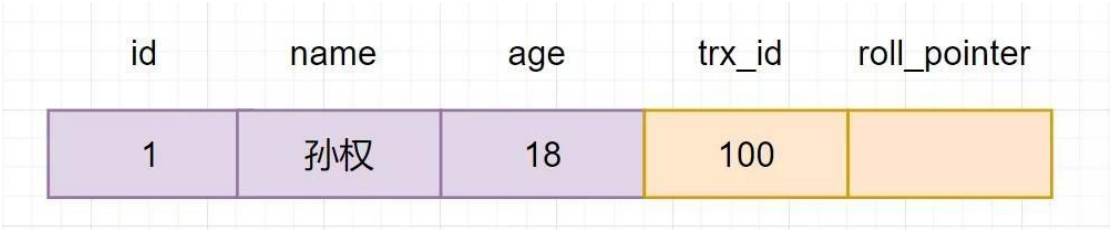


基于 MVCC，我们来看看执行流程

1. A 开启事务，首先得到一个事务 ID 为 100
2. B 开启事务，得到事务 ID 为 101
3. 事务 A 生成一个 Read View，read view 对应的值如下

变量	值
m_ids	100，101
max_limit_id	102
min_limit_id	100

creator_trx_ id	100



然后回到版本链：开始从版本链中挑选可见的记录：

由图可以看出，最新版本的列 name 的内容是孙权，该版本是 trx_id 值为 100。开始执行 read view 可见性规则 校验：

```
min_limit_id(100)=<trx_id（100）<102;creator_trx_id =trx_id =100;
```

由此可得，trx_id=100 的这个记录，当前事务是可见的。所以查到的 name 为孙权的记录。

事务 B 进行修改操作，把名字改为曹操。把原数据拷贝到 undo log,然后对数据进行修改，标记事务 ID 和上一个数据版本在 undo log 的地址

事务 B 提交事务

事务 A 再次执行查询操作，因为是 RR (可重复读) 隔离级别，因此会 复用老的 Read View 副本 ，Read View 对应的值如下

变量	值
m_ids	100，101

max_limit_id	102
min_limit_id	100
creator_trx_ id	100

然后再次回到版本链：从版本链中挑选可见✂记录：

从图可得，最新版本的列 name 的内容是曹操，该版本的 trx_id 值为 101。

开始执行 read view 可见性规则校验

```
1 min_limit_id(100)=<trx_id（101）<max_limit_id（102);
2 因为 m_ids{100,101}包含 trx_id（101），
3 并且 creator_trx_id (100) 不等于 trx_id（101）
```

所以，trx_id=101 这个记录，对于当前事务是不可见的。这时候呢，版本链roll_pointer跳到下一个版本， trx_id=100 这个记录，再次校验是否可见

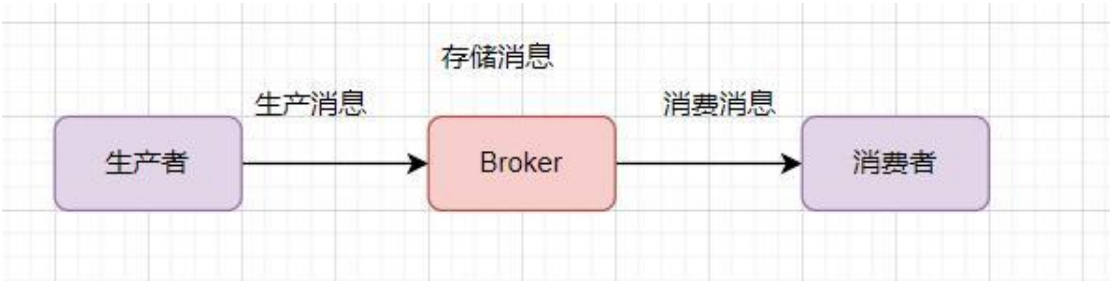
```
1 min_limit_id(100)=<trx_id（100）< max_limit_id（102);
2 因为 m_ids{100,101}包含 trx_id（100），
3 并且 creator_trx_id (100) 等于 trx_id（100）
```

所以，trx_id=100 这个记录，对于当前事务是可见的，所以两次查询结果，都是 name=孙权的那个记录。

即在可重复读（RR）隔离级别下，复用老的 Read View 副本，解决了不可重复读的问题。

54.12 你们项目使用了 RocketMQ 对吧？那你知道如何保证消息不丢失吗？

一个消息从生产者产生，到被消费者消费，主要经过这 3 个过程：



1. 生产者产生消息
2. 消息发送到存储端，保存下来
3. 消息推送到消费者，消费者消费完，ack 应答

因此如何保证 MQ 不丢失消息，可以从这三个阶段阐述：

- 生产者保证不丢消息
- 存储端不丢消息
- 消费者不丢消息

生产者保证不丢消息

生产端如何保证不丢消息呢？确保生产的消息能顺利到达存储端。

如果是 RocketMQ 消息中间件的话， Producer 生产者提供了三种发送消息的方式，分别是：

- 同步发送
- 异步发送
- 单向发送

生产者要想发消息时保证消息不丢失，可以：

- 采用同步方式发送，send 消 息方法返回成功状态，即消息正常到达了存储端 Broker
- 如果 send 消息异常或者返回非成功状态，可以发起重试。
- 可以使用事务消息， RocketMQ 的事务消息机制就是为了保证零丢失来设计的

存储端不丢消息

如何保证存储端的消息不丢失呢？确保消息持久化到磁盘，那就是刷盘机制嘛。

刷盘机制分同步刷盘和异步刷盘：

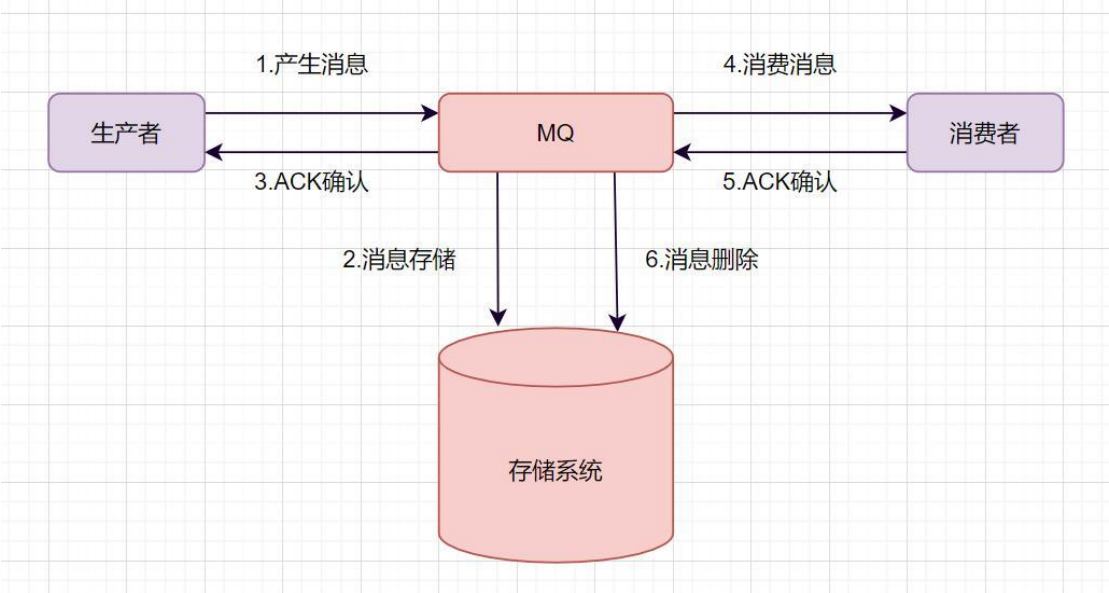
- 同步刷盘：生产者消息发过来时，只有持久化到磁盘， RocketMQ 的存储端 Broker 才返回一个成功的 ACK 响应。它保证消息不丢失，但是影响了性能。
- 异步刷盘：只要消息写入 PageCache 缓存，就返回一个成功的 ACK 响应。这样提高了 MQ 的性能，但是如果这时候机器断电了，就会丢失消息出了同步刷盘机制，还有一个维度需要考虑。 Broker 一般是集群部署的，有主节点和从节点。消息到 Broker 存储端，只有主节点和从节点都写入成功，才反馈成功的 ack 给生产者。这就是 同步复制 ，它保证了消息不丢失，但是降低了 系统的吞吐量。与之对应的是异步复制，只要消息写入主节点成功，就返回成功的 ack ，它速度快，但是会有性能问题

消费阶段不丢

消费者执行完业务逻辑，再反馈会 Broker 说消费成功，这样才可以保证消费阶段不丢消息。

54.13 事务消息是否了解？场景题：比如下单清空购物车，你是如何设计

事务消息主要用来解决消息生产者和消息消费者的数据一致性问题。我们先来回忆一下：一条普通的消息队列消息，从产生到被消费，经历的流程：



1. 生产者产生消息，发送带 MQ 服务器

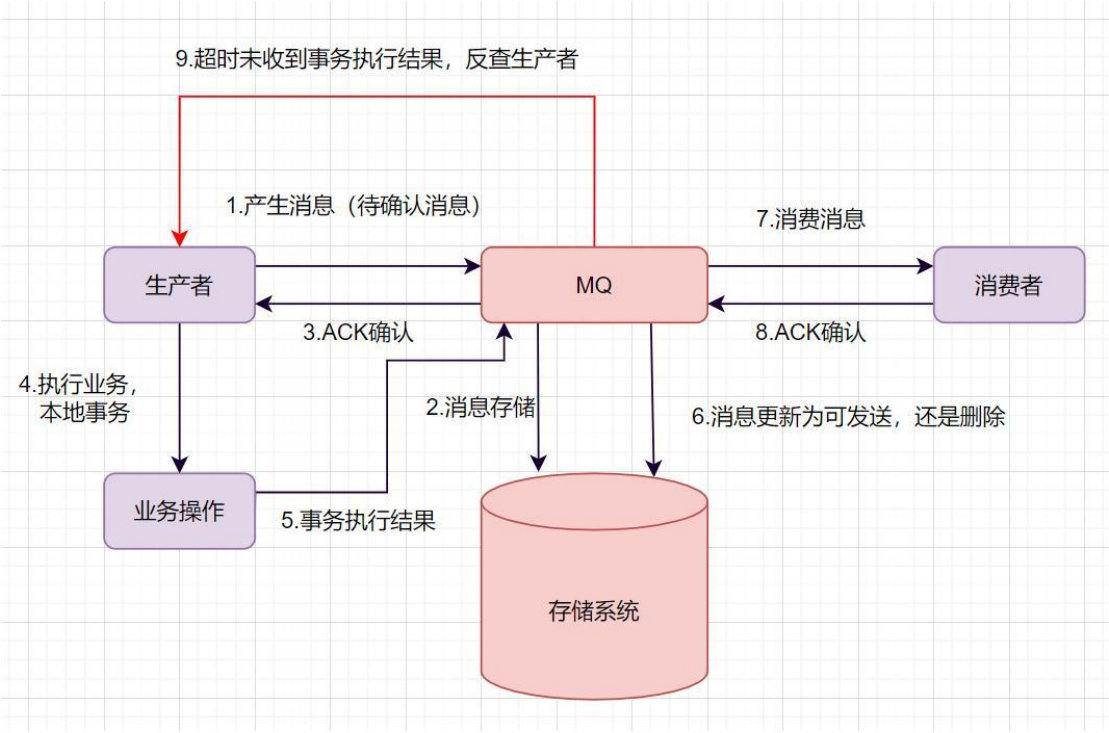
2. MQ 收到消息后，将消息持久化到存储系统。

3. MQ 服务器返回 ACk 到生产者。

4. MQ 服务器的消息 push 给消费者

5. 消费者消费完消息，响应 ACK

6. MQ 服务器收到 ACK，认为消息消费成功，即在存储中删除消息



消息队列的事务消息流程 是怎样 的呢?

1. 生产者产生消息，发送一条半事务消息到 MQ 服务器

2. MQ 收到消息后，将消息持久化到存储系统，这条消息的状态是待发送状态。

3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件

4. 生产者执行本地事务

5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。

6. 如果是正常的 commit，MQ 服务器更新消息状态为可发送；如果是 rollback，即删除消息。

7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者。消费者消费完就回 ACK。

8. 如果 MQ 服务器长时间没有收到生产者✂ commit 或者 rollback，它会反查生产者，然后根据查询到的结果执行最终状态

我们举个下订单清空购物车的例子吧。订单系统创建完订单后，然后发消息给下游系统购物车系统，清空购物车。

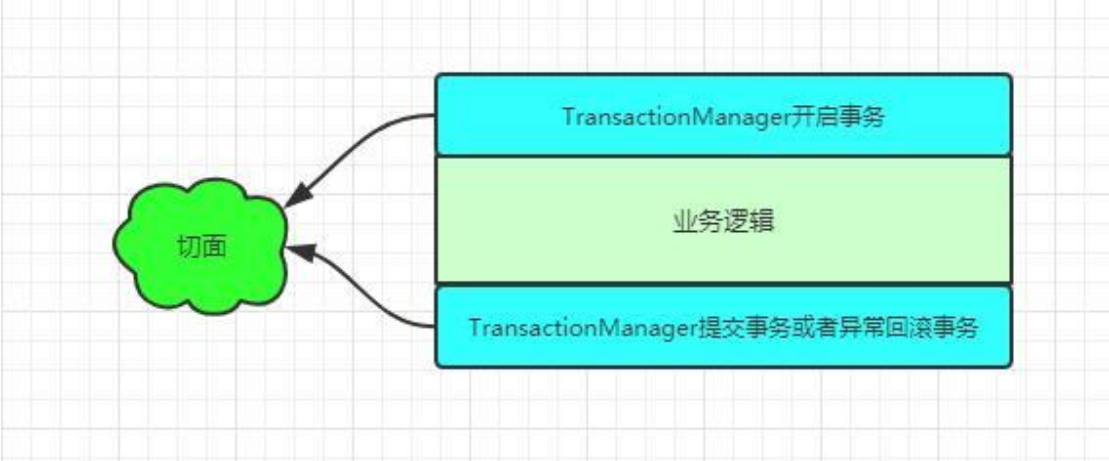
1. 生产者（订单系统）产生消息，发送一条半事务消息到 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统，这条消息的状态是待发送状态。
3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件
4. 生产者执行本地事务（订单创✂成功，提交事务消息）
5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。
6. 如果是 commit 正常提交，MQ 服务器更新消息状态为可发送；如果是 rollback，即删除消息。
7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者（购物车系统）。消费者消费完（即拿到订单消息，清空购物车成功）就应答 ACK。
8. 如果 MQ 服务器长时间没有收到生产者的 commit 或者 rollback，它会反查生产者，然后根据查询到的结果（回滚操作或者重新发送消息）执行最终状态

有些伙伴可能有疑惑，如果消费者消费失败怎么办呢？那数据是不是不一致啦？所以就需要消费者消费成功，执行业务逻辑成功，再反馈 ack 嘛。如果消费者消费失败，那就自动重试嘛，接口支持幂等即可

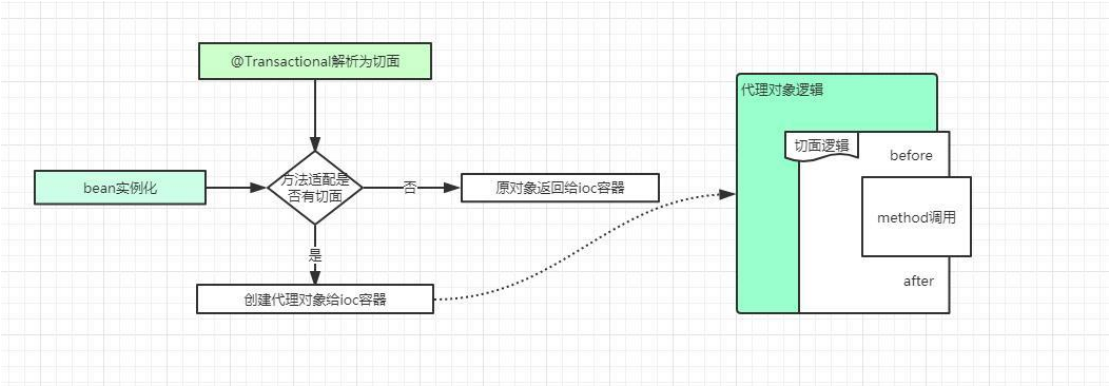
54.14 Spring 声明式事务原理？哪些场景事务会失效？

声明式事务原理

spring 声明式事务，即 @Transactional ,它可以帮助我们实现事务开启、提交或者回滚的操作，通过 Aop 的方式进行管理



在 spring 的 bean 的初始化过程中，就需要对实例化的 bean 进行代理，并且生成代理对象。生成代理对象的代理逻辑中，进行方法调用时，需要先获取切面逻辑，@Transactional 注解的切面逻辑类似于@Around，在 spring 中实现一种类似代理逻辑



54.15 spring 声明式事务哪些场景会失效

1. 方法的访问权限必须是 public，其他 private 等权限，事务失效
2. 方法被定义成了 final 的，这样会导致事务失效。
3. 在同一个类中的方法直接内部调用，会导致事务失效。
4. 一个方法如果没交给 spring 管理，就不会生成 spring 事务。
5. 多线程调用，两个方法不在同一个线程中，获取到的数据库连接不一样✂。
6. 表的存储引擎不支持事务
7. 如果自己 try...catch 误吞了异常，事务失效。
8. 错误的事务传播机制

54.16 你们 是微服务架构嘛？如果你来设计一个类似淘宝的系统，你怎么划分微服务？

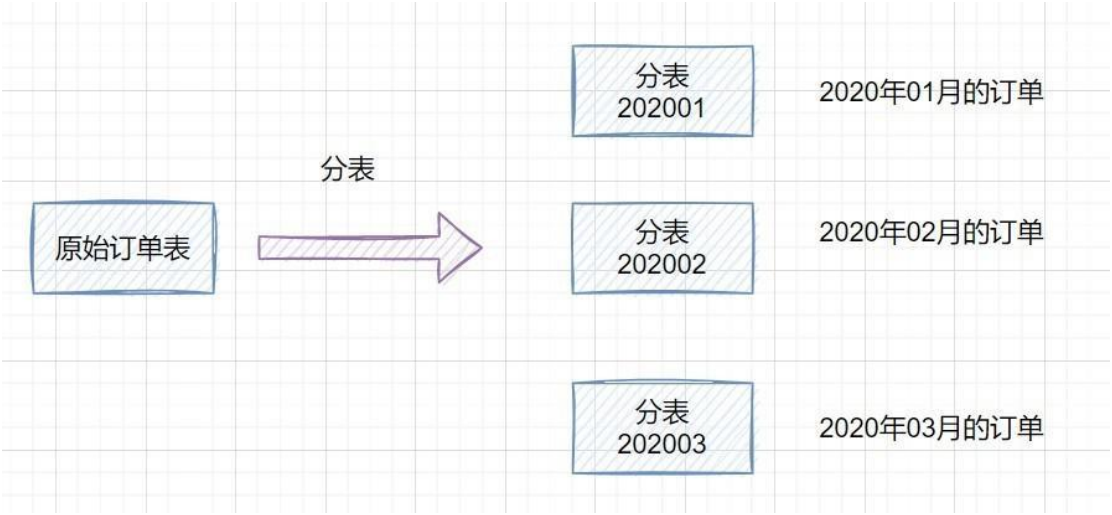
1. 可以按业务领域、功能、重要程度进行划分。
2. 可以按业务领域，按照用户、社区、商品信息、消息等模块等划分。
3. 单一功能职责，按功能拆分，比如订单、支付、物流、权限。
4. 按重要程度划分，区分核心和非核心功能，比如支付、订单就是核心功能

54.17 你们 是怎么分库分表 的？分布式 ID 如何生成？

如果是我们公司的话，使用了水平分库的方式，就是一个用户注册时，就划分了属于哪个数据库，然后具体的表结构是一样的。

业界还有垂直分库，就是按照不同的系统中的不同业务进行拆分，比如拆分成用户库、订单库、积分库、商品库，把它们部署在不同的数据库服务器。

分表的话也有水平分表和垂直分表，垂直分表就是将一些不常用的、数据较大或者长度较长的列拆分到另外一张表，水平分表就是可以按照某种规则 (如 hash 取模、range)，把数据切分到多张表去。一张订单表，按时间 range 拆分如下



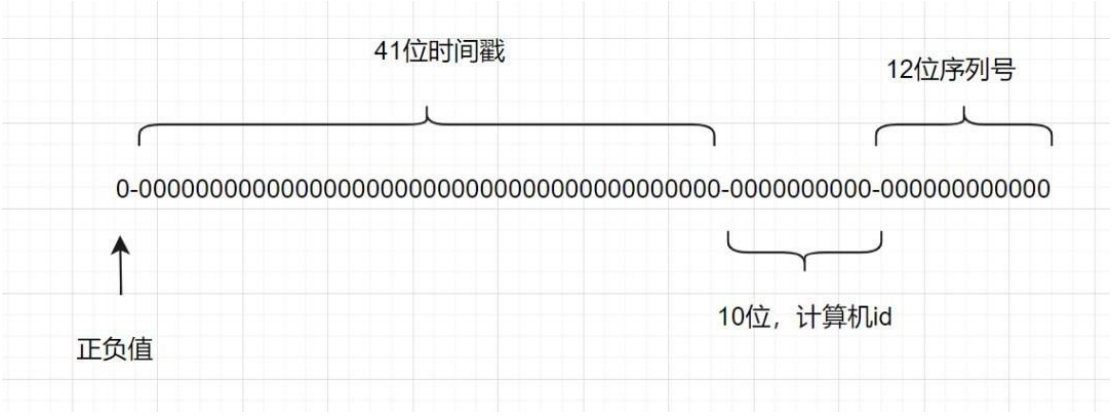
range 划分利于数据迁移，但是存在数据热点问题。hash 取模划分，不会存在明显的热点问题，但是不利于扩容。可以 range+hash 取模结合使用。

分布式 ID 可以使用雪花算法生成

雪花算法是一种生成分布式全局唯一 ID 的算法，生成的 ID 称为 SnowflakeIDs。这种算法由 Twitter 创的。

分布式 ID 可以使用雪花算法生成一个 Snowflake ID 有 64 位。

- 第 1 位：Java 中 long 的最高位是符号位代表正负，正数是 0，负数是 1，一般生 成 ID 都为正数，所以默认为 0。
- 接下来前 41 位的时间戳，表示了自选定的时期以来的毫秒数。
- 接下来的 10 位代表计算机 ID，防止冲突。
- 其余 12 位代表每台机器上生成 ID 的序列号，这允许在同一毫秒内创建多个 Snowflake ID



54.18 Redis 的 key 和 value 可以存储的最大值分别是多少?

- 虽然 Key 的大小上限为 512M,但是一般建议 key 大小不要超过 1KB，这样既可以节约存储空间，又有利于 Redis 进行检索。
- value 的最大值也是 512M。对于 String 类型的 value 值上限为 512M，而集合、链表、哈希等 key 类型，单个元素✂ value 上限也为 512M。

54.19 怎么利用 Redis 实现数据的去重?

Redis 的 set：它可以去除重复元素，也可以快速判断某一个元素是否存在于集合中，如果元素很多（比如上亿的计数），占用内存很大

Redis 的 bit：它可以用来实现比 set 内存高度压缩的计数，它通过一个 bit 设置为 1 或者 0，表示存储某个元素是否存在信息。例如网站唯一访客计数，可以把 user_id 作为 bit 的偏移量 offset，如设置为 1 表示有访问，使用 1 MB 的空间就可以存放 800 多万用户的一天访问计数情况。

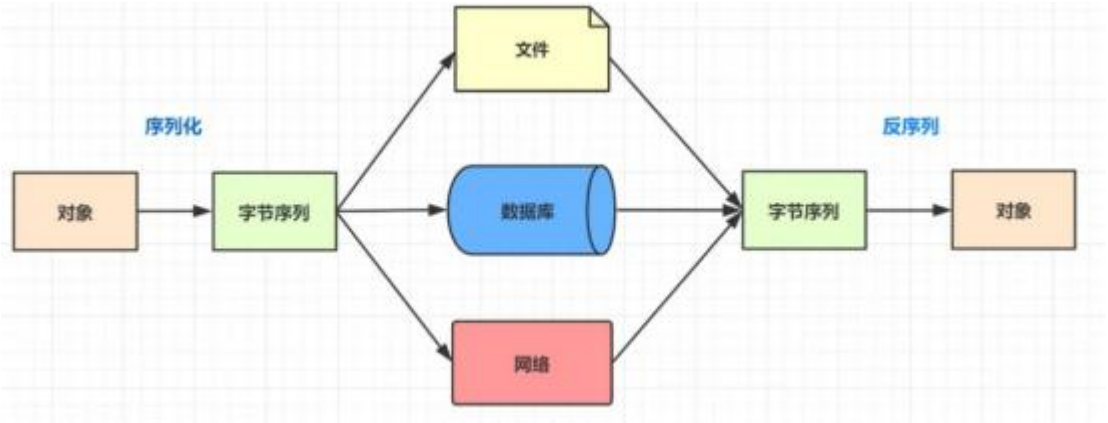
HyperLogLog：实现超大数据量精确的唯一计数都是比较困难额， HyperLogLog 可以仅仅使用 12 k 左右的内存，实现上亿的唯一计数，而且误差控制在百分之一左右。

bloomfilter 布隆过滤器：布隆过滤器是一种占用空间很小的数据结构，它由一个很长的二进制向量和一组 Hash 映射函数组成，它用于检索一个元素是否在一个集合中

54.20 Redis 什么时候需要序列化？Redis 序列化的方式有哪些?

大家先回忆下 Java 序列化，什么时候需要序列化?

- 序列化：将 Java 对象转换成字节流的过程。
- 反序列化：将字节流转换成 Java 对象的过程。



打个比喻：作为大城市漂泊的码农，搬家是常态。当我们搬书桌时，桌子比较大了就通不过比较小的门，因此我们需要把它拆开再搬过去，这个拆桌子的过程
就是序列化。而我们的书桌复原回来（安装）的过程就是反序列化啦。

为什么需要序列化呢?

- 比如想把内存中的对象状态保存到一个文件中或者数据库中的时候（最常用，如保存到 redis)；
- 再比喻想用套接字在网络上传送对象的时候，都需要序列化。

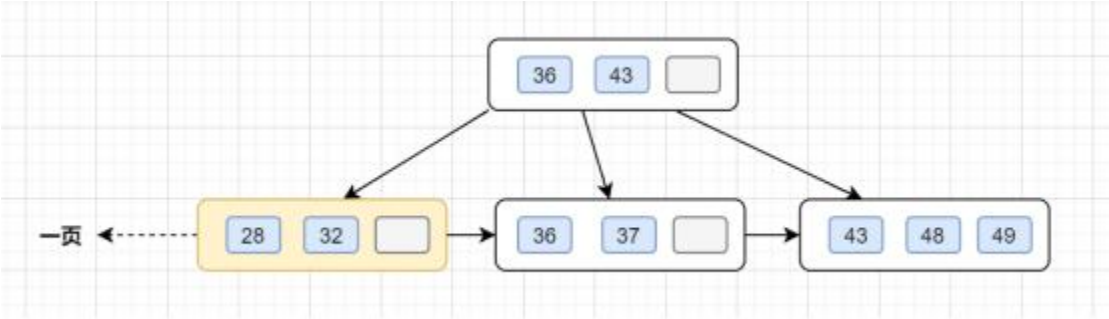
RedisSerializer 接口是 Redis 序列化接口，用于 Redis KEY 和 VALUE d 额 序列化

- JDK 序列化方式（默认）
- String 序列化方式
- JSON 序列化方式
- XML 序列化方式

54.21 MySQL 的 B+树的高度怎么计算？（比如有 100w 的数据，字段为 int 类型）

InnoDB 存储引擎最小储存单元是页，一页大小就是 16k。

B+树叶子存的是数据，内部节点存的是键值+指针。索引组织表通过非叶子节点的二分查找法以及指针确定数据在哪个页中，进而再去数据页中找到需要的 数据；



假设 B+树的高度为 2 的话，即有一个根结点和若干个叶子结点。这棵 B+树的 存放总记录数为=根结点指针数是单个叶子节点记录行数。

- 如果一行记录的数据大小为 1k，那么单个叶子节点可以存的记录数 =16k/1k=16.
- 非叶子节点内存放多少指针呢？我们假设主键 ID 为 **bigint 类型，长度为 8 字节**(**面试官问你 int 类型，一个 int 就是 32 位，4 字节**)，而指针大小在 InnoDB 源码中设置为 6 字节，所以就是 8+6=14 字节，16k/14B =16*1024B/14B =1170

因此，一棵高度为 2 的 B+树，能存放 1170 * 16=18720 条这样的数据记录。同理一棵高度为 3 的 B+树，能存放 1170 *1170 *16 =21902400，也就是说，可以存放两千万左右的记录。B+树高度一般为 1-3 层，已经满足千万级别的数据存储。

54.22 线程池的状态有哪些？获取多线程并发执行结果的方式有哪些?

线程池和线程的状态是不一样的哈，线程池有这几个状态：

RUNNING,SHUTDOWN,STOP,TIDYING,TERMINATED。

> 线程池状态

```
1 private static final int RUNNING = -1 << COUNT_BITS;
2 private static final int SHUTDOWN = 0 << COUNT_BITS;
3 private static final int STOP = 1 << COUNT_BITS;
4 private static final int TIDYING = 2 << COUNT_BITS;
5 private static final int TERMINATED = 3 << COUNT_BITS;
```

线程池各个状态切换状态如下：

54.22.1 RUNNING

- 该状态的线程池会接收新任务，并处理阻塞队列中的任务;
- 调用线程池的 shutdown()方法，可以切换到 SHUTDOWN 状态;
- 调用线程池的 shutdownNow()方法，可以切换到 STOP 状态;

54.22.2 SHUTDOWN

- 该状态的线程池不会接收新任务，但会处理阻塞队列中的任务；
- 队列为空，并且线程池中执行✂任务也为空,进入 TIDYING 状态;

54.22.3 STOP

- 该状态的线程不会接收新任务，也不会处理阻塞队列中的任务，而且会中断正在运行的任务；
- 线程池中执行的任务为空,进入 TIDYING 状态;

54.22.4 TIDYING

- 该状态表明所有的任务已经运行终止，记录的任务数量为 0。
- terminated()执行完毕，进入 TERMINATED 状态

54.22.5 TERMINATED

- 该状态表示线程池彻底终止

54.23 线程池原理？各个参数的作用。

ThreadPoolExecutor 构造函数：

```
1  public ThreadPoolExecutor(int corePoolSize, int maximumPoolSize,long keepAliveTime,
   TimeUnit unit,
2          BlockingQueue<Runnable> workQueue,
3          ThreadFactory threadFactory,
4          RejectedExecutionHandler handler)
```

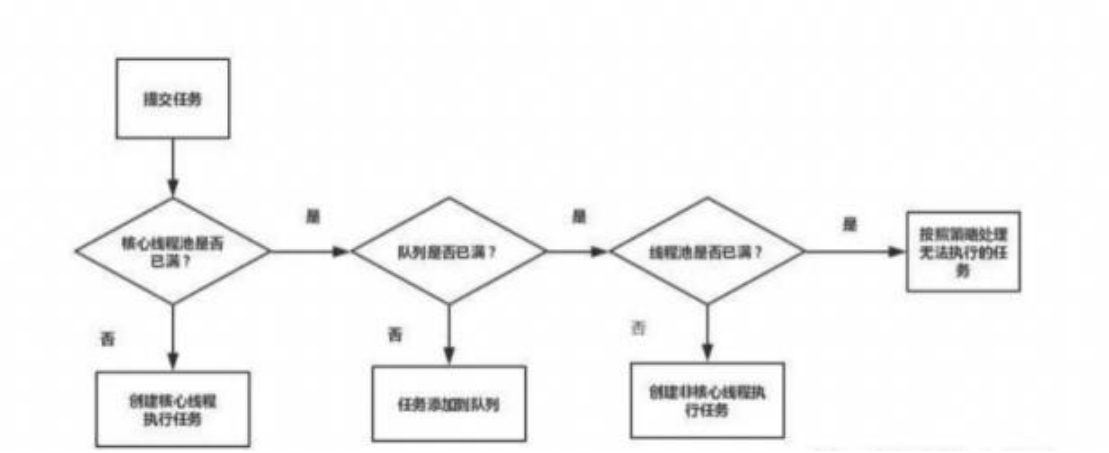
54.23.1 几个核心参数的作用

- corePoolSize： 线程池核心线程数最大值
- maximumPoolSize： 线程池最大线程数大小
- keepAliveTime： 线程池中非核心线程空闲✂存活时间大小
- unit： 线程空闲存活时间单位
- workQueue：存放任务的阻塞队列
- threadFactory：用于设置创建线程的工厂，可以给创建的线程设置有意义的名字，可方便排查问题。
- handler： 线城池✂饱和策略事件，主要有四种类型。

54.23.2 四种饱和拒绝策略

- AbortPolicy(抛出一个异常，默认策略)
- DiscardPolicy(直接丢弃任务)
- DiscardOldestPolicy（丢弃队列里最老的任务，将当前这个任务继续提交给线程池)
- CallerRunsPolicy（交给线程池调用所在的线程进行处理)

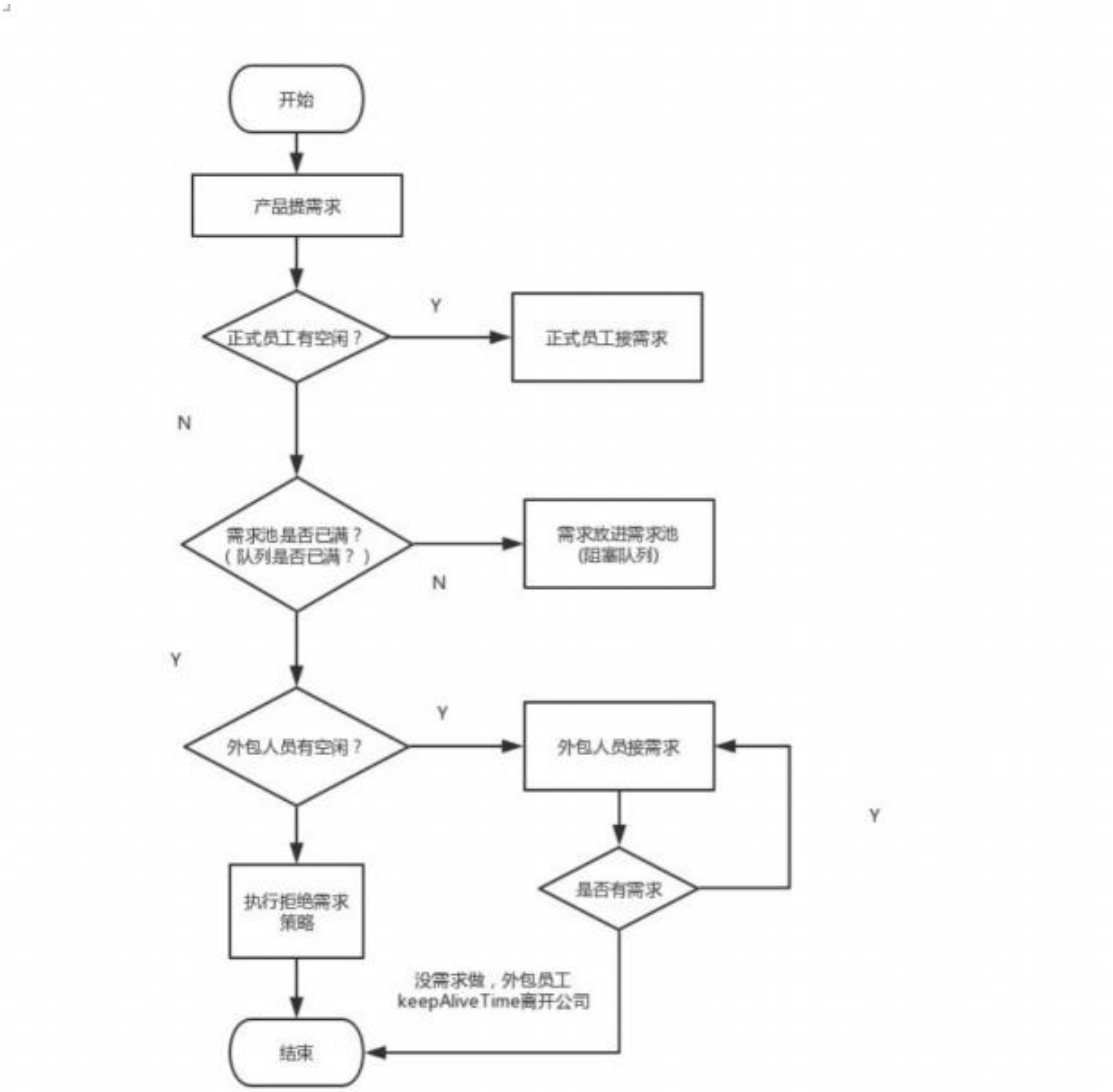
54.23.3 线程池原理



- 提交一个任务，线程池里存活的核心线程数小于线程数 corePoolSize 时，线程池会创建一个核心线程去处理提交的任务。
- 如果线程池核心线程数已满，即线程数已经等于 corePoolSize，一个新提交的任务，会被放进任务队列 workQueue 排队等待执行。
- 当线程池里面存活✂线程数已经等于 corePoolSize 了,并且任务队列 workQueue 也满，判断线程数量否达到 maximumPoolSize，即最大线程数量否已满，如果没到达，创建一个非核心线程执行提交的任务。
- 如果当前的线程数达到了 maximumPoolSize，还有新的任务过来的话，直接采用拒绝策略处理

> 为了形象描述线程池执行，我打个比喻：

- 核心线程比作公司正式员工
- 非核心线程比作外包员工
- 阻塞队列比作需求池
- 提交任务比作提需求



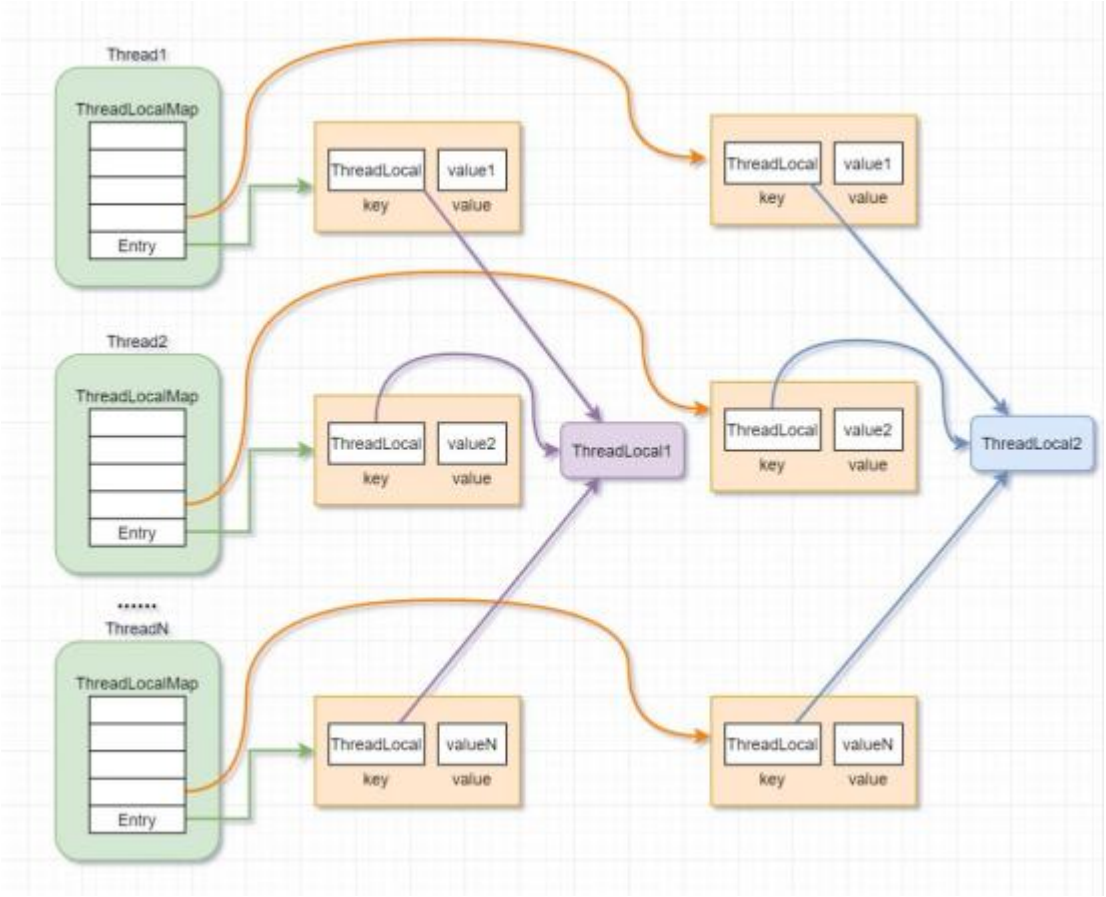
- 当产品提个需求，正式员工（核心线程）先接需求（执行任务）
- 如果正式员工都有需求在做，即核心线程数已满），产品就把需求先放需求池（阻塞队列）。
- 如果需求池(阻塞队列)也满了，但是这时候产品继续提需求,怎么办呢？那就请外包（非核心线程）来做。
- 如果所有员工（最大线程数也满了）都有需求在做了，那就执行拒绝策略。
- 如果外包员工的需求做完了，它经过一段（keepAliveTime）空闲时间，就离开公司了。

54.24 ThreadLocal 的使用场景有哪些？原理？内存泄漏?

ThreadLocal，即线程本地变量。如果你创了一个 ThreadLocal 变量，那么访问这个变量的每个线程都会有这个变量的一个本地拷贝，多个线程操作这个变量的时候，实际操作自己本地内存里面的变量，从而起到线程隔离的作用，避免了线程安全问题。

54.24.1 ThreadLocal 的应用场景

- 数据库连接池
- 会话管理中使用



54.24.2 ThreadLocal 内存结构图

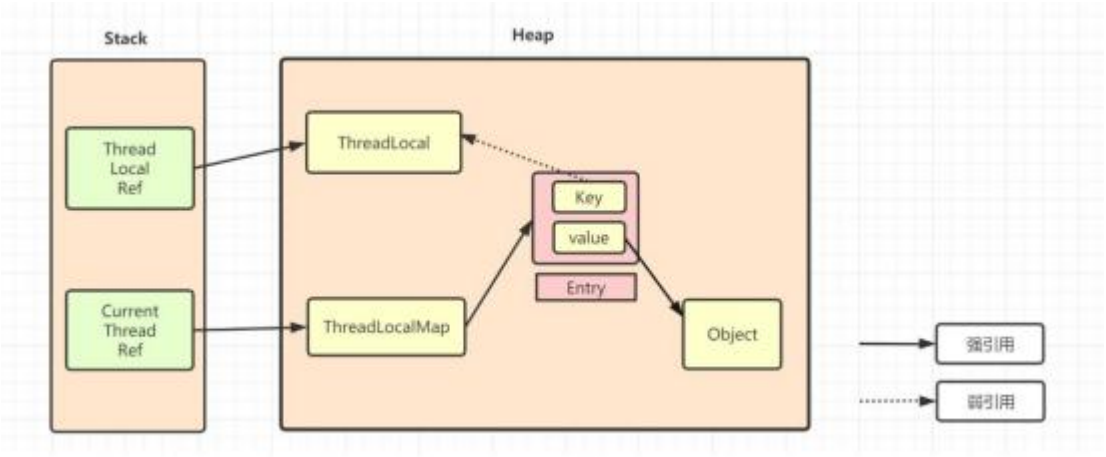
ThreadLocal 原理

- Thread 对象中持有一个 ThreadLocal.ThreadLocalMap ✂成员变量。
- ThreadLocalMap 内部维护了 Entry 数组，每个 Entry 代表一个完整✂对象， key ✂ ThreadLocal 本身，value ✂ ThreadLocal ✂泛型值。

- 每个线程在往 ThreadLocal 里设置值✂时候，都✂往自己✂ ThreadLocalMap 里存，读也✂以某个 ThreadLocal 作为引用，在自己✂ map 里找对应✂ key，从而实现了线程隔离。

54.24.3 ThreadLocal 内存泄露问题

先看看一下 TreadLocal 的引用示意图哈，



ThreadLocalMap 中使用✂ key 为 ThreadLocal ✂弱引用，如下

```
static class ThreadLocalMap {
    static class Entry extends WeakReference<ThreadLocal<?>> {
        /** The value associated with this ThreadLocal. */
        Object value;

        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
}
```

弱引用：只要垃圾回收机制一运行，不管 JVM 的内存空间是否充足，都会回收该对象占用的内存。

弱引用比较容易被回收。因此，如果 ThreadLocal（ThreadLocalMap 的 Key）被垃圾回收器回收了，但是因为 ThreadLocalMap 生命周期和 Thread 是一样的，它这时候如果不被回收，就会出现这种情况：ThreadLocalMap 的

key 没了，value 还在，这就会造成了**内存泄漏问题。**

如何**解决内存泄漏问题**？使用完 ThreadLocal 后，及时调用 remove()方法释放内存空间。

54.25 如何防止内存泄露

首先，ThreadLocal 里面的数据都是以 entry 的数组保存在我们的 thread 里的，其中 entry

的 key 为 threadLocal 对象，value 是你设置的值。

entry 的 key 设置为弱引用

因为 thread 这个对象的生命周期其实是跟 thread 绑定的。那么如果 key 为强引用的话。

ThreadLocaL 的对象一直会被线程的 entry 持有，然后 ThreadLocaL 对象回收不了，但是外

面其实是可以把 threadLocal 对象强引用去掉的。那么这个时候，这个对象以及整个 entry

对象都是无效的，本来应该是需要回收的。但是因为强引用回收不了

所以 thread 的 key 是弱引用，这样我无效的时候我能 GC 回收对象。一定程度上面防止了

内存泄漏问题。

但是不够

因为虽然 key 的对象能被回收了，。但是 entry 还是不能回收，但是这些 entry 的 key 为

null 了，也没用了，需要回收。

所以，。threadLocal 通过各种机制，在使用的时候，。会去回收 key=null 的 entry 对象

比如当我去 set 的时候，会从当前冲突的位置向前或者向后找到第一个 null 的为止，并且

把找到的所有的被 GC 回收掉的 entry 清除！！并且 rehash，供其他对象使用，达到一个时

间与空间的平衡。

还是不够，因为如果没有 hash 冲突，自身是不能走清除的。所以我们一般使用的使用，用

完要 remove.

54.26 Nacos 的选举机制了解嘛？说下 Raft 算法?

Nacos 作为配置中心功能是基于 **Raft 算法**来实现的。

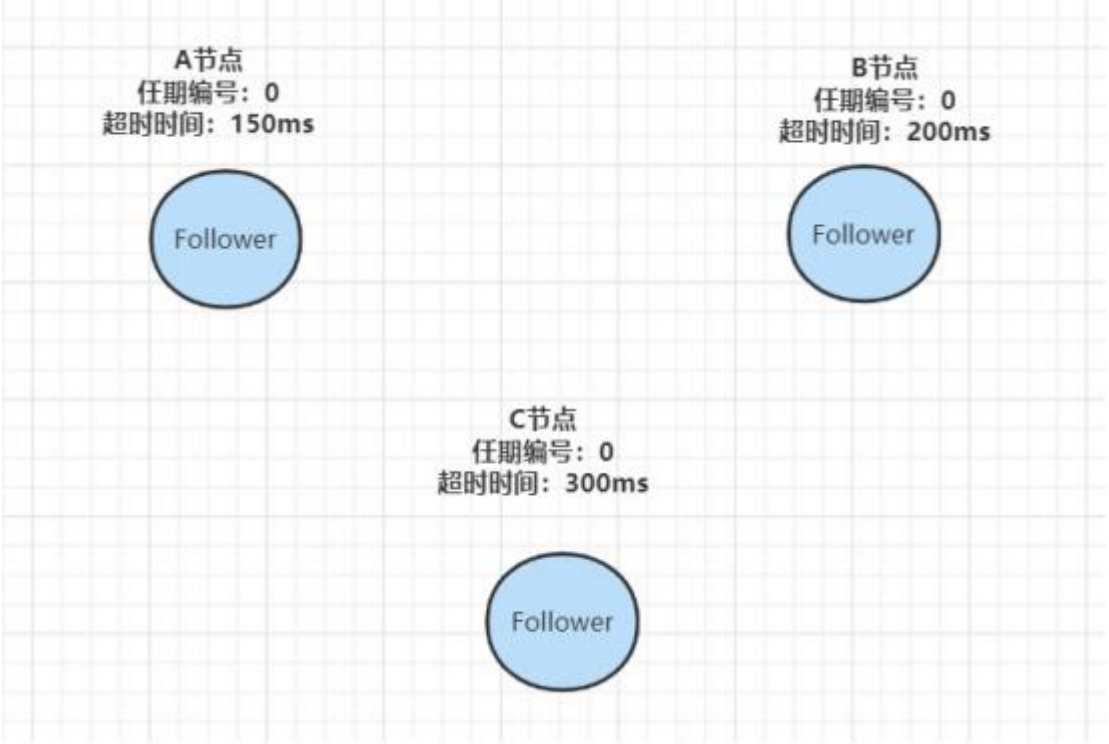
Raft 算法是分布式系统开发首选的共识算法，它通过“一切以领导者为准”的方式，实现一系列值的共识和各节点日志的一致。

Raft 选举规程涉及三种角色和任期（Term），

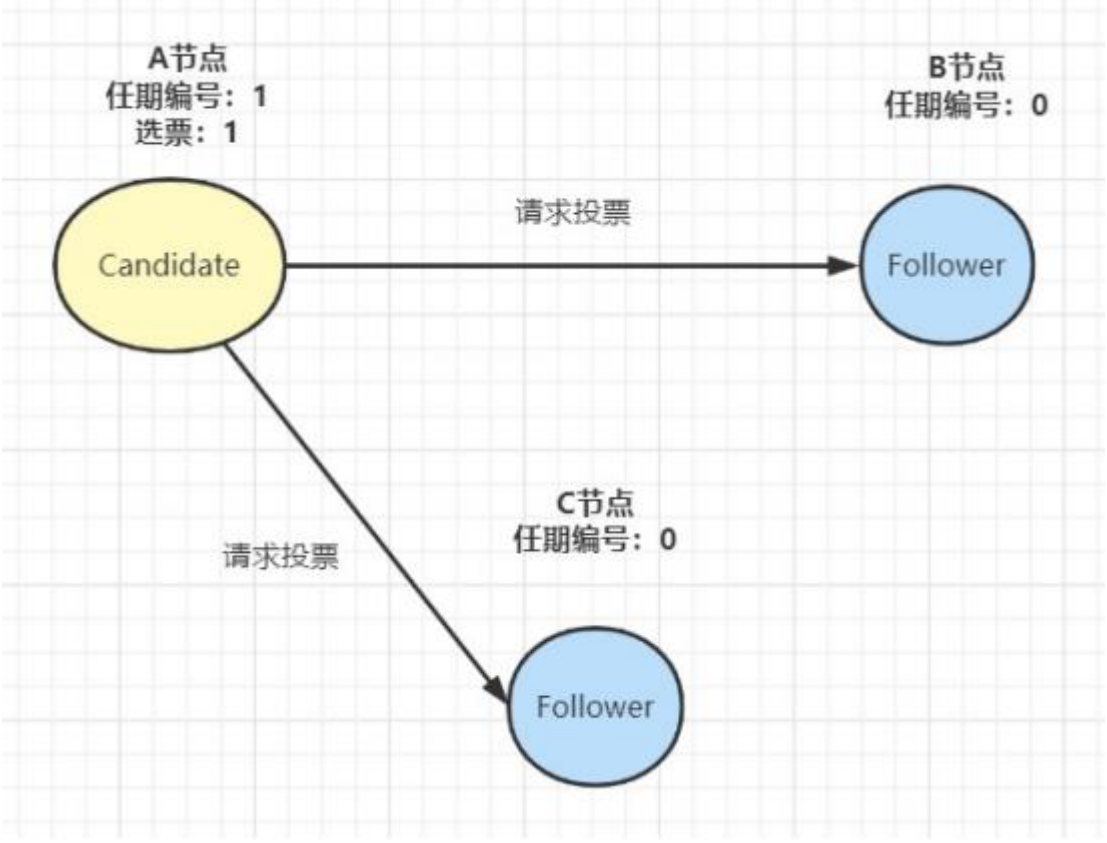
- Follower：默默地接收和处理来自 Leader 的消息，当等待 Leader 心跳信息超时
  的时候，就主动站出来，推荐自己当 Candidate。
- Candidate：向其他节点发送投票请求，通知其他节点来投票，如果赢得了大多数
  （N/2+1）选票，就晋升 Leader。
- Leader：负责处理客户端请求，进行日志复制等操作，每一轮选举的目标就是选出一个领导者；领导者会不断地发送心跳信息，通知其他节点“我是领导者，我还 活着，你们不要发起新的选举，不用找个新领导者来替代我。”
- Term：这跟民主社会的选举很像，每一届新的履职期称之为一届任期

领导选举过程

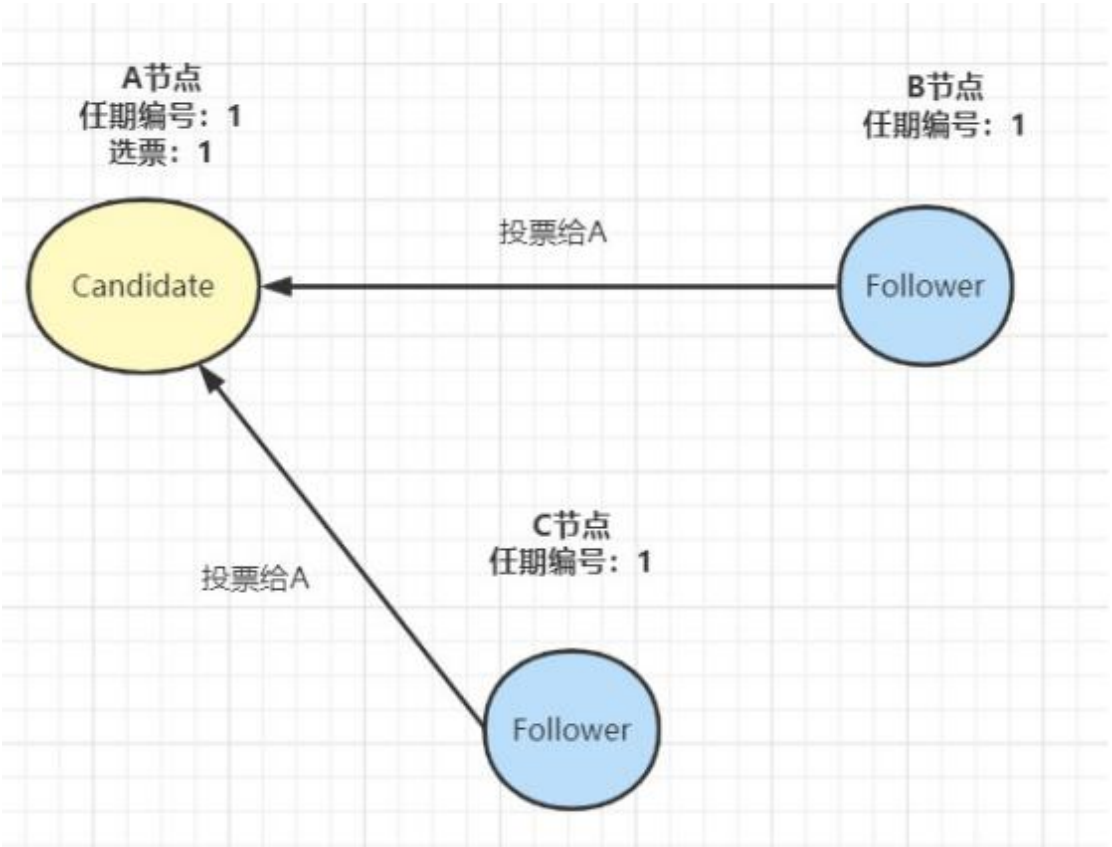
1. 在初始时，集群中所有的节点都是 Follower 状态，都被设定一个随机选举超时时间（一般 150ms-300ms）：



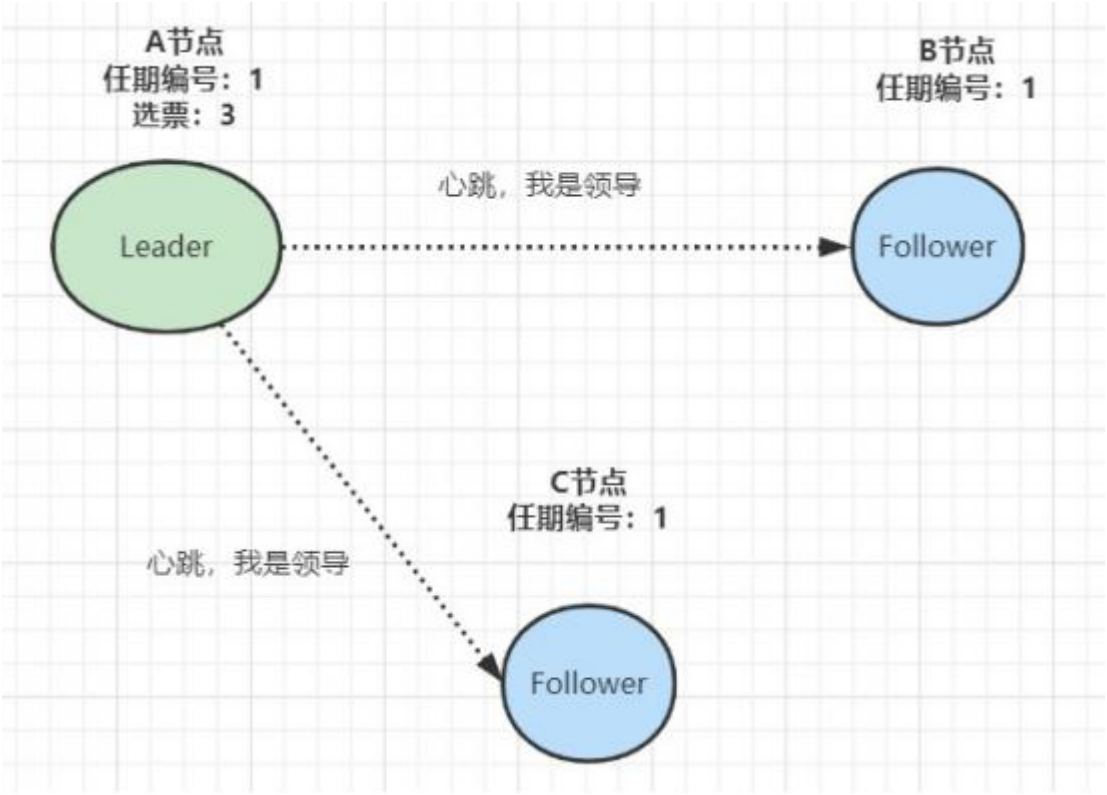
1. 如果 Follower 在规定的超时时间，都没有收到来自 Leader 的心跳，它就发起选举：将自己的状态切为 Candidate，增加自己✂任期编号，然后向集群中的其它 Follower 节点发送请求，询问其是否选举自己成为 Leader：



1. 其他节点收到候选人 A 的请求投票信息后，如果在编号为 1 的这届任期内还没有进行过投票，那么它将的选票投给节点 A，并增加自己的任期编号：



1. 当收到来自集群中过半节点的接受投票后，A 节点即成为本届任期内 Leader，他将周期性地发送心跳信息，通知其他节点我是 Leader，阻止 Follower 发起新的 选举：



54.27 聊聊项目，好的设计，好的代码

项目的话，你可以聊聊你平时做的项目，尤其有亮点的项目。如果没有什么特别亮点的项目，也可以说说一些好的设计，或者你优化了什么接口，性能提升了多少，优化了什么慢 SQL 都可以。甚至是一些好的代码写法都可以。

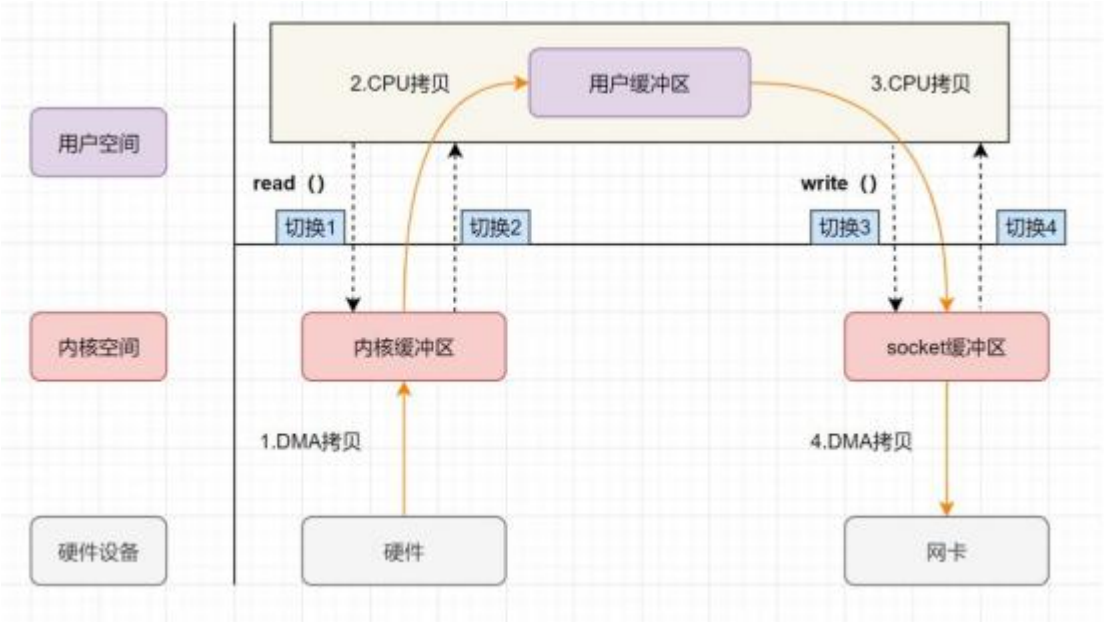
54.28 谈谈什么是零拷贝？

零拷贝是指计算机执行 IO 操作时，CPU 不需要将数据从一个存储区域复制到另一个存储区域，从而可以减少上下文切换以及 CPU 的拷贝时间。它是一种 I/O 操作优化技术。

54.29 传统 IO 的执行流程

传统的 IO 流程，包括 read 和 write 的过程。

- read：把数据从磁盘读取到内核缓冲区，再拷贝到用户缓冲区
- write：先把数据写入到 socket 缓冲区，最后写入网卡设备。



1. 用户应用进程调用 read 函数，向操作系统发起 IO 调用，上下文从用户态转为内核态（切换 1）
2. DMA 控制器的数据从磁盘中，读取到内核缓冲区。
3. CPU 把内核缓冲区数据，拷贝到用户应用缓冲区，上下文从内核态转为用户态（切换 2），read 函数返回
4. 用户应用进程通过 write 函数，发起 IO 调用，上下文从用户态转为内核态（切换 3）
5. CPU 将用户缓冲区中的数据，拷贝到 socket 缓冲区
6. DMA 控制器把数据从 socket 缓冲区，拷贝到网卡设备，上下文从内核态切换回用户态（切换 4），write 函数返回

传统 IO 的读写流程，包括了 **4 次上下文切换**（4 次用户态和内核态的切换），**4 次数据拷贝**（两次 CPU 拷贝以及两次 DMA 拷贝)。

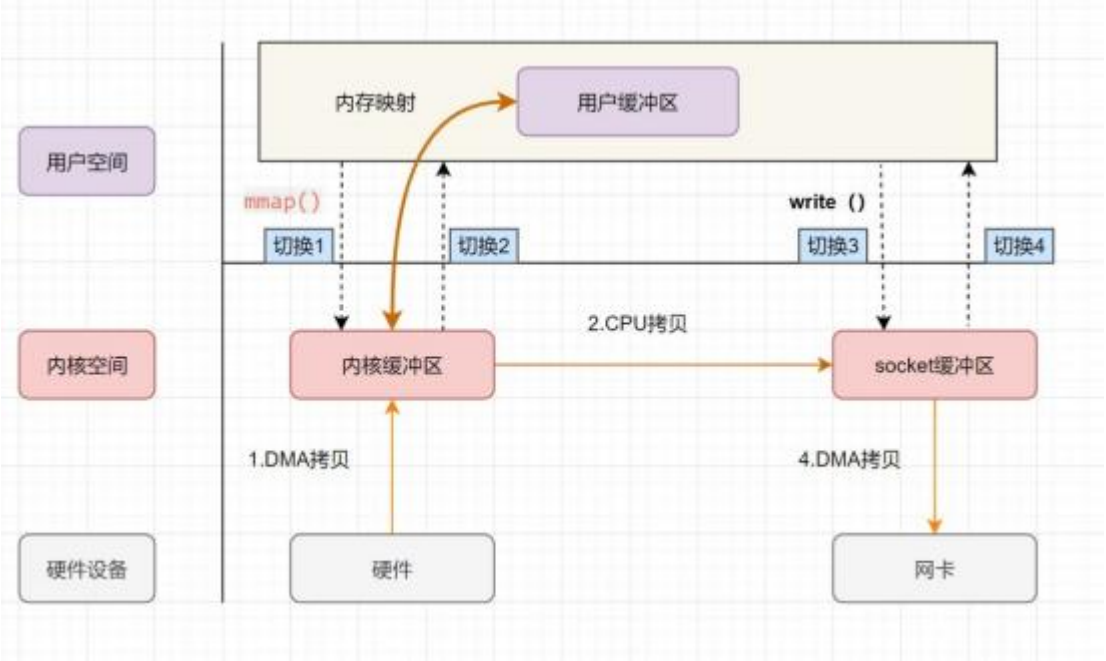
54.30 零拷贝实现方式

零拷贝并不是没有拷贝数据，而是减少用户态/内核态的切换次数以及 CPU 拷贝的次数。零拷贝一般有这三种实现方式：

- mmap+write
- sendfile
- 带有 DMA 收集拷贝功能的 sendfile

mmap+write

mmap 就是用了虚拟内存这个特点，它将内核中的读缓冲区与用户空间的缓冲区进行映射，以减少数据拷贝次数！

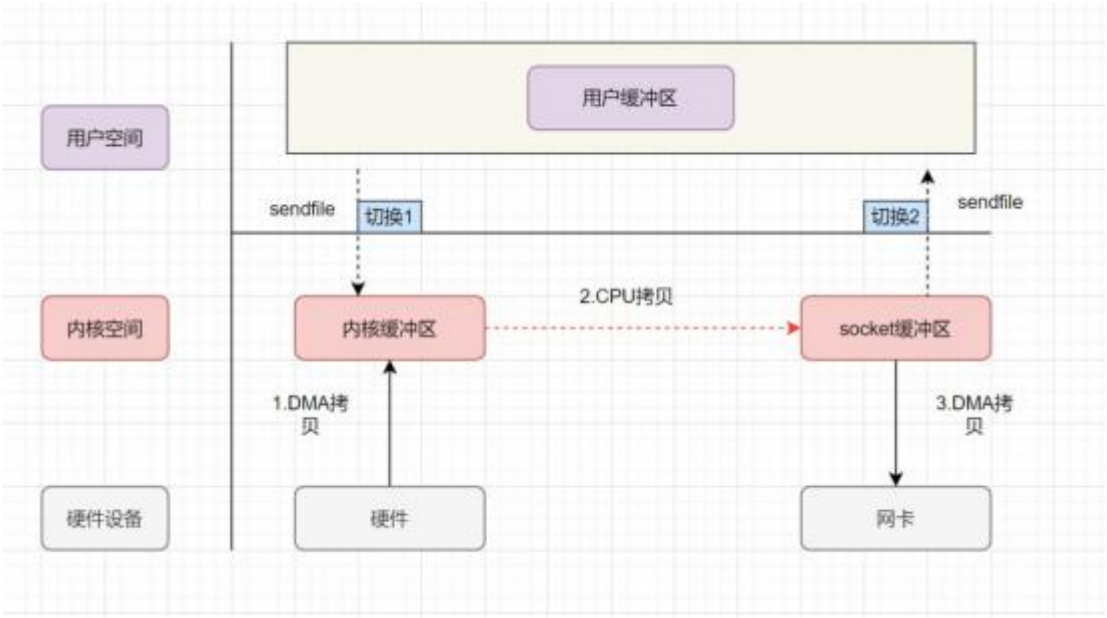


1. 用户进程通过 mmap 方法向操作系统内核发起 IO 调用，上下文从用户态切换为内核态。
2. CPU 利用 DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。
3. 上下文从内核态切换回用户态，mmap 方法返回。
4. 用户进程通过 write 方法向操作系统内核发起 IO 调用，上下文从用户态切换为内核态。
5. CPU 将内核缓冲区把数据拷贝到 socket 缓冲区。
6. CPU 利用 DMA 控制器，✂数据从 socket 缓冲区拷贝到网卡，上下文从内核态切换回用户态，write 调用返回。

mmap+write 实现零拷贝，I/O 发生了 4 次用户空间与内核空间的上下文切换，以及 3 次数据拷贝（包括了 2 次 DMA 拷贝和 1 次 CPU 拷贝）。

sendfile

sendfile 表示在两个文件描述符之间传输数据，它是在操作系统内核中操作的，



避免了数据从内核缓冲区和用户缓冲区之间的拷贝操作

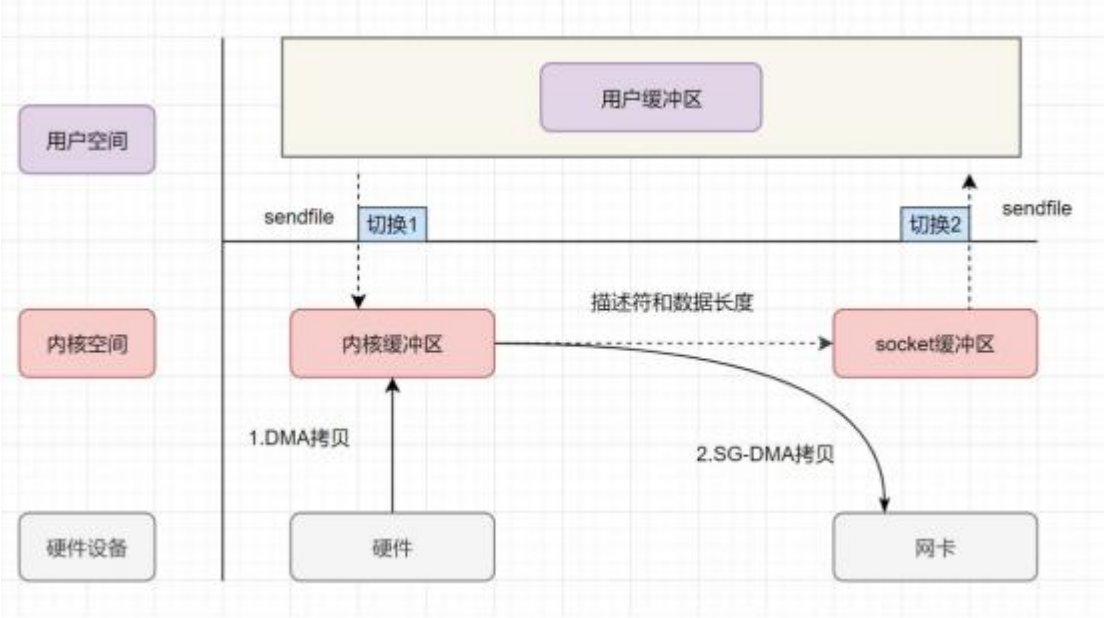
7. 用户进程发起 sendfile 系统调用，上下文（切换 1）从用户态转向内核态
8. DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。
9. CPU 将读缓冲区中数据拷贝到 socket 缓冲区
10. DMA 控制器，异步把数据从 socket 缓冲区拷贝到网卡，
11. 上下文（切换 2）从内核态切换回用户态，sendfile 调用返回。

sendfile 实现零拷贝，I/O 发生了 2 次用户空间与内核空间的上下文切换，以及 3 次数据拷贝。

其中 3 次数据拷贝中，包括了 2 次 DMA 拷贝和 1 次 CPU 拷贝。

带有 DMA 收集拷贝功能的 sendfile

linux 2.4 版本之后，对 sendfile 做了优化升级，引入 SG-DMA 技术，其实就是对 DMA 拷贝加入了 scatter/gather 操作，它可以直接从内核空间缓冲区中



将数据读取到网卡。使用这个特点搞零拷贝，即还可以多省去一次 CPU 拷贝。

12. 用户进程发起 sendfile 系统调用，上下文（切换 1）从用户态转向内核态
13. DMA 控制器，把数据从硬盘中拷贝到内核缓冲区。
14. CPU 把内核缓冲区中的文件描述符信息（包括内核缓冲区✂内存地址和偏移量）发送到 socket 缓冲区
15. DMA 控制器根据文件描述符信息，直接把数据从内核缓冲区拷贝到网卡
16. 上下文（切换 2）从内核态切换回用户态，sendfile 调用返回。

可以发现，**sendfile+DMA scatter/gather** 实现的零拷贝，I/O 发生了 2 次用户空间与内核空间的上下文切换，以及 2 次数据拷贝。其中 2 次数据拷贝都是 DMA 拷贝。这就是真正的 零拷贝（Zero-copy）技术，全程都没有通过 CPU 来搬运数据，所有的数据都是通过 DMA 来进行传输的。

54.31 请你说下对 InnoDB 索引数据结构的理解?

1.面试官：今天问你一个简单的问题 请你说下对 InnoDB 索引数据结构的理解?

灰灰老师：好的面试官

首先，在说 Mysql InnoDB 索引之前，可以先讲一下 InnoDB 的数据存储。

2.面试官：我在问你 InnoDB 索引数据结构的理解，为什么你要先讲一下 InnoDB 的数据存储

灰灰老师：

面试官是这样的，因为 Mysql 的索引要讲清楚跟数据存储有一定的关系。

在 InnoDB 中的数据都是以行的形式存储在 page 页，并且 page 页跟 page 页以双向链表进行关联。

我们在去查询数据的时候，如果基于我们的链表去查询，由于链表的查询性能是 O(n)，数据越大，查询的性能会越来越慢。

所以，Mysql 提出了一个索引的概念，不同的储存引擎的索引方式可能不一样。

3.面试官：恩，那 InnoDB 的采用了哪些实现方式呢?

InnoDB 的实现方式采取了跟书目录一样的思想，为每个 page 页创建一个目录记录，这些目录记录也都会保存到 page 页，由于 page 页大小有限，所以就会有多个目录 page。 当有多个目录 page 的时候，再为目录 page 创建一个目录，直到根节点。

最后，形成了一个，叶子节点是存有真实数据的 page 页的双向链表，而非叶子节点是存储目录的 page 页的树形结构。这样的数据结构我们就称为 B+树。

4.面试官：那你简单讲一下他带了哪些优势

灰灰老师：

第一呢之前是链表遍历查询，现在是树的查询，树的查询性能比链表要高很多，查询次数由树的高度来决定，树越矮跟磁盘的 io 次数也就越低

第二呢查询相对稳定，因为数据全部在叶子节点，不管查询什么数据，都需要查询到叶子节点

5.面试官：那为什么不用二叉树或者红黑树?

灰灰老师：

这里也有两点原因

首先二叉树会有斜树的可能，会退化成链表，那么索引树就没有作用

其次像红黑树，树的高度会很高。同样的，B-树的数据结构虽然跟 B+很类似，但是 B-没有目录节点的说话，每个节点都存储真实的数据，那样由于每个 page 要存的数据变大，节点就会越多，树层级就越高，查询来讲整体变慢。

所以，InnoDB 索引的目的是通过空间换时间，并且尽可能的去减少磁盘的 IO 次数。

这也是为什么用 B+树，而不用跳表等其他数据结构的原因。

54.32 请你说下对 Redis 锁的理解?

1.面试官：看了你的简历，外包工作三年，精通 Redis，那我们现在开始面试，请你简单说下你对 Redis 锁的理解

灰灰老师：好的面试官

首先，我们得讲下什么是锁，所谓锁，就是某些共享的资源，我不希望有并发执行的可能，而是让这些资源串行执行。

比如下单，同一个订单我只能由一个人同时去支付。

所以，锁的核心作用，我有线程在做的时候，其他线程不能做。

2.面试官：那既然说到锁了，那你怎么去实现一把锁：

灰灰老师：实现锁的必要条件有个标记这个标记来决定是否有线程在执行，并且这个标记获取必须是安全的，2 这个标记必须是线程可见的

比如厕所的门，如果 2 个人看到门开着，只能有 1 个人进入，还有这个门对盲人无效，盲人必须通过其他方式来确认是否有人。

Redis 实现分布式锁也满足了上述条件：

第一呢.因为 Redis 是 k.v 结构，我们可以把 key 做为一个标记，比如支付订单 111，我们可以把 pay:111 作为 key

第二呢.并且这个 key 是线程可见的， 因为 Redis 里面的指令执行是单线程的。

还有就是要保证获取这个 key 的安全性就可以，由于是单线程指令执行，我们可以用 setnx，如果不存在就能设置成功。

或者采用 lua 脚本来保证多个指令的原子性。

所以在我看来，Redis 能实现分布式锁最关键的点在于 Redis 的命令执行是单线程。

3.面试官：回答还不错，那再问一个问题 Redisson 做了什么事情呢?

灰灰老师：Redisson 呢 其实是 Redis 的一个客户端，它对 Redis 能做的一些事情都进行了封装，用户使用的时候只要去用已经封装的功能就行。

比如分布式锁，Redisson 就进行了很好的封装，主要实现了 5 件事情.

第一.基于 Redis 的发布与订阅 以及 Semaphore 实现了锁的等待，就是没拿到锁我可以等待一段时间。

第二.实现了可重入锁，基于 Redis 的 Hash 数据结构

第三.为了死锁，给锁添加了过期时间

第四.为了防止业务没有执行完，锁被释放，利用时间轮添加了续期机制

第五.因为 Redis 是属于 ap 模型，在发生分区容错的时候，优先保证高可用，所以会牺牲一定的数据一致性。为了防止锁丢失，也提供了联锁的概念。

54.33 我看你在项目中有用到线程池，那你能说下线程池的执行原理么?

2.面试官：哼 你也别不服气，随便考你一道题，说下对线程池的理解?

灰灰：表情思考一下

首先多线程，是我们在开发中提升并发能力的一个常用的方案，但是我们一般不会自己去手动创建线程，因为这样线程不易于管理，每个人都去创建线程，可能会出现无限的线程创建，导致响应速度反而越来越慢。

所以，我们一般会用线程池来创建线程，线程池的核心思想其实是线程复用，我希望用固定的线程数，来执行不定的任务。

我们 juc 包下的 Executors 其实封装了很多线程池的实现方式，比如

newFixedThreadPool newCachedThreadPool 等等

但是我们在工作中不建议直接用封装好的方式，因为如果用封装好的，很多人不会去看底层的实现原理，导致乱用。线程

如果没有用好，影响的不是某断代码或者某个服务，而是整台机器，所以要使用，建议把线程池的原理弄清楚。

这是我对线程池的理解。

3.面试官，那你说下线程池要传哪些参数?

在讲到线程池的原理的时候，我们必须要知道线程池的 7 个参数

1.核心线程数 默认会开启的线程数量

2.最大线程数 任务过多，我会额外开启的最大线程

3.线程等待时间 没有任务的时候，这个线程过多久会回收（如果能回收的话）

4.等待单位

5.阻塞队列 放任务的队列

6.线程的创建工厂

7.拒绝策略 在某些情况下，我会丢失任务 如果有拒绝策略 ，会进入拒绝策略

这些参数都会在流程中能使用到,每个参数都有每个参数的作用。

4.面试官，你刚才说线程池的流程会用到这些参数，那分别在哪里用到?

参数的使用，要先捋下线程池的执行流程。

线程池去执行任务最终是通过 exectue 方法去执行,传的任务是实现了 Runnable 接口的对象并实现 run 方法。实现流程如下

1.先会判断已经开启的线程数是否小于核心线程，如果是，会创建一个 worker 对象，这个 worker 是实现了 runnable 接口的，并且 worker 对象持有一个 Thread 对象。通过启动这个 Thread 去开启线程。 这是核心线程与线程工厂的使用场景，核心线程来判断我是否需要创建新的 thread，线程工厂决定我怎么去创建线程。

2.如果线程池已创建的线程数大于等于核心线程数，会把任务给到阻塞队列，已经开启的线程会自旋去阻塞队列获取任务。

阻塞队列就是用来存储我来不及执行的任务的。

3.如果我已经开启了核心线程数的线程，并且阻塞队列放不下了，那么我们会去创建额外的线程来帮助执行，但是总线程数不能超过最大线程。

最大线程是用来限制我线程池能开启的最大线程

4.如果阻塞队列满 了，并且也开启了最大线程的线程数，将不在接收任务，会进入拒绝策略。拒绝策略可以自己实现，实现 RejectedExecutionHandler 接口即可。

现在我们知道了 5 个参数的作用,还有 2 个参数在细节里面,就是过期时间与过期单位。这 2 个参数 是当阻塞队列没有任务的时候，我要去回收多余的线程或者回收所有的线程的时候，一个超时等待时间，也就是阻塞队列的 poll 的时间。

5.面试官：我看你提到了核心线程数 那这个核心线程跟最大线程有什么区别?

其实 7 大参数里面的核心线程数与最大核心线程数，其实就是一个线程的上限判断。核心线程，是来一个任务我就会开启一个线程，直到达到核心线程数为止。

最大线程是我的阻塞队列满了的时候，我需要开启线程来帮助处理任务。但是他们的开启回收逻辑是一模一样的。

默认会保证核心线程数的线程在阻塞等待，不被回收，那么至于哪些回收，哪些等待，就看线程哪个执行到判断现有线程数是否大于核心线程数的逻辑。

所以核心线程数与最大线程数只是用来做线程判断，对于线程本身来讲，没有任何区别。

54.34 我们都知道 Redis 属于一个内存型数据库，那么内存如果满了，怎么处理?

Redis 是一个内存型数据库，所有的数据是基于内存操作的，内存都是有大小限制的，在启动实例的时候还能指定内存大小。

首先，我们在设计的时候，尽量去节省内存或者防止内存不足，比如设置 key 的过期时间，到了时间会自动从内存移除，在存储数据的时候，会进行压缩，会搭建 cluster 集群来做负载。

如果真的发生了内存满了，Redis 会提供不同的策略来处理

1.默认是 novication,就是内存满了的时候 会报 OOM 错误，这个时候，数据能读但是不能写。

2.如果发现内存满了后，为了保证这次的数据能顺利写入，会去想办法淘汰一些之前的数据，那么淘汰哪些数据？Redis 会提供一些算法，比如 LRU、LFU、随机、根据过期时间来。

还会提供淘汰数据的范围选择：是去所有的 key 里面淘汰还是去设置了过期时间的 key 里淘汰。

面试官：LFU 根据最少次数来淘汰，会有一个时效性问题，就是有些数据可能不是热点数据，但是由于之前的点击次数还在，那就导致旧数据淘汰不了，新的数据一直被淘汰，怎么解决?

Redis 对象内部会用一个 24bit 的字段来记录对象的访问时间跟次数，前面 16bit 代表的是最后访问时间，后 8bit 代表的是次数，会根据前面的时间计算出这个对象多久没访问，如果很久没访问，会根据配置来减少对应的次数。

这样，越久时间不访问，次数减得越多。就能解决时效性问题。

54.35 reentantLock 的源码看过嘛，它是怎么实现的?

reentantLock 是基于 aqs 实现的一个可重入锁。它的实现主要有 2 种不同方式，一种是公平锁、一种是非公平锁。

他通过 aqs 中的 state 字段来维护是否获得锁或者重入了几次，并且这个字段需要用 volatile 来修饰，保证线程之间的可见性。

并且去抢占这个标记必须是安全的，所以会通过 CAS 来保证抢占过程的安全性。

如果是不同的线程，通过 cas 判断是否能拿到锁，如果是同一个线程重复抢占锁，那么 state+1，这样就实现了可重入。

如果拿到锁的线程，去执行相关业务代码，如果拿不到锁，会加入一个等待队列，是一个双向链表，这个队列也属于 aqs 维护，加入队列后 park，这个时候线程就等待拿到锁的线程释放锁去唤醒。

当拿锁的线程 unlock 后，会去唤醒等待队列的第一个等待的线程。自旋去获取抢占锁。

面试官：你刚才讲了 2 个实现，公平锁与非公平锁，他们的区别在哪?

他们的大致流程都是一样的，他们的区别主要在于去获取锁的时候，公平锁会去判断是不是等待队列的第一个线程或者没有线程在等待，如果是，就能获取锁；如果有线程在等待，必须按照队列的顺序来获取锁。新的线程必须排在等待队列后面。这也是公平的由来，任何线程都必须按照等待队列的顺序来获取锁

而非公平，只要锁释放，没有加入等待队列的线程能提前通过 cas 去抢占锁，可能拿到锁的时间节点比等待队列的线程更早。

54.36 Redis 主从数据一致性怎么保证

求职者：

年底了，贵司的这个岗位也是急着要人，给我一次面试机会，面完再决定要不要发 offer

求职者：

Redis 主从数据一致性，主要是讲 Redis 主从数据是怎么同步的，主要有 3 种，分别是全量

同步、增量同步、指令同步。每种同步方式的时机不一样。

****全量同步****

1.全量同步，一般发生在第一次建立主从关系、或者跟主断开时间比较久的场景。

它的步骤为：

1.slave 向 master 发起同步指令，master 收到以后，会通过 bgsave 指令生成一个快照 RDB

文件，同步给 slave，slave 拿到后，会通过 master 同步的快照文件进行加载。这个时候，

主生成 rdb 文件时候的所有数据都以及同步给了 slave。

2.但是 bgsave 指令是不会阻塞其他指令执行的，所以 master 在生成快照文件时，还是能接

收新的指令执行。这些指令 master 会先保存到一个叫 replication_buffer 的内存区间，等

slave 加载完快照文件后会同步。

面试官：

那为什么还需要增量同步呢?

求职者：

数据同步，slave 是定时会发起的，假如每次同步，都把主的所有数据都进行同步，那么性

能会很慢，大部分时候，slave 可能只跟 master 相差一部分数据。那么只需要同步这部分

数据。

slave 发起同步的时候，还会带有上次同步的偏移量，然后跟 master 的最新的偏移量比较，

如果相差的数据在 master 的积压缓存（一个专门存储 master 最新数据并且会覆盖的内存区

间）能查询到的话，那么只需要把相差的数据同步给 slave。这就叫做增量同步。

指令同步：master 输入的指令会异步同步给 slave。

54.37 ThreadLocaL 如何防止内存泄漏?

求职者：

是的，今天来公司面试 Java 中级开发的岗位，还请面试官多多指导

求职者：

首先，ThreadLocal 里面的数据都是以 entry 的数组保存在我们的 thread 里的，其中 entry

的 key 为 threadLocal 对象，value 是你设置的值。

entry 的 key 设置为弱引用

因为 thread 这个对象的生命周期其实是跟 thread 绑定的。那么如果 key 为强引用的话。

ThreadLocaL 的对象一直会被线程的 entry 持有，然后 ThreadLocaL 对象回收不了，但是外

面其实是可以把 threadLocal 对象强引用去掉的。那么这个时候，这个对象以及整个 entry

对象都是无效的，本来应该是需要回收的。但是因为强引用回收不了

所以 thread 的 key 是弱引用，这样我无效的时候我能 GC 回收对象。一定程度上面防止了

内存泄漏问题。

但是不够

因为虽然 key 的对象能被回收了，。但是 entry 还是不能回收，但是这些 entry 的 key 为

null 了，也没用了，需要回收。

所以，。threadLocal 通过各种机制，在使用的时候，。会去回收 key=null 的 entry 对象

比如当我去 set 的时候，会从当前冲突的位置向前或者向后找到第一个 null 的为止，并且

把找到的所有的被 GC 回收掉的 entry 清除！！并且 rehash，供其他对象使用，达到一个时

间与空间的平衡。

还是不够，因为如果没有 hash 冲突，自身是不能走清除的。所以我们一般使用的使用，用

完要 remove。

54.38 HashMap 是怎么解决哈希冲突的?

求职者

首先，我们得知道下 HashMap 的存储方式，HashMap 是 k-v 的数据结构， 底层是一个 entry 数组的数据结构， 并且根据 key 计算出一个 hash 值，取模 entry 数组长度-1，得到一个下标位置来存储数据。

所谓 hash 冲突 ，就是不同数据 hash 算法计算出来的结果可能是相同的，那么不同的数据就会放在同一个下标位置。

通常解决 hash 冲突的方法有 4 种。

1.开放寻址法，包含线性探测、平方探测等等，就是从发生冲突的那个位置开始，按照一定的次序从 hash 表中找到一个空闲的位置，然后把发生冲突的元素存入到这个空闲位置中。ThreadLocal 就用到了线性探测法来解决 hash 冲突的。

2.链式寻址法，这是一种非常常见的方法，简单理解就是把存在 hash 冲突的 key，以单向链表的方式来存储。

3.再 hash 法，就是当通过某个 hash 函数计算的 key 存在冲突时，再用另外一个 hash 函数对这个 key 做 hash，一直运算直到不再产生冲突。这种方式会增加计算时间，性能影响较大。

4.建立公共溢出区， 就是把 hash 表分为基本表和溢出表两个部分，凡事存在冲突的元素，一律放入到溢出表中。

HashMap 就是通过链式寻址法来解决的，但是链式地址有个问题：当链表过长的时候，会影响查询性能，所以在 HashMap 里，当链表长度大于 8 的时候，会转为红黑树，提升查询性能。但是树在添加数据的时候也会有树的分裂合并，所以在树节点小于 6 的时候，又会转为链表

这个是我对 HashMap 哈希冲突的理解。

面试官

54.39 HashMap 在并发的时候会有什么问题?

求职者

因为 HashMap 添加数据的时候，不是线程安全的，所以当并发添加数据的时候，有可能导致数据丢失，比如：当 2 个数据同时链地址到一个节点的时候，只会有 1 个是成功的。

还有，因为 entry 节点会有数据结构的变化，比如当链表长度达到 8 并且数组长度达到 64 后，会转为链表。那么在并发的时候，可能会产生数据结构类型转换错误。因为有可能在操作的时候，其他线程把数据结构变更。

所以，如果在有并发要求的场景，可以用 currentHashMap 或者 hashTable。它们是线程安全的。

54.40 你说下跳表跟 B+树的区别?

求职者

首先，跳表，其实就是一个链表结构。添加数据时都会随机一个层级，然后把每个数据层级相同的节点以链表的方式进行链接。

查询的时候，从最外层的链表开始查询，思想跟二分法的思想很类似。

B+树，叶子节点存有真实数据，非叶子节点是这些真实节点的目录页，查询的时候，从根目录开始查询，树的高度越高，查询的次数也就越多。

跳表跟 B+树都是用空间来换取时间，用额外的空间来保存链表或者目录页，来提升查询性能，但是也有一定的区别。

1.层高，B+树三层就能支持千万级别的数据，但是跳表，存储相同的数据量需要更高的层级，这也是为什么我们 innoDB 索引用 B+树而不用跳表的原因，InnoDb 需要跟磁盘 IO,层级越高，IO 次数也就越多，但是 Redis 的 zset 是用的跳表，因为 Redis 是基于内存操作，也就没有磁盘 IO 的概念，所以跳表反而更简单。

2.操作数据，跳表要比 B+树快，因为 B+树，在数据操作的时候，要去维护 B+树，所以会有树的分裂与合并，但是跳表是随机一个层次，实现相对简单。

54.41 Redis sentinel 集群和 Cluster 集群有什么区别?

Redis sentinel 集群和 Cluster 集群有什么区别?

求职者:

对于这个问题，我认为可以从 3 个方面来回答

首先，集群，我们首先想到的是主从集群，但是 Redis 的主从，只能做数据备份，但是当主发生宕机的情况下，从是不能够自动升级为主的。

所以我们需要一些其他的自动灾备集群方案。最常见的是 2 种，一种是 sentinel 哨兵集群，一种的 cluster 集群。

至于它们的区别主要有以下几个点：

1. Redis 哨兵集群是单独启动哨兵服务对主从节点进行监控，所以原则上还是主从，只不过哨兵实现了主从自动故障转移，它可以实现读写分离，分担 Redis 读操作的压力。而 Redis Cluster 集群的 Slave 节点只是实现冷备机制，它只有在 Master 宕机之后才会工作。

2. 从集群架构上来说，Redis 哨兵集群是一主多从， 而 Redis Cluster 是多主多从

3. 哨兵集群是一主多从，所有的数据都会存储在我的主，而 cluster 集群数据会通过虚拟槽的分片机制分配到不同的主服务器，所以扛压这块，哨兵集群受限于单个服务器的资源配置，而 cluster 是多主联合扛压。

4. RedisCluster 集群，采用虚拟槽的分片机制，能够灵活的在线进行扩容、缩容，以及灵活的进行数据隔离。

54.42 Redis 分布式锁跟 zk 分布式锁的区别在哪?

看你简历上，写的精通 Redis

那你详细描述下 Redis 分布式锁跟 zk 分布式锁的区别?

求职者

我们先来分析下他们 2 个的实现逻辑

1.Redis 是基于它的命令执行是单线程的这个特性，所以在获取数据的时候不会有并发问题来去做分布式锁的，其中 Redisson 客户端，对分布式锁进行了很好的包装，redisson 分布式锁 基于 lua+hash 来实现加锁逻辑，同时利用 Redis 的发布与订阅以及 seamphore 来实现锁的等待。并且添加了锁续期等功能。

2.zk 的分布式锁，主要是基于 zk 的临时有序节点+watch 机制来实现的，每个线程去尝试加锁都会创建一个临时有序节点，然后拿锁的时候，判断是否是最小的那几个节点，如果不是，就线程安全的去监听上一个节点。然后进行 wait。

区别主要体现在以下几个方面：

1.因为 Zk 是基于临时有序节点去获取锁，所以每个线程去抢占锁，都会去创建一个临时节点，当抢占锁的线程比较多的时候，对 ZK 集群的压力会比较大。而 Redis 则是通过判断 key 是否存在来获取锁。

2.Redis 属于 cap 里面的 ap，优先去保证高可用，所以在某些场景下，比如主从切换，可能导致数据丢失，导致锁丢失，而 zk 则是使用 zap 协议优先保证数据一致性，所以安全性方面，Redis 要比 ZK 低，但是也有像联锁的方式去提升可靠性。

3.性能方面，Redis 基于内存操作，所以性能比 zk 要高。

54.43 什么是缓存雪崩、缓存穿透、缓存击穿？你怎么解决?

请具体分析一下什么是缓存雪崩、缓存穿透、缓存击穿?

如果在开发过程中遇到了这些问题，你会怎么解决?

其实，缓存雪崩、穿透以及击穿，最终导致的问题都是我的请求绕过了我们的缓存中间件，直接打到了 DB 的集群，导致 DB 的压力过大或者造成 DB 崩溃的现象。只不过场景稍微有些不一样。

- 缓存雪崩

缓存雪崩主要有大量的 key 同时打到 DB，那么产生这种场景有 2 种主要原因。

1.缓存中间件服务不可用，缓存失效，导致所有的请求全部打到 DB，那么这种解决方案就是尽可能保证缓存中间件的高可用，比如 Redis 缓存，那么就采用 Redis 的 cluster 集群，以及提前做好缓存组件报警机制，杜绝宕机。

2.缓存中大部分 key 同时过期，比如 Redis 中 key 同时过期，对于这样的情况，可以在失效时间上增加一个 1 到 5 分钟的随机值。或者对热数据，进行访问的时候，自动续期。

- 缓存穿透

缓存穿透是指缓存和数据库中都没有的数据，但是用户一直请求不存在的数据！这时的用户很可能就是攻击者，恶意搞你们公司的，攻击会导致数据库压力过大。

1.我们可以在缓存之外再加一层性能更好的过滤方案，比如布隆过滤器或者布谷鸟过滤器。如果数据不存在，直接在过滤器进行过滤掉，不会打到 Redis 跟 DB。

2.如果发现是恶意攻击，可以对 IP 进行处理，加入 IP 黑名单。

- 缓存击穿

单个 key 过期的时候有大量并发，我们可以采用 dcl 锁等方式，但是我觉得如果要采用锁来保证，那么就没有必要用到缓存。

所以，在我看来，您提出来的这个问题，有点过于放大了它带来的影响。

首先，在一个成熟的系统里面，对于比较重要的热点数据，必然会有一个专门缓存系统来维护，同时它的过期时间的维护必然和其他业务的 key 会有一定的差别。而且非常重要的场景，我们还会设计多级缓存系统。

其次，即便是触发了缓存雪崩，数据库本身的容灾能力也并没有那么脆弱，数据库的主从、双主、读写分离这些策略都能够很好的缓解并发流量。

最后，数据库本身也有最大连接数的限制，超过限制的请求会被拒绝，再结合熔断机制，也能够很好的保护数据库系统，最多就是造成部分用户体验不好。

54.44 缓存跟 DB 的一致性怎么产生的?

我看你之前的项目开发中用过 Redis 做缓存，你有遇到过缓存跟 db 的一致性的问题吗

求职者

Redis 缓存这一块，我的实战经验还是非常丰富的

首先，我们缓存是为了减少跟 DB 的压力。

当我们去访问数据时，我们先去缓存读取，如果缓存有，直接不需要跟 DB 进行交互，如果缓存没有，那么从 DB 获取，然后存到缓存中。

当数据发生变更的时候，我们也会做好缓存依赖，会同步到缓存，以 Redis 为例，我们会删除 Redis 的数据。

但是，就算做好了缓存依赖，但是因为操作 DB 跟删除 Redis 不是一个原子操作，所以，在同步到到 Redis 之前，可能有其他的线程改动了 DB 的数据。简单举个例子。

1.线程 A 请求缓存，没有缓存，从 DB 拿到 1， 要去更新到缓存，但是还没更新到缓存

2.线程 B 将 1 更新为 2，并且删除缓存，DB 的值为 2

3.线程 A 更新缓存，redis 为 1

所以，根本原因其实就是并发带来的原子性问题。

面试官

那你们是如何解决的呢?

求职者

不可取的强一致性方案

延时双删

所谓延时双删，就是在更新 DB 后，等待一段时间，再进行 Redis 删除！来等待其他的线程拿到的都是最新数据！

也会产生很多问题。

1.延时多久？不知道其他线程要多久。

2.不够优雅，代码中写入延时代码。

> 采用锁机制，不让有并发

在更新的时候，采取锁的机制，不让其他线程进行删除操作！

但是会拖慢整个性能，违背了 Redis 的初衷

所以，我们只能采用最终一致性，不应该去保证强一致性

> 最终一致性方案

> 每个缓存设置过期时间

设置过期时间，就算数据不一致，也只是在有效时间内的不一致。

> Mysql canal 等数据同步工具

捕捉到 DB 的更改，同步到相关 Redis，相对比较复杂，要知道每个数据对应的缓存。

RocketMQ

比如基于 RocketMQ 的可靠性消息通信，来实现最终一致性。

54.45 请说下对动态代理的理解

长沙跟深圳的薪资那是天差地别

建议你这边还是要调整一下心理预期

我们先面试吧

请具体说说你对动态代理的理解

首先，所谓动态代理，它的核心思想在于拦截，去拦截被代理类的方法，当方法被代理类的方法的时候，走到自定义的拦截器

动态代理的实现方式有 2 种：

1.JDK 动态代理

jdk 代理里面有个很重要的类，Proxy，会通过这个 Proxy 类的 newProxyInstance 方法去反射生成一个代理类，生成的代理类必须实现被代理类实现的接口。

newProxyInstance 的 3 个参数，第一个：classLoader 类加载器 去加载我生成的代理类 2.要代理的接口 3.调用接口方法的时候要走到哪个拦截器，拦截器必须实现 InvocationHandler 接口的 invoke 方法，后面拦截流程全部走到 invoke 方法。

因为 JDK 动态代理是基于实现接口去做的，所以代理只能代理接口 或者实现了接口的类。

2.CGLIB 代理

CGLIB 代理的关键类是 Enhancer，通过 Enhancer 去创建代理类以及设置被代理类，CGLIB 是通过继承被代理类的方式去实现拦截，所以会设置 Superclass 以及 Callback，Callback 就是我们的拦截类。

CGLIB 是通过 ASM 框架去生成代理类的，所以生成比较复杂。

使用场景：我们的 Spring 里面的 aop 支持 jdk 以及 cglib 代理，我们 Mybatis 的主流程以及插件就是用到了 jdk 动态代理。

54.46 项目中有用过 Redis 么，用在哪里?

面试官

先问一点专业问题吧

54.46.1 项目中有用过 Redis 么，用在哪里?

我们项目中有用到，主要用了 3 个场景：

1.缓存相关的场景，我们是做在线教育的，内容模板会有很多课程相关，这些数据在 DB 单表有 5.600W；如果走 mysql 查询会很比较慢，用户体验感比较差，并发也上不去。所以我们做了些接口缓存、课程内容的对象缓存。提升了性能的同时，还解决了本地缓存不一致问题。

2.同时，我们由于它是分布式的，并且可以设置过期时间，也会用来保存用户 token。因为 token 也是有过期时间的，用 Redis 来保存刚好满足。

3.我们还用它去基于日期+incr 自增指令实现了一个分布式 ID。因为我们的课程 ID 比较大，需要分库分表，数据库自增满足不了我们需求。

54.46.2 你们缓存的单个 key 是多大的？会去控制 key 的大小么

我们的 key 因为有压缩，所以我们的 key 一般会控制在 10K 之内，比较大的 key 我们是不会放入 Redis 的。

因为如果 Key 过大，会由于带宽、网络等原因导致指令返回速度过慢，Redis 又是单线程的，会阻塞其他指令执行。

54.46.3 有遇到过缓存跟 DB 的数据不一致么，怎么产生的？怎么解决？

遇到过的，主要是因为操作 DB 跟 redis 这 2 个操作，不是原子性操作，所以在并发的时候，可能会导致数据不一致。其实就是并发导致的原子性问题。在拿到数据准备放入 Redis 的時候，另外的线程对 DB 数据进行了改动。

解决方案：

最优最好的方案：给 Redis 的缓存数据加上过期时间。达到最终一致性，就算不一致，在过期时间到了后，就能一致。

当然网上也有说通过锁、延迟双删；但是我觉得都不太适合我们的应用场景的。因为我们 Redis 本来就是为了提升性能。如果再加上一些消耗性能的方式来保证强一致性，那么你干脆不要用 Redis。

设置过期时间会从 Redis 删除，你知道怎么删的吗？

删除的话，Redis 主要有 2 种方式：

1.惰性删除　当我去获取这个 key 的时候，判断有没有过期，过期了就删除

2.定期过期　Redis 会有个事件驱动机制，每隔一段时间会去扫描我的 key 里面是否有过期的数据，如果有，我就去删除。

隔多久可以根据 config 文件配置，扫描也不会扫描所有的，而是分批去扫描。

54.46.4 还知道 Redis 的其他的一些数据类型的使用场景么?

我们工作中，刚才我讲的缓存、自增 ID、保存 token 其实都是基于 String 做的。

但是由于 Redis 性能比 DB 高，所以根据不同的缓存场景，我可以用不同的数据类型来支撑，比如 hash 我就可以用来做对象型的数据缓存，比如我们之前课程会有浏览数、购买数　这些数量可以用 hash 来存储。

还有一些集合类的缓存，可以用 list、set 存储。　如果我的数据有分数排行的，可以用 zset 做排行缓存。

Redis2.8.9 后引入了基于 HyperLogLog 算法的 HyperLogLog 数据类型。可以做一些统计类的数据，比如 UV，这个的好处就是内存会占用很小很小。但是会有一定的误差，也只能做统计。

Redis3.2 后，也加入了 GEO 的数据类型。可以做一些查询附近地址等功能。可以存储经纬度，然后可以拿到彼此的相差距离，比如搜索饭店，哪个离我最近，有多远。

Redis5.0 后又引入了 Stream，可以做类似 mq 的发布订阅功能，并且支持 ack 回执。

54.47 Redis 怎么做缓存设计的?

你刚提到 Redis 做缓存，那么性能肯定比 Mysql 要高，

那 Redis 是怎么设计的?

1.Redis 是基于内存操作的，节省了跟磁盘的实时 IO，但是后台还是会异步进行磁盘备份。所以数据一致性不强，会有数据丢失。

2.由于是基于内存操作，所以单次执行非常快，瓶颈不在 cpu。设计的是单线程指令执行。减少了执行指令时候的线程切换

3.Redis 本身就是一个 k-v 保存结构。查询数据的时间接近 O(1),只有当数据发生冲突的时候才会遍历链表，跟 HashMap 很类似。

4.Redis 底层的数据结构支持，思想就是空间换时间，比如 SDS、跳表

5.网络模型采用的是多路复用。大大减少了网络 IO 的时间消耗

面试官

跳表用在哪里？跳表的思路是什么

跳表主要用在 zset 的有序数据集合类型。

跳表的思路跟 Mysql 的 b+树差不多，主要是我的数据会保存一个随机的层级，每个层级会跟相同层级的数据建立一个双向链表。

找数据的时候，会从最外层开始，然后数据是有序的，查询过程跟二分法一样。

核心思想就是用空间来换取时间，提升查询性能。

面试官

你说 k-v 跟 HashMap 很类似，解决冲突用的是链表，那为什么 Redis 用头插，而 HashMap 用的是尾插

HashMap 用尾插，是因为在并发的场景下，可能会导致链表的死链。

但是 Redis 是单线程执行，所以不会有并发导致死链，头插法比尾插速度要快很多。所以用头插法即可。

面试官

那 Redis 有没有像 HashMap 一样的扩容机制

灰灰

Redis 肯定也有扩容机制，因为如果没有扩容的话，会导致链表越来越长，从而降低查询性能。

只不过 Redis 的扩容机制跟 HashMap 有点不一样，Redis 会有 2 个 hashTable，第二个 table 只有再扩容的时候使用，当第一个 table 的容量达到一定量，这个量正常是已有的数据是 table 大小的时候就会扩容，但是当有在进行持久化的时候，使用量是 table 容量的 5 倍的时候扩容

扩容也不会一下子都扩容完成，因为一下子把所有的数据从第一个 table 移到到第二个 table 耗时太长。

所以会采用渐进式 rehash，分批次的将数据迁移到第二个 table。然后把第一个 table 变量指向新 table。第二个 table 赋空。

54.48 Redis 内存满了，如何处理

我们接着聊技术

你有遇到过内存满的情况么，这种一般怎么处理？

灰灰

有遇到过的，内存满了的话，会报 OOM 错误，数据能读不能写了，因为没有足够的内存去接收新的数据。

至于解决方案：

1.首先想到的肯定是去加内存，包括加大单机的内存，或者横向扩容

当然，如果你允许一些老的、或者冷门的数据删除的话，你也可以在 config 文件设置淘汰策略。

默认官方提供 8 种淘汰策略：有 3 种算法(LRU、LFU、random)，然后每种算法的淘汰范围都可以设置淘汰范围 allkeys 还是 volatile 是淘汰所有的数据还是只淘汰设置了过期时间的。再加上 novication 不淘汰、根据 volatile-ttl 过期时间淘汰

1. volatile-lru：从已设置过期时间的数据中挑选最近最少使用的数据淘汰
2. volatile-lfu：从已设置过期时间的数据中挑选使用次数最少的数据淘汰
3. volatile-ttl：从已设置过期时间的数据中挑选将要过期的数据淘汰
4. volatile-random：从已设置过期时间的数据中任意选择数据淘汰
5. allkeys-lru：从所有数据中挑选最近最少使用的数据淘汰
6. allkeys-lfu：从所有数据中挑选使用次数最少的数据淘汰
7. allkeys-random：从所有数据中任意选择数据淘汰
8. noeviction（驱逐）：禁止驱逐数据，新写入操作会报错

面试官

Redis 中有个淘汰池的概念，它的具体作用是什么?

灰灰

首先，淘汰是根据淘汰算法去找到需要淘汰的数据，比如 lru 就是找到最久未使用的，LFU 是找到最少使用的。

但是如果去遍历所有的 key，那么就会导致要精确的查出来要淘汰的数据会很慢，从而阻塞数据的添加。

所以 Redis 中采用的是取样淘汰，取样数量可以设置，从取样的数据中找到最适合淘汰的。这样就大大的影响了淘汰的精确度。

所以，加了一个淘汰池的概念，默认是 16，如果淘汰池没满，那么取样出来适合淘汰的数据，假如淘汰池。

如果满了，跟里面的数据做比较，如果比淘汰池里面的数据更适合淘汰。就顶掉之前的。

然后淘汰的时候，就从淘汰池中末尾淘汰。

所以，在我看来，淘汰池的作用就是去提升我们淘汰的精确度。

面试官

基于内存操作，怎么去保证我宕机的时候，数据不会丢失?

灰灰

虽然是基于内存操作，但是持久化还是会做的，Redis 重启的时候，都会从我们的磁盘加载数据，然后再基于内存操作。

数据也会持久化到磁盘，只不过实时性不是特别高，同步方式主要有 2 种，RDB 与 AOF。

RDB 默认的持久化方式，触发时机可以通过 save 指令来配置。同时 shutdown、flushall、save/bgsave 指令也会触发

AOF 默认不开启，但是开启了就会默认用 aof 来加载数据。

aof 是指令追加的形式来刷新到磁盘，但是如果每次都要保证刷到磁盘的话，就每次都需要跟磁盘 IO，所以 Redis 会提供一些选择方案：

比如默认每 s 刷到磁盘，或者每次刷到磁盘，或者交给操作系统，方案不一样，数据一致性跟性能就会有不同的影响。

面试官

AOF 指令追加，那么文件越来越大怎么办?

灰灰

aof 文件会有重写的过程，重写就是当我的文件过大的时候，我会触发重写，这个过大到底多少才大，是可以配置的。

重写其实就是压缩的过程，会以 RDB 的文件格式来压缩现有的 aof。所以也是 AOF+RDB 的混合模式。

54.49 说下对 Redis 集群的理解

面试官

你们之前公司用的 Redis 集群是什么?

灰灰

我们用的 Redis cluster 集群

面试官

你们用 cluster 集群是基于什么考虑的?

灰灰

cluster 是一个去中心化多主多从的集群架构。

cluster 集群除了能实现主从自动切换以外，cluster 还可以做到数据分片，我可以把我的数据灵活的分散到不同的主，然后主对应的从会读取主的数据。

面试官

那数据怎么知道我要往哪个主写

灰灰

cluster 里面会有个虚拟槽的概念，其实就是 0-16383 的一个数值。

然后我们的 key 会通过一个 hash 算法，然后取模 16384 得到一个 0 到 16383 的值。

这个虚拟槽又会对应到不同的机器，并且虚拟槽跟机器的对应关系是能手动维护的，这样我就知道这个 key 会放在哪台机器呢。

并且整个集群维护会非常灵活，一些热点的 key 我都可以单独部署一台主。

面试官

你说 key 是根据一个 hash 算法得到虚拟槽的，那我希望一个订单相关的 key 都到一台机器，比如订单的统计数据、订单的基本信息等等，是不同的 key，会分散到不同的机器，查询就会查询多台机器，怎么解决?

灰灰

作者也想到了这个具体的业务问题，所以，当你希望相关的数据放在同一个 key 的话，你可以在 key 上加{}.这样 hash 的算法只会去根据{}里面的值来进行虚拟槽的计算了。

面试官

cluster 里面的 slave 是怎么保证跟 master 一致的?

灰灰

它的流程是这样的:

slave 会每隔一段时间发起同步指令，这个同步指令会包含我之前同步的数据的偏移量以及我对应的 masterId;

2.master 收到指令后，会去判断传过来的 masterid 是否跟自己的 ID 一致，如果不一致，那么说明之前没有同步过，master 会通过 bgsave 生成一个 rdb 文件给到 slave，slave 会清空自己的数据然后基于 rdb 文件去加载数据。

但是由于 bgsave 是不会阻塞主线程的，所以 master 会有新的指令，这些新的指令会暂存到 master 的一个内存空间，这个内存空间 每个 slave 独立。等 slave rdb 文件加载完后，把新的指令给到 slave 执行。

这个就是我们经常说的全量同步。

但是有一种场景，我的 slave 可能只是断开了一段时间，只有一部分数据没跟 master 同步。所以会有个增量过程。

master 收到指令后，如果 masterId 跟自己的 ID 一致，那么会根据数据的偏移量去一个内存缓存中找相差的数据，这个缓存叫做积压缓存，大小可以设置，跟 mysql 的 redolog 很类似，是覆盖写。如果能找到相差的数据，那么就同步给 slave。

但是因为覆盖写，所以可能断开太久，这个相差的数据找不到了，也会触发全量同步。

总结下：master 如果根据 slave 的参数，能找到相差的数据，就把相差的数据同步给 slave.如果找不到，那么就触发全量同步。

同时，master 的指令也会异步同步给 slave。

54.50 看门狗的实现原理是什么

面试官：

你的项目是真实的吗？有没有包装?

灰灰：

这个都是我日常做过的

绝对没有掺假

面试官：

你们项目中，Redis 用的是哪个客户端?

灰灰：

Redis 对不同的语言都支持不同的客户端，因为我们是基于 Springboot 开发,所以我们的客户端主要是 Springboot 自带的 letuce

也用过 Redisson,因为它对于我们的业务做了很好的封装，用起来也比较简单。比如锁啊这些。

面试官：

那 Redisson 对分布式锁做了哪些改进？

灰灰：

首先，redis 能做分布式锁，主要是因为是单线程执行的，所以如果能在一个指令里面操作完获取以及设置锁的话，就不会有并发问题。

比如 setnx 指令，以及新版的 set 支持相关的参数。当然也可以用 lua 脚本来保证多个指令的原子性。

但是基于简单的一条指令去做锁的话，

1.是一个不可重入锁

2.另外一个锁过期时间不好把控，可能会出现业务没有执行完锁过期。

所以 redisson 主要解决这 2 个问题，当然还有一些比如读写锁、联锁等等。

可重入的话基于 lua+hash 去实现可重入锁。

然后时间把控，如果你不知道锁多久过期，redisson 会基于看门狗来续期。

当然还有些基础的，比如我没拿到锁，会通过 redis 的发布与订阅+Semaphore 去实现锁等待。

面试官：

看门狗具体怎么实现的？能讲讲么

灰灰：

看门狗其实是时间轮+递归的方式去实现

时间轮就是 hashWheelTimer，是一个 netty 包下的类，它主要实现的功能是延时执行任务。

它的实现逻辑是会启动一个线程去轮询数组，然后任务根据延时多久添加到对应数组，如果轮询到了的话，通过多线程去执行相关的任务。

基于 hashWheelTimer 根据设置的看门狗时间/3 去延时执行任务：这个任务：判断 key 是否存在，如果存在，就续期，并且递归，,递归就是自己、调用自己，10s 后再去判断 key 是否存在。做到只要 key 还在就一直续期。如果 key 不存在，就不再递归。

面试官：

Redisson 为什么有联锁，目的是什么?

灰灰:

联锁的目的是因为 redis 是属于 ap 模式的中间件，会存在数据丢失，那么锁就会失效。所以联锁主要做的一件事情就是尽可能的去保证数据不丢失，加锁会加在不同的独立集群机器。当满足一半成功就成功。其实主要思想就是把鸡蛋分散到不同的篮子，降低风险。只要不是超过一半的失败，就是成功的。

54.51 Redis 线上阻塞要如何排查

面试官:

假如 Redis 线上突然阻塞了，你会怎么排查的?

灰灰:

指令阻塞其他指令

因为 Redis 执行指令是单线程的，因为单次执行速度会非常快，但是如果你让单次执行变慢了，那么也会阻塞后续的指令执行。

哪些情况会让指令变慢:

1.指令获取的数据很多，比如大数据量下执行 keys、hgetall、smembers 等指令。我们可以通过查看 Redis 的慢查找到问题，不要去执行慢查操作

2.大 Key,我单次查询的的数据过大,也会导致单次执行变慢。所以我们需要拆分大 key。

CPU 扛不住了

因为 Redis 处理命令只用到一个 cpu，所以当 cpu 超过我们负荷的时候，也会导致阻塞。

1.Redis 是否有牺牲 cpu 去换取内存的一些配置，比如数据类型底层数据结构的配置。有些数据类型是去追求极致空间，所以会牺牲 CPU。

2.如果也没有，那就是需要加机器了

持久化阻塞

1.虽然说 Redis 的持久化是异步做的，但是 for 子进程的时候就慢，那么也会导致问题。

2.AOF 刷盘的时候，由于磁盘压力，导致操作时间够久，主线程为了数据一致性会阻塞指令。

其他原因

1.其他的程序跟 Redis 部署在一台机器，导致 cpu 之间的相互影响

2.网络问题　包括网络是否稳定，是否有延迟、带宽是否足够

3.客户端连接数是否达到了 Redis 的最大连接数

54.52 项目访问量激增，如何应对

面试官：

你们公司的接口有做过压测么，你的接口线上 qps 最大的是多少?

灰灰：

有的，我们之前的话由于是做在线教育的，所以我们的高峰期一般是在晚上，因为晚上才是学习的高峰期，所以晚上的话 qps 能够达到 2000 左右，而白天平时的话也就 4.500 的样子。

然后我们压测的话，单机 qps 必须达到 1500 到 2000，然后线上会有 5 台机器来负载

面试官：

刚才你讲你们是会有课程售卖，假如双 11 了，请求量可能会暴增，你们怎么处理**

灰灰：

我们的双 11 其实是比较弱化的，重要的是双 12。

1.预估双 12 的量，但是这个量也没法去准确的评估，大概只能根据以往的量跟活动力度来预估一个大概的量，活动的话比如有没有秒杀活动，还是说是组团活动，如果有秒杀那么并发量可能会更高。

还有根据往年的双 12 的流量情况来预估一个大概值，流量大概是在平时流量的 5 到 10 倍。因为活动也不只有双 12 当天，而是一个周期，所以流量也会分散。

2.全链路压测由于是微服务，并且用到了比较多的组件，比如 MQ、redis、mysql 等等，所以整个链路会比较长，我们压测不能局限在单个接口，需要进行一个全

链路压测。

3.找到链路瓶颈、哪个服务、哪个接口、还是说哪个组件服务拖慢了性能,比如是 mysql 还是 redis，找到了链路瓶颈，我们就会去解决，先会去优化相关代码，比如代码是否有低级错误，有不应该的重复请求等等。查询语句是否有走到索引。有哪些数据可以做预热等等。

4.如果代码或者用法没有问题，那么就是到了机器的瓶颈，就会对应的去加机器，对于热点接口以及服务进行扩容。

5.降级对于复杂的但是不是主流的场景，进行静态降级或者暂时不可用

6.像实时监控与报警、与限流这些常见的高可用策略我们平时就会有。

面试官

刚才双 12 是知道流量暴增的点，那么突发的情况怎么解决?

灰灰:

首先，我们会有限流，会控制请求流量，导致系统被过多请求压垮。但是可能会导致很多请求是进不来的。

如果要做到请求尽可能的能处理，我们之前没有对应的措施，但是我觉得可以使用一些监控工具来监控流量的暴增，

可以让这个监控工具自动调用容器编排系统的钩子函数来进行动态的扩容。比如说我有一个 Docker 服务集群，里面有 100 个服务，能够同时承载 10 万的并发量，此时，如果流量监控工具监控到有 20 万的流量打过来，那么，这个流量监控工具就会自动去调用 Docker 服务集群的一个接口，这个接口会帮助我们把 100 个服务动态扩容到 200 个服务。

但是具体的，我之前没有接触过，只是一个思路。

54.53 不同时机面试系列 Mysql 事务

面试官：请你说下你对 InnoDB 事务隔离级别的了解，你在工作中关注隔离级别么?

好的面试官

聊到事务隔离级别，我们得先聊下事务

所谓事务，就是工作的原子单元，可以去进行提交、回滚。在数据库中的工作，就是执行的 sql 语句，这些语句可以是单条语句也可以是多条语句。所以，其实就是去保证单条语句或者多条 sql 语句作为一个原子操作，要么全部成功要么全部回滚。

那么这也是我们数据库 ACID 设计原则里面的 A,所谓 ACID，是我们去设置数据库的时候，尽可能去保证数据的一致性与可靠性的一些设计原则。就像我们写代码的时候会去考虑设计原则，比如依赖倒置原则。

A Atomicity ætəˈmɪsəti 原子性 就是通过事务的提交回滚来实现的

C Consistency kənˈsɪstənsi 一致性 保证数据进来一致的

I Isolation aɪsəˈleɪʃn 每个更改数据的会话希望是相互不影响的

D Durability dʊrəˈbɪləti 持久性 保证数据不会丢失

那么其中的 I 就是事务的隔离级别。

为什么会有这些隔离级别呢?

因为事务是可以并行执行的，那么如果并行执行，就会导致一定的数据一致性问题。比如我们经常讲的脏读、不可重复读、幻读。

那么如果业务不能容忍这些数据一致性问题，作者就得去解决它，但是如果解决了一致性问题，性能就会有很大的影响。所以，作者干脆将这个难题交给了用户自己选择，这样不同的隔离级别就出来了。

分别是

RU

RC

RR

串行化

这 4 个隔离级别，越往上，性能越差，但是数据一致性越好。

RU 读到未提交的数据，所以会有脏读、不可重复读、幻读问题

RC 读已提交的，但是没有解决不可重复读跟幻读问题

RR 在 innoDB 里面通过 MVCC 以及 LBCC 来解决了不可重复读跟幻读问题。也是 innodb 默认的隔离级别

至于串行化，就是不让事务有并行执行的可能，性能最低，一般不会选择

那么在工作中，也可以基于性能与数据一致性，来灵活的选择隔离级别，用得比较多的 RC。

面试官：你刚才讲 RR 通过 MVCC 以及 LBCC 来解决幻读问题，但是 RC 也有啊，为什么 RC 没有解决不可重复读跟幻读。

【灰灰老师回答问题】

这块其实是因为不同的隔离级别在 MVCC 于 LBCC 的实现逻辑有点不一样。

首先，我们来讲下 MVCC 的流程，MVCC 其实就是查询的时候，会去生成一个 readView 的 class 结构，里面会保存一些事务 ID 数据，比如当前存活的事务 ID 有哪些，然后跟数据的事务 ID 进行比对，来决定数据是否展示。

RR 只有在第一次查询的时候会生成一个 readView，后面的查询比对的都是第一个查询的时候的结果，所以后面查不到新的值

RC 是每次查询都会生成一个新的 readView，这样每次查询我就能拿到最新的数据了，也就产生了不可重复读跟幻读问题

而 LBCC 是通过锁的形式去解决不可重复读跟幻读问题，就是我在改数据同一条或者同一批数据的时候，其他事务不能进行改动。但是 RR 相比于 RC 多了一个间隙锁，所谓间隙锁，就是会根据你锁的条件会锁定一个区间，所以没有幻读，但是 RC 隔离级别下，间隙锁只会用在重复键的检查。

这是我对这个问题的理解。

54.54 请你说下线程池的参数，具体什么含义是什么?

面试官：请你说下线程池的参数，具体什么含义是什么?

你好，面试官

线程池其实是线程从池化技术，主要思路是在固定的线程数去执行不定的 task 任务，并且提供一些线程管理相关的钩子方法。

怎么做到线程复用的，主要就是开启线程后，如果我还有任务，线程就会自旋去阻塞队列获取任务，当队列没有任务的时候，线程通过阻塞队列进行阻塞，这样能保证线程是不会回收的，如果需要回收，那么可以通过阻塞队列的 poll 方法去获取任务，在阻塞一定时间后回收，如果不需要回收，则通过 take 方法一直阻塞获取。

这样我就会有固定的线程一直在自旋获取任务，就做到了线程复用。

面试官：那线程池一般开启几个线程?

【灰灰老师回答】这个线程的开启数量，一般要看具体的业务场景。

1.考虑你任务的数量，如果任务并发过多，导致阻塞队列放不下，那么需要较大的线程来提升消费的速度

2.要看你任务消耗的资源是 io 多还是 cpu 多，如果 io 多可以稍微多点线程，cpu 偏多的话可以相对来说少点，比如就 cpu 核数。

3.机器的配置也会影响具体的线程数的配置

这个如果要达到一个理想值，要通过压测才能得到一个适当的值，不能天马行空去猜。过多会导致 cpu 浪费，一直在空旋，过少就没法达到比较好的性能要求。

54.55 线程结束的方式有哪些?

面试官：线程结束的方式有哪些?

1.线程执行完毕后就会自动回收

2.我们也可以采用 stop、suspend 去强制关闭，但是不建议实现，现在 Thread 类中这个方法也是过时的，因为强制关闭，你没法跟踪到线程里面执行到哪里 导致业务异常或者数据一致性问题。

3.如果不强制关闭，那么可以用 interrupt 方法来优雅的关闭，所谓优雅的关闭，其实就是告诉线程一个标记，线程里面可以通过这个标记来判断，外面是否有人要我关闭，如果需要我关闭，那么线程自己来决定要不要执行完 run 方法。所以是否结束线程还是取决于线程自己。

这也是为什么我们的 sleep 等方法需要抛出 InterruptedException 的原因，因为线程在阻塞的时候，如果外面有人告诉你需要中断，你得进行 InterruptedException 响应。

还有一个异常结束，当线程中发生了未捕获的异常，线程就会异常结束，可以通过 Thread.UncaughtExceptionHandler 处理未捕获的异常。

54.56 Mysql 里面的 RedoLog 和 BinLog，他们的作用在哪?

好的面试官，RedoLog 是 innoDB 里面的事务日志，既然是事务日志，所以它的最终目的是保证我的数据一致性，也是 InnoDB 满足 ACID 模型的一块很重要的支持。

binlog 不属于存储引擎级别，不管是什么存储引擎都会有的，是 Mysql 服务自己去实现的。

binlog 其实就是一些二进制日志记录文件，主要目的是数据灾备以及主从节点之间的数据同步。

面试官：那 innoDb 为什么不直接每次数据操作的时候，都保证同步到磁盘，而要加一个 RedoLog?

【灰灰老师回答】那为什么要设计一个 redoLog，是因为 InnoDB 存储引擎为了提高数据交互性能，加了一个 bufferpool 内存缓存，如果我数据每次操作都同步给磁盘，就会很慢。

所以，InnoDB 采用的是异步的刷盘机制，但是异步刷盘又会导致数据丢失。因此，InnoDB 设计了一个 RedoLog 的重做日志，每次操作实时同步到 RedoLog 的 file 文件，当然，默认也会实时刷新到磁盘，但是解决了数据直接同步到磁盘的 2 个性能问题：第一个数据交互的最小单位是 page，单次交互的量比较大，第二个是数据属于随机 IO，而 redolog 属于顺序 IO。

面试官：你刚才说 binlog 是用来同步主从数据的，那假如我 RedoLog 写成功了，但是 binlog 没有成功，那是不是会出现主节点有数据，但是没有同步到 binlog?

【灰灰老师回答】好的，redoLog 如果成功了，binLog 没有，那么就会出现主有，从没有，如果先写 binlog 再写 redolog，也会出现从有，但是主可能没有。

因而，这 2 个日志是尽可能的去保证都要写成功的。

所以，在 Mysql 里面，他们实现的机制是 2pc 二阶段提交，redoLog 先是准备状态，然后再去写 binLog，只有 binLog 写成功了，redolog 才会真正的刷新到磁盘。只有当 redoLog 跟 binLog 都有，才能保证数据一致性。否则，数据回滚。

54.57 一条 mysql 查询语句很慢，怎么去排查优化

面试官：你们公司用的 mysql，假如，一条 mysql 查询语句很慢。你会怎么去排查优化?

面试者：好的。面试官。

1.首先，我会去检查是否是语句查询问题，比如是否是连接数的问题，因为在高并发的场景，这个问题也会经常出现的，可以通过 show status 来查看相关连接，里面包含现在的连接数等信息

2.如果不是网络跟连接问题，那么就需要去优化 sql 语句。

优化 sql 语句，最主要的是要用 explain 进行分析，explain 里面有几个关键字段，一个 type 连接类型，会有 all、index 、range 等等，一般最好能达到 index，如果是 all 那么就是全表扫描。

还有就是 key 里面用到的字段。还有 rows,代表估计检查扫描的行，数量越大，性能越慢，

extra ˈekstrə 附加信息，order by 排序是否走到索引等等，可以在附加信息查看。

通过 explain 能找出一些比较明显的问题，比如是否用到索引，where 条件、orderby、group by 是否用到了索引，如果没有用到，就要去看下索引是否满足最左匹配，或者用到了运算或者范围查询等等使下一个索引列失效。

就需要去变动相关的索引列的顺序或者更改查询语句。

当然，还有一些小表驱动大表，这个就是存储引擎与 mysql server 层的一些交互优化，尽量减少 server 跟存储引擎的一些交互。

当不能快速找到问题的时候，就具体情况去具体分析，比如一些复杂的 sql 语句，那么可以一个一个的去优化，把每个语句、或者每个查询条件单独拆开去分析，找到哪个语句或者哪个查询条件比较慢，然后就能找到慢的问题了。

54.58 MySql count 性能对比

【面试官】

Mysql 在获取表的总数的时候，你知道 count(*)、count(1)、count(字段)的区别么?

求职者:

好的面试官

count，其实是一个聚合函数，聚合数据的总数。

不同的存储引擎实现的方式可能又稍许不一样，我们以默认的 innoDB 存储引擎为例。

其实就是根据 where 条件去进行扫描，得到扫描结果后，一行一行去判断，如果 count 括号里的不是 null，那么累计值+1，否则不加，最后返回一个累计的总数。

所以，括号里面的值在一定程度是会影响性能的。

count(*) ，也就是整条数据，由于整条数据肯定不会为 null，所以，专门做了优化，不会去取全部字段。直接扫描多少数据统计数据即可。

count(1) 扫描到一条数据，直接返回一个数字 1，不会取扫描的字段，也不会去判断是否为 null。

这 2 个也是我们用得最多的，性能最好的。

但是如果你 count(字段)，有 2 种情况。

1.首先去看字段是否能走索引，如果没索引，那么扫描数据不会走索引树，比有索引的字段慢。

2.字段是否可为 null，如果可为 null，那么必须要去判断扫描出来的数据是否为 null，为 null 不统计，所以不可为 null 的比可为 null 的要快。

这是我对 count 聚合函数的理解。

54.59 假如我 Myql 的表很大，进行分页的时候，limit 1000000 加载很慢的话，你是怎么解决的呢?

【面试官】

假如我 Myql 的表很大，进行分页的时候，limit 1000000 加载很慢的话，你是怎么解决的呢?

求职者：好的，面试官，这个问题在生产环境也是比较常见的。

limit m，n ;其实去扫描 m+n 条数据，然后过滤掉前面的 m 条数据，当 m 越大，那么需要扫描的数据也就越多，性能也会越来越慢。

针对这种情况，有以下几种方案可以进行一定的优化。

1.如果 id 是趋势递增的，那么每次查询都可以返回这次查询最大的 ID，然后下次查询，加上大于 上次最大 id 的条件，这样会通过主键索引去扫描，并且扫描数量会少很多很多。因为只需要扫描 where 条件的数据

2.先 limit 出来主键 ID，然后用主表跟查询出来的 ID 进行 inner join 内连接，这样，也能一定上提速，因为减少了回表，查询 ID 只需要走聚集索引就行。

3.当然，如果 mysql 级别优化不了了。我们也可以对分页数据进行缓存，比如 Redis 缓存，数据进行变动的时候，做好缓存依赖即可。

4.因为越往后，一般用户行为触及不到，比如你去看淘宝，不会去翻后面几百页的数据，所以，业务层面也可以做一些让步，比如不做后面几百页的数据。

54.60 谈谈你对分布式 ID 的理解

求职者：

我研究生毕业后，一直在杭州某互联网大厂担任 Java 开发。来贵司应聘高级开发，我的期

待薪资年包 50W

求职者：

没问题，您可以随便考察

求职者：

首先，分布式 ID，是因为随着业务的发展，数据或者 ID 的生成会在不同的地方，比如常见

的数据的分库分表，在分库分表的情况下，假如不同的表生成的 ID 不能保证唯一性，那么

就会导致一个 ID 不能保证一条数据的唯一性，就会带来问题。

所以分布式 ID 的主要目的：不论何时何地，生成的 ID 都是唯一的。

实现的方式有很多：

1.Mysql 的全局 ID 生成表

2.UUID（有些版本会重复）

3.Redis 的 incr 自增 ID

4.zk 的有序节点序号

5.mongoDB 的 objectId、

等等这些方案都能解决唯一性的问题。

但是除了唯一性以外，我们还需要考虑更多的其他因素：

1.比如是否趋势递增，否则在 Mysql 会导致主键索引树的分裂与合并。

2.安全性，通过 ID 是否会导致数据泄露

3.高可用以及高性能，满足业务的需求，不能生成一个 ID 很慢或者动不动不可用。

所以，为了性能与可靠性以及成本，市面上还有一种雪花算法是用得比较多，他是一种基于

位运算来实现的，并且实现简单，没有太多外部依赖。

很多大公司也有基于雪花写了自己的 ID 生成方案，比如美。

54.61 MySQL 索引使用有哪些注意事项呢?

可以从三个维度回答这个问题：索引哪些情况会失效，索引不适合哪些场景， 索引一些规则特性

54.61.1 索引哪些情况会失效

InnoDB 引擎里面有两种索引类型，一种是主键索引、一种是普通索引。

InnoDB 用了 B+树的结构来存储索引数据。

当使用索引列进行数据查询的时候，最终会到主键索引树中查询对应的数据行进行返回。

理论上来说，使用索引列查询，就能很好的提升查询效率，但是不规范的使用

会导致索引失效，从而无法发挥索引本身的价值。

导致索引失效的情况有很多：

在索引列上做运算，比如使用函数，Mysql 在生成执行计划的时候，它是根据统计信息来判断是否要使用索引的。

而在索引列上加函数运算，导致 Mysql 无法识别索引列，也就不会再走索引了。

不过从 Mysql8 开始，增加了函数索引可以解决这个问题。

在一个由多列构成的组合索引中，需要按照最左匹配法则，也就是从索引的最左列开始顺序检索，否则不会走索引。

在组合索引中，索引的存储结构是按照索引列的顺序来存储的，因此在 sql 中也需要按照这个顺序才能进行逐一匹配。

否则 InnoDB 无法识别索引导致索引失效。

当索引列存在隐式转化的时候，比如索引列是字符串类型，但是在 sql 查询中没有使用引号。

那么 Mysql 会自动进行类型转化，从而导致索引失效在索引列使用不等于号、not 查询的时候，由于索引数据的检索效率非常低，因此 Mysql 引擎会判断不走索引。

使用 like 通配符匹配后缀%xxx 的时候，由于这种方式不符合索引的最左匹配原则，所以也不会走索引。

但是反过来，如果通配符匹配的是前缀 xxx%，符合最左匹配，也会走索引。

使用 or 连接查询的时候，or 语句前后没有同时使用索引，那么索引会失效。只有 or 左右查询字段都是索引列的时候，才会生效。

除了这些场景以外，对于多表连接查询的场景中，连接顺序也会影响索引的使用。

不过最终是否走索引，我们可以使用 explain 命令来查看 sql 的执行计划，然后针对性的进行调优即可。

54.61.2 索引不适合哪些场景

1. 数据量少的不适合加索引
2. 更新比较频繁的也不适合加索引
3. 区分度低的字段不适合加索引（如性别）

54.61.3 索引的一些潜规则

4. 覆盖索引
5. 回表
6. 索引数据结构（B+树）
7. 最左前缀原则
8. 索引下推

54.62 扫码登录到底是怎么实现的?

昨天下午，我的一个粉丝群里大家为了“扫码登录“的实现方式争论了 2 个多小时，最后还没得出结论。

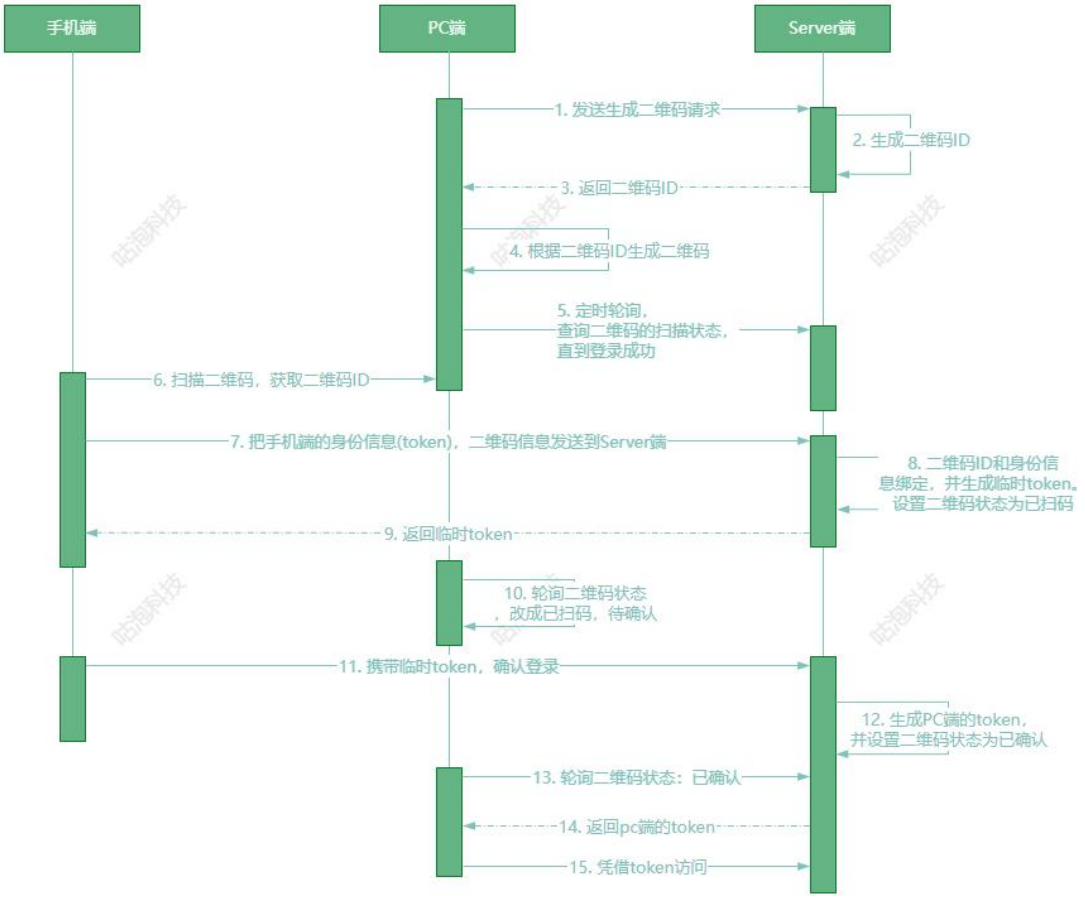
原因是群里面有个工作 5 年的粉丝去面试的时候被问到了这个问题，没有回答出来。

今天就给大家分享一下扫码登录的原理，认真听完，你会发现真的很简单。

扫码登录的本质是，通过已经登录过的 App 应用，扫描未登录的 Web 端程序中的二维码， 通过某种机制触发登录凭证的写入从而实现 Web 端自动登录的过程。

[如图]，

43. 首先，在网页端打开登录页面，展示一个二维码，这个二维码有一个唯一编号是服务端生成的。然后浏览器定时轮询这个二维码的状态
44. 接着，APP 扫描这个二维码，把 APP 的 token 信息、二维码 ID 发送给 Server 端，Server 收到请求后修改二维码的扫码状态，并生成一个临时 token
45. 此时，网页端展示的二维码状态会提示已扫码，待确认。 而 APP 端扫码之后，会提示确认授权的操作。
46. 于是，用户确认登录后，携带临时 token 给到 server，server 端修改二维码状态并为网页端生成授权 token
47. 最后，网页端轮询到状态变化并获取到 token，从而完成扫码授权。



不难发现，整个实现过程并不难，核心的设计就是一个特定的二维码，去连接了 APP 端和网页端。

好了，今天的分享就到这里，如果大家手机上看不清楚这个图片，可以在评论区置顶中去领取，我是 Mic，我们下期见。

图片地址：https://www.processon.com/view/link/63a689d9b6046e79daf68567

54.63 为什么阿里巴巴的 Java 开发手册不建议使用 Java 自带的线程池

Hi，大家好，我是 Mic

之前我分享了一次线程池的工作原理，然后有一些小伙伴就跑过来问我。

Mic，既然线程池这么好，为什么阿里巴巴的 Java 开发手册不建议使用呢？

我从没有看过这个手册，但是我不相信手册上会这么说。

于是我去手册上看到了原文内容。（如图）

发现是 “线程池不允许使用 Executors 去创建”。

4. 【强制】线程池不允许使用 Executors 去创建，而是通过 ThreadPoolExecutor 的方式，这样的处理方式让写的同学更加明确线程池的运行规则，规避资源耗尽的风险。

看到这句话，我就理解原因了。

Executors 里面默认提供的几个线程池是有一些弊端的，如果是不懂多线程、或者是新手直接盲目使用，就可能会造成比较严重的生产事故。

48. 第一个，FixedThreadPool 和 SingleThreadPool 中，阻塞队列长度是 Integer.Max_Value，一旦请求量增加，就会堆积大量请求阻塞在队列中，可能会造成内存溢出的问题。
49. 第二个，CachedThreadPool 和 ScheduledThreadPool 中最大线程数量是 Integer.Max_value，一旦请求量增加，导致创建大量的线程，使得处理性能下降。甚至可能会出现宕机的问题。

为了避免这类问题出现，我们可以直接实例化 ThreadPoolExecutor，然后自己设置参数的值，从而确保线程池的可控性。

实际上，很多源码或者中间件里面，都是使用这类的方式。

好了，今天的分享就到这里了

另外，关于多线程并发方面的面试题，我整理过大概 10W 字的问题和解答，有需要的小伙伴可以在评论区置顶中领取。

我是 Mic，我们下期见。

54.64 字节一面，Redis 为什么这么快?

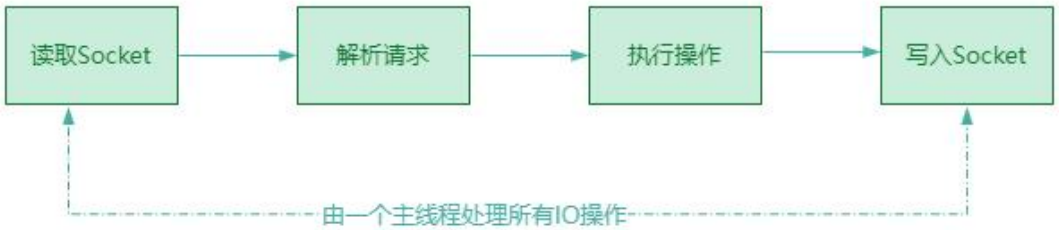
这是一道非常经典并且面试频率很高的面试题。

却很少有人回答得比较全面，今天就给大家分享一下我的理解。

决定 Redis 请求效率的因素主要是三个方面，分别是网络、cpu、内存。

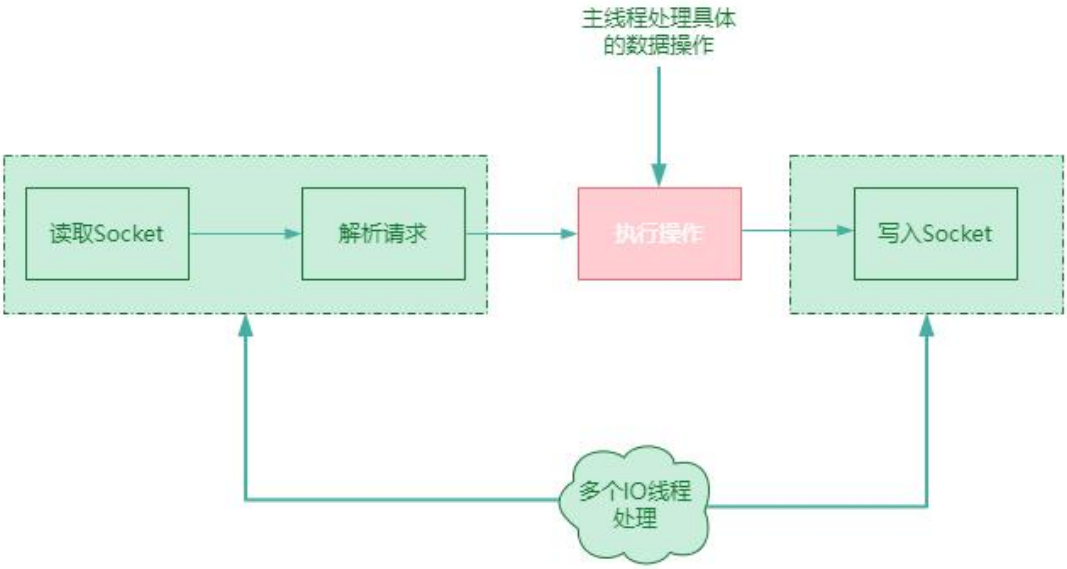
在网络层面，Redis 采用多路复用的设计，提升了并发处理的连接数，不过这个阶段，{如图} Server 端的所有 IO 操作，都是由同一个主线程处理的

这个时候 IO 的瓶颈就会影响到 Redis 端的整体处理性能。



所以从 Redis6.0 开始 {如图}，在多路复用及层面增加了多线程的处理，来优化 IO 处理的能力。

不过，具体的数据操作仍然是由主线程来处理的，所以我们可以认为 Redis 对于数据 IO 的处理依然是单线程。



从 CPU 层面来说，Redis 只需要采用单线程即可，原因有两个。

50. 如果采用多线程，对于 Redis 中的数据操作，都需要通过同步的方式来保证线程安全性，这反而会影响到 redis 的性能
51. 在 Linux 系统上 Redis 通过 pipelining 可以处理 100w 个请求每秒，而应用程序的计算复杂度主要是 O(N) 或 O(log(N))，不会消耗太多 CPU

从内存层面来说，Redis 本身就是一个内存数据库，内存的 IO 速度本身就很快，所以内存的瓶颈只是受限于内存大小。

最后，Redis 本身的数据结构也做了很多的优化，比如压缩表、跳跃表等方式降低了时间复杂读，同时还提供了不同时间复杂度的数据类型。

使得开发人员能够有更多合适的选择。

54.65 为什么一线互联网公司严禁使用存储过程?

一个刚刚从传统行业跳槽到互联网公司的粉丝跑过来问我。

他说在他们公司的数据库开发规范上明确禁止使用存储过程，有点困惑。

因为他在之前传统公司的时候在解决一些复杂的逻辑的时候都是习惯性写存储过程的，突然不让用了都不习惯。

之所以互联网公司不让用，主要有几个方面的原因：

52. 存储过程不好调试，一旦涉及到非常复杂的逻辑，定位问题的时候比较麻烦

53. 存储过程的一致性很差， 如果从 Oracle 迁移到 Mysql，涉及到部分数据库独有特性的时候，整个存储过程就需要重写，不仅成本高，而且还有可能对上游业务造成影响
54. 存储过程的管理比较困难，一旦存储过程的量比较大的时候，就会陷入到一个混乱的状态
55. 存储过程的优化和维护很麻烦， 随着业务的发展，数据库的表结构也许会发生变化，这些变化需要同步给到存储过程，并且有可能原来的 SQL 语句的执行计划不是最优的，也需要重新维护。

在互联网公司，业务的迭代速度是日新月异的，把业务逻辑沉淀在应用程序中，反而更加灵活。

好了，今天的分享就到这里。

最后，我给关注了我的粉丝准备了一个 20W 字的面试文档，大家可以在我的评论区置顶中领取，仅限 10 份，先到先得！

54.66 订单超时自动取消功能如何设计?

“订单超时自动取消功能如何设计？”

最近一个工作 7 年的粉丝去面试，因为这个问题没回答出来导致错过了一个高薪 Offer。

屏幕前很多小伙伴看到这个问题的时候，第一反应肯定是：

这个问题简单，直接写个定时任务去轮询数据库，根据订单时间找到超时的订单把它取消就行了。

那个工作 7 年的粉丝也是这么回答的，结果挂了。

这种方式存在几个问题

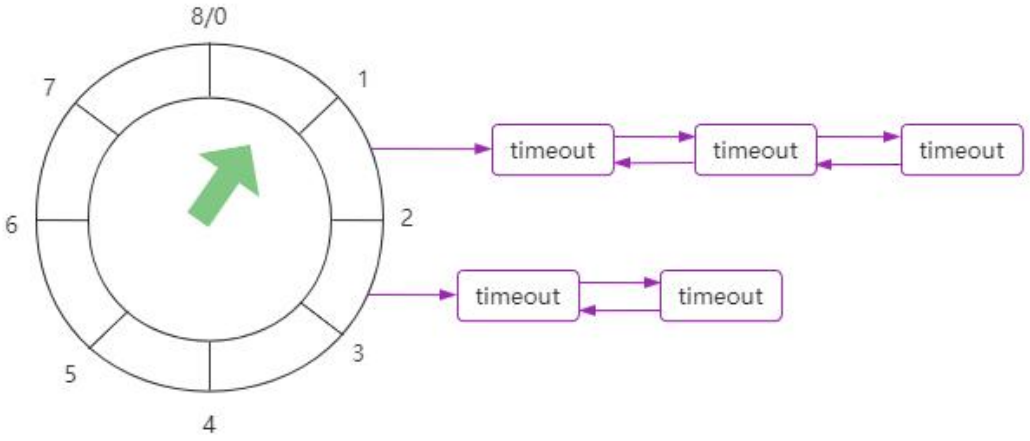
56. 轮询会存在延迟时间，也就是没办法准时实现订单的取消
57. 轮询数据库，会给数据库造成很大的压力，如果订单表的数据量比较大的情况下，轮询的效率也会比较低。

如果要考虑到性能、又要考虑到实时性，有没有更好的方案呢?

当然有，比如

58. 时间轮算法（如图），这种算法是采用了一个环状数组+链表的方式来管理延迟任务，我们只需要计算这个订单的超时时间，再加入到时间轮里面即可。

时间轮算法唯一的缺点就是无法持久化，所以需要在服务重启后做一次数据预热。



59. 利用主流 MQ 中的延迟消息功能，消息发送到 Broker 上以后并不会立刻投递，而是根据消息中设置的延迟时间去投递。我们只需要把新的订单并计算这个订单的超时时间发送到 MQ 中即可。

MQ 这种实现方式在性能、可扩展性、稳定性上都比较好，是一个不错的选择。

好了，我今天的分享就到这里了，如果大家有更好的实现方案，可以在评论区留言。

最后，给大家分享一套我花了一年多时间整理的 20W 字的面试题，正好可以应对金三银四的面试，由于这是给粉丝的福利，所以只发 10 份，大家可以在评论区置顶中去领取。

54.67 Mysql 中的 RR 隔离级别，到底有没有解决幻读问题?

昨天在我的粉丝群里面

就 Mysql 中的 RR（Repeatable Reads）事务隔离级别，到底有没有解决幻读问题发起了激烈的讨论。

一部分人说有，一部分人说没有。

我先说结论，Mysql 中的 RR 事务隔离级别，在特定的情况下会出现幻读的问题。

所谓的幻读，表示在同一个事务中的两次相同条件的查询得到的数据条数不一样。

那在 RR 级别下，具体什么情况下会出现幻读呢?

来看这样一种情况｛如图｝，在事务 1 里面通过 update 语句触发当前读的情况下，就会导致在该事务中的前后两次查询的数据行数不一致，从而出现幻读的现象。

事务1	事务2
begin;	
select * from t1;	
	begin;
	insert into t1 values(4,"Mic");
	commit;
当前读 ← update t1 set name='Mic123';	
select * from t1;	

导致幻读的根本原因是，update 触发的当前读操作，绕过了快照读，从而导致 MVCC 机制在当前场景下失效。

最终读取到了事务 2 中已经提交的数据。

为了避免出现这类的情况，我们可以通过 for update 语句加锁。

好了，今天的分享就到这里，另外，我收藏了基本 Mysql 底层工作原理的电子书，我自己看过收获很大，分享给关注了我的粉丝，仅限 10 份。

大家可以在我评论区指定中去领取。

54.68 有没有排查过线上 oom 的问题，如何排查的

就因为简历上写了一句“有解决生产环境下的 OOM 问题的经验”。

导致面试官直接来问我排查的过程和解决方法。

好了，今天就给大家分享一下线上排查 OOM 问题的经验。

OOM 是 out of memory 的简称，表示程序需要的内存空间大于 JVM 分配的内存空间。

OOM 后果就是导致程序崩溃；可以通俗理解：程序申请内存过大，虚拟机无法满足。

导致 OOM 错误的情况一般是：

60. 给 JVM 虚拟机分配的内存太小，实际业务需求对内存的消耗比较多
61. Java 应用里面存在内存泄漏的问题，或者应用中有大量占用内存的对象，并且没办法及时释放。

我给大家解释一下内存泄漏和内存溢出是两个完全不一样的情况

内存泄露：申请使用完的内存没有释放，导致虚拟机不能再次使用该内存，此时这段内存就泄露了，因为申请者不用了，而又不能被虚拟机分配给别人用。

内存溢出：申请的内存超出了 JVM 能提供的内存大小，此时称之为溢出。

常见的 OOM 异常情况有两种

- java.lang.OutOfMemoryError: Java heap space ------>java 堆内存溢出，此种情况最常见，一般由于内存泄露或者堆的大小设置不当引起。对于内存泄露，需要通过内存监控软件查找程序中的泄露代码，而堆大小可以通过虚拟机参数 -Xms,-Xmx 来修改。
- java.lang.OutOfMemoryError: PermGen space 或 java.lang.OutOfMemoryError：MetaSpace ------>java 方法区，溢出了，一般出现在大量 Class、或者采用 cglib 等反射机制的情况，因为这些情况会产生大量的 Class 信息存储于方法区。

这种情况可以通过更改方法区的大小来解决，使用类似-XX:PermSize=64m -XX:MaxPermSize=256m 的形式修改。

另外，过多的常量尤其是字符串也会导致方法区溢出。

遇到这类问题，通常的排查方式是，先获取内存的 Dump 文件。

Dump 文件有两种方式来生成，

- 第一种是配置 JVM 启动参数，当触发了 OOM 异常的时候自动生成
- 第二种是使用 jmap 工具来生成。

然后使用 MAT 工具来分析 Dump 文件。

- 如果是内存泄漏，可进一步通过工具查看泄漏对象到 GC Roots 的引用链。掌握了泄漏对象的类信息和 GC Roots 引用链的信息，就可以比较准确地定位泄漏代码的位置。
- 如果是普通的内存溢出，确实有很多占用内存的对象，那就只需要提升堆内存空间即可。

好了，今天的分享就到这里。

关于 JVM 相关的面试题，我整理了大概有 1000 道，并且都有对应的回答， 有需要的小伙伴可以在评论区置顶中去领取

因为确实是花了很多时间来整理的，所以只分享 10 份，先到先得。bye bye

54.69 怎么理解接口幂等，项目中如何保证的接口幂等

都 2024 年了，竟然还有人不知道接口幂等是什么东西。

hi，大家好，我是 Mic

今天正好有空，给大家分享一下幂等的实现。

首先，什么是幂等?

简单来说，就是一个接口，使用相同的参数重复执行的情况下，对数据造成的改变只发生一次。

比如支付操作，如果支付接口被重复调了 N 次，那资金的扣减只发生一次，这就是幂等。

有同学会比较好奇， 这个事情不是很正常吗？为什么还要单独拧出来说。

原因很简单｛如图｝， 在分布式架构中，由于引入了网络通信导致一个请求，除了成功/失败以外，还多了一个未知状态。

也就是如果一次远程接口调用失败，有可能这个请求在服务端执行成功了。

而客户端为了确保本次请求执行成功，可能会发起重试的操作，导致同一个接口被重复调用了多次。

为了保证服务端接口的幂等性，我们就需要在服务端的接口中去识别当前请求是重复请求，从而不再进行数据的变更操作。

通常的解决方案有几种。

62. 使用数据库唯一索引的方式实现， 我们可以专门创建一个消息表，里面有一个消息内容的字段并且设置为唯一索引，每次收到消息以后生成 md5 值插入到这个消息表里面。一旦出现重复消息，就会抛异常，我们可以捕获这个异常来避免重复对数据做变更。
63. 使用 Redis 里面的 setNx 命令,我们可以把当前请求中带有唯一标识的信息存储到 Redis 里面，根据 setNx 命令返回的结果来判断是否是重复执行，如果是则丢弃该请求。

64. 使用状态机的方式来实现幂等，在很多的业务场景中，都会存在业务状态的流转，并且这些状态流转只会前进，所以我们在对数据进行修改的时候，只需要在条件里面带上状态，就能避免数据被重复修改的问题。

当然，除了这几种方法以外，肯定还有其他更多的解决方案。

不管采用哪种方案，核心本质都是需要去识别当前请求是重复请求。

当然，如果大家还有更好的方案，可以在评论区留言。

另外，关于分布式高并发相关的场景问题，我最近也整理了 10W 字的面试文档，可以免费分享给关注我的粉丝。

仅限 10 份，可以在评论区置顶中领取。好了，今天的分享就到这里，bye bye。

54.70 消息推送中的已读消息和未读消息设计难题

“站内信”有两个基本功能：

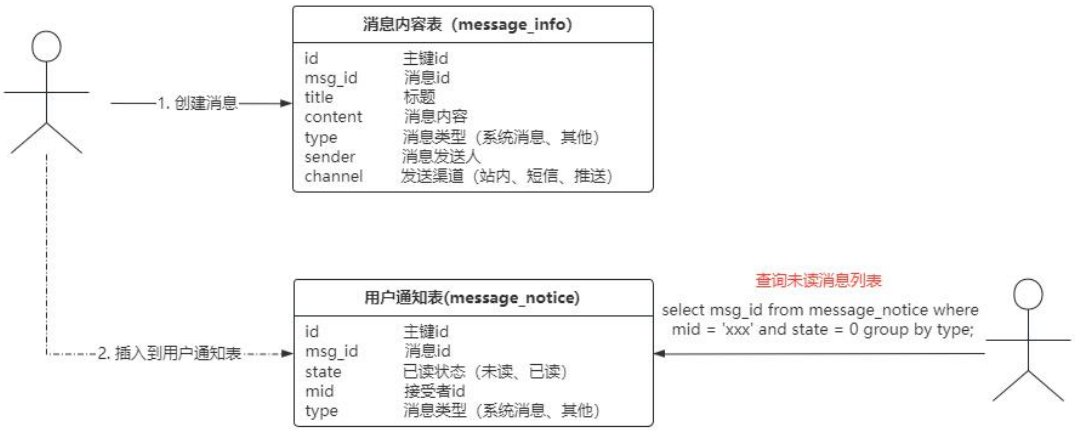
- **点到点的消息传送。**用户给用户发送站内信，管理员给用户发送站内信。
- **点到面的消息传送。**管理员给用户（指定满足某一条件的用户群）群发消息

这两个功能实现起来也很简单｛如图｝。

只需要设计一个消息内容表和一个用户通知表，

当创建一条系统通知后，数据插入到消息内容表。消息内容包含了发送渠道，根据发送渠道决定后续动作。

如果是站内渠道，在插入消息内容后异步的插入记录到用户通知表。



这个方案看起来没什么问题，但实际上，我们把所有用户通知的消息全部放在一个表里面，如果有 10W 个用户，那么同样的消息需要存储 10W 条。

很明显，会带来两个问题：

65. 随着用户量的增加，发送一次消息需要插入到数据库中的数据量会越来越大，导致耗时会越来越长
66. 用户通知表的数据量会非常大，对未读消息的查询效率会严重下降

所以上面这种方案很明显行不通，要解决这两个问题，我有两个参考解决思路。

第一个方式（如图），先取消用户通知表， 避免在发送平台消息的时候插入大量重复数据问题。

其次增加一个“message_offset”站内消息进度表，每个用户维护一个消息消费的进度 Offset。

每个用户去获取未读消息的时候，只需要查询大于当前维护的 msg_id_offset 的数据即可。

在这种设计方式中，即便我们发送给 10W 人，也只需要在消息内容表里面插入一条记录即可。

在性能上和数据量上都有较大的提升。

第二种方式，和第一种方式类似，使用 Redis 中的 Set 集合来保存已经读取过的消息 id。使用 `userid_read_message` 作为 key，这样就可以为每个用户保存已经读取过的所有消息的 id。

当用户读取了未读消息后， 就直接在 redis 的已读消息 id 的 set 中新增一条记录。

这样，在已经得知到已读消息的数量和具体消息 id 的情况下，我们可以直接使用消息 id 来查询没有消费过的数据。

你们看，一个小小的方案设计的优化，就能带来性能上的巨大提升，这就是为什么很多企业宁愿多花点钱也要招技术厉害的人的原因了吧。